
coreTools Command Reference Index

Information by Category

[Application-Programming-Interface](#)

[Activity](#)

[Attributes](#)

[Commands](#)

[Executables](#)

[Item Types](#)

[Strategy](#)

[TCL-Built-in-Commands](#)

[Variables](#)

Alphabetical Listing

[A](#) [B](#) [C](#) [D](#) [E](#) [F](#) [G](#) [H](#) [I](#) [J](#) [K](#) [L](#) [M](#) [N](#) [O](#) [P](#) [Q](#) [R](#) [S](#) [T](#) [U](#) [V](#) [W](#) [X](#) [Y](#) [Z](#) _ _ _ _ _

A

[Abstraction](#)

Indicates the abstraction of an interface instance or the abstraction in which a given interface port should be present.

[Activity](#)

Activity bound to a menu item.

[ActivityGroup](#)

Group/Menu to put this to-level activity in

[AddInterfaces](#)

Add interfaces to the top-level of the core for automated assembly of this core in a System-on-a-Chip subsystem.

[AddLockUpLatch](#)

When true lockup latches will be inserted between clock domain boundaries on scan chains.

[addressBank](#)

This item represents an address bank in a memory map.

[addressBlock](#)

This item represents a block of memory in a memory map.

[AddressOffset](#)

Offset from the base address of the address block for this register or registerArray

[AddressSpaceRef](#)

The name of the address space.

[add activity hook](#)

Add a hook to an existing activity

[add command hook](#)

Add hook into command execution

coreTools Command Reference Index

<u>add files to filegroup</u>	Add files to the specified file group in the Bill of Materials; create the file group if it does not already exist
<u>add hdl pragma</u>	A reuse-pragma statement external to the HDL
<u>add help menu item</u>	Add an item to the console Help menu and link it to relevant documentation
<u>add html report link</u>	Add an HTML report link to the index page
<u>add instantiate component hook</u>	Add hook to execute a Tcl plugin command when instantiate_component is completed
<u>add to collection</u>	Add object(s) to a collection. Result is new collection
<u>add verify subsystem hook</u>	Execute a Tcl checker command to verify subsystem at various places
<u>add workspace hook</u>	Execute a Tcl script when a workspace operation occurs
<u>after</u>	Execute a command after a time delay
<u>alias</u>	Creates a pseudo-command that expands to one or more words, or lists current alias definitions.
<u>AllowOpenProvider</u>	Flag disabling the requirement that provider interfaces must provide a driver when the consumer has an input port.
<u>all components</u>	Return sorted list of all currently loaded components in the sub-system.
<u>all designs</u>	Create a collection of all currently loaded designs
<u>all inputs</u>	Return a collection of all input and inout ports of the current_design
<u>all outputs</u>	Return a collection of all output and inout ports of the current_design
<u>all subdesigns</u>	Return a collection of all subdesigns of the current_design
<u>AltValue</u>	Alternate value for the attribute.
<u>analyze filegroup</u>	Source intent files for this filegroup.
<u>annotate trace</u>	Control annotation of traced command execution to the shell output.
<u>API coreAssembler</u>	Defines the application programming interface for coreAssembler
<u>API coreBuilder</u>	Defines the application programming interface for coreBuilder
<u>API coreConsultant</u>	Defines the application programming interface for coreConsultant
<u>append</u>	Append to variable
<u>append to collection</u>	Add object(s) to a collection. Modifies variable

coreTools Command Reference Index

<u>apply</u>	Apply an anonymous function
<u>apropos</u>	Searches the command database for a pattern.
<u>AreaEstimate</u>	Estimated cell area of a design. Indicates relative importance of a parameter with respect to area estimation. By default all parameters have a weight of 1 but parameters that have a bigger influence on the area of a design can have their match importance increased by raising the AreaWeight value.
<u>AreaWeight</u>	
<u>array</u>	Manipulate array variables
<u>ArrayEnd</u>	Final index for the array entries.
<u>ArrayFieldSize</u>	Number of bits per element in the array
<u>ArrayStart</u>	Initial index for the array entries.
<u>AssociationComplete</u>	Is true if all ports and parameters of the interface instance are completely 'linked'. This attribute does NOT indicate whether the association is valid and error-free. Provides specific completeness information on interface ports and parameters.
<u>AssociationFormula</u>	Formula to determine the alternative value to use during interface-design association.
<u>as_collection</u>	Find a collection or iter_collection by name; result a collection
<u>AtpgTclAuxScript</u>	Specifies a TestMAX Tcl script to be executed at specific points during test vector generation.
<u>AtpgTclAuxScriptComment</u>	Specifies a comment to be issued for the corresponding AtpgTclAuxScript
<u>ATPG basic scan strategy</u>	The ATPG_basic_scan_strategy will create test vectors using the Basic_scan_only engine of TetraMax. This is the recommended starting point for generating vectors, and it will provide fast, good coverage for full scan designs.
<u>attach_interface</u>	Attach a component interface to another component.
<u>attrDefn</u>	Definition of an attribute
<u>autocomplete activity</u>	Automatically complete activities
<u>AutoConnectComplete</u>	Indicates this design was auto connected.
<u>AutoConnectIfUnconnected</u>	Connection to make if left unconnected during auto-connection step.

coreTools Command Reference Index

<u>AutoConnectionPreCallback</u>	Defines user-callback to be invoked before making all automatic connections at a given level of hierarchy.
<u>AutoConnectWhen</u>	Indicates if/when the given interface should be automatically connected.
<u>AutoFixAsync</u>	Enables/Disables automatic fixing of asynchronous preset/clear violations. When set to gate, specifies that the asynchronous sets and resets are autofixed with a gate driven by the test_mode signal. When set to mux sets and resets are with a multiplexer. The select input of the mux is the test_mode signal and a primary input is controlling the asynchronous set or reset in test mode.
<u>AutoFixAsyncLogicGate</u>	
<u>AutoFixAsyncTestData</u>	Port to be used as TestData for fixing set/reset violations. The port must have been defined with a DftExistingSignalType of Reset.
<u>AutoFixAsyncWithScanEnable</u>	When true the test scan enable signal will be used to autofix asynchronous set and reset violations.
<u>AutoFixBidirectional</u>	Enables/Disables automatic bidirectional disabling.
<u>AutoFixClocks</u>	Enables/Disables automatic fixing of uncontrollable clock violations Port to be used as TestData for fixing clock violations. The port must be associated with a clock that has TestClock==1 or have been defined with a DftExistingSignalType of a clock type.
<u>AutoFixClockTestData</u>	
<u>AutoloadFilegroup</u>	Are the files in this group loaded automatically?

B

<u>BalanceBistSegments</u>	By default, SoCBIST rebalances the scan channels in BIST mode. Setting BalanceBistSegments to false will override this, and SoCBIST will not concatenate the scan segments.
<u>BankAlignment</u>	The memory alignment of this bank
<u>BaseAddress</u>	The base address of this item.
<u>batch_install</u>	Install a packaged coreKit into the specified installation root directory.

coreTools Command Reference Index

<u>bgerror</u>	Command invoked to process background errors
<u>BidirectionalMode</u>	Determines whether bidirectional ports are treated as inputs, outputs or left untouched during scan shift.
<u>binary</u>	Insert and extract fields from binary strings
<u>bind tech cell mapping</u>	Specify binding requirements for mapping to a technology specific cell.
<u>BistAutoFixBusses</u>	Indicates floating bus and bus contention issues should be fixed automatically.
<u>BistAutoFixXprop</u>	Should automatic fixing of x propagation be enabled?
<u>BistBlockIndividually</u>	Causes insertion of BIST logic into this design. By default, DFT Compiler concatenates lower level scan segments to arrive at the optimal number of scan chains during BIST controller insertion. You can override this number by specifying the scan configuration
<u>BistChainCount</u>	NumberOfScanChains or MaximumScanChainLength attributes. You must also use BistChainCount to control how many channels are hooked up to the controller. Setting this attribute to 0 allows DFT compiler to choose the number of scan chains.
<u>BistCodecCount</u>	Number of BIST CODECS
<u>BistDiagOutputs</u>	The number of diagnostic outputs for DBIST
<u>BistInvertPrpgClock</u>	Invert the BIST clock that feeds the PRPG Registers
<u>BistMaxChainLength</u>	Controls control the size of the counter, which increases the default upper limit on the length of the reconfigured scan chains. Setting BistMaxChainLength to 0 will allow DFT Compiler to determine the size of the counter.
<u>BistMode</u>	BIST operations to be performed. Choosing 'all' will cause the design to be made bist ready and for the codec to be inserted and integrated with the controller. For hierarchical bist insertion choose codec_insertion for the lower levels of hierarchy and all at the level where the controller is to be integrated with the codecs. Choose bist_ready to make many BIST scan chains or channels within the design without X-propagation and without wrapping the design.
<u>BistObserveOutputs</u>	Controls the number of scan output pins that are inserted into the design for BistMode == xdbist. By default, the number of scan outputs

coreTools Command Reference Index

	created depends on the number of scan chains: scan outputs = (scan chains/32). Setting this attribute to 0 in coreTools lets DFT compiler use the default value.
<u>BistPrpgLength</u>	By default, SoCBIST creates a 479-bit PRPG. You can override this by setting BistPrpgLength to 257. This will cause a 257-bit PRPG to be created. For DBIST, you can use up to 512 scan chains with the 257-bit PRPG. For X-tolerant DBIST, you can use 2048 scan chains with the 257-bit PRPG.
<u>BistPrpgShadowSi</u>	By default, DFT Compiler creates from one to twelve shadow scan chains for the PRPG shadow. The number of shadow scan chains created is optimized based on the length of the longest scan chain in the design. You can stipulate that DFT Compiler create a specific number of shadow scan chains instead by setting BistPrpgShadowSi to n, where n is a number from one to twelve. Setting BistPrpgShadowSi to default allows DFT Compiler to determine the number of shadow scan chains. BistPrpgShadowSi does not affect the length of the longest scan chain. It only affects the number of shadow scan chains.
<u>BistSelectorShadowSi</u>	The number of selector shadow chains. Setting BistSelectorShadowSi will allow DFT Compiler to determine the number of selector shadow chains based on the length of the longest scan chain.
<u>BistSubblocksIndividually</u>	Causes BIST logic to be inserted into the subdesigns of this design.
<u>BistType</u>	Type of BIST to implement
<u>BistUseTristateMux</u>	Specifies that tri-state muxes be used to construct the XDBIST codec
<u>BitAddressOffset</u>	Offset in bits from the base address of the register or memory block
<u>BitRange</u>	Returns the bit range of this field as [start]:[end]. Range is not evaluated if the start or end bit indicator is a formula.
<u>BitsInLAU</u>	The number of bits in the Least Addressable Unit (LAU)
<u>BitsToRepresent</u>	The number of bits needed to represent valid values for this type.
<u>BitWidth</u>	The number of bits in the bitfield.
<u>break</u>	Abort looping command
<u>Bridge</u>	Indicates that there is a bridge from this interface across the component.

coreTools Command Reference Index

<u>BuildcoreKit</u>	Build the coreKit for this core.
<u>BuildIPXACTComponent</u>	Build the IPX-ACT for this core.
<u>BuildSubsystemcoreKit</u>	Run coreBuilder and package the subsystem as a coreKit.
<u>build_debug_tarfile</u>	Build a debug tar-file Usually an interface port needs 'full' association to a design port with the same number of bits as InterfaceSize. Some vectored ports (busses) allow partial connection to an opposite design port. BusAlignment indicates how a narrower design port aligns within a bus of full interface size, and whether it connects on the left or right hand side of a full opposite port, or by slice-indices on a full port with a 0..InterfaceSize-1 indexing and same bit order. The bitwise alignment value is currently only available for IP-XACT components and should not be used within coreBuilder.
<u>BusAlignment</u>	
<u>busBit</u>	Individual bit of a bus
<u>byte_compile_file</u>	Byte compile a TCL file or text string.
<u>C</u>	
<u>CanUngroup</u>	Indicates that the design can be replaced by a hierarchy component.
<u>CanUngroupInfo</u>	Information about design HDL file and/or design contents when CanUngroup value is false.
<u>Capacitance</u>	Explicit load capacitance.
<u>case</u>	Evaluate one of several scripts, depending on a given value Static value that the port or pin will take on for the synthesis session.
<u>CaseAnalysisValue</u>	Unlike ConstantPort, CaseAnalysisValue will not cause logic to be optimized away.
<u>catch</u>	Evaluate script and trap exceptional returns
<u>cA supports hierarchy</u>	Controls whether or not hierarchy is supported by coreAssembler.
<u>cd</u>	Change working directory
<u>cell</u>	Cell item
<u>chan</u>	Read, write and manipulate channels
<u>Channel</u>	Indicates that there is a channel from this interface across the component.

coreTools Command Reference Index

<u>CHeaderValue</u>	Defines the value for the C header file for the given attribute on this register or register field.
<u>CheckCmd</u>	Command used to check values
<u>CheckExpr</u>	Check expression for a parameter
<u>CheckExprMessage</u>	Check expression message for a parameter
<u>CheckExprWhenDisabled</u>	Specifies whether or not CheckExpr expressions should be checked for non-visible or non-enabled parameters.
<u>check bom</u>	Check Bill Of Materials
<u>check component for parameter loops</u>	Check design parameters for the top design for loops return error count.
<u>check env vars</u>	Check for the existence of a list of environment variables.
<u>check error list</u>	List of messages IDs that should cause Design Compiler to stop in the case where one or more of the message appears during a call to the 'compile' command.
<u>check executables</u>	Check for the existence of a list of executables.
<u>check file for errors</u>	Check the specified file for error messages and return the number of error messages found
<u>check license</u>	Get information about a license
<u>check ParameterInfo</u>	Check the syntax and semantics of a ParameterInfo list string
<u>check parameter attribute formulas</u>	Check the formula value of an attribute on a parameter for any issues like dependancy loops.
<u>check script</u>	Statically check a script using the a static Tcl syntax checker based on TclPro Procheck and the Synopsys checking extensions for this tool.
<u>check workspace project id</u>	Compares the Project ID specified against the Project ID used to create the workspace
<u>class</u>	class of all classes
<u>clear prime profile validation exprs</u>	Clears all validation expressions for Prime Profiles.
<u>clock</u>	Clock item
<u>ClockFallLatency</u>	The time it takes a clock signal to propagate from the clock definition point to a register clock pin for a falling transition at the clock

coreTools Command Reference Index

<u>ClockGatingFallHoldCheck</u>	definition point. Falling edge hold check for gating control signals for this clock.
<u>ClockGatingFallSetupCheck</u>	Falling edge setup check for gating control signals for this clock.
<u>ClockGatingRiseHoldCheck</u>	Rising edge hold check for gating control signals for this clock.
<u>ClockGatingRiseSetupCheck</u>	Rising edge setup check for gating control signals for this clock.
<u>ClockGatingSignals</u>	Use this attribute to specify a list of signals to include/exclude when inferring flip-flops with gated clocks.
<u>ClockHoldUncertainty</u>	Hold uncertainty applied to all endpoints of the given clock.
<u>ClockMixing</u>	Specifies whether cells from different clock domains may be put in the same scan chain
<u>ClockName</u>	Name of the clock object (possibly associated with a port) The time it takes a clock signal to propagate from the clock definition point to a register clock pin for a rising transition at the clock definition point.
<u>ClockRiseLatency</u>	Setup uncertainty applied to all endpoints of the given clock. The time it takes a clock signal to propagate from its ideal waveform origin point to the clock definition point for a falling transition at the waveform origin point.
<u>ClockSetupUncertainty</u>	The time it takes a clock signal to propagate from its ideal waveform origin point to the clock definition point for a rising transition at the waveform origin point.
<u>ClockSourceFallLatency</u>	
<u>ClockSourceRiseLatency</u>	
<u>clone component</u>	clone a component in the subsystem.
<u>close</u>	Close an open channel
<u>close workspace</u>	Close the current workspace.
<u>CnctClass</u>	Connection class(es) for this connection.
<u>collection print item limit</u>	Number of item names to print when creating a collection.
<u>CombineInheritValues</u>	Inherit attribute values from other items and combine values to form a single result

coreTools Command Reference Index

<u>combineValues</u>	Combine the values of the specified attribute on the specified items according to the specified command
<u>compare dc version</u>	Compare a version string with the version of Design Compiler currently in use
<u>CompleteConnections</u>	Connect and disconnect component pins and (new) subsystem ports manually until the subsystem design is complete.
<u>complete custom activity definition</u>	Complete a custom activity definition
<u>complete interface definition</u>	Complete an interface definition, and mark the definition as unchangable
<u>ComponentOfItem</u>	The component name of an item
<u>compress sdc</u>	Inputs an SDC file and writes out a bus-compressed version of the input file
<u>concat</u>	Join lists together
<u>ConfigActivity</u>	The activity name that to configure this filegroup.
<u>ConfigDependsOnActivities</u>	List of activity names that should be completed (if they exist) before this filegroup is configured.
<u>ConfigDependsOnGroup</u>	List of filegroups that must be configured before this group.
<u>ConfigIntentSearchPath</u>	A list of directories to search for intent files
<u>Configurable</u>	Is this filegroup configurable?
<u>ConnectionDialogCmd</u>	Defines a custom command to be called for interactive connections.
<u>ConnectionRequired</u>	Used to indicate that this interface does not require a connection.
<u>ConnectToExportedInstance</u>	This instance need to connect to its exported instance
<u>connect interface</u>	Connect interfaces
<u>connect or export unconnected pins</u>	Connect or export unconnected pins in current context.
<u>ConstantPort</u>	Indicates that a port has a fixed logic value.
<u>ConstraintAnalysis</u>	Generate and (optionally) run ptc_shell synthesis constraint analysis scripts.
<u>ConstraintsVerification</u>	set up the rtl config file and generate the specified assertions
<u>ConstraintsVerificationWithAssertions</u>	Execute the simulation generated list of log files
<u>continue</u>	Skip to the next iteration of a loop
<u>ControlPointsPerScanCell</u>	

coreTools Command Reference Index

	Specifies the number of control points that can be shared per scan cell. When TestabilityMethod==control_and_observe for XG mode the larger of ObservePointsPerScanCell and ControlPointsPerScanCell will be used for the -test_points_per_scan_cell value for set_testability_configuration.
<u>ConvertSingleBitBus</u>	Convert single bit bus to bit type
<u>copy</u>	create copies of objects and classes Should this component be copied into the template? A value of false means that the template must be used with an install area for this component. A value of true means that the template does not require an install area for this component
<u>CopyToTemplate</u>	
<u>copy_configuration</u>	Copy configuration parameter values from source to target(s).
<u>copy_target_interface_parameters</u>	Copy parameter values from given interfaces to opposite target interfaces.
<u>coreAssembler</u>	Invokes the coreAssembler tool in either GUI or shell mode.
<u>coreBuilder</u>	Invokes the coreBuilder tool in either GUI or shell mode.
<u>coreBuilderBomTemplateVersion</u>	The version of the Bill-of-Materials template syntax used by a template.
<u>coreConsultant</u>	Invokes the coreConsultant tool in either GUI or shell mode.
<u>coretools_home_page</u>	Page to show as the tool home page.
<u>coroutine</u>	Create and produce values from coroutines Specify the set of files that are to be incorporated into the bill of materials. This is the definition of the set of all files that are part of the core design kit.
<u>CreateBillOfMaterials</u>	Set to true for designs that are created using create_hierarchical_component -fromComponent.
<u>CreatedViaUngrouping</u>	
<u>CreateIntegrationWorkspaces</u>	This activity packages up a core kit, installs the core kit, and then creates one or more integration (coreConsultant) work spaces from that

coreTools Command Reference Index

<u>CreateSkewGroup</u>	installation. These work spaces can be used to do validation of the core kit. Specifies the clock for which the skew group needs to be created. The skew group created will follow the naming convention SKEW_GROUP_QiK_CK_GEN_\ Refer to man page for 'create_clock_skew_group' in Fusion Compiler reference documentation.
<u>CreateSubsystemTemplate</u>	Create a subsystem template. The template is created as a single file and is located in the ./export directory of the current workspace.
<u>CreateTestbenchWorkspace</u>	Create testbench workspace for the current DUT.
<u>create address bank</u>	Create an address bank in a memory map or bank.
<u>create address block</u>	Create an address block in a memory map or bank.
<u>create address space</u>	Create a new address space or update an existing one.
<u>create autoload filegroup</u>	Create an autoloaded filegroup
<u>create command group</u>	Creates a new command group.
<u>create component view</u>	Create an IP-XACT component view.
<u>create configuration parameter</u>	Create a parameter for a configurable filegroup
<u>create connection</u>	Create connections in between component pins and subsystem ports.
<u>create custom activity</u>	Create a custom activity
<u>create custom activity parameter</u>	Create a parameter for a custom activity
<u>create custom tech cell</u>	Specify binding requirements for mapping to a technology specific cell.
<u>create generated clock</u>	Create a generated clock for use within the specified design
<u>create hierarchical component</u>	Create a new hierarchial component in the subsystem
<u>create interface</u>	Create named interface definition
<u>create interface instance</u>	Create an instance of an interface to associate it on the core
<u>create interface parameter</u>	Create interface parameter on an interface definition
<u>create interface port</u>	Create interface port on an interface definition
<u>create memory map</u>	Create a memory map.
<u>create memory mode</u>	

coreTools Command Reference Index

<u>create_plugin_kb</u>	Create a memory mode for the current/selected design
<u>create_prime_profile_file</u>	Create a plug-in KB from the files in the given directory
<u>create_reference_clock</u>	Creates a Prime Profile file to be packaged with the coreKit or shipped externally.
<u>create_register</u>	Create a reference clock for use within the specified design
<u>create_register_array</u>	Create register in an address block.
<u>create_register_field</u>	Create a register array in a memory block.
<u>create_register_field_value</u>	Create a register field in a register.
<u>create_subsystem_parameter</u>	Create a value for a register field.
<u>create_virtual_clock</u>	Create a parameter on the subsystem
<u>create_workspace</u>	Create a virtual clock for use within the specified design
<u>CriticalRange</u>	Create a new workspace (working area).
<u>CriticalRangeCoveringViolators</u>	Default critical range for this design.
<u>CriticalTiming</u>	Set the critical_range value to cover all violating paths in the design.
<u>crm_file_patterns</u>	Indicates whether the port will likely be part of a critical timing path.
	Identifies files that are part of your source code control system.
<u>CTLModels</u>	CTLModels design attribute lists the ctl models to be read in by the synthesis tool when bottom-up synthesis is enabled. It is applicable only for hierarchical designs which have SynthesizeAsSubblock set to 1. CTL files for hardened blocks which are already synthesized may be specified at this level. For example a Subsystem has an IP (A), IP (A) is to be synthesized as a sub-block with DFT models cell1.ctl and cell2.ctl. So one can set the design attributes to include the test models during synthesis.
<u>current_activity</u>	Return current activity
<u>current_design</u>	Set or get the current design
<u>current_kb</u>	Set or get the current knowledge database (KB)
<u>CustomizedTestbenchCode</u>	Customized testbench code to be inserted
<u>CycleTime</u>	Cycle time for a clock.

D

<u>date</u>	Returns a string containing the current date and time.
<u>DCLSPaths</u>	Specify the paths as a list-of-list. Generates scripts that do a 'quick map' of this core to the target technology. This is the fastest possible mapping to gates, and should be used as an estimation path only. You can also select the Stop at GTECH parameter to elaborate your configured core to the Synopsys technology-independent GTECH library, without actually compiling the core to your target technology. Generating a GTECH version of the core is an easy way to create a simulation model for a core that is delivered as encrypted HDL.
<u>DCTCL quick_map_strategy</u>	
<u>dc_shellStopMessageIds</u>	Messages IDs which should cause Design Compiler to terminate.
<u>dc_shellVariable</u>	Variable settings to be used for dc_shell. Example: set_design_attribute {dc_shellVariable[varName]} value.
<u>dc_shellVariableComment</u>	Comment to be issued for the corresponding subscript of dc_shellVariable.
<u>dde</u>	Execute a Dynamic Data Exchange command
<u>debug_script</u>	Debug a script using the TclDevKit Tcl debugger.
<u>DedicatedScanPorts</u>	Determines whether dedicated scan out ports will be created, or whether the functional ports will be used as scan ports.
<u>DedicatedWrapperCell</u>	Specifies the default dedicated wrapper cell type.
<u>DefaultConstantPort</u>	Specifies a default value to be connected to a design input port when the interface connection only links to a design port on one side

coreTools Command Reference Index

<u>DefaultHdlValue</u>	of connected interfaces. Store the Default HdlValue based on the unconfigured core
<u>DefaultInstallDir</u>	Default directory into which to install a file or file group
<u>DefaultLoadPath</u>	A list glob-style patterns specifying how to load this deliverable. Relative paths start with the root of the current workspace.
<u>DefaultParamRange</u>	Store the Parameter Range based on the unconfigured core
<u>DefaultValue</u>	Default value for a parameter
<u>DefaultValueMessage</u>	Message to display along with the default value
<u>DeferEvaluation</u>	If true, port will be present in elaborated design irrespective of generateIf condition. Its conditionality will be preserved in generated toplevel netlist on subsystem level.
<u>define</u>	define and configure classes and objects
<u>DefineComponentViews</u>	Add or delete IPXACT views for the component.
<u>DefineLibClocks</u>	Defines hard macro libraries containing clocks to be defined for constraints.
<u>DefineMemoryMaps</u>	This activity is used to create and view memory maps.
<u>define activity subproc params</u>	Define sub-process launch/result-viewing parameters for an activity or filegroup
<u>define array field parameters</u>	define parameters for each field of an array.
<u>define lib clocks</u>	Extract clock definitions from a hard macro library file
<u>define proc attributes</u>	Defines attributes of a Tcl procedure, including an information string for help, a command group, a set of argument descriptions for help, and so on. The command returns the empty string.
<u>define report</u>	Support creation and viewing of user defined reports
<u>define solution</u>	Define a DesignWare IP Solution

coreTools Command Reference Index

<u>define_split_interface</u>	Group the interfaces belonging to two different components.
<u>define_tech_cell_mapping</u>	Define mapping requirements to a technology specific cell.
<u>DeliverableType</u>	The type of information stored in this deliverable.
<u>Description</u>	Brief description of the item
<u>design</u>	Design item
<u>DesignConfigurationIntent</u>	Review/specify intent attributes associated with the HDL parameters for the core design.
<u>Design Compiler Reference Methodology</u>	Performs a one pass, top down compile from RTL using compile_ultra. This flow implements the Design Compiler Reference Methodology associated with the version of Design Compiler being used.
<u>detach_interface</u>	Remove an attached interface instance from the given component.
<u>DftExistingSignalActiveState</u>	Active logic state for the signal.
<u>DftExistingSignalTestMode</u>	The name of the test mode the dft signal specification applies to.
<u>DftExistingSignalTiming</u>	Waveform to be used for the test signal.
<u>DftExistingSignalType</u>	Test signal to be used for the 'existing_dft' view of set_dft_signal. These are signals that describe structures which already exist that must be understood for the design to pass DRC (dft_drc): clocks, resets, constants, existing scan chains, etc.
<u>DftExistingSignalUsage</u>	The usage scenario(s) the dft signal specification applies to.
<u>DftSpecSignalActiveState</u>	Active logic state for the signal.
<u>DftSpecSignalTestMode</u>	The name of the test mode the dft signal specification applies to.
<u>DftSpecSignalType</u>	Test signal to be used for the 'spec' view of set_dft_signal. These are signals that describe structures that DFT Compiler should use for insert_dft: scan in, scan out, scan enable, etc.
<u>DftSpecSignalUsage</u>	The usage scenario(s) the dft signal specification applies to.

coreTools Command Reference Index

<u>dict</u>	Manipulate dictionaries
<u>DigitsPrecision</u>	Number of digits of precision to interpret and display
<u>DisableAggregation</u>	Set on a pin/port to disable aggregate connections to it in the netlist.
<u>disable_prime_profile</u>	Disable Prime Profile related commands in builder, consultant and assembler (Synopsys only).
<u>DocAddressOffset</u>	Text relating to the AddressOffset attribute, for inclusion in documentation
<u>DocBlockTableAddressOffset</u>	Text relating to the BlockTableAddressOffset attribute, for inclusion in documentation
<u>docbook to html</u>	Convert the given DocBook XML file to HTML format.
<u>DocDefaultValue</u>	Documentation oriented default value for a parameter.
<u>DocDescription</u>	Documentation oriented version of the description of the given item. The text may include DocBook tags.
<u>DocDescriptionField</u>	Defines extra tagged information to be included in the table cell describing the given item. Each value is included, tagged with its corresponding subscript. The text may include DocBook tags.
<u>DocEnabled</u>	Documentation oriented description of when the given item is enabled. The text may include DocBook tags. Must be set for each subscript for which Enabled is set.
<u>DocGenerateIf</u>	Documentation oriented description of when this port exists. The text may include DocBook tags.
<u>DocGroup</u>	Indicates that this object belongs to a group of objects that will be documented as a group.
<u>DocGroupName</u>	Grouping value used to group objects for documentation generation.
<u>DocInclude</u>	Used to conditionally include an item in generated documentation.
<u>DocLabelName</u>	Report oriented label and name combo for a parameter.
<u>DocMaster</u>	

	Indicates that this object is the 'master' element among the group of elements with the same value for DocGroup, and is the element from which the documentation should be generated. The value of this attribute indicates the name of the looping variable and the min and max loop indices for the group (for documentation purposes). It can contain three entries "var min max" or five entries, with the two added entries being the post-elaboration min/max values. The latter two values can be parameter expressions if needed. Note a 4th/6th argument was added that allows a list of attribute/value pairs which can override the existing attribute value [list Name "UseMyName" GroupName "UseMyGroupName"].
<u>DocMemoryAccess</u>	Text relating to the MemoryAccess attribute, for inclusion in documentation
<u>DocOffset</u>	Replaces register table pretext overriding GlobalAddressOffset value normally displayed.
<u>DocPortWidth</u>	Documentation oriented description of the width of the port.
<u>DocPowerDomain</u>	Documentation oriented description of the power domain associated with a port.
<u>DocRangeDecoratedName</u>	Report oriented value for the RangeDecoratedName of a port.
<u>DocRegistered</u>	Documentation oriented description of whether or not a port is registered.
<u>DocRegisteredBy</u>	Documentation oriented description of whether or not a port is registered.
<u>DocRegisterResetMask</u>	Documentation oriented description of the RegisterResetMask attribute. The text may include DocBook tags.
<u>DocRegisterResetValue</u>	Documentation oriented description of the reset value for this register or field. The text may

coreTools Command Reference Index

<u>DocRegisterSize</u>	include DocBook tags. Documentation oriented description of the size of this register or field. The text may include DocBook tags.
<u>DocRegTableAddressOffset</u>	Text relating to the RegTableAddressOffset attribute, for inclusion in documentation
<u>DocShortDescription</u>	Shortened version of the regular DocDescription.
<u>DocSynchronousTo</u>	Documentation oriented description of the clocks to which the port is synchronized.
<u>DocTestable</u>	Documentation oriented description of the Testable attribute. The text may include DocBook tags.
<u>DocVisible</u>	Documentation oriented description of when this register or field is visible. The text may include DocBook tags.
<u>DocVolatileMemory</u>	Documentation oriented description of the VolatileMemory attribute. The text may include DocBook tags.
<u>DocWaiver</u>	Used to document waivers for use of Doc* attributes. Subscript indicates attribute the waiver applies to.
<u>DontTouchNetwork</u>	Set dont_touch_network on this port.
<u>Drive</u>	Drive resistance for input or inout ports.
<u>DrivingCell</u>	Specifies how to generate a set_driving_cell constraint for an input port.
<u>duplicate_component</u>	Duplicate a component in the subsystem
<u>duplicate_workspace</u>	Save the current workspace as duplicate.

E

<u>echo</u>	Echos arguments to standard output.
<u>Enabled</u>	Enable or disable this parameter in the GUI
<u>EnableDftAutoFix</u>	Enables the DFT Compiler AutoFix utility.
<u>EnableDftShadowLogic</u>	

coreTools Command Reference Index

<u>EnableScanCompression</u>	Enables the DFT Compiler Shadow LogicDFT utility for this design. When true enables scan compression. For details on DFTMAX compression options, please refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.
<u>enable_prime_profile</u>	Enable Prime Profile related commands in builder, consultant and assembler (Synopsys only).
<u>encoding</u>	Manipulate encodings
<u>EncryptText</u>	Encrypt text as it is being written.
<u>encrypt_solution</u>	Encrypt TCL files associated with a given DW IP Solution directory.
<u>EndBit</u>	Tag specification ending position. Indicates the original EndBit expression (subscripted representation) which was read in from RTL file in LoadDesigns activity
<u>EndBitExpr</u>	Indicates the original EndBits complete expression which was read in from RTL file in LoadDesigns activity
<u>EndBitsExpr</u>	Are the elements in this address block or interface big or little endian
<u>Endian</u>	an enumeration of the legal attribute values
<u>EnumValues</u>	EnvCheck is a subscripted attribute that may be attached to a coreKit. The user may call verify_environment from coreConsultant to verify the environment based on the CheckEnv attribute.
<u>EnvCheck</u>	Check for end of file condition on channel
<u>eof</u>	Generate an error
<u>error</u>	Prints extended information on errors from the last command.
<u>error_info</u>	Converts escaped name to internally compatible mangled name
<u>escaped_name</u>	Evaluate a Tcl script
<u>eval</u>	evaluates the code in component context
<u>eval_in_component</u>	evaluate a SV expression from ipxact file.
<u>eval_ipxact_expr</u>	Evaluate the given parameter expression.
<u>eval_param</u>	Will cause a set_dont_use \ true to be placed on the library cells specified.
<u>ExcludeLibCells</u>	A list of glob-style patterns to match file names against. Files that match an
<u>ExcludeLoadPatterns</u>	

coreTools Command Reference Index

	entry in this list will not be added to the coreKit.
<u>exec</u>	Invoke subprocesses
<u>Exists</u>	Does this deliverable exist?
<u>exit</u>	End the application
<u>explicit capacitance</u>	Formula to specify an explicit capacitance value for PinLoadType
	Generate and (optionally) run RTL Architect scripts. This activity is used to generate RTLA Reference Methodology scripts. By default, the RTLArchitect run is started immediately after the scripts are generated, but this behavior can be changed by checking the 'Generate Scripts Only' check-box. Once the RTLA Activity is finished, results will be available in the following locations: Logs - rtl/run_'flow type'/logs Reports - rtl/run_'flow type'/reports Outputs - rtl/run_'flow type'/outputs If you choose to run the rtl yourself, or wish to review the generated scripts or constraints, these can be found in the following locations: Flat Makefile: rm_setup/Makefile_rtl_flat Explore Makefile: rm_setup/Makefile_rtl_explore
<u>ExploreRTLArchitecture</u>	Indicates a 'define whose value should be written to the indicated file as part of the configuration process.
<u>ExportDefineToFile</u>	Export all interfaces in the current context that are set up for auto-export.
<u>export all interfaces</u>	Export a component interface out of the subsystem.
<u>export interface</u>	Export pins currently connected to open
<u>export open pins</u>	Export the pin to a port on the subsystem.
<u>export pin</u>	Export port description attributes up one level of hierarchy.
<u>export port descriptions</u>	Export unconnected interfaces in the current hierarchy.
<u>export unconnected interfaces</u>	Export unconnected pins in the current hierarchy.
<u>export unconnected pins</u>	Export the workspace as a component.
<u>export workspace as component</u>	Evaluate an expression
<u>expr</u>	Determines the disabling option during scan shift for all tristate nets that drive output ports of a design.
<u>ExternalTristates</u>	disable_all disables all drivers.

coreTools Command Reference Index

	enable_one disables all but one driver. no_disabling specifies not to insert disabling logic.
<u>ExtraSearchPathAppend</u>	List of additional appended search_path entries for Design Compiler.
<u>ExtraSearchPathPrepend</u>	List of additional prepended search_path entries for Design Compiler.
E	
<u>fblocked</u>	Test whether the last input operation exhausted all available input
<u>FcAuxSynthesisScript</u>	Specifies a FC script to be executed prior to specific optimization phase.
<u>FcAuxSynthesisScriptComment</u>	Specifies a comment to be issued for the corresponding FcAuxSynthesisScript.
<u>FCExcludeLibCells</u>	The attribute specifies the purposes for the library cells. The purpose is passed into the set_lib_cell_purpose command.
<u>FCIncludeLibCells</u>	The attribute specifies the purposes for the library cells. The purpose is passed into the set_lib_cell_purpose command.
<u>fconfigure</u>	Set and get options on a channel
<u>fcopy</u>	Copy data from one channel to another
<u>fc_shellAppOption</u>	Sets application options to the specified value on the specified block. Settings to be used for Fusion Compiler (fc_shell).
<u>fc_shellAppOptionComment</u>	Add a comment for the application options being introduced in the tool intent.
<u>FeedThroughConnect</u>	This input port feeds-through to the output ports listed. Used in automatic connections to prevent connections to these pins. Connections will bypass the component and connect items that would be connected to both pins directly.
<u>file</u>	File item

coreTools Command Reference Index

<u>FileContentType</u>	Content type of a file or file group
<u>fileevent</u>	Execute a script when a channel becomes readable or writable
<u>filegroup</u>	File group item Represents a collection of deliverables (filegroups) defined via a Group list in a BoM template.
<u>filegroupGroup</u>	File name conventions supported by Tcl commands
<u>filename</u>	Read/write/execute permissions for a file or file group
<u>FilePerms</u>	Locate the specified design or specified instance of the design and create a collection that contains that design
<u>find design</u>	Locate the specified interface instances.
<u>find interface instances</u>	Create a collection of items that match the specified name pattern
<u>find item</u>	Find KB with the given name and type.
<u>find kb</u>	Fix hold time violations for this clock.
<u>FixHold</u>	Flush buffered output for a channel
<u>flush</u>	Run formal verification on a design.
<u>FM formal verification strategy</u>	Variable settings to be used for fm_shell. Example: set_design_attribute {fm_shellVariable[varName]} value.
<u>fm_shellVariable</u>	Comment to be issued for the corresponding subscript of fm_shellVariable.
<u>fm_shellVariableComment</u>	'For' loop
<u>for</u>	If GenericAttr[ComponentUndefFile] is set on the design, use that while writing out technology cell macro undef directives. The file option is used otherwise.
<u>ForceTCBToUseComponentUndefFile</u>	Iterate over all elements in one or more lists
<u>foreach</u>	loop through the parameters for the fields of an array.
<u>foreach array field parameter</u>	

coreTools Command Reference Index

<u>foreach in collection</u>	Iterate over a collection
<u>FormalVerification</u>	Generate and (optionally) run formal verification scripts. Specifies a script that is to be run in Formality after the reference and implementation designs have been loaded but just before the verify command.
<u>FormalVerificationAuxScript</u>	Specifies a comment to be issued for the corresponding FormalVerificationAuxScript.
<u>FormalVerificationAuxScriptComment</u>	Format a string in the style of sprintf
<u>format</u>	Arguments to be passed to the set_fpga_pad_type command for FPGA synthesis.
<u>FpgaPadType</u>	Specifies that this port is a primary I/O for FPGA synthesis.
<u>FpgaPortIsPad</u>	Specifies which synthetic library FPGA synthesis will use for synthetic operators. See the man page for fpga_prefer_tmg in fpga_shell.
<u>FpgaPreferTmg</u>	Generate and (optionally) run FPGA synthesis scripts. This activity is used to generate FPGA synthesis scripts for SynplifyPro or SynplifyPremier, customized for the particular core configuration, Synplify version and FPGA device. By default, the Synplify run is started immediately after the scripts are generated, but this behavior can be changed by checking the 'Generate Scripts Only' check-box. Once the synthesis is finished, results will be available in the following locations: EDIF netlist - syn/final/db/<top_design>.edif HDL netlist - syn/final/db/<top_design>.v[hdl] Reports - syn/final/report If you choose to run the synthesis yourself, or wish to review the generated scripts or constraints, these can be found in the following locations; Launch script - syn/run.scr Synplify script -
<u>FPGASynthesis</u>	

coreTools Command Reference Index

FPGA optimization strategy

syn/synplify.tcl Constraints -
syn/constrain/script/*.cscr
Performs synthesis from RTL
targeting an FPGA using synplify.

FunctionDefinition

The Tcl code to implement this
function

Fusion Compiler Reference Methodology

Performs a one pass, top down
compile from RTL using
compile_fusion. This flow
implements the Fusion Compiler
Reference Methodology associated
with the version of Fusion
Compiler being used.

G

GenerateIf

This cell or port is conditionally
generated

GenerateSubsystemRTL

Generate the HDL source code for
top-level design of the subsystem.

GenerateTestVectors

Generate test vectors using
TestMAX.

generate_gtech_sim_model

Write scripts and return shell
command to generate a GTECH model
of the core

generate_qtm

Generate a QTM (quick timing
model) for the top design.

generate_reports

Generate the indicated reports.

generate_simulation_file_list

Generate a list of simulation
files

generate_simulation_launch_command

Generate the command to launch the
simulation

generate_views

Generate the indicated optional
views.

getenv

Returns the value of a system
environment variable.

gets

Read a line from a channel

get_activity_parameter

Get the value of parameter for an
activity

get_address_bank_attribute

Get the value of an attribute from
an address bank.

get_address_block_attribute

Get the value on an attribute from
an address block.

get_all_bits

Return all bits of the specified
port/pin, or port/pin itself if it
is a single bit item.

coreTools Command Reference Index

<u>get_app_var</u>	Gets the value of an application variable.
<u>get_associated_instance_parameter</u>	Get parameter value of linked instantiation parameter.
<u>get_attribute</u>	Get the current value(s) of the specified attribute(s)
<u>get_bitfield_value</u>	Get value of bit range with a given parameter or numeric value.
<u>get_bit_driver</u>	Get the driver for this connection.
<u>get_bit_loads</u>	Get the bit loads for this source connection.
<u>get_cell</u>	Command to get the cell from a cell name or path.
<u>get_cell_attribute</u>	Get the value of an attribute on a design instance
<u>get_children</u>	Get the child/children of the given item
<u>get_clocks</u>	Create a collection of all clocks associated with the specified design
<u>get_clock_attribute</u>	Get the value of an attribute on a clock
<u>get_command_hooks</u>	Get registered hooks for this command
<u>get_command_option_values</u>	Queries current or default option values.
<u>get_component_design</u>	Command to get the elaborated or unelaborated design for a component.
<u>get_component_name</u>	Get the component name of the given item.
<u>get_component_view</u>	Get the view associated with the current/selected component.
<u>get_configuration_parameter</u>	Get configuration (design) parameter value
<u>get_configuration_parameter_attribute</u>	Get the value of an attribute from the specified configuration parameter
<u>get_connections</u>	get connections for the specified single bit port/pin/busbit
<u>get_current_component</u>	Returns the current component.
<u>get_current_hook_command</u>	Get the currently running command string
<u>get_defined_commands</u>	Get information on defined commands and groups.

coreTools Command Reference Index

<u>get design attribute</u>	Get the value of an attribute on the specified design
<u>get design prefix</u>	Returns the prefix to be associated with each design name.
<u>get field parameter for array</u>	returning the field parameter for a given index of an array parameter.
<u>get filegroup attribute</u>	Get an attribute from a filegroup
<u>get filegroup parameter</u>	Get a parameter for a filegroup
<u>get file prefix</u>	Returns the file prefix to be associated with each design name.
<u>get generator parameter</u>	Get component generator parameter value
<u>get hdl file list</u>	Return the list of HDL files that DC must analyze (in order of analysis)
<u>get hdl language</u>	Get the HDL languages used
<u>get hdl library names</u>	Return the list of HDL libraries used by the currently loaded core
<u>get hdl pragma</u>	Get reuse-pragma statement external to the HDL
<u>get hierarchies</u>	Get information about the subsystem hierarchies.
<u>get include dirs</u>	return all include dirs for master leaf components in current or provided context
<u>get installation dir</u>	Get full installation directory name for the current core
<u>get installed component names</u>	Return installed component names and versions (for array set).
<u>get instance parameter value</u>	Command to get the value of instance parameter in terms of the subsystem parameter.
<u>get interface attribute</u>	Get the value of an attribute from the specified interface
<u>get interface link</u>	Get evaluated InterfaceLink for the given interface port/param.
<u>get interface parameter</u>	Get a parameter for an interface
<u>get interface parameter attribute</u>	Get the value of an attribute from the specified interface parameter
<u>get interface port attribute</u>	Get the value of an attribute from the specified interface port
<u>get launch activities status</u>	Get status for all the activities or few activities launched with launch_activities_on_grid
<u>get logical dir</u>	

coreTools Command Reference Index

<u>get_macro_prefix</u>	Get full logical directory name for the specified workspace directory
<u>get_memory_map_attribute</u>	Returns the macro prefix to be associated with each macro name.
<u>get_memory_mode</u>	Get the value on an attribute from a memory map.
<u>get_message_ids</u>	Get the current memory mode associated with the current/selected design
<u>get_message_info</u>	Get application message ids
<u>get_object_name</u>	Returns information about diagnostic messages.
<u>get_parameter_attribute</u>	Converts a collection into a list of item names
<u>get_port_attribute</u>	Get the value of an attribute on a parameter
<u>get_power_domain_voltage</u>	Get the value of an attribute on a port
<u>get_prime_profile_info</u>	Returns the voltage for the specified power domain.
<u>get_register_array_attribute</u>	Get Prime Profile info from the data model..
<u>get_register_attribute</u>	Get the value of an attribute from a register array.
<u>get_register_field_attribute</u>	Get the value of an attribute from a register.
<u>get_register_field_value_attribute</u>	Get the value of an attribute from a register field.
<u>get_slave_base_address</u>	Get the value of an attribute from a register field value.
<u>get_solution_data</u>	Get the remap range and/or base address of the given slave interface.
<u>get_solution_file</u>	Get all solution data or an individual field.
<u>get_solution_root</u>	Get IP solution file if present.
<u>get_solution_variable</u>	Get root context of current DesignWare IP Solution.
<u>get_strategy_parameter</u>	Get all solution variables or an individual variable.
<u>get_supply_voltage</u>	Get the value of a parameter for a strategy
	Command used to insert a voltage value into a set_voltage command in a power intent file.

coreTools Command Reference Index

<u>get_sv_expr</u>	Convert a tcl expression to equivalent sv expression
<u>get_synthesis_dir</u>	Return the full directory name for the specified synthesis subdirectory
<u>get_tcl_expr</u>	convert a SV expression to tcl.
<u>get_template_naming_style</u>	Get the template_naming_style value.
<u>get_template_parameter_style</u>	Get the template_parameter_style value.
<u>get_template_separator_style</u>	Get the template_separator_style value.
<u>get_TMR_separation_distance</u>	Retruns the TMR Separation Distance
<u>get_tool_preference</u>	Get the tool preference parameter
<u>get_tool_root</u>	Get the root directory for the specified tool.
<u>get_top_design</u>	Returns the top design.
<u>get_top_design_name</u>	Return the name of the top level design
<u>get_trace_option</u>	Get the current option value controlling behavior of command tracing and output annotation..
<u>get_unix_variable</u>	This is a synonym for the \fBgetenv\fP command.
<u>get_upf_attribute</u>	Get the value of an attribute on a UPF item
<u>get_upf_port_names</u>	Returns the names of the ports in in the specified power domain.
<u>get_value_from_interface</u>	Get value for interface parameter from the specified interface on the same component.
<u>get_VLNV_search_path_entries</u>	Get elements of given type and VLNV from the cache.
<u>get_workspace_kb</u>	Returns the current top-level knowledge database.
<u>get_workspace_license_info</u>	Get details about license and license type associated with workspace creation.
<u>get_workspace_name</u>	Returns the name of the current top-level knowledge database.
<u>glob</u>	Return names of files that match patterns
<u>global</u>	Access global variables
<u>GlobalSlotNumber</u>	Provides access to the global slot number stored in the SlotNumber

coreTools Command Reference Index

<u>GroupImage</u>	parameter on the given interface Image to use in pre-text for signal, parameter, and memory map groups.
<u>GroupImageAttrs</u>	Attributes to use for xml docbook:imagedata generated from GroupImage.
<u>GroupImageTitle</u>	Title to use for figures displayed with GroupText or GroupImage.
<u>GroupName</u>	Group similar items together in GUI displays
<u>GroupText</u>	Text that is displayed as pre-text for signal, parameter, and memory map groups.
<u>GroupUserLabel</u>	User label of the parameter group
<u>group_components</u>	Group the following components into a lower hierarchical level
<u>guiBehavior</u>	Array variable that specifies different aspects of the GUI behavior.
<u>gui_get setting</u>	Get a setting on the specified window
<u>gui_set setting</u>	Set a setting on the specified window
<u>gui_source</u>	Source script in the GUI without GUI updates.
<u>gui_start</u>	Start GUI
<u>gui_stop</u>	Stop GUI
 H	
<u>HasLibertyModel</u>	Indicates that this design has a Liberty model and therefore no HDL file analysis is required for synthesis.
<u>hdlFunc</u>	Internal model of a VHDL function
<u>hdlin enable upf compatible naming</u>	Tracks the value of the like-named dc_shell variable
<u>HdlLibrary</u>	HDL library into which to analyze this file or file group
<u>HdlSourceFileNames</u>	Name(s) of the file that defines this design or package, and any included files.
<u>HdlTypeName</u>	HDL type name of this parameter
<u>HdlUnquoted</u>	

coreTools Command Reference Index

	Indicates that when string param is written in HDL it should not be quoted.
<u>HdlValue</u>	How the value is formatted in the given language
<u>HeaderNameFormat</u>	Defines the name format for the header reference for the given attribute on this register or register field.
<u>help</u>	Displays quick help for one or more commands.
<u>HelpUrl</u>	URL of the documentation file for an item
<u>HierarchicalIsolation</u>	When true, hierarchical isolation logic is built, so that dedicated subdesign scan-out signals are gated by the design scan enable signal.
<u>HighFanout</u>	This input port has a high fanout.
<u>history</u>	Displays or modifies the commands recorded in the history list.
<u>http</u>	Client-side implementation of the HTTP/1.1 protocol
 L	
<u>IconPath</u>	Path to an icon for this item
<u>IdealPort</u>	This port is 'ideal'. It has infinite drive, no resistance, and directly attached nets are not checked for transition or capacitance violations. When true , insert_dft/compile -scan identifies shift-registers in the design and keep only the first flop as a scan flop. Scan Architecting in preview_dft/insert_dft will group the flops that form shift registers as a single scan group named shift_register_identified_??. preview_dft -show all will show the shift registers identified. preview_dft -script writes out the shift-registers as scan groups. If the scan style is not multiplexed_flip_flop, insert_dft/compile -scan ignores this option.
<u>IdentifyShiftRegisters</u>	
<u>if</u>	Execute scripts conditionally

coreTools Command Reference Index

<u>IfUnconnected</u>	Connection to make if left unconnected.
<u>Importance</u>	Indicates whether the deliverables for this filegroup are required or optional
<u>ImportedWorkspaceComponentFile</u>	Indicates the file used to import a workspace component.
<u>import_component</u>	Import an unpackaged component into the subsystem.
<u>import_ipxact_data</u>	Import partial IP-XACT component data.
<u>import_workspace_as_component</u>	Import the workspace as a component.
<u>IncludeIf</u>	Replace text if the parameter expression is true
<u>IncludeLibCells</u>	Will cause a set_dont_use \ false to be placed on the library cells specified.
<u>incr</u>	Increment the value of a variable
<u>index_collection</u>	Extract object from collection. Result is new collection
<u>infinite_drive</u>	Formula to indicate infinite drive for an input port
<u>info</u>	Return information about the state of the Tcl interpreter
<u>InheritValue</u>	Inherit an attribute value from an item up or down the design hierarchy or from a pin
<u>InitializeSubsystemConstraints</u>	Create system level clocks, and initialize system level constraints from component level constraints.
<u>InputDelay</u>	Delay constraint for an input port
<u>InputResistance</u>	Specifies the resistance on the net driven by an input port.
<u>InsertEndOfChainLockupLatch</u>	When true lockup latches will be inserted at the end of scan chains.
<u>InsertTechCellBindMacroAtStart</u>	Set to true to the file item if tech cell defn insertion should be done at top
<u>InsertTestPoints</u>	Controls whether observe and control points are inserted into this design to enhance testability.
<u>Install</u>	Should this group be installed
<u>InstallUserWorkDir</u>	How to install a file or file group into a user workspace
<u>InstallWhen</u>	When should file group be installed?
<u>install_filegroup</u>	

coreTools Command Reference Index

<u>InstanceClockNames</u>	Install the specified filegroup from the Bill of Materials
<u>instances of</u>	Names to use when creating clocks during InitializeSubsystemConstraints.
<u>instantiate component</u>	Get instances of given design name, recursively.
<u>instantiate dut</u>	Instantiate a component in the subsystem.
<u>instantiate interface</u>	Instantiate a DUT into the testbench subsystem
<u>instantiate solution</u>	Instantiate an exported interface without any attachment.
<u>IntentFilesProcessed</u>	Initiate creation of a DesignWare IP Solution
<u>InterClockHoldFallFallUncertainty</u>	Set to true after intent files or processed.
<u>InterClockHoldFallRiseUncertainty</u>	Uncertainty applied from falling edge of source clock to the falling edge of the destination clock for hold delay calculations
<u>InterClockHoldRiseFallUncertainty</u>	Uncertainty applied from falling edge of source clock to the rising edge of the destination clock for hold delay calculations
<u>InterClockHoldRiseRiseUncertainty</u>	Uncertainty applied from rising edge of source clock to the falling edge of the destination clock for hold delay calculations
<u>InterClockSetupFallFallUncertainty</u>	Uncertainty applied from rising edge of source clock to the rising edge of the destination clock for hold delay calculations
<u>InterClockSetupFallRiseUncertainty</u>	Uncertainty applied from falling edge of source clock to the falling edge of the destination clock for setup delay calculations
<u>InterClockSetupRiseFallUncertainty</u>	Uncertainty applied from falling edge of source clock to the rising edge of the destination clock for setup delay calculations
<u>InterClockSetupRiseRiseUncertainty</u>	Uncertainty applied from rising edge of source clock to the falling edge of the destination clock for setup delay calculations

coreTools Command Reference Index

	the destination clock for setup delay calculations
<u>interfaceDefn</u>	interfaceDefn item
<u>InterfaceGroup</u>	Used to indicate that an interface is part of a 'connection' group.
<u>interfaceInstance</u>	interfaceInstance item
<u>InterfaceIsUsed</u>	Indicates that the interface object is used on its parent interface instance.
<u>InterfaceLabel</u>	The label to be used in the display of interfaces/connections in the ad component tree dialog
<u>InterfaceLink</u>	Indicates the association to a design port or parameter (top-level core design). The design name can include \@\' and {\} in the attribute value, and allows a [\@\'] index annotation.
<u>interfacePort</u>	interfacePort item
<u>InterfaceSize</u>	Number of required bits on a design port associated with this interface port.
<u>InterfaceType</u>	Indicates the interface type for the given interface.
<u>InterfaceTypeName</u>	The customized name for each interface type.
<u>InternalTristates</u>	Determines the disabling option during scan shift for all tristate nets that do not drive output ports of a design. disable_all disables all drivers. enable_one disables all but one driver. no_disabling specifies not to insert disabling logic.
<u>interp</u>	Create and manipulate Tcl interpreters
<u>intfPin</u>	Represents an interface instance on a cell.
<u>intfPort</u>	Represents an interface instance on a design.
<u>invoke_generator</u>	invoke a component generator.
<u>invoke_ralgen</u>	Invoke ralgen from \$VCS_HOME or within cT image.
<u>in_autocomplete activity</u>	Is an activity currently being autocompleted?
<u>IPEnvList</u>	Allows additional IP specific env vars to be considered for running an activity. These will be overridden by UserEnvList activity parameter if

coreTools Command Reference Index

	same exist
<u>IsAddressable</u>	Is true if the interface bus definition is addressable.
<u>IsArray</u>	Indicates that the parameter is an array or not.
<u>IsBehavioral</u>	Indicates that the given port/net represents a behavioral connection.
	Concurrent Clock and Data (CCD) Optimization technology to enable users to meet frequency goals on high performance designs. When the attribute is set to true the clock is becomes part of the path groups for targeted concurrent clock and data (CCD) optimization. Refer to man page for 'ccd.targeted_ccd_path_groups' in Fusion Compiler reference documentation.
<u>IsCCDTarget</u>	
<u>IsComplete</u>	This activity is complete
<u>IsConnected</u>	Is the subsystem port, component pin, or interface connected?
	Indicates that the interface parameter value is used as control-only value. As a consequence the parameter value impacts interface setup but not the interface configuration.
<u>IsControlValue</u>	
<u>IsDanglingIntfAllowed</u>	Indicates whether an interface can be left without the corresponding consumer/provider.
<u>IsEnabled</u>	This activity is enabled
<u>IsExternalReset</u>	The attribute is set to true when any subscript of ResetProperties is explicitly set.
<u>IsFullyModeled</u>	Indicates that the design contents are fully modeled in coreTools.
<u>IsHexVal</u>	This value is in hexadecimal format
<u>IsTriState</u>	Is the port a tri-state?
<u>IsUpToDate</u>	Is this deliverable up-to-date when compared to disk?
<u>IsWorkspaceComponent</u>	Indicates the file used to import a workspace component.
<u>is_false</u>	Tests the value of a specified variable, and returns 1 if the value is 0 or the case-insensitive string \bfalse\b; returns 0 if the value is 1 or the case-insensitive string

coreTools Command Reference Index

<u>is_prime_profile_enabled</u>	\fBtrue\fP. Returns 1 if Prime Profile is enabled in this session, 0 otherwise.
<u>is_true</u>	Tests the value of a specified variable, and returns 1 if the value is 1 or the case-insensitive string \fBtrue\fP; returns 0 if the value is 0 or the case-insensitive string \fBfalse\fP.

J

<u>join</u>	Create a string by joining together list elements
-------------	---

K

<u>KbVersion</u>	Knowledge base version identifier.
<u>knowledgeBase</u>	Knowledge database item

L

<u>Label</u>	Label to be used for this item in GUI dialogs
<u>lappend</u>	Append list elements onto a variable
<u>lassign</u>	Assign list elements to variables
<u>launch_activities_on_grid</u>	Launch the sub-processes associated with multiple activities on a grid from a user workspace in coreConsultant
<u>launch_activity_subproc</u>	Launch a sub-process for an activity or filegroup
<u>library</u>	standard library of Tcl procedures
<u>LicenseForOpen</u>	The license used for opening the knowledgebase.
<u>lindex</u>	Retrieve an element from a list
<u>linsert</u>	Insert elements into a list
<u>list</u>	Create a list
<u>list_kb</u>	List all currently loaded knowledge databases
<u>llength</u>	Count the number of elements in a list
<u>lmap</u>	Iterate over all elements in one or more lists and collect results
<u>lminus</u>	Removes one or more named elements from a list and returns a new list.
<u>load</u>	Load machine code and initialize new commands
<u>LoadConsultantPlugins</u>	

coreTools Command Reference Index

	This activity is used to create plugin KBs that are used by the core integrator of this IP. These KBs are added to the Plugins file group for installation in the consultant workspace. As a side-effect, all non-object Tcl files are sourced. This activity is used to get design data into coreBuilder. Currently the tool supports input from either VHDL, Verilog or SystemVerilog Hdl. The HDL files may contain special comment based annotations. After the HDL is analyzed, then Tcl (and SDC) based intent files are sourced.
<u>LoadDesigns</u>	
<u>LoadInterfaceDefinitions</u>	Load definitions for each interface to be instantiated in the top-level of this core for automated assembly within coreAssembler.
<u>load autoload filegroup</u>	Populate an autoloading filegroup
<u>load component memory maps</u>	Loads all visible component memory maps from knowledgebase files in cC and cA into memory.
<u>load file into kb</u>	Load a file or directory into the current knowledge database
<u>load interface definitions</u>	Load interface definitions.
<u>load plugin</u>	Load a plugin KB and install non-configurable files.
<u>load solution definition</u>	Load a DesignWare IP Solution definition file.
<u>LockedInTemplate</u>	Indicate if the parameter/cell/interfaceinstance should be locked in template
<u>lock parameter</u>	Lock the configuration (design) parameter value so it cannot be changed.
<u>LogicalLeft</u>	Indicates the logical left bit index of the interface port to be connected.
<u>LogicalName</u>	Indicates the logical (abstract) name for this port/pin. Used to enable connections with matching LogicalName values.
<u>LogicalRight</u>	Indicates the logical right bit index of the interface port to be connected.
<u>log trace</u>	Control creation of a replay log of traced commands.
<u>lrange</u>	Return one or more adjacent elements from a list
<u>lrepeat</u>	Build a list by repeating elements
<u>lreplace</u>	Replace elements in a list with new elements
<u>lreverse</u>	Reverse the order of a list
<u>ls</u>	Lists the contents of a directory.

coreTools Command Reference Index

<u>lsearch</u>	See if a list contains a particular element
<u>lset</u>	Change an element in a list
<u>lsort</u>	Sort the elements of a list

M

<u>MacroPrefixFileList</u>	User specified list of files which need to be processed for macro prefixing. The paths should correspond to the installation path in the workspace.
<u>man</u>	Displays reference manual pages.
<u>MapBlockIndividually</u>	Map this design individually in Design Compiler. Note that this attribute does not control mapping for PSYN strategies.
<u>MapSubblocksIndividually</u>	Map this design by individually compiling its subdesigns.
<u>mathfunc</u>	Mathematical functions for Tcl expressions
<u>mathop</u>	Mathematical operators as Tcl commands
<u>MaxArea</u>	Maximum area constraint for current design.
<u>MaxCap</u>	Maximum capacitance value for a port or design.
<u>MaxControlPoints</u>	Maximum number of control points to be inserted to enhance testability. When TestabilityMethod==control_and_observe for XG mode the larger of MaxObservePoints and MaxControlPoints will be used for the -max_test_points value for set_testability_configuration.
<u>MaxFallInputDelay</u>	Maximum falling edge delay constraint for an input port
<u>MaxFallInputDelayFallingEdge</u>	Maximum falling edge delay constraint for an input port relative to a falling clock edge
<u>MaxFallOutputDelay</u>	Maximum falling edge delay constraint for an output port
<u>MaxFallOutputDelayFallingEdge</u>	Maximum falling edge delay constraint for an output port relative to a falling clock edge
<u>MaxFallTransitionDelay</u>	Maximum transition fall time for an ideal clock.
<u>MaxFanout</u>	Maximum fanout for an input port or a design.
<u>MaximumScanChainLength</u>	The maximum length a scan chain may be. Setting the value to 0 will allow DFT Compiler to determine the maximum scan

coreTools Command Reference Index

<u>maximum bit blast size</u>	chain length.
<u>MaxInputDelay</u>	Largest size bus that can be bit blasted.
<u>MaxInputDelayFallingEdge</u>	Maximum delay constraint for an input port
<u>MaxLoads</u>	Maximum delay constraint for an input port (relative to the falling clock edge)
<u>MaxObservePoints</u>	Maximum number of loads for this connection.
<u>MaxOutputDelay</u>	Maximum number of observe points to be inserted to enhance testability. When TestabilityMethod==control_and_observe for XG mode the larger of MaxObservePoints and MaxControlPoints will be used for the -max_test_points value for set_testability_configuration.
<u>MaxOutputDelayFallingEdge</u>	Maximum delay constraint for an output port
<u>MaxRiseInputDelay</u>	Maximum delay constraint for an output port (relative to the falling clock edge)
<u>MaxRiseInputDelayFallingEdge</u>	Maximum rising edge delay constraint for an input port
<u>MaxRiseOutputDelay</u>	Maximum rising edge delay constraint for an input port relative to a falling clock edge
<u>MaxRiseOutputDelayFallingEdge</u>	Maximum rising edge delay constraint for an output port
<u>MaxRiseTransitionDelay</u>	Maximum rising edge delay constraint for an output port relative to a falling clock edge
<u>MaxTransition</u>	Maximum transition rise time for an ideal clock.
<u>MaxValue</u>	Maximum transition time for an input port or design.
<u>MaxWriteConstraint</u>	Maximum value allowed
<u>memMap</u>	Defines a maximum value constraint to be met when writing the given register field.
<u>memory</u>	Describes a memory map in this component.
<u>MemoryAccess</u>	Control Tcl memory debugging capabilities
<u>MemoryAccessDescription</u>	Access type for this memory item
<u>MemoryMap</u>	Access type for this memory item
<u>MemoryMapSpecification</u>	Holds the memory map that this slave interface refers to.
<u>MemoryMode</u>	This activity is used to configure memory map.
<u>MemoryRange</u>	Set all the valid Memory Modes active for the design

coreTools Command Reference Index

	Specifies the address range of the registerArray, addressBlock or addressBank
<u>MemoryUsage</u>	Specifies the usage of this addressBlock or addressBank.
<u>MemoryWidth</u>	Specifies the bit width of an address block or bank
<u>MergeCandidate</u>	Indicates that a pin/port is a candidate for port merging via merge_ports.
<u>merge_ports</u>	Merge ports in current hierarchy, which are exporting adjacent bits of same pin.
<u>MessageLimitDC</u>	For each subscript, injects 'set_message_info -id \ -limit \' in the synthesis RM scripts.
<u>MessageLimitFC</u>	For each subscript, injects 'set_message_info -id \ -limit \' in the synthesis RM scripts.
<u>MessageStopOnDC</u>	If true, for each subscript, injects 'set_message_info -id \ -stop_on' in the synthesis RM scripts.
<u>MessageStopOnFC</u>	If true, for each subscript, injects 'set_message_info -id \ -stop_on' in the synthesis RM scripts.
<u>MinCap</u>	Minimum capacitance value for a port or a design.
<u>MinFallInputDelay</u>	Minimum falling edge delay constraint for an input port
<u>MinFallInputDelayFallingEdge</u>	Minimum falling edge delay constraint for an input port relative to a falling clock edge
<u>MinFallOutputDelay</u>	Minimum falling edge delay constraint for an output port
<u>MinFallOutputDelayFallingEdge</u>	Minimum falling edge delay constraint for an output port relative to a falling clock edge
<u>MinFallTransitionDelay</u>	Minimum transition fall time for an ideal clock.
<u>MinInputDelay</u>	Minimum delay constraint for an input port
<u>MinInputDelayFallingEdge</u>	Minimum delay constraint for an input port (relative to the falling clock edge)
<u>MinOutputDelay</u>	Minimum delay constraint for an output port
<u>MinOutputDelayFallingEdge</u>	Minimum delay constraint for an output port (relative to the falling clock edge)
<u>MinRiseInputDelay</u>	Minimum rising edge delay constraint for an input port
<u>MinRiseInputDelayFallingEdge</u>	

coreTools Command Reference Index

	Minimum rising edge delay constraint for an input port relative to a falling clock edge
<u>MinRiseOutputDelay</u>	Minimum rising edge delay constraint for an output port
<u>MinRiseOutputDelayFallingEdge</u>	Minimum rising edge delay constraint for an output port relative to a falling clock edge
<u>MinRiseTransitionDelay</u>	Minimum transition rise time for an ideal clock.
<u>MinValue</u>	Minimum value allowed
<u>MinWriteConstraint</u>	Defines a minimum value constraint to be met when writing the given register field.
<u>MonitoredComponent</u>	Defines a component as a monitor and indicates which type of component can be monitored. Value should be a component VLNV.
<u>MonitoredInterface</u>	Defines a component as application side VIP, identifying the type of interface to be connected to (SPIRIT VLN), the SPIRIT interface type, and the name of the VIP interface to connect to.
<u>move_components</u>	Move one or more components into a different hierarchical level.
<u>mpexpr</u>	Evaluate an expression with multiple precision math
<u>mpformat</u>	Evaluate an expression with multiple precision math
<u>msgcat</u>	Tcl message catalog
<u>msg_box</u>	Displays a modal message box
<u>my</u>	invoke any method of current object
 N	
<u>Name</u>	The name of the item
<u>namespace</u>	create and manipulate contexts for commands and variables
<u>net</u>	Net item
<u>next</u>	invoke superclass method implementations
<u>NumberOfScanChains</u>	The number of scan chains to be created. This value is passed to the set_scan_configuration command with the -chain_count option. For Internal_scan, this is also the number of scan IO ports created. The default value is the number of test clocks in the design.

coreTools Command Reference Index

<u>NumberOfScanCompressionChains</u>	In ScanCompression_mode, this parameter sets the number of scan chains to be created internally. Setting it to 0 will let coreTools decide the number of scan chains in the design based on the number of ScanCompressionIOCount strategy parameter.
<u>Q</u>	
<u>object</u>	root class of the class hierarchy
<u>ObservePointsPerScanCell</u>	Specifies the number of observe points that can be shared per scan cell. When TestabilityMethod==control_and_observe for XG mode the larger of ObservePointsPerScanCell and ControlPointsPerScanCell will be used for the -test_points_per_scan_cell value for set_testability_configuration.
<u>OneRequiredGroup</u>	Specifies that either this deliverable (filegroup) or another deliverable with the same value for this attribute in the the parent group is required.
<u>OneWrapperClock</u>	If the option is set to TRUE, a single wrapper clock is used for all wrapper cells of the design. If shared wrapper cells are used in wrapping the design, the functional clocks of the shared wrapper cells are muxed with the wrapper clock in wrapper test modes by the core wrapper application. If the option is set to FALSE, a dedicated wrapper clock is used for all dedicated wrapper cells of the design and functional clocks are used for shared wrapper cells of the design.
<u>open</u>	Open a file-based or command pipeline channel
<u>OpenAllTreeItems</u>	should all tree items be displayed open for tree dialogs
<u>open_workspace</u>	Open the workspace.
<u>OperatingConditionsBest</u>	Best-case operating condition to use when evaluating timing for this design.
<u>OperatingConditionsWorst</u>	Worst-case operating condition to use when evaluating timing for this design.
<u>OptimizationPriorities</u>	Primary and secondary implementation goals for this design.
<u>OptimizeArithmetic</u>	Should this design be optimized at the arithmetic level prior to initial mapping?
<u>Optional</u>	Indicates that the interface object may not be linked to a design object, or may not exist.

coreTools Command Reference Index

<u>OptionalAssociation</u>	Indicates that the interface object must not be associated to a design object in the parent interface instance.
<u>OutputDelay</u>	Delay constraint for an output port
<u>Overconstrain</u>	Percentage by which delay constraints should be tightened during initial mapping.
 P	
<u>package</u>	Facilities for package loading and version control
<u>packaged_prime_profile_report</u>	Generate an HTML report for Prime Profiles
<u>packagens</u>	Construct an appropriate 'package ifneeded' command for a given package specification
<u>ParallelJobCpuLimit</u>	Specifies the number of parallel compile jobs that may be run.
<u>param</u>	Parameter item
<u>ParameterCheckCmd</u>	Global check command for a number of parameters.
<u>ParameterInfo</u>	This attribute holds hints to build parameter dialog.
<u>Parameters</u>	Parameters used to configure this item. Subscript for that param.
<u>ParamValueFromDesign</u>	Indicates that the interface parameter comes from the corresponding design parameter.
<u>ParentWireLoad</u>	wireload model that will be used one level above this design.
<u>parse_proc_arguments</u>	Parses the arguments passed into a Tcl procedure.
<u>percent_of_period</u>	Define delay as a function of clock's cycle time
<u>permanent_proc</u>	Wrapper around proc to make a permanent hidden proc.
<u>PhysicalLeft</u>	Indicates the physical left bit index of the interface port to be connected.
<u>PhysicalRegion</u>	Denotes that this design is to be contained in one contiguous physical region of the chip. Also controls the mapping of the the design in Physical Compiler as MapBlockIndividually controls the mapping of designs in Design

coreTools Command Reference Index

	Compiler. In Design Compiler there are no restrictions on subdesigns during compilation, but in Physical Compiler a design that is compiled may not contain any subdesigns that are have been compiled separately. When in doubt leave this attribute set to FALSE.
<u>PhysicalRight</u>	Indicates the physical right bit index of the interface port to be connected.
<u>pid</u>	Retrieve process identifiers
<u>pin</u>	Pin item
<u>PingTestMask</u>	Obsolete attribute - Use UVM testing now.
<u>PinLoadCount</u>	Expected external fanout value for a port.
<u>PinLoadType</u>	Type of cell expected to load this port (externally).
<u>pin_cap_of</u>	Return the capacitance of the specified pin.
<u>pkgMkIndex</u>	Build an index for automatic loading of packages
<u>platform</u>	System identification support code and utilities
<u>platform_shell</u>	System identification support code and utilities
<u>plugin_proc</u>	Define a procedure in coreKit context or workspace context
<u>port</u>	Port item
<u>PortDirection</u>	Signal flow direction of this port
<u>PortFunction</u>	Function of the port/pin/bit: S(ource), L(oad), or B(oth)
<u>PortWidth</u>	Indicates the width of the port.
<u>PostPromptCmd</u>	Command(s) to be evaluated when completing an activity, after the parameters are successfully validated.
<u>PowerDomain</u>	Indicates that the given port is associated with the indicated power domain (which must be enumerated in the PowerDomains attribute on the containing design).
<u>PowerDomains</u>	Defines the set of valid power domain names for this design.
<u>PredefinedInoutPorts</u>	List of inout ports to create prior

<u>PredefinedInputPorts</u>	to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.
<u>PredefinedOutputPorts</u>	List of input ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.
<u>prefix</u>	List of output ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.
<u>prefix defines</u>	facilities for prefix matching
<u>prefix identifiers</u>	Prefix parameters in RTL
<u>prefix macros</u>	Prefix the specified identifiers in the listed source files.
<u>PrePromptCmd</u>	Prefix all of the macros found in source files.
<u>prereq activities complete</u>	Command(s) to be evaluated before querying and validating the parameters for an activity
<u>PreserveDesignWare</u>	Check if prerequisite activities have completed yet!
<u>PreserveHierarchyFromTop</u>	Indicates that the DesignWare parts in this design should not be ungrouped.
<u>PreserveMuxOps</u>	Defines lower level designs and cells to be preserved during synthesis.
<u>PrimeProfileAllowNone</u>	Indicates that the MuxOps in this design should not be ungrouped.
<u>PrimeProfileCurrent</u>	If true, users are allowed to skip Prime Profile application in SelectPrimeProfile activity.
<u>PrimeProfileDefaults</u>	Design attribute to store the current prime profile.
<u>PrimeProfileNote</u>	Design attribute to store Default Prime Profile tags.
<u>PrimeProfilesValid</u>	A note to be displayed in SelectPrimeProfile activity.
<u>printenv</u>	Design attribute to store valid prime profile tags.
<u>printvar</u>	Prints the value of environment variables.

coreTools Command Reference Index

<u>print_message_info</u>	Prints information about diagnostic messages that have occurred or have been limited.
<u>print_suppressed_messages</u>	Displays an alphabetical list of message IDs that are currently suppressed.
<u>proc</u>	Create a Tcl procedure
<u>process_solution_files</u>	Explicitly process solution files for the indicated phase.
<u>proc_args</u>	Displays the formal parameters of a procedure.
<u>proc_body</u>	Displays the body of a procedure.
<u>ProjectID</u>	Used to record the project ID associated with a workspace.
<u>ProjectRootDir</u>	The logical root directory of a core project.
<u>propagate_memory_map</u>	Propagate the memory maps visible from an exported slave interface to that slave
<u>protected_proc</u>	Wrapper around proc to make a protected (permanent but visible) proc.
<u>pt_shellVariable</u>	Variable settings to be used for pt_shell. Example: set_design_attribute {pt_shellVariable[varName]} value.
<u>pt_shellVariableComment</u>	Comment to be issued for the corresponding subscript of pt_shellVariable.
<u>PT_static_timing_analysis_strategy</u>	Run Static Timing Analysis on a design using PrimeTime
<u>puts</u>	Write to a channel
<u>pwd</u>	Return the absolute path of the current working directory
<u>py_eval</u>	Evaluate python code. This is an optional feature, not all applications support Python.

Q

<u>quit</u>	Exits the shell.
-------------	------------------

R

<u>RALAccessType</u>

<u>RALAdditionalFieldAttributeText</u>	Defines the VMM RAL access type for this register field. In most cases this is calculated via the MemoryAccess, WriteBehavior, and ReadAction values for the field. This is typically only set explicitly when the a0 or a1 types are desired. For RAL (Register Abstraction Language) files, this attribute defines additional text to be added to the definition of each RAL field. This attribute can be used to specify RAL constraints.
<u>RalListInfo</u>	Specifies the Ral list info for tb mode used for the workspace.
<u>RALSuppressionComment</u>	Reason for skipping test on register, register field or register array. Must not be left empty.
<u>randomize_parameters</u>	randomize the parameters with the help of VCS constraints solver
<u>rand_proc</u>	Create SV function which is equivalent of tcl proc which will be used by randomize_parameters command. Generally created in coreBuilder via plugin files. Can be overridden in cC and cA
<u>read</u>	Read from a channel
<u>ReadAction</u>	Indicates that reading the given field will cause the field value to be modified as described by the value of this attribute.
<u>ReadActionDescription</u>	Indicates that reading the given field will cause the field value to be modified as described by the value of this attribute.
<u>ReadActionModifier</u>	Indicates user-defined reading action when ReadAction is set to 'modify'. It implies this read action is not covered by any of other allowed ReadAction values which are: {clear, set}. This attribute is only valid when ReadAction is set to 'modify'
<u>ReadOnlyInterface</u>	Indicates that this interface connection cannot be modified manually.

coreTools Command Reference Index

<u>ReadOnlyParam</u>	The value of this parameter is read-only
<u>ReadOnlyWorkspaceComponent</u>	Indicates the workspace component will remain read-only after completing specify subsystem.
<u>read_attribute table</u>	Read an attribute table file.
<u>read_component constraints</u>	Read component SDC files to apply component constraints.
<u>read_ipxact file</u>	Read and process an IP-XACT file of the specified type.
<u>read_parameter info table</u>	Read a parameter info table file.
<u>read_parameter table</u>	Read a parameter table file.
<u>read_pin connection table</u>	Read a pin connection table file.
<u>read_sdc</u>	Read a Synopsys Design Constraints format script.
<u>read_subsystem table</u>	Read a subsystem table file.
<u>redirect</u>	Redirects the output of a command to a file.
<u>refchan</u>	command handler API of reflected channels
<u>ReferenceClock</u>	Indicates a generated clock defined for reference purposes only. A reference clock is used to enable clock linking in subsystem assembly but is not written to any SDC file.
<u>ReferenceClockName</u>	For a virtual clock, stores name of the reference real clock in the same hierarchy.
<u>refItem</u>	Defines a reference to a memory map in a component.
<u>regexp</u>	Match a regular expression against a string
<u>register</u>	Describes a register within an address block.
<u>registerArray</u>	Describes a registerArray within an address block or register array.
<u>RegisterArrayDimensions</u>	Specifies the size of the RegisterArray.
<u>Registered</u>	Indicates that the given port is registered within the component. It is either directly connected to a register or connected via only buffers or inverters with no intervening logic.
<u>RegisteredBy</u>	Indicates that the given port is registered within the component. It

coreTools Command Reference Index

<u>registerField</u>	is either directly connected to a register or connected via only buffers or inverters with no intervening logic.
<u>RegisterResetMask</u>	Describes a field within a register.
<u>RegisterResetValue</u>	Mask to be anded before comparing to the RegisterResetValue
<u>RegisterSize</u>	Value of the register at reset
<u>register_netlist</u>	Specifies the size of the Register or register field in bits
<u>registry</u>	Register a user-defined netlist for use in Generate Subsystem RTL.
<u>regsub</u>	Manipulate the Windows registry
<u>reinitialize component</u>	Perform substitutions based on regular expression pattern matching
<u>reinitialize workspace</u>	Re-initialize a component through resetting all design parameters in coreAssembler or coreConsultant.
<u>RemoveIfEmpty</u>	Re-initialize workspace through resetting all design parameters and activity parameters for each given activity
<u>remove address bank</u>	Specifies that this file can be deleted from disk if it is empty.
<u>remove address block</u>	Remove an address bank.
<u>remove address space</u>	Remove an address block.
<u>remove allow_paths</u>	Remove an address space.
<u>remove area estimates</u>	Remove -allow_paths from the named clock group.
<u>remove attached interface</u>	Remove area estimation values for the current design.
<u>remove command hook</u>	Delete attached interfaces.
<u>remove component</u>	Remove hook from command execution
<u>remove component view</u>	Remove the specified component instance from the subsystem.
<u>remove connection</u>	Remove an IP-XACT component view.
<u>remove constraints</u>	Remove the specified connections.
<u>remove exported interface</u>	Remove constraints from the specified or current component and from all levels above.
<u>remove from collection</u>	Remove an exported interface instance from the subsystem.
	Remove object(s) from a collection. Result is new collection

coreTools Command Reference Index

<u>remove generated clock</u>	Remove the specified generated clock
<u>remove html report link</u>	Remove an HTML report link from the index page
<u>remove interface</u>	Remove an interface definition and all its instances
<u>remove interface instance</u>	Remove an instance of an interface associate with the core
<u>remove ipxact file</u>	remove a spirit file and related objects defined in the file.
<u>remove memory map</u>	Remove a memory map.
<u>remove register</u>	Remove a register with the given name.
<u>remove register array</u>	Remove a registerArray.
<u>remove register field</u>	Remove a register field.
<u>remove register field value</u>	Remove a register field_value.
<u>remove virtual clock</u>	Remove the specified virtual clock
<u>rename</u>	Rename or delete a command
<u>rename component</u>	rename a component in the subsystem.
<u>rename exported interface</u>	Rename an exported or attached interface instance in the subsystem.
<u>rename port</u>	Rename a hierarchical port
<u>ReplaceConstantParam</u>	Replace the definition of a constant
<u>ReplaceDesignParams</u>	Replace a set of design parameters with user-specified values
<u>ReplaceFormatParam</u>	Replace text with a formatted parameter value
<u>ReplaceInHDL</u>	If true on a VHDL package parameter, then automatically replace the parameter definition in the source code. Defaults to false if there are no annotations on the parameter in the VHDL. For verilog 2001, set to false for conditional cells defined through generate-conditional statements.
<u>ReplaceSingleBitBus</u>	Convert single bit declarations into bit type
<u>replace component</u>	replace a component in the subsystem.
<u>replace interface</u>	This command takes a connected interface (with SpecifySubsystem completed) and removes the auto

coreTools Command Reference Index

<u>replay connections</u>	connections of the interfaces and replaces them with manual connections all without incompleting SpecifySubsystem. Replay manual connections.
<u>report activities</u>	Lists all activities and their current states
<u>report activity parameters</u>	List all parameters for each activity in the activity list.
<u>report activity subproc</u>	Report on activity/filegroup sub-process.
<u>report app var</u>	Shows the application variables.
<u>report attribute</u>	Report on the specified (or all) attributes for the specified item(s)
<u>report bom</u>	Report Bill Of Materials
<u>report connections</u>	Report on connected and unconnected component pins and subsystem ports.
<u>report filegroup</u>	Report file groups
<u>report interfaces</u>	generate textual or HDL description of the given interface definitions
<u>report interface instances</u>	generate textual or HDL description of the given interface instances
<u>report ip</u>	Report on Synopsys IP used.
<u>report license</u>	List all currently checked out licenses
<u>report memory maps</u>	Report on memory maps in a component.
<u>report prime profiles</u>	Generates a Prime Profile report for a hierarchy/component.
<u>report prime profile validation expr</u>	Report validation expressions for Prime Profiles.
<u>report solutions</u>	Generate simple report of available solutions.
<u>report suppressed ral tests</u>	Generates a JSON report for all memory items that are untested and items which are tested with a non-empty string in the RALSuppressionComment attribute
<u>report tech cell mapping</u>	Report technology specific cell mapping definitions and bindings.
<u>report timing exceptions</u>	Report timing exception objects
<u>report trace</u>	Produce a report of performance profile metrics collected for traced commands.
<u>RequiredExPortDirection</u>	

coreTools Command Reference Index

<u>RequiredPortDirection</u>	Required design port direction when associated with an exported interface port.
<u>Reserved</u>	Required design port direction when associated with an interface port. Indicates that the bits Reserved (meaning unspecified or undocumented bitFields). Support modeling of external reset signals. active-low : active low reset active-high : active high reset synchronous : reset is synchronous ideal : treat as ideal network for synthesis no-dft : do not include DFT reset signal definition no-vcspyglass : do not include SpyGlass reset signal definition no-synthesis : do not include in synthesis constraints
<u>ResetProperties</u>	Reset specified paths to single cycle timing
<u>reset_path</u>	Restructure the current workspace.
<u>restructure_workspace</u>	Return from a procedure, or set return code of a script
<u>return</u>	A type of stylized comment used to configure text files when used in coreConsultant and coreAssembler. For a more detailed description with more examples, see the coreBuilder manual.
<u>reuse-pragma</u>	Syntax of Tcl regular expressions
<u>re_syntax</u>	Specifies a RTLA script to be executed prior to specific optimization phase.
<u>RtlaAuxScript</u>	Specifies a comment to be issued for the corresponding RtlaAuxScript.
<u>RtlaAuxScriptComment</u>	Perform Design Verification of the subsystem and its components.
<u>RTLStaticChecking</u>	The attribute indicates that design needs abstract generation and to verified using bottom up flow using VC Spyglass in coreAssembler.
<u>RTLStaticCheckingAsSubblock</u>	Runs RTL Architect under the RTL Architect Reference Methodology associated with the version of RTL Architect being used.
<u>RTL_Architect_Reference_Methodology</u>	

<u>rtl_shellAppOption</u>	Sets application options to the specified value on the specified block. Settings to be used for RTL Architect (rtl_shell).
<u>rtl_shellAppOptionComment</u>	Add a comment for the application options being introduced in the tool intent.
<u>RunOptions</u>	Additional options for the given run style. For remote this should be a hostname. For lsf and uge this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
 <u>S</u>	
<u>safe</u>	Creating and manipulating safe interpreters
<u>save_workspace</u>	Save the current workspace.
<u>scale_to_current_units</u>	Scale a time, area, or capacitance value to the unit currently in use
<u>scan</u>	Parse string using conversion specifiers in the style of sscanf
<u>ScanBlockIndividually</u>	Causes scan insertion to be performed on this design when it is stand-alone. The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, please refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.
<u>ScanCompressionConfiguration</u>	
<u>ScanCompressionTestClock</u>	Sets a test clock for scan compression from the list of test clocks available.
<u>ScanCompressionUsage</u>	The value passed along with -usage option for ScanCompression_mode test mode in define_test_mode command. See

ScanExclude

DFTMAX Compression and DFTMAX Ultra/SEQ Compression sections in the TestMAX DFT user guide for more details.

List of sequential cells, hierarchical cells containing sequential elements, references, library cells, or designs that should not be replaced by scan cells.

When true, internal clocks are treated as separate clocks for creating scan chains. In XG mode, setting this attribute to false is equivalent to '-internal_clocks none'. When this attribute is set to TRUE it is Used in conjunction with ScanJumpBuffersInverters.

ScanInternalClocks

This attribute will only have an effect when ScanInternalClocks is set to TRUE. For XG mode, the value of this attribute will be the value used for the -internal_clocks option to set_scan_configuration. When not in XG mode, setting

ScanJumpBuffersInverters

ScanJumpBuffersInverters controls the setting of the DC variable test_jump_over_bufs_invs. Setting ScanJumpBuffersInverters to single sets test_jump_over_bufs_invs to true. Setting the attribute ScanJumpBuffersInverters to 'multi' sets test_jump_over_bufs_invs to false.

ScanMethodology

Selects the scan methodology. In full_scan methodology insert_scan makes all sequential cells scannable if they do not violate scan design rules. A setting of none selects no scan methodology for the design.

ScanReplace

Should non-scan registers be replaced by scannable registers?

ScanStyle

Sets the test scan implementation style for this design.

ScanSubblocksIndividually

Causes scan insertion to be performed on the sub-designs directly instantiated in this design.

ScratchInstallIndex

Indicates the index for the components being installed in scratch/ungroup folder by

<u>scratch_dir</u>	'create_hierarchical_component -fromComponent'. On the Subsystem KB this attribute is used to store the current count, on the design it is used to indicate the index to the install path. The install path can be created as Vscratch/ungroup/Install_\ Directory for storage of temporary files.
<u>SDCConstraints</u>	Specifies information about an externally provided SDC file for a component.
<u>SDCPropagateText</u>	SDC text to be propagated to next level of hierarchy. The text insertion point is based on the subscript specified.
<u>SDCWriteCallback</u>	Name of callback command to invoke after writing SDC file
<u>seek</u>	Change the access position for an open channel
<u>SelectPrimeProfile</u>	Prime Profiles set on components can be selected in this activity.
<u>select_by_class</u>	Select a representative library cell by its function class
<u>select_by_function</u>	Select a representative library cell by its logic function
<u>select_by_name</u>	Select a library cell (or pin) by name
<u>self</u>	method call internal introspection
<u>SeqCompressionInputs</u>	Total No of SEQ Compression Input Ports
<u>SeqCompressionOutputs</u>	Total No of SEQ Compression Output Ports
<u>Sequence</u>	Sequence number of an item
<u>set</u>	Read and write variables
<u>SetDesignFileMacroPrefixes</u>	Set the design, file, and macro prefixes. The design prefix is used to make each design in your core have a unique name. Setting the design prefix is only needed when you need to have two or more versions of one component, i.e., you plan to instantiate two or more versions of one component in the same design database, each with different values of configuration parameters. When the

<u>setenv</u>	RTL code is written to disk, the prefix is prepended to every design name reference \ (and package name references for VHDL\). All design names and references to design names will be changed to {prefix}_{original name}. The macro prefix is used to replace any 'define {var's} and usages of the '{var}' with a prefixed value throughout the RTL code.
<u>SetInHex</u>	Sets the value of a system environment variable. This parameter value can be, must be, or cannot be specified in hexadecimal format
<u>SetUpHelpMenu</u>	Add documentation links to the Help menu in the console window.
<u>set activity incomplete</u>	Set the specified activities to the incomplete state
<u>set activity parameter</u>	Set the value of an activity parameter
<u>set address bank attribute</u>	Set the value of an attributes on an address bank.
<u>set address block attribute</u>	Set the value of an attribute on an address block.
<u>set app var</u>	Sets the value of an application variable.
<u>set area estimate</u>	Store area estimation value for a given technology and configuration.
<u>set attribute</u>	Set the value of an attribute on one or more items
<u>set cell attribute</u>	Set the value of an attribute on a list of design instances
<u>set clock attribute</u>	Set the value of an attribute on a list of clocks
<u>set command option value</u>	Set a command option default or current value.
<u>set component view</u>	Set the view associated with the current/selected component.
<u>set configuration parameter</u>	Set configuration (design) parameter value
<u>set configuration parameter attribute</u>	Set an attribute on the specified configuration parameter
<u>set current command mode</u>	
<u>set current component</u>	Set the current component to enable component level operations.

coreTools Command Reference Index

<u>set default prime profiles</u>	Set Default Prime Profiles on the design in order of priority.
<u>set dependent controls</u>	Set dependent controls for a parameter dialog.
<u>set design attribute</u>	Set the value of an attribute on one or more designs
<u>set design prefix</u>	Sets the prefix to be prepended to each design name.
<u>set disable timing</u>	Disable timing arcs in a circuit
<u>set editing mode</u>	Set workspace editing mode
<u>set false path</u>	Removes timing constraints from particular paths
<u>set filegroup attribute</u>	Set an attribute on a list of filegroups
<u>set filegroup parameter</u>	Set a parameter for a filegroup
<u>set file prefix</u>	Sets the file prefix to be prepended to each design name.
<u>set generator parameter</u>	Set component generator parameter value
<u>set hdl file list</u>	Defines an alternate HDL file list for a component.
<u>set instance parameter</u>	Command to set the value of instance parameter in terms of the subsystem parameter.
<u>set instance wire names</u>	Store mapping of port names to wire names.
<u>set interface attribute</u>	Set an attribute on the specified interface
<u>set interface parameter</u>	Set a parameter on an interface
<u>set interface parameter attribute</u>	Set an attribute on the specified interface parameter
<u>set interface port attribute</u>	Set an attribute on the specified interface port
<u>set macro prefix</u>	Sets the macro prefix to be prepended to each macro name.
<u>set max delay</u>	Specifies a maximum delay target for paths in the current design
<u>set memory map attribute</u>	Set the value of an attribute on a memory map.
<u>set memory mode</u>	Set the memory mode associated with the current/selected design
<u>set message info</u>	Set some information about diagnostic messages.
<u>set min delay</u>	Specifies a minimum delay target for paths in the current design

coreTools Command Reference Index

<u>set_multicycle_path</u>	Modifies the single-cycle timing relationship of a constrained path
<u>set_parameter attribute</u>	Set the value an attribute on one or more parameters
<u>set_port attribute</u>	Set the value of an attribute on one or more ports
<u>set_prime_profile</u>	Set a Prime Profile.
<u>set_prime_profile validation expr</u>	Set a validation expression for Prime Profiles.
<u>set_register_array attribute</u>	Set the value of an attributes on a register array.
<u>set_register attribute</u>	Set the value of an attribute on a register.
<u>set_register field attribute</u>	Set the value of an attribute on a register field.
<u>set_register field value attribute</u>	Set the value of an attribute on a register field value.
<u>set_slave base address</u>	Set the remap range and/or base address of the given slave interface.
<u>set_solution preference</u>	Set solution preference
<u>set_strategy parameter</u>	Set the value of a parameter for a stragety
<u>set_synthesis dir</u>	Set the name of the directory where synthesis will be performed.
<u>set_tclconfig variables</u>	Set TCL variables for design parameters defined to have a TCL configuration variable.
<u>set_template naming style</u>	Set the template_naming_style value.
<u>set_template parameter style</u>	Set the template_parameter_style value.
<u>set_template separator style</u>	Set the template_separator_style value.
<u>set_tool preference</u>	Set the tool preference parameter
<u>set_tool root</u>	Set the root directory for the specified tool.
<u>set_trace option</u>	Set an option value controlling behavior of command tracing and output annotation..
<u>set_unix variable</u>	This is a synonym for the \fBsetenv\fP command.
<u>set_unused interface</u>	set the Unused attribute for unused provider
<u>set_upf attribute</u>	Set the value of an attribute on a list of UPF items
<u>set_upf cells</u>	User routine to specify technology cells for UPF file configuration.

coreTools Command Reference Index

<u>set upf supply voltage</u>	Set the voltage on a supply created from set_voltage.
<u>set upf voltage</u>	User routine to specify voltage values for UPF file configuration.
<u>set workspace options</u>	Set one or more workspace options, customizing the user flow.
<u>sh</u>	Executes a command in a child process.
<u>SharedWrapperCell</u>	Specifies the default shared wrapper cell type.
<u>ShareTestPointsAcrossHierarchy</u>	Specifies that test point scan cells may be shared across hierarchy.
<u>ShowIcons</u>	When shown in a tree display an icon if it is specified.
<u>ShowRoute</u>	Indicates whether or not routes from this component should be drawn in the schematic window.
<u>show color dialog</u>	Displays a color browser dialog
<u>show exit dialog</u>	Displays a color browser dialog
<u>show prime profile prompt</u>	Show/Don't show Prime Profile selection prompt in consultant and assembler.
<u>show spreadsheet for connections</u>	Generate and show a spreadsheet for making connections to interfaces with the type of the given interface.
<u>show url</u>	Display URL internally
<u>show url external</u>	Display URL externally
<u>show user parameter dialog</u>	Show the user parameter dialog to build and return the values of a set of user defined parameters
<u>sh allow tcl with set app var</u>	Allows the \fBset_app_var\fP and \fBget_app_var\fP commands to work with application variables.
<u>sh allow tcl with set app var no message list</u>	Suppresses CMD-104 messages for variables in this list.
<u>sh arch</u>	Indicates the system architecture of your machine.
<u>sh command abbrev mode</u>	Sets the command abbreviation mode for interactive convenience.
<u>sh command abbrev options</u>	Turns off abbreviation of command dash option names when false.
<u>sh command log file</u>	Specifies the name of the file to which the application logs the commands you executed during the session.
<u>sh continue on error</u>	

sh deprecated is error

sh dev null

sh disabled is error

sh enable page mode

sh enable stdout redirect

sh executable name

sh help shows group overview

sh language

sh new variable message

sh new variable message in proc

sh new variable message in script

sh obsolete is error

sh product version

sh script stop severity

sh show tcl stack on error

sh source emits line numbers

sh source logging

sh source uses search path

Allows processing to continue when errors occur during script execution with the \fBsource\fP command.

Raise a Tcl error when a deprecated command is executed.

Indicates the current null device.

Raise a Tcl error when a disabled command is executed.

Displays long reports one page at a time (similar to the UNIX \fBmore\fP command).

Allows the redirect command to capture output to the Tcl stdout channel.

Full path name of executable for the current tool session.

Changes the behavior of the "help" command.

Controls the tool language used by the application interactive console.

Controls a debugging feature for tracing the creation of new variables.

Controls a debugging feature for tracing the creation of new variables in a Tcl procedure.

Controls a debugging feature for tracing the creation of new variables within a sourced script.

Raise a Tcl error when an obsolete command is executed.

Tracks the value of the like-named pt_shell variable

Indicates the error message severity level that would cause a script to stop running before it completes.

Print or do not print the TCL stack when a TCL error occurs.

Indicates the error message severity level that causes an informational message to be issued, listing the script name and line number where that message occurred.

Indicates if individual commands from a sourced script should be logged to the command log file.

<u>sh Tcllib app dirname</u>	Indicates if the \fBsource\fP command uses the \fBsearch_path\fP variable to search for files.
<u>sh user man path</u>	Indicates the name of a directory where application-specific Tcl files are found.
<u>SideConstraint</u>	Indicates a directory root where you can store man pages for display with the \fBman\fP command.
<u>SideEffects</u>	Specifies the definite pin side details. The values for DesignCompiler are left, right, top, bottom \& for FusionCompiler the side should be an integer value starting from 1.
<u>SimTieOff</u>	Obsolete attribute - Use UVM testing now.
<u>SimulateSubsystemTestbench</u>	Indicates how a pin should be tied off in the testbench if left unconnected.
<u>SimulationModel</u>	Simulate the current subsystem testbench.
<u>sizeof collection</u>	Indicates that this design is a simulation model and is not to be synthesized.
<u>SlotNumber</u>	Get the number of objects in collection
<u>SlotNumberOffset</u>	Indicates the slot number on the bus for the given interface.
<u>SlotOrder</u>	Indicates the offset (relative to 0) for numbering slots associated with this interface.
<u>SlotWidth</u>	Indicates the order of the slot on the bus for the given interface.
<u>snps.cmd</u>	Indicates the number of continuous slots allocated by attribute SlotNumber and any other 'unique-value' interface parameter on the instance. The attribute annotates an eval_param expression in context of the instance.
<u>snps.collection</u>	Python object that provides access to application commands.
<u>snps.redirect</u>	Python class for Synopsys collection objects.
	Python class used to redirect application output.

coreTools Command Reference Index

<u>snps.value</u>	Python class for application command return values.
<u>socket</u>	Open a TCP network connection
<u>SolutionData</u>	Stores the definition of the solution (from define_solution).
<u>SolutionEnvVarValues</u>	The solution environment vars stored on disk for use in open workspace.
<u>SolutionItemPhase</u>	Indicates the solution phase the item was created in when installing a solution.
<u>SolutionProcessed</u>	Internal status attribute indicating whether solution files have been processed or not.
<u>source</u>	Read a file and evaluate it as a Tcl script.
<u>source solution script</u>	Invoke specified solution script.
<u>SpecifyClocks</u>	Define the set of clocks (virtual and real) within the core design.
<u>SpecifyConfiguration</u>	Specify values for HDL configuration parameters.
<u>SpecifyOperatingConditionsAndWireLoads</u>	Specify information which defines the context in which the core is going to be used (operating conditions and wireloads)
<u>SpecifyPortConstraints</u>	Set and/or review intent attributes on ports.
<u>SpecifyPowerIntent</u>	This activity is used to capture power intent scripts.
<u>SpecifySubsystem</u>	Select and configure the components and any associated memory maps for the subsystem.
<u>SpecifySynthesisMethodology</u>	Set and/or review intent attributes on designs.
<u>SpecifyTestMethododology</u>	Set and/or review Design for Test intent attributes on designs.
<u>SpecifyTimingExceptions</u>	Define the timing exceptions within the core design.
<u>SpiritContainer</u>	A place holder for spirit data that we don't model so we can export as is
<u>SpiritFile</u>	Indicates the file from which a SPIRIT element originated.
<u>SpiritFileType</u>	Specifies the value of the fileType element. Default is based on FileContentType and LoadDesigns language version.
<u>SpiritInterfaceType</u>	SPIRIT interface type

coreTools Command Reference Index

<u>SpiritLibrary</u>	IP-XACT library name
<u>SpiritName</u>	IP-XACT name
<u>SpiritVendor</u>	IP-XACT vendor name
<u>SpiritVersion</u>	IP-XACT version
<u>split</u>	Split a string into a proper Tcl list
<u>SplitInterfaceMembers</u>	Indicates the list of interfaces linked with the master interface
<u>SplitInterfaceName</u>	Indicates the Master interface pointed from the slave interface.
<u>StartBit</u>	Tag specification starting position. Indicates the original StartBit expression (subscripted representation) which was read in from RTL file in LoadDesigns activity
<u>StartBitExpr</u>	Indicates the original StartBits complete expression which was read in from RTL file in LoadDesigns activity
<u>StartBitsExpr</u>	
<u>StaticChecking</u>	Perform static checking of the subsystem and its components.
<u>StaticDefineExpr</u>	Defines the expression which causes the given port to exist. Assumed to be an expression of parameters that are not in the coreTools data model.
<u>StaticDefineExprs</u>	Specifies which static defines referenced in StaticDefineExpr values on ports should be treated as defined for constraint generation purposes.
<u>StaticTimingAnalysis</u>	Generate and (optionally) run static timing analysis scripts.
<u>StaticTimingAuxScript</u>	Specifies a script that is to be run in PrimeTime after the design has been loaded and all clocks and constraints have been applied, just before the 'update_timing' command.
<u>StaticTimingAuxScriptComment</u>	Specifies a comment to be issued for the corresponding StaticTimingAuxScript.
<u>Strategy</u>	Item used to model a strategy (command script) to drive a design tool
<u>string</u>	Manipulate strings
<u>subst</u>	Perform backslash, command, and variable substitutions
<u>SubstituteInFile</u>	Substitute text in this file at the time indicated by the subscript
<u>suppress_message</u>	

<u>switch</u>	Disables printing of one or more informational or warning messages.
<u>SymbolicNames</u>	Evaluate one of several scripts, depending on a given value
<u>SymbolLibraryPath</u>	List of symbolic names for valid values
<u>SymbolName</u>	The path to the symbol library to be loaded when the component is displayed in a schematic.
<u>SymbolPinPrefix</u>	The name of the desired symbol to use in the schematic display.
<u>SymbolPinSide</u>	The desired pin prefix name (Master).
<u>SymbolType</u>	The desired side of the symbol to place the pin
<u>SymmetricBus</u>	The desired symbol type to be generated
<u>SymmetricBusLink</u>	Indicates the interface is symmetric which allow direct master-slave connection. The bus definition should be defined as 'slave' type, then 'master' type is just symmetric to the slave. Imply 'provider-consumer' interface type for interface instance.
<u>SynchronousTo</u>	Indicates the association to a design port when a symmetric bus is used as a provider or a consumer. The design name can include \@ and {} in the attribute value, and allows a [\@] index annotation.
<u>synopsys program name</u>	Indicates that the given port is synchronous with respect to the given clock or other designer specified conditions.
<u>synopsys root</u>	Indicates the name of the program currently running.
<u>SynplifyAuxSynthesisScript</u>	Indicates the root directory from which the application was run.
<u>SynplifyAuxSynthesisScriptComment</u>	Specifies a synplify script to be executed prior to specific optimization phase.
<u>Synthesis API</u>	Specifies a comment to be issued for the corresponding SynplifyAuxSynthesisScript.
	This man page describes the different methodologies available for generating a gate-level

Synthesize

implementation of a component configured in coreConsultant or a subsystem assembled in coreAssembler.

Generate and (optionally) run synthesis scripts. This activity is used to generate synthesis scripts customized for the particular core configuration, DC/FC version compatibility, and target technology. Now user have the option between Design Compiler and Fusion Compiler to run synthesis. By default synthesis is fired right after the scripts are generated, using the preferred tool. But this behavior can be changed by checking the Generate Scripts Only check-box. Once the synthesis is finished, results will be available in the following locations: For DC only : DDC file - syn/final/db/<top design>.ddc For DC & FC : Netlist - syn/final/db/<top design>.v[hd] Reports - syn/final/report/ If you choose to run the synthesis yourself, or wish to review the generated scripts or constraints, these files can be found in the following locations: Launch script - syn/[scriptFile] Where the name of the 'scriptFile' can be obtained with the command: get_activity_parameter Synthesize ScriptFile Constraints - syn/constrain/script/*.[sdc/tcl] If you wish to integrate the generated scripts into your own chip-level synthesis environment, please see the 'Synthesis_API' man page for details about how to access the generated scripts from your own environment.

The attribute indicates that design needs to be synthesized separately and a DDC (when synthesized with Design Compiler) or NDM libs (when synthesized with Fusion Compiler) will be generated for the design. The generated DDC or NDM will be consumed by the top design during hierarchical synthesis flow in coreAssembler.

SynthesizeAsSubblocksysCmd

Global array variable containing paths to different system commands.

I

<u>tailcall</u>	Replace the current procedure with another command
<u>TargetConnectionBoundary</u>	Indicates that this is a TargetConnection boundry point for exported workspace components.
<u>TCBDisableUndefMacros</u>	Disable writing out undefine directives for tech cell bound in TechnologyCellBinding activity.
<u>Tcl</u>	Tool Command Language
<u>TclAuxSynthesisScript</u>	Specifies a DC Tcl script to be executed prior to specific optimization phase.
<u>TclAuxSynthesisScriptComment</u>	Specifies a comment to be issued for the correcsponding DcTclAuxSynthesisScript.
<u>TCLConfigVariable</u>	Indicates a parameter to be defined as a TCL variable by set_tclconfig_variables.
<u>TCLConfigVariableName</u>	Indicates a parameter to be defined as a TCL variable by set_tclconfig_variables. The attribute value defines the TCL variable name.
<u>TCLConfigVariableSetArgs</u>	Defines default arguments for the set_tclconfig_variables command when invoked with arguments in the context of this design.
<u>TclPhysicalConstraintFile</u>	Tcl format physical constraints to be placed on the design during synthesis.
<u>tcltest</u>	Test harness support code and utilities
<u>tclvars</u>	Variables used by Tcl
<u>TechCellMappingBind</u>	Used to store technology cell mapping bindings from specify_tech_cell_mapping.
<u>TechCellMappingDefn</u>	Used to store technology cell mapping definitions from define_tech_cell_mapping.
<u>TechnologyCellBinding</u>	This activity implements the technology cell mapping functionality.
<u>TechSetup</u>	This activity gets variable settings from DesignCompiler, and extracts technology specific information from

coreTools Command Reference Index

<u>tell</u>	the specified technology libraries. Return current access position for an open channel
<u>TestabilityClockSignal</u>	Specifies the clock to be used for control or observe registers.
<u>TestabilityClockType</u>	Specifies the clock type to be used for testability enhancements. Selects the control signal to be used for test points. The signal should be predefined as having the TestMode signal type. If no control signal is specified, the test points will be controlled by any signal with the TestMode signal type. If no TestMode signal is found, DFT Compiler will create a new TestMode port.
<u>TestabilityControlSignal</u>	Specifies the maximum percentage of area overhead due to the observe test point structure insertion in the design. Setting this attribute to 0 leaves this as unspecified in DFT Compiler.
<u>TestabilityMaxArea</u>	Testability method to be used. Note that the tdvr and observe methods are identical. The tdvr setting is for backward compatibility.
<u>TestabilityMethod</u>	Indicates that the given field is testable in a simple register test, provided the specified constraints are met.
<u>Testable</u>	This clock is used in test mode.
<u>TestClock</u>	Cycle time for clock in test mode.
<u>TestClockCycleTime</u>	Default cycle time for test clocks.
<u>TestClockDefaultCycleTime</u>	When true, dedicated test clocks are created according to different system clocks. Thus, scan cells that have different system clocks have different test clocks. When false, the default, test clocks are created without considering system clocks.
<u>TestClocksFollowSystemDomains</u>	Waveform for clock in test mode.
<u>TestClockWaveform</u>	Causes set_test_isolate to be issued for a port. This functionality is not supported in XG mode.
<u>TestIsolate</u>	Adds logic structure (and gate) for the power saving on the XOR tree
<u>TestpointPowerSavingOn</u>	
<u>TestsuiteSetup</u>	

coreTools Command Reference Index

<u>throw</u>	Specify testsuite filegroups and parameters.
<u>time</u>	Generate a machine-readable error
<u>timerate</u>	Time the execution of a script
<u>timingException</u>	Calibrated performance measurements of script execution time
<u>TimingExceptionCmd</u>	Item used to model a timing exception command
<u>TimingExceptionFall fromValue</u>	Command associated with this timing exception
<u>TimingExceptionFall throughValue</u>	List of path startpoints. The path must fall from objects specified
<u>TimingExceptionFall toValue</u>	List of path through points. Applied to paths with a falling transition at specified objects
<u>TimingExceptionFromValue</u>	List of path endpoints. Applies to paths falling at the endpoint
<u>TimingExceptionGroup path</u>	List of path startpoints
<u>TimingExceptionOptions</u>	Name of group for paths
<u>TimingExceptionRise fromValue</u>	Options associated with this timing exception
<u>TimingExceptionRise throughValue</u>	List of path startpoints. The path must rise from objects specified
<u>TimingExceptionRise toValue</u>	List of path through points. Applies to paths with a rising transition at specified objects
<u>TimingExceptionThroughValue</u>	List of path endpoints. Applies to paths rising at the endpoint
<u>TimingExceptionToValue</u>	List of path through points
<u>TimingExceptionValue</u>	List of path endpoints
<u>tm</u>	Value for timing exception command
<u>TMAX Reference Methodology</u>	Facilities for locating and loading of Tcl Modules
<u>tmax shellVariable</u>	This flow implements the TestMAX Reference Methodology. For details refer to the TMAX Reference Methodology.
<u>tmax shellVariableComment</u>	Sets application options to the specified value on the specified block. Settings to be used for Synopsys TestMAX (tmax). The app variables can only be applied at setup and report generation.
	Add a comment for the application options being introduced in the tool intent.

coreTools Command Reference Index

<u>TmmAuxScript</u>	Specifies a TestMaxManager script to be executed prior to specific phase.
<u>TmmAuxScriptComment</u>	Specifies a comment to be issued for the corresponding TmmAuxScript. The attribute is used for storing the updated values for the Test Max Manager Reference Methodology variables, which will then be updated in the Reference Methodology scripts if the strategy parameter EnableTestMaxManagerFlow is set to 1
<u>TMMRMVariable</u>	Sets application options to the specified value on the specified block. Settings to be used for TestMax Manager (tmm_shell).
<u>tmm_shellAppOption</u>	Add a comment for the application options being introduced in the tool intent.
<u>tmm_shellAppOptionComment</u>	Specifies a TMR intent script to be executed post elaboration.
<u>TMRIntentFile</u>	Specifies a comment to be issued for the corresponding TMR Intent file exported in the tool intent.
<u>TMRIntentFileComment</u>	Monitor variable accesses, command usages and command executions
<u>trace</u>	command handler API of channel transforms
<u>transchan</u>	This command takes a connected interface (with SpecifySubsystem completed) and removes the auto connections of the interfaces and replaces them with manual connections all without incompleting SpecifySubsystem.
<u>translate interface connection</u>	Translate a specified design from HDL files to IP-XACT.
<u>translate netlist</u>	Translate Hierarchical Pin SDC constraints to be compatible with DW IP Solution flow
<u>translate phy constraints</u>	Traverse a component boundary returning the above/below equivalent connection item.
<u>traverse boundary</u>	Traverse a component connections for the connection object.
<u>traverse connection</u>	Trap and process errors and exceptions
<u>try</u>	Generic class of SV typedef for the item: type, enum, struct, union.
<u>TypedefType</u>	

coreTools Command Reference Index

TypeName

The type of an item

U

<u>unalias</u>	Removes one or more aliases.
<u>UnconnectedPort</u>	Indicates that an output port remains unconnected.
<u>unconnect interface</u>	Unconnect interfaces
<u>UndefinedBitValue</u>	Indicates the default reset value for undefined bits in a register.
<u>Underconstrain</u>	Percentage by which I/O delay constraints should be relaxed during initial mapping.
<u>UnelaboratedName</u>	the unelaborated name for this design
<u>ungroup component</u>	Ungroup the hierarchical component into current level.
<u>ungroup filegroup</u>	ungroup a filegroup
<u>union collection</u>	Combine two collections with set union semantics.
<u>UniqueConsumerConnections</u>	Show each consumer connection for this provider independently on schematic.
<u>unknown</u>	Handle attempts to use non-existent commands
<u>unload</u>	Unload machine code
<u>unload autoload filegroup</u>	Removes loaded files from an autoloading filegroup
<u>unload component memory maps</u>	Unloads all non-visible component memory maps from knowledgebase files in cC and cA from memory or a specific one if -map option is used.
<u>unload file from kb</u>	Write the contents from a file item to the host system.
<u>unload plugin</u>	Unload a plugin KB.
<u>unset</u>	Delete variables
<u>unsetenv</u>	Removes a system environment variable.
<u>unsuppress message</u>	Enables printing of one or more suppressed informational or suppressed warning messages.
<u>unused interface instance</u>	Returns 1 if the given interface instance is not being used.
<u>update</u>	Process pending events and idle callbacks
<u>update dut</u>	Update the DUT to the latest version if possible.
<u>update workspace links</u>	Change symbolic links in a workspace
<u>UPPFile</u>	

coreTools Command Reference Index

	Specifies a UPF file to load with load_upf in Design Compiler.
<u>uplevel</u>	Execute a script in a different stack frame
<u>upvar</u>	Create link to variable in a different stack frame
<u>UsedOnInstance</u>	Specifies condition whether the interface port or parameter is used in the parent interface instance. The condition can include \@\'s in the attribute value, which makes the interface usage conditional.
<u>UserPortName</u>	Indicates a port name specified with create_connection -port to use with merge_ports.
<u>Utilization</u>	Indicates how utilized the given interface is.
<u>UVMRALAccessType</u>	This is a read-only attribute. This stores UVM Ral Attributes based on MemoryAccess ReadAction, WriteBehaviour. Please refer to UVM 1.1 users guide section 5.5.1.4. Note : As per IP-Xact mapping table, the User-defined fields will be se as rw as UVM RALF format does not support any appropriate field type
<u>UVMTestText</u>	This feature provides the user ability to add user specific steps in the UVM Test phases of RAL tests in the UVM RAL Subsystem flow. The user can configure, drive and control signals in any core by using this feature. Not all phases of the RAL test can be modified, the most important ones are available for modification.
 <u>V</u>	
<u>Value</u>	Value of this parameter
<u>ValueRequired</u>	Value is required for this parameter
<u>variable</u>	create and initialize a namespace variable
<u>verbose messages</u>	Enables verbose messaging mode.
<u>VerifyIntent</u>	This activity attempts to flag common mistakes that are made when specifying constraints.
<u>verify dwf</u>	Verify the DesignWare Foundation version.
<u>verify environment</u>	Verify the user's environment using the EnvCheck attribute.
<u>verify tool</u>	Verify the proper environment variable (if it has one; otherwise, verify that the tool's

coreTools Command Reference Index

	executable is in PATH), version and licensing for a tool.
<u>VerilogHeaderValue</u>	Defines the value for the Verilog header file for the given attribute on this register or register field.
<u>Version</u>	The version of the Bill-of-Materials template.
<u>VHDLDesignLibrary</u>	VHDL library for source code generated by coreAssembler
<u>VhdlHeaderValue</u>	Defines the value for the VHDL header file for the given attribute on this register or register field.
<u>VipConnection</u>	Indicates the name of the interface to be connected to. If two values are returned, indicates an alternate instance to be connected to. Can access global variable \::iip_instance_name via formulas if needed.
<u>VipInitializationCall</u>	Indicates the string required to be inserted into the testbench to initialize the instantiated VIP component.
<u>VipModelName</u>	Specifies the name of the vmm vip model
<u>VipPackage</u>	Specifies the name of the vmm vip package
<u>VipRandomizable</u>	Specifies if the specified VMM variable is randomizable
<u>VipRandomRange</u>	Specifies the VMM variable's range of possible settings
<u>VipReasonableRandomRange</u>	Specifies the VMM variable's range of reasonable settings
<u>VipScope</u>	Specifies the scope of the VIP configuration field
<u>Visible</u>	Make this parameter visible in the GUI
<u>visible fields</u>	Formula that can be used on a register to determine if there are visible fields.
<u>VolatileMemory</u>	True if this item refers to volatile memory
<u>vwait</u>	Process events until a variable is written

W

<u>wait activities on grid</u>	Wait for all or a specified set of activities started with launch_activities_on_grid to complete.
<u>wait for activity subproc</u>	Wait for an activity sub-process started with launch_activity_subproc to complete.
<u>Waveform</u>	clock waveform
<u>which</u>	

coreTools Command Reference Index

	Locates a file and displays its pathname.
<u>while</u>	Execute script repeatedly as long as a condition is met
<u>WireLoad</u>	Wireload model to use for a design.
<u>WireLoadGroup</u>	Wireload model selection group to use for automatic wireload model selection.
<u>WireLoadMinBlockSize</u>	Minimum block size to use for automatic wireload selection.
<u>WireLoadMode</u>	Indicates the wireload mode for the design.
<u>WrapBlockIndividually</u>	Causes test wrapper insertion to be performed on this design.
<u>WrapperDefaultSafeState</u>	Specifies the default safe state for the wrapper cells.
<u>WrapperExcludePorts</u>	List of ports that should not be wrapped.
	If the option is set to swap registers connected to ports of the design are replaced with equivalent shared wrapper cells. If the option is set to in_place, additional glue logic will be added to registers attached to ports of the design to reuse the registers in core wrapper chains.
<u>WrapperRegisterImplementation</u>	If the option is set to TRUE, shared wrapper cells will be used for register bound ports. The registers are reused in core wrapper chains.
<u>WrapperUseRegisterIO</u>	Causes test wrapper insertion to be performed on the subdesigns of this design.
<u>WrapSubblocksIndividually</u>	Indicates that writing the given field will cause a write of a value other than the one written. The written value will be modified as described by the value of this attribute.
<u>WriteBehavior</u>	Indicates user-defined writing action when WriteBehaviour is set to 'modify'. It implies this write action is not covered by any of other allowed WriteBehavior values which are: {oneToClear, oneToSet, oneToToggle, zeroToClear, zeroToSet, zeroToToggle, clear, set}. This attribute is only valid when WriteBehavior is set to 'modify'
<u>WriteBehaviorModifier</u>	

coreTools Command Reference Index

<u>WriteComponentInHDL</u>	Set to false if this component should not be written in the final netlist written by coreAssembler.
<u>WriteConstraint</u>	Defines a constraint to be met when writing the given register field.
<u>write_app_var</u>	Writes a script to set the current variable values.
<u>write_attribute table</u>	Write an attribute table file.
<u>write_automotive synthesis files</u>	Export the Automotive Synthesis files
<u>write_batch script</u>	Generate a script to recreate this workspace
<u>write_component register headers</u>	Write Component Register Headers files for component
<u>write_config tar file</u>	Write out a coreConsultant configuration tar file.
<u>write_failure modes file</u>	To export SGDC files
<u>write_ipxact abstractor</u>	Export an IP-XACT description of the current abstractor.
<u>write_ipxact component</u>	Export an IP-XACT description of the current component.
<u>write_ipxact design</u>	Export an IP-XACT description of the current design.
<u>write_ipxact designConfiguration</u>	Export an IP-XACT design configuration file.
<u>write_ipxact typeDefinitions</u>	Export an IP-XACT description of the current typeDefinition.
<u>write_ParameterInfo</u>	Write the configuration intent for a set of parameters out in the format of a template for use as a starting point when generating ParameterInfo.
<u>write_pin connection table</u>	Write a pin connection table file.
<u>write_ral file</u>	Write RAL (Register Abstraction Layer) file for component
<u>write_sdc</u>	Writes out a script in Synopsys Design Constraints (SDC) format.
<u>write_subsystem table</u>	Write a subsystem table file.

X

<u>xml_translate</u>	Export an IP-XACT description of the current component.
<u>xml_verify</u>	Verify IP-XACT syntax and semantics.

Y Z

zlib

compression and decompression operations

[API](#) [Command](#)
[Index](#) [Index](#)

NAME

API_coreAssembler

Defines the application programming interface (API) for coreAssembler.

DESCRIPTION

This man page provides an indexed listing of the API for coreAssembler. The API is the set of all user-level commands, attributes, variables, and item types. The first section below lists the general groups of [coreAssembler](#) and the commands that you use to accomplish those tasks. The second section below lists the support item types and the attributes that apply to each item type.

Tasks and Commands Index

Activity Related Tasks:

[Activity Reporting](#)

[Completing and Re-Completing
Activities](#)

[Getting/Setting Activity
Parameters](#)

[Report and View Generation](#)

Constraints/Synthesis Intent Specification:

[Finding design objects](#)

[Getting and setting
attributes](#)

[Intent specification
formulas/commands](#)

[Synopsys Design Constraints](#)

[Working with timing exceptions](#)

Design Object Related:

[Designs](#)

IPXACT Features:

[Generators](#) [Import/Export](#)

SPIRIT Features:

[Generators](#)

Working with Plug-ins:

<u>Building and loading plug-ins</u>	<u>Common plug-in utility functions</u>
<u>Creating custom activities</u>	<u>Environment Verification</u>

Workspace Related Tasks:

<u>Closing a workspace</u>	<u>Creating a new workspace</u>
<u>Dumping workspace contents</u>	<u>Installing a coreKit</u>
<u>Opening a workspace</u>	<u>Re initializing a component</u>
<u>Re initializing a workspace</u>	<u>Saving a workspace</u>
<u>Updating a workspace</u>	<u>Workspace related reporting</u>

Miscellaneous:

<u>Area Estimation</u>	<u>Automatic IP Checking</u>
<u>Collection Manipulation</u>	<u>External Tool Setup</u>
<u>Commands</u>	<u>HDL Configuration</u>
<u>Generating a Reports</u>	<u>HTML Generation</u>
<u>Index Web Page</u>	<u>MemoryMap Specification</u>
<u>HDL Text Substitution</u>	<u>Parameter Configuration</u>
<u>Memory Maps</u>	<u>Structure Control</u>
<u>Miscellaneous</u>	<u>Set HDL Naming Style</u>
<u>Parameters</u>	<u>Subsystem Assembly</u>
<u>Strategy Specification</u>	<u>Synthesis API</u>
<u>Support</u>	<u>Technology Cell Mapping</u>
<u>Table Related</u>	
<u>Verification Related</u>	

Item Types and Attributes Index

Assembly:

<u>Port Creation</u>	<u>Ports</u>
<u>Subsystem</u>	
<u>Assembly</u>	

Bill of Materials:

<u>Files and file groups</u>

Design Object Related:

[Bus](#)
[Bits](#) [Cells](#)
[Clocks](#) [Designs](#)
[Nets](#) [Pins](#)
[Ports](#) [Power](#)
 [Intent](#)

Interfaces:

[Interface Definitions](#) [Interface Instances](#)
[Interface Parameters](#) [Interface Ports](#)

Port Object Related:

[Power Intent](#)

Spirit related:

[Spirit integration](#)

Testbench Generation:

[Miscellaneous](#) [VIP Instantiation](#)

Working with Plug-ins:

[Environment Verification](#)

Miscellaneous:

Activities	Activity Specification
Backward Compatibility	Documentation Generation
Generating a Reports Index	Knowledge Database
Web Page	Miscellaneous Attributes and
MemoryMap Specification	Variables
Parameters	Ports
Subsystem Assembly	Synthesis Strategies
Technology Cell Mapping	Timing Exceptions

TASKS AND ASSOCIATED COMMANDS

: [\[index\]](#)

- [get_bit_driver](#)
- [get_bit_loads](#)

Activity Reporting: [\[index\]](#)

- [report_activities](#)

Design Object Related:

- [report activity parameters](#)

Area Estimation: [index] _____

- [remove area estimates](#)
- [set area estimate](#)

Automatic IP Checking: [index] _____

- [report ip](#)

Building and loading plug-ins: [index] _____

- [add activity hook](#)
- [add instantiate component hook](#)
- [add verify subsystem hook](#)
- [add workspace hook](#)
- [create plugin kb](#)
- [current activity](#)
- [load plugin](#)
- [unload plugin](#)

Closing a workspace: [index] _____

- [close workspace](#)
- [exit](#)
- [quit](#)
- [show exit dialog](#)

Collection Manipulation Commands: [index] _____

- [add to collection](#)
- [append to collection](#)
- [as collection](#)
- [foreach in collection](#)
- [index collection](#)
- [remove from collection](#)
- [sizeof collection](#)
- [union collection](#)

Common plug-in utility functions: [index] _____

- [check file for errors](#)
- [check license](#)
- [check script](#)
- [compare dc version](#)
- [current kb](#)
- [debug script](#)
- [define proc attributes](#)
- [error info](#)
- [eval in component](#)

coreTools Command Reference Index

- [generate gtech sim model](#)
- [get VLNV search path entries](#)
- [get attribute](#)
- [get bitfield value](#)
- [get component name](#)
- [get hdl file list](#)
- [get hdl language](#)
- [get hdl library names](#)
- [get include dirs](#)
- [get installation dir](#)
- [get installed component names](#)
- [get logical dir](#)
- [get object name](#)
- [get synthesis dir](#)
- [get workspace kb](#)
- [get workspace name](#)
- [gui get setting](#)
- [gui set setting](#)
- [in autocomplete activity](#)
- [launch activity subproc](#)
- [load file into kb](#)
- [msg box](#)
- [parse proc arguments](#)
- [permanent proc](#)
- [plugin proc](#)
- [protected proc](#)
- [report activity subproc](#)
- [report license](#)
- [set attribute](#)
- [set synthesis dir](#)
- [show url](#)
- [show url external](#)
- [show user parameter dialog](#)
- [unload file from kb](#)
- [unused interface instance](#)
- [wait for activity subproc](#)
- [xml translate](#)
- [xml verify](#)

Completing and Re-Completing Activities: [index] _____

- [autocomplete activity](#)
- [prereq activities complete](#)
- [set activity incomplete](#)

Creating a new workspace: [index] _____

- [create workspace](#)
- [duplicate workspace](#)
- [set editing mode](#)
- [set workspace options](#)

Creating custom activities: [index] _____

- check parameter attribute formulas _____
- complete custom activity definition _____
- create custom activity _____
- create custom activity parameter _____
- define activity subproc params _____
- get children _____
- get parameter attribute _____
- set parameter attribute _____

Designs: [index] _____

- disable prime profile _____
- enable prime profile _____
- get prime profile info _____
- is prime profile enabled _____
- report prime profile validation expr _____
- report prime profiles _____
- report tech cell mapping _____
- set prime profile _____
- show prime profile prompt _____

Dumping workspace contents: [index] _____

- write batch script _____

Environment Verification: [index] _____

- check env vars _____
- check executables _____
- verify dwf _____
- verify environment _____
- verify tool _____

External Tool Setup: [index] _____

- get tool root _____
- set tool root _____

Finding design objects: [index] _____

- all components _____
- all designs _____
- all inputs _____
- all outputs _____
- all subdesigns _____
- current design _____
- find design _____
- find item _____
- find kb _____

- [get cell](#)
- [get clocks](#)
- [get component design](#)
- [get current component](#)
- [get top design](#)
- [get top design name](#)
- [set current component](#)

Generating a Reports Index Web Page: [index] _____

- [add html report link](#)
- [remove html report link](#)

Generators: [index] _____

- [get generator parameter](#)
- [invoke generator](#)
- [set generator parameter](#)

Getting and setting attributes: [index] _____

- [get all bits](#)
- [get cell attribute](#)
- [get clock attribute](#)
- [get connections](#)
- [get design attribute](#)
- [get port attribute](#)
- [get power domain voltage](#)
- [get supply voltage](#)
- [get upf attribute](#)
- [get upf port names](#)
- [report attribute](#)
- [set cell attribute](#)
- [set clock attribute](#)
- [set design attribute](#)
- [set port attribute](#)
- [set upf attribute](#)
- [set upf cells](#)
- [set upf supply voltage](#)
- [set upf voltage](#)

Getting/Setting Activity Parameters: [index] _____

- [get activity parameter](#)
- [set activity parameter](#)

HDL Configuration: [index] _____

- [get configuration parameter](#)
- [lock parameter](#)
- [prefix defines](#)

- [set configuration parameter](#)

HDL Text Substitution: [index] _____

- [escaped name](#)
- [get file prefix](#)
- [prefix identifiers](#)
- [prefix macros](#)

HTML Generation: [index] _____

- [docbook to html](#)

Import/Export: [index] _____

- [read ipxact file](#)
- [remove ipxact file](#)
- [translate netlist](#)
- [write ipxact abstractor](#)
- [write ipxact component](#)
- [write ipxact design](#)
- [write ipxact designConfiguration](#)
- [write ipxact typeDefinitions](#)

Installing a coreKit: [index] _____

- [batch install](#)

Intent specification formulas/commands: [index] _____

- [CombineInheritValues](#)
- [InheritValue](#)
- [combineValues](#)
- [explicit capacitance](#)
- [infinite drive](#)
- [percent of period](#)
- [pin cap of](#)
- [scale to current units](#)
- [select by class](#)
- [select by function](#)
- [select by name](#)

Memory Maps: [index] _____

- [propagate memory map](#)
- [report memory maps](#)
- [report suppressed ral tests](#)

MemoryMap Specification: [index] _____

- [create address bank](#)

coreTools Command Reference Index

- create address block
- create address space
- create memory map
- create memory mode
- create register
- create register array
- create register field
- create register field value
- get address bank attribute
- get address block attribute
- get memory map attribute
- get memory mode
- get register array attribute
- get register attribute
- get register field attribute
- get register field value attribute
- get slave base address
- invoke ralgen
- load component memory maps
- remove address bank
- remove address block
- remove address space
- remove memory map
- remove register
- remove register array
- remove register field
- remove register field value
- set address bank attribute
- set address block attribute
- set memory map attribute
- set memory mode
- set register array attribute
- set register attribute
- set register field attribute
- set register field value attribute
- set slave base address
- unload component memory maps
- visible fields
- write component register headers
- write ral file

Miscellaneous: [index] _____

- alias
- apropos
- byte compile file
- date
- echo
- eval ipxact expr
- get sv expr
- get tcl expr

coreTools Command Reference Index

- [get tool preference](#)
- [getenv](#)
- [gui source](#)
- [gui start](#)
- [gui stop](#)
- [help](#)
- [history](#)
- [ls](#)
- [printenv](#)
- [printvar](#)
- [redirect](#)
- [set tool preference](#)
- [setenv](#)
- [sh](#)
- [show color dialog](#)
- [source](#)
- [unalias](#)

Opening a workspace: [index] _____

- [open workspace](#)

Parameter Configuration Structure Control: [index] _____

- [check ParameterInfo](#)
- [write ParameterInfo](#)

Parameters: [index] _____

- [set tclconfig variables](#)

Re initializing a component: [index] _____

- [reinitialize component](#)

Re initializing a workspace: [index] _____

- [reinitialize workspace](#)

Report and View Generation: [index] _____

- [generate reports](#)
- [generate views](#)
- [get launch activities status](#)
- [launch activities on grid](#)
- [wait activities on grid](#)

Saving a workspace: [index] _____

- [save workspace](#)

Set HDL Naming Style: [index] _____

- get template naming style _____
- get template parameter style _____
- get template separator style _____
- set template naming style _____
- set template parameter style _____
- set template separator style _____

Strategy Specification: [index] _____

- get strategy parameter _____
- set strategy parameter _____

Subsystem Assembly: [index] _____

- attach interface _____
- check component for parameter loops _____
- clone component _____
- connect interface _____
- connect or export unconnected pins _____
- copy configuration _____
- copy target interface parameters _____
- create connection _____
- create hierarchical component _____
- create subsystem parameter _____
- define lib clocks _____
- define solution _____
- define split interface _____
- detach interface _____
- duplicate component _____
- encrypt solution _____
- export all interfaces _____
- export interface _____
- export open pins _____
- export pin _____
- export port descriptions _____
- export unconnected interfaces _____
- export unconnected pins _____
- export workspace as component _____
- find interface instances _____
- get component view _____
- get hierarchies _____
- get instance parameter value _____
- get interface link _____
- get interface parameter _____
- get solution data _____
- get solution file _____
- get solution root _____
- get solution variable _____
- group components _____

coreTools Command Reference Index

- import component
- import workspace as component
- instances of
- instantiate component
- instantiate dut
- instantiate interface
- instantiate solution
- load interface definitions
- load solution definition
- merge ports
- move components
- process solution files
- read component constraints
- register netlister
- remove attached interface
- remove component
- remove connection
- remove exported interface
- rename component
- rename exported interface
- rename port
- replace component
- replace interface
- replay connections
- report connections
- report solutions
- restructure workspace
- set component view
- set dependent controls
- set instance parameter
- set instance wire names
- set interface parameter
- set solution preference
- set unused interface
- source solution script
- translate interface connection
- traverse boundary
- traverse connection
- unconnect interface
- ungroup component
- ungroup filegroup
- update dut

Support: [index]_____

- build debug tarfile

Synopsys Design Constraints: [index] _____

- compress sdc
- get TMR separation distance

- [read sdc](#)
- [remove allow paths](#)
- [remove constraints](#)
- [translate phy constraints](#)
- [write automotive synthesis files](#)
- [write sdc](#)

Synthesis API: [index] _____

- [Synthesis API](#)

Table Related: [index] _____

- [read attribute table](#)
- [read parameter info table](#)
- [read parameter table](#)
- [read pin connection table](#)
- [read subsystem table](#)
- [write attribute table](#)
- [write pin connection table](#)
- [write subsystem table](#)

Technology Cell Mapping: [index] _____

- [bind tech cell mapping](#)
- [create custom tech cell](#)

Updating a workspace: [index] _____

- [update workspace links](#)

Verification Related: [index] _____

- [generate simulation file list](#)
- [generate simulation launch command](#)

Working with timing exceptions: [index] _____

- [report timing exceptions](#)
- [reset path](#)
- [set disable timing](#)
- [set false path](#)
- [set max delay](#)
- [set min delay](#)
- [set multicycle path](#)

Workspace related reporting: [index] _____

- [eval in component](#)
- [get workspace kb](#)
- [get workspace license info](#)

- get workspace name
- list kb

ITEM TYPES AND ASSOCIATED ATTRIBUTES

Activities: [index] _____

- Activity

Activity Specification: [index] _____

- BuildSubsystemcoreKit
- CompleteConnections
- CreateSubsystemTemplate
- CreateTestbenchWorkspace
- ExploreRTLArchitecture
- GenerateSubsystemRTL
- InitializeSubsystemConstraints
- MemoryMapSpecification
- RTLStaticChecking
- SimulateSubsystemTestbench
- SpecifySubsystem
- StaticChecking

Backward Compatibility: [index] _____

- cA supports hierarchy

Bus Bits: [index] _____

- busBit
- maximum bit blast size

Cells: [index] _____

- CopyToTemplate
- LockedInTemplate
- cell

Clocks: [index] _____

- ClockFallLatency
- ClockGatingFallHoldCheck
- ClockGatingFallSetupCheck
- ClockGatingRiseHoldCheck
- ClockGatingRiseSetupCheck
- ClockHoldUncertainty
- ClockRiseLatency

coreTools Command Reference Index

- ClockSetupUncertainty
- ClockSourceFallLatency
- ClockSourceRiseLatency
- CreateSkewGroup
- CycleTime
- FixHold
- InterClockHoldFallFallUncertainty
- InterClockHoldFallRiseUncertainty
- InterClockHoldRiseFallUncertainty
- InterClockHoldRiseRiseUncertainty
- InterClockSetupFallFallUncertainty
- InterClockSetupFallRiseUncertainty
- InterClockSetupRiseFallUncertainty
- InterClockSetupRiseRiseUncertainty
- IsCCDTarget
- MaxFallTransitionDelay
- MaxRiseTransitionDelay
- MinFallTransitionDelay
- MinRiseTransitionDelay
- ReferenceClock
- ReferenceClockName
- TestClock
- TestClockCycleTime
- TestClockWaveform
- Waveform
- clock

Designs: [index]_____

- AddLockUpLatch
- AreaEstimate
- AtpgTclAuxScript
- AtpgTclAuxScriptComment
- AutoConnectComplete
- AutoFixAsync
- AutoFixAsyncLogicGate
- AutoFixAsyncTestData
- AutoFixAsyncWithScanEnable
- AutoFixBidirectional
- AutoFixClockTestData
- AutoFixClocks
- BalanceBistSegments
- BidirectionalMode
- BistAutoFixBusses
- BistAutoFixXprop
- BistBlockIndividually
- BistChainCount
- BistCodecCount
- BistDiagOutputs
- BistInvertPrpgClock
- BistMaxChainLength

- BistMode
- BistObserveOutputs
- BistPrpgLength
- BistPrpgShadowSi
- BistSelectorShadowSi
- BistSubblocksIndividually
- BistType
- BistUseTristateMux
- BitRange
- CTLModels
- ClockGatingSignals
- ClockMixing
- ControlPointsPerScanCell
- CriticalRange
- CriticalRangeCoveringViolators
- CustomizedTestbenchCode
- DCLSPaths
- DedicatedScanPorts
- DedicatedWrapperCell
- DefaultHdlValue
- DefaultParamRange
- DefineLibClocks
- EnableDftAutoFix
- EnableDftShadowLogic
- EnableScanCompression
- ExcludeLibCells
- ExternalTristates
- FCExcludeLibCells
- FCIncludeLibCells
- FcAuxSynthesisScript
- FcAuxSynthesisScriptComment
- FormalVerificationAuxScript
- FormalVerificationAuxScriptComment
- FpgaPreferTmg
- HasLibertyModel
- HierarchicalIsolation
- IPEnvList
- IdentifyShiftRegisters
- IncludeLibCells
- InsertEndOfChainLockupLatch
- InsertTestPoints
- InternalTristates
- MacroPrefixFileList
- MapBlockIndividually
- MapSubblocksIndividually
- MaxArea
- MaxControlPoints
- MaxObservePoints
- MaximumScanChainLength
- MessageLimitDC
- MessageLimitFC

coreTools Command Reference Index

- [MessageStopOnDC](#)
- [MessageStopOnFC](#)
- [NumberOfScanChains](#)
- [NumberOfScanCompressionChains](#)
- [ObservePointsPerScanCell](#)
- [OneWrapperClock](#)
- [OperatingConditionsBest](#)
- [OperatingConditionsWorst](#)
- [OptimizationPriorities](#)
- [OptimizeArithmetic](#)
- [Overconstrain](#)
- [ParallelJobCpuLimit](#)
- [ParentWireLoad](#)
- [PhysicalRegion](#)
- [PreserveDesignWare](#)
- [PreserveHierarchyFromTop](#)
- [PreserveMuxOps](#)
- [PrimeProfileCurrent](#)
- [PrimeProfileDefaults](#)
- [PrimeProfileNote](#)
- [PrimeProfilesValid](#)
- [RTLStaticCheckingAsSubblock](#)
- [RtlaAuxScript](#)
- [RtlaAuxScriptComment](#)
- [RunOptions](#)
- [SDCConstraints](#)
- [ScanBlockIndividually](#)
- [ScanCompressionConfiguration](#)
- [ScanCompressionTestClock](#)
- [ScanCompressionUsage](#)
- [ScanExclude](#)
- [ScanInternalClocks](#)
- [ScanJumpBuffersInverters](#)
- [ScanMethodology](#)
- [ScanReplace](#)
- [ScanStyle](#)
- [ScanSubblocksIndividually](#)
- [SeqCompressionInputs](#)
- [SeqCompressionOutputs](#)
- [ShareTestPointsAcrossHierarchy](#)
- [SharedWrapperCell](#)
- [StaticTimingAuxScript](#)
- [StaticTimingAuxScriptComment](#)
- [SynplifyAuxSynthesisScript](#)
- [SynplifyAuxSynthesisScriptComment](#)
- [SynthesizeAsSubblock](#)
- [TMMRMVariable](#)
- [TMRIntentFile](#)
- [TMRIntentFileComment](#)
- [TclAuxSynthesisScript](#)
- [TclAuxSynthesisScriptComment](#)

coreTools Command Reference Index

- TclPhysicalConstraintFile
- TestClockDefaultCycleTime
- TestClocksFollowSystemDomains
- TestabilityClockSignal
- TestabilityClockType
- TestabilityControlSignal
- TestabilityMaxArea
- TestabilityMethod
- TestpointPowerSavingOn
- TmmAuxScript
- TmmAuxScriptComment
- UPFFile
- Underconstrain
- WireLoad
- WireLoadGroup
- WireLoadMinBlockSize
- WireLoadMode
- WrapBlockIndividually
- WrapSubblocksIndividually
- WrapperDefaultSafeState
- WrapperExcludePorts
- WrapperRegisterImplementation
- WrapperUseRegisterIO
- dc_shellStopMessageIds
- dc_shellVariable
- dc_shellVariableComment
- design
- fc_shellAppOption
- fc_shellAppOptionComment
- fm_shellVariable
- fm_shellVariableComment
- hdlin_enable_upf_compatible_naming
- pt_shellVariable
- pt_shellVariableComment
- rtl_shellAppOption
- rtl_shellAppOptionComment
- tmax_shellVariable
- tmax_shellVariableComment
- tmm_shellAppOption
- tmm_shellAppOptionComment

Documentation Generation: [index] _____

- DocDefaultValue
- DocInclude
- DocLabelName
- DocOffset
- DocRangeDecoratedName
- DocShortDescription

Environment Verification: [index] _____

ITEM TYPES AND ASSOCIATED ATTRIBUTES

- [EnvCheck](#)

Files and file groups: [index] _____

- [file](#)
- [filegroup](#)
- [filegroupGroup](#)

Generating a Reports Index Web Page: [index] _____

- [coretools home page](#)

Interface Definitions: [index] _____

- [MemoryMap](#)
- [Utilization](#)

Interface Instances: [index] _____

- [Abstraction](#)
- [GlobalSlotNumber](#)
- [InterfaceLabel](#)
- [IsDanglingIntfAllowed](#)
- [LockedInTemplate](#)
- [ShowRoute](#)
- [SlotNumber](#)
- [SlotOrder](#)

Interface Parameters: [index] _____

- [AssociationFormula](#)
- [InterfaceIsUsed](#)
- [IsControlValue](#)
- [UsedOnInstance](#)
- [param](#)

Interface Ports: [index] _____

- [AllowOpenProvider](#)
- [BusAlignment](#)
- [DefaultConstantPort](#)
- [InterfaceIsUsed](#)
- [InterfaceLink](#)
- [InterfaceSize](#)
- [Optional](#)
- [OptionalAssociation](#)
- [RequiredExportDirection](#)
- [RequiredPortDirection](#)
- [UsedOnInstance](#)
- [interfacePort](#)

Knowledge Database: [index] _____

- [KbVersion](#)
- [knowledgeBase](#)

MemoryMap Specification: [index] _____

- [AddressOffset](#)
- [AddressSpaceRef](#)
- [BankAlignment](#)
- [BaseAddress](#)
- [BitAddressOffset](#)
- [BitsInLAU](#)
- [CHeaderValue](#)
- [DocAddressOffset](#)
- [DocMemoryAccess](#)
- [DocPowerDomain](#)
- [DocRegisterResetMask](#)
- [DocRegisterResetValue](#)
- [DocRegisterSize](#)
- [DocTestable](#)
- [DocVolatileMemory](#)
- [Endian](#)
- [HeaderNameFormat](#)
- [IsAddressable](#)
- [MaxWriteConstraint](#)
- [MemoryAccess](#)
- [MemoryAccessDescription](#)
- [MemoryMode](#)
- [MemoryRange](#)
- [MemoryUsage](#)
- [MemoryWidth](#)
- [MinWriteConstraint](#)
- [PingTestMask](#)
- [RALAccessType](#)
- [RALAdditionalFieldAttributeText](#)
- [RALSuppressionComment](#)
- [RalListInfo](#)
- [ReadAction](#)
- [RegisterArrayDimensions](#)
- [RegisterResetMask](#)
- [RegisterResetValue](#)
- [RegisterSize](#)
- [Reserved](#)
- [SideEffects](#)
- [Testable](#)
- [UVMRALAccessType](#)
- [UVMTestText](#)
- [UndefinedBitValue](#)
- [VerilogHeaderValue](#)
- [VhdlHeaderValue](#)

coreTools Command Reference Index

- VolatileMemory
- WriteBehavior
- WriteBehaviorModifier
- WriteConstraint
- addressBank
- addressBlock
- memMap
- refItem
- register
- registerArray
- registerField

Miscellaneous: [index] _____

- SimTieOff

Miscellaneous Attributes and Variables: [index] _____

- ComponentOfItem
- CreatedViaUngrouping
- ExtraSearchPathAppend
- ExtraSearchPathPrepend
- LicenseForOpen
- Name
- OpenAllTreeItems
- Parameters
- ProjectID
- ProjectRootDir
- ScratchInstallIndex
- TimingExceptionCmd
- TimingExceptionFall fromValue
- TimingExceptionFall throughValue
- TimingExceptionFall toValue
- TimingExceptionFromValue
- TimingExceptionGroup path
- TimingExceptionOptions
- TimingExceptionRise fromValue
- TimingExceptionRise throughValue
- TimingExceptionRise toValue
- TimingExceptionThroughValue
- TimingExceptionToValue
- TimingExceptionValue
- TypeName
- VHDLDesignLibrary
- attrDefn
- check error list
- collection print item limit
- crm file patterns
- guiBehavior
- scratch dir
- sh arch

- [sh command abbrev mode](#)
- [sh command log file](#)
- [sh continue on error](#)
- [sh dev null](#)
- [sh enable page mode](#)
- [sh executable name](#)
- [sh product version](#)
- [sh script stop severity](#)
- [sh show tcl stack on error](#)
- [sh source uses search path](#)
- [sh tcllib app dirname](#)
- [sysCmd](#)
- [verbose messages](#)

Nets: [\[index\]](#)

- [net](#)

Parameters: [\[index\]](#)

- [CanUngroup](#)
- [CanUngroupInfo](#)
- [GroupUserLabel](#)
- [HdlSourceFileNames](#)
- [HdlUnquoted](#)
- [HdlValue](#)
- [IntentFilesProcessed](#)
- [LockedInTemplate](#)
- [ShowIcons](#)
- [StaticDefineExprs](#)
- [UnelaboratedName](#)
- [param](#)

Pins: [\[index\]](#)

- [pin](#)

Port Creation: [\[index\]](#)

- [PredefinedInoutPorts](#)
- [PredefinedInputPorts](#)
- [PredefinedOutputPorts](#)
- [SolutionItemPhase](#)

Ports: [\[index\]](#)

- [Capacitance](#)
- [CaseAnalysisValue](#)
- [ClockName](#)
- [ConstantPort](#)
- [CriticalTiming](#)

- DftExistingSignalActiveState
- DftExistingSignalTestMode
- DftExistingSignalTiming
- DftExistingSignalType
- DftExistingSignalUsage
- DftSpecSignalActiveState
- DftSpecSignalTestMode
- DftSpecSignalType
- DftSpecSignalUsage
- DisableAggregation
- DontTouchNetwork
- Drive
- DrivingCell
- EndBit
- EndBitExpr
- EndBitsExpr
- FpgaPadType
- FpgaPortIsPad
- HighFanout
- IdealPort
- IfUnconnected
- InputDelay
- InputResistance
- IsBehavioral
- IsConnected
- IsExternalReset
- IsTriState
- LogicalName
- MaxCap
- MaxFallInputDelay
- MaxFallInputDelayFallingEdge
- MaxFallOutputDelay
- MaxFallOutputDelayFallingEdge
- MaxFanout
- MaxInputDelay
- MaxInputDelayFallingEdge
- MaxLoads
- MaxOutputDelay
- MaxOutputDelayFallingEdge
- MaxRiseInputDelay
- MaxRiseInputDelayFallingEdge
- MaxRiseOutputDelay
- MaxRiseOutputDelayFallingEdge
- MaxTransition
- MergeCandidate
- MinCap
- MinFallInputDelay
- MinFallInputDelayFallingEdge
- MinFallOutputDelay
- MinFallOutputDelayFallingEdge
- MinInputDelay

coreTools Command Reference Index

- [MinInputDelayFallingEdge](#)
- [MinOutputDelay](#)
- [MinOutputDelayFallingEdge](#)
- [MinRiseInputDelay](#)
- [MinRiseInputDelayFallingEdge](#)
- [MinRiseOutputDelay](#)
- [MinRiseOutputDelayFallingEdge](#)
- [OutputDelay](#)
- [PinLoadCount](#)
- [PinLoadType](#)
- [PortDirection](#)
- [PortFunction](#)
- [PortWidth](#)
- [Registered](#)
- [RegisteredBy](#)
- [ResetProperties](#)
- [SDCPropagateText](#)
- [SDCWriteCallback](#)
- [SideConstraint](#)
- [StartBit](#)
- [StartBitExpr](#)
- [StartBitsExpr](#)
- [TestIsolate](#)
- [UnconnectedPort](#)
- [UserPortName](#)
- [port](#)

Power Intent: [index] _____

- [PowerDomain](#)
- [PowerDomains](#)

Spirit integration: [index] _____

- [ConnectToExportedInstance](#)
- [SpiritContainer](#)
- [SpiritFile](#)
- [SpiritFileType](#)
- [SpiritInterfaceType](#)
- [SpiritLibrary](#)
- [SpiritName](#)
- [SpiritVendor](#)
- [SpiritVersion](#)
- [SymmetricBus](#)
- [SymmetricBusLink](#)

Subsystem Assembly: [index] _____

- [AutoConnectionPreCallback](#)
- [ImportedWorkspaceComponentFile](#)
- [InstanceClockNames](#)

coreTools Command Reference Index

- [IsWorkspaceComponent](#)
- [ReadOnlyWorkspaceComponent](#)
- [SolutionData](#)
- [SolutionEnvVarValues](#)
- [SolutionProcessed](#)
- [SplitInterfaceMembers](#)
- [SplitInterfaceName](#)
- [TargetConnectionBoundary](#)

Synthesis Strategies: [index] _____

- [Strategy](#)

Technology Cell Mapping: [index] _____

- [TechCellMappingBind](#)
- [TechCellMappingDefn](#)

Timing Exceptions: [index] _____

- [timingException](#)

VIP Instantiation: [index] _____

- [MonitoredComponent](#)
- [MonitoredInterface](#)
- [VipConnection](#)
- [VipInitializationCall](#)
- [VipModelName](#)
- [VipPackage](#)
- [VipRandomRange](#)
- [VipRandomizable](#)
- [VipReasonableRandomRange](#)
- [VipScope](#)

[API](#) [Command](#)
[Index](#) [Index](#)

NAME

API_coreBuilder

Defines the application programming interface (API) for coreBuilder.

DESCRIPTION

This man page provides an indexed listing of the API for coreBuilder. The API is the set of all user-level commands, attributes, variables, and item types. The first section below lists the general groups of [coreBuilder](#) and the commands that you use to accomplish those tasks. The second section below lists the support item types and the attributes that apply to each item type.

Tasks and Commands Index

Activity Related Tasks:

[Activity Reporting](#)

[Completing and Re-Completing
Activities](#)

[Getting/Settng Activity
Parameters](#)

[Report and View Generation](#)

Configuration:

[Parameter Randomization](#)

Constraints/Synthesis Intent Specification:

[Defining clocks](#)

[Finding design objects](#)

[Getting and setting attributes](#)

[Intent specification formulas/commands](#)

[Synopsys Design Constraints](#)

[Working with timing exceptions](#)

Design Object Related:

[Designs](#)

Working with Plug-ins:

[Building and loading plug-ins](#)

[Common plug-in utility functions](#)

[Creating custom activities](#)

[Environment Verification](#)

Workspace Related Tasks:

coreTools Command Reference Index

<u>Closing a workspace</u>	<u>Creating a new workspace</u>
<u>Dumping workspace contents</u>	<u>Opening a workspace</u>
<u>Re initializing a workspace</u>	<u>Saving a workspace</u>
<u>Updating a workspace</u>	<u>Workspace related reporting</u>

Miscellaneous:

<u>Area Estimation</u>	<u>Bill of Materials Specification</u>
<u>Changing HDL Files</u>	<u>Collection Manipulation Commands</u>
<u>External Tool Setup</u>	<u>Generating a Reports Index Web Page</u>
<u>HDL Configuration</u>	<u>HDL Text Substitution</u>
<u>HTML Generation</u>	<u>IP XACT import</u>
<u>Interface Specification</u>	<u>Memory Maps</u>
<u>MemoryMap Specification</u>	<u>Miscellaneous</u>
<u>Parameter Configuration</u>	<u>Parameters</u>
<u>Structure Control</u>	<u>Strategy Specification</u>
<u>Plugin</u>	<u>Synthesis API</u>
<u>Subsystem Assembly</u>	<u>Technology Cell Mapping</u>
<u>Table Related</u>	<u>View Specification</u>
<u>Verification Related</u>	

Item Types and Attributes Index

Assembly:

Ports

Bill of Materials:

Bill of Materials Template Files and file groups

Design Object Related:

<u>Bus</u>	<u>Cells</u>
<u>Bits</u>	
<u>Clocks</u>	<u>Designs</u>
<u>Nets</u>	<u>Pins</u>
<u>Ports</u>	<u>Power</u>
	<u>Intent</u>

Interfaces:

<u>Interface Definitions</u>	<u>Interface Instances</u>
<u>Interface Parameters</u>	<u>Interface Params</u>
<u>Interface Ports</u>	

Port Object Related:

[Power Intent](#)

Spirit related:

[Spirit integration](#)

Testbench Generation:

[Miscellaneous](#) [VIP Instantiation](#)

Working with Plug-ins:

[Environment Verification](#)

Miscellaneous:

<u>Activities</u>	<u>Activity Specification</u>
<u>Area Estimation</u>	<u>Documentation Generation</u>
<u>Generating a Reports Index</u>	<u>HDL Functions</u>
<u>Web Page</u>	<u>Knowledge Database</u>
<u>HDL Text Substitution</u>	<u>Miscellaneous Attributes and</u>
<u>MemoryMap Specification</u>	<u>Variables</u>
<u>Parameters</u>	<u>Ports</u>
<u>Synthesis Strategies</u>	<u>Technology Cell Mapping</u>
<u>Timing Exceptions</u>	

TASKS AND ASSOCIATED COMMANDS

Activity Reporting: [index] _____

- [report activities](#)
- [report activity parameters](#)

Area Estimation: [index] _____

- [remove area estimates](#)
- [set area estimate](#)

Bill of Materials Specification: [index] _____

- [add files to filegroup](#)
- [add help menu item](#)
- [analyze filegroup](#)

Interfaces:

- [check bom](#)
- [create autoloading filegroup](#)
- [create configuration parameter](#)
- [get filegroup attribute](#)
- [get filegroup parameter](#)
- [install filegroup](#)
- [load autoloading filegroup](#)
- [report bom](#)
- [report filegroup](#)
- [set filegroup attribute](#)
- [set filegroup parameter](#)
- [unload autoloading filegroup](#)

Building and loading plug-ins: [index] _____

- [add activity hook](#)
- [add instantiate component hook](#)
- [add verify subsystem hook](#)
- [add workspace hook](#)
- [create plugin kb](#)
- [current activity](#)
- [load plugin](#)
- [unload plugin](#)

Changing HDL Files: [index] _____

- [set hdl file list](#)

Closing a workspace: [index] _____

- [close workspace](#)
- [exit](#)
- [quit](#)
- [show exit dialog](#)

Collection Manipulation Commands: [index] _____

- [add to collection](#)
- [append to collection](#)
- [as collection](#)
- [foreach in collection](#)
- [index collection](#)
- [remove from collection](#)
- [sizeof collection](#)
- [union collection](#)

Common plug-in utility functions: [index] _____

- [check file for errors](#)
- [check license](#)
- [check script](#)

coreTools Command Reference Index

- [compare dc version](#)
- [current kb](#)
- [debug script](#)
- [define proc attributes](#)
- [error info](#)
- [eval in component](#)
- [generate gtech sim model](#)
- [get VLNV search path entries](#)
- [get attribute](#)
- [get bitfield value](#)
- [get component name](#)
- [get hdl file list](#)
- [get hdl language](#)
- [get hdl library names](#)
- [get include dirs](#)
- [get installation dir](#)
- [get installed component names](#)
- [get logical dir](#)
- [get object name](#)
- [get synthesis dir](#)
- [get workspace kb](#)
- [get workspace name](#)
- [gui get setting](#)
- [gui set setting](#)
- [in autocomplete activity](#)
- [launch activity subprocess](#)
- [load file into kb](#)
- [msg box](#)
- [parse proc arguments](#)
- [permanent proc](#)
- [plugin proc](#)
- [protected proc](#)
- [report activity subprocess](#)
- [report license](#)
- [set attribute](#)
- [set synthesis dir](#)
- [show url](#)
- [show url external](#)
- [show user parameter dialog](#)
- [unload file from kb](#)
- [unused interface instance](#)
- [wait for activity subprocess](#)
- [xml translate](#)
- [xml verify](#)

Completing and Re-Completing Activities: [index] _____

- [autocomplete activity](#)
- [prereq activities complete](#)
- [set activity incomplete](#)

Creating a new workspace: [index] _____

- create workspace _____
- set editing mode _____

Creating custom activities: [index] _____

- check parameter attribute formulas _____
- complete custom activity definition _____
- create custom activity _____
- create custom activity parameter _____
- define activity subproc params _____
- get children _____
- get parameter attribute _____
- set parameter attribute _____

Defining clocks: [index] _____

- create generated clock _____
- create reference clock _____
- create virtual clock _____
- remove generated clock _____
- remove virtual clock _____

Designs: [index] _____

- clear prime profile validation exprs _____
- create prime profile file _____
- disable prime profile _____
- enable prime profile _____
- get prime profile info _____
- is prime profile enabled _____
- packaged prime profile report _____
- report prime profile validation expr _____
- report prime profiles _____
- report tech cell mapping _____
- set default prime profiles _____
- set prime profile validation expr _____

Dumping workspace contents: [index] _____

- write batch script _____

Environment Verification: [index] _____

- check env vars _____
- check executables _____
- verify dwf _____
- verify environment _____
- verify tool _____

External Tool Setup: [index] _____

- [get tool root](#) _____
- [set tool root](#) _____

Finding design objects: [index] _____

- [all designs](#) _____
- [all inputs](#) _____
- [all outputs](#) _____
- [all subdesigns](#) _____
- [current design](#) _____
- [find design](#) _____
- [find item](#) _____
- [find kb](#) _____
- [get cell](#) _____
- [get clocks](#) _____
- [get component design](#) _____
- [get current component](#) _____
- [get top design](#) _____
- [get top design name](#) _____
- [set current component](#) _____

Generating a Reports Index Web Page: [index] _____

- [add html report link](#) _____
- [remove html report link](#) _____

Getting and setting attributes: [index] _____

- [get all bits](#) _____
- [get cell attribute](#) _____
- [get clock attribute](#) _____
- [get connections](#) _____
- [get design attribute](#) _____
- [get port attribute](#) _____
- [get power domain voltage](#) _____
- [get supply voltage](#) _____
- [get upf attribute](#) _____
- [get upf port names](#) _____
- [report attribute](#) _____
- [set cell attribute](#) _____
- [set clock attribute](#) _____
- [set design attribute](#) _____
- [set port attribute](#) _____
- [set upf attribute](#) _____
- [set upf cells](#) _____
- [set upf supply voltage](#) _____
- [set upf voltage](#) _____

Getting/Setting Activity Parameters: [index] _____

coreTools Command Reference Index

- [get activity parameter](#)
- [set activity parameter](#)

HDL Configuration: [\[index\]](#) _____

- [add hdl pragma](#)
- [define array field parameters](#)
- [eval param](#)
- [foreach array field parameter](#)
- [get configuration parameter](#)
- [get configuration parameter attribute](#)
- [get field parameter for array](#)
- [get hdl pragma](#)
- [get macro prefix](#)
- [lock parameter](#)
- [mpexpr](#)
- [mpformat](#)
- [reuse-pragma](#)
- [set configuration parameter](#)
- [set configuration parameter attribute](#)

HDL Text Substitution: [\[index\]](#) _____

- [IncludeIf](#)
- [ReplaceConstantParam](#)
- [ReplaceDesignParams](#)
- [ReplaceFormatParam](#)
- [ReplaceSingleBitBus](#)
- [escaped name](#)
- [get design prefix](#)
- [get file prefix](#)
- [prefix identifiers](#)
- [prefix macros](#)
- [set file prefix](#)
- [set macro prefix](#)

HTML Generation: [\[index\]](#) _____

- [docbook to html](#)

IP XACT import: [\[index\]](#) _____

- [import ipxact data](#)

Intent specification formulas/commands: [\[index\]](#) _____

- [CombineInheritValues](#)
- [InheritValue](#)
- [combineValues](#)
- [explicit capacitance](#)
- [infinite drive](#)

- percent of period
- pin cap of
- scale to current units
- select by class
- select by function
- select by name

Interface Specification: [index] _____

- complete interface definition
- create interface
- create interface instance
- create interface parameter
- create interface port
- get associated instance parameter
- get interface attribute
- get interface parameter attribute
- get interface port attribute
- get value from interface
- remove interface
- remove interface instance
- report interface instances
- report interfaces
- set interface attribute
- set interface parameter attribute
- set interface port attribute
- show spreadsheet for connections

Memory Maps: [index] _____

- report memory maps
- report suppressed ral tests

MemoryMap Specification: [index] _____

- create address bank
- create address block
- create address space
- create memory map
- create memory mode
- create register
- create register array
- create register field
- create register field value
- get address bank attribute
- get address block attribute
- get memory map attribute
- get memory mode
- get register array attribute
- get register attribute
- get register field attribute

coreTools Command Reference Index

- get register field value attribute
- invoke ralgen
- load component memory maps
- remove address bank
- remove address block
- remove address space
- remove memory map
- remove register
- remove register array
- remove register field
- remove register field value
- set address bank attribute
- set address block attribute
- set memory map attribute
- set memory mode
- set register array attribute
- set register attribute
- set register field attribute
- set register field value attribute
- unload component memory maps
- visible fields
- write component register headers
- write ral file

Miscellaneous: [index] _____

- alias
- apropos
- byte compile file
- date
- echo
- eval ipxact expr
- get sv expr
- get tcl expr
- get tool preference
- getenv
- gui source
- gui start
- gui stop
- help
- history
- ls
- printenv
- printvar
- redirect
- set tool preference
- setenv
- sh
- show color dialog
- source
- unalias

Opening a workspace: [index] _____

- open_workspace

Parameter Configuration Structure Control: [index] _____

- check_ParameterInfo
- write_ParameterInfo

Parameter Randomization: [index] _____

- rand_proc

Parameters: [index] _____

- set_tclconfig_variables

Plugin: [index] _____

- define_report

Re initializing a workspace: [index] _____

- reinitialize_workspace

Report and View Generation: [index] _____

- generate_reports
- generate_views

Saving a workspace: [index] _____

- save_workspace

Strategy Specification: [index] _____

- get_strategy_parameter
- set_strategy_parameter

Subsystem Assembly: [index] _____

- set_dependent_controls

Synopsys Design Constraints: [index] _____

- compress_sdc
- get_TMR_separation_distance
- read_sdc
- remove_allow_paths
- remove_constraints
- translate_phy_constraints

coreTools Command Reference Index

- write automotive synthesis files
- write sdc

Synthesis API: [index] _____

- Synthesis API

Table Related: [index] _____

- read attribute table
- read parameter info table
- read parameter table
- write attribute table

Technology Cell Mapping: [index] _____

- define tech cell mapping

Updating a workspace: [index] _____

- update workspace links

Verification Related: [index] _____

- generate simulation file list
- generate simulation launch command

View Specification: [index] _____

- create component view
- remove component view

Working with timing exceptions: [index] _____

- report timing exceptions
- reset path
- set disable timing
- set false path
- set max delay
- set min delay
- set multicycle path

Workspace related reporting: [index] _____

- eval in component
- get workspace kb
- get workspace license info
- get workspace name
- list kb

ITEM TYPES AND ASSOCIATED ATTRIBUTES

Activities: [index] _____

- [Activity](#)_____
- [ActivityGroup](#)_____
- [IsComplete](#)_____
- [IsEnabled](#)_____
- [ParameterCheckCmd](#)_____
- [ParameterInfo](#)_____

Activity Specification: [index] _____

- [AddInterfaces](#)_____
- [BuildIPXACTComponent](#)_____
- [BuildcoreKit](#)_____
- [CreateBillOfMaterials](#)_____
- [CreateIntegrationWorkspaces](#)_____
- [DefineComponentViews](#)_____
- [DefineMemoryMaps](#)_____
- [DesignConfigurationIntent](#)_____
- [LoadConsultantPlugins](#)_____
- [LoadDesigns](#)_____
- [LoadInterfaceDefinitions](#)_____
- [SetUpHelpMenu](#)_____
- [SpecifyClocks](#)_____
- [SpecifyPortConstraints](#)_____
- [SpecifyPowerIntent](#)_____
- [SpecifySynthesisMethodology](#)_____
- [SpecifyTestMethododology](#)_____
- [SpecifyTimingExceptions](#)_____
- [TechSetup](#)_____
- [TestsuiteSetup](#)_____
- [VerifyIntent](#)_____

Area Estimation: [index] _____

- [AreaWeight](#)_____

Bill of Materials Template: [index] _____

- [DefaultLoadPath](#)_____
- [DeliverableType](#)_____
- [ExcludeLoadPatterns](#)_____
- [Exists](#)_____
- [Importance](#)_____
- [IsUpToDate](#)_____
- [OneRequiredGroup](#)_____
- [Version](#)_____
- [coreBuilderBomTemplateVersion](#)_____

Bus Bits: [index]_____

- busBit_____
- maximum bit blast size_____

Cells: [index]_____

- ConvertSingleBitBus_____
- DocGenerateIf_____
- GenerateIf_____
- cell_____

Clocks: [index]_____

- ClockFallLatency_____
- ClockGatingFallHoldCheck_____
- ClockGatingFallSetupCheck_____
- ClockGatingRiseHoldCheck_____
- ClockGatingRiseSetupCheck_____
- ClockHoldUncertainty_____
- ClockRiseLatency_____
- ClockSetupUncertainty_____
- ClockSourceFallLatency_____
- ClockSourceRiseLatency_____
- CreateSkewGroup_____
- CycleTime_____
- FixHold_____
- InterClockHoldFallFallUncertainty_____
- InterClockHoldFallRiseUncertainty_____
- InterClockHoldRiseFallUncertainty_____
- InterClockHoldRiseRiseUncertainty_____
- InterClockSetupFallFallUncertainty_____
- InterClockSetupFallRiseUncertainty_____
- InterClockSetupRiseFallUncertainty_____
- InterClockSetupRiseRiseUncertainty_____
- IsCCDTarget_____
- MaxFallTransitionDelay_____
- MaxRiseTransitionDelay_____
- MinFallTransitionDelay_____
- MinRiseTransitionDelay_____
- ReferenceClock_____
- ReferenceClockName_____
- TestClock_____
- TestClockCycleTime_____
- TestClockWaveform_____
- Waveform_____
- clock_____

Designs: [index]_____

- AddLockUpLatch_____

- [AreaEstimate](#)
- [AtpgTclAuxScript](#)
- [AtpgTclAuxScriptComment](#)
- [AutoConnectComplete](#)
- [AutoFixAsync](#)
- [AutoFixAsyncLogicGate](#)
- [AutoFixAsyncTestData](#)
- [AutoFixAsyncWithScanEnable](#)
- [AutoFixBidirectional](#)
- [AutoFixClockTestData](#)
- [AutoFixClocks](#)
- [BalanceBistSegments](#)
- [BidirectionalMode](#)
- [BistAutoFixBusses](#)
- [BistAutoFixXprop](#)
- [BistBlockIndividually](#)
- [BistChainCount](#)
- [BistCodecCount](#)
- [BistDiagOutputs](#)
- [BistInvertPrpgClock](#)
- [BistMaxChainLength](#)
- [BistMode](#)
- [BistObserveOutputs](#)
- [BistPrpgLength](#)
- [BistPrpgShadowSi](#)
- [BistSelectorShadowSi](#)
- [BistSubblocksIndividually](#)
- [BistType](#)
- [BistUseTristateMux](#)
- [BitRange](#)
- [CTLModels](#)
- [ClockGatingSignals](#)
- [ClockMixing](#)
- [ControlPointsPerScanCell](#)
- [CriticalRange](#)
- [CriticalRangeCoveringViolators](#)
- [CustomizedTestbenchCode](#)
- [DCLSPaths](#)
- [DedicatedScanPorts](#)
- [DedicatedWrapperCell](#)
- [DefaultHdlValue](#)
- [DefaultParamRange](#)
- [DefineLibClocks](#)
- [EnableDftAutoFix](#)
- [EnableDftShadowLogic](#)
- [EnableScanCompression](#)
- [ExcludeLibCells](#)
- [ExternalTristates](#)
- [FCExcludeLibCells](#)
- [FCIncludeLibCells](#)
- [FcAuxSynthesisScript](#)

- FcAuxSynthesisScriptComment
- FormalVerificationAuxScript
- FormalVerificationAuxScriptComment
- EpgaPreferTmg
- HasLibertyModel
- HierarchicalIsolation
- IPEnvList
- IconPath
- IdentifyShiftRegisters
- IncludeLibCells
- InsertEndOfChainLockupLatch
- InsertTestPoints
- InternalTristates
- MacroPrefixFileList
- MapBlockIndividually
- MapSubblocksIndividually
- MaxArea
- MaxControlPoints
- MaxObservePoints
- MaximumScanChainLength
- MessageLimitDC
- MessageLimitFC
- MessageStopOnDC
- MessageStopOnFC
- NumberOfScanChains
- NumberOfScanCompressionChains
- ObservePointsPerScanCell
- OneWrapperClock
- OperatingConditionsBest
- OperatingConditionsWorst
- OptimizationPriorities
- OptimizeArithmetic
- Overconstrain
- ParallelJobCpuLimit
- ParameterCheckCmd
- ParameterInfo
- ParentWireLoad
- PhysicalRegion
- PreserveDesignWare
- PreserveHierarchyFromTop
- PreserveMuxOps
- PrimeProfileAllowNone
- PrimeProfileDefaults
- PrimeProfileNote
- PrimeProfilesValid
- RTLStaticCheckingAsSubblock
- RtlaAuxScript
- RtlaAuxScriptComment
- RunOptions
- SDCConstraints
- ScanBlockIndividually

coreTools Command Reference Index

- [ScanCompressionConfiguration](#)
- [ScanCompressionTestClock](#)
- [ScanCompressionUsage](#)
- [ScanExclude](#)
- [ScanInternalClocks](#)
- [ScanJumpBuffersInverters](#)
- [ScanMethodology](#)
- [ScanReplace](#)
- [ScanStyle](#)
- [ScanSubblocksIndividually](#)
- [SeqCompressionInputs](#)
- [SeqCompressionOutputs](#)
- [ShareTestPointsAcrossHierarchy](#)
- [SharedWrapperCell](#)
- [SimulationModel](#)
- [StaticTimingAuxScript](#)
- [StaticTimingAuxScriptComment](#)
- [SymbolLibraryPath](#)
- [SymbolName](#)
- [SymbolType](#)
- [SynplifyAuxSynthesisScript](#)
- [SynplifyAuxSynthesisScriptComment](#)
- [SynthesizeAsSubblock](#)
- [TMMRMVariable](#)
- [TMRIntentFile](#)
- [TMRIntentFileComment](#)
- [TclAuxSynthesisScript](#)
- [TclAuxSynthesisScriptComment](#)
- [TestClockDefaultCycleTime](#)
- [TestClocksFollowSystemDomains](#)
- [TestabilityClockSignal](#)
- [TestabilityClockType](#)
- [TestabilityControlSignal](#)
- [TestabilityMaxArea](#)
- [TestabilityMethod](#)
- [TestpointPowerSavingOn](#)
- [TmmAuxScript](#)
- [TmmAuxScriptComment](#)
- [UPFFile](#)
- [Underconstrain](#)
- [WireLoad](#)
- [WireLoadGroup](#)
- [WireLoadMinBlockSize](#)
- [WireLoadMode](#)
- [WrapBlockIndividually](#)
- [WrapSubblocksIndividually](#)
- [WrapperDefaultSafeState](#)
- [WrapperExcludePorts](#)
- [WrapperRegisterImplementation](#)
- [WrapperUseRegisterIO](#)
- [dc_shellStopMessageIds](#)

coreTools Command Reference Index

- [dc_shellVariable](#)
- [dc_shellVariableComment](#)
- [design](#)
- [fc_shellAppOption](#)
- [fc_shellAppOptionComment](#)
- [fm_shellVariable](#)
- [fm_shellVariableComment](#)
- [hdlin_enable_upf_compatible_naming](#)
- [pt_shellVariable](#)
- [pt_shellVariableComment](#)
- [rtl_shellAppOption](#)
- [rtl_shellAppOptionComment](#)
- [tmax_shellVariable](#)
- [tmax_shellVariableComment](#)
- [tmm_shellAppOption](#)
- [tmm_shellAppOptionComment](#)

Documentation Generation: [index] _____

- [DocDefaultValue](#)
- [DocInclude](#)
- [DocLabelName](#)
- [DocOffset](#)
- [DocRangeDecoratedName](#)
- [DocShortDescription](#)

Environment Verification: [index] _____

- [EnvCheck](#)

Files and file groups: [index] _____

- [AutoloadFilegroup](#)
- [ConfigActivity](#)
- [ConfigDependsOnActivities](#)
- [ConfigDependsOnGroup](#)
- [ConfigIntentSearchPath](#)
- [Configurable](#)
- [DefaultInstallDir](#)
- [EncryptText](#)
- [FileContentType](#)
- [FilePerms](#)
- [HdlLibrary](#)
- [Install](#)
- [InstallUserWorkDir](#)
- [InstallWhen](#)
- [PostPromptCmd](#)
- [PrePromptCmd](#)
- [RemoveIfEmpty](#)
- [SubstituteInFile](#)
- [file](#)

- [filegroup](#)
- [filegroupGroup](#)

Generating a Reports Index Web Page: [\[index\]](#) _____

- [coretools home page](#)

HDL Functions: [\[index\]](#) _____

- [FunctionDefinition](#)
- [hdlFunc](#)

HDL Text Substitution: [\[index\]](#) _____

- [WriteComponentInHDL](#)

Interface Definitions: [\[index\]](#) _____

- [AutoConnectWhen](#)
- [ConnectionDialogCmd](#)
- [InterfaceLabel](#)
- [InterfaceType](#)
- [InterfaceTypeName](#)
- [MemoryMap](#)
- [ShowRoute](#)

Interface Instances: [\[index\]](#) _____

- [Abstraction](#)
- [AssociationComplete](#)
- [Bridge](#)
- [Channel](#)
- [ConnectionDialogCmd](#)
- [ConnectionRequired](#)
- [InterfaceGroup](#)
- [InterfaceLabel](#)
- [ReadOnlyInterface](#)
- [ShowRoute](#)
- [SlotNumberOffset](#)
- [SlotWidth](#)
- [SymbolPinPrefix](#)
- [SymbolPinSide](#)
- [UniqueConsumerConnections](#)
- [interfaceDefn](#)
- [interfaceInstance](#)
- [intfPin](#)
- [intfPort](#)

Interface Parameters: [\[index\]](#) _____

- [AssociationFormula](#)

- [InterfaceIsUsed](#)
- [IsControlValue](#)
- [UsedOnInstance](#)
- [param](#)

Interface Params: [index] _____

- [ParamValueFromDesign](#)

Interface Ports: [index] _____

- [AllowOpenProvider](#)
- [BusAlignment](#)
- [DefaultConstantPort](#)
- [FeedThroughConnect](#)
- [InterfaceIsUsed](#)
- [InterfaceLink](#)
- [InterfaceSize](#)
- [LogicalLeft](#)
- [LogicalRight](#)
- [Optional](#)
- [OptionalAssociation](#)
- [PhysicalLeft](#)
- [PhysicalRight](#)
- [RequiredExPortDirection](#)
- [RequiredPortDirection](#)
- [UsedOnInstance](#)
- [interfacePort](#)

Knowledge Database: [index] _____

- [KbVersion](#)
- [knowledgeBase](#)

MemoryMap Specification: [index] _____

- [AddressOffset](#)
- [AddressSpaceRef](#)
- [BankAlignment](#)
- [BaseAddress](#)
- [BitAddressOffset](#)
- [BitsInLAU](#)
- [CHeaderValue](#)
- [DocAddressOffset](#)
- [DocMemoryAccess](#)
- [DocPowerDomain](#)
- [DocRegisterResetMask](#)
- [DocRegisterResetValue](#)
- [DocRegisterSize](#)
- [DocTestable](#)
- [DocVolatileMemory](#)

coreTools Command Reference Index

- Endian
- HeaderNameFormat
- IsAddressable
- MaxWriteConstraint
- MemoryAccess
- MemoryAccessDescription
- MemoryMode
- MemoryRange
- MemoryUsage
- MemoryWidth
- MinWriteConstraint
- PingTestMask
- RALAccessType
- RALAdditionalFieldAttributeText
- RALSuppressionComment
- RalListInfo
- ReadAction
- RegisterArrayDimensions
- RegisterResetMask
- RegisterResetValue
- RegisterSize
- Reserved
- SideEffects
- Testable
- UVMRALAccessType
- UVMTestText
- UndefinedBitValue
- VerilogHeaderValue
- VhdlHeaderValue
- VolatileMemory
- WriteBehavior
- WriteBehaviorModifier
- WriteConstraint
- addressBank
- addressBlock
- memMap
- refItem
- register
- registerArray
- registerField

Miscellaneous: [index] _____

- SimTieOff

Miscellaneous Attributes and Variables: [index] _____

- CreatedViaUngrouping
- ExtraSearchPathAppend
- ExtraSearchPathPrepend
- LicenseForOpen

coreTools Command Reference Index

- Name
- OpenAllTreeItems
- Parameters
- ProjectID
- ProjectRootDir
- ScratchInstallIndex
- TimingExceptionCmd
- TimingExceptionFall fromValue
- TimingExceptionFall throughValue
- TimingExceptionFall toValue
- TimingExceptionFromValue
- TimingExceptionGroup path
- TimingExceptionOptions
- TimingExceptionRise fromValue
- TimingExceptionRise throughValue
- TimingExceptionRise toValue
- TimingExceptionThroughValue
- TimingExceptionToValue
- TimingExceptionValue
- TypeName
- attrDefn
- check error list
- collection print item limit
- crm file patterns
- guiBehavior
- scratch dir
- sh arch
- sh command abbrev mode
- sh command log file
- sh continue on error
- sh dev null
- sh enable page mode
- sh executable name
- sh product version
- sh script stop severity
- sh show tcl stack on error
- sh source uses search path
- sh tcllib app dirname
- sysCmd
- verbose messages

Nets: [index] _____

- net

Parameters: [index] _____

- AltValue
- ArrayEnd
- ArrayFieldSize
- ArrayStart

- [BitWidth](#)
- [BitsToRepresent](#)
- [CanUngroup](#)
- [CanUngroupInfo](#)
- [CheckCmd](#)
- [CheckExpr](#)
- [CheckExprMessage](#)
- [CheckExprWhenDisabled](#)
- [DefaultValue](#)
- [DefaultValueMessage](#)
- [DeferEvaluation](#)
- [Description](#)
- [DigitsPrecision](#)
- [DocBlockTableAddressOffset](#)
- [DocDescription](#)
- [DocDescriptionField](#)
- [DocEnabled](#)
- [DocGroup](#)
- [DocGroupName](#)
- [DocMaster](#)
- [DocPortWidth](#)
- [DocRegTableAddressOffset](#)
- [DocRegistered](#)
- [DocRegisteredBy](#)
- [DocSynchronousTo](#)
- [DocVisible](#)
- [DocWaiver](#)
- [Enabled](#)
- [EnumValues](#)
- [ExportDefineToFile](#)
- [GroupImage](#)
- [GroupImageAttrs](#)
- [GroupImageTitle](#)
- [GroupName](#)
- [GroupText](#)
- [GroupUserLabel](#)
- [HdlSourceFileNames](#)
- [HdlTypeName](#)
- [HdlUnquoted](#)
- [HdlValue](#)
- [HelpUrl](#)
- [IntentFilesProcessed](#)
- [IsArray](#)
- [IsFullyModeled](#)
- [IsHexVal](#)
- [Label](#)
- [MaxValue](#)
- [MinValue](#)
- [ReadActionDescription](#)
- [ReadActionModifier](#)
- [ReadOnlyParam](#)

- ReplaceInHDL
- Sequence
- SetInHex
- ShowIcons
- StaticDefineExpr
- StaticDefineExprs
- SymbolicNames
- SynchronousTo
- TCLConfigVariable
- TCLConfigVariableName
- TCLConfigVariableSetArgs
- TypedefType
- UnelaboratedName
- Value
- ValueRequired
- Visible
- param

Pins: [index]_____

- pin

Ports: [index]_____

- AutoConnectIfUnconnected
- Capacitance
- CaseAnalysisValue
- ClockName
- CnctClass
- ConstantPort
- CriticalTiming
- DftExistingSignalActiveState
- DftExistingSignalTestMode
- DftExistingSignalTiming
- DftExistingSignalType
- DftExistingSignalUsage
- DftSpecSignalActiveState
- DftSpecSignalTestMode
- DftSpecSignalType
- DftSpecSignalUsage
- DisableAggregation
- DontTouchNetwork
- Drive
- DrivingCell
- EndBit
- EndBitExpr
- EndBitsExpr
- EpgaPadType
- EpgaPortIsPad
- HighFanout
- IdealPort

- IfUnconnected
- InputDelay
- InputResistance
- IsBehavioral
- IsConnected
- IsExternalReset
- IsTriState
- LogicalName
- MaxCap
- MaxFallInputDelay
- MaxFallInputDelayFallingEdge
- MaxFallOutputDelay
- MaxFallOutputDelayFallingEdge
- MaxFanout
- MaxInputDelay
- MaxInputDelayFallingEdge
- MaxLoads
- MaxOutputDelay
- MaxOutputDelayFallingEdge
- MaxRiseInputDelay
- MaxRiseInputDelayFallingEdge
- MaxRiseOutputDelay
- MaxRiseOutputDelayFallingEdge
- MaxTransition
- MergeCandidate
- MinCap
- MinFallInputDelay
- MinFallInputDelayFallingEdge
- MinFallOutputDelay
- MinFallOutputDelayFallingEdge
- MinInputDelay
- MinInputDelayFallingEdge
- MinOutputDelay
- MinOutputDelayFallingEdge
- MinRiseInputDelay
- MinRiseInputDelayFallingEdge
- MinRiseOutputDelay
- MinRiseOutputDelayFallingEdge
- OutputDelay
- PinLoadCount
- PinLoadType
- PortDirection
- PortFunction
- PortWidth
- Registered
- RegisteredBy
- ResetProperties
- SDCPropagateText
- SDCWriteCallback
- SideConstraint
- StartBit

- [StartBitExpr](#)
- [StartBitsExpr](#)
- [TestIsolate](#)
- [UnconnectedPort](#)
- [UserPortName](#)
- [port](#)

Power Intent: [index] _____

- [PowerDomain](#)
- [PowerDomains](#)

Spirit integration: [index] _____

- [SpiritFileType](#)

Synthesis Strategies: [index] _____

- [Strategy](#)

Technology Cell Mapping: [index] _____

- [ForceTCBToUseComponentUndefFile](#)
- [InsertTechCellBindMacroAtStart](#)
- [TCBDisableUndefMacros](#)
- [TechCellMappingDefn](#)

Timing Exceptions: [index] _____

- [timingException](#)

VIP Instantiation: [index] _____

- [MonitoredComponent](#)
- [MonitoredInterface](#)
- [VipConnection](#)
- [VipInitializationCall](#)
- [VipModelName](#)
- [VipPackage](#)
- [VipRandomRange](#)
- [VipRandomizable](#)
- [VipReasonableRandomRange](#)
- [VipScope](#)

[API](#) [Command](#)
[Index](#) [Index](#)

NAME

API_coreConsultant

Defines the application programming interface (API) for coreConsultant.

DESCRIPTION

This man page provides an indexed listing of the API for coreConsultant. The API is the set of all user-level commands, attributes, variables, and item types. The first section below lists the general groups of [coreConsultant](#) and the commands that you use to accomplish those tasks. The second section below lists the support item types and the attributes that apply to each item type.

Tasks and Commands Index

Activity Related Tasks:

[Activity Reporting](#)

[Completing and Re-Completing
Activities](#)

[Getting/Setting Activity
Parameters](#)

[Report and View Generation](#)

Configuration:

[Parameter Randomization](#)

Constraints/Synthesis Intent Specification:

[Finding design objects](#)

[Getting and setting
attributes](#)

[Intent specification
formulas/commands](#)

[Synopsys Design Constraints](#)

[Working with timing exceptions](#)

Design Object Related:

[Designs](#)

Working with Plug-ins:

[Building and loading plug-ins](#)

[Common plug-in utility functions](#)

[Creating custom activities](#)

[Environment Verification](#)

Workspace Related Tasks:

<u>Closing a workspace</u>	<u>Creating a new workspace</u>
<u>Dumping workspace contents</u>	<u>Installing a coreKit</u>
<u>Opening a workspace</u>	<u>Re initializing a component</u>
<u>Re initializing a workspace</u>	<u>Saving a workspace</u>
<u>Updating a workspace</u>	<u>Workspace related reporting</u>

Miscellaneous:

<u>Automatic IP Checking</u>	<u>Collection Manipulation Commands</u>
<u>External Tool Setup</u>	<u>Generating a Reports Index Web Page</u>
<u>HDL Configuration</u>	<u>HDL Text Substitution</u>
<u>HTML Generation</u>	<u>MemoryMap Specification</u>
<u>Miscellaneous</u>	<u>Parameter Configuration Structure Control</u>
<u>Parameters</u>	<u>Prototype Commands</u>
<u>Strategy Specification</u>	<u>Subsystem Assembly</u>
<u>Support</u>	<u>Synthesis API</u>
<u>Table Related</u>	<u>Technology Cell Mapping</u>
<u>Verification Related</u>	

Item Types and Attributes Index

Bill of Materials:

Files and file groups

Design Object Related:

<u>Bus</u>	<u>Cells</u>
<u>Bits</u>	
<u>Clocks</u>	<u>Designs</u>
<u>Nets</u>	<u>Pins</u>
<u>Ports</u>	<u>Power Intent</u>

Interfaces:

<u>Interface Parameters</u>	<u>Interface Ports</u>
-----------------------------	------------------------

Port Object Related:Power Intent**Spirit related:**Spirit integration**Working with Plug-ins:**Environment Verification**Miscellaneous:**ActivitiesDocumentation GenerationKnowledge DatabaseMiscellaneous Attributes andVariablesStrategy SpecificationTechnology Cell MappingActivity SpecificationGenerating a Reports IndexWeb PageMemoryMap SpecificationParametersSynthesis StrategiesTiming Exceptions

TASKS AND ASSOCIATED COMMANDS

: [index]

- write config tar file

Activity Reporting: [index]

- report activities
- report activity parameters

Automatic IP Checking: [index]

- report ip

Building and loading plug-ins: [index]

- add activity hook
- add instantiate component hook
- add verify subsystem hook
- add workspace hook
- create plugin kb
- current activity
- load plugin
- unload plugin

Closing a workspace: [index]**Port Object Related:**

- close workspace
- exit
- quit
- show exit dialog

Collection Manipulation Commands: [index] _____

- add to collection
- append to collection
- as collection
- foreach in collection
- index collection
- remove from collection
- sizeof collection
- union collection

Common plug-in utility functions: [index] _____

- check file for errors
- check license
- check script
- check workspace project id
- compare dc version
- current kb
- debug script
- define proc attributes
- error info
- eval in component
- generate gtech sim model
- get VLNV search path entries
- get attribute
- get bitfield value
- get component name
- get hdl file list
- get hdl language
- get hdl library names
- get include dirs
- get installation dir
- get installed component names
- get logical dir
- get object name
- get synthesis dir
- get workspace kb
- get workspace name
- gui get setting
- gui set setting
- in autocomplete activity
- launch activity subproc
- load file into kb
- msg box
- parse proc arguments

- permanent proc
- plugin proc
- protected proc
- report activity subproc
- report license
- set attribute
- set synthesis dir
- show url
- show url external
- show user parameter dialog
- unload file from kb
- unused interface instance
- wait for activity subproc
- xml translate
- xml verify

Completing and Re-Completing Activities: [index] _____

- autocomplete activity
- prereq activities complete
- set activity incomplete

Creating a new workspace: [index] _____

- create workspace
- set editing mode
- set workspace options

Creating custom activities: [index] _____

- check parameter attribute formulas
- complete custom activity definition
- create custom activity
- create custom activity parameter
- define activity subproc params
- get children
- get parameter attribute
- set parameter attribute

Designs: [index] _____

- disable prime profile
- enable prime profile
- get prime profile info
- is prime profile enabled
- report prime profile validation expr
- report prime profiles
- report tech cell mapping
- set prime profile
- show prime profile prompt

Dumping workspace contents: [index] _____

- write batch script _____

Environment Verification: [index] _____

- check env vars _____
- check executables _____
- verify dwf _____
- verify environment _____
- verify tool _____

External Tool Setup: [index] _____

- get tool root _____
- set tool root _____

Finding design objects: [index] _____

- all designs _____
- all inputs _____
- all outputs _____
- all subdesigns _____
- current design _____
- find design _____
- find item _____
- find kb _____
- get cell _____
- get clocks _____
- get component design _____
- get current component _____
- get top design _____
- get top design name _____
- set current component _____

Generating a Reports Index Web Page: [index] _____

- add html report link _____
- remove html report link _____

Getting and setting attributes: [index] _____

- get all bits _____
- get cell attribute _____
- get clock attribute _____
- get connections _____
- get design attribute _____
- get port attribute _____
- get power domain voltage _____
- get supply voltage _____
- get upf attribute _____

coreTools Command Reference Index

- [get upf port names](#)
- [report attribute](#)
- [set cell attribute](#)
- [set clock attribute](#)
- [set design attribute](#)
- [set port attribute](#)
- [set upf attribute](#)
- [set upf cells](#)
- [set upf supply voltage](#)
- [set upf voltage](#)

Getting/Setting Activity Parameters: [index] _____

- [get activity parameter](#)
- [set activity parameter](#)

HDL Configuration: [index] _____

- [get configuration parameter](#)
- [lock parameter](#)
- [prefix defines](#)
- [set configuration parameter](#)
- [set design prefix](#)

HDL Text Substitution: [index] _____

- [escaped name](#)
- [get file prefix](#)
- [prefix identifiers](#)
- [prefix macros](#)
- [set file prefix](#)
- [set macro prefix](#)

HTML Generation: [index] _____

- [docbook to html](#)

Installing a coreKit: [index] _____

- [batch install](#)

Intent specification formulas/commands: [index] _____

- [CombineInheritValues](#)
- [InheritValue](#)
- [combineValues](#)
- [explicit capacitance](#)
- [infinite drive](#)
- [percent of period](#)
- [pin cap of](#)
- [scale to current units](#)

coreTools Command Reference Index

- [select by class](#)
- [select by function](#)
- [select by name](#)

MemoryMap Specification: [index] _____

- [invoke ralgen](#)
- [load component memory maps](#)
- [unload component memory maps](#)
- [write component register headers](#)
- [write ral file](#)

Miscellaneous: [index] _____

- [alias](#)
- [apropos](#)
- [byte compile file](#)
- [date](#)
- [echo](#)
- [eval ipxact expr](#)
- [get sv expr](#)
- [get tcl expr](#)
- [get tool preference](#)
- [getenv](#)
- [gui source](#)
- [gui start](#)
- [gui stop](#)
- [help](#)
- [history](#)
- [ls](#)
- [printenv](#)
- [printvar](#)
- [redirect](#)
- [set tool preference](#)
- [setenv](#)
- [sh](#)
- [show color dialog](#)
- [source](#)
- [unalias](#)

Opening a workspace: [index] _____

- [open workspace](#)

Parameter Configuration Structure Control: [index] _____

- [check ParameterInfo](#)
- [write ParameterInfo](#)

Parameter Randomization: [index] _____

- [rand_proc](#)
- [randomize_parameters](#)

Parameters: [index] _____

- [set_tclconfig_variables](#)

Prototype Commands: [index] _____

- [generate_qtm](#)

Re initializing a component: [index] _____

- [reinitialize_component](#)

Re initializing a workspace: [index] _____

- [reinitialize_workspace](#)

Report and View Generation: [index] _____

- [generate_reports](#)
- [generate_views](#)
- [get_launch_activities_status](#)
- [launch_activities_on_grid](#)
- [wait_activities_on_grid](#)

Saving a workspace: [index] _____

- [save_workspace](#)

Strategy Specification: [index] _____

- [get_strategy_parameter](#)
- [set_strategy_parameter](#)

Subsystem Assembly: [index] _____

- [set_dependent_controls](#)

Support: [index] _____

- [build_debug_tarfile](#)

Synopsys Design Constraints: [index] _____

- [compress_sdc](#)
- [get_TMR_separation_distance](#)
- [read_sdc](#)
- [remove_allow_paths](#)
- [remove_constraints](#)

coreTools Command Reference Index

- [translate_phy_constraints](#)
- [write_automotive_synthesis_files](#)
- [write_failure_modes_file](#)
- [write_sdc](#)

Synthesis API: [index] _____

- [Synthesis_API](#)

Table Related: [index] _____

- [read_attribute_table](#)
- [read_parameter_info_table](#)
- [read_parameter_table](#)
- [write_attribute_table](#)

Technology Cell Mapping: [index] _____

- [bind_tech_cell_mapping](#)
- [create_custom_tech_cell](#)

Updating a workspace: [index] _____

- [update_workspace_links](#)

Verification Related: [index] _____

- [generate_simulation_file_list](#)
- [generate_simulation_launch_command](#)

Working with timing exceptions: [index] _____

- [report_timing_exceptions](#)
- [reset_path](#)
- [set_disable_timing](#)
- [set_false_path](#)
- [set_max_delay](#)
- [set_min_delay](#)
- [set_multicycle_path](#)

Workspace related reporting: [index] _____

- [eval_in_component](#)
- [get_workspace_kb](#)
- [get_workspace_license_info](#)
- [get_workspace_name](#)
- [list_kb](#)

ITEM TYPES AND ASSOCIATED ATTRIBUTES

Activities: [index] _____

- Activity _____

Activity Specification: [index] _____

- ConstraintAnalysis _____
- ConstraintsVerification _____
- ConstraintsVerificationWithAssertions _____
- ExploreRTLArchitecture _____
- FPGASynthesis _____
- FormalVerification _____
- GenerateTestVectors _____
- SelectPrimeProfile _____
- SetDesignFileMacroPrefixes _____
- SpecifyClocks _____
- SpecifyConfiguration _____
- SpecifyOperatingConditionsAndWireLoads _____
- SpecifyPortConstraints _____
- SpecifyPowerIntent _____
- SpecifySynthesisMethodology _____
- SpecifyTestMethododology _____
- StaticTimingAnalysis _____
- Synthesize _____
- TechSetup _____
- TechnologyCellBinding _____

Bus Bits: [index] _____

- busBit _____
- maximum bit blast size _____

Cells: [index] _____

- cell _____

Clocks: [index] _____

- ClockFallLatency _____
- ClockGatingFallHoldCheck _____
- ClockGatingFallSetupCheck _____
- ClockGatingRiseHoldCheck _____
- ClockGatingRiseSetupCheck _____
- ClockHoldUncertainty _____
- ClockRiseLatency _____
- ClockSetupUncertainty _____
- ClockSourceFallLatency _____
- ClockSourceRiseLatency _____

coreTools Command Reference Index

- [CreateSkewGroup](#)
- [CycleTime](#)
- [FixHold](#)
- [InterClockHoldFallFallUncertainty](#)
- [InterClockHoldFallRiseUncertainty](#)
- [InterClockHoldRiseFallUncertainty](#)
- [InterClockHoldRiseRiseUncertainty](#)
- [InterClockSetupFallFallUncertainty](#)
- [InterClockSetupFallRiseUncertainty](#)
- [InterClockSetupRiseFallUncertainty](#)
- [InterClockSetupRiseRiseUncertainty](#)
- [IsCCDTarget](#)
- [MaxFallTransitionDelay](#)
- [MaxRiseTransitionDelay](#)
- [MinFallTransitionDelay](#)
- [MinRiseTransitionDelay](#)
- [ReferenceClock](#)
- [ReferenceClockName](#)
- [TestClock](#)
- [TestClockCycleTime](#)
- [TestClockWaveform](#)
- [Waveform](#)
- [clock](#)

Designs: [index]_____

- [AddLockUpLatch](#)
- [AreaEstimate](#)
- [AtpgTclAuxScript](#)
- [AtpgTclAuxScriptComment](#)
- [AutoConnectComplete](#)
- [AutoFixAsync](#)
- [AutoFixAsyncLogicGate](#)
- [AutoFixAsyncTestData](#)
- [AutoFixAsyncWithScanEnable](#)
- [AutoFixBidirectional](#)
- [AutoFixClockTestData](#)
- [AutoFixClocks](#)
- [BalanceBistSegments](#)
- [BidirectionalMode](#)
- [BistAutoFixBusses](#)
- [BistAutoFixXprop](#)
- [BistBlockIndividually](#)
- [BistChainCount](#)
- [BistCodecCount](#)
- [BistDiagOutputs](#)
- [BistInvertPrpgClock](#)
- [BistMaxChainLength](#)
- [BistMode](#)
- [BistObserveOutputs](#)
- [BistPrpgLength](#)

- BistPrpgShadowSi
- BistSelectorShadowSi
- BistSubblocksIndividually
- BistType
- BistUseTristateMux
- BitRange
- CTLModels
- ClockGatingSignals
- ClockMixing
- ControlPointsPerScanCell
- CriticalRange
- CriticalRangeCoveringViolators
- CustomizedTestbenchCode
- DCLSPaths
- DedicatedScanPorts
- DedicatedWrapperCell
- DefaultHdlValue
- DefaultParamRange
- DefineLibClocks
- EnableDftAutoFix
- EnableDftShadowLogic
- EnableScanCompression
- ExcludeLibCells
- ExternalTristates
- FCExcludeLibCells
- FCIncludeLibCells
- FcAuxSynthesisScript
- FcAuxSynthesisScriptComment
- FormalVerificationAuxScript
- FormalVerificationAuxScriptComment
- FpgaPreferTmg
- HasLibertyModel
- HierarchicalIsolation
- IPEnvList
- IdentifyShiftRegisters
- IncludeLibCells
- InsertEndOfChainLockupLatch
- InsertTestPoints
- InternalTristates
- MacroPrefixFileList
- MapBlockIndividually
- MapSubblocksIndividually
- MaxArea
- MaxControlPoints
- MaxObservePoints
- MaximumScanChainLength
- MessageLimitDC
- MessageLimitFC
- MessageStopOnDC
- MessageStopOnFC
- NumberOfScanChains

coreTools Command Reference Index

- NumberOfScanCompressionChains
- ObservePointsPerScanCell
- OneWrapperClock
- OperatingConditionsBest
- OperatingConditionsWorst
- OptimizationPriorities
- OptimizeArithmetic
- Overconstrain
- ParallelJobCpuLimit
- ParentWireLoad
- PhysicalRegion
- PreserveDesignWare
- PreserveHierarchyFromTop
- PreserveMuxOps
- PrimeProfileCurrent
- PrimeProfileDefaults
- PrimeProfileNote
- PrimeProfilesValid
- RTLStaticCheckingAsSubblock
- RtlaAuxScript
- RtlaAuxScriptComment
- RunOptions
- SDCConstraints
- ScanBlockIndividually
- ScanCompressionConfiguration
- ScanCompressionTestClock
- ScanCompressionUsage
- ScanExclude
- ScanInternalClocks
- ScanJumpBuffersInverters
- ScanMethodology
- ScanReplace
- ScanStyle
- ScanSubblocksIndividually
- SeqCompressionInputs
- SeqCompressionOutputs
- ShareTestPointsAcrossHierarchy
- SharedWrapperCell
- StaticTimingAuxScript
- StaticTimingAuxScriptComment
- SynplifyAuxSynthesisScript
- SynplifyAuxSynthesisScriptComment
- SynthesizeAsSubblock
- TMMRMVariable
- TMRIntentFile
- TMRIntentFileComment
- TclAuxSynthesisScript
- TclAuxSynthesisScriptComment
- TclPhysicalConstraintFile
- TestClockDefaultCycleTime
- TestClocksFollowSystemDomains

coreTools Command Reference Index

- [TestabilityClockSignal](#)
- [TestabilityClockType](#)
- [TestabilityControlSignal](#)
- [TestabilityMaxArea](#)
- [TestabilityMethod](#)
- [TestpointPowerSavingOn](#)
- [TmmAuxScript](#)
- [TmmAuxScriptComment](#)
- [UPFFile](#)
- [Underconstrain](#)
- [WireLoad](#)
- [WireLoadGroup](#)
- [WireLoadMinBlockSize](#)
- [WireLoadMode](#)
- [WrapBlockIndividually](#)
- [WrapSubblocksIndividually](#)
- [WrapperDefaultSafeState](#)
- [WrapperExcludePorts](#)
- [WrapperRegisterImplementation](#)
- [WrapperUseRegisterIO](#)
- [dc_shellStopMessageIds](#)
- [dc_shellVariable](#)
- [dc_shellVariableComment](#)
- [design](#)
- [fc_shellAppOption](#)
- [fc_shellAppOptionComment](#)
- [fm_shellVariable](#)
- [fm_shellVariableComment](#)
- [hdlin_enable_upf_compatible_naming](#)
- [pt_shellVariable](#)
- [pt_shellVariableComment](#)
- [rtl_shellAppOption](#)
- [rtl_shellAppOptionComment](#)
- [tmax_shellVariable](#)
- [tmax_shellVariableComment](#)
- [tmm_shellAppOption](#)
- [tmm_shellAppOptionComment](#)

Documentation Generation: [index] _____

- [DocDefaultValue](#)
- [DocInclude](#)
- [DocLabelName](#)
- [DocOffset](#)
- [DocRangeDecoratedName](#)
- [DocShortDescription](#)

Environment Verification: [index] _____

- [EnvCheck](#)

Files and file groups: [index] _____

- [file](#)
- [filegroup](#)
- [filegroupGroup](#)

Generating a Reports Index Web Page: [index] _____

- [coretools_home_page](#)

Interface Parameters: [index] _____

- [AssociationFormula](#)
- [InterfaceIsUsed](#)
- [IsControlValue](#)
- [UsedOnInstance](#)
- [param](#)

Interface Ports: [index] _____

- [AllowOpenProvider](#)
- [BusAlignment](#)
- [DefaultConstantPort](#)
- [InterfaceIsUsed](#)
- [InterfaceLink](#)
- [InterfaceSize](#)
- [Optional](#)
- [OptionalAssociation](#)
- [RequiredExPortDirection](#)
- [RequiredPortDirection](#)
- [UsedOnInstance](#)
- [interfacePort](#)

Knowledge Database: [index] _____

- [KbVersion](#)
- [knowledgeBase](#)

MemoryMap Specification: [index] _____

- [UndefinedBitValue](#)

Miscellaneous Attributes and Variables: [index] _____

- [CreatedViaUngrouping](#)
- [ExtraSearchPathAppend](#)
- [ExtraSearchPathPrepend](#)
- [LicenseForOpen](#)
- [Name](#)
- [OpenAllTreeItems](#)
- [Parameters](#)

- ProjectID
- ProjectRootDir
- ScratchInstallIndex
- TimingExceptionCmd
- TimingExceptionFall fromValue
- TimingExceptionFall throughValue
- TimingExceptionFall toValue
- TimingExceptionFromValue
- TimingExceptionGroup path
- TimingExceptionOptions
- TimingExceptionRise fromValue
- TimingExceptionRise throughValue
- TimingExceptionRise toValue
- TimingExceptionThroughValue
- TimingExceptionToValue
- TimingExceptionValue
- TypeName
- attrDefn
- check error list
- collection print item limit
- crm file patterns
- guiBehavior
- scratch dir
- sh arch
- sh command abbrev mode
- sh command log file
- sh continue on error
- sh dev null
- sh enable page mode
- sh executable name
- sh product version
- sh script stop severity
- sh show tcl stack on error
- sh source uses search path
- sh tcllib app dirname
- sysCmd
- verbose messages

Nets: [index] _____

- net

Parameters: [index] _____

- GroupUserLabel
- HdlUnquoted
- HdlValue
- IntentFilesProcessed
- ShowIcons
- StaticDefineExprs
- UnelaboratedName

- param

Pins: [index]_____

- pin

Ports: [index]_____

- Capacitance
- CaseAnalysisValue
- ClockName
- ConstantPort
- CriticalTiming
- DftExistingSignalActiveState
- DftExistingSignalTestMode
- DftExistingSignalTiming
- DftExistingSignalType
- DftExistingSignalUsage
- DftSpecSignalActiveState
- DftSpecSignalTestMode
- DftSpecSignalType
- DftSpecSignalUsage
- DontTouchNetwork
- Drive
- DrivingCell
- EndBit
- EndBitExpr
- EndBitsExpr
- EpgaPadType
- EpgaPortIsPad
- HighFanout
- IdealPort
- InputDelay
- InputResistance
- IsBehavioral
- IsExternalReset
- IsTriState
- LogicalName
- MaxCap
- MaxFallInputDelay
- MaxFallInputDelayFallingEdge
- MaxFallOutputDelay
- MaxFallOutputDelayFallingEdge
- MaxFanout
- MaxInputDelay
- MaxInputDelayFallingEdge
- MaxLoads
- MaxOutputDelay
- MaxOutputDelayFallingEdge
- MaxRiseInputDelay
- MaxRiseInputDelayFallingEdge

- [MaxRiseOutputDelay](#)
- [MaxRiseOutputDelayFallingEdge](#)
- [MaxTransition](#)
- [MergeCandidate](#)
- [MinCap](#)
- [MinFallInputDelay](#)
- [MinFallInputDelayFallingEdge](#)
- [MinFallOutputDelay](#)
- [MinFallOutputDelayFallingEdge](#)
- [MinInputDelay](#)
- [MinInputDelayFallingEdge](#)
- [MinOutputDelay](#)
- [MinOutputDelayFallingEdge](#)
- [MinRiseInputDelay](#)
- [MinRiseInputDelayFallingEdge](#)
- [MinRiseOutputDelay](#)
- [MinRiseOutputDelayFallingEdge](#)
- [OutputDelay](#)
- [PinLoadCount](#)
- [PinLoadType](#)
- [PortDirection](#)
- [PortFunction](#)
- [PortWidth](#)
- [Registered](#)
- [RegisteredBy](#)
- [ResetProperties](#)
- [SDCPropagateText](#)
- [SDCWriteCallback](#)
- [SideConstraint](#)
- [StartBit](#)
- [StartBitExpr](#)
- [StartBitsExpr](#)
- [TestIsolate](#)
- [UnconnectedPort](#)
- [UserPortName](#)
- [port](#)

Power Intent: [index] _____

- [PowerDomain](#)
- [PowerDomains](#)

Spirit integration: [index] _____

- [SpiritFileType](#)

Strategy Specification: [index] _____

- [ATPG basic scan strategy](#)
- [DCTCL quick map strategy](#)
- [Design Compiler Reference Methodology](#)

coreTools Command Reference Index

- [FM formal verification strategy](#)
- [FPGA optimization strategy](#)
- [Fusion Compiler Reference Methodology](#)
- [PT static timing analysis strategy](#)
- [RTL Architect Reference Methodology](#)
- [TMAX Reference Methodology](#)

Synthesis Strategies: [index] _____

- [Strategy](#)

Technology Cell Mapping: [index] _____

- [TechCellMappingBind](#)
- [TechCellMappingDefn](#)

Timing Exceptions: [index] _____

- [timingException](#)

ATPG_basic_scan_strategy

The ATPG_basic_scan_strategy will create test vectors using the Basic_scan_only engine of TetraMax. This is the recommended starting point for generating vectors, and it will provide fast, good coverage for full scan designs.

Parameters

Strategy has no parameters.

See Also

Abstraction

Indicates the abstraction of an interface instance or the abstraction in which a given interface port should be present.

Definition

Type: string

Flags:

Default value: Value calculated internally.

Valid on: param_____

Description

Indicates the IP-XACT abstraction definition associated with this interface. This attribute is never set directly. It is set automatically from the value in the spirit:abstractionType element when instantiating an IP-XACT component or from the value explicitly passed to the create_interface_instance command via the -abstraction argument.

See Also

Activity Index

[AddInterfaces](#)

Add interfaces to the top-level of the core for automated assembly of this core in a System-on-a-Chip subsystem.

[BuildcoreKit](#)

Build the coreKit for this core.

[BuildIPXACTComponent](#)

Build the IPX-ACT for this core.

[BuildSubsystemcoreKit](#)

Run coreBuilder and package the subsystem as a coreKit.

[CompleteConnections](#)

Connect and disconnect component pins and (new) subsystem ports manually until the subsystem design is complete.

[ConstraintAnalysis](#)

Generate and (optionally) run ptc_shell synthesis constraint analysis scripts.

[ConstraintsVerification](#)

set up the rtl config file and generate the specified assertions

[ConstraintsVerificationWithAssertions](#)

Execute the simulation generated list of log files

[CreateBillOfMaterials](#)

Specify the set of files that are to be incorporated into the bill of materials. This is the definition of the set of all files that are part of the core design kit.

[CreateIntegrationWorkspaces](#)

This activity packages up a core kit, installs the core kit, and then creates one or more integration (coreConsultant) work spaces from that installation. These work spaces can be used to do validation of the core kit.

[CreateSubsystemTemplate](#)

Create a subsystem template. The template is created as a single file and is located in the ./export directory of the current workspace.

[CreateTestbenchWorkspace](#)

Create testbench workspace for the current DUT.

[DefineComponentViews](#)

Add or delete IPXACT views for the component.

[DefineMemoryMaps](#)

This activity is used to create

DesignConfigurationIntent

and view memory maps.

Review/specify intent attributes associated with the HDL parameters for the core design.

Generate and (optionally) run RTL Architect scripts. This activity is used to generate RTLA Reference Methodology scripts. By default, the RTLArchitect run is started immediately after the scripts are generated, but this behavior can be changed by checking the 'Generate Scripts Only' check-box. Once the RTLA Activity is finished, results will be available in the

ExploreRTLArchitecture

following locations: Logs - rtl/run_'flow type'/logs Reports - rtl/run_'flow type'/reports Outputs - rtl/run_'flow type'/outputs If you choose to run the rtl yourself, or wish to review the generated scripts or constraints, these can be found in the following locations: Flat Makefile:

rm_setup/Makefile_rtl_flat

Explore Makefile:

rm_setup/Makefile_rtl_explore

FormalVerification

Generate and (optionally) run formal verification scripts.

FPGASynthesis

Generate and (optionally) run FPGA synthesis scripts. This activity is used to generate FPGA synthesis scripts for SynplifyPro or SynplifyPremier, customized for the particular core configuration, Synplify version and FPGA device. By default, the Synplify run is started immediately after the scripts are generated, but this behavior can be changed by checking the 'Generate Scripts Only' check-box. Once the synthesis is finished, results will be available in the following locations: EDIF netlist - syn/final/db/<top_design>.edif HDL netlist -

<u>GenerateSubsystemRTL</u>	syn/final/db/<top_design>.v[hd] Reports - syn/final/report If you choose to run the synthesis yourself, or wish to review the generated scripts or constraints, these can be found in the following locations; Launch script - syn/run.scr Synplify script - syn/synplify.tcl Constraints - syn/constrain/script/*.cscr Generate the HDL source code for top-level design of the subsystem.
<u>GenerateTestVectors</u>	Generate test vectors using TestMAX. Create system level clocks, and initialize system level constraints from component level constraints.
<u>InitializeSubsystemConstraints</u>	This activity is used to create plugin KBs that are used by the core integrator of this IP. These KBs are added to the Plugins file group for installation in the consultant workspace. As a side-effect, all non-object Tcl files are sourced.
<u>LoadConsultantPlugins</u>	This activity is used to get design data into coreBuilder. Currently the tool supports input from either VHDL, Verilog or SystemVerilog Hdl. The HDL files may contain special comment based annotations. After the HDL is analyzed, then Tcl (and SDC) based intent files are sourced.
<u>LoadDesigns</u>	Load definitions for each interface to be instantiated in the top-level of this core for automated assembly within coreAssembler.
<u>LoadInterfaceDefinitions</u>	This activity is used to configure memory map.
<u>MemoryMapSpecification</u>	Perform Design Verification of the subsystem and its components.
<u>RTLStaticChecking</u>	Prime Profiles set on components can be selected in this activity.
<u>SelectPrimeProfile</u>	

<u>SetDesignFileMacroPrefixes</u>	Set the design, file, and macro prefixes. The design prefix is used to make each design in your core have a unique name. Setting the design prefix is only needed when you need to have two or more versions of one component, i.e., you plan to instantiate two or more versions of one component in the same design database, each with different values of configuration parameters. When the RTL code is written to disk, the prefix is prepended to every design name reference \ (and package name references for VHDL\). All design names and references to design names will be changed to {prefix}_{original name}. The macro prefix is used to replace any 'define {var}'s and usages of the '{var}' with a prefixed value throughout the RTL code.
<u>SetUpHelpMenu</u>	Add documentation links to the Help menu in the console window.
<u>SimulateSubsystemTestbench</u>	Simulate the current subsystem testbench.
<u>SpecifyClocks</u>	Define the set of clocks (virtual and real) within the core design.
<u>SpecifyConfiguration</u>	Specify values for HDL configuration parameters.
<u>SpecifyOperatingConditionsAndWireLoads</u>	Specify information which defines the context in which the core is going to be used (operating conditions and wireloads)
<u>SpecifyPortConstraints</u>	Set and/or review intent attributes on ports.
<u>SpecifyPowerIntent</u>	This activity is used to capture power intent scripts.
<u>SpecifySubsystem</u>	Select and configure the components and any associated memory maps for the subsystem.
<u>SpecifySynthesisMethodology</u>	Set and/or review intent attributes on designs.
<u>SpecifyTestMethododology</u>	Set and/or review Design for Test intent attributes on designs.
<u>SpecifyTimingExceptions</u>	

StaticChecking

Define the timing exceptions within the core design.

Perform static checking of the subsystem and its components.

StaticTimingAnalysis

Generate and (optionally) run static timing analysis scripts.

Generate and (optionally) run synthesis scripts. This activity is used to generate synthesis scripts customized for the particular core configuration, DC/FC version compatibility, and target technology. Now user have the option between Design Compiler and Fusion Compiler to run synthesis. By default synthesis is fired right after the scripts are generated, using the preferred tool. But this behavior can be changed by checking the Generate Scripts Only check-box. Once the synthesis is finished, results will be available in the following locations: For DC only : DDC file - syn/final/db/<top design>.ddc For DC & FC : Netlist - syn/final/db/<top design>.v[hd] Reports - syn/final/report/ If you choose to run the synthesis yourself, or wish to review the generated scripts or constraints, these files can be found in the following locations: Launch script - syn/[scriptFile] Where the name of the 'scriptFile' can be obtained with the command: get_activity_parameter Synthesize ScriptFile Constraints - syn/constrain/script/*.[sdc/tcl] If you wish to integrate the generated scripts into your own chip-level synthesis environment, please see the 'Synthesis_API' man page for details about how to access the generated scripts from your own environment.

Synthesize

This activity implements the technology cell mapping functionality.

TechnologyCellBinding

coreTools Command Reference Index

TechSetup

This activity gets variable settings from DesignCompiler, and extracts technology specific information from the specified technology libraries.

TestsuiteSetup

Specify testsuite filegroups and parameters.

VerifyIntent

This activity attempts to flag common mistakes that are made when specifying constraints.

[API Index](#) [Command Reference](#)

Activity

Activity bound to a menu item.

Definition

Type: itemList

Flags:

Default value:

Valid on: knowledgeBase

Description

Examples

See Also

ActivityGroup

Group/Menu to put this to-level activity in

Definition

Type: string

Flags:

Default value: TOP_LEVEL_GROUP

Valid on: filegroup_____

Description

The ActivityGroup attribute assigns an activity to a top-level group of related activities. In coreConsultant, the pre-defined top-level activity groups are Setup, Verification, and Synthesis. In coreBuilder, the pre-defined top-level activity groups are Setup, Synthesis, and Packaging.

To set the value of ActivityGroup on a custom activity, use the -group option with the create_custom_activity command. You can assign a custom activity to either an existing activity group or a custom activity group. If you specify a group that does not already exist, coreConsultant or coreBuilder creates the custom activity group when it reads the plug-in knowledge database that contains the custom activity.

To get the value of the ActivityGroup attribute on an activity, use the get_attribute command.

Examples

To get the value of ActivityGroup for the SpecifyPortIntent activity:

```
coreBuilder> get_attribute -attrs ActivityGroup SpecifyPortIntent  
Synthesis
```

See Also

[create_custom_activity \(2\)](#), [get_attribute \(2\)](#)_____

[API Index](#) [Command Reference](#)

Add Interfaces

Add interfaces to the top-level of the core for automated assembly of this core in a System-on-a-Chip subsystem.

Parameters

Activity has no parameters.

See Also

[Load Interface Definitions.](#)

[API Index](#) [Command Reference](#)

AddLockUpLatch

When true lockup latches will be inserted between clock domain boundaries on scan chains.

Definition

Type: boolean

Flags:

Default value: =InheritValue up TRUE

Valid on: design_____

Description

When set to true lockup latches will be inserted between clock domain boundaries on scan chains. To disable lockup latch insertion, set this attribute to false. If the ClockMixing is set to no_mix insert_scan ignores this option.

Examples

Specify that lockup latches should be inserted between clock domains on scan chains.

```
coreBuilder> set_design_attribute AddLockUpLatch TRUE
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#), [ClockMixing \(3\)](#)

AddressOffset

Offset from the base address of the address block for this register or registerArray

Definition

Type: bitfield

Flags:

Default value: 0x0

Valid on:

Description

This attribute specifies the address offset from the base address of the memory map in bytes for registers.

Examples

To specify the register reg1's address offset to be 0x1000

```
coreBuilder> set_register_attribute reg1 AddressOffset 0x1000
```

See Also

[API Index](#) [Command Reference](#)

AddressSpaceRef

The name of the address space.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

The name of the IP-Xact addressSpace referenced by this interface instance.

Examples

See Also

[create_address_space \(2\)](#), [remove_address_space \(2\)](#), [BaseAddress \(3\)](#)

AllowOpenProvider

Flag disabling the requirement that provider interfaces must provide a driver when the consumer has an input port.

Definition

Type: boolean

Flags:

Default value: 0

Valid on:

Description

By default, coreBuilder and coreAssembler require that a fromProvider interface port of a provider interface instance must be linked to a design port (e.g. the value of InterfaceLink cannot be "<open>"). This is to ensure that something is available to drive a consumer side input. This restriction is lifted if the interface port has a default value defined via DefaultConstantPort. There may still be cases where you explicitly want to say that it is okay to allow the driver to be missing on the provider side. This can be indicated by setting the AllowOpenProvider attribute to true on the provider side interface port.

Examples

In coreAssembler, set AllowOpenProvider on the optPort interface port of the Prov interface instance of component instance U0.

```
set_interface_port_attribute -instance U0/Prov optPort AllowOpenProvider true
```

AltValue

Alternate value for the attribute.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: actual altValueLabel symbolicName valueLabel

Default subscript: actual

Valid on: attrDefn

Description

The AltValue attribute specifies an alternate legal value for a parameter. An alternate value is a value other than the legal range of values specified by MinValue and MaxValue for the parameter.

For example, you may have a design parameter that has 0-256 as its legal range and an additional legal value -1 that has a different meaning than a value between 0 and 256. In this case, -1 is the alternate value.

The subscripts for AltValue are:

- **actual** Specifies the actual alternate value (for example, -1). For a design parameter, AltValue[actual] is the value that coreConsultant writes into the HDL source code when the user selects the alternate value during the Specify Configuration activity.
- **valueLabel** Specifies the string used in the GUI as a label for the non-alternate value. For the previously described example, if AltValue[valueLabel] = "Hardwired Value", the label "Hardwired Value" appears next to the text box where the user enters a value from 0 to 256.
- **symbolicName** An alias for the alternate value.
- **altValueLabel** The label to be used for the radio button that selects the alternate value for the parameter. If you specify AltValue[altValueLabel], coreConsultant generates radio buttons to select between the parameter's legal range of values and the alternate value. AltValue[altValueLabel] is the label for the radio button that selects the alternate value. For example, if AltValue[altValueLabel] = Programmable, the label "Programmable" appears next to the radio button that selects the alternate value. If you do not specify AltValue[altValueLabel], coreConsultant generates a single checkbox that either enables the value range selection or selects the alternate value.

Examples

The following example VHDL annotations illustrate how to use AltValue to create a parameter dialog where the user can select "Hardwired" to enter a value from 0 to 256 for the ID_reg parameter or select "Programmable" to set ID_reg to -1:

```
generic (
--reuse-pragma attr MinValue 0
--reuse-pragma attr MaxValue 256
--reuse-pragma attr Label ID Register
--reuse-pragma attr AltValue[actual] -1
--reuse-pragma attr AltValue[valueLabel] Hardwired
--reuse-pragma attr AltValue[altValueLabel] Programmable
ID_reg : integer := 0;
);
```

See Also

[set_parameter_attribute \(2\)](#), [MinValue \(3\)](#), [MaxValue \(3\)](#), [Label \(3\)](#)

Application-Programming-Interface Index

Command Reference Index

<u>API coreAssembler</u>	Defines the application programming interface (API) for coreAssembler
<u>API coreBuilder</u>	Defines the application programming interface (API) for coreBuilder
<u>API coreConsultant</u>	Defines the application programming interface (API) for coreConsultant

AreaEstimate

Estimated cell area of a design.

Definition

Type: float**Flags:****Default value:** =CombineInheritValues down sum**Valid on:** design_____

Description

The AreaEstimate attribute specifies the estimated cell area of a design. If PhysicalRegion is true on a design and you do not explicitly set a value for the WireLoad attribute on that design, coreConsultant uses the value of AreaEstimate on that design to select a wire load model for the initial compile. For subsequent compiles, coreConsultant lets Design Compiler automatically select wire load models based on the actual design area.

You can specify the AreaEstimate value with or without units. If you do not specify an area unit, the default area unit is the area unit used by the currently loaded technology library. If you do specify an area unit, coreConsultant automatically scales the area value to the area unit used by the currently loaded technology library.

The default value for AreaEstimate is sum of the AreaEstimate values on all subblocks contained in the design.

Examples

To set the estimated cell area of the current_design to 1200:

```
coreConsultant> set_design_attribute AreaEstimate 1200
```

To set the estimated cell area of Subblock_A in the current_design to 600:

```
coreConsultant> set_design_attribute -designs Subblock_A AreaEstimate 600
```

See Also

[set_design_attribute \(2\)](#), [WireLoad \(3\)](#), [PhysicalRegion \(3\)](#), _____
[WireLoadMinBlockSize \(3\)](#)

AreaWeight

Indicates relative importance of a parameter with respect to area estimation. By default all parameters have a weight of 1 but parameters that have a bigger influence on the area of a design can have their match importance increased by raising the AreaWeight value.

Definition

Type: long

Flags:

Default value: 1

Valid on: param_____

Description

This attribute is used to weight parameters that have a large influence on the area of a design. A high weight value indicates that a match of this parameter is more important for determining which area estimate is appropriate than a match of a parameter with a lower weight. The weights of all matching parameters are added together to determine the merit of a given configuration specific area estimation. The higher the sum, the closer the area estimate is to being a match.

Examples

Add weights to key parameters in a component

```
prompt>set_parameter_attribute Has_FPU AreaWeight 10
```

```
prompt>set_parameter_attribute Has_ALU AreaWeight 5
```

See Also

[set_area_estimate \(2\)](#) _____

[API Index](#) [Command Reference](#)

ArrayEnd

Final index for the array entries.

Definition

Type: short

Flags:

Default value:

Valid on: param_____

Description

Examples

See Also

[define_array_field_parameters \(2\)](#), [get_field_parameter_for_array \(2\)](#), _____

[foreach_array_field_parameter \(2\)](#), [isArray \(3\)](#), [ArrayStart \(3\)](#), _____

[ArrayFieldSize \(3\)](#)_____

[API Index](#) [Command Reference](#)

ArrayFieldSize

Number of bits per element in the array

Definition

Type: short

Flags:

Default value: 1

Valid on: param_____

Description

Examples

See Also

[define_array_field_parameters \(2\)](#), [get_field_parameter_for_array \(2\)](#), _____

[foreach_array_field_parameter \(2\)](#), [isArray \(3\)](#), [ArrayStart \(3\)](#), [ArrayEnd](#) _____
(3)

[API Index](#) [Command Reference](#)

ArrayStart

Initial index for the array entries.

Definition

Type: short

Flags:

Default value: 0

Valid on: param_____

Description

Examples

See Also

[define_array_field_parameters \(2\)](#), [get_field_parameter_for_array \(2\)](#), _____

[foreach_array_field_parameter \(2\)](#), [isArray \(3\)](#), [ArrayEnd \(3\)](#), _____

[ArrayFieldSize \(3\)](#)

AssociationComplete

Is true if all ports and parameters of the interface instance are completely 'linked'. This attribute does NOT indicate whether the association is valid and error-free. Provides specific completeness information on interface ports and parameters.

Definition

Type: boolean

Flags: readOnly

Default value: =sIntf::isAssociationComplete_[get_attribute %item
-attr TypeName] %item

Valid on: param_____

Description

This is a read-only attribute that indicates whether or not the given interface instance has been fully associated (linked) to the top-level design in the current coreBuilder workspace. If this attribute value is not true, then it is likely that there will be problems when trying to automatically connect this core within a coreAssembler subsystem.

Examples

See Also

[API Index](#) [Command Reference](#)

AssociationFormula

Formula to determine the alternative value to use during interface-design association.

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

Interface parameters are associated with corresponding design parameters using the InterfaceLink attribute. By default, the value of the interface parameter is set directly on the linked design parameter. However, there are situations where the value to be set on the design parameter is based on the value of the interface parameter, but not exactly the same. The AssociationFormula attribute can be used to define a formula for calculating the value to store on the design parameter. The value of the attribute should be a standard TCL arithmetic expression, except that it can contain the string '<interface parameter name>'. That string is replaced with the value of the interface parameter, and then the entire expression is evaluated, and the result is stored as the value for the corresponding design parameter.

Examples

Consider the case where interface parameter I (on interface instance II) is associated with design parameter D. The value of D is always supposed to be one less than the value of I. This can be accomplished as follows:

```
prompt> set_interface_parameter_attribute -instance II  
I AssociationFormula {@I-1}
```

See Also

[InterfaceLink \(3\)](#)

AtpgTclAuxScript

Specifies a TestMAX Tcl script to be executed at specific points during test vector generation.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: cleanup drc netlist post_run run setup

Default subscript:

Valid on: design_____

Description

AtpgTclAuxScript can be used to specify Tcl scripts that will be executed during test vector generation. The scripts specified by the subscripts will be executed at the following points in the test vector generation process. These scripts are configurable, so reuse-pragmas may be used to tailor the scripts to different design configurations, ATPG strategies, etc.

- setup - This script will be executed after the test simulation libraries have been loaded, but before the design is loaded into TetraMax.
- netlist - This script will be executed after the netlist is read-in but before 'run_build_model' command is issued.
- drc - This script will be executed after the design is loaded into TetraMax and just before drc is run.
- run - This script will be executed after the design has been loaded, run_drc has been executed, and add_faults -all has been executed. Immediately after this script is sourced, run_atpg will be executed. The AtpgTclAuxScript[run] is also used as the default value for ATPG_user_defined_strategy. This can be used by IP providers to package a script for vector generation with the core.
- cleanup - This script will be executed after all processing is complete, just before quitting.

Examples

Specify a script to be used after the libraries and design have been loaded, design rules checked, and the faults added.

```
coreConsultant> set_design_attribute {AtpgTclAuxScript[run]} \
/full/path/runAtpg.tcl
```

See Also

[set_design_attribute \(2\)](#), [reuse-pragma \(2\)](#), [TclAuxSynthesisScript \(3\)](#).

[API Index](#) [Command Reference](#)

AtpgTclAuxScriptComment

Specifies a comment to be issued for the corresponding AtpgTclAuxScript

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: cleanup drc netlist post_run run setup

Default subscript:

Valid on: design_____

Description

The AtpgTclAuxScriptComment attribute can be used to output a comment to document why the corresponding AtpgTclAuxScript is being used. The comment will be put into the file where the script is sourced.

Examples

Specify an aux script to be used in the setup stage, and document why it is being used.

```
set_design_attribute AtpgTclAuxScript myAuxScr.tcl
set_design_attribute AtpgTclAuxScriptComment "This script is used to set
up custom variables for the test pattern generation."
```

See Also

[AtpgTclAuxScript \(3\)](#)

Attributes Index

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z _ _ _ _ _

A

Abstraction

Indicates the abstraction of an interface instance or the abstraction in which a given interface port should be present.

Activity

Activity bound to a menu item.

ActivityGroup

Group/Menu to put this to-level activity in
When true lockup latches will be inserted between clock domain boundaries on scan chains.

AddLockUpLatch

AddressOffset

Offset from the base address of the address block for this register or registerArray

AddressSpaceRef

The name of the address space.

AllowOpenProvider

Flag disabling the requirement that provider interfaces must provide a driver when the consumer has an input port.

AltValue

Alternate value for the attribute.

AreaEstimate

Estimated cell area of a design.

AreaWeight

Indicates relative importance of a parameter with respect to area estimation. By default all parameters have a weight of 1 but parameters that have a bigger influence on the area of a design can have their match importance increased by raising the AreaWeight value.

ArrayEnd

Final index for the array entries.

ArrayFieldSize

Number of bits per element in the array

ArrayStart

Initial index for the array entries.

AssociationComplete

Is true if all ports and parameters of the interface instance are completely 'linked'.
This attribute does NOT indicate whether the association is valid and error-free. Provides specific completeness information on interface ports and parameters.

AssociationFormula

Formula to determine the alternative value to use during interface-design association.

AtpgTclAuxScript

Specifies a TestMAX Tcl script to be executed at specific points during test vector generation.

AtpgTclAuxScriptComment

coreTools Command Reference Index

	Specifies a comment to be issued for the corresponding AtpgTclAuxScript
<u>AutoConnectComplete</u>	Indicates this design was auto connected.
<u>AutoConnectIfUnconnected</u>	Connection to make if left unconnected during auto-connection step.
<u>AutoConnectionPreCallback</u>	Defines user-callback to be invoked before making all automatic connections at a given level of hierarchy.
<u>AutoConnectWhen</u>	Indicates if/when the given interface should be automatically connected.
<u>AutoFixAsync</u>	Enables/Disables automatic fixing of asynchronous preset/clear violations. When set to gate, specifies that the asynchronous sets and resets are autofixed with a gate driven by the test_mode signal.
<u>AutoFixAsyncLogicGate</u>	When set to mux sets and resets are with a multiplexer. The select input of the mux is the test_mode signal and a primary input is controlling the asynchronous set or reset in test mode.
<u>AutoFixAsyncTestData</u>	Port to be used as TestData for fixing set/reset violations. The port must have been defined with a DftExistingSignalType of Reset.
<u>AutoFixAsyncWithScanEnable</u>	When true the test scan enable signal will be used to autofix asynchronous set and reset violations.
<u>AutoFixBidirectional</u>	Enables/Disables automatic bidirectional disabling.
<u>AutoFixClocks</u>	Enables/Disables automatic fixing of uncontrollable clock violations
<u>AutoFixClockTestData</u>	Port to be used as TestData for fixing clock violations. The port must be associated with a clock that has TestClock==1 or have been defined with a DftExistingSignalType of a clock type.
<u>AutoloadFilegroup</u>	Are the files in this group loaded automatically?

B

<u>BalanceBistSegments</u>	By default, SoCBIST rebalances the scan channels in BIST mode. Setting BalanceBistSegments to false will override this, and SoCBIST will not concatenate the scan segments.
----------------------------	---

coreTools Command Reference Index

<u>BankAlignment</u>	The memory alignment of this bank
<u>BaseAddress</u>	The base address of this item.
<u>BidirectionalMode</u>	Determines whether bidirectional ports are treated as inputs, outputs or left untouched during scan shift.
<u>BistAutoFixBusses</u>	Indicates floating bus and bus contention issues should be fixed automatically.
<u>BistAutoFixXprop</u>	Should automatic fixing of x propagation be enabled?
<u>BistBlockIndividually</u>	Causes insertion of BIST logic into this design. By default, DFT Compiler concatenates lower level scan segments to arrive at the optimal number of scan chains during BIST controller insertion. You can override this number by specifying the scan configuration
<u>BistChainCount</u>	NumberOfScanChains or MaximumScanChainLength attributes. You must also use BistChainCount to control how many channels are hooked up to the controller. Setting this attribute to 0 allows DFT compiler to choose the number of scan chains.
<u>BistCodecCount</u>	Number of BIST CODECS
<u>BistDiagOutputs</u>	The number of diagnostic outputs for DBIST
<u>BistInvertPrpgClock</u>	Invert the BIST clock that feeds the PRPG Registers Controls control the size of the counter, which increases the default upper limit on the length of the reconfigured scan chains. Setting BistMaxChainLength to 0 will allow DFT Compiler to determine the size of the counter.
<u>BistMaxChainLength</u>	BIST operations to be performed. Choosing 'all' will cause the design to be made bist ready and for the codec to be inserted and integrated with the controller. For hierarchical bist insertion choose codec_insertion for the lower levels of hierarchy and all at the level where the controller is to be integrated with the codecs. Choose bist_ready to make many BIST scan chains or channels within the design without X-propagation and without wrapping the design.
<u>BistMode</u>	
<u>BistObserveOutputs</u>	Controls the number of scan output pins that are inserted into the design for BistMode == xdbist. By default, the number of scan outputs created depends on the number of scan chains: scan outputs = (scan chains/32). Setting this

coreTools Command Reference Index

	attribute to 0 in coreTools lets DFT compiler use the default value.
<u>BistPrpgLength</u>	By default, SoCBIST creates a 479-bit PRPG. You can override this by setting BistPrpgLength to 257. This will cause a 257-bit PRPG to be created. For DBIST, you can use up to 512 scan chains with the 257-bit PRPG. For X-tolerant DBIST, you can use 2048 scan chains with the 257-bit PRPG.
<u>BistPrpgShadowSi</u>	By default, DFT Compiler creates from one to twelve shadow scan chains for the PRPG shadow. The number of shadow scan chains created is optimized based on the length of the longest scan chain in the design. You can stipulate that DFT Compiler create a specific number of shadow scan chains instead by setting BistPrpgShadowSi to n, where n is a number from one to twelve. Setting BistPrpgShadowSi to default allows DFT Compiler to determine the number of shadow scan chains. BistPrpgShadowSi does not affect the length of the longest scan chain. It only affects the number of shadow scan chains.
<u>BistSelectorShadowSi</u>	The number of selector shadow chains. Setting BistSelectorShadowSi will allow DFT Compiler to determine the number of selector shadow chains based on the length of the longest scan chain.
<u>BistSubblocksIndividually</u>	Causes BIST logic to be inserted into the subdesigns of this design.
<u>BistType</u>	Type of BIST to implement
<u>BistUseTristateMux</u>	Specifies that tri-state muxes be used to construct the XDBIST codec
<u>BitAddressOffset</u>	Offset in bits from the base address of the register or memory block
<u>BitRange</u>	Returns the bit range of this field as [start]:[end]. Range is not evaluated if the start or end bit indicator is a formula.
<u>BitsInLAU</u>	The number of bits in the Least Addressable Unit (LAU)
<u>BitsToRepresent</u>	The number of bits needed to represent valid values for this type.
<u>BitWidth</u>	The number of bits in the bitfield.
<u>Bridge</u>	Indicates that there is a bridge from this interface across the component.
<u>BusAlignment</u>	Usually an interface port needs 'full' association to a design port with the same number of bits as InterfaceSize. Some vectored

ports (busses) allow partial connection to an opposite design port. BusAlignment indicates how a narrower design port aligns within a bus of full interface size, and whether it connects on the left or right hand side of a full opposite port, or by slice-indices on a full port with a 0..InterfaceSize-1 indexing and same bit order. The bitwise alignment value is currently only available for IP-XACT components and should not be used within coreBuilder.

C

<u>CanUngroup</u>	Indicates that the design can be replaced by a hierarchy component.
<u>CanUngroupInfo</u>	Information about design HDL file and/or design contents when CanUngroup value is false.
<u>Capacitance</u>	Explicit load capacitance. Static value that the port or pin will take on for the synthesis session.
<u>CaseAnalysisValue</u>	Unlike ConstantPort, CaseAnalysisValue will not cause logic to be optimized away.
<u>Channel</u>	Indicates that there is a channel from this interface across the component.
<u>CHeaderValue</u>	Defines the value for the C header file for the given attribute on this register or register field.
<u>CheckCmd</u>	Command used to check values
<u>CheckExpr</u>	Check expression for a parameter
<u>CheckExprMessage</u>	Check expression message for a parameter
<u>CheckExprWhenDisabled</u>	Specifies whether or not CheckExpr expressions should be checked for non-visible or non-enabled parameters.
<u>ClockFallLatency</u>	The time it takes a clock signal to propagate from the clock definition point to a register clock pin for a falling transition at the clock definition point.
<u>ClockGatingFallHoldCheck</u>	Falling edge hold check for gating control signals for this clock.
<u>ClockGatingFallSetupCheck</u>	Falling edge setup check for gating control signals for this clock.
<u>ClockGatingRiseHoldCheck</u>	Rising edge hold check for gating control signals for this clock.

coreTools Command Reference Index

<u>ClockGatingRiseSetupCheck</u>	Rising edge setup check for gating control signals for this clock.
<u>ClockGatingSignals</u>	Use this attribute to specify a list of signals to include/exclude when inferring flip-flops with gated clocks.
<u>ClockHoldUncertainty</u>	Hold uncertainty applied to all endpoints of the given clock.
<u>ClockMixing</u>	Specifies whether cells from different clock domains may be put in the same scan chain
<u>ClockName</u>	Name of the clock object (possibly associated with a port)
<u>ClockRiseLatency</u>	The time it takes a clock signal to propagate from the clock definition point to a register clock pin for a rising transition at the clock definition point.
<u>ClockSetupUncertainty</u>	Setup uncertainty applied to all endpoints of the given clock.
<u>ClockSourceFallLatency</u>	The time it takes a clock signal to propagate from its ideal waveform origin point to the clock definition point for a falling transition at the waveform origin point.
<u>ClockSourceRiseLatency</u>	The time it takes a clock signal to propagate from its ideal waveform origin point to the clock definition point for a rising transition at the waveform origin point.
<u>CnctClass</u>	Connection class(es) for this connection.
<u>ComponentOfItem</u>	The component name of an item
<u>ConfigActivity</u>	The activity name that to configure this filegroup.
<u>ConfigDependsOnActivities</u>	List of activity names that should be completed (if they exist) before this filegroup is configured.
<u>ConfigDependsOnGroup</u>	List of filegroups that must be configured before this group.
<u>ConfigIntentSearchPath</u>	A list of directories to search for intent files
<u>Configurable</u>	Is this filegroup configurable?
<u>ConnectionDialogCmd</u>	Defines a custom command to be called for interactive connections.
<u>ConnectionRequired</u>	Used to indicate that this interface does not require a connection.
<u>ConnectToExportedInstance</u>	This instance need to connect to its exported instance

coreTools Command Reference Index

<u>ConstantPort</u>	Indicates that a port has a fixed logic value.
<u>ControlPointsPerScanCell</u>	Specifies the number of control points that can be shared per scan cell. When TestabilityMethod==control_and_observe for XG mode the larger of ObservePointsPerScanCell and ControlPointsPerScanCell will be used for the -test_points_per_scan_cell value for set_testability_configuration.
<u>ConvertSingleBitBus</u>	Convert single bit bus to bit type
<u>CopyToTemplate</u>	Should this component be copied into the template? A value of false means that the template must be used with an install area for this component. A value of true means that the template does not require an install area for this component
<u>coreBuilderBomTemplateVersion</u>	The version of the Bill-of-Materials template syntax used by a template.
<u>CreatedViaUngrouping</u>	Set to true for designs that are created using create_hierarchical_component -fromComponent.
<u>CreateSkewGroup</u>	Specifies the clock for which the skew group needs to be created. The skew group created will follow the naming convention SKEW_GROUP_QiK_CK_GEN_\. Refer to man page for 'create_clock_skew_group' in Fusion Compiler reference documentation.
<u>CriticalRange</u>	Default critical range for this design.
<u>CriticalRangeCoveringViolators</u>	Set the critical_range value to cover all violating paths in the design.
<u>CriticalTiming</u>	Indicates whether the port will likely be part of a critical timing path. CTLModels design attribute lists the ctl models to be read in by the synthesis tool when bottom-up synthesis is enabled. It is applicable only for hierarchical designs which have SynthesizeAsSubblock set to 1. CTL files for hardened blocks which are already synthesized may be specified at this level. For example a Subsystem has an IP (A), IP (A) is to be synthesized as a sub-block with DFT models cell1.ctl and cell2.ctl. So one can set the design attributes to include the test models during synthesis.
<u>CTLModels</u>	
<u>CustomizedTestbenchCode</u>	Customized testbench code to be inserted

coreTools Command Reference Index

<u>CycleTime</u>	Cycle time for a clock.
D	
<u>DCLSPaths</u>	Specify the paths as a list-of-list.
<u>dc_shellStopMessageIds</u>	Messages IDs which should cause Design Compiler to terminate.
<u>dc_shellVariable</u>	Variable settings to be used for dc_shell. Example: set_design_attribute {dc_shellVariable[varName]} value.
<u>dc_shellVariableComment</u>	Comment to be issued for the corresponding subscript of dc_shellVariable.
<u>DedicatedScanPorts</u>	Determines whether dedicated scan out ports will be created, or whether the functional ports will be used as scan ports.
<u>DedicatedWrapperCell</u>	Specifies the default dedicated wrapper cell type.
<u>DefaultConstantPort</u>	Specifies a default value to be connected to a design input port when the interface connection only links to a design port on one side of connected interfaces.
<u>DefaultHdlValue</u>	Store the Default HdlValue based on the unconfigured core
<u>DefaultInstallDir</u>	Default directory into which to install a file or file group
<u>DefaultLoadPath</u>	A list glob-style patterns specifying how to load this deliverable. Relative paths start with the root of the current workspace.
<u>DefaultParamRange</u>	Store the Parameter Range based on the unconfigured core
<u>DefaultValue</u>	Default value for a parameter
<u>DefaultValueMessage</u>	Message to display along with the default value
<u>DeferEvaluation</u>	If true, port will be present in elaborated design irrespective of generateIf condition. Its conditionality will be preserved in generated toplevel netlist on subsystem level.
<u>DefineLibClocks</u>	Defines hard macro libraries containing clocks to be defined for constraints.
<u>DeliverableType</u>	The type of information stored in this deliverable.
<u>Description</u>	Brief description of the item
<u>DftExistingSignalActiveState</u>	Active logic state for the signal.

coreTools Command Reference Index

<u>DftExistingSignalTestMode</u>	The name of the test mode the dft signal specification applies to.
<u>DftExistingSignalTiming</u>	Waveform to be used for the test signal. Test signal to be used for the 'existing_dft' view of set_dft_signal.
<u>DftExistingSignalType</u>	These are signals that describe structures which already exist that must be understood for the design to pass DRC (dft_drc): clocks, resets, constants, existing scan chains, etc.
<u>DftExistingSignalUsage</u>	The usage scenario(s) the dft signal specification applies to.
<u>DftSpecSignalActiveState</u>	Active logic state for the signal.
<u>DftSpecSignalTestMode</u>	The name of the test mode the dft signal specification applies to. Test signal to be used for the 'spec' view of set_dft_signal. These are signals that describe structures that DFT Compiler should use for insert_dft: scan in, scan out, scan enable, etc.
<u>DftSpecSignalType</u>	
<u>DftSpecSignalUsage</u>	The usage scenario(s) the dft signal specification applies to.
<u>DigitsPrecision</u>	Number of digits of precision to interpret and display
<u>DisableAggregation</u>	Set on a pin/port to disable aggregate connections to it in the netlist.
<u>DocAddressOffset</u>	Text relating to the AddressOffset attribute, for inclusion in documentation
<u>DocBlockTableAddressOffset</u>	Text relating to the BlockTableAddressOffset attribute, for inclusion in documentation
<u>DocDefaultValue</u>	Documentation oriented default value for a parameter.
<u>DocDescription</u>	Documentation oriented version of the description of the given item. The text may include DocBook tags.
<u>DocDescriptionField</u>	Defines extra tagged information to be included in the table cell describing the given item. Each value is included, tagged with its corresponding subscript. The text may include DocBook tags.
<u>DocEnabled</u>	Documentation oriented description of when the given item is enabled. The text may include DocBook tags. Must be set for each subscript for which Enabled is set.
<u>DocGenerateIf</u>	Documentation oriented description of when this port exists. The text may include

coreTools Command Reference Index

	DocBook tags.
<u>DocGroup</u>	Indicates that this object belongs to a group of objects that will be documented as a group.
<u>DocGroupName</u>	Grouping value used to group objects for documentation generation.
<u>DocInclude</u>	Used to conditionally include an item in generated documentation.
<u>DocLabelName</u>	Report oriented label and name combo for a parameter.
	Indicates that this object is the 'master' element among the group of elements with the same value for DocGroup, and is the element from which the documentation should be generated. The value of this attribute indicates the name of the looping variable and the min and max loop indices for the group (for documentation purposes). It can contain three entries "var min max" or five entries, with the two added entries being the post-elaboration min/max values. The latter two values can be parameter expressions if needed. Note a 4th/6th argument was added that allows a list of attribute/value pairs which can override the existing attribute value [list Name "UseMyName" GroupName "UseMyGroupName"].
<u>DocMaster</u>	
<u>DocMemoryAccess</u>	Text relating to the MemoryAccess attribute, for inclusion in documentation
<u>DocOffset</u>	Replaces register table pretext overriding GlobalAddressOffset value normally displayed.
<u>DocPortWidth</u>	Documentation oriented description of the width of the port.
<u>DocPowerDomain</u>	Documentation oriented description of the power domain associated with a port.
<u>DocRangeDecoratedName</u>	Report oriented value for the RangeDecoratedName of a port.
<u>DocRegistered</u>	Documentation oriented description of whether or not a port is registered.
<u>DocRegisteredBy</u>	Documentation oriented description of whether or not a port is registered.
<u>DocRegisterResetMask</u>	Documentation oriented description of the RegisterResetMask attribute. The text may include DocBook tags.
<u>DocRegisterResetValue</u>	Documentation oriented description of the reset value for this register or field. The

<u>DocRegisterSize</u>	text may include DocBook tags. Documentation oriented description of the size of this register or field. The text may include DocBook tags.
<u>DocRegTableAddressOffset</u>	Text relating to the RegTableAddressOffset attribute, for inclusion in documentation
<u>DocShortDescription</u>	Shortened version of the regular DocDescription.
<u>DocSynchronousTo</u>	Documentation oriented description of the clocks to which the port is synchronized.
<u>DocTestable</u>	Documentation oriented description of the Testable attribute. The text may include DocBook tags.
<u>DocVisible</u>	Documentation oriented description of when this register or field is visible. The text may include DocBook tags.
<u>DocVolatileMemory</u>	Documentation oriented description of the VolatileMemory attribute. The text may include DocBook tags.
<u>DocWaiver</u>	Used to document waivers for use of Doc* attributes. Subscript indicates attribute the waiver applies to.
<u>DontTouchNetwork</u>	Set dont_touch_network on this port.
<u>Drive</u>	Drive resistance for input or inout ports.
<u>DrivingCell</u>	Specifies how to generate a set_driving_cell constraint for an input port.

E

<u>Enabled</u>	Enable or disable this parameter in the GUI
<u>EnableDftAutoFix</u>	Enables the DFT Compiler AutoFix utility.
<u>EnableDftShadowLogic</u>	Enables the DFT Compiler Shadow LogicDFT utility for this design.
<u>EnableScanCompression</u>	When true enables scan compression. For details on DFTMAX compression options, please refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.
<u>EncryptText</u>	Encrypt text as it is being written.
<u>EndBit</u>	Tag specification ending position.
<u>EndBitExpr</u>	Indicates the original EndBit expression (subscripted representation) which was read in from RTL file in LoadDesigns activity
<u>EndBitsExpr</u>	Indicates the original EndBits complete

coreTools Command Reference Index

	expression which was read in from RTL file in LoadDesigns activity
<u>Endian</u>	Are the elements in this address block or interface big or little endian
<u>EnumValues</u>	an enumeration of the legal attribute values
<u>EnvCheck</u>	EnvCheck is a subscripted attribute that may be attached to a coreKit. The user may call verify_environment from coreConsultant to verify the environment based on the CheckEnv attribute.
<u>ExcludeLibCells</u>	Will cause a set_dont_use \ true to be placed on the library cells specified.
<u>ExcludeLoadPatterns</u>	A list of glob-style patterns to match file names against. Files that match an entry in this list will not be added to the coreKit.
<u>Exists</u>	Does this deliverable exist?
<u>ExportDefineToFile</u>	Indicates a 'define whose value should be written to the indicated file as part of the configuration process.
<u>ExternalTristates</u>	Determines the disabling option during scan shift for all tristate nets that drive output ports of a design. disable_all disables all drivers. enable_one disables all but one driver. no_disabling specifies not to insert disabling logic.
<u>ExtraSearchPathAppend</u>	List of additional appended search_path entries for Design Compiler.
<u>ExtraSearchPathPrepend</u>	List of additional prepended search_path entries for Design Compiler.

E

<u>FcAuxSynthesisScript</u>	Specifies a FC script to be executed prior to specific optimization phase.
<u>FcAuxSynthesisScriptComment</u>	Specifies a comment to be issued for the corresponding FcAuxSynthesisScript.
<u>FCExcludeLibCells</u>	The attribute specifies the purposes for the library cells. The purpose is passed into the set_lib_cell_purpose command.
<u>FCIncludeLibCells</u>	The attribute specifies the purposes for the library cells. The purpose is passed into the set_lib_cell_purpose command.
<u>fc_shellAppOption</u>	Sets application options to the specified value on the specified

coreTools Command Reference Index

<u>fc_shellAppOptionComment</u>	block. Settings to be used for Fusion Compiler (fc_shell). Add a comment for the application options being introduced in the tool intent.
<u>FeedThroughConnect</u>	This input port feeds-through to the output ports listed. Used in automatic connections to prevent connections to these pins. Connections will bypass the component and connect items that would be connected to both pins directly.
<u>FileContentType</u>	Content type of a file or file group
<u>FilePerms</u>	Read/write/execute permissions for a file or file group
<u>FixHold</u>	Fix hold time violations for this clock.
<u>fm_shellVariable</u>	Variable settings to be used for fm_shell. Example: set_design_attribute {fm_shellVariable[varName]} value.
<u>fm_shellVariableComment</u>	Comment to be issued for the corresponding subscript of fm_shellVariable.
<u>ForceTCBToUseComponentUndefFile</u>	If GenericAttr[ComponentUndefFile] is set on the design, use that while writing out technology cell macro undef directives. The file option is used otherwise.
<u>FormalVerificationAuxScript</u>	Specifies a script that is to be run in Formality after the reference and implementation designs have been loaded but just before the verify command.
<u>FormalVerificationAuxScriptComment</u>	Specifies a comment to be issued for the corresponding FormalVerificationAuxScript.
<u>FpgaPadType</u>	Arguments to be passed to the set_fpga_pad_type command for FPGA synthesis.
<u>FpgaPortIsPad</u>	Specifies that this port is a primary I/O for FPGA synthesis.
<u>FpgaPreferTmg</u>	Specifies which synthetic library FPGA synthesis will use for synthetic operators. See the man page for fpga_prefer_tmg in fpga_shell.
<u>FunctionDefinition</u>	The Tcl code to implement this

G

<u>GenerateIf</u>	This cell or port is conditionally generated
<u>GlobalSlotNumber</u>	Provides access to the global slot number stored in the SlotNumber parameter on the given interface
<u>GroupImage</u>	Image to use in pre-text for signal, parameter, and memory map groups.
<u>GroupImageAttrs</u>	Attributes to use for xml docbook:imagedata generated from GroupImage.
<u>GroupImageTitle</u>	Title to use for figures displayed with GroupText or GroupImage.
<u>GroupName</u>	Group similar items together in GUI displays
<u>GroupText</u>	Text that is displayed as pre-text for signal, parameter, and memory map groups.
<u>GroupUserLabel</u>	User label of the parameter group

H

<u>HasLibertyModel</u>	Indicates that this design has a Liberty model and therefore no HDL file analysis is required for synthesis.
<u>hdlin_enable_upf_compatible_naming</u>	Tracks the value of the like-named dc_shell variable
<u>HdlLibrary</u>	HDL library into which to analyze this file or file group
<u>HdlSourceFileNames</u>	Name(s) of the file that defines this design or package, and any included files.
<u>HdlTypeName</u>	HDL type name of this parameter
<u>HdlUnquoted</u>	Indicates that when string param is written in HDL it should not be quoted.
<u>HdlValue</u>	How the value is formatted in the given language
<u>HeaderNameFormat</u>	Defines the name format for the header reference for the given attribute on this register or register field.
<u>HelpUrl</u>	URL of the documentation file for an item
<u>HierarchicalIsolation</u>	When true, hierarchical isolation logic is built, so that dedicated

<u>HighFanout</u>	subdesign scan-out signals are gated by the design scan enable signal. This input port has a high fanout.
L	
<u>IconPath</u>	Path to an icon for this item This port is 'ideal'. It has infinite drive, no resistance, and directly attached nets are not checked for transition or capacitance violations.
<u>IdealPort</u>	When true , insert_dft/compile -scan identifies shift-registers in the design and keep only the first flop as a scan flop. Scan Architecting in preview_dft/insert_dft will group the flops that form shift registers as a single scan group named shift_register_identified_??. preview_dft -show all will show the shift registers identified. preview_dft -script writes out the shift-registers as scan groups. If the scan style is not multiplexed_flip_flop, insert_dft/compile -scan ignores this option.
<u>IdentifyShiftRegisters</u>	
<u>IfUnconnected</u>	Connection to make if left unconnected. Indicates whether the deliverables for this filegroup are required or optional
<u>Importance</u>	
<u>ImportedWorkspaceComponentFile</u>	Indicates the file used to import a workspace component. Will cause a set_dont_use \ false to be placed on the library cells specified.
<u>IncludeLibCells</u>	
<u>InputDelay</u>	Delay constraint for an input port Specifies the resistance on the net driven by an input port.
<u>InputResistance</u>	
<u>InsertEndOfChainLockupLatch</u>	When true lockup latches will be inserted at the end of scan chains. Set to true to the file item if tech cell defn insertion should be done at top
<u>InsertTechCellBindMacroAtStart</u>	
<u>InsertTestPoints</u>	Controls whether observe and control points are inserted into this design

coreTools Command Reference Index

<u>Install</u>	to enhance testability.
<u>InstallUserWorkDir</u>	Should this group be installed
<u>InstallWhen</u>	How to install a file or file group into a user workspace
<u>InstanceClockNames</u>	When should file group be installed? Names to use when creating clocks during
<u>IntentFilesProcessed</u>	InitializeSubsystemConstraints. Set to true after intent files or processed.
<u>InterClockHoldFallFallUncertainty</u>	Uncertainty applied from falling edge of source clock to the falling edge of the destination clock for hold delay calcuations
<u>InterClockHoldFallRiseUncertainty</u>	Uncertainty applied from falling edge of source clock to the rising edge of the destination clock for hold delay calcuations
<u>InterClockHoldRiseFallUncertainty</u>	Uncertainty applied from rising edge of source clock to the falling edge of the destination clock for hold delay calcuations
<u>InterClockHoldRiseRiseUncertainty</u>	Uncertainty applied from rising edge of source clock to the rising edge of the destination clock for hold delay calcuations
<u>InterClockSetupFallFallUncertainty</u>	Uncertainty applied from falling edge of source clock to the falling edge of the destination clock for setup delay calcuations
<u>InterClockSetupFallRiseUncertainty</u>	Uncertainty applied from falling edge of source clock to the rising edge of the destination clock for setup delay calcuations
<u>InterClockSetupRiseFallUncertainty</u>	Uncertainty applied from rising edge of source clock to the falling edge of the destination clock for setup delay calcuations
<u>InterClockSetupRiseRiseUncertainty</u>	Uncertainty applied from rising edge of source clock to the rising edge of the destination clock for setup delay calcuations
<u>InterfaceGroup</u>	Used to indicate that an interface is part of a 'connection' group.
<u>InterfaceIsUsed</u>	Indicates that the interface object is used on its parent interface instance.

coreTools Command Reference Index

<u>InterfaceLabel</u>	The label to be used in the display of interfaces/connections in the ad component tree dialog
<u>InterfaceLink</u>	Indicates the association to a design port or parameter (top-level core design). The design name can include \@ and {} in the attribute value, and allows a [\@] index annotation.
<u>InterfaceSize</u>	Number of required bits on a design port associated with this interface port.
<u>InterfaceType</u>	Indicates the interface type for the given interface.
<u>InterfaceTypeName</u>	The customized name for each interface type.
<u>InternalTristates</u>	Determines the disabling option during scan shift for all tristate nets that do not drive output ports of a design. disable_all disables all drivers. enable_one disables all but one driver. no_disabling specifies not to insert disabling logic.
<u>IPEnvList</u>	Allows additional IP specific env vars to be considered for running an activity. These will be overridden by UserEnvList activity parameter if same exist
<u>IsAddressable</u>	Is true if the interface bus definition is addressable.
<u>IsArray</u>	Indicates that the parameter is an array or not.
<u>IsBehavioral</u>	Indicates that the given port/net represents a behavioral connection.
<u>IsCCDTarget</u>	Concurrent Clock and Data (CCD) Optimization technology to enable users to meet frequency goals on high performance designs. When the attribute is set to true the clock is becomes part of the path groups for targeted concurrent clock and data (CCD) optimization. Refer to man page for 'ccd.targeted_ccd_path_groups' in Fusion Compiler reference documentation.
<u>IsComplete</u>	This activity is complete
<u>IsConnected</u>	Is the subsystem port, component pin, or interface connected?
<u>IsControlValue</u>	

coreTools Command Reference Index

	Indicates that the interface parameter value is used as control-only value. As a consequence the parameter value impacts interface setup but not the interface configuration.
<u>IsDanglingIntfAllowed</u>	Indicates whether an interface can be left without the corresponding consumer/provider.
<u>IsEnabled</u>	This activity is enabled
<u>IsExternalReset</u>	The attribute is set to true when any subscript of ResetProperties is explicitly set.
<u>IsFullyModeled</u>	Indicates that the design contents are fully modeled in coreTools.
<u>IsHexVal</u>	This value is in hexadecimal format
<u>IsTriState</u>	Is the port a tri-state?
<u>IsUpToDate</u>	Is this deliverable up-to-date when compared to disk?
<u>IsWorkspaceComponent</u>	Indicates the file used to import a workspace component.

J K

<u>KbVersion</u>	Knowledge base version identifier.
------------------	------------------------------------

L

<u>Label</u>	Label to be used for this item in GUI dialogs
<u>LicenseForOpen</u>	The licensed used for opening the knowledgebase.
<u>LockedInTemplate</u>	Indicate if the parameter/cell/interfaceinstance should be locked in template
<u>LogicalLeft</u>	Indicates the logical left bit index of the interface port to be connected.
<u>LogicalName</u>	Indicates the logical (abstract) name for this port/pin. Used to enable connections with matching LogicalName values.
<u>LogicalRight</u>	Indicates the logical right bit index of the interface port to be connected.

M

<u>MacroPrefixFileList</u>	User specified list of files which need to be processed for macro prefixing. The
----------------------------	--

coreTools Command Reference Index

<u>MapBlockIndividually</u>	paths should correspond to the installation path in the workspace.
<u>MapSubblocksIndividually</u>	Map this design individually in Design Compiler. Note that this attribute does not control mapping for PSYN strategies.
<u>MaxArea</u>	Map this design by individually compiling its subdesigns.
<u>MaxCap</u>	Maximum area constraint for current design.
<u>MaxControlPoints</u>	Maximum capacitance value for a port or design.
<u>MaxFallInputDelay</u>	Maximum number of control points to be inserted to enhance testability. When TestabilityMethod==control_and_observe for XG mode the larger of MaxObservePoints and MaxControlPoints will be used for the -max_test_points value for set_testability_configuration.
<u>MaxFallInputDelayFallingEdge</u>	Maximum falling edge delay constraint for an input port
<u>MaxFallOutputDelay</u>	Maximum falling edge delay constraint for an input port relative to a falling clock edge
<u>MaxFallOutputDelayFallingEdge</u>	Maximum falling edge delay constraint for an output port
<u>MaxFallTransitionDelay</u>	Maximum falling edge delay constraint for an output port relative to a falling clock edge
<u>MaxFanout</u>	Maximum transition fall time for an ideal clock.
<u>MaximumScanChainLength</u>	Maximum fanout for an input port or a design.
<u>MaxInputDelay</u>	The maximum length a scan chain may be. Setting the value to 0 will allow DFT Compiler to determine the maximum scan chain length.
<u>MaxInputDelayFallingEdge</u>	Maximum delay constraint for an input port
<u>MaxLoads</u>	Maximum delay constraint for an input port (relative to the falling clock edge)
<u>MaxObservePoints</u>	Maximum number of loads for this connection.
	Maximum number of observe points to be inserted to enhance testability. When TestabilityMethod==control_and_observe for XG mode the larger of MaxObservePoints and MaxControlPoints will be used for the -max_test_points value for

coreTools Command Reference Index

<u>MaxOutputDelay</u>	set_testability_configuration. Maximum delay constraint for an output port
<u>MaxOutputDelayFallingEdge</u>	Maximum delay constraint for an output port (relative to the falling clock edge)
<u>MaxRiseInputDelay</u>	Maximum rising edge delay constraint for an input port
<u>MaxRiseInputDelayFallingEdge</u>	Maximum rising edge delay constraint for an input port relative to a falling clock edge
<u>MaxRiseOutputDelay</u>	Maximum rising edge delay constraint for an output port
<u>MaxRiseOutputDelayFallingEdge</u>	Maximum rising edge delay constraint for an output port relative to a falling clock edge
<u>MaxRiseTransitionDelay</u>	Maximum transition rise time for an ideal clock.
<u>MaxTransition</u>	Maximum transition time for an input port or design.
<u>MaxValue</u>	Maximum value allowed
<u>MaxWriteConstraint</u>	Defines a maximum value constraint to be met when writing the given register field.
<u>MemoryAccess</u>	Access type for this memory item
<u>MemoryAccessDescription</u>	Access type for this memory item
<u>MemoryMap</u>	Holds the memory map that this slave interface refers to.
<u>MemoryMode</u>	Set all the valid Memory Modes active for the design
<u>MemoryRange</u>	Specifies the address range of the registerArray, addressBlock or addressBank
<u>MemoryUsage</u>	Specifies the usage of this addressBlock or addressBank.
<u>MemoryWidth</u>	Specifies the bit width of an address block or bank
<u>MergeCandidate</u>	Indicates that a pin/port is a candidate for port merging via merge_ports.
<u>MessageLimitDC</u>	For each subscript, injects 'set_message_info -id \ -limit \' in the synthesis RM scripts.
<u>MessageLimitFC</u>	For each subscript, injects 'set_message_info -id \ -limit \' in the synthesis RM scripts.
<u>MessageStopOnDC</u>	If true, for each subscript, injects 'set_message_info -id \ -stop_on' in the synthesis RM scripts.
<u>MessageStopOnFC</u>	

coreTools Command Reference Index

	If true, for each subscript, injects 'set_message_info -id \ -stop_on' in the synthesis RM scripts.
<u>MinCap</u>	Minimum capacitance value for a port or a design.
<u>MinFallInputDelay</u>	Minimum falling edge delay constraint for an input port
<u>MinFallInputDelayFallingEdge</u>	Minimum falling edge delay constraint for an input port relative to a falling clock edge
<u>MinFallOutputDelay</u>	Minimum falling edge delay constraint for an output port
<u>MinFallOutputDelayFallingEdge</u>	Minimum falling edge delay constraint for an output port relative to a falling clock edge
<u>MinFallTransitionDelay</u>	Minimum transition fall time for an ideal clock.
<u>MinInputDelay</u>	Minimum delay constraint for an input port
<u>MinInputDelayFallingEdge</u>	Minimum delay constraint for an input port (relative to the falling clock edge)
<u>MinOutputDelay</u>	Minimum delay constraint for an output port
<u>MinOutputDelayFallingEdge</u>	Minimum delay constraint for an output port (relative to the falling clock edge)
<u>MinRiseInputDelay</u>	Minimum rising edge delay constraint for an input port
<u>MinRiseInputDelayFallingEdge</u>	Minimum rising edge delay constraint for an input port relative to a falling clock edge
<u>MinRiseOutputDelay</u>	Minimum rising edge delay constraint for an output port
<u>MinRiseOutputDelayFallingEdge</u>	Minimum rising edge delay constraint for an output port relative to a falling clock edge
<u>MinRiseTransitionDelay</u>	Minimum transition rise time for an ideal clock.
<u>MinValue</u>	Minimum value allowed
<u>MinWriteConstraint</u>	Defines a minimum value constraint to be met when writing the given register field.
<u>MonitoredComponent</u>	Defines a component as a monitor and indicates which type of component can be monitored. Value should be a component VLNV.
<u>MonitoredInterface</u>	Defines a component as application side VIP, identifying the type of interface to be connected to (SPIRIT VLN), the SPIRIT

coreTools Command Reference Index

interface type, and the name of the VIP interface to connect to.

N

<u>Name</u>	The name of the item
<u>NumberOfScanChains</u>	The number of scan chains to be created. This value is passed to the set_scan_configuration command with the -chain_count option. For Internal_scan, this is also the number of scan IO ports created. The default value is the number of test clocks in the design.
<u>NumberOfScanCompressionChains</u>	In ScanCompression_mode, this parameter sets the number of scan chains to be created internally. Setting it to 0 will let coreTools decide the number of scan chains in the design based on the number of ScanCompressionIOCount strategy parameter.

O

<u>ObservePointsPerScanCell</u>	Specifies the number of observe points that can be shared per scan cell. When TestabilityMethod==control_and_observe for XG mode the larger of ObservePointsPerScanCell and ControlPointsPerScanCell will be used for the -test_points_per_scan_cell value for set_testability_configuration.
<u>OneRequiredGroup</u>	Specifies that either this deliverable (filegroup) or another deliverable with the same value for this attribute in the the parent group is required.
<u>OneWrapperClock</u>	If the option is set to TRUE, a single wrapper clock is used for all wrapper cells of the design. If shared wrapper cells are used in wrapping the design, the functional clocks of the shared wrapper cells are muxed with the wrapper clock in wrapper test modes by the core wrapper application. If the option is set to FALSE, a dedicated wrapper clock is used for all dedicated wrapper cells of the design and functional clocks are used for shared wrapper cells of the design.
<u>OpenAllTreeItems</u>	should all tree items be displayed open for tree dialogs

coreTools Command Reference Index

<u>OperatingConditionsBest</u>	Best-case operating condition to use when evaluating timing for this design.
<u>OperatingConditionsWorst</u>	Worst-case operating condition to use when evaluating timing for this design.
<u>OptimizationPriorities</u>	Primary and secondary implementation goals for this design.
<u>OptimizeArithmetic</u>	Should this design be optimized at the arithmetic level prior to initial mapping?
<u>Optional</u>	Indicates that the interface object may not be linked to a design object, or may not exist.
<u>OptionalAssociation</u>	Indicates that the interface object must not be associated to a design object in the parent interface instance.
<u>OutputDelay</u>	Delay constraint for an output port
<u>Overconstrain</u>	Percentage by which delay constraints should be tightened during initial mapping.

P

<u>ParallelJobCpuLimit</u>	Specifies the number of parallel compile jobs that may be run.
<u>ParameterCheckCmd</u>	Global check command for a number of parameters.
<u>ParameterInfo</u>	This attribute holds hints to build parameter dialog.
<u>Parameters</u>	Parameters used to configure this item. Subscript for that param.
<u>ParamValueFromDesign</u>	Indicates that the interface parameter comes from the corresponding design parameter.
<u>ParentWireLoad</u>	wireload model that will be used one level above this design.
<u>PhysicalLeft</u>	Indicates the physical left bit index of the interface port to be connected. Denotes that this design is to be contained in one contiguous physical region of the chip. Also controls the mapping of the design in Physical Compiler as MapBlockIndividually controls the mapping of designs in Design Compiler. In Design Compiler there are no restrictions on subdesigns during compilation, but in Physical Compiler a design that is compiled may not contain any subdesigns that have been compiled separately. When in doubt leave this attribute set to FALSE.
<u>PhysicalRegion</u>	
<u>PhysicalRight</u>	Indicates the physical right bit index of the interface port to be connected.

coreTools Command Reference Index

<u>PingTestMask</u>	Obsolete attribute - Use UVM testing now.
<u>PinLoadCount</u>	Expected external fanout value for a port.
<u>PinLoadType</u>	Type of cell expected to load this port (externally).
<u>PortDirection</u>	Signal flow direction of this port
<u>PortFunction</u>	Function of the port/pin/bit: S(ource), L(oad), or B(oth)
<u>PortWidth</u>	Indicates the width of the port.
<u>PostPromptCmd</u>	Command(s) to be evaluated when completing an activity, after the parameters are successfully validated.
<u>PowerDomain</u>	Indicates that the given port is associated with the indicated power domain (which must be enumerated in the PowerDomains attribute on the containing design).
<u>PowerDomains</u>	Defines the set of valid power domain names for this design.
<u>PredefinedInoutPorts</u>	List of inout ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.
<u>PredefinedInputPorts</u>	List of input ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.
<u>PredefinedOutputPorts</u>	List of output ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.
<u>PrePromptCmd</u>	Command(s) to be evaluated before querying and validating the parameters for an activity
<u>PreserveDesignWare</u>	Indicates that the DesignWare parts in this design should not be ungrouped.
<u>PreserveHierarchyFromTop</u>	Defines lower level designs and cells to be preserved during synthesis.
<u>PreserveMuxOps</u>	Indicates that the MuxOps in this design should not be ungrouped.
<u>PrimeProfileAllowNone</u>	If true, users are allowed to skip Prime Profile application in SelectPrimeProfile activity.
<u>PrimeProfileCurrent</u>	Design attribute to store the current prime profile.
<u>PrimeProfileDefaults</u>	Design attribute to store Default Prime Profile tags.
<u>PrimeProfileNote</u>	A note to be displayed in SelectPrimeProfile activity.

coreTools Command Reference Index

<u>PrimeProfilesValid</u>	Design attribute to store valid prime profile tags.
<u>ProjectID</u>	Used to record the project ID associated with a workspace.
<u>ProjectRootDir</u>	The logical root directory of a core project.
<u>pt_shellVariable</u>	Variable settings to be used for pt_shell. Example: set_design_attribute {pt_shellVariable[varName]} value.
<u>pt_shellVariableComment</u>	Comment to be issued for the corresponding subscript of pt_shellVariable.

Q R

<u>RALAccessType</u>	Defines the VMM RAL access type for this register field. In most cases this is calculated via the MemoryAccess, WriteBehavior, and ReadAction values for the field. This is typically only set explicitly when the a0 or a1 types are desired.
<u>RALAdditionalFieldAttributeText</u>	For RAL (Register Abstraction Language) files, this attribute defines additional text to be added to the definition of each RAL field. This attribute can be used to specify RAL constraints.
<u>RalListInfo</u>	Specifies the Ral list info for tb mode used for the workspace.
<u>RALSuppressionComment</u>	Reason for skipping test on register, register field or register array. Must not be left empty.
<u>ReadAction</u>	Indicates that reading the given field will cause the field value to be modified as described by the value of this attribute.
<u>ReadActionDescription</u>	Indicates that reading the given field will cause the field value to be modified as described by the value of this attribute.
<u>ReadActionModifier</u>	Indicates user-defined reading action when ReadAction is set to 'modify'. It implies this read action is not covered by any of other allowed ReadAction values which are: {clear, set}. This attribute is only valid when ReadAction is set to 'modify'
<u>ReadOnlyInterface</u>	Indicates that this interface connection cannot be modified manually.

coreTools Command Reference Index

<u>ReadOnlyParam</u>	The value of this parameter is read-only
<u>ReadOnlyWorkspaceComponent</u>	Indicates the workspace component will remain read-only after completing specify subsystem.
<u>ReferenceClock</u>	Indicates a generated clock defined for reference purposes only. A reference clock is used to enable clock linking in subsystem assembly but is not written to any SDC file.
<u>ReferenceClockName</u>	For a virtual clock, stores name of the reference real clock in the same hierarchy.
<u>RegisterArrayDimensions</u>	Specifies the size of the RegisterArray.
<u>Registered</u>	Indicates that the given port is registered within the component. It is either directly connected to a register or connected via only buffers or inverters with no intervening logic.
<u>RegisteredBy</u>	Indicates that the given port is registered within the component. It is either directly connected to a register or connected via only buffers or inverters with no intervening logic.
<u>RegisterResetMask</u>	Mask to be anded before comparing to the RegisterResetValue
<u>RegisterResetValue</u>	Value of the register at reset
<u>RegisterSize</u>	Specifies the size of the Register or register field in bits
<u>RemoveIfEmpty</u>	Specifies that this file can be deleted from disk if it is empty.
<u>ReplaceInHDL</u>	If true on a VHDL package parameter, then automatically replace the parameter definition in the source code. Defaults to false if there are no annotations on the parameter in the VHDL. For verilog 2001, set to false for conditional cells defined through generate-conditional statements.
<u>RequiredExPortDirection</u>	Required design port direction when associated with an exported interface port.
<u>RequiredPortDirection</u>	Required design port direction when associated with an interface port.
<u>Reserved</u>	Indicates that the bits Reserved (meaning unspecified or undocumented bitFields).
<u>ResetProperties</u>	

	Support modeling of external reset signals. active-low : active low reset active-high : active high reset synchronous : reset is synchronous ideal : treat as ideal network for synthesis no-dft : do not include DFT reset signal definition no-vcspyglass : do not include SpyGlass reset signal definition no-synthesis : do not include in synthesis constraints
<u>RtlaAuxScript</u>	Specifies a RTLA script to be executed prior to specific optimization phase.
<u>RtlaAuxScriptComment</u>	Specifies a comment to be issued for the corresponding RtlaAuxScript.
<u>RTLStaticCheckingAsSubblock</u>	The attribute indicates that design needs abstract generation and to verified using bottom up flow using VC Spyglass in coreAssembler.
<u>rtl_shellAppOption</u>	Sets application options to the specified value on the specified block. Settings to be used for RTL Architect (rtl_shell).
<u>rtl_shellAppOptionComment</u>	Add a comment for the application options being introduced in the tool intent.
<u>RunOptions</u>	Additional options for the given run style. For remote this should be a hostname. For lsf and uge this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
<u>S</u>	
<u>ScanBlockIndividually</u>	Causes scan insertion to be performed on this design when it is stand-alone.
<u>ScanCompressionConfiguration</u>	The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, please refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter

coreTools Command Reference Index

	4, 'Managing X Values in Scan Compression'.
<u>ScanCompressionTestClock</u>	Sets a test clock for scan compression from the list of test clocks available.
	The value passed along with -usage option for ScanCompression_mode test mode in define_test_mode command. See DFTMAX Compression and DFTMAX Ultra/SEQ Compression sections in the TestMAX DFT user guide for more details.
<u>ScanExclude</u>	List of sequential cells, hierarchical cells containing sequential elements, references, library cells, or designs that should not be replaced by scan cells.
	When true, internal clocks are treated as separate clocks for creating scan chains. In XG mode, setting this attribute to false is equivalent to '-internal_clocks none'. When this attribute is set to TRUE it is Used in conjunction with
<u>ScanInternalClocks</u>	ScanJumpBuffersInverters.
	This attribute will only have an effect when ScanInternalClocks is set to TRUE. For XG mode, the value of this attribute will be the value used for the -internal_clocks option to set_scan_configuration. When not in XG mode, setting ScanJumpBuffersInverters controls the setting of the DC
<u>ScanJumpBuffersInverters</u>	variable test_jump_over_bufs_invs. Setting ScanJumpBuffersInverters to single sets test_jump_over_bufs_invs to true. Setting the attribute ScanJumpBuffersInverters to 'multi' sets test_jump_over_bufs_invs to false.
	Selects the scan methodology. In full_scan methodology insert_scan makes all sequential cells scannable if they do not violate scan design rules. A setting of none selects no scan methodology for the design.
<u>ScanMethodology</u>	
<u>ScanReplace</u>	Should non-scan registers be replaced by scannable registers?
<u>ScanStyle</u>	

coreTools Command Reference Index

	Sets the test scan implementation style for this design.
<u>ScanSubblocksIndividually</u>	Causes scan insertion to be performed on the sub-designs directly instantiated in this design.
	Indicates the index for the components being installed in scratch/ungroup folder by 'create_hierarchical_component -fromComponent'. On the Subsystem KB this attribute is used to store the current count, on the design it is used to indicate the index to the install path. The install path can be created as \scratch/ungroup/Install_\
<u>ScratchInstallIndex</u>	Specifies information about an externally provided SDC file for a component.
<u>SDCConstraints</u>	SDC text to be propagated to next level of hierarchy. The text insertion point is based on the subscript specified.
<u>SDCPropagateText</u>	Name of callback command to invoke after writing SDC file
<u>SDCWriteCallback</u>	Total No of SEQ Compression Input Ports
<u>SeqCompressionInputs</u>	Total No of SEQ Compression Output Ports
<u>SeqCompressionOutputs</u>	Sequence number of an item
<u>Sequence</u>	This parameter value can be, must be, or cannot be specified in hexadecimal format
<u>SetInHex</u>	Specifies the default shared wrapper cell type.
<u>SharedWrapperCell</u>	Specifies that test point scan cells may be shared across hierarchy.
<u>ShareTestPointsAcrossHierarchy</u>	When shown in a tree display an icon if it is specified.
<u>ShowIcons</u>	Indicates whether or not routes from this component should be drawn in the schematic window.
<u>ShowRoute</u>	Tracks the value of the like-named pt_shell variable
<u>sh_product_version</u>	Specifies the definite pin side details. The values for DesignCompiler are left, right, top, bottom \& for FusionCompiler the side should be an
<u>SideConstraint</u>	

coreTools Command Reference Index

<u>SideEffects</u>	integer value starting from 1. Obsolete attribute - Use UVM testing now.
<u>SimTieOff</u>	Indicates how a pin should be tied off in the testbench if left unconnected.
<u>SimulationModel</u>	Indicates that this design is a simulation model and is not to be synthesized.
<u>SlotNumber</u>	Indicates the slot number on the bus for the given interface.
<u>SlotNumberOffset</u>	Indicates the offset (relative to 0) for numbering slots associated with this interface.
<u>SlotOrder</u>	Indicates the order of the slot on the bus for the given interface.
<u>SlotWidth</u>	Indicates the number of continuous slots allocated by attribute SlotNumber and any other 'unique-value' interface parameter on the instance. The attribute annotates an eval_param expression in context of the instance.
<u>SolutionData</u>	Stores the definition of the solution (from define_solution).
<u>SolutionEnvVarValues</u>	The solution environment vars stored on disk for use in open workspace.
<u>SolutionItemPhase</u>	Indicates the solution phase the item was created in when installing a solution.
<u>SolutionProcessed</u>	Internal status attribute indicating whether solution files have been processed or not.
<u>SpiritContainer</u>	A place holder for spirit data that we don't model so we can export as is
<u>SpiritFile</u>	Indicates the file from which a SPIRIT element originated.
<u>SpiritFileType</u>	Specifies the value of the fileType element. Default is based on FileContentType and LoadDesigns language version.
<u>SpiritInterfaceType</u>	SPIRIT interface type
<u>SpiritLibrary</u>	IP-XACT library name
<u>SpiritName</u>	IP-XACT name
<u>SpiritVendor</u>	IP-XACT vendor name
<u>SpiritVersion</u>	IP-XACT version
<u>SplitInterfaceMembers</u>	

coreTools Command Reference Index

	Indicates the list of interfaces linked with the master interface
<u>SplitInterfaceName</u>	Indicates the Master interface pointed from the slave interface.
<u>StartBit</u>	Tag specification starting position.
	Indicates the original StartBit expression (subscripted representation) which was read in from RTL file in LoadDesigns activity
<u>StartBitExpr</u>	
	Indicates the original StartBits complete expression which was read in from RTL file in LoadDesigns activity
<u>StartBitsExpr</u>	
	Defines the expression which causes the given port to exist. Assumed to be an expression of parameters that are not in the coreTools data model.
<u>StaticDefineExpr</u>	
	Specifies which static defines referenced in StaticDefineExpr values on ports should be treated as defined for constraint generation purposes.
<u>StaticDefineExprs</u>	
	Specifies a script that is to be run in PrimeTime after the design has been loaded and all clocks and constraints have been applied, just before the 'update_timing' command.
<u>StaticTimingAuxScript</u>	
	Specifies a comment to be issued for the corresponding StaticTimingAuxScript.
<u>StaticTimingAuxScriptComment</u>	
<u>SubstituteInFile</u>	Substitute text in this file at the time indicated by the subscript
<u>SymbolicNames</u>	List of symbolic names for valid values
<u>SymbolLibraryPath</u>	The path to the symbol library to be loaded when the component is displayed in a schematic.
<u>SymbolName</u>	The name of the desired symbol to use in the schematic display.
<u>SymbolPinPrefix</u>	The desired pin prefix name (Master).
<u>SymbolPinSide</u>	The desired side of the symbol to place the pin
<u>SymbolType</u>	The desired symbol type to be generated
<u>SymmetricBus</u>	Indicates the interface is symmetric which allow direct master-slave connection. The bus definition should be defined as 'slave' type, then 'master' type is just symmetric to the

SymmetricBusLink

slave. Imply 'provider-consumer' interface type for interface instance.

Indicates the association to a design port when a symmetric bus is used as a provider or a consumer. The design name can include \@\' and {\} in the attribute value, and allows a [\@\' index annotation.

SynchronousTo

Indicates that the given port is synchronous with respect to the given clock or other designer specified conditions.

SynplifyAuxSynthesisScript

Specifies a synplify script to be executed prior to specific optimization phase.

SynplifyAuxSynthesisScriptComment

Specifies a comment to be issued for the corresponding SynplifyAuxSynthesisScript.

The attribute indicates that design needs to be synthesized separately and a DDC (when synthesized with Design Compiler) or NDM libs (when synthesized with Fusion Compiler) will be generated for the design. The generated DDC or NDM will be consumed by the top design during hierarchical synthesis flow in coreAssembler.

SynthesizeAsSubblock

T

TargetConnectionBoundary

Indicates that this is a TargetConnection boundary point for exported workspace components.

TCBDisableUndefMacros

Disable writing out undefine directives for tech cell bound in TechnologyCellBinding activity.

TclAuxSynthesisScript

Specifies a DC Tcl script to be executed prior to specific optimization phase.

TclAuxSynthesisScriptComment

Specifies a comment to be issued for the corresponding DcTclAuxSynthesisScript.

TCLConfigVariable

Indicates a parameter to be defined as a TCL variable by set_tclconfig_variables.

TCLConfigVariableName

Indicates a parameter to be defined as a TCL variable by

coreTools Command Reference Index

<u>TCLConfigVariableSetArgs</u>	set_tclconfig_variables. The attribute value defines the TCL variable name. Defines default arguments for the set_tclconfig_variables command when invoked with arguments in the context of this design.
<u>TclPhysicalConstraintFile</u>	Tcl format physical constraints to be placed on the design during synthesis.
<u>TechCellMappingBind</u>	Used to store technology cell mapping bindings from specify_tech_cell_mapping.
<u>TechCellMappingDefn</u>	Used to store technology cell mapping definitions from define_tech_cell_mapping.
<u>TestabilityClockSignal</u>	Specifies the clock to be used for control or observe registers.
<u>TestabilityClockType</u>	Specifies the clock type to be used for testability enhancements.
<u>TestabilityControlSignal</u>	Selects the control signal to be used for test points. The signal should be predefined as having the TestMode signal type. If no control signal is specified, the test points will be controlled by any signal with the TestMode signal type. If no TestMode signal is found, DFT Compiler will create a new TestMode port.
<u>TestabilityMaxArea</u>	Specifies the maximum percentage of area overhead due to the observe test point structure insertion in the design. Setting this attribute to 0 leaves this as unspecified in DFT Compiler.
<u>TestabilityMethod</u>	Testability method to be used. Note that the tdvr and observe methods are identical. The tdvr setting is for backward compatibility.
<u>Testable</u>	Indicates that the given field is testable in a simple register test, provided the specified constraints are met.
<u>TestClock</u>	This clock is used in test mode.
<u>TestClockCycleTime</u>	Cycle time for clock in test mode.
<u>TestClockDefaultCycleTime</u>	Default cycle time for test clocks.
<u>TestClocksFollowSystemDomains</u>	When true, dedicated test clocks are created according to different system clocks. Thus, scan cells that have

coreTools Command Reference Index

	different system clocks have different test clocks. When false, the default, test clocks are created without considering system clocks.
<u>TestClockWaveform</u>	Waveform for clock in test mode.
<u>TestIsolate</u>	Causes set_test_isolate to be issued for a port. This functionality is not supported in XG mode.
<u>TestpointPowerSavingOn</u>	Adds logic structure (and gate) for the power saving on the XOR tree
<u>TimingExceptionCmd</u>	Command associated with this timing exception
<u>TimingExceptionFall fromValue</u>	List of path startpoints. The path must fall from objects specified
<u>TimingExceptionFall throughValue</u>	List of path through points. Applied to paths with a falling transition at specified objects
<u>TimingExceptionFall toValue</u>	List of path endpoints. Applies to paths falling at the endpoint
<u>TimingExceptionFromValue</u>	List of path startpoints
<u>TimingExceptionGroup path</u>	Name of group for paths
<u>TimingExceptionOptions</u>	Options associated with this timing exception
<u>TimingExceptionRise fromValue</u>	List of path startpoints. The path must rise from objects specified
<u>TimingExceptionRise throughValue</u>	List of path through points. Applies to paths with a rising transition at specified objects
<u>TimingExceptionRise toValue</u>	List of path endpoints. Applies to paths rising at the endpoint
<u>TimingExceptionThroughValue</u>	List of path through points
<u>TimingExceptionToValue</u>	List of path endpoints
<u>TimingExceptionValue</u>	Value for timing exception command
	Sets application options to the specified value on the specified block.
<u>tmax shellVariable</u>	Settings to be used for Synopsys TestMAX (tmax). The app variables can only be applied at setup and report generation.
	Add a comment for the application options being introduced in the tool intent.
<u>tmax shellVariableComment</u>	
<u>TmmAuxScript</u>	Specifies a TestMaxManager script to be executed prior to specific phase.
<u>TmmAuxScriptComment</u>	Specifies a comment to be issued for the corresponding TmmAuxScript.

coreTools Command Reference Index

<u>TMMRMVariable</u>	The attribute is used for storing the updated values for the Test Max Manager Reference Methodology variables, which will then be updated in the Reference Methodology scripts if the strategy parameter EnableTestMaxManagerFlow is set to 1
<u>tmm_shellAppOption</u>	Sets application options to the specified value on the specified block. Settings to be used for TestMax Manager (tmm_shell).
<u>tmm_shellAppOptionComment</u>	Add a comment for the application options being introduced in the tool intent.
<u>TMRIntentFile</u>	Specifies a TMR intent script to be executed post elaboration.
<u>TMRIntentFileComment</u>	Specifies a comment to be issued for the corresponding TMR Intent file exported in the tool intent.
<u>TypedefType</u>	Generic class of SV typedef for the item: type, enum, struct, union.
<u>TypeName</u>	The type of an item
<u>U</u>	
<u>UnconnectedPort</u>	Indicates that an output port remains unconnected.
<u>UndefinedBitValue</u>	Indicates the default reset value for undefined bits in a register.
<u>Underconstrain</u>	Percentage by which I/O delay constraints should be relaxed during initial mapping.
<u>UnelaboratedName</u>	the unelaborated name for this design
<u>UniqueConsumerConnections</u>	Show each consumer connection for this provider independently on schematic.
<u>UPFFile</u>	Specifies a UPF file to load with load_upf in Design Compiler.
<u>UsedOnInstance</u>	Specifies condition whether the interface port or parameter is used in the parent interface instance. The condition can include \@\'s in the attribute value, which makes the interface usage conditional.
<u>UserPortName</u>	Indicates a port name specified with create_connection -port to use with merge_ports.
<u>Utilization</u>	Indicates how utilized the given interface is.
<u>UVMRALAccessType</u>	

coreTools Command Reference Index

UVMTestText

This is a read-only attribute. This stores UVM Ral Attributes based on MemoryAccess ReadAction, WriteBehaviour. Please refer to UVM 1.1 users guide section 5.5.1.4. Note : As per IP-Xact mapping table, the User-defined fields will be se as rw as UVM RALF format does not support any appropriate field type

This feature provides the user ability to add user specific steps in the UVM Test phases of RAL tests in the UVM RAL Subsystem flow. The user can configure, drive and control signals in any core by using this feature. Not all phases of the RAL test can be modified, the most important ones are available for modification.

V

Value Value of this parameter

ValueRequired Value is required for this parameter

VerilogHeaderValue Defines the value for the Verilog header file for the given attribute on this register or register field.

Version The version of the Bill-of-Materials template.

VHDLDesignLibrary VHDL library for source code generated by coreAssembler

VhdlHeaderValue Defines the value for the VHDL header file for the given attribute on this register or register field.

VipConnection Indicates the name of the interface to be connected to. If two values are returned, indicates an alternate instance to be connected to. Can access global variable \::iip_instance_name via formulas if needed.

VipInitializationCall Indicates the string required to be inserted into the testbench to initialize the instantiated VIP component.

VipModelName Specifies the name of the vmm vip model

VipPackage Specifies the name of the vmm vip package

VipRandomizable Specifies if the specified VMM variable is randomizable

VipRandomRange Specifies the VMM variable's range of possible settings

VipReasonableRandomRange Specifies the VMM variable's range of reasonable settings

VipScope

coreTools Command Reference Index

	Specifies the scope of the VIP configuration field
<u>Visible</u>	Make this parameter visible in the GUI
<u>VolatileMemory</u>	True if this item refers to volatile memory
 W	
<u>Waveform</u>	clock waveform
<u>WireLoad</u>	Wireload model to use for a design.
<u>WireLoadGroup</u>	Wireload model selection group to use for automatic wireload model selection.
<u>WireLoadMinBlockSize</u>	Minimum block size to use for automatic wireload selection.
<u>WireLoadMode</u>	Indicates the wireload mode for the design.
<u>WrapBlockIndividually</u>	Causes test wrapper insertion to be performed on this design.
<u>WrapperDefaultSafeState</u>	Specifies the default safe state for the wrapper cells.
<u>WrapperExcludePorts</u>	List of ports that should not be wrapped. If the option is set to swap registers connected to ports of the design are replaced with equivalent shared wrapper cells. If the option is set to in_place, additional glue logic will be added to registers attached to ports of the design to reuse the registers in core wrapper chains.
<u>WrapperRegisterImplementation</u>	If the option is set to TRUE, shared wrapper cells will be used for register bound ports. The registers are reused in core wrapper chains.
<u>WrapperUseRegisterIO</u>	Causes test wrapper insertion to be performed on the subdesigns of this design.
<u>WrapSubblocksIndividually</u>	Indicates that writing the given field will cause a write of a value other than the one written. The written value will be modified as described by the value of this attribute.
<u>WriteBehavior</u>	Indicates user-defined writing action when WriteBehaviour is set to 'modify'. It implies this write action is not covered by any of other allowed WriteBehavior values which are: {oneToClear, oneToSet, oneToToggle, zeroToClear, zeroToSet,
<u>WriteBehaviorModifier</u>	

coreTools Command Reference Index

WriteComponentInHDL

zeroToToggle, clear, set}. This attribute is only valid when WriteBehavior is set to 'modify'

Set to false if this component should not be written in the final netlist written by coreAssembler.

WriteConstraint

Defines a constraint to be met when writing the given register field.

[API Index](#) [Command Reference](#)

AutoConnectComplete

Indicates this design was auto connected.

Definition

Type: boolean

Flags:

Default value:

Valid on: design_____

Description

Examples

See Also

AutoConnectIfUnconnected

Connection to make if left unconnected during auto-connection step.

Definition

Type: string

Flags:

Default value:

Valid on: busBit pin port _____

Description

This attribute is used to indicate the value that a particular pin should be tied to in coreAssembler, if the pin is not connected during the automatic connection process. Legal values are: zero, one, and open.

It can be set on the ports of a top-level design packaged by coreBuilder. When the design is instantiated in a coreAssembler subsystem, the attribute value will apply to the pin corresponding to the design port on which the attribute was set.

At the end of the automatic connection process, any unconnected pins which have this attribute set will be connected to the specified value. This attribute is similar to the IfUnconnected attribute, but that attribute is utilized to connect unconnected pins at the end of the manual connection process, instead of at the end of the automatic connection process.

Examples

To indicate that port A should tied to 0 if left unconnected:

```
prompt> set_port_attribute A AutoConnectIfUnconnected zero
```

To indicate that port B should be left open in the subsystem if left unconnected:

```
prompt> set_port_attribute B IfUnconnected open
```

See Also

[IfUnconnected \(3\)](#)

AutoConnectWhen

Indicates if/when the given interface should be automatically connected.

Definition

Type: string

Flags:

Default value: always

Valid on:

Description

Valid values for this attribute are 'always' and 'never', with 'always' being the default value.

This attribute is used to indicate whether or not an interface instance should be automatically connected or not. The most common use of this attribute is to specify that all instances of a particular interface should never be automatically connected. For example, this may be done for interrupts as it is difficult to connect them automatically. In this case, AutoConnectWhen would be set to 'never' on the interrupt interface definition.

Examples

To specify that interrupts should never be automatically connected:

```
prompt> set_interface_attribute InterruptSource AutoConnectWhen never
```

See Also

[API Index](#) [Command Reference](#)

AutoConnectionPreCallback

Defines user-callback to be invoked before making all automatic connections at a given level of hierarchy.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute specifies the name of a user-defined procedure to be called prior to making any automatic connections, but after predefined ports have been created.

Examples

See Also

[API Index](#) [Command Reference](#)

AutoFixAsync

Enables/Disables automatic fixing of asynchronous preset/clear violations.

Definition

Type: boolean

Flags:

Default value: =InheritValue up TRUE

Valid on: design_____

Description

Enables/Disables automatic fixing of asynchronous preset/clear violations.

Examples

Specify that asynchronous violations should be automatically fixed.

```
coreBuilder> set_design_attribute AutoFixAsync TRUE
```

See Also

[set_design_attribute \(2\)](#), [EnableDftAutoFix \(3\)](#), [ScanBlockIndividually \(3\)](#)

AutoFixAsyncLogicGate

When set to gate, specifies that the asynchronous sets and resets are autofixed with a gate driven by the test_mode signal. When set to mux sets and resets are with a multiplexer. The select input of the mux is the test_mode signal and a primary input is controlling the asynchronous set or reset in test mode.

Definition

Type: string

Flags:

Default value: =InheritValue up mux

Valid on: design_____

Description

When set to gate, specifies that the asynchronous sets and resets are autofixed with a gate driven by the test_mode signal. When set to mux sets and resets are with a multiplexer. The select input of the mux is the test_mode signal and a primary input is controlling the asynchronous set or reset in test mode.

Examples

Specify that set and reset violations are to be fixed using a gate.

```
coreBuilder> set_design_attribute AutoFixAsyncLogicGate gate
```

See Also

[set_design_attribute \(2\)](#), [EnableDftAutoFix \(3\)](#), [AutoFixAsync \(3\)](#),
[ScanBlockIndividually \(3\)](#)_____

AutoFixAsyncTestData

Port to be used as TestData for fixing set/reset violations. The port must have been defined with a DftExistingSignalType of Reset.

Definition

Type: string

Flags:

Default value: =expr {[string equal \$::shell_activity_mode ""] ? "" : {<NONE>}}

Valid on: design_____

Description

When EnableDftAutoFix is true, this attribute is used to specify a reset signal that is to be used for fixing set/reset violations. This signal will be used as:

```
set_autofix_configuration -type reset -test_data <signal>
set_autofix_configuration -type set -test_data <signal>
```

Examples

Specify that auto fix is enabled, set and reset violations are to be fixed, and that the port autoFixReset_ is active low, and is to be used as test data for fixing set and reset violations.

```
coreBuilder> set_port_attribute autoFixReset_ DftExistingSignalType Reset
coreBuilder> set_port_attribute autoFixReset_
DftExistingSignalActiveState 0
coreBuilder> set_design_attribute EnableDftAutoFix TRUE
coreBuilder> set_design_attribute AutoFixAsync TRUE
coreBuilder> set_design_attribute AutoFixAsyncTestData autoFixReset_
```

See Also

[set_design_attribute \(2\)](#), [set_port_attribute \(2\)](#), [DftExistingSignalType \(3\)](#), [DftExistingSignalActiveState \(3\)](#), [EnableDftAutoFix \(3\)](#), [AutoFixAsync \(3\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

AutoFixAsyncWithScanEnable

When true the test scan enable signal will be used to autofix asynchronous set and reset violations.

Definition

Type: boolean

Flags:

Default value: =InheritValue up FALSE

Valid on: design_____

Description

When true the test scan enable signal will be used to autofix asynchronous set and reset violations.

Examples

Specify that set and reset violations are to be fixed using the test scan enable signal.

```
coreBuilder> set_design_attribute AutoFixAsyncWithScanEnable TRUE
```

See Also

[set_design_attribute \(2\)](#), [EnableDftAutoFix \(3\)](#), [AutoFixAsync \(3\)](#),
[ScanBlockIndividually \(3\)](#)_____

[API Index](#) [Command Reference](#)

AutoFixBidirectional

Enables/Disables automatic bidirectional disabling.

Definition

Type: boolean

Flags:

Default value: =InheritValue up TRUE

Valid on: design_____

Description

When set to true this will cause bidirectionals to be disabled during test.

Examples

```
coreBuilder> set_design_attribute AutoFixBidirectional TRUE
```

See Also

[set_design_attribute \(2\)](#), [EnableDftAutoFix \(3\)](#), [ScanBlockIndividually \(3\)](#)

AutoFixClockTestData

Port to be used as TestData for fixing clock violations. The port must be associated with a clock that has TestClock==1 or have been defined with a DftExistingSignalType of a clock type.

Definition

Type: string

Flags:

Default value: =expr {[string equal \$::shell_activity_mode ""] ? "" : {<NONE>}}

Valid on: design_____

Description

When EnableDftAutoFix is true, this attribute is used to specify a clock signal that is to be used for fixing clock violations. This signal will be used as:

```
set_autofix_configuration -type reset -test_data <signal>
set_autofix_configuration -type set -test_data <signal>
```

Examples

Specify that auto fix is enabled, clock violations are to be fixed, and that the port autoFixClock is to be used as test data for fixing set and reset violations.

```
coreBuilder> set_port_attribute autoFixClock DftExistingSignalType
ScanClock
coreBuilder> set_design_attribute EnableDftAutoFix TRUE
coreBuilder> set_design_attribute AutoFixClocks TRUE
coreBuilder> set_design_attribute AutoFixClockTestData autoFixClock
```

See Also

[set_design_attribute \(2\)](#), [set_port_attribute \(2\)](#), [DftExistingSignalType \(3\)](#), [EnableDftAutoFix \(3\)](#), [AutoFixClocks \(3\)](#)

[API Index](#) [Command Reference](#)

AutoFixClocks

Enables/Disables automatic fixing of uncontrollable clock violations

Definition

Type: boolean

Flags:

Default value: =InheritValue up TRUE

Valid on: design_____

Description

Enables/Disables automatic fixing of uncontrollable clock violations

Examples

Specify that uncontrollable clock violations should be automatically fixed.

```
coreBuilder> set_design_attribute AutoFixClocks TRUE
```

See Also

[set_design_attribute \(2\)](#), [EnableDftAutoFix \(3\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

AutoloadFilegroup

Are the files in this group loaded automatically?

Definition

Type: boolean

Flags:

Default value: 0

Valid on: filegroup filegroupGroup

Description

This attribute is set by the `create_autoload_filegroup` command. You can determine if a filegroup is autoloaded by querying this attribute.

Examples

To determine if the Documentation group is autoloaded:

```
coreConsultant> get_filegroup_attribute Documentation AutoloadFilegroup
```

See Also

[get_filegroup_attribute \(2\)](#), [create_autoload_filegroup \(2\)](#),
[DefaultLoadPath \(3\)](#)

[API Index](#) [Command Reference](#)

BalanceBistSegments

By default, SoCBIST rebalances the scan channels in BIST mode. Setting BalanceBistSegments to false will override this, and SoCBIST will not concatenate the scan segments.

Definition

Type: boolean

Flags:

Default value: 1

Valid on: design_____

Description

Set this attribute to false if you do not want SoCBIST to concatenate the scan segments.

Examples

Specify that the BIST channels are not to be balanced by DFT compiler.

```
coreBuilder> set_design_attribute BalanceBistSegments FALSE
```

See Also

[set_design_attribute \(2\)](#), [BistBlockIndividually \(3\)](#)

BankAlignment

The memory alignment of this bank

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This attribute describes how the children of this address bank (the blocks and or banks) are organized in memory.

If the value is "serial", then the children are all organized sequentially in memory like so:

```
+-----+
| block0 |
+-----+
| block1 |
+-----+
| block2 |
+-----+
```

In this case, the BaseAddress of block0 is the same as the BaseAddress of this address bank. The BaseAddress of the next block is the BaseAddress of the previous block plus the MemoryRange of the previous block.

If the value is "parallel", then the children are all organized in memory like so:

```
+-----+
| block0 | block1 | block2 |
+-----+
```

In this case the BaseAddress all the blocks is the same as the BaseAddress of this bank. The BitAddressOffset for each block are adjusted to accommodate the MemoryWidth of the preceding blocks.

Examples

See Also

BaseAddress (3), MemoryRange (3), MemoryWidth (3), BitAddressOffset (3)

[API Index](#) [Command Reference](#)

BaseAddress

The base address of this item.

Definition

Type: bitfield**Flags:****Default value:** =sMem::defaultBaseAddress %item**Valid on:**

Description

This attribute specifies the base address of an address block or address bank. It can also be used on an interface instance to store the IP-XAct base address from an addressSpace.

Examples

To specify the base address of the address block map1/block1 as 0x1000:

```
coreBuilder> set_address_block_attribute map1/block1 BaseAddress 0x1000
```

See Also

[set_address_block_attribute \(2\)](#), [get_address_block_attribute \(2\)](#),[set_address_bank_attribute \(2\)](#), [set_address_bank_attribute \(2\)](#),[MemoryRange \(3\)](#), [MemoryWidth \(3\)](#), [BankAlignment \(3\)](#), [create_address_space](#)[\(2\)](#), [remove_address_space \(2\)](#), [AddressSpaceRef \(3\)](#)

[API Index](#) [Command Reference](#)

BidirectionalMode

Determines whether bidirectional ports are treated as inputs, outputs or left untouched during scan shift.

Definition

Type: string

Flags:

Default value: =InheritValue up input

Valid on: design_____

Description

Determines whether bidirectional ports are treated as inputs, outputs or left untouched during scan shift.

Examples

Specify that bidirectional ports should be left untouched during scan shift.

```
coreBuilder> set_design_attribute BidirectionalMode no_disabling
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#), [ExternalTristates \(3\)](#), [InternalTristates \(3\)](#)_____

[API Index](#) [Command Reference](#)

BistAutoFixBusses

Indicates floating bus and bus contention issues should be fixed automatically.

Definition

Type: boolean

Flags:

Default value: 1

Valid on: design_____

Description

Setting BistAutoFixBusses will cause floating busses and bus contention issues to be fixed automatically during insert_dft when EnableDftAutoFix is enabled.

Examples

Enable automatic fixing of floating busses and bus contention issues on the current design.

```
coreBuilder> set_design_attribute EnableDftAutoFix TRUE
```

```
coreBuilder> set_design_attribute BistEnableAutoFixBusses TRUE
```

See Also

[set_design_attribute \(2\)](#), [EnableDftAutoFix \(3\)](#), [BistBlockIndividually \(3\)](#), [WrapBlockIndividually \(3\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

BistAutoFixXprop

Should automatic fixing of x propagation be enabled?

Definition

Type: boolean

Flags:

Default value: =string compare [get_attribute %item -attr BistType]

xdbist

Valid on: design_____

Description

When set to true BistAutoFixXprop enables automatic fixing of x-propagation issues.

Examples

Enable automatic fixing of x-propagation issues on the current design.

```
coreBuilder> set_design_attribute EnableDftAutoFix TRUE
```

```
coreBuilder> set_design_attribute BistAutoFixXprop TRUE
```

See Also

[set_design_attribute \(2\)](#), [EnableDftAutoFix \(3\)](#), [BistBlockIndividually \(3\)](#), [WrapBlockIndividually \(3\)](#), [ScanBlockIndividually \(3\)](#)

BistBlockIndividually

Causes insertion of BIST logic into this design.

Definition

Type: boolean

Flags:

Default value: =InheritValue up -attr BistSubblocksIndividually 0

Valid on: design_____

Description

Setting BistBlockIndividually to true on a design will cause BIST logic to be inserted into the design stand-alone in its own DFT Compiler session. It also causes set_bist_configuration to be issued in DFT Compiler. The correspondence between coreTools attributes and set_bist_configuration options is as follows:

BistType -type

BistCodecCount -codec_count

BalanceBistSegments -balance_bist_segments

BistPrpgLength -prpg_length

BistChainCount -bist_chain_count

BistMaxChainLength -max_chain_length

BistPrpgShadowSi -prpg_shadow_si

BistObserveOutputs -observe_output

BistSelectorShadowSi -selector_shadow_si

BistInvertPrpgClock -invert_prpg_clock

BistDiagOutputs -diag_output

BistUseTristateMux -tri_state_mux

Examples

This example will cause BIST logic insertion to be enabled on the current design.

```
coreBuilder> set_design_attribute BistBlockIndividually 1
```

See Also

[set_design_attribute \(2\)](#), [BistSubblocksIndividually \(3\)](#),
[WrapBlockIndividually \(3\)](#), [ScanBlockIndividually \(3\)](#) [BistAutoFixBusses \(3\)](#),
[BistAutoFixXprop \(3\)](#), [BistObserveOutputs \(3\)](#), [BistPrpgLength \(3\)](#),
[BistPrpgShadowSi \(3\)](#), [BistChainCount \(3\)](#), [BistCodecCount \(3\)](#),
[BistSubblocksIndividually \(3\)](#), [BistDiagOutputs \(3\)](#), [BistType \(3\)](#),
[BistInvertPrpgClock \(3\)](#), [BistUseTristateMux \(3\)](#), [BistMaxChainLength \(3\)](#)

BistChainCount

By default, DFT Compiler concatenates lower level scan segments to arrive at the optimal number of scan chains during BIST controller insertion. You can override this number by specifying the scan configuration NumberOfScanChains or MaximumScanChainLength attributes. You must also use BistChainCount to control how many channels are hooked up to the controller. Setting this attribute to 0 allows DFT compiler to choose the number of scan chains.

Definition

Type: short

Flags:

Default value: 0

Valid on: design_____

Description

By default, DFT Compiler concatenates lower level scan segments to arrive at the optimal number of scan chains during BIST controller insertion. You can override this number by specifying the scan configuration NumberOfScanChains or MaximumScanChainLength attributes. You must also use BistChainCount to control how many channels are hooked up to the controller. Setting this attribute to 0 allows DFT compiler to choose the number of scan chains.

If you do not want to concatenate lower level BIST segments, set the attribute BalanceBistSegments to false. When this attribute is set to false BistChainCount is ignored.

Important: Although in BIST-ready mode you can override the calculated number of scan chains, the design will then not have the optimum number of scan chains, which affects the test data volume and test cycle savings.

Examples

Specify that the number of BIST channels is to be determined by DFT compiler for the current design.

```
coreBuilder> set_design_attribute BistChainCount 0
```

See Also

[set_design_attribute \(2\)](#), [BistBlockIndividually \(3\)](#), [BalanceBistSegments \(3\)](#), [BistChainCount \(3\)](#), [NumberOfScanChains \(3\)](#), [MaximumScanChainLength \(3\)](#)

[API Index](#) [Command Reference](#)

BistCodecCount
Number of BIST CODECS

Definition

Type: short

Flags:

Default value: 1

Valid on: design_____

Description

Specifies the number of CODECS to be used for the current design when inserting BIST logic.

Examples

Set the number of codecs to be used for the current design to 2.

```
coreBuilder> set_design_attribute BistCodecCount 2
```

See Also

[set_design_attribute \(2\)](#), [BistBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

BistDiagOutputs

The number of diagnostic outputs for DBIST

Definition

Type: string**Flags:****Default value:** 1**Valid on:** design_____

Description

Specifies the number of diagnostic outputs to be used when dbist is being inserted into a design.

Examples

Set the number of diagnostic outputs to 4 on the current design.

```
coreBuilder> set_design_attribute BistDiagOutputs 4
```

See Also

[set_design_attribute \(2\)](#), [BistBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

BistInvertPrpgClock

Invert the BIST clock that feeds the PRPG Registers

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

When set to true the clock that feeds the PRPG registers will be inverted.

Examples

Specify that the BIST clock that feeds the PRPG registers is to be inverted.

```
coreBuilder> set_design_attribute BistInvertPrpgClock 1
```

See Also

[set_design_attribute \(2\)](#), [BistBlockIndividually \(3\)](#)

BistMaxChainLength

Controls control the size of the counter, which increases the default upper limit on the length of the reconfigured scan chains. Setting BistMaxChainLength to 0 will allow DFT Compiler to determine the size of the counter.

Definition

Type: short

Flags:

Default value: 0

Valid on: design_____

Description

Controls control the size of the counter, which increases the default upper limit on the length of the reconfigured scan chains. Setting BistMaxChainLength to 0 will allow DFT Compiler to determine the size of the counter. If the length specified is shorter than the longest scan chain in BIST mode, the specification is ignored.

Note: The BistMaxChainLength indicates the length of the longest chain, which is used to determine the size of the counter. It does not control the length of the chain. To control the length of the longest chain use MaximumScanChainLength

Setting BistMaxChainLength to 0 in coreTools specifies that the max chain length is not to be set and will allow DFT compiler to choose the counter length.

Examples

Specify that DFT compiler is to determine the counter length for the BIST controller.

```
coreBuilder> set_design_attribute BistMaxChainLength 0
```

See Also

[set_design_attribute \(2\), BistBlockIndividually \(3\)](#)
[MaximumScanChainLength \(3\)](#)

BistMode

BIST operations to be performed. Choosing 'all' will cause the design to be made bist ready and for the codec to be inserted and integrated with the controller. For hierarchical bist insertion choose `codec_insertion` for the lower levels of hierarchy and all at the level where the controller is to be integrated with the codecs. Choose `bist_ready` to make many BIST scan chains or channels within the design without X-propagation and without wrapping the design.

Definition

Type: string**Flags:****Default value:** all**Valid on:** design_____

Description

BistMode, along with BistType, controls the built in self-test circuitry that is inserted into the design. For BistMode of all and BistType DBIST, a two step BIST insertion flow is followed by the generated coreTools scripts.

- Group design logic into a new level of hierarchy, {designName}_DUT.
- Set the `current_design` to {designName}_DUT.
- Set test configuration commands.
- Insert a test wrapper and make the design BIST ready.
- Set the current design to {designName}
- Set test configuration commands
- Insert the BIST controller and integrate it with the codecs.

Note that even though the scripts follow a two step process, the user will see all of this done in a single Design Compiler session. If the BistType is set to `xbist` with BistMode all, the design will not be wrapped.

Setting BistMode to `codec_insertion` will cause the design to be made BIST ready, for the BIST codec to be inserted. The design will not be wrapped. Use `codec_insertion` in conjunction with a BistMode of integration at a higher level of hierarchy to perform hierarchical BIST insertion.

Setting BistMode to integration will cause the integration of BIST codecs at lower levels of the hierarchy. The design will not be wrapped.

Setting BistMode to `bist_ready` will cause the creation of many small scan chains, BIST channels, to be inserted into the design. The design will not be wrapped.

Examples

The following example will cause all of the steps necessary to insert BIST channels, codecs, and controllers to be inserted into the current design. In this example the design will also be wrapped for SocTest methodology.

```
coreBuilder> set_design_attribute BistMode all
coreBuilder> set_design_attribute BistType dbist
```

The following example will also cause all of the steps necessary to insert BIST, but the design will not be wrapped in this case.

```
coreBuilder> set_design_attribute BistMode all
coreBuilder> set_design_attribute BistMode xdbist
```

The following example causes the sub-designs directly instantiated in the current design to be made BIST ready and have codecs inserted. It will also cause integration and insertion of a BIST controller in the current design. None of the designs will be wrapped regardless of the BistType setting.

```
coreBuilder> set_design_attribute BistSubblocksIndividually
coreBuilder> set_design_attribute BistBlockIndividually
coreBuilder> set_design_attribute BistMode integration
coreBuilder> set_design_attribute -design [all_subdesigns] BistMode
codec_insertion
```

The following will cause the current_design to be made BIST ready. The design will not be wrapped.

```
coreBuilder> set_design_attribute BistMode bist_ready
```

See Also

set_design_attribute (2), BistBlockIndividually (3),
BistSubblocksIndividually (3), BistType (3)

BistObserveOutputs

Controls the number of scan output pins that are inserted into the design for BistMode == xdbist.

By default, the number of scan outputs created depends on the number of scan chains: scan outputs = (scan chains/32). Setting this attribute to 0 in coreTools lets DFT compiler use the default value.

Definition

Type: short

Flags:

Default value: 0

Valid on: design_____

Description

Controls the number of scan output pins that are inserted into the design for BistMode == xdbist.

By default, the number of scan outputs created depends on the number of scan chains: scan outputs = (scan chains/32). Setting this attribute to 0 in coreTools lets DFT compiler use the default value.

Reducing the number of scan outputs reduces the size of the selector, but it also reduces the number of scan chains that can be observed during a pattern and increases the number of patterns necessary to test the design. If there are 16 scan outputs, although all of the scan chains might capture during a pattern, only 16 chains can be observed by shifting the captured data out through an observe output. If there are 4 scan outputs, then only 4 scan chains can be observed during a pattern. You cannot decrease the number of scan outputs below the default number.

Note: BistObserveOutputs and BistChainCount are mutually exclusive. If you specify both options, BistChainCount takes precedence.

Examples

Specify that DFT compiler is to use the default for the number of scan outputs for xdbist.

```
coreBuilder> set_design_attribute BistObserveOutputs 0
```

See Also

[set_design_attribute \(2\)](#), [BistBlockIndividually \(3\)](#), [BistChainCount \(3\)](#)

[API Index](#) [Command Reference](#)

BistPrpgLength

By default, SoCBIST creates a 479-bit PRPG. You can override this by setting BistPrpgLength to 257. This will cause a 257-bit PRPG to be created. For DBIST, you can use up to 512 scan chains with the 257-bit PRPG. For X-tolerant DBIST, you can use 2048 scan chains with the 257-bit PRPG.

Definition

Type: string

Flags:

Default value: 479

Valid on: design_____

Description

By default, SoCBIST creates a 479-bit PRPG. You can override this by setting BistPrpgLength to 257. This will cause a 257-bit PRPG to be created. For DBIST, you can use up to 512 scan chains with the 257-bit PRPG. For X-tolerant DBIST, you can use 2048 scan chains with the 257-bit PRPG.

Examples

Override the default behavior and specify that a 257 bit PRPG is to be created.

```
coreBuilder> set_design_attribute BistPrpgLength 257
```

See Also

[set_design_attribute \(2\)](#), [BistBlockIndividually \(3\)](#)

BistPrpgShadowSi

By default, DFT Compiler creates from one to twelve shadow scan chains for the PRPG shadow. The number of shadow scan chains created is optimized based on the length of the longest scan chain in the design. You can stipulate that DFT Compiler create a specific number of shadow scan chains instead by setting BistPrpgShadowSi to n, where n is a number from one to twelve. Setting BistPrpgShadowSi to default allows DFT Compiler to determine the number of shadow scan chains.

BistPrpgShadowSi does not affect the length of the longest scan chain. It only affects the number of shadow scan chains.

Definition

Type: string

Flags:

Default value: default

Valid on: design_____

Description

By default, DFT Compiler creates from one to twelve shadow scan chains for the PRPG shadow. The number of shadow scan chains created is optimized based on the length of the longest scan chain in the design. You can stipulate that DFT Compiler create a specific number of shadow scan chains instead by setting BistPrpgShadowSi to n, where n is a number from one to twelve. Setting BistPrpgShadowSi to default allows DFT Compiler to determine the number of shadow scan chains.

BistPrpgShadowSi does not affect the length of the longest scan chain. It only affects the number of shadow scan chains.

Examples

In this example DFT Compiler is allowed to determine the optimal number of shadow scan chains.

```
coreBuilder> set_design_attribute BistPrpgShadowSi default
```

In this example DFT Compiler will be constrained to create 4 shadow scan chains.

```
coreBuilder> set_design_attribute BistPrpgShadowSi 4
```

See Also

[set design attribute \(2\)](#), [BistBlockIndividually \(3\)](#)

BistSelectorShadowSi

The number of selector shadow chains. Setting BistSelectorShadowSi will allow DFT Compiler to determine the number of selector shadow chains based on the length of the longest scan chain.

Definition

Type: string

Flags:

Default value: default

Valid on: design_____

Description

SoCBIST by default chooses the number of selector shadow chains based on the length of the longest scan chain. You can set the number of selector shadow chains by setting BistSelectorShadowSi.

Examples

In this example DFT Compiler will determine the number of selector shadow chains based on the length of the longest scan chain.

```
coreBuilder> set_desgin_attribute BistSelectorShadowSi default
```

In this example DFT Compiler is constrained to use 2 selector shadow chains.

```
coreBuilder> set_desgin_attribute BistSelectorShadowSi 2
```

See Also

[set design attribute \(2\)](#), [BistBlockIndividually \(3\)](#)

BistSubblocksIndividually

Causes BIST logic to be inserted into the subdesigns of this design.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Setting BistSubblocksIndividually to TRUE will cause BIST logic to be inserted into the sub-designs directly instantiated in the current design.

Examples

Assume that the design top is the current design, and that the designs A, B, and C are instantiated in top. The following example would cause A, B, and C to have BIST logic inserted into them, and for each of the designs to be wrapped for SocTest methodology.

```

set_design_attribute BistSubblocksIndividually
set_design_attribute -design {A B C} BistMode all
set_design_attribute -design {A B C} BistType dbist
set_design_attribute (2), BistBlockIndividually (3),
WrapBlockIndividually (3), ScanBlockIndividually (3) BistAutoFixBusses
(3), BistAutoFixXprop (3), BistObserveOutputs (3), BistPrpgLength (3),
BistPrpgShadowSi (3), BistChainCount (3), BistCodecCount (3),
BistSubblocksIndividually (3), BistDiagOutputs (3), BistType (3),
BistInvertPrpgClock (3), BistUseTristateMux (3), BistMaxChainLength (3)

```

[API Index](#) [Command Reference](#)

BistType

Type of BIST to implement

Definition

Type: string**Flags:****Default value:** dbist**Valid on:** design_____

Description

BistType selects the type of BIST circuitry that will be inserted into a design if BIST insertion is enabled. Select dbist for deterministic logic bist, xdbist for X-tolerant BIST, and sbist for streaming dbist.

Examples

Set the current design to have dbist circuitry inserted into it when BIST is enabled.

```
coreBuilder> set_design_attribute BistType dbist
```

See Also

[set_design_attribute \(2\)](#), [BistBlockIndividually \(3\)](#), [BistMode \(3\)](#),
[WrapBlockIndividually \(3\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)**BistUseTristateMux**

Specifies that tri-state muxes be used to construct the XDBIST codec

Definition

Type: boolean**Flags:****Default value:** 1**Valid on:** design_____

Description

By default, X-tolerant DBIST synthesizes designs with tri-state multiplexers when the target library contains them. In some cases, you may not want to use tri-state multiplexers. To change the default behavior, set BistUseTristateMux to FALSE.

Specifying false results in higher test coverage in controller mode, but also increased propagation delay.

Examples

Specify that DFT Compiler not use tristate multiplexers for xdbist.

```
coreBuilder> set_design_attribute BistUseTristateMux FALSE
```

See Also

[set_design_attribute \(2\)](#), [BistBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

BitAddressOffset

Offset in bits from the base address of the register or memory block

Definition

Type: long

Flags:

Default value: =sMem::defaultBitAddressOffset %item

Valid on:

Description

This attribute specifies the offset from the base address of the address block of the first register in the address block. It also used to specify the offset in bits for this register field from the start of the register.

Examples

To specify the register field field1's bit address offset to be 16:

```
coreBuilder> set_register_field_attribute map/block0/reg1/field1  
BitAddressOffset 16
```

See Also

[BankAlignment \(3\)](#), [MemoryRange \(3\)](#) [MemoryWidth \(3\)](#)

BitRange

Returns the bit range of this field as [start]:[end]. Range is not evaluated if the start or end bit indicator is a formula.

Definition

Type: string

Flags: readOnly

Default value: =sMem::fieldBitRange %item

Valid on:

Description

Examples

See Also

[API Index](#) [Command Reference](#)

BitWidth

The number of bits in the bitfield.

Definition

Type: long

Flags:

Default value:

Valid on: attrDefn param _____

Description

This attribute is valid on bitfield parameters. It specifies the number of bits for this bitfield value. The value of this attribute is generally extracted automatically from the HDL (both VHDL and Verilog).

Examples

The following Verilog statement will cause coreBuilder to create a bitfield parameter, and to set its BitWidth to 32:

```
// reuse-pragma attr Label Rom Location
parameter rom_addr = 32'hffa
```

The MinValue will be automatically set to 0, and the MaxValue will be set to 4294967295 (The largest value to represent in 32 bits).

See Also

[MinValue \(3\)](#), [MaxValue \(3\)](#), [set parameter attribute \(2\)](#),
[get parameter attribute \(2\)](#)

BitsInLAU

The number of bits in the Least Addressable Unit (LAU)

Definition

Type: long

Flags:

Default value: 8

Valid on:

Description

All memory maps have a concept of the a Least Addressable Unit (LAU). The default is byte addressable.

Examples

See Also

BitsToRepresent

The number of bits needed to represent valid values for this type.

Definition

Type: short

Flags:

Default value:

Valid on:

Description

Examples

See Also

Bridge

Indicates that there is a bridge from this interface across the component.

Definition

Type: itemList

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on:

Description

Used to indicate a path through the component from one interface to another. The Bridge attribute indicates both that there is a path through the component and that the path does not alter the address mapping properties of the subsystem. If the path terminates and restarts are a transaction then the Channel attribute should be set instead. The subscript of this attribute indicates the incoming slot number for a multi-consumer provider. On most bridges the only subscript for which the attribute must be set is the default (0) so the subscript specification can be omitted.

Examples

See Also

[Channel \(3\)](#)

Build IP-XACT Component
Build the IPX-ACT for this core.

Parameters

Parameters Table

Name	Label	Description
Basic		
coreKitName	IP-XACT Component Name:	The name of the IP-XACT component. This name is also used for the generated XML file.
coreKitVersion	IP-XACT Component Version:	The version of the IP-XACT component.
KitVendor	IP-XACT vendor:	The 'KitVendor' field is used to sort components in the coreAssembler Add Component dialog.
KitLibrary	IP-XACT library:	The 'KitLibrary' field is used to sort components in the coreAssembler Add Component dialog.
IncludeVendorExtensions	Include Vendor Extensions	Include Synopsys vendor extensions in IP-XACT output.
IncludeSynthesisIntent	Include Synthesis Intent	Include synthesis intent in IP-XACT output.
IncludeNonHDLFileSets	Include Non-HDL File Sets	Include non-HDL file sets in IP-XACT output.
Basic/File Relocation		
CommonPrefixReplacement	Common Prefix Replacement	String to use as a replacement for the common file prefix when generating the IP-XACT component. Used to make file names relative or environment variable dependent.
CommonFilePrefix	Common File Prefix	Common prefix on files to be replaced by the value of the CommonPrefixReplacement parameter.

See Also

[Design Configuration Intent.](#)
[Add Interfaces.](#)
[Define Component Views.](#)

Build Subsystem coreKit

Run coreBuilder and package the subsystem as a coreKit.

Parameters

Parameters Table

Name	Label	Description
Basic		
ScriptsOnly	Generate Packaging Scripts only?	The packaging scripts are always generated by this activity. In many cases you need to modify these scripts to add more value to the subsystem coreKit. If this is not required (yet), then you only need to generate the packaging scripts. Otherwise, if the option is disabled, then coreBuilder is started and will package the subsystem coreKit using the just-generated scripts. Refer Execution Options about details.
coreKitName	coreKit name:	The name of the coreKit file (.coreKit will be appended).
coreKitVersion	coreKit version:	The version associated with this build of the coreKit. The version string can be in any format, but cannot contain [tab], [newline], [space], or '':.
KitVendor	coreKit vendor:	The 'KitVendor' field is used to sort components in the coreAssembler Add Component dialog.
KitLibrary	coreKit library:	The 'KitLibrary' field is used to sort components in

coreTools Command Reference Index

Name	Label	Description
		the coreAssembler Add Component dialog.
SupportFile	Support file:	The name of a file containing support contact information for this core.
LicenseFile	License file:	The file that contains the license agreement.
ScriptName	Script name:	Script to run after the coreKit is installed.
CompleteOnError	Complete with BOM check error	Complete creating coreKit with BOM check error?
GenerateIPXACTComponent	Generate SPIRIT IP-XACT component	Generate SPIRIT IP-XACT component and design files upon completion of the 'Build coreKit' activity.
PreventDesignPrefix	Prevent design prefix	Prevent coreConsultant users from setting a design prefix.
PreventSVPrefix	Prevent SV prefix	Prevent coreConsultant from inserting a prefix for SystemVerilog interfaces and packages.
PreventFilePrefix	Prevent file prefix	Prevent coreConsultant users from setting a file prefix.
PreventMacroPrefix	Prevent macro prefix	Prevent coreConsultant users from setting a macro prefix.
Execution Options		
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to

coreTools Command Reference Index

Name	Label	Description
		specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.
Implementation Tools		
dc_shellImplementationAllowed	dc_shell implementation allowed	Indicates whether or not 'dc_shell' is a valid implementation tool for this coreKit.
fc_shellImplementationAllowed	fc_shell implementation allowed	Indicates whether or not 'fc_shell' is a valid implementation tool for this coreKit.
fm_shellImplementationAllowed	fm_shell implementation allowed	Indicates whether or not 'fm_shell' is a valid implementation tool for this coreKit.
dft_shellImplementationAllowed	dft_shell implementation allowed	Indicates whether or not 'dft_shell' is a valid implementation tool for this coreKit.
QIKImplementationAllowed	QIK implementation allowed	Indicates whether or not 'QIK' is a valid implementation tool for this coreKit.
tcm_shellImplementationAllowed	tcm_shell implementation allowed	Indicates whether or not 'tcm_shell' is a valid implementation tool for this coreKit.

See Also

Initialize Subsystem Constraints.

Build coreKit

Build the coreKit for this core.

Parameters

Parameters Table

Name	Label	Description
Basic		
coreKitName	coreKit name:	The name of the coreKit file (.coreKit will be appended).
coreKitVersion	coreKit version:	The version associated with this build of the coreKit. The version string can be in any format, but cannot contain [tab], [newline], [space], or ':'.
KitVendor	coreKit vendor:	The 'KitVendor' field is used to sort components in the coreAssembler Add Component dialog.
KitLibrary	coreKit library:	The 'KitLibrary' field is used to sort components in the coreAssembler Add Component dialog.
SupportFile	Support file:	The name of a file containing support contact information for this core.
LicenseFile	License file:	The file that contains the license agreement.
ScriptName	Script name:	Script to run after the coreKit is installed.
CompleteOnError	Complete with BOM check error	Complete creating coreKit with BOM check error?
GenerateIPXACTComponent	Generate SPIRIT IP-XACT component	Generate SPIRIT IP-XACT component and design files upon completion of the 'Build coreKit' activity.
PreventDesignPrefix	Prevent design prefix	Prevent coreConsultant users from setting a design prefix.

coreTools Command Reference Index

Name	Label	Description
PreventSVPrefix	Prevent SV prefix	Prevent coreConsultant from inserting a prefix for SystemVerilog interfaces and packages.
PreventFilePrefix	Prevent file prefix	Prevent coreConsultant users from setting a file prefix.
PreventMacroPrefix	Prevent macro prefix	Prevent coreConsultant users from setting a macro prefix.
Implementation Tools		
dc_shellImplementationAllowed	dc_shell implementation allowed	Indicates whether or not 'dc_shell' is a valid implementation tool for this coreKit.
fc_shellImplementationAllowed	fc_shell implementation allowed	Indicates whether or not 'fc_shell' is a valid implementation tool for this coreKit.
fm_shellImplementationAllowed	fm_shell implementation allowed	Indicates whether or not 'fm_shell' is a valid implementation tool for this coreKit.
dft_shellImplementationAllowed	dft_shell implementation allowed	Indicates whether or not 'dft_shell' is a valid implementation tool for this coreKit.
QIKImplementationAllowed	QIK implementation allowed	Indicates whether or not 'QIK' is a valid implementation tool for this coreKit.
tcm_shellImplementationAllowed	tcm_shell implementation allowed	Indicates whether or not 'tcm_shell' is a valid implementation tool for this coreKit.

See Also

[Load Consultant Plugins.](#)
[Design Configuration Intent.](#)
[Specify Clocks.](#)
[Specify Synthesis Methodology.](#)
[Specify Port Constraints.](#)
[Specify Power Intent.](#)
[Add Interfaces.](#)
[Create Bill of Materials.](#)
[Set Up Help Menu.](#)

BusAlignment

Usually an interface port needs 'full' association to a design port with the same number of bits as InterfaceSize. Some vectored ports (busses) allow partial connection to an opposite design port. BusAlignment indicates how a narrower design port aligns within a bus of full interface size, and whether it connects on the left or right hand side of a full opposite port, or by slice-indices on a full port with a 0..InterfaceSize-1 indexing and same bit order. The bitwise alignment value is currently only available for IP-XACT components and should not be used within coreBuilder.

Definition

Type: string

Flags:

Default value: full

Valid on:

Description

This is an internal attribute which is used to track the type of bus-alignment supported by the given interface port. This is set using the -align option to the create_interface_port command.

When connecting from a narrow design port to a wider design port, this attribute is used to indicate how to make the connection. By default, both sides must have the same width. If this attribute is specified, it is legal to connect from a narrower port to a wider one. For example, a 'fromProvider' interface port may support connecting an 8 bit provider port to a 16 bit consumer port. Valid values include: full, left, right, and slice. The default is 'full' which implies that both sides must connect fully (i.e. be the same width). Values of left and right indicate to connect to the left or right side of the bus. The value of slice means that the bit specification of the provider is used to determine which bits to connect to. For example, using the same example as before, if using -align slice and connecting provider bits 2:4, then bits 2:4 will be utilized (and must be present) on the consumer.

The broadcast connection allows connecting a single bit output port to all the bits in a multi-bit input port. The output port must be single bit

Note : This attribute should never be set manually.

Examples

See Also

[create_interface_port \(2\)](#)

CHeaderValue

Defines the value for the C header file for the given attribute on this register or register field.

Definition

Type: string

Flags: subscripted

Default value: =sMem::default_HeaderValue C %item %subscript

Valid subscripts: AddressOffset BitAddressOffset MemoryAccess
RegisterResetMask RegisterResetValue RegisterSize

Default subscript:

Valid on:

Description

This attribute is used to alter how a register attribute is written to the automatically generated C header file. The attribute to be altered is indicated by the attribute subscript. This attribute is used to customize how the indicated attribute is written to the header file. The value written will be the value returned by this attribute, instead of directly using the value returned by the subscript.

Examples

Write the RegisterSize attribute value as 1 << 5 instead of 32

```
set_register_attribute MyReg {CHeaderValue[RegisterSize]} {(1 << 5)}
```

See Also

[VerilogHeaderValue \(3\)](#), [VhdlHeaderValue \(3\)](#)

CTLModels

CTLModels design attribute lists the ctl models to be read in by the synthesis tool when bottom-up synthesis is enabled. It is applicable only for hierarchical designs which have SynthesizeAsSubblock set to 1. CTL files for hardened blocks which are already synthesized may be specified at this level. For example a Subsystem has an IP (A), IP (A) is to be synthesized as a sub-block with DFT models cell1.ctl and cell2.ctl. So one can set the design attributes to include the test models during synthesis.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

Examples

See Also

CanUngroup

Indicates that the design can be replaced by a hierarchy component.

Definition

Type: boolean

Flags: subscripted

Default value: =sHdl::canUngroup %item %attrSubscript

Valid subscripts: structural synthesis

Default subscript: structural

Valid on: design_____

Description

This attribute indicates that a component can be "ungrouped" and replaced with a coreAssembler hierarchical component. The value is true if the Verilog or SystemVerilog source code for the top design of the component consists of structural HDL and other constructs that can be completely modeled by a hierarchical component. The value is false if the HDL contains anything that will be lost by replacing the design with a hierarchical component. Note that the CanUngroup value is set based on the HDL code that is visible during the LoadDesign activity (i.e. conditional code that hidden with 'ifdef is not considered). The following requirements must be met for this attribute to be set to true:

The design is the only module in the HDL file.

The design has module instantiations, and those instances are of modules that reside in their own file. The instantiated modules may have behavioral code. The instances may not be done using "generate".

There are no behavioral constructs such as initial or always in the module.

There are no constructs such as interface definitions, packages, typedefs, functions, tasks, SystemVerilog assertions, in the same file as the module. These items would be lost by ungrouping.

Certain assign statements are not modeled completely in coreTools and thus cause CanUngroup to be false. These assigns include those with delays, logic expressions, function calls, or differing number of dimensions (even if they are correct Verilog code). For example:

```
assign #10 w1 = w2;
assign w1 = !w2;
assign w1 = func_call(w2);
assign w_2_dim[0] = w_1_dim;
```

Incorrect pin connections in the HDL, such as an instance with a connection to a pin that is not present on the module, will cause CanUngroup to be false.

Certain synthesis constraints will cause the CanUngroup value with subscript "synthesis" to be false. These include generated clocks, virtual clocks, timing exceptions, and ignored SDC commands.

Verilog gate primitives (and, or, etc.) are not modeled so a design containing them cannot be ungrouped.

coreKits with plugins cannot be ungrouped.

Examples

See Also

[CanUngroupInfo \(3\)](#), [IsFullyModeled \(3\)](#)

[API Index](#) [Command Reference](#)

CanUngroupInfo

Information about design HDL file and/or design contents when CanUngroup value is false.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

The CanUngroupInfo contains a short description of why the design attribute CanUngroup is set to false. The man page for CanUngroup provides additional detail about what file and design contents will cause CanUngroup to be false.

Examples

For a design that contains processes that aren't modeled, the CanUngroupInfo value would be:

Can't ungroup items: process

For a design that has no module instantiations (cells):

No child cells.

See Also

[CanUngroup \(3\)](#), [IsFullyModeled \(3\)](#)

[API Index](#) [Command Reference](#)

Capacitance
Explicit load capacitance.

Definition

Type: float

Flags:

Default value: =InheritValue up {}

Valid on: busBit port _____

Description

This attribute can be used to specify a specific capacitance on an output port. The number specified is assumed to be in the units of the currently loaded target technology unless units are explicitly specified as part of the number (e.g. 10pf).

Use of this attribute in coreBuilder is discouraged as it implies the use of technology specific information. When packaging cores it is better to use PinLoadType and PinLoadCount to specify output loading as this is automatically converted to the proper cells and units of the users target technology.

Examples

To specify an explicit capacitance value on an output port:

```
prompt> set_port_attribute data Capacitance 100pf
```

See Also

[PinLoadType \(3\)](#), [PinLoadCount \(3\)](#)

CaseAnalysisValue

Static value that the port or pin will take on for the synthesis session. Unlike ConstantPort, CaseAnalysisValue will not cause logic to be optimized away.

Definition

Type: string

Flags:

Default value:

Valid on: busBit pin port_____

Description

Static value that the port will take on for the synthesis session. Will cause a 'set_case_analysis <port> <0 | 1>' to be placed on the port for Design Compiler. To remove static value from the port set the CaseAnalysisValue to 'none'.

Examples

To set test_mode to be a static signal with a value of logic 0 for synthesis:

```
set_port_attribute test_mode CaseAnalysisValue 0
```

To remove test_mode from the list of static signals for synthesis:

```
set_port_attribute test_mode CaseAnalysisValue none
```

See Also

Channel

Indicates that there is a channel from this interface across the component.

Definition

Type: itemList

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on:

Description

Used to indicate a path through the component from one interface to another. The Channel attribute indicates both that there is a path through the component and that the path terminates one address transaction and begins another. If the path does not alter the address mapping properties of the subsystem, then the Bridge attribute should be set instead. The subscript of this attribute indicates the incoming slot number for a multi-consumer provider.

Examples

See Also

[Bridge \(3\)](#)

CheckCmd

Command used to check values

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

The CheckCmd attribute specifies the name of a Tcl command procedure that you want to use to check a value. CheckCmd is most commonly used to implement parameter value checking for design parameters or for custom activity parameters.

For example, consider a design where three Subblocks have configurable address ranges defined by six parameters A_lower, A_upper, B_lower, and so on. All parameters use the same MinValue and MaxValue, but there cannot be any overlapping addresses. In such a case, you can create a Tcl command procedure that checks for overlapping address ranges and returns an error if any address ranges overlap.

To invoke your parameter checking command when the user specifies a value for a parameter, set the value of the CheckCmd attribute to the name of your parameter checking command. Assuming that values are required for all six parameters and that your parameter checking command procedure checks for overlap between all three address ranges, you only need to set the CheckCmd attribute on one of the 6 parameters.

The command procedure that you specify for CheckCmd must be available when coreConsultant or coreBuilder checks the user-specified parameter value. To make the command procedure available, include the command procedure definition in a coreConsultant or coreBuilder plug-in. To build a core-specific plug-in, use the coreBuilder Load Consultant Plugins activity. To build a custom activity plug-in for use in coreBuilder only, use the create_plugin_kb command.

Examples

To set the CheckCmd attribute on parameter A_upper to the name of your custom parameter checking command (MyParamCheck):

```
coreBuilder> set_parameter_attribute A_upper CheckCmd MyParamCheck
```

See Also

set_parameter_attribute (2), create_plugin_kb (2), CheckExpr (3)

CheckExpr

Check expression for a parameter

Definition

Type: boolean**Flags:** subscripted**Default value:** 1**Valid subscripts:** <valid subscripts are not statically defined>**Default subscript:** 1**Valid on:** param_____

Description

The CheckExpr attribute enables parameter value checking through the use of basic Tcl expressions. The most common use for CheckExpr is to set up dependencies between parameter values when you cannot simply set the Value attribute of a parameter to be a function of another parameter value.

CheckExpr enables either simple or complex parameter dependency checking through the use of CheckExpr subscripts. The final value of the CheckExpr attribute is the logical AND of each value expression for each subscript that evaluates to true. If the final value of CheckExpr is true, coreConsultant or coreBuilder accepts the user-specified value for the parameter. If the final value of CheckExpr is false, coreConsultant or coreBuilder rejects the user-specified value for the parameter and returns an error message that indicates why the value is not valid.

If you want to display a custom message associated with the check failure, set the CheckExprMsg attribute using the same attribute subscript. If the check fails because of the specified subscript, then the value of the CheckExprMsg attribute is shown instead of the tool generated message.

The checking procedure is called more than once during attribute checking. The implication is that CheckExpr commands should not have side effects such as posting their own messages or changing parameter or attribute values in any way.

Simple Dependency Checking with Default Subscript

The default subscript for CheckExpr is 1, which means that the CheckExpr value alone determines whether coreConsultant or coreBuilder accepts the parameter value. For example, consider a simple dependency where design parameter A_upper must be greater than parameter A_lower. You can implement this dependency by setting CheckExpr on A_upper to "\@A_upper>\@A_lower". In this case, the CheckExpr subscript is [1] (the default). Therefore, coreConsultant always checks that the user-specified

value of A_upper is greater than A_lower.

More Complex Dependency Checking with Single Subscript

As an example of using a single CheckExpr subscript, consider a design with parameters A and B. The legal values for A are 0 and 1; the legal values for B are 0 through 9. If the value of A is 0, B can be any value from 0 through 9. If the value of A is 1, the value of B must be greater than 4. In such a case, you cannot automatically set B as a function of A, but you must restrict the value of B depending on the value of A.

To enforce this dependency, set CheckExpr on parameter B. The required expression for the CheckExpr subscript is "@A==1" because B has value restrictions only if the value of A is 1.

The expression that you specify as the value of CheckExpr imposes the actual restriction on the value of parameter B. In this example, the required value for {CheckExpr[@A==1]} is "@B>4" because the value of B must be greater than 4 if the value of A is 1. (This example assumes B has MinValue = 0 and MaxValue = 9.) If "@B>4" evaluates to true (the value of B is greater than 4), the coreTool accepts the value of parameter B. If "@B>4" evaluates to false, the coreTool returns an error message that indicates why the value of B is not valid.

Complex Dependency Checking with Multiple Subscripts

To implement more complex parameter dependency checking, use multiple CheckExpr subscripts. The coreTool uses the logic AND of all value expressions for which the associated subscript is true to determine whether the parameter value is valid. For example, consider a design with parameters X, Y, and Z. If X and Y are both 0, Z is restricted only by its MinValue and MaxValue. If X = 1, Z must be greater than 10. If Y = 1, Z must be less than 20.

The following CheckExpr attribute assignments on parameter Z implement the above dependencies:

```
set_parameter_attribute Z {CheckExpr[@X==1]} @Z>10
```

```
set_parameter_attribute Z {CheckExpr[@Y==1]} @Z<20
```

The coreTool logically ANDs each value for which the subscript evaluates to true. Therefore, if X and Y are both equal to 1, the coreTool only accepts a value between 10 and 20 for parameter Z.

Examples

The following annotations in a Verilog source file impose the simple dependency that A_upper must be greater than A_lower:

```
//reuse-pragma attr MinValue 0x00000000
//reuse-pragma attr MaxValue 0xFFFFFFFF
parameter A_lower = 0x00000000;
//reuse-pragma attr MinValue 0x00000000
```



```
//reuse-pragma attr MaxValue 0xFFFFFFFF
//reuse-pragma attr CheckExpr @A_upper>@A_lower
parameter A_upper = 0x00000000;
```

The following annotations in a Verilog source file impose the dependency that if parameter A = 1, then parameter B must greater than 4:

```
//reuse-pragma attr MinValue 0
//reuse-pragma attr MaxValue 1
parameter A = 0;
//reuse-pragma attr MinValue 0
//reuse-pragma attr MaxValue 9
//reuse-pragma attr CheckExpr[@A==1] @B>4
parameter B = 0;
```

The following lines in an _Obj.tcl file for a custom activity impose the following restrictions on custom activity parameter Z:

- If X = 1, Z must be greater than 10.
- If Y = 1, Z must be less than 20.

```
set_parameter_attribute X MinValue 0
set_parameter_attribute X MaxValue 1
set_parameter_attribute Y MinValue 0
set_parameter_attribute Y MaxValue 1
set_parameter_attribute Z MinValue 0
set_parameter_attribute Z MaxValue 100
set_parameter_attribute Z {CheckExpr[@X==1]} @Z>10
set_parameter_attribute Z {CheckExpr[@Y==1]} @Z<20
```

See Also

set_parameter_attribute (2), Value (3), MinValue (3), MaxValue (3),
CheckExprMessage (3), CheckCmd (3)

CheckExprMessage

Check expression message for a parameter

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 1

Valid on: param_____

Description

The CheckExprMessage attribute specifies the message for a given CheckExpr case which is subscripted the same as CheckExpr. When the expression fails, this message will be displayed as the error message for the failure. If this attribute is not set, a default message will be generated.

Examples

X, Y, Z are three parameters for a custom activity, The following lines in an _Obj.tcl file for the custom activity impose the following restrictions and set the corresponding error messages to display when the restrictions are not met:

- If X = 1, Z must be greater than 10.
- If Y = 1, Z must be less than 20.

```

set_parameter_attribute X MinValue 0
set_parameter_attribute X MaxValue 1
set_parameter_attribute Y MinValue 0
set_parameter_attribute Y MaxValue 1
set_parameter_attribute Z MinValue 0
set_parameter_attribute Z MaxValue 100
set_parameter_attribute Z {CheckExpr[@X==1]} @Z>10
set_parameter_attribute Z {CheckExpr[@Y==1]} @Z<20
set_parameter_attribute Z {CheckExprMessage[@X==1]}
"If X = 1, Z must be greater than 10."
set_parameter_attribute Z {CheckExprMessage[@Y==1]}
"If X = 1, Z must be less than 20."

```

See Also

[set_parameter_attribute \(2\)](#), [CheckExpr \(3\)](#), [Value \(3\)](#), [MinValue \(3\)](#),
[MaxValue \(3\)](#), [CheckCmd \(3\)](#)

CheckExprWhenDisabled

Specifies whether or not CheckExpr expressions should be checked for non-visible or non-enabled parameters.

Definition

Type: long

Flags:

Default value:

Minimum value: 0

Maximum value: 2

Valid on: param_____

Description

Checking expressions defined via the CheckExpr attribute are not validated by default for parameters that are not visible (Visible == 0) or not enabled (Enabled == 0). This behavior can be changed by setting the CheckExprWhenDisabled attribute on the parameter in question. Set this attribute to the value 'check' to cause the CheckExpr validation to occur regardless of the Enabled/Visible values.

This attribute can also be used to turn off checking for visible/enabled parameters. This is generally discouraged but can be accomplished by setting The CheckExprWhenDisabled attribute to 'dontcheck'.

The default value of this attribute is 'undefined' which implies the attribute has not be explicitly set. The legal values described here (undefined, dontcheck, check) are symbolic names corresponding to actual attribute values 0, 1, and 2.

Examples

Enable checking of a non-visible parameter via annotations in RTL:

```
// reuse-pragma attr Visible 0
// reuse-pragma attr CheckExpr @B > @A
// reuse-pragma attr CheckExprWhenDisabled check
`define B 2'b0
```

See Also

[CheckExpr \(3\)](#), [CheckExprMessage \(3\)](#), [Enabled \(3\)](#), [Visible \(3\)](#)

[API Index](#) [Command Reference](#)

ClockFallLatency

The time it takes a clock signal to propagate from the clock definition point to a register clock pin for a falling transition at the clock definition point.

Definition

Type: float

Flags:

Default value: 0

Valid on: clock_____

Description

The clock network latency is the time it takes a clock signal to propagate from the clock definition point to a register clock pin. The fall latencies are the latencies for a falling transitions at the clock definition point.

The total clock latency at a register clock pin is the sum of clock source latency (ClockSourceRiseLatency and ClockSourceFallLatency) and clock network latency (ClockRiseLatency and ClockFallLatency).

Examples

To set the clock "clk" latency for a falling transition to 1ns:

```
coreConsultant> set_clock_attribute clk ClockFallLatency 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseRiseUncertainty \(3\)](#),
[InterClockHoldRiseFallUncertainty \(3\)](#), [InterClockHoldFallFallUncertainty](#)
[\(3\)](#), [InterClockHoldFallRiseUncertainty \(3\)](#),
[InterClockSetupRiseRiseUncertainty \(3\)](#),
[InterClockSetupRiseFallUncertainty \(3\)](#),
[InterClockSetupFallFallUncertainty \(3\)](#),
[InterClockSetupFallRiseUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#),
[ClockHoldUncertainty \(3\)](#), [ClockRiseLatency \(3\)](#), [ClockSourceRiseLatency](#)
[\(3\)](#), [ClockSourceFallLatency \(3\)](#), [MaxRiseTransitionDelay \(3\)](#),
[MaxFallTransitionDelay \(3\)](#), [MinRiseTransitionDelay \(3\)](#),
[MinFallTransitionDelay \(3\)](#)

[API Index](#) [Command Reference](#)

ClockGatingFallHoldCheck

Falling edge hold check for gating control signals for this clock.

Definition

Type: float

Flags:

Default value: 0

Valid on: clock_____

Description

Sets the falling clock edge hold check for gating control signals associated with the clock.

Examples

To set the falling edge hold check for gating control signals to 0.1 for the clock gatedClk:

```
set_clock_attribute gatedClk ClockGatingFallHoldCheck 0.1
```

See Also

[set_clock_attribute \(2\)](#), [ClockGatingRiseHoldCheck \(3\)](#),
[ClockGatingFallSetupCheck \(3\)](#), [ClockGatingRiseSetupCheck \(3\)](#)

[API Index](#) [Command Reference](#)

ClockGatingFallSetupCheck

Falling edge setup check for gating control signals for this clock.

Definition

Type: float

Flags:

Default value: 0

Valid on: clock_____

Description

Sets the falling clock edge setup check for gating control signals associated with the clock.

Examples

To set the falling edge setup check for gating control signals to 0.1 for the clock gatedClk:

```
set_clock_attribute gatedClk ClockGatingFallSetupCheck 0.1
```

See Also

[set_clock_attribute \(2\)](#), [ClockGatingRiseSetupCheck \(3\)](#)

[ClockGatingFallHoldCheck \(3\)](#), [ClockGatingRiseHoldCheck \(3\)](#)

[API Index](#) [Command Reference](#)

ClockGatingRiseHoldCheck

Rising edge hold check for gating control signals for this clock.

Definition

Type: float

Flags:

Default value: 0

Valid on: clock_____

Description

Sets the rising clock edge hold check for gating control signals associated with the clock.

Examples

To set the rising edge hold check for gating control signals to 0.1 for the clock gatedClk:

```
set_clock_attribute gatedClk ClockGatingRiseHoldCheck 0.1
```

See Also

[set_clock_attribute \(2\)](#), [ClockGatingFallHoldCheck \(3\)](#),
[ClockGatingFallSetupCheck \(3\)](#), [ClockGatingRiseSetupCheck \(3\)](#)

[API Index](#) [Command Reference](#)

ClockGatingRiseSetupCheck

Rising edge setup check for gating control signals for this clock.

Definition

Type: float**Flags:****Default value:** 0**Valid on:** clock_____

Description

Sets the rising clock edge setup check for gating control signals associated with the clock.

Examples

To set the rising edge setup check for gating control signals to 0.1 for the clock gatedClk:

```
set_clock_attribute gatedClk ClockGatingRiseSetupCheck 0.1
```

See Also

[set_clock_attribute \(2\)](#), [ClockGatingFallSetupCheck \(3\)](#), _____

[ClockGatingFallHoldCheck \(3\)](#), [ClockGatingRiseHoldCheck \(3\)](#) _____

ClockGatingSignals

Use this attribute to specify a list of signals to include/exclude when inferring flip-flops with gated clocks.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: exclude include

Default subscript:

Valid on: design_____

Description

This attribute is used to provide extra control over insertion of clock gating logic which occurs during elaboration when clock gating is enabled. By default, gating is done on signals based on information such as register width. However, this attribute can be used to specifically include or exclude signals from consideration for clock gating.

Designs which have this attribute set on them will have a corresponding `set_clock_gating_signals` command generated for them if clock gating is enabled during synthesis. This translates directly into:

```
set_clock_gating_signals -include <signal list> -exclude <signal list>
where the included signals are derived from the value of
ClockGatingSignals with the 'include' subscript and the excluded signals
are derived from the value of ClockGatingSignals with the 'exclude'
signal.
```

Examples

Clock gating is going to be done for the current design, but the signals X and Y must not be gated, and the signal Z must be gated:

```
prompt> set_design_attribute {ClockGatingSignals[include]} {X Y}
prompt> set_design_attribute {ClockGatingSignals[exclude]} {Z}
```

See Also

[API Index](#) [Command Reference](#)

ClockHoldUncertainty

Hold uncertainty applied to all endpoints of the given clock.

Definition

Type: float

Flags:

Default value: =sCstr::get_default_clock_skew

Valid on: clock_____

Description

This attribute specify clock hold uncertainty applied to all paths to the endpoint.

Examples

To set the clock "clk" hold uncertainty to 1ns:

```
coreConsultant> set_clock_attribute clk ClockHoldUncertainty 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseRiseUncertainty \(3\)](#),
[InterClockHoldRiseFallUncertainty \(3\)](#), [InterClockHoldFallFallUncertainty](#)
[\(3\)](#), [InterClockHoldFallRiseUncertainty \(3\)](#),
[InterClockSetupRiseRiseUncertainty \(3\)](#),
[InterClockSetupRiseFallUncertainty \(3\)](#),
[InterClockSetupFallFallUncertainty \(3\)](#),
[InterClockSetupFallRiseUncertainty \(3\)](#), [ClockRiseLatency \(3\)](#),
[ClockFallLatency \(3\)](#), [ClockSourceRiseLatency \(3\)](#), [ClockSourceFallLatency](#)
[\(3\)](#), [MaxRiseTransitionDelay \(3\)](#), [MaxFallTransitionDelay \(3\)](#),
[MinRiseTransitionDelay \(3\)](#), [MinFallTransitionDelay \(3\)](#)

[API Index](#) [Command Reference](#)

ClockMixing

Specifies whether cells from different clock domains may be put in the same scan chain

Definition

Type: string

Flags:

Default value: =InheritValue up no_mix

Valid on: design_____

Description

Specifies whether insert_dft can include cells from different clock domains in the same scan chain. The following allowed values control the clocking of cells in scan chains to be inserted by insert_dft.

no_mix: Cells must be clocked by the same edge of the same clock.

mix_edges: Cells must be clocked by the same clock, but the clock edges can be different.

mix_clocks_not_edges: Cells must be clocked by the same clock edge, but the clocks can be different.

mix_clocks: Cells can be clocked by different clocks and different clock edges.

Examples

Specify that cells may be clocked by different clocks and different clock edges.

```
set_design_attribute ClockMixing mix_clocks
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

ClockName

Name of the clock object (possibly associated with a port)

Definition

Type: string

Flags:

Default value:

Valid on: busBit clock port _____

Description

The ClockName attribute specifies the name of a clock, either a real clock or a virtual clock. When applied to a port, the ClockName attribute identifies that port as a clock port and associates the port with the specified clock.

If the specified clock does not exist, coreBuilder creates a real clock with the specified ClockName and associates the clock with the port.

For virtual clocks, you specify ClockName when you create the virtual clock using the create_virtual_clock command.

Examples

To identify port clk_in as the clock port for the clock named clk and create the clock object clk if it does not already exist:

```
coreBuilder> set_port_attribute clk_in ClockName clk
```

To get the value of the ClockName attribute on the clock named clk:

```
coreBuilder> get_clock_attribute clk ClockName
clk
```

See Also

[set_port_attribute \(2\)](#), [get_clock_attribute \(2\)](#), [set_clock_attribute \(2\)](#),
[create_virtual_clock \(2\)](#), [get_clocks \(2\)](#)

ClockRiseLatency

The time it takes a clock signal to propagate from the clock definition point to a register clock pin for a rising transition at the clock definition point.

Definition

Type: float

Flags:

Default value: 0

Valid on: clock_____

Description

The clock network latency is the time it takes a clock signal to propagate from the clock definition point to a register clock pin. The rise latencies are the latencies for a rising transitions at the clock definition point.

The total clock latency at a register clock pin is the sum of clock source latency (ClockSourceRiseLatency and ClockSourceFallLatency) and clock network latency (ClockRiseLatency and ClockFallLatency).

Examples

To set the clock "clk" latency for a rising transition to 1ns:

```
coreConsultant> set_clock_attribute clk ClockRiseLatency 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseRiseUncertainty \(3\)](#),
[InterClockHoldRiseFallUncertainty \(3\)](#), [InterClockHoldFallFallUncertainty](#)
[\(3\)](#), [InterClockHoldFallRiseUncertainty \(3\)](#),
[InterClockSetupRiseRiseUncertainty \(3\)](#),
[InterClockSetupRiseFallUncertainty \(3\)](#),
[InterClockSetupFallFallUncertainty \(3\)](#),
[InterClockSetupFallRiseUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#),
[ClockHoldUncertainty \(3\)](#), [ClockFallLatency \(3\)](#), [ClockSourceRiseLatency](#)
[\(3\)](#), [ClockSourceFallLatency \(3\)](#), [MaxRiseTransitionDelay \(3\)](#),
[MaxFallTransitionDelay \(3\)](#), [MinRiseTransitionDelay \(3\)](#),
[MinFallTransitionDelay \(3\)](#)

ClockSetupUncertainty

Setup uncertainty applied to all endpoints of the given clock.

Definition

Type: float

Flags:

Default value: =sCstr::get_default_clock_skew

Valid on: clock_____

Description

This attribute specifies clock setup uncertainty applied to all paths to the endpoint.

Examples

To set the clock "clk" setup uncertainty to 1ns:

```
coreConsultant> set_clock_attribute clk ClockSetupUncertainty 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseRiseUncertainty \(3\)](#), _____

[InterClockHoldRiseFallUncertainty \(3\)](#), [InterClockHoldFallFallUncertainty](#)

[\(3\)](#), [InterClockHoldFallRiseUncertainty \(3\)](#), _____

[InterClockSetupRiseRiseUncertainty \(3\)](#), _____

[InterClockSetupRiseFallUncertainty \(3\)](#), _____

[InterClockSetupFallFallUncertainty \(3\)](#), _____

[InterClockSetupFallRiseUncertainty \(3\)](#), [ClockHoldUncertainty \(3\)](#), _____

[ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#), [ClockSourceRiseLatency \(3\)](#), _____

[ClockSourceFallLatency \(3\)](#), [MaxRiseTransitionDelay \(3\)](#), _____

[MaxFallTransitionDelay \(3\)](#), [MinRiseTransitionDelay \(3\)](#), _____

[MinFallTransitionDelay \(3\)](#) _____

ClockSourceFallLatency

The time it takes a clock signal to propagate from its ideal waveform origin point to the clock definition point for a falling transition at the waveform origin point.

Definition

Type: float

Flags:

Default value: 0

Valid on: clock_____

Description

Clock source latency (also called insertion delay) is the time it takes for a clock signal to propagate from its actual ideal waveform origin point to the clock definition point in the design. It can be used to model off-chip clock latency when clock generation circuit is not part of the current design. For generated clocks clock source latency can be used to model the delay from master-clock to generated clock definition point. ClockSourceFallLatency specifies clock source latency for a falling transition at the waveform origin point.

Clock source latency can be specified for ideal or propagated clocks. The total clock latency at a register clock pin is the sum of clock source latency (ClockSourceRiseLatency and ClockSourceFallLatency) and clock network latency (ClockRiseLatency and ClockFallLatency).

Examples

To set the clock "clk" source latency for a falling transition to 1ns:

```
coreConsultant> set_clock_attribute clk ClockSourceFallLatency 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseRiseUncertainty \(3\)](#), _____

[InterClockHoldRiseFallUncertainty \(3\)](#), [InterClockHoldFallFallUncertainty](#) _____

[\(3\)](#), [InterClockHoldFallRiseUncertainty \(3\)](#), _____

[InterClockSetupRiseRiseUncertainty \(3\)](#), _____

[InterClockSetupRiseFallUncertainty \(3\)](#), _____

[InterClockSetupFallFallUncertainty \(3\)](#), _____

[InterClockSetupFallRiseUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#), _____

[ClockHoldUncertainty \(3\)](#), [ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#), _____

[ClockSourceRiseLatency \(3\)](#), [MaxRiseTransitionDelay \(3\)](#), _____

[MaxFallTransitionDelay \(3\)](#), [MinRiseTransitionDelay \(3\)](#), _____

[MinFallTransitionDelay \(3\)](#) _____

ClockSourceRiseLatency

The time it takes a clock signal to propagate from its ideal waveform origin point to the clock definition point for a rising transition at the waveform origin point.

Definition

Type: float

Flags:

Default value: 0

Valid on: clock_____

Description

Clock source latency (also called insertion delay) is the time it takes for a clock signal to propagate from its actual ideal waveform origin point to the clock definition point in the design. It can be used to model off-chip clock latency when clock generation circuit is not part of the current design. For generated clocks clock source latency can be used to model the delay from master-clock to generated clock definition point. ClockSourceRiseLatency specifies clock source latency for a rising transition at the waveform origin point.

Clock source latency can be specified for ideal or propagated clocks. The total clock latency at a register clock pin is the sum of clock source latency (ClockSourceRiseLatency and ClockSourceFallLatency) and clock network latency (ClockRiseLatency and ClockFallLatency).

Examples

To set the clock "clk" source latency for a rising transition to 1ns:

```
coreConsultant> set_clock_attribute clk ClockSourceRiseLatency 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseRiseUncertainty \(3\)](#),
[InterClockHoldRiseFallUncertainty \(3\)](#), [InterClockHoldFallFallUncertainty](#)
[\(3\)](#), [InterClockHoldFallRiseUncertainty \(3\)](#),
[InterClockSetupRiseRiseUncertainty \(3\)](#),
[InterClockSetupRiseFallUncertainty \(3\)](#),
[InterClockSetupFallFallUncertainty \(3\)](#),
[InterClockSetupFallRiseUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#),
[ClockHoldUncertainty \(3\)](#), [ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#),
[ClockSourceFallLatency \(3\)](#), [MaxRiseTransitionDelay \(3\)](#),
[MaxFallTransitionDelay \(3\)](#), [MinRiseTransitionDelay \(3\)](#),
[MinFallTransitionDelay \(3\)](#)

CnctClass

Connection class(es) for this connection.

Definition

Type: string

Flags:

Default value:

Valid on: pin port _____

Description

This attribute is available and can be set but is not yet supported in any way by coreAssembler.

Indicates the connection classes associated with this port or pin. The attribute value is assumed to be a space separated list of classes to which the given connection object belongs. Two pins cannot be connected together unless they have at least one connection class in common. For example if pin A has connection class value "Generic Data", pin B has connection class value "Generic", and pin C has connection class value "Test", then the only legal connections between these pins would from pin A to pin B.

Examples

To define the port data to belong to connection classes Generic and Data:

```
prompt> set_port_attribute data CnctClass "Generic Data"
```

See Also

CombineInheritValues

Inherit attribute values from other items and combine values to form a single result

Syntax

```
string CombineInheritValues [-attr <attr>] [-sub <sub>] [-item <item>]
<up|down|pin> <cmd>
    string <attr>
    string <sub>
    string <item>
    string <up|down|pin>
    string <cmd>
```

Parameters

	Specifies an attribute from which to inherit the value instead of the attribute that you are currently specifying.
-attr <attr>	For example, you can specify a value for MaxInputDelay on an input port as a function of the MaxOutputDelay values inherited from the output ports that drive that input port.
-sub <sub>	Specifies an attribute subscript from which to inherit the value instead of the current context.
-item <item>	Item to use, instead of the context
<up down pin>	Direction from which to inherit the value. (Values: up, down, pin)
	The command to use to combine the list of values.
<cmd>	The commands typically used are logic_or, logic_and, min, max, and sum. However, you can specify any valid Tcl command.

Description

The CombineInheritValues command inherits the values of an attribute from one or more items and combines the values to form a result. The coreTools use CombineInheritValues to determine default values for several of the attributes that apply to designs. For example, the default value for the AreaEstimate attribute is {=CombineInheritValues down sum}, which means that the default value for AreaEstimate on a design is the sum of the AreaEstimate values on that designs subblocks.

You must specify the name of the Tcl command you want to use to combine the values (<cmd> argument). You can use any valid Tcl command to combine

the values. The commands that the coreTools most commonly use for the default attribute values are `logic_or`, `logic_and`, and `sum`.

You must also specify the direction from which to inherit the attribute value. The behavior of the `up`, `down`, and `pin` options differ for design attributes and port attributes. In general, you use `up` or `down` for designs and `up`, `down`, or `pin` for ports.

For designs, if you specify `down`, `CombineInheritValues` looks one level down in the hierarchy and extracts the value of the attribute you are setting from each design it finds, then combines the values according to your specified `<cmd>`. If you specify `up`, `CombineInheritValues` looks one level up in the hierarchy and extracts the value of the attribute you are setting from each design it finds, then combines the values according to your specified `<cmd>`. In general, the `down` direction is more useful for specifying design attributes.

For ports, if you specify `down`, `CombineInheritValues` looks one level down in the hierarchy and extracts the values of the attribute you are setting from the connected port on each lower level design, then combines the values according to your specified `<cmd>`. If you specify `up`, `CombineInheritValues` looks one level up in the hierarchy and extracts the value of the attribute you are setting from the connected port on each higher level design, then combines the values according to your specified `<cmd>`. For both `up` and `down`, `CombineInheritValues` looks up or down the design hierarchy for connected ports of the same directionality as the one to which you are currently specifying the attribute value. For example, if you use `CombineInheritValues up` to specify an attribute value on an input pin, `CombineInheritValues` extracts the attribute value from all input and inout pins on the next higher level design that are connected to the specified input pin and combines the values according to `<cmd>`.

If you specify `pin`, `CombineInheritValues` extracts the value of the specified from each pin of the opposite directionality that is electrically connected to the port you are setting the attribute on and combines the values according to your specified `<cmd>`. For input ports, `CombineInheritValues` gets the attribute value from each electrically connected output or inout port. For output ports, `CombineInheritValues` gets the attribute value from each electrically connected input or inout port.

To inherit from a pin, you must usually specify an attribute to inherit by using the `-attr` option, because ports of opposite directionality do not use the same attributes. For example, to specify a value for `MaxInputDelay` on an input port as a function of the `MaxOutputDelay` values inherited from the output ports that drive that input port, you must use the `-attr MaxOutputDelay` option.

In general, the `up` and `pin` directions are more useful for specifying port attributes.

Examples

To set HasArithmetic to true if HasArithmetic is true on any of the designs subblocks:

```
coreBuilder> set_design_attribute HasArithmetic
{=CombineInheritValues down logic_or}
```

To set MaxInputDelay on input port i4 to the clock cycle time minus the maximum MaxOutputDelay value on the ports that drive i4:

```
coreBuilder> set_port_attribute i4
{MaxInputDelay[clk]}
{=expr {[get_clock_attribute clk CycleTime]
- [CombineInheritValues -attr MaxOutputDelay -sub clk pin max]}}
```

See Also

[combineValues \(2\)](#), [InheritValue \(2\)](#), _____

Commands Index

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z _ _ _ _ _

A

<u>add activity hook</u>	Add a hook to an existing activity
<u>add files to filegroup</u>	Add files to the specified file group in the Bill of Materials; create the file group if it does not already exist
<u>add hdl pragma</u>	A reuse-pragma statement external to the HDL
<u>add help menu item</u>	Add an item to the console Help menu and link it to relevant documentation
<u>add html report link</u>	Add an HTML report link to the index page
<u>add instantiate component hook</u>	Add hook to execute a Tcl plugin command when instantiate_component is completed
<u>add to collection</u>	Add object(s) to a collection. Result is new collection
<u>add verify subsystem hook</u>	Execute a Tcl checker command to verify subsystem at various places
<u>add workspace hook</u>	Execute a Tcl script when a workspace operation occurs
<u>alias</u>	Creates a pseudo-command that expands to one or more words, or lists current alias definitions.
<u>all components</u>	Return sorted list of all currently loaded components in the sub-system.
<u>all designs</u>	Create a collection of all currently loaded designs
<u>all inputs</u>	Return a collection of all input and inout ports of the current_design
<u>all outputs</u>	Return a collection of all output and inout ports of the current_design
<u>all subdesigns</u>	Return a collection of all subdesigns of the current_design
<u>analyze filegroup</u>	Source intent files for this filegroup.
<u>append to collection</u>	Add object(s) to a collection. Modifies variable
<u>apropos</u>	Searches the command database for a pattern.
<u>as collection</u>	Find a collection or iter_collection by name; result a collection

coreTools Command Reference Index

<u>attach interface</u>	Attach a component interface to another component.
<u>autocomplete activity</u>	Automatically complete activities

B

<u>batch install</u>	Install a packaged coreKit into the specified installation root directory.
<u>bind tech cell mapping</u>	Specify binding requirements for mapping to a technology specific cell.
<u>build debug tarfile</u>	Build a debug tar-file
<u>byte compile file</u>	Byte compile a TCL file or text string.

C

<u>check bom</u>	Check Bill Of Materials
<u>check component for parameter loops</u>	Check design parameters for the top design for loops return error count.
<u>check env vars</u>	Check for the existence of a list of environment variables.
<u>check executables</u>	Check for the existence of a list of executables.
<u>check file for errors</u>	Check the specified file for error messages and return the number of error messages found
<u>check license</u>	Get information about a license
<u>check ParameterInfo</u>	Check the syntax and semantics of a ParameterInfo list string
<u>check parameter attribute formulas</u>	Check the formula value of an attribute on a parameter for any issues like dependancy loops.
<u>check script</u>	Statically check a script using the a static Tcl syntax checker based on TclPro Procheck and the Synopsys checking extensions for this tool.
<u>check workspace project id</u>	Compares the Project ID specified against the Project ID used to create the workspace
<u>clear prime profile validation exprs</u>	Clears all validation expressions for Prime Profiles.
<u>clone component</u>	clone a component in the subsystem.
<u>close workspace</u>	Close the current workspace.
<u>CombineInheritValues</u>	Inherit attribute values from other

coreTools Command Reference Index

<u>combineValues</u>	items and combine values to form a single result Combine the values of the specified attribute on the specified items according to the specified command
<u>compare_dc_version</u>	Compare a version string with the version of Design Compiler currently in use
<u>complete_custom_activity_definition</u>	Complete a custom activity definition
<u>complete_interface_definition</u>	Complete an interface definition, and mark the definition as unchangable
<u>compress_sdc</u>	Inputs an SDC file and writes out a bus-compressed version of the input file
<u>connect_interface</u>	Connect interfaces
<u>connect_or_export_unconnected_pins</u>	Connect or export unconnected pins in current context.
<u>copy_configuration</u>	Copy configuration parameter values from source to target(s).
<u>copy_target_interface_parameters</u>	Copy parameter values from given interfaces to opposite target interfaces.
<u>create_address_bank</u>	Create an address bank in a memory map or bank.
<u>create_address_block</u>	Create an address block in a memory map or bank.
<u>create_address_space</u>	Create a new address space or update an existing one.
<u>create_autoload_filegroup</u>	Create an autoloaded filegroup
<u>create_component_view</u>	Create an IP-XACT component view.
<u>create_configuration_parameter</u>	Create a parameter for a configurable filegroup
<u>create_connection</u>	Create connections in between component pins and subsystem ports.
<u>create_custom_activity</u>	Create a custom activity
<u>create_custom_activity_parameter</u>	Create a parameter for a custom activity
<u>create_custom_tech_cell</u>	Specify binding requirements for mapping to a technology specific cell.
<u>create_generated_clock</u>	Create a generated clock for use within the specified design
<u>create_hierarchical_component</u>	Create a new hierarchial component in the subsystem

coreTools Command Reference Index

<u>create interface</u>	Create named interface definition
<u>create interface instance</u>	Create an instance of an interface to associate it on the core
<u>create interface parameter</u>	Create interface parameter on an interface definition
<u>create interface port</u>	Create interface port on an interface definition
<u>create memory map</u>	Create a memory map.
<u>create memory mode</u>	Create a memory mode for the current/selected design
<u>create plugin kb</u>	Create a plug-in KB from the files in the given directory
<u>create prime profile file</u>	Creates a Prime Profile file to be packaged with the coreKit or shipped externally.
<u>create reference clock</u>	Create a reference clock for use within the specified design
<u>create register</u>	Create register in an address block.
<u>create register array</u>	Create a register array in a memory block.
<u>create register field</u>	Create a register field in a register.
<u>create register field value</u>	Create a value for a register field.
<u>create subsystem parameter</u>	Create a parameter on the subsystem
<u>create virtual clock</u>	Create a virtual clock for use within the specified design
<u>create workspace</u>	Create a new workspace (working area).
<u>current activity</u>	Return current activity
<u>current design</u>	Set or get the current design
<u>current kb</u>	Set or get the current knowledge database (KB)

D

<u>date</u>	Returns a string containing the current date and time.
<u>debug script</u>	Debug a script using the TclDevKit Tcl debugger.
<u>define activity subproc params</u>	Define sub-process launch/result-viewing parameters for an activity or filegroup
<u>define array field parameters</u>	define parameters for each field of an array.

coreTools Command Reference Index

<u>define lib clocks</u>	Extract clock definitions from a hard macro library file
<u>define proc attributes</u>	Defines attributes of a Tcl procedure, including an information string for help, a command group, a set of argument descriptions for help, and so on. The command returns the empty string.
<u>define report</u>	Support creation and viewing of user defined reports
<u>define solution</u>	Define a DesignWare IP Solution
<u>define split interface</u>	Group the interfaces belonging to two different components.
<u>define tech cell mapping</u>	Define mapping requirements to a technology specific cell.
<u>detach interface</u>	Remove an attached interface instance from the given component.
<u>disable prime profile</u>	Disable Prime Profile related commands in builder, consultant and assembler (Synopsys only).
<u>docbook to html</u>	Convert the given DocBook XML file to HTML format.
<u>duplicate component</u>	Duplicate a component in the subsystem
<u>duplicate workspace</u>	Save the current workspace as duplicate.

E

<u>echo</u>	Echos arguments to standard output.
<u>enable prime profile</u>	Enable Prime Profile related commands in builder, consultant and assembler (Synopsys only).
<u>encrypt solution</u>	Encrypt TCL files associated with a given DW IP Solution directory.
<u>error info</u>	Prints extended information on errors from the last command.
<u>escaped name</u>	Converts escaped name to internally compatible mangled name
<u>eval in component</u>	evaluates the code in component context
<u>eval ipxact expr</u>	evaluate a SV expression from ipxact file.
<u>eval param</u>	Evaluate the given parameter expression.
<u>explicit capacitance</u>	Formula to specify an explicit capacitance value for PinLoadType
<u>export all interfaces</u>	Export all interfaces in the current context that are set up for auto-export.
<u>export interface</u>	Export a component interface out of the subsystem.

coreTools Command Reference Index

<u>export open pins</u>	Export pins currently connected to open
<u>export pin</u>	Export the pin to a port on the subsystem.
<u>export port descriptions</u>	Export port description attributes up one level of hierarchy.
<u>export unconnected interfaces</u>	Export unconnected interfaces in the current hierarchy.
<u>export unconnected pins</u>	Export unconnected pins in the current hierarchy.
<u>export workspace as component</u>	Export the workspace as a component.

F

<u>find design</u>	Locate the specified design or specified instance of the design and create a collection that contains that design
<u>find interface instances</u>	Locate the specified interface instances.
<u>find item</u>	Create a collection of items that match the specified name pattern
<u>find kb</u>	Find KB with the given name and type.
<u>foreach array field parameter</u>	loop through the parameters for the fields of an array.
<u>foreach in collection</u>	Iterate over a collection

G

<u>generate gtech sim model</u>	Write scripts and return shell command to generate a GTECH model of the core
<u>generate qtm</u>	Generate a QTM (quick timing model) for the top design.
<u>generate reports</u>	Generate the indicated reports.
<u>generate simulation file list</u>	Generate a list of simulation files
<u>generate simulation launch command</u>	Generate the command to launch the simulation
<u>generate views</u>	Generate the indicated optional views.
<u>getenv</u>	Returns the value of a system environment variable.
<u>get activity parameter</u>	Get the value of parameter for an activity
<u>get address bank attribute</u>	Get the value of an attribute from an address bank.
<u>get address block attribute</u>	

coreTools Command Reference Index

<u>get_all_bits_____</u>	Get the value on an attribute from an address block.
<u>get_associated_instance_parameter_____</u>	Return all bits of the specified port/pin, or port/pin itself if it is a single bit item.
<u>get_attribute_____</u>	Get parameter value of linked instantiation parameter.
<u>get_bitfield_value_____</u>	Get the current value(s) of the specified attribute(s)
<u>get_bit_driver_____</u>	Get value of bit range with a given parameter or numeric value.
<u>get_bit_loads_____</u>	Get the driver for this connection.
<u>get_cell_____</u>	Get the bit loads for this source connection.
<u>get_cell_attribute_____</u>	Command to get the cell from a cell name or path.
<u>get_children_____</u>	Get the value of an attribute on a design instance
<u>get_clocks_____</u>	Get the child/children of the given item
<u>get_clock_attribute_____</u>	Create a collection of all clocks associated with the specified design
<u>get_component_design_____</u>	Get the value of an attribute on a clock
<u>get_component_name_____</u>	Command to get the elaborated or unelaborated design for a component.
<u>get_component_view_____</u>	Get the component name of the given item.
<u>get_configuration_parameter_____</u>	Get the view associated with the current/selected component.
<u>get_configuration_parameter_attribute_____</u>	Get configuration (design) parameter value
<u>get_connections_____</u>	Get the value of an attribute from the specified configuration parameter
<u>get_current_component_____</u>	get connections for the specified single bit port/pin/busbit
<u>get_design_attribute_____</u>	Returns the current component.
<u>get_design_prefix_____</u>	Get the value of an attribute on the specified design
<u>get_field_parameter_for_array_____</u>	Returns the prefix to be associated with each design name.

coreTools Command Reference Index

<u>get_filegroup_attribute</u>	returning the field parameter for a given index of an array parameter.
<u>get_filegroup_parameter</u>	Get an attribute from a filegroup
<u>get_file_prefix</u>	Get a parameter for a filegroup
<u>get_generator_parameter</u>	Returns the file prefix to be associated with each design name.
<u>get_hdl_file_list</u>	Get component generator parameter value
<u>get_hdl_language</u>	Return the list of HDL files that DC must analyze (in order of analysis)
<u>get_hdl_library_names</u>	Get the HDL languages used
<u>get_hdl_pragma</u>	Return the list of HDL libraries used by the currently loaded core
<u>get_hierarchies</u>	Get reuse-pragma statement external to the HDL
<u>get_include_dirs</u>	Get information about the subsystem hierarchies.
<u>get_installation_dir</u>	return all include dirs for master leaf components in current or provided context
<u>get_installed_component_names</u>	Get full installation directory name for the current core
<u>get_instance_parameter_value</u>	Return installed component names and versions (for array set).
<u>get_interface_attribute</u>	Command to get the value of instance parameter in terms of the subsystem parameter.
<u>get_interface_link</u>	Get the value of an attribute from the specified interface
<u>get_interface_parameter</u>	Get evaluated InterfaceLink for the given interface port/param.
<u>get_interface_parameter_attribute</u>	Get a parameter for an interface
<u>get_interface_port_attribute</u>	Get the value of an attribute from the specified interface parameter
<u>get_launch_activities_status</u>	Get the value of an attribute from the specified interface port
<u>get_logical_dir</u>	Get status for all the activities or few activities launched with launch_activities_on_grid
<u>get_macro_prefix</u>	Get full logical directory name for the specified workspace directory
	Returns the macro prefix to be associated with each macro name.

coreTools Command Reference Index

<u>get_memory_map_attribute</u>	Get the value on an attribute from a memory map.
<u>get_memory_mode</u>	Get the current memory mode associated with the current/selected design
<u>get_object_name</u>	Converts a collection into a list of item names
<u>get_parameter_attribute</u>	Get the value of an attribute on a parameter
<u>get_port_attribute</u>	Get the value of an attribute on a port
<u>get_power_domain_voltage</u>	Returns the voltage for the specified power domain.
<u>get_prime_profile_info</u>	Get Prime Profile info from the data model..
<u>get_register_array_attribute</u>	Get the value of an attribute from a register array.
<u>get_register_attribute</u>	Get the value of an attribute from a register.
<u>get_register_field_attribute</u>	Get the value of an attribute from a register field.
<u>get_register_field_value_attribute</u>	Get the value of an attribute from a register field value.
<u>get_slave_base_address</u>	Get the remap range and/or base address of the given slave interface.
<u>get_solution_data</u>	Get all solution data or an individual field.
<u>get_solution_file</u>	Get IP solution file if present.
<u>get_solution_root</u>	Get root context of current DesignWare IP Solution.
<u>get_solution_variable</u>	Get all solution variables or an individual variable.
<u>get_strategy_parameter</u>	Get the value of a parameter for a strategy
<u>get_supply_voltage</u>	Command used to insert a voltage value into a set_voltage command in a power intent file.
<u>get_sv_expr</u>	Convert a tcl expression to equivalent sv expression
<u>get_synthesis_dir</u>	Return the full directory name for the specified synthesis subdirectory
<u>get_tcl_expr</u>	convert a SV expression to tcl.
<u>get_template_naming_style</u>	Get the template_naming_style value.

coreTools Command Reference Index

<u>get template parameter style</u>	Get the template_parameter_style value.
<u>get template separator style</u>	Get the template_separator_style value.
<u>get TMR separation distance</u>	Retruns the TMR Separation Distance
<u>get tool preference</u>	Get the tool preference parameter
<u>get tool root</u>	Get the root directory for the specified tool.
<u>get top design</u>	Returns the top design.
<u>get top design name</u>	Return the name of the top level design
<u>get upf attribute</u>	Get the value of an attribute on a UPF item
<u>get upf port names</u>	Returns the names of the ports in in the specified power domain.
<u>get value from interface</u>	Get value for interface parameter from the specified interface on the same component.
<u>get VLNV search path entries</u>	Get elements of given type and VLNV from the cache.
<u>get workspace kb</u>	Returns the current top-level knowledge database.
<u>get workspace license info</u>	Get details about license and license type associated with workspace creation.
<u>get workspace name</u>	Returns the name of the current top-level knowledge database.
<u>group components</u>	Group the following components into a lower hierarchical level
<u>gui get setting</u>	Get a setting on the specified window
<u>gui set setting</u>	Set a setting on the specified window
<u>gui source</u>	Source script in the GUI without GUI updates.
<u>gui start</u>	Start GUI
<u>gui stop</u>	Stop GUI

H

<u>help</u>	Displays quick help for one or more commands.
<u>history</u>	Displays or modifies the commands recorded in the history list.

L

<u>import component</u>	Import an unpackaged component into the subsystem.
<u>import ipxact data</u>	Import partial IP-XACT component data.
<u>import workspace as component</u>	Import the workspace as a component.
<u>IncludeIf</u>	Replace text if the parameter expression is true
<u>index collection</u>	Extract object from collection. Result is new collection
<u>infinite drive</u>	Formula to indicate infinite drive for an input port
<u>InheritValue</u>	Inherit an attribute value from an item up or down the design hierarchy or from a pin
<u>install filegroup</u>	Install the specified filegroup from the Bill of Materials
<u>instances of</u>	Get instances of given design name, recursively.
<u>instantiate component</u>	Instantiate a component in the subsystem.
<u>instantiate dut</u>	Instantiate a DUT into the testbench subsystem
<u>instantiate interface</u>	Instantiate an exported interface without any attachment.
<u>instantiate solution</u>	Initiate creation of a DesignWare IP Solution
<u>invoke generator</u>	invoke a component generator.
<u>invoke ralgen</u>	Invoke ralgen from \$VCS_HOME or within cT image.
<u>in autocomplete activity</u>	Is an activity currently being autocompleted?
<u>is prime profile enabled</u>	Returns 1 if Prime Profile is enabled in this session, 0 otherwise.

LKL _

<u>launch activities on grid</u>	Launch the sub-processes associated with multiple activities on a grid from a user workspace in coreConsultant
<u>launch activity subproc</u>	Launch a sub-process for an activity or filegroup
<u>list kb</u>	List all currently loaded knowledge databases
<u>load autoload filegroup</u>	Populate an autoloaded filegroup
<u>load component memory maps</u>	Loads all visible component memory maps from knowledgebase files in cC and cA into memory.
<u>load file into kb</u>	

coreTools Command Reference Index

	Load a file or directory into the current knowledge database
<u>load interface definitions</u>	Load interface definitions.
<u>load plugin</u>	Load a plugin KB and install non-configurable files.
<u>load solution definition</u>	Load a DesignWare IP Solution definition file.
<u>lock parameter</u>	Lock the configuration (design) parameter value so it cannot be changed.
<u>ls</u>	Lists the contents of a directory.

M

<u>merge ports</u>	Merge ports in current hierarchy, which are exporting adjacent bits of same pin.
<u>move components</u>	Move one or more components into a different hierarchical level.
<u>mpexpr</u>	Evaluate an expression with multiple precision math
<u>mpformat</u>	Evaluate an expression with multiple precision math
<u>msg_box</u>	Displays a modal message box

N Q

<u>open workspace</u>	Open the workspace.
-----------------------	---------------------

P

<u>packaged prime profile report</u>	Generate an HTML report for Prime Profiles
<u>parse proc arguments</u>	Parses the arguments passed into a Tcl procedure.
<u>percent of period</u>	Define delay as a function of clock's cycle time
<u>permanent proc</u>	Wrapper around proc to make a permanent hidden proc.
<u>pin cap of</u>	Return the capacitance of the specified pin.
<u>plugin proc</u>	Define a procedure in coreKit context or workspace context
<u>prefix defines</u>	Prefix parameters in RTL
<u>prefix identifiers</u>	Prefix the specified identifiers in the listed source files.
<u>prefix macros</u>	Prefix all of the macros found in source files.

coreTools Command Reference Index

<u>prereq activities complete</u>	Check if prerequisite activities have completed yet!
<u>printenv</u>	Prints the value of environment variables.
<u>printvar</u>	Prints the values of one or more variables.
<u>process solution files</u>	Explicitly process solution files for the indicated phase.
<u>propagate memory map</u>	Propagate the memory maps visible from an exported slave interface to that slave
<u>protected proc</u>	Wrapper around proc to make a protected (permanent but visible) proc.
 Q	
<u>quit</u>	Exits the shell.
 R	
<u>randomize parameters</u>	randomize the parameters with the help of VCS constraints solver
<u>rand proc</u>	Create SV function which is equivalent of tcl proc which will be used by randomize_parameters command. Generally created in coreBuilder via plugin files. Can be overridden in cC and cA
<u>read attribute table</u>	Read an attribute table file.
<u>read component constraints</u>	Read component SDC files to apply component constraints.
<u>read ipxact file</u>	Read and process an IP-XACT file of the specified type.
<u>read parameter info table</u>	Read a parameter info table file.
<u>read parameter table</u>	Read a parameter table file.
<u>read pin connection table</u>	Read a pin connection table file.
<u>read sdc</u>	Read a Synopsys Design Constraints format script.
<u>read subsystem table</u>	Read a subsystem table file.
<u>redirect</u>	Redirects the output of a command to a file.
<u>register netlister</u>	Register a user-defined netlister for use in Generate Subsystem RTL.
<u>reinitialize component</u>	Re-initialize a component through resetting all design parameters in coreAssembler or coreConsultant.

coreTools Command Reference Index

<u>reinitialize_workspace</u>	Re-initialize workspace through resetting all design parameters and activity parameters for each given activity
<u>remove_address_bank</u>	Remove an address bank.
<u>remove_address_block</u>	Remove an address block.
<u>remove_address_space</u>	Remove an address space.
<u>remove_allow_paths</u>	Remove -allow_paths from the named clock group.
<u>remove_area_estimates</u>	Remove area estimation values for the current design.
<u>remove_attached_interface</u>	Delete attached interfaces.
<u>remove_component</u>	Remove the specified component instance from the subsystem.
<u>remove_component_view</u>	Remove an IP-XACT component view.
<u>remove_connection</u>	Remove the specified connections.
<u>remove_constraints</u>	Remove constraints from the specified or current component and from all levels above.
<u>remove_exported_interface</u>	Remove an exported interface instance from the subsystem.
<u>remove_from_collection</u>	Remove object(s) from a collection. Result is new collection
<u>remove_generated_clock</u>	Remove the specified generated clock
<u>remove_html_report_link</u>	Remove an HTML report link from the index page
<u>remove_interface</u>	Remove an interface definition and all its instances
<u>remove_interface_instance</u>	Remove an instance of an interface associate with the core
<u>remove_ipxact_file</u>	remove a spirit file and related objects defined in the file.
<u>remove_memory_map</u>	Remove a memory map.
<u>remove_register</u>	Remove a register with the given name.
<u>remove_register_array</u>	Remove a registerArray.
<u>remove_register_field</u>	Remove a register field.
<u>remove_register_field_value</u>	Remove a register field_value.
<u>remove_virtual_clock</u>	Remove the specified virtual clock
<u>rename_component</u>	rename a component in the subsystem.
<u>rename_exported_interface</u>	Rename an exported or attached interface instance in the subsystem.

coreTools Command Reference Index

<u>rename_port</u>	Rename a hierarchical port
<u>ReplaceConstantParam</u>	Replace the definition of a constant
<u>ReplaceDesignParams</u>	Replace a set of design parameters with user-specified values
<u>ReplaceFormatParam</u>	Replace text with a formatted parameter value
<u>ReplaceSingleBitBus</u>	Convert single bit declarations into bit type
<u>replace_component</u>	replace a component in the subsystem. This command takes a connected interface (with SpecifySubsystem completed) and removes the auto connections of the interfaces and replaces them with manual connections all without incompleting SpecifySubsystem.
<u>replace_interface</u>	Replay manual connections.
<u>replay_connections</u>	Lists all activities and their current states
<u>report_activities</u>	List all parameters for each activity in the activity list.
<u>report_activity_parameters</u>	Report on activity/filegroup sub-process.
<u>report_activity_subproc</u>	Report on the specified (or all) attributes for the specified item(s)
<u>report_attribute</u>	Report Bill Of Materials
<u>report_bom</u>	Report on connected and unconnected component pins and subsystem ports.
<u>report_connections</u>	Report file groups
<u>report_filegroup</u>	generate textual or HDL description of the given interface definitions
<u>report_interfaces</u>	generate textual or HDL description of the given interface instances
<u>report_interface_instances</u>	Report on Synopsys IP used.
<u>report_ip</u>	List all currently checked out licenses
<u>report_license</u>	Report on memory maps in a component.
<u>report_memory_maps</u>	Generates a Prime Profile report for a hierarchy/component.
<u>report_prime_profiles</u>	Report validation expressions for Prime Profiles.
<u>report_prime_profile_validation_expr</u>	
<u>report_solutions</u>	

coreTools Command Reference Index

<u>report suppressed ral tests</u>	Generate simple report of available solutions.
<u>report tech cell mapping</u>	Generates a JSON report for all memory items that are untested and items which are tested with a non-empty string in the RALSuppressionComment attribute
<u>report timing exceptions</u>	Report technology specific cell mapping definitions and bindings.
<u>reset path</u>	Report timing exception objects
<u>restructure workspace</u>	Reset specified paths to single cycle timing
<u>reuse-pragma</u>	Restructure the current workspace.
	A type of stylized comment used to configure text files when used in coreConsultant and coreAssembler. For a more detailed description with more examples, see the coreBuilder manual.
 S	
<u>save workspace</u>	Save the current workspace.
<u>scale to current units</u>	Scale a time, area, or capacitance value to the unit currently in use
<u>select by class</u>	Select a representative library cell by its function class
<u>select by function</u>	Select a representative library cell by its logic function
<u>select by name</u>	Select a library cell (or pin) by name
<u>setenv</u>	Sets the value of a system environment variable.
<u>set activity incomplete</u>	Set the specified activities to the incomplete state
<u>set activity parameter</u>	Set the value of an activity parameter
<u>set address bank attribute</u>	Set the value of an attributes on an address bank.
<u>set address block attribute</u>	Set the value of an attribute on an address block.
<u>set area estimate</u>	Store area estimation value for a given technology and configuration.
<u>set attribute</u>	Set the value of an attribute on one or more items

coreTools Command Reference Index

<u>set_cell_attribute</u>	Set the value of an attribute on a list of design instances
<u>set_clock_attribute</u>	Set the value of an attribute on a list of clocks
<u>set_component_view</u>	Set the view associated with the current/selected component.
<u>set_configuration_parameter</u>	Set configuration (design) parameter value
<u>set_configuration_parameter_attribute</u>	Set an attribute on the specified configuration parameter
<u>set_current_component</u>	Set the current component to enable component level operations.
<u>set_default_prime_profiles</u>	Set Default Prime Profiles on the design in order of priority.
<u>set_dependent_controls</u>	Set dependent controls for a parameter dialog.
<u>set_design_attribute</u>	Set the value of an attribute on one or more designs
<u>set_design_prefix</u>	Sets the prefix to be prepended to each design name.
<u>set_disable_timing</u>	Disable timing arcs in a circuit
<u>set_editing_mode</u>	Set workspace editing mode
<u>set_false_path</u>	Removes timing constraints from particular paths
<u>set_filegroup_attribute</u>	Set an attribute on a list of filegroups
<u>set_filegroup_parameter</u>	Set a parameter for a filegroup
<u>set_file_prefix</u>	Sets the file prefix to be prepended to each design name.
<u>set_generator_parameter</u>	Set component generator parameter value
<u>set_hdl_file_list</u>	Defines an alternate HDL file list for a component.
<u>set_instance_parameter</u>	Command to set the value of instance parameter in terms of the subsystem parameter.
<u>set_instance_wire_names</u>	Store mapping of port names to wire names.
<u>set_interface_attribute</u>	Set an attribute on the specified interface
<u>set_interface_parameter</u>	Set a parameter on an interface
<u>set_interface_parameter_attribute</u>	Set an attribute on the specified interface parameter
<u>set_interface_port_attribute</u>	Set an attribute on the specified interface port

coreTools Command Reference Index

<u>set_macro_prefix</u>	Sets the macro prefix to be prepended to each macro name.
<u>set_max_delay</u>	Specifies a maximum delay target for paths in the current design
<u>set_memory_map_attribute</u>	Set the value of an attribute on a memory map.
<u>set_memory_mode</u>	Set the memory mode associated with the current/selected design
<u>set_min_delay</u>	Specifies a minimum delay target for paths in the current design
<u>set_multicycle_path</u>	Modifies the single-cycle timing relationship of a constrained path
<u>set_parameter_attribute</u>	Set the value an attribute on one or more parameters
<u>set_port_attribute</u>	Set the value of an attribute on one or more ports
<u>set_prime_profile</u>	Set a Prime Profile.
<u>set_prime_profile_validation_expr</u>	Set a validation expression for Prime Profiles.
<u>set_register_array_attribute</u>	Set the value of an attributes on a register array.
<u>set_register_attribute</u>	Set the value of an attribute on a register.
<u>set_register_field_attribute</u>	Set the value of an attribute on a register field.
<u>set_register_field_value_attribute</u>	Set the value of an attribute on a register field value.
<u>set_slave_base_address</u>	Set the remap range and/or base address of the given slave interface.
<u>set_solution_preference</u>	Set solution preference
<u>set_strategy_parameter</u>	Set the value of a parameter for a strategy
<u>set_synthesis_dir</u>	Set the name of the directory where synthesis will be performed.
<u>set_tclconfig_variables</u>	Set TCL variables for design parameters defined to have a TCL configuration variable.
<u>set_template_naming_style</u>	Set the template_naming_style value.
<u>set_template_parameter_style</u>	Set the template_parameter_style value.
<u>set_template_separator_style</u>	Set the template_separator_style value.
<u>set_tool_preference</u>	Set the tool preference parameter
<u>set_tool_root</u>	

coreTools Command Reference Index

<u>set_unused_interface</u>	Set the root directory for the specified tool.
<u>set_upf_attribute</u>	set the Unused attribute for unused provider
<u>set_upf_cells</u>	Set the value of an attribute on a list of UPF items
<u>set_upf_supply_voltage</u>	User routine to specify technology cells for UPF file configuration.
<u>set_upf_voltage</u>	Set the voltage on a supply created from set_voltage.
<u>set_workspace_options</u>	User routine to specify voltage values for UPF file configuration.
<u>sh</u>	Set one or more workspace options, customizing the user flow.
<u>show_color_dialog</u>	Executes a command in a child process.
<u>show_exit_dialog</u>	Displays a color browser dialog
<u>show_prime_profile_prompt</u>	Displays a color browser dialog
<u>show_spreadsheet_for_connections</u>	Show/Don't show Prime Profile selection prompt in consultant and assembler.
<u>show_url</u>	Generate and show a spreadsheet for making connections to interfaces with the type of the given interface.
<u>show_url_external</u>	Display URL internally
<u>show_user_parameter_dialog</u>	Display URL externally
<u>sizeof_collection</u>	Show the user parameter dialog to build and return the values of a set of user defined parameters
<u>source</u>	Get the number of objects in collection
<u>source_solution_script</u>	Read a file and evaluate it as a Tcl script.
<u>Synthesis_API</u>	Invoke specified solution script.
	This man page describes the different methodologies available for generating a gate-level implementation of a component configured in coreConsultant or a subsystem assembled in coreAssembler.

T

coreTools Command Reference Index

<u>translate interface connection</u>	This command takes a connected interface (with SpecifySubsystem completed) and removes the auto connections of the interfaces and replaces them with manual connections all without incompleted SpecifySubsystem.
<u>translate netlist</u>	Translate a specified design from HDL files to IP-XACT.
<u>translate phy constraints</u>	Translate Hierarchical Pin SDC constraints to be compatible with DW IP Solution flow
<u>traverse boundary</u>	Traverse a component boundary returning the above/below equivalent connection item.
<u>traverse connection</u>	Traverse a component connections for the connection object.

U

<u>unalias</u>	Removes one or more aliases.
<u>disconnect interface</u>	Disconnect interfaces
<u>ungroup component</u>	Ungroup the hierarchical component into current level.
<u>ungroup filegroup</u>	ungroup a filegroup
<u>union collection</u>	Combine two collections with set union semantics.
<u>unload autoload filegroup</u>	Removes loaded files from an autoloading filegroup
<u>unload component memory maps</u>	Unloads all non-visible component memory maps from knowledgebase files in cC and cA from memory or a specific one if -map option is used.
<u>unload file from kb</u>	Write the contents from a file item to the host system.
<u>unload plugin</u>	Unload a plugin KB.
<u>unused interface instance</u>	Returns 1 if the given interface instance is not being used.
<u>update dut</u>	Update the DUT to the latest version if possible.
<u>update workspace links</u>	Change symbolic links in a workspace

V

<u>verify dwf</u>	Verify the DesignWare Foundation version.
<u>verify environment</u>	

coreTools Command Reference Index

	Verify the user's environment using the EnvCheck attribute.
<u>verify tool</u>	Verify the proper environment variable (if it has one; otherwise, verify that the tool's executable is in PATH), version and licensing for a tool.
<u>visible fields</u>	Formula that can be used on a register to determine if there are visible fields.

W

<u>wait activities on grid</u>	Wait for all or a specified set of activities started with launch_activities_on_grid to complete.
<u>wait for activity subproc</u>	Wait for an activity sub-process started with launch_activity_subproc to complete.
<u>write attribute table</u>	Write an attribute table file.
<u>write automotive synthesis files</u>	Export the Automotive Synthesis files
<u>write batch script</u>	Generate a script to recreate this workspace
<u>write component register headers</u>	Write Component Register Headers files for component
<u>write config tar file</u>	Write out a coreConsultant configuration tar file.
<u>write failure modes file</u>	To export SGDC files
<u>write ipxact abstractor</u>	Export an IP-XACT description of the current abstractor.
<u>write ipxact component</u>	Export an IP-XACT description of the current component.
<u>write ipxact design</u>	Export an IP-XACT description of the current design.
<u>write ipxact designConfiguration</u>	Export an IP-XACT design configuration file.
<u>write ipxact typeDefinitions</u>	Export an IP-XACT description of the current typeDefinition.
<u>write ParameterInfo</u>	Write the configuration intent for a set of parameters out in the format of a template for use as a starting point when generating ParameterInfo.
<u>write pin connection table</u>	Write a pin connection table file.
<u>write ral file</u>	Write RAL (Register Abstraction Layer) file for component
<u>write sdc</u>	Writes out a script in Synopsys Design Constraints (SDC) format.
<u>write subsystem table</u>	Write a subsystem table file.

X

<u>xml_translate</u>	Export an IP-XACT description of the current component.
<u>xml_verify</u>	Verify IP-XACT syntax and semantics.

[API Index](#) [Command Reference](#)

Complete Connections

Connect and disconnect component pins and (new) subsystem ports manually until the subsystem design is complete.

Parameters

Activity has no parameters.

See Also

[Specify Subsystem.](#)

[API Index](#) [Command Reference](#)

ComponentOfItem

The component name of an item

Definition

Type: string

Flags: readOnly

Default value: Value calculated internally.

Valid on: Strategy attrDefn busBit cell clock design file filegroup

filegroupGroup hdlFunc knowledgeBase net param pin port

timingException

Description

This attribute holds the fully qualified component name that this item refers to.

Examples

```
coreAssembler> get_port_attribute clk ComponentOfItem
/apb1/i_gpio
```

See Also

[get_current_component \(2\)](#), [set_current_component \(2\)](#),

[instantiate_component \(2\)](#), [create_hierarchical_component \(2\)](#)

ConfigActivity

The activity name that to configure this filegroup.

Definition

Type: string

Flags:

Default value:

Valid on: filegroup_____

Description

This attribute should be specified for all Configurable (3) filegroups. The value of this attribute should be the name of the activity that will configure the filegroup.

If this filegroup has parameters, then an activity will automatically be created in the coreConsultant flow when the core integrator creates a workspace for this core.

Examples

To add documentation to the coreKit that will be customized for the user's configuration:

```
coreBuilder> create_autoload_filegroup Documentation
-patterns doc/ coreBuilder
-install doc
coreBuilder> set_filegroup_attribute Documentation Configurable
coreBuilder> set_filegroup_attribute Documentation ConfigActivity
SpecifyConfiguration
```

See Also

[create_autoload_filegroup \(2\)](#), [analyze_filegroup \(2\)](#), _____
[create_configuration_parameter \(2\)](#), [Configurable \(3\)](#), _____
[ConfigIntentSearchPath \(3\)](#) _____

[API Index](#) [Command Reference](#)

ConfigDependsOnActivities

List of activity names that should be completed (if they exist) before this filegroup is configured.

Definition

Type: string

Flags:

Default value:

Valid on: filegroup_____

Description

This attribute is useful for filegroups that create activities when the user creates a workspace based on this core. This attribute should contain the names of any activities that need to be completed before the activity for this filegroup is enabled.

Examples

When creating a regression environment, it is customary to not enable the regression test activity until the core has been configured, as follows:

```
coreBuilder> set_filegroup_attribute TRE ConfigActivity RunRegressions
coreBuilder> set_filegroup_attribute TRE ConfigDependsOnActivities
SpecifyConfiguration
```

See Also

[analyze_filegroup \(2\)](#), [ConfigDependsOnGroup \(3\)](#), [ConfigActivity \(3\)](#) _____

[API Index](#) [Command Reference](#)

ConfigDependsOnGroup

List of filegroups that must be configured before this group.

Definition

Type: itemList

Flags:

Default value:

Valid on: filegroup_____

Description

If filegroups should be configured before this one, then set this attribute to the filegroups that must be configured before this one.

Examples

See Also

[analyze filegroup \(2\)](#), [ConfigActivity \(3\)](#), [ConfigDependsOnActivities \(3\)](#)

[API Index](#) [Command Reference](#)

ConfigIntentSearchPath

A list of directories to search for intent files

Definition

Type: string

Flags:

Default value:

Valid on: filegroup_____

Description

This attribute is used with Configurable (3) filegroups. When the filegroup is analyzed with analyze_filegroup (2), this attribute is used to search for in intent files. See the analyze_filegroup (2) manpage for more information.

Examples

See Also

[analyze_filegroup \(2\)](#), [create_configuration_parameter \(2\)](#), _____

[ReplaceFormatParam \(2\)](#), [Configurable \(3\)](#), [ConfigActivity \(3\)](#)_____

[API Index](#) [Command Reference](#)

Configurable

Is this filegroup configurable?

Definition

Type: boolean

Flags:

Default value: 0

Valid on: `filegroup filegroupGroup`

Description

This attribute is used to indicate that the specified filegroup is configurable. Files in configurable filegroups are scanned for reuse-pragmas. Files containing pragmas are run through the text substitution process when the filegroup gets installed.

Examples

```
prompt> set_filegroup_attribute myGroup Configurable true
```

See Also

[create_autoload_filegroup \(2\)](#), [add_files_to_filegroup \(2\)](#),
[analyze_filegroup \(2\)](#), [create_configuration_parameter \(2\)](#),
[ReplaceFormatParam \(2\)](#), [ConfigActivity \(3\)](#)

[API Index](#) [Command Reference](#)

ConnectToExportedInstance

This instance need to connect to its exported instance

Definition

Type: boolean

Flags:

Default value: 0

Valid on:

Description

This attribute indicates that the instance need to connect to its exported instance. It is used to export/import Spirit compliant component or design.

Examples

See Also

ConnectionDialogCmd

Defines a custom command to be called for interactive connections.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Use this command to use a custom dialog when changing connections in the AddSubsystemComponents activity. The standard dialog works well for most situations, but is not compact enough in certain situations such as connecting interrupt lines.

In general, anytime you have many providers on different components (i.e. the interrupt source), and many consumers on a single component (i.e. an interrupt controller), you want to set this attribute to use the builtin command `show_spreadsheet_for_connections`.

Examples

See Also

[show_spreadsheet_for_connections\(2\)](#)

ConnectionRequired

Used to indicate that this interface does not require a connection.

Definition

Type: boolean

Flags:

Default value: 1

Valid on:

Description

By default all interfaces require a connection except for providers with `MinConsumers == 0`. Setting `ConnectionRequired` to false implies that the given interface does not require a connection. This means that the interface will not be connected automatically unless something that cannot to it requires a connection, and that a connection is not required to make a valid subsystem.

Examples

```
set_interface_attribute -instance AXI_Master02 ConnectionRequired false
```

See Also

[API Index](#) [Command Reference](#)

ConstantPort

Indicates that a port has a fixed logic value.

Definition

Type: string

Flags:

Default value: =InheritValue up {}

Valid on: busBit port _____

Description

This attribute is used to indicate that the specified port has a fixed constant value for synthesis. This is enabled implementation tools to propagate the constant value which typically enables a reduction in the area of a design.

Examples

See Also

[UnconnectedPort \(3\)](#)

Constraint Analysis

Generate and (optionally) run ptc_shell synthesis constraint analysis scripts.

Parameters

Parameters Table

Name	Label	Description
ControlFiles		
IncrementalConstraintsFile	Incremental constraints file	Incremental constraints file to be sourced before reading SDC.
WaiversFile	Waivers file	Waivers file to be source before generating reports.
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieved using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and

coreTools Command Reference Index

Name	Label	Description
		qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
EmailAddress	To:	If this parameter is set to a value other than "", then email is sent when the command script completes or is killed. The default value is the current user's name.
NetlistData		
UseExternalNetlist	Use externally-generated netlist	Check this box to specify a netlist from a source outside this workspace. Leave unchecked to choose a netlist in this workspace.
InputStageName	Input stage name:	Name of the stage that contains the design to be analyzed.
NetlistFile	External netlist file:	Path to the target netlist, required if an externally-generated netlist is to be analyzed.
ConstraintsFile	Constraints File	SDC Constraints File to be applied to netlist

See Also

Perform ASIC Synthesis.

Constraints Verification

set up the rtl config file and generate the specified assertions

Parameters

Parameters Table

Name	Label	Description
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieved using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
ParallelJobCpuLimit	Parallel job CPU limit:	Specifies the number of parallel compile jobs that may be run.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.
Setup/Verification Goals		
VerifyAll	Verify All	This goal performs all 3 verifications : FalsePath, GeneratedClocks &

coreTools Command Reference Index

Name	Label	Description
		MultiCyclePath.
VerifyFalsePath	Verify False Path	This goal performs False Path verification.
VerifyGeneratedClocks	Verify Generated Clocks	This goal performs Generated Clocks verification
VerifyMultiCyclePath	Verify MultiCycle Path	This goal performs MultiCycle Path verification.
Setup/Waiver File		
WaiverFile	Global Waiver File	Specifies the name of a global waiver tcl file. The activity parameter is disabled in coreConsultant as the waiver file is provided by the IP provider.

See Also

Specify Configuration.

Constraints Verification With Assertions
Execute the simulation generated list of log files

Parameters

Parameters Table

Name	Label	Description
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieving using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.
Setup/Extract SDC Status Flags		
PassStatic	Pass Static	-pass_static option, report inconclusive (vacuous) paths as Pass Static instead of Inconclusive.
RemoveFeedbackPaths	Remove Feedback Paths	-remove_feedback_paths option : Do not report paths where the startpoint and endpoint are the same design object
Setup/Input File		
SimulationLogs		

Name	Label	Description
	Simulation Log List File	File containing a list of the simulation log files. Supports both Absoulte and relative path.relative path must be calculated from the logical directory.

See Also

Constraints Verification.

[API Index](#) [Command Reference](#)

ControlPointsPerScanCell

Specifies the number of control points that can be shared per scan cell. When TestabilityMethod==control_and_observe for XG mode the larger of ObservePointsPerScanCell and ControlPointsPerScanCell will be used for the -test_points_per_scan_cell value for set_testability_configuration.

Definition

Type: long

Flags:

Default value: 8

Minimum value: 1

Valid on: design_____

Description

Specifies the number of control points that can be shared per scan cell for test point insertion.

Examples

Set the number of control points that can be shared per scan cell to 4.

```
coreBuilder> set_design_attribute ControlPointsPerScanCell 4
```

See Also

[set_design_attribute \(2\)](#), [InsertTestPoints \(3\)](#), [ScanBlockIndividually \(3\)](#), [BistBlockIndividually \(3\)](#), [ObservePointsPerScanCell \(3\)](#)

[API Index](#) [Command Reference](#)

ConvertSingleBitBus

Convert single bit bus to bit type

Definition

Type: boolean

Flags:

Default value: 0

Valid on: port _____

Description

When set to true on a design port, coreConsultant will convert busses that are only a single bit wide into a bit type. See the [ReplaceSingleBitBus manpage](#) for a more detailed description and examples.

Examples

Please see [ReplaceSingleBitBus for examples](#)

See Also

[GenerateIf \(3\)](#), [ReplaceSingleBitBus \(2\)](#)

CopyToTemplate

Should this component be copied into the template? A value of false means that the template must be used with an install area for this component. A value of true means that the template does not require an install area for this component

Definition

Type: boolean

Flags:

Default value: =get_attribute %item -attr IsImportedComponent

Valid on: cell _____

Description

This attribute specify should a component be copied into template when building a coreAssembler template. A value of false means that the template must be used with an install area for this component. A value of true means that the template does not require an install area for this component and the install area will be copied into template.

Examples

if you want to copy the i_AHB component install area into the template, use the following command:

```
set_cell_attribute i_AHB CopyToTemplate true
```

See Also

[API Index](#) [Command Reference](#)

Create Bill of Materials

Specify the set of files that are to be incorporated into the bill of materials. This is the definition of the set of all files that are part of the core design kit.

Parameters

Activity has no parameters.

See Also

[Load Designs.](#)

[Verify Intent.](#)

Create Integration Workspaces

This activity packages up a core kit, installs the core kit, and then creates one or more integration (coreConsultant) work spaces from that installation. These work spaces can be used to do validation of the core kit.

Parameters

Parameters Table

Name	Label	Description
Filegroups		
ExcludeFilegroups	Filegroup names:	Indicates which file groups should be excluded from the coreKit. File groups should be specified as a simple space separated list. Pre-defined file groups such as Hdl can notbe excluded.
Root directory		
CleanupRootDir	Clean up existing root directory	Indicates that the specified root directory should be cleaned up if it exists. All installation and workspace directories within the root directory will be removed.
RootDir	Root directory:	Specify the directory which will be used as the root for holding the installed core kit and all integration (coreConsultant) work spaces.
Workspaces		
StartGUIOnWorkspace	Start GUI on workspace	Indicates that the given workspace should be loaded into the coreConsultant GUI. This option is ignored if more than one workspace is being created.
WorkspaceNames	Workspace names:	List the names of the workspace(s) to be created. At least one entry is required. This should be specified as a simple space separated list.

See Also

[Design Configuration Intent.](#)
[Specify Clocks.](#)
[Specify Synthesis Methodology.](#)
[Specify Port Constraints.](#)
[Specify Power Intent.](#)

[API Index](#) [Command Reference](#)

CreateSkewGroup

Specifies the clock for which the skew group needs to be created. The skew group created will follow the naming convention

SKEW_GROUP_QiK_CK_GEN_<CLOCK NAME>. Refer to man page for 'create_clock_skew_group' in Fusion Compiler reference documentation.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: clock_____

Description

Examples

See Also

Create Subsystem Template

Create a subsystem template. The template is created as a single file and is located in the ./export directory of the current workspace.

Parameters

Parameters Table

Name	Label	Description
TemplateName	Template name:	Specify name of the template to creat
TopDesignName	Top design name:	Top design name of the subsystem assembled
PluginDirectories	Template plugin directories	The list of directories to build plugin Kbs for. The default name of the plugin is the same as the name of the directory. If a different name is desired, then touch a file called [name].cfg to name the plugin [name].kb

See Also

Create Testbench Workspace

Create testbench workspace for the current DUT.

Parameters

Parameters Table

Name	Label	Description
TestbenchWorkspaceName	Testbench Workspace Name	Name to use for the testbench workspace to be created.
TestbenchWorkspaceDir	Testbench Workspace Directory	Root directory for the new testbench workspace.
TestbenchType	Testbench Strategy and Language	A System Verilog testbench using the test methodology described in the 'Verification Methodology Manual' (VMM) or UVM with RVM-VMT DesignWare VIP. Note: VMM/UVM can only be used with Synopsys VCS simulator - EnumValues: uvm_subsystem uvm_ral_ping - Symbolic Names: {UVM RAL Subsystem(default)/ UVM Subsystem} {UVM RAL Subsystem Ping Test}
TestbenchInstantiateVIP	Automatically Instantiate DesignWare VIP	If true, automatically instantiate and connect DW VIP where appropriate connections are available on the DUT
TestbenchCloseDUT	Close DUT Workspace	If true, create testbench workspace in this coreAssembler process. Otherwise launch a second coreAssembler to create the testbench.

See Also

[Generate Subsystem RTL](#)

[API Index](#) [Command Reference](#)

CreatedViaUngrouping

Set to true for designs that are created using
create_hierarchical_component -fromComponent.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design knowledgeBase

Description

Examples

See Also

[API Index](#) [Command Reference](#)

CriticalRange

Default critical range for this design.

Definition

Type: float

Flags:

Default value: =InheritValue up {}

Valid on: design_____

Description

The CriticalRange attribute specifies the Design Compiler critical_range value to use for incremental compiles of the selected design.

For the initial compile, coreConsultant generates a strategy that compiles the design based on the design constraints. For incremental compiles, coreConsultant generates a set_critical_range command for the design to set the design's critical_range attribute equal to CriticalRange. If CriticalRange = 0 (or is not set on the design), Design Compiler only reoptimizes the most critical path(s) in the design (the path(s) with the most negative slack). If CriticalRange is a non-zero number, Design Compiler reoptimizes all paths that are within CriticalRange of the most critical (worst) path.

For example, if the worst path after the initial compile has a negative slack of 6.2 and CriticalRange is set to 1.1 on that design, then for the incr1 compile phase, Design Compiler will reoptimize any path that has a negative slack between 7.3 and 5.1.

Examples

To reoptimize all near-critical paths in the current_design and define a path as near-critical if its delay is within 2 target technology time units of the worst path:

```
coreConsultant> set_design_attribute CriticalRange 2
```

See Also

[set_design_attribute \(2\)](#), [CriticalRangeCoveringViolators \(3\)](#)

[API Index](#) [Command Reference](#)**CriticalRangeCoveringViolators**

Set the critical_range value to cover all violating paths in the design.

Definition

Type: boolean**Flags:****Default value:** 0**Valid on:** design_____

Description

The CriticalRangeCoveringViolators attribute determines whether to reoptimize all violating paths during incremental compiles or just those paths that are within CriticalRange of the worst path.

If CriticalRangeCoveringViolators is true on a design, coreConsultant generates a synthesis strategy that reoptimizes all violating paths during incremental compile phases. That is, instead of using CriticalRange as the value for the Design Compiler set_critical_range command, coreConsultant uses the negative slack of the most critical path in the design as the value for set_critical_range.

If CriticalRangeCoveringViolators is false, coreConsultant uses the CriticalRange attribute to determine which paths to reoptimize during incremental compiles.

Setting CriticalRangeCoveringViolators to true can cause a large increase in compile time because Design Compiler attempts to reoptimize all paths that are causing timing constraint violations.

Examples

To reoptimize all violating paths during incremental compiles of the current_design:

```
coreConsultant> set_design_attribute CriticalRangeCoveringViolators true
```

See Also

[set_design_attribute \(2\)](#), [CriticalRange \(3\)](#)

[API Index](#) [Command Reference](#)

CriticalTiming

Indicates whether the port will likely be part of a critical timing path.

Definition

Type: boolean

Flags:

Default value: =InheritValue up 0

Valid on: busBit port _____

Description

The CriticalTiming attribute indicates whether the port is likely to be part of a critical timing path. If CriticalTiming is true, coreConsultant generates more aggressive optimization strategies for the port.

If CriticalTiming is not explicitly set on a port, the port inherits the CriticalTiming value from the connecting port on the next higher level design. If there is no CriticalTiming value to inherit, CriticalTiming defaults to 0 (false).

Examples

To indicate that the data_in port is part of a critical timing path:

```
coreConsultant> set_port_attribute data_in CriticalTiming true
```

See Also

[set_port_attribute \(2\)](#)

CustomizedTestbenchCode

Customized testbench code to be inserted

Definition

Type: string**Flags:** subscripted**Default value:****Valid subscripts:** Verilog_Additional_Footer

Verilog_Additional_Header Verilog_Additional_Instance

Verilog_Additional_Package Verilog_Additional_Post_Instance

Verilog_Additional_Pre_Instance Verilog_Additional_Pre_Wire

Verilog_Additional_Wire Vhdl_Additional_Footer

Vhdl_Additional_Header Vhdl_Additional_Instance

Vhdl_Additional_Package Vhdl_Additional_Post_Instance

Vhdl_Additional_Pre_Instance Vhdl_Additional_Pre_Signal

Vhdl_Additional_Signal Vhdl_Additional_UseClauseForArchitecture

Vhdl_Additional_UseClauseForPackage

Default subscript:**Valid on:** cell design _____

Description

This attribute can be used by core developers and/or core integrators to enable customization of the generated testbench. This attribute is used to define text that can be inserted into the generated HDL at different points within the file. The different attribute subscripts indicate where in the testbench the custom code will be inserted.

The testbench is generated as part of the SpecifyConfiguration activity in coreConsultant and as part of the GenerateSubsystemRTL activity in coreAssembler. If this attribute is set by a core integrator in one of these two tools, the corresponding activity will need to be recompleted in order for the attribute setting to have an impact on the generated testbench.

Examples

To insert simple text containing additional verilog instances:

```
set_design_attribute \
{CustomizedTestbenchCode[Verilog_Additional_Instance]} \
"//your customized code: additional instance to instantiate"
```

To insert extra signal declarations from a file:

```
set_design_attribute \
{CustomizedTestbenchCode[Vhdl_Additional_Signal]} \
```


"=exec cat file_containing_code_to_insert"

See Also

set_design_attribute (2), get_design_attribute (2)

[API Index](#) [Command Reference](#)

CycleTime

Cycle time for a clock.

Definition

Type: float**Flags:****Default value:****Valid on:** clock_____

Description

The CycleTime attribute indicates the length of the clock period for a clock. If you do not specify values for RiseEdge and/or FallEdge for the clock, the coreTools assume the clock has a 50 percent duty cycle, with the rising clock edge at time 0 and the falling clock edge at CycleTime/2.

You can specify the CycleTime value either with or without units. If you do not specify a time unit, the default time unit is the time unit used by the currently loaded technology library. If you do specify a time unit (for example, 25ns), the coreTools automatically scale the time value to the time unit used by the currently loaded technology library. For example, if you specify 25ns and the currently loaded technology library uses ps as the time unit, the coreTools automatically scale 25ns to 25,000ps.

Examples

To set the clk cycle time to 10ns:

```
coreConsultant> set_clock_attribute clk CycleTime 10ns
```

See Also

[set_clock_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

DCLSPaths

Specify the paths as a list-of-list.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

Examples

See Also

DCTCL_quick_map_strategy

Generates scripts that do a 'quick map' of this core to the target technology. This is the fastest possible mapping to gates, and should be used as an estimation path only.

You can also select the Stop at GTECH parameter to elaborate your configured core to the Synopsys technology-independent GTECH library, without actually compiling the core to your target technology. Generating a GTECH version of the core is an easy way to create a simulation model for a core that is delivered as encrypted HDL.

Parameters

Parameters Table

Name	Label	Description
Basic		
UseDcExplorer	Use DC-Explorer	Use DC-Explorer to run synthesis. This option is only available if using DC-Explorer G-2012.06-SP2 or newer.
StopAtGtech	Stop at GTECH	Don't actually map this design to the target technology. Setting this to true, causes the design to be analyzed and elaborated only. Mapping to GTECH is not supported by DC-Explorer.
ReadConstraints	Read Constraints	When enabled the SDC constraints will be read in during the GTECH Synthesis run.
OnlySimpleGtechGates	Use only simple GTECH gates	Use a minimal set of GTECH gates to implement the netlist. The parameter only takes effect if StopAtGtech is selected. The cells used are: GTECH_NOR2, GTECH_NOT, GTECH_FD2, GTECH_F
ExcludeGtechCellSet	Exclude cell set:	Set of GTECH cells that should not be used when mapping the design.
IncludeGtechCellSet	Include cell set:	Set of GTECH cells to be used in addition to the cells used when OnlySimpleGtechGates is specified.
UngroupAllAfterCompile	Ungroup all after compile	Indicates that the design should be totally ungrouped after the final compile stage. Checking this option will result in a completely flat netlist regardless of any PreserveHierarchy settings.
Basic/Compile Ultra		
CompileUltraInitOptions	Additional Initial Options	Extra options to pass to an initial compile_ultra command.

coreTools Command Reference Index

Name	Label	Description
Basic/ICC2 Write Files		
WriteICC2Files	Write ICC2 files after synthesis	Indicates whether an ICC2 file should be generated when synthesis completes. If enabled, the ICC2 file will be written to [workspace]/syn/[stage]/db/ICC2_files/*.
Basic/Netlist Generation/Netlist Generation		
NetlistFormat	Netlist format:	Causes synthesis checkpoints to be written in the specified netlist format. The netlist will be located in the 'db' directory for the checkpoint. - EnumValues: .v .vhd .edif .NET .neted .pla .st .tdl .xnf .fnc - Symbolic Names: verilog vhdl ed lsi mentor pla st tdl xnf equation
ChangeNamesRuleName	change_names Name Rules:	Indicates the name of a change_names naming rule. Prior to writing the netlist in the specified format, change_names will be called with the specified rules.
RemoveUnconnectedPorts	Remove unconnected ports	Removes unconnected ports from netlist
Basic/Netlist Generation/set_fix_multiple_port_nets		
FixMultiplePortNetsDefault	Fix Multiple Port Nets Default	Will cause set_fix_multiple_port_nets -default to be used in Design Compiler. No buffers will be inserted to fix multiple port nets.
FixMultiplePortNetsFeedthroughs	Fix Multiple Port Nets Feedthroughs	Will cause set_fix_multiple_port_nets -feedthroughs to be used in Design Compiler.
FixMultiplePortNetsOutputs	Fix Multiple Port Nets Outputs.	Will cause set_fix_multiple_port_nets -outputs to be used in Design Compiler.
FixMultiplePortNetsConstants	Fix Multiple Port Nets Constants	Will cause set_fix_multiple_port_nets -constants to be used in Design Compiler.
FixMultiplePortNetsBufferConstants	Fix Multiple Port Nets BufferConstants	Will cause set_fix_multiple_port_nets -buffer_constants to be used in Design Compiler.
Basic/SDC Generation		
WriteSDCArgs	Options for write_sdc	Defines arguments to be passed to the write_sdc command when writing SDC constraints for synthesis.
Basic/SDF Generation		
WriteSDF	Write SDF after synthesis	Indicates whether an SDF file should be generated when synthesis completes. If

coreTools Command Reference Index

Name	Label	Description
		enabled, the SDF file will be written to [workspace]/syn/[stage]/db/[design].sdf.
Basic/Static Aware Flow		
EnableStaticAwareFlow	Enable Static Aware Flow	The VC SpyGlass Static Aware DC/FC Flow generates an extra database that must be read in synthesis tool to guarantee that netlist compilation/optimization does not introduce new CDC issues. The activation of the feature is limited to coreConsulant for DesignWare IP's. To enable the feature, VC Spyglass Engineering Council must be at version 2023.09 or higher, and the runVcSpyglass activity must be complete.
StaticAwareDBPath	Static Aware synthesis database	Path to Static Aware synthesis DB.
Basic/Technology Node		
TechnologyNode	Technology Node	Specifies the technology node to be used in set_technology command in Design Compiler NXT. This helps in enabling setting in Design Compiler NXT for technology node specific synthesis. coreTools will not validate the Technology Node value and validation will be performed by Design Compiler.
Clock Gating		
ClockGating	Insert Clock Gating	Add clock gating logic using, insert_clock_gating, as defined by the other clock gating parameters.
SelfGating	Insert Self Gating	Self-gating is an clock gating technique that optimizes dynamic power by gating the clock signal in those cycles in which the data saved in a register remains unchanged. An enable condition is computed by comparing the stored data with the new data arriving at the data pin, and that signal is used to drive the inserted self-gating cell. Enabling this adds -self_gating option to compile_ultra command. This option is available only when Physical Synthesis & Clock Gating enabled. This parameter updates the control variable INSERT_SELF_GATING.ct_variable_setup.tcl, present in rm_setup folder.
GatingMinimumBitWidth		

coreTools Command Reference Index

Name	Label	Description
	Gating minimum bit width:	Minimum register bank width that will have its clock gated
GatingLatchCellName	Gating latch cell name:	Select between latch style gating or combinational style gating. For latch style the name of a library cell to use for latching the gated clock signal may be entered. If it is left blank then Design Compiler will choose a latch cell from the target library.
GatingAndFunction	Gating And function:	Circuitry used to implement And/Nand functionality for clock gating. If latch style clock gating is selected circuitry will be inserted into the clock network to gate positive edge triggered registers. If combinational style gating is chosen this circuitry will be used to gate negative edge triggered devices. If the appropriate ICG cells are available in the library, this value will default to integrated. EnumValues: integrated and {buf and} {and buf} {buf and buf} {nand inv} {buf nand inv} {inv or inv} {inv nor} {inv nor buf} {and inv} {buf and inv} nand {buf nand} {nand buf} {buf nand buf} {inv or} {inv or buf} {inv nor inv}
IntegratedAndCell	Integrated And cell:	Integrated library cell to be used for AND type clock gating. Please consult the Design Compiler man page for set_clock_gating_style for more information about using integrated clock gating cells.
GatingOrFunction	Gating Or function:	Circuitry used to implement Or/Nor functionality for clock gating. If latch style clock gating is selected, then this circuitry will be inserted into the clock network to gate negative edge triggered registers. If combinational style gating is chosen then this circuitry will be used to gate positive edge triggered devices. If the appropriate ICG cells are available in the library, this value will default to integrated. EnumValues: integrated or {buf or} {or buf} {buf or buf} {nor inv}

coreTools Command Reference Index

Name	Label	Description
		{buf nor inv} {inv and inv} {inv nand} {inv nand buf} {or inv} {b or inv} nor {buf nor} {nor buf} {buf nor buf} {inv and} {inv and buf} {inv nand inv}
IntegratedOrCell	Integrated Or cell:	Integrated library cell to be used for OR type clock gating. Please consult the Design Compiler man page for set_clock_gating_style for more information about using integrated clock gating cells.
GatingSetupTime	Gating setup time:	Setup time to clock enable pin
GatingHoldTime	Gating hold time:	Hold time at clock enable pin
GatingControlPoint	Gating control point:	Insert an OR gate to control the gating signal for testability EnumValues: none before after
GatingControlSignal	Gating control signal:	Signal used at control point during test. Note that if GatingControlPoint is none the value of GatingControlSignal will be ignored EnumValues: scan_enable test_m
GatingObservationLogicDepth	Gating observation logic depth:	Max depth of XOR observability tree
GatingCellMaxFanout	Gating cell max fanout value:	Specifies the maximum fanout of a single clock-gating cell. If no value is specified default Design Compiler will use its default.
Design for Test		
TestReadyCompile	Test Ready Compile	Enable test-ready compile. Primary effect of this parameter is to cause the -scan option to be added to the initial compile call.
PhysicalSynthesis		
PhysicalSynthesis	Physical Synthesis	Use physical synthesis in Design Compiler. Setting this parameter to true causes the physical synthesis placement engine to be used during optimization. Net loads and delays will be estimated by placement, and wireload models will not be used. This flow may be used with either a physical library in ddc format or Milkyway reference library in Milkyway

coreTools Command Reference Index

Name	Label	Description
		format. Note that no placement data is saved with the ddc or Milkyway database.
EnableSPG	Enable SPG	Enable Synopsys Physical Guidance (SPG) flow. This flow performs refined placement and enables tighter integration with IC Compiler, facilitating timing closure and resulting in closer correlation with ICC. This feature requires DC 2013.03 or later.
FloorplanFileDataFormat	Floorplan Data Format	Format of file containing floorplan constraints. - EnumValues: floorplan def tcl - Symbolic Names: Floorplan DEF
PhysicalConstraintFile	Physical Constraint File	Floorplan constraint file to be applied during synthesis.
EnablePortSideConstraint	Enable Port Side Constraint	Generates a port side file which is inserted into the synthesis flow for better pin placement. This file can only be generated when there is no floorplan file given and SideConstraint attribute is set on the design port by the user. This can be used only in topographical mode. This feature is enabled only when the DC version is >= 2018.06.
UtilizationRatio	Utilization Ratio	Specify a utilisation ratio as part of synthesis intent. Its default value is 0.7 and ranges from 0.0 to 1.0 . This utilization ratio is applicable only when floorplan data is not provided.
MinRoutingLayer	Min Routing Layer Name	Specifies the minimum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers below the specified layer as fully blocked for RC and congestion estimation.
MaxRoutingLayer	Max Routing Layer Name	Specifies the maximum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers above the specified layer as fully blocked for RC and congestion estimation.
PhysicalLibrary	Physical Library	The physical library to be used for physical synthesis.

coreTools Command Reference Index

Name	Label	Description
TechFileFormat	Tech File Format	Select 'tf' to use the tech file to create the Milkyway design library, or select 'plib' to use the physical library source (.plib) file EnumValues: tf plib
TechFile	Tech File	Technology file used to create the Milkyway design library. The file must match the format specified in TechFormat
MaxTluPlusFile	Max Tlu Plus File	The Max TLU+ file.
MinTluPlusFile	Min Tlu Plus File	The Min TLU+ file.
Tech2ItfMap	Tech file to ITF Mapping File	TECH2ITF mapping file name.
MaxEmulationTluPlus	Max Emulation TLU+ file name	To be used for the -max_emulation_tluplu switch to set_tlu_plus_files. See Design Compiler documentation for information about the set_tlu_plus_files command.
MinEmulationTluPlus	Min Emulation TLU+ file name	To be used for the -min_emulation_tluplu switch to set_tlu_plus_files. See Design Compiler documentation for information about the set_tlu_plus_files command.
CreateCongestionMap	Create Congestion Map	Create graphical congestion map when synthesis completes. To avail of this feature, you must synthesize with DC-Topographical D-2010.03 or newer. This feature also requires DC-Graphical with a DC-Extension license. Your DISPLAY variable must also be set in order to use this feature.
PhysicalSynthesis/Milkyway		
UseMilkyway	Use Milkyway	Use the Milkyway database format, It is supported for DC version older than S-2021.06-SP4. Note that the Milkyway design library must have been created prior to specifying the Milkyway parameters. Please see Milkyway documentation for more information on the Milkyway format.
MilkywayDesignLibrary	Milkyway Design Library	This parameter corresponds to the Design Compiler variable mw_design_library. Specify the path to the Milkyway design library. If a relative path is given, the directory will be created in the workspace 'syn' directory.
CreateMilkywayDesign		

coreTools Command Reference Index

Name	Label	Description
	Create Milkyway Design	If TRUE, coreTools will create the Milkyway design library.
MilkywayReferenceLibrary	Milkyway Reference Library	This parameter corresponds to Design Compiler variable mw_reference_library. Specify the full path to the Milkyway reference library. The Milkyway reference library must exist before specifying this parameter. coreTools will not create it.
ExtendMWLayers	Extend MW Layers	Insert extend_mw_layers command into DRC and topo scripts.
CreateMwDesignOptions	Create Mw Design Options	Additional options to the create_mw_lib command. Please see Design Compiler or Milkyway documentation for details.
MilkywayLogic0Net	Milkyway Logic 0 Net	This parameter corresponds to the Design Compiler variable mw_logic0_net. Contains the milkyway logic0(1'b0) net for the design.
MilkywayLogic1Net	Milkyway Logic 1 Net	This parameter corresponds to the Design Compiler variable mw_logic1_net. Contains the milkyway logic1(1'b1) net for the design.
WriteMilkywayCell	Save design in Milkyway Cell format	Set this parameter to 'true' if you require DC-Topographical to write out a Milkyway cell representation of the design after synthesis. Recommended if your implementation flow uses IC Compiler. Note, however, that creation of the Cell will succeed only if all cells used in the design have appropriate Milkyway views, especially FRAM, available.
PhysicalSynthesis/Multi Vth Constraint		
AvailableVthGroups	Available Threshold Voltage Groups	The Voltage Groups Available to the user.
LowThresholdVoltageGroups	Low Threshold Voltage Groups	Specifies the threshold-voltage groups considered to be low-threshold-voltage groups. The groups argument is a list of threshold-voltage group identifiers, which are names defined by attributes in the Liberty (.lib) descriptions of the cells and cell libraries.
LowThresholdVoltagePercentage	Low Threshold Voltage Percentage	Specifies the maximum allowable percentage of low-threshold-voltage cells for the current design. The percent_value argument is a floating-point number between 0 and 100

coreTools Command Reference Index

Name	Label	Description
LowThresholdVoltageConstraintType	Low Threshold Voltage Constraint Type	Specifies the constraint type, which can be either hard or soft. The default is soft; the tool meets this constraint only if it does not adversely affect the timing constraint. EnumValues: soft hard
PhysicalSynthesis/NDM Library		
UseNDM	Use NDM	This switch enables NDM view to extract the physical information. NDM view is only supported in DCRM version S-2021.06-SP4 or greater. In older versions of DCRM templates, milkyway v is supported.
NDMLibraryName	NDM Library Name	Specify the name of the new NDM library. The path to the NDM library can be a relative, or an absolute path. If a relative path is given, the directory will be created in the workspace 'syn' directory. The name cannot contain space or the colon (:) character, which is used to delimit the library and design name
NDMReferenceLibrary	NDM Reference Library	Specifies the reference libraries for the NDM library 'NDMlibraryName'. You can specify the reference libraries and physical source data using absolute or relative paths. If you specify the libraries with a relative path or with no path, the tool uses the TechSetup's activity parameter SearchPath to locate the libraries. The NDM reference library must exist before specifying this parameter. coreTools will not create it.
CreateLibDesignOptions	Create Lib Design Options	Additional options to the create_lib command which creates NDM library. Please see Design Compiler NXT or NDM library documentation for details.
Reports		
GenerateRMReports	Generate supplementary Reports	When enabled generates additional reports as mentioned in the Reference Methodology.
DisableCongestionReport	Skip Congestion Report	Disables congestion report generation. May be used for runtime improvements.

See Also

[API Index](#) [Command Reference](#)

DedicatedScanPorts

Determines whether dedicated scan out ports will be created, or whether the functional ports will be used as scan ports.

Definition

Type: boolean

Flags:

Default value: =InheritValue up FALSE

Valid on: design_____

Description

Determines whether dedicated scan out ports will be created, or whether the functional ports will be used as scan ports.

Examples

Specify that functional ports are not to be shared for the current design.

```
coreBuilder> set_design_attribute DedicatedScanPorts TRUE
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

DedicatedWrapperCell

Specifies the default dedicated wrapper cell type.

Definition

Type: string

Flags:

Default value: WC_D1

Valid on: design_____

Description

Specifies the default dedicated wrapper cell to be used for the design.

Examples

Specify that the default dedicated wrapper cell should be WC_D1.

```
coreBuilder> set_design_attribute DedicatedWrapperCell WC_D1
```

See Also

[set_design_attribute \(2\)](#), [WrapBlockIndividually \(3\)](#),
[WrapSubblocksIndividually \(3\)](#)

DefaultConstantPort

Specifies a default value to be connected to a design input port when the interface connection only links to a design port on one side of connected interfaces.

Definition

Type: string

Flags:

Default value: none

Valid on:

Description

Legal values for this attribute are: none, zero, one, and dc.

This attribute is used to determine how to tie-off an input pin when two connected interface ports only have links to design ports on one side. More specifically, this occurs when the InterfaceLink value of the provider-side interface port is "<open>" and that on the consumer-side interface is linked to a design port (or vice-versa). If the available design port is an input, then the input will be connected to the indicated tie-off value. The value 'none' implies no connection should be made.

Examples

For interface port 'thePort' specify that when there is no driver, connect the input pin to logic zero.

```
set_interface_port_attribute thePort DefaultConstantPort zero
```

See Also

[set_interface_port_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

DefaultHdlValue

Store the Default HdlValue based on the unconfigured core

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

DefaultInstallDir

Default directory into which to install a file or file group

Definition

Type: string**Flags:****Default value:****Valid on:** file [filegroup](#) [filegroupGroup](#)

Description

The DefaultInstallDir attribute specifies the directory into which to install the specified file or file group.

When coreConsultant installs the coreKit, the installer selects the installation root directory. Then, coreConsultant unpacks and installs the files and file groups into the subdirectories specified by the DefaultInstallDir attribute on the files and file groups.

The add_files_to_file_group automatically sets DefaultInstallDir on files to the value that you specify for the -install <install directory> option. You can also set DefaultInstallDir on a file or file group directly by using the set_attribute command.

To specify how to copy the file from the coreKit installation directory to a user workspace, set the InstallUserWorkDir attribute.

Examples

To add file to file group and then check value of DefaultInstallDir:

```
coreBuilder> add_files_to_filegroup Example \  
-files {file1.v file2.v}\  
-install example  
coreBuilder> get_attribute -attrs DefaultInstallDir file1.v  
example/
```

See Also

[add_files_to_filegroup \(2\)](#), [InstallUserWorkDir \(3\)](#)

DefaultLoadPath

A list glob-style patterns specifying how to load this deliverable.
Relative paths start with the root of the current workspace.

Definition

Type: string**Flags:****Default value:****Valid on:** filegroup_____

Description

This attribute controls how an autoloader filegroup actually loads its files.

When an autoloader filegroup is loaded (by `load_autoload_filegroup(2)`, or by the BuildcoreKit activity).

This attribute should be set to a list of patterns that specify how to create this autoloader filegroup. Every entry in the list is expanded with the glob command. Relative paths start with the root of the current workspace. If the workspace is located at `"/user/userx/corea/Builder_ws"`, then the pattern `tb/*.v` refers to `/user/userx/corea/tb/*.v`.

Examples

This attribute is generally set in the BomTemplate, or by the `create_autoload_filegroup(2)` command. See these for examples.

See Also

[create_autoload_filegroup\(2\)](#), [load_autoload_filegroup\(2\)](#),
[unload_autoload_filegroup\(2\)](#), [DefaultInstallDir\(3\)](#), [ExcludeLoadPatterns\(3\)](#)

[API Index](#) [Command Reference](#)

DefaultParamRange

Store the Parameter Range based on the unconfigured core

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

Examples

See Also

DefaultValue

Default value for a parameter

Definition

Type: string**Flags:****Default value:****Valid on:** attrDefn_param _____

Description

The DefaultValue attribute specifies a default value for a parameter.

For HDL parameters, the primary method to specify a default parameter value is to initialize the parameter to the desired default value in the HDL code. For cases where you want to calculate a default value for an HDL parameter in a manner that cannot be achieved with standard VHDL or Verilog syntax, you can use DefaultValue to specify the default value for the parameter as a parameter expression. The parameter takes on the default value calculated from the specified parameter expression, and users can specify a different value when they perform the Specify Configuration activity. If you do not want users to modify the value of the parameter, set the ReadOnlyParam to true on the parameter.

For custom activity parameters, you set DefaultValue on the parameter by using the -default option to the create_custom_activity_parameter command. The initial Value of the parameter is the value of DefaultValue. Users specify a new value for the activity parameter by specifying a new value in the custom activity dialog or by executing the set_activity_parameter command.

Examples

The following VHDL code annotation sets the default value of parameter C as a function of parameters A and B:

```
--reuse-prAGMA startSub Generics [ReplaceDesignParams TOP %subText]
generic (
--reuse-prAGMA attr MinValue 1
--reuse-prAGMA attr MaxValue 4
A : integer := 2;
--reuse-prAGMA attr MinValue 1
--reuse-prAGMA attr MaxValue 16
B : integer := 2;
--reuse-prAGMA attr MinValue 1
--reuse-prAGMA attr MaxValue 16
```

```
--reuse-pragma attr DefaultValue =(@A+@B)/(2*@A)
```

```
C : integer
```

```
);
```

```
--reuse-pragma endSub Generics
```

The following line in a `_Obj.tcl` file for a custom activity sets the DefaultValue of the custom activity parameter `MyParam` to 4:

```
set P [create_custom_activity_parameter -activity MyActivity \
-name MyParam -type integer -label "Test Number" -default 4 \
-description "Which test results do you want to view?"]
```

See Also

[set_parameter_attribute\(2\)](#), [create_custom_activity_parameter\(2\)](#), [Value\(3\)](#), [ReadOnlyParam\(3\)](#)

[API Index](#) [Command Reference](#)

DefaultValueMessage

Message to display along with the default value

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

The DefaultValueMessage attribute specifies a string to be displayed with the default value when displaying parameter help text.

Examples

```
set_attribute $param -attr DefaultValueMessage \  
-value "Value is based on the number of fields"
```

Help text display for the parameter would give you the following output which includes the default value and the default value message:

```
get_attribute -attr HelpText $param
```

Description: No description available.

DefaultValue: 16 (Value is based on the number of fields)

See Also

[set_parameter_attribute \(2\)](#), [create_custom_activity_parameter \(2\)](#), [Value \(3\)](#), [ReadOnlyParam \(3\)](#)

DeferEvaluation

If true, port will be present in elaborated design irrespective of generateIf condition. Its conditionality will be preserved in generated toplevel netlist on subsystem level.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: param pin port _____

Description

Setting this attribute to true (1) instructs coreAssembler to leave the corresponding port/pin in the design, regardless of the result of evaluating its existence condition. The existence condition is generally defined via the GenerateIf attribute (inferred when a port is defined within a ifdef/endif section). If the ifdef macro is statically defined (i.e. it is defined via a define statement without reuse-pragma annotations), or it is statically defined via the CompileDefines parameter of the Load Designs activity, then the StaticDefineExpr attribute must be explicitly set to the existence condition as it cannot be inferred in this case.

DeferEvaluation can be set either through use of reuse-pragma in rtl or _____
by explicitly setting it through set_attribute command. _____

Examples

Set DeferEvaluation on ports in Verilog RTL where the existence condition is inferred from the code:

```
// reuse-pragma attr DefaultValue false
`define EXTENDED_MODE
module test (
input in1,
`ifdef EXTENDED_MODE
// reuse-pragma attr DeferEvaluation 1
input [3:0] in2,
`else
// reuse-pragma attr DeferEvaluation 1
input in3,
`endif
output out1
`ifndef EXTENDED_MODE
// reuse-pragma attr DeferEvaluation 1
```

```
, output out2  
'endif  
);
```

Set `DeferEvaluation` on ports in Verilog RTL where the existence condition is static:

```
// EXTENDED_MODE defined for coreBuilder via CompileDefines in Load  
Designs.  
module test (  
  input in1,  
  `ifdef EXTENDED_MODE  
    // reuse-pragma attr StaticDefineExpr @EXTENDED_MODE  
    // reuse-pragma attr DeferEvaluation 1  
    input [3:0] in2,  
  `endif  
  output out1  
);
```

See Also

[StaticDefineExpr \(3\)](#), [reuse-pragma \(2\)](#), [set_attribute \(2\)](#) _____

[API Index](#) [Command Reference](#)

Define Component Views

Add or delete IPXACT views for the component.

Parameters

Activity has no parameters.

See Also

[Create Bill of Materials.](#)

DefineLibClocks

Defines hard macro libraries containing clocks to be defined for constraints.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This attribute is used to associate clocks from a hard macro library with the given design. The attribute subscript defines the instance path within the design to where the hard macro resides. The value of the attribute indicates the name of the hard macro db file containing the generated clock definitions.

When this attribute is set the read_component_constraints command will extract clock names from the hard macro db file and define those clocks so that they can be referenced in other constraints defined for the component. These clocks are defined automatically for Design Compiler but also need to be visible within the coreTools to enable references from other constraints.

Work is done by invoking the define_lib_clocks command with the indicated library and instance path.

Examples

The current design contains a hard macro at phy0/pma. The macro is defined in dwc_test_lib.db:

```
set_design_attribute {DefineLibClocks[phy0/pma]} dwc_test_lib.db
```

See Also

[define_lib_clocks \(2\)](#)_____

Define Memory Maps

This activity is used to create and view memory maps.

Parameters

Activity has no parameters.

See Also

[API Index](#) [Command Reference](#)

DeliverableType

The type of information stored in this deliverable.

Definition

Type: string

Flags:

Default value: Other

Valid on: filegroup filegroupGroup

Description

This attribute describes the type of information stored in this deliverable.

Examples

See Also

Description

Brief description of the item

Definition

Type: string

Flags:

Default value: No description available.

Valid on: [Strategy](#) [attrDefn](#) [busBit](#) [cell](#) [clock](#) [design](#) [file](#) [filegroup](#)

[filegroupGroup](#) [hdlFunc](#) [knowledgeBase](#) [net](#) [param](#) [pin](#) [port](#)

[timingException](#)

Description

The Description attribute provides a brief textual explanation of the selected item. When a user selects "What's This?" help for the item, the coreTool displays a help message that includes the text specified by the Description attribute.

As a core developer, you most commonly set the Description attribute on design parameters and custom activity parameters to provide a "quick help" description of the parameter. To provide more detailed help text for the parameter, place the help text in a file and set the HelpUrl attribute on the parameter to the location of the detailed help text file.

Examples

To set a value for Description on the custom activity parameter named TestLog:

```
coreBuilder> set_parameter_attribute TestLog
Description "View simulation log for which test?"
```

See Also

[set_parameter_attribute \(2\)](#), [HelpUrl \(3\)](#)

[API Index](#) [Command Reference](#)

Design Configuration Intent

Review/specify intent attributes associated with the HDL parameters for the core design.

Parameters

Activity has no parameters.

See Also

[Load Designs.](#)

Design_Compiler_Reference_Methodology

Performs a one pass, top down compile from RTL using compile_ultra. This flow implements the Design Compiler Reference Methodology associated with the version of Design Compiler being used.

Parameters

Parameters Table

Name	Label	
Automotive		
AutomotiveSynthesisFlowType	AutomotiveSynthesis Flow Type	Supported automotive EnumValues: TMR_and_DC
TMRSeparationDistance	TMR Separation Distance	Specifies the distance repelling bound constr group. It could be either an {x y} pair or a single radial distance.
DCLSDistance	DCLS Distance	Used to specify the DC pair of {width height} group bound with the dimensions.
Basic		
QorEffort	QoR effort:	How much CPU time using this strategy. EnumValues:
OptimizationFlow	Optimization flow:	For optimization flow out of hplp hc rtm_e high performance low For high connectivity runtime exploration se timing optimization se - EnumValues: - Symbolic Name Performance I Connectivity} {Timing Optim
ReducedEffortOptimizationFlow	Reduced Effort Optimization Flow	This parameter when e -reduced_effort option set_qor_strategy comm set_qor_strategy -redu adjusts selected Optim

coreTools Command Reference Index

Name	Label	
		or timing) for reduced expense of timing QOR. Physical Synthesis in I
EnableOptimizeNetlist	EnableOptimizeNetlist	Performs a high-effort the current design for results (QoR).
OptimizeNetlistArgs	OptimizeNetlistArgs	Performs monotonic g optimization to improv degrading timing or le only when EnableOpti is enabled.By default n optimize_netlist -area also take further these (-no_boundary_optimi hierachical boundary c -no_seq_output_invers sequential output inver by DC.
Basic/Compile Ultra		
AdaptiveRetiming	Enable Adaptive Retiming	Use adaptive retiming '-retime' with compile allows Design Compil and latches to improve
CompileUltraInitOptions	Additional Initial Options	Extra options to pass t compile_ultra comman
CompileUltraIncrOptions	Additional Incremental Options	Extra options to pass t compile_ultra comman
Basic/ICC2 Write Files		
WriteICC2Files	Write ICC2 files after synthesis	Indicates whether an I generated when synthe enabled, the ICC2 file [workspace]/syn/[stag
Basic/Netlist Generation/Netlist Generation		
NetlistFormat	Netlist format:	Causes synthesis chec written in the specified The netlist will be loca directory for the check • EnumValues: .neted .pla .st • Symbolic Nam lsi mentor pla
ChangeNamesRuleName	change_names Name Rules:	Indicates the name of a naming rule. Prior to v in the specified format will be called with the
RemoveUnconnectedPorts		Removes unconnected

coreTools Command Reference Index

Name	Label	
	Remove unconnected ports	
Basic/Netlist Generation/set_fix_multiple_port_nets		
FixMultiplePortNetsDefault	Fix Multiple Port Nets Default	Will cause set_fix_mu -default to be used in l No buffers will be inse multiple port nets.
FixMultiplePortNetsFeedthroughs	Fix Multiple Port Nets Feedthroughs	Will cause set_fix_mu -feedthroughs to be us Compiler.
FixMultiplePortNetsOutputs	Fix Multiple Port Nets Outputs.	Will cause set_fix_mu -outputs to be used in
FixMultiplePortNetsConstants	Fix Multiple Port Nets Constants	Will cause set_fix_mu -constants to be used i
FixMultiplePortNetsBufferConstants	Fix Multiple Port Nets BufferConstants	Will cause set_fix_mu -buffer_constants to be Compiler.
Basic/SDC Generation		
WriteSDCArgs	Options for write_sdc	Defines arguments to l write_sdc command w constraints for synthe
Basic/SDF Generation		
WriteSDF	Write SDF after synthesis	Indicates whether an S generated when synthe enabled, the SDF file v [workspace]/syn/[stag
Basic/Static Aware Flow		
EnableStaticAwareFlow	Enable Static Aware Flow	The VC SpyGlass Stat generates an extra data read in synthesis tool t netlist compilation/opt introduce new CDC is of the feature is limite coreConsultant for Des enable the feature, VC Engineering Council n 2023.09 or higher, and activity must be comp
StaticAwareDBPath	Static Aware synthesis database	Path to Static Aware s
Basic/Technology Node		
TechnologyNode	Technology Node	Specifies the technolo in set_technology com Compiler NXT. This h setting in Design Com technology node speci

coreTools Command Reference Index

Name	Label	
		coreTools will not validate Technology Node value. This operation can be performed by Design Compiler.
Clock Gating		
ClockGating	Insert Clock Gating	Add clock gating logic to the design. The insert_clock_gating command, as well as other clock gating parameters, are used to control the logic.
SelfGating	Insert Self Gating	Self-gating is an clock gating technique that optimizes dynamic power by gating the clock signal in those registers where the data saved in a register is unchanged. An enable signal is generated by comparing the current data with the new data arriving at the pin, and that signal is used to enable the inserted self-gating cell. The command adds -self_gating option to the insert_clock_gating command. This option is enabled when Physical Synthesis is enabled. This parameter is a control variable INSEL_SELF_GATE in the setup.tcl, jtag_setup.tcl, or jtag_rm_setup folder.
GatingMinimumBitWidth	Gating minimum bit width:	Minimum register bandwidth. Registers with less than this width have its clock gated.
GatingLatchCellName	Gating latch cell name:	Select between latch style and combinational style gate. The name of a library cell for latching the gated clock signal can be entered. If it is left empty, the Design Compiler will select the cell from the target library.
GatingAndFunction	Gating And function:	Circuitry used to implement clock gating functionality for clock gating. The combinational style clock gating is selected. The circuitry will be inserted into the design to gate positive edge triggered registers. If combinational style is chosen this circuitry will gate negative edge triggered registers. The appropriate ICG cell is selected from the library, this value is used to be integrated. - EnumValues: - and} {and buf - inv} {buf name {inv nor} {inv

coreTools Command Reference Index

Name	Label	
		{buf and inv} buf} {buf nan or buf} {inv n
IntegratedAndCell	Integrated And cell:	Integrated library cell AND type clock gating. Design Compiler man set_clock_gating_style information about usin gating cells.
GatingOrFunction	Gating Or function:	Circuitry used to imple functionality for clock style clock gating is se circuitry will be insert network to gate negati registers. If combinati is chosen then this circ used to gate positive e devices. If the appropr available in the library default to integrated. EnumValues: {or buf} {buf {buf nor inv} nand} {inv na or inv} nor {b {buf nor buf} buf} {inv nan
IntegratedOrCell	Integrated Or cell:	Integrated library cell type clock gating. Plea Design Compiler man set_clock_gating_style information about usin gating cells.
GatingSetupTime	Gating setup time:	Setup time to clock en
GatingHoldTime	Gating hold time:	Hold time at clock ena
GatingControlPoint	Gating control point:	Insert an OR gate to co signal for testability EnumValues:
GatingControlSignal	Gating control signal:	Signal used at control Note that if GatingCor the value of GatingCor ignored EnumValues:
GatingObservationLogicDepth	Gating observation logic depth:	Max depth of XOR ob

coreTools Command Reference Index

Name	Label	
GatingCellMaxFanout	Gating cell max fanout value:	Specifies the maximum clock-gating cell. If no specified default Design use its default.
Design for Test		
TestReadyCompile	Test Ready Compile	Enable test-ready compile of this parameter is to option to be added to the call.
InsertDft	Insert Dft	Enables design for test inserted based on the Specify Test Method
NumberOfScanChains	Internal_scan mode chain count.	The number of scan chains. This value is passed to set_scan_configuration-chain_count option. For this is also the number to be created. The default design attribute Number on the implementation
ClockMixing	Clock Mixing	Specifies whether cells clock domains may be chain. The default value attribute ClockMixing implementation context - EnumValues: mix_clocks m - Symbolic Name mix_clocks m
DFTSynthesisOptimization	DFT Synthesis Optimization:	Specify value to perform during DFT insertion. all optimizations, and all optimizations. The specifies the argument 'set_dft_insertion_com-synthesis_optimization topographical mode, the value is none; it cannot - EnumValues: - Symbolic Name
CTLModels	CTL Models	CTL Model files are used test model required to vectors for the design. parameter is read in the top-level synthesis (during bottom-up synthesis) v

coreTools Command Reference Index

Name	Label	
		attribute CTLModels i bottom-up synthesis ru variable DCRM_DFT (DCRM) or CTL_FOF along with the user spe in design attributes and TechnologyCellBindin
Design for Test/Scan Compression Settings		
EnableScanCompression	Enable scan compression	Enables ScanCompres implementation. The c design attribute Enable set on the implementa more information abou refer to DFTMax and guides.
ScanCompressionIOCount	ScanCompression_mode IO count.	When the hybrid DFT number of IOs for Inte ScanCompression_mo This strategy paramete of scan inputs that fee logic when scan comp (either scan_compress streaming_compression the number of scan ou Compressor logic. Thi must be set explicitly t value when Enabled, e 'set_scan_compression -input 3 -output 3 -cha used for configuration (where 12 comes from NumberOfScanCompr
NumberOfScanCompressionChains	Scan compression chain count.	In ScanCompression_ sets the number of sca created internally. The the design attribute NumberOfScanCompr implementation contex
ScanCompressionUsage	Scan compression usage	The value passed along for ScanCompression_ define_test_mode com DFTMAX Ultra and D sections in the TestMA for more details. The c this strategy parameter design attribute 'Scan set on the implementa

coreTools Command Reference Index

Name	Label	
		- EnumValues: streaming_compression - Symbolic Name (DFTMAX) (DFTMAX UI) (DFTMAX SE)
ScanCompressionTestClock	Scan compression test clock	Sets a test clock for scan from the list of test clocks.
ScanCompressionConfiguration	Scan compression configuration	The value of this parameter is passed directly to the 'set_scan_compression' command. For details on compression options, see DFTMAX User Guide, DFTMAX Compression, and 'Managing X Values in
Hierarchical Synthesis		
EnableHierarchicalSynthesis	Enable Hierarchical Synthesis	Setting the parameter 'enable_hierarchical_flow' to true enables hierarchical flow. When set to true, the flow for the sub-block is used for subsystem level synthesis. This is only enabled in coreAssembler.
IncrementalCompileForHierarchicalSynthesis	Perform Incremental Compile	Setting the parameter 'compiler_ultra' to be 'compile_ultra -incremental'. The flow is used for incremental compilation in coreAssembler. The flow is used for creating block abstraction. The flow is used for 'compile_ultra -incremental' and 'dont_touch' attribute options are set to false.
Physical Synthesis		
PhysicalSynthesis	Physical Synthesis	Use physical synthesis in the Compiler. Setting this parameter to true causes the physical synthesis engine to be used during Net loads and delays verification, placement, and wirelength calculation. This flow may be used. This flow may be used either a physical library or Milkyway reference library format. Note that no project is saved with the ddc or l
EnableSPG	Enable SPG	Enable Synopsys Physical Synthesis flow. This flow performs placement and enables synthesis with IC Compiler, facilitating closure and resulting in

coreTools Command Reference Index

Name	Label	
		correlation with ICC. requires DC 2013.03 or later.
FloorplanFileDataFormat	Floorplan Data Format	Format of file containing constraints. - EnumValues: - Symbolic Name
PhysicalConstraintFile	Physical Constraint File	Floorplan constraint file used during synthesis.
EnablePortSideConstraint	Enable Port Side Constraint	Generates a port side file inserted into the synthesis intent for better pin placement. This file is generated when the file given and SideConstraint is set on the design port. This can be used only in the Design Compiler mode. This feature is enabled if the DC version is >= 2013.03.
UtilizationRatio	Utilization Ratio	Specify a utilization ratio for synthesis intent. Its default value is 0.7 and ranges from 0.5 to 1.0. If utilization ratio is applied, floorplan data is not processed.
MinRoutingLayer	Min Routing Layer Name	Specifies the minimum routing layer in the design. The router uses this setting. To ensure that congestion analysis starts with the router, when this option is used, the tool marks the layers below the specified layer as blocked for RC and congestion analysis.
MaxRoutingLayer	Max Routing Layer Name	Specifies the maximum routing layer in the design. The router uses this setting. To ensure that congestion analysis starts with the router, when this option is used, the tool marks the layers above the specified layer as blocked for RC and congestion analysis.
CongestionOptimizationFactor	Congestion Optimization Factor	Enables user to specify a congestion optimization factor for congestion optimization. This specifies the Design Compiler can densely route the uncongested region. The default value is set to 0.9 in the Design_Compiler_Reference_Guide. This feature can be used only when Physical Synthesis is enabled.

coreTools Command Reference Index

Name	Label	
		'OptimizationFlow' set to Performance Low Power Optimization(Tim).
PhysicalLibrary	Physical Library	The physical library to use for physical synthesis.
TechFileFormat	Tech File Format	Select 'tf' to use the technology file to create the Milkyway design library. To select 'plib' to use the technology source (.plib) file EnumValues:
TechFile	Tech File	Technology file used to create the Milkyway design library. The format must match the format specified in the technology file.
MaxTluPlusFile	Max Tlu Plus File	The Max TLU+ file.
MinTluPlusFile	Min Tlu Plus File	The Min TLU+ file.
Tech2ItfMap	Tech file to ITF Mapping File	TECH2ITF mapping file.
MaxEmulationTluPlus	Max Emulation TLU+ file name	To be used for the -max_emulation switch to set_tlu_plus. See the Compiler documentation about the set_tlu_plus.
MinEmulationTluPlus	Min Emulation TLU+ file name	To be used for the -min_emulation switch to set_tlu_plus. See the Compiler documentation about the set_tlu_plus.
CreateCongestionMap	Create Congestion Map	Create graphical congestion map after synthesis completes. To use this feature, you must synthesize the design with the DC-Topographical Design features also requires the design to be a DC-Extension license. The variable must also be set to use this feature.
PhysicalSynthesis/Milkyway		
UseMilkyway	Use Milkyway	Use the Milkyway design library supported for DC version S-2021.06-SP4. Note that the design library must be specified prior to specifying the other parameters. Please see the documentation for more details on the Milkyway format.
MilkywayDesignLibrary	Milkyway Design Library	This parameter corresponds to the Compiler variable mw_design_library. Specify the path to the design library. If a relative path is used, it is relative to the current directory.

coreTools Command Reference Index

Name	Label	
		directory will be created in the workspace 'syn' directory.
CreateMilkywayDesign	Create Milkyway Design	If TRUE, coreTools will create a Milkyway design library.
MilkywayReferenceLibrary	Milkyway Reference Library	This parameter corresponds to the Compiler variable mw_ref_lib. Specify the full path to the reference library. The library must exist before the command. coreTools will check the parameter. coreTools will create the library if it does not exist.
ExtendMWLayers	Extend MW Layers	Insert extend_mw_layers into the topo scripts.
CreateMwDesignOptions	Create Mw Design Options	Additional options to the command. Please see the Milkyway documentation for more details.
MilkywayLogic0Net	Milkyway Logic 0 Net	This parameter corresponds to the Compiler variable mw_logic0_net. Specify the milkyway logic0(1) net design.
MilkywayLogic1Net	Milkyway Logic 1 Net	This parameter corresponds to the Compiler variable mw_logic1_net. Specify the milkyway logic1(1) net design.
WriteMilkywayCell	Save design in Milkyway Cell format	Set this parameter to 'true' to require DC-Topography for the Milkyway cell representation of the design after synthesis. This is required for your implementation for the Compiler. Note, however, that the Cell will succeed only if the design has been used in the design having Milkyway views, especially if the Milkyway views are available.
PhysicalSynthesis/Multi Vth Constraint		
AvailableVthGroups	Available Threshold Voltage Groups	The Voltage Groups Available for the Design.
LowThresholdVoltageGroups	Low Threshold Voltage Groups	Specifies the threshold voltage groups considered to be low-threshold voltage groups. The groups are specified as a list of threshold-voltage group names which are names defined in the Liberty (.lib) description of the cells and cell libraries.
LowThresholdVoltagePercentage	Low Threshold Voltage Percentage	Specifies the maximum percentage of low-threshold voltage for the current design. The argument is a floating-point number.

coreTools Command Reference Index

Name	Label	
		between 0 and 100
LowThresholdVoltageConstraintType	Low Threshold Voltage Constraint Type	Specifies the constraint type. The constraint can be either hard or soft. If the constraint is soft; the tool meets this constraint if it does not adversely affect the timing constraint. EnumValues:
PhysicalSynthesis/NDM Library		
UseNDM	Use NDM	This switch enables NDM. It is used to store the physical information. NDM is only supported in DCRM S-2021.06-SP4 or greater versions of DCRM tool. NDM is supported.
NDMLibraryName	NDM Library Name	Specify the name of the NDM library. The path to the NDM library can be relative, or an absolute path. If a relative path is given, the path will be created in the working directory. The name cannot contain the colon (:) character to delimit the library name.
NDMReferenceLibrary	NDM Reference Library	Specifies the reference NDM library. The NDM library 'NDMLib' specifies the reference library. The physical source data uses the relative paths. If you specify the libraries with a relative path, the tool uses the activity parameter SearchPath to find the libraries. The NDM library must exist before specifying the activity parameter. coreTools v
CreateLibDesignOptions	Create Lib Design Options	Additional options to the createLib command which create the library. See Design Compiler NDM documentation for details.
Power/Power Optimization		
EnableSaifAnnotation	Enable SAIF annotation	Enable the annotation of activity from a SAIF file. The activity from SAIF is used for power optimization. Power activity data also helps in the accuracy of power analysis. PrimeTime-PX.
SaifFile	Saif File Path	SAIF file for power analysis.

coreTools Command Reference Index

Name	Label	
SaifInstanceName	Saif Instance Name	Name of the instance of the design as it appears in the command line. The argument is passed on the command with the -str
ReportPowerOptions	Report Power options	Dash options to be passed to the report_power command
Power/UPF		
EnableUPFFlow	Enable UPF Flow	When Enable UPF Flow is set to 1, the UPF and power constraints are generated during synthesis. In case of DCTCL_opto_strategy, the UPF file can be compiled separately using the UPFFile design attribute.
UPFFlowType	UPF Flow Type	Use the default UPF flow type. The Golden UPF flow and the UPF file are written by synthesis. - EnumValues: - Symbolic Name: UPF}
IncompleteUPFAllowed	Incomplete UPF Allowed	Allow UPF synthesis to complete if the configured UPF file is not found.
UPFFile	UPF File	The name of the Unified Power Format (UPF) file to be used by the Design Compiler load_upf command.
PowerConstraintsFile	Power Constraints File	The power constraints file. The Design Compiler after synthesis generates the file. This file can be used by the 'set_voltage' command to set the dont_touch elements if needed.
Reports		
GenerateRMReports	Generate supplementary Reports	When enabled generates the Reports as mentioned in the Report Methodology.
DisableCongestionReport	Skip Congestion Report	Disables congestion report. May be used for runtime.

See Also

[API Index](#) [Command Reference](#)

DftExistingSignalActiveState
Active logic state for the signal.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: busBit port _____

Description

Specifies the active logic state for the signal in the DFT 'spec' view.

Examples

Specify that the port 'rst_n' is an active asynchronous reset.

```
coreBuilder> set_port_attribute rst_n DftExistingSignalType Reset
coreBuilder> set_port_attribute rst_n DftExistingSignalActiveState 0
```

See Also

[set_port_attribute \(2\)](#), [DftSpecSignalType \(3\)](#), [DftExistingSignalType \(3\)](#).

DftExistingSignalTestMode

The name of the test mode the dft signal specification applies to.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: busBit port _____

Description

When a DftExistingSignalType attribute has been specified for a port, DftExistingSignalTestMode may be used to associate that specification with test mode.

Examples

Specify that port 'myPort' is to be held at a constant logic level of 1 for the test mode 'myTestMode'.

```
coreBuilder> set_port_attribute myPort DftExistingSignalType Constant
coreBuilder> set_port_attribute myPort DftExistingSignalActiveState 1
coreBuilder> set_port_attribute DftExistingSignalTestMode myTestMode
```

See Also

[set_port_attribute \(2\), DftExistingSignalType \(3\),](#)
[DftExistingSignalActiveState \(3\)](#)

[API Index](#) [Command Reference](#)

DftExistingSignalTiming

Waveform to be used for the test signal.

Definition

Type: string

Flags: subscripted

Default value: =sDft::defaultDftTiming

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: busBit port _____

Description

Specifies the waveform timing for the DFT signal.

Examples

Create a test clock on port tclk, and specify the waveform.

```
coreBuilder> set_port_attribute tclk DftExistingSignalType ScanClock
```

```
coreBuilder> set_port_attribute tclk DftExistingSignalTiming {45 55}
```

See Also

[set_port_attribute \(2\)](#), [DftExistingSignalType \(3\)](#)

DftExistingSignalType

Test signal to be used for the 'existing_dft' view of set_dft_signal.

These are signals that describe structures which already exist that must be understood for the design to pass DRC (dft_drc): clocks, resets, constants, existing scan chains, etc.

Definition

Type: string

Flags: subscripted

Default value: =InheritValue up NONE

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: busBit port _____

Description

Test signal to be used for the 'existing_dft' view of set_dft_signal.

These are signals that describe structures which already exist that must be understood for the design to pass DRC (dft_drc): clocks, resets, constants, existing scan chains, etc.

Examples

Specify that the port rst_ is an active low asynchronous reset in all test modes.

```
coreBuilder> set_port_attribute rst_ DftExistingSignalType Reset
coreBuilder> set_port_attribute rst_ DftExistingSignalActiveState 0
```

See Also

[set_port_attribute \(2\), DftSpecSignalType \(3\), _____](#)
[DftExistingSignalActiveState \(3\) _____](#)

DftExistingSignalUsage

The usage scenario(s) the dft signal specification applies to.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: busBit port _____

Description

This attribute defines the usage mode of a DFT signal in the 'dft_existing' view. Valid values are: "scan", "clock_gating", and "scan clock_gating". This corresponds to the -usage argument of the set_dft_signal command in Design Compiler.

Examples

[DftExistingSignalType \(3\)](#), [DftSpecSignalType \(3\)](#), [DftSpecSignalUsage \(3\)](#)

[API Index](#) [Command Reference](#)

DftSpecSignalActiveState
Active logic state for the signal.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: busBit port _____

Description

Specifies the active logic state for the signal in the DFT 'spec' view.

Examples

Specify that the port 'se_n' is an active low scan enable.

```
coreBuilder> set_port_attribute se_n DftSpecSignalType Constant
```

```
coreBuilder> set_port_attribute se_n DftSpecSignalActiveState 0
```

See Also

[set_port_attribute \(2\)](#), [DftSpecSignalType \(3\)](#), [DftExistingSignalType \(3\)](#).

DftSpecSignalTestMode

The name of the test mode the dft signal specification applies to.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: busBit port _____

Description

When a DftSpecSignalType attribute has been specified for a port, DftSpecSignalTestMode may be used to associate that specification with test mode.

Examples

Specify that port 'test_mode_b' is used as test mode signal for the test mode 'myTestMode'.

```
coreBuilder> set_port_attribute test_mode_b DftSpecSignalType TestData
```

```
coreBuilder> set_port_attribute test_mode_b DftSpecSignalActiveState 1
```

```
coreBuilder> set_port_attribute DftSpecSignalTestMode myTestMode
```

See Also

[set_port_attribute \(2\)](#), [DftExistingSignalType \(3\)](#), [DftSpecSignalType \(3\)](#), [DftSpecSignalActiveState \(3\)](#)

DftSpecSignalType

Test signal to be used for the 'spec' view of set_dft_signal. These are signals that describe structures that DFT Compiler should use for insert_dft: scan in, scan out, scan enable, etc.

Definition

Type: string

Flags: subscripted

Default value: =InheritValue up NONE

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: busBit port _____

Description

Test signal to be used for the 'spec' view of set_dft_signal. These are signals that describe structures that DFT Compiler should use for insert_dft: scan in, scan out, scan enable, etc.

Examples

Specify that test_si is a scan in port.

```
coreBuilder> set_port_attribute test_si DftSpecSignalType ScanDataIn
```

See Also

[set_port_attribute \(2\)](#), [DftExistingSignalType \(3\)](#), _____

[API Index](#) [Command Reference](#)

DftSpecSignalUsage

The usage scenario(s) the dft signal specification applies to.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: busBit port _____

Description

This attribute defines the usage mode of a DFT signal in the 'spec' view. Valid values are: "scan", "clock_gating", and "scan clock_gating". This corresponds to the -usage argument of the set_dft_signal command in Design Compiler.

Examples

See Also

[DftSpecSignalType \(3\)](#), [DftExistingSignalType \(3\)](#), [DftExistingSignalUsage \(3\)](#)

[API Index](#) [Command Reference](#)

DigitsPrecision

Number of digits of precision to interpret and display

Definition

Type: long**Flags:****Default value:****Valid on:** attrDefn_____

Description

The DigitsPrecisions attribute specifies the number of digits of precision that the coreTool command interpreter will process and display for a parameter.

For example, if you create a custom activity parameter of type float, you can use DigitsPrecision to specify the number of digits after the decimal point that you want to use in the parameter value. If you set DigitsPrecision to three and the parameter value is five, the value 5.000 will appear in the parameter dialog. If you set the parameter value to 5.1234567, the coreTool will round the value to 5.123 before processing the value. If you re-invoke the parameter dialog, the value 5.123 will be displayed.

Examples

The following line in an _Obj.tcl file for a custom activity specifies that the value of parameter A should have 5 digits after the decimal point:

```
set_parameter_attribute A DigitsPrecisions 5
```

See Also

[set_parameter_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

DisableAggregation

Set on a pin/port to disable aggregate connections to it in the netlist.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: pin port _____

Description

Set to true on a pin or the corresponding port to disable use of an aggregate assignment on the pin in the netlist written by activity GenerateSubsystemRTL. This will prevent the use of aggregate assignments on the specified pin regardless of the value of the "Allow Aggregate Assignments" activity parameter (for GenerateSubsystemRTL). If DisableAggregation is set to true on a design port, then aggregate assignments will not be used on connections to the corresponding component pin(s).

Examples

```
set_attribute [find_item -type pin U1/in1] -attr DisableAggregation  
-value true
```

See Also

[API Index](#) [Command Reference](#)

DocAddressOffset

Text relating to the AddressOffset attribute, for inclusion in documentation

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This is the 'documentation' version of the AddressOffset attribute. If set, this attribute will be used in generated documentation in place of the AddressOffset attribute.

Examples

See Also

[AddressOffset \(3\)](#)

[API Index](#) [Command Reference](#)

DocBlockTableAddressOffset

Text relating to the BlockTableAddressOffset attribute, for inclusion in documentation

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

[API Index](#) [Command Reference](#)

DocDefaultValue

Documentation oriented default value for a parameter.

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

Examples

See Also

DocDescription

Documentation oriented version of the description of the given item. The text may include DocBook tags.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This is the 'documentation' version of the Description attribute. If set, this attribute will be used in generated documentation in place of the Description attribute.

Examples

See Also

[Description \(3\)](#)

DocDescriptionField

Defines extra tagged information to be included in the table cell describing the given item. Each value is included, tagged with its corresponding subscript. The text may include DocBook tags.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: paramport _____

Description**Examples****See Also**

DocEnabled

Documentation oriented description of when the given item is enabled. The text may include DocBook tags. Must be set for each subscript for which Enabled is set.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 1

Valid on: param port _____

Description

This is the 'documentation' version of the Enabled attribute. If set, this attribute will be used in generated documentation in place of the Enabled attribute.

Examples

See Also

[Enabled \(3\)](#)

DocGenerateIf

Documentation oriented description of when this port exists. The text may include DocBook tags.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 1

Valid on: port _____

Description

This is the 'documentation' version of the GenerateIf attribute. If set, this attribute will be used in generated documentation in place of the GenerateIf attribute.

Examples

See Also

[GenerateIf \(3\)](#)

[API Index](#) [Command Reference](#)

DocGroup

Indicates that this object belongs to a group of objects that will be documented as a group.

Definition

Type: string

Flags:

Default value:

Valid on: param port _____

Description

The attributes DocGroup and DocMaster provide a mechanism for grouping where multiple objects are documented as a single object (single entry) in the table or in the schematic.

All items with the same value of DocGroup are documented together. The item that has DocMaster set on it is the one from which the Description is taken if no Description is specified, and from which the range information is gathered.

Examples

```
// reuse-pragma attr EnumValues {0 2 4}
'define checker 4
// reuse-pragma attr DocMaster i 0 2 0 @checker
// reuse-pragma attr Description Master Info on Port
// reuse-pragma attr DocGroup some_INFO(i)
input [1:0] someport0,
// reuse-pragma attr DocGroup some_INFO(i)
// reuse-pragma attr GenerateIf @checker>2
input [2:0] someport1,
// reuse-pragma attr Description Child Info on Port
// reuse-pragma attr DocGroup some_INFO(i)
input [3:0] someport2,
For the above example the three ports having the same DocGroup will
appear in a single row in the IO report.
```

As no Description is provided for 'someport1' the description will be gathered from the Description attribute applied to the DocMaster pertaining to this DocGroup.

See Also

[DocMaster \(3\)](#)

[API Index](#) [Command Reference](#)

DocGroupName

Grouping value used to group objects for documentation generation.

Definition

Type: string

Flags:

Default value:

Valid on: [param port](#) _____

Description

This is the 'documentation' version of the GroupName attribute. If set, this attribute will be used in generated documentation in place of the GroupName attribute.

Examples

See Also

[GroupName](#) (3)

DocInclude

Used to conditionally include an item in generated documentation.

Definition

Type: boolean

Flags: subscribed

Default value: 1

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 1

Valid on: Strategy busBit cell design file filegroup filegroupGroup

[hdlFunc](#) [net](#) [param](#) [pin](#) [port](#) [timingException](#)

Description

Examples

See Also

[API Index](#) [Command Reference](#)

DocLabelName

Report oriented label and name combo for a parameter.

Definition

Type: string

Flags: readOnly

Default value: =sDocBook::reportLabelName %item

Valid on: param_____

Description

Examples

See Also

DocMaster

Indicates that this object is the 'master' element among the group of elements with the same value for DocGroup, and is the element from which the documentation should be generated. The value of this attribute indicates the name of the looping variable and the min and max loop indices for the group (for documentation purposes). It can contain three entries "var min max" or five entries, with the two added entries being the post-elaboration min/max values. The latter two values can be parameter expressions if needed. Note a 4th/6th argument was added that allows a list of attribute/value pairs which can override the existing attribute value [list Name "UseMyName" GroupName "UseMyGroupName"].

Definition

Type: string

Flags:

Default value:

Valid on: param port _____

Description

Examples

Specify the loop variable, min and max values, and override the name attribute for reports.

```
set_attribute $myParam -attribute DocMaster "i 0 7 [list Name "My
Display"]
```

Specify loop variable, min and max values, post-elaboration min and max, and override the Name and Label attributes.

```
//reuse-pragma DocMaster n 0 15 0 @myParam-1 {Name "Better Name" Label
"Better Label"}
```

See Also

[DocGroup\(3\)](#)

DocMemoryAccess

Text relating to the MemoryAccess attribute, for inclusion in documentation

Definition

Type: string

Flags:

Default value: =sMem::defaultDocMemoryAccess %item

Valid on: param_____

Description

This is the 'documentation' version of the MemoryAccess attribute. If set, this attribute will be used in generated documentation in place of the MemoryAccess attribute.

Examples

See Also

[API Index](#) [Command Reference](#)

DocOffset

Replaces register table pretext overriding GlobalAddressOffset value normally displayed.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

[API Index](#) [Command Reference](#)

DocPortWidth

Documentation oriented description of the width of the port.

Definition

Type: string

Flags:

Default value:

Valid on: port _____

Description

This is the 'documentation' version of the PortWidth attribute. If set, this attribute will be used in generated documentation in place of the PortWidth attribute.

Examples

See Also

[PortWidth \(3\)](#)

[API Index](#) [Command Reference](#)

DocPowerDomain

Documentation oriented description of the power domain associated with a port.

Definition

Type: string

Flags:

Default value:

Valid on: port _____

Description

Examples

See Also

DocRangeDecoratedName

Report oriented value for the RangeDecoratedName of a port.

Definition

Type: string

Flags:

Default value: =sDocBook::docRangeDecoratedName %item

Valid on: port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

DocRegTableAddressOffset

Text relating to the RegTableAddressOffset attribute, for inclusion in documentation

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

DocRegisterResetMask

Documentation oriented description of the RegisterResetMask attribute.

The text may include DocBook tags.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

[API Index](#) [Command Reference](#)

DocRegisterResetValue

Documentation oriented description of the reset value for this register or field. The text may include DocBook tags.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This is the 'documentation' version of the RegisterResetValue attribute. If set, this attribute will be used in generated documentation in place of the RegisterResetValue attribute.

Examples

See Also

[RegisterResetValue \(3\)](#)

DocRegisterSize

Documentation oriented description of the size of this register or field.
The text may include DocBook tags.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This is the 'documentation' version of the RegisterSize attribute. If set, this attribute will be used in generated documentation in place of the RegisterSize attribute.

Examples

See Also

[RegisterSize \(3\)](#)

[API Index](#) [Command Reference](#)

DocRegistered

Documentation oriented description of whether or not a port is registered.

Definition

Type: string

Flags:

Default value:

Valid on: busBit port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

DocRegisteredBy

Documentation oriented description of whether or not a port is registered.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: Default

Valid on: busBit port _____

Description

Examples

See Also

DocShortDescription

Shortened version of the regular DocDescription.

Definition

Type: string

Flags:

Default value: =sDocBook::shortStringAttrValue %item DocDescription

Valid on:

Description

Examples

See Also

[API Index](#) [Command Reference](#)

DocSynchronousTo

Documentation oriented description of the clocks to which the port is synchronized.

Definition

Type: string

Flags:

Default value:

Valid on: port _____

Description

Examples

See Also

DocTestable

Documentation oriented description of the Testable attribute. The text may include DocBook tags.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

DocVisible

Documentation oriented description of when this register or field is visible. The text may include DocBook tags.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 1

Valid on:

Description

This is the 'documentation' version of the Visible attribute. If set, this attribute will be used in generated documentation in place of the Visible attribute.

Examples

See Also

[Visible \(3\)](#)

DocVolatileMemory

Documentation oriented description of the VolatileMemory attribute. The text may include DocBook tags.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

DocWaiver

Used to document waivers for use of Doc* attributes. Subscript indicates attribute the waiver applies to.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: DocBitRange DocDefaultValue DocEnabled
DocGenerateIf DocInclude DocMemoryAccess DocOffset DocPowerDomain
DocRangeDecoratedName DocRegisterResetMask DocRegisterResetValue
DocRegisteredBy DocSynchronousTo DocTestable DocVisible
DocVolatileMemory Enum2Big EnumData EnumMixed

Default subscript:

Valid on: param port _____

Description

This attribute is used to document reasons why other Doc* attributes are used for documentation generation. For example, when setting DocVisible on a register, it is expected that the DocWaiver attribute (using the DocVisible subscript) is set on the same register indicating why DocVisible needs to be set instead of relying on the value generated directly from Visible.

Examples

See Also

DontTouchNetwork

Set dont_touch_network on this port.

Definition

Type: boolean

Flags:

Default value: =InheritValue up {}

Valid on: busBit port _____

Description

The DontTouchNetwork attribute indicates whether the selected port should be marked dont_touch_network for synthesis.

If DontTouchNetwork is set to true on a port, coreConsultant generates a Design Compiler set_dont_touch_network command for the port to prevent cells and nets in the transitive fanout of the port from being modified or replaced during optimization.

The coreTools automatically set DontTouchNetwork to true on clock ports (ClockName has a value) and asynchronous reset ports (IsAsynchronousReset is true). If you do not want to set DoneTouchNetwork on a clock or asynchronous reset port, you must explicitly set DoneTouchNetwork to false.

If DontTouchNetwork is not explicitly set to true or false on a port, the port inherits the DontTouchNetwork value from the connected port on the next higher level design.

Examples

To not generate a set_dont_touch_network command for the rst_in port:

```
coreConsultant> set_port_attribute rst_in DontTouchNetwork false
```

See Also

[set_port_attribute \(2\), ClockName \(3\)](#)_____

Drive

Drive resistance for input or inout ports.

Definition

Type: float

Flags:

Default value: =InheritValue up {}

Valid on: busBit port _____

Description

Specifies a nonnegative resistance value. The resistance is the output resistance of the cell that drives the port, such that a higher drive strength (resistance) means less drive capability and longer delays. Thus, a resistance of 0 is infinite drive, or no delay between the ports and all that is connected to them. The resistance must be in units consistent with the technology library used during optimization.

Note: set the DrivingCell attribute is more convenient and accurate than setting the Drive attribute for describing the drive capability of a port. Setting the Drive attribute removes any corresponding rise or fall driving cell attributes on the specified ports. Use the DrivingCell attribute instead of Drive, if possible.

Examples

The following example sets the drive of ports "A", "B", and "C" to 2.0:

```
coreConsultant> set_port_attribute {A B C} Drive 2.0
```

See Also

[set_port_attribute \(2\)](#), [DrivingCell \(3\)](#)_____

DrivingCell

Specifies how to generate a set_driving_cell constraint for an input port.

Definition

Type: string

Flags:

Default value: =InheritValue up {=sCstr::budget_driving_cell %item %attr}

Supported formulas: infinite_drive select_by_class
select_by_function select_by_name

Valid on: busBit port _____

Description

The DrivingCell attribute specifies the expected driving cell and (optionally) pin for an input port. Specify one of the following technology-independent formulas as the value for DrivingCell:

- {=select_by_class <class> [<drive_strength>]} - Select a representative library cell of the specified drive strength for the specified class of cells (combinational, sequential, or tristate). <drive_strength> is optional (low, median, or high) and defaults to median if not specified.
- {=select_by_function <function> [<drive_strength>]} - Select a representative library cell of the specified drive strength for the specified cell function (nand2, buf, inv, mux21, dff, latch, or xor2). <drive_strength> is optional (low, median, or high) and defaults to median if not specified.
- {=select_by_name [<library_name>]/<cell_name>/[<pin_name>]} - Select a library cell by cell name and, optionally, by pin name. Use this formula if you know the technology-specific cell name (and optional pin name) that you want to specify as the driving cell for synthesis.
- {=infinite_drive} - Do not generate a driving cell constraint for the port. Instead, Design Compiler assumes infinite drive and does not add buffers to support the loads on the nets connected to the input port. The coreTools set DrivingCell to {=infinite_drive} by default on clock and reset ports, because designers typically use external tools to buffer clock and reset ports.

If you do not specify a value for DrivingCell, the port inherits the DrivingCell value from the connected port on the next higher level design.

Examples

To select the median drive strength sequential cell in the currently loaded target technology library as the driving cell for the data_in port:

```
coreBuilder> set_port_attribute data_in DrivingCell \
{=select_by_class "sequential" "median"}
```

To select a known library cell and pin as the driving cell for the data_in port, assuming the surrounding logic has already been mapped:

```
coreConsultant> set_port_attribute data_in DrivingCell \
{=select_by_name fd1a1/q}
```

See Also

set_port_attribute (2), select_by_class (2), select_by_function (2), _____
select_by_name (2), infinite_drive (2) _____

[API Index](#) [Command Reference](#)

EnableDftAutoFix

Enables the DFT Compiler AutoFix utility.

Definition

Type: boolean

Flags:

Default value: =InheritValue up FALSE

Valid on: design_____

Description

Enables the DFT Compiler autofix utility to fix the specified test violations.

Setting EnableDftAutoFix to true will cause a set_autofix_configuration command to be issued in DFT Compiler. The correspondence between coreTools attributes and set_autofix_configuration options is as follows:

AutoFixClocks -clock

AutoFixAsync -async

AutoFixAsyncWithScanEnable -fix_async_with_scan_en

AutoFixAsyncLogicGate -async_fix

BistAutoFixBusses -bus

BistAutoFixXprop -xprop

Examples

Specify that clocks, asynchronous set/reset violations, floating busses, bus contention, and x-propagation violation should be fixed automatically for all designs.

```
coreBuilder> current_design [get_top_design]
coreBuilder> set_design_attribute EnableDftAutoFix 1
coreBuilder> set_design_attribute AutoFixClocks 1
coreBuilder> set_design_attribute AutoFixAsync 1
coreBuilder> set_design_attribute BistAutoFixBusses 1
coreBuilder> set_design_attribute BistAutoFixXprop 1
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#),
[BistBlockIndividually \(3\)](#), [AutoFixClocks \(3\)](#), [AutoFixAsync \(3\)](#),
[AutoFixAsyncWithScanEnable](#) [AutoFixAsyncLogicGate](#) [BistAutoFixBusses \(3\)](#),
[BistAutoFixXprop \(3\)](#)

[API Index](#) [Command Reference](#)

EnableDftShadowLogic

Enables the DFT Compiler Shadow LogicDFT utility for this design.

Definition

Type: boolean

Flags:

Default value: =InheritValue up FALSE

Valid on: design_____

Description

Enables the DFT Compiler Shadow LogicDFT utility for this design. Setting this attribute to true will cause the -shadow_wrapper option to be used with set_dft_configuration in DFT compiler.

Examples

Enable insertion of a shadow logic for this design.

```
coreBuilder> set_design_attribute EnableDftShadowLogic 1
```

See Also

[set_design_attribute \(2\)](#)_____

EnableScanCompression

When true enables scan compression. For details on DFTMAX compression options, please refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Scan compression is used to reduce number of scan IOs at the top level, and at the same time, facilitating faster testing by increasing the number of scan chains inside the design. Two usage types are supported 'scan_compression' and 'streaming_compression'. The chain count for scan compression is calculated based upon the number of scan chains (either using NumberOfScanChains or the number of test clocks).

Examples

Disable scan compression for the current design.

```
coreConsultant> set_design_attribute EnableScanCompression 0
```

See Also

[NumberOfScanChains \(3\)](#) [ScanCompressionUsage \(3\)](#)_____

[ScanCompressionConfiguration \(3\)](#)_____

Enabled

Enable or disable this parameter in the GUI

Definition

Type: boolean

Flags: subscripted

Default value: 1

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 1

Valid on: Strategy param _____

Description

The Enabled attribute specifies whether the item should be enabled in the coreTool GUI.

As a core developer, you typically use the Enabled attribute to enable or disable user-modification of parameters in a coreTool GUI. For example, you can set the Enabled attribute on a design parameter or custom activity parameter so that user-modification of the parameter is only possible under certain conditions. This is different from setting ReadOnlyParam, which simply disables users from modifying the value of a parameter under any conditions.

To determine whether to enable a parameter in the GUI according to the Enabled attribute, the coreTool determines the logical AND of the set of Enabled values for which the Enabled subscript evaluates to true. If the logical AND of those values is true, the coreTool enables the parameter in the GUI. If the logical AND of those values is false, the coreTool disables (grays out) the parameter in the GUI.

Simple Dependency Checking with Default Subscript

The default subscript for Enabled is 1, which means that the Enabled value alone determines whether the coreTool enables the parameter for user modification. The most simple form of using the default subscript is to simply set the value of Enabled (Enabled[1]) to either 0 or 1 to disable or enable the parameter.

As an example of a simple dependency, you can enable parameter A only if the value of parameter B is 1 by setting the value of Enabled on parameter A to "@B==1". In this case, the Enabled subscript is [1] (the default). Therefore, the coreTool enables parameter A in the GUI only if the value of parameter B is equal to 1.

More Complex Dependency Checking with Single Subscript

As an example of using a single Enabled subscript, consider a design with parameters A, B, C, and D such that:

Description

- If the value of parameter A is 2, parameter C should be enabled only if the value of parameter B is greater than 5 and less than the value of parameter D.
- If the value of parameter A is not 2, parameter C should be enabled regardless of the values of the other parameters.

To enforce this dependency, set the Enabled attribute on parameter C. The required expression for the Enabled subscript is "`@A==2`" because C has restrictions only if the value of A is 2.

The expression that you specify as the value of Enabled determines whether parameter C should be enabled when A = 2. In this example, the required value for `{Enabled[@A==2]}` is "`@B>5&&@B<@D`" because the value of B must be greater than 5 and less than the value of D if the value of A is 2. If "`@B>5&&@B<@D`" evaluates to true (the value of B is greater than 5 and less than the value of D), the coreTool enables parameter C in the GUI. If "`@B>5&&@B<@D`" evaluates to false, the coreTool disables parameter C in the GUI.

Complex Dependency Checking with Multiple Subscripts

To implement more complex parameter dependencies, use multiple Enabled subscripts. The coreTool uses the logic AND of all value expressions for which the associated subscript is true to determine whether to enable the parameter. For example, consider a design with parameters W, X, Y, and Z such that:

- If the value of parameter W is 0, parameter Z should be enabled only if the value of parameter X less than 20.
- If the value of parameter Y is 1, parameter Z should be enabled only if the value of parameter X is greater than 10.

The following Enabled attribute assignments on parameter Z implement the above dependencies:

```
set_parameter_attribute Z {Enabled[@W==0]} @X<20
```

```
set_parameter_attribute Z {Enabled[@Y==1]} @X>10
```

The coreTool logically ANDs each value for which the Enabled subscript evaluates to true. Therefore, if W = 0 and Y = 1, the coreTool enables parameter Z only if the value of parameter X is between 10 and 20.

Examples

The following line in a coreBuilder Tcl script enables parameter A in the GUI only if the value of parameter B is 1:

```
set_parameter_attribute A Enabled @B==1
```

The following annotation in a Verilog source file imposes the dependency that if parameter A = 2, then parameter C is enabled only if parameter B is greater than 5 and less than the value of parameter D:

```
//reuse-pragma attr Enabled[@A==2] @B>5&&@B<@D
parameter C = 0;
```

The following lines in an `_Obj.tcl` file for a custom activity impose the following restrictions on custom activity parameter Z:

- If `W = 0`, Z is enabled only if the value of X is less than 20.
- If `Y = 1`, Z is enabled only if the value of X is greater than 10.

```
set_parameter_attribute Z {CheckExpr[@W==0]} @X<20
set_parameter_attribute Z {CheckExpr[@Y==1]} @X>10
```

See Also

[set_parameter_attribute \(2\)](#), [ReadOnlyParam \(3\)](#), [Visible \(3\)](#)

[API Index](#) [Command Reference](#)

EncryptText

Encrypt text as it is being written.

Definition

Type: boolean**Flags:****Default value:** 0**Valid on:** file [filegroup](#) [filegroupGroup](#)

Description

This attribute is used to indicate that the associated file should be written in encrypted format when written to disk. Core developers can only set this attribute on files whose `FileContentType` is `vhdl`, `verilog`, or `dc_shell`. Note that it will only be possible to read the encrypted file using Synopsys synthesis or verification tools.

Examples

To protect the contents of a verification testbench file:

```
set_attribute myTestBench.vhd -attr EncryptText -value true
```

See Also

[API Index](#) [Command Reference](#)

EndBit

Tag specification ending position.

Definition

Type: string**Flags:** readOnly subscribed**Default value:****Valid subscripts:** <valid subscripts are not statically defined>**Default subscript:** 0**Valid on:** net param port _____

Description

Used to indicate the ending bit position of a ranged element. Typically applies to ports and pins. Value can be an integer or a parameter expression that resolves to an integer.

Examples

This attribute is never set explicitly. It's value comes from parsing HDL or IP-XACT files within coreBuilder or coreAssembler.

See Also

[StartBit \(3\)](#)

EndBitExpr

Indicates the original EndBit expression (subscripted representation)
which was read in from RTL file in LoadDesigns activity

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: param pin port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

EndBitsExpr

Indicates the original EndBits complete expression which was read in from RTL file in LoadDesigns activity

Definition

Type: string

Flags:

Default value:

Valid on: param pin port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

Endian

Are the elements in this address block or interface big or little endian

Definition

Type: string

Flags:

Default value: little

Valid on:

Description

Defines the memory layout for this address block. Valid values are "big", and "little". The default value is "big".

Examples

See Also

[addressBlock \(3\)](#)

EnumValues

an enumeration of the legal attribute values

Definition

Type: string

Flags:

Default value:

Valid on: attrDefn_____

Description

The EnumValues attribute specifies the set of acceptable value for a parameter. If a user attempts to set a parameter value to a value that is not included in the EnumValues list, the coreTool returns an error message.

For example, if a design parameter has legal values 16 and 32, set EnumValues to "16 32" so that coreConsultant only accepts either 16 or 32 as the user-specified value for the parameter.

For a parameter that has a range of legal values instead of a set of discreet legal values, use MinValue and MaxValue instead of EnumValues.

Examples

To set the legal values for the bus_width parameter to 16 and 32:

```
coreBuilder> set_parameter_attribute bus_width EnumValues "16 32"
```

See Also

[set_parameter_attribute \(2\)](#), [MinValue \(3\)](#), [MaxValue \(3\)](#) _____

EnvCheck

EnvCheck is a subscripted attribute that may be attached to a coreKit.

The user may call `verify_environment` from `coreConsultant` to verify the environment based on the `CheckEnv` attribute.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: custom env_vars executables tools

Default subscript:

Valid on: design_____

Description

- env_vars - list of env vars; the argument to `check_env_vars`
- executables - list of executables; the argument to `check_executables`
- tools - list of lists `{{tool_1 [min_ver [max_ver]]}{tool_2 [min_ver [max_ver]]} ...}`; the argument(s) to `verify_tool`. NOTE: dwf is a special case `{dwf min_ver [min_dc_ver]}` and must have a min_ver.
- custom - custom user command string to be executed. The custom command must return a 1 if successful; 0 otherwise.

Examples

```
set_design_attribute -designs [current_design] \
{EnvCheck[env_vars]} "DISPLAY ARCH"
set_design_attribute -designs [current_design] \
{EnvCheck[executables]} "netscape perl"
set_design_attribute -designs [current_design] \
{EnvCheck[tools]} {{vcs 4.1 5.2} {dwf 0301 2001.08} vx1}
set_design_attribute -designs [current_design] \
{EnvCheck[custom]} "custom_user_command arg_1 ... arg_n"
```

See Also

[check_env_vars \(2\)](#), [check_executables \(2\)](#), [verify_dwf \(2\)](#), [verify_tool](#)
[\(2\)](#), [verify_environment \(2\)](#)

[API Index](#) [Command Reference](#)

ExcludeLibCells

Will cause a set_dont_use <libCells> true to be placed on the library cells specified.

Definition

Type: string

Flags:

Default value: =InheritValue up ""

Valid on: design_____

Description

Use this attribute to specify a set of technology library cells that should not be used to compile this design. If no value is the design inherits the ExcludeLibCells value from the next higher level design. The default value of this attribute is null. Wildcards may be used to disable multiple cells with the same base name.

In Fusion Compiler intent ExcludeLibCells will issue the 'set_lib_cell_purpose -include none' command.

Examples

Disble the use of the library cells AND4H and XOR2L for the current_design.

```
coreConsultant> set_design_attribute ExcludeLibCells [list AND4H XOR2L]
```

Disable the use of all of the AOI cells in the library myLib that is contained in the library file myLib.db

```
coreConsultant> set_design_attribute ExcludeLibCells myLib/AOI*
```

See Also

[set_design_attribute \(2\)](#), [IncludeLibCells \(3\)](#)

ExcludeLoadPatterns

A list of glob-style patterns to match file names against. Files that match an entry in this list will not be added to the coreKit.

Definition

Type: string

Flags:

Default value: *.*### CVS RCS

Valid on: [filegroup](#) [filegroupGroup](#)

Description

This attribute controls how an autoloading filegroup actually loads its files. It also has impact on a standard filegroup, when a directory is added to the group.

When the `add_files_to_filegroup(2)` command is passed a directory name to the `-files` option, then all files in the directory minus the files that match the patterns in this attribute are loaded (recursively) into the BoM.

When a pattern (contains '?' or '*' characters) is used in the `DefaultLoadPath(3)` attribute, all files that match that pattern minus the files that match the patterns in this attribute are loaded into the BoM when `load_autoload_filegroup(2)` is run or when the `BuildcoreKit` activity is completed.

Examples

When setting this attribute it is common to add additional patterns into the default value. The default value of this attribute matches version control database for RCS and CVS, as well as auto-save and backup files for Emacs.

To exclude object files use the following command:

```
coreBuilder> set epat [get_filegroup_attribute fgroup
ExcludeLoadPatterns]
coreBuilder> lappend epat *.o
coreBuilder> set_filegroup_attribute fgroup $epat
```

See Also

[create_autoload_filegroup\(2\)](#), [add_files_to_filegroup\(2\)](#),

[load_autoload_filegroup\(2\)](#), [DefaultLoadPath\(3\)](#)

Executables Index

<u>coreAssembler</u>	Invokes the coreAssembler tool in either GUI or shell mode.
<u>coreBuilder</u>	Invokes the coreBuilder tool in either GUI or shell mode.
<u>coreConsultant</u>	Invokes the coreConsultant tool in either GUI or shell mode.

[API Index](#) [Command Reference](#)

Exists

Does this deliverable exist?

Definition

Type: boolean

Flags: readOnly

Default value: =sBom::filesExist %item

Valid on: filegroup filegroupGroup

Description

This ReadOnly attribute is true if there are files in for that deliverable. For autoload filegroups this attribute returns true if the DefaultLoadPath attribute matches any files.

Examples

To determine if the Documentation group has any files in it:

```
coreConsultant> get_filegroup_attribute Documentation Exists
```

See Also

get_filegroup_attribute (2), create_autoload_filegroup (2),
DefaultLoadPath (3), IsUpToDate (3)

Explore RTL Architecture

Generate and (optionally) run RTL Architect scripts.

This activity is used to generate RTLA Reference Methodology scripts. By default, the RTLArchitect run is started immediately after the scripts are generated, but this behavior can be changed by checking the 'Generate Scripts Only' check-box.

Once the RTLA Activity is finished, results will be available in the following locations:

Logs - rtl/run_'flow type'/logs

Reports - rtl/run_'flow type'/reports

Outputs - rtl/run_'flow type'/outputs

If you choose to run the rtl yourself, or wish to review the generated scripts or constraints, these can be found in the following locations:

Flat Makefile: rm_setup/Makefile_rtl_flat Explore Makefile:
rm_setup/Makefile_rtl_explore

Parameters

Parameters Table

Name	Label	Description
Constraint Settings		
WriteSDCArgs	Options for write_sdc	Defines arguments to be passed to the write_sdc command when writing SDC constraints for synthesis.
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieved using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run Style	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options	

coreTools Command Reference Index

Name	Label	Description
		Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
ParallelJobCpuLimit	Parallel Job CPU Limit	Specifies the number of parallel compile jobs that may be run.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.
Explore Flow Settings		
ExploreUtilizationList	Utilization values	Set the EXPLORE_UTILIZATION_LIST values for explore flow
ExploreAspectRatioList	Aspect ratio values	Set the EXPLORE_ASPECT_RATIO_LIST values for explore flow
ExploreMaxRoutingLayerList	Max routing layer value(s)	Set the EXPLORE_MAX_ROUTING_LAYER_LIST/MAX_ROUTING_LAYER variables in the explore flow. Specifying more than one value sets the EXPLORE_MAX_ROUTING_LAYER_LIST variable in the explore scripts. Specifying exactly one value sets the MAX_ROUTING_LAYER variable in the rm script.
ExploreMinRoutingLayer	Min routing layer value	Set the MIN_ROUTING_LAYER value for explore flow
ExploreFlowRunStyle	Explore Flow Run Style	Defines the protocol for launching worker processes on remote machines. The protocol corresponds to the type of compute farm manager used by RTL Architect to submit its jobs. - EnumValues: uge lsf grd - Symbolic Names: uge lsf grd
ExploreFlowGrdOptions	Explore Flow Run Options	UGE or LSF options for launching distributed jobs. Distributed run is set to 'true' for explore flow and 'false' for flat flow. This attribute value is passed to set_host_options command with the -submit_command argument. The set_host_options command is used when the 'DISTRIBUTED' variable is set to true in rtl_setup.tcl
Flat Flow Settings		
UtilizationRatio	Utilization Ratio	Specifies the utilization of the core area. The utilization is the total area of the core

coreTools Command Reference Index

Name	Label	Description
		occupied by all standard cells and macro cells divided by the total core area. You can specify a value between 0 and 1. For example, a core utilization of 0.8 specifies that 80 percent of the core area is used for cell placement at this stage. By default, the core utilization is 0.7. The value specified is used to update the control variable 'AUTO_FLOORPLAN_UTILIZATION' in Architect reference methodology scripts.
EnableFlatFlowMetricsReports	Enable Metrics Reports	Enables writing out reports of worst performing candidates for power/timing/congestion using the get_metrics -nWorst option. Reports specifying RTL lines leading to the worst outcomes in the reported metrics are also generated.
EnableFlatFlowCongestionMetrics	Enable Congestion Metrics	Enable congestion metrics in flat flow
EnableFlatFlowTimingMetrics	Enable Timing Metrics	Enable timing metrics in flat flow
EnableFlatFlowPowerMetrics	Enable Power Metrics	Enable power metrics in flat flow
Floorplan		
FloorplanFileDataFormat	Floorplan File Data Format	Format of file containing floorplan constraints. - EnumValues: def - Symbolic Names: DEF
PhysicalConstraintFile	Physical Constraint File	Path to the TCL floorplan file populates 'TCL_FLOORPLAN_FILE' RTLARM setup path to the DEF floorplan file populates 'DEF_FLOORPLAN_FILES' RTLARM setup path
Flow Settings		
FlowType	RTL Architect Flow Type	Selects the RTLA flow type for this block. - flat : Run FAST synthesis (and optionally use auto floorplanning) and generate congestion/timing/power metrics - explore : Run data exploration based on utilization, aspect ratio and max routing layers - EnumValues: flat explore - Symbolic Names: flat explore
DesignStyle	RTL Architect Design Style	Sets the DESIGN_STYLE variable for this block - flat : flat place and route flow (flat PnR for short) - hier : hierarchical place and route flow (hier PnR for short)

coreTools Command Reference Index

Name	Label	Description
		◆ EnumValues: flat hier ◆ Symbolic Names: flat hier
InsertDft	Insert DFT	Enables design for test circuitry to be inserted based on the settings from the Specify Test Methodology activity.
DFTFlowType	DFT Flow Type	sets the RTLA DFT_FLOW control variable to in_compile or pre_compile. • EnumValues: in_compile pre_compile • Symbolic Names: in_compile pre_compil
UseTechFile	Use Technology File	Use technology file to specify technology information. When enabled the target library (TECH_LIB) setup is not used.
TechFile	TechFile	Technology file used to create the Milkyway design library. The file must match the format specified in TechFormat.
SAIF File Settings		
SAIFFile	SAIF file	Specify an optional SAIF file for accurate power computation. Please specify single SAIF file, RTLA does not support multiple SAIF files as FO SAIF file can be generated using vcd2saif or fsdb2saif verdi command.
SAIFFilePowerScenario	SAIF file power scenario	Specify a power scenario where the SAIF is to be applied. If left empty, current scenario is selected.
SAIFFileSourceInstance	SAIF file source instance	Name of the instance of the current design as it appears in SAIF file. This argument is passed on to the read_saif command with the -strip_path option
TLUPlus		
MinTluPlusName	Early parasitic tech specification name	Specifies the early parasitic tech specification name
MinTluPlusFile	Min TLUPlus file name	TLU+ files to read for Min specification
MinTluPlusLayerMapFile	Min TLUPlus Layer map file	Layer mapping file between ITF and Min tech specification
MaxTluPlusName	Late parasitic tech specification name	Specifies the late parasitic tech specification name
MaxTluPlusFile	Max TLUPlus file name	TLU+ files to read for Max specification
MaxTluPlusLayerMapFile	MAX TLUPlus Layer map file	Layer mapping file between ITF and Max tech specification

Name	Label	Description
UPF		
EnableUPFFlow	Enable UPF Flow	When Enable UPF Flow is set to true, the UPF and power constraints files are used during synthesis.
UPFFlowType	UPF Flow Type	Use the default UPF flow - EnumValues: UPFPrime - Symbolic Names: UPF
UPFFile	UPF File	The name of the Unified Power Format (UPF) file to be used with the Fusion Compiler.
PowerConstraintsFile	Power Constraints File	The power constraints file is sourced in Fusion Compiler after loading the UPF file. This file can be used to issue 'set_voltage' commands and remove such elements on library cells, as needed.

See Also

[Specify Reference Technology.](#)
[Technology Cell Binding.](#)
[Initialize Subsystem Constraints.](#)
[Specify Clocks.](#)
[Specify Port Constraints.](#)
[Specify Power Intent.](#)
[Specify Synthesis Methodology.](#)

ExportDefineToFile

Indicates a ‘define whose value should be written to the indicated file as part of the configuration process.

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

This attribute is set on a parameter to cause it’s value to be written to the indicated external file (assuming it is defined). The value of the attribute indicates the file where the ‘define should be written. The file name can take on one of three different formats.

- Simple Name a simple file name like local.v, which is written to the component source directory (src).
- Simple name in directory defined by environment variable this is specified as \$ENV_VAR/localname. The environment variable must be defined when the configuration activity is completed.
- File name defined by environment variable this is specified as \$ENV_VAR. The environment variable must be defined when the configuration activity is completed.

Each parameter with this attribute set is written along with it’s Description attribute if specified. The value is written as one of the following depending on the original definition and current value:

```
'define PARAM1
```

```
'define PARAM2 14
```

```
// 'define PARAM3
```

Prior to each `define` statement, the parameter will be undefined using `undef` to prevent redefinition warnings in downstream tools.

There are certain cases where making the `define` visible as part of the design definition can cause parsing problems. To address this, the `define` can be given a different name (than any used in the RTL), and then the `RTLName` attribute can be set via annotation. When the ‘define is written to the file, the `RTLName` value will be used as the name of the macro.

Examples

```
// reuse-pragma attr Description Always exported to local.v
// reuse-pragma ExportDefineToFile local.v
`define checker 4
// reuse-pragma attr Description Written to $ABC but may be commented out
// reuse-pragma ExportDefineToFile $ABC
`define might_be_there
// reuse-pragma attr Description This one gets renamed when written to
file
// reuse-pragma attr ExportDefineToFile local.v
// reuse-pragma attr RTLName AgoodName
`define AbadName
```

See Also

[API Index](#) [Command Reference](#)

ExternalTristates

Determines the disabling option during scan shift for all tristate nets that drive output ports of a design. `disable_all` disables all drivers. `enable_one` disables all but one driver. `no_disabling` specifies not to insert disabling logic.

Definition

Type: string

Flags:

Default value: =InheritValue up `disable_all`

Valid on: `design`

Description

Determines the disabling option during scan shift for all tristate nets that drive output ports of a design. `disable_all` disables all drivers. `enable_one` disables all but one driver. `no_disabling` specifies not to insert disabling logic.

Examples

Specify that all tristate outputs are to be disabled during scan shift.

```
coreBuilder> set_design_attribute ExternalTristates disable_all
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#), [BidirectionalMode \(3\)](#), [InternalTristates \(3\)](#)

[API Index](#) [Command Reference](#)

ExtraSearchPathAppend

List of additional appended search_path entries for Design Compiler.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is provides a mechanism for users to specify additional entries for the Design Compiler search_path variable. The value of this attribute is appended to the default search_path.

Examples

```
set_design_attribute ExtraSearchPathAppend {./otherDirectory  
/my/other/location}
```

See Also

[ExtraSearchPathPrepend \(3\)](#)

[API Index](#) [Command Reference](#)

ExtraSearchPathPrepend

List of additional prepended search_path entries for Design Compiler.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is provides a mechanism for users to specify additional entries for the Design Compiler search_path variable. The value of this attribute is prepended to the default search_path.

Examples

```
set_design_attribute ExtraSearchPathPrepend {./otherDirectory  
/my/other/location}
```

See Also

[ExtraSearchPathAppend \(3\)](#)

FCExcludeLibCells

The attribute specifies the purposes for the library cells. The purpose is passed into the `set_lib_cell_purpose` command.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: all cts hold optimization power

Default subscript: all

Valid on: design_____

Description

The attribute specifies valid purposes for library cells. The subscripts indicates the purpose for the library cells. If the attribute is set a 'dontUseLibCells.tcl' file is exported in the workspace/syn folder which contain 'set_lib_cell_purpose -exclude' command. The file is sourced in the Fusion Compiler intent.

- cts - For clock tree synthesis
- optimization - For delay and electrical design rule optimization
- hold - For hold fixing
- power - For power optimization
- all - For no restrictions on inserting the specified cells

Examples

The command 'set_lib_cell_purpose -exclude optimization [get_lib_cells myLib/OR2]' will be issued to exclude myLib/OR2 cell from the synthesis run.

The following command will be generated in 'dontUseLibCells.tcl'

```
coreConsultant> set_attribute [current_design -quiet] -attr  
FCExcludeLibCells -subscript optimization -value {myLib/OR2}
```

See Also

[FCIncludeLibCells \(3\)](#) [ExcludeLibCells \(3\)](#) [IncludeLibCells \(3\)](#)_____

FCIncludeLibCells

The attribute specifies the purposes for the library cells. The purpose is passed into the `set_lib_cell_purpose` command.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: all cts hold none optimization power

Default subscript: all

Valid on: design_____

Description

The attribute specifies valid purposes for library cells. The subscripts indicates the purpose for the library cells. If the attribute is set a 'dontUseLibCells.tcl' file is exported in the workspace/syn folder which contain 'set_lib_cell_purpose -include' command. The file is sourced in the Fusion Compiler intent.

- cts - For clock tree synthesis
- optimization - For delay and electrical design rule optimization
- hold - For hold fixing
- power - For power optimization
- all - For no restrictions on inserting the specified cells
- none - To prevent insertion of the specified cells for any purpose

Examples

The command 'set_lib_cell_purpose -include all [get_lib_cells myLib/OR2]' will be issued to include myLib/OR2 cell in the synthesis run.

The following command will be generated in 'dontUseLibCells.tcl'

```
coreConsultant> set_attribute [current_design -quiet] -attr  
FCIncludeLibCells -subscript all -value {myLib/OR2}
```

See Also

[FCExcludeLibCells \(3\)](#) [ExcludeLibCells \(3\)](#) [IncludeLibCells \(3\)](#)

FM_formal_verification_strategy
Run formal verification on a design.

Parameters

Strategy has no parameters.

See Also

Perform FPGA Synthesis

Generate and (optionally) run FPGA synthesis scripts.

This activity is used to generate FPGA synthesis scripts for SynplifyPro or SynplifyPremier, customized for the particular core configuration, Synplify version and FPGA device. By default, the Synplify run is started immediately after the scripts are generated, but this behavior can be changed by checking the 'Generate Scripts Only' check-box.

Once the synthesis is finished, results will be available in the following locations:

EDIF netlist - syn/final/db/<top_design>.edif HDL netlist - syn/final/db/<top_design>.v[hd] Reports - syn/final/report

If you choose to run the synthesis yourself, or wish to review the generated scripts or constraints, these can be found in the following locations;

Launch script - syn/run.scr Synplify script - syn/synplify.tcl
Constraints - syn/constrain/script/*.cscr

Parameters

Parameters Table

Name	Label	Description
Execution		
UseSynplifyElite	Use Synplify Elite	Use synplify_elite or if using synplicity/2021.09-SP2 or older then use synplify_premier to perform FPGA synthesis. ActivityAliasNames : UseSynplifyPremier
UseSynplifyBase	Use Synplify Base	Use synplify_base or if using synplicity/2021.09-SP2 or older then use synplify_pro to perform FPGA synthesis. ActivityAliasNames : UseSynplifyPro
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script

coreTools Command Reference Index

Name	Label	Description
		does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieving using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
ParallelJobCpuLimit	Parallel job CPU limit:	Specifies the number of parallel compile jobs that may be run.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.
Netlist		
ProjectResultFormat	Project Result Format	Save an EDIF netlist when the design is synthesized. EnumValues: edif
WriteVerilogNetlist	Write Simulation Verilog Netlist	Save a Verilog netlist when the design is synthesized.
WriteVhdlNetlist	Write Simulation Vhdl Netlist	Save a VHDL netlist when the design is synthesized.

coreTools Command Reference Index

Name	Label	Description
OptimizationAndMapping		
EnableDesignware	Compile with Synplicity DesignWare Library	Include the -enable_designware option in the generated script.
EnableSynopsysDesignware	Compile with Synopsys DWBB (Design Compiler) DesignWare Library	Compile using Synopsys DesignWare library. This feature requires Synplify Premier version D-2010.03 or newer. You must also provide a valid root directory for Design Compiler in the Tool Roots GUI or using the 'set_tool_root' command. DC C-2009.06-SP4 or newer is recommended. Older versions of DC require DWBB_0912 to be overlaid in the DC installation tree. Selecting this parameter will include the -dw_foundation and -dc_root options in the generated script
EnableRetiming	Enable Retiming	Enable retiming optimizations. Includes the -retiming option in the generated script.
EnablePipelining	Enable Pipelining	Enable pipeline optimizations. Includes the -pipe option in the generated script.
EnableResourceSharing	Enable Resource Sharing	Enable resource sharing optimizations. Includes the -resource_sharing option in the generated script.
UserOptions	User Options	List of additional options to be set in the generated script. Value should contain an even number of elements where each odd element is a legal option for the set_option command and the even elements are the values for the corresponding option. For example: -frequency auto -fanout_limit 10 would cause set_option -frequency auto set_option -fanout_limit 10 to appear in the generated script.
GlobalAttributes	Global Attributes	List of global attribute names and values to be included in the generated script. The value

coreTools Command Reference Index

Name	Label	Description
		should contain an even number of elements where each odd element is a legal global attribute name and each even element is the value for the preceeding global attribute. For example, a value of: <pre>syn_useioff 1 syn_noclockbuf 1</pre> would cause <pre>define_global_attribute syn_useioff 1 define_global_attribute syn_noclockbuf 1</pre> to appear in the generated script.
PandR		
AddPRImplementation	Add Place-and-Route implementation	Create a Place-and-Route implementation using Synplify and Vendor place-and-route tools. Currently, Xilinx/ISE and Altera/Quartus flows are supported.
RunBackannotation	Run Backannotation following synthesis	For Xilinx-ISE with Synplify-Premier only: Create a 'coreloc.sdc' file containing placement information for all core or anchor instances in your design.
ParUseXflow	Run Xilinx-ISE using xflow	For Xilinx-ISE only: use 'xflow' to run ISE executable rather than xtclsh.
PlaceAndRouteOptionsFile	Place and Route Options file	Optional file containing place-and-route options for vendor tools.
ProjectSettings		
ImplementationName	Implementation Name	Name to use for the implementation.

See Also

[Specify Clocks.](#)
[Specify Port Constraints.](#)

FPGA_optimization_strategy

Performs synthesis from RTL targeting an FPGA using synplify.

Parameters

Parameters Table

Name	Label	Description
Options/Execution		
UseSynplifyPro	Use Synplify Pro	Use synplify_pro to run the tool.
UseSynplifyPremier	Use Synplify Premier	Use synplify_premier to run the tool.
Options/Netlist		
ProjectResultFormat	Project Result Format	Save an EDIF netlist when the design is synthesized. EnumValues: edif
WriteVerilogNetlist	Write Verilog Netlist	Save a Verilog netlist when the design is synthesized.
WriteVhdlNetlist	Write Vhdl Netlist	Save a VHDL netlist when the design is synthesized.
Options/Optimization and Mapping		
EnableDesignware	Compile with Synplicity DesignWare Library	Include the -enable_designware option in the generated script.
EnableSynopsysDesignware	Compile with Synopsys DWBB (Design Compiler) DesignWare Library	Compile using Synopsys DesignWare library. This feature requires Synplify Premier version D-2010.03 or newer. You must also provide a valid root directory for Design Compiler in the Tool Roots GUI or using the 'set_tool_root' command. DC C-2009.06-SP4 or newer is recommended. Older versions of DC require DWBB_0912 to be overlaid in the DC installation tree. Selecting this parameter will include the -dw_foundation and -dc_root options in the generated script
EnableRetiming	Enable Retiming	Enable retiming optimizations. Includes the -retiming option in the generated script.

coreTools Command Reference Index

Name	Label	Description
EnablePipelining	Enable Pipelining	Enable pipeline optimizations. Includes the -pipe option in the generated script.
EnableResourceSharing	Enable Resource Sharing	Enable resource sharing optimizations. Includes the -resource_sharing option in the generated script.
UserOptions	User Options	List of additional options to be set in the generated script. Value should contain an even number of elements where each odd element is a legal option for the set_option command and the even elements are the values for the corresponding option. For example: -frequency auto -fanout_limit 10 would cause set_option -frequency auto set_option -fanout_limit 10 to appear in the generated script.
GlobalAttributes	Global Attributes	List of global attribute names and values to be included in the generated script. The value should contain an even number of elements where each odd element is a legal global attribute name and each even element is the value for the preceding global attribute. For example, a value of: syn_useioff 1 syn_noclockbuf 1 would cause define_global_attribute syn_useioff 1 define_global_attribute syn_noclockbuf 1 to appear in the generated script.

See Also

FcAuxSynthesisScript

Specifies a FC script to be executed prior to specific optimization phase.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: AddToTmmDFTSetup AddToDFTSetup DFTSetup cleanup

post_compile post_init_design post_place_opt post_read_rtl

pre_compile pre_compile_unified pre_insert_dft pre_place_opt

pre_read_rtl report write_data

Default subscript:

Valid on: design_____

Description

The FcAuxSynthesisScript attribute specifies the name of a Fusion Compiler (FC) script that you want to execute before or after a particular phase of synthesis for the selected design. As an example, you may want to perform special structuring or grouping on a design before a certain synthesis phase to improve timing or area.

The subscript indicates the stage in the FCRM flow where you want to execute the auxiliary synthesis script:

- pre_read_rtl - Execute script before reading RTL.
- post_read_rtl - Execute script after reading RTL.
- post_init_design - Execute script at the very end of init_design stage before save_block
- pre_compile - Execute script before compile.
- DFTSetup - Sourced in DFT.setup.tcl, this DFT setup script overrides the DFT constraints packaged with the IP.
- AddToDFTSetup - Sourced in DFT.setup.tcl, this script adds the DFT constraints on top of the DFT constraints packaged with the IP.
- AddToTmmDFTSetup - Sourced in fc_tmm_dft_task.tcl, this scripts adds the DFT constraints on top of the DFT constraints generated by the TMM tool.
- pre_insert_dft - Execute script before the call to insert_dft.
- post_compile - Execute script after compile.
- pre_compile_unified - Execute script in the UNIFIED_FLOW before compile_fusion -from final_place
- pre_place_opt - Execute script before place_opt.
- post_place_opt - Execute script after place_opt.
- write_data - Execute script before data writeout.
- report - Execute script at the end of data writeout, can be used to include report generation commands.

coreTools Command Reference Index

- cleanup - Execute script after all processing is complete, just before quitting.

When using the FcAuxSynthesisScript , some targets may have corresponding Fusion Compiler insertion points defined in the Reference Methodology.

The compiled table captures the mapping between the tool provided insertion points and the corresponding Fusion Compiler control variable.

If the user need to use a Fusion Compiler insertion point that is not explicitly mapped by FcAuxSynthesisScript, you can manually set the corresponding FCRM control variable using the 'set_rm_variable' command.

FcAuxSynthesisScript to FCRM insertion point mapping

FcAuxSynthesisScript Subscript	FCRM variable Name
pre_read_rtl	TCL_USER_READ_RTL_PRE_SCRIPT
post_read_rtl	TCL_USER_READ_RTL_POST_SCRIPT
post_init_design	TCL_USER_INIT_DESIGN_POST_SCRIPT
pre_compile	TCL_USER_COMPILE_PRE_SCRIPT
post_compile	TCL_USER_COMPILE_POST_SCRIPT
pre_place_opt	TCL_USER_PLACE_OPT_PRE_SCRIPT
post_place_opt	TCL_USER_PLACE_OPT_POST_SCRIPT

Examples

See Also

set_rm_variable (2)

[API Index](#) [Command Reference](#)

FcAuxSynthesisScriptComment

Specifies a comment to be issued for the corresponding

FcAuxSynthesisScript.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: AddToTmmDFTSetup AddToDFTSetup DFTSetup cleanup

post_compile post_init_design post_place_opt post_read_rtl

pre_compile pre_compile_unified pre_insert_dft pre_place_opt

pre_read_rtl report write_data

Default subscript:

Valid on: design_____

Description

The FcAuxSynthesisScriptComment attribute can be used to output a comment to document why the corresponding FcAuxSynthesisScriptComment is being used. The comment will be put into the file where the script is sourced.

Examples

See Also

[API Index](#) [Command Reference](#)

FeedThroughConnect

This input port feeds-through to the output ports listed. Used in automatic connections to prevent connections to these pins. Connections will bypass the component and connect items that would be connected to both pins directly.

Definition

Type: string

Flags:

Default value:

Valid on: port _____

Description

Set this attribute on an input port if that port feeds through the design completely, and you don't want coreAssembler to make connections to this port, but to connect directory the items on both sides of the feedthrough.

The ports listed in the value of this attribute, must have a width specified exactly the same as this input port. The output ports listed may be optional (i.e. have GenerateIf set on them).

Examples

```
coreBuilder> set_port_attribute data_m1 FeedThroughConnect data_s1
```

See Also

[GenerateIf \(3\)](#)

FileContentType

Content type of a file or file group

Definition

Type: string

Flags:

Default value:

Valid on: file filegroup filegroupGroup

Description

The FileContentType attribute indicates the type of content of a file or file group. Current content types are:

- unknown - Not known
- ascii - ASCII file
- vhdl - VHDL source file
- verilog - Verilog source file
- dc_shell - Design Compiler shell script
- tcl - Tcl script
- perl - Perl script
- csh - C-shell script
- html - HTML file
- frame - FrameMaker document file
- kb - Knowledge database file
- sdc - SDC file
- systemVerilog - System Verilog file
- xml - XML file (typically IP-XACT)
- directory - directory (for empty directories)
- python - Python script
- binary - binary file
- upf - UPF file
- pdf - PDF file
- make - Make script
- csource - C Source file
- cheader - C Header file

If not set the default file content type is "unknown". However for the following file extensions the file type is automatically inferred: .v, .vh, .sv, .svh, .pl, .pm, .py, .upf, .pdf, .mk, .c, .h

Examples

To indicate that myfile.txt is an ASCII text file:

```
coreBuilder> set_attribute -attr FileContentType -value ascii myfile.txt
```


See Also

[FilePerms \(3\)](#)

[API Index](#) [Command Reference](#)

FilePerms

Read/write/execute permissions for a file or file group

Definition

Type: short**Flags:****Default value:****Valid on:** file filegroup filegroupGroup

Description

The FilePerms attribute specifies the read/write/execute permissions for a file or file group. The format of the FilePerms value is the same as UNIX chmod "nnn" format.

For a file that is included in the Bill of Materials (BoM) for a coreKit, the default value of FilePerms is the permission set on the file when the file when the file was loaded into coreBuilder.

Examples

To set the file permissions on myfile.txt so that owner has read/write/execute permission, all others have read-only permission:

```
coreBuilder> set_attribute -attr FilePerms -value 744 myfile.txt
```

See Also

[FileContentType \(3\)](#)

[API Index](#) [Command Reference](#)

FixHold

Fix hold time violations for this clock.

Definition

Type: boolean**Flags:****Default value:** 0**Valid on:** clock_____

Description

FixHold specifies whether to fix hold time violations for the selected clock. If FixHold is true, coreConsultant generates a set_fix_hold command for the clock. The set_fix_hold command causes Design Compiler to insert a delay, if necessary, to fix hold time violations at registers that are driven by the selected clock.

Examples

To not fix hold time violations for the clock named pclk:

```
coreConsultant> set_clock_attribute pclk FixHold false
```

See Also

[set_clock_attribute \(2\)](#)_____

ForceTCBToUseComponentUndefFile

If `GenericAttr[ComponentUndefFile]` is set on the design, use that while writing out technology cell macro undef directives. The file option is used otherwise.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

When a component undef file is used, tech cell may or may not be written into it based on version of BCM libraries used (tech cell undefs for BCM tech cell definitions using macro args are not written into undef file). This attribute forces the TCB flow to write undefs into the file name specified in the `GenericAttr[ComponentUndefFile]` design attribute.

Examples

```
# example coreBuilder script
create_workspace -name ws_cb_compK -replace
set_activity_parameter LoadDesigns InputLanguage SystemVerilog
set_activity_parameter LoadDesigns InputFileNames
{SOURCE/DWbb_bcm21_and.v SOURCE/DWbb_bcm21_or.v SOURCE/DWbb_bcm21.v
SOURCE/DWbb_bcm21_inst.v SOURCE/compK-undef.svh SOURCE/compK.v}
set_activity_parameter LoadDesigns TopDesignName compK
autocomplete_activity LoadDesigns
set_design_attribute ForceTCBToUseComponentUndefFile 1
set_attribute -attr GenericAttr -subs ComponentUndefFile [get_top_design]
-value "compK-undef.svh"
source -echo -verbose SOURCE/bcm_defines.tcl
source -echo -verbose SOURCE/DWbb_bcm21_tech_cell_mappings.tcl
# example
define_tech_cell_mapping \
-module DWbb_bcm21 \
-macro DWC_SYNC_1_STAGE_SRC \
-description "1-stage Synchronizer Cell" \
-instance U_sync_1 \
-width {[WIDTH-1:0]} \
-enable_macro_args \
-extra_macro_args {WIDTH} \
-port_names "clk_d rst_d_n data_s data_d" \
-port_descriptions { "Destination Clock" "Destination Asynchronous Reset"
"Source Domain Data Vector" "Destination Domain Data Vector" } \
```

-file DWbb_bcm21.v

autocomplete_activity BuildcoreKit

On completion of the TechnologyCellBinding activity in coreConsultant or coreAssembler, the undefine directives will be written into a single file compK-undef.svh.

See Also

[InsertTechCellBindMacroAtStart \(3\)](#) [TCBDisableUndefMacros \(3\)](#)

[TechCellMappingDefn \(3\)](#) [TechCellMappingBind \(3\)](#)

Formal Verification

Generate and (optionally) run formal verification scripts.

Parameters

Parameters Table

Name	Label	Description
Activity Controls		
ReferenceStageName	Reference stage name:	Name of the stage that contains the 'golden' reference design. EnumValues: RTL
UseReferenceStageNetlist	Use reference stage netlist	Use the netlist from the reference stage instead of the db.
ReferenceStageNetlist	Reference stage netlist:	The path to the netlist from the reference stage.
TargetStageName	Target stage name:	Name of the stage that contains the design to be verified. EnumValues: {} RTL2RTL
UseTargetStageNetlist	Use target stage netlist	Use the netlist from the reference stage instead of the db.
UseUPFPGNetlist	Use UPF PG Netlist	Use netlist with PG (power/ground) for Golden UPF flow. When set to false, PG info is in UPF files.
TargetStageNetlist	Target stage netlist:	Name of target stage netlist ([design]_pg netlist has power/ground info for UPF Golden Flow).
UPFAuxiliaryScriptName	UPF auxiliary script name:	Defines the name of a script to be sourced when using a UPF flow. This script is sourced after the implementation design is loaded but before issuing the 'set_top' command. It can be used for specification of retention register models that match in both the reference and implementation designs.
AuxiliaryScriptName		

coreTools Command Reference Index

Name	Label	Description
	Auxiliary script name:	Defines the name of an auxiliary script to be sourced prior to invoking the actual verification commands and after the design and technology libraries have been loaded.
UseGeneratedSvf	Use Generated SVF	Controls whether the svf generated during synthesis is used in Formality.
ForceUPFSuppliesOn	Force UPF Supplies On	Forces UPF supplies on to be propagated into the FM script via the verification_force_upf_supplies_on variable.
Alternate Strategies		
ValidAlternateStrategies	Valid alternate strategies names	List of valid alternate strategy names.
AlternateStrategies	Alternate strategies list	Run the Formality command run_alternate_strategies using the specified strategies. Use whitespace to separate each strategy in the list. E.g. set_activity_parameter FormalVerification AlternateStrategies 's1 o1' For a list of allowed strategies, refer to the Valid Alternate Strategies field above. To avoid running the run_alternate_strategies command, leave this field blank.
RunAllValidAlternateStrategies	Run all valid alternate strategies	Run all valid alternate strategies defined in the version of Formality being used. This parameter is not related to the -run_all switch used in the run_alternate_strategies command.
ParallelAlternateStrategies	Parallel Alternate Strategies	Number of alternate strategies run in parallel.
AlternateStrategiesExtraArgs	Alternate Strategies Extra Arguments	Extra args appended to the run_alternate_strategies Formality command.
AlternateStrategiesRunStyle	Alternate Strategies	Defines the protocol for launching worker processes on remote. The

coreTools Command Reference Index

Name	Label	Description
	Run Style	protocol corresponds to the type of compute farm manager you are submitting to. - EnumValues: lsf uge grd - Symbolic Names: lsf uge grd
AlternateStrategiesRunOptions	Alternate Strategies Run Options	LSF or Grid options for launching distributed jobs. This attribute value is passed to set_host_options command with the -submit_command argument.
DPX Flow		
EnableDPXFlow	Enable DPX Flow	DPX allows more partitions to be verified in parallel compared to the standard Formality equivalence checking solution. The DPX Manager (the main Formality process you interact with) saves a session file that contains the entire design space and verification context. Each worker process reads the session file and accepts tasks from the manager that consists of instructions to verify a partition using a specific solver strategy. Workers can accept one task per core and can run N number of tasks in parallel. When a task completes a specific effort level, the status is returned to the DPX Manager. The feature can be enabled when the FormalVerification RunStyle activity parameter is set to 'uge' && license check for 'Formality-DPX' is successful. Additional information on the DPX flow can be found in the Formality User Guide.
DPXRunStyle	DPX Run Style	Defines the protocol for launching worker processes on remote machines. The protocol corresponds to the type of compute farm manager used by Formality to submit its jobs. - EnumValues: uge lsf grd - Symbolic Names: uge lsf grd
DPXRunOptions		

coreTools Command Reference Index

Name	Label	Description
	DPX Run Options	LSF or Grid options for launching distributed jobs. This attribute value is passed to set_dpx_options command with the -submit_command argument.
MaxWorkers	Specifies the number of machines	Specifies the maximum number of machines requested for processing distributed tasks. The default is 4 workers
MaxCores	Number of parallel tasks per machine	Specifies the maximum number of compute cores per machine assumed to be available for processing distributed tasks. The default is 4 core per worker.
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieved using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
ParallelJobCpuLimit	Parallel job CPU limit:	Specifies the number of parallel compile jobs that may be run.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes

coreTools Command Reference Index

Name	Label	Description
		or is killed. The default value is the current user's name.
RTL2RTL Flow		
TargetWorkspace	Target workspace	Target workspace for RTL to RTL verification.
VerificationDesign	Verification design	Set a design name to be verified in RTL2RTL verification. The verification will run only for this design.
EnableBlackBoxing	Enable black boxing	When enabled, blackboxes the designs in the design attribute 'BlackBoxDesigns' for RTL2RTL verification.

See Also

Perform ASIC Synthesis.

[API Index](#) [Command Reference](#)

FormalVerificationAuxScript

Specifies a script that is to be run in Formality after the reference and implementation designs have been loaded but just before the verify command.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute can be used by the IP provider to specify an auxiliary script that will be packaged with the core to be run in Formality. The script will be run after loading both the reference and implementation designs and just before running the 'verify' command.

To specify commands to be run prior to loading the designs use the design attribute `fm_shellVariable`.

Examples

Specify that the script `formalAux.tcl` is to be packaged with the core and run in Formality.

```
coreBuilder> current_design [get_top_design]
coreBuilder> set_design_attribute FormalVerificationAuxScript
/full/path/to/formalAux.tcl
```

See Also

[set design_attribute \(2\), fm_shellVariable \(2\)](#)

[API Index](#) [Command Reference](#)

FormalVerificationAuxScriptComment
Specifies a comment to be issued for the corresponding
FormalVerificationAuxScript.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

The FormalVerificationAuxScriptComment attribute can be used to output a comment to document why the corresponding FormalVerificationAuxScriptComment is being used. The comment will be put into the file where the script is sourced.

Examples

Specify an aux script to be used, and document why it is being used.

```
set_design_attribute FormalVerificationAuxScript myAuxScr.tcl  
set_design_attribute FormalVerificationAuxScriptComment "This script is  
used to set up custom variables for formal verification."
```

See Also

[API Index](#) [Command Reference](#)

FpgaPadType

Arguments to be passed to the set_fpga_pad_type command for FPGA synthesis.

Definition

Type: string

Flags:

Default value:

Valid on: port _____

Description

The command set_fpga_pad_type is used by the pad mapping algorithms in FPGA synthesis to specify what type of pad should be mapped to a port. The value of this attribute will be passed to the set_fpga_pad_type command in fpga_shell.

Examples

The following command sets the pad type to OBUF, the IOSTANDARD to pci, the slew to S, and the drive to 12 on port outPort:

```
set_port_attribute outPut FpgaPadType -pad OBUF \  
-iostandard PCI \  
-slew S \  
-drive 12
```

See Also

[FpgaPortIsPad \(3\)](#)

[API Index](#) [Command Reference](#)

FpgaPortIsPad

Specifies that this port is a primary I/O for FPGA synthesis.

Definition

Type: boolean

Flags:

Default value:

Valid on: port _____

Description

Specifies that this port is a primary I/O for FPGA synthesis. In `fpga_shell` the command `set_port_is_pad` will be issued for the specified port. This attribute is only used for FPGA synthesis.

Examples

To set the port is pad to true on inPort:

```
set_port_attribute inPort FpgaPortIsPad 1
```

See Also

[FpgaPadType \(3\)](#)

[API Index](#) [Command Reference](#)

FpgaPreferTmg

Specifies which synthetic library FPGA synthesis will use for synthetic operators. See the man page for `fpga_prefer_tmg` in `fpga_shell`.

Definition

Type: string

Flags:

Default value: =InheritValue up fpgadw

Valid on: design_____

Description

Specifies which synthetic library FPGA synthesis will use for synthetic operators. The valid values are 'dwfpga' and 'tmg'. If it is set to tmg, `fpga_shell` will use tmg implementations for synthetic operators that are implemented in both tmg and DesignWare. When it is set to dwfpga, tmg implementations will not be used.

Examples

To set the FpgaPreferTmg attribute to tmg on the current_design:

```
set_design_attribute FpgaPreferTmg tmg
```

See Also

[set_design_attribute \(2\)](#)_____

FunctionDefinition

The Tcl code to implement this function

Definition

Type: string

Flags:

Default value:

Valid on: hdlFunc_____

Description

The FunctionDefinition attribute specifies the Tcl code needed to implement an existing VHDL function. You need to specify FunctionDefinition for any VHDL function that is required for design configuration.

For example, your VHDL design may use a VHDL function to calculate the value of a design parameter. In such a case, you must write Tcl code that implements the VHDL function and specify that code as the value of FunctionDefinition on the VHDL function.

The mechanism to set FunctionDefinition on a VHDL function is to insert a coreBuilder reuse-pragma with the FunctionDefinition attribute definition immediately before the VHDL function in the VHDL file. When coreBuilder parses the VHDL, it applies the FunctionDefinition attribute to the VHDL function. When a core integrator executes the Specify Configuration activity, coreConsultant evaluates the FunctionDefinition Tcl code to perform the calculation implemented in the VHDL function.

If your VHDL function has arguments, you can reference the arguments by name in your Tcl code for the function and assume that the arguments will exist in the scope of the Tcl proc that coreBuilder creates to implement the function. You do not need to declare the arguments explicitly in the Tcl code for the function.

Examples

The following example illustrates how to use FunctionDefinition to specify Tcl code that implements the VHDL function `addr_width_c`. In this example, the VHDL function does not have arguments. The code shown in bold is the coreBuilder reuse-pragma that you must insert to set FunctionDefinition on the function:

```
-- reuse-pragma beginAttr FunctionDefinition
-- expr { ($ram_low_c < 1 || !$ram_interface_c ) ? 1 : $ram_low_c }
-- reuse-pragma endAttr\fp
```



```
function addr_width_c return integer is
begin
if ram_low_c < 1 or ram_interface_c = false
then return 1;
else return ram_low_c;
end if;
end addr_width_c;
```

The following example sets FunctionDefinition on a VHDL function that has arguments:

```
package body timer_pkg is
-- reuse-pragma beginAttr FunctionDefinition
-- expr {($icpt==0 && $ocmp==2) ? 1 : $icpt}
-- reuse-pragma endAttr\fp
function sel_icrl(icpt, ocmp : integer)
return integer is
variable value : integer;
begin
if (icpt = 0 and ocmp = 2) then
value := 1;
else
value := icpt;
end if;
return value;
end sel_icrl;
end timer_pkg;
```

See Also

Fusion_Compiler_Reference_Methodology

Performs a one pass, top down compile from RTL using compile_fusion. This flow implements the Fusion Compiler Reference Methodology associated with the version of Fusion Compiler being used.

Parameters

Parameters Table

Name	Label	
Automotive		
AutomotiveSynthesisFlowType	AutomotiveSynthesis Flow Type	Supported automotive synthesis flows EnumValues: disabled TMR D
TMRSeparationDistance	TMR Separation Distance	Specifies the distance values for repelling for redundancy group. It could be either y} pair or a single number denoting rad
DCLSDistance	DCLS Distance	Used to specify the DCLS distance as a height}. This creates a group bound with dimensions.
Basic/FloorPlan		
FloorplanFileDataFormat	Floorplan Data Format	Format of file containing floorplan con - EnumValues: def tcl - Symbolic Names: DEF TCL
EnableDesignPlanFlatFlow	Enable Design Planning Flat Flow	Enables the flat Design Planning FLOW design planning flow is the generation floorplan. The primary inputs to the floor constraints, technology node setup, and flow will export a completed design floor floorplan is a primary input into the P& exported floorplan will be tcl based floor feature is available with Fusion Compiler 2022.03-SP1.
PhysicalConstraintFile	Physical Constraint File	Path to the TCL floorplan file populated FCRM setup parameter. Path to the DEF 'DEF_FLOORPLAN_FILES' FCRM s
EnablePortSideConstraint	Enable Port Side Constraint	Generates a port side file which is inserted flow for better pin placement. This file when there is no floorplan file given an attribute is set on the design port by the
UtilizationRatio	Utilization Ratio	Specify utilisation ratio as part of synthesis default value is 0.7 and ranges from 0.0 utilization ratio is applicable only when provided.

coreTools Command Reference Index

Name	Label	
Basic/Flow		
EnableQIK	Enable QIK for DesignWare Interface IP	Enables the Quick Implementation Kit Interface IP. This flow is a version of the Reference Methodology which has been modified for DesignWare Interface IP. Requires Fusion Compiler later to be enabled. ActivityAliasNames : EnableQIK
FinalFCRMFlowStep	Final FCRM Flow Step	Indicates how far the FCRM flow should go. This will be the last one run in the FCRM flow. - EnumValues: place_opt compile - Symbolic Names: {place_opt (compile (unified flow to final_opto + compile (unified flow to final_opto + compile
Basic/QoR		
QorStrategy	Qor Strategy	The selected strategy is passed into set_qor [strategy]. Appropriate application options will optimize the specified target strategy - EnumValues: timing leakage_opt - Symbolic Names: Timing {Leakage
EnableReducedEffort	Enable Reduced Effort	Setting it to true to enable the -reduced_effort set_qor_strategy command
Basic/SDC Generation		
WriteSDCArgs	Options for write_sdc	Defines arguments to be passed to the write_sdc writing SDC constraints for synthesis.
Basic/Static Aware Flow		
EnableStaticAwareFlow	Enable Static Aware Flow	The VC SpyGlass Static Aware DC/FCM database that must be read in synthesis. If netlist compilation/optimization does not work, issues. The activation of the feature is limited to coreConsultant for DesignWare IP's. To use Spyglass Engineering Council must be enabled higher, and the runVcSpyglass activity must be enabled.
StaticAwareDBPath	Static Aware synthesis database	Path to Static Aware synthesis DB.
Basic/TLUPlus		
MinTluPlusName	Early parasitic tech specification name	Specifies the early parasitic tech specification name
MinTluPlusFile	Min TLUPlus file name	TLU+ files to read for Min specification
MinTluPlusLayerMapFile	Min TLUPlus Layer map file	Layer mapping file between ITF and MCM
MaxTluPlusName		Specifies the late parasitic tech specification name

coreTools Command Reference Index

Name	Label	
	Late parasitic tech specification name	
MaxTluPlusFile	Max TLUPlus file name	TLU+ files to read for Max specification
MaxTluPlusLayerMapFile	MAX TLUPlus Layer map file	Layer mapping file between ITF and M
Basic/TechFile		
UseTechFile	Use Technology File	Use technology file to specify technology enabled the target library (TECH_LIB)
TechFile	TechFile	Technology file used to create the Milk file must match the format specified in
EnableLibraryConfigurationFlow	Enable Library Configuration Flow	Setting it to true enables library configuration calls the library manager under the hood
Basic/Technology Node		
TechnologyNode	Technology Node	Specifies the technology node to be used in command in Fusion Compiler. This helps Fusion Compiler for technology node selection. coreTools will not validate the TechnologyNode validation will be performed by Fusion Compiler. It is disabled when the QIK flow is enabled and scripts are being used.
TechnologyRMScripts	Technology Reference Methodology Scripts	Specifies the path to the RM technology node (rm_tech_scripts) delivered by Synopsys. It is recommended for the user to specify the path to rm_tech_scripts.
SetTechnologyAfterFloorplan	Set technology after floorplan	Setting of technology node to be performed after floorplan. The feature is available if Fusion Compiler version is 2022.03-SP1 or greater when TechnologyNode is set to the either PhysicalConstraintFile is specified or Flat Flow (EnableDesignPlanFlatFlow) is enabled.
Basic/Technology Side File		
TechnologySideFileSettings	Technology Sidefile Settings	Set the Reference Methodology control variables. It accepts a list of control variables and values. The output is exported in the file 'syn/rm_snps_ip_scripts/snps_ip_techrm_sidefile'. The parameter is enabled when the QIK flow is enabled (EnableQIK is set to true).
Cell Pruning		
DontUseLibCells	Don't Use Library Cells	Defines library cells that should not be used during floorplan flow. Typically consists of very small area cells, strength cells, and non-typical use cells. It updates the FCRM control variable DC
ValidOptBuffers	Valid Optimization Buffers	Defines buffer cells to be used during optimization flow except for 'cts'. Typically includes low capacitance buffers across all voltage threshold classes. Cells listed here will be blocked from use. The

coreTools Command Reference Index

Name	Label	
		the FCRM control variable VALID_OPT_INVERTER_LIST.
ValidOptInverters	Valid Optimization Inverters	Defines inverter cells to be used during optimization stages except for 'cts'. Typically includes high strength inverters across all voltage thresholds. Inverter cells not listed here will be blocked from use. This parameter updates the FCRM control variable VALID_OPT_INVERTER_LIST.
ValidHoldBuffers	Valid Hold Buffers	Defines buffer cells to be use for fixing hold times. Typically includes delay buffers and low strength buffers, usually of the slowest voltage threshold class available. This parameter updates the FCRM control variable VALID_HOLD_BUFFER_LIST.
ValidICGCells	Valid ICG Cells	Defines the integrated clock gating (ICG) cells. Typically includes ICG cells of a variety of strength, usually of the fastest voltage threshold class available. Cells not listed here will be blocked from use. This parameter updates the FCRM control variable VALID_ICG_CELL_LIST.
ValidCTSRepeaters	Valid CTS Repeaters	Defines valid buffer/inverter cells to be used for the 'cts' stage. Typically includes buffers and inverters of high drive strength, usually of the fastest voltage threshold class available. Library cells not listed here will be blocked from use for the 'cts' stage. This parameter updates the FCRM control variable VALID_CTS_REPEATER_LIST.
ValidSkewCells	Valid Skew Cells	Defines valid skew cells that should not be blocked from optimization. Cells in this list will be blocked from all optimization stages. This parameter updates the FCRM control variable SKEW_CELL_LIST.
ValidTieCells	Valid Tie Cells	Defines the valid tie high and low cells to be used for optimization. Typically includes tie high and tie low cells of lowest voltage threshold class available. Cells not listed here will have any existing dont_touch design attribute removed and defined for use in all optimization stages. This parameter updates the FCRM control variable VALID_TIE_CELL_LIST.
ClockISOBuffer	Clock Isolation Buffer	Defines a single clock tree repeater cell to be used for high strength used to isolate clock ports. This parameter updates the FCRM control variable CLOCK_ISO_BUFFER_LIST.
Constraints/Multi Vth Constraint		
AvailableVthGroups	Available Vth Groups	The Voltage Groups Available to the user.
LowThresholdVoltageGroups	Low Threshold Voltage Groups	A list of threshold voltage group names for low threshold voltage groups, stored as attributes present in the liberty or NDM file..
LowThresholdVoltagePercentage	Low Threshold Voltage Percentage	Specifies the allowable low vth cell percentage for the current design, based on cost type. This parameter updates the FCRM control variable LOW_VTH_PERCENTAGE. Default value is 100.
LowThresholdVoltageCost	Low Threshold Voltage Cost	Specifies how low vth percentage limit is calculated. vth percentage can be based on cell cost or cell area.

coreTools Command Reference Index

Name	Label	
		value is cell_count. • EnumValues: area cell_count
Constraints/Routing Layer		
MinRoutingLayer	Min Routing Layer Name	Specifies the minimum routing layer for congestion analysis. To ensure that RC congestion analysis stay synchronized with routing, set this option, the tool marks all routing specified layer as fully blocked for RC estimation.
MaxRoutingLayer	Max Routing Layer Name	Specifies the maximum routing layer for congestion analysis. To ensure that RC congestion analysis stay synchronized with routing, set this option, the tool marks all routing specified layer as fully blocked for RC estimation.
Corner Definition		
PrimaryCorner	Index of the Primary Corner	Sets application option cts.compile.primaryCorner to selected corner. • EnumValues: 1 2 3 4 5 6 7 8 9 19 20
Corner Definition/Corner 1		
CornerName1	Name for Corner 1	Defines a corner to be created using the cornerName1. A corner being used for set-up time analysis must have HOLD in the name. Similarly, a corner used for routing must have HOLD in the name.
CornerConstraintFile1	Corner Constraint File for Corner 1	Specifies the corner constraint file associated with Corner 1.
Corner Definition/Corner 10		
CornerName10	Name for Corner 10	Defines a corner to be created using the cornerName10. A corner being used for set-up time analysis must have HOLD in the name. Similarly, a corner used for routing must have HOLD in the name.
CornerConstraintFile10	Corner Constraint File for Corner 10	Specifies the corner constraint file associated with Corner 10.
Corner Definition/Corner 11		
CornerName11	Name for Corner 11	Defines a corner to be created using the cornerName11. A corner being used for set-up time analysis must have HOLD in the name. Similarly, a corner used for routing must have HOLD in the name.
CornerConstraintFile11	Corner Constraint File for Corner 11	Specifies the corner constraint file associated with Corner 11.
Corner Definition/Corner 12		
CornerName12	Name for Corner 12	Defines a corner to be created using the cornerName12. A corner being used for set-up time analysis must have HOLD in the name. Similarly, a corner used for routing must have HOLD in the name.

coreTools Command Reference Index

Name	Label	
		in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile12	Corner Constraint File for Corner 12	Specifies the corner constraint file asso
Corner Definition/Corner 13		
CornerName13	Name for Corner 13	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile13	Corner Constraint File for Corner 13	Specifies the corner constraint file asso
Corner Definition/Corner 14		
CornerName14	Name for Corner 14	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile14	Corner Constraint File for Corner 14	Specifies the corner constraint file asso
Corner Definition/Corner 15		
CornerName15	Name for Corner 15	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile15	Corner Constraint File for Corner 15	Specifies the corner constraint file asso
Corner Definition/Corner 16		
CornerName16	Name for Corner 16	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile16	Corner Constraint File for Corner 16	Specifies the corner constraint file asso
Corner Definition/Corner 17		
CornerName17	Name for Corner 17	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile17	Corner Constraint File for Corner 17	Specifies the corner constraint file asso
Corner Definition/Corner 18		
CornerName18	Name for Corner 18	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile18		Specifies the corner constraint file asso

coreTools Command Reference Index

Name	Label	
	Corner Constraint File for Corner 18	
Corner Definition/Corner 19		
CornerName19	Name for Corner 19	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile19	Corner Constraint File for Corner 19	Specifies the corner constraint file associated
Corner Definition/Corner 2		
CornerName2	Name for Corner 2	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile2	Corner Constraint File for Corner 2	Specifies the corner constraint file associated
Corner Definition/Corner 20		
CornerName20	Name for Corner 20	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile20	Corner Constraint File for Corner 20	Specifies the corner constraint file associated
Corner Definition/Corner 3		
CornerName3	Name for Corner 3	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile3	Corner Constraint File for Corner 3	Specifies the corner constraint file associated
Corner Definition/Corner 4		
CornerName4	Name for Corner 4	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile4	Corner Constraint File for Corner 4	Specifies the corner constraint file associated
Corner Definition/Corner 5		
CornerName5	Name for Corner 5	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile5	Corner Constraint File for Corner 5	Specifies the corner constraint file associated
Corner Definition/Corner 6		

coreTools Command Reference Index

Name	Label	
CornerName6	Name for Corner 6	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile6	Corner Constraint File for Corner 6	Specifies the corner constraint file asso
Corner Definition/Corner 7		
CornerName7	Name for Corner 7	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile7	Corner Constraint File for Corner 7	Specifies the corner constraint file asso
Corner Definition/Corner 8		
CornerName8	Name for Corner 8	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile8	Corner Constraint File for Corner 8	Specifies the corner constraint file asso
Corner Definition/Corner 9		
CornerName9	Name for Corner 9	Defines a corner to be created using the A corner being used for set-up time and in the name. Similarly, a corner used for must have HOLD in the name.
CornerConstraintFile9	Corner Constraint File for Corner 9	Specifies the corner constraint file asso
Design for Test		
InsertDft	Insert Dft	Enables design for test circuitry to be in settings from the Specify Test Methodo
DFTFlowType	DFT Flow Type	sets the FCRM DFT_Flow control vari pre_compile. - EnumValues: in_compile pre_c - Symbolic Names: in_compile p
NumberOfScanChains	Internal_scan mode chain count.	The number of scan chains to be create to the set_scan_configuration commande option. For Internal_scan, this is also th ports to be created. The default value is NumberOfScanChains set on the imple
ClockMixing	Clock Mixing	Specifies whether cells from different c in the same scan chain. The default val attribute ClockMixing set on the imple EnumValues: no_mix mix_edges mix_clocks_not_edges

coreTools Command Reference Index

Name	Label	
		Symbolic Names: no_mix mix_mix_clocks_not_edges
CTLModels	CTL Models	CTL Model files are used to describe the design and generate test vectors for the design. This is read in the context of top-level synthesis (or bottom-up synthesis) whereas the design is only used during bottom-up synthesis. The variable DCRM_DFT_CTL_FOR_ABSTRACT_BLOCKS (For CTLModels in design attributes and the activity.
Design for Test/Scan Compression Settings		
EnableScanCompression	Enable scan compression	Enables ScanCompression_mode in DFTMAX. The value is the design attribute EnableScanCompression implementation context. For more information on scan compression, refer to DFTMAX and DFTMAX Ultra.
EnableDistributedCodec	Enable Distributed Codec	Enables distributed codec feature in DFTMAX. The app option user can enable distributed codec. Best-In Class PPA for DFT Compression. The following features: - Splitting of DFTMAX IP Codec IP into multiple sub-units. - DFTMAX will be kept. - Tool will perform scan compression. Tool will place compressors and decompressors in chain clustering.
ScanCompressionIOCount	ScanCompression_mode IO count.	When the hybrid DFT flow is used, the Internal_scan and ScanCompression_mode strategy parameter sets the number of scan outputs of the Decompressor logic when scan compression or streaming compression. The number of scan outputs of the Decompressor parameter must be set explicitly to a non-zero value. Enabled, e.g if set to 3, 'set_scan_compression -input 3 -output 3 -chain_count 12' is used for scan compression. (where 12 comes from NumberOfScanCompressionChains).
NumberOfScanCompressionChains	Scan compression chain count.	In ScanCompression_mode, this parameter sets the number of scan compression chains to be created internally. The default value is 12. The design attribute NumberOfScanCompressionChains implementation context.
ScanCompressionUsage	Scan compression usage	The value passed along with -usage option. ScanCompression_mode test mode in DFTMAX, DFTMAX Ultra and DFTMAX TestMAX DFT user guide for more details. This strategy parameter is the value of design attribute 'ScanCompressionUsage' set on the implementation context. EnumValues: scan_compression_usage seq_compression

coreTools Command Reference Index

Name	Label	
		Symbolic Names: {Scan comp compression (DFTMAX Ultra SEQ)}
SeqCompressionInputs	SEQ Compression Inputs	Total No of SEQ Compression Input P
SeqCompressionOutputs	SEQ Compression Outputs	Total No of SEQ Compression Output
ScanCompressionTestClock	Scan compression test clock	Sets a test clock for scan compression f clocks available.
ScanCompressionConfiguration	Scan compression configuration	The value of this parameter is passed d 'set_scan_compression_configuration' DFTMAX compression options, please Guide, Chapter 2, 'Using DFTMAX C 'Managing X Values in Scan Compres
Design for Test/TestMAX Manager Settings		
EnableTestMaxManagerFlow	Enable TestMAX Manager Flow	Enables TestMAX Manager to introduc the RTL prior to synthesis. Such RTL v processed by Fusion Compiler. To enab following conditions are to be met : 1. InsertDft set to 1 2. The leaf level components shor ready. 3. Fusion Compiler version 2022. 4. ScanCompressionUsage cannot 5. Test Max Manager version 202 6. Spyglass version 2023.03 or gr 7. Not supported for encrypted li
EnableTPI	Enable Test point insertion	Enables TestMAX Manager to introduc RTL prior to synthesis. Such RTL with by Fusion Compiler. Can only be enabl ScanCompressionUsage is not set to sc
Hierarchical Synthesis		
EnableHierarchicalSynthesis	Enable Hierarchical Synthesis	Setting the parameter to true enables th When enabled, the ndm for the sub-blo Subsystem level synthesis. The flow is coreAssembler. To enable this strategy Compiler 2022.03-SP1 or greater shoul
Mode Definition/Mode 1		
ModeName1	Name for Mode 1	Defines a mode to be created using the Each mode must have a unique name.
ModeConstraintFile1	Mode Constraint File for Mode 1	Constraint file used to set up mode 1.
Mode Definition/Mode 2		
ModeName2	Name for Mode 2	Defines a mode to be created using the Each mode must have a unique name.

coreTools Command Reference Index

Name	Label	
ModeConstraintFile2	Mode Constraint File for Mode 2	Constraint file used to set up mode 2.
Mode Definition/Mode 3		
ModeName3	Name for Mode 3	Defines a mode to be created using the Each mode must have a unique name.
ModeConstraintFile3	Mode Constraint File for Mode 3	Constraint file used to set up mode 3.
Mode Definition/Mode 4		
ModeName4	Name for Mode 4	Defines a mode to be created using the Each mode must have a unique name.
ModeConstraintFile4	Mode Constraint File for Mode 4	Constraint file used to set up mode 4.
Parasitic Definition/Parasitic 1		
ParasiticModelName1	Parasitic Model Name for Model 1	Specifies the unique name which identifies the model.
TluPlusFileForParasitic1	TLUPlus File for Model 1	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile1	Layer Map File for Model 1	Specifies the names of the layer map files, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 10		
ParasiticModelName10	Parasitic Model Name for Model 10	Specifies the unique name which identifies the model.
TluPlusFileForParasitic10	TLUPlus File for Model 10	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile10	Layer Map File for Model 10	Specifies the names of the layer map files, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 11		
ParasiticModelName11	Parasitic Model Name for Model 11	Specifies the unique name which identifies the model.
TluPlusFileForParasitic11	TLUPlus File for Model 11	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile11	Layer Map File for Model 11	Specifies the names of the layer map files, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 12		
ParasiticModelName12	Parasitic Model Name for Model 12	Specifies the unique name which identifies the model.
TluPlusFileForParasitic12	TLUPlus File for Model 12	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile12	Layer Map File for Model 12	Specifies the names of the layer map files, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 13		
ParasiticModelName13	Parasitic Model Name for Model 13	Specifies the unique name which identifies the model.
TluPlusFileForParasitic13		

coreTools Command Reference Index

Name	Label	
	TLUPlus File for Model 13	Specifies the names of the TLUPlus file to read for this parasitic model.
LayerMapFile13	Layer Map File for Model 13	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 14		
ParasiticModelName14	Parasitic Model Name for Model 14	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic14	TLUPlus File for Model 14	Specifies the names of the TLUPlus file to read for this parasitic model.
LayerMapFile14	Layer Map File for Model 14	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 15		
ParasiticModelName15	Parasitic Model Name for Model 15	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic15	TLUPlus File for Model 15	Specifies the names of the TLUPlus file to read for this parasitic model.
LayerMapFile15	Layer Map File for Model 15	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 16		
ParasiticModelName16	Parasitic Model Name for Model 16	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic16	TLUPlus File for Model 16	Specifies the names of the TLUPlus file to read for this parasitic model.
LayerMapFile16	Layer Map File for Model 16	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 17		
ParasiticModelName17	Parasitic Model Name for Model 17	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic17	TLUPlus File for Model 17	Specifies the names of the TLUPlus file to read for this parasitic model.
LayerMapFile17	Layer Map File for Model 17	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 18		
ParasiticModelName18	Parasitic Model Name for Model 18	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic18	TLUPlus File for Model 18	Specifies the names of the TLUPlus file to read for this parasitic model.
LayerMapFile18	Layer Map File for Model 18	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 19		
ParasiticModelName19	Parasitic Model Name for Model 19	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic19	TLUPlus File for Model 19	Specifies the names of the TLUPlus file to read for this parasitic model.

coreTools Command Reference Index

Name	Label	
LayerMapFile19	Layer Map File for Model 19	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 2		
ParasiticModelName2	Parasitic Model Name for Model 2	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic2	TLUPlus File for Model 2	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile2	Layer Map File for Model 2	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 20		
ParasiticModelName20	Parasitic Model Name for Model 20	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic20	TLUPlus File for Model 20	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile20	Layer Map File for Model 20	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 3		
ParasiticModelName3	Parasitic Model Name for Model 3	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic3	TLUPlus File for Model 3	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile3	Layer Map File for Model 3	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 4		
ParasiticModelName4	Parasitic Model Name for Model 4	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic4	TLUPlus File for Model 4	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile4	Layer Map File for Model 4	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 5		
ParasiticModelName5	Parasitic Model Name for Model 5	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic5	TLUPlus File for Model 5	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile5	Layer Map File for Model 5	Specifies the names of the layer map file, technology file and the ITF file for this model.
Parasitic Definition/Parasitic 6		
ParasiticModelName6	Parasitic Model Name for Model 6	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic6	TLUPlus File for Model 6	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile6	Layer Map File for Model 6	Specifies the names of the layer map file, technology file and the ITF file for this model.

Name	Label	
Parasitic Definition/Parasitic 7		
ParasiticModelName7	Parasitic Model Name for Model 7	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic7	TLUPlus File for Model 7	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile7	Layer Map File for Model 7	Specifies the names of the layer map files, the technology file and the ITF file for this model.
Parasitic Definition/Parasitic 8		
ParasiticModelName8	Parasitic Model Name for Model 8	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic8	TLUPlus File for Model 8	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile8	Layer Map File for Model 8	Specifies the names of the layer map files, the technology file and the ITF file for this model.
Parasitic Definition/Parasitic 9		
ParasiticModelName9	Parasitic Model Name for Model 9	Specifies the unique name which identifies the parasitic model.
TluPlusFileForParasitic9	TLUPlus File for Model 9	Specifies the names of the TLUPlus files to read for this parasitic model.
LayerMapFile9	Layer Map File for Model 9	Specifies the names of the layer map files, the technology file and the ITF file for this model.
Power/Power Optimization		
SaifFile	Saif File Path	List of SAIF files for power aware synthesis.
SaifInstanceName	Saif Instance Name	Name of the instance of the current design that is associated with the SAIF file. This argument is passed on to the power aware synthesis with the -strip_path option.
Power/UPF		
EnableUPFFlow	Enable UPF Flow	When Enable UPF Flow is set to true, the power aware synthesis constraints files are used during synthesis.
UPFFile	UPF File	The name of the Unified Power Format file to be used by the Fusion Compiler.
PowerConstraintsFile	Power Constraints File	The power constraints file is sourced in the UPF file during loading the UPF file. This file can be used to remove 'set_voltage' commands and remove dummy cells, as needed.
MVSetupFile	Multi Voltage Setup File	Tcl script placeholder for your MV setup script, such as, create_voltage_area, placement, power aware synthesis creation and level shifter insertion, etc.
Reports		
GenerateRMReports	Generate supplementary Reports	Enables report generation commands such as generate_rm_reports Methodology.
ReportQOR	Generate Stage Based QOR reports	FCRM templates generate QOR data at the end of synthesis by this parameter.

coreTools Command Reference Index

Name	Label	
ReportQORHistogram	Generate QOR Histogram	Report a histogram for QOR data using write_data stage.
DisableCongestionReport	Skip Congestion Report	Disables congestion report generation. improvements.
Scenario Definition		
PrimaryScenario	Index of the Primary Scenario	Select the primary scenario at the end of the simulation. EnumValues: 1 2 3 4 5 6 7 8 9 19 20
Scenario Definition/Active Scenario List		
ActiveScenarioListcompile	Active Scenario List for compile	Defines the set of active scenarios for the compile stage. Values must match entries in the scenario parameters. If not specified, all scenarios are considered active.
ActiveScenarioListclock_opt_cts	Active Scenario List for clock_opt_cts	Defines the set of active scenarios for the clock_opt_cts stage. Values must match entries in the scenario parameters. If not specified, active scenarios for the compile stage will be used. If not specified for a stage, all scenarios are considered active.
ActiveScenarioListclock_opt_opto	Active Scenario List for clock_opt_opto	Defines the set of active scenarios for the clock_opt_opto stage. Values must match entries in the scenario parameters. If not specified, active scenarios for the clock_opt_cts stage will be used. If not specified for a stage, all scenarios are considered active.
ActiveScenarioListroute_auto	Active Scenario List for route_auto	Defines the set of active scenarios for the route_auto stage. Values must match entries in the scenario parameters. If not specified, active scenarios for the clock_opt_opto stage will be used. If not specified for a stage, all scenarios are considered active.
ActiveScenarioListroute_opt	Active Scenario List for route_opt	Defines the set of active scenarios for the route_opt stage. Values must match entries in the scenario parameters. If not specified, active scenarios for the route_auto stage will be used. If not specified for a stage, all scenarios are considered active.
Scenario Definition/Scenario 1		
ScenarioName1	Name for Scenario 1	Defines a scenario to be created with the scenario command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode. The entries must be defined in the defined corner.
Scenario Definition/Scenario 10		
ScenarioName10	Name for Scenario 10	Defines a scenario to be created with the scenario command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode. The entries must be defined in the defined corner.
Scenario Definition/Scenario 11		
ScenarioName11	Name for Scenario 11	

coreTools Command Reference Index

Name	Label	
		Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 12		
ScenarioName12	Name for Scenario 12	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 13		
ScenarioName13	Name for Scenario 13	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 14		
ScenarioName14	Name for Scenario 14	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 15		
ScenarioName15	Name for Scenario 15	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 16		
ScenarioName16	Name for Scenario 16	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 17		
ScenarioName17	Name for Scenario 17	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 18		
ScenarioName18	Name for Scenario 18	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 19		
ScenarioName19	Name for Scenario 19	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 2		

coreTools Command Reference Index

Name	Label	
ScenarioName2	Name for Scenario 2	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 20		
ScenarioName20	Name for Scenario 20	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 3		
ScenarioName3	Name for Scenario 3	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 4		
ScenarioName4	Name for Scenario 4	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 5		
ScenarioName5	Name for Scenario 5	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 6		
ScenarioName6	Name for Scenario 6	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 7		
ScenarioName7	Name for Scenario 7	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 8		
ScenarioName8	Name for Scenario 8	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.
Scenario Definition/Scenario 9		
ScenarioName9	Name for Scenario 9	Defines a scenario to be created with the command. The entries must be named <MODE NAME> where <MODE NAME> is a defined mode defined corner.

See Also

GenerateIf

This cell or port is conditionally generated

Definition

Type: string**Flags:** subscripted**Default value:****Valid subscripts:** <valid subscripts are not statically defined>**Default subscript:** 0**Valid on:** cell param pin port _____

Description

The GenerateIf attribute specifies the conditions under which a conditionally generated cell or port will exist in a design. coreConsultant uses GenerateIf to update its internal data model of the core based on the user-specified configuration, so that it does not refer to a non-existent cell or port.

For VHDL designs, coreBuilder automatically sets GenerateIf on any VHDL cell that is conditionally generated with a VHDL if-generate construct. Therefore, you do not need to explicitly set GenerateIf in a VHDL design.

For Verilogs designs, you must set GenerateIf on any cell that is conditionally generated based on design parameter values. If the code is Verilog 2001, then the generate construct is supported and the GenerateIf attributes do not need to be explicitly set.

For ports, neither language provides a mechanism to conditionally generate the port. So in that case the attribute must be explicitly set.

Conditional cells and ports must appear 'conditional' in both the internal data model of the design (i.e. within the tool), and within the HDL written out for implementation. In most cases, the tool will automatically handle insertion of the text substitution pragmas required to include or exclude the conditional port or cell. Note that for VHDL, text substitution is required only for ports as conditional cells are supported by native language constructs. The only time that the user must specifically include a substitution pragma to handle conditionality of a port or cell is when the port or cell is already contained within a manually inserted conditional text substitution block. When needed, use the IncludeIf command inside a reuse-pragma startSub/endSub block within the code to conditionally include the code block. The parameter expression passed to IncludeIf should exactly match the expression used for the GenerateIf value. See the EXAMPLES section for an example of this.

The GenerateIf subscript indicates the level of nesting of condition statements. GenerateIf[0] (the default subscript) indicates the outer-most condition or a single condition. If there are multiple conditions (for example, a VHDL if-generate within another if-generate), the subscript indicates the level of nesting. GenerateIf[0] is the outer-most level, GenerateIf[1] is the next level in, and so on.

When the GenerateIf attribute is explicitly set (i.e. not part of a language generate construct), then the LoadDesigns activity will automatically place text substitution pragmas that use the IncludeIf command to conditionally remove port or cell from the RTL code.

Examples

The following Verilog code example shows how to implement a conditionally generated cell. The GenerateIf attribute indicates to the coreTool that cell U2 is generated only if the value of parameter W is 3. The conditional text pragmas will be inserted automatically in this case.

```
U1 mysubmodule(in1, out1);
// reuse-pragma attr GenerateIf @W==3
U2 mysubmodule(in2, out2);
```

Alternatively, you could remove the GenerateIf attribute assignment from the Verilog file and, instead, insert the following command in the Tcl intent command file for the design:

```
set_cell_attribute U2 GenerateIf @W=3
```

The following VHDL code fragment shows nested if-generate constructs. When coreBuilder parses this code, it automatically sets GenerateIf[0] to @W==8 and GenerateIf[1] to @D==16 on cell U1:

```
LOOP1: if (W = 8) generate
LOOP2: if (D = 16) generate
U1: mysubmodule(...);
end generate LOOP1;
end generate LOOP2;
```

The following two blocks of Verilog code show two different ways to implement the conditional generation of cell U1. In both cases, U1 is generated if parameter W is 8 and parameter D is 16:

```
// reuse-pragma attr GenerateIf (@W==8)&&(@D==16)
U1 mysubmodule(...);
is the same as:
```

```
// reuse-pragma attr GenerateIf[0] @W==8
// reuse-pragma attr GenerateIf[1] @D==16
U1 mysubmodule(...);
```

The following example shows the situation where the text substitution pragmas must be manually inserted because the conditional port instantiation is included within a pre-existing text substitution block.

```
// reuse-pragma startSub myBlk [IncludeIf @myParam %subText]
<... Verilog code ...>
// reuse-pragma attr GenerateIf @T==2
U5 conditional_module(...);
<... more Verilog code ...>
/ reuse-pragma endSub myBlk
```

See Also

set_cell_attribute (2), IncludeIf (3), ConvertSingleBitBus (3)

Generate Subsystem RTL

Generate the HDL source code for top-level design of the subsystem.

Parameters

Parameters Table

Name	Label	Description
LanguageChoice	Output language	Indicates the language to be used when generating the top-level design(s) of the subsystem. EnumValues: VHDL, SystemVerilog, Verilog
PreferredDataType	Preferred VHDL logic type	Use this logic data type for top-level ports and Verilog-to-VHDL port connections when creating a VHDL subsystem. EnumValues: std_logic, std_ulogic
IncludeTestbenchProbeSignals	Include testbench probe signals	When checked, an additional package is generated containing definitions of global signals for all pins in the design. This is required if you intend to create a VHDL testbench that utilizes monitors whose connections will be made by being routed through the hierarchy. In VHDL, this connectivity requires the definition of these global signals to enable the connections.
IncludeSVIncludeFiles	Write System Verilog 'include files	When generating System Verilog HDL write 'include files information into the files.
UseCompactInstanceSyntax	Use compact instance syntax	When generating System Verilog HDL use the compact instance syntax. This excludes the pin name when the pin name and signal name are the same.
IncludeModPortInInterfaceConnections	Include modport in interface connections	When checked if you connect system Verilog pin using modport.

coreTools Command Reference Index

Name	Label	Description
		system Verilog interface, the underlying system Verilog port in the component is connected with a modport, then we should include the modport in the connection.
IncludePortDescriptionsInRTL	Include port descriptions in RTL	When checked, port descriptions will be written into the generated RTL where they are defined (VHDL entity or Verilog module). Descriptions are specified by setting the Description attribute on the port.
WriteAllExportPinComments	WriteAllExportPinComments	Set to true to also print export_pin comments for intermediate levels of hierarchy.
SignalNameFormat	Signal name format	Defines the format for automatically created signal names. The default format translates to [instance name]_[port name], as taken from the signal driver. The default format can be overridden by setting a preference on the 'assembly' page of the Edit-->Preferences dialog. The format string can have additional characters in it. It supports the following substitutions: c --> component instance name l --> use load component instance of driver (single load only) p --> component port name d --> component port direction ('o', or 'b') D --> component port direction ('I', 'O', or 'B')
UseUniqueLogicNets	Use unique logic nets	Use a unique net for each logical constant connection.
UseNetsForOpenOutputPins	Use nets for open output pins	Use a unique, floating net for each open output pin instead of an empty connection.
AssignForInitialization		

coreTools Command Reference Index

Name	Label	Description
	Use assign for initialization	Use separate assignment statement for initialization of constant nets.
AllowReadingOfOutputPorts	Allow reading of output ports	Generated netlist may utilize reading of output ports to eliminate assign statements. This will occur when a pin is driving 1 or more other pins or a single output port. If enabled, the output port net will be used as the wire connected the pins and ports. Otherwise an assign statement would be utilized to connect the output port.
DontWriteHeader	Don't Write Synopsys Header	Don't write the Synopsys header to Subsystem RTL Files
AllowAggregateAssignments	Allow Aggregate Assignments	Allow aggregate assignment of pins to reduce use of assign statements.
DontTieOffInputs	Don't tie open inputs to logic X.	Don't tie open inputs to logic X.
DefineWiresForAllPorts	Define Wires for all Ports.	Define wires for all ports if not strictly required.
DelimitBitVectors	Write bit vector literals in binary format.	When this parameter is enabled, bit vector literals are reformatted to binary in the generated Verilog/SystemVerilog subsystem RTL, written in groups of 4, separated by a delimiter.
ExportConnectionCommentsInline	Set the comment on port to occur Inline	A Comment on port appears on port assignment. By setting ExportConnectionCommentsInline, the comment appears inline on port assignment
UserComment	User comment inserted into each generated file	Comment text that will be inserted into the header of the generated subsystem RTL files.

See Also

[Complete Connections.](#)

Generate Test Vectors

Generate test vectors using TestMAX.

Parameters

Parameters Table

Name	Label	Description
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieved using the following command: 'get_logical_dir syn/run.scr'
ParallelJobCpuLimit	Parallel job CPU limit:	Specifies the number of parallel compile jobs that may be run.
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
EmailAddress	To:	If this parameter is set to a value other than "", then email is sent when the command script completes or is killed. The default value is the current user's name.
UseTestMAX_ATPG	Use TestMAX ATPG	Indicates if TestMAX ATPG is to be used for Test vector generation. TestMAX ATPG is available starting L-2016.03 version of TestMAX.
Setup		

coreTools Command Reference Index

Name	Label	Description
OutputStageName	Output stage name:	Synthesis stage to use for test vector generation.
InputStageName	Input stage name:	Synthesis stage to use for test vector generation.
InputNetlistFormat	Input Netlist format:	The format of the design netlist to be read into TestMAX. - EnumValues: .v .vhd .edif - Symbolic Names: verilog vhd edif
TestLibraries	Test Libraries	The test libraries are the verilog simulation models for the target libraries used during synthesis. They provide the description of the logical functionality of all the cells in the standard cell library. They are needed for the ATPG tool to comprehend the logic function implemented by each standard cell instantiated, to generate test vectors for the faults. The value may be a list and may contain wildcards that are acceptable to TestMAX.
LibsMissingCellDefine	Libs Missing Cell Define	If the libraries specified in TestLibraries do not have <code>celldefine</code> and <code>endcelldefine</code> around the module definitions, setting this parameter to true will cause the libraries to be read using the <code>-library</code> switch.
TestPatternFormat	Test Pattern Format	Format for saving the test patterns. - EnumValues: .bin .stil .stil .vhd .vhd .wgl .tst .ftdl .tdl .wgl - Symbolic Names: Binary Stil STIL99 VHdl VHDL93 Wgl TStl2 Ftdl TDl91 WGL_Flat
WriteTestPatterns	Write Test Patterns	Set this parameter to FALSE to disable saving the test patterns.
UserStrategyFile	TestMAX command file:	The name of the file containing the TestMAX commands to execute for the ATPG_user_defined_strategy.
Strategy		
StrategyName	Strategy	Selects the strategy for generating test vectors. EnumValues: TMAX_Reference_Methodology

Name	Label	Description
		ATPG_basic_scan_strategy ATPG_user_defined_strategy

See Also

Perform ASIC Synthesis.

GlobalSlotNumber

Provides access to the global slot number stored in the SlotNumber parameter on the given interface

Definition

Type: long

Flags: readOnly

Default value: =get_attribute [get_attribute %item -attr Children -subscript SlotNumber] -attr Value

Valid on:

Description

This read-only attribute indicates the slot associated with a connection to a bus or other interface 'source' which connects to multiple 'loads' (typically bus masters or slaves). This represents which bus 'slot' the master or slave is associated with. This number is assigned automatically as components are added to the subsystem, but it can be changed by setting the SlotNumber attribute on the interface instance in a batch script or via the "Switch Slots" menu option in the schematic window.

The SlotOrder specifies the continuous sequence of all slots that make up the SlotNumber. Slots can have a SlotWidth which could be different from 1 (multiple slots occupied by a single interface), and the provider interface defines the first slot number with SlotNumberOffset.

In a hierarchical subsystem the SlotNumber represents the 'local' value for slot number (within the containing hierarchy). If the overall (flat) slot number is required, then the GlobalSlotNumber attribute should be used. In a non-hierarchical subsystem the SlotNumber and GlobalSlotNumber are always equivalent. In either case, only the SlotNumber can actually be set.

Examples

See Also

[SlotNumberOffset \(3\)](#), [SlotWidth \(3\)](#), [SlotOrder \(3\)](#)

[API Index](#) [Command Reference](#)

GroupImage

Image to use in pre-text for signal, parameter, and memory map groups.

Definition

Type: string

Flags:

Default value:

Valid on: param port _____

Description

Examples

See Also

GroupImageAttrs

Attributes to use for xml docbook:imagedata generated from GroupImage.

Definition

Type: string

Flags:

Default value: align="center"

Valid on: param port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

GroupImageTitle

Title to use for figures displayed with GroupText or GroupImage.

Definition

Type: string

Flags:

Default value:

Valid on: param port _____

Description

Examples

See Also

GroupName

Group similar items together in GUI displays

Definition

Type: string

Flags:

Default value:

Valid on: param port _____

Description

The GroupName attribute determines how the coreTools group parameters or ports together in GUI spreadsheets and dialogs.

Parameters that have the same value for the GroupName attribute will appear on the same tab of a parameter dialog. Ports that have the same value for the GroupName attribute will appear in the same port group in the Specify Port Intent property sheet.

If you do not specify the GroupName attribute on parameters, the coreTool automatically groups the parameters into groups of ten. By specifying the GroupName attribute you can group design parameters or custom activities into logical groups with meaningful names.

For custom activities, if you specify the -group_boxes with the complete_custom_activity_definition command, the parameter groups appear in separate boxes on a single-page parameter dialog instead of on separate tabs.

The coreTools automatically group the ports into two tab groups: input ports and output ports. By specifying the GroupName attribute on a design's ports, you can further subdivide the ports into groups according to function or any other common feature. On the Inputs or Outputs tab of the Specify Port Intent property sheet, users can select subgroups by GroupName from the Port Group combo box.

Examples

To assign design parameter device_id to the Identification group:

```
coreBuilder> set_parameter_attribute device_id GroupName Identification
```

To assign the ram_addr port to the Memory Interface group:

```
coreBuilder> set_port_attribute ram_addr GroupName "Memory Interface"
```

See Also

set_parameter_attribute (2), set_port_attribute (2),
complete_custom_activity_definition (2)

[API Index](#) [Command Reference](#)

GroupText

Text that is displayed as pre-text for signal, parameter, and memory map groups.

Definition

Type: string

Flags:

Default value:

Valid on: param port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

GroupUserLabel

User label of the parameter group

Definition

Type: string

Flags:

Default value: =get_attribute %item -attr GroupName

Valid on: param_____

Description

User label of the parameter group, usually get from ParameterInfo.

Examples

See Also

[GroupName \(3\)](#)

HasLibertyModel

Indicates that this design has a Liberty model and therefore no HDL file analysis is required for synthesis.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

This attribute is used to indicate that the given component will be fully represented by a Liberty model during synthesis. Generated synthesis scripts will not analyze any of the HDL files associated with the component. It is assumed that references to this component will be resolved via a cell reference within a cell library. Typically this file would be associated with the component via the ExtraLinkLibs activity parameter in the Load Designs activity in coreBuilder. Alternatively, for IP-XACT componednts the file would be stored in a fileSet with the name ExtraLinkLibraries.

Setting this attribute on any design other than the top design of a component will not have any effect.

Examples

See Also

HdlLibrary

HDL library into which to analyze this file or file group

Definition

Type: string

Flags:

Default value:

Valid on: file filegroup filegroupGroup

Description

The HdlLibrary attribute specifies the HDL library into which to analyze the selected file or file group.

For VHDL designs, coreBuilder automatically sets HdlLibrary to the library that you specify for the file when you execute the Load Designs activity. To change the value of HdlLibrary for a VHDL file, you must repeat the Load Designs activity. You cannot change the value of HdlLibrary directly. The default library for VHDL designs is WORK.

For VHDL designs, you can specify more than one design library when you execute the Load Designs activity. However, each file can only be analyzed into one library. To get a list of existing HDL library names, use the `get_hdl_library_names` command. To list the HDL files that are currently specified to be analyzed into a specified library, use the `-library` option to the `get_hdl_file_list` command.

For Verilog designs, coreBuilder automatically sets HdlLibrary to WORK. You cannot change the value of HdlLibrary for a Verilog design file.

In the core integrator's environment, coreConsultant directs Design Compiler to analyze the design(s) contained in a file into the library specified by the HdlLibrary attribute on that file.

Examples

To get the VHDL library that the file MyCore.vhd will be analyzed into:

```
coreConsultant> get_attribute -attrs HdlLibrary MyCore.vhd
WORK
```

See Also

[get_hdl_library_names \(2\)](#), [get_hdl_file_list \(2\)](#), [get_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

HdlSourceFileNames

Name(s) of the file that defines this design or package, and any included files.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

List of the HDL files that define the design. The last file in the list should be the file that contains the Verilog or SystemVerilog module. Other files in the list are for related files that defined the design, for example include files.

Examples

See Also

[API Index](#) [Command Reference](#)

HdlTypeName

HDL type name of this parameter

Definition

Type: string**Flags:****Default value:****Valid on:** param_____

Description

The HdlTypeName attribute indicates the HDL type of a design parameter. coreBuilder automatically sets HdlTypeName on a design parameter if the parameter has a type declared in the HDL source code. Do not change the value of HdlTypeName directly in coreBuilder. If you need to change the type of the HDL parameter, change it in the HDL code, then repeat the Load Designs activity.

If there is no parameter type declared in the HDL source code, coreBuilder does not set HdlTypeName on the parameter.

Examples

To get the HDL type of design parameter bus_width:

```
coreConsultant> get_parameter_attribute bus_width HdlTypeName  
integer
```

See Also

[get_parameter_attribute \(2\), TypeName \(3\)](#)_____

HdlUnquoted

Indicates that when string param is written in HDL it should not be quoted.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: param_____

Description

This attribute is used to indicate that a string parameter should be written to HDL without quotes. Generally this will only work for Verilog or System Verilog parameters defined via ‘define. By default string parameters are always written quoted, but when HdlUnquoted is set to true the value is written without quotes.

Note that the original value (even for a ‘define) must be defined with quotes for coreBuilder to properly parse as a string parameter.

Examples

Create define to be written unquoted:

```
// reuse-pragma attr HdlUnquoted true
‘define StringParam "OrigValue"
This will be written as:
```

```
‘define StringParam OrigValue
in the configured RTL.
```

See Also

[API Index](#) [Command Reference](#)

HdlValue

How the value is formatted in the given language

Definition

Type: string

Flags: readOnly subscripted

Default value: Value calculated internally.

Valid subscripts: C systemverilog verilog vhdl

Default subscript:

Valid on: param_____

Description

This attribute returns the value of a parameter formatted correctly for the given language. The language is specified via the subscript. Currently Verilog and VHDL are supported.

Examples

The parameter "interruptValue" is a design parameter that was extracted from the HDL as a bitfield:

```
coreBuilder> get_parameter_attribute interruptValue Value
0xabc
coreBuilder> get_parameter_attribute interruptValue {HdlValue[verilog]
32'habc
coreBuilder> get_parameter_attribute interruptValue {HdlValue[vhdl]
x"abc"
32h'abc
0xabc
```

See Also

[get_parameter_attribute \(2\)](#), [ReplaceFormatParam \(2\)](#), [Value \(3\)](#)

HeaderNameFormat

Defines the name format for the header reference for the given attribute on this register or register field.

Definition

Type: string

Flags: subscripted

Default value: =sMem::default_HeaderNameFormat %item

Valid subscripts: AddressOffset BaseAddress BitAddressOffset

MemoryAccess RegisterResetMask RegisterResetValue RegisterSize

Default subscript:

Valid on:

Description

This attribute is used to alter the name of a definition in the automatically generated header file for a memory map. By default constants are defined as <register name><attribute name> for registers and <register name><field name><attribute name> for register fields. The format strings look like "%r_%a" and "%r_%f_%a" by default. This attribute is used to modify the default format.

Examples

Change how the constant is defined for the register size constant.

```
set_register_attribute MyReg {HeaderNameFormat[RegisterSize]} {%r_Size}
```

See Also

[CHeaderValue \(3\)](#), [VerilogHeaderValue \(3\)](#), [VhdlHeaderValue \(3\)](#)

HelpUrl

URL of the documentation file for an item

Definition

Type: string

Flags:

Default value: =sHtml::defaultUrl %item

Valid on:

Description

The HelpUrl attribute specifies the location of the documentation file for the selected item. When a user requests detailed help for the item, the coreTool displays the contents of the file located at HelpUrl in the user's web browser.

You can specify the URL in either of two forms: local to the coreKit installation or as a fully qualified URL.

To specify a HelpUrl that is local to the coreKit installation (for documentation that you deliver with in or with the coreKit), specify HelpUrl as a relative path name. For example, if the documentation file for an item is named inputs.html and is located in the doc subdirectory of the coreKit installation directory (<install_dir>/doc/inputs.html), specify HelpUrl as `"/doc/inputs.html"`.

To specify a fully-qualified URL use the form: `transport://machine:port/path`. An example of a fully qualified URL is `"http://www.synopsys.com"`.

Examples

To put the documentation file `doc/MyCoreDatabook.html` for the design MyCore in the coreKit installation directory:

```
coreBuilder> set_design_attribute -designs MyCore \
HelpUrl "/doc/MyCoreDatabook.html"
```

To point to the documentation file for the design MyCore at <http://www.MyCompany.com/Documentation/MyCoreDatabook.html>:

```
coreBuilder> set_design_attribute -designs MyCore HelpUrl \
"http://www.MyCompany.com/Documentation/MyCoreDatabook.html"
```

See Also

Description (3)

[API Index](#) [Command Reference](#)

HierarchicalIsolation

When true, hierarchical isolation logic is built, so that dedicated subdesign scan-out signals are gated by the design scan enable signal.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

When true, hierarchical isolation logic is built, so that dedicated subdesign scan-out signals are gated by the design scan enable signal.

Examples

Specify that hierarchical isolation logic is to be built.

```
coreBuilder> set_design_attribute HierarchicalIsolation TRUE
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

HighFanout

This input port has a high fanout.

Definition

Type: boolean

Flags:

Default value:

Valid on: busBit port _____

Description

The HighFanout attribute indicates that the associated input port has a high fanout internal to the design. If HighFanout is true on an input port, coreConsultant invokes special Design Compiler optimizations that resolve the problems caused by high fanout situations when generating a synthesis strategy for the design.

Whether a design benefits from the high-fanout related optimization depends not only on the number of fanout loads on the input port, but also on the types of the loads, which is largely technology-dependent. However, as a general rule, it is best to set HighFanout to true if an input port is directly connected to eight or more internal loads.

Examples

The following line in a coreTool Tcl script indicates that input port InputA of the current_design has a high fanout:

```
set_port_attribute InputA HighFanout true
```

See Also

set_port_attribute (2), set_port_attribute (2)

[API Index](#) [Command Reference](#)

IPEnvList

Allows additional IP specific env vars to be considered for running an activity. These will be overridden by UserEnvList activity parameter if same exist

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

IconPath

Path to an icon for this item

Definition

Type: string**Flags:****Default value:****Valid on:**

Description

Path to show to an icon file to display for the given items. This will typically be a logo associated with a design which is then shown in the schematic window within a component instance.

Examples

See Also

IdealPort

This port is 'ideal'. It has infinite drive, no resistance, and directly attached nets are not checked for transition or capacitance violations.

Definition

Type: boolean

Flags:

Default value: =InheritValue up {}

Valid on: busBit port _____

Description

The IdealPort attribute indicates that the given port has infinite drive (unless explicitly indicated otherwise via the DrivingCell attribute). In addition, the net(s) attached to the port have the following properties: no resistance, no transition or capacitance DRC violations.

This attribute is typically set to true on reset and test lines which are going to be buffered by a post-synthesis tool.

If you do not specify a value for IdealPort, the port inherits the IdealPort value from the connected port on the next higher level design.

Examples

To treat the rst_n input port as an ideal port:

```
coreConsultant> set_port_attribute rst_n IdealPort true
```

See Also

[set_port_attribute \(2\), DrivingCell \(3\)](#) _____

IdentifyShiftRegisters

When true , insert_dft/compile -scan identifies shift-registers in the design and keep only the first flop as a scan flop. Scan Architecting in preview_dft/insert_dft will group the flops that form shift registers as a single scan group named shift_register_identified_??. preview_dft -show all will show the shift registers identified. preview_dft -script writes out the shift-registers as scan groups. If the scan style is not multiplexed_flip_flop, insert_dft/compile -scan ignores this option.

Definition

Type: boolean

Flags:

Default value: =sCommonKb::GetIdentifyShiftRegistersDefault

Valid on: design_____

Description

When true , insert_dft/compile -scan identifies shift-registers in the design and keep only the first flop as a scan flop. Scan Architecting in preview_dft/insert_dft will group the flops that form shift registers as a single scan group named shift_register_identified_??. preview_dft -show all will show the shift registers identified. preview_dft -script writes out the shift-registers as scan groups. If the scan style is not multiplexed_flip_flop, insert_dft/compile -scan ignores this option.

Examples

```
set_design_attribute IdentifyShiftRegisters true
```

See Also

IfUnconnected

Connection to make if left unconnected.

Definition

Type: string

Flags:

Default value:

Valid on: busBit pin port _____

Description

This attribute is used to indicate the value that a particular pin should be tied to in coreAssembler, if the pin is never connected to anything during the automatic or manual connection processes. Legal values are: zero, one, dc, open, and export.

It can be set on the ports of a top-level design packaged by coreBuilder. When the design is instantiated in a coreAssembler subsystem, the attribute value will apply to the pin corresponding to the design port on which the attribute was set.

At the end of the manual connection process, all pins must be connected to something or else the subsystem is considered incomplete. Prior to issuing an error, coreAssembler will look at the unconnected pins and see if this attribute is set on the corresponding port of the component design. If set, the appropriate constant connection, or connection to 'open', or export of the pin will be made and the pin will therefore no longer be considered unconnected.

If you want to make a connection at the end of the automatic connection process (as opposed to the manual connection process) you can use the AutoConnectIfUnconnected attribute to achieve that purpose.

Examples

To indicate that port A should be tied to 0 if left unconnected:

```
prompt> set_port_attribute A IfUnconnected zero
```

To indicate that port B should be exported from the subsystem if left unconnected:

```
prompt> set_port_attribute B IfUnconnected export
```

See Also

[AutoConnectIfUnconnected \(3\)](#)

Importance

Indicates whether the deliverables for this filegroup are required or optional

Definition

Type: string

Flags:

Default value: Optional

Valid on: filegroup filegroupGroup

Description

This attribute indicates the default criticality of including the deliverables in the given filegroup. A value of 'Required' implies that the filegroup must have at least one file in it when the coreKit is built. A value of 'Optional' means that it is okay if the filegroup does not have any files it when the coreKit is built.

This attribute can be set directly on a filegroup, but is most often implied by definition of a deliverable in a BoM template. See the example below for more details.

Examples

To define a BoM template that has one required deliverable and one optional deliverable:

```
set BOMTemplate {
  BOMTemplate
  {Name {Example Importance BOM Template}}
  {coreBuilderBOMTemplateVersion 1.0}
  {Version 1.0}
  {Deliverable
    {Name "Verification Environment"}
    {Importance Required}
  }
  {Deliverable
    {Name "Documentation"}
    {Importance Optional}
  }
}
```

See Also

[API Index](#) [Command Reference](#)

ImportedWorkspaceComponentFile

Indicates the file used to import a workspace component.

Definition

Type: string

Flags:

Default value:

Valid on: cell design _____

Description

This attribute indicates that the component cell or design was created by importing a workspace component that was not editable.

Examples

This attribute should not be set by the user and only gets set when using `import_workspace_as_component` command without the `-editable` option.

See Also

`IsWorkspaceComponent (3)`, `ReadOnlyWorkspaceComponent (3)`,
`export_workspace_as_component (2)`, `import_workspace_as_component (2)`

IncludeIf

Replace text if the parameter expression is true

Syntax

```
string IncludeIf [-design <design>] [-comment <chars>] pExpr text
    string <design>
    string <chars>
    string pExpr
    string text
```

Parameters

-design <design>	The scope to evaluate the expression in. Default = current_design.
-comment <chars>	Strip comment characters from text This option tells the tool to strip these character from the beginning of each line of text.
pExpr	The parameter expression to evaluate. A parameter expression is a Tcl expression that involves one or more parameters and is a valid expression for the Tcl expr command after each @<parameter> string has been replaced by the actual parameter value.
text	The text to be conditionally included in a file. In most cases, it is more useful to use the string "%subText" instead of actual text. Using "%subText" allows you to enter the conditional text directly into the file so that, when coreConsultant writes out the file, it will only write the conditional text if the parameter expression evaluates to true. This text can include special strings described below that support an if-elseif-else construct.

Description

The IncludeIf command is part of the coreTools text substitution mechanism. By using Include with "reuse-pragma startSub", you can designate text to either include or exclude from a text file depending on if parameter expression evaluates to true. The most common use for Include is to generate custom text files based on user-specified parameter values.

<pExpr> can be a simple parameter expression like "@A==1", which evaluates to true if the value of parameter A is 1, or a more complex parameter expression like "(@A==1)&&(@B==2)", which evaluates to true if the value of parameter A is 1 and the value of parameter B is 2.

The <text> argument can contain special strings so that the IncludeIf command can choose from one of several sections to include. The special strings must be specified on a line by itself (with a leading comment character). The special strings are '[ElseIncludeIf <pExpr>]' and '[ElseInclude]'. These commands are used like an if-elseif-else command, and are optional. The ElseInclude string must be the last directive. See the examples section for a more concrete example.

The -comment argument is used to uncomment the text that is replaced. This is especially handy for Verilog and VHDL files, allowing the replaced section to be uncommented.

Refer to the Text Substitution section of the *coreBuilder User Guide* for more information about how to implement text substitution in files.

Examples

In the following code fragment, coreConsultant chooses the appropriate section of code from a verilog file:

```
// reuse-pragma startSub example
// [IncludeIf @param==1 %subText]
this text is included if param is 1
// [ElseIncludeIf @param==2]
this text is included if param is 2
// [ElseIncludeIf @param==3]
this text is included if param is 3
// [ElseInclude]
```

Otherwise we get this text

```
// reuse-pragma endSub example
```

The following example makes sure the given macro is defined (although it is not legal to have this macro defined in the default configuration, i.e. during the LoadDesigns activity):

```
// reuse-pragma startSub e2 [IncludeIf -comment // @p==2 %subText]
// 'define INCLUDE_FEATURE_X
// reuse-pragma endSub e2
```

The following example shows that if a proc ::RCEP::isSnps true then its following text added

```
# reuse-pragma startSub [IncludeIf {[::RCEP::isSnps]}
"RCE_WA_VCSPYGLASS-0003: sensitive"] endSub
```

See Also

[ReplaceConstantParam \(2\)](#), [ReplaceDesignParams \(2\)](#), [SubstituteInFile \(3\)](#)

IncludeLibCells

Will cause a set_dont_use <libCells> false to be placed on the library cells specified.

Definition

Type: string

Flags:

Default value: =InheritValue up ""

Valid on: design_____

Description

Use this attribute to specify a set of technology library cells that have been disabled for elsewhere that can be used to compile this design. If no value is the design inherits the IncludeLibCells value from the next higher level design. The default value of this attribute is null. Wildcards may be used to enable multiple cells with the same base name.

Cells which are enabled in this manner will also become visible (if appropriate) within the Specify Power Intent activity. For example, if you include an isolation cell in the IncludeLibCells value, that isolation cell will appear for selection within the Specify Power Intent activity.

In Fusion Compiler intent IncludeLibCells will issue the 'set_lib_cell_purpose -include all' command.

Examples

Enable the use of the library cells AND4H and XOR2L for the current_design.

```
coreConsultant> set_design_attribute IncludeLibCells [list AND4H XOR2L]
```

Enable the use of all of the AOI cells in the library myLib that is contained in the library file myLib.db

```
coreConsultant> set_design_attribute IncludeLibCells myLib/AOI*
```

See Also

[set_design_attribute \(2\)](#), [ExcludeLibCells \(3\)](#)

InheritValue

Inherit an attribute value from an item up or down the design hierarchy or from a pin

Syntax

```
string InheritValue [-component <comp>] [-attr <attr>] [-sub <sub>]
[-item <item>] [-nocache] <up|down|pin|from> <default>
    string <comp>
    string <attr>
    string <sub>
    string <item>
    string <up|down|pin|from>
    string <default>
```

Parameters

-component <comp>	Specifies the component this attribute value came from.
-attr <attr>	Specifies an attribute from which to inherit the value instead of the attribute that you are currently specifying. For example, you can specify a value for MaxInputDelay on an input port as a function of the MaxOutputDelay value inherited from the output port that drives that input port.
-sub <sub>	Specifies an attribute subscript from which to inherit the value instead of the current context.
-item <item>	Item to use, instead of the context
-nocache	Don't cache intermediate results
<up down pin from>	Direction from which to inherit the value. (Values: up, down, pin, from)
<default>	The default value to use if there is no value to inherit.

Description

The InheritValue command inherits the value of an attribute from an item either higher or lower in the design hierarchy (up or down) or from the driver or load on a port (pin). The coreTools use InheritValue to determine default values for several of the attributes that apply to designs. For example, the default value for the CanFlatten attribute is {=InheritValue up 0}, which means that the default value for CanFlatten

on a design is the value of CanFlatten on the next higher level design. The default value (0) is used if there is no higher level design or the higher level design has no value for CanFlatten.

You must specify the direction from which to inherit the attribute value. The behavior of the up, down, and pin options differ for design attributes and port attributes. In general, you use up or down for designs and up, down, or pin for ports.

For designs, if you specify up, InheritValue looks one level up in the hierarchy and extracts the value of the attribute you are setting from the next higher level design. If you specify down, InheritValue looks one level down in the hierarchy and extracts the value of the attribute you are setting from the next lower level design. In general, the up direction is more useful for specifying design attributes; CombineInheritValues is better for inheriting down because there is usually more than one lower level design.

For ports, if you specify up, InheritValue looks one level up in the hierarchy and extracts the value of the attribute you are setting from the connected port on the higher level design. If you specify down, InheritValue looks one level down in the hierarchy and extracts the value of the attribute you are setting from the connected port on the lower level design. For both up and down, InheritValue looks up or down the design hierarchy for a connected port of the same directionality as the one to which you are currently specifying the attribute value. For example, if you use InheritValue up to specify an attribute value on an input pin, InheritValue extracts the attribute value from the connected input or inout pin on the next higher level design.

If you specify pin, InheritValue extracts the value of the specified from a pin of the opposite directionality that is electrically connected to the port you are setting the attribute on. For input ports, InheritValue gets the attribute value from the output or inout port that is electrically connected to the input pin. For output ports, InheritValue gets the attribute value from the electrically connected input or inout port.

To inherit from a pin, you must usually specify an attribute to inherit by using the -attr option, because ports of opposite directionality do not use the same attributes. For example, to specify a value for MaxInputDelay on an input port as a function of the MaxOutputDelay value inherited from the output port that drives that input port, you must use the -attr MaxOutputDelay option.

In general, the up and pin directions are more useful for specifying port attributes.

Examples

To set CanFlatten to the value of CanFlatten on the next higher level design:

```
coreBuilder> \set_design_attribute CanFlatten {=InheritValue up 0}
```

To set MaxInputDelay on input port i4 to the clock cycle time minus the MaxOutputDelay value on the port that drives i4:

```
coreBuilder> set_port_attribute i4 {MaxInputDelay[clk]}  
{=expr {[get_clock_attribute clk CycleTime]  
- [InheritValue -attr MaxOutputDelay -sub clk pin 0]}}
```

See Also

[combineValues \(2\)](#), [CombineInheritValues \(2\)](#)

Initialize Subsystem Constraints

Create system level clocks, and initialize system level constraints from component level constraints.

Parameters

Parameters Table

Name	Label	Description
PropagateAllConstraints	Propagate all constraints	If true, all Synopsys Design Constraints are propagated upward from the leaf components. If false, only delay load constraints and timing exceptions between clocks are propagated. This case is consistent with the constraints that were propagated in versions of coreTools prior to C-2009.06
PropagateUpwardOnly	Propagate upward only	If true, all Synopsys Design Constraints are propagated upward only. If constraints are manually modified after propagation, they must be manually propagated upwards. This applies to all constraints except clocks.
ImplementationContext	Implementation Context	Defines the context at which the constraints are going to be implemented. Synthesis is performed in the 'syn' directory by default, or the chosen context.
IgnoreClockDefinitionOrMappingErrors	Ignore clock definition or mapping errors	Indicates whether or not errors in clock definition or clock mapping files cause the constraint propagation to stop.
ClockMappingFile	Clock mapping file name	The named file contains commands for the SystemClockName and GeneralClockName attributes on component level clocks.
ClockDefinitionFileName	Clock definition file name	The named file contains system level clock definition commands. This file contains only clock related SDC commands: create_clock, set_clock_uncertainty, set_clock_transition, set_false_path (from/to clocks), set_multicycle_path (from/to clocks), and set_max_delay (from/to clocks).
VirtualClockNamingStyle	Virtual clock naming style	Indicates the system level virtual clock naming style. Defaults to h_c, where h is replaced with the component level clock name and h is replaced with c.

coreTools Command Reference Index

Name	Label	Description
		hierarchical path to the component as a hierarchy separator. Multiple may be specified separated with spaces. If multiple are specified, each potential virtual clock name is generated in sequence to check for a pre-existing virtual clock. If a virtual clock is found with the given name, that name will be used. Otherwise a virtual clock based on first name in the set will be created during clock linking.
GeneratedClockNamingStyle	Generated clock naming style	Indicates the system level generated clock naming style. Defaults to i_c, where i is replaced with the component level clock name and c is replaced with the instance name of the component. Multiple styles may be specified separated with spaces. If multiple are specified, each potential generated clock name is generated in sequence to check for a pre-existing clock. If a clock is found with the given name, that name will be used. Otherwise a clock based on the given naming format will be created during clock linking.
RealClockNamingStyle	Real clock naming style	Indicates the system level real clock naming style. Defaults to c where c is replaced with the component level clock name. Multiple styles may be specified in the set to be merged have the same format. If they do not, the format reverts to the default. The format string can also reference the component level clock name. The key p is replaced with the system level port name. The key i is replaced with the instance name of the component. If multiple clocks are being propagated upwards, the first component (alphabetically) is chosen. If the format string is used with the p key, then the port name is from the chosen component's input port. Multiple formats may be specified separated with spaces.
Link virtual clocks using real clocks		
LinkVirtualClocksUsingRealClocks	Link virtual clocks using real clocks	Indicates that linking of component level virtual clocks should be attempted by matching component level real clocks and system level real clocks. If a link is found at both

Name	Label	Description
		then the corresponding virtual clock is linked across levels. Links at a common level are done using the format strings specified in the <code>LinkVirtualClocksUsingRealClocksNamingStyle</code> parameter. This methodology for linking is only used when a format string (or strings) is specified in the <code>LinkVirtualClocksUsingRealClocksNamingStyle</code> parameter, and a clock definition file name was specified to pre-define system clocks.
<code>LinkVirtualClocksUsingRealClocksNamingStyle</code>	Naming style	Specifies the format strings used for linking of component level virtual clocks. Links at a common level are done using the specified format strings with the clock name substituted into each string, replacing c with the component clock name, and p with the component port name with which the clock is associated. This methodology for linking is only used when a format string (or strings) is specified in this parameter and a clock definition file name was specified to pre-define system clocks.

See Also

[Generate Subsystem RTL](#),

InputDelay

Delay constraint for an input port

Definition

Type: float**Flags:** subscripted**Default value:****Valid subscripts:** <valid subscripts are not statically defined>**Default subscript:****Valid on:** busBit port _____

Description

InputDelay is a shorthand notation that you can use to set both MinInputDelay and MaxInputDelay to the same value using the set_port_attribute command. The InputDelay attribute does not appear in the coreConsultant port intent specification spreadsheets.

The subscript to the InputDelay attribute is the name of the clock. For example, InputDelay[clk] is the minimum and maximum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of InputDelay. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify InputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period \<percentage> [\<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

Examples

To set both MinInputDelay and MaxInputDelay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {InputDelay[clk]} \
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in InputDelay  
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MinInputDelay (3),
MaxInputDelay (3)

[API Index](#) [Command Reference](#)

InputResistance

Specifies the resistance on the net driven by an input port.

Definition

Type: float

Flags:

Default value: =InheritValue up {}

Valid on: busBit port _____

Description

This attribute defines the resistance on a net driven by an input port.

This attribute is passed along as a constraint to Design Compiler using the set_input_resistance command.

Examples

```
prompt> set_port_attribute data InputResistance 0
```

See Also

[DrivingCell \(3\)](#)

[API Index](#) [Command Reference](#)

InsertEndOfChainLockupLatch

When true lockup latches will be inserted at the end of scan chains.

Definition

Type: boolean

Flags:

Default value: =InheritValue up FALSE

Valid on: design_____

Description

When true lockup latches will be inserted at the end of scan chains for multiplexed_flip_flop scan style.

Examples

Specify that lockup latches are not to be inserted at the end of scan chains.

```
coreBuilder> set_design_attribute InsertEndOfChainLockupLatch 0
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

InsertTechCellBindMacroAtStart

Set to true to the file item if tech cell defn insertion should be done at top

Definition

Type: boolean

Flags:

Default value: 0

Valid on: file _____

Description

Examples

```
coreAssembler> set_attribute [find_item -type file DWbb_bcm00_sync1.v
-should_match 1] -attr InsertTechCellBindMacroAtStart -value 1
set_activity_parameter LoadConsultantPlugins PluginDirectories ./plugins
set_activity_parameter LoadDesigns InputLanguage SystemVerilog
set_activity_parameter LoadDesigns InputFileNames
{SOURCE/include_dir/compI_include_3.v
SOURCE/include_dir/compI_include_2.v SOURCE/include_dir/compI_include.vh
SOURCE/compI.v}
set_activity_parameter LoadDesigns TopDesignName compI
autocomplete_activity LoadDesigns
set_fileItem [filter_collection [get_hdl_file_list -items] -filter
Name=~.*\/compI_include_2.v$]
set_attribute $fileItem -attr ReadOnlyItem -value 0
set_attribute $fileItem -attr InsertTechCellBindMacroAtStart -value 1
set_attribute $fileItem -attr ReadOnlyItem -value 1
```

See Also

ForceUseTCBUndefFile (3) TechCellMappingDefn (3) TechCellMappingBind (3)

[API Index](#) [Command Reference](#)

InsertTestPoints

Controls whether observe and control points are inserted into this design to enhance testability.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Controls the insertion of test points to enhance the testability of this design. If this attribute is true, the -testability option will be used with the set_dft_configuration in DFT Compiler. The correspondence between coreTools attributes and set_testability_configuration options is as follows:

TestabilityMethod -method
 MaxControlPoints -max_control_points
 MaxObservePoints -max_observe_points
 ControlPointsPerScanCell -control_points_per_scan_cell
 ObservePointsPerScanCell -observe_points_per_scan_cell
 TestabilityClockType -clock_type
 ShareTestPointsAcrossHierarchy -share_across_hierarchy

Examples

Enable test point insertion on the current design.

```
coreBuilder> set_design_attribute InsertTestPoints TRUE
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#), [TestabilityMethod \(3\)](#), [MaxControlPoints \(3\)](#), [MaxObservePoints \(3\)](#), [ControlPointsPerScanCell \(3\)](#), [ObservePointsPerScanCell \(3\)](#), [TestabilityClockType \(3\)](#), [ShareTestPointsAcrossHierarchy \(3\)](#)

Install

Should this group be installed

Definition

Type: boolean

Flags:

Default value: 1

Valid on: file filegroup filegroupGroup

Description

This attribute is used to support conditional installation of a filegroup. If the attribute is set to true, the filegroup is installed, and if set to false, the filegroup is not installed.

To support conditional installation, this attribute should be set to a formula which evaluates to true for those conditions which imply that the files should be installed.

Examples

Assume that you have a filegroup called Doc/MacDoc which contains files which document the MAC block in your core. If the MAC block is only utilized when the 'UseMAC' parameter is set to true, then it might be nice to only install the documentation when the parameter is true. This can be accomplished by setting the Install attribute to {=eval_param @UseMAC}. This can be done in the BoM activity GUI or in batch mode as shown below.

```
prompt> set_filegroup_attribute Doc/MacDoc Install {=eval_param @UseMAC}
```

See Also

InstallUserWorkDir

How to install a file or file group into a user workspace

Definition

Type: string

Flags:

Default value: link

Valid on: file filegroup filegroupGroup

Description

The InstallUserWorkDir attribute specifies how to install the file or filegroup from the Bill of Materials (BoM) into a user workspace when a core integrator creates a user workspace for a core.

Valid values for InstallUserWorkDir are:

- link - Create a link from the coreKit installation directory to the user workspace. This is the default behavior.
- copy - Copy the file or file group from the coreKit installation directory to the user workspace.
- ignore - Ignore the selected file or file group when creating a user workspace.

Examples

To copy (not link) myfile from the coreKit installation directory to the user workspace when creating workspace:

```
coreBuilder> set_attribute -attr InstallUserWorkDir -value copy myfile
```

See Also

set_attribute (2), DefaultInstallDir (3)

[API Index](#) [Command Reference](#)

InstallWhen

When should file group be installed?

Definition

Type: string

Flags:

Default value: installation

Valid on: filegroup filegroupGroup_____

Description

InstallWhen specifies when to install the selected file group. Valid values for InstallWhen are:

- installation - Install the files during standard design kit installation
- custom - Package the files, but install the file only through a custom flow which has explicit code to install this specific filegroup
- used - Package the files in a plugin filegroup only, and install when the plugin activity is activated for the first time

The 'used' value is only supported for use within a plugin KB, and cannot be used on a configurable filegroup (Configurable==1).

Examples

See Also

InstanceClockNames

Names to use when creating clocks during InitializeSubsystemConstraints.

Definition

Type: string**Flags:** subscripted**Default value:****Valid subscripts:** <valid subscripts are not statically defined>**Default subscript:****Valid on:** design_____

Description

For leaf components, clock naming at the next level up can be controlled by setting SystemClockName on the leaf component clock. However, when there are shared instances of a component (leaf or hierarchical) a different approach is required because different names may be needed per instance. The InstanceClockNames attribute can be used for this purpose. This attribute is set in the upper design (where clocks will be created). The attribute is subscripted, and the subscript values indicate the component instance name and component clock name. The value of the attribute indicates the name to use for the clock within the design where the attribute is set.

Examples

Consider the following instance hierarchy:

- /L1/L2a/i_ahb (with clock hclk)
- /L1/L2b (shared copy of L1/L2a)
- /L1 (parent of L2a and L2b)

With this hierarchy, L2a and L2b are instances of the same design (D2). The clock 'hclk' in i_ahb will result in 2 clocks in L1 because of the multiple instances of design D2. In order to control the names of the propagated clock within L1, the following can be done.

```
set_current_component L1
set_design_attribute {InstanceClockNames[L1a/hclk]} hclk_a
set_design_attribute {InstanceClockNames[L1b/hclk]} hclk_b
set_current_component /
```

With these settings, clock propagation (via InitializeSubsystemConstraints) will create clocks named hclk_a and hclk_b, logically connected to the two instances, L1/L2a and L1/L2b, respectively.

See Also

SystemClockName

[API Index](#) [Command Reference](#)

IntentFilesProcessed

Set to true after intent files or processed.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: filegroup_____

Description

If IntentFilesProcessed is true it indicates that we have already processed intent files for the filegroup.

Examples

```
set fgIsProcessed [get_attribute -attr IntentFilesProcessed $filegroup]
```

See Also

InterClockHoldFallFallUncertainty

Uncertainty applied from falling edge of source clock to the falling edge of the destination clock for hold delay calculations

Definition

Type: float

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: clock_____

Description

The inter-clock uncertainty allows you to specify different skew between various clock domains. The InterClockHoldFallFallUncertainty attribute specifies inter-clock uncertainty applied to the falling edge of the destination clock relative to a falling edge source clock for hold delay calculations.

The subscript to the InterClockHoldFallFallUncertainty attribute is the name of the destination clock. For example, the InterClockHoldFallFallUncertainty[PHI2] attribute set on clock PHI1 defines inter-clock uncertainty between the PHI1 and PHI2 clock domains.

Examples

To set the inter-clock uncertainty from the falling edge of clock PHI1 to the falling edge of clock PHI2 to 1ns for hold delay calculations:

```
coreConsultant> set_clock_attribute PHI1
InterClockHoldFallFallUncertainty[PHI2] 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseRiseUncertainty \(3\)](#),
[InterClockHoldRiseFallUncertainty \(3\)](#), [InterClockHoldFallRiseUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#), [ClockHoldUncertainty \(3\)](#),
[ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#), [ClockSourceRiseLatency \(3\)](#),
[ClockSourceFallLatency \(3\)](#).

InterClockHoldFallRiseUncertainty

Uncertainty applied from falling edge of source clock to the rising edge of the destination clock for hold delay calculations

Definition

Type: float

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: clock_____

Description

The inter-clock uncertainty allows you to specify different skew between various clock domains. The InterClockHoldFallRiseUncertainty attribute specifies inter-clock uncertainty applied to the rising edge of the destination clock relative to a falling edge source clock for hold delay calculations.

The subscript to the InterClockHoldFallRiseUncertainty attribute is the name of the destination clock. For example, the InterClockHoldFallRiseUncertainty[PHI2] attribute set on clock PHI1 defines inter-clock uncertainty between the PHI1 and PHI2 clock domains.

Examples

To set the inter-clock uncertainty from the falling edge of clock PHI1 to the rising edge of clock PHI2 to 1ns for hold delay calculations:

```
coreConsultant> set_clock_attribute PHI1
InterClockHoldFallRiseUncertainty[PHI2] 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseRiseUncertainty \(3\)](#),
[InterClockHoldRiseFallUncertainty \(3\)](#), [InterClockHoldFallFallUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#), [ClockHoldUncertainty \(3\)](#),
[ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#), [ClockSourceRiseLatency \(3\)](#),
[ClockSourceFallLatency \(3\)](#).

InterClockHoldRiseFallUncertainty

Uncertainty applied from rising edge of source clock to the falling edge of the destination clock for hold delay calculations

Definition

Type: float

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: clock_____

Description

The inter-clock uncertainty allows you to specify different skew between various clock domains. The InterClockHoldRiseFallUncertainty attribute specifies inter-clock uncertainty applied to the falling edge of the destination clock relative to a rising edge source clock for hold delay calculations.

The subscript to the InterClockHoldRiseFallUncertainty attribute is the name of the destination clock. For example, the InterClockHoldRiseFallUncertainty[PHI2] attribute set on clock PHI1 defines inter-clock uncertainty between the PHI1 and PHI2 clock domains.

Examples

To set the inter-clock uncertainty from the rising edge of clock PHI1 to the falling edge of clock PHI2 to 1ns for hold delay calculations:

```
coreConsultant> set_clock_attribute PHI1
InterClockHoldRiseFallUncertainty[PHI2] 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseRiseUncertainty \(3\)](#),
[InterClockHoldFallRiseUncertainty \(3\)](#), [InterClockHoldFallFallUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#), [ClockHoldUncertainty \(3\)](#),
[ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#), [ClockSourceRiseLatency \(3\)](#),
[ClockSourceFallLatency \(3\)](#).

InterClockHoldRiseRiseUncertainty

Uncertainty applied from rising edge of source clock to the rising edge of the destination clock for hold delay calculations

Definition

Type: float

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: clock_____

Description

The inter-clock uncertainty allows you to specify different skew between various clock domains. The InterClockHoldRiseRiseUncertainty attribute specifies inter-clock uncertainty applied to the rising edge of the destination clock relative to a rising edge source clock for hold delay calculations.

The subscript to the InterClockHoldRiseRiseUncertainty attribute is the name of the destination clock. For example, the InterClockHoldRiseRiseUncertainty[PHI2] attribute set on clock PHI1 defines inter-clock uncertainty between the PHI1 and PHI2 clock domains.

Examples

To set the inter-clock uncertainty from the rising edge of clock PHI1 to the rising edge of clock PHI2 to 1ns for hold delay calculations:

```
coreConsultant> set_clock_attribute PHI1
InterClockHoldRiseRiseUncertainty[PHI2] 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockHoldRiseFallUncertainty \(3\)](#),
[InterClockHoldFallRiseUncertainty \(3\)](#), [InterClockHoldFallFallUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#), [ClockHoldUncertainty \(3\)](#),
[ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#), [ClockSourceRiseLatency \(3\)](#),
[ClockSourceFallLatency \(3\)](#).

InterClockSetupFallFallUncertainty

Uncertainty applied from falling edge of source clock to the falling edge of the destination clock for setup delay calculations

Definition

Type: float

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: clock_____

Description

The inter-clock uncertainty allows you to specify different skew between various clock domains. The InterClockSetupFallFallUncertainty attribute specifies inter-clock uncertainty applied to the falling edge of the destination clock relative to a falling edge source clock for setup delay calculations.

The subscript to the InterClockSetupFallFallUncertainty attribute is the name of the destination clock. For example, the InterClockSetupFallFallUncertainty[PHI2] attribute set on clock PHI1 defines inter-clock uncertainty between the PHI1 and PHI2 clock domains.

Examples

To set the inter-clock uncertainty from the falling edge of clock PHI1 to the falling edge of clock PHI2 to 1ns for setup delay calculations:

```
coreConsultant> set_clock_attribute PHI1
InterClockSetupFallFallUncertainty[PHI2] 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockSetupRiseRiseUncertainty \(3\)](#),
[InterClockSetupRiseFallUncertainty \(3\)](#),
[InterClockSetupFallRiseUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#),
[ClockHoldUncertainty \(3\)](#), [ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#),
[ClockSourceRiseLatency \(3\)](#), [ClockSourceFallLatency \(3\)](#),

InterClockSetupFallRiseUncertainty

Uncertainty applied from falling edge of source clock to the rising edge of the destination clock for setup delay calculations

Definition

Type: float

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: clock_____

Description

The inter-clock uncertainty allows you to specify different skew between various clock domains. The InterClockSetupFallRiseUncertainty attribute specifies inter-clock uncertainty applied to the rising edge of the destination clock relative to a falling edge source clock for setup delay calculations.

The subscript to the InterClockSetupFallRiseUncertainty attribute is the name of the destination clock. For example, the InterClockSetupFallRiseUncertainty[PHI2] attribute set on clock PHI1 defines inter-clock uncertainty between the PHI1 and PHI2 clock domains.

Examples

To set the inter-clock uncertainty from the falling edge of clock PHI1 to the rising edge of clock PHI2 to 1ns for setup delay calculations:

```
coreConsultant> set_clock_attribute PHI1
InterClockSetupFallRiseUncertainty[PHI2] 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockSetupRiseRiseUncertainty \(3\)](#),
[InterClockSetupRiseFallUncertainty \(3\)](#),
[InterClockSetupFallFallUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#),
[ClockHoldUncertainty \(3\)](#), [ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#),
[ClockSourceRiseLatency \(3\)](#), [ClockSourceFallLatency \(3\)](#),

InterClockSetupRiseFallUncertainty

Uncertainty applied from rising edge of source clock to the falling edge of the destination clock for setup delay calculations

Definition

Type: float

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: clock_____

Description

The inter-clock uncertainty allows you to specify different skew between various clock domains. The InterClockSetupRiseFallUncertainty attribute specifies inter-clock uncertainty applied to the falling edge of the destination clock relative to a rising edge source clock for setup delay calculations.

The subscript to the InterClockSetupRiseFallUncertainty attribute is the name of the destination clock. For example, the InterClockSetupRiseFallUncertainty[PHI2] attribute set on clock PHI1 defines inter-clock uncertainty between the PHI1 and PHI2 clock domains.

Examples

To set the inter-clock uncertainty from the rising edge of clock PHI1 to the falling edge of clock PHI2 to 1ns for setup delay calculations:

```
coreConsultant> set_clock_attribute PHI1
InterClockSetupRiseFallUncertainty[PHI2] 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockSetupRiseRiseUncertainty \(3\)](#),
[InterClockSetupFallRiseUncertainty \(3\)](#),
[InterClockSetupFallFallUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#),
[ClockHoldUncertainty \(3\)](#), [ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#),
[ClockSourceRiseLatency \(3\)](#), [ClockSourceFallLatency \(3\)](#),

InterClockSetupRiseRiseUncertainty

Uncertainty applied from rising edge of source clock to the rising edge of the destination clock for setup delay calculations

Definition

Type: float

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: clock_____

Description

The inter-clock uncertainty allows you to specify different skew between various clock domains. The InterClockSetupRiseRiseUncertainty attribute specifies inter-clock uncertainty applied to the rising edge of the destination clock relative to a rising edge source clock for setup delay calculations.

The subscript to the InterClockSetupRiseRiseUncertainty attribute is the name of the destination clock. For example, the InterClockSetupRiseRiseUncertainty[PHI2] attribute set on clock PHI1 defines inter-clock uncertainty between the PHI1 and PHI2 clock domains.

Examples

To set the inter-clock uncertainty from the rising edge of clock PHI1 to the rising edge of clock PHI2 to 1ns for setup delay calculations:

```
coreConsultant> set_clock_attribute PHI1
InterClockSetupRiseRiseUncertainty[PHI2] 1ns
```

See Also

[set_clock_attribute \(2\)](#), [InterClockSetupRiseFallUncertainty \(3\)](#),
[InterClockSetupFallRiseUncertainty \(3\)](#),
[InterClockSetupFallFallUncertainty \(3\)](#), [ClockSetupUncertainty \(3\)](#),
[ClockHoldUncertainty \(3\)](#), [ClockRiseLatency \(3\)](#), [ClockFallLatency \(3\)](#),
[ClockSourceRiseLatency \(3\)](#), [ClockSourceFallLatency \(3\)](#),

InterfaceGroup

Used to indicate that an interface is part of a 'connection' group.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Used to group a set of provider or consumer interfaces on a given component. Grouped interfaces are handled differently during interface connection actions. For automatic interface connection, the entire group is treated as a single available interface so the connection is not automatically ambiguous. Also, in the Change Connections dialog, the group name is shown as the interface to be connected to instead of listing every member of the group (except when connecting to a monitor). The following rules must be followed when setting this attribute:

- all interfaces in a group must have the same interface definition
- all interfaces in a group must have the same interface type
- provider interfaces must have MaxConsumers == 1

Examples

```
set_interface_attribute -instance AXI_Slave01 InterfaceGroup AXI_Slave
set_interface_attribute -instance AXI_Slave02 InterfaceGroup AXI_Slave
set_interface_attribute -instance AXI_Slave03 InterfaceGroup AXI_Slave
set_interface_attribute -instance AXI_Slave04 InterfaceGroup AXI_Slave
```

See Also

[API Index](#) [Command Reference](#)

InterfaceIsUsed

Indicates that the interface object is used on its parent interface instance.

Definition

Type: boolean

Flags: readOnly

Default value:

Valid on: param_____

Description

This attribute indicates the static result of the UsedOnInstance condition: An interface port or parameter is used if this attribute is true, otherwise not.

The differences to UsedOnInstance are: This attribute describes a boolean value, not an 'expr' expression, and has a static value inside coreAssembler for all coreKit interfaces.

The most important usage is within [find_item -filter "InterfaceIsUsed==1"], to select interface ports and parameters only which are used on the instance.

Examples

To find all used ports of the interface instance \$intf:

```
set ports [find_item -quiet \  
-type interfacePort \  
-filter InterfaceIsUsed==1 \  
[get_attribute $intf -attr Children] \  
]
```

See Also

[UsedOnInstance \(3\)](#)

InterfaceLabel

The label to be used in the display of interfaces/connections in the add component tree dialog

Definition

Type: string

Flags:

Default value:

Valid on:

Description

The label is used when showing connections in the add component tree or in a component inspector window. The label is prepended to the standard interface information, to generally its value should end with ": " if it is set.

Examples

To have consumers of a provider interface show up in the tree with the string "Target 0: " use a setting like this:

```
set_interface_attribute ProviderInterfaceName InterfaceLabel \  
{=set slot [eval_param -context %item @SlotNumber]  
if {$slot == -1} { format "" } else { format "Target %d: " $slot }  
}
```

See Also

InterfaceLink

Indicates the association to a design port or parameter (top-level core design). The design name can include @<paramName> and {<expr>} in the attribute value, and allows a [@<paramName>] index annotation.

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

This attribute specifies the association of an interface port or parameter to a design port or parameter.

The design name can include @<paramName> and {<expr>} in the attribute value, and allows a [@<paramName>] index annotation.

"\<open\>" is a reserved value used to specify that an interface parameter or port is not associated to any design parameter/port.

Examples

Example 1

If you defined an interface definition using the following scripts:

```
create_interface AHB-Slave
create_interface_parameter UsingExternalDecoder \
-interface AHB-Slave \
-type boolean \
-specify_on provider \
-default 0 \
-label "Is external decoder being used?" \
-description \
"Indicates that the internal decoder is NOT being used."
create_interface_port xhsel \
-interface AHB-Slave \
-direction fromProvider \
-separate \
-description \
"Select line from AHB decoder."
complete_interface_definition AHB-Slave
```

And you created an interface instance based on the above definition:

create_interface_instance -interface AHB-Slave -type provider AHB_Slave
You can set the interface parameter InterfaceLink to "<open>" using the following command:

```
coreBuilder> set_interface_parameter_attribute -instance AHB_Slave \  
UsingExternalDecoder InterfaceLink "<open>"
```

You can also set the interface port InterfaceLink attribute:

```
coreBuilder> set_interface_port_attribute -instance AHB_Slave xhsel \  
InterfaceLink "{@UsingExternalDecoder ? {x} : {}}sel"
```

Example 2

You defined another interface definition as below:

```
create_interface AHB-External-Decoder \  
-description \  
"AHB decode interface. Utilized only when an external decoder is desired"  
create_interface_port xhsel \  
-interface AHB-External-Decoder \  
-direction fromConsumer \  
-constant zero \  
-description \  
"Asserted to indicate that default slave should be selected."  
complete_interface_definition AHB-External-Decoder
```

And you created an interface instance based on the above definition:

```
create_interface_instance -interface AHB-External-Decoder -type provider  
\  
ExternalDecode
```

See Also

[create_interface_port \(2\)](#), [set_interface_port_attribute \(2\)](#)

InterfaceSize

Number of required bits on a design port associated with this interface port.

Definition

Type: string

Flags:

Default value: 1

Valid on:

Description

This attribute is used to indicate the number bits that are expected to be connected to on the design side. For provider connections, the interface port and design port are typically the same size, and that common size is typically the desired InterfaceSize. For consumer connections, the InterfaceSize is likely to match the size of the interface port, but may be smaller than the design port as there may be multiple consumers connected to different bits of the same design port.

This attribute is initialized by the '-size' option to the `create_interface_port` command. It can be modified in the Port Association dialog in coreBuilder.

Examples

To set the interface size of the interface myBus to match the (variable) size of the interface port:

```
prompt> set_interface_port_attribute myBus myPort InterfaceSize @width
```

To set the interface size for the same example to a fixed size of 4:

```
prompt> set_interface_port_attribute myBus myPort InterfaceSize 4
```

See Also

[BusAlignment \(3\)](#), [set_interface_port_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

InterfaceType

Indicates the interface type for the given interface.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Type of the interface could be: provider, consumer, monitor, internal-consumer, provider-consumer.

Examples

See Also

InterfaceTypeName

The customized name for each interface type.

Definition

Type: string

Flags: subscripted

Default value: Value calculated internally.

Valid subscripts: consumer consumer-monitor interface-monitor
internal-consumer provider provider-consumer

Default subscript:

Valid on:

Description

This attribute is used to assign a 'user friendly' name to the different types of interfaces instantiated from the given interface definition. Whenever a reference is made to a 'provider' instance of the interface definition, the string InterfaceTypeName[provider] is used instead of the string 'provider'. This works similarly for the other interface types. Typically it's only useful to set this attribute for the provider and consumer subscripts as the other values are automatically derived from the provider and consumer values. For example, instead of using the type 'monitor' directory, a reference to a monitor instantiation would use the string 'monitor for <value of InterfaceTypeName[consumer]>.

Examples

To set up 'user friendly' names for the AHB Master provider and consumer types:

```
prompt> set_interface_attribute AHB-Master \
{InterfaceTypeName[provider]} AHB
prompt> set_interface_attribute AHB-Master \
{InterfaceTypeName[consumer]} "AHB Master"
```

See Also

[InterfaceType \(3\)](#)

[API Index](#) [Command Reference](#)

InternalTristates

Determines the disabling option during scan shift for all tristate nets that do not drive output ports of a design. `disable_all` disables all drivers. `enable_one` disables all but one driver. `no_disabling` specifies not to insert disabling logic.

Definition

Type: string

Flags:

Default value: =InheritValue up `disable_all`

Valid on: `design`

Description

Determines the disabling option during scan shift for all tristate nets that do not drive output ports of a design. `disable_all` disables all drivers. `enable_one` disables all but one driver. `no_disabling` specifies not to insert disabling logic.

Examples

Specify that all internal tristates are to be disabled during scan shift.

```
coreBuilder> set_design_attribute InternalTristates disable_all
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#), [BidirectionalMode \(3\)](#), [ExternalTristates \(3\)](#)

[API Index](#) [Command Reference](#)

IsAddressable

Is true if the interface bus definition is addressable.

Definition

Type: boolean

Flags:

Default value: `:::sXML::getIsAddressable %item`

Valid on:

Description

This attribute is set on interface bus definitions if it is addressable.

Examples

See Also

IsArray

Indicates that the parameter is an array or not.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: param_____

Description

The coreTools support array parameters which are used to set the values of fields within a bitfield parameter. This attribute is used to indicate if the parameter is an array or not.

Use `define_array_field_parameters` to define each entry for an array parameter. Field parameters are defined based on the following attributes of the array parameter: `IsArray`, `ArrayStart`, `ArrayEnd`, `ArrayFieldSize`. Field parameters can be accessed by `get_field_parameter_for_array` and `foreach_array_field_parameter`.

Examples

See Also

[define_array_field_parameters \(2\)](#), [get_field_parameter_for_array \(2\)](#),
[foreach_array_field_parameter \(2\)](#), [IsArray \(3\)](#), [ArrayStart \(3\)](#), [ArrayEnd \(3\)](#), [ArrayFieldSize \(3\)](#)

[API Index](#) [Command Reference](#)

IsBehavioral

Indicates that the given port/net represents a behavioral connection.

Definition

Type: boolean

Flags:

Default value:

Valid on: net port _____

Description

Examples

See Also

IsCCDTarget

Concurrent Clock and Data (CCD) Optimization technology to enable users to meet frequency goals on high performance designs. When the attribute is set to true the clock is becomes part of the path groups for targeted concurrent clock and data (CCD) optimization. Refer to man page for 'ccd.targeted_ccd_path_groups' in Fusion Compiler reference documentation.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: clock_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

IsComplete

This activity is complete

Definition

Type: boolean**Flags:** readOnly**Default value:****Valid on:**

Description

IsComplete is a read-only attribute that indicates whether a coreTool activity is complete.

The coreTool automatically sets IsComplete to true on an activity after you complete that activity either through the GUI or by using the `autocomplete_activity` command. In the coreTool GUI Activity List, an activity is checked if the IsComplete attribute is true on that activity.

If the activity launches a subprocess, the activity may be in the complete state even if the subprocess is still active. For example, when the Synthesize activity launches Design Compiler to execute synthesis, the Synthesize activity is complete even though Design Compiler may be still running as a subprocess.

To list all activities and their current states (complete and/or enabled), use the `report_activities` command.

Examples

To determine if the Specify Port Intent activity is complete:

```
coreBuilder> get_attribute -attrs IsComplete SpecifyPortIntent  
1
```

See Also

[get_attribute \(2\)](#), [report_activities \(2\)](#), [IsEnabled \(3\)](#)

[API Index](#) [Command Reference](#)

IsConnected

Is the subsystem port, component pin, or interface connected?

Definition

Type: short

Flags: readOnly

Default value:

Valid on: busBit pin port _____

Description

This is a read-only attribute which indicates whether or not the given connection object is attached to something. This attribute is used to filter connections between those that are 'in use' and those that are not.

Legal values: 0 object is unconnected 1 object is partially connected (bussed pin/ports only) 2 object is fully connected

Examples

See Also

[API Index](#) [Command Reference](#)

IsControlValue

Indicates that the interface parameter value is used as control-only value. As a consequence the parameter value impacts interface setup but not the interface configuration.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: param_____

Description

This attribute is set with the "-control" argument of the `create_interface_parameter` command and specifies that the parameter is a control only parameter. See the manpage for `create_interface_parameter` for more information.

Examples

See Also

[create_interface_parameter\(2\)](#)

[API Index](#) [Command Reference](#)

IsDanglingIntfAllowed

Indicates whether an interface can be left without the corresponding consumer/provider.

Definition

Type: boolean

Flags:

Default value: 0

Valid on:

Description

Examples

See Also

[API Index](#) [Command Reference](#)

IsEnabled

This activity is enabled

Definition

Type: boolean**Flags:** readOnly**Default value:****Valid on:**

Description

IsEnabled is a read-only attribute that indicates whether a coreTool activity is enabled.

The coreTool automatically sets IsEnabled to true on an activity when all pre-requisite activities for that activity are complete. In the coreTool GUI Activity List, an activity is grayed out if the IsEnabled attribute is false on that activity.

To list all activities and their current states (complete and/or enabled), use the `report_activities` command.

Examples

To determine if the Specify Port Intent activity is enabled:

```
coreBuilder> get_attribute -attrs IsEnabled SpecifyPortIntent  
1
```

See Also

[get_attribute \(2\)](#), [report_activities \(2\)](#), [IsComplete \(3\)](#)

[API Index](#) [Command Reference](#)

IsExternalReset

The attribute is set to true when any subscript of ResetProperties is explicitly set.

Definition

Type: boolean

Flags: readOnly

Default value: =sCstr::getIsExternalReset %item

Valid on: port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

IsFullyModeled

Indicates that the design contents are fully modeled in coreTools.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

This attribute indicates whether the design has been modeled completely in coreTools. It is used to determine whether an design can be replaced with a hierarchical component in coreAssembler. The man page for CanUngroup has more information on what things are not modeled in coreTools.

Examples

See Also

[CanUngroup \(3\)](#), [CanUngroupInfo \(3\)](#)_____

IsHexVal

This value is in hexadecimal format

Definition

Type: boolean

Flags:

Default value:

Valid on: attrDefn _____

Description

The IsHexVal attribute indicates whether a parameter value is in hexadecimal format. If IsHexVal is true on a parameter, the coreTools automatically convert the user-specified value for the parameter to hexadecimal format before processing the parameter.

For example, if IsHexVal is true on a design parameter, and the user specifies the parameter value as 32 (decimal), coreConsultant automatically converts the value to "0x20" when generating the configured HDL code for the design.

If you want to force the user to specify the parameter value in hexadecimal format, set the SetInHex attribute value on the parameter to Required.

Examples

To specify that parameter Slave1_addr is in hexadecimal format:

```
coreBuilder> set_parameter_attribute Slave1_addr IsHexVal true
```

See Also

[set_parameter_attribute \(2\)](#), [SetInHex \(3\)](#) _____

[API Index](#) [Command Reference](#)

IsTriState

Is the port a tri-state?

Definition

Type: boolean

Flags:

Default value: =InheritValue up 0

Valid on: busBit port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

IsUpToDate

Is this deliverable up-to-date when compared to disk?

Definition

Type: boolean

Flags:

Default value: 1

Valid on: filegroup filegroupGroup

Description

For configurable and autoloading filegroups, this attribute returns true if the contents of the filegroup is up to date with respect to the disk contents.

For a configurable filegroup, the IsUpToDate attribute returns false if the set of intent files matched by the ConfigIntentSearchPath(3) attribute has changed since the last analyze_filegroup(2) command.

For an autoloading filegroup, the IsUpToDate attribute returns false if the set of files matched by the DefaultLoadPath(3) attribute has changed since the last load_autoload_filegroup(2) command.

Examples

To determine if the LoadDesigns activity should be recompleted:

```
coreConsultant> get_filegroup_attribute Synopsys/Hdl IsUpToDate
```

If this command returned false, then the filegroup could be repopulated with the following commands:

```
coreConsultant> set_activity_incomplete LoadDesigns
```

```
coreConsultant> autocompleate LoadDesigns
```

See Also

get_filegroup_attribute (2), create_autoload_filegroup (2),

set_activity_incomplete (2), unload_autoload_filegroup (2),

DefaultLoadPath (3), ConfigIntentSearchPath (3), IsUpToDate (3)

[API Index](#) [Command Reference](#)

IsWorkspaceComponent

Indicates the file used to import a workspace component.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: cell design _____

Description

Examples

See Also

[ImportedWorkspaceComponentFile \(3\)](#), [ReadOnlyWorkspaceComponent \(3\)](#),
[export_workspace as component \(2\)](#), [import_workspace as component \(2\)](#)

Item_Types Index

<u>addressBank</u>	This item represents an address bank in a memory map.
<u>addressBlock</u>	This item represents a block of memory in a memory map.
<u>attrDefn</u>	Definition of an attribute
<u>busBit</u>	Individual bit of a bus
<u>cell</u>	Cell item
<u>clock</u>	Clock item
<u>design</u>	Design item
<u>file</u>	File item
<u>filegroup</u>	File group item
<u>filegroupGroup</u>	Represents a collection of deliverables (filegroups) defined via a Group list in a BoM template.
<u>hdlFunc</u>	Internal model of a VHDL function
<u>interfaceDefn</u>	interfaceDefn item
<u>interfaceInstance</u>	interfaceInstance item
<u>interfacePort</u>	interfacePort item
<u>intfPin</u>	Represents an interface instance on a cell.
<u>intfPort</u>	Represents an interface instance on a design.
<u>knowledgeBase</u>	Knowledge database item
<u>memMap</u>	Describes a memory map in this component.
<u>net</u>	Net item
<u>param</u>	Parameter item
<u>pin</u>	Pin item
<u>port</u>	Port item
<u>refItem</u>	Defines a reference to a memory map in a component.
<u>register</u>	Describes a register within an address block.
<u>registerArray</u>	Describes a registerArray within an address block or register array.
<u>registerField</u>	Describes a field within a register.
<u>Strategy</u>	Item used to model a strategy (command script) to drive a design tool
<u>timingException</u>	Item used to model a timing exception command

[API Index](#) [Command Reference](#)

KbVersion

Knowledge base version identifier.

Definition

Type: string

Flags:

Default value:

Valid on: [knowledgeBase](#)

Description

Examples

See Also

Label

Label to be used for this item in GUI dialogs

Definition

Type: string

Flags:

Default value: Value calculated internally.

Valid on: Strategy attrDefn busBit cell clock design file filegroup

filegroupGroup hdlFunc knowledgeBase net param pin port

timingException

Description

The Label attribute determines the label string to be used in the coreTool GUI dialogs for the selected item.

Label is commonly used on design parameters or custom activity parameters to provide a meaningful label for the parameter in the associated parameter dialog. The default value for Label is the value of the Name attribute on the item.

For example, if a design has a parameter named "intrs" that specifies the number of interrupt inputs for a core, you can set the value of the Label attribute on the parameter to "Number of Interrupts" to create a more intuitive GUI dialog for the user. If you do not specify a value for the Label attribute, the label "intrs" will appear in the parameter dialog for the parameter.

Examples

To use the label "Number of Interrupts" in the GUI dialog where the user specifies the value for the intrs parameter:

```
coreBuilder> set_parameter_attribute intrs Label "Number of Interrupts"
```

See Also

[set_parameter_attribute \(2\), Name \(3\)](#)

[API Index](#) [Command Reference](#)

LicenseForOpen

The licensed used for opening the knowledgebase.

Definition

Type: string

Flags:

Default value:

Valid on: [knowledgeBase](#)

Description

Examples

See Also

[ProjectID \(3\)](#), [check_license \(2\)](#),

Load Consultant Plugins

This activity is used to create plugin KBs that are used by the core integrator of this IP. These KBs are added to the Plugins file group for installation in the consultant workspace. As a side-effect, all non-object Tcl files are sourced.

Parameters

Parameters Table

Name	Label	Description
Plugins		
PluginDirectories	Plugin directories	The list of directories to build plugin Kbs for. The default name of the plugin is the same as the name of the directory. If a different name is desired, then touch a file called [name].cfg to name the plugin [name].kb

See Also

Load Designs

This activity is used to get design data into coreBuilder. Currently the tool supports input from either VHDL, Verilog or SystemVerilog Hdl. The HDL files may contain special comment based annotations. After the HDL is analyzed, then Tcl (and SDC) based intent files are sourced.

Parameters

Parameters Table

Name	Label	Description
Advanced		
StrictLanguageSemantics	Strict Language Semantics	Enforce strict language semantics. When false, commonly supported non-compliant language semantics are accepted without error.
PreventDesignPrefix	PreventDesignPrefix	Prevent insertion of design and cell prefixes in the source code.
SkipDesigns	Skip Designs	List of patterns that indicate which designs from the RTL should be skipped. A pattern is a glob-style expression. If a pattern starts with '!', then a design must match that pattern, otherwise it is skipped. It is possible to have multiple patterns listed. ActivityAliasNames : SkipDesignPatterns
PreventSVPrefix	PreventSVPrefix	Prevent insertion of the design prefix for SystemVerilog interface and package names in the configured source code.
SkipPrefixNames	Skip Prefix Names	List of names and patterns to indicate modules and SystemVerilog interfaces and packages to not prefix. A pattern is a glob-style expression. If a pattern

coreTools Command Reference Index

Name	Label	Description
		starts with '!', then only names that match the pattern are prefixed.
AutoReadDCSupported	Autoread Supported?	Allows usage of autoread mode in DC to analyze the RTL files given that the RTL code is autoread compatible. The files should have file structure where everything kept under one top directory
SourceLicenseString	Source License	If this is set to a non-empty string, then coreBuilder will encrypt the source files when writing them, unless the specified license is available. This option is only available within Synopsys. ActivityAliasNames : UnencryptLicenseString
IntentSearchPath	Tcl intent search path	The list of directories to search in when looking for Tcl (and SDC) intent files. Intent files should be named with the same basename as an HDL file, with a '.tcl' (or '.sdc') extension. If the extension is '.elab.tcl' (or '.elab.sdc'), then the file will be sourced after configuration/elaboration in the coreBuilder flow.
ExtraLinkLibs	Extra link libraries	The list of additional libraries to be linked in - hard macros, timing shells, etc. Worst case/best case timing libraries may be specified using a list of lists. Assume lib1 is to be used for all conditions while lib2 is to be used for worst case analysis and lib2Min for best case analysis. In batch mode this would be specified as:

coreTools Command Reference Index

Name	Label	Description
		set_activity_parameter LoadDesigns ExtraLinkLibs lib1.db {lib2.db lib2Min.db} lib3.db In the gui lib1.db and lib3.db should be placed on lines by themselves, and lib2.db and lib2Min.db should be placed on the same line.
Options		
InputLanguage	Input language	Chooses the HDL input language EnumValues: VHDL Verilog SystemVerilog
HonorTranslatePragmas	Honor Translate Pragmas	Honor translate on/off pragmas when compiling RTL files.
CompileDefines	Compile Defines	Defines to pass to HDL source compiler, e.g. 'DEF1 DEF2=10'.
VCSOptions	VCS Options	Additional command line options for VCS.
LedaConfigFile	Leda Config File	Leda configuration filename.
TopDesignName	Top Design Name	Indicates the name of the top design. Can be calculated if there is only one design which does not have any references (instances). NOTE: In coreAssembler, this must be specified explicitly.
Options/SystemVerilog		
SystemVerilogVersion	SystemVerilog Version	The SystemVerilog version used to analyze the input files. - EnumValues: 1800-2005 1800-2009 1800-2012 - Symbolic Names: 1800-2005 1800-2009 1800-2012
Options/VHDL		
VhdlVersion	VHDL Version	The VHDL version used to analyze the input files.

coreTools Command Reference Index

Name	Label	Description
		- EnumValues: VHDL93 VHDL87 - Symbolic Names: VHDL-93 VHDL-87
VhdlLibrary	Default Library	The default library to analyze VHDL files into. See the help on InputFileNames for multi-library support
Options/Verilog		
VerilogVersion	Verilog Version	The Verilog version used to analyze the input files. - EnumValues: Verilog95 Verilog01 Verilog05 - Symbolic Names: Verilog-95 Verilog-2001 Verilog-2005
IncludePath	Include File Directories	List of directories to search in for 'include files. Only those directories where 'included files are actually found will be saved as 'include directories' for the coreKit.
SourceFiles		
InputFileNames	HDL Files	The list of all HDL files (glob-style) that should be analyzed. This list should be in the proper analysis order. For multi-language and multi-library designs, entries of the form 'language [language]' and 'library [library]' can also be 'included in the file list. All files following the library or language line are assumed to belong to that library or language.
InputStrategy	HDL file location strategy	Indicates the strategy to use to locate HDL files. EnumValues: File Directory coreDeveloper
InputStrategyArguments	Arguments to input strategy procedure	Arguments to be passed to the input strategy procedure.
AnalyzeSubsetOfFiles		

coreTools Command Reference Index

Name	Label	Description
	Analyze Subset of Files	When true, indicates that not all of the HDL files should be analyzed. Only those files specified via the FilesToBeAnalyzed parameter in batch mode or those entered into the list control labeled 'HDL Files To Be Analyzed' in GUI mode will actually be analyzed by the tool.
FilesToBeAnalyzed	HDL Files To Be Analyzed	List of files to be analyzed during LoadDesigns. By default this parameter has the same value as the InputFileNames parameter (i.e. all HDL files are analyzed). If some of the files should not be analyzed (for performance or other reasons), check the 'Analyze Subset of Files' check-box (or set the AnalyzeSubsetOfFiles parameter in batch mode) and specify the files to be analyzed via this parameter, which is labeled 'HDL Files To Be Analyzed' in the graphical interface.

See Also

[API Index](#) [Command Reference](#)

Load Interface Definitions

Load definitions for each interface to be instantiated in the top-level of this core for automated assembly within coreAssembler.

Parameters

Parameters Table

Name	Label	Description
Interfaces		
InterfaceDefinitionFileNames	Interface Definition File Names	List of TCL source files containing interface definition commands.

See Also

LockedInTemplate

Indicate if the parameter/cell/interfaceinstance should be locked in template

Definition

Type: boolean

Flags:

Default value: 0

Valid on: cell param _____

Description

This attribute is used by platform designer to enforce certain platform requirements in coreAssembler. It indicates if an item is locked. If a cell or exported interface instance is locked, the platform user could not remove the locked item. If a interface or configuration parameter is locked, the platform user could not change the value of the locked parameter.

Examples

See Also

LogicalLeft

Indicates the logical left bit index of the interface port to be connected.

Definition

Type: short

Flags:

Default value:

Valid on:

Description

This attribute is part of a set of attributes used to indicate which bits of a physical port are associated with which bits of the logical port in the bus interface. This attribute corresponds to the value from the IP-XACT port mapping element portMaps/portMap/logicalPort/left which indicates the left index of the logical port to be mapped to within a bus interface port map.

This attribute is only applicable in coreBuilder if the interface instance was defined using an IP-XACT bus definition and abstraction definition. Otherwise InterfaceSize is the appropriate attribute to set.

Examples

See Also

[InterfaceLink](#) [InterfaceSize](#) [LogicalRight](#) [PhysicalLeft](#) [PhysicalRight](#)

[API Index](#) [Command Reference](#)

LogicalName

Indicates the logical (abstract) name for this port/pin. Used to enable connections with matching LogicalName values.

Definition

Type: string

Flags:

Default value:

Valid on: busBit pin port _____

Description

Examples

See Also

LogicalRight

Indicates the logical right bit index of the interface port to be connected.

Definition

Type: short

Flags:

Default value:

Valid on:

Description

This attribute is part of a set of attributes used to indicate which bits of a physical port are associated with which bits of the logical port in the bus interface. This attribute corresponds to the value from the IP-XACT port mapping element portMaps/portMap/logicalPort/right which indicates the right index of the logical port to be mapped to within a bus interface port map.

This attribute is only applicable in coreBuilder if the interface instance was defined using an IP-XACT bus definition and abstraction definition. Otherwise InterfaceSize is the appropriate attribute to set.

Examples

See Also

[InterfaceLink](#) [InterfaceSize](#) [LogicalLeft](#) [PhysicalLeft](#) [PhysicalRight](#)

[API Index](#) [Command Reference](#)

MacroPrefixFileList

User specified list of files which need to be processed for macro prefixing. The paths should correspond to the installation path in the workspace.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

Examples

See Also

MapBlockIndividually

Map this design individually in Design Compiler. Note that this attribute does not control mapping for PSYN strategies.

Definition

Type: boolean

Flags:

Default value: =expr { [InheritValue up -attr
MapSubblocksIndividually 0] || (![get_attribute %item -attr
Uniquify] && ![get_attribute %item -attr Ungroup]) }

Valid on: design_____

Description

The MapBlockIndividually attribute determines whether to map a design individually or to map the design with other designs as part of the compile of the containing design. If MapBlockIndividually is true on a design, coreConsultant generates a synthesis that compiles that design as an individual unit. If MapBlockIndividually is false on a design, coreConsultant generates a synthesis that compiles that design as part together with other designs as part of the compile of the next higher-level design.

To compile all subblocks of a design individually, you do not have to explicitly set MapBlockIndividually to true on each subblock. Instead, you can just set MapSubblocksIndividually to true on the higher-level design.

If you do not specify a value for MapBlockIndividually, coreConsultant automatically sets MapBlockIndividually to the same value as the MapSubblocksIndividually attribute (if set) on the containing design. If MapSubblocksIndividually is not set on the containing design, then MapBlockIndividually defaults to false (0).

Examples

To compile Subblock_A and Subblock_B of My_Core individually and then compile the remaining subblocks of My_Core together as part of the My_Core compile:

```
coreConsultant> current_design My_Core
coreConsultant> set_design_attribute MapSubblocksIndividually false
coreConsultant> set_design_attribute -designs
{Subblock_A Subblock_B} MapBlockIndividually true
```

See Also

set_design_attribute (2), MapSubblocksIndividually (3) PhysicalRegion (3)

MapSubblocksIndividually

Map this design by individually compiling its subdesigns.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

The MapSubblocksIndividually attribute determines whether to map the selected design through individual compiles of its subblocks.

If MapSubblocksIndividually is true on a design, coreConsultant generates a synthesis strategy that compiles each of the design's subblocks individually, with the exception of any subblocks that have the MapBlockIndividually attribute explicitly set to false. Those subblocks on which MapBlockIndividually is false are mapped together as part of the compile of the containing design.

If MapSubblocksIndividually is false, coreConsultant generates a synthesis strategy that maps the selected design without individually compiling its subblocks, with the exception of any subblocks that have the MapBlockIndividually attribute explicitly set to true. Those subblocks on which MapBlockIndividually is true are mapped individually. Those subblocks on which MapBlockIndividually is false or is not set are mapped together as part of the compile of the containing design.

The set of subblocks in the design hierarchy that are to be compiled individually define the implementation frontier of the design. The implementation frontier is the lowest level set of subblocks that are to be compiled as individual units. In general, a lower implementation frontier results in faster compiles. However, the quality of synthesis results depends on how well the design budgets are set on the lower level subblocks.

Examples

To compile the subblocks of My_Core individually:

```
coreConsultant> current_design My_Core
coreConsultant> set_design_attribute MapSubblocksIndividually true
```

See Also

set_design_attribute (2), MapBlockIndividually (3)

MaxArea

Maximum area constraint for current design.

Definition

Type: float

Flags:

Default value:

Valid on: design_____

Description

The MaxArea attribute specifies the maximum area constraint for a design. coreConsultant uses MaxArea as a value for the Design Compiler set_max_area command.

You can specify the MaxArea value with or without units. If you do not specify an area unit, the default area unit is the area unit used by the currently loaded technology library. If you do specify an area unit, coreConsultant automatically scales the area value to the area unit used by the currently loaded technology library.

As a core developer, you specify MaxArea in coreBuilder in terms of nand2 cell units. As a core integrator, you specify MaxArea in coreConsultant in terms of the area unit used by your target technology library.

To get the smallest possible design, set MaxArea to 0. To set a more realistic area goal, set MaxArea to the desired cell area for the design.

In most cases, you do not need to set MaxArea explicitly. coreConsultant automatically sets MaxArea to 0 on a design if OptimizationPriorities is set to anything other than "timing" on that design.

Examples

To set the maximum area constraint for the current_design to 8000 nand2 cell units:

```
coreBuilder> set_design_attribute MaxArea 8000
```

To optimize the current_design for the smallest possible area:

```
coreConsultant> set_design_attribute MaxArea 0.0
```

See Also

[set_design_attribute \(2\)](#), [AreaEstimate \(3\)](#), [OptimizationPriorities \(3\)](#)

[API Index](#) [Command Reference](#)

MaxCap

Maximum capacitance value for a port or design.

Definition

Type: float**Flags:****Default value:** =InheritValue up {}**Valid on:** busBit design port _____

Description

The MaxCap attribute specifies a maximum capacitance value for the selected input or output port. coreConsultant uses MaxCap directly as a value for the Design Compiler set_max_capacitance command.

You can specify the MaxCap value with or without units. If you do not specify a capacitance unit, the default capacitance unit is the capacitance unit used by the currently loaded technology library. If you do specify a capacitance unit (for example, 5pf), coreConsultant automatically scales the time value to the capacitance unit used by the currently loaded technology library. For example, if you specify 5pf and the currently loaded technology library uses ff as the capacitance unit, coreConsultant automatically scales 5pf to 5,000ff.

Examples

To set the maximum capacitance on the int0 port to 10 picofarads:

```
coreConsultant> set_port_attribute int0 MaxCap 10pf
```

See Also

[set_port_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

MaxControlPoints

Maximum number of control points to be inserted to enhance testability.
When TestabilityMethod==control_and_observe for XG mode the larger of
MaxObservePoints and MaxControlPoints will be used for the
-max_test_points value for set_testability_configuration.

Definition

Type: long

Flags:

Default value: 1000

Valid on: design_____

Description

Maximum number of control points to be inserted to enhance testability.

Examples

Disable insertion of control points.

```
coreBuilder> set_design_attribute MaxControlPoints 0
```

See Also

[set_design_attribute \(2\)](#), [InsertTestPoints \(3\)](#), [MaxObservePoints \(3\)](#) _____

MaxFallInputDelay

Maximum falling edge delay constraint for an input port

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port

Description

The MaxFallInputDelay attribute specifies the maximum falling edge input delay for the input port with respect to the rising edge of the specified clock. The MaxFallInputDelay value is the maximum propagation delay of a falling edge of the incoming signal from the rising edge of the specified clock to the selected input port on the current design.

The subscript to the MaxFallInputDelay attribute is the name of the clock. For example, MaxFallInputDelay[clk] is the maximum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxFallInputDelay. The default subscript is the single existing clock in the design. To set both the maximum rise and fall input delay values to the same value you can use the MaxInputDelay attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxFallInputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxFallInputDelay, the port inherits the MaxFallInputDelay value from the connected port on the next higher level design.

To specify maximum input delay relative to a falling clock edge, use the MaxFallInputDelayFallingEdge attribute instead of MaxFallInputDelay.

Examples

To set the maximum falling edge input delay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {MaxFallInputDelay[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MaxFallInputDelay
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MaxRiseInputDelay (3), _____
MinRiseInputDelay (3), MinFallInputDelay (3), MaxInputDelay (3), _____
MaxFallInputDelayFallingEdge (3) _____

MaxFallInputDelayFallingEdge

Maximum falling edge delay constraint for an input port relative to a falling clock edge

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MaxFallInputDelayFallingEdge attribute specifies the maximum falling edge input delay for the input port with respect to the falling edge of the specified clock. The MaxFallInputDelayFallingEdge value is the maximum propagation delay of a falling edge of the incoming signal from the falling edge of the specified clock to the selected input port on the current design.

The subscript to the MaxFallInputDelayFallingEdge attribute is the name of the clock. For example, MaxFallInputDelayFallingEdge[clk] is the maximum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxFallInputDelayFallingEdge. The default subscript is the single existing clock in the design. To set both the maximum rise and fall input delay values to the same value you can use the MaxInputDelayFallingEdge attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxFallInputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

{=percent_of_period <percentage> [<clock_name>]}

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxFallInputDelayFallingEdge, the port inherits the MaxFallInputDelayFallingEdge value from the connected port on the next higher level design.

To specify maximum input delay relative to a rising clock edge, use the `MaxFallInputDelay` attribute instead of `MaxFallInputDelayFallingEdge`.

Examples

To set the maximum falling edge input delay on the `data_in` port to 15 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_in
{MaxFallInputDelayFallingEdge[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MaxFallInputDelayFallingEdge
{=percent_of_period 15.0}
```

See Also

[percent_of_period \(2\)](#), [set_port_attribute \(2\)](#)

[MaxRiseInputDelayFallingEdge \(3\)](#), [MinRiseInputDelayFallingEdge \(3\)](#)

[MinFallInputDelayFallingEdge \(3\)](#), [MaxInputDelayFallingEdge \(3\)](#)

[MaxFallInputDelay \(3\)](#)

MaxFallOutputDelay

Maximum falling edge delay constraint for an output port

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MaxFallOutputDelay attribute specifies the maximum falling edge output delay for the output port with respect to the rising edge of the specified clock. The MaxFallOutputDelay value is the maximum propagation delay of a falling edge of the incoming signal from the rising edge of the specified clock to the selected output port on the current design.

The subscript to the MaxFallOutputDelay attribute is the name of the clock. For example, MaxFallOutputDelay[clk] is the maximum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxFallOutputDelay. The default subscript is the single existing clock in the design. To set both the maximum rise and fall output delay values to the same value you can use the MaxOutputDelay attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxFallOutputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxFallOutputDelay, the port inherits the MaxFallOutputDelay value from the connected port on the next higher level design.

To specify maximum output delay relative to a falling clock edge, use the MaxFallOutputDelayFallingEdge attribute instead of MaxFallOutputDelay.

Examples

To set the maximum falling edge output delay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {MaxFallOutputDelay[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MaxFallOutputDelay
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MaxRiseOutputDelay (3),
MinRiseOutputDelay (3), MinFallOutputDelay (3), MaxOutputDelay (3),
MaxFallOutputDelayFallingEdge (3)

MaxFallOutputDelayFallingEdge

Maximum falling edge delay constraint for an output port relative to a falling clock edge

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MaxFallOutputDelayFallingEdge attribute specifies the maximum falling edge output delay for the output port with respect to the falling edge of the specified clock. The MaxFallOutputDelayFallingEdge value is the maximum propagation delay of a falling edge of the incoming signal from the falling edge of the specified clock to the selected output port on the current design.

The subscript to the MaxFallOutputDelayFallingEdge attribute is the name of the clock. For example, MaxFallOutputDelayFallingEdge[clk] is the maximum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxFallOutputDelayFallingEdge. The default subscript is the single existing clock in the design. To set both the maximum rise and fall output delay values to the same value you can use the MaxOutputDelayFallingEdge attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxFallOutputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

{=percent_of_period <percentage> [<clock_name>]}

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxFallOutputDelayFallingEdge, the port inherits the MaxFallOutputDelayFallingEdge value from the connected port on the next higher level design.

To specify maximum output delay relative to a rising clock edge, use the `MaxFallOutputDelay` attribute instead of `MaxFallOutputDelayFallingEdge`.

Examples

To set the maximum falling edge output delay on the `data_in` port to 15 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_in  
{MaxFallOutputDelayFallingEdge[clk]}  
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MaxFallOutputDelayFallingEdge  
{=percent_of_period 15.0}
```

See Also

[percent_of_period \(2\), set_port_attribute \(2\),](#)
[MaxRiseOutputDelayFallingEdge \(3\), MinRiseOutputDelayFallingEdge \(3\),](#)
[MinFallOutputDelayFallingEdge \(3\), MaxOutputDelayFallingEdge \(3\),](#)
[MaxFallOutputDelay \(3\)](#)

MaxFallTransitionDelay

Maximum transition fall time for an ideal clock.

Definition

Type: float

Flags:

Default value:

Valid on: clock_____

Description

The MaxFallTransitionDelay attribute specifies the maximum falling edge transition time for the specified ideal clock. MaxFallTransitionDelay does not apply to propagated clocks because Design Compiler uses the calculated transition delay for propagated clocks.

coreConsultant uses the MaxFallTransitionDelay attribute to generate the clock transition value for Design Compiler to use for maximum delay analysis.

The primary use for MaxFallTransitionDelay is to force the maximum transition delay to 0 for generated clocks. Non-generated clocks have a transition delay of 0 by default.

Examples

To use 0 maximum falling edge transition delay for the generated clock named int_clk:

```
coreConsultant> set_clock_attribute int_clk MaxFallTransitionDelay 0
```

See Also

[set_clock_attribute \(2\)](#), [MinFallTransitionDelay \(3\)](#), _____

[MaxRiseTransitionDelay \(3\)](#), [MinRiseTransitionDelay \(3\)](#), _____

MaxFanout

Maximum fanout for an input port or a design.

Definition

Type: float

Flags:

Default value: =InheritValue up {}

Valid on: busBit design port _____

Description

The MaxFanout attribute specifies a maximum fanout value for the selected input port or design. coreConsultant uses MaxFanout directly as the value for the Design Compiler set_max_fanout command for the specified input port or design.

If you set MaxFanout on an input port, Design Compiler attempts to ensure that the sum of all fanout_load attributes on the input pins of all nets that are driven by the specified input port is less than the MaxFanout value on the input port.

If you set MaxFanout on a design, Design Compiler attempts to ensure that the sum of all fanout_load attributes on all input pins of each net in the design is less than the MaxFanout value on the design.

If you specify MaxFanout on both a design and an input port of that design, Design Compiler attempts to meet the more restrictive value. Design Compiler also attempts to meet the more restrictive value of the MaxFanout attribute or the Maximum Fanout design rule constraint of the target technology library.

If you do not specify MaxFanout on an input port of a design, the input port inherits the MaxFanout value from the connected port on the next higher level design. If you do not specify MaxFanout on a design, the design inherits the MaxFanout value from the next higher level design.

Note: Fusion compiler has no support for the "set_max_fanout" command. Hence, when the synthesis strategy is set to Fusion Compiler Reference Methodology, the tool does not write the maxFanout command in sdc file. Instead, the tool automatically handles it by setting the app options in the init_design phase. So, the tool writes the following commands in init_design.tcl:

```
set_app_options -name opts.common.max_fanout -value <MaxFanoutValue>
set_app_options -name cts.common.max_fanout -value <MaxFanoutValue>
```

Examples

To set the maximum fanout of the req_n input port to 8:

```
coreConsultant> set_port_attribute req_n MaxFanout 8
```

To set the maximum fanout for input pins of all nets in the current_design to 6:

```
coreConsultant> set_design_attribute MaxFanout 6
```

See Also

[set_port_attribute \(2\), set_design_attribute \(2\)](#)

MaxInputDelay

Maximum delay constraint for an input port

Definition

Type: float

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: busBit port _____

Description

The MaxInputDelay attribute specifies the maximum input delay for the input port with respect to the rising edge of the specified clock. The MaxInputDelay value is the maximum propagation delay of the incoming signal from the rising edge of the specified clock to the selected input port on the current design. Setting this attribute is equivalent to setting both MaxRiseInputDelay and MaxFallInputDelay to the same value.

The subscript to the MaxInputDelay attribute is the name of the clock. For example, MaxInputDelay[clk] is the maximum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxInputDelay. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxInputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxInputDelay, the port inherits the MaxInputDelay value from the connected port on the next higher level design.

To specify maximum input delay relative to a falling clock edge, use the MaxInputDelayFallingEdge attribute instead of MaxInputDelay.

Examples

To set the maximum input delay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {MaxInputDelay[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MaxInputDelay
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MinInputDelay (3),
MaxRiseInputDelay (3), MaxFallInputDelay (3), InputDelay (3),
MaxInputDelayFallingEdge (3)

MaxInputDelayFallingEdge

Maximum delay constraint for an input port (relative to the falling clock edge)

Definition

Type: float

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: busBit port _____

Description

The MaxInputDelayFallingEdge attribute specifies the maximum input delay for the input port with respect to the falling edge of the specified clock. The MaxInputDelayFallingEdge value is the maximum propagation delay of the incoming signal from the falling edge of the specified clock to the selected input port on the current design. Setting this attribute is equivalent to setting both MaxRiseInputDelayFallingEdge and MaxFallInputDelayFallingEdge to the same value.

The subscript to the MaxInputDelayFallingEdge attribute is the name of the clock. For example, MaxInputDelayFallingEdge[clk] is the maximum input delay with respect to the falling edge of clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxInputDelayFallingEdge. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxInputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxInputDelayFallingEdge, the port inherits the MaxInputDelayFallingEdge value from the connected port on the next higher level design.

To specify maximum input delay relative to a rising clock edge, use the MaxInputDelay attribute.

Examples

To set the maximum input delay on the data_in port, relative to the falling edge of the clk signal, to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in
{MaxInputDelayFallingEdge[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in
MaxInputDelayFallingEdge
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2),

MaxRiseInputDelayFallingEdge (3), MaxFallInputDelayFallingEdge (3),

MaxInputDelay (3), InputDelay (3), MinInputDelayFallingEdge (3)

[API Index](#) [Command Reference](#)

MaxLoads

Maximum number of loads for this connection.

Definition

Type: short

Flags: subscribed

Default value: 9999

Valid subscripts: pin port

Default subscript: port

Valid on: pin port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

MaxObservePoints

Maximum number of observe points to be inserted to enhance testability.
When TestabilityMethod==control_and_observe for XG mode the larger of
MaxObservePoints and MaxControlPoints will be used for the
-max_test_points value for set_testability_configuration.

Definition

Type: long

Flags:

Default value: 1000

Valid on: design_____

Description

Maximum number of observe points to be inserted to enhance testability.

Examples

Disable insertion of observe points.

```
coreBuilder> set_design_attribute MaxObservePoints 0
```

See Also

[set_design_attribute \(2\)](#), [InsertTestPoints \(3\)](#), [MaxControlPoints \(3\)](#)

MaxOutputDelay

Maximum delay constraint for an output port

Definition

Type: float**Flags:** subscripted**Default value:****Valid subscripts:** <valid subscripts are not statically defined>**Default subscript:****Valid on:** busBit port _____

Description

The MaxOutputDelay attribute specifies the maximum output delay for the output port with respect to the rising edge of the specified clock. The MaxOutputDelay value is the maximum amount of time that the outgoing signal must be available at the output port before the next rising clock edge. In most cases, MaxOutputDelay represents the longest combinational path delay from the design's output port to a register that receives the signal, plus that register's library setup time. Setting this attribute is equivalent to setting both MaxRiseOutputDelay and MaxFallOutputDelay to the same value.

The subscript to the MaxOutputDelay attribute is the name of the clock. For example, MaxOutputDelay[clk] is the maximum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxOutputDelay. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxOutputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxOutputDelay, the port inherits the MaxOutputDelay value from the connected port on the next higher level design.

To specify maximum output delay relative to a falling clock edge, use the MaxOutputDelayFallingEdge attribute instead of MaxOutputDelay.

Examples

To set the maximum output delay on the data_out port to 20 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_out
{MaxOutputDelay[clk]}
{=percent_of_period 20.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_out MaxOutputDelay
{=percent_of_period 20.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MinOutputDelay (3), _____
MaxRiseOutputDelay (3), MaxFallOutputDelay (3), OutputDelay (3), _____
MaxOutputDelayFallingEdge (3)_____

MaxOutputDelayFallingEdge

Maximum delay constraint for an output port (relative to the falling clock edge)

Definition

Type: float

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: busBit port _____

Description

The MaxOutputDelayFallingEdge attribute specifies the maximum output delay for the output port with respect to the falling edge of the specified clock. The MaxOutputDelayFallingEdge value is the maximum amount of time that the outgoing signal must be available at the output port before the next falling clock edge. In most cases, MaxOutputDelay represents the longest combinational path delay from the design's output port to a register that receives the signal, plus that register's library setup time. Setting this attribute is equivalent to setting both MaxRiseOutputDelayFallingEdge and MaxFallOutputDelayFallingEdge to the same value.

The subscript to the MaxOutputDelayFallingEdge attribute is the name of the clock. For example, MaxOutputDelayFallingEdge [clk] is the maximum output delay with respect to the falling edge of clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxOutputDelayFallingEdge. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxOutputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxOutputDelayFallingEdge, the port inherits the MaxOutputDelayFallingEdge value from the connected port on the next higher level design.

To specify maximum output delay relative to a rising clock edge, use the `MaxOutputDelay` attribute.

Examples

To set the maximum output delay on the `data_out` port, relative to the falling edge of the `clk` signal, to 20 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_out
{MaxOutputDelayFallingEdge[clk]}
{=percent_of_period 20.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_out
MaxOutputDelayFallingEdge
{=percent_of_period 20.0}\fP
```

See Also

percent_of_period (2), set_port_attribute (2), MaxOutputDelay (3),
MaxRiseOutputDelayFallingEdge (3), MaxFallOutputDelayFallingEdge (3),
OutputDelay (3), MinOutputDelayFallingEdge (3)

MaxRiseInputDelay

Maximum rising edge delay constraint for an input port

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MaxRiseInputDelay attribute specifies the maximum rising edge input delay for the input port with respect to the rising edge of the specified clock. The MaxRiseInputDelay value is the maximum propagation delay of a rising edge of the incoming signal from the rising edge of the specified clock to the selected input port on the current design.

The subscript to the MaxRiseInputDelay attribute is the name of the clock. For example, MaxRiseInputDelay[clk] is the maximum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxRiseInputDelay. The default subscript is the single existing clock in the design. To set both the maximum rise and fall input delay values to the same value you can use the MaxInputDelay attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxRiseInputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxRiseInputDelay, the port inherits the MaxRiseInputDelay value from the connected port on the next higher level design.

To specify maximum input delay relative to a falling clock edge, use the MaxRiseInputDelayFallingEdge attribute instead of MaxRiseInputDelay.

Examples

To set the maximum rising edge input delay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {MaxRiseInputDelay[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MaxRiseInputDelay
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MaxFallInputDelay (3), _____
MinRiseInputDelay (3), MinFallInputDelay (3), MaxInputDelay (3), _____
MaxRiseInputDelayFallingEdge (3) _____

MaxRiseInputDelayFallingEdge

Maximum rising edge delay constraint for an input port relative to a falling clock edge

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MaxRiseInputDelayFallingEdge attribute specifies the maximum rising edge input delay for the input port with respect to the falling edge of the specified clock. The MaxRiseInputDelayFallingEdge value is the maximum propagation delay of a rising edge of the incoming signal from the falling edge of the specified clock to the selected input port on the current design.

The subscript to the MaxRiseInputDelayFallingEdge attribute is the name of the clock. For example, MaxRiseInputDelayFallingEdge[clk] is the maximum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxRiseInputDelayFallingEdge. The default subscript is the single existing clock in the design. To set both the maximum rise and fall input delay values to the same value you can use the MaxInputDelayFallingEdge attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxRiseInputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxRiseInputDelayFallingEdge, the port inherits the MaxRiseInputDelayFallingEdge value from the connected port on the next higher level design.

To specify maximum input delay relative to a rising clock edge, use the `MaxRiseInputDelay` attribute instead of `MaxRiseInputDelayFallingEdge`.

Examples

To set the maximum rising edge input delay on the `data_in` port to 15 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_in
{MaxRiseInputDelayFallingEdge[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MaxRiseInputDelayFallingEdge
{=percent_of_period 15.0}
```

See Also

[percent_of_period \(2\)](#), [set_port_attribute \(2\)](#)

[MaxFallInputDelayFallingEdge \(3\)](#), [MinRiseInputDelayFallingEdge \(3\)](#)

[MinFallInputDelayFallingEdge \(3\)](#), [MaxInputDelayFallingEdge \(3\)](#)

[MaxRiseInputDelay \(3\)](#)

MaxRiseOutputDelay

Maximum rising edge delay constraint for an output port

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up { }

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MaxRiseOutputDelay attribute specifies the maximum rising edge output delay for the output port with respect to the rising edge of the specified clock. The MaxRiseOutputDelay value is the maximum propagation delay of a rising edge of the incoming signal from the rising edge of the specified clock to the selected output port on the current design.

The subscript to the MaxRiseOutputDelay attribute is the name of the clock. For example, MaxRiseOutputDelay[clk] is the maximum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxRiseOutputDelay. The default subscript is the single existing clock in the design. To set both the maximum rise and fall output delay values to the same value you can use the MaxOutputDelay attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxRiseOutputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxRiseOutputDelay, the port inherits the MaxRiseOutputDelay value from the connected port on the next higher level design.

To specify maximum output delay relative to a falling clock edge, use the MaxRiseOutputDelayFallingEdge attribute instead of MaxRiseOutputDelay.

Examples

To set the maximum rising edge output delay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {MaxRiseOutputDelay[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MaxRiseOutputDelay
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MaxFallOutputDelay (3),
MinRiseOutputDelay (3), MinFallOutputDelay (3), MaxOutputDelay (3),
MaxRiseOutputDelayFallingEdge (3)

MaxRiseOutputDelayFallingEdge

Maximum rising edge delay constraint for an output port relative to a falling clock edge

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MaxRiseOutputDelayFallingEdge attribute specifies the maximum rising edge output delay for the output port with respect to the falling edge of the specified clock. The MaxRiseOutputDelayFallingEdge value is the maximum propagation delay of a rising edge of the incoming signal from the falling edge of the specified clock to the selected output port on the current design.

The subscript to the MaxRiseOutputDelayFallingEdge attribute is the name of the clock. For example, MaxRiseOutputDelayFallingEdge[clk] is the maximum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MaxRiseOutputDelayFallingEdge. The default subscript is the single existing clock in the design. To set both the maximum rise and fall output delay values to the same value you can use the MaxOutputDelayFallingEdge attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MaxRiseOutputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

{=percent_of_period <percentage> [<clock_name>]}

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MaxRiseOutputDelayFallingEdge, the port inherits the MaxRiseOutputDelayFallingEdge value from the connected port on the next higher level design.

To specify maximum output delay relative to a rising clock edge, use the `MaxRiseOutputDelay` attribute instead of `MaxRiseOutputDelayFallingEdge`.

Examples

To set the maximum rising edge output delay on the `data_in` port to 15 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_in
{MaxRiseOutputDelayFallingEdge[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MaxRiseOutputDelayFallingEdge
{=percent_of_period 15.0}
```

See Also

[percent_of_period \(2\)](#), [set_port_attribute \(2\)](#)

[MaxFallOutputDelayFallingEdge \(3\)](#), [MinRiseOutputDelayFallingEdge \(3\)](#)

[MinFallOutputDelayFallingEdge \(3\)](#), [MaxOutputDelayFallingEdge \(3\)](#)

[MaxRiseOutputDelay \(3\)](#)

[API Index](#) [Command Reference](#)

MaxRiseTransitionDelay

Maximum transition rise time for an ideal clock.

Definition

Type: float**Flags:****Default value:****Valid on:** clock_____

Description

The MaxRiseTransitionDelay attribute specifies the maximum rising edge transition time for the specified ideal clock. MaxRiseTransitionDelay does not apply to propagated clocks because Design Compiler uses the calculated transition delay for propagated clocks.

coreConsultant uses the MaxRiseTransitionDelay attribute to generate the clock transition value for Design Compiler to use for maximum delay analysis.

The primary use for MaxRiseTransitionDelay is to force the maximum transition delay to 0 for generated clocks. Non-generated clocks have a transition delay of 0 by default.

Examples

The MaxRiseTransitionDelay attribute specifies the maximum rising edge transition time for the specified ideal clock. MaxRiseTransitionDelay does not apply to propagated clocks because Design Compiler uses the calculated transition delay for propagated clocks.

coreConsultant uses the MaxRiseTransitionDelay attribute to generate the clock transition value for Design Compiler to use for maximum delay analysis.

The primary use for MaxRiseTransitionDelay is to force the maximum transition delay to 0 for generated clocks. Non-generated clocks have a transition delay of 0 by default.

See Also

[set_clock_attribute \(2\), MinRiseTransitionDelay \(3\),](#)[MaxFallTransitionDelay \(3\), MinFallTransitionDelay \(3\),](#)

MaxTransition

Maximum transition time for an input port or design.

Definition

Type: float

Flags:

Default value: =InheritValue up {}

Valid on: busBit design port _____

Description

The MaxTransition attribute specifies a maximum transition time for the selected input port or design. coreConsultant uses MaxTransition directly as the value for the Design Compiler set_max_transition command for the specified input port or design.

If you set MaxTransition on an input port, Design Compiler attempts to ensure that the transition time for each net that is driven by that input port is less than the MaxTransition value on the input port.

If you set MaxTransition on a design, Design Compiler attempts to ensure that the transition time for each net in the design is less than the MaxTransition value on the design.

If you specify MaxTransition on both a design and an input port of that design, Design Compiler attempts to meet the more restrictive value. Design Compiler also attempts to meet the more restrictive value of the MaxTransition attribute or the Maximum Transition design rule constraint of the target technology library.

You can specify the MaxTransition time value with or without units. If you do not specify a time unit, the default time unit is the time unit used by the currently loaded technology library. If you do specify a time unit (for example, 2ns), coreConsultant automatically scales the time value to the time unit used by the currently loaded technology library. For example, if you specify 2ns and the currently loaded technology library uses ps as the time unit, coreConsultant automatically scales 2ns to 2,000ps.

If you do not specify MaxTransition on an input port of a design, the input port inherits the MaxTransition value from the connected port on the next higher level design. If you do not specify MaxTransition on a design, the design inherits the MaxTransition value from the next higher level design.

Examples

To set the maximum transition time on the int0 port to 2ns:

```
coreConsultant> set_port_attribute int0 MaxTransition 2ns
```

To set the maximum transition time for Subblock_A to 2ns:

```
coreConsultant> set_design_attribute -designs
```

```
Subblock_A MaxTransition 2ns
```

See Also

[set_port_attribute \(2\), set_design_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

MaxValue
Maximum value allowed

Definition

Type: string

Flags:

Default value:

Valid on: attrDefn param _____

Description

The MaxValue attribute specifies the maximum acceptable value for a parameter. If a user attempts to set a parameter value to a value greater than MaxValue, coreConsultant or coreBuilder returns an error message.

Use MaxValue with MinValue to set a range of legal values for a parameter. For a parameter that has a set of individual legal values instead of a range of values, use EnumValues instead of MinValue and MaxValue.

If you do not explicitly set a value for MaxValue on an HDL parameter, coreBuilder automatically sets MaxValue to the maximum value for the parameter's type. If there is a range declared for the parameter in the HDL code, coreBuilder sets MaxValue to the maximum value of the declared range.

Examples

To set the maximum value for the intrs parameter to 8:

```
coreBuilder> set_parameter_attribute intrs MaxValue 8
```

See Also

[set_parameter_attribute \(2\)](#), [MinValue \(3\)](#), [EnumValues \(3\)](#) _____

[API Index](#) [Command Reference](#)

MaxWriteConstraint

Defines a maximum value constraint to be met when writing the given register field.

Definition

Type: bitfield

Flags:

Default value:

Valid on:

Description

Indicates the maximum value that can be written to the given register field.

Examples

See Also

[MinWriteConstraint](#)

[API Index](#) [Command Reference](#)

MaximumScanChainLength

The maximum length a scan chain may be. Setting the value to 0 will allow DFT Compiler to determine the maximum scan chain length.

Definition

Type: long

Flags:

Default value: 0

Valid on: design_____

Description

The maximum length a scan chain may be. Setting the value to 0 will allow DFT Compiler to determine the maximum scan chain length.

Examples

Specify that DFT Compiler is to determine the maximum scan chain length.

```
coreBuilder> set_design_attribute MaximumScanChainLength 0
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#), [NumberOfScanChains \(3\)](#), [BistChainCount \(3\)](#), [BistMaxChainLength \(3\)](#)

[API Index](#) [Command Reference](#)

MemoryAccess

Access type for this memory item

Definition

Type: string**Flags:** subscripted**Default value:** =sMem::defaultMemoryAccess %item**Valid subscripts:** <valid subscripts are not statically defined>**Default subscript:** DEFAULT**Valid on:** param_____

Description

This attribute specifies the access to this memory map or register. Value values are 'read-write', 'read-only', 'write-only', 'writeOnce', and 'read-writeOnce'.

Examples

To specify the memory map map1's access type to be 'read-only':

```
coreBuilder> set_memory_map_attribute map1 MemoryAccess "read-only"
```

See Also

[API Index](#) [Command Reference](#)

MemoryAccessDescription
Access type for this memory item

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

MemoryMap

Holds the memory map that this slave interface refers to.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This attribute holds the name of the memory map that this slave interface refers to.

Examples

See Also

Memory Map Specification

This activity is used to configure memory map.

Parameters

Activity has no parameters.

See Also

[API Index](#) [Command Reference](#)

MemoryMode

Set all the valid Memory Modes active for the design

Definition

Type: string

Flags:

Default value: DEFAULT

Valid on: design_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

MemoryRange

Specifies the address range of the registerArray, addressBlock or addressBank

Definition

Type: bitfield

Flags:

Default value: =sMem::defaultRange %item

Valid on:

Description

This attribute specifies the address range of the memory element. This value is expressed as the number of addressable units of size BitsInLAU for the containing memory map.

Examples

To set the address blocks memory range to be 4M:

```
coreBuilder> set_memory_map_attribute map1/block1 MemoryRange 0x400000
```

See Also

[BankAlignment \(3\)](#)

MemoryUsage

Specifies the usage of this addressBlock or addressBank.

Definition

Type: string

Flags:

Default value: =InheritValue up {}

Valid on:

Description

This attribute specifies the usage of the memory map. Possible values are 'memory', 'register', and 'reserved'.

Examples

To set the memory map map1's usage to be 'register':

```
coreBuilder> set_memory_map_attribute map1 MemoryUsage register
```

See Also

[API Index](#) [Command Reference](#)

MemoryWidth

Specifies the bit width of an address block or bank

Definition

Type: long

Flags:

Default value: =sMem::defaultWidth %item

Valid on:

Description

This attribute specifies the bit width of the address block or bank.

Examples

```
coreBuilder> set_address_block_attribute map1/block1 MemoryWidth 32
```

See Also

MergeCandidate

Indicates that a pin/port is a candidate for port merging via `merge_ports`.

Definition

Type: boolean

Flags:

Default value: 1

Valid on:

Description

This attribute can be used to prevent ports from being merged by the `merge_ports` command. Ports (or pins) with `MergeCandidate == false` will be excluded from consideration for merging. This is equivalent to passing the indicated ports/pins by name to the `merge_ports` command using the `-exclude` option.

Examples

See Also

[merge_ports \(2\)](#)

[API Index](#) [Command Reference](#)

MessageLimitDC

For each subscript, injects 'set_message_info -id <subscript> -limit <value>' in the synthesis RM scripts.

Definition

Type: short

Flags: subscripted

Default value: 0

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This attribute is used to inject 'set_message_info -id <msg id> -limit <count>' in the synthesis RM scripts to limit messages in the synthesis log files.

Examples

```
coreAssembler> set_design_attribute MessageLimitDC\[FILE-007\] 3
```

This introduces the following command in the Design Compiler synthesis scripts.

```
set_message_info -id FILE-007 -limit 3
```

See Also

[MessageStopOnDC \(3\)](#) [MessageStopOnFC \(3\)](#) [MessageLimitFC \(3\)](#)

[API Index](#) [Command Reference](#)

MessageLimitFC

For each subscript, injects 'set_message_info -id <subscript> -limit <value>' in the synthesis RM scripts.

Definition

Type: short

Flags: subscripted

Default value: 0

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This attribute is used to inject 'set_message_info -id <msg id> -limit <count>' in the synthesis RM scripts to limit messages in the synthesis log files.

Examples

```
coreAssembler> set_design_attribute MessageLimitDC\[FILE-007\] 3
```

This introduces the following command in the Fusion Compiler synthesis scripts.

```
set_message_info -id FILE-007 -limit 3
```

See Also

[MessageStopOnDC \(3\)](#) [MessageStopOnFC \(3\)](#) [MessageLimitFC \(3\)](#)

[API Index](#) [Command Reference](#)

MessageStopOnDC

If true, for each subscript, injects 'set_message_info -id <subscript> -stop_on' in the synthesis RM scripts.

Definition

Type: boolean

Flags: subscripted

Default value: 0

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This attribute is used to inject 'set_message_info -id <msg id> -stop_on' in the synthesis RM scripts to stop the synthesis run on encountering the message.

Examples

```
coreAssembler> set_design_attribute MessageStopOnDC[FILE-007\] true
```

This introduces the following command in the Design Compiler synthesis scripts.

```
set_message_info -id FILE-007 -stop_on
```

See Also

[MessageStopOnFC \(3\)](#) [MessageLimitDC \(3\)](#) [MessageLimitFC \(3\)](#)

[API Index](#) [Command Reference](#)

MessageStopOnFC

If true, for each subscript, injects 'set_message_info -id <subscript> -stop_on' in the synthesis RM scripts.

Definition

Type: boolean

Flags: subscripted

Default value: 0

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This attribute is used to inject 'set_message_info -id <msg id> -stop_on' in the synthesis RM scripts to stop the synthesis run on encountering the message.

Examples

coreAssembler> set_design_attribute MessageStopOnDC[FILE-007\] true

This introduces the following command in the Design Compiler synthesis scripts.

```
set_message_info -id FILE-007 -stop_on
```

See Also

[MessageStopOnDC \(3\)](#) [MessageLimitDC \(3\)](#) [MessageLimitFC \(3\)](#)

[API Index](#) [Command Reference](#)

MinCap

Minimum capacitance value for a port or a design.

Definition

Type: float

Flags:

Default value: =InheritValue up {}

Valid on: busBit design port _____

Description

The MinCap attribute specifies a minimum capacitance value for the selected input or output port. coreConsultant uses MinCap directly as a value for the Design Compiler set_min_capacitance command.

You can specify the MinCap value with or without units. If you do not specify a capacitance unit, the default capacitance unit is the capacitance unit used by the currently loaded technology library. If you do specify a capacitance unit (for example, 5pf), coreConsultant automatically scales the time value to the capacitance unit used by the currently loaded technology library. For example, if you specify 5pf and the currently loaded technology library uses ff as the capacitance unit, coreConsultant automatically scales 5pf to 5,000ff.

Examples

To set the minimum capacitance on the int0 port to 10 picofarads:

```
coreConsultant> set_port_attribute int0 MinCap 10pf
```

See Also

[set_port_attribute \(2\)](#)

MinFallInputDelay

Minimum falling edge delay constraint for an input port

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port

Description

The MinFallInputDelay attribute specifies the minimum falling edge input delay for the input port with respect to the rising edge of the specified clock. The MinFallInputDelay value is the minimum propagation delay of a falling edge of the incoming signal from the rising edge of the specified clock to the selected input port on the current design.

The subscript to the MinFallInputDelay attribute is the name of the clock. For example, MinFallInputDelay[clk] is the minimum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinFallInputDelay. The default subscript is the single existing clock in the design. To set both the minimum rise and fall input delay values to the same value you can use the MinInputDelay attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinFallInputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinFallInputDelay, the port inherits the MinFallInputDelay value from the connected port on the next higher level design.

To specify minimum input delay relative to a falling clock edge, use the MinFallInputDelayFallingEdge attribute instead of MinFallInputDelay.

Examples

To set the minimum falling edge input delay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {MinFallInputDelay[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MinFallInputDelay
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MinRiseInputDelay (3), _____
MaxRiseInputDelay (3), MaxFallInputDelay (3), MinInputDelay (3), _____
MinFallInputDelayFallingEdge (3) _____

MinFallInputDelayFallingEdge

Minimum falling edge delay constraint for an input port relative to a falling clock edge

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MinFallInputDelayFallingEdge attribute specifies the minimum falling edge input delay for the input port with respect to the falling edge of the specified clock. The MinFallInputDelayFallingEdge value is the minimum propagation delay of a falling edge of the incoming signal from the falling edge of the specified clock to the selected input port on the current design.

The subscript to the MinFallInputDelayFallingEdge attribute is the name of the clock. For example, MinFallInputDelayFallingEdge[clk] is the minimum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinFallInputDelayFallingEdge. The default subscript is the single existing clock in the design. To set both the minimum rise and fall input delay values to the same value you can use the MinInputDelayFallingEdge attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinFallInputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

{=percent_of_period <percentage> [<clock_name>]}

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinFallInputDelayFallingEdge, the port inherits the MinFallInputDelayFallingEdge value from the connected port on the next higher level design.

To specify minimum input delay relative to a rising clock edge, use the `MinFallInputDelay` attribute instead of `MinFallInputDelayFallingEdge`.

Examples

To set the minimum falling edge input delay on the `data_in` port to 15 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_in
{MinFallInputDelayFallingEdge[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MinFallInputDelayFallingEdge
{=percent_of_period 15.0}
```

See Also

[percent_of_period \(2\)](#), [set_port_attribute \(2\)](#), _____

[MinRiseInputDelayFallingEdge \(3\)](#), [MaxRiseInputDelayFallingEdge \(3\)](#), _____

[MaxFallInputDelayFallingEdge \(3\)](#), [MinInputDelayFallingEdge \(3\)](#), _____

[MinFallInputDelay \(3\)](#) _____

MinFallOutputDelay

Minimum falling edge delay constraint for an output port

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MinFallOutputDelay attribute specifies the minimum falling edge output delay for the output port with respect to the rising edge of the specified clock. The MinFallOutputDelay value is the minimum propagation delay of a falling edge of the incoming signal from the rising edge of the specified clock to the selected output port on the current design.

The subscript to the MinFallOutputDelay attribute is the name of the clock. For example, MinFallOutputDelay[clk] is the minimum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinFallOutputDelay. The default subscript is the single existing clock in the design. To set both the minimum rise and fall output delay values to the same value you can use the MinOutputDelay attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinFallOutputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinFallOutputDelay, the port inherits the MinFallOutputDelay value from the connected port on the next higher level design.

To specify minimum output delay relative to a falling clock edge, use the MinFallOutputDelayFallingEdge attribute instead of MinFallOutputDelay.

Examples

To set the minimum falling edge output delay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {MinFallOutputDelay[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MinFallOutputDelay
{=percent_of_period 15.0}
```

See Also

[percent_of_period \(2\)](#), [set_port_attribute \(2\)](#), [MinRiseOutputDelay \(3\)](#),
[MaxRiseOutputDelay \(3\)](#), [MaxFallOutputDelay \(3\)](#), [MinOutputDelay \(3\)](#),
[MinFallOutputDelayFallingEdge \(3\)](#)

MinFallOutputDelayFallingEdge

Minimum falling edge delay constraint for an output port relative to a falling clock edge

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MinFallOutputDelayFallingEdge attribute specifies the minimum falling edge output delay for the output port with respect to the falling edge of the specified clock. The MinFallOutputDelayFallingEdge value is the minimum propagation delay of a falling edge of the incoming signal from the falling edge of the specified clock to the selected output port on the current design.

The subscript to the MinFallOutputDelayFallingEdge attribute is the name of the clock. For example, MinFallOutputDelayFallingEdge[clk] is the minimum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinFallOutputDelayFallingEdge. The default subscript is the single existing clock in the design. To set both the minimum rise and fall output delay values to the same value you can use the MinOutputDelayFallingEdge attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinFallOutputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

{=percent_of_period <percentage> [<clock_name>]}

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinFallOutputDelayFallingEdge, the port inherits the MinFallOutputDelayFallingEdge value from the connected port on the next higher level design.

To specify minimum output delay relative to a rising clock edge, use the `MinFallOutputDelay` attribute instead of `MinFallOutputDelayFallingEdge`.

Examples

To set the minimum falling edge output delay on the `data_in` port to 15 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_in
{MinFallOutputDelayFallingEdge[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MinFallOutputDelayFallingEdge
{=percent_of_period 15.0}
```

See Also

[percent_of_period \(2\)](#), [set_port_attribute \(2\)](#), [MinRiseOutputDelayFallingEdge \(3\)](#), [MaxRiseOutputDelayFallingEdge \(3\)](#), [MaxFallOutputDelayFallingEdge \(3\)](#), [MinOutputDelayFallingEdge \(3\)](#), [MinFallOutputDelay \(3\)](#)

[API Index](#) [Command Reference](#)

MinFallTransitionDelay

Minimum transition fall time for an ideal clock.

Definition

Type: float**Flags:****Default value:****Valid on:** clock_____

Description

The MinFallTransitionDelay attribute specifies the minimum falling edge transition time for the specified ideal clock. MinFallTransitionDelay does not apply to propagated clocks because Design Compiler uses the calculated transition delay for propagated clocks.

coreConsultant uses the MinFallTransitionDelay attribute to generate the clock transition value for Design Compiler to use for minimum delay analysis.

The primary use for MinFallTransitionDelay is to force the minimum transition delay to 0 for generated clocks. Non-generated clocks have a transition delay of 0 by default.

Examples

To use 0 minimum falling edge transition delay for the generated clock named int_clk:

```
coreConsultant> set_clock_attribute int_clk MinFallTransitionDelay 0
```

See Also

[set_clock_attribute \(2\)](#), [MaxFallTransitionDelay \(3\)](#), _____

[MaxRiseTransitionDelay \(3\)](#), [MinRiseTransitionDelay \(3\)](#), _____

MinInputDelay

Minimum delay constraint for an input port

Definition

Type: float

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: busBit port _____

Description

The MinInputDelay attribute specifies the minimum input delay for the input port with respect to the rising edge of the specified clock. The MinInputDelay value is the minimum propagation delay of the incoming signal from the rising edge of the specified clock to the selected input port on the current_design. Setting this attribute is equivalent to setting both MinRiseInputDelay and MinFallInputDelay to the same value.

The subscript to the MinInputDelay attribute is the name of the clock. For example, MinInputDelay[clk] is the minimum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinInputDelay. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinInputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinInputDelay, the port inherits the MinInputDelay value from the connected port on the next higher level design.

To specify MinInputDelay relative to a falling clock edge, use the MinInputDelayFallingEdge attribute instead of MinInputDelay.

Examples

To set the minimum input delay on the data_in port to 10 percent of the clk cycle time:

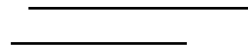
```
coreConsultant> set_port_attribute data_in
{MinInputDelay[clk]}
{=percent_of_period 10.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MinInputDelay
{=percent_of_period 10.0}\fP
```

See Also

percent_of_period (2), set_port_attribute (2), MaxInputDelay (3),
MinRiseInputDelay (3), MinFallInputDelay (3), InputDelay (3),
MinInputDelayFallingEdge (3)



MinInputDelayFallingEdge

Minimum delay constraint for an input port (relative to the falling clock edge)

Definition

Type: float

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: busBit port _____

Description

The MinInputDelayFallingEdge attribute specifies the minimum input delay for the input port with respect to the falling edge of the specified clock. The MinInputDelayFallingEdge value is the minimum propagation delay of the incoming signal from the falling edge of the specified clock to the selected input port on the current_design. Setting this attribute is equivalent to setting both MinRiseInputDelayFallingEdge and MinFallInputDelayFallingEdge to the same value.

The subscript to the MinInputDelayFallingEdge attribute is the name of the clock. For example, MinInputDelayFallingEdge[clk] is the minimum input delay with respect to the falling edge of clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinInputDelayFallingEdge. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinInputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinInputDelayFallingEdge, the port inherits the MinInputDelayFallingEdge value from the connected port on the next higher level design.

To specify minimum input delay relative to a rising clock edge, use the MinInputDelay attribute.

Examples

To set the minimum input delay on the data_in port, relative to the falling edge of the clk signal, to 10 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in
{MinInputDelayFallingEdge[clk]}
{=percent_of_period 10.0 clk}\fP
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in
MinInputDelayFallingEdge
{=percent_of_period 10.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MinInputDelay (3),
MinRiseInputDelayFallingEdge (3), MinFallInputDelayFallingEdge (3),
InputDelay (3), MaxInputDelayFallingEdge (3)

MinOutputDelay

Minimum delay constraint for an output port

Definition

Type: float**Flags:** subscripted**Default value:****Valid subscripts:** <valid subscripts are not statically defined>**Default subscript:****Valid on:** busBit port _____

Description

The MinOutputDelay attribute specifies the minimum output delay for the output port with respect to the rising edge of the specified clock. The MinOutputDelay value is the minimum amount of time that the outgoing signal must be available at the output port before the next rising clock edge. In most cases, MinOutputDelay represents the shortest combinational path delay from the design's output port to a register that receives the signal, minus the library hold time. Setting this attribute is equivalent to setting both MinRiseOutputDelay and MinFallOutputDelay to the same value.

The subscript to the MinOutputDelay attribute is the name of the clock. For example, MinOutputDelay[clk] is the minimum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinOutputDelay. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinOutputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinOutputDelay, the port inherits the MinOutputDelay value from the connected port on the next higher level design.

To specify minimum output delay relative to a falling clock edge, use the MinOutputDelayFallingEdge attribute instead of MinOutputDelay.

Examples

To set the minimum output delay on the data_out port to 5 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_out
{MinOutputDelay[clk]}
{=percent_of_period 5.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_out
MinOutputDelay
{=percent_of_period 5.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MaxOutputDelay (3), _____
MinRiseOutputDelay (3), MinFallOutputDelay (3), OutputDelay (3), _____
MinOutputDelayFallingEdge (3)_____

MinOutputDelayFallingEdge

Minimum delay constraint for an output port (relative to the falling clock edge)

Definition

Type: float

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: busBit port _____

Description

The MinOutputDelayFallingEdge attribute specifies the minimum output delay for the output port with respect to the falling edge of the specified clock. The MinOutputDelayFallingEdge value is the minimum amount of time that the outgoing signal must be available at the output port before the next falling clock edge. In most cases, MinOutputDelay represents the shortest combinational path delay from the design's output port to a register that receives the signal, minus the library hold time. Setting this attribute is equivalent to setting both MinRiseOutputDelayFallingEdge and MinFallOutputDelayFallingEdge to the same value.

The subscript to the MinOutputDelayFallingEdge attribute is the name of the clock. For example, MinOutputDelayFallingEdge[clk] is the minimum output delay with respect to the falling edge of clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinOutputDelayFallingEdge. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinOutputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinOutputDelayFallingEdge, the port inherits the MinOutputDelayFallingEdge value from the connected port on the next higher level design.

To specify minimum output delay relative to a rising clock edge, use the `MinOutputDelay` attribute.

Examples

To set the minimum output delay on the `data_out` port, relative to the falling edge of the `clk` signal, to 5 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_out
{MinOutputDelayFallingEdge[clk]}
{=percent_of_period 5.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_out
MinOutputDelayFallingEdge
{=percent_of_period 5.0}\fP
```

See Also

percent_of_period (2), set_port_attribute (2), MinOutputDelay (3), _____
MinRiseOutputDelayFallingEdge (3), MinFallOutputDelayFallingEdge (3), _____
OutputDelay (3), MaxOutputDelayFallingEdge (3) _____

MinRiseInputDelay

Minimum rising edge delay constraint for an input port

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port

Description

The MinRiseInputDelay attribute specifies the minimum rising edge input delay for the input port with respect to the rising edge of the specified clock. The MinRiseInputDelay value is the minimum propagation delay of a rising edge of the incoming signal from the rising edge of the specified clock to the selected input port on the current design.

The subscript to the MinRiseInputDelay attribute is the name of the clock. For example, MinRiseInputDelay[clk] is the minimum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinRiseInputDelay. The default subscript is the single existing clock in the design. To set both the minimum rise and fall input delay values to the same value you can use the MinInputDelay attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinRiseInputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinRiseInputDelay, the port inherits the MinRiseInputDelay value from the connected port on the next higher level design.

To specify minimum input delay relative to a falling clock edge, use the MinRiseInputDelayFallingEdge attribute instead of MinRiseInputDelay.

Examples

To set the minimum rising edge input delay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {MinRiseInputDelay[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MinRiseInputDelay
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MinFallInputDelay (3), _____
MaxRiseInputDelay (3), MaxFallInputDelay (3), MinInputDelay (3), _____
MinRiseInputDelayFallingEdge (3) _____

MinRiseInputDelayFallingEdge

Minimum rising edge delay constraint for an input port relative to a falling clock edge

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MinRiseInputDelayFallingEdge attribute specifies the minimum rising edge input delay for the input port with respect to the falling edge of the specified clock. The MinRiseInputDelay value is the minimum propagation delay of a rising edge of the incoming signal from the falling edge of the specified clock to the selected input port on the current design.

The subscript to the MinRiseInputDelayFallingEdge attribute is the name of the clock. For example, MinRiseInputDelayFallingEdge[clk] is the minimum input delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinRiseInputDelayFallingEdge. The default subscript is the single existing clock in the design. To set both the minimum rise and fall input delay values to the same value you can use the MinInputDelayFallingEdge attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinRiseInputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

{=percent_of_period <percentage> [<clock_name>]}

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinRiseInputDelayFallingEdge, the port inherits the MinRiseInputDelayFallingEdge value from the connected port on the next higher level design.

To specify minimum input delay relative to a rising clock edge, use the `MinRiseInputDelay` attribute instead of `MinRiseInputDelayFallingEdge`.

Examples

To set the minimum rising edge input delay on the `data_in` port to 15 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_in
{MinRiseInputDelayFallingEdge[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MinRiseInputDelayFallingEdge
{=percent_of_period 15.0}
```

See Also

[percent_of_period \(2\)](#), [set_port_attribute \(2\)](#), _____

[MinFallInputDelayFallingEdge \(3\)](#), [MaxRiseInputDelayFallingEdge \(3\)](#), _____

[MaxFallInputDelayFallingEdge \(3\)](#), [MinInputDelay \(3\)](#), [MinRiseInputDelay \(3\)](#) _____

MinRiseOutputDelay

Minimum rising edge delay constraint for an output port

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up { }

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MinRiseOutputDelay attribute specifies the minimum rising edge output delay for the output port with respect to the rising edge of the specified clock. The MinRiseOutputDelay value is the minimum propagation delay of a rising edge of the incoming signal from the rising edge of the specified clock to the selected output port on the current design.

The subscript to the MinRiseOutputDelay attribute is the name of the clock. For example, MinRiseOutputDelay[clk] is the minimum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinRiseOutputDelay. The default subscript is the single existing clock in the design. To set both the minimum rise and fall output delay values to the same value you can use the MinOutputDelay attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinRiseOutputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinRiseOutputDelay, the port inherits the MinRiseOutputDelay value from the connected port on the next higher level design.

To specify minimum output delay relative to a falling clock edge, use the MinRiseOutputDelayFallingEdge attribute instead of MinRiseOutputDelay.

Examples

To set the minimum rising edge output delay on the data_in port to 15 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_in {MinRiseOutputDelay[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MinRiseOutputDelay
{=percent_of_period 15.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MinFallOutputDelay (3),
MaxRiseOutputDelay (3), MaxFallOutputDelay (3), MinOutputDelay (3),
MinRiseOutputDelayFallingEdge (3)

MinRiseOutputDelayFallingEdge

Minimum rising edge delay constraint for an output port relative to a falling clock edge

Definition

Type: float

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Supported formulas: percent_of_period

Valid on: busBit port _____

Description

The MinRiseOutputDelayFallingEdge attribute specifies the minimum rising edge output delay for the output port with respect to the falling edge of the specified clock. The MinRiseOutputDelay value is the minimum propagation delay of a rising edge of the incoming signal from the falling edge of the specified clock to the selected output port on the current design.

The subscript to the MinRiseOutputDelayFallingEdge attribute is the name of the clock. For example, MinRiseOutputDelayFallingEdge[clk] is the minimum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of MinRiseOutputDelayFallingEdge. The default subscript is the single existing clock in the design. To set both the minimum rise and fall output delay values to the same value you can use the MinOutputDelayFallingEdge attribute instead.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify MinRiseOutputDelayFallingEdge as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify <clock_name>. coreConsultant uses the single existing clock.

If you do not specify a value for MinRiseOutputDelayFallingEdge, the port inherits the MinRiseOutputDelayFallingEdge value from the connected port on the next higher level design.

To specify minimum output delay relative to a rising clock edge, use the `MinRiseOutputDelay` attribute instead of `MinRiseOutputDelayFallingEdge`.

Examples

To set the minimum rising edge output delay on the `data_in` port to 15 percent of the `clk` cycle time:

```
coreConsultant> set_port_attribute data_in
{MinRiseOutputDelayFallingEdge[clk]}
{=percent_of_period 15.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_in MinRiseOutputDelayFallingEdge
{=percent_of_period 15.0}
```

See Also

[percent_of_period \(2\)](#), [set_port_attribute \(2\)](#)

[MinFallOutputDelayFallingEdge \(3\)](#), [MaxRiseOutputDelayFallingEdge \(3\)](#)

[MaxFallOutputDelayFallingEdge \(3\)](#), [MinOutputDelay \(3\)](#), [MinRiseOutputDelay \(3\)](#)

MinRiseTransitionDelay

Minimum transition rise time for an ideal clock.

Definition

Type: float

Flags:

Default value:

Valid on: clock_____

Description

The MinRiseTransitionDelay attribute specifies the minimum rising edge transition time for the specified ideal clock. MinRiseTransitionDelay does not apply to propagated clocks because Design Compiler uses the calculated transition delay for propagated clocks.

coreConsultant uses the MinRiseTransitionDelay attribute to generate the clock transition value for Design Compiler to use for minimum delay analysis.

The primary use for MinRiseTransitionDelay is to force the minimum transition delay to 0 for generated clocks. Non-generated clocks have a transition delay of 0 by default.

Examples

To use 0 minimum rising edge transition delay for the generated clock named int_clk:

```
coreConsultant> set_clock_attribute int_clk MinRiseTransitionDelay 0
```

See Also

[set_clock_attribute \(2\)](#), [MinFallTransitionDelay \(3\)](#),
[MaxFallTransitionDelay \(3\)](#), [MaxRiseTransitionDelay \(3\)](#),

[API Index](#) [Command Reference](#)

MinValue
Minimum value allowed

Definition

Type: string

Flags:

Default value:

Valid on: attrDefn param _____

Description

The MinValue attribute specifies the minimum acceptable value for a parameter. If a user attempts to set a parameter value to a value less than MinValue, the coreTool returns an error message.

Use MinValue with MaxValue to set a range of legal values for a parameter. For a parameter that has a set of individual legal values instead of a range of legal values, use EnumValues instead of MinValue and MaxValue.

If you do not explicitly set a value for MinValue on an HDL parameter, coreBuilder automatically sets MinValue to the minimum value for the parameter's type. If there is a range declared for the parameter in the HDL code, coreBuilder sets MinValue to the minimum value of the declared range.

Examples

To set the minimum value for the intrs parameter to 1:

```
coreBuilder> set_parameter_attribute intrs MinValue 1
```

See Also

[set_parameter_attribute \(2\)](#), [MaxValue \(3\)](#), [EnumValues \(3\)](#) _____

[API Index](#) [Command Reference](#)

MinWriteConstraint

Defines a minimum value constraint to be met when writing the given register field.

Definition

Type: bitfield

Flags:

Default value:

Valid on:

Description

Indicates the minimum value that can be written to the given register field.

See Also

[MaxWriteConstraint](#)

MonitoredComponent

Defines a component as a monitor and indicates which type of component can be monitored. Value should be a component VLNV.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is used to specify that the given design can be used as a monitor, and that it monitors parts with the specified VLNV. This attribute is used during testbench generation to enable automatic instantiation of monitors in the testbench. Monitors are located by looking in the search path for components with this attribute set. The monitor is then instantiated for each instance which matches the specified VLNV.

This attribute is typically set as part of the packaging of a verification IP component.

Examples

Set up to monitor all versions of the DesignWare DW_ahb component.

```
set_design_attribute MonitoredComponent {Synopsys DesignWareLibrary  
DW_ahb}
```

See Also

/ref MonitoredInterface (3)

MonitoredInterface

Defines a component as application side VIP, identifying the type of interface to be connected to (SPIRIT VLN), the SPIRIT interface type, and the name of the VIP interface to connect to.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is used to specify that the given design can be used as application side verification IP. This means that the given component attaches to a testbench via the specified interface.

This attribute is used during testbench generation to enable automatic instantiation of application VIP in the testbench. Candidates are located by looking in the search path for components with this attribute set. If a DUT in the testbench contains an unconnected interface which matches the described conditions, then the specified component is instantiated and connected to the interface.

This attribute is typically set as part of the packaging of a verification IP component.

Examples

Set up an application VIP component to attach to the serial I/O interface of a DUT using the 'SerialIO' interface on the VIP. The SPIRIT type of the DUT interface must be 'mirroredSlave'.

```
/code set_design_attribute MonitoredInterface {Synopsys DesignWareLibrary  
DW-SoC::Serial-IO mirroredSlave SerialIO} /endcode
```

See Also

/ref MonitoredComponent (3)

Name

The name of the item

Definition

Type: string

Flags: readOnly

Default value:

Valid on: Strategy attrDefn busBit cell clock design file filegroup

filegroupGroup hdlFunc knowledgeBase net param pin port

timingException

Description

The Name attribute is a read-only attribute that indicates the name of the selected item.

For items that you can create, you specify the item name when you create the item. For example, when you create a virtual clock, you specify the item name as part of the create_virtual_clock command. In coreBuilder, you also specify names for custom activities and custom activity parameters.

Otherwise, coreConsultant and coreBuilder automatically set the Name attribute on items to the actual item name. For example, coreBuilder automatically sets the value of the Name attribute on a design to the name of the design as specified in the HDL.

Examples

To get the value of the Name attribute on the data_in port:

```
coreConsultant> get_port_attribute data_in Name
data_in
```

See Also

[get_attribute \(2\)](#)

NumberOfScanChains

The number of scan chains to be created. This value is passed to the `set_scan_configuration` command with the `-chain_count` option. For `Internal_scan`, this is also the number of scan IO ports created. The default value is the number of test clocks in the design.

Definition

Type: long

Flags:

Default value: `=sDft::getTestClocks "all" "current" 1`

Valid on: `design`

Description

The number of scan chains to be created. Setting the value to 0 will allow DFT Compiler to determine the number of scan chains, based on the number of test clocks in the design. This number decides the pairs of scan IOs created for the block. When scan compression is enabled, the number of internal scan chains is calculated by coreTools based on the number of scan chains and usage type selected for scan compression.

Examples

Specify that DFT Compiler is to determine the number of scan chains in the design.

```
coreBuilder> set_desgin_attribute NumberOfScanChains 0
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#),
[MaximumScanChainLength \(3\)](#), [BistChainCount \(3\)](#), [BistMaxChainLength \(3\)](#)
[ScanCompressionUsage \(3\)](#) [ScanCompressionConfiguration \(3\)](#)

NumberOfScanCompressionChains

In ScanCompression_mode, this parameter sets the number of scan chains to be created internally. Setting it to 0 will let coreTools decide the number of scan chains in the design based on the number of ScanCompressionIOCount strategy parameter.

Definition

Type: long

Flags:

Default value: =sDft::getScanCompressionChainCount

Valid on: design_____

Description

Sets the number of scan compression chains explicitly. The default value is set to the values mentioned in the last 2 columns from the table below based on the ScanCompressionUsage design attr or strategy parameter. These are guidelines and should not be interpreted as strict values. Actual values may be different based on size of register count, PPA requirements, atpg fault & test coverage requirements.

In the below table, column 1 is the count of scan IO pairs that feed the decompressor/compressor logic. In most cases, this is represented by NumberOfScanChains. But when Hybrid DFT flow is used (e.g. bottom-up synthesis is used with scan compression enabled on sub blocks), this column represents the ScanCompressionIOCount strategy parameter.

# of scan compression IO pairs (NumberOfScanChains) (or ScanCompressionIOCount if Hybrid DFT is used)	NumberOfScanCompressionChains	
	ScanCompressionUsage "scan_compression"	ScanCompressionUsage "streaming_compression"
1	not allowed in DC/FC	10
2	4	20
3	12	30
4	32	40
5	50	50
6	90	90
7	105	105
8	120	120
9	135	135

coreTools Command Reference Index

10	150	150
11	220	220
12	240	240
13	260	260
14 and higher	280 and higher	280 and higher

Examples

```
coreAssembler> set_design_attribute NumberOfScanChains 4
coreAssembler> set_design_attribute EnableScanCompression 1
coreAssembler> set_design_attribute ScanCompressionUsage
"streaming_compression"
coreAssembler> set_design_attribute NumberOfScanCompressionChains 48 # If
not set, the default value for this attr would've been 40 based on the
table above.
coreAssembler> set strategy Fusion_Compiler_Reference_Methodology
coreAssembler> set_strategy_parameter $strategy InsertDft 1
coreAssembler> set_strategy_parameter $strategy NumberOfScanChains 3
coreAssembler> set_strategy_parameter $strategy EnableScanCompression 1
coreAssembler> set_strategy_parameter $strategy ScanCompressionUsage
"scan_compression"
coreAssembler> echo [get_strategy_parameter $strategy
NumberOfScanCompressionChainsa]
12
# This value is the default that coreConsultant/coreAssembler sets as
default based on the table above.
```

See Also

set_design_attribute (2), ScanBlockIndividually (3),
MaximumScanChainLength (3), NumberOfScanChains (3) EnableScanCompression
(3) ScanCompressionUsage (3) ScanCompressionConfiguration (3)

[API Index](#) [Command Reference](#)

ObservePointsPerScanCell

Specifies the number of observe points that can be shared per scan cell. When TestabilityMethod==control_and_observe for XG mode the larger of ObservePointsPerScanCell and ControlPointsPerScanCell will be used for the -test_points_per_scan_cell value for set_testability_configuration.

Definition

Type: long

Flags:

Default value: 8

Minimum value: 1

Valid on: design_____

Description

Specifies the number of observe points that can be shared per scan cell for test point insertion.

Examples

Set the number of observe points that can be shared per scan cell to 4.

```
coreBuilder> set_design_attribute ObservePointsPerScanCell 4
```

See Also

[set_design_attribute \(2\)](#), [InsertTestPoints \(3\)](#), [ScanBlockIndividually \(3\)](#), [BistBlockIndividually \(3\)](#), [ControlPointsPerScanCell \(3\)](#)

OneRequiredGroup

Specifies that either this deliverable (filegroup) or another deliverable with the same value for this attribute in the the parent group is required.

Definition

Type: string

Flags:

Default value:

Valid on: `filegroup filegroupGroup`

Description

This attribute is used to indicate that among a given set of deliverables (filegroups), at least one of the deliverables must be present (i.e. have files in it) when the coreKit is built. When the coreKit is built, filegroups with the OneRequiredGroup attribute set on them are broken up into groups which have the same value for the attribute. Within each of these groups of filegroups, at least one filegroup must have files in it, or else the coreKit cannot be built.

Examples

The following BoM template fragment defines two verification-related deliverables, one of which must be present when the coreKit is built:

```
{Group
{Name "Verification Views"}
{Deliverable
{Name "Vera BFMs"}
{OneRequiredGroup sim-views}
}
{Deliverable
{Name "RTL BFMs"}
{OneRequiredGroup sim-views}
}
}
```

See Also

OneWrapperClock

If the option is set to TRUE, a single wrapper clock is used for all wrapper cells of the design. If shared wrapper cells are used in wrapping the design, the functional clocks of the shared wrapper cells are muxed with the wrapper clock in wrapper test modes by the core wrapper application. If the option is set to FALSE, a dedicated wrapper clock is used for all dedicated wrapper cells of the design and functional clocks are used for shared wrapper cells of the design.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

If the option is set to TRUE, a single wrapper clock is used for all wrapper cells of the design. If shared wrapper cells are used in wrapping the design, the functional clocks of the shared wrapper cells are muxed with the wrapper clock in wrapper test modes by the core wrapper application. If the option is set to FALSE, a dedicated wrapper clock is used for all dedicated wrapper cells of the design and functional clocks are used for shared wrapper cells of the design.

Examples

Specify that a dedicated wrapper clock is used for all dedicated wrapper cells of the design and functional clocks are used for shared wrapper cells.

```
coreBuilder> set_design_attribute OneWrapperClock 0
```

See Also

[set_design_attribute \(2\)](#), [WrapBlockIndividually \(3\)](#),
[WrapSubblocksIndividually \(3\)](#)_____

OpenAllTreeItems

should all tree items be displayed open for tree dialogs

Definition

Type: boolean

Flags:

Default value: 0

Valid on:

Description

This option forces to tree dialog to be displayed initially with all hierarchical items open displaying all leaf nodes.

Examples

See Also

OperatingConditionsBest

Best-case operating condition to use when evaluating timing for this design.

Definition

Type: string

Flags:

Default value: =InheritValue up {}

Valid on: design_____

Description

OperatingConditionsBest (and OperatingConditionsWorst) specify which technology-specific operating conditions you want Design Compiler to use to evaluate timing for the design. OperatingConditionsBest specifies the best case operating conditions to use for minimum delay (hold time) timing analysis. OperatingConditionsWorst specifies the worst case operating conditions to use for maximum delay (setup time) timing analysis.

You must specify a value for OperatingConditionsWorst. If you do not specify OperatingConditionsBest, coreConsultant uses OperatingConditionsWorst for both setup and hold time analysis.

Specify OperatingConditionsBest and OperatingConditionsWorst on the top-level design only. coreConsultant automatically propagates the OperatingConditionsBest and OperatingConditionsWorst values to the design's subblocks.

Examples

To use typ_25_3.25 as the best case operating condition for timing analysis on the design MyCore:

```
coreConsultant> current_design MyCore
Information: Current design changed to MyCore. (DES-4)
{MyCore}
coreConsultant> set_design_attribute OperatingConditionsBest typ_25_3.25
```

See Also

[set_design_attribute \(2\)](#), [OperatingConditionsWorst \(3\)](#)

OperatingConditionsWorst

Worst-case operating condition to use when evaluating timing for this design.

Definition

Type: string

Flags:

Default value: =InheritValue up

{=sCstr::default_operating_conditions}

Valid on: design_____

Description

OperatingConditionsWorst (and OperatingConditionsBest) specify which technology-specific operating conditions value you want Design Compiler to use to evaluate timing for the design. OperatingConditionsWorst specifies the worst case operating conditions to use for maximum delay (setup time) timing analysis. OperatingConditionsBest specifies the best case operating conditions to use for minimum delay (hold time) timing analysis.

You must specify a value for OperatingConditionsWorst. If you do not specify OperatingConditionsBest, coreConsultant uses OperatingConditionsWorst for both setup and hold time analysis.

Specify OperatingConditionsWorst and OperatingConditionsBest on the top-level design only. coreConsultant automatically propagates the OperatingConditionsBest and OperatingConditionsWorst values to the design's subblocks.

Examples

To use typ_75_3.25 as the worst case operating condition for timing analysis on the design MyCore:

```
coreConsultant> current_design MyCore
```

```
Information: Current design changed to MyCore. (DES-4)
```

```
{MyCore}
```

```
coreConsultant> set_design_attribute OperatingConditionsWorst typ_75_3.25
```

See Also

[set_design_attribute \(2\), OperatingConditionsBest \(3\)](#)_____

OptimizationPriorities

Primary and secondary implementation goals for this design.

Definition

Type: string

Flags:

Default value: =InheritValue up timing_area

Valid on: design_____

Description

The OptimizationPriorities attribute specifies the optimization priorities for the selected design. Possible values are:

- timing - Focus on achieving the best timing, with minimal consideration to area impact.
- timing_area - Achieve timing objectives first, then try to reduce area.
- area_timing - Achieve the smallest design possible that meets timing constraints.
- area - The design should easily meet timing objectives, so focus the optimization on reducing area.

If you do not explicitly set OptimizationPriorities on a design, the design inherits the OptimizationPriorities value from the next higher level design in the design hierarchy. If OptimizationPriorities is not set on a higher level design in the hierarchy, then OptimizationPriorities defaults to timing_area.

Examples

To optimize the current_design for best timing results:

```
coreConsultant> set_design_attribute OptimizationPriorities timing
```

To meet timing constraints for Subblock_A, then optimize Subblock_A for area:

```
coreConsultant> set_design_attribute -designs
Subblock_A OptimizationPriorities timing_area
```

See Also

[set_design_attribute \(2\)](#)_____

[API Index](#) [Command Reference](#)

OptimizeArithmetic

Should this design be optimized at the arithmetic level prior to initial mapping?

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Examples

See Also

Optional

Indicates that the interface object may not be linked to a design object, or may not exist.

Definition

Type: boolean

Flags:

Default value: 0

Valid on:

Description

Examples

See Also

OptionalAssociation

Indicates that the interface object must not be associated to a design object in the parent interface instance.

Definition

Type: boolean

Flags: readOnly

Default value: =sIntf::isOptionalAssociation_[get_attribute %item
-attr TypeName] %item

Valid on: param_____

Description

This read-only attribute is used to indicate whether or not the specified interface port or interface parameter requires an association with a design object. If 'association' is not option, the interface port (or param) must be associated with a design port (or param) or else the interface itself is not considered complete. Association is optional if the definition of the port or parameter specifies the Optional attribute to be true, and under certain conditions for monitor interfaces. For example, ports on monitor interfaces are always optional.

This is a convenience attribute shown in the interface association dialogs in coreBuilder as an indication of whether or not the interface port or parameter needs to be associated with a design port or parameter.

Examples

There are no reasonable examples of usage as this is primarily for viewing within a spreadsheet.

See Also

[Optional \(3\)](#)

OutputDelay

Delay constraint for an output port

Definition

Type: float

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: busBit port _____

Description

OutputDelay is a shorthand notation that you can use to set both MinOutputDelay and MaxOutputDelay to the same value using the set_port_attribute command. The OutputDelay attribute does not appear in the coreConsultant port intent specification spreadsheets.

The subscript to the OutputDelay attribute is the name of the clock. For example, OutputDelay[clk] is the minimum and maximum output delay with respect to clk. If there is only one clock in the design, you do not have to specify a subscript when you set or get the value of OutputDelay. The default subscript is the single existing clock in the design.

You can specify the delay value either as a numerical value (with or without units) or as a percentage of the clock period. coreConsultant automatically scales the specified value to the time unit used by the currently loaded technology library.

To specify OutputDelay as a percentage of the clock period, use the following technology-independent formula:

```
{=percent_of_period <percentage> [<clock_name>]}
```

If there is only one clock in the design, you do not need to specify a clock name. coreConsultant uses the single existing clock.

Examples

To set both MinOutputDelay and MaxOutputDelay on the data_out port to 10 percent of the clk cycle time:

```
coreConsultant> set_port_attribute data_out
{OutputDelay[clk]}
{=percent_of_period 10.0 clk}
```

For a design with only one clock, you can set the above attribute value as follows:

```
coreConsultant> set_port_attribute data_out OutputDelay  
{=percent_of_period 10.0}
```

See Also

percent_of_period (2), set_port_attribute (2), MinOutputDelay (3),
MaxOutputDelay (3)

Overconstrain

Percentage by which delay constraints should be tightened during initial mapping.

Definition

Type: float

Flags:

Default value: =InheritValue up {}

Valid on: design_____

Description

The Overconstrain attribute specifies the factor by which the timing constraints should be tightened when compiling the design.

When Overconstrain is specified, the generated synthesis script is updated to include calls to the `set_path_margin` command for each clock in the design. Prior to calling the 'compile' command, `set_path_margin` commands are included to reduce required times for paths to and from each clock. Then prior to generation of the final reports, `set_path_margin` commands are included to reset the required times to their original values. The net effect is that during compile, the tool will work to meet tighter constraints, but then the final reporting will be relative to the original constraints.

In general, you should not need to set Overconstrain unless you cannot meet your timing constraints using the standard reference methodology flow.

If you do not explicitly specify a value for Overconstrain on a design, the design inherits the Overconstrain value from the next higher level design in the hierarchy.

Examples

The following line in a coreTools Tcl script sets the Overconstrain value to 0.1 on the `current_design`, which means that timing constraints are tightened by 10 percent during compile:

```
set_design_attribute Overconstrain 0.1
```

Assuming a simple design with two clocks (C1, C2), each with a 10ns cycle time, the setting above would result in the following commands being added prior to compile:

```
set_path_margin -from [get_clocks C1] [expr {0.1*10}]
set_path_margin -to [get_clocks C1] [expr {0.1*10}]
```

coreTools Command Reference Index

`set_path_margin -from [get_clocks C2] [expr {0.1*10}]`

`set_path_margin -to [get_clocks C2] [expr {0.1*10}]`

Then prior to final report generation the following commands are used to reset the path margin:

`set_path_margin -from [get_clocks C1] -reset`

`set_path_margin -to [get_clocks C1] -reset`

`set_path_margin -from [get_clocks C2] -reset`

`set_path_margin -to [get_clocks C2] -reset`

See Also

[set_design_attribute \(2\)](#)

PT_static_timing_analysis_strategy

Run Static Timing Analysis on a design using PrimeTime

Parameters

Strategy has no parameters.

See Also

[API Index](#) [Command Reference](#)

ParallelJobCpuLimit

Specifies the number of parallel compile jobs that may be run.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

Examples

The design attribute is passed into the synthesis intent and sets the parallel compile job limit for the synthesis run.

```
set_design_attribute ParallelJobCpuLimit 4
```

See Also

ParamValueFromDesign

Indicates that the interface parameter comes from the corresponding design parameter.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: param_____

Description

Used to indicate that propagation should occur from the design parameter to the interface parameter instead of the other way around. This design parameter must be an 'instantiation' parameter. Instantiation parameters are defined implicitly if a parameter appears in the -used statement for create_interface_instance, or explicitly if DefinesInstanceExistence is set on the parameter.

Examples

```
set_interface_parameter_attribute -instance AXI_Slave axi_dw  
ParamValueFromDesign true
```

See Also

ParameterCheckCmd

Global check command for a number of parameters.

Definition

Type: string**Flags:****Default value:****Valid on:** Strategy design _____

Description

ParameterCheckCmd is valid on the objects which can have parameters as children. Like CheckCmd, it specifies the name of a Tcl command procedure that you want to use to check the value of the children parameters. It is invoked when the dialog for the parameters is checked. ParameterCheckCmd is most commonly used to impose restrictions on a set of children parameters.

Also like CheckCmd, The command procedure that you specify for ParameterCheckCmd must be available when the coreTool checks the user-specified parameter value. To make the command procedure available, include the command procedure definition in a coreTool plug-in. To build a core-specific plug-in, use the coreBuilder Load Consultant Plugins activity. To build a custom activity plug-in for use in coreBuilder only, use the create_plugin_kb command.

Examples

To set the ParameterCheckCmd attribute on design "top" to the name of your custom parameter checking command (DesignParamCheck) in which a check is done to make sure a set of configurable address parameters are non overlapping, the command takes the design itself as argument:

```
coreBuilder> set_design_attribute top ParameterCheckCmd
{DesignParamCheck %item}
```

See Also

set_design_attribute (2), create_plugin_kb (2), CheckCmd (3), CheckExpr (3)

ParameterInfo

This attribute holds hints to build parameter dialog.

Definition

Type: string

Flags:

Default value:

Valid on: Strategy design filegroup _____

Description

The ParameterInfo attribute is a Tcl list string which contains structural information for building the parameter configuration dialog, and group hierarchy information for a set of parameters. It is valid on the owner of a group of parameters. The value is a Tcl list string where each element is a statement-value pair to specify the value for a set of valid statements.

The following table lists the valid statements used in ParameterInfo:

Statement	Args	Description
Group	<string>	Hierarchical name for a given group, required to be unique across sibling groups. Group ordering is determined by the order the groups appear in the parameter info or by the lowest sequence number of the specified parameters in the group if the group is not found in the parameter info. You can override the default SequenceNumber calculations by explicitly

coreTools Command Reference Index

		specifying it in the group parameter info. Specifies that a text string should be added to the group of parameters. It can have a sub-expression item can to specify the sequence for the text. By default it will be positioned at the beginning of the group of parameters.
Text	<string>	
Label	<string>	Label to be shown to the user for this
ColumnLabels	<string>	Specifies a list of labels to be used as column headers when displaying a spreadsheet or table.
SequenceNumber	<string>	Specifies the sequence to display multiple groups in. If not set it defaults to the lowest sequence number of the specified parameters in the group.
GroupType	<enum>	This statement controls the type of relationship for the group of parameters. Additional statements can be specified to

coreTools Command Reference Index

		control the layout. There are two allowed values: <i>subparam</i> - specifies a group of parameters which are subparameters. <i>layout</i> - controls the layout of a group of parameters. The Default Value is "layout." This is set to the top level group to specify the container type of the dialog. The three allowed values are: <i>dialog</i> - a single dialog will be built. <i>tab</i> - pages are contained in tabs within a dialog <i>tree</i> - pages are contained in a tree property sheet within a dialog. The Default Value is "dialog."
ContainerType	<enum>	
ShowSelectAllButton	<int>	Set to 1 to display the Select All button for the dialog. When the Select All button is pressed all checkbox items

coreTools Command Reference Index

		on the current page or dialog will be unchecked. Default Value: 0
ShowDeselectAllButton	<int>	Set to 1 to display the Deselect All button for the dialog. When the Deselect All button is pressed all checkbox items on the current page or dialog will be unchecked. Default Value: 0
		Disable automatically setting the group SequenceNumber based on the order the groups appear in the parameter info.
DisableAutoSequence	<int>	If SequenceNumber is explicitly specified for a group it overrides the default behaviors of determining the group SequenceNumber. Default Value: 0
LayoutAs	<enum>	For group with the GroupType set to layout, this attribute provide a hint concerning how the group should be added to the dialog. The allowed values

are:

none - specifies the parameters should be put into a tab. Tabs within tables will force the generation of a tree dialog, as well as when there are more than MaxPageCount tabs.

groupbox - controls in the group will be placed into a group box.

spreadsheet - the parameters are placed into a spreadsheet control.

No subgroups are allowed for this layout type.

Parameter statements must occur with Row statements nested with this group.

table - subgroups in the group will be placed into a table.

Parameters in a subgroup will be placed in a row.

Multiple subgroups will create a table with multiple rows.

Default Value:

If the group is at the top level of the hierarchy

coreTools Command Reference Index

		the default is "none," otherwise the default is "groupbox." Specifies the maximum number of pages allowed to be contained in a dialog. If the number of tabs required is greater than this number, a tree dialog is forced to be built. Default Value: 10
MaxPageCount	<int>	
		This statement is a list of patterns specifying the parameters in a given group. It overrides the GroupName set on the parameters themselves. It is valid to have more than one parameter statement in a given Group and all of the parameters specified will be appended to the groups. If a parameter matches the parameter statement for more than one group then there will be a warning generated and the first statement will be obeyed.
Parameters	<item_list>	

coreTools Command Reference Index

MinRowsVisible	<int>	The number of rows to display for a spreadsheet group. If this group is the last group on a page, then the spreadsheet will grow to fill the available display area.
Row	<string>	This statement can only appear with a Group with LayoutAs set to spreadsheet. This command is similar to the Group statemnt, in that it starts a new level of hierarchy, and accepts the Label and Parameters statements within it. The Label value defines the string shown in the first column. The Parameters statement defines the parameter values displayed in the remaining columns. It is valid for the number of parameters to be less than the number of remaining columns, but an error is flagged

coreTools Command Reference Index

		if there are too many parameters.		
AutoHideRows	<int>	The row will be hidden or not visible if all of the parameters in the row are disabled.	AutoHideColumns	<int>
		Specifies the type of grouping decoration. The legal values are: <i>none</i> - provide a normal group box with no outline. <i>outline</i> - provide a normal group box. <i>indent</i> - simply indent the group of parameters, but don't add a box around them. Default Value: outline		The column will be hidden or not visible if all of the parameters in the column are disabled.
BoxType	<enum>			

Examples

The following example shows how to control the generation of tabs and group boxes for a set of parameters. It generates a dialog with two tabs: "Top-level General" and "Top-level Advanced". The AdrWidth and BusWidth parameters are in a group box and "Sizes" is on the "Top-level General" tab:

```
coreBuilder> set paramInfo \
{ContainerType tab}
{MaxPageCount 12}
{Group General
{Lable { Top-Level General}}
{Parameters {f66mhz}}
{Group Sizes
{Parameter {AdrWidth BusWidth}} }
{Group Advanced
{Lable { Top-Level Advanced }}
}
```


coreTools Command Reference Index

```
coreBuilder>set_design_attribute top ParameterInfo $paramInfo
```

This example could also be specified using the GroupName attribute on the parameters to provide the structure of the parameter hierarchy.

```
coreBuilder>set_parameter_attribute AdrWidth GroupName  
{Top-Level General/Sizes}
```

```
coreBuilder>set_parameter_attribute BusWidth GroupName  
{Top-Level General/Sizes}
```

```
coreBuilder>set_parameter_attribute f66mhz GroupName  
{Top-Level Advanced}
```

See Also

set_design_attribute (2), set_parameter_attribute (2), GroupName (3)

Parameters

Parameters used to configure this item. Subscript for that param.

Definition

Type: itemList

Flags: readOnly subscripted

Default value: Value calculated internally.

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: *

Valid on:

Description

This attribute holds that set of parameters that configure this item.

This attribute is useful on designs, activities, and filegroupe. Without a subscript, it returns all the parameters for this item. If a subscript is specified, then a parameter that matches that name will be returned if it exists (empty collection otherwise).

Examples

To get the set of parameters that configures the current design:

```
coreConsultant> get_design_attribute Parameters
```

To get the set of parameter named extended_interrupt:

```
coreConsultant> get_design_attribute {Parameters[extended_interrupt]}
```

See Also

[API Index](#) [Command Reference](#)

ParentWireLoad

wireload model that will be used one level above this design.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

The ParentWireLoad attribute specifies the wire load model that is to be used for the design one level above the selected design. If the core is to be integrated at the chip level, then the ParentWireLoad for the core is the wire load model to be used for the chip.

coreConsultant uses ParentWireLoad to set a wire load model for each of the top level ports of the core. If you do not specify a value for ParentWireLoad, coreConsultant does not select any wire load model for the top level ports.

Specify ParentWireLoad for the top-level design only.

Examples

To specify a wire load model for the design that contains My_Core is io30x30:

```
coreConsultant> current_design My_Core
coreConsultant> set_design_attribute ParentWireLoad io30x30
```

See Also

[set_design_attribute \(2\)](#), [WireLoad \(3\)](#)_____

PhysicalLeft

Indicates the physical left bit index of the interface port to be connected.

Definition

Type: short

Flags:

Default value:

Valid on:

Description

This attribute is part of a set of attributes used to indicate which bits of a physical port are associated with which bits of the logical port in the bus interface. This attribute corresponds to the value from the IP-XACT port mapping element portMaps/portMap/physicalPort/left which indicates the left index of the physical port to be mapped to within a bus interface port map.

This attribute is only applicable in coreBuilder if the interface instance was defined using an IP-XACT bus definition and abstraction definition. Otherwise InterfaceSize is the appropriate attribute to set.

Examples

See Also

[InterfaceLink](#) [InterfaceSize](#) [LogicalLeft](#) [LogicalRight](#) [PhysicalRight](#)

PhysicalRegion

Denotes that this design is to be contained in one contiguous physical region of the chip. Also controls the mapping of the the design in Physical Compiler as MapBlockIndividually controls the mapping of designs in Design Compiler. In Design Compiler there are no restrictions on subdesigns during compilation, but in Physical Compiler a design that is compiled may not contain any subdesigns that are have been compiled separately. When in doubt leave this attribute set to FALSE.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

The PhysicalRegion attribute indicates whether the selected design is to be contained within one contiguous region of the chip during physical design.

If PhysicalRegion is true on a design and the core integrator does not explicitly specify a value for the WireLoad attribute on that design, coreConsultant uses the value of AreaEstimate on that design (set explicitly or calculated by default by summing AreaEstimate of the design's subblocks) to automatically select a wire load model for all designs in the same physical region. coreConsultant uses its automatically selected wire load model only for the initial compile of the design. For subsequent compiles, coreConsultant lets Design Compiler automatically select wire load models based on the actual design area.

If there is no AreaEstimate value for the design, coreConsultant does not automatically select a wire load. Instead, Design Compiler performs automatic wire load model selection.

PhysicalRegion is also used to determine the compile frontier for Physical Compiler strategies, such as PSYN_rtl_to_placed_gates, just as MapBlockIndividually is used to determine the compile frontier for Design Compiler strategies. There are no restrictions on the compile frontier for Design Compiler. In Physical Compiler it is not allowed to have a subdesign compiled if its parent design will also be compiled.

Examples

To specify that My_Core is to be located in one contiguous physical region of the chip:

coreTools Command Reference Index

```
coreConsultant> current_design My_Core
```

```
coreConsultant> set_design_attribute PhysicalRegion true
```

For the following examples assume that the design top contains the designs A B and C.

The following is an example of a valid compile frontier for Physical Compiler:

```
coreConsultant> set_design_attribute -designs {A B C} PhysicalRegion true
```

The following is also an example of a valid compile frontier for Physical Compiler:

```
coreConsultant> set_design_attribute -designs top PhysicalRegion true
```

The following is an example of an illegal compile frontier for Physical Compiler:

```
coreConsultant> set_design_attribute -designs {top A} PhysicalRegion true
```

See Also

[set_design_attribute \(2\)](#), [AreaEstimate \(3\)](#), [WireLoad \(3\)](#)

[API Index](#) [Command Reference](#)

PhysicalRight

Indicates the physical right bit index of the interface port to be connected.

Definition

Type: short

Flags:

Default value:

Valid on:

Description

This attribute is part of a set of attributes used to indicate which bits of a physical port are associated with which bits of the logical port in the bus interface. This attribute corresponds to the value from the IP-XACT port mapping element portMaps/portMap/physicalPort/right which indicates the right index of the physical port to be mapped to within a bus interface port map.

This attribute is only applicable in coreBuilder if the interface instance was defined using an IP-XACT bus definition and abstraction definition. Otherwise InterfaceSize is the appropriate attribute to set.

See Also

[InterfaceLink](#) [InterfaceSize](#) [LogicalLeft](#) [LogicalRight](#) [PhysicalLeft](#)

PinLoadCount

Expected external fanout value for a port.

Definition

Type: short

Flags:

Default value: =InheritValue up {=sCstr::default_pin_load_count
%item %attr}

Minimum value: 0

Valid on: busBit port _____

Description

The PinLoadCount attribute specifies the number of cells (of type PinLoadType) that you expect to load the specified port. For both input ports and output ports, PinLoadCount indicates the number of loads external to the design.

If you do not specify a value for PinLoadCount, the port inherits the PinLoadCount value from the connected port on the next higher level design. For top-level output ports, the default value for PinLoadCount is three.

Examples

To set the expected load of the pdata_enable pin to 16 median drive strength sequential cells:

```
coreConsultant> set_port_attribute pdata_enable PinLoadCount 16
coreConsultant> set_port_attribute pdata_enable PinLoadType
{=select_by_class "sequential" "median"}
```

See Also

[set_port_attribute \(2\), PinLoadType \(3\)](#) _____

PinLoadType

Type of cell expected to load this port (externally).

Definition

Type: string

Flags:

Default value: =InheritValue up {=sCstr::budget_pin_load_type %item %attr}

Supported formulas: explicit_capacitance select_by_class
select_by_function select_by_name

Valid on: busBit port

Description

The PinLoadType attribute specifies the type of cell that you expect to load the specified port. For both input ports and output ports, PinLoadType and PinLoadCount indicate the type and number, respectively, of loads external to the design.

Use one of the following formulas to specify the cell type:

- `{=select_by_class <class> [<drive_strength>]}` - Select a representative library cell of the specified drive strength for the specified class of cells (combinational, sequential, or tristate). `<drive_strength>` is optional (low, median, or high) and defaults to median if not specified.
- `{=select_by_function <function> [<drive_strength>]}` - Select a representative library cell of the specified drive strength for the specified cell function (nand2, buf, inv, mux21, dff, latch, or xor2). `<drive_strength>` is optional (low, median, or high) and defaults to median if not specified.
- `{=select_by_name <cell_name>/[<pin_name>]}` - Select a library cell by cell name and, optionally, by pin name. Use this formula if you know the technology-specific cell name (and optional pin name) of the expected load on this output pin.
- `{=explicit_capacitance <cap_value>}` - Specify the pin load as an explicit capacitance value instead of as a type of library cell. You can specify `<cap_value>` either with or without a capacitance unit. coreConsultant automatically scales the capacitance value to the capacitance unit used by the currently loaded technology library.

If you do not specify a value for PinLoadType, the port inherits the PinLoadType value from the connected port on the next higher level design. For top-level output ports, the default PinLoadType value is `{=select_by_class combinational}`.

Examples

To set the expected load of the pdata_out pin to 16 high drive strength buffers:

```
coreConsultant> set_port_attribute pdata_out PinLoadCount 16
coreConsultant> set_port_attribute pdata_out PinLoadType
{=select_by_function "buf" "high"}
```

See Also

set_port_attribute (2), select_by_class (2), select_by_function (2), _____
select_by_name (2), explicit_capacitance (2), PinLoadCount (3) _____

[API Index](#) [Command Reference](#)

PortDirection

Signal flow direction of this port

Definition

Type: string**Flags:** readOnly**Default value:****Valid on:** port _____

Description

PortDirection is a read-only attribute that indicates the direction of signal flow for a port (in, out, or inout).

coreBuilder automatically sets PortDirection based on the port direction declaration in the HDL code for the design. Other port attributes are valid or not valid on a port depending on PortDirection. For example, MaxInputDelay is not valid on a port that has a PortDirection of out.

Examples

To get the direction of the dout port:

```
coreConsultant> get_port_attribute dout PortDirection
out
```

To find all ports of the current_design of direction inout:

```
coreConsultant> find_item -type port -filter {PortDirection==inout}
{addr_bus}
```

See Also

[get_port_attribute \(2\)](#), [find_item \(2\)](#)_____

[API Index](#) [Command Reference](#)

PortFunction

Function of the port/pin/bit: S(ource), L(oad), or B(oth)

Definition

Type: string

Flags: readOnly

Default value:

Valid on: busBit pin port _____

Description

Indicates whether the given port/pin/bit is a source, a load, or both.

Sources are input ports and output pins. Loads are output ports and input pins. Items listed as both have directionality inout.

Examples

See Also

[PortDirection \(3\)](#)

[API Index](#) [Command Reference](#)

PortWidth

Indicates the width of the port.

Definition

Type: string

Flags: readOnly

Default value: =sHdl::getPortWidth %item

Valid on: port _____

Description

Read-only attribute which provides the width of the port. Used to get a width value to be included in generated documentation.

Examples

See Also

PostPromptCmd

Command(s) to be evaluated when completing an activity, after the parameters are successfully validated.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: Cancel Ok

Default subscript: Ok

Valid on: filegroup_____

Description

The PostPromptCmd attribute stores the name(s) of one or more Tcl commands to be executed after coreConsultant or coreBuilder posts the dialog for an activity. If the user clicks the OK button in the activity dialog, the coreTool executes the command(s) listed in PostPromptCmd[Ok]. If the user clicks the Cancel button in the activity dialog, coreConsultant or coreBuilder executes the command(s) listed in PostPromptCmd[Cancel].

To add a Tcl command to PostPromptCmd[Ok] use the add_activity_hook command with the -after option. To add a Tcl command to PostPromptCmd[Cancel] use the add_activity_hook command with the -cancel option.

Examples

To check existing PostPromptCmd[Ok] for BuildcoreKit activity and then add the custom command My_Build_command:

```
coreBuilder> get_attribute -attr PostPromptCmd
-subscript Ok BuildcoreKit
sDevKb::buildDesignKit %item
coreBuilder> add_activity_hook -after BuildcoreKit
My_Build_command
coreBuilder> get_attribute -attr PostPromptCmd
-subscript Ok BuildcoreKit
sDevKb::buildDesignKit %item ; My_Build_command
```

See Also

[add_activity_hook \(2\)](#), [plugin_proc \(2\)](#), [get_attribute \(2\)](#), [PrePromptCmd \(3\)](#)

[API Index](#) [Command Reference](#)

PowerDomain

Indicates that the given port is associated with the indicated power domain (which must be enumerated in the PowerDomains attribute on the containing design).

Definition

Type: string

Flags:

Default value: =get_attribute [get_attribute [sUtils::getTopConnItem %item] -attr Parent] -attr TopLevelPowerDomain

Valid on: busBit port

Description

Indicates the power domain to which this port belongs. The value must also be in the list of defined power domains for the design as specified in the list of values set in the PowerDomains attribute on the containing design.

The ports of a given power domain can be accessed using the get_upf_port_names command. This is typically used to enable configuration of a UPF file where the set ports in a given power domain is dynamic.

Examples

```
set_port_attribute MyPort PowerDomain AlwaysOn
```

See Also

[PowerDomains \(3\)](#), [get_upf_port_names \(2\)](#)

PowerDomains

Defines the set of valid power domain names for this design.

Definition

Type: string

Flags:

Default value: =sUtils::collection_to_string [find_item -type
genHierItem UPF_power_domain* -filter Visible==1 -quiet] UPFDomain

Valid on: design_____

Description

This attribute is used to define the complete set of valid power domains for the design. This list controls the legal value for the PowerDomain attribute on the ports of this design.

set_design_attribute PowerDomains {AlwaysOn Hibernation}

See Also

[PowerDomain \(3\)](#)

PrePromptCmd

Command(s) to be evaluated before querying and validating the parameters for an activity

Definition

Type: string

Flags:

Default value: sHdl::defaultCurrentDesignToTop

Valid on: filegroup_____

Description

The PrePromptCmd attribute stores the name(s) of one or more Tcl commands to be executed before coreConsultant or coreBuilder posts the dialog for an activity. For example, before posting the user dialog for the Load Designs activity, coreBuilder executes the default PrePromptCmd (sHdl::defaultCurrentDesignToTop).

To add a custom Tcl command to the PrePromptCmd attribute, use the add_activity_hook command with the -before option.

Examples

To check an existing PrePromptCmd for LoadDesigns activity and then add a custom command My_LD_command:

```
coreBuilder> get_attribute -attr PrePromptCmd LoadDesigns
sHdl::defaultCurrentDesignToTop
coreBuilder> add_activity_hook -before LoadDesigns My_LD_command
coreBuilder> get_attribute -attr PrePromptCmd LoadDesigns
sHdl::defaultCurrentDesignToTop ; My_LD_command
```

See Also

[add_activity_hook \(2\)](#), [get_attribute \(2\)](#), [PostPromptCmd \(3\)](#) _____

[API Index](#) [Command Reference](#)

PredefinedInoutPorts

List of inout ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is used to define a list of inout ports that should be created prior to making automatic connections between interfaces. This enables the creation of ports that are wider than those needed for the automatic connections so that the same port can be used in multiple automatic or manual connections. The value of this attribute is a simple list of port names which may include bit designators.

Examples

Define two extra input ports:

```
coreAssembler> set_design_attribute PredefinedInoutPorts {Data  
Address[63:0]}
```

See Also

[PredefinedOutputPorts \(3\)](#) [PredefinedInputPorts \(3\)](#)

[API Index](#) [Command Reference](#)

PredefinedInputPorts

List of input ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is used to define a list of input ports that should be created prior to making automatic connections between interfaces. This enables the creation of ports that are wider than those needed for the automatic connections so that the same port can be used in multiple automatic or manual connections. The value of this attribute is a simple list of port names which may include bit designators.

Examples

Define two extra input ports:

```
coreAssembler> set_design_attribute PredefinedInputPorts {Data  
Address[63:0]}
```

See Also

[PredefinedOutputPorts \(3\)](#) [PredefinedInoutPorts \(3\)](#)

[API Index](#) [Command Reference](#)

PredefinedOutputPorts

List of output ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is used to define a list of output ports that should be created prior to making automatic connections between interfaces. This enables the creation of ports that are wider than those needed for the automatic connections so that the same port can be used in multiple automatic or manual connections. The value of this attribute is a simple list of port names which may include bit designators.

Examples

Define two extra input ports:

```
coreAssembler> set_design_attribute PredefinedOutputPorts {Data  
Address[63:0]}
```

See Also

[PredefinedInputPorts \(3\)](#) [PredefinedInoutPorts \(3\)](#)

[API Index](#) [Command Reference](#)

PreserveDesignWare

Indicates that the DesignWare parts in this design should not be ungrouped.

Definition

Type: boolean

Flags:

Default value: =InheritValue up 0

Valid on: design_____

Description

This attribute can be set on a design to indicate that the DesignWare parts within this design should not be ungrouped if and when the this design is ungrouped. This is equivalent to putting a dont_touch attribute on each of the DesignWare parts within this design.

Examples

To prevent ungrouping of DesignWare parts in the current design:

```
prompt> set_design_attribute PreserveDesignWare true
```

See Also

[PreserveMuxOps \(3\)](#)

PreserveHierarchyFromTop

Defines lower level designs and cells to be preserved during synthesis.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: cells designs hierCells

Default subscript:

Valid on: design_____

Description

This attribute is used to prevent ungrouping of lower level designs and/or cells during synthesis. To specify design names, use the 'designs' subscript. To specify cell (instance) names, use the 'cells' subscript. Names can be simple names, or glob-style names. If lower level instances need to be specified without a full instance path from the top design, use the 'hierCells' subscript. Those cells will be located using [get_cells -hierarchical <pattern>] instead of just a call to get_cells. Use of this approach is discouraged since the partial instance pattern can match additional cells in a coreAssembler generated subsystem. The specified names are passed to the appropriate get_<object> commands in the generated Fusion Compiler and Design Compiler scripts to locate the objects to which the set_ungroup command will apply.

Examples

Prevent ungrouping of design D1, designs beginning with D2, and the design D3:

```
prompt> set_design_attribute {PreserveHierarchyFromTop[designs]} {D1 D2* D3}
```

Prevent ungrouping of cell U1 and all cells in U2/U3 beginning with U.

```
prompt> set_design_attribute {PreserveHierarchyFromTop[cells]} {U1 U2/U3/U*}
```

See Also

[API Index](#) [Command Reference](#)

PreserveMuxOps

Indicates that the MuxOps in this design should not be ungrouped.

Definition

Type: boolean

Flags:

Default value: =InheritValue up 0

Valid on: design_____

Description

This attribute can be set on a design to indicate that the MUX operators within this design should not be ungrouped if and when the this design is ungrouped. This is equivalent to putting a dont_touch attribute on each of the MUX operators within this design.

Examples

To prevent ungrouping of MUX operators in the current design:

```
prompt> set_design_attribute PreserveMuxOps true
```

See Also

[PreserveDesignWare \(3\)](#)

PrimeProfileAllowNone

If true, users are allowed to skip Prime Profile application in SelectPrimeProfile activity.

Definition

Type: boolean

Flags:

Default value: 1

Valid on: design_____

Description

During packaging IP designers can set (default) or unset this boolean attribute to control whether a dedicated option '<None>' appears in the SelectPrimeProfile activity's Prime Profile selection drop down list.

- When default Prime Profiles are available, and PrimeProfileAllowNone is 1, then the '<None>' option appears after the default profiles in the drop down list.
- When default Prime Profiles are available, and PrimeProfileAllowNone 0, the '<None>' option is absent altogether.
- When default Prime Profiles are not available, and PrimeProfileAllowNone is 1, then the '<None>' option appears at the top of the drop down list and is the implicit selection unless modified by users.
- When default Prime Profiles are not available, and PrimeProfileAllowNone is 0, then the first entry in the drop down is '<Please Select A Prime Profile>' implying users must explicitly select a Prime Profile in order to complete the activity successfully.

Note

To make it explicit that a user has selected the '<None>' option, batch scripts generated include the following command before SelectPrimeProfile activity is completed. This is done to ensure default Prime Profiles are not applied when not intended.

```
set_prime_profile -tag <None>
autocomplete_activity SelectPrimeProfile
```

Examples

```
coreBuilder> set_design_attribute PrimeProfileAllowNone 0
```

See Also

set_prime_profile (2) set_default_prime_profiles (2) PrimeProfileDefaults
(3) PrimeProfilesValid (3)

[API Index](#) [Command Reference](#)

PrimeProfileCurrent

Design attribute to store the current prime profile.

Definition

Type: string**Flags:** readOnly**Default value:****Valid on:** design_____

Description

In consultant and assembler, if triggered prime profiles exist for a leaf component design, a prime profile must be set on the component design. This restricts the configuration to the one specified by the prime profile file.

Examples

The following command will set the current prime profile to Test001 on the current component. The current prime profile must be in the list of triggered prime profiles set by configuring trigger parameters and values. The triggered prime profiles may be set after current prime profile but SpecifyConfiguration or SpecifySubsystem activities will only proceed if current prime profile is found in the list of triggered prime profiles.

```
coreConsultant> set_prime_profile -tag Test001
```

```
coreConsultant> get_attribute [current_design -quiet] -attr  
PrimeProfileCurrent Test001
```

```
coreAssembler> set_prime_profile -tag Test001 -component /i_core1
```

See Also

[set_prime_profile \(2\)](#)

[API Index](#) [Command Reference](#)

PrimeProfileDefaults

Design attribute to store Default Prime Profile tags.

Definition

Type: string

Flags: readOnly

Default value:

Valid on: design_____

Description

The list of Prime Profile tags specified in the `set_default_prime_profiles` command. It should be noted that profiles stored in this attribute may not always be valid (e.g. if the license string associated with the Prime Profile is false during creating or opening a workspace).

Examples

```
coreBuilder> set_default_prime_profiles -tags "DisableRom1
DummyCheckbox2"
coreBuilder> get_design_attribute PrimeProfileDefaults
DisableRom1 DummyCheckbox2
coreConsultant> get_design_attribute PrimeProfileDefaults
DisableRom1 DummyCheckbox2
```

See Also

[PrimeProfileCurrent \(3\)](#) [PrimeProfilesValid \(3\)](#) [set_default_prime_profiles \(2\)](#) [set_prime_profile \(2\)](#)

[API Index](#) [Command Reference](#)

PrimeProfileNote

A note to be displayed in SelectPrimeProfile activity.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

Stores the text that appears in the SelectPrimeProfile activity page.

This note can be used by the IP team to provide additional information about Prime Profile usage.

Examples

For ease of use this attribute should be in packaging scripts where Prime Profile validation expressions are defined.

```
coreBuilder> set_design_attribute PrimeProfileNote "Test note for Prime Profile."
```

See Also

[PrimeProfileCurrent \(3\)](#) [PrimeProfilesValid \(3\)](#) [set_default_prime_profiles](#)

[\(2\)](#) [set_prime_profile \(2\)](#)

PrimeProfilesValid

Design attribute to store valid prime profile tags.

Definition

Type: string

Flags: readOnly

Default value:

Valid on: design_____

Description

When a workspace is created or opened in coreConsultant or coreAssembler, this attribute is updated with a list of all Prime Profiles which have a valid license string. This attribute only stores those Prime Profiles which do not have trigger parameters or check expressions associated with them. Only profiles names listed in this design attribute will be displayed in the SelectPrimeProfile activity.

Examples

In coreBuilder, it simply stores a list of all Prime Profiles created with the create_prime_profile_file command.

```
coreBuilder> get_design_attribute PrimeProfilesValid
BadLicense BadLicense2 DisableRom1 DisableRom2 DontLockProfile
DummyCheckbox DummyCheckbox2 DummyExpr1 DummyRadio LockRamSize Test003
```

If a user does not have valid licenses for the profiles 'BadLicense' and 'BadLicense2' from the above list, coreConsultant and coreAssembler workspaces will update the design attribute as follows.

```
coreBuilder> get_design_attribute PrimeProfilesValid
DisableRom1 DisableRom2 DontLockProfile DummyCheckbox DummyCheckbox2
DummyExpr1 DummyRadio LockRamSize Test003
```

If the PrimeProfilesValid design attribute is empty, the SelectPrimeProfile activity will not be visible.

See Also

[create_prime_profile_file \(2\)](#) [set_prime_profile \(2\)](#) [report_prime_profiles \(2\)](#) [PrimeProfileDefaults \(3\)](#) _____

[API Index](#) [Command Reference](#)

ProjectID

Used to record the project ID associated with a workspace.

Definition

Type: string

Flags: readOnly

Default value:

Valid on: knowledgeBase

Description

Examples

See Also

[LicenseForOpen \(3\)](#), [check_license \(2\)](#),

ProjectRootDir

The logical root directory of a core project.

Definition

Type: string

Flags:

Default value: .

Valid on:

Description

Examples

See Also

RALAccessType

Defines the VMM RAL access type for this register field. In most cases this is calculated via the MemoryAccess, WriteBehavior, and ReadAction values for the field. This is typically only set explicitly when the a0 or a1 types are desired.

Definition

Type: string

Flags:

Default value: =sMem::RALAccessType %item

Valid on:

Description

This attribute indicates the access type for a register field in RAL. It is not normally set explicitly as the value is calculated from other attributes (MemoryAccess, VolatileMemory, ReadAction, WriteBehavior). If the default RAL access type is not being calculated as desired, then the value can be set explicitly to any legal RAL access type value: rw, ro, wo, w1, w1c, ru, rc, a1, a0, dc, other, user1, user2, user3.

Explicit setting of this attribute is normally only required to get one of the a* or user* values.

Calculation of the default value for this attribute works as follows:

If the field is volatile, then the RAL access type is 'ru' for a read-only field and 'other' for all other access types.

If the field does not have any defined write behavior or read action, then the RAL access type is defined by the field access type.

If the field has both a write behavior and a read action, then the RAL access type is set to 'other'.

If the field has only a write behavior defined, then the RAL access type is set to 'other' except for read-write fields where the write behavior is set to 'oneToClear', in which case the RAL access type is set to 'w1c'.

If the field has only a read action defined, then the RAL access type is set to 'other' except for read-only fields where there read action is set to 'clear', in which case the RAL access type is set to 'rc'.

Examples

set_register_field_attribute \$field RALAccessType user2

See Also

MemoryAccess (3), VolatileMemory (3), ReadAction (3), WriteBehavior (3) _____

RALAdditionalFieldAttributeText

For RAL (Register Abstraction Language) files, this attribute defines additional text to be added to the definition of each RAL field. This attribute can be used to specify RAL constraints.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This attribute can be used to supply any additional text to add as part of the RAL description of a register field. Use this attribute to specify constraints on the field value when it is randomized.

Examples

To insert a constraint associated with field 'my_field':

```
cB> set r "my_map/my_block/my_register/my_field"
cB> set_register_field_attribute $r RALAdditionalFieldAttributeText \
"constraint valid { value != 2'b00 }"
```

See Also

RALSuppressionComment

Reason for skipping test on register, register field or register array.

Must not be left empty.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

RTL Static Checking

Perform Design Verification of the subsystem and its components.

Parameters

Parameters Table

Name	Label	Description
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieving using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
ParallelJobCpuLimit	Parallel job CPU limit:	Specifies the number of parallel compile jobs that may be run.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.
UseBlockingHoldJob	Report generation waits for grd/lsf jobs	If verification jobs uses RunStyle=grd lsf they are launched with qsub/bsub. The report generation job holds for the verification jobs to complete but can run in 2 ways. 1. A non blocking hold job. Use qsub/bsub to schedule a job that will hold for the verification jobs to complete. - this means the activity will return as soon as the verification job is created.

coreTools Command Reference Index

Name	Label	Description
		1. A blocking hold job. Use qrsh/bsub -K to create a job that will hold for the verification jobs to complete. • this means the activity will wait until the report job is complete before returning
Setup/Constraints		
TopConstraintFiles	Additional SDC Files	List of toplevel additional constraints files containing the clock definition and other delay related informations. There can be multiple set of files and specified as {FileA FileB FileC}. coreTools will by default generate a top level sdc which will be placed in vcs SpyGlass/design_constraints folder. The additional files specified by the user, will be appended along with the coreTools generated sdc when it is read during the verification.
ConstraintFiles	Constraint Files	List of constraints files containing the quasistatic signals information. There can be multiple set of files provided as {FileA FileB FileC}
ResetFiles	Reset Files	List of toplevel reset files. There can be multiple set of files and specified as {FileA FileB FileC}.
WaiverFiles	Waiver Files	List of waiver files. There can be multiple set of files and specified as {FileA FileB FileC}.
WriteSdcArgs	Write SDC Args	Arguments to the write_sdc call
Setup/DW Components		
DwComponentPreCompileEnable	Use Pre-compiled	Use the pre-compiled DesignWare components that are part of the VC SpyGlass release. The versions of the components are aligned with the latest release of DC at the time of the VC SpyGlass release.
DwComponentCompileDir	Compile Dir	Folder with pre-compiled DesignWare Components. You can pre-compile the components to the folder myDwComponentDir with the following command <pre>\$VC_STATIC_HOME/auxx/monet/scripts/vsi -full64 \$SYNOPSYS myDwComponentDir</pre> This feature is optionally enabled in the coreKit. This feature is disabled if using the pre-compiled libraries is enabled.
Setup/Engineering Council Rule		
EngineeringCouncilRule	Engineering Council Rule File	VC SpyGlass Engineering Council Rule File. The default rule file is calculated using the algorithm: 1. Gather all Engineering Council versions from the components. 2. Identify the latest Engineering Council version from the collected versions.

coreTools Command Reference Index

Name	Label	Description
		3. Check if the identified version is available in the tool image. If yes, use it. If no, find the most recent Engineering Council rule file from the tool image that is closest to the identified version in step 2.
Setup/Other		
AnalyzeVcsArgs	Analyze VCS Args	Extra args to VCS as part of the VC SpyGlass command analyze. See VC SpyGlass documentation for more information.
ElaborateArgs	Elaborate Args	Extra arguments to the VC SpyGlass command elaborate. See VC SpyGlass documentation for more information.
AdditionalRtlFiles	Additional RTL Files	List of Additional RTL Files.
AdditionalIncludeDirs	Additional Include Directories	List of include directories to be used for searching files specified in Parameter AdditionalRtlFiles.
VerifyThreads	Verify Threads	The check_<verif> supports multiple threads.
SessionMode	Generate Sessions	Generate Session. See VC SpyGlass documentation for more details on sessions. • EnumValues: Disabled Enable Auto
Setup/TechSetup		
UseTechLibraries	Use Technology Libraries	Use Technology Libraries defined in the activity TechSetup. This activity must be completed.
Setup/VC Static		
VcStaticHome	VC_STATIC_HOME	Tool root for vc_static_shell, specified in tool root settings.
Setup/Verification		
EnableBottomUpFlow	Enable Bottom Up Verification	Enable Bottom Up Verification. This allows abstract generation at the Sub Block in a hierarchy and read in at the top. This activity parameter can be only enabled when the design attribute RTLStaticCheckingAsSubblock is set to atleast one Sub Block.
LintEnable	Lint Verification	Run Lint Verification. Verification is performed in the workspace folder vcspyglass/lint. The toplevel report vcspyglass/vcspyglass.html has links to the associated log and report files.
CdcEnable	CDC Verification	Run CDC Verification. Verification is performed in the workspace folder vcspyglass/cdc. The toplevel report vcspyglass/vcspyglass.html has links to the associated log and report files.
RdcEnable	RDC Verification	Run RDC Verification. Verification is performed in the workspace folder vcspyglass/rdc. The toplevel report vcspyglass/vcspyglass.html has links to the associated log and report files.

Name	Label	Description
ConstraintsEnable	Constraints Verification	Run Constraints Verification. Verification is performed in the workspace folder vcspyglass/constraints. The toplevel report vcspyglass/vcspyglass.html has links to the associated log and report files.
RcaEnable	Enable Root Cause Analysis	Enable Root Cause Analysis. This is a feature of VC SpyGlass and the user should refer to the VC SpyGlass documentation for more information.
AbstractEnable	Enable Abstract	Enable Abstract Generation. When enabled, cdc and rd verification will generate an abstract model in the verification subfolder abstract. See VC SpyGlass documentation for more information.

See Also

Initialize Subsystem Constraints,

[API Index](#) [Command Reference](#)

RTLStaticCheckingAsSubblock

The attribute indicates that design needs abstract generation and to verified using buttom up flow using VC Spyglass in coreAssembler.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Examples

See Also

RTL_Architect_Reference_Methodology

Runs RTL Architect under the RTL Architect Reference Methodology associated with the version of RTL Architect being used.

Parameters

Strategy has no parameters.

See Also

[API Index](#) [Command Reference](#)

RalListInfo

Specifies the Ral list info for tb mode used for the workspace.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

Examples

See Also

ReadAction

Indicates that reading the given field will cause the field value to be modified as described by the value of this attribute.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: DEFAULT

Valid on:

Description

This attribute indicates the impact of reading the given register field. This information can be used when writing register tests to determine whether the test program can recover from the impact of reading the register.

Valid values are: clear, set, and modify. The latter value is meant to indicate that reading causes a change in value which cannot be described by the other enumerations.

Examples

See Also

[WriteBehavior](#)

ReadActionDescription

Indicates that reading the given field will cause the field value to be modified as described by the value of this attribute.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

ReadActionModifier

Indicates user-defined reading action when ReadAction is set to 'modify'.

It implies this read action is not covered by any of other allowed

ReadAction values which are: {clear, set}. This attribute is only valid when ReadAction is set to 'modify'

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: DEFAULT

Valid on:

Description

Examples

See Also

[API Index](#) [Command Reference](#)

ReadOnlyInterface

Indicates that this interface connection cannot be modified manually.

Definition

Type: boolean

Flags:

Default value: 0

Valid on:

Description

This attribute is usually used on an internal-consumer interface so that interface connections cannot be changed.

Examples

See Also

ReadOnlyParam

The value of this parameter is read-only

Definition

Type: boolean

Flags:

Default value: 0

Valid on: param_____

Description

The ReadOnlyParam attribute indicates whether the parameter's value can be modified by a user. If ReadOnlyParam is true, the parameter's value is read-only and cannot be modified by a user.

If Visible and ReadOnlyParam are both true on a parameter, the parameter appears in the GUI parameter dialog, but its value cannot be modified by a user. To exclude the parameter from the parameter dialog, set the parameter's Visible attribute to false.

ReadOnlyParam is typically set to true on a parameter that has a value that is a function of another parameter and cannot be set to a different value.

Examples

The following VHDL code annotations set the default value of parameter B as a function of parameter A. The value of B is read-only (cannot be modified by the user):

```

--reuse-pragma startSub Generics [ReplaceDesignParams TOP %subText]
generic (
--reuse-pragma attr MinValue 1
--reuse-pragma attr MaxValue 4
A : integer := 2;
--reuse-pragma attr MinValue 1
--reuse-pragma attr MaxValue 8
--reuse-pragma attr DefaultValue =@A*2
--reuse-pragma attr ReadOnlyParam true
B : integer
);
--reuse-pragma endSub Generics

```

The following line in a _Obj.tcl file specifies that activity parameter MyParam is a read-only parameter:

```
set_parameter_attribute MyParam ReadOnlyParam true
```


See Also

set_parameter_attribute (2), DefaultValue (3), Visible (3), ReplaceInHDL
(3)

[API Index](#) [Command Reference](#)

ReadOnlyWorkspaceComponent

Indicates the workspace component will remain read-only after completing specify subsystem.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: cell _____

Description

Examples

See Also

[IsWorkspaceComponent \(3\)](#), [ImportedWorkspaceComponentFile \(3\)](#),
[export workspace as component \(2\)](#), [import_workspace as component \(2\)](#)

ReferenceClock

Indicates a generated clock defined for reference purposes only. A reference clock is used to enable clock linking in subsystem assembly but is not written to any SDC file.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: clock_____

Description

Reference clocks are created to guide the clock linking process in coreAssembler for cases where the tool will not see that the two clocks are explicitly connected. The common use case for this type of clock is within clock generation blocks where an input clock is defined on a primary input and then a reference clock is defined on a primary output to indicate that the clock is propagated out of the module. Reference clocks are created when there is not going to be any explicitly created generated clock. The reference clock links the input clock to the output port so that any clocks driven by the output port can be properly associated (linked) to the clock defined on the input port.

Examples

Block clkgen has an input named clkin and an output named clkout. A clock named clkin is defined on the input port clkin. This clock propagated out of the clkgen module via the port clkout, though it is sometimes muxed off. Definition of the reference clock would look like:

```
create_generated_clock -name refclk -master_clock clkin \
-source [get_ports clkin] -divide_by 1 [get_ports clkout]
set_clock_attribute refclk ReferenceClock true
```

Note that the -master_clock option must be used when defining the clock so that the connection to the master clock is explicitly defined.

See Also

[create_generated_clock \(2\)](#)_____

[API Index](#) [Command Reference](#)

ReferenceClockName

For a virtual clock, stores name of the reference real clock in the same hierarchy.

Definition

Type: string

Flags:

Default value:

Valid on: clock_____

Description

For designs synthesized using Fusion Compiler, this attribute is used to add commands to the generated intent for QiK synthesis run.

The following command added to the intent:

```
# virtual clocks relations
set_latency_adjustment_options -clocks_to_update $virt_clocks
-reference_clock $ref_clock ;
```

Examples

This attribute should be set by IP designers during packaging as follows:

```
coreConsultant> set_clock_attribute virClk1 ReferenceClockName realClk1
```

See Also

[set_clock_attribute \(2\)](#)_____

[API Index](#) [Command Reference](#)

RegisterArrayDimensions

Specifies the size of the RegisterArray.

Definition

Type: long

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on:

Description

This attribute specifies the register arrays for the address block or register array.

Examples

See Also

coreBuilder> set_memory_map_attribute map1/block1/regArray1

RegisterArrayDimensions[0] 4

addressBlock (3), create_register_array (2) remove_register_array (2)

set_register_array_attribute (2), get_register_array_attribute (2),

AddressOffset (3), MemoryRange (3), RegisterArrayDimensions (3), register

(3)

[API Index](#) [Command Reference](#)

RegisterResetMask

Mask to be anded before comparing to the RegisterResetValue

Definition

Type: bitfield

Flags:

Default value: =sMem::calcRegisterAttrValue %item %attrValue

RegisterResetMask

Valid on:

Description

The value to use as a mask when comparing against the RegisterResetValue.

The default value is all ones.

Examples

See Also

[RegisterResetValue \(3\)](#)

[API Index](#) [Command Reference](#)

RegisterResetValue

Value of the register at reset

Definition

Type: bitfield

Flags:

Default value: =sMem::calcRegisterAttrValue %item %attrValue

RegisterResetValue

Minimum value: 0x0

Maximum value: =sMem::maxValForSize %item

Valid on:

Description

The value of this register in the reset state.

Examples

See Also

[RegisterResetMask \(3\)](#)

[API Index](#) [Command Reference](#)

RegisterSize

Specifies the size of the Register or register field in bits

Definition

Type: long

Flags:

Default value:

Valid on:

Description

This attribute specifies the size of the register or register field in bits.

Examples

To specify the register reg1's size to be 64:

```
coreBuilder> set_register_attribute reg1 RegisterSize 64
```

To specify the register field field1's size to be 2:

```
coreBuilder> set_register_field_attribute field1 RegisterSize 2
```

See Also

Registered

Indicates that the given port is registered within the component. It is either directly connected to a register or connected via only buffers or inverters with no intervening logic.

Definition

Type: boolean

Flags:

Default value:

Valid on: busBit pin port_____

Description

Indicates that the given port is registered within the component. It is either directly connected to a register or connected via only buffers or inverters with no intervening logic.

Examples

See Also

RegisteredBy

Indicates that the given port is registered within the component. It is either directly connected to a register or connected via only buffers or inverters with no intervening logic.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: Default

Valid on: busBit pin port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

RemoveIfEmpty

Specifies that this file can be deleted from disk if it is empty.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: file _____

Description

This attribute is used to indicate that the specified file should be removed from disk if, when it is written, it has a size of 0 bytes. This is only useful for configurable files, which are sometimes empty and sometimes not, based on the configuration of the design. Typically this would be done by putting the entire body of the file inside of a substitution pragma which conditionally includes the body of the file. If the body of the file is not included, then the file is removed from disk, effectively appearing as if it was never written to disk.

Examples

To define a file to be removed if empty:

```
prompt> set_attribute theFile -attr RemoveIfEmpty -value true
```

See Also

ReplaceConstantParam

Replace the definition of a constant

Syntax

string **ReplaceConstantParam** *text*
string *text*

Parameters

	The text to replace. Error if format is bad
	In most cases, it is more useful to use the string "%subText" instead of actual text. Using "%subText" allows you to enter
text	the actual constant definition in the the HDL file. When coreConsultant generates configured HDL, it substitutes the value of the constant (parameter) with the user-specified value.

Description

The ReplaceConstantParam command is part of the coreTools text substitution mechanism and is used for designs that implement configurability through the use of HDL constants (constant in VHDL, 'define in Verilog). There is a similar command (ReplaceDesignParams) that you can use with designs that implement configurability through VHDL generics or Verilog parameters. (In the coreTools, the term "design parameter" refers to any VHDL constant or generic or Verilog 'define or parameter for which the user modifies the value to configure the reusable core.)

By using ReplaceConstantParam with "reuse-pragma startSub", you can designate a VHDL constant definition or Verilog 'define to be replaced with a similar line that contains the user-defined value for the parameter when coreConsultant generates the user-specified configuration of the core.

You can specify the constant definition directly as the value of the <text> argument. However, it is generally more useful to specify "%subText" as the value of the <text> argument. This enables you to enter the actual VHDL constant or Verilog 'define directly in the HDL source code file. When coreConsultant generates a user-configuration of the file, coreConsultant substitutes the original constant definition with a similar line of code that contains the user-specified value for the parameter.

Refer to the Text Substitution section of the *coreBuilder User Guide* for

more information about how to implement text substitution in files.

Examples

To replace the value of the Width parameter in a VHDL package file with the user-specified value when the user executes the coreConsultant Specify Configuration activity, add the following reuse-pragma:

```
-- reuse-pragma startSub [ReplaceConstantParam %subText]  
constant Width : integer := 5;
```

To replace the value of the Width parameter in a Verilog include file with the user-specified value when the user executes the coreConsultant Specify Configuration activity, add the following reuse-pragma:

```
// reuse-pragma startSub [ReplaceConstantParam %subText]  
'define Width 5
```

See Also

[ReplaceDesignParams \(2\)](#), [IncludeIf \(2\)](#)

ReplaceDesignParams

Replace a set of design parameters with user-specified values

Syntax

string **ReplaceDesignParams** *design text*

string *design*

string *text*

Parameters

design	The design for which to replace the design parameter values. The text to replace In most cases, it is more useful to use the string "%subText" instead of actual text. Using "%subText" allows
text	you to enter the actual VHDL generic or Verilog parameter definitions directly into the HDL source file. When coreConsultant generates configured HDL, it substitutes the values of the generics/parameters with the user-specified value.

Description

The ReplaceDesignParams command is part of the coreTools text substitution mechanism and is used for designs that implement configurability through the use of HDL parameters (generics in VHDL, parameters in Verilog). There is a similar command (ReplaceConstantParam) that you can use with designs that implement configurability through VHDL constants or Verilog 'define statements. (In the coreTools, the term "design parameter" refers to any VHDL constant or generic or Verilog 'define or parameter for which the user modifies the value to configure the reusable core.)

By using ReplaceConstantParam with "reuse-pragma startSub" and "reuse-pragma endSub", you can designate a VHDL generic block or set of Verilog parameter definitions to be replaced when coreConsultant generates the user-specified configuration of the core. The block of text that coreConsultant inserts into the file at configuration time is the same as the original text, but contains the user-specified values for the generics/parameters.

You can specify the generic or parameter definition directly as the value of the <text> argument. However, it is generally more useful to specify "%subText" as the value of the <text> argument. This enables you to enter the actual VHDL generic block or Verilog parameter block directly

in the HDL source code file. When coreConsultant generates a user-configuration of the file, coreConsultant substitutes the original generic/parameter definitions with similar lines of code that contain the user-specified values for the generics/parameters.

Refer to the Text Substitution section of the *coreBuilder User Guide* for more information about how to implement text substitution in files.

Examples

To replace the values of the VHDL generics width and height with the user-specified values when the user executes the coreConsultant Specify Configuration activity, add the following reuse-pragmas:

```
entity TOP is
-- reuse-pragma startSub GenericBlock1 [ReplaceDesignParams TOP %subtext]
generic (width : integer := 4,
height : integer := 8);
-- reuse-pragma endSub GenericBlock1
```

To replace the values of the Verilog parameters width and height with the user-specified values when the user executes the coreConsultant Specify Configuration activity, add the following reuse-pragmas:

```
module TOP
(port list);
// reuse-pragma startSub ParameterBlock1 [ReplaceDesignParams TOP
%subtext]
parameter width = 4;
parameter height = 8;
// reuse-pragma endSub ParameterBlock1
```

See Also

[ReplaceConstantParam \(2\), IncludeIf \(2\)](#)

ReplaceFormatParam

Replace text with a formatted parameter value

Syntax

```
string ReplaceFormatParam format param
      string format
      string param
```

Parameters

	The format string to direct the replacement
format	This string can be any value that is legal for the <code>mpformat(2)</code> command.
param	The name of the parameter to replace

Description

The ReplaceFormatParam command is part of the coreTools text substitution mechanism. It is used mainly by core developers to format parameter values within a configurable file.

By using ReplaceFormatParam with "reuse-pragma startSub", you can designate a line or region in a file as configurable and insert a formatted parameter value when the filegroup containing the configurable file is written to disk.

Refer to the Text Substitution section of the *coreBuilder User Guide* for more information about how to implement text substitution in files.

Examples

To replace the value of a testbench parameter in a VHDL file with the user-specified value, add the following reuse-pragma:

```
-- reuse-pragma startSub [ReplaceFormatParam {constant p1 : integer :=
"%d"} p1]
constant p1 : integer := 10001101;
```

Note: The reuse-pragma should not contain any line breaks.

See Also

[ReplaceDesignParams \(2\)](#), [ReplaceConstantParam \(2\)](#), [analyze filegroup \(2\)](#),
[IncludeIf \(2\)](#) reuse-pragma (2)

ReplaceInHDL

If true on a VHDL package parameter, then automatically replace the parameter definition in the source code. Defaults to false if there are no annotations on the parameter in the VHDL. For verilog 2001, set to false for conditional cells defined through generate-conditional statements.

Definition

Type: boolean

Flags:

Default value: 1

Valid on: cell param port _____

Description

This attribute can be used to override the default replacement insertion coreBuilder. By default, coreBuilder inserts replacement pragmas into your source code to replace all parameters with annotations on them. In certain situations, you do not want coreBuilder to do this replacement. Setting this attribute to false will prevent that. This attribute is generally used in conjunction with ReadOnlyParam.

Examples

The following VHDL code annotations set up a ReadOnlyParam that is needed to elaborate the design. The person who packaged this core did not want the value replaced:

```
-- reuse-pragma attr SymbolicNames false true
'define WIDE_MODE 1'b0
-- reuse-pragma attr ReadOnlyParam 1
-- reuse-pragma attr Visible 0
-- reuse-pragma attr ReplaceInHDL 0
'define PORT_WIDTH ((WIDE_MODE) ? 32 : 16)
```

See Also

[set_parameter_attribute \(3\)](#), [DefaultValue \(3\)](#), [ReadOnlyParam \(3\)](#) _____

ReplaceSingleBitBus

Convert single bit declarations into bit type

Syntax

```
string ReplaceSingleBitBus -design <des> -port <port> text
    string <des>
    string <port>
    string text
```

Parameters

-design <des>	The design name for this port
-port <port>	The port to convert
text	The text to replace

Description

The ReplaceSingleBitBus command is part of the coreTools text substitution mechanism and is automatically inserted by the LoadDesigns activity to replace port declarations on design ports that have the ConvertSingleBitBus attribute set to true.

If the port specified is a single bit, rather than a bus, then the declaration is re-written to define the port as a bit.

Examples

Given the following Verilog code fragment:

```
module top ( // reuse-pragma attr ConvertSingleBitBus true
sel
);
input ['NUM_SELECTS-1:0] sel;
```

In coreConsultant, if the user chose NUM_SELECTS to be 1, then the code generated would be:

```
module top (
sel
);
input sel;
```

It is up to you to write the code so that it is legal in all configurations. For example having a wire makes this simple:

```
wire ['NUM_SELECTS-1:0] sel_i
assign sel_i['WIDTH-1:0] = sel;
Now just use sel_i internally within the module.
```

The same example in VHDL would look like this:

```
entity top is
port (
-- reuse-pragma attr ConvertSingleBitBus true
sel : in std_logic_vector(NUM_SELECTS-1 downto 0)
);
end top;
```

After configuration the generated code would be:

```
entity top is
port (
sel : in std_logic
);
end top;
```

See Also

[ConvertSingleBitBus \(3\)](#), [GenerateIf \(3\)](#), [IncludeIf \(2\)](#)

RequiredExPortDirection

Required design port direction when associated with an exported interface port.

Definition

Type: string

Flags: readOnly

Default value: =sIntf::getRequiredExPortDirection %item

Valid on:

Description

This is a read-only attribute which appears in port association spreadsheets as an aide to determine the required directionality of associated ports. This attribute indicates the port directionality required on an exported design port if it is going to be associated with the given interface port.

Examples

Consider a 'fromProvider' interface port on an interface instantiated as an exported provider. If this port is going to be associated with a design port (i.e. one that goes to the outside of the subsystem), then the design port must have directionality 'input' since the actual provider (driver) is outside the subsystem.

See Also

[RequiredPortDirection \(3\)](#)

RequiredPortDirection

Required design port direction when associated with an interface port.

Definition

Type: string

Flags: readOnly

Default value: Value calculated internally.

Valid on:

Description

This is a read-only attribute which appears in port association spreadsheets as an aide to determine the required directionality of associated ports. This attribute indicates the port directionality required on an internal design port if it is going to be associated with the given interface port.

Examples

Consider a 'fromProvider' interface port on an interface instantiated as a provider. If this port is going to be associated with an internal design port (i.e. one that belongs to a component within the subsystem), then the design port must have directionality 'output' since the design port is the driver of the consumers associated with the provider interface port.

See Also

[RequiredExPortDirection \(3\)](#)

Reserved

Indicates that the bits Reserved (meaning unspecified or undocumented bitFields).

Definition

Type: boolean

Flags:

Default value: 0

Valid on:

Description

Indicates that the regster field is Reserved (meaning unspecified or undocumented bitFields). The attribute when set ensures that the field will be marked reserved in the component XML.

```
<ipxact:reserved>1</ipxact:reserved>
```

write_ral_file command also respects the 'Reserved' attribute on the field by introduced an attribute NO_RAL_TEST (VMM) or NO_REG_TESTS (UVM) in the generated file.

See Also

[write_ral_file \(2\)](#)

ResetProperties

Support modeling of external reset signals. active-low : active low reset
 active-high : active high reset synchronous : reset is synchronous ideal
 : treat as ideal network for synthesis no-dft : do not include DFT reset
 signal definition no-vcspyglass : do not include SpyGlass reset signal
 definition no-synthesis : do not include in synthesis constraints

Definition

Type: boolean

Flags: subscripted

Default value: 0

Valid subscripts: active-high active-low ideal no-dft no-synthesis
no-vcspyglass synchronous

Default subscript:

Valid on: busBit port _____

Description

The subscript indicates the attributes associated with the reset

- active-low : active low reset
- active-high : active high reset
- synchronous : reset is synchronous
- ideal : treat as ideal network for synthesis
- no-dft : do not include DFT reset signal definition
- no-vcspyglass : do not include SpyGlass reset signal definition
- no-synthesis : do not include in synthesis constraints

Examples

The attributes ResetProperties[active-low] will be set to true for the port 'presetn' This will help generate the set_dft_signal command.

- command exported : set_dft_signal -port [get_ports {presetn}] -type Reset -view existing_dft -active_state 0).

```
coreConsultant> set_attribute [find_item -type port presetn -quiet] -attr
ResetProperties -subs active-low -value 1
```

The attributes ResetProperties[active-low] and ResetProperties[synchronous] are set to true for the port 'presetn' This will export a 'create_reset' command in the VC Spyglass intent.

- command exported : create_reset presetn -sync -sense low

```
coreConsultant> set_attribute [find_item -type port presetn -quiet] -attr
ResetProperties -subs synchronous -value 1
```

coreTools Command Reference Index

```
coreConsultant> set_attribute [find_item -type port presetn -quiet] -attr  
ResetProperties -subs active-low -value 1
```

The attributes ResetProperties[ideal] will be set to true for the port 'presetn' This will help generate the set_ideal_network command. A side effect of setting ResetProperties[ideal] or attr IdealPort is that The attr DrivingCell gets explicitly set so in the SDC a 'set_drive' command is issued for the port.

- command exported : set_ideal_network -no_propagate [get_ports {presetn}]

```
coreConsultant> set_attribute [find_item -type port presetn -quiet] -attr  
ResetProperties -subs ideal -value 1
```

See Also

DrivingCell IdealPort

RtlaAuxScript

Specifies a RTL A script to be executed prior to specific optimization phase.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: cleanup post_commit post_conditioning
post_estimation post_explore post_floorplanning post_init_design
post_metrics post_read_rtl post_split_constraints pre_commit
pre_conditioning pre_estimation pre_explore pre_floorplanning
pre_metrics pre_read_rtl pre_split_constraints

Default subscript:

Valid on: design_____

Description

The RtlaAuxScript attribute specifies the name of a RTL Architect (RTLA) script to execute before or after a particular step of flat or explore flow for the selected design. As an example, you may want to perform special structuring or grouping on a design before a certain synthesis phase to improve timing or area.

The subscript indicates the stage in the RTL Architect flow where you want to execute the auxiliary script:

- cleanup - Execute script after all processing is complete, just before quitting.
- post_init_design - Execute script after init_design step.
- pre_conditioning - Execute script before conditioning
- post_conditioning - Execute script after conditioning
- pre_estimation - Execute script before estimation
- post_estimation - Execute script after estimation
- pre_split_constraints - Execute script before split constraints
- post_split_constraints - Execute script after split constraints
- pre_commit - Execute script before commit
- post_commit - Execute script before commit
- pre_floorplanning - Execute script before floorplanning
- post_floorplanning - Execute script after floorplanning
- pre_metrics - Execute script before metrics
- post_metrics - Execute script after metrics
- pre_explore - Execute script before explore
- post_explore - Execute script after explore

Examples

```
coreBuilder> set_design_attribute {RtlaAuxScript[post_metrics]}  
"./rtla_script.tcl"
```

See Also

RtlaAuxScriptComment

Specifies a comment to be issued for the corresponding RtlaAuxScript.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: cleanup post_commit post_conditioning
post_estimation post_explore post_floorplanning post_init_design
post_metrics post_read_rtl post_split_constraints pre_commit
pre_conditioning pre_estimation pre_explore pre_floorplanning
pre_metrics pre_read_rtl pre_split_constraints

Default subscript:

Valid on: design_____

Description

The RtlaAuxScriptComment attribute can be used to output a comment to document why the corresponding FcAuxSynthesisScriptComment is being used. The comment will be put into the file where the script is sourced.

Examples

See Also

[API Index](#) [Command Reference](#)

RunOptions

Additional options for the given run style. For remote this should be a hostname. For lsf and uge this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

When RunStyle is set to uge or lsf the RunOptions attribute will be passed into the synthesis launch jobs. If hierarchical synthesis is enabled the RunStyle design attribute setting will be used for the sub-block synthesis run.

Examples

```
set_attribute [current_design] -attr RunOptions -value { -P bnormal -l  
'os_distribution=centos,mem_free=40G,qsc=s' }
```

See Also

SDCConstraints

Specifies information about an externally provided SDC file for a component.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: component external fpgaTimingExceptions
timingExceptionInstances timingExceptions translator

Default subscript: component

Valid on: design_____

Description

The 'component' subscript is used to specify a component level SDC file. This file will be sourced during Initialize Subsystem Constraints in coreAssembler prior to constraint propagation up the hierarchy. The 'external' subscript is used to specify an SDC file for the top-level design in coreAssembler. When specified, this external SDC file is used for synthesis instead of anything generated by coreAssembler. The 'translator' subscript is used to indicate the name of a procedure to be invoked to translate constraints (specified using the 'component' subscript) prior to reading the SDC. The translator proc must take two arguments (input SDC file name and output SDC file name) and should return an indication of whether or not any changes were actually made. The output file only needs to be written if changes are made.

The 'timingExceptions' subscript is used to specify timing exceptions which require use of non-standard commands to locate the objects on which the timing exception apply. This file will be appended to the generated SDC file without modification. In coreAssembler, all files will be sourced from the top context without modification. If multiple components reference a file with the same base name, the files are sourced by order of instance nesting level. To change this behavior, set the SDCConstraints attribute on the top design using the timingExceptionInstances attribute to indicate the order in which the files should be included.

Examples

Specify component level SDC file and use the default translation procedure:

```
set_design_attribute {SDCConstraints[component]} /path/to/component.sdc
```

```
set_design_attribute {SDCConstraints[translator]}
```

```
translate_phy_constraints
```

Given a design with leaf instances i0, i1, hier0/i2, where each instance has SDCConstraints[timingExceptions] set, the default order the files will be included in coreAssembler is hier0/i2, and then i0 or i1 followed by i1 or i0 (order random due to matching nesting level). In order to force the order to be i1, hier0/i2, i0, use the following:

```
set_design_attribute {SDCConstraints[timingExceptionInstances]} {i1  
hier0/i2 i0}
```

If instead you only want to apply the timing exception from i1:

```
set_design_attribute {SDCConstraints[timingExceptionInstances]} {i1}
```

See Also

[translate_phy_constraints \(2\)](#)

SDCPropagateText

SDC text to be propagated to next level of hierarchy. The text insertion point is based on the subscript specified.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: AfterClocks BeforeClocks End Unsupported

Default subscript:

Valid on: design

Description

This attribute can be used to pass fragments of SDC scripts up the hierarchy for subsystem constraint propagation in coreAssembler. The subscripts indicate where in the SDC file the fragment should be inserted. The Unsupported subscript is provided to indicate a script fragment which is relevant to a component but which is not written in a way that it can work from a different context. It is expected that these unsupported fragments are documented so that a subsystem developer is aware that something will need to be done manually.

It is expected that this attribute is only used for fragments of constraint files that cannot be processed by the coreTools.

Examples

Insert an SDC fragment after clocks have been defined.

```
set_design_attribute {SDCPropagateText[AfterClocks]} {
# Here is an SDC fragment to be applied from the top.
SDC-scommand
}
```

See Also

read_sdc(2), write_sdc(2),

[API Index](#) [Command Reference](#)

SDCWriteCallback

Name of callback command to invoke after writing SDC file

Definition

Type: string**Flags:****Default value:****Valid on:** design_____

Description

This attribute is used to cause an update the SDC file generated by the `write_sdc` command. The attribute value should be the name of TCL procedure to be called to make the SDC file updates. The proc must be defined to take a single argument, which is the name of the SDC file to be updated.

Examples

Add a comment to the generated SDC file. First define the callback proc, and then set the callback attribute to cause this proc to be called whenever an SDC file is generated.

```
proc sdc_callback {fileName} {  
  set fid [open $fileName "a"]  
  puts $fid "# Commented added via SDCWriteCallback"  
  close $fid  
}  
set_design_attribute SDCWriteCallback sdc_callback
```

See Also

[write_sdc\(2\)](#)

ScanBlockIndividually

Causes scan insertion to be performed on this design when it is stand-alone.

Definition

Type: boolean

Flags:

Default value: =expr {[InheritValue up -attr
ScanSubblocksIndividually 0] || [get_attribute %item -attr
WrapBlockIndividually] || [get_attribute %item -attr
BistBlockIndividually] || [sDft::defaultScanBlockIndividually
%item]}

Valid on: design_____

Description

Setting ScanBlockIndividually to true on a design will cause insert_dft to be run on the design stand-alone in its own Design Compiler session. ScanBlockIndividually can be set explicitly on a design, or it may inherit it from other attributes. Setting ScanSubblocksIndividually on a design's direct parent design will cause ScanBlockIndividually to be set to true on the design. Setting WrapBlockIndividually or BistBlockIndividually to true on a design will also cause ScanBlockIndividually to be set to true. If WrapBlockIndividually or BistBlockIndividually is set to true, on a design ScanBlockIndividually may not be set false on that design.

ScanBlockIndividually defines the test frontier in the same way that MapBlockIndividually defines the compile frontier. The test frontier does not have to match the compile frontier. Designs that are on the compile frontier do not have to be on the test frontier. Conversely a design that is not on the compile frontier may be on the test frontier.

When ScanBlockIndividually is set to true, the set_scan_configuration command will be issued in DFT Compiler. The correspondence between coreTools attributes and set_scan_configuration options is given below.

ScanStyle -style

ScanMethodology -methodology

ScanReplace -replace

ClockMixing -clock_mixing

TestClocksFollowSystemDomains -create_test_clocks_by_system_clock_domain

AddLockUpLatch -add_lockup

InsertEndOfChainLockupLatch -insert_end_of_chain_lockup_latch

DedicatedScanPorts -dedicated_scan_ports

BidirectionalMode -bidi_mode

ExternalTristates -external_tristates

InternalTristates -internal_tristates
 HierarchicalIsolation -hierarchical_isolation
 ScanInternalClocks -internal_clocks
 NumberOfScanChains -chain_count
 MaximumScanChainLength -longest_chain_length
 The correspondence between coreTools attributes and set_dft_configuration options is given below.

EnableDftAutoFix -autofix
 EnableDftShadowLogic -shadow_wrapper
 WrapBlockIndividually -core_wrapper
 InsertTestPoints -testability
 BistBlockIndividually -bist

Examples

See Also

set design attribute (2), MapBlockIndividually (3),
ScanSubblocksIndividually (3), BistBlockIndividually (3),
WrapBlockIndividually (3), ScanStyle (3), ScanMethodology (3),
ScanReplace (3), ClockMixing (3), TestClocksFollowSystemDomains (3),
AddLockUpLatch (3), InsertEndOfChainLockupLatch (3), DedicatedScanPorts
(3), BidirectionalMode (3), ExternalTristates (3), InternalTristates (3),
HierarchicalIsolation (3), ScanInternalClocks (3), NumberOfScanChains
(3), MaximumScanChainLength (3)

ScanCompressionConfiguration

The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, please refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.

Definition

Type: string

Flags:

Default value: =sDft::getArgsForScanCompressionConfiguration

Valid on: design_____

Description

The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, please refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.

Note that when using set_design_attribute a blank space must be the first character. When using set_attribute this is not necessary.

Use set_design_attribute to set the scan compression configuration.

```
coreConsultant>set_design_attribute ScanCompressionConfiguration \
"-xtolerance high -min_power true"
```

Use set_attribute to set the scan compression configuration.

```
coreConsultant>set_attribute [current_design -quiet] \
ScanCompressionConfiguration \
"-xtolerance high -min_power true"
```

See Also

[EnableScanCompression \(3\)](#)

[API Index](#) [Command Reference](#)

ScanCompressionTestClock

Sets a test clock for scan compression from the list of test clocks available.

Definition

Type: string

Flags:

Default value: =sDft::getTestClocks "default" "current"

Valid on: design_____

Description

For streaming compression, a test clock must be selected from the available test clocks in the current context. The default value is the alphabetically first test clock in the design.

Examples

```
coreAssembler> set_design_attribute EnableScanCompression 1
coreAssembler> set_design_attribute NumberOfScanChains 7
coreAssembler> set_design_attribute ScanCompressionUsage
"streaming_compression"
coreAssembler> set_design_attribute ScanCompressionTestClock "pclk"
```

See Also

[set_design_attribute \(2\)](#), [NumberOfScanChains \(3\)](#) [EnableScanCompression \(3\)](#) [ScanCompressionUsage \(3\)](#) [NumberOfScanCompressionChains \(3\)](#) [ScanCompressionConfiguration \(3\)](#)

ScanCompressionUsage

The value passed along with -usage option for ScanCompression_mode test mode in define_test_mode command. See DFTMAX Compression and DFTMAX Ultra/SEQ Compression sections in the TestMAX DFT user guide for more details.

Definition

Type: string

Flags:

Default value: scan_compression

Valid on: design_____

Description

This attribute accepts 'scan_compression', 'streaming_compression' or 'seq_cpmpression'. Setting it to 'streaming_compression' adds '-streaming true' and '-clock <codec clock name>' to the set_scan_compression_configuration command in <IP>.DFT.setup.tcl file, while for TMM Based flow the scan configuration commands are added in tmm_dft_constraints.tcl file, which is read in during synthesis. Selecting 'scan_compression' will not add those lines to the command, disabling streaming compression. Scan compression uses DFTMAX, Streaming compression uses DFTMAX Ultra and Seq compression requires DFTMAX Seq.

Examples

```
coreConsultant> set_design_attribute ScanCompressionUsage
"scan_compression"
coreConsultant> set_design_attribute ScanCompressionUsage
"streaming_compression"
coreConsultant> set_design_attribute ScanCompressionUsage
"seq_compression"
```

See Also

[NumberOfScanChains \(3\)](#) [EnableScanCompression \(3\)](#)
[ScanCompressionConfiguration \(3\)](#)

ScanExclude

List of sequential cells, hierarchical cells containing sequential elements, references, library cells, or designs that should not be replaced by scan cells.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This will cause the Design Compiler command `set_scan_element false` to be issued for the items specified by this attribute. This attribute must be set on the compile frontier at the point where the item to be excluded from scan will be compiled.

See the DC man page for `set_scan_element` for more details.

Examples

To set `ScanExclude` such that the sequential elements U1, U2, and U3 in the current design will not be replaced by scan equivalent cells:

```
coreBuilder> set_design_attribute ScanExclude {U1 U2 U3}
```

See Also

[ScanStyle \(3\)](#)

[API Index](#) [Command Reference](#)

ScanInternalClocks

When true, internal clocks are treated as separate clocks for creating scan chains. In XG mode, setting this attribute to false is equivalent to '-internal_clocks none'. When this attribute is set to TRUE it is Used in conjunction with ScanJumpBuffersInverters.

Definition

Type: boolean

Flags:

Default value: =InheritValue up FALSE

Valid on: design_____

Description

Applies only to the multiplexed flip-flop scan style; ignored for other scan styles. When set to true, insert_dft treats any internal clocks in the design as separate clocks for the purpose of architecting scan chains. When false, insert_dft does not treat internal clocks as separate clocks. An internal clock is defined as an internal signal driven by a multiplexer (or multiple input gate) output pin.

Examples

Specify that internal clocks are to be treated as separate clocks for scan chains.

```
coreBuilder> set_design_attribute ScanInternalClocks 1
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#)_____

ScanJumpBuffersInverters

This attribute will only have an effect when ScanInternalClocks is set to TRUE. For XG mode, the value of this attribute will be the value used for the -internal_clocks option to set_scan_configuration. When not in XG mode, setting ScanJumpBuffersInverters controls the setting of the DC variable test_jump_over_bufs_invs. Setting ScanJumpBuffersInverters to single sets test_jump_over_bufs_invs to true. Setting the attribute ScanJumpBuffersInverters to 'multi' sets test_jump_over_bufs_invs to false.

Definition

Type: string

Flags:

Default value: =InheritValue up single

Valid on: design_____

Description

This attribute will only have an effect when ScanInternalClocks is set to TRUE. For XG mode, the value of this attribute will be the value used for the -internal_clocks option to set_scan_configuration. When not in XG mode, setting ScanJumpBuffersInverters controls the setting of the DC variable test_jump_over_bufs_invs. Setting ScanJumpBuffersInverters to single sets test_jump_over_bufs_invs to true. Setting the attribute ScanJumpBuffersInverters to 'multi' sets test_jump_over_bufs_invs to false.

Examples

Specify that set_scan_configuration -internal_clocks single is to be used in DFT Compiler.

```
set_design_attribute ScanInternalClocks TRUE
set_design_attribute ScanJumpBuffersInverters single
```

See Also

[set_design_attribute \(2\)](#), [ScanInternalClocks \(3\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

ScanMethodology

Selects the scan methodology. In full_scan methodology insert_scan makes all sequential cells scannable if they do not violate scan design rules. A setting of none selects no scan methodology for the design.

Definition

Type: string

Flags:

Default value: =InheritValue up full_scan

Valid on: design_____

Description

Selects the scan methodology. In full_scan methodology insert_scan makes all sequential cells scannable if they do not violate scan design rules. A setting of none selects no scan methodology for the design.

Examples

Set the scan methodology to full_scan on the current design.

```
coreBuilder> set_design_attribute ScanMethodology full_scan
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

ScanReplace

Should non-scan registers be replaced by scannable registers?

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

When true, insert_scan replaces sequential cells with scan cells, if the sequential cells are not violated by scan design rule checking. To disable scan replacement, set this option to false.

Examples

Specify that sequential cells should be replaced by scan cells.

```
coreBuilder> set_design_attribute ScanReplace 1
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

ScanStyle

Sets the test scan implementation style for this design.

Definition

Type: string

Flags:

Default value: =InheritValue up [sCstr::get_default_scan_style]

Valid on: design_____

Description

This attribute is used to indicate the scan implementation style for a given design. The value is passed on to DC via the -style option to the set_scan_configuration command. Valid values include:

- multiplexed_flip_flop
- clocked_scan
- lssd
- aux_clock_lssd
- combinational
- none

The value of this attribute defaults to the value of the DC variable 'test_default_scan_style' if specified in the system or user set-up file for DC (.synopsys_dc.setup). Otherwise it defaults to multiplexed_flip_flop. See the DC man page for set_scan_configuration for details about the different implementation styles.

Examples

To set ScanStyle to use an lssd based implementation style:

```
coreBuilder> set_design_attribute ScanStyle lssd
```

See Also

[ScanExclude \(3\)](#)

[API Index](#) [Command Reference](#)

ScanSubblocksIndividually

Causes scan insertion to be performed on the sub-designs directly instantiated in this design.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Setting ScanSubblocksIndividually to true on a design will cause scan insertion to be performed stand-alone on the sub-designs that are directly instantiated in it. ScanBlockIndividually will be set to true on the sub-designs.

Examples

Assume that the current design, top, has three designs, left, right, and middle, instantiated in it. The following will cause ScanBlockIndividually to be set to TRUE on left, right, and middle, and each design will have insert_dft run on it stand alone.

```
coreBuilder> set_design_attribute ScanSubblocksIndividually TRUE
```

See Also

[set_design_attribute \(2\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

ScratchInstallIndex

Indicates the index for the components being installed in scratch/ungroup folder by 'create_hierarchical_component -fromComponent'. On the Subsystem KB this attribute is used to store the current count, on the design it is used to indicate the index to the install path. The install path can be created as

<workspace>/scratch/ungroup/Install_<ScratchInstallIndex>

Definition

Type: long

Flags:

Default value: 0

Valid on: design knowledgeBase

Description

Examples

See Also

Select Prime Profile

Prime Profiles set on components can be selected in this activity.

Parameters

Activity has no parameters.

See Also

[API Index](#) [Command Reference](#)

SeqCompressionInputs

Total No of SEQ Compression Input Ports

Definition

Type: string

Flags:

Default value: =sDft::seqInputDefaultValue

Valid on: design_____

Description

Used when SEQ Compression is used during TMM based synthesis flow. This values are added to the 'set_logicbist_configuration' command with the switch '-input'.

Examples

```
coreConsultant> set_design_attribute SeqCompressionInputs 10
```

See Also

[NumberOfScanChains \(3\)](#) [EnableScanCompression \(3\)](#) _____
[ScanCompressionConfiguration \(3\)](#) _____

[API Index](#) [Command Reference](#)

SeqCompressionOutputs

Total No of SEQ Compression Output Ports

Definition

Type: string**Flags:****Default value:** 1**Valid on:** design_____

Description

Used when SEQ Compression is used during TMM based synthesis flow. This values are added to the 'set_logicbist_configuration' command with the switch '-output'.

Examples

```
coreConsultant> set_design_attribute SeqCompressionOutputs 10
```

See Also

[NumberOfScanChains \(3\)](#) [EnableScanCompression \(3\)](#)
[ScanCompressionConfiguration \(3\)](#)

Sequence

Sequence number of an item

Definition

Type: short

Flags:

Default value: 32767

Valid on: [Strategy](#) [attrDefn](#) [busBit](#) [cell](#) [clock](#) [design](#) [file](#) [filegroup](#) [filegroupGroup](#) [hdlFunc](#) [knowledgeBase](#) [net](#) [param](#) [pin](#) [port](#) [timingException](#)

Description

The Sequence attribute indicates the sequence number of the selected item.

For activities, Sequence determines the order in which the activities appear in the Activity List and the report_activities output. When you create a custom activity, you set the value of Sequence on the activity with the -sequence option to the coreBuilder complete_custom_activity_definition command.

For parameters, Sequence determines the order in which the parameters appear in the parameter dialog. For design parameters, you can determine the order of parameters in the Specify Configuration dialog by setting the Sequence attribute on the parameters. For custom activity parameters, you can determine the order of the parameters in the custom activity parameter dialog by setting the Sequence attribute on the custom activity parameters.

For timing exception objects, the coreTools set the Sequence attributes on the timing exception objects in the order in which they are created with the timing exception commands (for example, set_false_path). The Sequence attribute values on timing exception items are particularly important because the effect of timing exception commands can be cumulative. coreConsultant must generate timing exception commands for Design Compiler in the same order in which the timing exception items were created in the coreTools.

Examples

To put the support_idle parameter before the ram_size parameter in the Specify Configuration dialog:

```
coreBuilder> set_parameter_attribute support_idle Sequence 1
coreBuilder> set_parameter_attribute ram_size Sequence 2
```

See Also

set_parameter_attribute (2), get_attribute (2),
complete custom activity definition (2)

Set Design, File, and Macro Prefixes

Set the design, file, and macro prefixes. The design prefix is used to make each design in your core have a unique name. Setting the design prefix is only needed when you need to have two or more versions of one component, i.e., you plan to instantiate two or more versions of one component in the same design database, each with different values of configuration parameters.

When the RTL code is written to disk, the prefix is prepended to every design name reference (and package name references for VHDL). All design names and references to design names will be changed to {prefix}_{original name}.

The macro prefix is used to replace any `define {var's}` and usages of `the{var}` with a prefixed value throughout the RTL code.

Parameters

Parameters Table

Name	Label	Description
DesignPrefix	Design prefix	String to place at the start of all design names
FilePrefix	File prefix:	Prefix all generated source files to avoid name conflicts if the directory hierarchy is flattened.
MacroPrefix	Macro prefix:	Prefix all macro's to avoid name conflicts.

See Also

[API Index](#) [Command Reference](#)

SetInHex

This parameter value can be, must be, or cannot be specified in hexadecimal format

Definition

Type: string

Flags:

Default value:

Valid on: attrDefn

Description

The SetInHex attribute determines whether the user can, must, or cannot specify a parameter value in hexadecimal (0xNNN) format. Valid values for SetInHex are:

- Required - The user must specify the value in hexadecimal format.
- Allowed - The user can, but does not have to, specify the value in hexadecimal format.
- Illegal - The user cannot specify the value in hexadecimal format.

If the user specifies the parameter value in a format that is not allowed, the coreTool returns an error message.

Examples

To specify that the value for parameter Slave1_addr must be in hexadecimal format:

```
coreBuilder> set_parameter_attribute Slave1_addr SetInHex true
```

See Also

[set_parameter_attribute \(2\)](#), [IsHexVal \(3\)](#) _____

[API Index](#) [Command Reference](#)

Set Up Help Menu

Add documentation links to the Help menu in the console window.

Parameters

Activity has no parameters.

See Also

[Create Bill of Materials.](#)

[API Index](#) [Command Reference](#)

ShareTestPointsAcrossHierarchy

Specifies that test point scan cells may be shared across hierarchy.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Specifies that test point scan cells may be shared across hierarchy.

Examples

Enable the sharing of test points across hierarchy.

```
coreBuilder> set_design_attribute ShareTestPointsAcrossHierarchy 1
```

See Also

[set_design_attribute \(2\)](#), [InsertTestPoints \(3\)](#)_____

[API Index](#) [Command Reference](#)

SharedWrapperCell

Specifies the default shared wrapper cell type.

Definition

Type: string

Flags:

Default value: WC_S1

Valid on: design_____

Description

Specifies the default shared wrapper cell to be used for the design.

Examples

Specify that the default shared wrapper cell should be WC_D1.

```
coreBuilder> set_design_attribute SharedWrapperCell WC_D1
```

See Also

[set_design_attribute \(2\)](#), [WrapBlockIndividually \(3\)](#),
[WrapSubblocksIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

ShowIcons

When shown in a tree display an icon if it is specified.

Definition

Type: boolean

Flags:

Default value: 0

Valid on:

Description

If ShowIcons is set to true and the attribute IconPath is set to the path of an existing graphic symbol then the icon will be displayed when the item is shown in a view.

Examples

```
set_attribute $block -attr IconPath -value $iconPath
```

See Also

[IconPath \(3\)](#)

ShowRoute

Indicates whether or not routes from this component should be drawn in the schematic window.

Definition

Type: boolean

Flags:

Default value: 1

Valid on:

Description

This attribute is used to indicate whether or not routes from the specified interface instance should be shown in the schematic. This attribute is typically set as part of the packaging of a component on the 'provider' interface. In coreAssembler, it is not usually set directly, but is set via the 'Show/Hide Routes' menu option.

Examples

To set the default so that the routes for interface instance 'BAR' are not shown:

```
prompt> set_interface_attribute BAR ShowRoute false
```

See Also

SideConstraint

Specifies the definite pin side details. The values for DesignCompiler are left, right, top, bottom & for FusionCompiler the side should be an integer value starting from 1.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: dc_shell fc_shell

Default subscript: dc_shell

Valid on: port _____

Description

This attribute is used for defining port side constraint for synthesis engine.

The constraints will be used to generate a coarse floorplan during synthesis. If port side constraints are provided, the ports will be snapped to the specified side. Otherwise, in default, the ports will be snapped to the side nearest to the port location assigned by the synthesis engine.

Design Compiler:

The values for attribute SideConstraint in DesignCompiler are left, right, top, bottom.

File generated is DC_port_side_constraints.tcl and it will be generated in the syn/constraint/scripts

The file will be generated when DC version is > 2018.06 & physical synthesis is enabled in addition to the SideConstraint for DesignCompiler is specified.

Fusion Compiler:

The values for attribute SideConstraint in FusionCompiler should be an integer starting from 1.

File generated is FC_port_side_constraints.tcl and it will be generated in the syn/constraint/scripts

Examples

For Fusion Compiler:

```
coreConsultant>set_attribute [find_item -type port paddr] -attr
SideConstraint -subscript fc_shell -value 2
Generated FusionCompiler command:
```

```
set_individual_pin_constraints [get_ports paddr] -sides 2
```

For Design Compiler:

```
coreConsultant>set_attribute [find_item -type port paddr] -attr
SideConstraint -subscript dc_shell -value top
Generated DesignCompiler command :
```

```
set_port_side -side top [get_ports paddr]
```

NOTE: The above command also updates the 'SideConstraint' attribute for fc_shell to 2.

To set the SideConstraint for both subscripts to bottom

```
coreConsultant>set_port_attribute paddr SideConstraint bottom
The command is equivalent to:
```

```
coreConsultant>set_attribute [find_item -type port paddr] -attr
SideConstraint -subscript dc_shell -value bottom
coreConsultant>set_attribute [find_item -type port paddr] -attr
SideConstraint -subscript fc_shell -value 4
```

See Also

[API Index](#) [Command Reference](#)

SideEffects

Obsolete attribute - Use UVM testing now.

Definition

Type: boolean

Flags:

Default value:

Valid on:

Description

Indicates whether or not writing/reading this field or register has any side effects.

Examples

```
coreBuilder> set_register_attribute $reg SideEffects false
```

See Also

[PingTestMask \(3\)](#)

SimTieOff

Indicates how a pin should be tied off in the testbench if left unconnected.

Definition

Type: string

Flags:

Default value:

Valid on: busBit pin port _____

Description

This attribute is used to indicate how to tie-off a DUT pin exported from a component within the DUT if left unconnected in the testbench. This tie-off occurs only within a coreAssembler generated testbench. Legal values include:

- One of the values 'zero', and 'one' to tie off to a constant value
- Any hexadecimal number with leading 0x to tie a bus to a constant value
- One of the names of the output ports of the clock and reset generator automatically instantiated in the testbench. These are standard AMBA clock and reset port names with the AMBA meanings implied: pclk, presetn, hclk, hresetn, aclk, aresetn
- The value pcie_clk is valid for an additional non-AMBA clock
- The value sio_rst_n is valid for an additional non-AMBA reset

Examples

Indicate that the 'slave_reset' port should be connected to the presetn port of the clock generator

```
set_port_attribute slave_reset SimTieOff presetn
```

See Also

Simulate Subsystem Testbench
 Simulate the current subsystem testbench.

Parameters

Parameters Table

Name	Label	Description
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieving using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.
CleanupOnRerun	Clean up files when repeating simulation	Indicates whether or not the contents of the 'sim' directory should be removed when recompleting the Simulate Subsystem Testbench activity.
Simulator Setup		
SimChoice	Simulator	

coreTools Command Reference Index

Name	Label	Description
		Choice of simulator to invoke for this testbench. It is assumed you have the simulator installed and properly set up in your environment already. EnumValues: VCS MTI_Verilog NC_Verilog
SimulatorOptions	Simulator Options	Choice of simulator options to use for this testbench. It is assumed you have the simulator installed and properly set up in your environment already.
AmbaSvtVipVersion	AMBA SVT VIP Version	Specify AMBA SVT VIP Version, By default it takes latest version.
MTIIncludePath	MTI Include Path	The path to the 'include' directory contained within your MTI simulator installation area. A valid directory will include the file veriusers.h.
CDSInstDir	Root Directory of Cadence Installation (formerly CDS_INST_DIR)	Directory under which Cadence's ncsim tool can be found.
DumpEnabled	Generate 'waves' file	Indicates whether or not a waves file should be created for debugging with a wave file browser after simulation ends. When enabled the the VPD (vcdplus.vpd) and a FSDB (fsdbfile.fsdb) wave files are generated
CCompiler	C Compiler (for VERA PLI)	Choice of which C Compiler to invoke to create a vera PLI. Choose cc if you have the platform native Ansi C-compiler. Choose gcc if you have GNU C-compiler installed. /usr/ucb/cc does not work well. - EnumValues: gcc cc - Symbolic Names: {GNU C-Compiler (gcc)} {Platform's Ansi C-Compiler (cc)}
VeraVroCache	Vera .vro cache directory	Indicates the directory where the compiled Vera .vro files will be written
Simulator Setup/Enable UVM Test List		
RunRegAccessTests	Register Access	Run UVM Register Access tests for all components
RunHWResetTests	Hardware Reset	

coreTools Command Reference Index

Name	Label	Description
		Run UVM H/W Reset tests for all components
RunBitBashTests	Bit-Bash	Run UVM Bit-Bash tests for all components
Simulator Setup/UVM RAL Coverage		
BitLevelEn	Bit-Level	Enable -c b option in ralgen. Generates coverage model for bit-level
AddressMapEn	Address Map	Enable -c a option in ralgen. Generates coverage model for address maps
FieldEn	Field Values	Enable -c F option in ralgen. Generates coverage model for field values, even if 'cover +f' is not specified in RALF spec
TestbenchDefinition/Slaves/Row1		
Slv_0	Slv_0	Indicates which master is going to test this slave.
Comp_0	Comp_0	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row10		
Slv_9	Slv_9	Indicates which master is going to test this slave.
Comp_9	Comp_9	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row100		
Slv_99	Slv_99	Indicates which master is going to test this slave.
Comp_99	Comp_99	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row11		
Slv_10	Slv_10	Indicates which master is going to test this slave.
Comp_10	Comp_10	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row12		
Slv_11	Slv_11	Indicates which master is going to test this slave.
Comp_11	Comp_11	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row13		
Slv_12	Slv_12	Indicates which master is going to test this slave.
Comp_12	Comp_12	Indicate the design associated with this slave.

coreTools Command Reference Index

Name	Label	Description
TestbenchDefinition/Slaves/Row14		
Slv_13	Slv_13	Indicates which master is going to test this slave.
Comp_13	Comp_13	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row15		
Slv_14	Slv_14	Indicates which master is going to test this slave.
Comp_14	Comp_14	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row16		
Slv_15	Slv_15	Indicates which master is going to test this slave.
Comp_15	Comp_15	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row17		
Slv_16	Slv_16	Indicates which master is going to test this slave.
Comp_16	Comp_16	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row18		
Slv_17	Slv_17	Indicates which master is going to test this slave.
Comp_17	Comp_17	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row19		
Slv_18	Slv_18	Indicates which master is going to test this slave.
Comp_18	Comp_18	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row2		
Slv_1	Slv_1	Indicates which master is going to test this slave.
Comp_1	Comp_1	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row20		
Slv_19	Slv_19	Indicates which master is going to test this slave.
Comp_19	Comp_19	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row21		
Slv_20	Slv_20	Indicates which master is going to test this slave.
Comp_20	Comp_20	

coreTools Command Reference Index

Name	Label	Description
		Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row22		
Slv_21	Slv_21	Indicates which master is going to test this slave.
Comp_21	Comp_21	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row23		
Slv_22	Slv_22	Indicates which master is going to test this slave.
Comp_22	Comp_22	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row24		
Slv_23	Slv_23	Indicates which master is going to test this slave.
Comp_23	Comp_23	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row25		
Slv_24	Slv_24	Indicates which master is going to test this slave.
Comp_24	Comp_24	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row26		
Slv_25	Slv_25	Indicates which master is going to test this slave.
Comp_25	Comp_25	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row27		
Slv_26	Slv_26	Indicates which master is going to test this slave.
Comp_26	Comp_26	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row28		
Slv_27	Slv_27	Indicates which master is going to test this slave.
Comp_27	Comp_27	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row29		
Slv_28	Slv_28	Indicates which master is going to test this slave.
Comp_28	Comp_28	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row3		
Slv_2	Slv_2	

coreTools Command Reference Index

Name	Label	Description
		Indicates which master is going to test this slave.
Comp_2	Comp_2	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row30		
Slv_29	Slv_29	Indicates which master is going to test this slave.
Comp_29	Comp_29	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row31		
Slv_30	Slv_30	Indicates which master is going to test this slave.
Comp_30	Comp_30	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row32		
Slv_31	Slv_31	Indicates which master is going to test this slave.
Comp_31	Comp_31	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row33		
Slv_32	Slv_32	Indicates which master is going to test this slave.
Comp_32	Comp_32	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row34		
Slv_33	Slv_33	Indicates which master is going to test this slave.
Comp_33	Comp_33	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row35		
Slv_34	Slv_34	Indicates which master is going to test this slave.
Comp_34	Comp_34	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row36		
Slv_35	Slv_35	Indicates which master is going to test this slave.
Comp_35	Comp_35	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row37		
Slv_36	Slv_36	Indicates which master is going to test this slave.
Comp_36	Comp_36	Indicate the design associated with this slave.

coreTools Command Reference Index

Name	Label	Description
TestbenchDefinition/Slaves/Row38		
Slv_37	Slv_37	Indicates which master is going to test this slave.
Comp_37	Comp_37	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row39		
Slv_38	Slv_38	Indicates which master is going to test this slave.
Comp_38	Comp_38	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row4		
Slv_3	Slv_3	Indicates which master is going to test this slave.
Comp_3	Comp_3	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row40		
Slv_39	Slv_39	Indicates which master is going to test this slave.
Comp_39	Comp_39	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row41		
Slv_40	Slv_40	Indicates which master is going to test this slave.
Comp_40	Comp_40	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row42		
Slv_41	Slv_41	Indicates which master is going to test this slave.
Comp_41	Comp_41	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row43		
Slv_42	Slv_42	Indicates which master is going to test this slave.
Comp_42	Comp_42	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row44		
Slv_43	Slv_43	Indicates which master is going to test this slave.
Comp_43	Comp_43	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row45		
Slv_44	Slv_44	Indicates which master is going to test this slave.
Comp_44	Comp_44	

coreTools Command Reference Index

Name	Label	Description
		Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row46		
Slv_45	Slv_45	Indicates which master is going to test this slave.
Comp_45	Comp_45	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row47		
Slv_46	Slv_46	Indicates which master is going to test this slave.
Comp_46	Comp_46	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row48		
Slv_47	Slv_47	Indicates which master is going to test this slave.
Comp_47	Comp_47	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row49		
Slv_48	Slv_48	Indicates which master is going to test this slave.
Comp_48	Comp_48	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row5		
Slv_4	Slv_4	Indicates which master is going to test this slave.
Comp_4	Comp_4	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row50		
Slv_49	Slv_49	Indicates which master is going to test this slave.
Comp_49	Comp_49	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row51		
Slv_50	Slv_50	Indicates which master is going to test this slave.
Comp_50	Comp_50	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row52		
Slv_51	Slv_51	Indicates which master is going to test this slave.
Comp_51	Comp_51	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row53		
Slv_52	Slv_52	

coreTools Command Reference Index

Name	Label	Description
		Indicates which master is going to test this slave.
Comp_52	Comp_52	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row54		
Slv_53	Slv_53	Indicates which master is going to test this slave.
Comp_53	Comp_53	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row55		
Slv_54	Slv_54	Indicates which master is going to test this slave.
Comp_54	Comp_54	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row56		
Slv_55	Slv_55	Indicates which master is going to test this slave.
Comp_55	Comp_55	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row57		
Slv_56	Slv_56	Indicates which master is going to test this slave.
Comp_56	Comp_56	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row58		
Slv_57	Slv_57	Indicates which master is going to test this slave.
Comp_57	Comp_57	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row59		
Slv_58	Slv_58	Indicates which master is going to test this slave.
Comp_58	Comp_58	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row60		
Slv_5	Slv_5	Indicates which master is going to test this slave.
Comp_5	Comp_5	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row61		
Slv_59	Slv_59	Indicates which master is going to test this slave.
Comp_59	Comp_59	Indicate the design associated with this slave.

coreTools Command Reference Index

Name	Label	Description
TestbenchDefinition/Slaves/Row61		
Slv_60	Slv_60	Indicates which master is going to test this slave.
Comp_60	Comp_60	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row62		
Slv_61	Slv_61	Indicates which master is going to test this slave.
Comp_61	Comp_61	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row63		
Slv_62	Slv_62	Indicates which master is going to test this slave.
Comp_62	Comp_62	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row64		
Slv_63	Slv_63	Indicates which master is going to test this slave.
Comp_63	Comp_63	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row65		
Slv_64	Slv_64	Indicates which master is going to test this slave.
Comp_64	Comp_64	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row66		
Slv_65	Slv_65	Indicates which master is going to test this slave.
Comp_65	Comp_65	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row67		
Slv_66	Slv_66	Indicates which master is going to test this slave.
Comp_66	Comp_66	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row68		
Slv_67	Slv_67	Indicates which master is going to test this slave.
Comp_67	Comp_67	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row69		
Slv_68	Slv_68	Indicates which master is going to test this slave.
Comp_68	Comp_68	

coreTools Command Reference Index

Name	Label	Description
		Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row7		
Slv_6	Slv_6	Indicates which master is going to test this slave.
Comp_6	Comp_6	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row70		
Slv_69	Slv_69	Indicates which master is going to test this slave.
Comp_69	Comp_69	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row71		
Slv_70	Slv_70	Indicates which master is going to test this slave.
Comp_70	Comp_70	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row72		
Slv_71	Slv_71	Indicates which master is going to test this slave.
Comp_71	Comp_71	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row73		
Slv_72	Slv_72	Indicates which master is going to test this slave.
Comp_72	Comp_72	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row74		
Slv_73	Slv_73	Indicates which master is going to test this slave.
Comp_73	Comp_73	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row75		
Slv_74	Slv_74	Indicates which master is going to test this slave.
Comp_74	Comp_74	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row76		
Slv_75	Slv_75	Indicates which master is going to test this slave.
Comp_75	Comp_75	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row77		
Slv_76	Slv_76	

coreTools Command Reference Index

Name	Label	Description
		Indicates which master is going to test this slave.
Comp_76	Comp_76	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row78		
Slv_77	Slv_77	Indicates which master is going to test this slave.
Comp_77	Comp_77	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row79		
Slv_78	Slv_78	Indicates which master is going to test this slave.
Comp_78	Comp_78	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row8		
Slv_7	Slv_7	Indicates which master is going to test this slave.
Comp_7	Comp_7	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row80		
Slv_79	Slv_79	Indicates which master is going to test this slave.
Comp_79	Comp_79	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row81		
Slv_80	Slv_80	Indicates which master is going to test this slave.
Comp_80	Comp_80	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row82		
Slv_81	Slv_81	Indicates which master is going to test this slave.
Comp_81	Comp_81	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row83		
Slv_82	Slv_82	Indicates which master is going to test this slave.
Comp_82	Comp_82	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row84		
Slv_83	Slv_83	Indicates which master is going to test this slave.
Comp_83	Comp_83	Indicate the design associated with this slave.

coreTools Command Reference Index

Name	Label	Description
TestbenchDefinition/Slaves/Row85		
Slv_84	Slv_84	Indicates which master is going to test this slave.
Comp_84	Comp_84	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row86		
Slv_85	Slv_85	Indicates which master is going to test this slave.
Comp_85	Comp_85	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row87		
Slv_86	Slv_86	Indicates which master is going to test this slave.
Comp_86	Comp_86	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row88		
Slv_87	Slv_87	Indicates which master is going to test this slave.
Comp_87	Comp_87	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row89		
Slv_88	Slv_88	Indicates which master is going to test this slave.
Comp_88	Comp_88	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row9		
Slv_8	Slv_8	Indicates which master is going to test this slave.
Comp_8	Comp_8	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row90		
Slv_89	Slv_89	Indicates which master is going to test this slave.
Comp_89	Comp_89	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row91		
Slv_90	Slv_90	Indicates which master is going to test this slave.
Comp_90	Comp_90	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row92		
Slv_91	Slv_91	Indicates which master is going to test this slave.
Comp_91	Comp_91	

coreTools Command Reference Index

Name	Label	Description
		Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row93		
Slv_92	Slv_92	Indicates which master is going to test this slave.
Comp_92	Comp_92	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row94		
Slv_93	Slv_93	Indicates which master is going to test this slave.
Comp_93	Comp_93	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row95		
Slv_94	Slv_94	Indicates which master is going to test this slave.
Comp_94	Comp_94	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row96		
Slv_95	Slv_95	Indicates which master is going to test this slave.
Comp_95	Comp_95	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row97		
Slv_96	Slv_96	Indicates which master is going to test this slave.
Comp_96	Comp_96	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row98		
Slv_97	Slv_97	Indicates which master is going to test this slave.
Comp_97	Comp_97	Indicate the design associated with this slave.
TestbenchDefinition/Slaves/Row99		
Slv_98	Slv_98	Indicates which master is going to test this slave.
Comp_98	Comp_98	Indicate the design associated with this slave.

See Also

Generate Subsystem RTL.

SimulationModel

Indicates that this design is a simulation model and is not to be synthesized.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

This attribute is set on designs which are not going to be synthesized because they are present for verification purposes only. This can be used on sub-designs within a core if the sub-design is not supposed to be synthesized. It can also be used on the top design, if for example, you are packaging a verification model for use in coreAssembler. All designs underneath a design with SimulationModel set to true are assumed to also be simulation models.

Examples

```
prompt> set_design_attribute SimulationModel true
```

See Also

SlotNumber

Indicates the slot number on the bus for the given interface.

Definition

Type: short

Flags: readOnly

Default value:

Valid on:

Description

This attribute indicates the slot associated with a connection to a bus or other interface 'source' which connects to multiple 'loads' (typically bus masters or slaves). This represents which bus 'slot' the master or slave is associated with. This number is assigned automatically as components are added to the subsystem, but it can be changed by setting this attribute on the interface instance in a batch script or via the "Switch Slots" menu option in the schematic window.

The SlotOrder specifies the continuous sequence of all slots that make up the SlotNumber. Slots can have a SlotWidth which could be different from 1 (multiple slots occupied by a single interface), and the provider interface defines the first slot number with SlotNumberOffset.

In a hierarchical subsystem the SlotNumber represents the 'local' value for slot number (within the containing hierarchy). If the overall (flat) slot number is required, then the GlobalSlotNumber attribute should be used.

Examples

To force the CPU to be in 'slot' 1 on the AHB bus:

```
prompt> set_interface_attribute CPU SlotNumber 1
```

See Also

[GlobalSlotNumber \(3\)](#), [SlotNumberOffset \(3\)](#), [SlotWidth \(3\)](#), [SlotOrder \(3\)](#)

[API Index](#) [Command Reference](#)

SlotNumberOffset

Indicates the offset (relative to 0) for numbering slots associated with this interface.

Definition

Type: short

Flags:

Default value: 0

Valid on:

Description

This attribute is used to define the smallest legal slot number associated with an interface 'provider' (typically a bus) when the starting slot is not 0. For example, if slot numbering should start at 1, then the SlotNumberOffset should be set to 1.

Examples

To specify that slot numbering for the AHB starts at 1:

```
prompt> set_interface_attribute -instance AHB SlotNumberOffset 1
```

See Also

[SlotNumber \(3\)](#), [SlotWidth \(3\)](#), [SlotOrder \(3\)](#) _____

SlotOrder

Indicates the order of the slot on the bus for the given interface.

Definition

Type: short

Flags:

Default value: -1

Valid on:

Description

The slot order defines a continuous sequence, beginning with 0, of all interface slots that make up the SlotNumber. Slots can have a SlotWidth which could be different from 1, and an interface defines the first slot number with SlotNumberOffset.

If the SlotOrder is changed to a value which is already occupied by another consumer interface then both interfaces swap their position in the sequence. New interfaces are appended to the sequence by connect_interface, and set_unused_interface and unconnect_interface remove the sequence gap if necessary.

Examples

Component IntrCtrl was instantiated last. The following command changes the slot order for its interface APB_Slave to be the first interface = occupy first slot:

```
prompt> set_interface_attribute IntrCtrl/APB_Slave SlotOrder 0
```

See Also

[SlotNumber \(3\)](#), [SlotNumberOffset \(3\)](#), [SlotWidth \(3\)](#), [connect_interface \(2\)](#), [unconnect_interface \(2\)](#), [set_unused_interface \(2\)](#)

SlotWidth

Indicates the number of continuous slots allocated by attribute SlotNumber and any other 'unique-value' interface parameter on the instance. The attribute annotates an eval_param expression in context of the instance.

Definition

Type: string

Flags:

Default value: 1

Valid on:

Description

The SlotWidth indicates the number of continuous interface slots on the interface provider instance which are occupied by a single, connected consumer interface. Default width is one slot.

The attribute is a string expression which can include @Parameter syntax for a dynamic width. In this case the parameter impacts the SlotNumber on the provider instance, but @-parameters are evaluated in the consumer context of the connected interface.

Examples

To specify that the memory controller requires two AHB slave slots:

```
prompt> set_interface_attribute -instance AHB_Slave SlotWidth 2
```

To derive the number of required slots dynamically from interface parameter NumSelectSlots for all instances of the AHB-Slaves definition:

```
prompt> set_interface_attribute AHB-Slaves SlotWidth @NumSelectSlots
```

See Also

[SlotNumber \(3\)](#), [SlotNumberOffset \(3\)](#), [SlotOrder \(3\)](#) _____

[API Index](#) [Command Reference](#)

SolutionData

Stores the definition of the solution (from `define_solution`).

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

Examples

See Also

SolutionEnvVarValues

The solution environment vars stored on disk for use in open workspace.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

Examples

See Also

SolutionItemPhase

Indicates the solution phase the item was created in when installing a solution.

Definition

Type: boolean

Flags: subscribed

Default value: 0

Valid subscripts: Instantiate Interfaces

Default subscript:

Valid on: cell _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

SolutionProcessed

Internal status attribute indicating whether solution files have been processed or not.

Definition

Type: boolean

Flags: subscribed

Default value: 0

Valid subscripts: Instances Interfaces

Default subscript:

Valid on: design_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

Specify Clocks

Define the set of clocks (virtual and real) within the core design.

Parameters

Activity has no parameters.

See Also

[Load Designs.](#)

[API Index](#) [Command Reference](#)

Specify Configuration

Specify values for HDL configuration parameters.

Parameters

Activity has no parameters.

See Also

[Set Design, File, and Macro Prefixes.](#)

[Select Prime Profile.](#)

[API Index](#) [Command Reference](#)

Specify Operating Conditions & WireLoads

Specify information which defines the context in which the core is going to be used (operating conditions and wireloads)

Parameters

Activity has no parameters.

See Also

[Specify Configuration.](#)

[Specify Reference Technology.](#)

[API Index](#) [Command Reference](#)

Specify Port Constraints

Set and/or review intent attributes on ports.

Parameters

Activity has no parameters.

See Also

[Load Designs.](#)

[Specify Reference Technology.](#)

[API Index](#) [Command Reference](#)

Specify Power Intent

This activity is used to capture power intent scripts.

Parameters

Parameters Table

Name	Label	Description
IntentFiles		
IntentFileNames	Power Intent Files	List of all power intent files.
UPFCells		
UseCommonIsolationCells	Use common isolation cell(s)	Use the same power isolation cells for all isolation strategies/domains.
UseCommonLevelShifterCells	Use common level shifter cell(s)	Use the same power level shifter cells for all level shifter strategies/domains.
UseCommonRetentionCells	Use common retention cell(s)	Use the same power retention cells for all retention strategies/domains.

See Also

[Load Designs.](#)

[Specify Reference Technology.](#)

Specify Subsystem

Select and configure the components and any associated memory maps for the subsystem.

Parameters

Parameters Table

Name	Label	Description
SearchPath	Installed coreKit and IP-XACT File Search Path	The list of directories to search to determine the set of available components. Entries should be directories containing installed coreKits or directories to search (recursively) for IP-XACT *.xml files. Initialized to \$DESIGNWARE_HOME/iip/latest if DESIGNWARE_HOME is defined.
NoWorkspaceSaves	No workspace saves	Indicates that the workspace should not be automatically saved after each component instantiation.
BusDefnSearchPath	BusDefnSearchPath	This parameter is no longer being utilized. The SearchPath variable should be used for components and all SPIRIT files.
Advanced		
StrictLanguageSemantics	Strict Language Semantics	Enforce strict language semantics. When false, commonly supported non-compliant language semantics are accepted without error.
PreventDesignPrefix	PreventDesignPrefix	Prevent insertion of design and cell prefixes in the source code.
SkipDesigns	Skip Designs	List of patterns that indicate which designs from the RTL should be skipped. A pattern is a glob-style expression. If a pattern starts with '!', then a design must match that pattern, otherwise it is skipped. It is possible to

coreTools Command Reference Index

Name	Label	Description
		have multiple patterns listed. ActivityAliasNames : SkipDesignPatterns
PreventSVPrefix	PreventSVPrefix	Prevent insertion of the design prefix for SystemVerilog interface and package names in the configured source code.
SkipPrefixNames	Skip Prefix Names	List of names and patterns to indicate modules and SystemVerilog interfaces and packages to not prefix. A pattern is a glob-style expression. If a pattern starts with '!', then only names that match the pattern are prefixed.
AutoReadDCSupported	Autoread Supported?	Allows usage of autoread mode in DC to analyze the RTL files given that the RTL code is autoread compatible. The files should have file structure where everything kept under one top directory
IntentSearchPath	Tcl intent search path	The list of directories to search in when looking for Tcl (and SDC) intent files. Intent files should be named with the same basename as an HDL file, with a '.tcl' (or '.sdc') extension. If the extension is '.elab.tcl' (or '.elab.sdc'), then the file will be sourced after configuration/elaboration in the coreAssembler flow.
TopDesignName	Top Design Name	Indicates the name of the top design. Can be calculated if there is only one design which does not have any references (instances). NOTE: In coreAssembler, this must be specified explicitly.
ExtraLinkLibs	Synthesis Link Libraries	The list of additional libraries to be linked in - hard macros, timing shells,

coreTools Command Reference Index

Name	Label	Description
		etc. Worst case/best case timing libraries may be specified using a list of lists. Assume lib1 is to be used for all conditions while lib2 is to be used for worst case analysis and lib2Min for best case analysis. In batch mode this would be specified as: <pre>set_activity_parameter LoadDesigns ExtraLinkLibs lib1.db {lib2.db lib2Min.db} lib3.db</pre> In the gui lib1.db and lib3.db should be placed on lines by themselves, and lib2.db and lib2Min.db should be placed on the same line.
Misc		
InstanceName	Instance name	HDL name for new component instance. The name must be a valid HDL identifier.
Options		
InputLanguage	Input language	Chooses the HDL input language EnumValues: VHDL Verilog SystemVerilog
HonorTranslatePragmas	Honor Translate Pragmas	Honor translate on/off pragmas when compiling RTL files.
CompileDefines	Compile Defines	Defines to pass to HDL source compiler, e.g. 'DEF1 DEF2=10'.
VCSOptions	VCS Options	Additional command line options for VCS.
LedaConfigFile	Leda Config File	Leda configuration filename.
Options/SystemVerilog		
SystemVerilogVersion	SystemVerilog Version	The SystemVerilog version used to analyze the input files. - EnumValues: 1800-2005 1800-2009 1800-2012 - Symbolic Names: 1800-2005 1800-2009 1800-2012

coreTools Command Reference Index

Name	Label	Description
Options/VHDL		
VhdlVersion	VHDL Version	The VHDL version used to analyze the input files. - EnumValues: VHDL93 VHDL87 - Symbolic Names: VHDL-93 VHDL-87
VhdlLibrary	Default Library	The default library to analyze VHDL files into. See the help on InputFileNames for multi-library support
Options/Verilog		
VerilogVersion	Verilog Version	The Verilog version used to analyze the input files. - EnumValues: Verilog95 Verilog01 Verilog05 - Symbolic Names: Verilog-95 Verilog-2001 Verilog-2005
IncludePath	Include Dirs	List of directories to search in for 'include files. Only those directories where 'included files are actually found will be saved as 'include directories' for the coreKit.
SourceFiles		
InputFileNames	Top-Level HDL Files in Analysis Order	The list of all HDL files (glob-style) that should be analyzed. This list should be in the proper analysis order. For multi-language and multi-library designs, entries of the form 'language [language]' and 'library [library]' can also be 'included in the file list. All files following the library or language line are assumed to belong to that library or language.
InputStrategy	HDL file location strategy	Indicates the strategy to use to locate HDL files. EnumValues: File Directory

coreTools Command Reference Index

Name	Label	Description
InputStrategyArguments	Arguments to input strategy procedure	Arguments to be passed to the input strategy procedure.
AnalyzeSubsetOfFiles	Analyze Subset of Files	When true, indicates that not all of the HDL files should be analyzed. Only those files specified via the FilesToBeAnalyzed parameter in batch mode or those entered into the list control labeled 'HDL Files To Be Analyzed' in GUI mode will actually be analyzed by the tool.
FilesToBeAnalyzed	HDL Files To Be Analyzed	List of files to be analyzed during LoadDesigns. By default this parameter has the same value as the InputFilesNames parameter (i.e. all HDL files are analyzed). If some of the files should not be analyzed (for performance or other reasons), check the 'Analyze Subset of Files' check-box (or set the AnalyzeSubsetOfFiles parameter in batch mode) and specify the files to be analyzed via this parameter, which is labeled 'HDL Files To Be Analyzed' in the graphical interface.

See Also

[API Index](#) [Command Reference](#)

Specify Synthesis Methodology
Set and/or review intent attributes on designs.

Parameters

Activity has no parameters.

See Also

[Load Designs.](#)
[Specify Reference Technology.](#)

[API Index](#) [Command Reference](#)

Specify Test Methodology

Set and/or review Design for Test intent attributes on designs.

Parameters

Activity has no parameters.

See Also

[Load Designs.](#)

[Specify Reference Technology.](#)

[API Index](#) [Command Reference](#)

Specify Timing Exceptions

Define the timing exceptions within the core design.

Parameters

Activity has no parameters.

See Also

[Specify Reference Technology.](#)

SpiritContainer

A place holder for spirit data that we don't model so we can export as is

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design param port _____

Description

A place holder for spirit data that we don't model so we can export as is

Examples

See Also

[API Index](#) [Command Reference](#)

SpiritFile

Indicates the file from which a SPIRIT element originated.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute indicates the file from which a SPIRIT element is originated.

Examples

See Also

[SpiritName \(3\)](#), [SpiritLibrary \(3\)](#), [SpiritVendor \(3\)](#), [SpiritVersion \(3\)](#)

[API Index](#) [Command Reference](#)

SpiritFileType

Specifies the value of the fileType element. Default is based on FileContentType and LoadDesigns language version.

Definition

Type: string

Flags:

Default value: ::=sXML::getSpiritFileType %item

Valid on: file filegroup

Description

The value of the IP-XACT fileType element of a file. It may be specified on a file or filegroup, in which case it applies to all files in the filegroup. The default value is derived from the value of the FileContentType attribute, and if applicable the current language version specified in the LoadDesigns activity. The legal values for this attribute depend on the IP-XACT schema version.

Examples

```
set_attribute -attr SpiritFileType [find_item -type file file1.v] -value  
verilogSource
```

See Also

add_files_to_filegroup (2), file (3) filegroup (3)

[API Index](#) [Command Reference](#)

SpiritInterfaceType
SPIRIT interface type

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This attribute defines Spirit interface type, valid values are: master, slave, system, mirroredMaster, mirroredSlave, mirroredSystem. This is only used to generate Spirit compliant component or design.

Examples

See Also

[SpiritVendor \(3\)](#), [SpiritLibrary \(3\)](#), [SpiritName \(3\)](#), [SpiritVersion \(3\)](#),
[SpiritContainer \(3\)](#), _____

[API Index](#) [Command Reference](#)

SpiritLibrary
IP-XACT library name

Definition

Type: string

Flags:

Default value: Value calculated internally.

Valid on: cell design _____

Description

The Spirit library name of the interface definition or design. Used to generate Spirit compliant component or design.

Examples

See Also

[SpiritName \(3\)](#), [SpiritVendor \(3\)](#), [SpiritVersion \(3\)](#), _____

[API Index](#) [Command Reference](#)

SpiritName
IP-XACT name

Definition

Type: string

Flags:

Default value: Value calculated internally.

Valid on: cell design _____

Description

The Spirit name of the interface definition or design. Used to generate Spirit compliant component or design.

Examples

See Also

[SpiritLibrary \(3\)](#), [SpiritVendor \(3\)](#), [SpiritVersion \(3\)](#), _____

[API Index](#) [Command Reference](#)

SpiritVendor
IP-XACT vendor name

Definition

Type: string

Flags:

Default value: Value calculated internally.

Valid on: cell design _____

Description

The Spirit vendor name of the interface definition or design. Used to generate Spirit compliant component or design.

Examples

See Also

[SpiritLibrary \(3\)](#), [SpiritName \(3\)](#), [SpiritVersion \(3\)](#), _____

[API Index](#) [Command Reference](#)

SpiritVersion
IP-XACT version

Definition

Type: string

Flags:

Default value: Value calculated internally.

Valid on: cell design _____

Description

The Spirit version of the interface definition or design. Used to generate Spirit compliant component or design.

Examples

See Also

[SpiritLibrary \(3\)](#), [SpiritVendor \(3\)](#), [SpiritName \(3\)](#), _____

[API Index](#) [Command Reference](#)

SplitInterfaceMembers

Indicates the list of interfaces linked with the master interface

Definition

Type: itemList

Flags:

Default value:

Valid on:

Description

This attribute is an ItemList attribute set only on the master interface for the split interface. The attribute contains a list of slave interfaces corresponding to the master.

Examples

See Also

[API Index](#) [Command Reference](#)

SplitInterfaceName

Indicates the Master interface pointed from the slave interface.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

[API Index](#) [Command Reference](#)

StartBit

Tag specification starting position.

Definition

Type: string

Flags: readOnly subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 0

Valid on: net param port _____

Description

Used to indicate the starting bit position of a ranged element. Typically applies to ports and pins. Value can be an integer or a parameter expression that resolves to an integer.

Examples

This attribute is never set explicitly. It's value comes from parsing HDL or IP-XACT files within coreBuilder or coreAssembler.

See Also

[EndBit \(3\)](#)

[API Index](#) [Command Reference](#)

StartBitExpr

Indicates the original StartBit expression (subscripted representation) which was read in from RTL file in LoadDesigns activity

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: param pin port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

StartBitsExpr

Indicates the original StartBits complete expression which was read in from RTL file in LoadDesigns activity

Definition

Type: string

Flags:

Default value:

Valid on: param pin port _____

Description

Examples

See Also

Static Checking

Perform static checking of the subsystem and its components.

Parameters

Parameters Table

Name	Label	Description
Configuration/Flow		
BottomUpFlow	Bottom Up Flow	Run subsystem Spyglass bottom up flow.
Configuration/Setup		
SpyglassVersion	Spyglass Version	Version of Spyglass that will be used.
GuideWareVersion	Guide Ware Version	Version of GuideWare that from which the goals will be sourced.
SubsystemRulesFiles	Subsystem Rules Files	List of rules files to be sourced for the subsystem verification.
SubsystemWaiversFiles	Subsystem Waivers Files	Optional user specified waivers to use at the subsystem level.
SubsystemSgdcFiles	Subsystem Sgdc Files	Optional user specified sgdc files to use at the subsystem level.
CleanComponents	Clean Components	A list of components for which to regenerate abstracts.
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieved using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this

coreTools Command Reference Index

Name	Label	Description
		specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
ParallelJobCpuLimit	Parallel job CPU limit:	Specifies the number of parallel compile jobs that may be run.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.

See Also

Initialize Subsystem Constraints.

[API Index](#) [Command Reference](#)

StaticDefineExpr

Defines the expression which causes the given port to exist. Assumed to be an expression of parameters that are not in the coreTools data model.

Definition

Type: string

Flags:

Default value:

Valid on: param pin port

Description

This attribute is used in conjunction with the `DeferEvaluation` attribute. It defines a static expression (static in that it is defined outside of the scope of `coreBuilder`) which defines when the associated port exists.

Examples

See the `DeferEvaluation` man page for an example using this attribute.

See Also

[DeferEvaluation \(3\)](#)

StaticDefineExprs

Specifies which static defines referenced in StaticDefineExpr values on ports should be treated as defined for constraint generation purposes.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is used to indicate a list of 'define macros that should be treated as defined for purposes of top-level synthesis. The value of this attribute is passed to the Design Compiler 'analyze' command using the -define option. Top-level ports with DeferredEvaluation set to true which have StaticDefineExpr or GenerateIf references to parameters referenced in the StaticDefineExprs value will be treated as defined (or not defined) for purposes of constraint generation.

For example, if a port has DeferredEvaluation==1 and StaticDefineExpr==@P1, then the constraints for the port will not be generated unless P1 is seen in the StaticDefineExprs value for the containing design. Note that the define SYNTHESIS is always treated as defined so ports with DeferredEvaluation==1 and StaticDefineExpr==@SYNTHESIS will have constraints generated and ports with DeferredEvaluation==1 and StaticDefineExpr==!@SYNTHESIS will not.

The defines stored in the attribute can be written out in IPXACT component by setting the value of the GUI preference variable 'include_defines_in_fileset' to true. The defines stored in this attribute are written out in file/define element of the 'Hdl' filegroup under filesets.

Examples

Indicate that the macros PWR_PINS and GND_PINS should be treated as defined:

```
set_design_attribute StaticDefineExprs {PWR_PINS GND_PINS}
```

There are two ways to write out defines in IPXACT component:

1. We can either set the macros in the activity parameter "CompileDefines" of the LoadDesigns activity in coreBuilder.

```
set_activity_parameter LoadDesigns CompileDefines { var1=10 var2  
var3=test_string }
```

2. Or set them in the design attribute "StaticDefineExprs"
set_design_attribute StaticDefineExprs { var1=10 var2
var3=test_string }

Set the value of GUI preference variable 'include_defines_in_fileset' to true to write out defines in IPXACT component

gui_set_pref_value -key include_defines_in_fileset -value true
The IPXACT for the above example will look like this:

```
<ipxact:define>
<ipxact:name>var1</ipxact:name>
<ipxact:value>10</ipxact:value>
</ipxact:define>
<ipxact:define>
<ipxact:name>var2</ipxact:name>
<ipxact:value>1'b1</ipxact:value>
</ipxact:define>
<ipxact:define>
<ipxact:name>var3</ipxact:name>
<ipxact:value>test_string</ipxact:value>
</ipxact:define>
```

The default value "1'b1" is written out if the value of the define or macro is not specified.

See Also

[DeferEvaluation \(3\)](#), [StaticDefineExpr \(3\)](#)

Static Timing Analysis

Generate and (optionally) run static timing analysis scripts.

Parameters

Parameters Table

Name	Label	Description
AnalysisSetup		
AnalysisType	Analysis Mode	Choose the type of analysis to be performed on the netlist. - EnumValues: on_chip_variation single - Symbolic Names: {On-Chip Variation} {Single Operating Condition}
TimingDerateDelayType	Set Delay Derating Factors	Set delay derating factors for either the design or a specified list of instances. - EnumValues: none early late - Symbolic Names: {No Derating} Early Late
TimingDerateValue	Timing Derate Factor	Timing Derate value to be applied
TimingDerateOptions	Timing Derate Options	Arguments to be passed to the 'set_timing_derate' command (in addition to '-early' or '-late')
UsePropagatedClocks	Use Propagated Clocks	Setting this parameter to 'true' specifies that delays be propagated through the clock network to determine latency at register clock pins. A value of 'false' means that ideal clocking is assumed.
AnalyzeInMissionMode	Analyze netlist in mission mode	Perform timing analysis with the netlist in 'mission' mode, i.e. with scan mode disabled. In this mode, any

coreTools Command Reference Index

Name	Label	Description
		scan-enable pin inserted by DC will be driven to the value which will disable scan mode. This parameter has no effect on the analysis of netlists which do not have ports which have been inserted during DFT synthesis by DC.
EnableECOFlow	Enable ECO Flow	Enables the ECO flow in PrimeTime. Enabling it, PrimeTime would run the command 'fix_eco_timing'. This fixes the setup timing violations, set as default, through engineering change order (ECO) changes.
FixECOTimingArgs	Options for fix_eco_timing	Defines arguments to be passed to the fix_eco_timing when writing the Static Timing Analysis intent file.
AuxiliaryScriptName	Auxiliary script name:	Defines the name of an auxiliary script to be sourced prior to the invocation of 'update_timing', after the design has been loaded and all clocks and constraints have been defined.
BottleneckAnalysis		
ReportBottleneck	Run Bottleneck Analysis	Report timing bottlenecks
ReportBottleneckOptions	Bottleneck Analysis Options	Specify arguments to pass to 'report_bottleneck'
ClockReports		
ReportClockTiming	Run Clock Timing Report	Run report_clock_timing report
ReportMinPulseWidth	Run Minimum Pulse Width report	Run report_min_pulse_width report
ReportMinPulseWidthOptions	Minimum Pulse Width Report Options	Arguments to pass to 'report_min_pulse_width' command
ReportClockGatingCheck	Run clock-gating report	Run report_clock_gating_check report
ReportClockGatingCheckOptions	Clock-Gating Report Options	Arguments to pass to 'report_clock_gating_check'

coreTools Command Reference Index

Name	Label	Description
CrossDomainPathReport		
ReportCrossDomainPaths	Report Cross-Domain Paths	Report paths which cross clock domains
NumCrossDomainPaths	Number of paths to report	Maximum number of paths to report (from clock group to clock group)
CrossDomainReportFormat	Report Format	Format of cross-domain report. 'Summary' returns the start and end points of each path, 'short' reports show the cells at the start and end of each path, full reports show each cross-domain path in its entirety. • EnumValues: summary short full • Symbolic Names: {Summary (start- and end-points)} {Short timing paths} {Full (detailed) paths}
SuppressDuplicateBusPaths	Suppress Duplicate Bus Paths	This parameter controls the reporting of bussed paths which cross clock domains. Setting this to 'true' will cause only one path per bus to be reported.
CustomScript		
PTScriptFile	Custom PrimeTime Script	Optional PrimeTime script. PrimeTime will execute this script at the end of its run, after all selected reports have been generated.
DesignAndConstraintChecks		
ReportDesign	Report Design Attributes	Setting this parameter to 'true' runs the 'report_design' command
ReportClocks	Report Clocks	Setting this parameter to 'true' runs the 'report_clocks' command
ReportRegisterInfo	Report Register Information	Setting this parameter to 'true' generates a report detailing the total number of registers in the design. The total number of latches,

coreTools Command Reference Index

Name	Label	Description
		positive- and negative-edge flops per clock domain is also reported. All sequential elements are also listed in the report.
ReportSyncPaths	Report Synchronization Flops	Setting this parameter to 'true' generates a report which shows all the synchronization flops in the design.
ReportCaseAnalysis	Report Case Analysis	Run the 'report_case_analysis' command
ReportExceptions	Report Timing Exceptions	Run the 'report_exceptions' command
ReportExceptionsOptions	Report Timing Exceptions Options	Options and switches to pass to 'report_exceptions'
ReportAnalysisCoverageCheckType	Report Analysis Coverage Check Type	Arguments for the check_type switch to 'report_analysis_coverage'
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieved using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands,

coreTools Command Reference Index

Name	Label	Description
		respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
ParallelJobCpuLimit	Parallel job CPU limit:	Specifies the number of parallel compile jobs that may be run.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.
NetlistData		
UseExternalNetlist	Use externally-generated netlist	Check this box to specify a netlist from a source outside this workspace. Leave unchecked to choose a netlist in this workspace.
UseUPFPGNetlist	Use UPF PG Netlist	Use netlist with PG (power/ground) for Golden UPF flow. When set to false, PG info is in UPF files.
InputStageName	Input stage name:	Name of the stage that contains the design to be analyzed.
NetlistFile	External netlist file:	Path to the target netlist, required if an externally-generated netlist is to be analyzed.
ConstraintsFile	Constraints File	SDC Constraints File to be applied to netlist
BackAnnotatedData	Back-annotated data:	Format of back-annotated data to apply to netlist EnumValues: None SPEF SDF
BackAnnotatedDataFile	Back-annotated data file:	File containing data to back-annotate onto netlist.
BackAnnotationOptions	Back-annotation command string:	Here you can specify any switches and options you wish to pass to 'read_sdf' or

coreTools Command Reference Index

Name	Label	Description
		'read_spef' to guide the backannotation of data
OverrideTechSetup		
OverrideSearchPath	Override Search Paths	A separate group of search paths for libraries which are required in Static Timing Analysis. These search paths are set in the variable ::search_path reflected in [StageName]_pt.ptsh file.
OverrideLinkLibraryMapping	Override Link Libraries	A separate group of link libraries which are specifically to be used for Static Timing Analysis. This attribute when enabled will override the previous TechSetup link libraries and set the new libraries. These libraries will be set in the variable ::link_libraries reflected in [StageName]_pt.ptsh file. This parameter uses a mapping technique, which is used to run Static Timing Analysis with the updated SDC. To provide the inputs to this parameter we must set it up in a list of list sequence. Eg <pre>{{Link_Library_Tech_Setup_1 Override_Link_Library_1} {Link_Library_Tech_Setup_2 Override_Link_Library_2} ... }</pre> Now when the Static Timing Analysis activity is fired, coreTools checks for the Link_Library_Tech_Setup_* attributes in the SDC and updates it to the parameters of Override_Link_Library_*. Only Override_Link_Library_1 will be used to extract the Operating Condition and update the Link_Library_Tech_Setup_1 OPCON in the SDC and pt intent file.

coreTools Command Reference Index

Name	Label	Description
OverrideDefaultOperatingCondition	Override Default Operating Condition	A optional paramter when using different set of libraries in Static Timing Analysis. Used to change the Operating Condition during STA. If not provided the tool will automatically extract the corresponding Operating Condition from the provided reference library given in OverrideLinkLibraryMapping.
PowerAnalysisSetup		
PowerAnalysisFlow	Power Analysis Flow	Power Analysis flow to use: select 'VCD/VCD+/FSDB' for analysis using waves, 'SAIF' for analysis based on a SAIF file, or 'Vector-free' to estimate power without simulation data • EnumValues: vcd saif vector-free • Symbolic Names: VCD/VCD+/FSDB SAIF Vector-Free
PowerAnalysisMode	Power Analysis Mode	Specify power analysis mode. Use this parameter to choose either averaged or time-based analysis. This feature is supported in PrimeTime-PX 2008.12 or newer • EnumValues: averaged time_based • Symbolic Names: Averaged Time-based
SwitchingActivityFile	Switching Activity File	Switching activity file to be used by PrimeTime-PX for power analysis. For vector-based analysis, this is either a VCD/VCD+/FSDB or SAIF file. For the vector-free flow, this is an optional file containing toggle and switching commands for PrimeTime-PX
SimulationType	Simulation Type	Use this parameter to specify whether your VCD (or VCD+ or

coreTools Command Reference Index

Name	Label	Description
		FSDB) wave file was generated from an RTL or gate-level simulation - EnumValues: gate rtl zero_delay - Symbolic Names: {Gate-level Simulation} {RTL Simulation} {Zero-Delay Gate-level Simulation}
SaifNameMapFile	SAIF Name Mapping File	For power analysis using data (waves or SAIF) captured from an RTL simulation, this optional file can be used to specify mapping between RTL and gate-level object names. As the names of RTL objects may change after synthesis, providing a mapping file may help to increase the accuracy of power calculations by helping PrimeTime-PX to annotate switching activity accurately to the gate-level netlist.
StripPath	Testbench Path to design instance	If the current design is instantiated as an instance in a testbench used to generate a VCD/VCD+/FSDB or SAIF file, this parameter is used to specify the instance of the current design as specified in the testbench, e.g. test_toplevel/U_instance.
StartTime	Start Time for power analysis	Start time of time window for power calculation. A value of zero means that calculation will start from the beginning of the wave file.
Mode	Mode	String to pass to '-mode' switch of 'read_vcd' command
CalculatePowerWhen	CalculatePowerWhen	String to pass to '-when' switch of 'read_vcd'/'read_fsdb' command. Requires PT-PX D-2009.12 or newer & VCD/VPD

coreTools Command Reference Index

Name	Label	Description
		waveform file or PT-PX K-2015.12 or new for FSDB waveform file; SAIF not supported
EndTime	End Time for power analysis	End time of time window for power calculation. A value of -1 means that calculation will cover the time window from the specified start time to the end of the wave file.
UseOldPtPxInterface	Use Pre-2008.12 PrimeTime-PX User Interface?	Use pre-2008.12 PT-PX User Interface. Set this parameter to 'true' if you want to run mode-based power calculations
PowerReport		
ReportPower	Run Power Analysis	Run Power Analysis using PrimeTime-PX
PowerAnalysisOptions	Power Analysis Options	Options to pass to 'set_power_analysis_options' command in PrimeTime-PX. This is only supported in PrimeTime-PX 2008.12 or newer
ReportPowerOptions	Power Report Options	Options to be passed to the 'report_power' command in PrimeTime-PX
ReportSwitchingActivity	Report Switching Activity	Run 'report_switching_activity' to generate a report of annotated switching activity on the design
ReportSwitchingActivityOptions	Switching Activity Report Options	Options and switches to pass to 'report_switching_activity'
ReportClockGateSavings	Report Clock Gate Savings	Run variations of the report_clock_gating_savings report
TestMAX		
LinkToATPG	Link To TestMax	Create timing files needed by TestMax. ActivityAliasNames : LinkToTetraMax
TimingReports		
ReportMaxTiming	Report Timing (Max Delay)	Run the 'report_timing' command to report 'max' path delays

coreTools Command Reference Index

Name	Label	Description
ReportMaxTimingOptions	Report Timing (Max Delay) Options	Additional options and switches to pass to 'report_timing -delay max'
ReportMinTiming	Report Timing (Min Delay)	Run the 'report_timing' command to report 'min' path delays
ReportMinTimingOptions	Report Timing (Min Delay) Options	Additional options and switches to pass to 'report_timing -delay min'
ReportConstraints	Report Constraints	Run the 'report_constraints' command
ReportConstraintsOptions	Report Constraints Options	Options to pass to 'report_constraints' command

See Also

Perform ASIC Synthesis.

[API Index](#) [Command Reference](#)

StaticTimingAuxScript

Specifies a script that is to be run in PrimeTime after the design has been loaded and all clocks and constraints have been applied, just before the 'update_timing' command.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute can be used by the IP provider to specify an auxiliary script that will be packaged with the core to be run in PrimeTime during static timing analysis. The script will be run after the design is loaded and constrained, before the 'update_timing' command is run.

Examples

Specify that the script staticTimingAuxiliary.tcl is to be packaged with the core and run in PrimeTime.

```
coreBuilder> current_design [get_top_design]
coreBuilder> set_design_attribute StaticTimingAuxScript
/full/path/to/staticTimingAuxiliary.tcl
```

See Also

[set_design_attribute \(2\)](#), _____

[API Index](#) [Command Reference](#)

StaticTimingAuxScriptComment

Specifies a comment to be issued for the corresponding StaticTimingAuxScript.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

The StaticTimingAuxScriptComment attribute can be used to output a comment to document why the corresponding StaticTimingAuxScript is being used. The comment will be put into the file where the script is sourced.

Examples

Specify an aux script to be used in the setup stage, and document why it is being used.

```
set_design_attribute StaticTimingAuxScript myAuxScr.tcl
set_design_attribute StaticTimingAuxScriptComment "This script is used to
set up custom variables for the static timing analysis."
```

See Also

[StaticTimingAuxScript \(3\)](#)_____

Strategy Index

[ATPG basic scan strategy](#)

The ATPG_basic_scan_strategy will create test vectors using the Basic_scan_only engine of TetraMax. This is the recommended starting point for generating vectors, and it will provide fast, good coverage for full scan designs.

Generates scripts that do a 'quick map' of this core to the target technology. This is the fastest possible mapping to gates, and should be used as an estimation path only. You can also select the Stop at GTECH parameter to elaborate your configured core to the Synopsys technology-independent GTECH library, without actually compiling the core to your target technology. Generating a GTECH version of the core is an easy way to create a simulation model for a core that is delivered as encrypted HDL.

[DCTCL quick map strategy](#)

Performs a one pass, top down compile from RTL using compile_ultra. This flow implements the Design Compiler Reference Methodology associated with the version of Design Compiler being used.

[Design Compiler Reference Methodology](#)

Run formal verification on a design.

[FM formal verification strategy](#)

Performs synthesis from RTL targeting an FPGA using synplify.

[FPGA optimization strategy](#)

Performs a one pass, top down compile from RTL using compile_fusion. This flow implements the Fusion Compiler Reference Methodology associated with the version of Fusion Compiler being used.

[Fusion Compiler Reference Methodology](#)

[PT static timing analysis strategy](#)

coreTools Command Reference Index

RTL Architect Reference Methodology

Run Static Timing Analysis on a design using PrimeTime

Runs RTL Architect under the RTL Architect Reference Methodology associated with the version of RTL Architect being used.

TMAX Reference Methodology

This flow implements the TestMAX Reference Methodology. For details refer to the TMAX Reference Methodology.

Strategy

Item used to model a strategy (command script) to drive a design tool

Description

The Strategy item type represents a set of files that together form a hierarchical set of command scripts used to drive a particular tool. In addition to a set of files, the strategy has other information associated with it, including: the name of the tool the strategy applies to, flexlm licenses required to use the strategy, tool version information, and parameters required to use the strategy.

See Also

[set_strategy_parameter \(2\)](#)

Supported Attributes

[ComponentOfItem \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [Enabled \(3\)](#), [Label \(3\)](#), [Name \(3\)](#), [ParameterCheckCmd \(3\)](#), [ParameterInfo \(3\)](#), [Sequence \(3\)](#), [TypeName \(3\)](#),

SubstituteInFile

Substitute text in this file at the time indicated by the subscript

Definition

Type: boolean

Flags: subscripted

Default value: 0

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: OnWrite

Valid on: file filegroup filegroupGroup

Description

The SubstituteInFile attribute directs coreConsultant to perform text substitution in the file or file group at the time indicated by the SubstituteInFile subscript.

If SubstituteInFile[OnWrite] = true, text substitution occurs each time coreConsultant writes the file or file group to disk, including during coreKit installation.

To prevent text substitution during coreKit installation and, instead, only perform text substitution when coreConsultant writes the file(s) to a user workspace, set SubstituteInFile to true with any other text string as a subscript.

For example, to enable text substitution when coreConsultant installs files from the BoM as part of a custom activity, you need to do the following things:

- Include reuse-pragma with parameter_conditional_text in the files that require text substitution.
- Set SubstituteInFile[<subscript>] to true on the file(s) or file group(s) that require text substitution.
- Use the install_filegroup command with the -substitute <when> option in the command procedure for the custom activity to install the file(s) from the BoM at the required time. The value of <when> must be the same as the <subscript> on the SubstituteInFile attribute on file(s) or file group(s) that require text substitution.

To control how text substitution occurs within a file, use reuse-pragma in the file to invoke a Tcl command procedure that substitutes the text as required. coreBuilder provides three built-in command procedures for text substitution. You can use the built-in command procedures, other Tcl commands, or your own custom command procedures. The three built-in command procedures for text substitution are:

- `ReplaceConstantParam` - Replace a VHDL constant or Verilog `'define` that provides a user-configurable "parameter" for the core.
- `ReplaceDesignParams` - Replace a block of VHDL generics or Verilog parameters that provides a set of user-configurable "parameters" for the core.
- `parameter_conditional_text` - Replace any other block of text in a text file.

Examples

To enable text substitution in `MyFile.txt` whenever `coreConsultant` writes `MyFile.txt` to disk, including during `coreKit` installation (no subscript required because `OnWrite` is the default subscript):

```
coreBuilder> set_attribute -attr SubstituteInFile  
-value true MyFile.txt
```

To enable text substitution in the files in file group "testbench" whenever `coreConsultant` writes file group "testbench" to disk, except during `coreKit` installation:

```
coreBuilder> set_attribute -attr SubstituteInFile  
-subscript ConfigTB  
-value true testbench
```

To install the file group from the previous example as part of a custom verification activity:

```
install_filegroup -substitute ConfigTB -subscript ConfigTB  
testbench
```

See Also

[install_filegroup \(2\)](#), [IncludeIf \(2\)](#), [ReplaceConstantParam \(2\)](#), [ReplaceDesignParams \(2\)](#), [set_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

SymbolLibraryPath

The path to the symbol library to be loaded when the component is displayed in a schematic.

Definition

Type: string

Flags:

Default value:

Valid on: cell design _____

Description

This attribute is used to indicate the symbol library containing the symbol for the component. Use SymbolName to specify the actual symbol name for the component.

Use library compiler to create your own customized symbols. Create a fileName.slib and use the read_lib and write_lib commands to read in the library and write out a fileName.sdb file.

Examples

To specify the symbol library path:

```
prompt> set_design_attribute SymbolLibraryPath ./amba.sdb
```

See Also

[SymbolName](#) (3),

[API Index](#) [Command Reference](#)

SymbolName

The name of the desired symbol to use in the schematic display.

Definition

Type: string

Flags:

Default value:

Valid on: cell design _____

Description

This attribute is used to indicate the name of the symbol to be used in this command display. The symbol must exist in a library that is specified using the SymbolLibraryPath attribute.

Examples

To indicate that a specific symbol to display for the DW_ahb component:

```
prompt> set_design_attribute SymbolName DW_ahb
```

See Also

[SymbolLibraryPath \(3\).](#)

SymbolPinPrefix

The desired pin prefix name (Master).

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This attribute is used to indicate the name to be shown for instances of this interface in the subsystem schematic. The attribute is called a 'prefix' because the most common usage is for busses where there is a separate pin on the schematic for each connection to the bus. In that case, the value of this attribute is used as the pin name, but with a colon and unique index value appended.

Examples

To set the schematic pin name for slaves of a bus to be 'Slv':

```
prompt> set_interface_attribute Bus_Slave SymbolPinPrefix Slv
```

See Also

SymbolPinSide

The desired side of the symbol to place the pin

Definition

Type: string

Flags:

Default value: =sIntf::getDefaultPinSide %item

Valid on:

Description

Indicates which side of the component pins for this interface should appear on. Valid values are 'left', 'right', 'top', 'bottom', and 'none'. When 'none' is specified, the pins will be placed on the left side for providers (typically busses) or the right side (typically slaves).

Examples

See Also

[API Index](#) [Command Reference](#)

SymbolType

The desired symbol type to be generated

Definition

Type: string

Flags:

Default value: normal

Valid on: design_____

Description

This attribute is used to indicate the generic type of a component. Valid values are: 'normal', and 'bus'. This attribute is set on the top-level design and determines the color of the symbol in the schematic. Since the default is 'normal', this attribute is typically only set explicitly on busses.

Examples

To indicate that the given design represents a bus:

```
prompt> set_design_attribute SymbolType bus
```

See Also

[SymbolPinPrefix \(3\)](#), [SymbolPinSide \(3\)](#)_____

SymbolicNames

List of symbolic names for valid values

Definition

Type: string

Flags:

Default value:

Valid on: attrDefn_____

Description

The SymbolicNames attribute specifies meaningful names for a parameter's legal values. The symbolic names appear in the parameter specification dialog instead of the actual parameter values.

For example, if your design has two possible internal RAM sizes that are selected by setting a parameter to 0 or 1, you can use the actual RAM sizes (for example, 128-byte and 256-byte) as the SymbolicNames for the parameter's legal values 0 and 1. If you do not specify SymbolicNames, the actual legal values (0 and 1) appear in the parameter specification dialog.

If you specify SymbolicNames, the SymbolicNames value must contain exactly one list element for each legal value of the parameter. If you use EnumValues to specify a set of legal parameter values, specify SymbolicNames as a list of symbolic names in the same order as their corresponding EnumValues.

If you use MinValue and MaxValue to set the legal range of values for the parameter, specify SymbolicNames as a list of symbolic names from the minimum legal parameter value to the maximum legal value. In cases where MinValue and MaxValue are automatically derived from the parameter declaration in the HDL code, specify SymbolicNames in order of increasing value even if the parameter value range declared in the HDL code is specified in descending order (for example, 2 downto 0).

Examples

The following example VHDL annotations illustrate how to set SymbolicNames for a variety of legal parameter values. For the serial_ports parameter, the SymbolicNames values are specified in increasing order even though the legal range declared in the HDL code is specified in decreasing order. This is necessary because coreBuilder automatically sets MinValue to 0 and MaxValue to 2 for the serial_ports parameter.

```
generic (  
--reuse-pragma attr EnumValues 16 32  
--reuse-pragma attr SymbolicNames "16 Bits" "32 Bits"  
bus_width : integer := 32;  
--reuse-pragma attr MinValue 1  
--reuse-pragma attr MaxValue 4  
--reuse-pragma attr SymbolicNames One Two Three Four  
intrs : integer := 2;  
--reuse-pragma attr SymbolicNames Zero One Two  
serial_ports : integer range 2 downto 0 :=1  
);
```

See Also

set_parameter_attribute (2), EnumValues (3), MinValue (3), MaxValue (3)

SymmetricBus

Indicates the interface is symmetric which allow direct master-slave connection. The bus definition should be defined as 'slave' type, then 'master' type is just symmetric to the slave. Imply 'provider-consumer' interface type for interface instance.

Definition

Type: boolean

Flags:

Default value: 0

Valid on:

Description

This attribute indicates the interface definition is symmetric which allow direct master-slave connection. The interfacePort of the bus definition should be defined as 'slave' type in terms of port direction, then 'master' type is just symmetric to the slave. If an interface instance is instantiated from symmetric bus, it implies a 'provider-consumer' interface type unless type is specified.

Examples

use '-symmetric' option in create_interface command to create a symmetric bus definition: coreAssembler>create_interface D1 -symmetric

See Also

[SymmetricBusLink \(3\)](#), [create_interface \(2\)](#), [create_interface_port \(2\)](#),
[create_interface_parameter \(2\)](#), [create_interface_instance](#)

SymmetricBusLink

Indicates the association to a design port when a symmetric bus is used as a provider or a consumer. The design name can include @<paramName> and {<expr>} in the attribute value, and allows a [@<paramName>] index annotation.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: consumer provider

Default subscript:

Valid on:

Description

This attribute indicates the association to a design port when a symmetric bus (provider-consumer type of interface) is used as a provider or a consumer. A symmetric interface can connect to either provider or consumer type of interface, this attribute is used to indicate the InterfaceLink to a design port under different connection.

Examples

set the association to a design port of provider-consumer interface instance I1 port p1 to different design ports using the following commands: coreBuilder>set_interface_port_attribute -instance I1 p1 SymmetricBusLink[provider] Q coreBuilder>set_interface_port_attribute -instance I1 p1 SymmetricBusLink[consumer] D

See Also

[InterfaceLink \(3\)](#), [set_interface_port_attribute \(2\)](#),
[get_interface_port_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

SynchronousTo

Indicates that the given port is synchronous with respect to the given clock or other designer specified conditions.

Definition

Type: string

Flags:

Default value:

Valid on: busBit port _____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

SynplifyAuxSynthesisScript

Specifies a synplify script to be executed prior to specific optimization phase.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: cleanup elab synthesis

Default subscript:

Valid on: design_____

Description

The SynplifyAuxSynthesisScript attribute specifies the name of a Synplify script that you want to execute before a particular phase of synthesis for the selected design. Typically this would be used to set up non-default options using the set_options command.

The subscript indicates the phase before which you want to execute the auxiliary synthesis script:

- elab - Execute the auxiliary script before elaboration.
- synthesis - Execute the auxiliary script before synthesizing the design
- cleanup - Execute the auxiliary script after all processing is complete, just before quitting.

Examples

See Also

[API Index](#) [Command Reference](#)

SynplifyAuxSynthesisScriptComment

Specifies a comment to be issued for the corresponding SynplifyAuxSynthesisScript.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: cleanup elab synthesis

Default subscript:

Valid on: design_____

Description

The SynplifyAuxSynthesisScriptComment attribute can be used to output a comment to document why the corresponding SynplifyAuxSynthesisScript is being used. The comment will be put into the file where the script is sourced.

Examples

Specify an aux script to be used in the setup stage, and document why it is being used.

```
set_design_attribute SynplifyAuxSynthesisScript myAuxScr.tcl
set_design_attribute SynplifyAuxSynthesisScriptComment "This script is
used to set up custom variables for the test pattern generation."
```

See Also

[SynplifyAuxSynthesisScript \(3\)](#)_____

Synthesis_API

This man page describes the different methodologies available for generating a gate-level implementation of a component configured in coreConsultant or a subsystem assembled in coreAssembler.

Syntax

Description

The recommended flow is to utilize the Synthesize activity within coreConsultant or coreAssembler. This activity utilizes a script generator which targets your particular version of Design Compiler and generates synthesis scripts customized for your subsystem and target technology. The IP can then be integrated with the chip by running Design Compiler, reading the ddc file <workspace>/syn/final/<topDesign>.ddc, and linking it into the rest of the chip. The advantage of this flow is that all of the coreTools synthesis optimizations are run for the selected QorEffort of the base strategy, and coreTools incremental strategies may be run prior to integrating the final db with the chip.

This process can be integrated into your chip-level synthesis scripts using the following script fragment:

```
# Save the current design and working directory
set currDes [current_design]
set saveDir [pwd]
# cd to workspace/syn, run synthesis, read in mapped design.
cd <workspace>/syn
exec run.scr
```

```
read_ddc ./final/db/<topDesignName>.ddc
```

Other integration methodologies are also supported. For example, the write_sdc command can be utilized to write configured constraints for a component or subsystem. These constraints and the configure RTL can then be utilized in a custom synthesis methodology. The write_sdc command can be utilized any time after the component (subsystem) has been configured. However, if a technology library is not loaded in the Technology Setup activity, the generated SDC file will contain constraints relative to a generic technology library. These constraints will need to be modified to match your target library cell names. This issue can be avoided by loading the technology library within the Technology Setup activity.

For dc_shell Tcl mode, scripts are also provided that will allow integration of the IP synthesis with an ad hoc chip synthesis flow. The scripts can integrated the core into the chip with constrained RTL, unconstrained RTL, and mapped gates. For mapped gates, integration takes place after only the initial compile phase has been performed, so the quality of results may be inferior to those obtained through the

recommended flow shown above. The scripts can be accessed in the <workspace>/export/synthesis directory.

The scripts are created when the DCTCL_opto_strategy is selected and the the Synthesize Activity is completed.

Once in Design Compiler source the appropriate script inside of your chip level synthesis scripts.

Examples

For all examples assume that the chip level synthesis is being run in the directory chipSynDir, the design to be integrated is in the directory chipSynDir/myWorkspace, and the top design from the workspace is topDesign.

Integrate a mapped design into the chip:

```
# Set the target_library and link_library first.  
source myWorkspace/export/synthesis/compile.tcl  
Integrate an unmapped design that has been constrained:
```

```
# Set the target_library and link_library first.  
source myWorkspace/export/synthesis/elab_and_constrain.tcl  
Integrate an unmapped, unconstrained design:
```

```
# Set the target_library and link_library first.  
source myWorkspace/export/synthesis/elab_only.tcl
```

See Also

Perform ASIC Synthesis

Generate and (optionally) run synthesis scripts.

This activity is used to generate synthesis scripts customized for the particular core configuration, DC/FC version compatibility, and target technology. Now user have the option between Design Compiler and Fusion Compiler to run synthesis. By default synthesis is fired right after the scripts are generated, using the preferred tool. But this behavior can be changed by checking the Generate Scripts Only check-box.

Once the synthesis is finished, results will be available in the following locations:

For DC only : DDC file - syn/final/db/<top design>.ddc

For DC & FC : Netlist - syn/final/db/<top design>.v[hd] Reports - syn/final/report/

If you choose to run the synthesis yourself, or wish to review the generated scripts or constraints, these files can be found in the following locations:

Launch script - syn/[scriptFile] Where the name of the 'scriptFile' can be obtained with the command: get_activity_parameter Synthesize ScriptFile

Constraints - syn/constrain/script/*.sdc/tcl]

If you wish to integrate the generated scripts into your own chip-level synthesis environment, please see the 'Synthesis_API' man page for details about how to access the generated scripts from your own environment.

Parameters

Parameters Table

Name	Label	Description
TclMode	TclMode	Obsolete parameter to choose between dc-tcl mode and dc eqn mode. Eqn mode is no longer supported.
Automotive Reports		
ReportSafetyRegisterRules	Generate safety register rules report	Reports safety register rules from the design. This report is only generated when TMR or DCLS intent are specified.
ReportSafetyRegisterGroups	Generate safety register	Reports safety register groups of the

coreTools Command Reference Index

Name	Label	Description
	groups report	design. This report is only generated when TMR or DCLS intent are specified
ReportSafetyStatus	Perform & report safety related checks	Perform and report checks related to safety_register_rules and safety_register_groups. This report is only generated when TMR or DCLS intent are specified
Checker Reports		
DcThroughPathReport	Generate Through-Path report	Should a report of all 'through' paths be generated along with the other DC reports. This report consists of a list of top-level inputs and outputs that are directly connected via combinational logic. These 'snake' paths are typically undesirable from a synthesis point of view. This report is optional because it can take a while to run if there are a large number of outputs which have a large number of combinational paths.
DcRegisteredIOReport	Generate registered I/O report	Should a report of all 'unregistered' I/O ports be generated along with the other DC reports. This report consists of a list of ports that are not directly connected to a register within the core design. Registering inputs and/or outputs makes the synthesis process easier and more reliable, so registered I/Os are generally a good goal to strive for. This report is optional because it can take a while to run if there are a large number of ports (particularly large busses).
DcSetupReport	Generate DC setup report	Controls whether or not a synthesis setup report is generated?
DcCheckDesignReport	Generate check design report	Should a DC check_design report be generated?
DcLatchReport	Generate latch report	Should a DC latch report be generated?
DcLoopsReport	Generate loops report	Should a DC combinational loops report be generated?
DcCheckTestReport	Generate check test report	Should a DC check_test report be generated?
DFT Reports		
DFTCoverageEstimateReport	Generate DFT coverage estimate report	Controls whether or not to generate the DFT coverage estimate report. Applicable only if the DFT flow is enabled.

coreTools Command Reference Index

Name	Label	Description
Execution Options		
ScriptsOnly	Generate scripts only?	Setting this causes the script run.scr to be generated in the WorkDirectory, but the script does not get executed. Users can then run the script at a later time. The path to the run.scr can be gotten by retrieved using the following command: 'get_logical_dir syn/run.scr'
RunStyle	Run style:	If 'Generate scripts only' is unchecked, then this selects how to run the command, which can be run locally, via LSF, via GRD, or via the remote shell. EnumValues: local lsf uge grd
RunOptions	Run Options:	Additional options for the given run style. For remote this should be a hostname. For lsf and uge/grd this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
ParallelJobCpuLimit	Parallel job CPU limit:	Specifies the number of parallel compile jobs that may be run.
EmailAddress	To:	If this parameter is set to a value other than '', then email is sent when the command script completes or is killed. The default value is the current user's name.
NoHomeInit	Skip reading \$HOME/.synopsys_dc.setup	Force tools not to read the .synopsys_dc.setup file for dc_shell & dcnxt_shell or .synopsys_fc.setup file for fc_shell from \$HOME.
UseDCNXT	Use Design Compiler NXT	Indicates if DC-NXT is to be used for synthesis. DC-NXT is available starting P-2019.03 version of Design Compiler.
EnableAutoReadDC	Enable Autoread Flow	Uses autoread mode in DC to analyze the RTL files. To enable this option 'select the AutoRead Supported?' in coreBuilder LoadDesign activity under Advanced option

coreTools Command Reference Index

Name	Label	Description
Licenses		
DC_Ultra_Opt	DC-Ultra-Opt	If checked, features requiring this license can be utilized by the synthesis script generator.
DC_Ultra_Features	DC-Ultra-Features	If checked, features requiring this license can be utilized by the synthesis script generator.
DesignWare	DesignWare	If checked, features requiring this license can be utilized by the synthesis script generator. ActivityAliasNames : DesignWare_Foundation
BOA_BRT	BOA-BRT	If checked, features requiring this license can be utilized by the synthesis script generator.
Test_Compiler	Test-Compiler	If checked, features requiring this license can be utilized by the synthesis script generator.
Test_Compiler_Plus	Test-Compiler-Plus	If checked, features requiring this license can be utilized by the synthesis script generator.
Test_DFTC_TMAX	Test-DFTC-TMAX	If checked, features requiring this license can be utilized by the synthesis script generator.
DC_Expert	DC-Expert	If checked, features requiring this license can be utilized by the synthesis script generator.
PrimeTime	PrimeTime	If checked, features requiring this license can be utilized by the synthesis script generator.
Physical Reports		
GenerateIgnoredLayersReport	Generate ignored layers report	Reports the routing layers that are ignored during congestion analysis and RC estimation. The report gets generated for DCRM synthesis (only when PhysicalSynthesis is enabled) & FCRM synthesis run. The name of producing report is [DesignName]_ignored_layers.rpt
Physical Tool Options		
PlacementFileFormat	Placement file format	File format that is used by the placement tool. EnumValues: PDEF DEF
ReadDefOptions	read_def Options	

coreTools Command Reference Index

Name	Label	Description
		Options to be passed to the read_def command.
RouterFileFormat	Router file format	File format that is used by the routing tool. EnumValues: PDEF DEF
WriteDefOptions	write_def options	Options to be passed to the write_def command.
Strategy		
XgMode	Use XG mode for Synthesis	Obsolete parameter to enable XG-Mode in DC
WriteDbInXgMode	Write DB In XG Mode.	Obsolete parameter which, if true, forces the design to be saved in db format.
StrategyName	Strategy:	Selects the strategy coreConsultant should use to synthesize this block. EnumValues: Design_Compiler_Reference_ DCTCL_quick_map_strategy DCTCL_one_pass_compile_u
Timing Reports		
DcConstraintReport	Generate constraint report	Controls whether a constraint report is generated.
DcInternalTimingReport	Generate internal timing report	Controls whether a register-register timing report is generated.
DcQtoDTimingReport	Generate register Q to D timing report	Controls whether a register output to register input timing report is generated.
DcShortIoTimingReport	Generate short I/O timing report	Controls wheter a short path timing report will be generated for each primary I/O. The short path report shows only the startpoint and endpoint for each path along with clock and external delay data.
DcLongIoTimingReport	Generate long I/O timing report	Controls wheter a path only timing report will be generated for each primary I/O. The path only report shows just the design elements in the timing path. No clock data are shown.
DcQorReport	Generate Qor report	Should a DC QoR report be generated.
DcClockReport	Generate clock report	Should a DC clock report be generated
DcTimingExceptionsReport	Generate timing exceptions report	Should a DC timing exceptions report (report_timing_requirements) be generated?
DcPathGroups		

coreTools Command Reference Index

Name	Label	Description
	Generate Path Group Report	Should a path group report be generated?
Utilization Reports		
DcHierarchyReport	Generate hierarchy report	Should a DC hierarchy report be generated?
DcClockGatingReport	Generate Clock Gating Report	Controls whether or not a clock gating report will be generated.
DcSelfGatingReport	Generate Self Gating Report	Controls whether or not a self gating report will be generated. Only applicable for Design Compiler Reference Methodology. The report will be written out when both SelfGating strategy parameter and DcSelfGatingReport parameter is enabled. The report gets generated as [design_name].mapped.self_gating.rpt
DcReferenceReport	Generate reference report	Controls whether or not a reference report will be generated, and whether it should be for the top level only, for designs on the compile frontier, or for all designs.
DcPowerDomainsReport	Generate power domains report	Controls whether to generate a power domain reports, and other other reports for a multi-voltage synthesis flow using UPF files. The other reports are: power switches, supply nets, power state table, level shifters, isolations cells, and retention cells.
DcResourcesReport	Generate resources report	Controls whether or not a resources report will be generated, and whether it should be for the top level only, for designs on the compile frontier, or for all designs.
DcRegisterCountReport	Generate register count report	Controls whether or not to generate a report detailing the total number of registers present in the synthesized design.
DcFaninFanoutReport	Generate fanin/fanout report	Controls whether or not to generate the fanin/fanout estimate report.
GenerateAreaReport	Generate area report	Controls whether or not to generate the area information report for the current design.
Wait For Licenses		
WaitForLicenses	Wait For Licenses	Number of minutes to wait for Design Compiler licenses. Will cause Design Compiler to be invoked using

Name	Label	Description
		'-checkout Design-Compiler -wait [minutes]'. Inside of Design Compiler the scripts will also wait before giving up when checking out needed licenses such as Ultra, Test-Compiler, etc.

See Also

[Technology Cell Binding.](#)
[Specify Clocks.](#)
[Specify Operating Conditions & WireLoads.](#)
[Specify Port Constraints.](#)
[Specify Synthesis Methodology.](#)
[Specify Test Methodology.](#)

SynthesizeAsSubblock

The attribute indicates that design needs to be synthesized separately and a DDC (when synthesized with Design Compiler) or NDM libs (when synthesized with Fusion Compiler) will be generated for the design. The generated DDC or NDM will be consumed by the top design during hierarchical synthesis flow in coreAssembler.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Examples

See Also

TCBDisableUndefMacros

Disable writing out undefine directives for tech cell bound in TechnologyCellBinding activity.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Setting this to 1 completely disables writing out of undef directives for cells bound in the TechnologyCellBinding activity. This must be used with caution and generally left unset (default value).

Examples

```
# example coreBuilder script
create_workspace -name ws_cb_compK -replace
set_activity_parameter LoadDesigns InputLanguage SystemVerilog
set_activity_parameter LoadDesigns InputFileNames
{SOURCE/DWbb_bcm21_and.v SOURCE/DWbb_bcm21_or.v SOURCE/DWbb_bcm21.v
SOURCE/DWbb_bcm21_inst.v SOURCE/compK-undef.svh SOURCE/compK.v}
set_activity_parameter LoadDesigns TopDesignName compK
autocomplete_activity LoadDesigns
set_design_attribute TCBDisableUndefMacros 1
set_attribute -attr GenericAttr -subs ComponentUndefFile [get_top_design]
-value "compK-undef.svh"
source -echo -verbose SOURCE/bcm_defines.tcl
source -echo -verbose SOURCE/DWbb_bcm21_tech_cell_mappings.tcl
# example
define_tech_cell_mapping \
-module DWbb_bcm21 \
-macro DWC_SYNC_1_STAGE_SRC \
-description "1-stage Synchronizer Cell" \
-instance U_sync_1 \
-width {[WIDTH-1:0]} \
-enable_macro_args \
-extra_macro_args {WIDTH} \
-port_names "clk_d rst_d_n data_s data_d" \
-port_descriptions { "Destination Clock" "Destination Asynchronous Reset"
"Source Domain Data Vector" "Destination Domain Data Vector" } \
-file DWbb_bcm21.v
autocomplete_activity BuildcoreKit
```

See Also

[InsertTechCellBindMacroAtStart \(3\)](#) [ForceTCBToUseComponentUndefFile \(3\)](#)
[TechCellMappingDefn \(3\)](#) [TechCellMappingBind \(3\)](#)

TCL-Built-in-Commands Index

<u>after</u>	Execute a command after a time delay
<u>append</u>	Append to variable
<u>apply</u>	Apply an anonymous function
<u>array</u>	Manipulate array variables
<u>berror</u>	Command invoked to process background errors
<u>binary</u>	Insert and extract fields from binary strings
<u>break</u>	Abort looping command
<u>case</u>	Evaluate one of several scripts, depending on a given value
<u>catch</u>	Evaluate script and trap exceptional returns
<u>cd</u>	Change working directory
<u>chan</u>	Read, write and manipulate channels
<u>class</u>	class of all classes
<u>close</u>	Close an open channel
<u>concat</u>	Join lists together
<u>continue</u>	Skip to the next iteration of a loop
<u>copy</u>	create copies of objects and classes
<u>coroutine</u>	Create and produce values from coroutines
<u>dde</u>	Execute a Dynamic Data Exchange command
<u>define</u>	define and configure classes and objects
<u>dict</u>	Manipulate dictionaries
<u>encoding</u>	Manipulate encodings
<u>eof</u>	Check for end of file condition on channel
<u>error</u>	Generate an error
<u>eval</u>	Evaluate a Tcl script
<u>exec</u>	Invoke subprocesses
<u>exit</u>	End the application
<u>expr</u>	Evaluate an expression
<u>fblocked</u>	Test whether the last input operation exhausted all available input
<u>fconfigure</u>	Set and get options on a channel
<u>fcopy</u>	Copy data from one channel to another
<u>fileevent</u>	Execute a script when a channel becomes readable or writable
<u>filename</u>	File name conventions supported by Tcl commands
<u>flush</u>	Flush buffered output for a channel
<u>for</u>	'For' loop
<u>foreach</u>	Iterate over all elements in one or more lists

coreTools Command Reference Index

<u>format</u>	Format a string in the style of sprintf
<u>gets</u>	Read a line from a channel
<u>glob</u>	Return names of files that match patterns
<u>global</u>	Access global variables
<u>http</u>	Client-side implementation of the HTTP/1.1 protocol
<u>if</u>	Execute scripts conditionally
<u>incr</u>	Increment the value of a variable
<u>info</u>	Return information about the state of the Tcl interpreter
<u>interp</u>	Create and manipulate Tcl interpreters
<u>join</u>	Create a string by joining together list elements
<u>lappend</u>	Append list elements onto a variable
<u>lassign</u>	Assign list elements to variables
<u>library</u>	standard library of Tcl procedures
<u>lindex</u>	Retrieve an element from a list
<u>linsert</u>	Insert elements into a list
<u>list</u>	Create a list
<u>llength</u>	Count the number of elements in a list
<u>lmap</u>	Iterate over all elements in one or more lists and collect results
<u>load</u>	Load machine code and initialize new commands
<u>lrange</u>	Return one or more adjacent elements from a list
<u>lrepeat</u>	Build a list by repeating elements
<u>lreplace</u>	Replace elements in a list with new elements
<u>lreverse</u>	Reverse the order of a list
<u>lsearch</u>	See if a list contains a particular element
<u>lset</u>	Change an element in a list
<u>lsort</u>	Sort the elements of a list
<u>mathfunc</u>	Mathematical functions for Tcl expressions
<u>mathop</u>	Mathematical operators as Tcl commands
<u>memory</u>	Control Tcl memory debugging capabilities
<u>msgcat</u>	Tcl message catalog
<u>my</u>	invoke any method of current object
<u>namespace</u>	create and manipulate contexts for commands and variables
<u>next</u>	invoke superclass method implementations
<u>object</u>	root class of the class hierarchy
<u>open</u>	Open a file-based or command pipeline channel
<u>package</u>	Facilities for package loading and version control
<u>packagens</u>	Construct an appropriate 'package ifneeded' command for a given package specification
<u>pid</u>	Retrieve process identifiers

coreTools Command Reference Index

<u>pkgMkIndex</u>	Build an index for automatic loading of packages
<u>platform</u>	System identification support code and utilities
<u>platform shell</u>	System identification support code and utilities
<u>prefix</u>	facilities for prefix matching
<u>proc</u>	Create a Tcl procedure
<u>puts</u>	Write to a channel
<u>pwd</u>	Return the absolute path of the current working directory
<u>read</u>	Read from a channel
<u>refchan</u>	command handler API of reflected channels
<u>regexp</u>	Match a regular expression against a string
<u>registry</u>	Manipulate the Windows registry
<u>regsub</u>	Perform substitutions based on regular expression pattern matching
<u>rename</u>	Rename or delete a command
<u>return</u>	Return from a procedure, or set return code of a script
<u>re_syntax</u>	Syntax of Tcl regular expressions
<u>safe</u>	Creating and manipulating safe interpreters
<u>scan</u>	Parse string using conversion specifiers in the style of sscanf
<u>seek</u>	Change the access position for an open channel
<u>self</u>	method call internal introspection
<u>set</u>	Read and write variables
<u>socket</u>	Open a TCP network connection
<u>split</u>	Split a string into a proper Tcl list
<u>string</u>	Manipulate strings
<u>subst</u>	Perform backslash, command, and variable substitutions
<u>switch</u>	Evaluate one of several scripts, depending on a given value
<u>tailcall</u>	Replace the current procedure with another command
<u>Tcl</u>	Tool Command Language
<u>tcltest</u>	Test harness support code and utilities
<u>tclvars</u>	Variables used by Tcl
<u>tell</u>	Return current access position for an open channel
<u>throw</u>	Generate a machine-readable error
<u>time</u>	Time the execution of a script
<u>timerate</u>	Calibrated performance measurements of script execution time
<u>tm</u>	Facilities for locating and loading of Tcl Modules
<u>trace</u>	Monitor variable accesses, command usages and command executions
<u>transchan</u>	command handler API of channel transforms

coreTools Command Reference Index

<u>try</u>	Trap and process errors and exceptions
<u>unknown</u>	Handle attempts to use non-existent commands
<u>unload</u>	Unload machine code
<u>unset</u>	Delete variables
<u>update</u>	Process pending events and idle callbacks
<u>uplevel</u>	Execute a script in a different stack frame
<u>upvar</u>	Create link to variable in a different stack frame
<u>variable</u>	create and initialize a namespace variable
<u>vwait</u>	Process events until a variable is written
<u>while</u>	Execute script repeatedly as long as a condition is met
<u>zlib</u>	compression and decompression operations

[API Index](#) [Command Reference](#)

TCLConfigVariable

Indicates a parameter to be defined as a TCL variable by `set_tclconfig_variables`.

Definition

Type: boolean

Flags:

Default value: Value calculated internally.

Valid on: `param_____`

Description

When set true, this attribute indicates that the parameter should be included in the set for which TCL variables are generated when the `set_tclconfig_variables` command. The attribute can set explicitly in which case the TCL variable name will be derived from the parameter name. If a different name is desired, the `TCLConfigVariableName` attribute should be set to indicate the name. In that case it is not required to set `TCLConfigVariable` as it defaults to true when `TCLConfigVariableName` is set.

Examples

See the `set_tclconfig_variables` man page for examples.

See Also

[TCLConfigVariableName \(3\)](#), [set_tclconfig_variables \(2\)](#)

[API Index](#) [Command Reference](#)

TCLConfigVariableName

Indicates a parameter to be defined as a TCL variable by `set_tclconfig_variables`. The attribute value defines the TCL variable name.

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

When set, this attribute indicates that the parameter should be included in the set for which TCL variables are generated when the `set_tclconfig_variables` command. This attribute is used for cases where the desired TCL variable name is not derived from the parameter name. The value of this attribute is used to derive the TCL variable name. If the name should be derived from the actual parameter name, all that is required is setting `TCLConfigVariable` (to true). In that case this variable should not be set.

Examples

See the `set_tclconfig_variables` man page for examples.

See Also

[TCLConfigVariable \(3\)](#), [set_tclconfig_variables \(2\)](#)

[API Index](#) [Command Reference](#)

TCLConfigVariableSetArgs

Defines default arguments for the `set_tclconfig_variables` command when invoked with arguments in the context of this design.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is used to define default arguments for the `set_tclconfig_variables` command. When the command is called with no arguments and this attribute is set. The value of the attribute is interpreted as the arguments to the command. This is available for the case where the command is called internal to coreAssembler (without arguments) prior to synthesis constraint processing for components. If the components expect TCL variable definition, are not set up for the default behavior, then this attribute can be used to indicate the desired output behavior.

Examples

Set up constraint reading flow so that TCL configuration variables are defined in the global array `::ConfigParam`. Note that the space at the beginning of the attribute value is needed to prevent the TCL interpreter from treating the attribute value as a 'dash option' to the `set_design_attribute` command.

```
set_design_attribute TCLConfigVariableSetArgs { -array ::ConfigParams }  
.
```

See Also

[set_tclconfig_variables \(2\)](#)_____

TMAX_Reference_Methodology

This flow implements the TestMAX Reference Methodology. For details refer to the TMAX Reference Methodology.

Parameters

Parameters Table

Name	Label	Description
Strategy Controls		
MaxTbConfigFile	TMAX Testbench Config	TestMAX testbench options file.
OptimizePatterns	Optimize Patterns	Set to TRUE for the final ATPG run to optimize pattern count at the expense of runtime. This optimization applies to the stuck-at and transition delay fault models; other fault models may not get better results so they are not supported in the RM.
PersistentFaultModel	Persistent Fault Model	Enables persistent fault modeling. By default, this is set to TRUE. It helps to manage multiple fault model flows easily. See the 'TestMAX ATPG User Guide' for details on the persistent fault model.
TMAXDebugMode	TMAX Debug Mode	Set to TRUE to print a more complete log file and additional reports for debugging.
IntegratePathDelayPatterns	Integrate Path Delay Patterns	If path delay patterns will be part of the product quality screen (TRUE), include them with other tests and fault grade them to reduce pattern count. If they are used only for characterization or speed binning (FALSE), keep them as a separate test.
RulesToDownGrade	Rules to Downgrade	Errors to downgrade without masking.
RulesToMask	Rules to Mask	Errors to mask.
BlackBoxModules	Black Box	Modules to replace by black

coreTools Command Reference Index

Name	Label	Description
	Modules	boxes.
BlackBoxInstances	Black Box Instances	Instances to replace by TIEX's.
ExceptionFiles	Exception Files	SDC files for timing exceptions. Path must be relative to the synthesis directory. The default is the output from static timing analysis when the LinkToATPG option was set to TRUE in the static timing analysis activity.
DelayPathFile	Delay Path File	Path file for path delay testing. The default is the output from static timing analysis when the LinkToATPG option was set to TRUE in the static timing analysis activity.
AddPiConstraints	Add Pi Constraints	Adds constraints to the listed primary inputs to disable during testing. The parameter value is a list of pairs of {[port] [logic value]}. For example set_strategy_parameter TMAX_Reference_Methodology {test_se 0 rst_n 1}
GlobalSlackFile	Global Slack File	Slack File for Slack-Based Transition Fault Testing. The default is the output from static timing analysis when the LinkToATPG option was set to TRUE in the static timing analysis activity.
UseScanCompressionProtocol	Use the scan compression Protocol	When TRUE, uses the scan compression configuration spf file.
AddNoFaultInstances	Add NoFault Instances	String list of instances added to the add_nofault commands.
TestSelection		
StuckAtTest	Stuck At Test	When TRUE, stuck-at fault testing will be performed.
TransitionDelayTest	Transition Delay Test	When TRUE, transition delay testing will be performed.
TransitionDelayCoverageGoal	Transition Delay Coverage Goal	When TransitionDelayTest is true, the parameter defines the transition delay coverage

coreTools Command Reference Index

Name	Label	Description
		goal.
AtSpeedTest	At Speed Test	When TRUE, at speed testing will be performed.
AtSpeedScript	At Speed SPF waveform update script	Script to run in order to update the SPF waveforms for at-speed testing.
PathDelayTest	Path Delay Test	When TRUE, path delay testing will be performed
RandomIncrementalAtpg	Random Incremental Atpg	When TRUE, causes random, incremental ATPG to be run for all enabled tests. The number of incremental runs for each test is controlled by the parameter IncrementalRuns.
IncrementalRuns	IncrementalRuns Runs	When RandomIncrementalAtpg is TRUE, this parameter determines the number of incremental runs for each enabled test.

See Also

TMMRMVariable

The attribute is used for storing the updated values for the Test Max Manager Reference Methodology variables, which will then be updated in the Reference Methodology scripts if the strategy parameter

EnableTestMaxManagerFlow is set to 1

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

TMRIntentFile

Specifies a TMR intent script to be executed post elaboration.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: dc_shell fc_shell

Default subscript:

Valid on: design_____

Description

Examples

See Also

TMRIntentFileComment

Specifies a comment to be issued for the corresponding TMR Intent file exported in the tool intent.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: dc_shell fc_shell

Default subscript:

Valid on: design_____

Description

Examples

See Also

TargetConnectionBoundary

Indicates that this is a TargetConnection boundry point for exported workspace components.

Definition

Type: boolean

Flags:

Default value: Value calculated internally.

Valid on:

Description

This modifies what TargetConnection returns by stopping at any interface that has TargetConnectionBoundary set to true on it.

Examples

See Also

NAME

Tcl - Tool Command Language

SYNOPSIS

Summary of Tcl language syntax.

DESCRIPTION

The following rules define the syntax and semantics of the Tcl language:

- [1] **Commands.** A Tcl script is a string containing one or more commands. Semi-colons and newlines are command separators unless quoted as described below. Close brackets are command terminators during command substitution (see below) unless quoted.
- [2] **Evaluation.** A command is evaluated in two steps. First, the Tcl interpreter breaks the command into *words* and performs substitutions as described below. These substitutions are performed in the same way for all commands. Secondly, the first word is used to locate a command procedure to carry out the command, then all of the words of the command are passed to the command procedure. The command procedure is free to interpret each of its words in any way it likes, such as an integer, variable name, list, or Tcl script. Different commands interpret their words differently.

coreTools Command Reference Index

- [3] **Words.** Words of a command are separated by white space (except for newlines, which are command separators).
- [4] **Double quotes.** If the first character of a word is double-quote then the word is terminated by the next double-quote character. If semi-colons, close brackets, or white space characters (including newlines) appear between the quotes then they are treated as ordinary characters and included in the word. Command substitution, variable substitution, and backslash substitution are performed on the characters between the quotes as described below. The double-quotes are not retained as part of the word.
- [5] **Argument expansion.** If a word starts with the string followed by a non-whitespace character, then the leading is removed and the rest of the word is parsed and substituted as any other word. After substitution, the word is parsed as a list (without command or variable substitutions; backslash substitutions are performed as is normal for a list and individual internal words may be surrounded by either braces or double-quote characters), and its words are added to the command being substituted. For instance, is equivalent to
- [6] **Braces.** If the first character of a word is an open brace and rule [5] does not apply, then the word is terminated by the matching close brace Braces nest within the word: for each additional open brace there must be an additional close brace (however, if an open brace or close brace within the word is quoted with a backslash then it is not counted in locating the matching close brace). No substitutions are performed on the characters between the braces except for backslash-newline substitutions described below, nor do semi-colons, newlines, close brackets, or white space receive any special interpretation. The word will consist of exactly the characters between the outer braces, not including the braces themselves.
- [7] **Command substitution.** If a word contains an open bracket then Tcl performs *command substitution*. To do this it invokes the Tcl interpreter recursively to process the characters following the open bracket as a Tcl script. The script may contain any

coreTools Command Reference Index

number of commands and must be terminated by a close bracket. The result of the script (i.e. the result of its last command) is substituted into the word in place of the brackets and all of the characters between them. There may be any number of command substitutions in a single word. Command substitution is not performed on words enclosed in braces.

[8] Variable substitution.

If a word contains a dollar-sign followed by one of the forms described below, then Tcl performs *variable substitution*: the dollar-sign and the following characters are replaced in the word by the value of a variable. Variable substitution may take any of the following forms:

<code><i>\$name</i></code>	<i>Name</i> is the name of a scalar variable; the name is a sequence of one or more characters that are a letter, digit, underscore, or namespace separators (two or more colons). Letters and digits are <i>only</i> the standard ASCII ones (0-9 , A-Z and a-z).
<code><i>\$name(index)</i></code>	<i>Name</i> gives the name of an array variable and <i>index</i> gives the name of an element within that array. <i>Name</i> must contain only letters, digits, underscores, and namespace separators, and may be an empty string. Letters and digits are only the standard ASCII ones (0-9 , A-Z and a-z). Command substitutions, variable substitutions, and backslash substitutions are performed on the characters of <i>index</i> .
<code><i>\${name}</i></code>	<i>Name</i> is the name of a scalar variable or array element. It may contain any characters whatsoever except for close braces. It indicates an array element if <i>name</i> is in the form where <i>arrayName</i> does not contain any open parenthesis characters,

coreTools Command Reference Index

or close brace characters, and index can be any sequence of characters except for close brace characters. No further substitutions are performed during the parsing of name.

There may be any number of variable substitutions in a single word. Variable substitution is not performed on words enclosed in braces.

*Note that variables may contain character sequences other than those listed above, but in that case other mechanisms must be used to access them (e.g., via the **set** command's single-argument form).*

[9] **Backslash substitution.**

If a backslash appears within a word then backslash substitution occurs. In all cases but those described below the backslash is dropped and the following character is treated as an ordinary character and included in the word. This allows characters such as double quotes, close brackets, and dollar signs to be included in words without triggering special processing. The following table lists the backslash sequences that are handled specially, along with the value that replaces each sequence.

\a	<i>Audible alert (bell) (Unicode U+000007).</i>
\b	<i>Backspace (Unicode U+000008).</i>
\f	<i>Form feed (Unicode U+00000C).</i>
\n	<i>Newline (Unicode U+00000A).</i>
\r	<i>Carriage-return (Unicode U+00000D).</i>
\t	<i>Tab (Unicode U+000009).</i>
\v	<i>Vertical tab (Unicode U+00000B).</i>

\<newline>*whiteSpace*

A single space character replaces the backslash, newline, and all spaces and tabs after the newline. This backslash sequence is unique in that it is replaced in a separate pre-pass before the command is actually parsed. This means that it will be replaced

coreTools Command Reference Index

even when it occurs between braces, and the resulting space will be treated as a word separator if it is not in braces or quotes.

`\\` Backslash

`\ooo` The digits *ooo* (one, two, or three of them) give a eight-bit octal value for the Unicode character that will be inserted, in the range 000-377 (i.e., the range U+000000-U+0000FF). The parser will stop just before this range overflows, or when the maximum of three digits is reached. The upper bits of the Unicode character will be 0.

`\xhh` The hexadecimal digits *hh* (one or two of them) give an eight-bit hexadecimal value for the Unicode character that will be inserted. The upper bits of the Unicode character will be 0 (i.e., the character will be in the range U+000000-U+0000FF).

`\uhhhh` The hexadecimal digits *hhhh* (one, two, three, or four of them) give a sixteen-bit hexadecimal value for the Unicode character that will be inserted. The upper bits of the Unicode character will be 0 (i.e., the character will be in the range U+000000-U+00FFFF).

`\Uhhhhhhhh`
The hexadecimal digits *hhhhhhhh* (one up to eight of them) give a twenty-one-bit hexadecimal value for the Unicode character that will be inserted, in the range U+000000-U+10FFFF. The parser will stop just before this range overflows, or when the maximum of eight digits is reached. The upper bits of the Unicode character will be 0.

The range U+010000-U+10FFFD is reserved for the future.

Backslash substitution is not performed on words enclosed in braces, except for backslash-newline as described above.

[10] Comments. *If a hash character appears at a point where Tcl is expecting the first character of the first word of a command, then the hash character and the*

coreTools Command Reference Index

characters that follow it, up through the next newline, are treated as a comment and ignored. The comment character only has significance when it appears at the beginning of a command.

[11] Order of substitution.

Each character is processed exactly once by the Tcl interpreter as part of creating the words of a command. For example, if variable substitution occurs then no further substitutions are performed on the value of the variable; the value is inserted into the word verbatim. If command substitution occurs then the nested command is processed entirely by the recursive call to the Tcl interpreter; no substitutions are performed before making the recursive call and no additional substitutions are performed on the result of the nested script.

Substitutions take place from left to right, and each substitution is evaluated completely before attempting to evaluate the next. Thus, a sequence like

```
set y [set x 0][incr x][incr x]
```

will always set the variable y to the value, 012.

[12] Substitution and word boundaries.

Substitutions do not affect the word boundaries of a command, except for argument expansion as specified in rule [5]. For example, during variable substitution the entire value of the variable becomes part of a single word, even if the variable's value contains spaces.

KEYWORDS

backslash, command, comment, script, substitution, variable

TclAuxSynthesisScript

Specifies a DC Tcl script to be executed prior to specific optimization phase.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: cleanup constrain dcrmBeforeWrite dcrmInitial dcrmPathGroupScript dftBistReady dftCleanup dftInsertion elab incr1 incr2 incrStrategy initial postelab qmap report setup

Default subscript:

Valid on: design_____

Description

The TclAuxSynthesisScript attribute specifies the name of a Design Compiler (DC) script that you want to execute before a particular phase of synthesis for the selected design. As an example, you may want to perform special structuring or grouping on a design before a certain synthesis phase to improve timing or area.

For the core developer, the script must be technology-independent; for the core integrator, the script can be technology-specific.

The subscript indicates the phase before which you want to execute the auxiliary synthesis script:

- setup - Execute script before any other processing.
- elab - Execute script before elaboration.
- postelab - Execute script after elaboration.
- constrain - Execute script after setting constraints.
- dcrmInitial - Execute script before initial compile. This script applies to the DC Reference Methodology script (DCTCL_one_pass_compile_utra) only.
- dcrmBeforeWrite - Execute script before final design/report generation
- cleanup - Execute script after all processing is complete, just before quitting.
- dftInsertion Execute script during dft insertion after set_*_configuration commands have been executed. This script will always be executed when dft insertion is chosen. For BistMode==all and BistType==dbist the script is executed after the design has been wrapped and made bist ready.
- dftCleanup Execute script during dft insertion after all processing is complete, just before quitting.

Within the DC script, you can use the value of a coreConsultant design attribute to control the flow of the script. For example, you can execute different sets of DC commands depending on the value of OptimizationPriorities, as shown in the following example:

```
set optprio [get_attribute [current_design] OptimizationPriorities]
if {[string equal $optprio "Area"]} {
<area priority commands>
} else {
<other commands>
}
```

Examples

To execute auxiliary script My_DC_script.tcl before the initial compile phase of the current_design:

```
coreConsultant> set_design_attribute
{TclAuxSynthesisScript[initial]}
My_DC_script.tcl
```

See Also

[API Index](#) [Command Reference](#)

TclAuxSynthesisScriptComment

Specifies a comment to be issued for the corresponding

DcTclAuxSynthesisScript.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: cleanup constrain dcrmBeforeWrite dcrmInitial
dcrmPathGroupScript dftBistReady dftCleanup dftInsertion elab incr1
incr2 incrStrategy initial postelab qmap report setup

Default subscript:

Valid on: design_____

Description

The TclAuxSynthesisScriptComment attribute can be used to output a comment to document why the corresponding TclAuxSynthesisScript is being used. The comment will be put into the file where the script is sourced.

Examples

Specify an aux script to be used in the setup stage, and document why it is being used.

```
set_design_attribute {TclAuxSynthesisScript[setup]} myAuxScr.tcl
set_design_attribute {TclAuxSynthesisScriptComment[setup]} "This script
is used to set up custom variables for the synthesis run."
```

See Also

[TclAuxSynthesisScript \(3\)](#)_____

[API Index](#) [Command Reference](#)

TclPhysicalConstraintFile

Tcl format physical constraints to be placed on the design during synthesis.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

TechCellMappingBind

Used to store technology cell mapping bindings from
specify_tech_cell_mapping.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This attribute is used to store binding information for technology cells.

Examples

N/A This attribute should not be set by a user but via the
bind_tech_cell_mapping command.

See Also

[bind_tech_cell_mapping \(2\)](#) [create_custom_tech_cell \(2\)](#)

[API Index](#) [Command Reference](#)

TechCellMappingDefn

Used to store technology cell mapping definitions from
define_tech_cell_mapping.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This attribute is used to store definitions for technology cells.

Examples

N/A This attribute should not be set by a user but via the
define_tech_cell_mapping command.

See Also

[define_tech_cell_mapping \(2\)](#) [create_custom_tech_cell \(2\)](#)

Specify Reference Technology

This activity gets variable settings from DesignCompiler, and extracts technology specific information from the specified technology libraries.

Parameters

Parameters Table

Name	Label	Description
ASIC		
SearchPath	Search Path	Value of search_path variable in Design Compiler and on Fusion platform.
LibraryCacheSearchPath	Library Kb Cache Search Directories	Search paths for library kbs cache.
TargetLibrary	Target Library	Value of Design Compiler target_library variable. Value of Fusion Compiler TECH_LIB variable
LinkLibrary	Link Library	Value of Design Compiler link_library variable. Value of Fusion Compiler LINK_LIBRARY variable.
MinLibrary	Min timing library	Indicate Minimum Timing Libraries utilized by set_min_library. It is specified either as a single best-case library, or as a list of pairs of worst-case and best-case libraries. The worst-case library must be in the target or link library list and the best-case library must NOT be in the target or link library list.

See Also

Technology Cell Binding

This activity implements the technology cell mapping functionality.

Parameters

Parameters Table

Name	Label	Description
TechnologyCellBinding		
UniquifyEnableMacros	Uniquify Tech Cell Enable Macros	Add a design prefix and IP name to the ENABLE_Tech_CELL_MAPPING macro while writing out BCM macro definitions inserted RTL. For coreConsultant, the synthesis scripts for reading in the RTL use the modified macro. In coreAssembler, the top level read_design.tcl uses the modified macro for components except those synthesized as sub-blocks. For sub-block synthesis, the operation is similar to coreConsultant.
SkipComponentsList	Skip Components List	Technology cell bindings will not be inserted for components specified here. This parameter expects a string list of full instance paths to components.

See Also

[Specify Configuration.](#)

[Specify Reference Technology.](#)

[API Index](#) [Command Reference](#)

TestClock

This clock is used in test mode.

Definition

Type: boolean

Flags:

Default value: =sCstr::testClockDefaultValue %item

Valid on: clock_____

Description

The TestClock attribute specifies whether the selected clock is used for test. When TestClock is TRUE, a create_test_clock command will be issued for the clock in DFT Compiler.

Examples

To specify that the clock named tclk is used for test purposes:

```
coreConsultant> set_clock_attribute tclk TestClock true
```

See Also

[set_clock_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

TestClockCycleTime

Cycle time for clock in test mode.

Definition

Type: float

Flags:

Default value: =get_design_attribute TestClockDefaultCycleTime

Valid on: clock_____

Description

The cycle time to be used for a clock in test mode.

Examples

Set the cycle time for clock, clk, to 100 for test modes.

```
coreBuilder> set_clock_attribute clk TestClockCycleTime 100
```

See Also

[set_clock_attribute \(2\)](#), [TestClock \(3\)](#), [TestClockWaveform \(3\)](#)

[API Index](#) [Command Reference](#)

TestClockDefaultCycleTime

Default cycle time for test clocks.

Definition

Type: float

Flags:

Default value: 100

Valid on: design_____

Description

Specifies the default cycle time for test clocks for the design.

Examples

Specify that the default cycle time for clocks in the current_design is 100.

```
coreBuilder> set_design_attribute TestClockDefaultCycleTime 100
```

See Also

[set_design_attribute \(2\)](#), [TestClock \(3\)](#), [DftExistingSignalType \(3\)](#)

[API Index](#) [Command Reference](#)

TestClockWaveform

Waveform for clock in test mode.

Definition

Type: string

Flags:

Default value: =sDft::defaultTcWaveform %item

Valid on: clock_____

Description

The waveform to use for a clock in test modes.

Examples

Specify that the waveform for the clock, clk, is a rising edge at 45 and a falling edge at 55.

```
coreBuilder> set_clock_attribute clk TestClockWaveform [list 45 55]
```

See Also

[set_clock_attribute \(2\)](#), [TestClock \(3\)](#), [TestClockCycleTime \(3\)](#)

[API Index](#) [Command Reference](#)

TestClocksFollowSystemDomains

When true, dedicated test clocks are created according to different system clocks. Thus, scan cells that have different system clocks have different test clocks. When false, the default, test clocks are created without considering system clocks.

Definition

Type: boolean

Flags:

Default value: =InheritValue up FALSE

Valid on: design_____

Description

When true, dedicated test clocks (for example, test_scan_clock_a) are created according to different system clocks. Thus, scan cells that have different system clocks have different test clocks. When false (the default), test clocks are created without considering system clocks. This attribute corresponds to the -create_test_clocks_by_system_clock_domain switch to the set_scan_configuration command in DFT Compiler.

Examples

Specify that test clocks are to be created without regard to the system clock domains.

```
coreBuilder> set_design_attribute TestClocksFollowSystemDomains 0
```

See Also

[set_design_attribute \(2\) ScanBlockIndividually \(3\)](#)_____

[API Index](#) [Command Reference](#)

TestIsolate

Causes set_test_isolate to be issued for a port. This functionality is not supported in XG mode.

Definition

Type: boolean

Flags:

Default value: =InheritValue up false

Valid on: port _____

Description

Causes set_test_isolate to be issued for a port. This functionality is not supported in XG mode. Use the design attribute ScanExclude remove other design elements from DFT Compiler control.

Examples

Specify that the port dontTest is to not be used for DFT in Design Compiler db mode.

```
coreBuilder> set_port_attribute dontTest TestIsolate true
```

See Also

[set_port_attribute \(2\)](#), [ScanExclude \(3\)](#), _____

[API Index](#) [Command Reference](#)

TestabilityClockSignal

Specifies the clock to be used for control or observe registers.

Definition

Type: string

Flags:

Default value: =InheritValue up "\<NONE\>"

Valid on: design_____

Description

Specifies the clock to be used for control or observe registers. The port must have been previously defined as a test clock.

Examples

Create a test clock on the port tp_clk, and specify that it be used for test points.

```
coreBuilder> set_port_attribute tp_clk DftExistingSignalType ScanClock
coreBuilder> set_design_attribute TestabilityClockSignal tp_clk
```

See Also

[set_port_attribute \(2\), DftExistingSignalType \(3\),](#)
[DftExistingSignalActiveState \(3\)](#)

[API Index](#) [Command Reference](#)

TestabilityClockType

Specifies the clock type to be used for testability enhancements.

Definition

Type: string

Flags:

Default value: dominant

Valid on: design_____

Description

Specifies the clock type to be used when inserting control and observe point scan cells.

Examples

Set the TestabilityClockType to dedicated.

```
coreBuilder> set_design_attribute TestabilityClockType dedicated
```

See Also

[set_design_attribute \(2\)](#), [InsertTestPoints \(3\)](#)_____

TestabilityControlSignal

Selects the control signal to be used for test points. The signal should be predefined as having the TestMode signal type. If no control signal is specified, the test points will be controlled by any signal with the TestMode signal type. If no TestMode signal is found, DFT Compiler will create a new TestMode port.

Definition

Type: string

Flags:

Default value: =InheritValue up "\<NONE\>"

Valid on: design_____

Description

Selects the control signal to be used for test points. The signal should be predefined as having the TestMode signal type. If no control signal is specified, the test points will be controlled by any signal with the TestMode signal type. If no TestMode signal is found, DFT Compiler will create a new TestMode port.

Examples

Specify that the signal tpControl is to be used for controlling test points, and that it is active high.

```
set_port_attribute tpControl DftSpecSignalType TestMode
set_port_attribute tpControl DftSpecSignalActiveState 1
set_design_attribute TestabilityControlSignal tpControl
```

See Also

[set_design_attribute \(2\)](#), [set_port_attribute \(2\)](#), [DftSpecSignalType \(3\)](#), [InsertTestPoints \(3\)](#), [ScanBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

TestabilityMaxArea

Specifies the maximum percentage of area overhead due to the observe test point structure insertion in the design. Setting this attribute to 0 leaves this as unspecified in DFT Compiler.

Definition

Type: short

Flags:

Default value: 0

Valid on: design_____

Description

Specifies the maximum percentage of area overhead due to the observe test point structure insertion in the design. Setting this attribute to 0 leaves this as unspecified in DFT Compiler.

Examples

Specify a max added area overhead of 25% for test point logic.

```
coreBuilder> set_design_attribute TestabilityMaxArea 25
```

Specify that DFT compiler will not be constrained by an area overhead.

```
coreBuilder> set_design_attribute TestabilityMaxArea 0
```

See Also

[set_design_attribute \(2\)](#), [InsertTestPoints \(3\)](#), [ScanBlockIndividually \(3\)](#)_____

[API Index](#) [Command Reference](#)

TestabilityMethod

Testability method to be used. Note that the tdvr and observe methods are identical. The tdvr setting is for backward compatibility.

Definition

Type: string

Flags:

Default value: control_and_observe

Valid on: design_____

Description

Determines whether bist or tdvr method will be used for inserting control and observe points in the design.

Examples

Set TestabilityMethod to tdvr.

```
coreBuilder> set_design_attribute TestabilityMethod tdvr
```

See Also

[set_design_attribute \(2\)](#), [InsertTestPoints \(3\)](#)

Testable

Indicates that the given field is testable in a simple register test, provided the specified constraints are met.

Definition**Type:** string**Flags:****Default value:****Valid on:****Description**

The testable attribute indicates if the field can be tested with automated register routine. The presumed value of testable is true if not specified.

If the testable attribute is present, the following is the IP-XACT description

- untestable - This maps to IP-XACT testable as set to false. The field will skip
- unconstrained - Read and write all legal values
- restore - Read and write legal values but the value must be restored to the
- writeAsRead - Value read before a write may be written to the field
- readOnly - Values may only be read from the field

VMM/UVM tests mapping**VMM RAL tests Support**

The RAL tests are suppressed for individual or complete Register Based on the VCS versions given in table below

UVM RAL tests support

The RAL tests are suppressed for the entire register based on the table given below. Currently no VCS version supports field level suppression in UVM

Test Support	untestable	unconstrained	restore writeAsRead readOnly
VMM Tests suppressed at field level VCS 2012.09 onwards	hw_reset bit_bash	N/A	bit_bash
VMM Tests suppressed at register level Prior to VCS 2012.09	hw_reset bit_bash	N/A	bit_bash
UVM Tests suppressed at register level	hw_reset bit_bash	N/A	bit_bash

Test Support	untestable	unconstrained	restore writeAsRead readOnly
for any VCS version			

Examples

```
coreBuilder> # Reg3 - Testable with unconstrained
create_register reg3 -block map/block -offset 0x2 -size 8
create_register_field reg3_f1 -register map/block/reg3 -offset 0x0 \
-size 2
set_register_field_attribute map/block/reg3/reg3_f1 Testable
unconstrained
```

See Also

[ReadAction](#) [WriteBehavior](#) [MinWriteConstraint](#) [MaxWriteConstraint](#)

[API Index](#) [Command Reference](#)

TestpointPowerSavingOn

Adds logic structure (and gate) for the power saving on the XOR tree

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Adds logic structure (and gate) for the power saving on the XOR tree.

This option is ignored when the TestabilityMethod is not tdvr.

Examples

Specify that logic is to be added for power saving when inserting control and observe points.

```
coreBuilder> set_design_attribute TestpointPowerSavingOn 1
```

See Also

[set_design_attribute \(2\)](#), [InsertTestPoints \(3\)](#), [TestabilityMethod \(3\)](#)

Testsuite Set-up

Specify testsuite filegroups and parameters.

Parameters

Activity has no parameters.

See Also

[API Index](#) [Command Reference](#)

TimingExceptionCmd

Command associated with this timing exception

Definition

Type: string

Flags:

Default value:

Valid on: [timingException](#)

Description

Internal attribute used to store information about a particular timing exception. This attribute can not be set directly. Timing exceptions should be specified using the GUI or via a file of SDC

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionFall_fromValue

List of path startpoints. The path must fall from objects specified

Definition

Type: string

Flags: readOnly

Default value: =sTe::getOptionValueAttr %item Fall_from

Valid on: [timingException](#)

Description

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionFall_throughValue

List of path through points. Applied to paths with a falling transition at specified objects

Definition

Type: string

Flags: readOnly

Default value: =sTe::getOptionValueAttr %item Fall_through

Valid on: timingException

Description

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionFall_toValue

List of path endpoints. Applies to paths falling at the endpoint

Definition

Type: string

Flags: readOnly

Default value: =sTe::getOptionValueAttr %item Fall_to

Valid on: [timingException](#)

Description

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionFromValue
List of path startpoints

Definition

Type: string

Flags: readOnly

Default value: =sTe::getOptionValueAttr %item From

Valid on: timingException

Description

Internal attribute used to store information about a particular timing exception. This attribute can not be set directly. Timing exceptions should be specified using the GUI or via a file of SDC commands.

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionGroup_path
Name of group for paths

Definition

Type: string

Flags:

Default value:

Valid on: [timingException](#)

Description

Internal attribute used to store information about a particular timing exception. This attribute can not be set directly. Timing exceptions should be specified using the GUI or via a file of SDC commands.

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionOptions

Options associated with this timing exception

Definition

Type: string

Flags:

Default value:

Valid on: [timingException](#)

Description

Internal attribute used to store information about a particular timing exception. This attribute can not be set directly. Timing exceptions should be specified using the GUI or via a file of SDC commands.

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionRise_fromValue

List of path startpoints. The path must rise from objects specified

Definition

Type: string

Flags: readOnly

Default value: =sTe::getOptionValueAttr %item Rise_from

Valid on: [timingException](#)

Description

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionRise_throughValue

List of path through points. Applies to paths with a rising transition at specified objects

Definition

Type: string

Flags: readOnly

Default value: =sTe::getOptionValueAttr %item Rise_through

Valid on: timingException

Description

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionRise_toValue

List of path endpoints. Applies to paths rising at the endpoint

Definition

Type: string

Flags: readOnly

Default value: =sTe::getOptionValueAttr %item Rise_to

Valid on: [timingException](#)

Description

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionThroughValue

List of path through points

Definition

Type: string

Flags: readOnly

Default value: =sTe::getOptionValueAttr %item Through

Valid on: timingException

Description

Internal attribute used to store information about a particular timing exception. This attribute can not be set directly. Timing exceptions should be specified using the GUI or via a file of SDC commands.

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionToValue
List of path endpoints

Definition

Type: string

Flags: readOnly

Default value: =sTe::getOptionValueAttr %item To

Valid on: [timingException](#)

Description

Internal attribute used to store information about a particular timing exception. This attribute can not be set directly. Timing exceptions should be specified using the GUI or via a file of SDC commands.

Examples

See Also

[API Index](#) [Command Reference](#)

TimingExceptionValue

Value for timing exception command

Definition

Type: string

Flags: readOnly subscribed

Default value: =sTe::getValueAttr %item -return %attrSubscript

Valid subscripts: eval no_eval

Default subscript: eval

Valid on: timingException

Description

Internal attribute used to store information about a particular timing exception. This attribute can not be set directly. Timing exceptions should be specified using the GUI or via a file of SDC commands.

Examples

See Also

TmmAuxScript

Specifies a TestMaxManager script to be executed prior to specific phase.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: output post_read_rtl pre_read_rtl report

Default subscript:

Valid on: design_____

Description

The TmmAuxScript attribute specifies the name of a TestMaxManager (TMM) script that you want to execute before or after a particular stage of the TMM run for the selected design.

The subscript indicates the stage in the TMM flow where you want to execute the auxiliary synthesis script:

- pre_read_rtl - Execute script before reading RTL.
- report - Execute script at the end of report generation stage, the script can be used to include report generation commands.
- output - Execute script after all the output views have been created, just before quitting.

Examples

See Also

TmmAuxScriptComment

Specifies a comment to be issued for the corresponding TmmAuxScript.

Definition

Type: string

Flags: subscribed

Default value:

Valid subscripts: output post_read_rtl pre_read_rtl report

Default subscript:

Valid on: design_____

Description

The TmmAuxScriptComment attribute can be used to output a comment to document why the corresponding TmmAuxScriptComment is being used. The comment will be put into the file where the script is sourced.

Examples

See Also

TypeName
The type of an item

Definition

Type: string

Flags: readOnly

Default value:

Valid on: [Strategy](#) [attrDefn](#) [busBit](#) [cell](#) [clock](#) [design](#) [file](#) [filegroup](#) [filegroupGroup](#) [hdlFunc](#) [knowledgeBase](#) [net](#) [param](#) [pin](#) [port](#) [timingException](#)

Description

TypeName is a read-only attribute that indicates the type of the specified item. For example, if you create a virtual clock with the `create_virtual_clock` command, the coreTool automatically sets the clock's TypeName attribute to "clock".

The value of the TypeName attribute is one of the coreTool item types listed in the `Item_Types` index.

Examples

To determine the item type name of the item named serial:

```
coreConsultant> get_attribute -attrs TypeName serial
param
```

See Also

[get_attribute \(2\)](#)

TypedefType

Generic class of SV typedef for the item: type, enum, struct, union.

Definition

Type: string

Flags:

Default value: =get_typedef_type %item

Valid on: busBit pin port _____

Description

Contains the generic class of an item that has a type defined by a SystemVerilog typedef. Values are type, enum, struct, union, or an empty value if the type is not from a typedef.

Examples

Port *in1* will have an empty value for TypedefType:

```
input logic in1;
```

The TypedefType value for port *in1_s* below is *struct*:

```
typedef struct { logic f1; } struct1_t;
input struct1_t in1_s;
```

See Also

[API Index](#) [Command Reference](#)

UPFFile

Specifies a UPF file to load with load_upf in Design Compiler.

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

UPFFile specifies the name of a Unified Power Format (UPF) file for synthesis and formal verification. For the DCTCL_opto_strategy synthesis strategy, the UPFFile attribute must be specified on each design to be compiled. For the DCTCL_one_pass_compile_ultra synthesis strategy, only the UPFFile attribute for the top-level design is used.

Note that the UPFFile attribute is ignored unless the parameter EnableUPFFlow is set on the synthesis strategy. Also, the UPF file specified in the synthesis strategy will override the UPFFile attribute for the top-level design (see the Power tab on the synthesis strategy Options dialog).

Examples

```
coreConsultant> set_design_attribute UPFFile /full/path/des.upf
```

See Also

[set design attribute \(2\)](#)

UVMRALAccessType

This is a read-only attribute. This stores UVM Ral Attributes based on MemoryAccess ReadAction, WriteBehaviour. Please refer to UVM 1.1 users guide section 5.5.1.4. Note : As per IP-Xact mapping table, the User-defined fields will be se as rw as UVM RALF format does not support any appropriate field type

Definition

Type: string

Flags:

Default value: =sMem::loadUVMRALAccessType %item

Valid on:

Description

This attribute indicates the UVMRALAccess type and the standard access types in UVM mode are

ro rw rc rs wrc wrs wc ws wsrc wcrs w1c w1s w1t w0c w0s w0t w1src w1crs w0src w0crs wo woc wos w1 wo1

This is mapped with MemoryAccess , ReadAction and WriteBehaviour attribute and once can refer to section 5.5.1.4 in UVM 1.1 users guide.

Examples

See Also

[RALAccessType](#)

UVMTestText

This feature provides the user ability to add user specific steps in the UVM Test phases of RAL tests in the UVM RAL Subsystem flow. The user can configure, drive and control signals in any core by using this feature.

Not all phases of the RAL test can be modified, the most important ones are available for modification.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: build_phase build_phase-bit_bash
 build_phase-hw_reset build_phase-reg_access configure_phase
 configure_phase-bit_bash configure_phase-hw_reset
 configure_phase-reg_access main_phase main_phase-bit_bash
 main_phase-hw_reset main_phase-reg_access reset_phase
 reset_phase-bit_bash reset_phase-hw_reset reset_phase-reg_access
 shutdown_phase shutdown_phase-bit_bash shutdown_phase-hw_reset
 shutdown_phase-reg_access start_of_simulation_phase
 start_of_simulation_phase-bit_bash
 start_of_simulation_phase-hw_reset
 start_of_simulation_phase-reg_access

Default subscript:

Valid on: design_____

Description

The UVMTestText is a design attribute and it can be used for any IPCore which needs text

%I%% Substitution

The %I%% substitution will help the tool to replace the top level instance name of IP core with the internal hierarchical path

Example

```
eval_in_component i_rap {
    set_design_attribute UVMTestText\[reset_phase-hw_reset\] {repeat (10) @ (posedge %I%.)}
}
```

OUTPUT : hw_reset_test.sv in reset_phase

```
....
repeat (10) @ (posedge simulation_top.tb_Subsystem.DUT.i_rap.i_rap_clk_gen.clk);
...
```

Subscripts Table : Mapping of supported UVM Phases and RAL tests

phases/Subscript	hw_reset	bit_bash
Initial Phase		
build_phase	build_phase-hw_reset	build_phase-bit_bash
start_of_simulation_phase	start_of_simulation_phase-hw_reset	start_of_simulation_phase-bit_bash
Runtime Phase		
reset_phase	reset_phase-hw_reset	reset_phase-bit_bash
configure_phase	configure_phase-hw_reset	configure_phase-bit_bash
main_phase	main_phase-hw_reset	main_phase-bit_bash
shutdown_phase	shutdown_phase-hw_reset	shutdown_phase-bit_bash

Note:

If a subscript does not have test associated , for eg UVMTestText\[build_phase\] and not UVMTestText\[build_phase-hw_reset\] then the code inserted in the build_phase only subscript will get written in all , hw_reset, bit_bash and reg_access tests

Examples

```
eval_in_component i_rap {
set_design_attribute UVMTestText\[reset_phase-hw_reset\] {repeat (10) @
(posedge %%I%%.i_rap_clk_gen.clk);}
}
eval_in_component i_usb3 {
set_design_attribute UVMTestText\[build_phase\] {
%%I%%.<internal_inst>.reg = 3'h2;}
set_design_attribute UVMTestText\[configure_phase\] {
fork
begin
$display("I am in first thread")
end
begin
%%I%%.<internal_inst>.usb_reg = 1'b0;
end
join_none
}
}
```

See Also

[API Index](#) [Command Reference](#)

UnconnectedPort

Indicates that an output port remains unconnected.

Definition

Type: boolean

Flags:

Default value:

Valid on: busBit port _____

Description

This attribute is used to indicate that the given output port is unconnected in the context in which it is used. This information can be used by Design Compiler to strip away unneeded logic. This can provide a large savings in area, but should not be used unless the output is truly unconnected, since logic driving the output is permanently removed.

Examples

```
coreBuilder> set_port_attribute unused_output UnconnectedPort true
```

See Also

[ConstantPort \(3\)](#)

[API Index](#) [Command Reference](#)

UndefinedBitValue

Indicates the default reset value for undefined bits in a register.

Definition

Type: string

Flags:

Default value: =InheritValue up {}

Valid on:

Description

When specified, this value will be used as the reset value for undefined bits within a register (bits not within a defined field). This applies to commands which infer fields for undefined bits such as `write_ral_file`.

See Also

[RegisterResetValue \(3\)](#), [write_ral_file \(2\)](#)

Underconstrain

Percentage by which I/O delay constraints should be relaxed during initial mapping.

Definition

Type: float

Flags:

Default value: =InheritValue up {}

Valid on: design_____

Description

The Underconstrain attribute specifies the factor by which the timing constraints should be loosened for the initial compile of the design.

coreConsultant generates a synthesis strategy that loosens the input and output timing constraints for the design by the specified factor for the initial compile to produce a slower, but smaller, circuit to be used as a starting point for subsequent incremental compiles. For example, if you set Underconstrain to 0.1, coreConsultant loosens the timing constraints by 10 percent for the initial compile. For incremental compiles, coreConsultant returns the timing constraints to the original values.

In general, you should not need to set Underconstrain unless you cannot meet your area constraints on an area-driven compile by setting the CompileMapEffort attribute to high and setting the Synthesis QorEffort parameter to high. In such a case, if the design meets timing constraints easily, you may get better area results by specifying a value for Underconstrain.

There may also be cases where you need to set Underconstrain to a non-zero value if you cannot meet the design's area objectives with your technology library.

coreConsultant ignores the Underconstrain attribute if the value of the design's OptimizationPriorities attribute is timing or timing_area.

If you do not explicitly specify a value for Underconstrain on a design, the design inherits the Underconstrain value from the next higher level design in the hierarchy.

Examples

The following line in a coreConsultant Tcl script sets the Underconstrain value to 0.1 on the current_design, which means that timing constraints are loosened by 10 percent for the initial compile:

set_design_attribute Underconstrain 0.1

The following line in a coreConsultant Tcl script sets the Underconstrain value to 0 on the design named Subblock_A, which means that Subblock_A will not be underconstrained for the initial compile:

```
set_design_attribute -designs Subblock_A Underconstrain 0
```

See Also

set_design_attribute (2), OptimizationPriorities (3), Overconstrain (3)

[API Index](#) [Command Reference](#)

UnelaboratedName

the unelaborated name for this design

Definition

Type: string

Flags: readOnly

Default value:

Valid on: design_____

Description

Examples

See Also

UniqueConsumerConnections

Show each consumer connection for this provider independently on schematic.

Definition

Type: boolean

Flags:

Default value: =sIntf::getDefaultUniqueConsumerConnections %item

Valid on:

Description

This attribute controls if an interface that can provide to many consumers is shown as one pin or a pin for each possible consumer. The default value is usually correct. Set if the Schematic does not look right.

Examples

See Also

UsedOnInstance

Specifies condition whether the interface port or parameter is used in the parent interface instance. The condition can include `@<paramName>`'s in the attribute value, which makes the interface usage conditional.

Definition

Type: string

Flags:

Default value: 1

Valid on: param_____

Description

The attribute is usually set with the `"-used \<condition\>"` argument of `create_interface_port` and `create_interface_parameter` and specifies the condition under which the interface port or parameter is used on its instance. By default the condition is true, and the interface item is always "used".

Then the interface item is used on both the consumer and the provider side, and the interface item must be associated to a design item using interface linkage.

If the attribute condition is specified using an `@Parameter` on-consumer interface parameter then the port is unused on a specific interface instance in case this condition (expression value) evaluates to false for the given, static `@-parameter` value.

Any valid 'expr' expression can be used as a condition, but a "static true" (always used) must be represent as "1" or "true", not as expression.

An unused interface item ignores any interface-to-design linkage, and is not considered during automatic connection (ports) or parameter propagation (parameters) in `coreAssembler`.

Examples

To define an interface port which is used only on split-capable bus connections, assuming that an on-consumer interface parameter "SplitCapable" exists on the same interface definition:

```
prompt> create_interface_port HSPLIT -interface AHB-Slave \
-used @SplitCapable ...
```

If `SplitCapable` is 0 by default then the interface port is not used until `SplitCapable` is set to true on the interface instance. The "is used"

concept requires that SplitCapable has a static value (is read-only in coreAssembler).

```
prompt> create_interface_instance -interface AHB-Slave ... slave
prompt> set_interface_port_attribute slave HSPLIT InterfaceLink hsplit
prompt> set_interface_parameter slave SplitCapable 1
prompt> set_interface_parameter_attr slave SplitCapable ReadOnlyParam
true
```

If SplitCapable would be false then no InterfaceLink is required. In this case the interface port seems not to exist and cannot connect.

See Also

InterfaceIsUsed (3), Optional (3), create_interface_port (2),
create_interface_parameter (2)

[API Index](#) [Command Reference](#)

UserPortName

Indicates a port name specified with `create_connection -port` to use with `merge_ports`.

Definition

Type: string

Flags:

Default value:

Valid on: pin port _____

Description

This attribute is set to the value specified with the `-port` option of the `create_connection` command. The attribute is used by `merge_ports` to preserve this name if the port is merged. When more than one port to be merged together have this attribute set, the merge is not done.

Examples

See Also

Utilization

Indicates how utilized the given interface is.

Definition

Type: string

Flags: readOnly

Default value:

Valid on:

Description

Indicates the utilization level of the given interface in coreAssembler.

Legal values are: empty tooempty halfempty full toofull. The value toofull implies more connections than are allowed. The value tooempty implies that there are not enough connections to meet the MinConsumers requirement. The value empty means the interface is not connected (and there is no minimum requirement). The value full means the interface is connected to the maximum allowed number of interfaces. If none of the above conditions is met, the value is set to halfempty.

Examples

See Also

[API Index](#) [Command Reference](#)

VHDLDesignLibrary

VHDL library for source code generated by coreAssembler

Definition

Type: string**Flags:****Default value:** =sHdl::getVHDLDesignLibrary %item**Valid on:** design_____

Description

When generating VHDL code in the GenerateSubsystemRTL, this attribute specifies the library to use. By default the library name is work, or the value of the containing level of hierarchy.

Examples

```
coreAssembler> set_activity_parameter GenerateSubsystemRTL LanguageChoice
```

```
VHDL
```

```
coreAssembler> set_design_attribute VHDLDesignLibrary audio
```

See Also

Value

Value of this parameter

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

The Value attribute is the current value of the selected parameter.

For activity parameters, you set or get the value of a parameter by using the `set_activity_parameter` or `get_activity_parameter` command.

For design parameters, you use `set_parameter_attribute` or `get_parameter_attribute` to set or get the value of the parameter's Value attribute.

In the core developer design flow, `coreBuilder` initializes the Value of a design parameter to the value specified in the HDL code or to `DefaultValue` if you set `DefaultValue` in the HDL code with a `reuse-pragma`.

Parameter values may or may not have symbolic names. If the `SymbolicNames` attribute is set on a parameter, the value that appears in the `coreConsultant` or `coreBuilder` GUI for the parameter is the symbolic name associated with the parameter's current value. When you set or get a parameter value using the commands listed above, you interact with the actual parameter value (through the Value attribute) and not the symbolic name for the attribute.

Examples

To return the current value of design parameter A, then set A to new value:

```
coreConsultant> get_parameter_attribute A Value
```

```
1
```

```
coreConsultant> set_parameter_attribute A Value 2
```

See Also

[get_activity_parameter \(2\)](#), [set_activity_parameter \(2\)](#),
[get_parameter_attribute \(2\)](#), [set_parameter_attribute \(2\)](#), [DefaultValue \(3\)](#), [SymbolicNames \(3\)](#)

[API Index](#) [Command Reference](#)

ValueRequired

Value is required for this parameter

Definition

Type: boolean**Flags:****Default value:****Valid on:** param_____

Description

The ValueRequired attribute indicates whether a value is required for a parameter. If ValueRequired is true on a design parameter, coreConsultant will not complete the Specify Configuration activity unless the user specifies a value for the parameter. If ValueRequired is true on a custom activity parameter, coreConsultant or coreBuilder will not complete the associated custom activity unless the user specifies a value for the parameter.

In GUI mode, if you attempt to execute the activity without specifying a value for the parameter, coreConsultant or coreBuilder keeps the activity dialog posted and displays an error message that indicates which parameter value is required.

Examples

The following annotation in a Verilog source file specifies that a user-specified value is required for the next design parameter declared in the file:

```
// reuse-pragma attr ValueRequired true
```

The following line in a _Obj.tcl file for a custom activity specifies that a user-specified value is required for custom activity parameter TestNum:

```
set_parameter_attribute TestNum ValueRequired true
```

See Also

[create custom activity parameter \(2\)](#)

Variables Index

<u>cA supports hierarchy</u>	Controls whether or not hierarchy is supported by coreAssembler.
<u>check_error_list</u>	List of messages IDs that should cause Design Compiler to stop in the case where one or more of the message appears during a call to the 'compile' command.
<u>collection_print_item_limit</u>	Number of item names to print when creating a collection.
<u>coretools home page</u>	Page to show as the tool home page.
<u>crm file patterns</u>	Identifies files that are part of your source code control system.
<u>guiBehavior</u>	Array variable that specifies different aspects of the GUI behavior.
<u>maximum bit blast size</u>	Largest size bus that can be bit blasted.
<u>scratch_dir</u>	Directory for storage of temporary files.
<u>sh arch</u>	Indicates the system architecture of your machine.
<u>sh command abbrev mode</u>	Sets the command abbreviation mode for interactive convenience.
<u>sh command log file</u>	Specifies the name of the file to which the application logs the commands you executed during the session.
<u>sh continue on error</u>	Allows processing to continue when errors occur during script execution with the \fBsource\fP command.
<u>sh dev null</u>	Indicates the current null device.
<u>sh enable page mode</u>	Displays long reports one page at a time (similar to the UNIX \fBmore\fP command.
<u>sh executable name</u>	Full path name of executable for the current tool session.
<u>sh script stop severity</u>	Indicates the error message severity level that would cause a script to stop running before it completes.
<u>sh show tcl stack on error</u>	Print or do not print the TCL stack when a TCL error occurs.
<u>sh source uses search path</u>	Indicates if the \fBsource\fP command uses the \fBsearch_path\fP variable to search for files.
<u>sh tcllib app dirname</u>	Indicates the name of a directory where application-specific Tcl files are found.
<u>sysCmd</u>	

coreTools Command Reference Index

Global array variable containing paths to different system commands.

verbose_messages

Enables verbose messaging mode.

Verify Intent

This activity attempts to flag common mistakes that are made when specifying constraints.

Parameters

Parameters Table

Name	Label	Description
SettingsForElaboration	SettingsForElaboration	Holds the values of the last configuration used to elaborate the design for budget checking.
Checks		
DoCompletenessCheck	Report budget completeness	Ensure constraints are complete. All clocks have a period specified. Ports have input/output delay and driving cell set.
DoConsistencyCheck	Report budget consistency	Ensure budgets are consistent. All electrically equivalent ports must have equivalent values for input/output delay, driving cell, and load type. Clocks has period greater than delay and uncertainty.

See Also

[Design Configuration Intent.](#)
[Specify Clocks.](#)
[Specify Synthesis Methodology.](#)
[Specify Timing Exceptions.](#)
[Specify Port Constraints.](#)

VerilogHeaderValue

Defines the value for the Verilog header file for the given attribute on this register or register field.

Definition

Type: string

Flags: subscripted

Default value: =sMem::default_HeaderValue verilog %item %subscript

Valid subscripts: AddressOffset BitAddressOffset MemoryAccess

RegisterResetMask RegisterResetValue RegisterSize

Default subscript:

Valid on:

Description

This attribute is used to alter how a register attribute is written to the automatically generated Verilog header file. The attribute to be altered is indicated by the attribute subscript. This attribute is used to customize how the indicated attribute is written to the header file. The value written will be the value returned by this attribute, instead of directly using the value returned by the subscript.

Examples

Write the RegisterSize attribute value as 1 << 5 instead of 32

```
set_register_attribute MyReg {VerilogHeaderValue[RegisterSize]} {(1 << 5)}
```

See Also

[CHeaderValue \(3\)](#), [VhdlHeaderValue \(3\)](#)

Version

The version of the Bill-of-Materials template.

Definition

Type: float

Flags:

Default value:

Valid on:

Description

This attribute indicates the version of a particular BoM template. This information is typically only useful to BoM template developers who need to track which version of a particular template is being used by core designers.

Examples

This attribute is set via the BoM template as shown in the sample template fragment below:

```
set BOMTemplate {  
  BOMTemplate  
  {Name {Synopsys RMM BOM Template}}  
  {coreBuilderBOMTemplateVersion 1.0}  
  {Version 1.0}  
}
```

See Also

VhdlHeaderValue

Defines the value for the VHDL header file for the given attribute on this register or register field.

Definition

Type: string

Flags: subscripted

Default value: =sMem::default_HeaderValue vhd1 %item %subscript

Valid subscripts: AddressOffset BitAddressOffset MemoryAccess
RegisterResetMask RegisterResetValue RegisterSize

Default subscript:

Valid on:

Description

This attribute is used to alter how a register attribute is written to the automatically generated VHDL header file. The attribute to be altered is indicated by the attribute subscript. This attribute is used to customize how the indicated attribute is written to the header file. The value written will be the value returned by this attribute, instead of directly using the value returned by the subscript.

Examples

Write the RegisterSize attribute value as 1 << 5 instead of 32

```
set_register_attribute MyReg {VhdlHeaderValue[RegisterSize]} {(1 << 5)}
```

See Also

[VerilogHeaderValue \(3\)](#), [CHeaderValue \(3\)](#)

VipConnection

Indicates the name of the interface to be connected to. If two values are returned, indicates an alternate instance to be connected to. Can access global variable `::iip_instance_name` via formulas if needed.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This attribute is used on monitor and application VIP components to indicate a connection from an interface on the VIP component to an interface on the component being connected to. The attribute is set on an interface instance and its value indicates which interface to connect to on the DUT or component being monitored. If the value contains two sub-elements then the first is assumed to be the name of an alternate instance to attach to. This can be used (via formula) when the VIP component ends up needing to connect to more than one component.

Examples

See Also

[/ref CustomizedTestbenchCode \(3\)](#), [/ref MonitoredComponent \(3\)](#), [/ref MonitoredInterface \(3\)](#), [/ref VipInitializationCall \(3\)](#)

VipInitializationCall

Indicates the string required to be inserted into the testbench to initialize the instantiated VIP component.

Definition

Type: string

Flags:

Default value:

Valid on: cell _____

Description

This attribute is set on a cell to indicate the required text to be inserted into the testbench to initialize the instance. It is typically used in conjunction with a chunk of text written out as the cell instance is written. For example a Verilog task may be written as part of the cell instantiation and then the call to the Verilog task would be stored in the VipInitialization attribute. The testbench generator can then get this attribute and know what text must be inserted into the testbench to initialize the IP.

Examples

```
set_cell_attribute my_vip VipInitializationCall my_task_name
```

See Also

[/ref CustomizedTestbenchCode \(3\)](#), [/ref MonitoredComponent \(3\)](#), [/ref MonitoredInterface \(3\)](#),

[API Index](#) [Command Reference](#)

VipModelName

Specifies the name of the vmm vip model

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute is used in the creation of the VMM environment files. The name of the VIP model is needed for instantiation. The name of the design is not the same since that name corresponds to the Verilog interface which acts as a connector between the System Verilog environment and the Verilog Testbench.

Examples

See Also

[API Index](#) [Command Reference](#)

VipPackage

Specifies the name of the vmm vip package

Definition

Type: string

Flags:

Default value:

Valid on: design_____

Description

This attribute contains the name of any package file that the VIP design needs to have included for simulation

Examples

See Also

[API Index](#) [Command Reference](#)

VipRandomRange

Specifies the VMM variable's range of possible settings

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

VipRandomizable

Specifies if the specified VMM variable is randomizable

Definition

Type: string

Flags:

Default value: No

Valid on: param_____

Description

This attribute indicates if a VIP parameter is randomizable and if so when it can be randomize: Prior to start of simulation or during simulation

Examples

See Also

[API Index](#) [Command Reference](#)

VipReasonableRandomRange

Specifies the VMM variable's range of reasonable settings

Definition

Type: string

Flags:

Default value:

Valid on: param_____

Description

Examples

See Also

[API Index](#) [Command Reference](#)

VipScope

Specifies the scope of the VIP configuration field

Definition

Type: string

Flags:

Default value: Public

Valid on: param_____

Description

This attribute is set on a parameter to indicate the scope of the VIP parameter within its class definition. It is a string, but the most typical values would be to indicate if this attribute is Public or not

Examples

See Also

Visible

Make this parameter visible in the GUI

Definition

Type: boolean

Flags: subscripted

Default value: 1

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: 1

Valid on: param port _____

Description

The Visible attribute determines whether the selected parameter is visible in the associated parameter dialog. For design parameters, a parameter appears in the coreBuilder Specify Configuration Intent dialog and the coreConsultant Specify Configuration dialog only if its Visible attribute is true. For custom activity parameters, a parameter appears in the custom activity parameter dialog only if its Visible attribute is true.

To determine whether to display a parameter in the GUI according to the Visible attribute, the coreTool determines the logical AND of the set of Visible values for which the Visible subscript evaluates to true. If the logical AND of those values is true, the coreTool displays the parameter in the GUI. If the logical AND of those values is false, the coreTool does not display the parameter in the GUI.

Simple Dependency Checking with Default Subscript

The default subscript for Visible is 1, which means that the Visible value alone determines whether the coreTool displays the parameter in the GUI. The most simple form of using the default subscript is to simply set the value of Visible (Visible[1]) to either 0 or 1 to display or not display the parameter.

As an example of a simple dependency, you can make parameter A visible only if the value of parameter B is 1 by setting the value of Visible on parameter A to "&@B==1". In this case, the Visible subscript is [1] (the default). Therefore, the coreTool displays parameter A in the GUI only if the value of parameter A is equal to 1.

More Complex Dependency Checking with Single Subscript

As an example of using a single Visible subscript, consider a design with parameters A, B, C, and D such that:

- If the value of parameter A is 2, parameter C should be visible only if the value of parameter B is greater than 5 and less than the

value of parameter D.

- If the value of parameter A is not 2, parameter C should be visible regardless of the values of the other parameters.

To enforce this dependency, set the Visible attribute on parameter C. The required expression for the Visible subscript is "`@A==2`" because C has restrictions only if the value of A is 2.

The expression that you specify as the value of Visible determines whether parameter C should be visible when $A = 2$. In this example, the required value for `{Visible[@A==2]}` is "`@B>5&&@B<@D`" because the value of B must be greater than 5 and less than the value of D if the value of A is 2. If "`@B>5&&@B<@D`" evaluates to true (the value of B is greater than 5 and less than the value of D), the coreTool displays parameter C in the GUI. If "`@B>5&&@B<@D`" evaluates to false, the coreTool does not display parameter C in the GUI.

Complex Dependency Checking with Multiple Subscripts

To implement more complex parameter dependencies, use multiple Visible subscripts. The coreTool uses the logic AND of all value expressions for which the associated subscript is true to determine whether to display the parameter. For example, consider a design with parameters W, X, Y, and Z such that:

- If the value of parameter W is 0, parameter Z should be visible only if the value of parameter X less than 20.
- If the value of parameter Y is 1, parameter Z should be visible only if the value of parameter X is greater than 10.

The following Visible attribute assignments on parameter Z implement the above dependencies:

```
set_parameter_attribute Z {Visible[@W==0]} @X<20
```

```
set_parameter_attribute Z {Visible[@Y==1]} @X>10
```

The coreTool logically ANDs each value for which the Visible subscript evaluates to true. Therefore, if $W = 0$ and $Y = 1$, the coreTool displays parameter Z only if the value of parameter X is between 10 and 20.

Examples

The following line in a coreBuilder Tcl script makes parameter A not visible in the GUI:

```
set_parameter_attribute A Visible 0
```

The following line in a coreBuilder Tcl script makes parameter A visible in the GUI only if the value of parameter B is 1:

```
set_parameter_attribute A Visible @B==1
```

The following annotation in a Verilog source file imposes the dependency that if parameter $A = 2$, then parameter C is visible only if parameter B

is greater than 5 and less than the value of parameter D:

```
//reuse-pragma attr Visible[@A==2] @B>5&&@B<@D  
parameter C = 0;
```

The following lines in an `_Obj.tcl` file for a custom activity impose the following restrictions on custom activity parameter Z:

- If `W = 0`, Z is visible only if the value of X is less than 20.
- If `Y = 1`, Z is visible only if the value of X is greater than 10.

```
set_parameter_attribute Z {CheckExpr[@W==0]} @X<20  
set_parameter_attribute Z {CheckExpr[@Y==1]} @X>10
```

See Also

[set_parameter_attribute \(2\)](#), [ReadOnlyParam \(3\)](#), [Enabled \(3\)](#)

VolatileMemory

True if this item refers to volatile memory

Definition

Type: boolean

Flags:

Default value: =InheritValue up 0

Valid on:

Description

The element implies there is some additional mechanism by which registers can acquire new values other than reads/writes/resets and other access methods.

In UVM environment register fields with VolatileMemory set to true will not participate in bit_bash tests & will skip failure comparison in hw_reset & reg_access tests.

Examples

To indicate that memory map map1 is volatile memory:

```
coreBuilder> set_memory_map_attribute map1 VolatileMemory true
```

See Also

[API Index](#) [Command Reference](#)

Waveform
clock waveform

Definition

Type: string

Flags:

Default value: =list 0 [percent_of_period 50]

Valid on: clock_____

Description

Waveform is a Tcl list which specifies the rise and fall edge times (by default in library time units) of the clock over an entire clock period. The first time in the list is a rising transition, typically the first rising transition after time zero. There must be an even number of increasing times, and they are assumed. The numbers must represent one full clock period. The default value of a clock waveform is (0.0, period_value/2). Units can be specified with edge times, and formulas can be used to specify a waveform (see the examples below).

Examples

To get the default value of the Waveform, assuming PHI1 is a clock with 10ns CycleTime:

```
coreBuilder> get_attribute PHI1 -attr Waveform -with_units
0ns 5ns
```

Either of the following two commands sets the clock Waveform to {0ns 2.5ns 5ns 7.5ns}:

```
coreBuilder> set_clock_attribute clk Waveform {0 2.5 5.0 7.5}
or
```

```
coreBuilder> set_clock_attribute clk Waveform
{{=percent_of_period 0} {=percent_of_period 25}
{=percent_of_period 50} {=percent_of_period 75}}
```

See Also

[set_clock_attribute \(2\), CycleTime \(3\)](#)_____

WireLoad

Wireload model to use for a design.

Definition

Type: string**Flags:****Default value:** =sCstr::determineWireLoad %item**Valid on:** busBit design port _____

Description

The WireLoad attribute specifies the wire load model to be used for synthesis. The core developer does not set WireLoad, because it is a technology-specific specification that must be selected by the core integrator.

If the core integrator does not explicitly specify a value for WireLoad, coreConsultant will either automatically select a wire load model for the design or leave automatic wire load model selection to Design Compiler. If PhysicalRegion is true on the design, coreConsultant uses the AreaEstimate value of the design (if AreaEstimate is either set on the design or can be calculated by summing the AreaEstimate values of the design's subblocks) to automatically select a wire load model for the initial compile. For subsequent compiles, coreConsultant lets Design Compiler automatically select wire load models based on the actual design area.

If the core integrator does not specify WireLoad and PhysicalRegion is not set to true on the design (or PhysicalRegion is true but no AreaEstimate value can be determined), coreConsultant does not select a wire load model and, instead, leaves automatic wire load model selection to Design Compiler.

Examples

To use wire load io30x30 for all nets in My_Core including all nets within subblocks:

```
coreConsultant> current_design My_Core
coreConsultant> set_design_attribute WireLoad io30x30
coreConsultant> set_design_attribute WireLoadMode top
```

To use wire load io10x10 for all nets fully contained in Subblock_A, wire load io20x20 for all nets fully contained in Subblock_B, and wire load io30x30 for all nets fully contained in My_Core:

```
coreConsultant> current_design My_Core
```

coreTools Command Reference Index

```
coreConsultant> set_design_attribute WireLoad io30x30
coreConsultant> set_design_attribute WireLoadMode enclosed
coreConsultant> set_design_attribute -designs Subblock_A WireLoad io10x10
coreConsultant> set_design_attribute -designs Subblock_B WireLoad io20x20
```

See Also

set_design_attribute (2), AreaEstimate (3), PhysicalRegion (3), _____
WireLoadGroup (3), ParentWireLoad (3), WireLoadMode (3) _____

WireLoadGroup

Wireload model selection group to use for automatic wireload model selection.

Definition

Type: string

Flags:

Default value: =InheritValue up

{=sTech::get_default_wire_load_selection_name}

Valid on: design

Description

The WireLoadGroup attribute specifies which of the technology library's wire load selection groups to use for automatic wire load model selection if the WireLoad attribute is not set on a design.

If you do not set WireLoadGroup on a design, the design inherits the WireLoadGroup value from the next higher level design. If there is no WireLoadGroup to inherit, coreConsultant uses the library's default wire load selection group, if one exists.

Examples

To use wire load model selection group m5c_70_wlm for the current_design:

```
coreConsultant> set_design_attribute WireLoadGroup m5c_70_wlm
```

See Also

[set_design_attribute \(2\)](#), [WireLoad \(3\)](#), [ParentWireLoad \(3\)](#), [WireLoadMode \(3\)](#)

[API Index](#) [Command Reference](#)

WireLoadMinBlockSize

Minimum block size to use for automatic wireload selection.

Definition

Type: float

Flags:

Default value:

Valid on: design_____

Description

Examples

See Also

WireLoadMode

Indicates the wireload mode for the design.

Definition

Type: string

Flags:

Default value: =InheritValue up {}

Valid on: design_____

Description

The WireLoadMode attribute specifies which wire load mode to use for the selected design. Valid values for WireLoadMode are:

- top - Sets the wire load model for all nets contained in the selected design to the wire load model assigned to the top-level design, regardless of what wire load models are assigned to subblocks contained in the selected design.
- enclosed - Sets the wire load for a net to the wire load model of the smallest subblock that fully contains the net.
- segmented - If a net crosses hierarchical boundaries, each segment of the net uses the wire load model of the hierarchical subblock that contains that segment of the net.

If you do not specify WireLoadMode on a design, the design inherits the WireLoadMode value from the next higher level design. If there is no WireLoadMode value to inherit, coreConsultant uses the library's default wire load mode, if one exists.

Examples

To use wire load io30x30 for all nets in My_Core including all nets within subblocks:

```
coreConsultant> current_design My_Core
coreConsultant> set_design_attribute WireLoad io30x30
coreConsultant> set_design_attribute WireLoadMode top
```

To use wire load io10x10 for all nets fully contained in Subblock_A, wire load io20x20 for all nets fully contained in Subblock_B, and wire load io30x30 for all nets fully contained in My_Core:

```
coreConsultant> current_design My_Core
coreConsultant> set_design_attribute WireLoad io30x30
coreConsultant> set_design_attribute WireLoadMode enclosed
coreConsultant> set_design_attribute -designs Subblock_A WireLoad io10x10
coreConsultant> set_design_attribute -designs Subblock_B WireLoad io20x20
```

See Also

set_design_attribute (2), ParentWireLoad (3), WireLoad (3), WireLoadGroup (3) _____

WrapBlockIndividually

Causes test wrapper insertion to be performed on this design.

Definition

Type: boolean

Flags:

Default value: =expr { [InheritValue up -attr
WrapSubblocksIndividually 0] || ([get_attribute %item -attr
BistBlockIndividually] && [string equal [get_attribute %item -attr
BistType] dbist] && [string equal [get_attribute %item -attr
BistMode] all]) }

Valid on: design_____

Description

Setting WrapBlockIndividually to TRUE will cause a test wrapper to be inserted into the design for a SocBIST testing methodology. Setting WrapBlockIndividually to true will also cause the set_core_wrapper_configuration command to be issued in DFT Compiler. The correspondence between coreTools attributes and set_core_wrapper_configuration options is as follows:

WrapperUseRegisterIO -use_register_io
WrapperRegisterImplementation -register_io_implementation
DedicatedWrapperCell -dedicated_wrapper_cell
SharedWrapperCell -shared_wrapper_cell
OneWrapperClock -one_wrapper_clock
WrapperDefaultSafeState -default_safe_state

Examples

The following will cause the current design to be wrapped for SocBIST testing.

```
coreConsultant> set_design_attribute WrapBlockIndividually 1
```

See Also

[set_design_attribute \(2\)](#), [WrapSubblocksIndividually \(3\)](#), _____

[ScanBlockIndividually \(3\)](#), [BistBlockIndividually \(3\)](#) _____

[API Index](#) [Command Reference](#)

WrapSubblocksIndividually

Causes test wrapper insertion to be performed on the subdesigns of this design.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

Setting WrapSubblocksIndividually to true specifies that the sub-designs directly instantiated in the current design should have test wrappers inserted in them.

Examples

Assume that the current design, top, has designs left, right and middle directly instantiated in it. The following commands would specify that designs left, right, and middle will be wrapped, and the wrappers will be integrated in top.

```
coreBuilder> set_design_attribute WrapSubblocksIndividually TRUE
coreBuilder> set_design_attribute IntegrateTestWrappers TRUE
set_design_attribute (2), WrapBlockIndividually (3), _____
```

[API Index](#) [Command Reference](#)

WrapperDefaultSafeState

Specifies the default safe state for the wrapper cells.

Definition

Type: string

Flags:

Default value: none

Valid on: design_____

Description

Specifies the default safe state value for the wrapper cells. Allowed values are 0, 1, or none (the default); these determine the value of the safe state logic inserted by the core wrapper application.

Examples

Set the default safe state for the wrapper to logic 0.

```
coreBuilder> set_design_attribute WrapperDefaultSafeState 0
```

See Also

[set_design_attribute \(2\)](#), [WrapBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

WrapperExcludePorts

List of ports that should not be wrapped.

Definition

Type: string**Flags:****Default value:****Valid on:** design_____

Description

WrapperExcludePorts specifies a list of ports that should not be wrapped. Since this reduces test coverage you should exclude output ports only when necessary. For example, wrapping clocked primary outputs leads to a potential race condition. You should never exclude input ports other than test signals, test holds, clocks and other asynchronous signals from being wrapped.

Examples

Specify that ports a, b, and c should not be wrapped.

```
set_design_attribute WrapperExcludePorts [list a b c]
```

See Also

[set_design_attribute \(2\)](#), [WrapBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

WrapperRegisterImplementation

If the option is set to swap registers connected to ports of the design are replaced with equivalent shared wrapper cells. If the option is set to `in_place`, additional glue logic will be added to registers attached to ports of the design to reuse the registers in core wrapper chains.

Definition

Type: string

Flags:

Default value: swap

Valid on: design

Description

If the option is set to swap registers connected to ports of the design are replaced with equivalent shared wrapper cells. If the option is set to `in_place`, additional glue logic will be added to registers attached to ports of the design to reuse the registers in core wrapper chains.

Examples

Specify that SocBIST is to reuse registers attached to ports on the design.

```
coreBuilder> set_design_attribute WrapperRegisterImplementation in_place
```

See Also

[set_design_attribute \(2\)](#), [WrapBlockIndividually \(3\)](#)

[API Index](#) [Command Reference](#)

WrapperUseRegisterIO

If the option is set to TRUE, shared wrapper cells will be used for register bound ports. The registers are reused in core wrapper chains.

Definition

Type: boolean

Flags:

Default value: 0

Valid on: design_____

Description

If the option is set to TRUE, shared wrapper cells will be used for register bound ports. The registers are reused in core wrapper chains. If this attribute is FALSE, shared wrapper cells are not used by the core wrapper application. Dedicated wrapper cells are used to wrap ports of the design.

Examples

Specify that register bound ports should use shared wrapper cells.

```
coreBuilder> set_design_attribute WrapperUseRegisterIO TRUE
```

See Also

[set_design_attribute \(2\)](#), [WrapBlockIndividually \(3\)](#)

WriteBehavior

Indicates that writing the given field will cause a write of a value other than the one written. The written value will be modified as described by the value of this attribute.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: DEFAULT

Valid on:

Description

This attribute indicates the impact of writing the given register field. This information can be used when writing register tests to determine whether the test program can recover from the impact of writing the register.

Valid values are: oneToClear, oneToSet, oneToToggle, zeroToClear, zeroToSet, zeroToToggle, clear, set, and modify. The latter value (modify) is meant to imply a transition not covered by any of the other values.

Examples

See Also

[ReadAction](#)

WriteBehaviorModifier

Indicates user-defined writing action when WriteBehaviour is set to 'modify'. It implies this write action is not covered by any of other allowed WriteBehavior values which are: {oneToClear, oneToSet, oneToToggle, zeroToClear, zeroToSet, zeroToToggle, clear, set}. This attribute is only valid when WriteBehavior is set to 'modify'

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript: DEFAULT

Valid on:

Description

Examples

See Also

[API Index](#) [Command Reference](#)

WriteComponentInHDL

Set to false if this component should not be written in the final netlist written by coreAssembler.

Definition

Type: boolean

Flags:

Default value: Value calculated internally.

Valid on: cell design _____

Description

Setting this attribute to false will prevent coreAssembler from writing a cell instance in the HDL for any instances of this component. This is only valid if all the input ports to this component have FeedThroughConnect set

Examples

```
coreBuilder> set_design_attribute WriteComponentInHDL false
```

See Also

[FeedThroughConnect \(3\)](#)

[API Index](#) [Command Reference](#)

WriteConstraint

Defines a constraint to be met when writing the given register field.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This attribute indicates a constraint that must be met when writing to the given register field. Legal values are: writeAsRead and useEnumeratedValues. When set to writeAsRead, the field can only be written with the value read from the field. When set to useEnumeratedValues, the field can only be written using one of the values defined in the IP-XACT description of the register field.

Examples

See Also

[Testable](#)

`add_activity_hook`

Add a hook to an existing activity

Syntax

```
string add_activity_hook [-before] [-after] [-cancel] [-prepend] [-once]
[-filegroup] [-sequence sequence] [-mode <activityMode>] activity command
    string sequence
    string <activityMode>
    string activity
    string command
```

Parameters

<code>-before</code>	Execute this command before executing the activity. In GUI mode this means the command is run before the activity dialog is shown. In batch mode it runs before the built-in code of the activity that runs when the activity is completed.
<code>-after</code>	Execute this command after executing the activity. The command is run after the built-in code of the activity that runs when the activity is completed, but note that this is still prior to the activity being marked complete.
<code>-cancel</code>	Run this command if the activity did not complete.
<code>-prepend</code>	Put this command before existing hooks, instead of after existing hooks.
<code>-once</code>	Run this hook one time in coreAssembler even if there are multiple instances.
<code>-filegroup</code>	Makes hook run before files are configured and written.
<code>-sequence <i>sequence</i></code>	The order to execute the hooks in. Note that for backward script compatibility hooks default to a sequence number of 999 if nothing is specified.
<code>-mode <<i>activityMode</i>></code>	Specify the activity mode in which the hook is run. (Values: builder, consultant, developer, assembler)
<code><i>activity</i></code>	The activity that you want to modify by adding a hook.
<code><i>command</i></code>	The name of the Tcl command to execute.

Description

The `add_activity_hook` command enables you to customize the execution of an existing coreTool activity, by inserting your own TCL command into the

coreTools Command Reference Index

activity execution sequence (see sequence definition below). You can specify whether to execute your Tcl command before the coreTool displays the prompt (dialog) for the activity, after the user clicks the OK button for the activity, or after the user clicks the Cancel button for the activity.

You must use a separate `add_activity_hook` command for each hook that you want to add.

The normal flow of activity completion is:

1. The coreTool runs the commands that are designated as pre-prompt commands for the activity. The pre-prompt commands include built-in (default) commands, plus any custom commands that have been added by executing `"add_activity_hook -before"`.
2. Post the dialog for the activity.
3. When the user clicks the OK button, the coreTool checks the specified activity parameter values for validity, then executes the commands that are designated as post-prompt-ok commands for the activity. The post-prompt-ok commands include built-in (default) commands, plus any custom commands that have been added by executing `"add_activity_hook -after"`.
4. If the user clicks the Cancel button, the coreTool executes the commands that are designated as post-prompt-cancel commands for the activity. The post-prompt-cancel commands include built-in (default) commands, plus any custom commands that have been added by executing `"add_activity_hook -cancel"`.

In batch mode, the coreTool does not post the activity dialog. The batch mode sequence of activity completion is: execute pre-prompt command(s), validate activity parameters, then execute post-prompt-ok command(s).

As described above, the `-before` switch to `add_activity_hook` appends the specified command to the list of pre-prompt commands for the specified activity. The `-after` switch appends the specified command to the list of post-prompt-ok commands. The `-cancel` switch appends the specified command to the list of post-prompt-cancel commands.

Using the `-prepend` switch causes the command to be prepended to the list of commands (for example, the list of pre-prompt commands), instead of appended.

If you execute the `add_activity_hook` command at the coreBuilder prompt, the activity modification remains in effect for the current coreBuilder session. To make the modification available permanently, create a coreBuilder or coreConsultant plug-in.

Use the `report_activities` command to obtain a list of activities to which a hook can be added. This should be done in the tool in which the hook is going to be utilized.

Using the -filegroup switch overrides the other switches and it causes the command to be run when the user OKs activity but before any files are written out, like any RTL files

Examples

To execute the Tcl command My_LD_command before posting the dialog for the coreBuilder Load Designs activity:

```
coreBuilder> add_activity_hook -before LoadDesigns My_LD_command
```

To execute the Tcl command My_Tech_cancel_cmd when the user cancels the the coreBuilder Specify Reference Technology activity:

```
coreBuilder> add_activity_hook -cancel TechSetup My_Tech_cancel_cmd
```

To modify the execution of a coreConsultant activity, place the add_activity_hook command in a .tcl file, then execute the coreBuilder Load Consultant Plugins activity to build the plug-in. If you create a coreConsultant plug-in from a .tcl file that contains the following command, coreConsultant executes the Tcl My_Synth_command after the user clicks OK for the coreConsultant Synthesize activity, before executing the default Tcl command for Synthesize:

```
add_activity_hook -after -prepend Synthesize My_Synth_command
```

See Also

[create_custom_activity \(2\)](#), [create_plugin_kb \(2\)](#), [report_activities \(2\)](#)

NAME

add_command_hook

Add hook into command execution

SYNTAX

```
string add_command_hook -before script  
                        -after script -replace script [-name name]  
                        commandName
```

```
string    script  
string    script  
string    script  
string    name  
string    commandName
```

ARGUMENTS

-before *script* Hook runs before command.

-after *script* Hook runs after command.

-replace *script*
Hook runs in place of command.

-name *name* Name of hook. If no name is specified a
name is automatically generated.

commandName Command to update.

DESCRIPTION

The **add_command_hook** adds a customization point into the execution of an application command. For example, these hooks can be used to either run a report or gather statistics before and after an application command is run.

Within the body of any hook the `get_current_hook_command` can be used to query the command and its arguments. For example, a before and after hook may use this information if the hook actions should only happen when specific options are specified. In a replace hook the hook can run the replaced command possibly updating the command options.

This command returns the name of the hook which can be used to remove the hook from the command if desired.

Python Support

When run from a Python, this command expects the supplied script to be a Python script. When the Python hook is run, it is evaluated as if the code is run from within an anonymous function declared at the current scope (global or package). The normal function rules apply. For example, if you want to modify global or package variables, you need to declare them as 'global' within the script.

EXAMPLES

Arrange for report timing to be executed every time `place_opt_is` run.

```
prompt> add_command_hook place_opt -before report_timing
before@
```

Force all `report_timing` calls to use the `-nosplit` option.

```
add_command_hook report_timing -replace {eval [get_current_hook_command] -nosp
```

Python Example

When evaluated from Python the supplied script is Python. A Python equivalent of the previous example, looks like this:

```
def report_timing_hook():
    (command, arg, kwargs) = cmd.get_current_hook_command()
    kwargs['nosplit'] = True
    cmd.result = command(*args, **kwargs) # Run command, and return the result
```

coreTools Command Reference Index

```
cmd.add_command_hook(cmd.report_timing, replace='report_timing_hook()')
```

SEE ALSO

```
get_command_hooks(2)  
remove_command_hook(2)  
get_current_hook_command(2)
```


add_files_to_filegroup

Add files to the specified file group in the Bill of Materials; create the file group if it does not already exist

Syntax

```
string add_files_to_filegroup -files <list of files> -install <install
directory> [group]
    string <list of files>
    string <install directory>
    string group
```

Parameters

	The file(s) to be added to the specified file group.
-files <list of files>	The specified files can include directory names. In that case the files within the directory (recursively) are individually added to the filegroup.
-install <install directory>	The directory into which the specified files should be installed when the coreKit is installed.
group	The name of the Bill of Materials file group into which to add the specified files.

Description

The `add_files_to_filegroup` command adds the specified list of files to the specified Bill of Materials (BoM) filegroup. `coreBuilder` automatically executes the `add_files_to_filegroup` command when you execute the Create Bill of Materials activity. Use `add_files_to_filegroup` to add files to the BoM in batch mode.

If the specified filegroup does not already exist, `coreBuilder` creates a new filegroup with the specified name and adds the specified files to the new filegroup. During `coreKit` installation, `coreConsultant` installs the specified files into the directory specified by the `-install` option.

The `add_files_to_filegroup` command may also be used to add files and filegroups to a plugin by calling the command in the `_Obj.tcl` file of the plugin. The pathnames to the files should be absolute pathnames, or relative paths to the plugin source directory or the current working directory. Note that files in the plugin source directory with a `.tcl` extension are treated as source code for the plugin, so it may be more

appropriate to place any .tcl files to be added to a plugin filegroup in a subdirectory.

For organizational convenience, filegroups can be defined hierarchically. For example, filegroups A, A/B, and A/C can be created to store different files, respectively. There is no implication of the hierarchy other than for naming organization. Filegroup attributes set on filegroup A do not propagate to filegroup A/B. In the coreTools data model, filegroups are named with their base name (A, B, C in the example above). This can lead to filegroups with the same name. In order to find a specific filegroup in this case, the HierName attribute can be used. For example, if filegroup B/C existed with the groups defined above, then B/C could be found using:

```
set fg [find_item -type filegroup C -filter HierName==B/C]
```

Examples

To add file1.v and file2.v to the Example filegroup and install them in the example directory when installing the coreKit:

```
coreBuilder> add_files_to_filegroup Example -files {file1.v file2.v}  
-install example
```

See Also

[create_autoload_filegroup \(2\)](#), [create_plugin_kb \(2\)](#), [load_plugin \(2\)](#),
[DefaultInstallDir \(3\)](#)

add_hdl_pragma

A reuse-pragma statement external to the HDL

Syntax

```

string add_hdl_pragma -ignore -attr <attribute> -process_ifdef <mode>
[-value <value>] -macro <name> -design <name> -package <name> -line <num>
-start_line <num> [-end_line <num>] -ignore_file <name> [-param <name>]
[-function <name>] [-port <name>] [-cell <name>] [-signal <name>] [-type
<name>] [-file <name>] [-library <name>]
    string <attribute>
    string <mode>
    string <value>
    string <name>
    int <num>

```

Parameters

-ignore	Ignore the object or line
-attr <attribute>	Set attribute on the object
-process_ifdef <mode>	Change parameter macro ifdef mode (Values: all_branches, standard)
-value <value>	Set the value of the given attribute
-macro <name>	Select the macro
-design <name>	Select the design
-package <name>	Select the package
-line <num>	Select the specified line
-start_line <num>	Select a range of lines
-end_line <num>	End a range lines
-ignore_file <name>	Specify the included file to be ignored in post processing
-param <name>	Select a parameter
-function <name>	Select a function
-port <name>	Select a port
-cell <name>	Select a cell
-signal <name>	Select a signal
-type <name>	Select a type
-file <name>	Select the file
-library <name>	Select the library

Description

The `add_hdl_pragma` command allows design intent to be added during the LoadDesigns activity, as if special reuse-pragma comments had been inserted into the source code for that activity.

This command must be placed into a special intent file that is sourced by LoadDesigns at the appropriate time. Executing this command at any other time results in an error. The LoadDesigns activity searches for intent files in the IntentSearchPath that end in `.pragma.tcl` and match the source files specified in InputFileNames. It then sources these files before analyzing the HDL code for that file.

The command has two ways for describing where the pragma belongs:

- *object_selector* - This format allows a user to refer to a specific item in the HDL file. The selector never refers to line numbers, but rather to entities in the HDL file. The following table describes the options that specify an *object_selector* and what source languages they are valid for.

Option	Description	VHDL	Verilog
-macro <name>	This refers to a macro definition		X
-library <name>	Override the current library. By default the library is the library of the associated HDL file.	X	
-design <name>	This refers to a design. This option is generally combined with another option to select an object within a design.	X	X
-package <name>	This refers to a package. This option is generally combined with another option to select an object with a package	X	
-param <name>	This refers to a parameter in the design or package	X	X
-function <name>	This refers a function name in the design or package	X	
-port <name>	This refers to a port in the design	X	X
-cell <name>	This refers to a cell in the design	X	X
-signal <name>	This refers to a signal or wire in the design	X	X
-type <name>	This refers to a type or subtype in the design or package	X	

- *line_selector* - This allows the user to refer to a specific line or series of lines in an HDL file. The following table describes the valid options for *line_selectors*

coreTools Command Reference Index

Option	Description
-file <name>	This option allows the user to specify a file. The default value is the HDL file associated to this intent file.
-line <num>	A line number in the file.
-start_line <num> -end_line <num>	A range of lines in the file. All lines greater than or equal to the start line and less than or equal to the end line are in the range
-ignore_file <name>	The included file is ignored during post processing.

The supported versions of `add_hdl_pragma` are described below. This command acts as if a `reuse-pragma` was actually inserted into the source code.

- `add_hdl_pragma -attr <name> -value <value> object_selector`
This command adds an attribute annotation to the specified object
- `add_hdl_pragma -ignore object_selector`
This command tells coreBuildt to ignore the given object.
- `add_hdl_pragma -ignore line_selector`
This command tells coreBuilder to ignore the line(s) specified. If possible, the object version of this command should be used. If the files are edited, line numbers may change, but objects can still be located.
- `add_hdl_pragma -process_ifdef all_branches|standard line_selector`
This command tells coreBuilder to change 'ifdef processing mode. The default processing mode at the start of every file is *all_branches*. The processing mode only applies to 'ifdef statements that refer to parameter macros. Note that this form only supports the `-line`, since this pragma is never paired.

Examples

To specify that a macro definition within a Verilog source file should become a configuration parameter:

```
add_hdl_pragma -macro fast_interrupt \
-attr Label -value "Include fast interrupt?"
add_hdl_pragma -macro fast_interrupt \
-attr MinValue -value 0
add_hdl_pragma -macro fast_interrupt \
-attr MaxValue -value 1
add_hdl_pragma -macro fast_interrupt \
-attr Description -value \
"When set to true, the core includes the fast interrupt
port fiq. When false, the fiq port is not available"
Generally MinValue and MaxValue are not specified for VHDL code, because
they are inferred directly from the type information in the VHDL source.
```

If you need to restrict the values further than those specified in the VHDL, then `MinValue`, `MaxValue`, and `EnumValues` may be used. To specify that a constant in a VHDL package should show up in the configuration GUI use code like the following:

```
add_hdl_pragma -package mycore_constants -param fast_interrupt \  
-attr Label -value "Include fast interrupt?"  
add_hdl_pragma -package mycore_constants -param fast_interrupt \  
-attr Description -value \  
"When set to true, the core includes the fast interrupt  
port fiq. When false, the fiq port is not available"  
To specify that a port is optional, and is only included when the  
configuration parameter fast_interrupt is true:
```

```
add_hdl_pragma -design top_des -port fiq \  
-attr GenerateIf -value @fast_interrupt  
add_hdl_pragma -file -ignore_file "inc.sv"  
add_hdl_pragma -file -ignore_file "../include/inc1.sv"  
add_hdl_pragma -file -ignore_file "path/src/inc2.sv"
```

See Also

[reuse-pragma \(2\)](#), [GenerateIf \(3\)](#), [MinValue \(3\)](#), [MaxValue \(3\)](#), [EnumValues \(3\)](#), [Label \(3\)](#), [Description \(3\)](#)

`add_help_menu_item`

Add an item to the console Help menu and link it to relevant documentation

Syntax

string **add_help_menu_item** *menuText* *helpUrl* [*description*]

string *menuText*

string *helpUrl*

string *description*

Parameters

<code>menuText</code>	The text that will appear in the coreConsultant Help menu.
<code>helpUrl</code>	The URL to display when the user selects this Help menu item.
<code>description</code>	The text to appear in the coreConsultant status bar when this Help menu item is highlighted.

Description

The `add_help_menu_item` command adds a new item to the coreTool Help menu and links the item to a URL that contains the relevant documentation.

When a core integrator installs the coreKit, the new item *<MenuText>* appears in the coreConsultant console Help menu.

When the user selects the *<MenuText>* Help item, coreTool invokes the user's selected web browser to display the document located at *<helpUrl>*. *<helpUrl>* can be the URL of a web site or the post-coreKit-installation location of a file that you include in the Bill of Materials.

The text specified by the *<description>* option appears in the console status bar when the menu option is highlighted.

coreBuilder automatically executes the `add_help_menu_item` command when you execute the Set Up Help Menu activity. A typical use for `add_help_menu_item` is to add Help menu items in batch mode.

As a coreTool user, you can also use `add_help_menu_item` to add a help menu item to your current coreTool session.

Examples

To create a new Help menu item named "Databook" and link it to the file `"/doc/databook.html"` in the user's workspace:

```
coreBuilder> add_help_menu_item Databook {./doc/databook.html} \
{My_Core Databook}
```

To create a new Help menu item named "ReleaseNotes" and link it to the file `"ReleaseNotes.pdf"` in the coreKit installation directory named `"/usr/cores/MyCore/doc"`:

```
coreConsultant> add_help_menu_item ReleaseNotes \
{/usr/cores/MyCore/doc/ReleaseNotes.pdf} \
{My_Core Release Notes}
```

See Also

[add_files_to_filegroup \(2\), HelpUrl \(3\)](#) _____

`add_html_report_link`

Add an HTML report link to the index page

Syntax

```
string add_html_report_link -title <title_str> -description <description>
-link <link>
    string <title_str>
    string <description>
    string <link>
```

Parameters

<code>-title <title_str></code>	The title of the link to add
<code>-description <description></code>	The description of the link
<code>-link <link></code>	The location of report to link to The link should either be an absolute path or be relative to the report directory.

Description

This command is used to add an HTML report to the top level reports table. This table is accessed via the View->Reports menu, or by using a web browser to open the file `<workspace>/report/index.html`. The specified report file is added to the list of available reports, including a brief description of the report, and a link to the actual report file.

Examples

To add a user defined report I/O pad mapping results to the set of top-level reports.

```
prompt> add_html_report_link -link ./report/padreport.html -title "I/O
Pad
Mapping Report" -description "Report indicating results of the I/O
pad mapping activity."
```

See Also

[remove_html_report_link \(2\)](#)

Add hook to execute a Tcl plugin command when `instantiate_component` is completed

Syntax

```
string add_instantiate_component_hook [-connect] command
      string command
```

Parameters

-connect	Execute command before autoconnections (Default behavior is to run the code after autoconnections)
command	Name of the plugin proc to execute as hook

Description

This command is only used by core developer to add a hook into a consultant plugin kb. The hook will be run after a component has been instantiated in coreAssembler. This can be used for adding additional components, automatically exporting interfaces, connecting interfaces, setting configuration values, etc. The hook command must be defined in core plugins. The command must accept three arguments: component name, kit name, and kit version.

Examples

To add a hook to execute a Tcl command 'after_instantiate_ahb' after a DW_ahb component is instantiated in coreAssembler, add the following command into the consultant plugin obj file:

add instantiate component hook after instantiate ahb

See Also

add verify subsystem hook (2), create plugin kb (2)

add_to_collection

Add object(s) to a collection. Result is new collection

Syntax

```
string add_to_collection [-unique] collection1 object_spec
      string collection1
      list object_spec
```

Parameters

-unique	Remove duplicates from the result
collection1	Base collection The add_to_collection command copies the base collection to a new collection, then adds to specified objects to the new collection. The base collection can be the empty collection (empty string). Object(s) to add If the name matches an existing collection, then collection is added to the new collection. Otherwise, add_to_collection searches for the specified objects in the currently loaded knowledge database as if the find_item command had been used to locate the named items.
object_spec	

Description

Because Tcl is a command-driven language, traditional operators like plus (+) and minus (-) have no special meaning unless a particular command (like expr) imposes some meaning. The add_to_collection command allows you to add elements to a collection. The result is a new collection representing the objects in the <object_spec> added to the objects in the base collection.

If an element exists in both the base collection and the <object_spec>, it is duplicated in the resulting collection. Duplicates are not removed unless the unique option is selected. If the <object_spec> is empty, the result is a copy of the base collection.

Examples

To create a collection of all ports that begin with "int", then add the "clk" port:

```
coreConsultant> set intrports [find_item -type port int*]  
{int1_n int3_n int5_n int0_n int2 int4}  
coreConsultant> set intandclk [add_to_collection $intrports  
[find_item -type port clk]  
{int1_n int3_n int5_n int0_n int2 int4 clk}]
```

See Also

all_inputs (2), all_outputs (2), find_item (2), foreach_in_collection
(2), remove_from_collection (2), sizeof_collection (2)

`add_verify_subsystem_hook`

Execute a Tcl checker command to verify subsystem at various places

Syntax

string **add_verify_subsystem_hook** [-before] [-after] *scripts*
string *scripts*

Parameters

-before	Run the hook before component is instantiated(default is to run the hook after built-in verify_subsystem)
-after	Run the hook after component is instantiated (default is to run the hook after built-in verify_subsystem)
scripts	Tcl scripts to execute

Description

The hook will be used to run a 'checker' proc at various places to check the subsystem. The hook can be run before a component is about to be instantiated, after a component is instantiated, or at the end (after coreAssembler built-in verify_subsystem).

The hook proc should return any errors found during verification via the format described below. If the hook is run before or after instantiate_component, errors will be returned. If the hook is run at the end, the errors will be presented to the user in the standard report.

The format of the error is a tcl list in which each element of the list is again a tcl list of 2 elements {componentName error_reason}. The first element is the component name of the erroneous component or interface instance, and the second element is a text string that states the reason for the error. An empty list means no error found in the hook. For example, you can format the following error:

```
set errorList {}
set reason1 "Too few consumer connections: requires2, has 1"
set reason2 "unused exported interface."
lappend errorList "i_ahb/AHB_Slave [list $reason1]"
lappend errorList "ex_AHB_Master [list $reason2]"
```

This command can be used in both coreBuilder and coreAssembler. In coreBuilder, this command is only used to add a hook into a consultant plugin kb. Only '-before' and '-after' can be used to add a hook to a plugin kb. In coreAssembler, the user can add verify_subsystem hooks to run in all three places (-before, -after and at the end).

Examples

To add hook to execute a Tcl command 'verify_before_instantiation' before instantiating a DW_ahb component in coreAssembler, add the following command into the consultant plugin obj file:

```
add_verify_subsystem_hook -before verify_before_instantiation
```

To add hook to execute a Tcl command 'verify_after_instantiation' after a DW_ahb component is instantiated in coreAssembler, add the following command into the consultant plugin obj file:

```
add_verify_subsystem_hook -before verify_after_instantiation
```

In coreAssembler, to add a hook to execute a Tcl command "customized_verify_subsystem" after coreAssembler's built-in verify_subsystem:

```
coreAssembler>add_verify_subsystem_hook customized_verify_subsystem
```

See Also

[add_instantiate_component_hook \(2\)](#), [create_plugin_kb \(2\)](#)

`add_workspace_hook`

Execute a Tcl script when a workspace operation occurs

Syntax

```
string add_workspace_hook [-mode <activityMode>] [-create] [-open]
[-close] [-line_index ] script
    string <activityMode>
    string
    string script
```

Parameters

	Required tool mode for the workspace to run in (default: plug-in required activity mode; consultant) (Values: builder, consultant, developer, assembler) If <code>add_workspace_hook</code> is called during a <code>create_plugin_kb</code> (the calling source code is plug-in source code) then the default value is the required activity mode for the plugin-kb. Otherwise, in case no mode is required, the default mode is consultant. The mode is ignored outside a plug-in environment. In case of a surrounding <code>create_plugin_kb</code> , it is not possible to specify a different mode than the explicitly required plug-in mode.
<code>-mode</code> <code><activityMode></code>	
<code>-create</code>	Run the code when a new workspace is created This option is valid only for plug-in source code.
<code>-open</code>	Run the code when the workspace is opened This option can be used with any workspace for plug-in source code or with the current workspace.
<code>-close</code>	Run the code when the workspace is closed At least one of the options <code>-create</code> , <code>-open</code> , or <code>-close</code> must be present. You can specify the same Tcl script for multiple workspace operations (<code>-options</code>).
<code>-line_index</code>	The line index of existing hooks to insert new script code after.
<code>script</code>	Tcl code to execute

Description

The given Tcl script will be executed when a workspace is created,

opened, or closed, according to which option(s) are selected. This hook either:

- Applies to workspace operations when the hook is part of a plug-in knowledge base, and this knowledge base is loaded when the operation occurs (implicit load via RT_*PLUGINS search path, or explicit load of core specific plug-ins during core installation); or
- Applies to the currently open workspace. In this case only workspace open and close can be hooked. A close-workspace hook will already apply with the close of the current workspace.

The mode specifies to which kind of workspaces, or tool activity mode, this hook applies: coreBuilder, coreConsultant or coreAssembler workspaces. The hook never applies to multiple modes.

RESTRICTIONS

A workspace hook cannot be removed. Therefore a close-workspace hook will always be executed when closing the workspace, not once only for the currently open workspace.

Examples

The context for these examples is a company- or site-specific plug-in. For every new coreConsultant workspace the synthesis setup is copied from the common project setup to the `<workspace>/syn`, and the TechSetup is completed as follows:

```
add_workspace_hook -mode consultant -create {
file copy [file join $::env(PROJECT_ROOT) setup .synopsys_dc.setup]
[file join [get_logical_dir syn] .synopsys_dc.setup]
autocomplete_activity TechSetup }
If code should apply to both opening a new workspace (create) and opening
an existing workspace then you can combine both:
```

```
add_workspace_hook -create -open {
# Check the project environment for changes, e.g. use {exec diff ...}
# to test whether the .synopsys_dc.setup above is updated in the
# PROJECT_ROOT, and in this case copy it again, inform the user and/or
# re-do TechSetup (otherwise a symbolic link would satisfy needs).
# -create is included here: if not exists (-create) or 'exec diff'
# fails (-open) then copy and proceed. }
```

See Also

[create_plugin_kb \(2\)](#), [create_workspace \(2\)](#), [open_workspace \(2\)](#), _____
[close_workspace \(2\)](#), [plugin_proc \(2\)](#), _____

addressBank

This item represents an address bank in a memory map.

Description

An address bank is a container to hold address blocks or address banks into a logical grouping. An address bank defines an alignment that determines how the address the address blocks and banks are organized in the memory map. See the [BankAlignment attribute](#) for details.

See Also

[memMap \(3\)](#), [create_memory_map \(2\)](#), [create_address_bank \(2\)](#),
[create_address_block \(2\)](#), [set_address_bank_attribute \(2\)](#),
[get_address_bank_attribute \(2\)](#)

Supported Attributes

[BankAlignment \(3\)](#), [BaseAddress \(3\)](#), [BitAddressOffset \(3\)](#), [ComponentOfItem \(3\)](#), [Description \(3\)](#), [DocAddressOffset \(3\)](#), [DocBlockTableAddressOffset \(3\)](#), [DocInclude \(3\)](#), [DocMemoryAccess \(3\)](#), [DocRegTableAddressOffset \(3\)](#),
[HeaderNameFormat \(3\)](#), [Label \(3\)](#), [MemoryAccess \(3\)](#),
[MemoryAccessDescription \(3\)](#), [MemoryRange \(3\)](#), [MemoryUsage \(3\)](#),
[MemoryWidth \(3\)](#), [Name \(3\)](#), [Sequence \(3\)](#), [SpiritContainer \(3\)](#), [TypeName \(3\)](#),
[UndefinedBitValue \(3\)](#), [Visible \(3\)](#)

addressBlock

This item represents a block of memory in a memory map.

Description

An address block represents a continuous block of memory with a memory map. The address block also acts as a container for a set of registers in this memory region.

See Also

[memMap \(3\)](#), [create_memory_map \(2\)](#), [create_address_block \(2\)](#),
[create_address_bank \(2\)](#), [create_register \(2\)](#), [set_address_block_attribute](#)
[\(2\)](#), [get_address_block_attribute \(2\)](#)

Supported Attributes

[BaseAddress \(3\)](#), [BitAddressOffset \(3\)](#), [ComponentOfItem \(3\)](#), [Description](#)
[\(3\)](#), [DocAddressOffset \(3\)](#), [DocBlockTableAddressOffset \(3\)](#), [DocGroup \(3\)](#),
[DocInclude \(3\)](#), [DocMaster \(3\)](#), [DocMemoryAccess \(3\)](#),
[DocRegTableAddressOffset \(3\)](#), [DocVisible \(3\)](#), [DocWaiver \(3\)](#), [Endian \(3\)](#),
[GroupImage \(3\)](#), [GroupImageAttrs \(3\)](#), [GroupImageTitle \(3\)](#), [GroupText \(3\)](#),
[HeaderNameFormat \(3\)](#), [Label \(3\)](#), [MemoryAccess \(3\)](#),
[MemoryAccessDescription \(3\)](#), [MemoryRange \(3\)](#), [MemoryUsage \(3\)](#),
[MemoryWidth \(3\)](#), [Name \(3\)](#), [Sequence \(3\)](#), [SpiritContainer \(3\)](#), [TypeName](#)
[\(3\)](#), [UndefinedBitValue \(3\)](#), [Visible \(3\)](#)

NAME

`after` - Execute a command after a time delay

SYNOPSIS

`after` *ms*

`after` *ms* *?script script script ...?*

`after` `cancel` *id*

`after` `cancel` *script script script ...*

`after` `idle` *?script script script ...?*

`after` `info` *?id?*

DESCRIPTION

This command is used to delay execution of the program or to execute a command in background sometime in the future. It has several forms, depending on the first argument to the command:

`after` *ms*

Ms must be an integer giving a time in milliseconds. The command sleeps for *ms* milliseconds and then returns. While the command is sleeping the application does not respond to events.

`after` *ms* *?script script script ...?*

In this form the command returns immediately, but it arranges for a Tcl command to be executed *ms* milliseconds later as an event handler. The command will be executed exactly once, at the given time. The delayed command is formed by concatenating all the *script* arguments in the same fashion as the `concat`

coreTools Command Reference Index

command. The command will be executed at global level (outside the context of any Tcl procedure). If an error occurs while executing the delayed command then the background error will be reported by the command registered with **interp bgerror**. The **after** command returns an identifier that can be used to cancel the delayed command using **after cancel**.

after cancel *id*

Cancels the execution of a delayed command that was previously scheduled. *Id* indicates which command should be canceled; it must have been the return value from a previous **after** command. If the command given by *id* has already been executed then the **after cancel** command has no effect.

after cancel *script script ...*

This command also cancels the execution of a delayed command. The *script* arguments are concatenated together with space separators (just as in the **concat** command). If there is a pending command that matches the string, it is canceled and will never be executed; if no such command is currently pending then the **after cancel** command has no effect.

after idle *script ?script script ...?*

Concatenates the *script* arguments together with space separators (just as in the **concat** command), and arranges for the resulting script to be evaluated later as an idle callback. The script will be run exactly once, the next time the event loop is entered and there are no events to process. The command returns an identifier that can be used to cancel the delayed command using **after cancel**. If an error occurs while executing the script then the background error will be reported by the command registered with **interp bgerror**.

after info *?id?*

This command returns information about existing event handlers. If no *id* argument is supplied, the command returns a list of the identifiers for all existing event handlers created by the **after** command for this interpreter. If *id* is supplied, it specifies an existing handler; *id* must have been the return value from some previous call to **after** and it must not have triggered yet or been canceled. In this case the command returns a list with two elements. The first element of the list is the script associated with *id*, and the second element is either **idle** or **timer** to indicate what kind of event handler it is.

The **after ms** and **after idle** forms of the command assume that the application is event driven: the delayed commands will not be executed unless the application enters the event loop. In applications that are not normally event-driven, such as **tclsh**, the event loop can be entered with the **vwait** and **update** commands.

EXAMPLES

This defines a command to make Tcl do nothing at all for **N** seconds:

```
proc sleep {N} {
    after [expr {int($N * 1000)}]
}
```

This arranges for the command *wake_up* to be run in eight hours (providing the event loop is active at that time):

```
after [expr {1000 * 60 * 60 * 8}] wake_up
```

The following command can be used to do long-running calculations (as represented here by *::my_calc::one_step*, which is assumed to return a boolean indicating whether another step should be performed) in a step-by-step fashion, though the calculation itself needs to be arranged so it can work step-wise. This technique is extra careful to ensure that the event loop is not starved by the rescheduling of processing steps (arranging for the next step to be done using an already-triggered timer event only when the event queue has been drained) and is useful when you want to ensure that a Tk GUI remains responsive during a slow task.

```
proc doOneStep {} {
    if {[:my_calc::one_step]} {
        after idle [list after 0 doOneStep]
    }
}
doOneStep
```

SEE ALSO

concat(n), interp(n), update(n), vwait(n)

KEYWORDS

cancel, delay, idle callback, sleep, time

NAME

alias Creates a pseudo-command that expands to one or more words, or lists current alias definitions.

SYNTAX

```
string alias [name] [def]
```

Data Types

<i>name</i>	string
<i>def</i>	string

ARGUMENTS

name Specifies a name of the alias to define or display. The name must begin with a letter, and can contain letters, underscores, and numbers.

def Expands the alias. That is, the replacement text for the alias name.

DESCRIPTION

The **alias** command defines or displays command aliases. With no arguments, the **alias** command displays all currently defined aliases and their expansions. With a single argument, the **alias** command displays the expansion for the given alias name. With more than one argument, an alias is created that is named by the first argument and expanding to the remaining arguments.

You cannot create an alias using the name of any existing command or procedure. Thus, you cannot use **alias** to redefine existing commands.

Aliases can refer to other aliases.

coreTools Command Reference Index

Aliases are only expanded when they are the first word in a command.

EXAMPLES

Although commands can be abbreviated, sometimes there is a conflict with another command. The following example shows how to use **alias** to get around the conflict:

```
prompt> alias q quit
```

The following example shows how to use **alias** to create a shortcut for commonly-used command invocations:

```
prompt> alias include {source -echo -verbose}

prompt> alias rt100 {report_timing -max_paths 100}
```

After the previous commands, the command **include script.tcl** is replaced with **source -echo -verbose script.tcl** before the command is interpreted.

The following examples show how to display aliases using **alias**. Note that when displaying all aliases, they are in alphabetical order.

```
prompt> alias rt100
rt100          report_timing -max_paths 100

prompt> alias
include        source -echo -verbose
q              quit
rt100          report_timing -max_paths 100
```

SEE ALSO

`unalias(2)`

`all_components`

Return sorted list of all currently loaded components in the sub-system.

Syntax

string **all_components** [-recursive] [-imported] [-instantiated]
[-duplicates_of] [-unfolded]

Parameters

-recursive	All components below this level of hierarchy
-imported	Select imported components only.
-instantiated	Select instantiated components only.
-duplicates_of	Return the shared components.
-unfolded	Return components as if no sharing has occurred.

Description

This command can be used in coreAssembler to create a list of the names of all components currently contained in the subsystem. This can be useful when writing procedures which need to iterate over all subsystem components. The names returned are those of the cells in the top-level design (i.e. the instance names passed to the `instantiate_component` command). The result of this command is typically used to perform an action within each component context.

If any action is limited to packaged instantiated components, then you can use option `-instantiated`. You can use option `-imported` to filter for unpackaged imported components.

Examples

The following code fragment could be used to list the clocks defined within each component in the subsystem:

```
foreach component [all_components] {set_current_component $component \
list_item [get_clocks] }
```

See Also

[instantiate_component\(2\)](#), [import_component\(2\)](#), [set_current_component\(2\)](#), [get_current_component\(2\)](#)

`all_designs`

Create a collection of all currently loaded designs

Syntax

string **`all_designs`**

Description

The `all_designs` command returns a collection of all currently loaded designs. The most common usage for `all_designs` is in a `coreBuilder` intent command file to set a common attribute on all designs or to iterate over all designs.

Examples

To set the `PreserveHierarchy` attribute to true on all currently loaded designs:

```
coreBuilder> set_design_attribute -designs [all_designs]  
PreserveHierarchy "true"
```

See Also

[`all\_subdesigns` \(2\)](#)

`all_inputs`

Return a collection of all input and inout ports of the current_design

Syntax

string **all_inputs** [-no_clocks] [-no_resets] [-no_ideals] [-no_primary]

Parameters

<code>-no_clocks</code>	Do not return clock ports.
<code>-no_resets</code>	This option is obsolete. <code>-no_ideals</code> is implied.
<code>-no_ideals</code>	Do not return any ideal ports.
<code>-no_primary</code>	Do not return ports that are electrically connected to the design's primary input/output ports.

Description

The `all_inputs` command returns a collection of all input ports of the current design that meet the criteria specified by the `all_inputs` command options.

If you do not include any of the available options, `all_inputs` returns a collection that includes all ports of direction in and inout. The `-no_clocks`, `-no_ideals`, and `-no_primary` options further reduce the number of ports returned in the collection.

The `-no_primary` option is useful for setting default attribute values on ports that are not directly connected to the top level ports. Top-level ports typically have more accurate intent specified for them and that data will implicitly be pushed down onto the electrically connected ports.

The `-no_clocks` and `-no_ideals` options exclude ports that have been identified as clock and ideal ports, respectively.

Examples

To apply an attribute to all input ports except clocks:

```
coreBuilder> set_port_attribute [all_inputs -no_clocks] DrivingCell
{=select_by_class "combinational"}
```

To apply default input delay to non-clock inputs that are not connected to primary inputs:

```
coreBuilder> set_port_attribute [all_inputs -no_clocks -no_primary]
InputDelay 10ns
```


See Also

[all_outputs \(2\)](#)

all_outputs

Return a collection of all output and inout ports of the current_design

Syntax

string **all_outputs** [-no_clocks] [-no_primary]

Parameters

-no_clocks	Do not return clock ports.
-no_primary	Do not return ports that are electrically connected to the design's primary input/output ports.

Description

The all_outputs command returns a collection of all output ports of the current design that meet the criteria specified by the all_outputs command options.

If you do not include any of the available options, all_outputs returns a collection that includes all ports of direction out and inout. The -no_clocks and -no_primary options further reduce the number of ports returned in the collection.

The -no_primary option is useful for setting default attribute values on ports that are not directly connected to the top level ports. Top-level ports typically have more accurate intent specified for them and that data will implicitly be pushed down onto the electrically connected ports.

The -no_clocks option excludes ports that have been identified as clock ports.

Examples

To apply an attribute to all output ports of the current_design:

```
coreBuilder> set_port_attribute [all_outputs] OutputDelay  
{=percent_of_period 25.0}
```

To apply default output delay to non-clock outputs that are not connected to primary outputs:

```
coreBuilder> set_port_attribute [all_outputs -no_clocks -no_primary]  
OutputDelay 10ns
```

See Also

[all_inputs \(2\)](#)

`all_subdesigns`

Return a collection of all subdesigns of the `current_design`

Syntax

string **all_subdesigns** [-recursive] [*design*]
 string *design*

Parameters

<code>-recursive</code>	Recursively traverse down to the leaf-level hierarchy and return all designs found.
-------------------------	---

Description

The `all_subdesigns` command returns a collection of the designs contained within the `current_design`, that is, instantiated in the current design.

If you specify the `-recursive` option, `all_subdesigns` returns all designs contained within the designs within the `current_design` (and so on, recursively) as part of the collection.

Examples

To set `PreserveHierarchy` to true on all subdesigns of the `current_design`:

```
coreBuilder> set_design_attribute -designs [all_subdesigns]  
PreserveHierarchy true
```

See Also

[all_designs \(2\)](#)

analyze_filegroup

Source intent files for this filegroup.

Syntax

```
string analyze_filegroup [-intent_path <path list>] group
      string <path list>
      string group
```

Parameters

-intent_path	Override the default ConfigIntentSearchPath, forces Configurable to true.
<path list>	This option is required if the filegroup is not already marked as Configurable(3).
group	BOM File group

Description

This command is used to analyze configurable filegroups. A configurable filegroup uses the text substitution mechanism to change the contents of certain files in the group. The filegroup is configured when the ConfigActivity(3) is completed during the coreConsultant flow.

A configurable filegroup may also be an autoloading filegroup. If so, then this command runs the load_autoload_filegroup(2) command prior to analyzing the filegroup.

This command looks at every file in the filegroup, and tries to find another file in the ConfigIntentSearchPath(3) that has the same basename, with a '.tcl' extension. If a file is found that meets these criteria, then it is sourced. These files generally have create_configuration_parameter(2) commands in them.

If this filegroup is configured by an existing activity in the coreConsultant flow, then the ConfigIntentSearchPath(3) should be empty. When intent files are found, and parameters are created for the filegroup, an activity will be automatically created when the coreConsultant creates a workspace. This is an alternative method to create an activity for the coreConsultant flow (see documentation on the LoadConsultantPlugins activity for the other method).

Note: When the BuildcoreKit activity is completed in the coreBuilder flow, all Configurable(3) filegroups that have not been analyzed will be analyzed automatically at that time. It is rarely necessary to run this

command yourself, unless you would like to edit the configuration parameters in GUI.

Examples

To create documentation that is configured based on the users selected implementation run the following commands:

```
coreBuilder> create_autoload_filegroup -patterns doc/* -install doc
Documentation
```

```
coreBuilder> set_filegroup_attribute Documentation Configurable
```

```
coreBuilder> set_filegroup_attribute Documentation ConfigActivity
SpecifyConfiguration
```

To force the analyze command (that is, to determine which files are configurable) before the BuildcoreKit activity is completed, run the following command:

```
coreBuilder> analyze_filegroup Documentation
```

To create a filegroup that is configured by a new activity in the coreConsultant flow, run the following commands:

```
coreBuilder> add_files_to_filegroup TestBench -files tb/Makefile
-install tb
```

```
coreBuilder> set_filegroup_attribute TestBench Configurable
```

```
coreBuilder> set_filegroup_attribute TestBench ConfigActivity
ChooseSimulator
```

```
coreBuilder> set_filegroup_attribute TestBench ConfigIntentSearchPath
tb/intent
```

```
coreBuilder> analyze_filegroup TestBench
```

The file tb/intent/Makefile.tcl will be sourced during the analyze_filegroup command.

See Also

[load_autoload_filegroup \(2\)](#) [create_configuration_parameter \(2\)](#)

[ReplaceFormatParam \(2\)](#) [Configurable \(3\)](#) [ConfigIntentSearchPath \(3\)](#)

[ConfigActivity \(3\)](#)

NAME

annoate_trace Control annotation of traced command execution to the shell output.

SYNTAX

```

annotate_trace [-start|-stop]
                  [-profile profile_annotation_type]
                  [-log log_string]
                  [-quiet]

```

string *log_string* string *profile_annotation_type*

ARGUMENTS

-start This option is mutually exclusive with **-stop**. The option starts the annotation of traced command execution to the shell output. Each executed command string is annotated to the output prepended with ";## ".

-stop This option is mutually exclusive with **-start**. This option stops annotation of traced command execution to the shell output.

-profile *profile_annotation_type*
 This option selects the profile data output behavior for command annotation. This option is only meaningful when given with the **-start** option. The argument is one of: *on*, *with_start*, *off*. The */fI**on*/*fP* value starts annotation with profile data output enabled in the default mode which is to output the data only at the end of a command invocation. The data is output only if the delta values from the beginning of the command invocation meet or exceed the thresholds set for each of the reported metrics. The */fI**with_start*/*fP* value starts annotation with profile data output both at the beginning and at the end of a

coreTools Command Reference Index

command invocation. The profile data is output at the beginning irregardless of any thresholds since delta values cannot be calculated at the beginning of the invocation. The profile data output at the end of command invocation is controlled by thresholds. The /fIoff/fP value disables profile data output. If profile data annotation is enabled but profile metric gathering has not been enabled with `set_trace_option`, all profile metrics are auto-enabled. If omitted, annotation is started without profile data output.

`-log log_string`

This option outputs the given `log_string` to the shell output prepended with `";## "`. If the given string contains newline characters, each line is prepended with `";## "`.

`-quiet`

If given, this option causes the command to ignore error conditions such as an attempt to log a comment string before annotation is started. The command will exit quietly when it encounters an error condition.

DESCRIPTION

The **`annotate_trace`** command performs operations related to annotating traced command execution to the shell output. If the application creates an output log, the annotation is made also to the output log. The command strings which are annotated to the output are those that are traced for creating a traced command replay log.

The **`annotate_trace`** with no options will report on the current status of annotation: on or off.

EXAMPLES

The following example starts annotation of executed commands to the output. Then the command is invoked again with no options to report on the status.

```
prompt> annoate_trace -start
prompt> annoate_trace
on
```


coreTools Command Reference Index

The following example configures profile metrics, then starts annotation of executed commands to the output along with the profile data.

```
prompt> set_trace_topion -profile all
prompt> set_trace_topion -memory_threshold 10
prompt> set_trace_topion -cpu_threshold 0
prompt> annotate_trace -start -profile with_start
```

SEE ALSO

```
set_trace_option(2)
get_trace_option(2)
log_trace(2)
```

NAME

`append` - Append to variable

SYNOPSIS

`append` *varName* *?value value value ...?*

DESCRIPTION

Append all of the *value* arguments to the current value of variable *varName*. If *varName* does not exist, it is given a value equal to the concatenation of all the *value* arguments. The result of this command is the new value stored in variable *varName*. This command provides an efficient way to build up long variables incrementally. For example, is much more efficient than if `$a` is long.

EXAMPLE

Building a string of comma-separated numbers piecemeal using a loop.

```
set var 0
for {set i 1} {$i<=10} {incr i} {
    append var "," $i
}
puts $var
# Prints 0,1,2,3,4,5,6,7,8,9,10
```

SEE ALSO

`concat(n)`, `lappend(n)`

KEYWORDS

`append`, `variable`

append_to_collection

Add object(s) to a collection. Modifies variable

Syntax

```
string append_to_collection [-dict dict_var] [-unique] var_name
object_spec
    string dict_var
    string var_name
    list object_spec
```

Parameters

-dict <i>dict_var</i>	Dict to update
-unique	Remove duplicates from the result
<i>var_name</i>	Variable holding collection
<i>object_spec</i>	Object(s) to add

Description

The `append_to_collection` command allows you to add elements to a collection. This command treats the variable name given by *var_name* as a collection, and appends all of the elements in *object_spec* to that collection. If the variable does not exist, it is created as a collection with elements from *object_spec* as its value. If the variable does exist, and it does not contain a collection, it is an error.

The result of the command is the new value of *var_name*.

The `append_to_collection` command provides the same semantics as a common use of `add_to_collection` but with a significant improvement in performance. An example of replacing `add_to_collection` with `append_to_collection` is given below.

```
set var_name [add_to_collection $var_name $objs]
```

Using `add_to_collection` this becomes:

```
append_to_collection var_name $objs
```

The `append_to_collection` command can be much more efficient than `add_to_collection` if you are building up a collection in a loop. The arguments of the command have the same restrictions as the `add_to_collection` command. Please see the `add_to_collection` man page for more information about those restrictions.

Examples

To create a collection of all ports that begin with "int", then add the "clk" port:

```
coreConsultant> set intrports [find_item -type port int*]
{int1_n int3_n int5_n int0_n int2 int4}
coreConsultant> append_to_collection intrports [find_item -type port clk]
{int1_n int3_n int5_n int0_n int2 int4 clk}
```

See Also

find_item (2), add_to_collection (2), foreach_in_collection (2),
index_collection (2), remove_from_collection (2), sizeof_collection (2).

NAME

`apply` - Apply an anonymous function

SYNOPSIS

`apply func ?arg1 arg2 ...?`

DESCRIPTION

The command **apply** applies the function *func* to the arguments *arg1 arg2 ...* and returns the result.

The function *func* is a two element list *{args body}* or a three element list *{args body namespace}* (as if the **list** command had been used). The first element *args* specifies the formal arguments to *func*. The specification of the formal arguments *args* is shared with the **proc** command, and is described in detail in the corresponding manual page.

The contents of *body* are executed by the Tcl interpreter after the local variables corresponding to the formal arguments are given the values of the actual parameters *arg1 arg2* When *body* is being executed, variable names normally refer to local variables, which are created automatically when referenced and deleted when **apply** returns. One local variable is automatically created for each of the function's arguments. Global variables can only be accessed by invoking the **global** command or the **upvar** command. Namespace variables can only be accessed by invoking the **variable** command or the **upvar** command.

The invocation of **apply** adds a call frame to Tcl's evaluation stack (the stack of frames accessed via **uplevel**). The execution of *body* proceeds in this call frame, in the namespace given by *namespace* or in the

coreTools Command Reference Index

global namespace if none was specified. If given, *namespace* is interpreted relative to the global namespace even if its name does not start with

The semantics of **apply** can also be described by:

```
proc apply {fun args} {
  set len [llength $fun]
  if {($len < 2) || ($len > 3)} {
    error "can't interpret \"$fun\" as anonymous function"
  }
  lassign $fun argList body ns
  set name ::$ns::[getGloballyUniqueName]
  set body0 {
    rename [lindex [info level 0] 0] {}
  }
  proc $name $argList ${body0}$body
  set code [catch {uplevel 1 $name $args} res opt]
  return -options $opt $res
}
```

EXAMPLES

This shows how to make a simple general command that applies a transformation to each element of a list.

```
proc map {lambda list} {
  set result {}
  foreach item $list {
    lappend result [apply $lambda $item]
  }
  return $result
}
map {x {return [string length $x]:$x}} {a bb ccc dddd}
-> 1:a 2:bb 3:ccc 4:dddd
map {x {expr {$x**2 + 3*$x - 2}}}{-4 -3 -2 -1 0 1 2 3 4}
-> 2 -2 -4 -4 -2 2 8 16 26
```

The **apply** command is also useful for defining callbacks for use in the **trace** command:

```
set vbl "123abc"
trace add variable vbl write {apply {{v1 v2 op} {
  upvar 1 $v1 v
  puts "updated variable to \"$v\""}
}}}
set vbl 123
set vbl abc
```

SEE ALSO

proc(n), uplevel(n)

KEYWORDS

anonymous function, argument, lambda, procedure,

NAME

apropos Searches the command database for a pattern.

SYNTAX

```
string apropos
      [-symbols_only]
      pattern
```

Data Types

<i>pattern</i>	string
----------------	--------

ARGUMENTS

-symbols_only Searches only command and option names.

pattern Searches for the specified *pattern*.

DESCRIPTION

The **apropos** command searches the command and option database for all commands that contain the specified *pattern*. The *pattern* argument can include the wildcard characters asterisk (*) and question mark (?). The search is case-sensitive. For each command that matches the search criteria, the command help is printed as though **help -verbose** was used with the command.

Whereas **help** looks only at command names, **apropos** looks at command names, the command one-line description, option names, and option value-help strings. The search can be restricted to only command and option names with the **-symbols_only** option.

When searching for dash options, do not include the leading dash. Search only for the name.

EXAMPLES

In the following example, assume that the **get_cells** and **get_designs** commands have the **-exact** option. Note that without the **-symbols_only** option, the first search picks up commands which have the string "exact" in the one-line description.

```
prompt> apropos exact
get_cells          # Create a collection of cells
  [-exact]         (Wildcards are considered as plain characters)
  patterns         (Match cell names against patterns)

get_designs        # Create a collection of designs
  [-exact]         (Wildcards are considered as plain characters)
  patterns         (Match design names against patterns)

real_time          # Return the exact time of day

prompt> apropos exact -symbols_only
get_cells          # Create a collection of cells
  [-exact]         (Wildcards are considered as plain characters)
  patterns         (Match cell names against patterns)

get_designs        # Create a collection of designs
  [-exact]         (Wildcards are considered as plain characters)
  patterns         (Match design names against patterns)
```

SEE ALSO

help(2)

NAME

`array` - Manipulate array variables

SYNOPSIS

`array option arrayName ?arg arg ...?`

DESCRIPTION

This command performs one of several operations on the variable given by *arrayName*. Unless otherwise specified for individual commands below, *arrayName* must be the name of an existing array variable. The *option* argument determines what action is carried out by the command. The legal *options* (which may be abbreviated) are:

array anymore *arrayName searchId*

Returns 1 if there are any more elements left to be processed in an array search, 0 if all elements have already been returned. *SearchId* indicates which search on *arrayName* to check, and must have been the return value from a previous invocation of **array startsearch**. This option is particularly useful if an array has an element with an empty name, since the return value from **array nextelement** will not indicate whether the search has been completed.

array donesearch *arrayName searchId*

This command terminates an array search and destroys all the state associated with that search. *SearchId* indicates which search on *arrayName* to destroy, and must have been the return value from a previous invocation of **array startsearch**. Returns an empty string.

array exists *arrayName*

coreTools Command Reference Index

Returns 1 if *arrayName* is an array variable, 0 if there is no variable by that name or if it is a scalar variable.

array get *arrayName* ?*pattern*?

Returns a list containing pairs of elements. The first element in each pair is the name of an element in *arrayName* and the second element of each pair is the value of the array element. The order of the pairs is undefined. If *pattern* is not specified, then all of the elements of the array are included in the result. If *pattern* is specified, then only those elements whose names match *pattern* (using the matching rules of **string match**) are included. If *arrayName* is not the name of an array variable, or if the array contains no elements, then an empty list is returned. If traces on the array modify the list of elements, the elements returned are those that exist both before and after the call to **array get**.

array names *arrayName* ?*mode*? ?*pattern*?

Returns a list containing the names of all of the elements in the array that match *pattern*. *Mode* may be one of **-exact**, **-glob**, or **-regexp**. If specified, *mode* designates which matching rules to use to match *pattern* against the names of the elements in the array. If not specified, *mode* defaults to **-glob**. See the documentation for **string match** for information on glob style matching, and the documentation for **regexp** for information on regexp matching. If *pattern* is omitted then the command returns all of the element names in the array. If there are no (matching) elements in the array, or if *arrayName* is not the name of an array variable, then an empty string is returned.

array nextelement *arrayName* *searchId*

Returns the name of the next element in *arrayName*, or an empty string if all elements of *arrayName* have already been returned in this search. The *searchId* argument identifies the search, and must have been the return value of an **array startsearch** command. Warning: if elements are added to or deleted from the array, then all searches are automatically terminated just as if **array donesearch** had been invoked; this will cause **array nextelement** operations to fail for those searches.

array set *arrayName* *list*

Sets the values of one or more elements in *arrayName*. *list* must have a form like that returned by **array get**, consisting of an even number of elements. Each odd-numbered element in *list* is treated as an element name within *arrayName*, and the following element in *list* is used as a new value for that array element. If the variable *arrayName* does not already exist and *list* is empty, *arrayName* is created with an empty array value.

array size *arrayName*

Returns a decimal string giving the number of elements in the array. If *arrayName* is not the name of an array then 0 is returned.

coreTools Command Reference Index

array startsearch *arrayName*

This command initializes an element-by-element search through the array given by *arrayName*, such that invocations of the **array nextelement** command will return the names of the individual elements in the array. When the search has been completed, the **array donesearch** command should be invoked. The return value is a search identifier that must be used in **array nextelement** and **array donesearch** commands; it allows multiple searches to be underway simultaneously for the same array. It is currently more efficient and easier to use either the **array get** or **array names**, together with **foreach**, to iterate over all but very large arrays. See the examples below for how to do this.

array statistics *arrayName*

Returns statistics about the distribution of data within the hashtable that represents the array. This information includes the number of entries in the table, the number of buckets, and the utilization of the buckets.

array unset *arrayName* *?pattern?*

Unsets all of the elements in the array that match *pattern* (using the matching rules of **string match**). If *arrayName* is not the name of an array variable or there are no matching elements in the array, no error will be raised. If *pattern* is omitted and *arrayName* is an array variable, then the command unsets the entire array. The command always returns an empty string.

EXAMPLES

```
array set colorcount {
    red    1
    green  5
    blue   4
    white  9
}

foreach {color count} [array get colorcount] {
    puts "Color: $color Count: $count"
}
-> Color: blue Count: 4
   Color: white Count: 9
   Color: green Count: 5
   Color: red Count: 1

foreach color [array names colorcount] {
    puts "Color: $color Count: $colorcount($color)"
}
-> Color: blue Count: 4
   Color: white Count: 9
   Color: green Count: 5
   Color: red Count: 1

foreach color [lsort [array names colorcount]] {
    puts "Color: $color Count: $colorcount($color)"
}
```

coreTools Command Reference Index

```
-> Color: blue Count: 4  
    Color: green Count: 5  
    Color: red Count: 1  
    Color: white Count: 9
```

array statistics colorcount

```
-> 4 entries in table, 4 buckets  
    number of buckets with 0 entries: 1  
    number of buckets with 1 entries: 2  
    number of buckets with 2 entries: 1  
    number of buckets with 3 entries: 0  
    number of buckets with 4 entries: 0  
    number of buckets with 5 entries: 0  
    number of buckets with 6 entries: 0  
    number of buckets with 7 entries: 0  
    number of buckets with 8 entries: 0  
    number of buckets with 9 entries: 0  
    number of buckets with 10 or more entries: 0  
    average search distance for entry: 1.2
```

SEE ALSO

list(n), string(n), variable(n), trace(n), foreach(n)

KEYWORDS

array, element names, search

[API Index](#) [Command Reference](#)

as_collection

Find a collection or iter_collection by name; result a collection

Syntax

```
string as_collection [-check] collection1  
      string collection1
```

Parameters

-check	Return 1 if collection1 is a collection
collection1	The collection to convert

Description

Examples

See Also

attach_interface

Attach a component interface to another component.

Syntax

string **attach_interface** -to <componentName> -name <newName> [-component <componentName>] -interface <instanceName> [-monitor] [-format <format>] [-channel]

string <componentName>

string <newName>

string <instanceName>

string <format>

Parameters

-to <componentName>	The component the interface will be attached to
-name <newName>	New attached interface instance name
-component <componentName>	The component from which the interface is used
-interface <instanceName>	The opposite instance for the new attached instance
-monitor	Attach as consumer-monitor or interface-monitor
-format <format>	The format to use for prefixing exported interfaces.
-channel	Symmetric SPIRIT interface to be connected to a channel

Description

Use this command to attach and connect an existing component interface to another component. In almost all cases the destination component is an imported component which starts without its own interfaces. In the rare case that some kind of interface is missing on a component during subsystem assembly, the interfaces could attach to regular instantiated components.

Attached interfaces are siblings of the exported interfaces: You could treat exported interfaces as interfaces which are attached to "the system," and source components "export" their interfaces to other components.

When an interface is attached to a component it is then addressed like a regular component interface. You can connect it to other interfaces, export it and even attach it a third component (if you find a need for this).

There is one important exception: Even as you address attached interfaces as `<compName>/<instName>`, or `-component <compName> -interface <instName>`, they share the name space with exported interfaces. "Attach" implies that coreAssembler treats it as if it belonged to the attached component although it is still "outside": The `-name` option must identify a unique name for all attached and exported interfaces. As a consequence, if an unpackaged component is imported twice and requires the same interfaces, then the two instances need different names for their attached interfaces.

Like regular component interfaces, the interface ports and parameters must be associated with design ports (and in rare cases with design parameters) of the component.

After creating the new attached interface, the attached interface and the source component interface are connected.

Even exported interfaces can connect and attach to a component: For this case option `-component` is optional, or you can use `-component ""`.

The attached interface is always the opposite interface to the component interface:

- A component provider attaches as consumer.
- A component consumer or internal-consumer attaches as provider.
- A component consumer-monitor attaches as consumer.
- A component interface-monitor attaches as provider.

If option `-monitor` is used then the attached interface becomes a monitor interface:

- A component provider with option `-monitor` attaches as interface-monitor.
- A component consumer with option `-monitor` attaches as consumer-monitor.

It is an error if option `-monitor` is used for other component interface types.

Examples

To attach an APB slave from the APB bridge (component `U_APB`) to the imported component `MyGPIO`:

coreTools Command Reference Index

`attach_interface -to MyGPIO -name GPIOslave -component U_APB -interface APB_Slaves`

To attach the system clock, which was already exported from U_APB, to the above component:

`attach_interface -to MyGPIO -name GPIOclock -interface APB_Clock`

If the instantiated component Timer with interrupt output already has an interface connection to the interrupt controller (provider) and the second CPU has a second interrupt controller which must be connected to the same interrupt output (consumer), connect it as an attachment and copy the port association:

`attach_interface -to Timer -name Intr2 -component Ictl2 -interface IRQ`

`set_interface_port_attribute Timer/Intr2 irq`

`InterfaceLink [get_interface_port_attribute Timer/Intr irq InterfaceLink]`

See Also

`detach_interface (2)`, `export_interface (2)`, `connect_interface (2)`, `import_component (2)`, `InterfaceLink (3)`

[API Index](#) [Command Reference](#)

attrDefn

Definition of an attribute

Description

The attrDefn item represents the definition of a coreTool attribute. For example, the MinInputDelay item is the definition of the MinInputDelay attribute.

See Also

Supported Attributes

[AltValue \(3\)](#), [ComponentOfItem \(3\)](#), [DefaultValue \(3\)](#), [Description \(3\)](#),
[Label \(3\)](#), [MaxValue \(3\)](#), [MinValue \(3\)](#), [Name \(3\)](#), [Sequence \(3\)](#),
[SymbolicNames \(3\)](#), [TypeName \(3\)](#), _____

autocomplete_activity
Automatically complete activities

Syntax

```
string autocomplete_activity [-recur] [-enable] [-component <component
name>] [-nosave] [-quiet] [<activity>]
    string <component name>
    list <activity>
```

Parameters

-recur	Recursively auto-complete dependent activities.
-enable	Enable the specified activity by auto-completing all prerequisite activities. Component activity is associated with (cA only). This option can be specified in coreAssembler to ensure that a component specific activity is completed for the proper component. This is only required when a component-specific activity with the same exists for two different components. This would typically only occur when a given component is instantiated more than once in the same subsystem.
-component <component name>	
-nosave	Dont call save_workspace on completion.
-quiet	Do not print messages.
<activity>	One or more activities to be completed or enabled.

Description

The autocomplete_activity command automatically completes one or more design flow activities. When invoked with no arguments, autocomplete_activity attempts to complete all enabled activities that are not currently completed. When invoked with the -recur option, autocomplete_activity attempts to complete the specified activity and all activities that depend on the specified activity.

When invoked with the -enable option, autocomplete_activity enables the specified activity by auto-completing all prerequisite activities, but does not auto-complete the specified activity.

To list the design flow activities for the currently loaded workspace and their current status (complete or enabled), use the report_activities command.

Examples

To complete all activities that are enabled but not complete:

```
coreConsultant> autocomplete_activity
```

To complete the VerifyBudgets activity:

```
coreConsultant> autocomplete_activity VerifyBudgets
```

To complete VerifyBudgets and any activities that depend on VerifyBudgets:

```
coreConsultant> autocomplete_activity VerifyBudgets -recur
```

To enable VerifyBudgets by completing all prerequisite activities:

```
coreConsultant> autocomplete_activity -enable VerifyBudgets
```

See Also

[set_activity_incomplete \(2\), report_activities \(2\)](#)

batch_install

Install a packaged coreKit into the specified installation root directory.

Syntax

```
string batch_install [-agree] coreKit installPath
      string coreKit
      string installPath
```

Parameters

-agree	Explicitly accept license agreement.
coreKit	Full pathname of the coreKit file (or KB for old-style kits).
installPath	Where to install the coreKit.

Description

The **batch_install** command performs a batch-mode installation of a coreKit. **coreConsultant** reads the specified coreKit file (*<name>.coreKit*) and installs the coreKit into the specified directory. For coreKits created with versions of **coreBuilder** older than 1.1, the installation KB (*Install.kb*) should be specified instead.

If the coreKit has a license agreement that you must accept, you can specify the **-agree** option to automatically accept the license agreement so that **coreConsultant** does not display the license agreement and prompt you to accept it. You should read the license agreement at least once before you use the **-agree** option.

Examples

To install a coreKit from */path/to/Builder1/install/coreA.coreKit* into *./coreA/install* and accept the license agreement:

```
coreConsultant> batch_install -agree
/path/to/Builder1/install/coreA.coreKit ./coreA/install
To invoke coreConsultant in shell mode, install old-style coreKit, then
quit:
```

```
% coreConsultant -shell -x "batch_install -agree
/path/to/coreA/Install.kb ./coreA/install; exit"
```

See Also

NAME

`bgerror` - Command invoked to process background errors

SYNOPSIS

`bgerror` *message*

DESCRIPTION

Release 8.5 of Tcl supports the **interp bgerror** command, which allows applications to register in an interpreter the command that will handle background errors in that interpreter. In older releases of Tcl, this level of control was not available, and applications could control the handling of background errors only by creating a command with the particular command name **bgerror** in the global namespace of an interpreter. The following documentation describes the interface requirements of the **bgerror** command an application might define to retain compatibility with pre-8.5 releases of Tcl. Applications intending to support only Tcl releases 8.5 and later should simply make use of **interp bgerror**.

The **bgerror** command does not exist as built-in part of Tcl. Instead, individual applications or users can define a **bgerror** command (e.g. as a Tcl procedure) if they wish to handle background errors.

A background error is one that occurs in an event handler or some other command that did not originate with the application. For example, if an error occurs while executing a command specified with the **after** command, then it is a background error. For a non-background error, the error can simply be returned up through nested Tcl command evaluations until it reaches the top-level code in the application; then the

coreTools Command Reference Index

application can report the error in whatever way it wishes. When a background error occurs, the unwinding ends in the Tcl library and there is no obvious way for Tcl to report the error.

When Tcl detects a background error, it saves information about the error and invokes a handler command registered by **interp bgerror** later as an idle event handler. The default handler command in turn calls the **bgerror** command. Before invoking **bgerror**, Tcl restores the **errorInfo** and **errorCode** variables to their values at the time the error occurred, then it invokes **bgerror** with the error message as its only argument. Tcl assumes that the application has implemented the **bgerror** command, and that the command will report the error in a way that makes sense for the application. Tcl will ignore any result returned by the **bgerror** command as long as no error is generated.

If another Tcl error occurs within the **bgerror** command (for example, because no **bgerror** command has been defined) then Tcl reports the error itself by writing a message to stderr.

If several background errors accumulate before **bgerror** is invoked to process them, **bgerror** will be invoked once for each error, in the order they occurred. However, if **bgerror** returns with a break exception, then any remaining errors are skipped without calling **bgerror**.

If you are writing code that will be used by others as part of a package or other kind of library, consider avoiding **bgerror**. The reason for this is that the application programmer may also want to define a **bgerror**, or use other code that does and thus will have trouble integrating your code.

EXAMPLE

This **bgerror** procedure appends errors to a file, with a timestamp.

```
proc bgerror {message} {
    set timestamp [clock format [clock seconds]]
    set fl [open mylog.txt {WRONLY CREAT APPEND}]
    puts $fl "$timestamp: bgerror in $::argv '$message'"
    close $fl
}
```

SEE ALSO

[after\(n\)](#), [errorCode\(n\)](#), [errorInfo\(n\)](#), [interp\(n\)](#)

KEYWORDS

background error, reporting

NAME

`binary` - Insert and extract fields from binary strings

SYNOPSIS

```

binary decode format ?-option value ...? data
binary encode format ?-option value ...? data
binary format formatString ?arg arg ...?
binary scan string formatString ?varName varName ...?

```

DESCRIPTION

This command provides facilities for manipulating binary data. The subcommand **binary format** creates a binary string from normal Tcl values. For example, given the values 16 and 22, on a 32-bit architecture, it might produce an 8-byte binary string consisting of two 4-byte integers, one for each of the numbers. The subcommand **binary scan**, does the opposite: it extracts data from a binary string and returns it as ordinary Tcl string values. The **binary encode** and **binary decode** subcommands convert binary data to or from string encodings such as base64 (used in MIME messages for example).

Note that other operations on binary data, such as taking a subsequence of it, getting its length, or reinterpreting it as a string in some encoding, are done by other Tcl commands (respectively **string range**, **string length** and **encoding convertfrom** in the example cases). A binary string in Tcl is merely one where all the characters it contains are in the range \u0000-\u00FF.

BINARY ENCODE AND DECODE

When encoding binary data as a readable string, the starting binary data is passed to the **binary encode** command, together with the name of the encoding to use and any encoding-specific options desired. Data which has been encoded can be converted back to binary form using **binary decode**. The following formats and options are supported.

base64

The **base64** binary encoding is commonly used in mail messages and XML documents, and uses mostly upper and lower case letters and digits. It has the distinction of being able to be rewrapped arbitrarily without losing information.

During encoding, the following options are supported:

-maxlen

length

Indicates that the output should be split into lines of no more than *length* characters. By default, lines are not split.

-wrapchar

character

Indicates that, when lines are split because of the **-maxlen** option, *character* should be used to separate lines. By default, this is a newline character,

During decoding, the following options are supported:

-strict

Instructs the decoder to throw an error if it encounters any characters that are not strictly part of the encoding itself. Otherwise it ignores them. RFC 2045 calls for base64 decoders to be non-strict.

hex

The **hex** binary encoding converts each byte to a pair of hexadecimal digits in big-endian form.

No options are supported during encoding. During decoding, the following options are supported:

-strict

Instructs the decoder to throw an error if it encounters whitespace characters. Otherwise it ignores them.

uuencode

The **uuencode** binary encoding used to be common for transfer of data between Unix systems and on USENET, but is less common these days, having been largely superseded by the **base64** binary encoding.

During encoding, the following options are supported (though changing them may produce files that other implementations of decoders cannot process):

-maxlen	
<i>length</i>	
Indicates the maximum number of characters to produce for each encoded line. The valid range is 5 to 85.	
Line lengths outside that range cannot be accommodated by the encoding format. The default value is 61.	
-wrapchar	
<i>character</i>	
Indicates the character(s) to use to mark the end of each encoded line. Acceptable values are a sequence of zero or more characters from the set { \x09 (TAB), \x0B (VT), \x0C (FF), \x0D (CR) } followed by zero or one newline \x0A (LF). Any other values are rejected because they would generate encoded text that could not be decoded. The default value is a single newline.	
During decoding, the following options are supported:	
-strict	
Instructs the decoder to throw an error if it encounters anything outside of the standard encoding format. Without this option, the decoder tolerates some deviations, mostly to forgive reflows of lines between the encoder and decoder.	
Note that neither the encoder nor the decoder handle the header and footer of the uuencode format.	

BINARY FORMAT

The **binary format** command generates a binary string whose layout is specified by the *formatString* and whose contents come from the additional arguments. The resulting binary value is returned.

The *formatString* consists of a sequence of zero or more field specifiers separated by zero or more spaces. Each field specifier is a single type character followed by an optional flag character followed by an optional numeric *count*. Most field specifiers consume one argument to obtain the value to be formatted. The type character specifies how the value is to be formatted. The *count* typically indicates how many items of the specified type are taken from the value. If present, the *count* is a non-negative decimal integer or *, which normally indicates that all of the items in the value are to be used. If the number of arguments does not match the number of fields in the format string that consume arguments, then an error is generated. The flag character is ignored for **binary format**.

Here is a small example to clarify the relation between the field specifiers and the arguments:

```
binary format d3d {1.0 2.0 3.0 4.0} 0.1
```

The first argument is a list of four numbers, but

coreTools Command Reference Index

because of the count of 3 for the associated field specifier, only the first three will be used. The second argument is associated with the second field specifier. The resulting binary string contains the four numbers 1.0, 2.0, 3.0 and 0.1.

Each type-count pair moves an imaginary cursor through the binary data, storing bytes at the current position and advancing the cursor to just after the last byte stored. The cursor is initially at position 0 at the beginning of the data. The type may be any one of the following characters:

- a** Stores a byte string of length *count* in the output string. Every character is taken as modulo 256 (i.e. the low byte of every character is used, and the high byte discarded) so when storing character strings not wholly expressible using the characters \u0000-\u00ff, the **encoding convertto** command should be used first to change the string into an external representation if this truncation is not desired (i.e. if the characters are not part of the ISO 8859-1 character set.) If *arg* has fewer than *count* bytes, then additional zero bytes are used to pad out the field. If *arg* is longer than the specified length, the extra characters will be ignored. If *count* is *, then all of the bytes in *arg* will be formatted. If *count* is omitted, then one character will be formatted. For example,
binary format a7a*a alpha bravo charlie
will return a string equivalent to
alpha\000\000bravoc,
binary format a* [encoding convertto utf-8 \u20ac]
will return a string equivalent to
\342\202\254 (which is the UTF-8 byte sequence for a Euro-currency character)
and
binary format a* [encoding convertto iso8859-15 \u20ac]
will return a string equivalent to **\244**
(which is the ISO 8859-15 byte sequence for a Euro-currency character). Contrast these last two with:
binary format a* \u20ac
which returns a string equivalent to **\254** (i.e. **\xac**) by truncating the high-bits of the character, and which is probably not what is desired.
- A** This form is the same as **a** except that spaces are used for padding instead of nulls. For example,
binary format A6A*A alpha bravo charlie
will return **alpha bravoc**.
- b** Stores a string of *count* binary digits in low-to-high order within each byte in the output string. *Arg* must contain a

coreTools Command Reference Index

sequence of **1** and **0** characters. The resulting bytes are emitted in first to last order with the bits being formatted in low-to-high order within each byte. If *arg* has fewer than *count* digits, then zeros will be used for the remaining bits. If *arg* has more than the specified number of digits, the extra digits will be ignored. If *count* is *****, then all of the digits in *arg* will be formatted. If *count* is omitted, then one digit will be formatted. If the number of bits formatted does not end at a byte boundary, the remaining bits of the last byte will be zeros. For example,
binary format b5b* 11100 111000011010
will return a string equivalent to **\x07\x87\x05**.

B This form is the same as **b** except that the bits are stored in high-to-low order within each byte. For example,
binary format B5B* 11100 111000011010
will return a string equivalent to **\xe0\xe1\xa0**.

H Stores a string of *count* hexadecimal digits in high-to-low within each byte in the output string. *Arg* must contain a sequence of characters in the set The resulting bytes are emitted in first to last order with the hex digits being formatted in high-to-low order within each byte. If *arg* has fewer than *count* digits, then zeros will be used for the remaining digits. If *arg* has more than the specified number of digits, the extra digits will be ignored. If *count* is *****, then all of the digits in *arg* will be formatted. If *count* is omitted, then one digit will be formatted. If the number of digits formatted does not end at a byte boundary, the remaining bits of the last byte will be zeros. For example,
binary format H3H*H2 ab DEF 987
will return a string equivalent to **\xab\x00\xde\xfd\x98**.

h This form is the same as **H** except that the digits are stored in low-to-high order within each byte. This is seldom required. For example,
binary format h3h*h2 AB def 987
will return a string equivalent to **\xba\x00\xed\x0f\x89**.

c Stores one or more 8-bit integer values in the output string. If no *count* is specified, then *arg* must consist of an integer value. If *count* is specified,

coreTools Command Reference Index

arg must consist of a list containing at least that many integers. The low-order 8 bits of each integer are stored as a one-byte value at the cursor position. If *count* is *, then all of the integers in the list are formatted. If the number of elements in the list is greater than *count*, then the extra elements are ignored. For example,
binary format c3cc* {3 -3 128 1} 260 {2 5}
will return a string equivalent to `\x03\xfd\x80\x04\x02\x05`, whereas
binary format c {2 5}
will generate an error.

s This form is the same as **c** except that it stores one or more 16-bit integers in little-endian byte order in the output string. The low-order 16-bits of each integer are stored as a two-byte value at the cursor position with the least significant byte stored first. For example,
binary format s3 {3 -3 258 1}
will return a string equivalent to `\x03\x00\xfd\xff\x02\x01`.

S This form is the same as **s** except that it stores one or more 16-bit integers in big-endian byte order in the output string. For example,
binary format S3 {3 -3 258 1}
will return a string equivalent to `\x00\x03\xff\xfd\x01\x02`.

t This form (mnemonically *tiny*) is the same as **s** and **S** except that it stores the 16-bit integers in the output string in the native byte order of the machine where the Tcl script is running. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the **tcl_platform** array.

i This form is the same as **c** except that it stores one or more 32-bit integers in little-endian byte order in the output string. The low-order 32-bits of each integer are stored as a four-byte value at the cursor position with the least significant byte stored first. For example,
binary format i3 {3 -3 65536 1}
will return a string equivalent to `\x03\x00\x00\x00\xfd\xff\xff\xff\x00\x00\x01\x00`

I This form is the same as **i** except that it stores one or more one or more 32-bit integers in big-endian byte order in the output string. For example,
binary format I3 {3 -3 65536 1}
will return a string equivalent to

\x00\x00\x00\x03\xff\xff\xff\xfd\x00\x01\x00\x00

- n** This form (mnemonically *number* or *normal*) is the same as **i** and **I** except that it stores the 32-bit integers in the output string in the native byte order of the machine where the Tcl script is running. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the **tcl_platform** array.
- w** This form is the same as **c** except that it stores one or more 64-bit integers in little-endian byte order in the output string. The low-order 64-bits of each integer are stored as an eight-byte value at the cursor position with the least significant byte stored first. For example,
binary format w 7810179016327718216
 will return the string **HelloTcl**
- W** This form is the same as **w** except that it stores one or more one or more 64-bit integers in big-endian byte order in the output string. For example,
binary format Wc 4785469626960341345 110
 will return the string **BigEndian**
- m** This form (mnemonically the mirror of **w**) is the same as **w** and **W** except that it stores the 64-bit integers in the output string in the native byte order of the machine where the Tcl script is running. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the **tcl_platform** array.
- f** This form is the same as **c** except that it stores one or more one or more single-precision floating point numbers in the machine's native representation in the output string. This representation is not portable across architectures, so it should not be used to communicate floating point numbers across the network. The size of a floating point number may vary across architectures, so the number of bytes that are generated may vary. If the value overflows the machine's native representation, then the value of FLT_MAX as defined by the system will be used instead. Because Tcl uses double-precision floating point numbers internally, there may be some loss of precision in the conversion to single-precision. For example, on a Windows system running on an Intel Pentium processor,

coreTools Command Reference Index

```
binary format f2 {1.6 3.4}
will return a string equivalent to
\xcd\xcc\xcc\x3f\x9a\x99\x59\x40.
```

```

r      This form (mnemonically real) is the
      same as f except that it stores the
      single-precision floating point numbers
      in little-endian order. This conversion
      only produces meaningful output when
      used on machines which use the IEEE
      floating point representation (very
      common, but not universal.)

```

R This form is the same as **r** except that it stores the single-precision floating point numbers in big-endian order.

d This form is the same as **f** except that it stores one or more one or more double-precision floating point numbers in the machine's native representation in the output string. For example, on a Windows system running on an Intel Pentium processor, **binary format d1 {1.6}** will return a string equivalent to `\x9a\x99\x99\x99\x99\x99\xf9\x3f`.

q This form (mnemonically the mirror of **d**) is the same as **d** except that it stores the double-precision floating point numbers in little-endian order. This conversion only produces meaningful output when used on machines which use the IEEE floating point representation (very common, but not universal.)

Q This form is the same as **q** except that it stores the double-precision floating point numbers in big-endian order.

x Stores *count* null bytes in the output string. If *count* is not specified, stores one null byte. If *count* is *, generates an error. This type does not consume an argument. For example, **binary format** a3xa3x2a3 abc def ghi will return a string equivalent to **abc\000def\000\000ghi**.

X Moves the cursor back *count* bytes in the output string. If *count* is * or is larger than the current cursor position, then the cursor is positioned at location 0 so that the next byte stored will be the first byte in the result string. If *count* is omitted then the cursor is moved back one byte. This type does not consume an argument. For example,
binary format a3X*a3X2a3 abc def ghi
will return **dghi**.

@ Moves the cursor to the absolute location in the output string specified by *count*. Position 0 refers to the first byte in the output string. If *count* refers to a position beyond the last byte stored so far, then null bytes will be placed in the uninitialized locations and the cursor will be placed at the specified location. If *count* is *, then the cursor is moved to the current end of the output string. If *count* is omitted, then an error will be generated. This type does not consume an argument. For example,
binary format a5@2a1@*a3@10a1 abcde f ghi j
 will return **abfdeghi\000\000j**.

BINARY SCAN

The **binary scan** command parses fields from a binary string, returning the number of conversions performed. *String* gives the input bytes to be parsed (one byte per character, and characters not representable as a byte have their high bits chopped) and *formatString* indicates how to parse it. Each *varName* gives the name of a variable; when a field is scanned from *string* the result is assigned to the corresponding variable.

As with **binary format**, the *formatString* consists of a sequence of zero or more field specifiers separated by zero or more spaces. Each field specifier is a single type character followed by an optional flag character followed by an optional numeric *count*. Most field specifiers consume one argument to obtain the variable into which the scanned values should be placed. The type character specifies how the binary data is to be interpreted. The *count* typically indicates how many items of the specified type are taken from the data. If present, the *count* is a non-negative decimal integer or *, which normally indicates that all of the remaining items in the data are to be used. If there are not enough bytes left after the current cursor position to satisfy the current field specifier, then the corresponding variable is left untouched and **binary scan** returns immediately with the number of variables that were set. If there are not enough arguments for all of the fields in the format string that consume arguments, then an error is generated. The flag character may be given to cause some types to be read as unsigned values. The flag is accepted for all field types but is ignored for non-integer fields.

A similar example as with **binary format** should explain the relation between field specifiers and arguments in case of the binary scan subcommand:
binary scan \$bytes s3s first second

This command (provided the binary string in the

coreTools Command Reference Index

variable *bytes* is long enough) assigns a list of three integers to the variable *first* and assigns a single value to the variable *second*. If *bytes* contains fewer than 8 bytes (i.e. four 2-byte integers), no assignment to *second* will be made, and if *bytes* contains fewer than 6 bytes (i.e. three 2-byte integers), no assignment to *first* will be made. Hence:

```
puts [binary scan abcdefg s3s first second]
```

```
puts $first
```

```
puts $second
```

will print (assuming neither variable is set previously):

```
1
```

```
25185 25699 26213
```

```
can't read "second": no such variable
```

It is *important* to note that the **c**, **s**, and **S** (and **i** and **I** on 64bit systems) will be scanned into long data size values. In doing this, values that have their high bit set (0x80 for chars, 0x8000 for shorts, 0x80000000 for ints), will be sign extended. Thus the following will occur:

```
set signShort [binary format s1 0x8000]
```

```
binary scan $signShort s1 val; # val == 0xFFFF8000
```

If you require unsigned values you can include the flag character following the field type. For example, to read an unsigned short value:

```
set signShort [binary format s1 0x8000]
```

```
binary scan $signShort su1 val; # val == 0x00008000
```

Each type-count pair moves an imaginary cursor through the binary data, reading bytes from the current position. The cursor is initially at position 0 at the beginning of the data. The type may be any one of the following characters:

a The data is a byte string of length *count*. If *count* is ***, then all of the remaining bytes in *string* will be scanned into the variable. If *count* is omitted, then one byte will be scanned. All bytes scanned will be interpreted as being characters in the range \u0000-\u00ff so the **encoding convertfrom** command will be needed if the string is not a binary string or a string encoded in ISO 8859-1. For example,
binary scan abcde\000fghi a6a10 var1 var2
will return **1** with the string equivalent to **abcde\000** stored in *var1* and *var2* left unmodified, and
binary scan \342\202\254 a* var1
set *var2* [encoding convertfrom utf-8 \$var1]
will store a Euro-currency character in *var2*.

A This form is the same as **a**, except trailing blanks and nulls are stripped from the scanned value before it is stored in the variable. For example,
binary scan "abc efghi \000" A* var1

coreTools Command Reference Index

will return **1** with **abc efghi** stored in *var1*.

- b** The data is turned into a string of *count* binary digits in low-to-high order represented as a sequence of and characters. The data bytes are scanned in first to last order with the bits being taken in low-to-high order within each byte. Any extra bits in the last byte are ignored. If *count* is *, then all of the remaining bits in *string* will be scanned. If *count* is omitted, then one bit will be scanned. For example, **binary scan \x07\x87\x05 b5b* var1 var2** will return **2** with **11100** stored in *var1* and **1110000110100000** stored in *var2*.
- B** This form is the same as **b**, except the bits are taken in high-to-low order within each byte. For example, **binary scan \x70\x87\x05 B5B* var1 var2** will return **2** with **01110** stored in *var1* and **1000011100000101** stored in *var2*.
- H** The data is turned into a string of *count* hexadecimal digits in high-to-low order represented as a sequence of characters in the set The data bytes are scanned in first to last order with the hex digits being taken in high-to-low order within each byte. Any extra bits in the last byte are ignored. If *count* is *, then all of the remaining hex digits in *string* will be scanned. If *count* is omitted, then one hex digit will be scanned. For example, **binary scan \x07\xC6\x05\x1f\x34 H3H* var1 var2** will return **2** with **07c** stored in *var1* and **051f34** stored in *var2*.
- h** This form is the same as **H**, except the digits are taken in reverse (low-to-high) order within each byte. For example, **binary scan \x07\x86\x05\x12\x34 h3h* var1 var2** will return **2** with **706** stored in *var1* and **502143** stored in *var2*.
- Note that most code that wishes to parse the hexadecimal digits from multiple bytes in order should use the **H** format.
- c** The data is turned into *count* 8-bit signed integers and stored in the corresponding variable as a list. If *count* is *, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one 8-bit integer will be scanned. For example, **binary scan \x07\x86\x05 c2c* var1 var2** will return **2** with **7 -122** stored in *var1*

coreTools Command Reference Index

and **5** stored in *var2*. Note that the integers returned are signed, but they can be converted to unsigned 8-bit quantities using an expression like:
set num [expr { \$num & 0xFF }]

s The data is interpreted as *count* 16-bit signed integers represented in little-endian byte order. The integers are stored in the corresponding variable as a list. If *count* is ***, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one 16-bit integer will be scanned. For example,
binary scan \x05\x00\x07\x00\xff s2s* var1 var2
will return **2** with **5 7** stored in *var1* and **-16** stored in *var2*. Note that the integers returned are signed, but they can be converted to unsigned 16-bit quantities using an expression like:
set num [expr { \$num & 0xFFFF }]

S This form is the same as **s** except that the data is interpreted as *count* 16-bit signed integers represented in big-endian byte order. For example,
binary scan \x00\x05\x00\x07\xff S2S* var1 var2
will return **2** with **5 7** stored in *var1* and **-16** stored in *var2*.

t The data is interpreted as *count* 16-bit signed integers represented in the native byte order of the machine running the Tcl script. It is otherwise identical to **s** and **S**. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the **tcl_platform** array.

i The data is interpreted as *count* 32-bit signed integers represented in little-endian byte order. The integers are stored in the corresponding variable as a list. If *count* is ***, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one 32-bit integer will be scanned. For example,
set str \x05\x00\x00\x00\x07\x00\x00\x00\xff\xff\xff
binary scan \$str i2i* var1 var2
will return **2** with **5 7** stored in *var1* and **-16** stored in *var2*. Note that the integers returned are signed, but they can be converted to unsigned 32-bit quantities using an expression like:
set num [expr { \$num & 0xFFFFFFFF }]

I This form is the same as **I** except that the data is interpreted as *count* 32-bit signed integers represented in big-endian byte order. For example,

coreTools Command Reference Index

- set str \x00\x00\x00\x05\x00\x00\x00\x07\xff\xff\xff\xf0
binary scan \$str I2I* var1 var2
will return 2 with 5 7 stored in var1
and -16 stored in var2.
- n** The data is interpreted as *count* 32-bit signed integers represented in the native byte order of the machine running the Tcl script. It is otherwise identical to **i** and **I**. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the **tcl_platform** array.
- w** The data is interpreted as *count* 64-bit signed integers represented in little-endian byte order. The integers are stored in the corresponding variable as a list. If *count* is *, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one 64-bit integer will be scanned. For example,
set str \x05\x00\x00\x00\x07\x00\x00\x00\xf0\xff\xff\xff
binary scan \$str wi* var1 var2
will return 2 with 30064771077 stored in var1 and -16 stored in var2. Note that the integers returned are signed and cannot be represented by Tcl as unsigned values.
- W** This form is the same as **w** except that the data is interpreted as *count* 64-bit signed integers represented in big-endian byte order. For example,
set str \x00\x00\x00\x05\x00\x00\x00\x07\xff\xff\xff\xf0
binary scan \$str WI* var1 var2
will return 2 with 21474836487 stored in var1 and -16 stored in var2.
- m** The data is interpreted as *count* 64-bit signed integers represented in the native byte order of the machine running the Tcl script. It is otherwise identical to **w** and **W**. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the **tcl_platform** array.
- f** The data is interpreted as *count* single-precision floating point numbers in the machine's native representation. The floating point numbers are stored in the corresponding variable as a list. If *count* is *, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one single-precision floating point number will be scanned. The size of a floating point number may vary across architectures, so the number of bytes that are scanned may vary. If the data does not represent a

coreTools Command Reference Index

	valid floating point number, the resulting value is undefined and compiler dependent. For example, on a Windows system running on an Intel Pentium processor, binary scan \x3f\xcc\xcc\xcd f var1 will return 1 with 1.6000000238418579 stored in <i>var1</i>.
r	This form is the same as f except that the data is interpreted as <i>count</i> single-precision floating point number in little-endian order. This conversion is not portable to the minority of systems not using IEEE floating point representations.
R	This form is the same as f except that the data is interpreted as <i>count</i> single-precision floating point number in big-endian order. This conversion is not portable to the minority of systems not using IEEE floating point representations.
d	This form is the same as f except that the data is interpreted as <i>count</i> double-precision floating point numbers in the machine's native representation. For example, on a Windows system running on an Intel Pentium processor, binary scan \x9a\x99\x99\x99\x99\x99\xf9\x3f d var1 will return 1 with 1.6000000000000001 stored in <i>var1</i>.
q	This form is the same as d except that the data is interpreted as <i>count</i> double-precision floating point number in little-endian order. This conversion is not portable to the minority of systems not using IEEE floating point representations.
Q	This form is the same as d except that the data is interpreted as <i>count</i> double-precision floating point number in big-endian order. This conversion is not portable to the minority of systems not using IEEE floating point representations.
x	Moves the cursor forward <i>count</i> bytes in <i>string</i>. If <i>count</i> is * or is larger than the number of bytes after the current cursor position, then the cursor is positioned after the last byte in <i>string</i>. If <i>count</i> is omitted, then the cursor is moved forward one byte. Note that this type does not consume an argument. For example, binary scan \x01\x02\x03\x04 x2H* var1 will return 1 with 0304 stored in <i>var1</i>.

x	Moves the cursor back <i>count</i> bytes in <i>string</i> . If <i>count</i> is * or is larger than the current cursor position, then the cursor is positioned at location 0 so that the next byte scanned will be the first byte in <i>string</i> . If <i>count</i> is omitted then the cursor is moved back one byte. Note that this type does not consume an argument. For example, binary scan \x01\x02\x03\x04 c2XH* var1 var2 will return 2 with 1 2 stored in var1 and 020304 stored in var2.
@	Moves the cursor to the absolute location in the data string specified by <i>count</i> . Note that position 0 refers to the first byte in <i>string</i> . If <i>count</i> refers to a position beyond the end of <i>string</i> , then the cursor is positioned after the last byte. If <i>count</i> is omitted, then an error will be generated. For example, binary scan \x01\x02\x03\x04 c2@1H* var1 var2 will return 2 with 1 2 stored in var1 and 020304 stored in var2.

PORTABILITY ISSUES

The **r**, **R**, **q** and **Q** conversions will only work reliably for transferring data between computers which are all using IEEE floating point representations. This is very common, but not universal. To transfer floating-point numbers portably between all architectures, use their textual representation (as produced by **format**) instead.

EXAMPLES

This is a procedure to write a Tcl string to a binary-encoded channel as UTF-8 data preceded by a length word:

```
proc writeString {channel string} {
    set data [encoding convertto utf-8 $string]
    puts -nonewline [binary format Ia* \
        [string length $data] $data]
}
```

This procedure reads a string from a channel that was written by the previously presented *writeString* procedure:

```
proc readString {channel} {
    if {[binary scan [read $channel 4] I length]} {
        error "missing length"
    }
}
```

coreTools Command Reference Index

```
}  
set data [read $channel $length]  
return [encoding convertfrom utf-8 $data]  
}
```

This converts the contents of a file (named in the variable *filename*) to base64 and prints them:

```
set f [open $filename rb]  
set data [read $f]  
close $f  
puts [binary encode base64 -maxlen 64 $data]
```

SEE ALSO

encoding(n), format(n), scan(n), string(n),
tcl_platform(n)

KEYWORDS

binary, format, scan

bind_tech_cell_mapping

Specify binding requirements for mapping to a technology specific cell.

Syntax

```

string bind_tech_cell_mapping [-module <module name>] [-macro <RTL macro
name>] [-design_name <Component design name>] [-tech_cell <cell name>]
[-port_map <map data>] [-attributes <attribute list>
(default=dont_touch)] [-ports_connected_to_high <port list>] [-ctl_file
<ctl file path>] [-unbind]
    string <module name>
    string <RTL macro name>
    string <Component design name>
    string <cell name>
    string <map data>
    list <attribute list> (default=dont_touch)
    string <port list>
    string <ctl file path>

```

Parameters

-module <module name>	Module of the technology cell mapping
-macro <RTL macro name>	Macro defining the technology cell mapping
-design_name <Component design name>	Design name of the component the tech cell belongs to
-tech_cell <cell name>	Technology specific cell name
-port_map <map data>	Map of module ports to tech cell pins
-attributes <attribute list> (default=dont_touch)	String list of attributes: dont_touch, size_only, dont_retime
-ports_connected_to_high <port list>	String list of unused input ports to be tied to 1 or 0. Ports can be tied to 0 using ~portName
-ctl_file <ctl file path>	Path to Test Model (ctl_file) corresponding to Tech Cell.
-unbind	Remove/Delete the technology cell map binding based on module and/or macro name

Description

coreTools Command Reference Index

This command is used to bind the definition of a technology cell mapping created with the `define_tech_cell_mapping` command to a particular technology cell. The mapping definition comes from the `-module` and optional `-macro` arguments which should match those specified for `define_tech_cell_mapping`, except that the `-macro` argument is optional unless multiple definitions with different macros exist for the same module.

The resulting mapping will be used to derive a macro mapping to the technology cell in a file called `<design>-techmap.vh` which is automatically written to the `<workspace>/src` directory and included in the source file list for synthesis and other implementation tools.

The `-attribute` option can be used to set constraints on tech cells. A file `tech_cell_binding_attributes.tcl` is written out, which is sourced by downstream synthesis tool.

For a `define_tech_cell_mapping` command issued as follows:

```
coreBuilder> define_tech_cell_mapping -module BCM00 \  
-macro BCM00_map \  
-instance I000 \  
-description "Sample tech map" \  
-port_names {m1 m2 m3} \  
-port_descriptions {{m1 port} {m2 port} {m3 port}} \  
-enable_macro_args \  
-extra_macro_args "a1 a2 a3" \  
-file "master.v" \  
-width "[WIDTH-1:0]" \  
-filter "Name=~ADDF" \  
-output_inversion_suffix "INVERTED_OUT"
```

The option `output_inversion_suffix` is optional and provided to support output inversion.

and `bind_tech_cell_mapping` command issued as follows:

For output Inversion

```
coreAssembler> bind_tech_cell_mapping -module BCM00 \  
-tech_cell OR2 \  
-port_map {{m1 A} {m2 B} {m3 Z 1}} \  
-attribute {dont_touch size_only dont_retime}
```

with no output Inversion:

```
coreAssembler> bind_tech_cell_mapping -module BCM00 \  
-tech_cell OR2 \  
-port_map {{m1 A} {m2 B} {m3 Z}} \  
-attribute {dont_touch size_only dont_retime}
```

The `bind_tech_cell_mapping` command is stored in `TechCellMappingBind` subscripted attribute on the top level design as follows.

```
coreAssembler> get_subscripts [current_design] -attr TechCellMappingBind  
BCM00-BCM00_map
```

coreTools Command Reference Index

In older coreTools versions (R2020.12-SP2 or older), the format of the subscript is as shown:

BCM00-BCM00_map-compA

where the macro fields are:

- BCM00 - tech cell module
- BCM00_map - tech cell macro
- compA - design name

If output inversion is supported and enabled.

```
coreAssembler> get_attribute [current_design ] -attr TechCellMappingBind
-subscript BCM00-BCM00_map-master OR2 {A B Z} /M {{Sample tech map} I001
{[width-1:0]} {m1 m2 m3} {{m1 port} {m2 port} {m3 port}} compA.v
Name=~ADDF 1 {a1 a2 a3} INVERTED_OUT} {dont_touch dont_retime size_only}
```

If output inversion is not supported and not enabled.

```
coreAssembler> get_attribute [current_design ] -attr TechCellMappingBind
-subscript BCM00-BCM00_map-master OR2 {A B Z} /M {{Sample tech map} I001
{[width-1:0]} {m1 m2 m3} {{m1 port} {m2 port} {m3 port}} compA.v
Name=~ADDF 1 {a1 a2 a3}} {dont_touch dont_retime size_only}
```

for older coreTools (R2020.12-SP2 or older), the attribute is as follows
(enable_macro_args and extra_macro_args are absent)

```
coreAssembler> get_attribute [current_design ] -attr TechCellMappingBind
-subscript BCM00-BCM00_map-master
OR2 {A B Z} / {{Sample tech map} I001 {[width-1:0]} {r1 r2 r3} {{r1 port}
{r2 port} {r3 port}} compA.v {}} {dont_touch dont_retime size_only}
```

where the fields are:

- OR2 - tech_cell
- {A B Z} - pin names
- /M - components
- {{Sample tech map} I000 {[WIDTH-1:0]} {m1 m2 m3} {{m1 port} {m2 port} {m3 port}} master.v Name=~ADDF 1 {a1 a2 a3}} - TechCellMappingDefn[BCM00-BCM00_map-master]
 - ◆ {Sample tech map} - description
 - ◆ I000 - instance
 - ◆ {[WIDTH-1:0]} - width
 - ◆ {m1 m2 m3} - port_names
 - ◆ {{m1 port} {m2 port} {m3 port}} - port_descriptions
 - ◆ master.v - file
 - ◆ Name=~ADDF - filter
 - ◆ 1 - enable_macro_args switch (boolean)
 - ◆ {a1 a2 a3} - extra_macro_args list
 - ◆ INVERTED_OUT -output_inversion_suffix
- {dont_touch size_only dont_retime} - attributes to be set on the tech_cell for synthesis run

Examples

Consider the following RTL module for which a technology specific mapping is desired:

```

module my_and(in0, in1, out1);
input in0;
input in1;
output out0;
// Map to technology specific cell if provided.
`ifdef my_tech_cell_macro
`my_tech_cell_macro
`else
assign out1 = in0 & in1;
`endif
endmodule

```

Create a mapping definition for the module above. This is typically done in coreBuilder:

```

define_tech_cell_mapping -module my_module \
-macro my_tech_cell_macro \
-description "Module mapped to an AND gate"
-instance I0 \
-port_names {in0 in0 out0} \
-port_descriptions {{port in0} {port in1} {port out0}} \
-file "compA.v"

```

Then create the binding to a particular cell in coreConsultant or coreAssembler:

```

bind_tech_cell_mapping -module my_module \
-tech_cell AN2 \
-port_map {{in0 A} {in1 B} {out0 Z}} \
-attributes {dont_touch size_only dont_retime}

```

This binding will result in the creation of tech mapping headers in the mandatory -file option for define_tech_cell_mapping as shown:

```

`ifdef SYNTHESIS
`ifndef my_tech_cell_macro
`define my_tech_cell_macro AN2 I0 (.A(in0),.B(in1),.Z(out1));
`endif
`endif

```

These headers are placed before the first occurrence of the module keyword. This file also includes an undef section at the bottom as follows:

```

`ifndef mytech_cell_macro
`undef mytech_cell_macro
`endif

```

For the Case, where output inversion is supported.

Create a mapping definition for the module above. This is typically done in coreBuilder:

```

define_tech_cell_mapping -module my_module \
-macro my_tech_cell_macro \
-description "Module mapped to an AND gate"

```

coreTools Command Reference Index

```
-instance I0 \  
-port_names {in0 in0 out0} \  
-port_descriptions {{port in0} {port in1} {port out0}} \  
-file "compA.v"  
-output_inversion_suffix "INVERTED_OUT"
```

Then create the binding to a particular cell and invert an output in coreConsultant or coreAssembler:

```
bind_tech_cell_mapping -module my_module \  
-tech_cell AN2 \  
-port_map {{in0 A} {in1 B} {out0 Z 1}} \  
-attributes {dont_touch size_only dont_retime}
```

This binding will result in the creation of tech mapping headers in the mandatory -file option for define_tech_cell_mapping as shown:

```
'ifdef SYNTHESIS  
'ifndef my_tech_cell_macro_INVERTED_OUT  
'define my_tech_cell_macro_INVERTED_OUT AN2 I0  
(.A(in0),.B(in1),.Z(out1_INVERTED_OUT));  
'endif  
'endif
```

Some more examples for bind_tech_cell_mapping command are:

If multiple tech cell definitions with the same module name are found, the macro name can be explicitly specified:

```
bind_tech_cell_mapping -module BCM61 \  
-macro BCM61_map \  
-tech_cell OR2 \  
-port_map {{r1 A} {r2 B} {r3 Z}}
```

Multiple definitions with same module names may exist across designs. In that case, the design name can be explicitly specified:

```
bind_tech_cell_mapping -module BCM61 \  
-design_name compA \  
-tech_cell OR2 \  
-port_map {{r1 A} {r2 B} {r3 Z}}
```

Existing bindings can be removed with the -unbind switch. The module/macro options should uniquely identify the tech_cell in question.

```
coreConsultant> bind_tech_cell_mapping -unbind -module BCM01  
Information: Unbinding technology cell mapping for module BCM01 via macro  
BCM01_map. (TECH-39)  
Information: Attribute TechCellMappingBind[BCM01-BCM01_map-compA] removed  
from item compA_6. (CMD5-64)  
for older coreTools versions (R2020.12-SP2 or older), the -design option  
may be required  
coreConsultant> bind_tech_cell_mapping -unbind -module BCM02 -macro  
BCM02_map -design_name compA  
Information: Unbinding technology cell mapping for module BCM02 via macro  
BCM02_map. (TECH-39)
```


Information: Attribute TechCellMappingBind[BCM02-BCM02_map-compA] removed from item compA_6. (CMDS-64)

Unconnected ports on technology cells can be tied to 1'b1 as follows

```
bind_tech_cell_mapping \
-macro DWC_AND_2_SRC \
-tech_cell AN3P \
-ports_connected_to_high "C" \
-port_map {{i_a A} {i_b B} {o_z Z}}
This will instantiate define the macro as follow:
```

```
'define DWC_AND_2_SRC AN3P U_AND \[AWIDTH-1:0\]
(.A(i_a),.B(i_b),.Z(o_z),.C(1'b1))
```

Unconnected ports on technology cells can be tied to 1'b0 as follows

```
bind_tech_cell_mapping \
-macro DWC_AND_2_SRC \
-tech_cell AN3P \
-ports_connected_to_high "~C" \
-port_map {{i_a A} {i_b B} {o_z Z}}
This will instantiate define the macro as follow:
```

```
'define DWC_AND_2_SRC AN3P U_AND \[AWIDTH-1:0\]
(.A(i_a),.B(i_b),.Z(o_z),.C(1'b0))
```

A CTL test model file can be specified as follows:

```
bind_tech_cell_mapping \
-macro DWC_SYNC_2_STAGE_SRC \
-tech_cell HSBSVT08_SYNC2RBMSFQ_Y2_1 \
-port_map {{clk_d CK} {rst_d_n RD} {data_s D} {data_d Q}} \
-ctl_file "<path to ctl files>/HSBSVT08_SYNC2RBMSFQ_Y2_1.ctl" \
-attributes {size_only}
```

See Also

[define_tech_cell_mapping\(2\)](#)

NAME

`break` - Abort looping command

SYNOPSIS

`break`

DESCRIPTION

This command is typically invoked inside the body of a looping command such as **for** or **foreach** or **while**. It returns a 3 (**TCL_BREAK**) result code, which causes a break exception to occur. The exception causes the current script to be aborted out to the innermost containing loop command, which then aborts its execution and returns normally. Break exceptions are also handled in a few other situations, such as the **catch** command, Tk event bindings, and the outermost scripts of procedure bodies.

EXAMPLE

Print a line for each of the integers from 0 to 5:

```
for {set x 0} {$x<10} {incr x} {  
    if {$x > 5} {  
        break  
    }  
    puts "x is $x"  
}
```

SEE ALSO

`catch(n)`, `continue(n)`, `for(n)`, `foreach(n)`, `return(n)`,
`while(n)`

KEYWORDS

`abort`, `break`, `loop`

build_debug_tarfile
Build a debug tar-file

Syntax

```
string build_debug_tarfile [-batch_script] [-environment] [-workspace]
[-sim_logs] [-syn_logs] [-keep_links] [-dwh_content] [-check_env]
[-tools] [-notes <notes>] file
    string <notes>
    string file
```

Parameters

-batch_script	Include the batch for the current workspace
-environment	Include environment variable settings
-workspace	Include the current workspace
-sim_logs	Include simulation log files
-syn_logs	Include synthesis log files
-keep_links	Preserve symbolic links in workspace
-dwh_content	Include report of DESIGNWARE_HOME contents
-check_env	Include environment check results
-tools	Include tool versions
-notes <notes>	Include notes
<i>file</i>	Filename for generated tarball

Description

This command is used to build a debug tarfile for support purposes. The command generates a README file with the date and time of generation, a list of requested items and information about tool versions and user notes, if selected. The command collects the README file along with any other requested files such as the workspace batch script, and creates a compressed archive, dereferencing links as they are encountered, in the given file name.

The '-batch_script', '-workspace', '-sim_logs', '-syn_logs', and '-check_env' options require a currently loaded workspace.

The '-sim_logs' and '-check_env' options are only available in coreConsultant.

'-sim_logs' and '-syn_logs' are ignored (a warning is issued) if the '-workspace' is also selected because the workspace already includes both

sets of files.

Examples

To build a tarfile including the script to build a workspace:

```
prompt> build_debug_tarfile -batch_script debug.tar.gz
```

To build a tarfile including the simulation logs, tool version and environment check information:

```
prompt> build_debug_tarfile -sim_logs -tools -check_env debug.tar.gz
```

To build a tarfile including just tool version and note information:

```
prompt> build_debug_tarfile -tools -notes "describe the problem"
debug.tar.gz
```

See Also

busBit

Individual bit of a bus

Description

A busBit item represents an individual bit of a bus in the coreBuilder/coreConsultant model of the design. Individual busBit items do not exist in the design knowledge database until the bus is blasted. A bus is automatically blasted when you set an attribute on an individual bus bit either through the GUI Bit-Level Intent button or by executing set_port_attribute on a bus bit.

See Also

[set_port_attribute \(2\)](#)

Supported Attributes

[AutoConnectIfUnconnected \(3\)](#), [Capacitance \(3\)](#), [CaseAnalysisValue \(3\)](#),
[ClockName \(3\)](#), [ComponentOfItem \(3\)](#), [ConstantPort \(3\)](#), [CriticalTiming \(3\)](#),
[Description \(3\)](#), [DftExistingSignalActiveState \(3\)](#),
[DftExistingSignalTestMode \(3\)](#), [DftExistingSignalTiming \(3\)](#),
[DftExistingSignalType \(3\)](#), [DftExistingSignalUsage \(3\)](#),
[DftSpecSignalActiveState \(3\)](#), [DftSpecSignalTestMode \(3\)](#),
[DftSpecSignalType \(3\)](#), [DftSpecSignalUsage \(3\)](#), [DocInclude \(3\)](#),
[DocRegisteredBy \(3\)](#), [DontTouchNetwork \(3\)](#), [Drive \(3\)](#), [DrivingCell \(3\)](#),
[HighFanout \(3\)](#), [IdealPort \(3\)](#), [IfUnconnected \(3\)](#), [InputDelay \(3\)](#),
[InputResistance \(3\)](#), [IsConnected \(3\)](#), [IsTriState \(3\)](#), [Label \(3\)](#),
[LogicalName \(3\)](#), [MaxCap \(3\)](#), [MaxFallInputDelay \(3\)](#),
[MaxFallInputDelayFallingEdge \(3\)](#), [MaxFallOutputDelay \(3\)](#),
[MaxFallOutputDelayFallingEdge \(3\)](#), [MaxFanout \(3\)](#), [MaxInputDelay \(3\)](#),
[MaxInputDelayFallingEdge \(3\)](#), [MaxOutputDelay \(3\)](#),
[MaxOutputDelayFallingEdge \(3\)](#), [MaxRiseInputDelay \(3\)](#),
[MaxRiseInputDelayFallingEdge \(3\)](#), [MaxRiseOutputDelay \(3\)](#),
[MaxRiseOutputDelayFallingEdge \(3\)](#), [MaxTransition \(3\)](#), [MinCap \(3\)](#),
[MinFallInputDelay \(3\)](#), [MinFallInputDelayFallingEdge \(3\)](#),
[MinFallOutputDelay \(3\)](#), [MinFallOutputDelayFallingEdge \(3\)](#), [MinInputDelay \(3\)](#),
[MinInputDelayFallingEdge \(3\)](#), [MinOutputDelay \(3\)](#),
[MinOutputDelayFallingEdge \(3\)](#), [MinRiseInputDelay \(3\)](#),
[MinRiseInputDelayFallingEdge \(3\)](#), [MinRiseOutputDelay \(3\)](#),
[MinRiseOutputDelayFallingEdge \(3\)](#), [Name \(3\)](#), [OutputDelay \(3\)](#),
[PinLoadCount \(3\)](#), [PinLoadType \(3\)](#), [PortFunction \(3\)](#), [PowerDomain \(3\)](#),
[RegisteredBy \(3\)](#), [ResetProperties \(3\)](#), [Sequence \(3\)](#), [SimTieOff \(3\)](#),
[SynchronousTo \(3\)](#), [TypeName \(3\)](#), [TypedefType \(3\)](#), [UnconnectedPort \(3\)](#),
[WireLoad \(3\)](#)

byte_compile_file

Byte compile a TCL file or text string.

Syntax

```
string byte_compile_file [-text] -out <file name> input
      string <file name>
      string input
```

Parameters

-text	Interpret input as text instead of a file name
-out <file name>	Output file name
input	Input file name or text to be compiled

Description

This command is used to byte-code compile a TCL file. The compiled file will be written to the file specified via the -out argument. If the -text argument is specified, the input argument is interpreted as the text to be compiled instead of as the name of the file to be compiled.

Examples

Compile file simple.tcl with output going to simple.tbc:

```
byte_compile_file -out simple.tbc simple.tcl
```

Compile incoming text with output going to text.tbc:

```
byte_compile_file -out text.tbc -text {set comment "Text to be
compiled."}
```

See Also

cA_supports_hierarchy

Controls whether or not hierarchy is supported by coreAssembler.

Syntax

```
string cA_supports_hierarchy = "true"
```

Description

Setting this variable to false turns off hierarchy support in coreAssembler. When this variable is false, it is not possible to create a hierarchical component or open a workspace with hierarchy in it.

The purpose of this variable is to ease migration for kit providers. It may be that the plugins provided with a coreKit break in the presence of hierarchy. Setting this variable to false should resolve that issue.

Setting this variable does the following:

- Prevents opening a workspace with hierarchical components in it.
- Prevents creation of a hierarchical component.
- Makes `get_current_component` return the old-style (non-absolute) component name. (`set_current_component` does likewise).

Examples

```
coreAssembler> get_current_component
/i_AXIMaster
coreAssembler> set cA_supports_hierarchy 0
0
coreAssembler> get_current_component
i_AXIMaster
coreAssembler> create_hierarchical_component hcomp
Error: Cannot create a hierarchical component.
Set cA_supports_hierarchy to true to allow it
Use error_info for more info. (CMD-013)
```

See Also

[create_hierarchical_component \(2\)](#), [open_workspace \(2\)](#)

NAME

`case` - Evaluate one of several scripts, depending on a given value

SYNOPSIS

```
case string ?in? patList body ?patList body ...?
```

```
case string ?in? {patList body ?patList body ...?}
```

DESCRIPTION

*Note: the **case** command is obsolete and is supported only for backward compatibility. At some point in the future it may be removed entirely. You should use the **switch** command instead.*

The **case** command matches *string* against each of the *patList* arguments in order. Each *patList* argument is a list of one or more patterns. If any of these patterns matches *string* then **case** evaluates the following *body* argument by passing it recursively to the Tcl interpreter and returns the result of that evaluation. Each *patList* argument consists of a single pattern or list of patterns. Each pattern may contain any of the wild-cards described under **string match**. If a *patList* argument is **default**, the corresponding body will be evaluated if no *patList* matches *string*. If no *patList* argument matches *string* and no default is given, then the **case** command returns an empty string.

Two syntaxes are provided for the *patList* and *body* arguments. The first uses a separate argument for each of the patterns and commands; this form is convenient if substitutions are desired on some of the patterns or commands. The second form places all of the patterns

coreTools Command Reference Index

and commands together into a single argument; the argument must have proper list structure, with the elements of the list being the patterns and commands. The second form makes it easy to construct multi-line case commands, since the braces around the whole list make it unnecessary to include a backslash at the end of each line. Since the *patList* arguments are in braces in the second form, no command or variable substitutions are performed on them; this makes the behavior of the second form different than the first form in some cases.

SEE ALSO

`switch(n)`

KEYWORDS

case, match, regular expression

NAME

`catch` - Evaluate script and trap exceptional returns

SYNOPSIS

`catch script ?resultVarName? ?optionsVarName?`

DESCRIPTION

The **catch** command may be used to prevent errors from aborting command interpretation. The **catch** command calls the Tcl interpreter recursively to execute *script*, and always returns without raising an error, regardless of any errors that might occur while executing *script*.

If *script* raises an error, **catch** will return a non-zero integer value corresponding to the exceptional return code returned by evaluation of *script*. Tcl defines the normal return code from script evaluation to be zero (0), or **TCL_OK**. Tcl also defines four exceptional return codes: 1 (**TCL_ERROR**), 2 (**TCL_RETURN**), 3 (**TCL_BREAK**), and 4 (**TCL_CONTINUE**). Errors during evaluation of a script are indicated by a return code of **TCL_ERROR**. The other exceptional return codes are returned by the **return**, **break**, and **continue** commands and in other special situations as documented. Tcl packages can define new commands that return other integer values as return codes as well, and scripts that make use of the **return -code** command can also have return codes other than the five defined by Tcl.

If the *resultVarName* argument is given, then the variable it names is set to the result of the script evaluation. When the return code from the script is 1 (**TCL_ERROR**), the value stored in *resultVarName* is an error message. When the return code from the script is

coreTools Command Reference Index

0 (**TCL_OK**), the value stored in *resultVarName* is the value returned from *script*.

If the *optionsVarName* argument is given, then the variable it names is set to a dictionary of return options returned by evaluation of *script*. Tcl specifies two entries that are always defined in the dictionary: **-code** and **-level**. When the return code from evaluation of *script* is not **TCL_RETURN**, the value of the **-level** entry will be 0, and the value of the **-code** entry will be the same as the return code. Only when the return code is **TCL_RETURN** will the values of the **-level** and **-code** entries be something else, as further described in the documentation for the **return** command.

When the return code from evaluation of *script* is **TCL_ERROR**, four additional entries are defined in the dictionary of return options stored in *optionsVarName*: **-errorinfo**, **-errorcode**, **-errorline**, and **-errorstack**. The value of the **-errorinfo** entry is a formatted stack trace containing more information about the context in which the error happened. The formatted stack trace is meant to be read by a person. The value of the **-errorcode** entry is additional information about the error stored as a list. The **-errorcode** value is meant to be further processed by programs, and may not be particularly readable by people. The value of the **-errorline** entry is an integer indicating which line of *script* was being evaluated when the error occurred. The value of the **-errorstack** entry is an even-sized list made of token-parameter pairs accumulated while unwinding the stack. The token may be in which case the parameter is a list made of the proc name and arguments at the corresponding level; or it may be in which case the parameter is the relative level (as in **uplevel**) of the previous **CALL**. The salient differences with respect to **-errorinfo** are that:

- | | | |
|-----|--|--|
| [1] | it is a machine-readable form that is amenable to processing with [foreach {tok prm} ...] , | |
| [2] | it contains the true (substituted) values passed to the functions, instead of the static text of the calling sites, and | |
| [3] | it is coarser-grained, with only one element per stack frame (like procs; no separate elements for foreach constructs for example). | |

The values of the **-errorinfo** and **-errorcode** entries of the most recent error are also available as values of the global variables **::errorInfo** and **::errorCode** respectively. The value of the **-errorstack** entry surfaces as **info errorstack**.

coreTools Command Reference Index

Tcl packages may provide commands that set other entries in the dictionary of return options, and the **return** command may be used by scripts to set return options in addition to those defined above.

EXAMPLES

The **catch** command may be used in an **if** to branch based on the success of a script.

```
if { [catch {open $someFile w} fid] } {  
    puts stderr "Could not open $someFile for writing\n$fid"  
    exit 1  
}
```

There are more complex examples of **catch** usage in the documentation for the **return** command.

SEE ALSO

`break(n)`, `continue(n)`, `dict(n)`, `error(n)`, `errorCode(n)`,
`errorInfo(n)`, `info(n)`, `return(n)`

KEYWORDS

`catch`, `error`, `exception`

NAME

`cd` - Change working directory

SYNOPSIS

`cd ?dirName?`

DESCRIPTION

Change the current working directory to *dirName*, or to the home directory (as specified in the HOME environment variable) if *dirName* is not given. Returns an empty string. Note that the current working directory is a per-process resource; the `cd` command changes the working directory for all interpreters and (in a threaded environment) all threads.

EXAMPLES

Change to the home directory of the user **fred**:

```
cd ~fred
```

Change to the directory **lib** that is a sibling directory of the current one:

```
cd ../lib
```


SEE ALSO

`filename(n)`, `glob(n)`, `pwd(n)`

KEYWORDS

working directory

cell
Cell item

Description

The cell item type represents a cell (design instance) in the coreConsultant or coreBuilder model of the design. When coreBuilder parses the HDL source code for the core, it creates a cell item for each cell (design instance) that it finds.

See Also

Supported Attributes

[ComponentOfItem \(3\)](#), [CopyToTemplate \(3\)](#), [CustomizedTestbenchCode \(3\)](#),
[Description \(3\)](#), [DocInclude \(3\)](#), [GenerateIf \(3\)](#),
[ImportedWorkspaceComponentFile \(3\)](#), [IsWorkspaceComponent \(3\)](#), [Label \(3\)](#),
[LockedInTemplate \(3\)](#), [Name \(3\)](#), [ReadOnlyWorkspaceComponent \(3\)](#),
[ReplaceInHDL \(3\)](#), [Sequence \(3\)](#), [SolutionItemPhase \(3\)](#), [SpiritLibrary \(3\)](#),
[SpiritName \(3\)](#), [SpiritVendor \(3\)](#), [SpiritVersion \(3\)](#), [SymbolLibraryPath](#)
[\(3\)](#), [SymbolName \(3\)](#), [TypeName \(3\)](#), [VipInitializationCall \(3\)](#),
[WriteComponentInHDL \(3\)](#)

NAME

`chan` - Read, write and manipulate channels

SYNOPSIS

`chan option ?arg arg ...?`

DESCRIPTION

This command provides several operations for reading from, writing to and otherwise manipulating open channels (such as have been created with the **open** and **socket** commands, or the default named channels **stdin**, **stdout** or **stderr** which correspond to the process's standard input, output and error streams respectively). *Option* indicates what to do with the channel; any unique abbreviation for *option* is acceptable. Valid options are:

chan blocked *channelId*

This tests whether the last input operation on the channel called *channelId* failed because it would have otherwise caused the process to block, and returns 1 if that was the case. It returns 0 otherwise. Note that this only ever returns 1 when the channel has been configured to be non-blocking; all Tcl channels have blocking turned on by default.

chan close *channelId* ?*direction*?

Close and destroy the channel called *channelId*. Note that this deletes all existing file-events registered on the channel. If the *direction* argument (which must be **read** or **write** or any unique abbreviation of them) is present, the channel will only be half-closed, so that it can go from being read-write to write-only or read-only respectively. If a read-only channel is closed for reading, it is the same as if the channel is fully

coreTools Command Reference Index

closed, and respectively similar for write-only channels. Without the *direction* argument, the channel is closed for both reading and writing (but only if those directions are currently open). It is an error to close a read-only channel for writing, or a write-only channel for reading.

As part of closing the channel, all buffered output is flushed to the channel's output device (only if the channel is ceasing to be writable), any buffered input is discarded (only if the channel is ceasing to be readable), the underlying operating system resource is closed and *channelId* becomes unavailable for future use (both only if the channel is being completely closed). If the channel is blocking and the channel is ceasing to be writable, the command does not return until all output is flushed. If the channel is non-blocking and there is unflushed output, the channel remains open and the command returns immediately; output will be flushed in the background and the channel will be closed when all the flushing is complete.

If *channelId* is a blocking channel for a command pipeline then **chan close** waits for the child processes to complete.

If the channel is shared between interpreters, then **chan close** makes *channelId* unavailable in the invoking interpreter but has no other effect until all of the sharing interpreters have closed the channel. When the last interpreter in which the channel is registered invokes **chan close** (or **close**), the cleanup actions described above occur. With half-closing, the half-close of the channel only applies to the current interpreter's view of the channel until all channels have closed it in that direction (or completely). See the **interp** command for a description of channel sharing.

Channels are automatically fully closed when an interpreter is destroyed and when the process exits. Channels are switched to blocking mode, to ensure that all output is correctly flushed before the process exits.

The command returns an empty string, and may generate an error if an error occurs while flushing output. If a command in a command pipeline created with **open** returns an error, **chan close** generates an error (similar to the **exec** command.)

Note that half-closes of sockets and command pipelines can have important side effects because they result in a shutdown() or close() of the underlying system resource, which can change how other processes or systems respond to the Tcl program.

value?...

chan configure *channelId* *?optionName?* *?value?*
?optionName

Query or set the configuration options of the channel named *channelId*.

If no *optionName* or *value* arguments are supplied, the command returns a list containing alternating option names and values for the channel. If *optionName* is supplied but no *value* then the command returns the current value of the given option. If one or more pairs of *optionName* and *value* are supplied, the command sets each of the named options to the corresponding *value*; in this case the return value is an empty string.

The options described below are supported for all channels. In addition, each channel type may add options that only it supports. See the manual entry for the command that creates each type of channel for the options supported by that specific type of channel. For example, see the manual entry for the **socket** command for additional options for sockets, and the **open** command for additional options for serial devices.

-blocking *boolean*

The **-blocking** option determines whether I/O operations on the channel can cause the process to block indefinitely. The value of the option must be a proper boolean value. Channels are normally in blocking mode; if a channel is placed into non-blocking mode it will affect the operation of the **chan gets**, **chan read**, **chan puts**, **chan flush**, and **chan close** commands; see the documentation for those commands for details. For non-blocking mode to work correctly, the application must be using the Tcl event loop (e.g. by calling **Tcl_DoOneEvent** or invoking the **vwait** command).

-buffering *newValue*

If *newValue* is **full** then the I/O system will buffer output until its internal buffer is full or until the **chan flush** command is invoked. If *newValue* is **line**, then the I/O system will automatically flush output for the channel whenever a newline character is output. If *newValue* is **none**, the I/O system will flush automatically after every output operation. The default is for **-buffering** to be set to **full** except for channels that connect to terminal-like devices; for these channels the initial setting is **line**. Additionally, **stdin** and **stdout** are initially set to **line**, and **stderr** is set to **none**.

-buffersize *newSize*

Newvalue must be an integer; its value is used to set the size of buffers, in bytes, subsequently allocated for this channel to store input or output. *Newvalue* must be a number of no more than one million, allowing buffers of up to one million bytes in size.

-encoding *name*

coreTools Command Reference Index

This option is used to specify the encoding of the channel as one of the named encodings returned by **encoding names** or the special value **binary**, so that the data can be converted to and from Unicode for use in Tcl. For instance, in order for Tcl to read characters from a Japanese file in **shiftjis** and properly process and display the contents, the encoding would be set to **shiftjis**. Thereafter, when reading from the channel, the bytes in the Japanese file would be converted to Unicode as they are read. Writing is also supported - as Tcl strings are written to the channel they will automatically be converted to the specified encoding on output.

If a file contains pure binary data (for instance, a JPEG image), the encoding for the channel should be configured to be **binary**. Tcl will then assign no interpretation to the data in the file and simply read or write raw bytes. The Tcl **binary** command can be used to manipulate this byte-oriented data. It is usually better to set the **-translation** option to **binary** when you want to transfer binary data, as this turns off the other automatic interpretations of the bytes in the stream as well.

The default encoding for newly opened channels is the same platform- and locale-dependent system encoding used for interfacing with the operating system, as returned by **encoding system**.

-eofchar *char*

-eofchar {*inChar outChar*}

This option supports DOS file systems that use Control-z (\x1A) as an end of file marker. If *char* is not an empty string, then this character signals end-of-file when it is encountered during input. For output, the end-of-file character is output when the channel is closed. If *char* is the empty string, then there is no special end of file character marker. For read-write channels, a two-element list specifies the end of file marker for input and output, respectively. As a convenience, when setting the end-of-file character for a read-write channel you can specify a single value that will apply to both reading and writing. When querying the end-of-file character of a read-write channel, a two-element list will always be returned. The default value for **-eofchar** is the empty string in all cases except for files under Windows. In that case the **-eofchar** is Control-z (\x1A) for reading and the empty string for writing. The acceptable range for **-eofchar** values is \x01 - \x7f; attempting to set **-eofchar** to a value outside of this range will generate an error.

-translation *mode*

-translation {*inMode outMode*}

In Tcl scripts the end of a line is always represented using a single newline character (\n). However, in actual files and devices the end of a line may be represented differently on different platforms, or even

coreTools Command Reference Index

for different devices on the same platform. For example, under UNIX newlines are used in files, whereas carriage-return-linefeed sequences are normally used in network connections. On input (i.e., with **chan gets** and **chan read**) the Tcl I/O system automatically translates the external end-of-line representation into newline characters. Upon output (i.e., with **chan puts**), the I/O system translates newlines to the external end-of-line representation. The default translation mode, **auto**, handles all the common cases automatically, but the **-translation** option provides explicit control over the end of line translations.

The value associated with **-translation** is a single item for read-only and write-only channels. The value is a two-element list for read-write channels; the read translation mode is the first element of the list, and the write translation mode is the second element. As a convenience, when setting the translation mode for a read-write channel you can specify a single value that will apply to both reading and writing. When querying the translation mode of a read-write channel, a two-element list will always be returned. The following values are currently supported:

auto

As the input translation mode, **auto** treats any of newline (**lf**), carriage return (**cr**), or carriage return followed by a newline (**crlf**) as the end of line representation. The end of line representation can even change from line-to-line, and all cases are translated to a newline. As the output translation mode, **auto** chooses a platform specific representation; for sockets on all platforms Tcl chooses **crlf**, for all Unix flavors, it chooses **lf**, and for the various flavors of Windows it chooses **crlf**. The default setting for **-translation** is **auto** for both input and output.

binary

No end-of-line translations are performed. This is nearly identical to **lf** mode, except that in addition **binary** mode also sets the end-of-file character to the empty string (which disables it) and sets the encoding to **binary** (which disables encoding filtering). See the description of **-eofchar** and **-encoding** for more information.

cr

The end of a line in the underlying file or device is represented by a single carriage return character. As the input translation mode, **cr** mode converts carriage returns to newline characters. As the output translation mode, **cr** mode translates newline characters to carriage returns.

crlf

The end of a line in the underlying file or device is represented by a carriage return character followed by a linefeed character. As the input translation mode, **crlf** mode converts carriage-return-linefeed sequences to newline characters. As the output translation mode,

coreTools Command Reference Index

crlf mode translates newline characters to carriage-return-linefeed sequences. This mode is typically used on Windows platforms and for network connections.

lf

The end of a line in the underlying file or device is represented by a single newline (linefeed) character. In this mode no translations occur during either input or output. This mode is typically used on UNIX platforms.

chan copy *inputChan outputChan* **?-size** *size?* **?-command** *callback?*

Copy data from the channel *inputChan*, which must have been opened for reading, to the channel *outputChan*, which must have been opened for writing. The **chan copy** command leverages the buffering in the Tcl I/O system to avoid extra copies and to avoid buffering too much data in main memory when copying large files to slow destinations like network sockets.

The **chan copy** command transfers data from *inputChan* until end of file or *size* bytes or characters have been transferred; *size* is in bytes if the two channels are using the same encoding, and is in characters otherwise. If no **-size** argument is given, then the copy goes until end of file. All the data read from *inputChan* is copied to *outputChan*. Without the **-command** option, **chan copy** blocks until the copy is complete and returns the number of bytes or characters (using the same rules as for the **-size** option) written to *outputChan*.

The **-command** argument makes **chan copy** work in the background. In this case it returns immediately and the *callback* is invoked later when the copy completes. The *callback* is called with one or two additional arguments that indicates how many bytes were written to *outputChan*. If an error occurred during the background copy, the second argument is the error string associated with the error. With a background copy, it is not necessary to put *inputChan* or *outputChan* into non-blocking mode; the **chan copy** command takes care of that automatically. However, it is necessary to enter the event loop by using the **vwait** command or by using Tk.

You are not allowed to do other I/O operations with *inputChan* or *outputChan* during a background **chan copy**. If either *inputChan* or *outputChan* get closed while the copy is in progress, the current copy is stopped and the command callback is *not* made. If *inputChan* is closed, then all data already queued for *outputChan* is written out.

Note that *inputChan* can become readable during a background copy. You should turn off any **chan event** or **fileevent** handlers during a background copy so those handlers do not interfere with the copy. Any I/O attempted by a **chan event** or **fileevent** handler will get a error.

coreTools Command Reference Index

Chan copy translates end-of-line sequences in *inputChan* and *outputChan* according to the **-translation** option for these channels (see **chan configure** above). The translations mean that the number of bytes read from *inputChan* can be different than the number of bytes written to *outputChan*. Only the number of bytes written to *outputChan* is reported, either as the return value of a synchronous **chan copy** or as the argument to the callback for an asynchronous **chan copy**.

Chan copy obeys the encodings and character translations configured for the channels. This means that the incoming characters are converted internally first UTF-8 and then into the encoding of the channel **chan copy** writes to (see **chan configure** above for details on the **-encoding** and **-translation** options). No conversion is done if both channels are set to encoding **binary** and have matching translations. If only the output channel is set to encoding **binary** the system will write the internal UTF-8 representation of the incoming characters. If only the input channel is set to encoding **binary** the system will assume that the incoming bytes are valid UTF-8 characters and convert them according to the output encoding. The behaviour of the system for bytes which are not valid UTF-8 characters is undefined in this case.

chan create mode cmdPrefix

This subcommand creates a new script level channel using the command prefix *cmdPrefix* as its handler. Any such channel is called a **reflected** channel. The specified command prefix, **cmdPrefix**, must be a non-empty list, and should provide the API described in the **refchan** manual page. The handle of the new channel is returned as the result of the **chan create** command, and the channel is open. Use either **close** or **chan close** to remove the channel.

The argument *mode* specifies if the new channel is opened for reading, writing, or both. It has to be a list containing any of the strings or The list must have at least one element, as a channel you can neither write to nor read from makes no sense. The handler command for the new channel must support the chosen mode, or an error is thrown.

The command prefix is executed in the global namespace, at the top of call stack, following the appending of arguments as described in the **refchan** manual page. Command resolution happens at the time of the call. Renaming the command, or destroying it means that the next call of a handler method may fail, causing the channel command invoking the handler to fail as well. Depending on the subcommand being invoked, the error message may not be able to explain the reason for that failure.

Every channel created with this subcommand knows which interpreter it was created in, and only ever executes its handler command in that interpreter, even if the channel was shared with and/or was moved into a different interpreter. Each reflected channel also

coreTools Command Reference Index

knows the thread it was created in, and executes its handler command only in that thread, even if the channel was moved into a different thread. To this end all invocations of the handler are forwarded to the original thread by posting special events to it. This means that the original thread (i.e. the thread that executed the **chan create** command) must have an active event loop, i.e. it must be able to process such events. Otherwise the thread sending them will *block indefinitely*. Deadlock may occur.

Note that this permits the creation of a channel whose two endpoints live in two different threads, providing a stream-oriented bridge between these threads. In other words, we can provide a way for regular stream communication between threads instead of having to send commands.

When a thread or interpreter is deleted, all channels created with this subcommand and using this thread/interpreter as their computing base are deleted as well, in all interpreters they have been shared with or moved into, and in whatever thread they have been transferred to. While this pulls the rug out under the other thread(s) and/or interpreter(s), this cannot be avoided. Trying to use such a channel will cause the generation of a regular error about unknown channel handles.

This subcommand is **safe** and made accessible to safe interpreters. While it arranges for the execution of arbitrary Tcl code the system also makes sure that the code is always executed within the safe interpreter.

chan eof *channelId*

Test whether the last input operation on the channel called *channelId* failed because the end of the data stream was reached, returning 1 if end-of-file was reached, and 0 otherwise.

chan event *channelId* *event* *?script?*

Arrange for the Tcl script *script* to be installed as a *file event handler* to be called whenever the channel called *channelId* enters the state described by *event* (which must be either **readable** or **writable**); only one such handler may be installed per event per channel at a time. If *script* is the empty string, the current handler is deleted (this also happens if the channel is closed or the interpreter deleted). If *script* is omitted, the currently installed script is returned (or an empty string if no such handler is installed). The callback is only performed if the event loop is being serviced (e.g. via **vwait** or **update**).

A file event handler is a binding between a channel and a script, such that the script is evaluated whenever the channel becomes readable or writable. File event handlers are most commonly used to allow data to be received from another process on an event-driven basis, so that the receiver can continue to interact with the user or with other channels while waiting for the data to arrive. If an application invokes **chan gets** or **chan**

coreTools Command Reference Index

read on a blocking channel when there is no input data available, the process will block; until the input data arrives, it will not be able to service other events, so it will appear to the user to With **chan event**, the process can tell when data is present and only invoke **chan gets** or **chan read** when they will not block.

A channel is considered to be readable if there is unread data available on the underlying device. A channel is also considered to be readable if there is unread data in an input buffer, except in the special case where the most recent attempt to read from the channel was a **chan gets** call that could not find a complete line in the input buffer. This feature allows a file to be read a line at a time in non-blocking mode using events. A channel is also considered to be readable if an end of file or error condition is present on the underlying file or device. It is important for *script* to check for these conditions and handle them appropriately; for example, if there is no special check for end of file, an infinite loop may occur where *script* reads no data, returns, and is immediately invoked again.

A channel is considered to be writable if at least one byte of data can be written to the underlying file or device without blocking, or if an error condition is present on the underlying file or device. Note that client sockets opened in asynchronous mode become writable when they become connected or if the connection fails.

Event-driven I/O works best for channels that have been placed into non-blocking mode with the **chan configure** command. In blocking mode, a **chan puts** command may block if you give it more data than the underlying file or device can accept, and a **chan gets** or **chan read** command will block if you attempt to read more data than is ready; no events will be processed while the commands block. In non-blocking mode **chan puts**, **chan read**, and **chan gets** never block.

The script for a file event is executed at global level (outside the context of any Tcl procedure) in the interpreter in which the **chan event** command was invoked. If an error occurs while executing the script then the command registered with **interp bgerror** is used to report the error. In addition, the file event handler is deleted if it ever returns an error; this is done in order to prevent infinite loops due to buggy handlers.

chan flush *channelId*

Ensures that all pending output for the channel called *channelId* is written.

If the channel is in blocking mode the command does not return until all the buffered output has been flushed to the channel. If the channel is in non-blocking mode, the command may return before all buffered output has been flushed; the remainder will be flushed in the background as fast as the underlying file or device is able to absorb it.

chan gets *channelId* ?*varName*?

Reads the next line from the channel called *channelId*. If *varName* is not specified, the result of the command will be the line that has been read (without a trailing newline character) or an empty string upon end-of-file or, in non-blocking mode, if the data available is exhausted. If *varName* is specified, the line that has been read will be written to the variable called *varName* and result will be the number of characters that have been read or -1 if end-of-file was reached or, in non-blocking mode, if the data available is exhausted.

If an end-of-file occurs while part way through reading a line, the partial line will be returned (or written into *varName*). When *varName* is not specified, the end-of-file case can be distinguished from an empty line using the **chan eof** command, and the partial-line-but-non-blocking case can be distinguished with the **chan blocked** command.

chan names ?*pattern*?

Produces a list of all channel names. If *pattern* is specified, only those channel names that match it (according to the rules of **string match**) will be returned.

chan pending *mode* *channelId*

Depending on whether *mode* is **input** or **output**, returns the number of bytes of input or output (respectively) currently buffered internally for *channelId* (especially useful in a readable event callback to impose application-specific limits on input line lengths to avoid a potential denial-of-service attack where a hostile user crafts an extremely long line that exceeds the available memory to buffer it). Returns -1 if the channel was not opened for the mode in question.

chan pipe

Creates a standalone pipe whose read- and write-side channels are returned as a 2-element list, the first element being the read side and the second the write side. Can be useful e.g. to redirect separately **stderr** and **stdout** from a subprocess. To do this, spawn with "2>@" or ">@" redirection operators onto the write side of a pipe, and then immediately close it in the parent. This is necessary to get an EOF on the read side once the child has exited or otherwise closed its output.

Note that the pipe buffering semantics can vary at the operating system level substantially; it is not safe to assume that a write performed on the output side of the pipe will appear instantly to the input side. This is a fundamental difference and Tcl cannot conceal it. The overall stream semantics are compatible, so blocking reads and writes will not see most of the differences, but the details of what exactly gets written when are not. This is most likely to show up when using pipelines for testing; care should be taken to ensure that deadlocks do not occur and that potential short reads are allowed for.

coreTools Command Reference Index

chan pop *channelId*

Removes the topmost transformation from the channel *channelId*, if there is any. If there are no transformations added to *channelId*, this is equivalent to **chan close** of that channel. The result is normally the empty string, but can be an error in some situations (i.e. where the underlying system stream is closed and that results in an error).

chan postevent *channelId eventSpec*

This subcommand is used by command handlers specified with **chan create**. It notifies the channel represented by the handle *channelId* that the event(s) listed in the *eventSpec* have occurred. The argument has to be a list containing any of the strings **read** and **write**. The list must contain at least one element as it does not make sense to invoke the command if there are no events to post.

Note that this subcommand can only be used with channel handles that were created/opened by **chan create**. All other channels will cause this subcommand to report an error.

As only the Tcl level of a channel, i.e. its command handler, should post events to it we also restrict the usage of this command to the interpreter that created the channel. In other words, posting events to a reflected channel from an interpreter that does not contain its implementation is not allowed. Attempting to post an event from any other interpreter will cause this subcommand to report an error.

Another restriction is that it is not possible to post events that the I/O core has not registered an interest in. Trying to do so will cause the method to throw an error. See the command handler method **watch** described in **refchan**, the document specifying the API of command handlers for reflected channels.

This command is **safe** and made accessible to safe interpreters. It can trigger the execution of **chan event** handlers, whether in the current interpreter or in other interpreters or other threads, even where the event is posted from a safe interpreter and listened for by a trusted interpreter. **Chan event** handlers are *always* executed in the interpreter that set them up.

chan push *channelId cmdPrefix*

Adds a new transformation on top of the channel *channelId*. The *cmdPrefix* argument describes a list of one or more words which represent a handler that will be used to implement the transformation. The command prefix must provide the API described in the **transchan** manual page. The result of this subcommand is a handle to the transformation. Note that it is important to make sure that the transformation is capable of supporting the channel mode that it is used with or this can make the channel neither readable nor writable.

chan puts *?-nonewline? ?channelId? string*

Writes *string* to the channel named *channelId* followed by a newline character. A trailing newline character is written unless the optional flag **-nonewline** is given. If *channelId* is omitted, the string is written to the standard output channel, **stdout**.

Newline characters in the output are translated by **chan puts** to platform-specific end-of-line sequences according to the currently configured value of the **-translation** option for the channel (for example, on PCs newlines are normally replaced with carriage-return-linefeed sequences; see **chan configure** above for details).

Tcl buffers output internally, so characters written with **chan puts** may not appear immediately on the output file or device; Tcl will normally delay output until the buffer is full or the channel is closed. You can force output to appear immediately with the **chan flush** command.

When the output buffer fills up, the **chan puts** command will normally block until all the buffered data has been accepted for output by the operating system. If *channelId* is in non-blocking mode then the **chan puts** command will not block even if the operating system cannot accept the data. Instead, Tcl continues to buffer the data and writes it in the background as fast as the underlying file or device can accept it. The application must use the Tcl event loop for non-blocking output to work; otherwise Tcl never finds out that the file or device is ready for more output data. It is possible for an arbitrarily large amount of data to be buffered for a channel in non-blocking mode, which could consume a large amount of memory. To avoid wasting memory, non-blocking I/O should normally be used in an event-driven fashion with the **chan event** command (do not invoke **chan puts** unless you have recently been notified via a file event that the channel is ready for more output data).

chan read *channelId ?numChars?***chan read** *?-nonewline? channelId*

In the first form, the result will be the next *numChars* characters read from the channel named *channelId*; if *numChars* is omitted, all characters up to the point when the channel would signal a failure (whether an end-of-file, blocked or other error condition) are read. In the second form (i.e. when *numChars* has been omitted) the flag **-nonewline** may be given to indicate that any trailing newline in the string that has been read should be trimmed.

If *channelId* is in non-blocking mode, **chan read** may not read as many characters as requested: once all available input has been read, the command will return the data that is available rather than blocking for more input. If the channel is configured to use a multi-byte encoding, then there may actually be some

coreTools Command Reference Index

bytes remaining in the internal buffers that do not form a complete character. These bytes will not be returned until a complete character is available or end-of-file is reached. The **-newline** switch is ignored if the command returns before reaching the end of the file.

chan read translates end-of-line sequences in the input into newline characters according to the **-translation** option for the channel (see **chan configure** above for a discussion on the ways in which **chan configure** will alter input).

When reading from a serial port, most applications should configure the serial port channel to be non-blocking, like this:

chan configure *channelId* **-blocking 0**.

Then **chan read** behaves much like described above. Note that most serial ports are comparatively slow; it is entirely possible to get a **readable** event for each character read from them. Care must be taken when using **chan read** on blocking serial ports:

chan read *channelId numChars*

In this form **chan read** blocks until *numChars* have been received from the serial port.

chan read *channelId*

In this form **chan read** blocks until the reception of the end-of-file character, see **chan configure -eofchar**. If there no end-of-file character has been configured for the channel, then **chan read** will block forever.

chan seek *channelId offset ?origin?*

Sets the current access position within the underlying data stream for the channel named *channelId* to be *offset* bytes relative to *origin*. *Offset* must be an integer (which may be negative) and *origin* must be one of the following:

- | | |
|----------------|--|
| start | The new access position will be <i>offset</i> bytes from the start of the underlying file or device. |
| current | The new access position will be <i>offset</i> bytes from the current access position; a negative <i>offset</i> moves the access position backwards in the underlying file or device. |
| end | The new access position will be <i>offset</i> bytes from the end of the file or device. A negative <i>offset</i> places the access position before the end of file, and a positive <i>offset</i> places the access position after the end of file. |

The *origin* argument defaults to **start**.

Chan seek flushes all buffered output for the channel before the command returns, even if the channel is in

coreTools Command Reference Index

non-blocking mode. It also discards any buffered and unread input. This command returns an empty string. An error occurs if this command is applied to channels whose underlying file or device does not support seeking.

Note that *offset* values are byte offsets, not character offsets. Both **chan seek** and **chan tell** operate in terms of bytes, not characters, unlike **chan read**.

chan tell *channelId*

Returns a number giving the current access position within the underlying data stream for the channel named *channelId*. This value returned is a byte offset that can be passed to **chan seek** in order to set the channel to a particular position. Note that this value is in terms of bytes, not characters like **chan read**. The value returned is -1 for channels that do not support seeking.

chan truncate *channelId* *?length?*

Sets the byte length of the underlying data stream for the channel named *channelId* to be *length* (or to the current byte offset within the underlying data stream if *length* is omitted). The channel is flushed before truncation.

EXAMPLES

This opens a file using a known encoding (CP1252, a very common encoding on Windows), searches for a string, rewrites that part, and truncates the file after a further two lines.

```
set f [open somefile.txt r+]
chan configure $f -encoding cp1252
set offset 0

# Search for string "FOOBAR" in the file
while {[chan gets $f line] >= 0} {
    set idx [string first FOOBAR $line]
    if {$idx > -1} {
        # Found it; rewrite line

        chan seek $f [expr {$offset + $idx}]
        chan puts -nonewline $f BARFOO

        # Skip to end of following line, and truncate
        chan gets $f
        chan gets $f
        chan truncate $f

        # Stop searching the file now
        break
    }

    # Save offset of start of next line for later
    set offset [chan tell $f]
}
chan close $f
```


coreTools Command Reference Index

A network server that does echoing of its input line-by-line without preventing servicing of other connections at the same time.

```
# This is a very simple logger...
proc log {message} {
    chan puts stdout $message
}

# This is called whenever a new client connects to the server
proc connect {chan host port} {
    set clientName [format <%s:%d> $host $port]
    log "connection from $clientName"
    chan configure $chan -blocking 0 -buffering line
    chan event $chan readable [list echoLine $chan $clientName]
}

# This is called whenever either at least one byte of input
# data is available, or the channel was closed by the client.
proc echoLine {chan clientName} {
    chan gets $chan line
    if {[chan eof $chan]} {
        log "finishing connection from $clientName"
        chan close $chan
    } elseif {![chan blocked $chan]} {
        # Didn't block waiting for end-of-line
        log "$clientName - $line"
        chan puts $chan $line
    }
}

# Create the server socket and enter the event-loop to wait
# for incoming connections...
socket -server connect 12345
vwait forever
```

SEE ALSO

close(n), eof(n), fblocked(n), fconfigure(n), fcopy(n),
file(n), fileevent(n), flush(n), gets(n), open(n),
puts(n), read(n), seek(n), socket(n), tell(n),
refchan(n), transchan(n)

KEYWORDS

channel, input, output, events, offset

check_ParameterInfo

Check the syntax and semantics of a ParameterInfo list string

Syntax

string **check_ParameterInfo** *paramInfoList* [*toplevelMaxParamCount*]
[*toplevelMaxPageCount*]

string *paramInfoList*

string *toplevelMaxParamCount*

string *toplevelMaxPageCount*

Parameters

paramInfoList	parameterInfo list string to be checked This list (a TCL list) is the value for the ParameterInfo attribute, typically for a design or custom activity. The value is passed to this command to enable checking the syntax of the dialog building data contained within the ParameterInfo list string.
---------------	--

Description

check_ParameterInfo does a simple syntax and semantics check for the ParameterInfo attribute value. It ensures that the string is a valid Tcl list, and ensures the enum values of some of the statements are valid. check_ParameterInfo returns a report of values for each statement and Invalid values will be indicated in the report.

Examples

The following commands write out all parameter configuration information for the current design parameters, and do a ParameterInfo check:

```

coreBuilder> set design [current_design]
coreBuilder> set params [get_attribute $design -attr Parameters]
coreBuilder> set a [write_ParameterInfo $params -all]
{LayoutAs tab/groupbox/spreadsheet}
{MaxParamCount 15}
{Text {put the text here}}
{Label {put the label here}}
{GroupType subparam/layout }
{BoxType outline/indent }
{Visible {<param_expr> } }
{ColumnAttrs {Name} {Value}}
{ColumnLabels {label1} {label2} }
```

```

{Parameters {P1 P2 P3 P4} }
{Group Group1
{Parameters {P5} }
}
coreBuilder> check_ParameterInfo $a
Label = put the label here
ColumnLabels = label1 label2
GroupNames = Group1
BoxType = outline/indent
Text = put the text here
GroupType = subparam/layout
MaxParamCount = 15
LayoutAs = tab/groupbox/spreadsheet
ColumnAttrs = Name Value
Parameters = P1 P2 P3 P4
Visible = <param_expr>
Error: value 'subparam/layout' is not valid for statement GroupType,
specify one of: subparam,layout
Error: value 'tab/groupbox/spreadsheet' is not valid for statement
LayoutAs, specify one of: tab,groupbox,spreadsheet
Error: value 'outline/indent' is not valid for statement BoxType,
specify one of: outline,indent
GroupName: Group1
Parameters = P5
MaxParamCount = 15
LayoutAs = groupbox
boxType = outline
GroupType = layout

```

See Also

[ParameterInfo \(3\), write_ParameterInfo \(2\)](#)

[API Index](#) [Command Reference](#)

check_bom
Check Bill Of Materials

Syntax

string **check_bom**

Description

Examples

To check the current BoM:

```
coreBuilder> check_bom
```

See Also

[report_attribute \(2\)](#), [report_bom \(2\)](#)

check_component_for_parameter_loops

Check design parameters for the top design for loops return error count.

Syntax

```
string check_component_for_parameter_loops [-attrList <-attrList>]  
[-paramList <-paramList>] [-show_progress] [component]  
    string <-attrList>  
    string <-paramList>  
    string component
```

Parameters

-attrList	The list of attributes to check, default is
<-attrList>	{Enabled Visible Hidden DefaultValue Value}
-paramList	The list of params to check, all if none
<-paramList>	specified
-show_progress	Show progress not if not specified just show errors
component	The component to eval in

Description

Examples

See Also

check_env_vars

Check for the existence of a list of environment variables.

Syntax

```
string check_env_vars [-verbose] [-quiet] env_var_list
      string env_var_list
```

Parameters

-verbose	Print extra informational messages to the screen.
-quiet	Suppress all error and warning messages.
env_var_list	The list of the environment variables to check.

Description

This command verifies that each environment variable in the argument list is set. It does not check the value of the environment variable.

Return Status:

1: All of the environment variable(s) are set.

0: Otherwise

Examples

```
coreConsultant> check_env_vars SYNOPSISYS
1
coreConsultant> check_env_vars {DISPLAY SYNOPSISYS} -verbose
Information: Value of SYNOPSISYS is /d/dwt1/synopsys/2000.11 (CMDS-222)
Information: Value of DISPLAY is 198.182.51.134:0.0 (CMDS-222)
1
coreConsultant> check_env_vars {SYNOPSISYS BOGUS_ENV_VAR} -verbose
Error: Environment variable is not set: BOGUS_ENV_VAR (CMDS-220)
Information: Value of SYNOPSISYS is /d/dwt1/synopsys/2000.11 (CMDS-222)
0
coreConsultant> check_env_vars {SYNOPSISYS BOGUS_ENV_VAR} -quiet
0
```

See Also

[verify_environment \(2\)](#), [EnvCheck \(3\)](#)

check_error_list

List of messages IDs that should cause Design Compiler to stop in the case where one or more of the message appears during a call to the 'compile' command.

Syntax

```
string check_error_list = "OPT-100 LINK-5 LINK-9 LNK-005"
```

Description

check_error_list is a global variable that specifies the set of Design Compile message IDs which should be considered as errors if they occur during the synthesis process. The value of this variable is translated to the like-named variable in Design Compiler when any of the coreTools generated synthesis strategies is utilized.

Examples

To add additional message MYMSG-001:

```
set ::check_error_list [concat MYMSG-001 $::check_error_list]
```

See Also

[API Index](#) [Command Reference](#)

check_executables

Check for the existence of a list of executables.

Syntax

```
string check_executables [-verbose] [-quiet] exe_list  
      string exe_list
```

Parameters

-verbose	Print extra informational messages to the screen.
-quiet	Suppress all error and warning messages.
<i>exe_list</i>	The list of executables to check.

Description

Verifies that each executable in the argument list can be found using the paths in the PATH environment variable.

Return Status:

1: All of the executable(s) can be found.

0: Otherwise

Examples

```
coreConsultant> check_executables netscape  
1  
coreConsultant> check_executables {netscape perl} -verbose  
Information: Found perl at /usr/local/bin/perl (CMDS-222)  
Information: Found netscape at /usr/local/bin/netscape (CMDS-222)  
1  
coreConsultant> check_executables {perl bogus_exe} -verbose  
Error: Executable was not found: bogus_exe (CMDS-220)  
Information: Found perl at /usr/local/bin/perl (CMDS-222)  
0  
coreConsultant> check_executables {perl bogus_exe} -quiet  
0
```

See Also

[verify_environment \(2\)](#), [EnvCheck \(3\)](#)

`check_file_for_errors`

Check the specified file for error messages and return the number of error messages found

Syntax

```
string check_file_for_errors filename [error_keys]  
    string filename  
    string error_keys
```

Parameters

<code>filename</code>	The name of the file to be checked for errors.
<code>error_keys</code>	List of error strings to check for. The default is <code>{{Error:}}</code> .

Description

The `check_file_for_errors` command checks a file for lines that begin with the specified error string(s). This command is useful for checking log files for error messages. The command returns the number of lines in the specified file that begin with the specified string.

Examples

To check `logFile` for error messages that begin with the string "Error:" and store the number of error messages in the "errors" variable:

```
coreBuilder> set errors [check_file_for_errors logFile]
```

To check `logFile` for error messages that begin with the string "Error:" and warning messages that begin with the string "Warning:". Return the total count of errors and warnings:

```
coreBuilder> check_file_for_errors logFile {{Error:}} {Warning:}}
```

See Also

check_license

Get information about a license

Syntax

```
string check_license [-authorized] [-available] [-checkout] [-checkedout]
[-wasauthorized] [-project <project name>] [-version] <license name>
    string <project name>
    string <license name>
```

Parameters

-authorized	Check if user is authorized to use the named license.
-available	Check to see if user can check out the named license.
-checkout	Check out the named license.
-checkedout	Is the specified license checked out?
-wasauthorized	Has the specified license been previously authorized?
-project <project name>	Check project ID as part of license checking operation. This option should always be used for Synopsys IP products (within reuse-pragmas for parameters and Prime Profiles). Since the customer project ID is not known in advance, the value "-" should be used as the project ID value. This special value tells the tool to use the project ID that was used to do the initial IP image installation.
-version	This key is part of a versioned key set.
<license name>	Name of the license to check.

Description

The check_license command checks the availability of the named license key. If you specify the -available option, the coreTool returns "1" if a key is available for the specified license or "0" if a key is not available. If you specify the -authorized option, the coreTool returns "1" if you are authorized to check out the specified license or "0" if you are not authorized to check out the specified license.

For more information about license key installation and license key file variables, refer to the coreTools Release Notes.

Examples

To check if user is authorized to check out a DW-IP-Developer license:

```
coreBuilder> check_license DW-IP-Developer -authorized
```

To check if a Design-Compiler license is available:

```
coreBuilder> check_license Design-Compiler -available
```

Within an RTL file, use a license to enable a parameter

```
// reuse-pragma attr Enabled [check_license My-Product-License -project  
-]
```

See Also

`check_parameter_attribute_formulas`

Check the formula value of an attribute on a parameter for any issues like dependancy loops.

Syntax

string **check_parameter_attribute_formulas** [-secondaryattr <-secondaryattr>] [-all_paths] [-errors_only] [-favor_design_params]

param attr

string <-secondaryattr>

string param

string attr

Parameters

-secondaryattr <-secondaryattr>	The name of the attribute to use for checking all recursive evaluations below the top.
-all_paths	If set outputs all paths otherwise only unique paths.
-errors_only	Only output error information.
-favor_design_params	If multiple parameters found from formula params favor the design parameters.
param	The name of the parameter for which you want to check the attribute for.
attr	The name of the attribute for which you want to check.

Description

This command is used to check for loops in parameter attribute formulas. A loop is a condition where the attribute value of parameter A depends on a formula. The values of one of the parameters in the formula for A, say param B, depends on the value of the parameter A. Loops can be detected at any depth level with this command.

Loops are a problem in the DefaultValue and Enabled parameter attributes most frequently since these are often formulas containing other parameters. If the attribute specified has subscripts then we iterate over all of the subscripts checking for loops.

If loops are detected they should be fixed because it can change the value depending on where the calculation starts yielding incorrect results. Also frequently the disabled value is different from the current

value for a parameter so if we incorrectly calculate it to be disabled it ends up changing the value the user sees to the wrong value.

Examples

```
coreConsultant> check_parameter_attribute_formulas $param $attr  
-errors_only  
coreConsultant> check_parameter_attribute_formulas MyParam1 Enabled
```

See Also

[get_parameter_attribute\(2\)](#) [set_parameter_attribute\(2\)](#)

NAME

check_script Statically check a script using the a static Tcl syntax checker based on TclPro Procheck and the Synopsys checking extensions for this tool.

SYNTAX

```
string snpsTclPro::check_script
[-no_tclchecker_warning]
    [-quiet] [-onepass] [-verbose] [-suppress
    messageID] [-application appName] [-W1] [-W2]
    [-W3] [-Wall] filePattern
```

```
string     messageID
string     appName
string     filePattern
```

ARGUMENTS

```
-no_tclchecker_warning
    Don't warn that TclDevKit cannot be
    found

-quiet                prints minimal error information

-onepass             perform a single pass without checking
                     proc args

-verbose             prints summary information

-suppess messageIDs
    prevent given messageIDs from being
    printed

-application appName
    Check script against application other
    than this one

-W1                  print only error messages
```

coreTools Command Reference Index

-W2	print only error and usage warnings
-W3	print all errors and warnings
-Wall	print all types of messages (same as W3)

filePattern file(s) to be checked

USING THE PACKAGE

The **check_script** command is in the snpsTclPro Tcl package, and all of the commands in this package are defined in the namespace snpsTclPro. The way to use this command is to load the package, which will import the commands it exports into the global namespace. This is shown below.

```
shell> package require snpsTclPro
1.0
```

These commands can be added to the .synopsys setup file for the system, user, or current directory, or the commands can be typed when the package is needed.

DESCRIPTION

The **check_script** command simplifies running the TclDevKit tclchecker program. This application is available for purchase from ActiveState (<http://www.activestate.com>). If the TclDevKit is not available then there is a limited Synopsys Tcl syntax checker that is used by this command. The Synopsys version of this code does not understand Tcl8.4 constructs, and some other checking is limited.

These checkers will check builtin Tcl commands, as well as the Synopsys extension commands which are found in the script. **check_script** will locate the Synopsys checking extensions for the application and will then invoke the checker. The limited checker does not require any external license. It is part of the Synopsys installation.

The **check_script** command will raise a Tcl error if the checker detects errors in the scripts being checked.

The **check_script** command is provided as a convenience

coreTools Command Reference Index

to streamline the use of the syntax checker within Synopsys applications. The checker can also be called directly on scripts, without requiring a license for the Synopsys application whose script is being checked. This checker script is available in the auxx/tcllib/bin/check_script in the installation directory of your application. When run standalone the -application option must be specified.

Synopsys Checker Extensions

The Synopsys extensions to the TclPro checker define a number of new warnings and errors. These messages are listed below, along with a short explanation of them.

Error **MisVal**

An option to a command which requires a value is missing its value.

Error **MisReq**

A required option or argument was not specified.

Error **Excl**

Two options which are mutually exclusive were both specified.

Error **ExtraPos**

An extra positional argument was specified to a command.

Error **BadRange**

The value specified for a given argument was outside of the legal range.

Error **UnkCmd**

The command specified is not known by the application.

Error **UnkOpt**

The specified option is not legal for the command.

Error **AmbOpt**

The option specified is not complete and matches 2 or more options for the command.

Warning **NotLit**

This warning indicates that the argument specified to an option was not a literal and therefore could not be checked. This message is suppressed by default. It can be enabled by setting the environment variable SNPS_TCL_NOLITERALWAN to true.

Warning **DupIgn**

An option was specified multiple times to the command, and the first value specified will be the one used. The other values will be ignored.

Warning **DupOver**

An option was specified multiple times to the command,

coreTools Command Reference Index

and the last value specified will be the one used. The other values will be ignored.

EXAMPLES

```
shell> check_script myscript.tcl
```

```
Loading snps_tcl.pcx...
Loading coreConsultant.pcx...
scanning: /u/user1/myscript.tcl
checking: /u/user1/myscript.tcl
test.tcl:3 (warnVarRef) variable reference used where variable name expected
set $a $b
    ^
test.tcl:5 (SnpsE-MisReq) Missing required positional options for foreach_in_col
foreach_in_collection x {
}
^
test.tcl:9 (SnpsE-BadRange) Value -1 for 'index_collection index' must be >= 0
index_collection $a -1
child process exited abnormally
Error: Errors found in script.
      Use error_info for more info. (CMD-013)
```

```
shell> check_script -Wall anotherScript.tcl
```

```
Loading snps_tcl.pcx...
scanning: /u/user1/anotherScript.tcl
checking: /u/user1/anotherScript.tcl
```

CAVEATS

The Synopsys extension messages cannot be suppressed. Aliases and abbreviated commands will be flagged as undefined procedures.

SEE ALSO

debug_script(2) **package(2)** **namespace(2)**

check_workspace_project_id

Compares the Project ID specified against the Project ID used to create the workspace

Syntax

```
string check_workspace_project_id -project <Project ID>  
      string <Project ID>
```

Parameters

-project <Project ID> ProjectID to compare against.

Description

The check_workspace_project_id compares the 'Project ID' specified against the 'Project ID' used to create the workspace. If the two 'Project ID' match the command returns '1' else returns '0'.

Examples

Check if the workspace was created with Project ID 'PID001'

```
coreConsultant> check_workspace_project_id -project PID001  
0
```

Check if the workspace was created with Project ID 'us01dwpid000'

```
coreConsultant> check_workspace_project_id -project us01dwpid000  
1
```

See Also

NAME

`oo::class` - class of all classes

SYNOPSIS

`package require TclOO`

`oo::class method ?arg ...?`

CLASS HIERARCHY

`oo::object`
 -> `oo::class`

DESCRIPTION

Classes are objects that can manufacture other objects according to a pattern stored in the factory object (the class). An instance of the class is created by calling one of the class's factory methods, typically either **create** if an explicit name is being given, or **new** if an arbitrary unique name is to be automatically selected.

The `oo::class` class is the class of all classes; every class is an instance of this class, which is consequently an instance of itself. This class is a subclass of `oo::object`, so every class is also an object. Additional metaclasses (i.e., classes of classes) can be defined if necessary by subclassing `oo::class`. Note that the `oo::class` object hides the **new** method on itself, so new classes should always be made using the **create** method.

coreTools Command Reference Index

CONSTRUCTOR

The constructor of the **oo::class** class takes an optional argument which, if present, is sent to the **oo::define** command (along with the name of the newly-created class) to allow the class to be conveniently configured at creation time.

DESTRUCTOR

The **oo::class** class does not define an explicit destructor. However, when a class is destroyed, all its subclasses and instances are also destroyed, along with all objects that it has been mixed into.

EXPORTED METHODS

cls create name ?arg ...?

This creates a new instance of the class *cls* called *name* (which is resolved within the calling context's namespace if not fully qualified), passing the arguments, *arg ...*, to the constructor, and (if that returns a successful result) returning the fully qualified name of the created object (the result of the constructor is ignored). If the constructor fails (i.e. returns a non-OK result) then the object is destroyed and the error message is the result of this method call.

cls new ?arg ...?

This creates a new instance of the class *cls* with a new unique name, passing the arguments, *arg ...*, to the constructor, and (if that returns a successful result) returning the fully qualified name of the created object (the result of the constructor is ignored). If the constructor fails (i.e., returns a non-OK result) then the object is destroyed and the error message is the result of this method call.

Note that this method is not exported by the **oo::class** object itself, so classes should not be created using this method.

NON-EXPORTED METHODS

The **oo::class** class supports the following non-exported methods:

cls createWithNamespace name nsName ?arg ...?

This creates a new instance of the class *cls* called *name* (which is resolved within the calling context's namespace if not fully qualified), passing the arguments, *arg ...*, to the constructor, and (if that returns a successful result) returning the fully qualified name of the created object (the result of the constructor is ignored). The name of the instance's internal namespace will be *nsName* unless that namespace already exists (when an arbitrary name will be chosen instead). If the constructor fails (i.e., returns a non-OK result) then the object is destroyed and the error message is the result of this method call.

EXAMPLES

This example defines a simple class hierarchy and creates a new instance of it. It then invokes a method of the object before destroying the hierarchy and showing that the destruction is transitive.

```
oo::class create fruit {
  method eat {} {
    puts "yummy!"
  }
}
oo::class create banana {
  superclass fruit
  constructor {} {
    my variable peeled
    set peeled 0
  }
  method peel {} {
    my variable peeled
    set peeled 1
    puts "skin now off"
  }
  method edible? {} {
    my variable peeled
    return $peeled
  }
  method eat {} {
    if {[my edible?]} {
      my peel
    }
    next
  }
}
set b [banana new]
$b eat                -> prints "skin now off" and "yummy!"
fruit destroy
$b eat                -> error "unknown command"
```

SEE ALSO

oo::define(n), oo::object(n)

KEYWORDS

class, metaclass, object

`clear_prime_profile_validation_exprs`
 Clears all validation expressions for Prime Profiles.

Syntax

string `clear_prime_profile_validation_exprs`

Description

Clears all Prime Profile validation expressions created in current session (coreBuilder only command). This command can be useful when IP designers are developing validation expressions for the Prime Profiles they have created in an interactive session. Clearing incorrectly designed validation expressions allows users to continue the current session to create correct Prime Profile validation expressions.

Examples

```
coreBuilder> set_prime_profile_validation_expr -tag_list "DisableRom1
DisableRom2" -expression "@dummy == 1" -message "DisableRom(1|2) profiles
must be applied when setting dummy to 1."
coreBuilder> set_prime_profile_validation_expr -tag_list "LockRamSize"
-expression "@dummy == 2"
coreBuilder> set_prime_profile_validation_expr -tag_list "Test003
BadLicense2" -expression "@dummy == 3"
coreBuilder> report_prime_profile_validation_expr
mycore:
0:
exprTxt: "@dummy == 1"
tagList: DisableRom1 DisableRom2
msg: "DisableRom(1|2) profiles must be applied when setting dummy to 1."
1:
exprTxt: "@dummy == 2"
tagList: LockRamSize
msg: ""
2:
exprTxt: "@dummy == 3"
tagList: Test003 BadLicense2
msg: ""
coreBuilder> clear_prime_profile_validation_exprs
1
coreBuilder> report_prime_profile_validation_expr
mycore:
```


See Also

[set_prime_profile \(2\)](#) [set_prime_profile_validation_expr \(2\)](#)

[report_prime_profile_validation_expr \(2\)](#) [report_prime_profiles \(2\)](#)

clock
Clock item

Description

The clock item type represents a clock in the coreConsultant or coreBuilder model of a design. coreBuilder creates a clock item when you set the ClockName attribute on a port or when you execute the create_virtual_clock command.

See Also

[create_virtual_clock \(2\)](#), [ClockName \(3\)](#)

Supported Attributes

[ClockFallLatency \(3\)](#), [ClockGatingFallHoldCheck \(3\)](#),
[ClockGatingFallSetupCheck \(3\)](#), [ClockGatingRiseHoldCheck \(3\)](#),
[ClockGatingRiseSetupCheck \(3\)](#), [ClockHoldUncertainty \(3\)](#), [ClockName \(3\)](#),
[ClockRiseLatency \(3\)](#), [ClockSetupUncertainty \(3\)](#), [ClockSourceFallLatency](#)
[\(3\)](#), [ClockSourceRiseLatency \(3\)](#), [ComponentOfItem \(3\)](#), [CreateSkewGroup](#)
[\(3\)](#), [CycleTime \(3\)](#), [Description \(3\)](#), [FixHold \(3\)](#),
[InterClockHoldFallFallUncertainty \(3\)](#), [InterClockHoldFallRiseUncertainty](#)
[\(3\)](#), [InterClockHoldRiseFallUncertainty \(3\)](#),
[InterClockHoldRiseRiseUncertainty \(3\)](#), [InterClockSetupFallFallUncertainty](#)
[\(3\)](#), [InterClockSetupFallRiseUncertainty \(3\)](#),
[InterClockSetupRiseFallUncertainty \(3\)](#),
[InterClockSetupRiseRiseUncertainty \(3\)](#), [IsCCDTarget \(3\)](#), [Label \(3\)](#),
[MaxFallTransitionDelay \(3\)](#), [MaxRiseTransitionDelay \(3\)](#),
[MinFallTransitionDelay \(3\)](#), [MinRiseTransitionDelay \(3\)](#), [Name \(3\)](#),
[ReferenceClock \(3\)](#), [ReferenceClockName \(3\)](#), [Sequence \(3\)](#), [TestClock \(3\)](#),
[TestClockCycleTime \(3\)](#), [TestClockWaveform \(3\)](#), [TypeName \(3\)](#), [Waveform \(3\)](#)

clone_component

clone a component in the subsystem.

Syntax

```
string clone_component componentName  
      string componentName
```

Parameters

componentName The name of the existing component to clone.

Description

The clone_component command finds all of the connected cells through out the hierarchy and inserts a new clone in each of the unique hierarchical contexts. It then replaces the original cells connections to the cells at each hierarchical level so that it is now connected to the cloned cell.

Examples

```
coreAssembler> clone_component i_driver
```

See Also

NAME

`close` - Close an open channel

SYNOPSIS

`close channelId ?r(ead)|w(rite)?`

DESCRIPTION

Closes or half-closes the channel given by *channelId*.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension.

The single-argument form is a simple all buffered output is flushed to the channel's output device, any buffered input is discarded, the underlying file or device is closed, and *channelId* becomes unavailable for use.

If the channel is blocking, the command does not return until all output is flushed. If the channel is nonblocking and there is unflushed output, the channel remains open and the command returns immediately; output will be flushed in the background and the channel will be closed when all the flushing is complete.

If *channelId* is a blocking channel for a command pipeline then **close** waits for the child processes to complete.

If the channel is shared between interpreters, then **close** makes *channelId* unavailable in the invoking

coreTools Command Reference Index

interpreter but has no other effect until all of the sharing interpreters have closed the channel. When the last interpreter in which the channel is registered invokes **close**, the cleanup actions described above occur. See the **interp** command for a description of channel sharing.

Channels are automatically closed when an interpreter is destroyed and when the process exits. From 8.6 on (TIP#398), nonblocking channels are no longer switched to blocking mode when exiting; this guarantees a timely exit even when the peer or a communication channel is stalled. To ensure proper flushing of stalled nonblocking channels on exit, one must now either (a) actively switch them back to blocking or (b) use the environment variable `TCL_FLUSH_NONBLOCKING_ON_EXIT`, which when set and not equal to "0" restores the previous behavior.

The command returns an empty string, and may generate an error if an error occurs while flushing output. If a command in a command pipeline created with **open** returns an error, **close** generates an error (similar to the **exec** command.)

The two-argument form is given a bidirectional channel like a socket or command pipeline and a (possibly abbreviated) direction, it closes only the sub-stream going in that direction. This means a shutdown() on a socket, and a close() of one end of a pipe for a command pipeline. Then, the Tcl-level channel data structure is either kept or freed depending on whether the other direction is still open.

A single-argument close on an already half-closed bidirectional channel is defined to just A half-close on an already closed half, or on a wrong-sided unidirectional channel, raises an error.

In the case of a command pipeline, the child-reaping duty falls upon the shoulders of the last close or half-close, which is thus allowed to report an abnormal exit error.

Currently only sockets and command pipelines support half-close. A future extension will allow reflected and stacked channels to do so.

EXAMPLE

This illustrates how you can use Tcl to ensure that files get closed even when errors happen by combining **catch**, **close** and **return**:

```
proc withOpenFile {filename channelVar script} {
    upvar 1 $channelVar chan
    set chan [open $filename]
    catch {
        uplevel 1 $script
    }
```

```
} result options
close $chan
return -options $options $result
}
```

SEE ALSO

file(n), open(n), socket(n), eof(n),
Tcl_StandardChannels(3)

KEYWORDS

blocking, channel, close, nonblocking, half-close

close_workspace

Close the current workspace.

Syntax

string **close_workspace** [-save]

Parameters

-save Save the workspace if it is modified before it is closed.

Description

The close_workspace command closes the currently loaded workspace, removes the workspace knowledge base (kb) and all kbs it requires from the memory. If -save is specified, it saves all modified kbs before closing, otherwise, kbs will be forced to remove. If there is no currently loaded workspace, an error will occur.

Usually this command is used before opening or creating another workspace. In the coreTools GUI, the user will be prompted to close the current workspace.

In the coreTools GUI, you can close a workspace from the menu File->Close.

Examples

The following command closes the currently loaded workspace without saving:

```
coreConsultant> close_workspace
```

The following command closes the currently loaded workspace and saves all modified kbs before closing:

```
coreConsultant> close_workspace -save
```

See Also

[create_workspace \(2\)](#), [open_workspace \(2\)](#), [save_workspace \(2\)](#),
[add_workspace_hook \(2\)](#)

collection_print_item_limit

Number of item names to print when creating a collection.

Syntax

```
string collection_print_item_limit = "10"
```

Description

The collection_print_item_limit variable determines how many item names coreConsultant or coreBuilder will print when returning a collection. For example, when you execute the all_inputs command, coreConsultant or coreBuilder creates a collection that includes all input and inout ports of the current_design, but only prints the number of port names specified by collection_print_item_limit. If collection_print_item_limit = 10, the coreTool prints the names of the first 10 ports in the collection, then prints an ellipsis (...) to indicate that there are additional items in the collection.

Examples

To print only three items when returning a collection:

```
coreConsultant> all_inputs
{clk input1 input2 input3 input4 input5 input6 input7 input8 input9 ...}
coreConsultant> set collection_print_item_limit 3
3
coreConsultant> all_inputs
{clk input1 input2 ...}
```

See Also

combineValues

Combine the values of the specified attribute
on the specified items according to the specified command

Syntax

string **combineValues** *items attr combineProc*

string *items*

string *attr*

string *combineProc*

Parameters

<i>items</i>	A list of items for which to retrieve and combine the values of the specified attribute.
<i>attr</i>	The attribute for which you want to extract and combine values. For subscripted attributes, use the attrName[subscript] syntax.
<i>combineProc</i>	The command to use to combine the list of values. Examples include logic_or, logic_and, min, max, and sum. However, you can specify any valid Tcl command or custom command procedure.

Description

The combineValues command extracts the values of an attribute from one or more items and combines the values to form a result.

You must specify the name of the Tcl command you want to use to combine the values (*<cmd> argument*). You can use any valid Tcl command to combine the values.

Examples

To get the maximum value of MaxOutputDelay[clk] on ports o1, o2, and o3:

```
coreBuilder> combineValues {o1 o2 o3} {MaxOutputDelay[clk]} max
20
```

To get the sum of AreaEstimate values on all subdesigns of the current_design:

```
coreBuilder> combineValues [all_subdesigns] AreaEstimate sum
850
```

See Also

[CombineInheritValues \(2\)](#), [InheritValue \(2\)](#), _____

compare_dc_version

Compare a version string with the version of Design Compiler currently in use

Syntax

string **compare_dc_version** *version*
 string *version*

Parameters

version	The version string to compare against the DC version that is currently in use.
---------	--

Description

The compare_dc_version command compares the specified Design Compiler version (<version>) with the Design Compiler version currently in use and returns one of the following results:

-1 if the version of Design Compiler currently in use is less (older) than (<version>).

0 if the version of Design Compiler currently in use is the same as (<version>).

1 if the version of Design Compiler currently in use is greater (newer) than (<version>).

As a core developer, you can use compare_dc_version to control the flow of a Tcl script depending on the version of Design Compiler a core integrator is currently using.

The command can only compare the DC version after activity 'TechSetup' is completed. If the tool root for dc_shell is updated in that cas:wpq the user needs to autocomplete TechSetup prior to successfully using compare_dc_version command.

Examples

To determine whether the Design Compiler version currently in use is less than (-1), equal to (0), or greater than (1) 1999.05:

```
coreBuilder> compare_dc_version 1999.05  
0
```

To set CanFlatten to true on TOP if the Design Compiler version in use is greater than or equal to 1999.05:

```
if { [compare_dc_version 1999.05] >= 0 } {  
set_design_attribute -designs TOP CanFlatten true  
}
```

See Also

complete_custom_activity_definition
Complete a custom activity definition

Syntax

```
string complete_custom_activity_definition -activity <activity> -sequence
<number> [-prereqs <activities>] [-postreqs <activities>] [-group_boxes]
    string <activity>
    int <number>
    string <activities>
```

Parameters

-activity <activity>	Specifies the custom activity for which you want to complete the definition. <activity> is the same activity name that you specified with the -name option to the create_custom_activity command that created the custom activity.
-sequence <number>	Specifies the sequence number for the custom activity. The sequence number determines where the custom activity will occur in the coreTool design flow. To determine the sequence numbers of existing activities, use the command "report_attribute -attrs Sequence [find_item -type activity]".
-prereqs <activities>	Specifies the names of activities that must be completed before the custom activity is enabled.
-postreqs <activities>	Specifies the names of activities that cannot be enabled until the custom activity has been completed.
-group_boxes	Group parameters in boxes instead of separate tabs.

Description

The complete_custom_activity_definition command completes the definition of a coreTool custom activity.

The create_custom_activity command creates the custom activity. The create_activity_parameter command creates a parameter for the custom activity. Then, the complete_custom_activity_definition command completes the definition of the custom activity, sets a sequence number for the custom activity, and sets up links to pre-requisite and post-requisite (dependent) activities.

The `-sequence` option assigns a sequence number to the custom activity. Every activity has a unique sequence number (Sequence attribute). Activities appear in each activity group of the coreTool Activity List in order of increasing sequence number.

The `-prereqs` and `-postreqs` options specify which activities must be completed before the custom activity and which activities can only be completed after the custom activity.

The **EXAMPLES** section below shows how to use the `complete_custom_activity_definition` command. For a complete example that shows how to use `create_custom_activity`, `create_activity_parameter`, and `complete_custom_activity_definition` together, refer to the `create_custom_activity` man page.

Examples

To complete the definition of custom coreConsultant activity `SetUpGateSim` with `Specify Synthesize` as a pre-requisite activity and `RunGateSim` as a post-requisite activity, using boxes to group activity parameters instead of using tabs:

```
complete_custom_activity_definition -activity SetUpGateSim \  
-sequence 580 \  
-prereqs Synthesize \  
-postreqs RunGateSim \  
-group_boxes
```

To complete the definition of a custom coreBuilder activity `DoLint` and specify `LoadDesigns` as a pre-requisite activity:

```
complete_custom_activity_definition -activity DoLint\  
-sequence 250 \  
-prereqs LoadDesigns
```

See Also

[create_custom_activity\(2\), create_custom_activity_parameter\(2\)](#)

`complete_interface_definition`

Complete an interface definition, and mark the definition as unchangable

Syntax

string **complete_interface_definition** *name*
 string *name*

Parameters

name Name of the interface to complete

Description

This command must be used to 'complete' the description of an interface definition. Interface definitions can not be utilized within coreBuilder if they have not been completed. Once an interface definition has been completed, it can no longer be modified in any way. This command indicates to the tool that the definition of the interface has been completed, and that it is now okay to create instances of the definition. This command is used exclusively within TCL command files which create interface definitions.

Examples

To create and complete a simple interface definition:

```
create_interface myInterface
complete_interface_definition myInterface
```

See Also

`create_interface (2)`, `create_interface_port (2)`, `create_interface_parameter (2)`

compress_sdc

Inputs an SDC file and writes out a bus-compressed version of the input file

Syntax

```
string compress_sdc [-verbose] in_filename out_filename  
    string in_filename  
    string out_filename
```

Parameters

-verbose	Print messages about compression as it occurs
in_filename	The Input SDC file name
out_filename	The Output SDC file name with bus compressed SDC commands

Description

This command can be used to compress a "bit-blasted" SDC file to improve the performance of the read_sdc command in the coreTools. This command currently only compresses delay constraints as they are the primary cause of slow read times. When all bits of a bus have the same delay constraint applied, the constraint is compressed into a single constraint on the port instead of individual constraints on each bit of the bus.

Examples

```
compress_sdc original.sdc compressed.sdc
```

See Also

[read_sdc \(2\)](#), [write_sdc \(2\)](#)

NAME

`concat` - Join lists together

SYNOPSIS

`concat ?arg arg ...?`

DESCRIPTION

This command joins each of its arguments together with spaces after trimming leading and trailing white-space from each of them. If all of the arguments are lists, this has the same effect as concatenating them into a single list. Arguments that are empty (after trimming) are ignored entirely. It permits any number of arguments; if no *args* are supplied, the result is an empty string.

EXAMPLES

Although **concat** will concatenate lists, flattening them in the process (so giving the following interactive session):

```
% concat a b {c d e} {f {g h}}  
a b c d e f {g h}
```

it will also concatenate things that are not lists, as can be seen from this session:

```
% concat " a b {c " d " e} f"  
a b {c d e} f
```

coreTools Command Reference Index

Note also that the concatenation does not remove spaces from the middle of values, as can be seen here:

```
% concat "a  b  c" { d e f }  
a  b  c d e f
```

(i.e., there are three spaces between each of the **a**, the **b** and the **c**).

SEE ALSO

`append(n)`, `eval(n)`, `join(n)`

KEYWORDS

`concatenate`, `join`, `list`

connect_interface
Connect interfaces

Syntax

```
string connect_interface [-from_component <component 1>] [-from_interface
<interface instance 1>] [-to_component <component 2>] [-to_interface
<interface instance 2>] [-hierarchy] [-name <interconnection name>]
    string <component 1>
    string <interface instance 1>
    string <component 2>
    string <interface instance 2>
    string <interconnection name>
```

Parameters

-from_component <component 1>	the first component name of the interface to be connected
-from_interface <interface instance 1>	name of the first interface instance to be connected
-to_component <component 2>	the second component name of the interface to be connected
-to_interface <interface instance 2>	name of the second interface instance to be connected
-hierarchy	This connection crosses hierarchical boundaries.
-name <interconnection name>	name to be associated with the connection between the interfaces

Description

The connect_interface command connects unique interfaces in the two components specified. If all options are specified, the command will connect the two specified interface instances in the specified components. If only -from_component and -to_component are specified, and the interface instances in both components are unique, the command will connect the two unique instance. If there are more than one set of unique interface instances in both components to connect, the user needs to specify which set to connect. Exported interface only need to specify the interface name. Only the following types of interface instances can be connected:

- Instance1 to Instance2
- provider to consumer

- consumer to provider
- provider to internal-consumer
- internal-consumer to provider
- provider to interface-monitor
- interface-monitor to provider
- consumer to consumer-monitor
- consumer-monitor to consumer

Generally, the interfaces instances must both be visible in the same level of hierarchy. This restriction is lifted when the current workspace was created as a testbench workspace. In that situation, interfaces in different hierarchical levels may be connected if one of them is within the DUT, and the other interface has only input ports.

It is also lifted if the `-hierarchy` option is utilized. In that case, connection crossing hierarchical boundaries can be made provided that both interfaces are visible within or below the current context. In this case interfaces from one or both of the connection points will be automatically exported up to the current context level and then a connection is made in the current context.

Examples

Example 1

For a subsystem with a Component BUS interface "Provider Instance DATA1" of interfaceDefn data" and a Component DUMMY interface "Consumer Instance DATA" of interfaceDefn data, all instances are unique and there is only one provider/consumer with the same interfaceDefn in the `-from/-to` component.
The following command will connect BUS/DATA1 to DUMMY/DATA: `@code coreAssembler> connect_interfaces -from_component BUS -to_component DUMMY \endcode`
Example 2
For a subsystem with Component BUS interfaces "Provider Instance CLK" of interfaceDefn clock and "Provider Instance DATA1" of interfaceDefn data, and Component DUMMY interfaces "Consumer Instance CLK" of interfaceDefn clock and "Consumer Instance DATA" of interfaceDefn data, there are two sets of unique provider/consumers with the same interfaceDefn in the `-from/-to` component: {BUS/CLK, DUMMY/CLK} and {BUS/DATA1 to DUMMY/DATA}. To be safe, the instance names need to be specified to make the correct connection. The following command produce an error: `@code coreAssembler> connect_interfaces -from_component BUS -to_component DUMMY \endcode` The following command will connect {BUS/CLK, DUMMY/CLK}: `@code coreAssembler> connect_interfaces -from_component BUS -from_interface CLK \ -to_component DUMMY -to_interface CLK \endcode` The following command will connect {BUS/DATA1 to DUMMY/DATA}: `@code coreAssembler> connect_interfaces -from_component BUS -from_interface DATA1 \ -to_component DUMMY -to_interface DATA \endcode`
Example 3
For a subsystem with Component BUS interfaces "Provider Instance CLK" of interfaceDefn clock, "Provider Instance DATA1" of interfaceDefn data, and "Provider Instance DATA2" of interfaceDefn data, and Component DUMMY interfaces "Consumer Instance CLK" of interfaceDefn clock and "Consumer

coreTools Command Reference Index

Instance DATA" of interfaceDefn data, provider BUS/CLK, consumer DUMMY/CLK, and consumer DUMMY/DATA are unique interface instances. Provider BUS/DATA1 and BUS/DATA2 are not unique instances. You will get an error if you perform the following command: @code coreAssembler> connect_interfaces -from_component BUS -to_comonent DUMMY \endcode You need to specify the instance name in order to correctly connect the interfaces BUS/DATA1 to DUMMY/DATA and BUS/CLK to DUMMY/CLK: @code coreAssembler> connect_interfaces -from_component BUS \ -from_interface DATA1 \ -to_component DUMMY \ [-to_interface DATA]\fP \endcode Note that in this case "-to_interface" is optional because DUMMY/DATA is unique.

To connect a monitor from the testbench to an interface located with the DUT, specify the path to dut component:

```
coreAssembler> connect_interfaces -from_component APB_Monitor \
-to_component dut/i_apb -to_interface APB_Slave
```

To make a hierarchical connection between two lower level components, use the -hierarchy options:

```
coreAssembler> connect_interfaces -hierarchy -from_component
i_hier1/i_ahb \
-from_interface AHB_Slave -to_component i_hier2/i_apb \
-to_interface AHB_Slave
```

See Also

[disconnect_interface \(2\)](#), [set_unused_interface \(2\)](#),
[translate_interface_connection \(2\)](#)

`connect_or_export_unconnected_pins`

Connect or export unconnected pins in current context.

Syntax

```
string connect_or_export_unconnected_pins [-components <instances>]
[-internal] [-format <format>]
    string <instances>
    string <format>
```

Parameters

<code>-components <instances></code>	Component names to export from.
<code>-internal</code>	Make internal connections only.
<code>-format <format></code>	Port name format.

Description

This command is used to connect and/or export unconnected pins in the current context. By default, all fully unconnected pins are processed by this command. Use the `-components` argument to restrict processing to only those unconnected pins on the specified components.

Unconnected pins are grouped by port name for processing. Like named pins are connected together. If there is no driver, an input port is created to drive the pins. Pins with no match are simply exported. The name matching can be altered by setting the `LogicalName` attribute on ports of the components. When `LogicalName` is set, that name overrides the port name when looking for a name match.

If the `-internal` option is specified, no pins are exported. In this case only like named pins with a driver will be connected.

The default port name format is 'p', meaning that the port name of exported pins will match the name of the port associated with the pin. For different behavior, specify the format using the `-format` option. This string can contain fixed characters along with the format specifiers p, d, and D. The latter two format options are replace with the port directionality (i,o,b for d, and I,O,B for D).

Examples

Connect or export all pins in the current hierarchy:

`connect_or_export_unconnected_pins`

Connect pins between instances U1 and U2 only, with no exporting.

`connect_or_export_unconnected_pins -components "U1 U2" -internal`

In a subsystem with unconnected pins U1/A and U2/a, set LogicalName so that these pins are connected and exported with port A:

```
eval_in_component U2 {
```

```
  set_port_attribute a LogicalName A
```

```
}
```

```
connect_or_export_unconnected_pins
```

See Also

`export_unconnected_pins (2)`

NAME

`continue` - Skip to the next iteration of a loop

SYNOPSIS

`continue`

DESCRIPTION

This command is typically invoked inside the body of a looping command such as **for** or **foreach** or **while**. It returns a 4 (**TCL_CONTINUE**) result code, which causes a continue exception to occur. The exception causes the current script to be aborted out to the innermost containing loop command, which then continues with the next iteration of the loop. Catch exceptions are also handled in a few other situations, such as the **catch** command and the outermost scripts of procedure bodies.

EXAMPLE

Print a line for each of the integers from 0 to 10
except 5:

```
for {set x 0} {$x<10} {incr x} {  
    if {$x == 5} {  
        continue  
    }  
    puts "x is $x"  
}
```

SEE ALSO

`break(n)`, `for(n)`, `foreach(n)`, `return(n)`, `while(n)`

KEYWORDS

`continue`, `iteration`, `loop`

NAME

`oo::copy` - create copies of objects and classes

SYNOPSIS

package require TclOO

oo::copy *sourceObject* ?*targetObject*? ?*targetNamespace*?

DESCRIPTION

The **oo::copy** command creates a copy of an object or class. It takes the name of the object or class to be copied, *sourceObject*, and optionally the name of the object or class to create, *targetObject*, which will be resolved relative to the current namespace if not an absolute qualified name and *targetNamespace* which is the name of the namespace that will hold the internal state of the object (**my** command, etc.); it *must not* refer to an existing namespace. If either *targetObject* or *targetNamespace* is omitted or is given as the empty string, a new name is chosen. Names, unless specified, are chosen with the same algorithm used by the **new** method of **oo::class**. The copied object will be of the same class as the source object, and will have all its per-object methods copied. If it is a class, it will also have all the class methods in the class copied, but it will not have any of its instances copied.

After the *targetObject* has been created and all definitions of its configuration (e.g., methods, filters, mixins) copied, the **<cloned>** method of *targetObject* will be invoked, to allow for customization of the created object such as installing related variable traces. The only argument given will be *sourceObject*. The default implementation of this method (in **oo::object**) just copies the procedures and

coreTools Command Reference Index

variables in the namespace of *sourceObject* to the namespace of *targetObject*. If this method call does not return a result that is successful (i.e., an error or other kind of exception) then the *targetObject* will be deleted and an error returned.

The result of the **oo::copy** command will be the fully-qualified name of the new object or class.

EXAMPLES

This example creates an object, copies it, modifies the source object, and then demonstrates that the copied object is indeed a copy.

```
oo::object create src
oo::objdefine src method msg {} {puts foo}
oo::copy src dst
oo::objdefine src method msg {} {puts bar}
src msg          -> prints "bar"
dst msg          -> prints "foo"
```

SEE ALSO

oo::class(n), oo::define(n), oo::object(n)

KEYWORDS

clone, copy, duplication, object

copy_configuration

Copy configuration parameter values from source to target(s).

Syntax

string **copy_configuration** -source <source instance> [-targets <target instances>] [-filter <filter>]

string <source instance>

string <target instances>

string <filter>

Parameters

-source <source instance>

Source component name.

-targets <target instances>

Target component names.

-filter <filter>

Filter for finding parameters.

Description

The copy_configuration command is used to copy common (like named) configuration parameters from the specified source component to the target components. Only like-named parameters are copied, and only for parameters which are non-visible and enabled in a given target. More specifically, parameters in the target are found using the filter (Visible==0&&Enabled==1&&ReadOnlyParam==0). This behavior can be changed by passing a different filter using the -filter argument.

If no targets are explicitly specified using the -targets option, then all components other than the source are considered as potential targets.

Examples

Copy like named configuration values from U1 to U2 and U3:

```
copy_configuration -source U1 -targets {U2 U3}
```

Copy configuration values from U7 to all other components in this hierarchy:

```
copy_configuration -source U7
```

See Also

Copy parameter values from given interfaces to opposite target interfaces.

Syntax

```
string copy_target_interface_parameters [-verbose] interface
string interface
```

Parameters

-verbose	Print messages about propagated parameters.
interface	Interface to be propagated to

Description

Examples

See Also

coreAssembler

Invokes the coreAssembler tool in either GUI or shell mode.

Syntax

```
string coreAssembler [-f script_file] [-x command_string] [-version]
[-shell] [workspace]
    string script_file
    string command_string
    string workspace
```

Parameters

-f <i>script_file</i>	Execute <i>script_file</i> before displaying the initial prompt. If the last statement in the script is quit, then the tool exists without entering interactive mode.
-x <i>command_string</i>	Execute <i>command_string</i> before displaying the initial prompt. Multiple statements may be entered, each separated by a semicolon. If the last statement is quit, then the tool exists without entering interactive mode.
-version	Displays the version number, build data, site id number, local administrator, and contact information; then exists.
-shell	Invokes the tool in shell mode instead of GUI mode.
<i>workspace</i>	Specifies a workspace to load at tool start-up. The name specified should be a directory path to the root of the workspace.

Description

The coreAssembler tool is used to automatically assemble and implement an SOC subsystem consisting of cores packages with coreBuilder. The cores are packaged with 'interface intent' which enables coreAssembler to automatically connect the cores to each other. See the user manual for more details.

See Also

[coreConsultant \(1\)](#), [coreBuilder \(1\)](#)

coreBuilder

Invokes the coreBuilder tool in either GUI or shell mode.

Syntax

```
string coreBuilder [-f script_file] [-x command_string] [-version]
[-shell] [workspace]
    string script_file
    string command_string
    string workspace
```

Parameters

-f <i>script_file</i>	Execute <i>script_file</i> before displaying the initial prompt. If the last statement in the script is quit, then the tool exists without entering interactive mode.
-x <i>command_string</i>	Execute <i>command_string</i> before displaying the initial prompt. Multiple statements may be entered, each separated by a semicolon. If the last statement is quit, then the tool exists without entering interactive mode.
-version	Displays the version number, build data, site id number, local administrator, and contact information; then exists.
-shell	Invokes the tool in shell mode instead of GUI mode.
<i>workspace</i>	Specifies a workspace to load at tool start-up. The name specified should be a directory path to the root of the workspace.

Description

The coreBuilder tool is used to package reusable IP blocks for use within coreConsultant and coreAssembler. See the user manuals or on-line help for details.

See Also

[coreConsultant \(1\)](#), [coreAssembler \(1\)](#)

coreBuilderBomTemplateVersion

The version of the Bill-of-Materials template syntax used by a template.

Definition

Type: string

Flags:

Default value:

Valid on:

Description

This attribute defines the 'format version' of a BoM template. This allows for multiple formats to be supported if needed. When BoM templates are read, they are scanned for the version and then the appropriate template reader is used, based on the version value. This attribute is only useful in the context of defining a template and the only currently supported value for this attribute is "1.0".

Examples

To define a BoM template which has a 'template version' of 1.0:

```
set BOMTemplate {  
  BOMTemplate  
  {Name {Example template version template}}  
  {coreBuilderBOMTemplateVersion 1.0}  
  {Version 5.0}  
  {Deliverable  
    {Name "Documentation"}}  
  }  
}
```

See Also

coreConsultant

Invokes the coreConsultant tool in either GUI or shell mode.

Syntax

```
string coreConsultant [-f script_file] [-x command_string] [-version]  
[-shell] [workspace]  
    string script_file  
    string command_string  
    string workspace
```

Parameters

-f <i>script_file</i>	Execute <i>script_file</i> before displaying the initial prompt. If the last statement in the script is quit, then the tool exists without entering interactive mode.
-x <i>command_string</i>	Execute <i>command_string</i> before displaying the initial prompt. Multiple statements may be entered, each separated by a semicolon. If the last statement is quit, then the tool exists without entering interactive mode.
-version	Displays the version number, build data, site id number, local administrator, and contact information; then exists.
-shell	Invokes the tool in shell mode instead of GUI mode.
<i>workspace</i>	Specifies a workspace to load at tool start-up. The name specified should be a directory path to the root of the workspace.

Description

The coreConsultant tool is used to implement an individual core packaged with the coreBuilder tool. See the reference manuals or on-line help for details.

See Also

[coreBuilder \(1\)](#), [coreAssembler \(1\)](#)

coretools_home_page

Page to show as the tool home page.

Syntax

```
string coretools_home_page = "/vobs/apbld/doc/dware/coretools.html"
```

Description

The coretools_home_page is a global variable that controls the home page to display in all of the coreTools. Set this variable in your .synopsys_rt.setup file or in the system wide version of this file if you want to customize the home page.

Examples

See Also

NAME

`coroutine`, `yield`, `yieldto` - Create and produce values from coroutines

SYNOPSIS

```
coroutine name command ?arg...?  
yield ?value?  
yieldto command ?arg...?  
name ?value...?
```

DESCRIPTION

The **coroutine** command creates a new coroutine context (with associated command) named *name* and executes that context by calling *command*, passing in the other remaining arguments without further interpretation. Once *command* returns normally or with an exception (e.g., an error) the coroutine context *name* is deleted.

Within the context, values may be generated as results by using the **yield** command; if no *value* is supplied, the empty string is used. When that is called, the context will suspend execution and the **coroutine** command will return the argument to **yield**. The execution of the context can then be resumed by calling the context command, optionally passing in the *single* value to use as the result of the **yield** call that caused the context to be suspended. If the coroutine context never yields and instead returns conventionally, the result of the **coroutine** command will be the result of the evaluation of the context.

The coroutine may also suspend its execution by use of the **yieldto** command, which instead of returning, cedes execution to some command called *command* (resolved in the context of the coroutine) and to which *any number*

coreTools Command Reference Index

of arguments may be passed. Since every coroutine has a context command, **yieldto** can be used to transfer control directly from one coroutine to another (this is only advisable if the two coroutines are expecting this to happen) but *any* command may be the target. If a coroutine is suspended by this mechanism, the coroutine processing can be resumed by calling the context command optionally passing in an arbitrary number of arguments. The return value of the **yieldto** call will be the list of arguments passed to the context command; it is up to the caller to decide what to do with those values.

The recommended way of writing a version of **yield** that allows resumption with multiple arguments is by using **yieldto** and the **return** command, like this:

```
proc yieldm {value} {  
    yieldto return -level 0 $value  
}
```

The coroutine can also be deleted by destroying the command *name*, and the name of the current coroutine can be retrieved by using **info coroutine**. If there are deletion traces on variables in the coroutine's implementation, they will fire at the point when the coroutine is explicitly deleted (or, naturally, if the command returns conventionally).

At the point when *command* is called, the current namespace will be the global namespace and there will be no stack frames above it (in the sense of **upvar** and **uplevel**). However, which command to call will be determined in the namespace that the **coroutine** command was called from.

EXAMPLES

This example shows a coroutine that will produce an infinite sequence of even values, and a loop that consumes the first ten of them.

```
proc allNumbers {} {  
    yield  
    set i 0  
    while 1 {  
        yield $i  
        incr i 2  
    }  
}  
coroutine nextNumber allNumbers  
for {set i 0} {$i < 10} {incr i} {  
    puts "received [nextNumber]"  
}  
rename nextNumber {}
```

In this example, the coroutine acts to add up the arguments passed to it.

coreTools Command Reference Index

```
coroutine accumulator apply {{} {
    set x 0
    while 1 {
        incr x [yield $x]
    }
}}
for {set i 0} {$i < 10} {incr i} {
    puts "$i -> [accumulator $i]"
}
```

This example demonstrates the use of coroutines to implement the classic Sieve of Eratosthenes algorithm for finding prime numbers. Note the creation of coroutines inside a coroutine.

```
proc filterByFactor {source n} {
    yield [info coroutine]
    while 1 {
        set x [$source]
        if {$x % $n} {
            yield $x
        }
    }
}
coroutine allNumbers apply {{} {while 1 {yield [incr x]}}}
coroutine eratosthenes apply {c {
    yield
    while 1 {
        set n [$c]
        yield $n
        set c [coroutine prime$n filterByFactor $c $n]
    }
}} allNumbers
for {set i 1} {$i <= 20} {incr i} {
    puts "prime#$i = [eratosthenes]"
}
```

This example shows how a value can be passed around a group of three coroutines that yield to each other:

```
proc juggler {name target {value ""}} {
    if {$value eq ""} {
        set value [yield [info coroutine]]
    }
    while {$value ne ""} {
        puts "$name : $value"
        set value [string range $value 0 end-1]
        lassign [yieldto $target $value] value
    }
}
coroutine j1 juggler Larry [
    coroutine j2 juggler Curly [
        coroutine j3 juggler Moe j1]] "Nyuck!Nyuck!Nyuck!"
```

DETAILED SEMANTICS

This example demonstrates that coroutines start from the global namespace, and that *command* resolution happens before the coroutine stack is created.

```
proc report {where level} {
    # Where was the caller called from?
```

coreTools Command Reference Index

```
set ns [uplevel 2 {namespace current}]
yield "made $where $level context=$ns name=[info coroutine]"
}
proc example {} {
    report outer [info level]
}
namespace eval demo {
    proc example {} {
        report inner [info level]
    }
    proc makeExample {} {
        puts "making from [info level]"
        puts [coroutine coroEg example]
    }
    makeExample
}
```

Which produces the output below. In particular, we can see that stack manipulation has occurred (comparing the levels from the first and second line) and that the parent level in the coroutine is the global namespace. We can also see that coroutine names are local to the current namespace if not qualified, and that coroutines may yield at depth (e.g., in called procedures).

```
making from 2
made inner 1 context=: name=:demo::coroEg
```

SEE ALSO

`apply(n)`, `info(n)`, `proc(n)`, `return(n)`

KEYWORDS

coroutine, generator

create_address_bank

Create an address bank in a memory map or bank.

Syntax

```
string create_address_bank [-map <map>] [-bank <bank>] [-base_address
<addr>] -alignment <alignment> [name]
    string <map>
    string <bank>
    string <addr>
    string <alignment>
    string name
```

Parameters

-map <map>	The map to put this bank into.
-bank <bank>	The bank to put this bank into.
-base_address <addr>	The base address of the bank.
-alignment <alignment>	The alignment of this bank (Values: parallel, serial)
name	Name of the address bank to create.

Description

This command creates an address bank. You must specify either -map or -bank. If -map is specified, then the the -base_address option must be provided, otherwise the BaseAddress is automatically computed from the rules of the address bank.

Examples

To create a parallel bank:

```
coreBuilder> create_address_bank -map map1 -base_address 0x1000
-alignment parallel
```

See Also

[BankAlignment \(3\)](#), [BaseAddress \(3\)](#), [remove_address_bank \(2\)](#)

create_address_block

Create an address block in a memory map or bank.

Syntax

string **create_address_block** [-map <map>] [-bank <bank>] [-base_address <addr>] -range <range> -width <bits> [name]

string <map>
string <bank>
string <addr>
string <range>
string <bits>
string name

Parameters

-map <map>	The map to put this block into.
-bank <bank>	The bank to put this block into.
-base_address <addr>	The base address of the block.
-range <range>	The range of this block.
-width <bits>	The width of an address in this block.
name	Name of the address block to create.

Description

This command creates an address block in the specified memory map or address bank. If -map is specified, then -base_address must also be specified, otherwise the BaseAddress is computed via the rules of the address bank.

Examples

```
coreBuilder> create_address_block -map map1 -base_address 0x0 -range 0x1000 -width 8
```

See Also

[BaseAddress \(3\)](#), [BankAlignment \(3\)](#), [MemoryRange \(3\)](#), [MemoryWidth \(3\)](#), [remove_address_block \(2\)](#)

create_address_space

Create a new address space or update an existing one.

Syntax

```
string create_address_space -name <name> -range <range> -width <width>
[-addressUnitBits <addressUnitBits>]
    string <name>
    string <range>
    int <width>
    string <addressUnitBits>
```

Parameters

-name <name>	The unique name of the address space.
-range <range>	The address range of the address space.
-width <width>	The bit width of a row in the address space.
-addressUnitBits <addressUnitBits>	The number of data bits in an addressable unit of the address space.

Description

This command is used to create or modify an addressSpace defined on a design. If the address space does not exist it will be created. If the address space already exists the values will be updated.

Examples

```
create_address_space -name lPort -range 1G -width 8
```

See Also

[remove_address_space \(2\)](#), [BaseAddress \(3\)](#), [AddressSpaceRef \(3\)](#)

create_autoload_filegroup
Create an autoloading filegroup

Syntax

```
string create_autoload_filegroup -patterns -install <install directory>
group
    string
    string <install directory>
    string group
```

Parameters

	A list of (glob-style) patterns for how to create the group
-patterns	Every entry in the list is expanded with the glob command. Relative paths start with the root of the current workspace. If the workspace is located at "/user/userx/coreA/Builder_ws", then the pattern "tb/\<filename>\.v" refers to "/user/userx/coreA/tb/\<filename>\.v".
-install	
<install directory>	Installation directory
group	BOM File group

Description

The create_autoload_filegroup command creates an autoloading filegroup and adds it to the Bill of Materials (BoM). When the BuildcoreKit activity is completed, the list of patterns is used to determine what files to put into the coreKit.

The create_autoload_filegroup command may also be used to add files and filegroups to a plugin by calling the command in the Obj.tcl file of the plugin. The pathnames to the files should be absolute pathnames, or a paths relative to the plugin source directory or the current working directory. Note that files in the plugin source directory with a .tcl extension are treated as source code for the plugin, so it may be more appropriate to place any .tcl files to be added to a plugin filegroup in a subdirectory.

Examples

To add all documentation files into the coreKit:

```
coreBuilder> create_autoload_filegroup -patterns doc/* -install doc  
Documentation
```

See Also

[add_files_to_filegroup \(2\)](#) [create_plugin_kb \(2\)](#) [load_plugin \(2\)](#)

NAME

create_command_group
Creates a new command group.

SYNTAX

```
string create_command_group [-info info_text]  
group_name
```

ARGUMENTS

-info info_text
Help string for the group

group_name Specifies the name of the new group.

DESCRIPTION

The **create_command_group** command is used to create a new command group, which you can use to separate related user-defined procedures into functional units for the online help facility. When a procedure is created, it is placed in the "Procedures" command group. With the **define_proc_attributes** command, you can move the procedure into the group you created.

The *group_name* can contain any characters, including spaces, as long as it is appropriately quoted. If *group_name* already exists, **create_command_group** quietly ignores the command. The result of **create_command_group** is always an empty string.

EXAMPLES

The following example demonstrates the use of the **create_command_group** command:

```
prompt> create_command_group {My Procedures} -info "Useful utilities"

prompt> proc plus {a b} { return [expr $a + $b] }

prompt> define_proc_attributes plus -command_group "My Procedures"

prompt> help
My Procedures:
  plus
...

```

SEE ALSO

```
define_proc_attributes(2)
help(2)
proc(2)
```


create_component_view

Create an IP-XACT component view.

Syntax

```
string create_component_view -name <view name> -envIdentifier
<identifier> [-hierarchyRef <VLNV>] [-displayName <name>] [-description
<description>] [-language <language>] [-modelName <model name>]
[-fileSetRef <filesets>] [-whiteboxElementRefs <XML>]
    string <view name>
    string <identifier>
    string <VLNV>
    string <name>
    string <description>
    string <language>
    string <model name>
    string <filesets>
    string <XML>
```

Parameters

-name <view name>	View name
-envIdentifier <identifier>	List of env identifier values
-hierarchyRef <VLNV>	VLNV reference to a design or configuration file.
-displayName <name>	Display name
-description <description>	View description
-language <language>	HDL language
-modelName <model name>	
-fileSetRef <filesets>	List of fileset references
-whiteboxElementRefs <XML>	Complete, valid XML for spirit:whiteboxElementRefs

Description

This command creates a new IPXACT view for the component.

Examples

```
create_component_view -name sim_view -envIdentifiers :*Simulation:
```

See Also

[remove_component_view \(2\)](#)

create_configuration_parameter

Create a parameter for a configurable filegroup

Syntax

```
string create_configuration_parameter -type <type> [-label <label>]
[-description <description string>] [-url <URL>] [-default <default
value>] [-group <file group name>] name
    string <type>
    string <label>
    string <description string>
    string <URL>
    string <default value>
    string <file group name>
    string name
```

Parameters

-type <type>	The type of the new parameter. (Values: integer, float, string, boolean, bitfield)
-label <label>	The label for the parameter in the configuration dialog
-description <description string>	The short help text for the parameter.
-url <URL>	Specifies the URL of the detailed help text for the parameter.
-default <default value>	Specifies the default value for the parameter.
-group <file group name>	The filegroup to create the parameter for, (default: from analyze_filegroup)
name	The name of the new parameter.

Description

The create_configuration_parameter command creates a new parameter that controls the configuration of a filegroup. This command is generally put into intent files, and then sourced via the analyze_filegroup(2) command.

The command line options set values for the following attributes on the parameter: Label (-label), Description (-description), HelpUrl (-url), DefaultValue (-default). There are several additional parameter attributes you can set to specify legal values for the parameter and control how the coreTool builds the activity dialog where the user specifies a value for the parameter. To set these additional parameter

attributes, use the `set_parameter_attribute` command. Refer to the *coreBuilder User Guide* for examples.

The `create_configuration_parameter` command returns a collection that contains the new configuration parameter. You can use the collection as input to the `set_parameter_attribute` commands.

Examples

See `create_custom_activity_parameter` for examples

See Also

[analyze_filegroup \(2\) ReplaceFormatParam \(2\)](#)
[create_custom_activity_parameter \(2\) Configurable \(3\)](#)
[ConfigIntentSearchPath \(3\)](#)

create_connection

Create connections in between component pins and subsystem ports.

Syntax

```
string create_connection [-constant <value>] [-nosource] [-hierarchy]
[-signal <signal name>] [-interface <interface name>] [-instance {
{param1 value1} {param2 value2} ... }] [-comment <text>] [-quiet]
[-multisource] [-broadcast] [-supply0 <port/pin>] [-supply1 <port/pin>]
[-port <port_name>] conns
    string <value>
    string <signal name>
    string <interface name>
    string { {param1 value1} {param2 value2} ... }
    string <text>
    string <port/pin>
    string <port_name>
    string conns
```

Parameters

-constant <value>	Connect to a constant
-nosource	Allow load to load connection
-hierarchy	Connect pins through hierarchy. This is no longer required for hierarchical connections.
-signal <signal name>	Signal name for the given connection
-interface <interface name>	Name of SV interface cell to connect with
-instance { {param1 value1} {param2 value2} ... }	Name/Value pairs for parameters on new SV interface cell
-comment <text>	Comment associated with the given connection
-quiet	Don't print status messages
-multisource	Create a connection with multiple sources.
-broadcast	connects a single bit driver(source) to multi-bit receiver(load) Must be used with -constant. Connects all zero bits of the value specified with -constant to specified SUPPLY/GND port/pin. In this case remaining one bits are tied to logic one or -supply1 if specified
-supply0 <port/pin>	

<code>-supply1 <port/pin></code>	Must be used with <code>-constant</code> . Connects all one bits of the value specified with <code>-constant</code> to specified SUPPLY/GND port/pin. In this case remaining zero bits are tied to logic zero or <code>-supply0</code> if specified
<code>-port <port_name></code>	Specifies the desired name for ports created while making a hierarchical connection.
<code>conns</code>	ports and pins to connect

Description

This command is used to make manual connections in a coreAssembler subsystem. It is typically used to deal with connections that can not be connected automatically. Connections can be made between pins and subsystem ports, with a few basic restrictions. Connections will not be made if the resulting signal will have more than one driver. Also, only drivers can be connected to the constant 'open', and only loads can be connected to constant values 'zero', 'one', and 'dc'.

This command also supports connecting a source to one or more loads via an inverter. This functionality is accessed by using '!' as the first character of the port or pin that should be inverted.

The `-signal` option can be used to control the name of the signals created in the generated RTL. Note that the names are only used when a signal is required to be created. The `-comment` option can be used to associate a comment with a particular connection. In default flow the comment appears in generated RTL above the portMaps. When the activity parameter `ExportConnectionCommentsInline` is set in `GenerateSubsystemRTL` activity, the connection comments appear inline after the portMaps.

The `-hierarchy` option can be used to make connections across different levels of hierarchy. If connecting into a lower level of hierarchy, the lower level pin(s) will be exported upwards into the current level of hierarchy and then a connection will be made.

The `-broadcast` option can be used to connect a single bit driver to multi-bit input pins. Partial bits of the input pins can also be connected

Examples

To connect two component pins:

```
coreAssembler> create_connection {U1/data U2/sum}
```

To connect a component pin and a top-level port:

```
coreAssembler> create_connection {appData U3/appData}
```

coreTools Command Reference Index

To connect two component pins of width 1, to a two bit port:

```
coreAssembler> export_pin -port inA[0] -dimensions 0:1 U4/A
coreAssembler> create_connection {inA[1] U5/A}
```

To connect unused inputs and outputs to constant values:

```
coreAssembler> create_connection -constant open U7/out
coreAssembler> create_connection -constant zero U8/in
```

To connect SystemVerilog interface pins and create interface instance:

```
coreAssembler> create_connection {mod1/intfp mod2/intfp} -interface
auto_intf
```

To connect SystemVerilog interface pins to instance of SV interface
coreKit:

```
coreAssembler> instantiate_component -name i_intf intf
coreAssembler> create_connection {mod1/intfp mod2/intfp} -interface i_intf
```

Broadcast connection : Can be used to connect single bit driver to multi
bit source. Partial Bits can also be connected For example

```
output data_out
input [7:0] data_in
input [7:0] data_in1
input [7:0] mem [7:0]
```

```
coreAssembler> create_connection -broadcast {top/data_out top/data_in
top/data_in1[3:0] top/mem[3:0][2:0] }
```

See Also

[export_pin \(2\)](#), [remove_connection \(2\)](#), [translate_interface_connection \(2\)](#)

create_custom_activity
Create a custom activity

Syntax

```
string create_custom_activity -name <name> [-type <type>] [-label
<label>] [-pre <command string>] [-post <command string>] [-cancel
<command string>] [-description <description string>] [-url <URL>]
[-group <name>]
    string <name>
    string <type>
    string <label>
    string <command string>
    string <description string>
    string <URL>
```

Parameters

-name <name>	Specifies the name of the new activity. By convention, the name of an activity is the activity label with the spaces removed (see -label below). For example, the activity name for Verify Budgets is VerifyBudgets and the label is Verify Budgets. Use report_activities to show a list of the existing activity names.
-type <type>	Specifies the type of the activity, either Global (the default) or Hier. (Values: Global, Design, Hier)
-label <label>	Specifies the text label that appears in the Activity List for the new activity.
-pre <command string>	Specifies the Tcl command to execute before posting the dialog for the new activity.
-post <command string>	Specifies the Tcl command to execute when the user clicks the OK button for the new activity.
-cancel <command string>	Specifies the Tcl command to execute when the user clicks the Cancel button for the new activity.
-description <description string>	Specifies the short help text to display.
-url <URL>	Specifies the URL of the help text or man page for the new activity.
-group <name>	Specifies the activity group into which you want to insert the new activity.

Description

The `create_custom_activity` command creates a new custom coreTool design flow activity. You can create a custom coreBuilder activity to insert into the coreBuilder design flow or you can create a custom coreConsultant or coreAssembler activity to insert into the coreConsultant or coreAssembler design flow. Core developers typically create custom coreConsultant activities and package them into the coreKit to support core-specific design tasks.

In addition to creating the custom activity, you must create custom Tcl command procedures that implement the functionality of the activity. You can create separate command procedures to be executed before the coreTool posts the dialog for the custom activity, when the user clicks OK in the dialog for the custom activity, and when the user clicks Cancel in the dialog for the custom activity.

`create_custom_activity` is the first of a sequence of commands required to create a custom activity and (optionally) parameters for the activity. The other commands associated with creating a custom activity are:

- `complete_custom_activity_definition` - Completes the definition of your new custom activity, specifies where the activity appears in the coreTool design flow, and specifies prerequisite and dependent activities.
- `create_activity_parameter` - Creates parameters for the custom activity. This command is optional because not all activities require parameters.
- `set_parameter_attribute` - Sets attributes on the custom activity parameters.
- `get_activity_parameter` - Used in Tcl command procedures to retrieve the user-specified value for an activity parameters.

The mechanism to implement a custom activity is a coreTool plug-in. To create a plug-in:

1. Place the `create_custom_activity`, `complete_custom_activity_definition`, `create_activity_parameter`, and `set_parameter_attribute` commands for the new activity in a file named `<activity>_Obj.tcl`.
2. Place Tcl code for the Tcl command procedures that implement the pre-prompt, post-prompt, and cancel functionality for the activity in a file named `<activity>.tcl`. Use `get_activity_parameter` in your custom Tcl commands procedures to get the user-specified values for activity parameters.
3. Build the plug-in either (1) using the `create_plugin_kb` command, or (2) executing the coreBuilder "Load Consultant Plugins" activity.

When you load the plug-in, either explicitly or automatically, the custom activity appears in the coreTool activity list and can be executed in

either GUI mode or in command line mode. For GUI mode, the coreTool automatically creates a user dialog for the activity according to the specifications defined by the custom activity creation commands options.

The `create_custom_activity` command returns a collection that contains the new activity. You can use the collection as input to the other custom activity creation commands, as shown in **Examples** below.

Examples

The following set of commands in an `<activity>_Obj.tcl` file creates a new custom activity with the following characteristics:

- Activity name is ViewSimulationResults.
- The activity appears in the Verification group with the label View Simulation Results.
- The activity has one parameter (TestNumber).
- coreConsultant executes the Tcl procedure `Check_Sim_Complete` before posting the activity dialog.
- coreConsultant executes the Tcl procedure `Display_Log` when the user clicks OK in the activity dialog.
- The activity sequence number is 570.
- RunSim is a prerequisite activity.

```
# File: ViewSimulationResults_Obj.tcl
# Create ViewSimulationResults activity.
set view_sim_log [create_custom_activity \e
-name ViewSimulationResults \e
-type Global \e
-label "View Simulation Results" \e
-pre "Check_Sim_Complete" \e
-post "Display_Log" \e
-group Verification \e
-description "Display simulation log in web browser."]
set P [create_custom_activity_parameter \e
-activity $view_sim_log \e
-name TestNumber \e
-type string \e
-label "Enter Test Number" \e
-description "Which test results do you want to view?"]
set_parameter_attribute $P ValueRequired true
complete_custom_activity_definition -activity $view_sim_log \e
-sequence 570 \e
-prereqs RunSim
```

See Also

[complete_custom_activity_definition\(2\), set_parameter_attribute\(2\),](#)

[get_activity_parameter\(2\), set_activity_parameter\(2\)](#)

create_custom_activity_parameter

Create a parameter for a custom activity

Syntax

```
string create_custom_activity_parameter -activity <activity> -name <name>
-type <type> [-label <label>] [-description <description string>] [-url
<URL>] [-default <default value>] [-design]
    string <activity>
    string <name>
    string <type>
    string <label>
    string <description string>
    string <URL>
    string <default value>
```

Parameters

-activity <activity>	Specifies the custom activity for which you want to create a parameter. <activity> is the same activity name that you specified with the -name option to the create_custom_activity command that created the custom activity.
-name <name>	Specifies the name of the new parameter.
-type <type>	Specifies the parameter type. (Values: integer, float, string, boolean, bitfield)
-label <label>	Specifies the text that will appear for the parameter in the parameter dialog for the custom activity.
-description <description string>	Specifies the short help text for the parameter.
-url <URL>	Specifies the URL of the detailed help text for the parameter.
-default <default value>	Specifies the default value for the parameter.
-design	Parameter is design dependent. Checking must be deferred.

Description

The create_custom_activity_parameter command creates a new activity parameter for a custom activity that you created with the

create_custom_activity command.

The command line options set values for the following attributes on the parameter: Name (-name), Label (-label), Description (-description), HelpUrl (-url), DefaultValue (-default). There are several additional parameter attributes you can set to specify legal values for the parameter and control how the coreTool builds the activity dialog where the user specifies a value for the parameter. To set these additional parameter attributes, use the set_parameter_attribute command. Refer to the *coreBuilder User Guide* for examples.

The create_custom_activity_parameter command returns a collection that contains the new activity parameter. You can use the collection as input to the set_parameter_attribute commands, as shown in **Examples** below.

Examples

To create a parameter named TestNumber for the custom activity named ViewSimulationResults, with the following characteristics:

- Parameter name is TestNumber; label that appears in activity dialog is "Enter Test Number".
- Parameter type is integer.
- Short (What's This?) help is "Which test results do you want to view?".
- coreConsultant invokes user's web browser to display `"/doc/ViewSimulationResults.html"` when the user requests detailed help for the parameter.

```
set P [create_custom_activity_parameter \
-activity ViewSimulationResults \
-name TestNumber \
-type integer \
-label "Enter Test Number" \
-url "/doc/ViewSimulationResults.html" \
-description "Which test results do you want to view?"]
To set additional attributes on the TestNumber parameter:
```

```
set_parameter_attribute $P ValueRequired true
set_parameter_attribute $P MinValue 1
set_parameter_attribute $P MaxValue 64
```

See Also

[create_custom_activity \(2\), complete_custom_activity_definition \(2\),
set_parameter_attribute \(2\)](#)

create_custom_tech_cell

Specify binding requirements for mapping to a technology specific cell.

Syntax

```
string create_custom_tech_cell -name <name> [-file_name <file_name>]
[-language <language>] [-port_definitions <ports>] [-undef]
    string <name>
    string <file_name>
    string <language>
    string <ports>
```

Parameters

-name <name>	Unique name of the library cell to define
-file_name <file_name>	File name of the RTL code
-language <language>	Language of the RTL code
-port_definitions <ports>	List of ports, name and port direction pairs {A input B output}
-undef	Remove/Delete the technology cell definition

Description

For RTL macros which should be mapped to custom tech cells that are not present in the technology library, this command creates a custom user defined cell definition to use with bind_tech_cell_mapping. Doing so will bind the RTL macro to the tech cell created with this command.

- For creating definition, -name, -design, and -file_name are required.
- For undefining an existing definition, only required arguments are -undef and -name.

Examples

```
# In coreConsultant or coreAssembler
# create source verilog if it doesn't exist
coreConsultant> set verilogModel {
module bot(I1, I2, I3, O1, O2, O3);
input I1, I2, I3;
output O1, O2, O3;
endmodule
}
coreConsultant> redirect "customCell3x3.v" {set verilogModel}
```

coreTools Command Reference Index

```
coreConsultant> create_custom_tech_cell -name CustomCell3x3 -design $des
-file_name customCell3x3.v -language verilog -port_definitions {I1 input
I2 input I3 input O1 output O2 output O3 output}
coreConsultant> set verilogModel {
module bot(I1, I2, O1);
input I1, I2;
output O1;
endmodule
}
coreConsultant> redirect "customCell2x1.v" {set verilogModel}
coreConsultant> create_custom_tech_cell -name CustomCell2x1 -design $des
-file_name customCell2x1.v -language verilog -port_definitions {I1 input
I2 input O1 output}
# Add technology cell binding to map to cell defined above
coreConsultant> bind_tech_cell_mapping -macro BCM05_map \
-tech_cell CustomCell2x1 \
-port_map {{q1 I1} {q2 I2} {q3 O1}}
# Undefine an existing custom tech cell definition
coreConsultant> create_custom_tech_cell -name CustomCell2x1 -undef
```

See Also

[bind_tech_cell_mapping\(2\)](#) [define_tech_cell_mapping\(2\)](#)

create_generated_clock

Create a generated clock for use within the specified design

Syntax

```
string create_generated_clock [-name <name>] [-add] -source <port or pin>
[-master_clock <clock>] [-divide_by <factor>] [-multiply_by <factor>]
[-duty_cycle <percent>] [-invert] [-preinvert] [-edges ] [-edge_shift ]
[-design <design>] [-combinational] [-comment <text>] clocks_list
    string <name>
    string <port or pin>
    string <clock>
    int <factor>
    double <percent>
    string
    string <design>
    string <text>
    string clocks_list
```

Parameters

-name <name>	The name for the generated clock Specifies the name of the generated clock. If you do not use this option, the clock receives the same name as the first clock source specified in the -source option. If you specify the -add option, you must use the -name option and the clocks with the same source must have different names. Specifies whether to add this clock to the existing clock or to overwrite.
-add	Use this option to capture the case where multiple generated clocks must be specified on the same source, because multiple clocks fan into the master pin. Ideally, one generated clock must be specified for each clock that fans into the master pin. If you specify this option, you must also use the -name option.
-source <port or pin>	Port or pin of the master clock. Specifies the master clock pin, which is either a master clock source pin or a pin in the fanout of the master clock and driving the generated clock definition pin. The clock waveform at the master pin is used for deriving the generated clock waveform.
-master_clock <clock>	Master clock for the generated clock. Used to specify the specific master clock when multiple clocks fan into the master pin.

coreTools Command Reference Index

-divide_by <factor>	frequency division factor.
-multiply_by <factor>	frequency multiplication factor 2
-duty_cycle <percent>	percent of period that clock is high This option is valid only with the -multiply_by option.
-invert	inverts the generated clock
-preinvert	Creates a generated clock based on the inverted clock signal only when the source clock on the master pin has a non-unate sense
-edges	A list of master clock edges from which to derive the generated clock.
-edge_shift	The amount of shift the specified edges undergo to yield the final generated clock waveform
-design <design>	Specifies the design in which to create the generated clock. If the -design switch is not present the generated clock will be created in the current design.
-combinational	Specifies that the source latency paths for this type of generated clock only includes the logic where the master clock propagates along combinational paths.
-comment <text>	Associate a string description with the command for tracking purposes.
clocks_list	List of ports or pins to define as generate clocks.

Description

This command creates a generated clock object in the current design. The command defines a list of objects as generated clock sources in the current design. You can specify a pin or a port as a generated clock object. The command also specifies the clock source, master clock, from which it is generated. The -master_clock argument must be either a clock name or a clock collection. The -source argument must be the pin or port on which the master clock is defined. The -source argument can be either a name or a collection. The master clock must be defined prior to using it in the create_generated_clock command.

The generated clock can be created as a frequency divided clock (-divide_by option), frequency multiplied clock (-multiply_by), or edge derived clock (-edges). In addition, the frequency divided/multiplied clock can be inverted with the -invert option. The shifting of edges of the edge-derived clock is specified with the -edge_shift option. The -edge_shift option is used for intentional edge shifts and not for clock latency.

Using `create_generated_clock` on an existing `generated_clock` produces an error.

For additional details and examples, see the `create_generated_clock` man page within Design Compiler for Fusion Compiler. This command is meant to mimic the behavior provided in those tools.

Examples

In the following examples, different commands are used to find port/pin/clock objects. If the `create_generated_clock` command is defined within an SDC file, then the standard SDC `get_<object type>` commands should be used to find the objects. If instead the command is defined within a coreTools TCL file, then the coreTools `find_item` command should be used to find the objects.

The following example creates a frequency divide_by 2 generated clock. The master clock source is the port named `clkPort`. The generated clock will be created on the port `genClkPort`

```
coreBuilder> create_generated_clock \
-name genClk \
-divide_by 2 \
-source clkPort \
genClkPort
```

The following example creates a frequency divide_by 3 generated_clock. If the master clock period is 30, and master waveform is {24 36}, the generated clock period will be 90 with waveform {72 108}. The master clock is the clock named `clk`. The generated clock is created on the pin `u_clk_div/div3Clk`. The `-name` option was not used, so the generated clock will be named `u_clk_div/div3Clk`. The clock '`clk`' is the name of the master clock used to derive the generated clock.

```
coreBuilder> create_generated_clock \
-divide_by 3 \
-source [find_item -type port clk] \
[find_item -type pin u_clk_div/div3Clk]
```

The following example creates a frequency multiply_by 2 generated_clock with a duty cycle of 60% on the pin `u_clkgen/clk2x`. The name of the generated clock will be `clk2x`. The `-source` argument is used, so the master clock used to derive `clk2x` is the clock that is defined on the pin '`clkPort`'.

```
coreBuilder> create_generated_clock \
-multiply_by 2 \
-duty_cycle 60 \
-source [find_item -type port clkPort] \
-name clk2x \
u_clkgen/clk2x
```

coreTools Command Reference Index

The following example creates a frequency multiply_by 3 generated_clock with a duty cycle equal to the master clock duty cycle. If the master clock period is 30, and master waveform is {24 36}, the generated clock period will be 10 with waveform {8 12}.

```
coreBuilder> create_generated_clock \  
-multiply_by 3 \  
-source [get_ports clk] \  
[get_pins div3/Q]
```

The following example creates a generated clock whose edges are edges 1, 3, and 5 of the master clock source.

```
coreBuilder> create_generated_clock \  
-edges {1 3 5} \  
-source clkPort \  
-name genClk \  
genClkPort
```

The following example shows the generated clock in the previous example with each derived edge shifted by 1 time unit.

```
coreBuilder> create_generated_clock \  
-edges {1 3 5} \  
-edge_shift {1 1 1} \  
-source clkPort \  
-name genClk \  
genClkPort
```

The following example shows the generated clock in the previous example. The first edge is shifted by 1 time unit. The second edge is shifted by 100 picoseconds, and the third edge is not shifted.

```
coreBuilder> create_generated_clock \  
-edges {1 3 5} \  
-edge_shift {1 100ps} \  
-source clkPort \  
-name genClk \  
genClkPort
```

The following example creates an inverted clock.

```
coreBuilder> create_generated_clock \  
-divide_by 1 \  
-invert \  
-source [get_ports posClk] \  
-name negClk \  
negClkPort
```

See Also

[get_clocks \(2\)](#) [set_clock_attribute \(2\)](#) [remove_generated_clock \(2\)](#)

`create_hierarchical_component`

Create a new hierarchial component in the subsystem

Syntax

string **create_hierarchical_component** [-fromComponent] [-silent] *name*
[*design*]

string *name*

string *design*

Parameters

<code>-fromComponent</code>	The component name to be ungrouped
<code>-silent</code>	instantiate without incompleting any activities
<code>name</code>	New component cell name
<code>design</code>	Name of the design

Description

This command is used to create a new hierarchical component with the given name, within the current hierarchy in the subsystem. The current context automatically switches to the new component so any subsequent component creation commands will create components in the new hierarchical context.

Examples

Create an instance of DW_ahb inside hierarchical component 'bus':

```
coreAssembler> create_hierarchical_component bus
coreAssembler> instantiate_component -name i_ahb DW_ahb
```

See Also

[instantiate_component \(2\)](#), [cA_supports_hierarchy \(3\)](#)

create_interface

Create named interface definition

Syntax

```
string create_interface [-no_provider] [-version <version>] [-description
<text>] [-url <URL>] [-min <#consumers>] [-max <#consumers>] [-require
<interfaces>] [-symmetric] name
    string <version>
    string <text>
    string <URL>
    string <#consumers>
    string <interfaces>
    string name
```

Parameters

-no_provider	The interface is provider-less Specifies the interface version for compliance checks
-version <version>	The version string must be of the form <digits>. When interface instances are connected together in coreAssembler, checks are done to ensure that the major version number (the first <digits>) is equivalent for all interfaces of the same interface definition.
-description <text>	Specifies the short help text to display
-url <URL>	Specifies the URL of the help text or man page for the new interface definition
-min < consumers>	Limits the minimum number of consumers It is not possible to successfully complete the AddSubsystemComponents activity if this limit is not met for every interface.
-max < consumers>	Limits the maximum number of consumers It is not possible to successfully complete the AddSubsystemComponents activity if this limit is not met for every interface.
-require <interfaces>	List of interface definitions required by the new definition For example, this can be used to ensure that a clock gating interface is present for a given bus type, when the master interface is going to be used.
-symmetric	

Indicate the interface is symmetric bus which allow direct mast-slave connection

This option is used to create a symmetric bus definition. In symmetric bus, all interface ports direction are defined as "slave" type. If an 'master' type of symmetric bus is instantiated, then interface port direction will be inverted.

name

Name of the new interface definition

Description

This is the command which actually defines a new interface definition. This command can only appear in files which define interface definitions. These files are loaded into coreBuilder via the LoadInterfaceDefns activity. After creating an interface, its ports and parameters must be defined using the `create_interface_port` and `create_interface_parameter` commands. Further information can be specified using the corresponding attribute setting commands for the interface definition and its ports and parameters.

Examples

For a complete example of an interface definition, including ports and parameters, refer to the interface definition files which ship with the coreBuilder tools. These files can be found in the tool installation area under the `auxx/dware/interface_defns` directory.

See Also

`create_interface_port (2)`, `create_interface_parameter (2)`,
`set_interface_attribute (2)`

create_interface_instance

Create an instance of an interface to associate it on the core

Syntax

```
string create_interface_instance [-interface <interfaceName>] [-version
<version>] [-abstraction <abstraction VLNV>] [-associationformat
<portNamePattern>] [-type <type>] [-auto <ports>] [-used
<parameterExpression>] [-max <#consumers>] [-busType <type>] [-definition
<definition_name>] [-attach] name
    string <interfaceName>
    string <version>
    string <abstraction VLNV>
    string <portNamePattern>
    string <type>
    string <ports>
    string <parameterExpression>
    string <#consumers>
    string <definition_name>
    string name
```

Parameters

-interface <interfaceName>	The definition for the new interface instance
-version <version>	The version of the interface definition to be used
-abstraction <abstraction VLNV>	The abstraction for the new interface instance
-associationformat <portNamePattern>	Pattern match string for linking interface ports to design ports This option defaults to '%s' which implies that design ports and interface ports will only match when their names match (ignoring case). In addition to the required %s, the pattern string can contain other fixed characters, the format specifiers %d and %D, and a glob style matching character (*). The %d and %D format specifiers are replaced with i/o/b or I/O/B, respectively, based on the required port direction. The glob matching character matches any string of characters. For example, the patterns "my_%s_*" and "my_%s_%d" could be used to match the interface port hresp

-type <type>

to the design port my_hresp_o. However, the latter is the preferred format string since my%s_* would match other things such as my_hresp_i, my_hresp_garbage, etc.

The type of the interface instance

(Values: provider, consumer, internal-consumer, interface-monitor, consumer-monitor, provider-consumer)

This option is required when -interface is specified. This option is used to indicate the role that this instance of the interface definition will play. This is critical as it defines how parameters are specified and how connections will be made. The provider and consumer types are the most common. The provider type indicates that the design in which the interface is being instantiated is going to act as a provider for the interface. This means that there will be output ports in the design associated with each 'fromProvider' port in the interface. Also, there will be parameters in the design associated with each provider parameter in the interface. This is not a 100% requirement, as optional ports and parameters do not need to have values and during the interface association process (see the user manual) it is possible to indicate that some interface ports and parameters are unused. The interface type consumer indicates that the design in which the interface is being instantiated is going to act as a consumer of the interface. The internal-consumer type is provided to model situations where the provider design also acts as a consumer. This can occur in bus IP when the arbiter sits inside the bus IP. In this case, there may be parameters linked from the interface instance to the design, but there are no ports, as they are assumed to be internal to the IP. If that is not the case, then the standard consumer interface type should be utilized.

The interface-monitor and consumer-monitor interface types are

coreTools Command Reference Index

	used for verification IP only. These types are used in designs which monitor the entire interface, or a specific consumer, respectively. All ports associated with these types of interfaces must be inputs as the monitors can not modify the subsystem in any way. Create the provider automatically from a list of ports
<code>-auto <ports></code>	This option automatically creates in interface definition that will match the set of ports provided. Specify expression defining when instance is used This option is used to indicate an interface which is conditionally present on the component. The interface is present when the specified parameter expression evaluates to true. Any parameters referenced in the expression may NOT be linked to interface parameters for this interface or any other interface. Parameters used in this expression are referred to as 'instantiation' parameters and when the component is instantiated in coreAssembler, these parameters are specified before the component is added to the subsystem (via the GUI). In batch mode these parameters must be set prior to completing the Add Subsystem Components activity.
<code>-used <parameterExpression></code>	Limits the maximum number of consumers Specifies an instance specific value for the maximum number of consumers that can be connected to the interface.
<code>-max < consumers></code>	Appropriate only for interfaces of type provider or provider-consumer. This value will override the value (if any) from the interface definition. The bus type of the interface instance (Values: master, slave, system, monitor)
<code>-busType <type></code>	This option is used when instantiating a symmetric bus. The interface ports of symmetric bus are defined as 'slave' type by default, you need to specify which type so interface port direction can be determined.

-definition	Use this option to create the interface
<definition_name>	definition. Must be used with -auto
-attach	Create an attached interface instance
name	Name of the new instance

Description

This command comes in one of two flavors, -auto and -instance.

When -auto is specified, a provider interface instance is automatically created that connects to the given port list. This interface is setup so that it will automatically be exported when this component is instantiated in coreAssembler.

When -instances is specified, this command is used in coreBuilder to indicate that the core provides the specified interface playing the role specified by the -type option. It causes the creation of an interface instance which can be further modified using the set_interface_attribute command. In addition, the set_interface_port_attribute and set_interface_parameter_attribute commands can be used to override (where supported) values of attributes specified on the ports and parameters of the interface definition.

Examples

To add a provider and internal-consumer interface instance to the current core, execute the following commands (interface definitions are loaded as part of the LoadInterfaceDefns activity and are not shown here).

```
coreBuilder> create_interface_instance MasterProvider \
-interface MyMaster \
-type provider
coreBuilder> create_interface_instance MasterArbiter \
-interface MyMaster \
-type internal-consumer
```

To create an interface from a set of ports that is automatically exported from the subsystem:

```
coreBuilder> create_interface_instance API \
-auto {serial* porta b}
```

See Also

[create_interface \(2\)](#), [set_interface_attribute \(2\)](#),
[set_interface_port_attribute \(2\)](#), [set_interface_parameter_attribute \(2\)](#)

create_interface_parameter

Create interface parameter on an interface definition

Syntax

```
string create_interface_parameter [-specify_on <side>] -type <param_type>
-interface <interfaceName> [-used <condition>] [-label <label>]
[-description <text>] [-url <URL>] [-default <default value>] [-unique]
[-control] name
    string <side>
    string <param_type>
    string <interfaceName>
    string <condition>
    string <label>
    string <text>
    string <URL>
    string <default value>
    string name
```

Parameters

-specify_on <side>	Indicates on which side parameter needs to be specified (Values: provider, consumer) Parameters marked -specify_on provider are set exactly once, on the provider instance of the interface. Parameters marked -specify_on consumer are set once for each consumer of the interface.
-type <param_type>	The type of the parameter (Values: integer, bitfield, float, string, boolean)
-interface <interfaceName>	The parent interface definition of the new interface parameter
-used <condition>	Condition under which the parameter is used on its instance By default the condition is true (1). Then the interface parameter is used on both the consumer and the provider side, and the parameter must be associated to a design parameter using interface linkage. If a condition is specified using an @Parameter on-consumer interface parameter then the parameter is unused on a specific interface instance in case this condition evaluates to false for the given, static @ parameter value. Any valid 'expr' expression can be used as a condition, but a "static true" (always used) must be represent as "1" or "true", not as expression. An unused parameter ignores any interface-to-design linkage,

coreTools Command Reference Index

	and is not considered during parameter propagation in coreAssembler. Specifies the text that will appear for the parameter in the parameter dialog for the interface Parameters specified on providers appear once, and parameters specified on consumers appear once for each consumer. The label specified with this option is used to define the parameter in the dialog. For provider parameters, the label appears exactly as specified. For consumer parameters, the text " (for XXX)" is added, where XXX is the name of a consumer. If this option is not specified, the parameter name is used by default.
-label <label>	
-description <text>	Specifies the short help text to display This description will appear in reports associated with the interface definition and any instances of the definition.
-url <URL>	Specifies the URL of the help text or man page for the new interface parameter
-default <default value>	Specifies the default value for the parameter
	The interface parameter values must be unique for all consumers This option can be specified in conjunction with '-specify_on consumer' to indicate that the parameter must have a unique value on each consumer. This is useful for parameters which represent data 'slots' or ids where each consumer must have a unique setting for the given parameter.
-unique	
	The interface parameter is has a control-only value Typically it indicates usage of interface ports and other parameters, or the SlotWidth of its interface instance. Control parameters have two special features: For coreKit interfaces their value becomes read-only in coreAssembler, and they do not appear anymore in the interface configuration dialog: automatic read-only and implicit invisible. The automatic read-only implies that the value is static (unless it is a formula) important for any -used usage. But for exported or attached interfaces they are fully configurable in coreAssembler, and impact related interface instances, ports or parameters.
-control	
name	Name of the new interface parameter

Description

This command is used to define a new interface parameter on the specified interface definition. This command can only be used inside files which define interfaces. These files are loaded into coreBuilder in the LoadInterfaceDefns activity. A new interface parameter is defined with all the specified properties. Further information can be associated with the parameter using the `set_interface_parameter_attribute` command. The actual value of the parameter can be set using the `set_interface_parameter` command.

Examples

To define the 'DataWidth' parameter for a master bus interface and allow it to be set to only 16 or 32, use the following code. Data ports connected to this bus must all have the a size matching DataWidth.

```
prompt> create_interface_parameter DataWidth -interface MyMaster \
-default 16 \
-label "Data Width" \
-specify_on provider \
-description "Defines the width of the master data bus."
prompt> set_interface_parameter_attribute MyMaster DataWidth EnumValues
{16 32}
```

To define the 'Slot' parameter for a master bus interface and set it to a unique value between 0 and the total number of consumers - 1 on each consumer.

```
prompt> create_interface_parameter Slot -interface MyMaster \
-label "Slave Slot" \
-specify_on consumer \
-unique \
prompt> set_interface_parameter_attribute MyMaster Slot MinValue 0
prompt> set_interface_parameter_attribute MyMaster Slot MaxValue
@NumConsumers-1
```

See Also

[create_interface \(2\)](#), [create_interface_port \(2\)](#),
[set_interface_parameter_attribute \(2\)](#), [set_interface_parameter \(2\)](#),
[SlotWidth \(3\)](#)

create_interface_port

Create interface port on an interface definition

Syntax

```
string create_interface_port -direction <direction> -interface
<interfaceName> [-abstraction <abstraction VLNV>] [-optional] [-separate]
[-common] [-size <bits>] [-used <condition>] [-constant <constantInput>]
[-align <busAlignment>] [-description <text>] [-url <URL>] name
    string <direction>
    string <interfaceName>
    string <abstraction VLNV>
    string <bits>
    string <condition>
    string <constantInput>
    string <busAlignment>
    string <text>
    string <URL>
    string name
```

Parameters

	Connection direction between design ports of the interface port (Values: fromProvider, fromConsumer, bidirectional)
	The value fromProvider, means that information flows through the port from the provider to the consumer. The implication is that a design port associated with a 'provider' instance of the interface must be an output, and that a design port associated with a 'consumer' instance of the interface must be an input. The value fromConsumer implies data from from consumer to provider, so port directionalities of the design ports are opposite those for fromProvider interface ports. A value of bidirectional implies that ports on both provider and consumer must be bidirectional.
-direction <direction>	
-interface <interfaceName>	The parent interface definition of the new interface port
-abstraction <abstraction VLNV>	The abstraction to which this port belongs
-optional	The interface port is optional on consumer side Specifying -optional means that there does not need to be a design port associated with the interface port on consumer interface instances.

coreTools Command Reference Index

	The port must still be present on provider interface instances. The consumers have separate connections to the provider (default for fromConsumer)
-separate	This option is mutually exclusive with the -common option.
	The consumers share a common connection to the provider (default for bidirectional)
-common	This option is mutually exclusive to the -separate option.
	Number of bits on design port
-size <bits>	This can be a static value or can be a parameter expression such as @width, indicating that the port connects to @width bits.
	Condition under which the port is used on its instance
-used <condition>	By default the condition is true (1). Then the interface port is used on both the consumer and the provider side, and the port must be associated to a design port using interface linkage. If a condition is specified using an @Parameter on-consumer interface parameter, then the port is unused on a specific interface instance in case this condition evaluates to false for the given, static parameter value. Any valid 'expr' expression can be used as a condition, but a "static true" (always used) must be represent as "1" or "true", not as expression. An unused port ignores any interface-to-design linkage, and is not considered during automatic connection in coreAssembler.
	Default value to drive if the port is open on the interface instance (Values: none, zero, one, dc)
-constant <constantInput>	If this option is not specified, it is not legal for the provider to be left unconnected.
	Bus alignment for narrow design ports (Values: full, left, right, slice)
-align <busAlignment>	By default, both sides must have the same width. If this option is specified, it is legal to connect from a narrower port to a wider one. For example, a 'fromProvider' interface port may support connecting an 8 bit provider port to a 16 bit consumer port. The default is 'full' which implies that both sides must connect fully (i.e. be the same width). Values of left and right indicate to connect to the left or right side of the bus. The value of slice means that the bit specification of the provider is used to determine which bits to connect to. For example, using the

	same example as before, if using -align slice and connecting provider bits 2:4, then bits 2:4 will be utilized (and must be present) on the consumer.
-description <text>	Specifies the short help text to display This description will appear in reports associated with the interface definition and any instances of the definition.
-url <URL>	Specifies the URL of the help text or man page for the new interface port
name	Name of the new interface port

Description

This command is used to define a new interface port on the specified interface definition. This command can only be used inside files which define interfaces. These files are loaded into coreBuilder in the LoadInterfaceDefns activity. A new interface port is defined with all the specified properties. Further information can be associated with the port using the set_interface_port_attribute command.

Examples

To define an interface port for a design connection which goes from provider to consumer with a port of fixed width 16 and driven by a constant 0 if there is no associated provider port, use the following code:

```
prompt> create_interface_port Data -interface MyMaster \
-size 16 \
-constant 0 \
-description "Data port for the master bus."
```

See Also

[create_interface \(2\), set_interface_port_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

create_memory_map
Create a memory map.

Syntax

```
string create_memory_map name  
    string name
```

Parameters

name Name of the memory map to create.

Description

This command creates a memory map with the given name. A memory map is a container used to hold different address regions. The address regions are described by adding address blocks and address banks to the memory map.

The memory map is then associated to an interface instance with the MemoryMap attribute to describe the memory exposed by that interface.

Examples

```
coreBuilder> create_memory_map map1
```

See Also

memMap (3), addressBank (3), addressBlock (3), remove_memory_map (2),
create_address_bank (2), create_address_block (2)

create_memory_mode

Create a memory mode for the current/selected design

Syntax

```
string create_memory_mode -name <name> [-description <description>]
[-shortdescription <shortdescription>] [-displayname <displayname>]
[-vendorextensions <vendorextensions>] [-condition <condition>]
[-overwrite]
    string <name>
    string <description>
    string <shortdescription>
    string <displayname>
    string <vendorextensions>
    string <condition>
```

Parameters

-name <name>	Name of the memory mode to create.
-description <description>	Description of the mode.
-shortdescription <shortdescription>	Short Description of the mode.
-displayname <displayname>	Display name of the mode.
-vendorextensions <vendorextensions>	Vendor Extensions of the mode.
-condition <condition>	Condition of the mode.
-overwrite	Overwrite existing mode

Description

This command creates a memory mode with the given name on the design. The mode can be associated with the registerField attributes like MemoryAccess, ReadAction and WriteBehaviour to support different field accesses based on different modes.

The modes created using these commands would be considered as valid modes and one of them could be set as current memory mode on design using set_memory_mode.

Examples

The following commands creates the 'cache' and 'cpu' memory mode respectively.

coreTools Command Reference Index

```
coreBuilder> create_memory_mode -displayname "Cache Mode" -name cache  
-description "Cache Mode"
```

```
coreBuilder> create_memory_mode -displayname "Cpu Mode" -name cpu  
-description "Cpu Mode"
```

The following command can be used to overwrite the existing mode like 'cpu' memory mode.

```
coreBuilder> create_memory_mode -displayname "Cpu Mode" -name cpu  
-description "Cpu Mode" -overwrite
```

The following commands sets the MemoryAccess attribute of the register 'reg1' for mode 'cache' to be 'read-only' and for 'cpu' mode to be 'read-write' :

```
coreBuilder> set_register_attribute map1/block1/reg1 MemoryAccess -mode  
cache read-only
```

```
coreBuilder> set_register_attribute map1/block1/reg1 MemoryAccess -mode  
cpu read-write
```

The following commands sets the MemoryAccess attribute of the register field 'field1' of register 'reg1' for mode 'cache' to be 'read-only' and for 'cpu' mode to be 'read-write' :

```
coreBuilder> set_register_field_attribute map1/block1/reg1/field1  
MemoryAccess -mode cache read-only
```

```
coreBuilder> set_register_field_attribute map1/block1/reg1/field1  
MemoryAccess -mode cpu read-write
```

See Also

set_register_field_attribute (2), set_register_attribute (3),
get_memory_mode (4), set_memory_mode (5)

`create_plugin_kb`

Create a plug-in KB from the files in the given directory

Syntax

```
string create_plugin_kb -name <name> [-mode <requiredActivityMode>]
[-no_search] [-work_dir <directory>] [-post <code>] [-no_summary]
[<directory>]
```

```
string <name>
string <requiredActivityMode>
string <directory>
string <code>
string <directory>
```

Parameters

<code>-name <name></code>	Name of the plug-in (with possible path). coreBuilder builds the plugin knowledge database (KB) as <code><name>.kb</code> .
<code>-mode <requiredActivityMode></code>	Specifies the tool mode(s) required to load the plug-in. Allowed values are consultant, builder, developer, assembler. If you specify a mode, you cannot load the plugin if you are using the tool in the other modes. For example, if you specify <code>-mode builder</code> , you can only use the plugin with coreBuilder; you cannot use the plugin when running coreConsultant or coreAssembler.
<code>-no_search</code>	Don't search by default.
<code>-work_dir <directory></code>	Directory in which temp files should be created.
<code>-post <code></code>	TCL code to invoke prior to saving the plugin KB to disk. This can be used to modify the plugin KB prior to it being made read-only.
<code>-no_summary</code>	Don't generate summary file list
<code><directory></code>	The name of the directory that contains the plug-in source.

Description

The `create_plugin_kb` command creates a plugin knowledge database (KB). A plugin is used to alter the default behavior of a coreTool. You can

coreTools Command Reference Index

create a plugin that can be used in any tool (the default) or you can specify that the plugin can only be used in coreConsultant, coreAssembler, or coreBuilder by using the -mode option.

The create_plugin_kb command sources all of the .tcl files in the specified directory (current directory by default). The .tcl files contain the Tcl scripts, including custom command procedures that provide the plugin's functionality. For a plugin that modifies the default behavior of an existing coreTool design activity, the .tcl file(s) must also contain the add_activity_hook command that associates the Tcl command procedures with the existing activity.

After sourcing the .tcl files, create_plugin_kb sources any file(s) in the specified directory that end with Obj.tcl. The Obj.tcl files are optional and contain the commands that create the data objects needed for the plugin. For example, if the plugin creates a custom activity, the Obj.tcl file(s) must contain the create_custom_activity, create_custom_activity_parameter, and complete_custom_activity_definition commands that create the custom activity and (optionally) parameters for the custom activity. To add other files (e.g. data files, test files, etc.) and filegroups to a plugin, use the commands add_files_to_filegroup and create_autoload_filegroup in the Obj.tcl file.

After sourcing the .tcl and Obj.tcl files, and creating the associated data objects, create_plugin_kb writes out the KB file with the name specified by the -name option. You can then read the <name>.kb file into the coreTool and access the functionality defined in the plugin.

When you read a plugin KB, the coreTool sources the .tcl files, which means that the procedures defined in the .tcl files are available as long as the plugin KB is loaded. If the plugin modifies an activity or creates a new activity, then the modification or new activity is present in the coreTool design flow.

To build a tool-specific (coreBuilder, coreConsultant, or coreAssembler) plugin, execute the create_plugin_kb command directly. To build a core-specific plugin, execute the coreBuilder "Load Consultant Plugins" activity, which automatically invokes create_plugin_kb.

Examples

To create a plugin KB named MyPlugIn.kb from the .tcl and Obj.tcl files in the plugin_source directory:

```
coreBuilder> ls plugin_source
myplugin.tcl
myplugin_Obj.tcl
coreBuilder> create_plugin_kb -name MyPlugIn plugin_source
coreBuilder> ls plugin_source
myplugin.tcl
```

myplugin_Obj.tcl

MyPlugIn.kb

To invoke coreBuilder in shell mode to create a coreBuilder-mode-only plugin named MyBuilderPlugin from the .tcl and Obj.tcl files in the current directory, and then quit:

```
% coreBuilder -shell -x "create_plugin_kb -name MyBuilderPlugin
-mode builder; quit"
```

See Also

[add_activity_hook \(2\)](#), [add_files_to_filegroup \(2\)](#), [add_workspace_hook](#)

[\(2\)](#), [create_custom_activity \(2\)](#), [create_autoload_filegroup \(2\)](#),

[complete_custom_activity_definition \(2\)](#), [create_custom_activity_parameter](#)

[\(2\)](#), [load_plugin \(2\)](#), [unload_plugin \(2\)](#), [plugin_proc \(2\)](#)

create_prime_profile_file

Creates a Prime Profile file to be packaged with the coreKit or shipped externally.

Syntax

```
string create_prime_profile_file -tag <Prime Profile name> -label <text>
-description <text> [-body <tcl config commands>] [-cfg_file <file name>]
[-out_path <directory>] [-lock <all | param list>] [-dont_lock <param
list>] [-analysis_lock <all | param list>] [-license <license string>]
[-filter_profiles <filter string>] [-early_access] [-group <group name>]
[-param_name <config param name>] [-value <config param value>]
[-expression <expression>]
    string <Prime Profile name>
    string <text>
    string <tcl config commands>
    string <file name>
    string <directory>
    string <all | param list>
    string <param list>
    string <license string>
    string <filter string>
    string <group name>
    string <config param name>
    string <config param value>
    string <expression>
```

Parameters

-tag <Prime Profile name>	Unique identifier for the Prime Profile selected.
-label <text>	A short one line description for the Prime Profile trigger being defined.
-description <text>	Detailed description of Prime Profile being created.
-body <tcl config commands>	Configuration settings for the selected Prime Profile.
-cfg_file <file name>	Path to an external config file (relative to current working dir, if full path is not used).
-out_path <directory>	Specifies the output directory for external Prime Profile generation.
-lock <all param list>	Parameters to be locked when Prime Profile is applied.
	All parameters except these are locked.

-dont_lock <param list>	
-analysis_lock <all param list>	Parameters to be locked in analysis mode when Prime Profile is applied.
-license <license string>	License string associated with a Prime Profile.
-filter_profiles <filter string>	Specify plugin procs, env vars or global vars for filtering out profiles during workspace creation or opening.
-early_access	Designates the Prime Profile as an early access profile.
-group <group name>	Assign a group to a Prime Profile for help with GUI selection.
-param_name <config param name>	(Deprecated) Name of the design parameter that triggers the Prime Profile (trigger parameter).
-value <config param value>	(Deprecated) Trigger value for the trigger parameter
-expression <expression>	(Deprecated) Parameter expression checked before a Prime Profile is triggered.

Description

In **coreBuilder**, IP designers can package application specific configurations for an IP by creating Prime Profiles. After defining a configuration (which makes up the body of the profile), a Prime Profile can be created using this command. Applying a profile will configure parameters as defined in the body, and lock parameters based on arguments specified with the **-lock**, **-analysis_lock** and **-dont_lock** options.

In **coreConsultant**, the activity **Select Prime Profile, which precedes Specify Configuration** activity can be used to select and apply one of the packaged Prime Profiles in an IP.

Note

- ◇ **Select Prime Profile** activity will not appear if an IP has been packaged with Prime Profiles using trigger parameters and expressions (deprecated usage).
- ◇ Prime Profiles for an IP must ALL be created either using triggers OR without using triggers. An error is issued in **coreBuilder** if the two approaches are mixed.

Designers may want to make an IP more configurable when a Prime Profile is applied. The **-analysis_lock** option can be used to specify and different set of parameters to be locked (usually a smaller subset then those specified in **-lock** option). However, RTL generation is not allowed in analysis mode.

Deprecated:

Creating Prime Profiles using trigger parameters, trigger values and expressions:

Using `-param_name`, `-value` and `-expression` options with this command is the legacy method to create Prime Profiles and is now deprecated. A coreConsultant workspace for an IP packaged with Prime Profiles using any of these options will NOT see the Select Prime Profile activity.

The `-param_name` and `-value` options associate Prime Profile with a trigger parameter and a value. Setting the trigger parameter to a trigger value will enable application of the Prime Profile associated with them in coreConsultant and coreAssembler. A Prime Profile can then be applied using the `set_prime_profile` command or via the GUI pop-up that appears when a trigger parameter is set.

It is NOT recommended to use these 3 options while creating a Prime Profile (including external ones) since support for creating Prime Profiles with this approach will be dropped in the future.

See 'Creating Prime Profiles with triggers' in the Examples section for usage.

Examples

Creating a basic Prime Profile

- The argument specified with the `-tag` is a unique identifier for the Prime Profile being generated. Both the Select Prime Profile activity and the Prime Profile selection pop-up on triggering a Prime Profile display the tag of a Prime Profile for allowing selection and application of a Prime Profile.
- The `-label` text appears in the Select Prime Profile activity (or the GUI pop-up) alongside the Prime Profile tag.
- The `-description` text is printed to the console when a Prime Profile is applied.

The following command generates a Prime Profile (named Test001) with the content specified with the `-body` option. Parameters to be locked are specified with the `-lock` option and parameters to be locked in analysis mode are specified using the `-analysis_lock` option.

```
coreBuilder> create_prime_profile_file \
-tag Test001 \
-description "Long Descr 001" \
-lock [list rom_size rom_word] \
-body {set_configuration_parameter ram_word 8} \
-analysis_lock "dummy" \
```


-label "Short description 001"

The following command creates a Prime Profile named `ProfileA` using the

-`cfg_file` option instead of -`body`. The contents of `iipregr/configs/prime_profiles/ProfileA.tcl` form the body of the Prime Profile. A full path may be provided here. If a relative path is provided, it is assumed to be relative to the current working dir, which is usually the directory above the workspace directory.

```
coreBuilder> create_prime_profile_file \
-tag ProfileA \
-description "Long Desc of ProfileA" \
-lock all \
-analysis_lock [list APB_DATA_WIDTH PERIPH_RDATA_WIDTH] \
-cfg_file iipregr/configs/prime_profiles/ProfileA.tcl \
-label "Short description of ProfileA"
```

Locking Parameters (-lock)

A Prime Profile configuration for an IP can also be locked using the -`lock` option. Parameter object collections or parameter name lists can be provided as arguments with this option. Specifying `all` with -`lock` will lock down all parameters on applying a Prime Profile. Designers must be cautious with locking down more parameters than required by the Prime Profile since it may be too restrictive for end users. In cases where parameters are configured in activity hooks for `SpecifyConfiguration`, locking down such parameters may result in errors which are only exposed in regressions. It is recommended to lock down those parameters that are configured explicitly by the Prime Profile to avoid unexpected or unvalidated configurations.

```
coreBuilder> set lParam [find_item -quiet -type param "ParamA ParamB
ParamC"]
coreBuilder> create_prime_profile_file \
-tag ProfileB \
...
-lock $lParam \
...
...
```

When only a few parameters are to be locked, the param names can be directly specified in the command as follows:

```
coreBuilder> create_prime_profile_file \
...
...
-lock "ParamA ParamB ParamC" \
...
```

For locking down all params, pass the `all` string as follows:

```
coreBuilder> create_prime_profile_file \
...
...
```

```
-lock "all" \
```

```
...
```

Keeping Parameters Unlocked (-dont_lock)

Users may want to lock all but some parameters in a Prime Profile. The `-dont_lock` option (not to be used with `-lock` option), can be used to specify parameters that should remain unlocked. All other design parameters except these will be locked on applying the Prime Profile.

```
coreBuilder> set oParam [find_item -quiet -type param "AUTO_BCM95_TMR
PARAMA PARAMB PARAMC"]
```

```
coreBuilder> create_prime_profile_file \
-tag ProfileB \
```

```
...
```

```
-dont_lock $oParam \
```

```
...
```

```
...
```

When only a few parameters are to be kept unlocked, the param names can be directly specified in the command as follows:

```
coreBuilder> create_prime_profile_file \
```

```
...
```

```
...
```

```
-dont_lock "include_rom rom_size rom_word" \
```

```
...
```

Applying the above Prime Profile will keep params `include_rom`, `rom_size`, and `rom_word` unlocked, while locking all others.

Adding Licenses and Filters (-license and -filter_profiles)

The `-license` switch can be used to specify license strings and `-filter_profiles` can be used to specify filter strings. In `coreConsultant` and `coreAssembler`, only those Prime Profiles with valid license and filter strings (or no license/filter strings) are displayed in the Select Prime Profile activity. License and filter checks are carried out on opening or creating a workspace. These are also evaluated while applying the Prime Profile and must be true for successful application. When adding license checks, the `-project` option should always be specified with a 'dash' as the `-project` value as shown below. This ensures that only project specific licenses will be utilized.

```
# Specify a single license check
```

```
create_prime_profile_file \
```

```
...
```

```
-license "check_license -project - -authorized Designware" \
```

```
...
```

```
...
```

```
# Specify multiple licenses
```

```

create_prime_profile_file \
...
-license {expr { [check_license -project - -authorized Designware] &&
[check_license -project - -authorized DC-Ultra-Features] }} \
...
...
# Specify a filter string
create_prime_profile_file \
...
-filter_profiles {expr {[info exists ::env(FILTER_PROFILE)] &&
$::env(FILTER_PROFILE) == 5}}
...
...

```

Grouping Prime Profiles (-group)

In the **Select Prime Profile** activity GUI coreConsultant and the Prime Profile selection pop-up in coreAssembler, a Prime Profile Group selection drop down appears if the -group option was used during packaging. This allows further filtering of the Prime Profiles that appear in the Valid Prime Profiles drop down.

```

create_prime_profile_file \
-tag ProfileA1 \
...
-group "GroupA" \
...
...
create_prime_profile_file \
-tag ProfileA2 \
...
-group "GroupA" \
...
...
create_prime_profile_file \
-tag ProfileB1 \
...
-group "GroupB" \
...
...

```

Note

This option does not affect batch mode or batch script generation in any manner and is only provided to facilitate easier Prime Profile selection.

Creating External Prime Profiles (-out_path)

An external Prime Profile file (not packaged in the IP coreKit), can be generated in coreBuilder or coreConsultant, using the -out_path option (this option is mandatory). It generates a file titled <tag>.pp.gz in the

directory specified using `-out_path`. The tag provided here must be a unique identifier and cannot collide with the tags packaged in the coreKit. The tool cannot verify tag name collision with other external Prime Profiles, so designers must be careful not to ship multiple external Prime Profiles with the same tag.

This Prime Profile file can be shipped externally and applied in `coreConsultant` or `coreAssembler` after the designs are loaded, using the `set_prime_profile` command with the `-file` option. External profiles are used to reduce turn around time for delivering new configurations to customers, and do not require IP designers to generate coreKits again. For ease of use, external Prime Profiles can also be generated in `coreConsultant`.

Note

Creating an external Prime Profile with triggers using the `-param_name` and `-value` options or expressions using the `-expression` option is NOT allowed. As such, the command exits with an error if these options are used with the `-out_path` option.

- For `coreBuilder`, the workspace used to create the coreKit must be used to generate an external Prime Profile file.
- For `coreConsultant`, a workspace based on the installed coreKit must be used to generate an external Prime Profile file.

A typical set of commands for creating an external Prime Profile using `coreBuilder` is as follows:

```
coreBuilder> open_workspace <coreBuilder workspace>
coreBuilder> set_activity_incomplete DesignConfigurationIntent
coreBuilder> set_ramConfigParams [find_item -quiet [get_design_attribute
Parameters] -filter "Name=~RAM_CONFIG"]
coreBuilder> create_prime_profile_file \
-tag TestExternal \
-out_path "/path/to/output/dir" \
-description "Long Desc" \
-lock $ramConfigParams \
-analysis_lock [list ENABLE_RAM_CONFIG] \
-cfg_file customPrimeProfile.tcl \
-label "Short Desc"
```

Using `coreConsultant`:

```
coreConsultant> create_workspace -name config1 -installation
$::env(DESIGNWARE_HOME)/iip/<name>/<version>
coreConsultant> set cAll [get_design_attribute Parameters]
coreConsultant> create_prime_profile_file \
-tag TestExternal \
-out_path "/path/to/output/dir" \
-description "Long Desc" \
-lock $cAll \
-analysis_lock [list RM_BCM00_MAJ] \
```

```
-cfg_file customPrimeProfile.tcl \
-label "Short Desc"
```

Note

If `coreBuilder` is used to generate external Prime Profiles, the resulting Prime Profile file will NOT be packaged in the coreKit.

Early Access Prime Profiles (-early_access)

IP designers may want to ship configurations which are in early access stage. They can create profiles using the `-early_access` switch as shown. These profiles are not suitable for tape-out and a message/prompt is displayed when such a profile is being applied.

```
coreConsultant> create_prime_profile_file \
-tag TestExternal \
...
-early_access \
...
...
```

Encryption

All Prime Profile files are encrypted by default. IP designers can use the hidden `-dont_encrypt` hidden switch to leave the Prime Profiles generated unencrypted. It must only be used for examining Prime Profiles and not in production.

```
create_prime_profile_file \
-tag Test003b \
...
-dont_encrypt
...
```

Creating Validation Expressions

Validation expressions (`set_prime_profile_validation_expr`) are used to prevent users from entering into configurations or enabling IP features that were intended to be restricted using Prime Profiles. After all the Prime Profiles are created, designers must come up with validation expressions and group Prime Profiles with common expressions together.

Designers must also ensure that two disjoint lists of Prime Profiles can never have their validation expressions true resulting in an error. This can be tested by simply applying each Prime Profile and completing `SpecifyConfiguration` or `SpecifySubsystem` activities. Since applying external Prime Profiles skips these validation expression checks, all testing for packaged Prime Profiles must be done using the standard process (selecting a Prime Profile via `Select Prime Profile` activity drop down selection or using `set_prime_profile -tag <Prime Profile Tag>` command.)

Designating Default Prime Profiles

Designers may want to apply a default Prime Profile if the end users do not explicitly select one. The `set_default_prime_profiles` command may be used after all Prime Profiles are created to designate a list of Prime Profiles as default Prime Profiles. These profiles appear at the top of the drop-down Prime Profile selection field in the `Select Prime Profile` activity. The first one in the list (if it has a valid license expr or no license expr) is auto-selected and applied if users do not explicitly select a different one.

Allow None option

The `PrimeProfileAllowNone` design attribute can be modified to control the appearance of a <None> option which allows skipping Prime Profile application for an IP.

Creating Prime Profiles with triggers (Deprecated)

Deprecated:

The following command creates a Prime Profile named Test001 with a trigger parameter and a value. When the trigger parameter `dummy` is set to value 1 in `coreConsultant` or `coreAssembler`, Prime Profile mode is triggered and Test001 and any other Profile with the same trigger parameter-value combination will be added to the list of triggered Prime Profiles for that component/design.

In shell mode, `set_prime_profile` must then be used to set a Prime Profile, e.g. Test001, from the list of triggered Prime Profiles.

In GUI mode, a Prime Profile selection pop-up will appear on setting a trigger parameter and one of the triggered profiles (or an external Prime Profile) must be selected.

```
coreBuilder> create_prime_profile_file \
-tag Test001 \
-description "Long Descr 001" \
-lock [list rom_size rom_word] \
-body {set_configuration_parameter ram_word 8} \
-analysis_lock "dummy" \
-param_name dummy \
-value 1 \
-label "Short description 001"
```

Creating Prime Profiles with expressions (Deprecated)

Deprecated:

An optional expression can also be specified in a `create_prime_profile_file` command. The Prime Profiles associated with this parameter/value combination will only be triggered if the expression evaluates to true.

```
create_prime_profile_file \
-tag ProfileB \
-expression {@include_rom==1 && @rom_size==8} \
-param_name dummy \
-value 3 \
-description "desc txt" \
-lock all \
-cfg_file profile.tcl \
-label "lbl txt"
```

Porting Prime Profiles with triggers to Prime Profiles without triggers

IP designers/packaging teams can port Prime Profiles that use triggers to Prime Profiles without triggers by dropping the `-param_name`, `-value`, and `-expression` options from the `create_prime_profile_file` command in their packaging scripts. However, it must be ensured that those parameters and expressions are set to the right values in the Prime Profiles they were associated with. If trigger parameters were introduced in a design for the sole purpose of triggering Prime Profiles and do not affect the functionality of the IP, designers may consider removing those parameters altogether. Trigger parameter/values and expressions can provide guidance in writing validation expressions for trigger-less Prime Profiles.

See Also

[set_prime_profile\(2\)](#) [report_prime_profiles\(2\)](#)
[set_prime_profile_validation_expr\(2\)](#) [set_default_prime_profiles\(2\)](#)
[PrimeProfilesValid\(3\)](#) [PrimeProfileDefaults\(3\)](#)

create_reference_clock

Create a reference clock for use within the specified design

Syntax

```
string create_reference_clock [-name <name>] -source <port or pin>
[-master_clock <clock>] [-comment <text>] port
    string <name>
    string <port or pin>
    string <clock>
    string <text>
    string port
```

Parameters

-name <name>	The name for the reference clock If the -name argument is not used the name of the reference clock will be the same as the object on which it is created.
-source <port or pin>	Port or pin of the master clock
-master_clock <clock>	Master clock for the generated clock
-comment <text>	Associate a string description with the command for tracking purposes
port	Output port to be associated with the reference clock

Description

This command creates a reference clock object in the current design.

Reference clocks are modeled as generated clocks with the ReferenceClock attribute set to 'true'. They are used as placeholders to enable clock linking in coreAssembler. Reference clocks should be used on output ports of clock generation modules when an actual generated clock is not going to be defined. Reference clocks are not written to the SDC file. They are used only to enable the master clock to be linked to any clock connected to the corresponding source pin.

Examples

```
coreBuilder> create_reference_clock \
-name genClk \
```


coreTools Command Reference Index

-source clkPort \

genClkPort

get_clocks (2) set_clock_attribute (2)

`create_register`

Create register in an address block.

Syntax

`string create_register -block <block> -registerArray <register array>``-alternate <alternate> -size <bits> -offset <bytes> <name>``string <block>``string <register array>``string <alternate>``string <bits>``string <bytes>``string <name>`

Parameters

`-block <block>`

Create the register in the given address block.

`-registerArray <register array>`

Create the register in the given register array.

`-alternate <alternate>`

The alternate register for this register.

`-size <bits>`

Size of the register

`-offset <bytes>`

Offset from the base address of the address block.

`<name>`

Name of the register to create

Description

This command creates a register in the given address block at the specified offset (in bytes). The size in bits of the register is specified by the `-size` argument.

Examples

```
coreBuilder> create_register -name reg1 -block map1/block0 -size 16
-offset 0x0
```

See Also

[memMap \(3\)](#), [addressBank \(3\)](#), [addressBlock \(3\)](#), [RegisterSize \(3\)](#), [AddressOffset \(3\)](#), [register \(3\)](#), [remove_register \(2\)](#), [set_register_attribute \(2\)](#), [get_register_attribute \(2\)](#)

create_register_array

Create a register array in a memory block.

Syntax

```

string create_register_array -block <block> -registerArray
<register_array array> -range <range> -offset <bytes> [-dimensions
<dimensions>] <name>
    string <block>
    string <register_array array>
    string <range>
    string <bytes>
    string <dimensions>
    string <name>

```

Parameters

-block <block>	Create the register array in the given address block.
-registerArray <register_array array>	Create the register array in the given register array.
-range <range>	The range of this register array.
-offset <bytes>	Offset from the base address of the address block.
-dimensions <dimensions>	Dimensions of the register array.
<name>	Name of the register array to create

Description

This command creates a registerArray in the given address block or register array at the specified offset (in bytes).

Examples

```

cB> set blk "map1/block0"
cB> create_register_array -name regArray -block $blk -range 10 -dimension
8 -offset 0x100

```

See Also

[memMap \(3\)](#), [addressBlock \(3\)](#), [AddressOffset \(3\)](#), [MemoryRange \(3\)](#), [RegisterArrayDimensions \(3\)](#), [register \(3\)](#), [remove_register_array \(2\)](#), [set_register_array_attribute \(2\)](#), [get_register_array_attribute \(2\)](#).

create_register_field

Create a register field in a register.

Syntax

```
string create_register_field -register <register> -offset <bits> -size
<bits> <name>
    string <register>
    string <bits>
    string <name>
```

Parameters

-register	Create the register field in the given register.
<register>	
-offset <bits>	Offset from the start of the register.
-size <bits>	Size of the field
<name>	Name of the register field to create

Description

This command creates a register field in the given register at the specified bit offset of size specified in the -size argument.

Examples

The following command creates a register field 'field1' in the given register 'reg1'. It starts from bit offset 0 and has 2 bits.

```
coreBuilder> create_register_field -name field1 -register
map1/block1/reg1 \
-offset 0 -size 2
```

See Also

[create_memory_map\(2\)](#), [create_register\(2\)](#), [create_register_field\(2\)](#)
[set_register_field_attribute\(2\)](#), [get_register_field_attribute\(2\)](#).

create_register_field_value

Create a value for a register field.

Syntax

string **create_register_field_value** -field <field> -value <value> [-usage <usage>] [-description <desc>] name

string <field>
string <value>
string <usage>
string <desc>
string name

Parameters

-field <field>	The register field to this value belongs to
-value <value>	The bit value
-usage <usage>	Usage context for this value (Values: read-write, read-only, write-only)
-description <desc>	Description of the value
name	Name of the value.

Description

This command creates a register field value. These "values" describe the value that a register field me take.

Examples

```
coreBuilder> create_register_field_value MOV \  
-field map1/block0/instr/code -value 0x5 \  
-description "The move instruction."
```

See Also

[registerField\(3\), set_register_field_value attribute\(2\),](#)
[remove_register_field_value\(2\)](#)

create_subsystem_parameter
Create a parameter on the subsystem

Syntax

```
string create_subsystem_parameter -type <param type> -default <Default Value> <parameter name>
    string <param type>
    string <Default Value>
    string <parameter name>
```

Parameters

-type <param type>	integer, bitfield, float, string, boolean, custom (Values: integer, bitfield, float, string, boolean, custom)
-default <Default Value>	Default value of the Subsystem parameter.
<parameter name>	Name of the parameter

Description

This command is used to define parameter within a level of hierarchy in a coreAssembler subsystem. These parameters can then be passed to lower level components using the set_instance_parameter command to enable parameter propagation within a hierarchical subsystem.

Examples

Define a parameter width and pass into an instance via it's DataWidth parameter.

```
cA> set p [create_subsystem_parameter -type integer -default 8 width]
cA> set_attribute $p -attr MinValue 2
cA> set_attribute $p -attr MaxValue 32
cA> set_instance_parameter -instance U1 -parameter DataWidth @width
```

See Also

[set_instance_parameter\(2\)](#)

create_virtual_clock

Create a virtual clock for use within the specified design

Syntax

```
string create_virtual_clock [-design <design>] name
      string <design>
      string name
```

Parameters

-design	Specifies the design in which to create the virtual clock.
<design>	If you do not specify a design, create_virtual_clock creates the virtual clock in the current_design.
name	The name for the virtual clock.

Description

The create_virtual_clock command creates a virtual clock within the specified design, or within the current_design if you do not specify a design. A virtual clock is useful as a reference for timing constraint specification for a design that does not contain a real clock.

If the named virtual clock already exists within another design, either as a real clock or a virtual clock, the coreTool assumes the new virtual clock to be the same as the existing clock of that name. In such a case, all characteristics (attributes) of the clock are the same in all designs that contain the clock, including the design(s) in which the clock is a virtual clock.

To specify a virtual clock with different characteristics of any existing clock, give the virtual clock a name that is not currently assigned to any other clock.

To create a real clock, assign a value for the ClockName port attribute on the clock port. When you use set_port_attribute to assign a ClockName to a port, the coreTool creates a real clock with the specified ClockName.

Examples

To create a virtual clock named vclk in the current_design:

```
coreConsultant> create_virtual_clock vclk
```

Subblock_B does not contain a real clock. So, to create a virtual clock in Subblock_B that has the same characteristics as the real clock named clk that is used in the current_design:

```
coreConsultant> create_virtual_clock -design Subblock_B clk
```

See Also

get_clocks (2), remove_virtual_clock (2), set_port_attribute (2),
ClockName (3)

create_workspace

Create a new workspace (working area).

Syntax

```
string create_workspace [-name <name>] [-root <root dir>] [-installation
<install dir>] [-bom_template <file>] [-design <name>] [-template
<template_file>] [-testbench] [-language <language>] [-ipxact] [-replace]
    string <name>
    string <root dir>
    string <install dir>
    string <file>
    string <template_file>
    string <language>
```

Parameters

-name <name>	The name of the new workspace and knowledge database. This name will be used to create the workspace directory under the <root dir>.
-root <root dir>	The root directory in which to create the new workspace. This directory must already exist.
-installation <install dir>	The path to the coreKit installation directory (coreConsultant only).
-bom_template <file>	Load the specified BOM template file (coreBuilder only). The BoM template defines filegroups and indicates which groups are required, optional, etc. This option should only be specified if you have defined a valid BoM template, or if you want to use a template other than the default.
-design <name>	The name of the new top-level design (coreAssembler only). Creating a cA workspace from a template.
-template <template_file>	A template is a compressed cA workspace that has partially configured subsystem to be started with.
-testbench	Create a cA workspace for testbench assembly A testbench workspace customizes cA for assembly of a testbench around a DUT instance.
-language <language>	Target language of generated RTL (coreAssembler only). (Values: Verilog, SystemVerilog, VHDL)
-ipxact	Create a cB workspace for creating ipxact output

`-replace` Replace an existing workspace while preserving RCS files.

Description

This command is used to create a workspace for one of the suite of the coreTools (coreBuilder, coreConsultant, or coreAssembler). The workspace defines a directory hierarchy, rooted inside a directory specified by the `-name` option. The workspace directory is created under the root directory specified with the `-root` option. The actual directories created underneath the workspace directory depend on which tool the workspace is being created for.

Examples

To create a coreConsultant workspace named myCore in the current directory:

```
coreConsultant> create_workspace -name myCore
```

To create a coreBuilder workspace named bigDesign under /usr/fred, with the core to be packaged using a site specified BoM template:

```
coreBuilder> create_workspace -name bigDesign -root /usr/fred
-bom_template \
/usr/fred/templates/company.tcl
```

To create a coreConsultant workspace named myCore under /usr/fred with all designs in the core prefixed with the string U1_:

```
coreBuilder> create_workspace -name myCore -root /usr/fred -prefix U1
```

See Also

[open_workspace \(2\)](#), [close_workspace \(2\)](#), [duplicate_workspace \(2\)](#),
[save_workspace \(2\)](#)

crm_file_patterns

Identifies files that are part of your source code control system.

Syntax

```
string crm_file_patterns = ".svn .cvs"
```

Description

crm_file_patterns is a global variable that defines name patterns which match files and/or directories which are part of your source code control system. This information is used when creating a workspace with the -replace option. Instead of deleting the entire workspace, everything except for directories and files matched by the CRM file patterns is removed.

Examples

Add .rcs directories to those that should not be removed:

```
set ::crm_file_patterns [concat .rcs $::crm_file_patterns]
```

See Also

current_activity
Return current activity

Syntax

string **current_activity**

Description

This command can be used to return the current activity as a collection. It can only be called during activity completion or while interacting with an activity in the GUI. It is generally available for use within files configured in different activities (so that the file can now which activity is configuring it), or within pre and post activity hook commands, again to enable knowing which activity is invoking the hook.

Examples

Configurable file in filegroup that wants to know which activity configured it:

```
# testfile
# reuse-pragma startSub [format "# Configured for %s" [get_attribute
[current_activity] -attr Name]] endSub
```

See Also

`current_design`

Set or get the current design

Syntax

```
string current_design [-quiet] [-unset] [<design name>]
      string <design name>
```

Parameters

<code>-quiet</code>	Do not print informational messages.
<code>-unset</code>	Unset the current design.
<code><design name></code>	Make the specified design the current design.

Description

The `current_design` command sets the current design to the specified design, and returns a collection that contains the (new) `current_design`. If you do not specify a design name, `current_design` just returns a collection that contains the current design.

Several coreTools commands, such as `all_inputs` and `all_subdesigns`, operate on the `current_design`. Other commands, such as `get_design_attribute` and `set_design_attribute`, operate on the `current_design` by default if you do not specify a design.

Examples

To specify Subblock_A as the current design:

```
coreConsultant> current_design Subblock_A
```

To set the value of the `my_design` variable to the name of the current design:

```
coreConsultant> set my_design [current_design]
```

See Also

[set_design_attribute \(2\)](#), [find_design \(2\)](#)

current_kb

Set or get the current knowledge database (KB)

Syntax

```
string current_kb [-quiet] [<KB name>]  
      string <KB name>
```

Parameters

-quiet	Do not print informational messages.
<KB name>	The name of the KB that you want to make the current KB.

Description

The `current_kb` command sets the current KB to the specified KB, and returns a collection that contains the (new) `current_kb`. If you do not specify a KB name, `current_kb` just returns a collection that contains the current KB.

The current KB is the one to which the coreTool adds all new items that you create. The `current_kb` also controls the default KB search list used when the coreTool looks for items in all loaded knowledge bases.

The coreTools automatically change the `current_kb` as needed during execution of activities.

Examples

To get the name of the `current_kb`:

```
coreConsultant> current_kb  
{config1}
```

See Also

[current_design \(2\)](#)

NAME

date Returns a string containing the current date and time.

SYNTAX

string **date**

DESCRIPTION

The **date** command generates a string containing the current date and time, and returns that string as the result of the command. The format is fixed as follows:

ddd mmm nn hh:mm:ss yyyy

Where:

ddd is an abbreviation for the day
mmm is an abbreviation for the month
nn is the day number
hh is the hour number (24 hour system)
mm is the minute number
ss is the second number
yyyy is the year

The **date** command is useful because it is native. It does not fork a process. On some operating systems, when the process becomes large, no further processes can be forked from it. With it, there is no need to call the operating system with **exec** to ask for the date and time.

EXAMPLES

The following command prints the date.

```
prompt> echo "Date and time: [date]"
Date and time: Thu Dec 9 17:29:51 1999
```

SEE ALSO

exec(2).

[API Index](#) [Command Reference](#)

`dc_shellStopMessageIds`

Messages IDs which should cause Design Compiler to terminate.

Definition

Type: string**Flags:****Default value:****Valid on:** design_____

Description

This design attribute is used to cause Design Compiler to stop when certain messages are emitted into the Design Compiler log output. The attribute is set to a list of message IDs corresponding to the messages which should cause Design Compiler to terminate. Note that the actual behavior of Design Compiler in this instance depends on the value of the `sh_script_stop_severity` variable. If set to its default value (None), the script will still not stop, but the message will be indicated by a "Severe Error" message prefix.

Examples

Cause DC to stop when the OPT-150 or OPT-151 messages are present:

```
set_design_attribute dc_shellStopMessageIds [list OPT-150 OPT-151]
set_design_attribute {dc_shellVariable[sh_script_stop_severity]} E
```

See Also

[`dc\_shellVariable` \(3\)](#)

dc_shellVariable

Variable settings to be used for dc_shell. Example: set_design_attribute {dc_shellVariable[varName]} value.

Definition

Type: string

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This subscripted attribute can be used to set the variables each time dc_shell is run for designs in the core or subsystem. If a design does not have an explicit setting for a subscript, the value is inherited from its parent design. The subscript of the attribute is the variable name and the value is the value that the variable will be set to in dc_shell.

The subscripts supported are as follows:

- <modeName>:internal:<variable>
- <modeName>:<variable>
- internal:<variable>
- <variable>

modeName: Currently, the mode name "topographical" is supported.

If the design has TclAuxSynthesisScript[setup], the variable settings will be applied in dc_shell after the setup aux script has been sourced.

Examples

Specify some variables that are to be applied at setup each time dc_shell is run for any of the designs on the compile frontier.

The following commands are written in intial.tcl or dc.tcl:

1. **set_app_var** command: This gets written out during the following set_design_attribute commands:

```
coreConsultant> set_design_attribute {dc_shellVariable[null_var]} ""
set_app_var null_value {}
```

```
coreConsultant> set_design_attribute {dc_shellVariable[my_var]} my_value
```

set_app_var my_var my_value

```
coreConsultant> set_design_attribute
{dc_shellVariable[topographical:my_topo_var]} my_value
set_app_var my_topo_var my_value
```

```
coreConsultant> set_design_attribute
{dc_shellVariable[topographical:::tag::my_topo_var]} my_value
set_app_var ::tag::my_topo_var my_value
```

```
coreConsultant> set_design_attribute {dc_shellVariable[list_var]} [list A
list of values]
set_app_var list_var {A list of values}
```

1. **set** command: This gets written out during the followings:

```
coreConsultant> set_design_attribute
{dc_shellVariable[internal:my_int_var]} my_value
set my_int_var my_value
```

```
coreConsultant> set_design_attribute
{dc_shellVariable[topographical:internal:my_topo_int_var]} my_value
set my_topo_int_var my_value
```

```
coreConsultant> set_design_attribute
{dc_shellVariable[topographical:internal:::tag::my_topo_int_var]}
my_value
set ::tag::my_topo_int_var my_value
```

See Also

set_design_attribute (2), TclAuxSynthesisScript (2)

[API Index](#) [Command Reference](#)

`dc_shellVariableComment`

Comment to be issued for the corresponding subscript of `dc_shellVariable`.

Definition

Type: string

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

The `dc_shellVariableComment` attribute can be used to output a comment to document why the corresponding `dc_shellVariable` is being used. The comment will be put into the file where the variable is set.

Examples

Set variable to be used in `dc_shell`, and specify a comment.

```
set_design_attribute {dc_shellVariable[myVar]} myValue
set_design_attribute {dc_shellVariableComment[myVar]} "Sample comment for
setting myVar."
```

See Also

[dc_shellVariable \(3\)](#)

NAME

dde - Execute a Dynamic Data Exchange command

SYNOPSIS

```
package require dde 1.4

dde servername ?-force? ?-handler proc? ?--? ?topic?

dde execute ?-async? ?-binary? service topic data      |
dde poke ?-binary? service topic item data
dde request ?-binary? service topic item
dde services service topic
dde eval ?-async? topic cmd ?arg arg ...?
```

DESCRIPTION

This command allows an application to send Dynamic Data Exchange (DDE) command when running under Microsoft Windows. Dynamic Data Exchange is a mechanism where applications can exchange raw data. Each DDE transaction needs a *service name* and a *topic*. Both the *service name* and *topic* are application defined; Tcl uses the service name **TclEval**, while the topic name is the name of the interpreter given by **dde servername**. Other applications have their own *service names* and *topics*. For instance, Microsoft Excel has the service name **Excel**.

DDE COMMANDS

The following commands are a subset of the full Dynamic Data Exchange set of commands.

dde servername *?-force? ?-handler proc? ?-? ?topic?*
dde servername registers the interpreter as a DDE server with the service name **TclEval** and the topic name specified by *topic*. If no *topic* is given, **dde servername** returns the name of the current topic or the empty string if it is not registered as a service. If the given *topic* name is already in use, then a suffix of the form *or* is appended to the name to make it unique. The command's result will be the name actually used. The **-force** option is used to force registration of precisely the given *topic* name.

The **-handler** option specifies a Tcl procedure that will be called to process calls to the dde server. If the package has been loaded into a safe interpreter then a **-handler** procedure must be defined. The procedure is called with all the arguments provided by the remote call.

dde execute *?-async? ?-binary? service topic data*
dde execute takes the *data* and sends it to the server indicated by *service* with the topic indicated by *topic*. Typically, *service* is the name of an application, and *topic* is a file to work on. The *data* field is given to the remote application. Typically, the application treats the *data* field as a script, and the script is run in the application. The **-async** option requests asynchronous invocation. The command returns an error message if the script did not run, unless the **-async** flag was used, in which case the command returns immediately with no error. Without the **-binary** option all data will be sent in unicode. For dde clients which don't implement the CF_UNICODE clipboard format, this will automatically be translated to the system encoding. You can use the **-binary** option in combination with the result of **encoding convertto** to send data in any other encoding.

dde poke *?-binary? service topic item data*
dde poke passes the *data* to the server indicated by *service* using the *topic* and *item* specified. Typically, *service* is the name of an application. *topic* is application specific but can be a command to the server or the name of a file to work on. The *item* is also application specific and is often not used, but it must always be non-null. The *data* field is given to the remote application. Without the **-binary** option all data will be sent in unicode. For dde clients which don't implement the CF_UNICODE clipboard format, this will automatically be translated to the system encoding. You can use the **-binary** option in combination with the result of **encoding convertto** to send data in any other encoding.

dde request *?-binary? service topic item*

coreTools Command Reference Index

dde request is typically used to get the value of something; the value of a cell in Microsoft Excel or the text of a selection in Microsoft Word. *service* is typically the name of an application, *topic* is typically the name of the file, and *item* is application-specific. The command returns the value of *item* as defined in the application. Normally this is interpreted to be a string with terminating null. If **-binary** is specified, the result is returned as a byte array.

dde services *service topic*

dde services returns a list of service-topic pairs that currently exist on the machine. If *service* and *topic* are both empty strings ({}), then all service-topic pairs currently available on the system are returned. If *service* is empty and *topic* is not, then all services with the specified topic are returned. If *service* is non-empty and *topic* is, all topics for a given service are returned. If both are non-empty, if that service-topic pair currently exists, it is returned; otherwise, an empty string is returned.

dde eval *?-async? topic cmd ?arg arg ...?*

dde eval evaluates a command and its arguments using the interpreter specified by *topic*. The DDE service must be the **TclEval** service. The **-async** option requests asynchronous invocation. The command returns an error message if the script did not run, unless the **-async** flag was used, in which case the command returns immediately with no error. This command can be used to replace send on Windows.

DDE AND TCL

A Tcl interpreter always has a service name of **TclEval**. Each different interpreter of all running Tcl applications must be given a unique name specified by **dde servername**. Each interp is available as a DDE topic only if the **dde servername** command was used to set the name of the topic for each interp. So a **dde services TclEval {}** command will return a list of service-topic pairs, where each of the currently running interps will be a topic.

When Tcl processes a **dde execute** command, the data for the execute is run as a script in the interp named by the topic of the **dde execute** command.

When Tcl processes a **dde request** command, it returns the value of the variable given in the dde command in the context of the interp named by the dde topic. Tcl reserves the variable **\$TCLEVAL\$EXECUTES\$RESULT** for internal use, and **dde request** commands for that variable will give unpredictable results.

An external application which wishes to run a script in Tcl should have that script store its result in a variable, run the **dde execute** command, and then run **dde**

request to get the value of the variable.

When using DDE, be careful to ensure that the event queue is flushed using either **update** or **vwait**. This happens by default when using **wish** unless a blocking command is called (such as **exec** without adding the **&** to place the process in the background). If for any reason the event queue is not flushed, DDE commands may hang until the event queue is flushed. This can create a deadlock situation.

EXAMPLE

This asks Internet Explorer (which must already be running) to go to a particularly important website:

```
package require dde
dde execute -async iexplore WWW_OpenURL http://www.tcl.tk/
```

SEE ALSO

tk(n), winfo(n), send(n)

KEYWORDS

application, dde, name, remote execution

NAME

debug_script Debug a script using the TclDevKit Tcl debugger.

SYNTAX

```
string debug_script script [port] [hostname]
```

ARGUMENTS

<i>script</i>	This specifies the Tcl script(s) to be debugged. This script will be sourced and instrumented for debugging.
<i>port</i>	Specifies the port number that the debugger is listening on for a remote debugging application. By default the debugger will be launched on an available port.
<i>hostname</i>	Specifies the host where Prodebug is already running. This is used to connect to an existing remote Prodebug session.
<i>::env(SNPS_TCLPRO_HOME)</i>	This Tcl variable is used to specify the root of the TclPro installation. It is initialized from the environment variable SNPS_TCLPRO_HOME in the user's environment.

USING THE PACKAGE

The **debug_script** command is in the snpsTclPro Tcl package, and all of the commands in this package are defined in the namespace snpsTclPro. The way to use this command is to load the package, which will import the commands it exports into the global namespace.

coreTools Command Reference Index

This is shown below.

```
shell> package require snpsTclPro
1.0
```

These commands can be added to the .synopsys setup file for the system, user, or current directory, or the commands can be typed when the package is needed.

DESCRIPTION

The **debug_script** command simplifies the use of the TclDevKit Tcl debugger available from ActiveState (<http://www.activestate.com>). First it ensures a connection to the Prodebug debugger either by connecting to a running debug session (given a *hostname* and *port*), or by launching the debugger on *localhost* using an available port, and then connecting to it. If there is already an active connection to the debugger then that debugger will simply be used.

Once the connection to the debugger is set up, the specified script will be sourced and debugged. The default behavior for the debugger is to stop at the first line of the source'd script.

The debugger must be run using a "remote" debugging project if you want to connect the Synopsys tools to a running prodebug session. The port number in use by the debugger can be viewed via the Application tab of the "File->Project" Settings dialog.

EXAMPLES

```
# Launch the debugger and then debug the script myscript.tcl.
shell> debug_script myscript.tcl
Selected port 2576 on host localhost
launching prodebug
```

```
# now debug another script using the same session
shell> debug_script anotherScript.tcl
```

```
# debug myscript.tcl, connecting to the specified debugger session or
# creating one if the connection fails.
shell> debug_script myscript.tcl 2576 localhost
```

CAVEATS

The debugger currently kills the process that started the debugger when you exit, despite the preferences being set differently in the debugger.

When the Synopsys tool has spawned a debugger, that debugger must be exited before the Synopsys tool can exit. If this is not the case then the Synopsys tool will hang on exit, until the debugger is exited.

The **debug_script** command waits for the debug process to start up, before it tries to connect to the debugger. If **debug_script** times out before the debugger is started due to machine or network loading, simply close the debugger that was launched, and then increase the timeout setting with the command **set_debugger_start_timeout**, and then re-run the debugger. The **set_debugger_start_timeout** command takes a single argument which is the value of the timeout in milliseconds, and the default value for the timeout is 10000.

SEE ALSO

check_script(2), **package(2)**, **namespace(2)**.

NAME

`oo::define`, `oo::objdefine` - define and configure classes and objects

SYNOPSIS

```
package require TclOO
```

```
oo::define class defScript  
oo::define class subcommand arg ?arg ...?  
oo::objdefine object defScript  
oo::objdefine object subcommand arg ?arg ...?
```

DESCRIPTION

The **oo::define** command is used to control the configuration of classes, and the **oo::objdefine** command is used to control the configuration of objects (including classes as instance objects), with the configuration being applied to the entity named in the *class* or the *object* argument. Configuring a class also updates the configuration of all subclasses of the class and all objects that are instances of that class or which mix it in (as modified by any per-instance configuration). The way in which the configuration is done is controlled by either the *defScript* argument or by the *subcommand* and following *arg* arguments; when the second is present, it is exactly as if all the arguments from *subcommand* onwards are made into a list and that list is used as the *defScript* argument.

CONFIGURING CLASSES

The following commands are supported in the *defScript* for **oo::define**, each of which may also be used in the *subcommand* form:

coreTools Command Reference Index

constructor *argList bodyScript*

This creates or updates the constructor for a class. The formal arguments to the constructor (defined using the same format as for the Tcl **proc** command) will be *argList*, and the body of the constructor will be *bodyScript*. When the body of the constructor is evaluated, the current namespace of the constructor will be a namespace that is unique to the object being constructed. Within the constructor, the **next** command should be used to call the superclasses' constructors. If *bodyScript* is the empty string, the constructor will be deleted.

deletemethod *name ?name ...?*

This deletes each of the methods called *name* from a class. The methods must have previously existed in that class. Does not affect the superclasses of the class, nor does it affect the subclasses or instances of the class (except when they have a call chain through the class being modified) or the class object itself.

destructor *bodyScript*

This creates or updates the destructor for a class. Destructors take no arguments, and the body of the destructor will be *bodyScript*. The destructor is called when objects of the class are deleted, and when called will have the object's unique namespace as the current namespace. Destructors should use the **next** command to call the superclasses' destructors. Note that destructors are not called in all situations (e.g. if the interpreter is destroyed). If *bodyScript* is the empty string, the destructor will be deleted. Note that errors during the evaluation of a destructor *are not returned* to the code that causes the destruction of an object. Instead, they are passed to the currently-defined **bgerror** handler.

export *name ?name ...?*

This arranges for each of the named methods, *name*, to be exported (i.e. usable outside an instance through the instance object's command) by the class being defined. Note that the methods themselves may be actually defined by a superclass; subclass exports override superclass visibility, and may in turn be overridden by instances.

filter *?-slotOperation? ?methodName ...?*

This slot (see **SLOTTED DEFINITIONS** below) sets or updates the list of method names that are used to guard whether method call to instances of the class may be called and what the method's results are. Each *methodName* names a single filtering method (which may be exposed or not exposed); it is not an error for a non-existent method to be named since they may be defined by subclasses. By default, this slot works by appending.

forward *name cmdName ?arg ...?*

This creates or updates a forwarded method called *name*. The method is defined be forwarded to the command called *cmdName*, with additional arguments, *arg* etc., added before those arguments specified by the caller of

coreTools Command Reference Index

the method. The *cmdName* will always be resolved using the rules of the invoking objects' namespaces, i.e., when *cmdName* is not fully-qualified, the command will be searched for in each object's namespace, using the instances' namespace's path, or by looking in the global namespace. The method will be exported if *name* starts with a lower-case letter, and non-exported otherwise.

method *name argList bodyScript*

This creates or updates a method that is implemented as a procedure-like script. The name of the method is *name*, the formal arguments to the method (defined using the same format as for the Tcl **proc** command) will be *argList*, and the body of the method will be *bodyScript*. When the body of the method is evaluated, the current namespace of the method will be a namespace that is unique to the current object. The method will be exported if *name* starts with a lower-case letter, and non-exported otherwise; this behavior can be overridden via **export** and **unexport**.

mixin *?-slotOperation? ?className ...?*

This slot (see **SLOTTED DEFINITIONS** below) sets or updates the list of additional classes that are to be mixed into all the instances of the class being defined. Each *className* argument names a single class that is to be mixed in. By default, this slot works by replacement.

renamemethod *fromName toName*

This renames the method called *fromName* in a class to *toName*. The method must have previously existed in the class, and *toName* must not previously refer to a method in that class. Does not affect the superclasses of the class, nor does it affect the subclasses or instances of the class (except when they have a call chain through the class being modified), or the class object itself. Does not change the export status of the method; if it was exported before, it will be afterwards.

self *subcommand arg ...*

self *script*

This command is equivalent to calling **oo::objdefine** on the class being defined (see **CONFIGURING OBJECTS** below for a description of the supported values of *subcommand*). It follows the same general pattern of argument handling as the **oo::define** and **oo::objdefine** commands, and operates identically to

superclass *?-slotOperation? ?className ...?*

This slot (see **SLOTTED DEFINITIONS** below) allows the alteration of the superclasses of the class being defined. Each *className* argument names one class that is to be a superclass of the defined class. Note that objects must not be changed from being classes to being non-classes or vice-versa, that an empty parent class is equivalent to **oo::object**, and that the parent classes of **oo::object** and **oo::class** may not be modified. By default, this slot works by replacement.

coreTools Command Reference Index

unexport *name ?name ...?*

This arranges for each of the named methods, *name*, to be not exported (i.e. not usable outside the instance through the instance object's command, but instead just through the **my** command visible in each object's context) by the class being defined. Note that the methods themselves may be actually defined by a superclass; subclass unexports override superclass visibility, and may be overridden by instance unexports.

variable *?-slotOperation? ?name ...?*

This slot (see **SLOTTED DEFINITIONS** below) arranges for each of the named variables to be automatically made available in the methods, constructor and destructor declared by the class being defined. Each variable name must not have any namespace separators and must not look like an array access. All variables will be actually present in the instance object on which the method is executed. Note that the variable lists declared by a superclass or subclass are completely disjoint, as are variable lists declared by instances; the list of variable names is just for methods (and constructors and destructors) declared by this class. By default, this slot works by appending.

CONFIGURING OBJECTS

The following commands are supported in the *defScript* for **oo::objdefine**, each of which may also be used in the *subcommand* form:

class *className*

This allows the class of an object to be changed after creation. Note that the class's constructors are not called when this is done, and so the object may well be in an inconsistent state unless additional configuration work is done.

deletemethod *name ?name ...*

This deletes each of the methods called *name* from an object. The methods must have previously existed in that object (e.g., because it was created through **oo::objdefine method**). Does not affect the classes that the object is an instance of, or remove the exposure of those class-provided methods in the instance of that class.

export *name ?name ...?*

This arranges for each of the named methods, *name*, to be exported (i.e. usable outside the object through the object's command) by the object being defined. Note that the methods themselves may be actually defined by a class or superclass; object exports override class visibility.

filter *?-slotOperation? ?methodName ...?*

This slot (see **SLOTTED DEFINITIONS** below) sets or updates the list of method names that are used to guard whether a method call to the object may be called and what the method's results are. Each *methodName* names a single filtering method (which may be exposed or not

coreTools Command Reference Index

exposed); it is not an error for a non-existent method to be named. Note that the actual list of filters also depends on the filters set upon any classes that the object is an instance of. By default, this slot works by appending.

forward *name cmdName ?arg ...?*

This creates or updates a forwarded object method called *name*. The method is defined be forwarded to the command called *cmdName*, with additional arguments, *arg* etc., added before those arguments specified by the caller of the method. Forwarded methods should be deleted using the **method** subcommand. The method will be exported if *name* starts with a lower-case letter, and non-exported otherwise.

method *name argList bodyScript*

This creates, updates or deletes an object method. The name of the method is *name*, the formal arguments to the method (defined using the same format as for the Tcl **proc** command) will be *argList*, and the body of the method will be *bodyScript*. When the body of the method is evaluated, the current namespace of the method will be a namespace that is unique to the object. The method will be exported if *name* starts with a lower-case letter, and non-exported otherwise.

mixin *?-slotOperation? ?className ...?*

This slot (see **SLOTTED DEFINITIONS** below) sets or updates a per-object list of additional classes that are to be mixed into the object. Each argument, *className*, names a single class that is to be mixed in. By default, this slot works by replacement.

renamemethod *fromName toName*

This renames the method called *fromName* in an object to *toName*. The method must have previously existed in the object, and *toName* must not previously refer to a method in that object. Does not affect the classes that the object is an instance of and cannot rename in an instance object the methods provided by those classes (though a **oo::objdefine** **forwarded** method may provide an equivalent capability). Does not change the export status of the method; if it was exported before, it will be afterwards.

unexport *name ?name ...?*

This arranges for each of the named methods, *name*, to be not exported (i.e. not usable outside the object through the object's command, but instead just through the **my** command visible in the object's context) by the object being defined. Note that the methods themselves may be actually defined by a class; instance unexports override class visibility.

variable *?-slotOperation? ?name ...?*

This slot (see **SLOTTED DEFINITIONS** below) arranges for each of the named variables to be automatically made available in the methods declared by the object being defined. Each variable name must not have any namespace separators and must not look like an array access. All variables will be actually present in the

object on which the method is executed. Note that the variable lists declared by the classes and mixins of which the object is an instance are completely disjoint; the list of variable names is just for methods declared by this object. By default, this slot works by appending.

SLOTTED DEFINITIONS |

Some of the configurable definitions of a class or object are *slotted definitions*. This means that the configuration is implemented by a slot object, that is an instance of the class **oo::Slot**, which manages a list of values (class names, variable names, etc.) that comprises the contents of the slot. The class defines three operations (as methods) that may be done on the slot:

slot -append ?member ...?

This appends the given *member* elements to the slot definition.

slot -clear

This sets the slot definition to the empty list.

slot -set ?member ...?

This replaces the slot definition with the given *member* elements.

A consequence of this is that any use of a slot's default operation where the first member argument begins with a hyphen will be an error. One of the above operations should be used explicitly in those circumstances.

SLOT IMPLEMENTATION

Internally, slot objects also define a method **--default-operation** which is forwarded to the default operation of the slot (thus, for the class slot, this is forwarded to and these methods which provide the implementation interface:

slot Get

Returns a list that is the current contents of the slot. This method must always be called from a stack frame created by a call to **oo::define** or **oo::objdefine**.

slot Set elementList

Sets the contents of the slot to the list *elementList* and returns the empty string. This method must always be called from a stack frame created by a call to **oo::define** or **oo::objdefine**.

The implementation of these methods is slot-dependent (and responsible for accessing the correct part of the class or object definition). Slots also have an unknown method handler to tie all these pieces together, and they hide their **destroy** method so that it is not invoked inadvertently. It is *recommended* that any user

changes to the slot mechanism be restricted to defining |
new operations whose names start with a hyphen.

EXAMPLES

This example demonstrates how to use both forms of the **oo::define** and **oo::objdefine** commands (they work in the same way), as well as illustrating four of the subcommands of them.

```
oo::class create c
c create o
oo::define c method foo {} {
    puts "world"
}
oo::objdefine o {
    method bar {} {
        my Foo "hello "
        my foo
    }
    forward Foo ::puts -nonewline
    unexport foo
}
o bar                                -> prints "hello world"
o foo                                -> error "unknown method foo"
o Foo Bar                            -> error "unknown method Foo"
oo::objdefine o renamemethod bar lollipop
o lollipop                            -> prints "hello world"
```

This example shows how additional classes can be mixed into an object. It also shows how **mixin** is a slot that supports appending:

```
oo::object create inst
inst m1                                -> error "unknown method m1"
inst m2                                -> error "unknown method m2"

oo::class create A {
    method m1 {} {
        puts "red brick"
    }
}
oo::objdefine inst {
    mixin A
}
inst m1                                -> prints "red brick"
inst m2                                -> error "unknown method m2"

oo::class create B {
    method m2 {} {
        puts "blue brick"
    }
}
oo::objdefine inst {
    mixin -append B
}
inst m1                                -> prints "red brick"
inst m2                                -> prints "blue brick"
```

SEE ALSO

`next(n)`, `oo::class(n)`, `oo::object(n)`

KEYWORDS

`class`, `definition`, `method`, `object`, `slot`

define_activity_subproc_params

Define sub-process launch/result-viewing parameters for an activity or filegroup

Syntax

```
string define_activity_subproc_params -workdir workdir -resultsfile
resultsfile [-resultstatusfile resultstatusfile] [-mailfile mailfile]
[-scriptfile scriptfile] [-hideparams] [-parallelJobs] activity
    string workdir
    string resultsfile
    string resultstatusfile
    string mailfile
    string scriptfile
    string activity
```

Parameters

-workdir <i>workdir</i>	Directory for generated scripts, guard files and command execution.
-resultsfile <i>resultsfile</i>	Location of results file, relative to working directory.
-resultstatusfile <i>resultstatusfile</i>	A file with results information to display in the activity summary report.
-mailfile <i>mailfile</i>	Mail file to append to the generic command complete results email.
-scriptfile <i>scriptfile</i>	Name of top-level script file associated with the sub-process.
-hideparams	Don't show these parameters when configuring the activity or filegroup This should only be used for sub-processes which run quickly and do not require background run support or result file viewing.
-parallelJobs	Activity can use parallel cpus.
<i>activity</i>	Activity or filegroup associated with the command

Description

Adds the parameters required to launch a sub-process for an activity or filegroup. Parameters added include:

coreTools Command Reference Index

- **RunStyle** - If ScriptsOnly is 0, then this selects how to run the command. The command can be run locally, through a load-sharing interface, or via a remote shell. Valid values are "local", "lsf", "grd", and "remote".
- **RemoteHost** - If RunStyle is remote, then this is the name of the machine to run the command on.
- **RunOptions** - If the RunStyle is grd or lsf, this parameter has any special options to pass to the corresponding job launch command.
- **ResultsFile** - Set this parameter to the location of the results file for the external command. The path should be relative to the WorkDirectory. The preferred format of this file is HTML. This parameter is initialized with the value specified with the -resultsfile option.
- **ScriptsOnly** - Setting this causes the generated script (run.scr by default), to be generated in the WorkDirectory but the script does not get executed. Users can then run the script at a later time.
- **EmailAddress** - If this parameter is set to a value other than "", then email is sent when the command script completes or is killed. The default value is the current users name.
- **MailFile** - If this parameter is set then the contents of the file are appended to the generic mail message that is sent when a command script completes or is killed. Mail is only sent if the parameter EmailAddress is also set. This parameter is initialized with the value specified with the -mailfile option.
- **WorkDirectory** - Set this parameter to the directory where the scripts and guard files go. The command will also run in this directory when executed using the launch_activity_subproc command. The value for this parameter should be a relative path name from the user's workspace. This parameter is initialized with the value specified with the -workdir option.

Defining these parameters enables a plugin writer to launch a command string (the sub-process) for an activity using launch_activity_subproc. The user can set the values of the above parameters and then the tool automatically handles launching using the proper run style. The result file is also automatically made available via a button associated with the activity in the activity checklist dialog. If the user has selected to send email then email will be sent when the command completes or is killed.

This command is also used to launch testsuite regression environment runs defined as TREs in coreBuilder.

Examples

The following is a sample TCL script that could be used as part of a plugin defining a custom activity or as part of a TCL intent file associated with a TRE testsuite. In the example, it is assumed that an activity named customAct has already been created using the create_custom_activity command.

```
# Define parameters associated with the sub-process
define_activity_subproc_params customAct -workdir ./sim -resultsfile
regress.log
# Don't let user change result file name
set parameter [get_activity_parameter customAct ResultsFile -param]
set_parameter_attribute $parameter Visible false
```

See Also

[create_custom_activity \(2\)](#), [launch_activity_subproc \(2\)](#),
[report_activity_subproc \(2\)](#), [wait_for_activity_subproc \(2\)](#),
[prereq_activities_complete \(2\)](#)

`define_array_field_parameters`
define parameters for each field of an array.

Syntax

```
string define_array_field_parameters [-resize] [-values ] param
    list
    string param
```

Parameters

<code>-resize</code>	Resize Parameter
<code>-values</code>	Resize Parameter values
<code>param</code>	Parameter to define array parameters for

Description

The coreTools support array parameters which are used to set the values of fields within a bitfield parameter. `define_array_field_parameters` is used to define each entry for an array parameter based on the following attributes of the array parameter: `IsArray`, `ArrayStart`, `ArrayEnd`, `ArrayFieldSize`. It creates field parameters and set the attributes based on the array parameter from which field parameters are defined. The field parameters can be accessed by `get_field_parameter_for_array` and `foreach_array_field_parameter`.

`define_array_field_parameters` and the access commands are mostly useful in Tcl intent file for simplify the definition of the array parameters and their linking to the underlying array in the HDL.

Examples

The following commands define three field parameters based on the array parameter AP, the value of the three field parameters will be 0xaaaa, 0xbbbb, and 0xcccc, respectively:

```
coreBuilder> create_parameter -type bitfield -package pac1 AP
coreBuilder> set_attribute AP -attr IsArray -value 1
coreBuilder> set_attribute AP -attr ArrayStart -value 1
coreBuilder> set_attribute AP -attr ArrayEnd -value 3
coreBuilder> set_attribute AP -attr ArrayFieldSize -value 16
coreBuilder> set_attribute AP -attr IsHexVal -value 1
coreBuilder> set_attribute AP -attr Value -value 0xaaaabbbbcccc
coreBuilder> define_array_field_parameters AP
```

coreTools Command Reference Index

```
coreBuilder> get_attribute [get_field_parameter_for_array AP 1] -attr  
Value  
0xaaaa  
coreBuilder> get_attribute [get_field_parameter_for_array AP 2] -attr  
Value  
0xbbbb  
coreBuilder> get_attribute [get_field_parameter_for_array AP 3] -attr  
Value  
0xcccc
```

See Also

get_field_parameter_for_array (2), foreach_array_field_parameter (2),
IsArray (3), ArrayStart (3), ArrayEnd (3), ArrayFieldSize (3)

define_lib_clocks

Extract clock definitions from a hard macro library file

Syntax

```
string define_lib_clocks -instance <name> library
    string <name>
    string library
```

Parameters

-instance <name>	Instance path of clock definition
library	Library (.db) file name

Description

This command is used to create local definitions for clocks defined in a hard macro library. This is required when SDC constraints for a component reference these clocks. In order for the references to be valid within coreAssembler, the clocks must be defined. This cannot be done automatically because coreAssembler doesn't always know where the hard macro is instantiated within a component.

When invoked, this command extracts generated clock definitions from the indicated hard macro library file and creates clocks whose names are built from the specified instance path and library clock name.

Examples

The current design contains a hard macro at phy0/pma. The macro is defined in dwc_test_lib.db:

```
define_lib_clocks -instance phy0/pma dwc_test_lib.db
```

See Also

[DefineLibClocks \(3\)](#)

NAME

define_proc_attributes

Defines attributes of a Tcl procedure, including an information string for help, a command group, a set of argument descriptions for help, and so on. The command returns the empty string.

SYNTAX

```
string define_proc_attributes
    proc_name
    [-info info_text]
    [-define_args arg_defs]
    [-define_arg_groups group_defs]
    [-command_group group_name]
    [-return type_name]
    [-hide_body]
    [-hidden]
    [-dont_abbrev]
    [-permanent]
```

Data Types

<i>proc_name</i>	string
<i>info_text</i>	string
<i>arg_defs</i>	list
<i>group_defs</i>	list
<i>group_name</i>	string
<i>type_name</i>	string

ARGUMENTS

proc_name Specifies the name of the existing procedure.

-info *info_text*
Provides a help string for the procedure. This is printed by the **help** command when you request help for the procedure. If you do not specify *info_text*, the default is "Procedure".

-define_args *arg_defs*

coreTools Command Reference Index

Defines each possible procedure argument for use with **help -verbose**. This is a list of lists where each list element defines one argument.

- define_arg_groups *group_defs*
Defines argument checking groups. These groups are checked for you in `parse_proc_arguments`. each list element defines one group
- command_group *group_name*
Defines the command group for the procedure. By default, procedures are placed in the "Procedures" command group.
- return *type_name*
Specifies the type of value returned by this proc. Any value may be specified. Some applications use this information for automatically generated dialogs etc. Please see the `type_name` option attribute described below.
- hide_body
Hides the body of the procedure from **info body**.
- hidden
Hides the procedure from **help** and **info proc**.
- dont_abbrev
Specifies that the procedure can never be abbreviated. By default, procedures can be abbreviated, subject to the value of the **sh_command_abbrev_mode** variable.
- permanent
Defines the procedure as permanent. You cannot modify permanent procedures in any way, so use this option carefully.
- deprecated
Defines the procedure as deprecated i.e. it should no longer be called but is still available.
- obsolete
Defines the procedure as obsolete i.e. it used to exist but can no longer be called.

DESCRIPTION

The **define_proc_attributes** command associates attributes with a Tcl procedure. These attributes are used to define help for the procedure, locate it in a particular command group, and protect it.

When a procedure is created with the **proc** command, it is placed in the Procedures command group. There is no help text for its arguments. You can view the body of the procedure with **info body**, and you can modify the procedure and its attributes. The **define_proc_attributes** command allows you to change these aspects of a procedure.

Note that the arguments to Tcl procedures are all named, positional arguments. They can be programmed with default values, and there can be optional arguments by using the special argument name *args*. The **define_proc_attributes** command does not relate the information that you enter for argument definitions with **-define_args** to the actual argument names. If you are describing anything other than positional arguments, it is expected that you are also using **parse_proc_arguments** to validate and extract your arguments.

The *info_text* is displayed when you use the **help** command on the procedure.

Use **-define_args** to define help text and constraints for individual arguments. This makes the help text for the procedure look like the help for an application command. The value for **-define_args** is a list of lists. Each element has the following format:

```
arg_name option_help value_help data_type attributes
```

The elements specify the following information:

- o *arg_name* is the name of the argument.
- o *option_help* is a short description of the argument.
- o *value_help* is the argument name for positional arguments, or a one word description for dash options. It has no meaning for a Boolean option.
- o *data_type* is optional and is used for option validation. The *data_type* can be any of: string, list, boolean, int, float, or one_of_string. The default is string.
- o *attributes* is optional and is used for option validation. The *attributes* is a list that can have any of the following entries:
 - o "required" - This argument must be specified. This attribute is mutually exclusive with optional.
 - o "optional" - Specifying this argument is optional.

coreTools Command Reference Index

This attribute is mutually exclusive with "required."

- o "value_help" - Indicates that the valid values for a one_of_string argument should be listed whenever argument help is shown.
- o "values {<list of allowable values>}" - If the argument type is one_of_string, you must specify the "values" attribute.
- o "type_name <name>" - Give a descriptive name to the type that this argument supports. Some applications may use this information to provide features for automatically generated dialogs, etc. Please see product documentation for details. This attribute is not supported on boolean options.
- o "merge_duplicates" - When this option appears more than once in a command, its values are concatenated into a list of values. The default behavior is that the right-most value for the option specified is used.
- o "remainder" - Specifies that any additional positional arguments should be returned in this option. This option is only valid for string option types, and by default the option is optional. You can require at least one item to be specified by also including the required option.
- o "deprecated" - Specifying this option is deprecated i.e. it should no longer be used but is still available. A warning will be output if this option is specified. This attribute cannot be combined with obsolete or required.
- o "obsolete" - Specifying this option is obsolete i.e. it used to exist but can no longer be used. A warning will be output if this option is specified. This attribute cannot be combined with deprecated or required.
- o "min_value" <value> - Specify the minimum value for this option. This attribute is only valid for integer and float types.
- o "max_value" <value> - Specify the maximum value for this option. This attribute is only valid for integer and float types.
- o "default" <value> - Specify the default value for this option. This attribute is only valid for string, integer and float option types. If the user does not specify this option when invoking the command this default value will be automatically passed to the associated tcl procedure.

The default for *attributes* is "required."

Use the **-define_arg_groups** to define argument checking groups. The format of this option is a list where each

coreTools Command Reference Index

element in the list defines an option group. Each element has the following format:

```
{<type> {<opt1> <opt2> ...} [<attributes>]}
```

The types of groups are

- o *"exclusive"* Only one option in an exclusive group is allowed. All the options in the group must have the same required/optional status. This group can contain any number of options.
- o *"together"* If the first option in the group is specified then the second argument is also required. This type of group can contain at most two options. The second option may be specified as "{optName optVal}" to indicate that the first option requires a specific value for the second option.
- o *"related"* These options are related to each other. An automatic dialog builder for this command may try to group these options together.

The supported attributes are:

- o *{"label" <text>}* An optional label text to identify this group. The label may be used by an application to automatically build a grouping in a generated dialog.
- o *"bidirectional"* This is only valid for a together group. It means that both options must be specified together.

Change the command group of the procedure using the **-command_group** command. Protect the contents of the procedure from being viewed by using **-hide_body**. Prevent further modifications to the procedure by using **-permanent**. Prevent abbreviation of the procedure by using **-dont_abbrev**.

EXAMPLES

The following procedure adds two numbers together and returns the sum. For demonstration purposes, unused arguments are defined.

```
prompt> proc plus {a b} { return [expr $a + $b]}

prompt> define_proc_attributes plus -info "Add two numbers" \
? -define_args {
  {a "first addend" a string required}
  {b "second addend" b string required}
  {"-verbose" "issue a message" "" boolean optional}}
```


coreTools Command Reference Index

```
prompt> help -verbose plus
Usage: plus      # Add two numbers
    [-verbose]      (issue a message)
    a                (first addend)
    b                (second addend)
```

```
prompt> plus 5 6
11
```

In the following example, the `argHandler` procedure accepts an optional argument of each type supported by **define_proc_attributes**, then displays the options and values received. Note the only one of `-Int`, `-Float`, or `-Bool` may be specified to the command. Additionally `-IDup` is only allowed when `-Int` is specified:

```
proc argHandler {args} {
    parse_proc_arguments -args $args results
    foreach argname [array names results] {
        echo $argname = $results($argname)
    }
}

define_proc_attributes argHandler \
    -info "Arguments processor" \
    -define_args {
        {-Oos "oos help"      AnOos  one_of_string
         {required value_help {values {a b}}}}

        {-Int "int help"      AnInt   int      optional}
        {-Float "float help"  AFloat  float    optional}
        {-Bool "bool help"    ""      boolean optional}
        {-String "string help" AString string optional}
        {-List "list help"    AList   list     optional}
        {-IDup "int dup help" AIDup   int      {optional merge_duplicates}}
    } \
    -define_arg_groups {
        {exclusive {-Int -Float -Bool}}
        {together  {-IDup -Int}}
    }
```

SEE ALSO

```
help(2)
info(2)
parse_proc_arguments(2)
proc(2)
sh_command_abbrev_mode(3)
```


define_report

Support creation and viewing of user defined reports

Syntax

string **define_report** -reportName -reportCallback *<proc name>* -activity
<act> -description *<description>* [-cleanupCallback] [-launchExternally]
[-externalReportPath *<report path>*] [-completeActivity] [-sequence]

string
string *<proc name>*
string *<act>*
string *<description>*
string *<report path>*

Parameters

-reportName	Name of the report. User must generate a report with this <i><name>.html</i> via reportCallback. The <i><name>.html</i> must reside in reports folder
-reportCallback <i><proc name></i>	Callback to be called for user report generation
-activity <i><act></i>	Name of the activity for which the report is defined
-description <i><description></i>	Description of the report
-cleanupCallback	Define a cleanup callback proc
-launchExternally	Enables the report in external Browser
-externalReportPath <i><report path></i>	Path to html report relative to report folder
-completeActivity	Generate report only if activity is complete (Default False)
-sequence	Sequence number to override the order in which the report is rendered. Default sequence number is 10000.

Description

The define_report command supports generation of external reports. This supports reports in coreAssembler as well as coreConsultant flows.

The define_report command needs to be called in Plugin folder in *Obj.tcl file . The callback procs can be implemented in *.tcl file

NOTE : The define_report command should NOT be specified under any workspace hooks (add_workspace_hook) or any activity hooks

Examples

For example the following report is adding in workspace/plugin/genReport_Obj.tcl and the callback procs are called in workspace/plugin/genReport.tcl

```
coreAssembler> define_report -reportName {DiskUsage} \
-activity {SpecifyConfiguration SpecifySubsystem} \
-reportCallback {genDiskSpaceReport DiskUsage} \
-launchExternally \
-cleanupCallback {diskSpaceCleanup} \
-externalReportPath {csd} \
-description "Checks disk usage"
```

In the command above

-reportName	: Specify the name of the report
-activity	: The report gets displayed in SpecifyConfiguration (coreConsultant A
-reportCallback	: Specifies proc genDiskSpaceReport with report name argument DiskUsa
-launchExternally	: Will launch report in external browser
-cleanupCallback	: If any explicit cleanup is required then a cleanupcallback proc must be specified
-externalReportPath	: All external report paths must be specified inside the component/ By default the reports are available in report folder
-description	: Description for the Report

The reports get displayed in cC or cA by clicking on reports page in the link.
To cleanup reports , specify generate_reports -show -cleanup -report {Report Name}

See Also

define_solution

Define a DesignWare IP Solution

Syntax

```
string define_solution -name <name> -version <name> -info <description>
-ip <text> [-env <text>] [-design <design>] [-docs <doc-details>] [-vars
<vars-and-values>] [-syscfg <sys-cfg-list>] [-hide <activity-names>]
[-show <activity-names>] [-internal] [-constraint_file <file>]
[-constraint_mode <model>] [-relocatable] [-as_hier] [-move <value>]
    string <name>
    string <description>
    string <text>
    string <design>
    string <doc-details>
    string <vars-and-values>
    string <sys-cfg-list>
    string <activity-names>
    string <file>
    string <model>
    string <value>
```

Parameters

-name <name>	Unique name for the solution
-version <name>	Version of the solution
-info <description>	Description of the solution
-ip <text>	Required IP information
-env <text>	Required environment variables and labels
-design <design>	Default design name
-docs <doc-details>	Documentation details
-vars <vars-and-values>	Variable definitions
-syscfg <sys-cfg-list>	System configuration definition
-hide <activity-names>	Activities to be hidden
-show <activity-names>	Activities to be shown
-internal	This is an internal solution - not for direct customer instantiation.
-constraint_file <file>	Top-level or external constraint file

-constraint_mode <model>	Constraint modeling approach (Values: standard, top-level, external)
-relocatable	Solution can be instantiated in any hierarchy
-as_hier	[OBSOLETE] Create solution as standard hierarchical component
-move <value>	When to move clock source pin references: never, before or after compile (Values: never, immediately, before, after)

Description

This command is used to define a DesignWare IP Solution product. Each of the options provides information about the product in terms of content such that the solution can be created (instantiated) by the `instantiate_solution` command. This command is not invoked directly. Instead it is meant to reside within a solution root directory within a file named `solution.tcl`. See the DW IP Solution documentation for details.

The mandatory options (`-name`, `-version`, `-info`) define the name, version, and description of the solution. The mandatory `-ip` option defines the list of components instantiated within the solution. The value associated with this option is a list of lists. Each top-level list defines an IP to be instantiated. The sub-list for an individual IP can have either 2 or 3 entries. The first two entries indicate the IP name and version, while the optional 3 entry defines the location of the IP installation. The IP version can be a single fixed version, a comma separated list of versions, or a version with a '+' character append, implying that any newer version is also valid. The IP location can be a fixed path, or a relative path within the solution directory. Paths can contain variable or environment variable references which are resolved at the time of the call to `instantiate_solution`. The `<solution>/installations` directory is automatically added to the search path so information about IP installation areas within the `-ip` argument is only needed when the IP is not expected to be installed within the same `DESIGNWARE_HOME` that the solution is installed in, or within the `installations` directory of the solution.

The `-env` option specifies environment variables which must be set in order to instantiate the solution. The value passed should be a list of alternating environment variable names and descriptions. The environment variables can be referenced within scripts and within the `-ip` specification as desired.

The `-design` option specifies the default design name for the top-level solution hierarchy created in `coreAssembler`.

The `-docs` options specifies documentation to be linked into the `coreAssembler` help menu. The value passed should be a list of alternate

file references and labels. The file references should be paths relative to the solution root directory. The label defines the text to put in the menu to define the link to access the document.

The `-vars` option specified TCL variables that can be referenced within solution script files. This allows the files to be shared over multiple solution products. The value passed should be a list of alternating TCL variable names and values. The variables can then be referenced using the `get_solution_variable` command.

The `-syscfg` option defines which variables defined via `-vars` or `-env` can be overridden by the user as a form of "system configuration". System configuration values can then be passed on the `instantiate_solution` command line or if the GUI is active a configuration dialog will appear for value specification. The value passed to `-syscfg` should be a TCL list of lists where each sublist has the format: `var-name var-type attr1 value1 attr2 value2 ...` where `var-name` matches a variable defined via `-vars` or `-env`, `var-type` indicates the variable type and is one of (integer, bitfield, float, string, boolean), and the optional `attrN/valueN` values define attribute value pairs that control what the user configuration dialog will look like. The format of each of these lists is as defined for the `show_user_parameter_dialog` command. See that man page for additional details. Note that the `DefaultValue` attribute should not be specified. The default value will be taken from the value passed to `-vars` or from the current setting of the environment variable if the variable is defined via `-env`.

The `-hide` option defines a list of activities that should be hidden upon instantiating the solution. Use of this option is discouraged as it makes the use model inconsistent between solution products.

The `-constraint_file` and `-constraint_mode` options are used when the solution is going to support synthesis constraints in a non-standard flow. In the standard flow these options are not used and constraints are propagated upward from the leaf IPs. When `-constraint_mode` is specified as anything other than "standard", the constraints are assumed to come from the file specified via the `-constraint_file` option. In 'top-level' mode, the specified constraints are applied to the coreTools data model and then constraints are written from that model as they would be in the standard flow (but in this case without any propagation since the constraints apply directly to the top-level). In 'external' mode, the constraints are not applied to the coreTools data model. Instead they are simply read directly into Design Compiler without modification.

The `-relocatable` option indicates that it is okay to instantiate this solution at a location within a coreAssembler workspace other than the top-level.

If the `-move` option is specified, it indicates that constraints in one or more of the components are defined on hierarchical pins but need to be

moved to leaf-level output pins once they have been defined. Legal values for this option are 'immediately', meaning move at the time the constraint is define, 'before', meaning before compile, and 'after', meaning after compile. If '-immediately' is used, the solution must define a postelab auxiliary script to pre-compile modules containing generated clock pins. If -before is used, the solution should define a constrain auxiliary synthesis script to pre-compile modules containing generated clock pins. The approach of using -immediately and a postelab auxiliary script are preferred as use of -before and -after will result in the loss of constraint information for moved clocks (because they must be removed and redefined).

The -internal option indicates a solution defined for internal usage. Internal solutions are not exposed via report_solutions nor are they visible in the coreAssembler home page for instantiation by clicking a link in that page. Solutions would typically be made internal to prevent them from being seen by customers or when they are meant to be used within another solution instead of being directly instantiated. Solution products delivered within other solutions (via the 'installations' directory) are automatically marked as internal.

Examples

A minimal solution with a single IP installed in a standard location:

```
define_solution -name "My Solution" -version "1.0" -ip {{DWC_test 1.4a}}
```

A solution requiring set-up of PHY_PATH environment variable, some variables, and a locally installed component:

```
define_solution -name "Another Solution" \
-version 1.1 \
-vars {NLANES 4 NPORTS 2} \
-env {PHY_PATH "Location of PHY installation"} \
-ip {{DWC_test1 1.60a+} {DWC_test2 1.70a ./local_install}}
```

A solution defined as in the previous example but with the ability to override the number of lanes, and to override the PHY_PATH environment variable as system configuration options:

```
define_solution -name "Another Solution" \
-version 1.1 \
-vars {NLANES 4 NPORTS 2} \
-env {PHY_PATH "Location of PHY installation"} \
-syscfg {
{NLANES integer Label {Number of lanes} MinValue 1}
{PHY_PATH string Label {PHY Installation Path}}
} \
-ip {{DWC_test1 1.60a+} {DWC_test2 1.70a ./local_install}}
```


See Also

[instantiate_solution \(2\)](#) [show_user_parameter_dialog \(2\)](#)

`define_split_interface`

Group the interfaces belonging to two different components.

Syntax

string **define_split_interface** [-ungroup] *interfaces*
string *interfaces*

Parameters

<code>-ungroup</code>	Ungroup the interfaces
<code>interfaces</code>	Interfaces to be grouped.

Description

The command defines an interface instance that is split into a group of interface instances of the same interface definition. The first interface in the collection is treated as the master and the remaining as the slaves. The master interface instance is used in commands for connecting and exporting to the entire split interface. While defining the split interface the following statement must be true for the interfaces.

- All interfaces must be in the same context
- All interfaces must have the same VLNV
- All interfaces must have the same value of InterfaceType
- There cannot be any overlap in logical ports across all of the instances (no interface port appears on more than one of the instances)
- No interface already has SplitInterfaceName set to true unless -ungroup is specified
- No interface already has SplitInterfaceMembers set to true unless -ungroup is specified
- All interfaces have IsConnected==0
- If -unlink is specified, the first interface must have SplitInterfaceMembers set and the list of interfaces must be exactly 1 in length

Some restrictions which apply on grouped interfaces are

- Connecting to an interface with SplitInterfaceName set is illegal.
- Exporting an interface with SplitInterfaceName set is illegal.
- Interfaces with SplitInterfaceName set should not be flagged as being unconnected.

Examples

To group the interface C1 on component A and C2 on component B. Interface C1 is the Mater Interface in the group.

```
set intf1 [find_interface_instance -component A -name C1]
set intf2 [find_interface_instance -component B -name C2]
set intf [add_to_collection $intf1 $intf2]
define_split_interface $intf
To ungroup interfaces C1 and C2.
```

```
set intf [find_interface_instance -component A -name C1]
define_split_interface -ungroup $intf
```

See Also

[SplitInterfaceMembers \(3\)](#) [SplitInterfaceName \(3\)](#)

`define_tech_cell_mapping`

Define mapping requirements to a technology specific cell.

Syntax

```
string define_tech_cell_mapping [-module <RTL module name>] -macro <RTL
macro name> [-description <text>] [-instance <instance name>] [-width
<width text>] [-port_names <port names>] [-port_descriptions <port
descriptions>] [-undef] [-file <RTL file name>] [-filter <filter>]
[-enable_macro_args] [-extra_macro_args <extra argument list>]
[-output_inversion_suffix <output inversion suffix>] [-required]
    string <RTL module name>
    string <RTL macro name>
    string <text>
    string <instance name>
    string <width text>
    string <port names>
    string <port descriptions>
    string <RTL file name>
    string <filter>
    list <extra argument list>
    string <output inversion suffix>
```

Parameters

-module <RTL module name>	Wrapper module containing the technology cell
-macro <RTL macro name>	Macro defining the technology cell mapping
-description <text>	Description of required technology specific cell
-instance <instance name>	Instance name for the technology cell being defined
-width <width text>	Width specification for arrayed cells
-port_names <port names>	List of port names from the technology cell wrapper
-port_descriptions <port descriptions>	List of port descriptions from the technology cell wrapper
-undef	Undefine the indicated technology cell mapping
-file <RTL file name>	Leaf name of the file where macros need to be inserted
-filter <filter>	The filter to use for limiting what you see when binding technology cells in the

	GUI
-enable_macro_args	Write out the macro with arguments (instance name,extra_macro_args,port names)
-extra_macro_args <extra argument list>	When macro arguments are enabled, these appear in the argument list
-output_inversion_suffix <output inversion suffix>	Output pin inversion will be enabled and the Tech Cell Bind macro name and output pin name will be suffixed with a custom string 'output_inversion_suffix'
-required	Specifies that the binding must be provided for the associated macro and is mandatory.

Description

This command is used to create the definition of a potential mapping to a target technology cell. The `bind_tech_cell_mapping` command is then used to bind to a specific cell as shown in the example section below. The definition includes the module containing the tech cell instance, the macro to be used to define the tech cell code, the instance name, instance width, and wires to be connected to the technology specific cell. Description information is also provided via the `-description` and `-port_description` options to guide a user when creating the specific binding. The `-file` option can be used if the macro definitions need to be written out into the component's source files instead of the generated <component name>-techmap.svh or vh files. The option accepts the leaf name of the file (E.g. compA.v). If the specified file (in the <component>/src dir) is encrypted, write protected, a symbolic link, or does not exist in the hdl file list, the macro definitions will be not be written into the techmap file. The `-output_inversion_suffix` option can be used to invert the outputPort. If the option `-output_inversion_suffix` is provided, then the output port is enabled for invert connection and macro and outPut port name is suffixed with the given `output_inversion_suffix` string. If the `output_inversion_suffix` is not provided, inverting outputPort is not supported.

```
coreBuilder> define_tech_cell_mapping -module BCM00 \
-macro BCM00_map \
-instance I000 \
-description "Sample tech map" \
-port_names {m1 m2 m3} \
-port_descriptions {{m1 port} {m2 port} {m3 port}} \
-enable_macro_args \
-extra_macro_args "a1 a2 a3"
-file "master.v" \
-width "[WIDTH-1:0]" \
-filter "Name=~ADDF"
-output_inversion_suffix "INVERTED_OUT"
```

coreTools Command Reference Index

-required

In older coreTools versions (V-2024.06-2 or older),

-output_inversion_suffix and -required are not supported.

In older coreTools versions (R2020.12-SP2 or older), -enable_macro_args and -extra_macro_args are not supported.

The define command is stored in a subscripted attribute, TechCellMappingDefn, as shown below:

```
coreBuilder> get_subscripts [current_design -quiet] -attr
```

```
TechCellMappingDefn
```

```
BCM00-BCM00_map
```

In older coreTools versions (R2020.12-SP2 or older), the format of the subscript is as shown:

```
BCM00-BCM00_map-compA
```

where the macro fields are:

- BCM00 - tech cell module
- BCM00_map - tech cell macro
- compA - design name

```
coreBuilder> get_attribute [current_design -quiet] -attr
```

```
TechCellMappingDefn -subscript BCM00-BCM00_map-master
```

```
{Sample tech map} I001 {[width-1:0]} {m1 m2 m3} {{m1 port} {m2 port} {m3  
port}} compA.v Name=~ADDF 1 {a1 a2 a3} INVERTED_OUT 1
```

for older coreTools versions (V-2024.06-2 or older),

-output_inversion_suffix and -required are not supported.

```
coreBuilder> get_attribute [current_design -quiet] -attr
```

```
TechCellMappingDefn -subscript BCM00-BCM00_map-master
```

```
{Sample tech map} I001 {[width-1:0]} {m1 m2 m3} {{m1 port} {m2 port} {m3  
port}} compA.v Name=~ADDF 1 {a1 a2 a3}
```

for older coreTools (R2020.12-SP2 or older), the attribute is as follows (macro arguments are not supported):

```
{Sample tech map} I001 {[width-1:0]} {r1 r2 r3} {{r1 port} {r2 port} {r3  
port}} compA.v Name=~ADDF
```

where the fields are:

- {Sample tech map} - description
- I000 - instance
- {[WIDTH-1:0]} - width
- {m1 m2 m3} - port_names
- {{m1 port} {m2 port} {m3 port}} - port_descriptions
- master.v - file
- Name=~ADDF - filter
- 1 - enable_macro_args switch (boolean)
- {a1 a2 a3} - extra_macro_args list
- INVERTED_OUT - output_inversion_suffix
- 1 - required switch (boolean)

Examples

Consider the following RTL module for which a technology specific mapping is desired:

```
module my_and(in0, in1, out1);
input in0;
input in1;
output out0;
// Map to technology specific cell if provided.
`ifdef my_tech_cell_macro
`my_tech_cell_macro
`else
assign out1 = in0 & in1;
`endif
endmodule
```

Create a mapping definition for the module above. This is typically done in coreBuilder:

```
define_tech_cell_mapping -module my_module \
-macro my_tech_cell_macro \
-description "Module mapped to an AND gate"
-instance I0 \
-port_names {in0 in0 out0} \
-port_descriptions {{port in0} {port in1} {port out0}}
-output_inversion_suffix "INVERTED_OUT"
-required
```

Then create the binding to a particular cell in coreConsultant or coreAssembler:

In coreTools Version where output_inversion_suffix is supported, 3rd argument is provided as '1' for port_map inversion.

```
bind_tech_cell_mapping -module my_module \
-tech_cell AN2 \
-port_map {{in0 A} {in1 B} {out0 Z 1}}
```

By default, 3rd argument of port_map is taken as '0' to support old versions.

In older coreTools versions (V-2024.06-2 or older),
-output_inversion_suffix and -required are not supported.

```
bind_tech_cell_mapping -module my_module \
-tech_cell AN2 \
-port_map {{in0 A} {in1 B} {out0 Z}}
```

In older coreTools versions (R2020.12-SP2 or older), -macro must also be specified.

This binding will result in the creation of tech mapping header file with the following content:

For the coreTools version where ouput Inversion is enabled and output is inverted.

```
`ifdef SYNTHESIS
```

coreTools Command Reference Index

```
'define mytech_cell_macro_INVERTED_OUT AN2 I0  
(.A(in0),.B(in1),.Z(out1_INVERTED_OUT));  
'endif
```

For the older coreTools version where output Inversion is not supported enabled or when output is not inverted.

```
'ifndef SYNTHESIS  
'define mytech_cell_macro AN2 I0 (.A(in0),.B(in1),.Z(out1));  
'endif
```

This file will automatically be included in the file list for synthesis and other implementation tool flows.

```
# Utilize technology cell mapping  
define_tech_cell_mapping -module BCM02 \  
-macro BCM02_map \  
-instance I002 \  
-description "Sample tech map" \  
-port_names {n1 n2 n3 n4} \  
-port_descriptions {{n1 port} {n2 port} {n3 port} {n4 port}} \  
-file "compA.v"
```

A tech cell definition can be removed (undefined) using the macro name as follows:

```
# Utilize technology cell mapping For  
coreConsultant> define_tech_cell_mapping -undef -macro BCM61_map  
Information: Undefined technology cell mapping for module BCM61 via  
macro BCM61_map. (TECH-32)  
Information: Attribute TechCellMappingDefn[BCM61-BCM61_map] removed from  
item compH. (CMD5-64)  
coreConsultant> define_tech_cell_mapping -undef -macro BCM62_map -module  
BCM62  
Warning: The '-module BCM62' option is ignored when -undef is used.  
(TECH-59)  
Information: Undefined technology cell mapping for module BCM62 via  
macro BCM62_map. (TECH-32)  
Information: Attribute TechCellMappingDefn[BCM62-BCM62_map] removed from  
item compH. (CMD5-64)  
The module name is ignored (for 2021.09-SP4 or later coreTools versions)  
while undefining a tech cell definition and all macros with different  
module names and same macro names are undefined.
```

For older ct versions (2021.09-SP3 or older), the -undef command is issued as follows:

```
coreConsultant> define_tech_cell_mapping -undef -macro BCM62_map -module  
BCM62  
Information: Undefined technology cell mapping for module BCM62 via  
macro BCM62_map. (TECH-32)  
Information: Attribute TechCellMappingDefn[BCM62-BCM62_map] removed from  
item compH. (CMD5-64)
```


See Also

[bind tech cell mapping \(2\)](#)

Deprecated List

Page [create_prime_profile_file](#)

Creating Prime Profiles using trigger parameters, trigger values and expressions:

Using `-param_name`, `-value` and `-expression` options with this command is the legacy method to create Prime Profiles and is now deprecated. A coreConsultant workspace for an IP packaged with Prime Profiles using any of these options will NOT see the [Select Prime Profile](#) activity.

The `-param_name` and `-value` options associate Prime Profile with a trigger parameter and a value. Setting the trigger parameter to a trigger value will enable application of the Prime Profile associated with them in coreConsultant and coreAssembler. A Prime Profile can then be applied using the `set_prime_profile` command or via the GUI pop-up that appears when a trigger parameter is set.

It is NOT recommended to use these 3 options while creating a Prime Profile (including external ones) since support for creating Prime Profiles with this approach will be dropped in the future.

See 'Creating Prime Profiles with triggers' in the Examples section for usage.

The following command creates a Prime Profile named Test001 with a trigger parameter and a value. When the trigger parameter `dummy` is set to value 1 in `coreConsultant` or `coreAssembler`, Prime Profile mode is triggered and Test001 and any other Profile with the same trigger parameter-value combination will be added to the list of triggered Prime Profiles for that component/design.

In shell mode, `set_prime_profile` must then be used to set a Prime Profile, e.g. Test001, from the list of triggered Prime Profiles.

In GUI mode, a Prime Profile selection pop-up will appear on setting a trigger parameter and one of the triggered profiles (or an external Prime Profile) must be selected.

An optional expression can also be specified in a `create_prime_profile_file` command. The Prime Profiles associated with this parameter/value combination will only be triggered if the expression evaluates to true.

Page [set_prime_profile](#)

In coreConsultant and coreAssembler, if Prime Profiles with triggers are packaged for a component, the trigger parameter values must be set and `set_prime_profile` must be used to select one of those

coreTools Command Reference Index

triggered Prime Profiles. In GUI mode, this command is implicitly executed using the Prime Profile selection GUI on setting trigger parameter if `set_prime_profile` is not already run. In shell mode, a Prime Profile must be set explicitly even if trigger parameters are set.

If Prime Profile, Test001, has trigger parameter `include_rom` and trigger value set to 1, then the Prime Profile must be triggered first and a Prime Profile must be selected from the list of triggered tags reported above.

design

Design item

Description

The design item type represents a design or subdesign in the coreConsultant or coreBuilder model of a design. When coreBuilder parses the HDL source code for a core, it creates a design item for each design (VHDL entity or Verilog module) that it finds.

See Also

Supported Attributes

[AddLockUpLatch \(3\)](#), [AreaEstimate \(3\)](#), [AtpgTclAuxScript \(3\)](#),
[AtpgTclAuxScriptComment \(3\)](#), [AutoConnectComplete \(3\)](#),
[AutoConnectionPreCallback \(3\)](#), [AutoFixAsync \(3\)](#), [AutoFixAsyncLogicGate \(3\)](#),
[AutoFixAsyncTestData \(3\)](#), [AutoFixAsyncWithScanEnable \(3\)](#),
[AutoFixBidirectional \(3\)](#), [AutoFixClockTestData \(3\)](#), [AutoFixClocks \(3\)](#),
[BalanceBistSegments \(3\)](#), [BidirectionalMode \(3\)](#), [BistAutoFixBusses \(3\)](#),
[BistAutoFixXprop \(3\)](#), [BistBlockIndividually \(3\)](#), [BistChainCount \(3\)](#),
[BistCodecCount \(3\)](#), [BistDiagOutputs \(3\)](#), [BistInvertPrpgClock \(3\)](#),
[BistMaxChainLength \(3\)](#), [BistMode \(3\)](#), [BistObserveOutputs \(3\)](#),
[BistPrpgLength \(3\)](#), [BistPrpgShadowSi \(3\)](#), [BistSelectorShadowSi \(3\)](#),
[BistSubblocksIndividually \(3\)](#), [BistType \(3\)](#), [BistUseTristateMux \(3\)](#),
[CTLModels \(3\)](#), [CanUngroup \(3\)](#), [CanUngroupInfo \(3\)](#), [ClockGatingSignals \(3\)](#),
[ClockMixing \(3\)](#), [ComponentOfItem \(3\)](#), [ControlPointsPerScanCell \(3\)](#),
[CreatedViaUngrouping \(3\)](#), [CriticalRange \(3\)](#),
[CriticalRangeCoveringViolators \(3\)](#), [CustomizedTestbenchCode \(3\)](#),
[DCLSPaths \(3\)](#), [DedicatedScanPorts \(3\)](#), [DedicatedWrapperCell \(3\)](#),
[DefineLibClocks \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [EnableDftAutoFix \(3\)](#),
[EnableDftShadowLogic \(3\)](#), [EnableScanCompression \(3\)](#), [EnvCheck \(3\)](#),
[ExcludeLibCells \(3\)](#), [ExternalTristates \(3\)](#), [ExtraSearchPathAppend \(3\)](#),
[ExtraSearchPathPrepend \(3\)](#), [FCExcludeLibCells \(3\)](#), [FCIncludeLibCells \(3\)](#),
[FcAuxSynthesisScript \(3\)](#), [FcAuxSynthesisScriptComment \(3\)](#),
[ForceTCBToUseComponentUndefFile \(3\)](#), [FormalVerificationAuxScript \(3\)](#),
[FormalVerificationAuxScriptComment \(3\)](#), [FpgaPreferTmg \(3\)](#),
[HasLibertyModel \(3\)](#), [HdlSourceFileNames \(3\)](#), [HierarchicalIsolation \(3\)](#),
[IPEnvList \(3\)](#), [IdentifyShiftRegisters \(3\)](#), [ImportedWorkspaceComponentFile \(3\)](#),
[IncludeLibCells \(3\)](#), [InsertEndOfChainLockupLatch \(3\)](#),
[InsertTestPoints \(3\)](#), [InstanceClockNames \(3\)](#), [InternalTristates \(3\)](#),
[IsFullyModeled \(3\)](#), [IsWorkspaceComponent \(3\)](#), [Label \(3\)](#),
[MacroPrefixFileList \(3\)](#), [MapBlockIndividually \(3\)](#),
[MapSubblocksIndividually \(3\)](#), [MaxArea \(3\)](#), [MaxCap \(3\)](#), [MaxControlPoints \(3\)](#),
[MaxFanout \(3\)](#), [MaxObservePoints \(3\)](#), [MaxTransition \(3\)](#),
[MaximumScanChainLength \(3\)](#), [MemoryMode \(3\)](#), [MessageLimitDC \(3\)](#),

coreTools Command Reference Index

[MessageLimitFC \(3\)](#), [MessageStopOnDC \(3\)](#), [MessageStopOnFC \(3\)](#), [MinCap \(3\)](#),
[MonitoredComponent \(3\)](#), [MonitoredInterface \(3\)](#), [Name \(3\)](#),
[NumberOfScanChains \(3\)](#), [NumberOfScanCompressionChains \(3\)](#),
[ObservePointsPerScanCell \(3\)](#), [OneWrapperClock \(3\)](#),
[OperatingConditionsBest \(3\)](#), [OperatingConditionsWorst \(3\)](#),
[OptimizationPriorities \(3\)](#), [OptimizeArithmetic \(3\)](#), [Overconstrain \(3\)](#),
[ParallelJobCpuLimit \(3\)](#), [ParameterCheckCmd \(3\)](#), [ParameterInfo \(3\)](#),
[ParentWireLoad \(3\)](#), [PhysicalRegion \(3\)](#), [PowerDomains \(3\)](#),
[PredefinedInoutPorts \(3\)](#), [PredefinedInputPorts \(3\)](#), [PredefinedOutputPorts](#)
[\(3\)](#), [PreserveDesignWare \(3\)](#), [PreserveHierarchyFromTop \(3\)](#), [PreserveMuxOps](#)
[\(3\)](#), [PrimeProfileAllowNone \(3\)](#), [PrimeProfileCurrent \(3\)](#),
[PrimeProfileDefaults \(3\)](#), [PrimeProfileNote \(3\)](#), [PrimeProfilesValid \(3\)](#),
[RTLStaticCheckingAsSubblock \(3\)](#), [RalListInfo \(3\)](#), [RtlaAuxScript \(3\)](#),
[RtlaAuxScriptComment \(3\)](#), [RunOptions \(3\)](#), [SDCCConstraints \(3\)](#),
[SDCPropagateText \(3\)](#), [SDCWriteCallback \(3\)](#), [ScanBlockIndividually \(3\)](#),
[ScanCompressionConfiguration \(3\)](#), [ScanCompressionTestClock \(3\)](#),
[ScanCompressionUsage \(3\)](#), [ScanExclude \(3\)](#), [ScanInternalClocks \(3\)](#),
[ScanJumpBuffersInverters \(3\)](#), [ScanMethodology \(3\)](#), [ScanReplace \(3\)](#),
[ScanStyle \(3\)](#), [ScanSubblocksIndividually \(3\)](#), [ScratchInstallIndex \(3\)](#),
[Sequence \(3\)](#), [ShareTestPointsAcrossHierarchy \(3\)](#), [SharedWrapperCell \(3\)](#),
[SimulationModel \(3\)](#), [SolutionData \(3\)](#), [SolutionEnvVarValues \(3\)](#),
[SolutionProcessed \(3\)](#), [SpiritContainer \(3\)](#), [SpiritFile \(3\)](#), [SpiritLibrary](#)
[\(3\)](#), [SpiritName \(3\)](#), [SpiritVendor \(3\)](#), [SpiritVersion \(3\)](#),
[StaticDefineExprs \(3\)](#), [StaticTimingAuxScript \(3\)](#),
[StaticTimingAuxScriptComment \(3\)](#), [SymbolLibraryPath \(3\)](#), [SymbolName \(3\)](#),
[SymbolType \(3\)](#), [SynplifyAuxSynthesisScript \(3\)](#),
[SynplifyAuxSynthesisScriptComment \(3\)](#), [SynthesizeAsSubblock \(3\)](#),
[TCBDisableUndefMacros \(3\)](#), [TCLConfigVariableSetArgs \(3\)](#), [TMMRMVariable](#)
[\(3\)](#), [TMRIntentFile \(3\)](#), [TMRIntentFileComment \(3\)](#), [TclAuxSynthesisScript](#)
[\(3\)](#), [TclAuxSynthesisScriptComment \(3\)](#), [TclPhysicalConstraintFile \(3\)](#),
[TechCellMappingBind \(3\)](#), [TechCellMappingDefn \(3\)](#),
[TestClockDefaultCycleTime \(3\)](#), [TestClocksFollowSystemDomains \(3\)](#),
[TestabilityClockSignal \(3\)](#), [TestabilityClockType \(3\)](#),
[TestabilityControlSignal \(3\)](#), [TestabilityMaxArea \(3\)](#), [TestabilityMethod](#)
[\(3\)](#), [TestpointPowerSavingOn \(3\)](#), [TmmAuxScript \(3\)](#), [TmmAuxScriptComment](#)
[\(3\)](#), [TypeName \(3\)](#), [UPFFFile \(3\)](#), [UVMTestText \(3\)](#), [Underconstrain \(3\)](#),
[UnelaboratedName \(3\)](#), [VHDLDesignLibrary \(3\)](#), [VipModelName \(3\)](#), [VipPackage](#)
[\(3\)](#), [WireLoad \(3\)](#), [WireLoadGroup \(3\)](#), [WireLoadMinBlockSize \(3\)](#),
[WireLoadMode \(3\)](#), [WrapBlockIndividually \(3\)](#), [WrapSubblocksIndividually](#)
[\(3\)](#), [WrapperDefaultSafeState \(3\)](#), [WrapperExcludePorts \(3\)](#),
[WrapperRegisterImplementation \(3\)](#), [WrapperUseRegisterIO \(3\)](#),
[WriteComponentInHDL \(3\)](#), [dc_shellStopMessageIds \(3\)](#), [dc_shellVariable](#)
[\(3\)](#), [dc_shellVariableComment \(3\)](#), [fc_shellAppOption \(3\)](#),
[fc_shellAppOptionComment \(3\)](#), [fm_shellVariable \(3\)](#),
[fm_shellVariableComment \(3\)](#), [pt_shellVariable \(3\)](#),
[pt_shellVariableComment \(3\)](#), [rtl_shellAppOption \(3\)](#),
[rtl_shellAppOptionComment \(3\)](#), [tmax_shellVariable \(3\)](#),
[tmax_shellVariableComment \(3\)](#), [tmm_shellAppOption \(3\)](#),
[tmm_shellAppOptionComment \(3\)](#)

`detach_interface`

Remove an attached interface instance from the given component.

Syntax

```
string detach_interface -from <componentName> name
      string <componentName>
      string name
```

Parameters

<code>-from</code>	The component the interface is attached to
<code><componentName></code>	
<code>name</code>	Name of the attached interface instance to remove

Description

If an interface was attached to the named component with `attach_interface`, then `detach_interface` will remove the interface instance and unconnect all existing interface connections.

Note: `remove_component` automatically detaches all interfaces from itself.

Examples

To detach the attached interface `GPIOslave` from imported component `MyGPIO`:

```
detach_interface -from MyGPIO -name GPIOslave
```

See Also

[attach_interface \(2\)](#), [remove_exported_interface \(2\)](#), [unconnect_interface \(2\)](#), [remove_component \(2\)](#)

NAME

`dict` - Manipulate dictionaries

SYNOPSIS

`dict option arg ?arg ...?`

DESCRIPTION

Performs one of several operations on dictionary values or variables containing dictionary values (see the **DICTIONARY VALUES** section below for a description), depending on *option*. The legal *options* (which may be abbreviated) are:

dict append *dictionaryVariable* *key* ?*string* ...?

This appends the given string (or strings) to the value that the given key maps to in the dictionary value contained in the given variable, writing the resulting dictionary value back to that variable. Non-existent keys are treated as if they map to an empty string. The updated dictionary value is returned.

dict create ?*key* *value* ...?

Return a new dictionary that contains each of the key/value mappings listed as arguments (keys and values alternating, with each key being followed by its associated value.)

dict exists *dictionaryValue* *key* ?*key* ...?

This returns a boolean value indicating whether the given key (or path of keys through a set of nested dictionaries) exists in the given dictionary value. This returns a true value exactly when **dict get** on that path will succeed.

dict filter *dictionaryValue* *filterType* *arg* ?*arg* ...?

coreTools Command Reference Index

This takes a dictionary value and returns a new dictionary that contains just those key/value pairs that match the specified filter type (which may be abbreviated.) Supported filter types are:

dict filter *dictionaryValue* **key** *?globPattern ...?*

The key rule only matches those key/value pairs whose keys match any of the given patterns (in the style of **string match**.)

script

dict filter *dictionaryValue* **script** {*keyVariable valueVariable*}

The script rule tests for matching by assigning the key to the *keyVariable* and the value to the *valueVariable*, and then evaluating the given script which should result in a boolean value (with the key/value pair only being included in the result of the **dict filter** when a true value is returned.) Note that the first argument after the rule selection word is a two-element list. If the *script* returns with a condition of **TCL_BREAK**, no further key/value pairs are considered for inclusion in the resulting dictionary, and a condition of **TCL_CONTINUE** is equivalent to a false result. The key/value pairs are tested in the order in which the keys were inserted into the dictionary.

dict filter *dictionaryValue* **value** *?globPattern ...?*

The value rule only matches those key/value pairs whose values match any of the given patterns (in the style of **string match**.)

dict for {*keyVariable valueVariable*} *dictionaryValue* *body*

This command takes three arguments, the first a two-element list of variable names (for the key and value respectively of each mapping in the dictionary), the second the dictionary value to iterate across, and the third a script to be evaluated for each mapping with the key and value variables set appropriately (in the manner of **foreach**.) The result of the command is an empty string. If any evaluation of the body generates a **TCL_BREAK** result, no further pairs from the dictionary will be iterated over and the **dict for** command will terminate successfully immediately. If any evaluation of the body generates a **TCL_CONTINUE** result, this shall be treated exactly like a normal **TCL_OK** result. The order of iteration is the order in which the keys were inserted into the dictionary.

dict get *dictionaryValue* *?key ...?*

Given a dictionary value (first argument) and a key (second argument), this will retrieve the value for that key. Where several keys are supplied, the behaviour of the command shall be as if the result of **dict get** *\$dictVal \$key* was passed as the first argument to **dict get** with the remaining arguments as second (and possibly subsequent) arguments. This facilitates

coreTools Command Reference Index

lookups in nested dictionaries. For example, the following two commands are equivalent:

```
dict get $dict foo bar spong
dict get [dict get [dict get $dict foo] bar] spong
```

If no keys are provided, **dict get** will return a list containing pairs of elements in a manner similar to **array get**. That is, the first element of each pair would be the key and the second element would be the value for that key.

It is an error to attempt to retrieve a value for a key that is not present in the dictionary.

dict incr *dictionaryVariable* *key* *?increment?*

This adds the given increment value (an integer that defaults to 1 if not specified) to the value that the given key maps to in the dictionary value contained in the given variable, writing the resulting dictionary value back to that variable. Non-existent keys are treated as if they map to 0. It is an error to increment a value for an existing key if that value is not an integer. The updated dictionary value is returned.

dict info *dictionaryValue*

This returns information (intended for display to people) about the given dictionary though the format of this data is dependent on the implementation of the dictionary. For dictionaries that are implemented by hash tables, it is expected that this will return the string produced by **Tcl_HashStats**, similar to **array statistics**.

dict keys *dictionaryValue* *?globPattern?*

Return a list of all keys in the given dictionary value. If a pattern is supplied, only those keys that match it (according to the rules of **string match**) will be returned. The returned keys will be in the order that they were inserted into the dictionary.

dict lappend *dictionaryVariable* *key* *?value ...?*

This appends the given items to the list value that the given key maps to in the dictionary value contained in the given variable, writing the resulting dictionary value back to that variable. Non-existent keys are treated as if they map to an empty list, and it is legal for there to be no items to append to the list. It is an error for the value that the key maps to to not be representable as a list. The updated dictionary value is returned.

dict map *{keyVariable valueVariable}* *dictionaryValue* *body*

This command applies a transformation to each element of a dictionary, returning a new dictionary. It takes three arguments: the first is a two-element list of variable names (for the key and value respectively of each mapping in the dictionary), the second the dictionary value to iterate across, and the third a script to be evaluated for each mapping with the key

coreTools Command Reference Index

and value variables set appropriately (in the manner of **lmap**). In an iteration where the evaluated script completes normally (**TCL_OK**, as opposed to an **error**, etc.) the result of the script is put into an accumulator dictionary using the key that is the current contents of the *keyVariable* variable at that point. The result of the **dict map** command is the accumulator dictionary after all keys have been iterated over.

If the evaluation of the body for any particular step generates a **break**, no further pairs from the dictionary will be iterated over and the **dict map** command will terminate successfully immediately. If the evaluation of the body for a particular step generates a **continue** result, the current iteration is aborted and the accumulator dictionary is not modified. The order of iteration is the natural order of the dictionary (typically the order in which the keys were added to the dictionary; the order is the same as that used in **dict for**).

dict merge *?dictionaryValue ...?*

Return a dictionary that contains the contents of each of the *dictionaryValue* arguments. Where two (or more) dictionaries contain a mapping for the same key, the resulting dictionary maps that key to the value according to the last dictionary on the command line containing a mapping for that key.

dict remove *dictionaryValue ?key ...?*

Return a new dictionary that is a copy of an old one passed in as first argument except without mappings for each of the keys listed. It is legal for there to be no keys to remove, and it also legal for any of the keys to be removed to not be present in the input dictionary in the first place.

dict replace *dictionaryValue ?key value ...?*

Return a new dictionary that is a copy of an old one passed in as first argument except with some values different or some extra key/value pairs added. It is legal for this command to be called with no key/value pairs, but illegal for this command to be called with a key but no value.

dict set *dictionaryVariable key ?key ...? value*

This operation takes the name of a variable containing a dictionary value and places an updated dictionary value in that variable containing a mapping from the given key to the given value. When multiple keys are present, this operation creates or updates a chain of nested dictionaries. The updated dictionary value is returned.

dict size *dictionaryValue*

Return the number of key/value mappings in the given dictionary value.

dict unset *dictionaryVariable key ?key ...?*

This operation (the companion to **dict set**) takes the name of a variable containing a dictionary value and

coreTools Command Reference Index

places an updated dictionary value in that variable that does not contain a mapping for the given key. Where multiple keys are present, this describes a path through nested dictionaries to the mapping to remove. At least one key must be specified, but the last key on the key-path need not exist. All other components on the path must exist. The updated dictionary value is returned.

dict update *dictionaryVariable key varName ?key varName ...? body*

Execute the Tcl script in *body* with the value for each *key* (as found by reading the dictionary value in *dictionaryVariable*) mapped to the variable *varName*. There may be multiple *key/varName* pairs. If a *key* does not have a mapping, that corresponds to an unset *varName*. When *body* terminates, any changes made to the *varNames* is reflected back to the dictionary within *dictionaryVariable* (unless *dictionaryVariable* itself becomes unreadable, when all updates are silently discarded), even if the result of *body* is an error or some other kind of exceptional exit. The result of **dict update** is (unless some kind of error occurs) the result of the evaluation of *body*.

Each *varName* is mapped in the scope enclosing the **dict update**; it is recommended that this command only be used in a local scope (**procedure**, lambda term for **apply**, or method). Because of this, the variables set by **dict update** will continue to exist after the command finishes (unless explicitly **unset**). Note that the mapping of values to variables does not use traces; changes to the *dictionaryVariable*'s contents only happen when *body* terminates.

dict values *dictionaryValue ?globPattern?*

Return a list of all values in the given dictionary value. If a pattern is supplied, only those values that match it (according to the rules of **string match**) will be returned. The returned values will be in the order of that the keys associated with those values were inserted into the dictionary.

dict with *dictionaryVariable ?key ...? body*

Execute the Tcl script in *body* with the value for each *key* in *dictionaryVariable* mapped (in a manner similarly to **dict update**) to a variable with the same name. Where one or more *keys* are available, these indicate a chain of nested dictionaries, with the innermost dictionary being the one opened out for the execution of *body*. As with **dict update**, making *dictionaryVariable* unreadable will make the updates to the dictionary be discarded, and this also happens if the contents of *dictionaryVariable* are adjusted so that the chain of dictionaries no longer exists. The result of **dict with** is (unless some kind of error occurs) the result of the evaluation of *body*.

The variables are mapped in the scope enclosing the **dict with**; it is recommended that this command only be used in a local scope (**procedure**, lambda term for **apply**, or method). Because of this, the variables set

coreTools Command Reference Index

by **dict with** will continue to exist after the command finishes (unless explicitly **unset**). Note that the mapping of values to variables does not use traces; changes to the *dictionaryVariable*'s contents only happen when *body* terminates.

If the *dictionaryVariable* contains a value that is not a dictionary at the point when the *body* terminates (which can easily happen if the name is the same as any of the keys in dictionary) then an error occurs at that point. This command is thus not recommended for use when the keys in the dictionary are expected to clash with the *dictionaryVariable* name itself. Where the contained key does map to a dictionary, the net effect is to combine that inner dictionary into the outer dictionary; see the **EXAMPLES** below for an illustration of this.

DICTIONARY VALUES

Dictionaries are values that contain an efficient, order-preserving mapping from arbitrary keys to arbitrary values. Each key in the dictionary maps to a single value. They have a textual format that is exactly that of any list with an even number of elements, with each mapping in the dictionary being represented as two items in the list. When a command takes a dictionary and produces a new dictionary based on it (either returning it or writing it back into the variable that the starting dictionary was read from) the new dictionary will have the same order of keys, modulo any deleted keys and with new keys added on to the end. When a string is interpreted as a dictionary and it would otherwise have duplicate keys, only the last value for a particular key is used; the others are ignored, meaning that, and are equivalent dictionaries (with different string representations).

Operations that derive a new dictionary from an old one (e.g., updates like **dict set** and **dict unset**) preserve the order of keys in the dictionary. The exceptions to this are for any new keys they add, which are appended to the sequence, and any keys that are removed, which are excised from the order.

EXAMPLES

Basic dictionary usage:

```
# Make a dictionary to map extensions to descriptions
set filetypes [dict create .txt "Text File" .tcl "Tcl File"]

# Add/update the dictionary
dict set filetypes .tcl "Tcl Script"
dict set filetypes .tm "Tcl Module"
dict set filetypes .gif "GIF Image"
```

coreTools Command Reference Index

```
dict set filetypes .png "PNG Image"

# Simple read from the dictionary
set ext ".tcl"
set desc [dict get $filetypes $ext]
puts "$ext is for a $desc"

# Somewhat more complex, with existence test
foreach filename [glob *] {
    set ext [file extension $filename]
    if {[dict exists $filetypes $ext]} {
        puts "$filename is a [dict get $filetypes $ext]"
    }
}

Constructing and using nested dictionaries:

# Data for one employee
dict set employeeInfo 12345-A forenames "Joe"
dict set employeeInfo 12345-A surname  "Schmoe"
dict set employeeInfo 12345-A street  "147 Short Street"
dict set employeeInfo 12345-A city    "Springfield"
dict set employeeInfo 12345-A phone  "555-1234"
# Data for another employee
dict set employeeInfo 98372-J forenames "Anne"
dict set employeeInfo 98372-J surname  "Other"
dict set employeeInfo 98372-J street  "32995 Oakdale Way"
dict set employeeInfo 98372-J city    "Springfield"
dict set employeeInfo 98372-J phone  "555-8765"
# The above data probably ought to come from a database...

# Print out some employee info
set i 0
puts "There are [dict size $employeeInfo] employees"
dict for {id info} $employeeInfo {
    puts "Employee #[incr i]: $id"
    dict with info {
        puts "    Name: $forenames $surname"
        puts "    Address: $street, $city"
        puts "    Telephone: $phone"
    }
}

# Another way to iterate and pick out names...
foreach id [dict keys $employeeInfo] {
    puts "Hello, [dict get $employeeInfo $id forenames]!"
}

A localizable version of string toupper:

# Set up the basic C locale
set capital [dict create C [dict create]]
foreach c [split {abcdefghijklmnopqrstuvwxyz} ""] {
    dict set capital C $c [string toupper $c]
}

# English locales can luckily share the "C" locale
dict set capital en [dict get $capital C]
dict set capital en_US [dict get $capital C]
dict set capital en_GB [dict get $capital C]

# ... and so on for other supported languages ...
```

coreTools Command Reference Index

```
# Now get the mapping for the current locale and use it.
set upperCaseMap [dict get $capital $env(LANG)]
set upperCase [string map $upperCaseMap $string]
```

Showing the detail of **dict with**:

```
proc sumDictionary {varName} {
    upvar 1 $varName vbl
    foreach key [dict keys $vbl] {
        # Manufacture an entry in the subdictionary
        dict set vbl $key total 0
        # Add the values and remove the old
        dict with vbl $key {
            set total [expr {$x + $y + $z}]
            unset x y z
        }
    }
    puts "last total was $total, for key $key"
}

set myDict {
    a {x 1 y 2 z 3}
    b {x 6 y 5 z 4}
}

sumDictionary myDict
#   prints: last total was 15, for key b

puts "dictionary is now \"$myDict\""
#   prints: dictionary is now "a {total 6} b {total 15}"

When dict with is used with a key that clashes with the
name of the dictionary variable:

set foo {foo {a b} bar 2 baz 3}
dict with foo {}
puts $foo
#   prints: a b foo {a b} bar 2 baz 3
```

SEE ALSO

append(n), array(n), foreach(n), incr(n), list(n),
lappend(n), lmap(n), set(n)

KEYWORDS

dictionary, create, update, lookup, iterate, filter,
map

`disable_prime_profile`

Disable Prime Profile related commands in builder, consultant and assembler (Synopsys only).

Syntax

string **`disable_prime_profile`**

Description

This command is only allowed at Synopsys site. Issuing `disable_prime_profile` in `coreConsultant` or `coreAssembler` will disable all Prime Profile related functionality and checks. Prime Profile validation expression checks are also disabled.

Examples

```
coreConsultant> disable_prime_profile
coreConsultant> autocomplete SpecifyConfiguration
coreConsultant> is_prime_profile_enabled
1
coreConsultant> disable_prime_profile
Warning: Prime Profile feature is disabled. Prime Profile related
commands will not have any effect. (CMDS-288)
1
coreConsultant> is_prime_profile_enabled
0
coreConsultant> enable_prime_profile
Information: Prime Profile feature and its related commands will be
enabled. (CMDS-297)
0
coreConsultant> is_prime_profile_enabled
1
```

See Also

[set_prime_profile \(2\) create_prime_profile_file \(2\)](#)
[set_configuration_parameter \(2\) enable_prime_profile \(2\)](#)
[is_prime_profile_enabled \(2\)](#)

`docbook_to_html`

Convert the given DocBook XML file to HTML format.

Syntax

```
string docbook_to_html [-xrefs <value>] [-docid <id>] [-target_db_doc
<olinkdb.xml>] [-xref_db_name <target.db>] [-expected_filename
<index.html>] [-copy_to_src_dir] docbook-files
    string <value>
    string <id>
    string <olinkdb.xml>
    string <target.db>
    string <index.html>
    string docbook-files
```

Parameters

<code>-xrefs <value></code>	Generate cross reference targets for linking. (Values: no, yes, only)
<code>-docid <id></code>	Current document ID
<code>-target_db_doc <olinkdb.xml></code>	Target database document
<code>-xref_db_name <target.db></code>	Name for the output when '-xrefs' is used. Defaults to target.db
<code>-expected_filename <index.html></code>	The file name output by xsltproc.
<code>-copy_to_src_dir</code>	The expected output file should be copied to the xml source directory.
<code>docbook-files</code>	DocBook format XML file names

Description

Converts a DocBook format XML file into HTML. The HTML file is installed in the same directory as the XML file.

Examples

This example converts a simple xml file with no links to other files:
`docbook_to_html /full/path/my_report.xml`

In the following example file1.xml contains no external links, file2.xml has links to file1.xml, and file3.xml has links to file2.xml. Each file is in the report directory of the current workspace. This example assumes

that the file olinkdb.xml exists and that each of the files has been given an xml id as in:

```
<docbook:book xml:id="file1"></docbook:book>
# First generate just the cross references
foreach fileName {file1 file2 file3} {
docbook_to_html [get_logical_dir report/${fileName}.xml] \
-xrefs only \
-xref_db_name ${fileName}.db
}
# Convert file1.xml to html.
docbook_to_html [get_logical_dir report/file1.xml]
# Now file2.xml and file3.xml can be converted to html.
foreach fileName {file2 file3} {
docbook_to_html [get_logical_dir report/${fileName}.xml] \
-docid $fileName \
-target_db_doc olinkdb.xml
}
```

See Also

[get_logical_dir \(2\).](#)

`duplicate_component`

Duplicate a component in the subsystem

Syntax

```
string duplicate_component [-share] [-allow_prefix] comp [new]
      string comp
      string new
```

Parameters

<code>-share</code>	Duplicate as a shared component with only one workspace area.
<code>-allow_prefix</code>	Allow prefixed instances within a hierarchical component.
<code>comp</code>	The name of the component to duplicate
<code>new</code>	The name of the component to create

Description

The `duplicate_component` command is used to instantiate a copy of a given component in the subsystem. It is the only way to have two instances of the same imported component.

Examples

To add another APB component:

```
coreAssembler> duplicate_component i_apb i_apb_1
```

See Also

[import_component \(2\), instantiate_component \(2\), remove_component \(2\)](#)

duplicate_workspace

Save the current workspace as duplicate.

Syntax

```
string duplicate_workspace [-name <name>] [-root <directory>] [-design
<name>] [-copy]
    string <name>
    string <directory>
```

Parameters

	Name of the new workspace (default to current workspace name).
-name <name>	This name will be used to create the workspace directory under the -root directory. By default the workspace name is the same as for the current workspace, but this requires a different -root directory.
-root <directory>	The root directory in which to create the new workspace (default to current directory). This directory must already exist. By default the workspace root is the same as for the current workspace, but this requires a different workspace -name.
-design <name>	The name of the new top-level design (coreAssembler only). This option allows a change of the design name while duplicating the workspace.
-copy	Remove all symbolic links to the installation but copy these files (coreConsultant and coreAssembler only). The saved workspace becomes independent from changes in the coreKit installation area, and you can easily ship the new workspace to another location (other site, or customer support). The costs are duplicated files, which is wasted disk space if both workspaces are locally used only.

Description

The current workspace is duplicated to another workspace, also known as "Save As". The location of the new workspace must either differ by a new directory root or by a new workspace name. Both the same is the default for the options: so either -root or -name must be present.

If activity Synthesize of the current workspace has already been completed then in the duplicated workspace this activity becomes incomplete and must be started again. This makes sure that no absolute path inside the synthesis directory structure is pointing back to the original workspace.

If you change the design name then a completed activity "Generate Subsystem RTL" becomes incomplete: The new design name requires new HDL files, and dependend activities like Synthesize or simulation need to run again with the changed name and files.

The current workspace is saved and closed, and the new duplicated workspace becomes the current workspace.

Examples

To save the current coreAssembler workspace "subsystem1" with the workspace name "AMBAreference" in parallel to the current workspace directory:

```
coreAssembler> duplicate_workspace -name AMBAreference
```

To save the current coreAssembler workspace with new design name "BtBus" into workspace "BtBus1" below the directory /u/bt/tests:

```
coreAssembler> duplicate_workspace -name BtBus1 -root /u/bt/tests -design  
BtBus
```

See Also

[create_workspace \(2\)](#), [open_workspace \(2\)](#), [close_workspace \(2\)](#)

NAME

echo Echos arguments to standard output.

SYNTAX

```
string echo  
      [-n]  
      [arguments]
```

<i>Data Types</i>	
<i>arguments</i>	string

ARGUMENTS

-n	Suppresses the new line. By default, echo adds a new line.
<i>arguments</i>	Specifies the arguments to be printed.

DESCRIPTION

The **echo** command prints out the value of the given arguments. Each of the arguments are separated by a space. The line is normally terminated with a new line, but if the **-n** option is specified, multiple **echo** command outputs are printed onto the same output line.

The **echo** command is used to print out the value of variables and expressions in addition to text strings. The output from **echo** can be redirected using the **>** and **>>** operators.

EXAMPLES

The following are examples of using the **echo** command:

coreTools Command Reference Index

```
prompt> echo
"Running version" $sh_product_version
Running version v3.0a

prompt> echo -n "Printing to" [expr (3 - 2)] > foo
prompt> echo " line." >> foo
prompt> sh cat foo
Printing to 1 line.
```

SEE ALSO

sh(2)

[API Index](#) [Command Reference](#)

`enable_prime_profile`

Enable Prime Profile related commands in builder, consultant and assembler (Synopsys only).

Syntax

string **enable_prime_profile**

Description

This command is only allowed at Synopsys site. Prime Profile feature and related commands will be enabled on issuing this.

Examples

```
coreConsultant> is_prime_profile_enabled
1
coreConsultant> disable_prime_profile
Warning: Prime Profile feature is disabled. Prime Profile related
commands will not have any effect. (CMDS-288)
1
coreConsultant> is_prime_profile_enabled
0
coreConsultant> enable_prime_profile
Information: Prime Profile feature and its related commands will be
enabled. (CMDS-297)
0
coreConsultant> is_prime_profile_enabled
1
```

See Also

[disable_prime_profile \(2\)](#) [is_prime_profile_enabled \(2\)](#) [set_prime_profile \(2\)](#) [create_prime_profile_file \(2\)](#)

NAME

`encoding` - Manipulate encodings

SYNOPSIS

`encoding option ?arg arg ...?`

INTRODUCTION

Strings in Tcl are logically a sequence of 16-bit Unicode characters. These strings are represented in memory as a sequence of bytes that may be in one of several encodings: modified UTF-8 (which uses 1 to 3 bytes per character), 16-bit (which uses 2 bytes per character, with an endianness that is dependent on the host architecture), and binary (which uses a single byte per character but only handles a restricted range of characters). Tcl does not guarantee to always use the same encoding for the same string.

Different operating system interfaces or applications may generate strings in other encodings such as Shift-JIS. The **encoding** command helps to bridge the gap between Unicode and these other formats.

DESCRIPTION

Performs one of several encoding related operations, depending on *option*. The legal *options* are:

encoding convertfrom *?encoding? data*
Convert *data* to Unicode from the specified *encoding*.
The characters in *data* are treated as binary data where

coreTools Command Reference Index

the lower 8-bits of each character is taken as a single byte. The resulting sequence of bytes is treated as a string in the specified *encoding*. If *encoding* is not specified, the current system encoding is used.

encoding convertto *?encoding?* *string*

Convert *string* from Unicode to the specified *encoding*. The result is a sequence of bytes that represents the converted string. Each byte is stored in the lower 8-bits of a Unicode character (indeed, the resulting string is a binary string as far as Tcl is concerned, at least initially). If *encoding* is not specified, the current system encoding is used.

encoding dirs *?directoryList?*

Tcl can load encoding data files from the file system that describe additional encodings for it to work with. This command sets the search path for ***.enc** encoding data files to the list of directories *directoryList*. If *directoryList* is omitted then the command returns the current list of directories that make up the search path. It is an error for *directoryList* to not be a valid list. If, when a search for an encoding data file is happening, an element in *directoryList* does not refer to a readable, searchable directory, that element is ignored.

encoding names

Returns a list containing the names of all of the encodings that are currently available. The encodings and are guaranteed to be present in the list.

encoding system *?encoding?*

Set the system encoding to *encoding*. If *encoding* is omitted then the command returns the current system encoding. The system encoding is used whenever Tcl passes strings to system calls.

EXAMPLE

It is common practice to write script files using a text editor that produces output in the euc-jp encoding, which represents the ASCII characters as single bytes and Japanese characters as two bytes. This makes it easy to embed literal strings that correspond to non-ASCII characters by simply typing the strings in place in the script. However, because the **source** command always reads files using the current system encoding, Tcl will only source such files correctly when the encoding used to write the file is the same. This tends not to be true in an internationalized setting. For example, if such a file was sourced in North America (where the ISO8859-1 is normally used), each byte in the file would be treated as a separate character that maps to the 00 page in Unicode. The resulting Tcl strings will not contain the expected Japanese characters. Instead, they will contain a sequence of Latin-1 characters that correspond to the bytes of the original string. The **encoding** command can

coreTools Command Reference Index

be used to convert this string to the expected Japanese Unicode characters. For example,

```
set s [encoding convertfrom euc-jp "\xA4\xCF"]
```

would return the Unicode string which is the Hiragana letter HA.

SEE ALSO

`Tcl_GetEncoding(3)`

KEYWORDS

encoding, unicode

encrypt_solution

Encrypt TCL files associated with a given DW IP Solution directory.

Syntax

```
string encrypt_solution [-encryption <encryption style>] [-exclude
<files>] [-target <directory>] directory
    string <encryption style>
    string <files>
    string <directory>
    string directory
```

Parameters

-encryption <encryption style>	Encryption style (Values: synenc, tbc, internal)
-exclude <files>	Leaf file names to be excluded
-target <directory>	Target output directory
directory	Solution directory to be encrypted

Description

This command is used to encrypt the TCL files contained in the specified DW IP Solution directory. By default, the following are encrypted: solution.tcl, tcl/*.tcl, and tcl/include/*.tcl.inc. If specific files should be left unencrypted, the base name of the file can be passed via -exclude.

If the -target option is used, the entire solution directory is copied to the target location, and then the TCL files are encrypted and renamed with .tcl replaced with .tbc in the file name. The specified target directory must not exist and the parent directory of the target directory must exist and be writable.

If the -target option is not specified, encrypted files will reside adjacent to the source TCL file with a name created by replacing .tcl with .tbc in the name. In this case the original TCL files are not removed. The command returns the original names of the files that were encrypted.

The command supports encryption using Synenc encryption (-encryption synenc) or using TCL byte-compile. The encryption methodology defaults to synenc. To use synenc encryption tool root for Design Compiler need to set. NOTE : Synenc encryption is a proprietary Synopsys encryption scheme.

Examples

Encrypt every file in the solution directory solution_home/mySolution/1.0 except for tcl/include/techsetup.tcl.inc:

```
encrypt_solution solution_home/mySolution/1.0 -exclude techsetup.tclinc
```

Encrypt every file and copy to encrypted_solution/mySolution/1.0

```
encrypt_solution solution_home/mySolution/1.0 -target  
encrypted_solution/mySolution/1.0
```

See Also

[define_solution\(2\)](#)

NAME

`eof` - Check for end of file condition on channel

SYNOPSIS

`eof` *channelId*

DESCRIPTION

Returns 1 if an end of file condition occurred during the most recent input operation on *channelId* (such as `gets`), 0 otherwise.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**`stdin`**, **`stdout`**, or **`stderr`**), the return value from an invocation of **`open`** or **`socket`**, or the result of a channel creation command provided by a Tcl extension.

EXAMPLES

Read and print out the contents of a file line-by-line:

```
set f [open somefile.txt]
while {1} {
    set line [gets $f]
    if {[eof $f]} {
        close $f
        break
    }
    puts "Read line: $line"
}
```

coreTools Command Reference Index

Read and print out the contents of a file by fixed-size records:

```
set f [open somefile.dat]
fconfigure $f -translation binary
set recordSize 40
while {1} {
    set record [read $f $recordSize]
    if {[eof $f]} {
        close $f
        break
    }
    puts "Read record: $record"
}
```

SEE ALSO

file(n), open(n), close(n), fblocked(n),
Tcl_StandardChannels(3)

KEYWORDS

channel, end of file

NAME

`error` - Generate an error

SYNOPSIS

`error message ?info? ?code?`

DESCRIPTION

Returns a **TCL_ERROR** code, which causes command interpretation to be unwound. *Message* is a string that is returned to the application to indicate what went wrong.

The **-errorinfo** return option of an interpreter is used to accumulate a stack trace of what was in progress when an error occurred; as nested commands unwind, the Tcl interpreter adds information to the **-errorinfo** return option. If the *info* argument is present, it is used to initialize the **-errorinfo** return options and the first increment of unwind information will not be added by the Tcl interpreter. In other words, the command containing the **error** command will not appear in the stack trace; in its place will be *info*. Historically, this feature had been most useful in conjunction with the **catch** command: if a caught error cannot be handled successfully, *info* can be used to return a stack trace reflecting the original point of occurrence of the error:

```
catch {...} errMsg
set savedInfo $::errorInfo
...
error $errMsg $savedInfo
```

When working with Tcl 8.5 or later, the following code

coreTools Command Reference Index

should be used instead:

```
catch {...} errMsg options
...
return -options $options $errMsg
```

If the *code* argument is present, then its value is stored in the **-errorcode** return option. The **-errorcode** return option is intended to hold a machine-readable description of the error in cases where such information is available; see the **return** manual page for information on the proper format for this option's value.

EXAMPLE

Generate an error if a basic mathematical operation fails:

```
if {1+2 != 3} {
    error "something is very wrong with addition"
}
```

SEE ALSO

catch(n), return(n)

KEYWORDS

error, exception

NAME

error_info Prints extended information on errors from the last command.

SYNTAX

string **error_info**

ARGUMENTS

This **error_info** command has no arguments.

DESCRIPTION

The **error_info** command is used to display information after an error has occurred. Tcl collects information showing the call stack of commands and procedures. When an error occurs, the **error_info** command can help you to focus on the exact line in a block that caused the error.

EXAMPLES

This example shows how **error_info** can be used to trace an error. The error is that the iterator variable "s" is not dereferenced in the 'if' statement. It should be '\$s == "a" '.

```
prompt> foreach s $my_list {
?         if { s == "a" } {
?         echo "Found 'a'!"
?         }
?     }
Error: syntax error in expression " s == "a" "
      Use error_info for more info. (CMD-013)
shell> error_info
Extended error info:
syntax error in expression " s == "a" "
```

coreTools Command Reference Index

```
while executing
"if { s == a } {
    echo "Found 'a'"
}"
    ("foreach" body line 2)
    invoked from within
"foreach s [list a b c] {
    if { s == a } {
        echo "Found 'a'"
    }
}"
-- End Extended Error Info
```

escaped_name

Converts escaped name to internally compatible mangled name

Syntax

```
string escaped_name rtl_name
      string rtl_name
```

Parameters

rtl_name Escaped name which is to be converted to mangled name

Description

The escaped_name command is used to reference Verilog/SystemVerilog escaped name object. It converts actual escaped name, which is present in RTL, to a mangled name which coreTools correctly understands. CoreTools uses this mangled name to identify the correct object in data-model.

Examples

The following command exports a port having escaped name, from a component i_test:

```
coreAssembler> set pinName "i_test/[escaped_name
{\%%$OUT2*%^d6&**87()57}][7:0]"
coreAssembler> export_pin -port ex_out2 -auto_dim $pinName
To set InterfaceLink attribute on an interface port, to a design port
with escaped name:
```

```
coreBuilder> set portName [escaped_name {\OUT1*%568s^&67s**()a}]
coreBuilder> set_interface_port_attribute intf_inst intf_port
InterfaceLink $portName
```

See Also

[export_pin \(2\)](#), [set_interface_port_attribute \(2\)](#)

NAME

`eval` - Evaluate a Tcl script

SYNOPSIS

`eval arg ?arg ...?`

DESCRIPTION

Eval takes one or more arguments, which together comprise a Tcl script containing one or more commands. **Eval** concatenates all its arguments in the same fashion as the **concat** command, passes the concatenated string to the Tcl interpreter recursively, and returns the result of that evaluation (or any error generated by it). Note that the **list** command quotes sequences of words in such a way that they are not further expanded by the **eval** command.

EXAMPLES

Often, it is useful to store a fragment of a script in a variable and execute it later on with extra values appended. This technique is used in a number of places throughout the Tcl core (e.g. in **fcopy**, **lsort** and **trace** command callbacks). This example shows how to do this using core Tcl commands:

```
set script {
    puts "logging now"
    lappend $myCurrentLogVar
}
set myCurrentLogVar log1
```

coreTools Command Reference Index

```
# Set up a switch of logging variable part way through!  
after 20000 set myCurrentLogVar log2
```

```
for {set i 0} {$i<10} {incr i} {  
  # Introduce a random delay  
  after [expr {int(5000 * rand())}]  
  update    ;# Check for the asynch log switch  
  eval $script $i [clock clicks]  
}
```

Note that in the most common case (where the script fragment is actually just a list of words forming a command prefix), it is better to use **{*}\$script** when doing this sort of invocation pattern. It is less general than the **eval** command, and hence easier to make robust in practice. The following procedure acts in a way that is analogous to the **lappend** command, except it inserts the argument values at the start of the list in the variable:

```
proc lprepend {varName args} {  
  upvar 1 $varName var  
  # Ensure that the variable exists and contains a list  
  lappend var  
  # Now we insert all the arguments in one go  
  set var [eval [list linsert $var 0] $args]  
}
```

However, the last line would now normally be written without **eval**, like this:

```
set var [linsert $var 0 {*}$args]
```

SEE ALSO

catch(n), concat(n), error(n), errorCode(n),
errorInfo(n), interp(n), list(n), namespace(n),
subst(n), uplevel(n)

KEYWORDS

concatenate, evaluate, script

`eval_in_component`
evaluates the code in component context

Syntax

```
string eval_in_component [-notify] [-source] <component name> <code>
    string <component name>
    string <code>
```

Parameters

<code>-notify</code>	Send notifications about KB and design changes.
<code>-source</code>	Source component plugin procs before code
<code><component name></code>	
<code><code></code>	code to be evaluated.

Description

This command is used to safely evaluate a chunk of TCL code within the context of the specified component. The following are functionality equivalent provided the `<code>` does not cause a TCL error.

```
set currentComponent [get_current_component]
set_current_component i_ahb
set name [get_current_component]
set_current_component $currentComponent
eval_in_component i_ahb {
set name [get_current_component]
}
```

Using `eval_in_component` is clearer, more concise, and less error prone. In the first code fragment if there is a TCL error while processing commands inside the component, the code will terminate without restoring the prior component context.

Note that you cannot use the TCL 'break' command within the code body if the break is for a loop initiated outside of the `eval_in_component` call. If the loop being broken is inside the code body, then it is okay. See the final examples below for details.

Examples

Get the name of the design associated with instance `i_ahb`:

```
eval_in_component i_ahb {
```

```
set name [get_top_design_name]
}
```

This is a legal use of 'break' within eval_in_component:

```
eval_in_component i_ahb {
for {set i 0} {$i < 10} {incr i} {
if {[expr {$i % 2}]} {
break
}
}
}
```

This is an illegal use of break since the break is for a loop initiated outside of eval_in_component:

```
for {set i 0} {$i < 10} {incr i} {
eval_in_component i_ahb$i {
if {[expr {$i % 2}]} {
break
}
}
}
```

See Also

set_current_component(2), get_current_component(2),
instantiate_component(2), create_hierarchical_component(2)

[API Index](#) [Command Reference](#)

eval_ipxact_expr

evaluate a SV expression from ipxact file.

Syntax

```
string eval_ipxact_expr [-init <filename>] [-clear] [<sv_expression>]  
    string <filename>  
    string <sv_expression>
```

Parameters

-init <filename>	read file containing table of parameter and values and cache it.
-clear	clear parameter cache.
<sv_expression>	SV expression to be evaluated.

Description

Examples

See Also

eval_param

Evaluate the given parameter expression.

Syntax

```
string eval_param [-context <evaluation context>] <string>
      string <evaluation context>
      string <string>
```

Parameters

-context	Context in which to evaluate the expression.
<evaluation context>	The context can be anything that has parameters associated with it (design, activity, strategy, or formula). The default context is current_design.
<string>	Expression to be evaluated.

Description

The eval_param command evaluates a parameter expression in the specified context. For example, you can evaluate an expression that involves parameters associated with the current_design. Or, you can evaluate an expression that involves parameters associated with an activity.

By default, eval_param evaluates the parameter expression in the context of the current_design (the expression involves a design parameter). By using the -context option, you can evaluate a parameter expression in another context. For example, if you specify the name of an activity, you can use eval_param to evaluate a parameter expression that involves an activity parameter.

Examples

To get the value of the serial parameter on the current_design:

```
coreConsultant> eval_param @serial
2
```

To determine if the value of the serial parameter is equal to 2 on the current_design:

```
coreConsultant> eval_param @serial==2
1
```

To determine whether either DoCompletenessCheck or DoConsistencyCheck is true on the Verify Budgets activity:

```
coreConsultant> eval_param -context VerifyBudgets  
{@DoCompletenessCheck||@DoConsistencyCheck}  
1
```

To determine if the value of the StopAtGTECH parameter on the DC_quick_map_strategy is equal to 1:

```
coreConsultant> eval_param -context DC_quick_map_strategy  
{@StopAtGTECH==1}  
0
```

See Also

report_activity_parameters (2), get_activity_parameter (2),
get_strategy_parameter

NAME

`exec` - Invoke subprocesses

SYNOPSIS

`exec` *?switches?* *arg* *?arg ...? ?&?*

DESCRIPTION

This command treats its arguments as the specification of one or more subprocesses to execute. The arguments take the form of a standard shell pipeline where each *arg* becomes one word of a command, and each distinct command becomes a subprocess.

If the initial arguments to **exec** start with `-` then they are treated as command-line switches and are not part of the pipeline specification. The following switches are currently supported:

-ignorestderr

Stops the **exec** command from treating the output of messages to the pipeline's standard error channel as an error case.

-keepnewline Retains a trailing newline in the pipeline's output. Normally a trailing newline will be deleted.

-- Marks the end of switches. The argument following this one will be treated as the first *arg* even if it starts with a `-`.

If an *arg* (or pair of *args*) has one of the forms described below then it is used by **exec** to control the flow of input and output among the subprocess(es). Such arguments will not be passed to the

coreTools Command Reference Index

subprocess(es). In forms such as *fileName* may either be in a separate argument from or in the same argument with no intervening space (i.e.

	Separates distinct commands in the pipeline. The standard output of the preceding command will be piped into the standard input of the next command.
&	Separates distinct commands in the pipeline. Both standard output and standard error of the preceding command will be piped into the standard input of the next command. This form of redirection overrides forms such as 2> and >&.
< <i>fileName</i>	The file named by <i>fileName</i> is opened and used as the standard input for the first command in the pipeline.
<@ <i>fileId</i>	<i>FileId</i> must be the identifier for an open file, such as the return value from a previous call to open . It is used as the standard input for the first command in the pipeline. <i>FileId</i> must have been opened for reading.
<< <i>value</i>	<i>Value</i> is passed to the first command as its standard input.
> <i>fileName</i>	Standard output from the last command is redirected to the file named <i>fileName</i> , overwriting its previous contents.
2> <i>fileName</i>	Standard error from all commands in the pipeline is redirected to the file named <i>fileName</i> , overwriting its previous contents.
>& <i>fileName</i>	Both standard output from the last command and standard error from all commands are redirected to the file named <i>fileName</i> , overwriting its previous contents.
>> <i>fileName</i>	Standard output from the last command is redirected to the file named <i>fileName</i> , appending to it rather than overwriting it.
2>> <i>fileName</i>	Standard error from all commands in the pipeline is redirected to the file named <i>fileName</i> , appending to it rather than overwriting it.
>>& <i>fileName</i>	Both standard output from the last command and standard error from all commands are redirected to the file named <i>fileName</i> , appending to it rather than overwriting it.

coreTools Command Reference Index

<code>>@ fileId</code>	<i>FileId</i> must be the identifier for an open file, such as the return value from a previous call to open . Standard output from the last command is redirected to <i>fileId</i> 's file, which must have been opened for writing.
<code>2>@ fileId</code>	<i>FileId</i> must be the identifier for an open file, such as the return value from a previous call to open . Standard error from all commands in the pipeline is redirected to <i>fileId</i> 's file. The file must have been opened for writing.
<code>2>@1</code>	Standard error from all commands in the pipeline is redirected to the command result. This operator is only valid at the end of the command pipeline.
<code>>&@ fileId</code>	<i>FileId</i> must be the identifier for an open file, such as the return value from a previous call to open . Both standard output from the last command and standard error from all commands are redirected to <i>fileId</i> 's file. The file must have been opened for writing.

If standard output has not been redirected then the **exec** command returns the standard output from the last command in the pipeline, unless was specified, in which case standard error is included as well. If any of the commands in the pipeline exit abnormally or are killed or suspended, then **exec** will return an error and the error message will include the pipeline's output followed by error messages describing the abnormal terminations; the **-errorcode** return option will contain additional information about the last abnormal termination encountered. If any of the commands writes to its standard error file and that standard error is not redirected and **-ignorestderr** is not specified, then **exec** will return an error; the error message will include the pipeline's standard output, followed by messages about abnormal terminations (if any), followed by the standard error output.

If the last character of the result or error message is a newline then that character is normally deleted from the result or error message. This is consistent with other Tcl return values, which do not normally end with newlines. However, if **-keepnewline** is specified then the trailing newline is retained.

If standard input is not redirected with or then the standard input for the first command in the pipeline is taken from the application's current standard input.

If the last *arg* is then the pipeline will be executed in background. In this case the **exec** command will return a list whose elements are the process identifiers for all of the subprocesses in the pipeline. The standard output from the last command in the pipeline will go to the application's standard

coreTools Command Reference Index

output if it has not been redirected, and error output from all of the commands in the pipeline will go to the application's standard error file unless redirected.

The first word in each command is taken as the command name; tilde-substitution is performed on it, and if the result contains no slashes then the directories in the PATH environment variable are searched for an executable by the given name. If the name contains a slash then it must refer to an executable reachable from the current directory. No expansion or other shell-like substitutions are performed on the arguments to commands.

PORTABILITY ISSUES

Windows (all versions)

Reading from or writing to a socket, using the notation, does not work. When reading from a socket, a 16-bit DOS application will hang and a 32-bit application will return immediately with end-of-file. When either type of application writes to a socket, the information is instead sent to the console, if one is present, or is discarded.

Note that the current escape resp. quoting of arguments for windows works only with executables using CommandLineToArgv, CRT-library or similar, as well as with the windows batch files (excepting the newline, see below). Although it is the common escape algorithm, but, in fact, the way how the executable parses the command-line (resp. splits it into single arguments) is decisive.

Unfortunately, there is currently no way to supply newline character within an argument to the batch files (.cmd or .bat) or to the command processor (cmd.exe /c), because this causes truncation of command-line (also the argument chain) on the first newline character. But it works properly with an executable (using CommandLineToArgv, etc).

The Tk console text widget does not provide real standard IO capabilities. Under Tk, when redirecting from standard input, all applications will see an immediate end-of-file; information redirected to standard output or standard error will be discarded. Either forward or backward slashes are accepted as path separators for arguments to Tcl commands. When executing an application, the path name specified for the application may also contain forward or backward slashes as path separators. Bear in mind, however, that most Windows applications accept arguments with forward slashes only as option delimiters and backslashes only in paths. Any arguments to an application that specify a path name with forward slashes will not automatically be converted to use the backslash character. If an argument contains forward slashes as the path separator, it may or may not be

coreTools Command Reference Index

recognized as a path name, depending on the program.

Two or more forward or backward slashes in a row in a path refer to a network path. For example, a simple concatenation of the root directory **c:/** with a subdirectory **/windows/system** will yield **c://windows/system** (two slashes together), which refers to the mount point called **system** on the machine called **windows** (and the **c:/** is ignored), and is not equivalent to **c:/windows/system**, which describes a directory on the current computer. The **file join** command should be used to concatenate path components.

Note that there are two general types of Win32 console applications:

- [1] CLI -- CommandLine Interface, simple stdio exchange. **netstat.exe** for example.
- [2] TUI -- Textmode User Interface, any application that accesses the console API for doing such things as cursor movement, setting text color, detecting key presses and mouse movement, etc. An example would be **telnet.exe** from Windows 2000. These types of applications are not common in a windows environment, but do exist.

exec will not work well with TUI applications when a console is not present, as is done when launching applications under **wish**. It is desirable to have console applications hidden and detached. This is a designed-in limitation as **exec** wants to communicate over pipes. The Expect extension addresses this issue when communicating with a TUI application.

When attempting to execute an application, **exec** first searches for the name as it was specified. Then, in order, **.com**, **.exe**, **.bat** and **.cmd** are appended to the end of the specified name and it searches for the longer name. If a directory name was not specified as part of the application name, the following directories are automatically searched in order when attempting to locate the application:

- o The directory from which the Tcl executable was loaded.
- o The current directory.
- o The Windows 32-bit system directory.
- o The Windows home directory.
- o The directories listed in the path.

In order to execute shell built-in commands like **dir** and **copy**, the caller must prepend the desired command with **because** built-in commands are not implemented using executables.

coreTools Command Reference Index

Unix (including Mac OS X)

The **exec** command is fully functional and works as described.

UNIX EXAMPLES

Here are some examples of the use of the **exec** command on Unix. To execute a simple program and get its result:

```
exec uname -a
```

WORKING WITH NON-ZERO RESULTS

To execute a program that can return a non-zero result, you should wrap the call to **exec** in **catch** and check the contents of the **-errorcode** return option if you have an error:

```
set status 0
if {[catch {exec grep foo bar.txt} results options]} {
    set details [dict get $options -errorcode]
    if {[lindex $details 0] eq "CHILDSTATUS"} {
        set status [lindex $details 2]
    } else {
        # Some other error; regenerate it to let caller handle
        return -options $options -level 0 $results
    }
}
```

This is more easily written using the **try** command, as that makes it simpler to trap specific types of errors. This is done using code like this:

```
try {
    set results [exec grep foo bar.txt]
    set status 0
} trap CHILDSTATUS {results options} {
    set status [lindex [dict get $options -errorcode] 2]
}
```

WORKING WITH QUOTED ARGUMENTS

When translating a command from a Unix shell invocation, care should be taken over the fact that single quote characters have no special significance to Tcl. Thus:

```
awk '{sum += $1} END {print sum}' numbers.list
```

would be translated into something like:

```
exec awk {{sum += $1} END {print sum}} numbers.list
```

WORKING WITH GLOBBING

If you are converting invocations involving shell globbing, you should remember that Tcl does not handle globbing or expand things into multiple arguments by default. Instead you should write things like this:

```
exec ls -l {[*]}[glob *.tcl]
```

WORKING WITH USER-SUPPLIED SHELL SCRIPT FRAGMENTS

One useful technique can be to expose to users of a script the ability to specify a fragment of shell script to execute that will have some data passed in on standard input that was produced by the Tcl program. This is a common technique for using the *lpr* program for printing. By far the simplest way of doing this is to pass the user's script to the user's shell for processing, as this avoids a lot of complexity with parsing other languages.

```
set lprScript [get from user...]
set postscriptData [generate somehow...]

exec $env(SHELL) -c $lprScript << $postscriptData
```

WINDOWS EXAMPLES

Here are some examples of the use of the **exec** command on Windows. To start an instance of *notepad* editing a file without waiting for the user to finish editing the file:

```
exec notepad myfile.txt &
```

To print a text file using *notepad*:

```
exec notepad /p myfile.txt
```

WORKING WITH CONSOLE PROGRAMS

If a program calls other programs, such as is common with compilers, then you may need to resort to batch files to hide the console windows that sometimes pop up:

```
exec cmp.bat somefile.c -o somefile
```

With the file *cmp.bat* looking something like:

```
@gcc %*
or like another variant using single parameters:
@gcc %1 %2 %3 %4 %5 %6 %7 %8 %9
```

WORKING WITH COMMAND BUILT-INS

Sometimes you need to be careful, as different programs may have the same name and be in the path. It can then happen that typing a command at the DOS prompt finds a *different program* than the same command run via **exec**. This is because of the (documented) differences in behaviour between **exec** and DOS batch files.

When in doubt, use the command **auto_execok**: it will return the complete path to the program as seen by the **exec** command. This applies especially when you want to run commands like *dir* from a Tcl script (if you just want to list filenames, use the **glob** command.) To do that, use this:

coreTools Command Reference Index

```
exec {*}[auto_execok dir] *.tcl
```

WORKING WITH NATIVE FILENAMES

Many programs on Windows require filename arguments to be passed in with backslashes as pathname separators. This is done with the help of the **file nativename** command. For example, to make a directory (on NTFS) encrypted so that only the current user can access it requires use of the *CIPHER* command, like this:

```
set secureDir "~/Desktop/Secure Directory"  
file mkdir $secureDir  
exec CIPHER /e /s:[file nativename $secureDir]
```

SEE ALSO

error(n), file(n), open(n)

KEYWORDS

execute, pipeline, redirection, subprocess

NAME

`exit` - End the application

SYNOPSIS

`exit ?returnCode?`

DESCRIPTION

Terminate the process, returning *returnCode* to the system as the exit status. If *returnCode* is not specified then it defaults to 0.

EXAMPLE

Since non-zero exit codes are usually interpreted as error cases by the calling process, the **exit** command is an important part of signaling that something fatal has gone wrong. This code fragment is useful in scripts to act as a general problem trap:

```
proc main {} {
    # ... put the real main code in here ...
}

if {[catch {main} msg options]} {
    puts stderr "unexpected script error: $msg"
    if {[info exists env(DEBUG)]} {
        puts stderr "---- BEGIN TRACE ----"
        puts stderr [dict get $options -errorinfo]
        puts stderr "---- END TRACE ----"
    }
}
```



```
# Reserve code 1 for "expected" error exits...  
exit 2  
}
```

SEE ALSO

`exec(n)`

KEYWORDS

abort, exit, process

explicit_capacitance

Formula to specify an explicit capacitance value for PinLoadType

Syntax

```
string explicit_capacitance cap_value  
      string cap_value
```

Parameters

cap_value The capacitance value, either with or without units.

Description

explicit_capacitance is a formula command that you can use to specify the value for the PinLoadType attribute as an explicit capacitance value, instead of using a library cell or one of the library cell selection formulas.

If you specify PinLoadType as an explicit capacitance, coreConsultant uses the specified capacitance value as a value for the set_load constraint on that port.

The default capacitance unit, if you do not specify a unit, is the capacitance unit used by the currently loaded technology library (as stored in the current_capacitance_unit variable). If you do specify a capacitance unit, the unit must be either pf or ff. In either case, when a different technology library is loaded, the coreTool automatically scales the capacitance value to the unit used by the new library.

For example, if a core developer specifies an explicit_capacitance value of 10pf, and a core integrator is using a technology library that uses ff as the capacitance unit, coreConsultant automatically scales the 10pf value to 10,000ff.

The pin_cap_of command returns the capacitance of a specific pin and can be used as the argument to explicit_capacitance.

Examples

To set PinLoadType on myPort to an explicit capacitance value of 10 (capacitance units of the currently loaded technology library):

```
coreConsultant> set_port_attribute myPort PinLoadType  
{=explicit_capacitance 10}
```

To set PinLoadType on myPort to an explicit capacitance value which is the capacitance of pin A on an ND2:

```
coreConsultant> set_port_attribute myPort PinLoadType  
{=explicit_capacitance [pin_cap_of MY_LIB/ND2/A]}
```

See Also

set_port_attribute (2), select_by_class (2), select_by_function (2), _____
select_by_name (2), pin_cap_of (2), PinLoadType (3)_____

`export_all_interfaces`

Export all interfaces in the current context that are set up for auto-export.

Syntax

string **export_all_interfaces** [-totop]

Parameters

-totop Export all interfaces to the top level component.

Description

Use this command to export all interfaces out of the subsystem that are set up for auto-export. This is a fast way to prepare a subsystem so that you can quickly complete the Add Subsystem Activity without having to individually export each unconnected interface.

Examples

`export_all_interfaces`

See Also

[export_interface \(2\)](#), _____

export_interface

Export a component interface out of the subsystem.

Syntax

```
string export_interface -name <newName> -component <componentName>
-interface <instanceName> [-copy] [-monitor] [-format <format>]
[-channel] [-noauto]
    string <newName>
    string <componentName>
    string <instanceName>
    string <format>
```

Parameters

-name <newName>	New exported interface instance name
-component <componentName>	The component from which the interface is exported
-interface <instanceName>	The opposite instance for the new exported instance
-copy	Copy interface parameter values to newly exported interface
-monitor	Export as consumer-monitor or interface-monitor
-format <format>	The format to use for prefixing exported interfaces.
-channel	Symmetric SPIRIT interface to be connected to a channel
-noauto	Don't run autoconnect_interface

Description

Use this command to export an interface out of the subsystem. Exporting the interface implies that an opposite component with this interface is going to exist outside of the subsystem. Therefore these "exported interfaces" are sometimes referenced as "system interfaces", in contrast to the component interfaces: The opposite side of the component interface is not another component interface but a system component outside the subsystem.

You configure the exported interface, and coreAssembler connects them, in a fashion similar to interfaces within the subsystem. Ports of the exported interface are automatically created as top-level ports of the subsystem design.

coreTools Command Reference Index

After creating the new exported interface ("system or subsystem interface") the exported interface and the component interface are connected.

Please note that exported interfaces are siblings of attached interfaces: You can treat exported interfaces like interfaces which are attached to "the system". Consequently, exported and attached interfaces share the name space: they must have a unique name (option -name) even if attached interfaces are addressed like component interfaces.

The exported interface is always the opposite interface to the component interface:

- A component provider exports as consumer.
- A component consumer or internal-consumer exports as provider.
- A component consumer-monitor exports as consumer.
- A component interface-monitor exports as provider.

If option -monitor is used then the exported interface becomes a monitor interface:

- A component provider with option -monitor exports as interface-monitor.
- A component consumer with option -monitor exports as consumer-monitor.

It is an error if option -monitor is used for other component interface types.

The impact of option -prefix requires a bit of background information: Subsystem ports which are separate per interface are always prefixed with the exported interface name. If option -prefix is selected then all port names use the prefix, even the 'common' ports. Use this option when:

- you know that there will be a single interface of this kind only, and you like to have a consistent prefix for all ports of this interface; or
- you have two exported interfaces for different providers of the same kind which would cause an unintended common port name (-prefix required).

This option provides better names but can impact synthesis results. Avoid the prefix when you would finally have multiple exported interfaces and a single provider only: because of different names this will cause multiple output ports but synthesis does not optimize well these multiple outputs driven by the same source.

Hint: At any time you can change the subsystem port names in the 'Port Association' dialog, if your decision now does not match final subsystem requirement, but this requires more manual work.

The -channel option is used only in conjunction with symmetric busses as defined by the SPIRIT consortium. If you are exporting a SPIRIT master interface of a symmetric bus and expect to connect it to a mirroredMaster interface instead of a slave interface, the -channel option must be used. Otherwise coreAssembler assumes that symmetric SPIRIT interfaces represent direct master to slave connections.

Examples

To export the clock interface when the 'provider' of the clock exists outside the subsystem usually the clock generator is not part of the subsystem:

```
export_interface -name SysClk \
-component theAHB \
-interface AHB_Clock
```

See Also

[remove_exported_interface \(2\), instantiate_interface \(2\),](#)
[attach_interface \(2\), instantiate_component \(2\), connect_interface \(2\)](#)

export_open_pins

Export pins currently connected to open

Syntax

string **export_open_pins** [-components <instances>] [-all] [-format <format>]

string <instances>

string <format>

Parameters

-components <instances>	Component names to export from.
-all	Do not restrict to connections made via interfaces.
-format <format>	Port name format.

Description

This command is used to export pins that are currently connected to 'open'. This should only be required in special cases where there is no control over the previous connection, such as when it was made due to an interface connection.

When invoked without arguments, the command will export all pins that were fully connected to 'open' because of interface connection. This is the case when there are output pins involved in an interface where the corresponding input on the other side of the interface is not defined. If involved with the -all option, then all 'open' pins are processed. To further restrict the behavior, one or more component names can be specified via -component to ensure that only 'open' pins on the specified component(s) are exported.

Examples

Export all pins from the u_phy instance that are 'open' due to interface connections:

```
prompt> export_open_pins -component u_phy
Export all pins that are currently connected to 'open'
```

```
prompt> export_open_pins -all
```


See Also

export_pin

Export the pin to a port on the subsystem.

Syntax

```
string export_pin -port <name> [-auto_dim] [-dimensions <bit_range>]
[-signal <signal name>] [-comment <text>] [-hierarchy] [-hier_port]
[-preserve] [-quiet] pin
    string <name>
    string <bit_range>
    string <signal name>
    string <text>
    string pin
```

Parameters

-port <name>	Name of the port to create and connect to with optional bit range
-auto_dim	Automatically determine port dimensions from pin
-dimensions <bit_range>	Explicitly specify the dimensions
-signal <signal name>	Signal name for the given connection
-comment <text>	Comment associated with the given connection
-hierarchy	This connection crosses hierarchical boundaries.
-hier_port	Use -port <name> for all intermediate ports if possible.
-preserve	Preserve all unspecified dimensions in exported port definition.
-quiet	Don't print status messages
pin	Pin to export with optional bit range By default, the entire pin is exported, unless the pin name includes a bit range, in which case only the specified range is exported. The first character of the pin may be '!', in which case the pin is connected to the port via an inverter.

Description

This command is used in batch mode to create a top-level subsystem port and connect a pin on a subsystem component to the port. By default, the dimensions of the port will match those of the pin, unless only a portion of the pin is being exported, or unless specific dimensions are specified

for the port using the `-dimensions` option. If the port already exists, then this command is equivalent to a `create_connection` command.

Ports created by the `export_pin` command are automatically removed when disconnected. To keep the ports used the `-keep_ports` option with the `remove_connection` command.

The `-signal` option can be used to control the name of the signals created in the generated RTL. Note that the names are only used when a signal is required to be created. The `-comment` option can be used to associate a comment with a particular connection. The comment will appear in the generated RTL just above the pin to which the comment was associated.

When the `-hier_port` option is used with the `-hierarchy` option to export a pin across multiple levels of hierarchy, any intermediate ports created will be named based on the name passed to the `-port` option. Otherwise the intermediate ports will be named using the exported port naming format.

Examples

To export two bits of a 4 bit pin:

```
coreAssembler> export_pin -auto_dim -port data U1/Z[0:2]
```

To export a single bit pin to port that is a bus. Note that `create_connection` can be used here too.

```
coreAssembler> export_pin -dimensions 0:2 -port data[0] U2/A
```

```
coreAssembler> export_pin -port data[1] U3/A
```

```
coreAssembler> create_connection {data[2] U4/A}
```

See Also

[create_connection\(2\)](#), [remove_connection\(2\)](#), [escaped_name\(2\)](#), [translate_interface_connection\(2\)](#)

export_port_descriptions

Export port description attributes up one level of hierarchy.

Syntax

```
string export_port_descriptions [-attributes <attributes>]  
    string <attributes>
```

Parameters

-attributes <attributes>

Attributes to be propagated.

Description

This command is used to push documentation associated with ports up one level of hierarchy. This is typically used to push IP level documentation up the hierarchy in coreAssembler. This allows for generation of I/O documentation for a subsystem. By default the following attributes are propagated: DocGroup DocMaster DocDescription DocRangeDecoratedName DocDescriptionField GroupText GroupImage. To propagate a different set of attributes, pass the desired list of attributes using the optional -attributes argument.

Examples

Propagate documentation attributes up one level of hierarchy:

```
export_port_descriptions
```

Propagate only the Description and GroupText attributes:

```
export_port_descriptions -attributes {Description GroupText}
```

See Also

`export_unconnected_interfaces`

Export unconnected interfaces in the current hierarchy.

Syntax

string **export_unconnected_interfaces** [-exclusions *<port names>*] [-format *<format>*]

string *<port names>*

string *<format>*

Parameters

-exclusions *<port names>*

Interface ports to be mapped by user

-format *<format>*

Format string for created ports.

Description

This command is used to export all currently unconnected interfaces in the current context. The InterfaceLink values in the exported interfaces are set up based on the -format option, which defaults to "%p" if not specified. This means the port name created by the exported interface will match the port name in the component of the exported interface. For other naming strategies, use the -format option. The format can include fixed text as well as p (replaced by port name), d (replaced by port direction: i, o, b), or D (replaced by port direction I, O, B).

If you want to specify some of the link values yourself, use the -exclusions option to exclude interface ports from having InterfaceLink updated. Entries in the -exclusions list should have the format *<instance>/<interface>/<interface port>*. For example, *i_ahb/AHB_Slave/haddr*.

Examples

Export all currently unconnected interfaces, naming the ports based on the component port name, but starting with a port direction:

```
export_unconnected_interfaces -format {%d_%p}
```

See Also

`export_unconnected_pins`

Export unconnected pins in the current hierarchy.

Syntax

```
string export_unconnected_pins [-partial] [-format <format>]  
    string <format>
```

Parameters

<code>-partial</code>	Export partially connected pins
<code>-format <format></code>	Format string for naming created ports.

Description

This command is used to export all unconnected pins in the current context. By default, only fully unconnected pins are exported. Specify the `-partial` option to add partially connected pins to the list to be processed. The `-format` option is used to specify the naming for created ports. The default format is `"%p"` which means the exported port name will match the name of the port associated with the pin (i.e. `U1/data_out` would be exported with port name `data_out`). The format string can also contain fixed text, and the format specifiers `d` and `D`, which are replaced by the port directionality (`i`, `o`, `b`) or (`I`, `O`, `B`), respectively.

Examples

Export all unconnected pins, adding a suffix containing the port directionality to the port name:

```
export_unconnected_pins -format "%p_%d"
```

See Also

[connect or export unconnected pins \(2\)](#)

`export_workspace_as_component`
Export the workspace as a component.

Syntax

```
string export_workspace_as_component [-directory <directory>] [-tarfile]
[-copy_installs]
    string <directory>
```

Parameters

<code>-directory</code> <code><directory></code>	The directory path to export to. Defaults to the workspace export directory under the directory <code>workspace_component</code> .
<code>-tarfile</code>	Export the workspace as a component compressed tar file.
<code>-copy_installs</code>	Copy all component installation areas into the exported workspace.

Description

`coreAssembler` allows the export of a component workspace file set or compressed tar file that can then be used as a component with the `import_workspace_as_component` or `replace_component` commands.

Examples

```
coreAssembler> export_workspace_as_component
coreAssembler> import_workspace_as_component
-name i_ws1
-file
./Subsystem1/export/workspace_component/Subsystem1
coreAssembler> export_workspace_as_component
-directory ./Subsystem1.tar.gz
coreAssembler> import_workspace_as_component
-name i_ws1
-file ./Subsystem1.tar.gz
coreAssembler> replace_component
-name i_ws1
./Subsystem1.tar.gz
```

See Also

[import_workspace_as_component \(2\)](#), [replace_component \(2\)](#)

NAME

`expr` - Evaluate an expression

SYNOPSIS

`expr arg ?arg arg ...?`

DESCRIPTION

Concatenates *args* (adding separator spaces between them), evaluates the result as a Tcl expression, and returns the value. The operators permitted in Tcl expressions include a subset of the operators permitted in C expressions. For those operators common to both Tcl and C, Tcl applies the same meaning and precedence as the corresponding C operators. Expressions almost always yield numeric results (integer or floating-point values). For example, the expression

`expr 8.2 + 6`

evaluates to 14.2. Tcl expressions differ from C expressions in the way that operands are specified. Also, Tcl expressions support non-numeric operands and string comparisons, as well as some additional operators not found in C.

OPERANDS

A Tcl expression consists of a combination of operands, operators, parentheses and commas. White space may be used between the operands and operators and parentheses (or commas); it is ignored by the expression's instructions. Where possible, operands are interpreted as integer values. Integer values may be specified in decimal (the normal case), in binary (if the first two characters of the operand are **0b**), in octal (if the first two characters of the operand are **0o**), or in

coreTools Command Reference Index

hexadecimal (if the first two characters of the operand are **0x**). For compatibility with older Tcl releases, an octal integer value is also indicated simply when the first character of the operand is **0**, whether or not the second character is also **o**. If an operand does not have one of the integer formats given above, then it is treated as a floating-point number if that is possible. Floating-point numbers may be specified in any of several common formats making use of the decimal digits, the decimal point **.**, the characters **e** or **E** indicating scientific notation, and the sign characters **+** or **-**. For example, all of the following are valid floating-point numbers: 2.1, 3., 6e4, 7.91e+16. Also recognized as floating point values are the strings **Inf** and **NaN** making use of any case for each character. If no numeric interpretation is possible (note that all literal operands that are not numeric or boolean must be quoted with either braces or with double quotes), then an operand is left as a string (and only a limited set of operators may be applied to it).

Operands may be specified in any of the following ways:

- [1] As a numeric value, either integer or floating-point.
- [2] As a boolean value, using any form understood by **string is boolean**.
- [3] As a Tcl variable, using standard **\$** notation. The variable's value will be used as the operand.
- [4] As a string enclosed in double-quotes. The expression parser will perform backslash, variable, and command substitutions on the information between the quotes, and use the resulting value as the operand
- [5] As a string enclosed in braces. The characters between the open brace and matching close brace will be used as the operand without any substitutions.
- [6] As a Tcl command enclosed in brackets. The command will be executed and its result will be used as the operand.
- [7] As a mathematical function whose arguments have any of the above forms for operands, such as **sin(\$x)**. See **MATH FUNCTIONS** below for a discussion of how mathematical functions are handled.

Where the above substitutions occur (e.g. inside quoted strings), they are performed by the expression's instructions. However, the command parser may already have performed one round of substitution before the expression processor was called. As discussed below, it is usually best to enclose expressions in braces to prevent the command parser from performing

coreTools Command Reference Index

substitutions on the contents.

For some examples of simple expressions, suppose the variable **a** has the value 3 and the variable **b** has the value 6. Then the command on the left side of each of the lines below will produce the value on the right side of the line:

expr {3.1 + \$a}	6.1
expr {2 + "\$a.\$b"}	5.6
expr {4*[llength "6 2"]}	8
expr {{word one} < "word \$a"}	0

OPERATORS

The valid operators (most of which are also available as commands in the **tcl::mathop** namespace; see the **mathop(n)** manual page for details) are listed below, grouped in decreasing order of precedence:

- + ~ !	Unary minus, unary plus, bit-wise NOT, logical NOT. None of these operators may be applied to string operands, and bit-wise NOT may be applied only to integers.
**	Exponentiation. Valid for any numeric operands. The maximum exponent value that Tcl can handle if the first number is an integer > 1 is 268435455.
* / %	Multiply, divide, remainder. None of these operators may be applied to string operands, and remainder may be applied only to integers. The remainder will always have the same sign as the divisor and an absolute value smaller than the absolute value of the divisor. When applied to integers, the division and remainder operators can be considered to partition the number line into a sequence of equal-sized adjacent non-overlapping pieces where each piece is the size of the divisor; the division result identifies which piece the divisor lay within, and the remainder result identifies where within that piece the divisor lay. A consequence of this is that the result of is always -6, and the result of is always 3.
+ -	Add and subtract. Valid for any numeric operands.
<< >>	Left and right shift. Valid for integer operands only. A right shift always propagates the sign bit.

coreTools Command Reference Index

< > <= >=	Boolean less, greater, less than or equal, and greater than or equal. Each operator produces 1 if the condition is true, 0 otherwise. These operators may be applied to strings as well as numeric operands, in which case string comparison is used.
== !=	Boolean equal and not equal. Each operator produces a zero/one result. Valid for all operand types.
eq ne	Boolean string equal and string not equal. Each operator produces a zero/one result. The operand types are interpreted only as strings.
in ni	List containment and negated list containment. Each operator produces a zero/one result and treats its first argument as a string and its second argument as a Tcl list. The in operator indicates whether the first argument is a member of the second argument list; the ni operator inverts the sense of the result.
&	Bit-wise AND. Valid for integer operands only.
^	Bit-wise exclusive OR. Valid for integer operands only.
 	Bit-wise OR. Valid for integer operands only.
&&	Logical AND. Produces a 1 result if both operands are non-zero, 0 otherwise. Valid for boolean and numeric (integers or floating-point) operands only. This operator evaluates lazily; it only evaluates its second operand if it must in order to determine its result.
 	Logical OR. Produces a 0 result if both operands are zero, 1 otherwise. Valid for boolean and numeric (integers or floating-point) operands only. This operator evaluates lazily; it only evaluates its second operand if it must in order to determine its result.
x ? y : z	If-then-else, as in C. If x evaluates to non-zero, then the

coreTools Command Reference Index

result is the value of **y**.
Otherwise the result is the value
of **z**. The **x** operand must have a
boolean or numeric value. This
operator evaluates lazily; it
evaluates only one of **y** or **z**.

See the C manual for more details on the results
produced by each operator. The exponentiation operator
promotes types like the multiply and divide operators,
and produces a result that is the same as the output of
the **pow** function (after any type conversions.) All of
the binary operators but exponentiation group left-to-
right within the same precedence level; exponentiation
groups right-to-left. For example, the command

```
expr {4*2 < 7}
```

returns 0, while

```
expr {2**3**2}
```

returns 512.

The **&&**, **||**, and **?:** operators have just as in C, which
means that operands are not evaluated if they are not
needed to determine the outcome. For example, in the
command

```
expr {$v?[a]:[b]}
```

only one of or will actually be evaluated, depending on
the value of **\$v**. Note, however, that this is only true
if the entire expression is enclosed in braces;
otherwise the Tcl parser will evaluate both and before
invoking the **expr** command.

MATH FUNCTIONS

When the expression parser encounters a mathematical
function such as **sin(\$x)**, it replaces it with a call to
an ordinary Tcl command in the **tcl::mathfunc** namespace.
The processing of an expression such as:

```
expr {sin($x+$y)}
```

is the same in every way as the processing of:

```
expr {[tcl::mathfunc::sin [expr {$x+$y}]]}
```

which in turn is the same as the processing of:

```
tcl::mathfunc::sin [expr {$x+$y}]
```

The executor will search for **tcl::mathfunc::sin** using
the usual rules for resolving functions in namespaces.
Either **::tcl::mathfunc::sin** or **[namespace
current]::tcl::mathfunc::sin** will satisfy the request,
and others may as well (depending on the current
namespace path setting).

Some mathematical functions have several arguments,
separated by commas like in C. Thus:

```
expr {hypot($x,$y)}
```

ends up as

```
tcl::mathfunc::hypot $x $y
```

See the **mathfunc(n)** manual page for the math functions that are available by default.

TYPES, OVERFLOW, AND PRECISION

All internal computations involving integers are done calling on the LibTomMath multiple precision integer library as required so that all integer calculations are performed exactly. Note that in Tcl releases prior to 8.5, integer calculations were performed with one of the C types *long int* or *Tcl_WideInt*, causing implicit range truncation in those calculations where values overflowed the range of those types. Any code that relied on these implicit truncations will need to explicitly add **int()** or **wide()** function calls to expressions at the points where such truncation is required to take place.

All internal computations involving floating-point are done with the C type *double*. When converting a string to floating-point, exponent overflow is detected and results in the *double* value of **Inf** or **-Inf** as appropriate. Floating-point overflow and underflow are detected to the degree supported by the hardware, which is generally pretty reliable.

Conversion among internal representations for integer, floating-point, and string operands is done automatically as needed. For arithmetic computations, integers are used until some floating-point number is introduced, after which floating-point is used. For example,

```
expr {5 / 4}
```

returns 1, while

```
expr {5 / 4.0}
expr {5 / ( [string length "abcd"] + 0.0 )}
```

both return 1.25. Floating-point values are always returned with a or an so that they will not look like integer values. For example,

```
expr {20.0/5.0}
```

returns **4.0**, not **4**.

STRING OPERATIONS

String values may be used as operands of the comparison operators, although the expression evaluator tries to do comparisons as integer or floating-point when it can, i.e., when all arguments to the operator allow numeric interpretations, except in the case of the **eq** and **ne** operators. If one of the operands of a comparison is a string and the other has a numeric

coreTools Command Reference Index

value, a canonical string representation of the numeric operand value is generated to compare with the string operand. Canonical string representation for integer values is a decimal string format. Canonical string representation for floating-point values is that produced by the **%g** format specifier of Tcl's **format** command. For example, the commands

```
expr {"0x03" > "2"}  
expr {"0y" > "0x12"}
```

both return 1. The first comparison is done using integer comparison, and the second is done using string comparison. Because of Tcl's tendency to treat values as numbers whenever possible, it is not generally a good idea to use operators like **==** when you really want string comparison and the values of the operands could be arbitrary; it is better in these cases to use the **eq** or **ne** operators, or the **string** command instead.

PERFORMANCE CONSIDERATIONS

Enclose expressions in braces for the best speed and the smallest storage requirements. This allows the Tcl bytecode compiler to generate the best code.

As mentioned above, expressions are substituted twice: once by the Tcl parser and once by the **expr** command. For example, the commands

```
set a 3  
set b {$a + 2}  
expr $b*4
```

return 11, not a multiple of 4. This is because the Tcl parser will first substitute for the variable **b**, then the **expr** command will evaluate the expression

Most expressions do not require a second round of substitutions. Either they are enclosed in braces or, if not, their variable and command substitutions yield numbers or strings that do not themselves require substitutions. However, because a few unbraced expressions need two rounds of substitutions, the bytecode compiler must emit additional instructions to handle this situation. The most expensive code is required for unbraced expressions that contain command substitutions. These expressions must be implemented by generating new code each time the expression is executed.

If it is necessary to include a non-constant expression string within the wider context of an otherwise-constant expression, the most efficient technique is to put the varying part inside a recursive **expr**, as this at least allows for the compilation of the outer part, though it does mean that the varying part must itself be evaluated as a separate expression. Thus, in this example the result is 20 and the outer expression

coreTools Command Reference Index

benefits from fully cached bytecode compilation.

```
set a 3
set b {$a + 2}
expr {[expr $b] * 4}
```

When the expression is unbraced to allow the substitution of a function or operator, consider using the commands documented in the **mathfunc**(n) or **mathop**(n) manual pages directly instead.

EXAMPLES

Define a procedure that computes an mathematical function:

```
proc tcl::mathfunc::calc {x y} {
    expr { ($x**2 - $y**2) / exp($x**2 + $y**2) }
}
```

Convert polar coordinates into cartesian coordinates:

```
# convert from ($radius,$angle)
set x [expr { $radius * cos($angle) }]
set y [expr { $radius * sin($angle) }]
```

Convert cartesian coordinates into polar coordinates:

```
# convert from ($x,$y)
set radius [expr { hypot($y, $x) }]
set angle [expr { atan2($y, $x) }]
```

Print a message describing the relationship of two string values to each other:

```
puts "a and b are [expr {$a eq $b ? {equal} : {different}}]"
```

Set a variable to whether an environment variable is both defined at all and also set to a true boolean value:

```
set isTrue [expr {
    [info exists ::env(SOME_ENV_VAR)] &&
    [string is true -strict $::env(SOME_ENV_VAR)]
}]
```

Generate a random integer in the range 0..99 inclusive:

```
set randNum [expr { int(100 * rand()) }]
```

SEE ALSO

array(n), **for**(n), **if**(n), **mathfunc**(n), **mathop**(n), **namespace**(n), **proc**(n), **string**(n), **Tcl**(n), **while**(n)

KEYWORDS

arithmetic, boolean, compare, expression, fuzzy
comparison

COPYRIGHT

Copyright (C) 1993 The Regents of the University of California.
Copyright (C) 1994-2000 Sun Microsystems Incorporated.
Copyright (C) 2005 Kevin B. Kenny <kennykb@acm.org>. All rights reserved.

NAME

fblocked - Test whether the last input operation exhausted all available input

SYNOPSIS

fblocked *channelId*

DESCRIPTION

The **fblocked** command returns 1 if the most recent input operation on *channelId* returned less information than requested because all available input was exhausted. For example, if **gets** is invoked when there are only three characters available for input and no end-of-line sequence, **gets** returns an empty string and a subsequent call to **fblocked** will return 1.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension.

EXAMPLE

The **fblocked** command is particularly useful when writing network servers, as it allows you to write your code in a line-by-line style without preventing the servicing of other connections. This can be seen in this simple echo-service:

```
# This is called whenever a new client connects to the
server proc connect {chan host port} {
    set clientName [format <%s:%d> $host $port]
    puts "connection from $clientName"
    fconfigure $chan -blocking 0 -buffering line
    fileevent $chan readable [list echoLine $chan
$clientName] }
```

```
# This is called whenever either at least one byte of
input # data is available, or the channel was closed by
the client. proc echoLine {chan clientName} {
    gets $chan line
```

coreTools Command Reference Index

```
if {[eof $chan]} {  
    puts "finishing connection from $clientName"  
    close $chan  
} elseif {![fblocked $chan]} {  
    # Didn't block waiting for end-of-line  
    puts "$clientName - $line"  
    puts $chan $line  
} }
```

```
# Create the server socket and enter the event-loop to  
wait # for incoming connections... socket -server  
connect 12345 vwait forever
```

SEE ALSO

gets(n), open(n), read(n), socket(n),
Tcl_StandardChannels(3)

KEYWORDS

blocking, nonblocking

fc_shellAppOption

Sets application options to the specified value on the specified block.
Settings to be used for Fusion Compiler (fc_shell).

Definition

Type: string**Flags:** subscripted**Default value:****Valid subscripts:** <valid subscripts are not statically defined>**Default subscript:****Valid on:** design_____

Description

This subscripted attribute can be used to issue set_app_options commands in the Fusion Compiler tool each time fc_shell is run.

For application options that should be set at the start of the init_design stage, the attribute subscript indicates the application option name and the attribute value is the value to assign to the application option. For application options that should be set at points other than init_design, the insertion point must be specified as part of the attribute subscript. For example, "read_rtl-hdlin.elaborate.option" would define an application option "hdlin.elaborate.option" which is set before RTL is analyzed and elaborated. The valid insertion points are defined in the table below.

Insertion points for app options in FCRM

Insertion Point	Description
init_design	At the start of init_design stage (Data preparation)
read_rtl	Before the RTL is analyzed and elaborated
compile	At the start of compile stage (before compile_fusion is called)
place_opt	Before start of placement and optimization in the classic flow
write_data	Before the view (Verilog, SDC, OASIS) & report generation stage

Examples

Issue the command 'set_app_options -name place.coarse.continue_on_missing_scandef -value true' in the generated reference methodology scripts at the start of init_design stage:

coreTools Command Reference Index

```
coreConsultant> set_design_attribute  
{fc_shellAppOption[place.coarse.continue_on_missing_scandef]} true  
Issue the command 'set_app_options -name  
hdlin.elaborate.maximum_while_loop_iterations -value 400000' in the  
generated reference methodology scripts before RTL is analysed and  
elaborated
```

```
coreConsultant> set_design_attribute  
{fc_shellAppOption[read_rtl-hdlin.elaborate.maximum_while_loop_iterations]}  
400000
```

See Also

fc_shellAppOptionComment

Add a comment for the application options being introduced in the tool intent.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

The fc_shellAppOptionComment attribute can be used to output a comment to document why the corresponding fc_shellAppOption is being used. The comment will be put into the file where the app option is set.

Examples

```
coreConsultant> current_design [get_top_design]
coreConsultant> set_design_attribute
{fc_shellAppOptionComment[place.coarse.continue_on_missing_scandef]}
"Overriding app option to true"
coreConsultant> set_design_attribute
{fc_shellAppOption[place.coarse.continue_on_missing_scandef]} true
Generates the following code fragment in the script
# Overriding app option to true
set_app_options -name place.coarse.continue_on_missing_scandef -value
true
```

See Also

NAME

`fconfigure` - Set and get options on a channel

SYNOPSIS

```
fconfigure channelId  
fconfigure channelId name  
fconfigure channelId name value ?name value ...?
```

DESCRIPTION

The **fconfigure** command sets and retrieves options for channels.

ChannelId identifies the channel for which to set or query an option and must refer to an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension.

If no *name* or *value* arguments are supplied, the command returns a list containing alternating option names and values for the channel. If *name* is supplied but no *value* then the command returns the current value of the given option. If one or more pairs of *name* and *value* are supplied, the command sets each of the named options to the corresponding *value*; in this case the return value is an empty string.

The options described below are supported for all channels. In addition, each channel type may add options that only it supports. See the manual entry for the command that creates each type of channels for the options that that specific type of channel supports. For example, see the manual entry for the **socket** command for additional options for sockets, and the

coreTools Command Reference Index

open command for additional options for serial devices.

-blocking *boolean*

The **-blocking** option determines whether I/O operations on the channel can cause the process to block indefinitely. The value of the option must be a proper boolean value. Channels are normally in blocking mode; if a channel is placed into nonblocking mode it will affect the operation of the **gets**, **read**, **puts**, **flush**, and **close** commands by allowing them to operate asynchronously; see the documentation for those commands for details. For nonblocking mode to work correctly, the application must be using the Tcl event loop (e.g. by calling **Tcl_DoOneEvent** or invoking the **vwait** command).

-buffering *newValue*

If *newValue* is **full** then the I/O system will buffer output until its internal buffer is full or until the **flush** command is invoked. If *newValue* is **line**, then the I/O system will automatically flush output for the channel whenever a newline character is output. If *newValue* is **none**, the I/O system will flush automatically after every output operation. The default is for **-buffering** to be set to **full** except for channels that connect to terminal-like devices; for these channels the initial setting is **line**. Additionally, **stdin** and **stdout** are initially set to **line**, and **stderr** is set to **none**.

-buffersize *newSize*

Newvalue must be an integer; its value is used to set the size of buffers, in bytes, subsequently allocated for this channel to store input or output. *Newvalue* must be between one and one million, allowing buffers of one to one million bytes in size.

-encoding *name*

This option is used to specify the encoding of the channel, so that the data can be converted to and from Unicode for use in Tcl. For instance, in order for Tcl to read characters from a Japanese file in **shiftjis** and properly process and display the contents, the encoding would be set to **shiftjis**. Thereafter, when reading from the channel, the bytes in the Japanese file would be converted to Unicode as they are read. Writing is also supported - as Tcl strings are written to the channel they will automatically be converted to the specified encoding on output.

If a file contains pure binary data (for instance, a JPEG image), the encoding for the channel should be configured to be **binary**. Tcl will then assign no interpretation to the data in the file and simply read or write raw bytes. The Tcl **binary** command can be used to manipulate this byte-oriented data. It is usually better to set the **-translation** option to **binary** when you want to transfer binary data, as this turns off the other automatic interpretations of the bytes in the stream as well.

The default encoding for newly opened channels is the

coreTools Command Reference Index

same platform- and locale-dependent system encoding used for interfacing with the operating system, as returned by **encoding system**.

-eofchar *char*

-eofchar {*inChar outChar*}

This option supports DOS file systems that use Control-z (\x1A) as an end of file marker. If *char* is not an empty string, then this character signals end-of-file when it is encountered during input. For output, the end-of-file character is output when the channel is closed. If *char* is the empty string, then there is no special end of file character marker. For read-write channels, a two-element list specifies the end of file marker for input and output, respectively. As a convenience, when setting the end-of-file character for a read-write channel you can specify a single value that will apply to both reading and writing. When querying the end-of-file character of a read-write channel, a two-element list will always be returned. The default value for **-eofchar** is the empty string in all cases except for files under Windows. In that case the **-eofchar** is Control-z (\x1A) for reading and the empty string for writing. The acceptable range for **-eofchar** values is \x01 - \x7F; attempting to set **-eofchar** to a value outside of this range will generate an error.

-translation *mode*

-translation {*inMode outMode*}

In Tcl scripts the end of a line is always represented using a single newline character (\n). However, in actual files and devices the end of a line may be represented differently on different platforms, or even for different devices on the same platform. For example, under UNIX newlines are used in files, whereas carriage-return-linefeed sequences are normally used in network connections. On input (i.e., with **gets** and **read**) the Tcl I/O system automatically translates the external end-of-line representation into newline characters. Upon output (i.e., with **puts**), the I/O system translates newlines to the external end-of-line representation. The default translation mode, **auto**, handles all the common cases automatically, but the **-translation** option provides explicit control over the end of line translations.

The value associated with **-translation** is a single item for read-only and write-only channels. The value is a two-element list for read-write channels; the read translation mode is the first element of the list, and the write translation mode is the second element. As a convenience, when setting the translation mode for a read-write channel you can specify a single value that will apply to both reading and writing. When querying the translation mode of a read-write channel, a two-element list will always be returned. The following values are currently supported:

auto

coreTools Command Reference Index

As the input translation mode, **auto** treats any of newline (**lf**), carriage return (**cr**), or carriage return followed by a newline (**crlf**) as the end of line representation. The end of line representation can even change from line-to-line, and all cases are translated to a newline. As the output translation mode, **auto** chooses a platform specific representation; for sockets on all platforms Tcl chooses **crlf**, for all Unix flavors, it chooses **lf**, and for the various flavors of Windows it chooses **crlf**. The default setting for **-translation** is **auto** for both input and output.

binary

No end-of-line translations are performed. This is nearly identical to **lf** mode, except that in addition **binary** mode also sets the end-of-file character to the empty string (which disables it) and sets the encoding to **binary** (which disables encoding filtering). See the description of **-eofchar** and **-encoding** for more information.

Internally, i.e. when it comes to the actual behaviour of the translator this value is identical to **lf** and is therefore reported as such when queried. Even if **binary** was used to set the translation.

cr

The end of a line in the underlying file or device is represented by a single carriage return character. As the input translation mode, **cr** mode converts carriage returns to newline characters. As the output translation mode, **cr** mode translates newline characters to carriage returns.

crlf

The end of a line in the underlying file or device is represented by a carriage return character followed by a linefeed character. As the input translation mode, **crlf** mode converts carriage-return-linefeed sequences to newline characters. As the output translation mode, **crlf** mode translates newline characters to carriage-return-linefeed sequences. This mode is typically used on Windows platforms and for network connections.

lf

The end of a line in the underlying file or device is represented by a single newline (linefeed) character. In this mode no translations occur during either input or output. This mode is typically used on UNIX platforms.

STANDARD CHANNELS

The Tcl standard channels (**stdin**, **stdout**, and **stderr**) can be configured through this command like every other channel opened by the Tcl library. Beyond the standard options described above they will also support any special option according to their current type. If,

coreTools Command Reference Index

for example, a Tcl application is started by the **inet** super-server common on Unix system its Tcl standard channels will be sockets and thus support the socket options.

EXAMPLES

Instruct Tcl to always send output to **stdout** immediately, whether or not it is to a terminal:

```
fconfigure stdout -buffering none
```

Open a socket and read lines from it without ever blocking the processing of other events:

```
set s [socket some.where.com 12345]
fconfigure $s -blocking 0
fileevent $s readable "readMe $s"
proc readMe chan {
    if {[gets $chan line] < 0} {
        if {[eof $chan]} {
            close $chan
            return
        }
        # Could not read a complete line this time; Tcl's
        # internal buffering will hold the partial line for us
        # until some more data is available over the socket.
    } else {
        puts stdout $line
    }
}
```

Read a PPM-format image from a file:

```
# Open the file and put it into Unix ASCII mode
set f [open teapot.ppm]
fconfigure $f -encoding ascii -translation lf

# Get the header
if {[gets $f] ne "P6"} {
    error "not a raw-bits PPM"
}

# Read lines until we have got non-comment lines
# that supply us with three decimal values.
set words {}
while {[llength $words] < 3} {
    gets $f line
    if {[string match "#" $line]} continue
    lappend words {[join [scan $line %d%d%d]]}
}

# Those words supply the size of the image and its
# overall depth per channel. Assign to variables.
lassign $words xSize ySize depth

# Now switch to binary mode to pull in the data,
# one byte per channel (red,green,blue) per pixel.
```

coreTools Command Reference Index

```
fconfigure $f -translation binary
set numDataBytes [expr {3 * $xSize * $ySize}]
set data [read $f $numDataBytes]

close $f
```

SEE ALSO

close(n), flush(n), gets(n), open(n), puts(n), read(n),
socket(n), Tcl_StandardChannels(3)

KEYWORDS

blocking, buffering, carriage return, end of line,
flushing, linemode, newline, nonblocking, platform,
translation, encoding, filter, byte array, binary

NAME

`fcopy` - Copy data from one channel to another

SYNOPSIS

`fcopy inchan outchan ?-size size? ?-command callback?`

DESCRIPTION

The **fcopy** command copies data from one I/O channel, *inchan* to another I/O channel, *outchan*. The **fcopy** command leverages the buffering in the Tcl I/O system to avoid extra copies and to avoid buffering too much data in main memory when copying large files to slow destinations like network sockets.

The **fcopy** command transfers data from *inchan* until end of file or *size* bytes or characters have been transferred; *size* is in bytes if the two channels are using the same encoding, and is in characters otherwise. If no **-size** argument is given, then the copy goes until end of file. All the data read from *inchan* is copied to *outchan*. Without the **-command** option, **fcopy** blocks until the copy is complete and returns the number of bytes or characters (using the same rules as for the **-size** option) written to *outchan*.

The **-command** argument makes **fcopy** work in the background. In this case it returns immediately and the *callback* is invoked later when the copy completes. The *callback* is called with one or two additional arguments that indicates how many bytes were written to *outchan*. If an error occurred during the background copy, the second argument is the error string associated with the error. With a background copy, it is not necessary to put *inchan* or *outchan* into non-

coreTools Command Reference Index

blocking mode; the **fcopy** command takes care of that automatically. However, it is necessary to enter the event loop by using the **vwait** command or by using Tk.

You are not allowed to do other input operations with *inchan*, or output operations with *outchan*, during a background **fcopy**. The converse is entirely legitimate, as exhibited by the bidirectional fcopy example below.

If either *inchan* or *outchan* get closed while the copy is in progress, the current copy is stopped and the command callback is *not* made. If *inchan* is closed, then all data already queued for *outchan* is written out.

Note that *inchan* can become readable during a background copy. You should turn off any **fileevent** handlers during a background copy so those handlers do not interfere with the copy. Any wrong-sided I/O attempted (by a **fileevent** handler or otherwise) will get a error.

Fcopy translates end-of-line sequences in *inchan* and *outchan* according to the **-translation** option for these channels. See the manual entry for **fconfigure** for details on the **-translation** option. The translations mean that the number of bytes read from *inchan* can be different than the number of bytes written to *outchan*. Only the number of bytes written to *outchan* is reported, either as the return value of a synchronous **fcopy** or as the argument to the callback for an asynchronous **fcopy**.

Fcopy obeys the encodings and character translations configured for the channels. This means that the incoming characters are converted internally first UTF-8 and then into the encoding of the channel **fcopy** writes to. See the manual entry for **fconfigure** for details on the **-encoding** and **-translation** options. No conversion is done if both channels are set to encoding and have matching translations. If only the output channel is set to encoding the system will write the internal UTF-8 representation of the incoming characters. If only the input channel is set to encoding the system will assume that the incoming bytes are valid UTF-8 characters and convert them according to the output encoding. The behaviour of the system for bytes which are not valid UTF-8 characters is undefined in this case.

EXAMPLES

The first example transfers the contents of one channel exactly to another. Note that when copying one file to another, it is better to use **file copy** which also copies file metadata (e.g. the file access permissions) where possible.

```
fconfigure $in -translation binary
fconfigure $out -translation binary
```

coreTools Command Reference Index

fcopy \$in \$out

This second example shows how the callback gets passed the number of bytes transferred. It also uses `vwait` to put the application into the event loop. Of course, this simplified example could be done without the command callback.

```
proc Cleanup {in out bytes {error {}}} {
    global total
    set total $bytes
    close $in
    close $out
    if {[string length $error] != 0} {
        # error occurred during the copy
    }
}
set in [open $file1]
set out [socket $server $port]
fcopy $in $out -command [list Cleanup $in $out]
vwait total
```

The third example copies in chunks and tests for end of file in the command callback.

```
proc CopyMore {in out chunk bytes {error {}}} {
    global total done
    incr total $bytes
    if {[string length $error] != 0} || [eof $in] {
        set done $total
        close $in
        close $out
    } else {
        fcopy $in $out -size $chunk \
            -command [list CopyMore $in $out $chunk]
    }
}
set in [open $file1]
set out [socket $server $port]
set chunk 1024
set total 0
fcopy $in $out -size $chunk \
    -command [list CopyMore $in $out $chunk]
vwait done
```

The fourth example starts an asynchronous, bidirectional `fcopy` between two sockets. Those could also be pipes from two `[open "|hal 9000" r+]` (though their conversation would remain secret to the script, since all four fileevent slots are busy).

```
set flows 2
proc Done {dir args} {
    global flows done
    puts "$dir is over."
    incr flows -1
    if {$flows<=0} {set done 1}
}
fcopy $sok1 $sok2 -command [list Done UP]
fcopy $sok2 $sok1 -command [list Done DOWN]
vwait done
```


SEE ALSO

`eof(n)`, `fblocked(n)`, `fconfigure(n)`, `file(n)`

KEYWORDS

blocking, channel, end of line, end of file,
nonblocking, read, translation

file

File item

Description

The file item type represents a file in a coreTools knowledge database.

See Also

[filegroup \(3\)](#)

Supported Attributes

[ComponentOfItem \(3\)](#), [DefaultInstallDir \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [EncryptText \(3\)](#), [FileContentType \(3\)](#), [FilePerms \(3\)](#), [HdlLibrary \(3\)](#), [InsertTechCellBindMacroAtStart \(3\)](#), [Install \(3\)](#), [InstallUserWorkDir \(3\)](#), [Label \(3\)](#), [Name \(3\)](#), [RemoveIfEmpty \(3\)](#), [Sequence \(3\)](#), [SpiritFileType \(3\)](#), [SubstituteInFile \(3\)](#), [TypeName \(3\)](#), _____

NAME

`fileevent` - Execute a script when a channel becomes readable or writable

SYNOPSIS

`fileevent channelId readable ?script?`

`fileevent channelId writable ?script?`

DESCRIPTION

This command is used to create *file event handlers*. A file event handler is a binding between a channel and a script, such that the script is evaluated whenever the channel becomes readable or writable. File event handlers are most commonly used to allow data to be received from another process on an event-driven basis, so that the receiver can continue to interact with the user while waiting for the data to arrive. If an application invokes **gets** or **read** on a blocking channel when there is no input data available, the process will block; until the input data arrives, it will not be able to service other events, so it will appear to the user to With **fileevent**, the process can tell when data is present and only invoke **gets** or **read** when they will not block.

The *channelId* argument to **fileevent** refers to an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension.

If the *script* argument is specified, then **fileevent** creates a new event handler: *script* will be evaluated

coreTools Command Reference Index

whenever the channel becomes readable or writable (depending on the second argument to **fileevent**). In this case **fileevent** returns an empty string. The **readable** and **writable** event handlers for a file are independent, and may be created and deleted separately. However, there may be at most one **readable** and one **writable** handler for a file at a given time in a given interpreter. If **fileevent** is called when the specified handler already exists in the invoking interpreter, the new script replaces the old one.

If the *script* argument is not specified, **fileevent** returns the current script for *channelId*, or an empty string if there is none. If the *script* argument is specified as an empty string then the event handler is deleted, so that no script will be invoked. A file event handler is also deleted automatically whenever its channel is closed or its interpreter is deleted.

A channel is considered to be readable if there is unread data available on the underlying device. A channel is also considered to be readable if there is unread data in an input buffer, except in the special case where the most recent attempt to read from the channel was a **gets** call that could not find a complete line in the input buffer. This feature allows a file to be read a line at a time in nonblocking mode using events. A channel is also considered to be readable if an end of file or error condition is present on the underlying file or device. It is important for *script* to check for these conditions and handle them appropriately; for example, if there is no special check for end of file, an infinite loop may occur where *script* reads no data, returns, and is immediately invoked again.

A channel is considered to be writable if at least one byte of data can be written to the underlying file or device without blocking, or if an error condition is present on the underlying file or device.

Event-driven I/O works best for channels that have been placed into nonblocking mode with the **fconfigure** command. In blocking mode, a **puts** command may block if you give it more data than the underlying file or device can accept, and a **gets** or **read** command will block if you attempt to read more data than is ready; a readable underlying file or device may not even guarantee that a blocking [read 1] will succeed (counter-examples being multi-byte encodings, compression or encryption transforms). In all such cases, no events will be processed while the commands block.

In nonblocking mode **puts**, **read**, and **gets** never block. See the documentation for the individual commands for information on how they handle blocking and nonblocking channels.

Testing for the end of file condition should be done after any attempts read the channel data. The eof flag is set once an attempt to read the end of data has

coreTools Command Reference Index

occurred and testing before this read will require an additional event to be fired.

The script for a file event is executed at global level (outside the context of any Tcl procedure) in the interpreter in which the **fileevent** command was invoked. If an error occurs while executing the script then the command registered with **interp bgerror** is used to report the error. In addition, the file event handler is deleted if it ever returns an error; this is done in order to prevent infinite loops due to buggy handlers.

EXAMPLE

In this setup **GetData** will be called with the channel as an argument whenever \$chan becomes readable. The **read** call will read whatever binary data is currently available without blocking. Here the channel has the fileevent removed when an end of file occurs to avoid being continually called (see above). Alternatively the channel may be closed on this condition.

```
proc GetData {chan} {
    set data [read $chan]
    puts "[string length $data] $data"
    if {[eof $chan]} {
        fileevent $chan readable {}
    }
}
```

```
fconfigure $chan -blocking 0 -encoding binary
fileevent $chan readable [list GetData $chan]
```

The next example demonstrates use of **gets** to read line-oriented data.

```
proc GetData {chan} {
    if {[gets $chan line] >= 0} {
        puts $line
    }
    if {[eof $chan]} {
        close $chan
    }
}
```

```
fconfigure $chan -blocking 0 -buffering line -translation crlf
fileevent $chan readable [list GetData $chan]
```

CREDITS

fileevent is based on the **addinput** command created by Mark Diekhans.

SEE ALSO

`fconfigure(n)`, `gets(n)`, `interp(n)`, `puts(n)`, `read(n)`,
`Tcl_StandardChannels(3)`

KEYWORDS

asynchronous I/O, blocking, channel, event handler,
nonblocking, readable, script, writable.

filegroup
File group item

Description

The filegroup item type represents a file group in a coreBuilder workspace or coreConsultant coreKit.

coreBuilder automatically creates the file groups Hdl and DesignKbs. Core developers can create additional file groups to be included in the coreKit Bill of Materials either through the Specify Bill of Materials GUI dialog or by executing the `add_files_to_filegroup` command. The `add_files_to_filegroup` command creates a filegroup item if the specified file group does not already exist.

See Also

[add_files_to_filegroup \(2\)](#), [file \(3\)](#)

Supported Attributes

[ActivityGroup \(3\)](#), [AutoloadFilegroup \(3\)](#), [ComponentOfItem \(3\)](#),
[ConfigActivity \(3\)](#), [ConfigDependsOnActivities \(3\)](#), [ConfigDependsOnGroup \(3\)](#), [ConfigIntentSearchPath \(3\)](#), [Configurable \(3\)](#), [DefaultInstallDir \(3\)](#),
[DefaultLoadPath \(3\)](#), [DeliverableType \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [EncryptText \(3\)](#), [ExcludeLoadPatterns \(3\)](#), [Exists \(3\)](#),
[FileContentType \(3\)](#), [FilePerms \(3\)](#), [HdlLibrary \(3\)](#), [Importance \(3\)](#),
[Install \(3\)](#), [InstallUserWorkDir \(3\)](#), [InstallWhen \(3\)](#),
[IntentFilesProcessed \(3\)](#), [IsUpToDate \(3\)](#), [Label \(3\)](#), [Name \(3\)](#),
[OneRequiredGroup \(3\)](#), [ParameterInfo \(3\)](#), [PostPromptCmd \(3\)](#), [PrePromptCmd \(3\)](#), [Sequence \(3\)](#), [SpiritFileType \(3\)](#), [SubstituteInFile \(3\)](#), [TypeName \(3\)](#),

filegroupGroup

Represents a collection of deliverables (filegroups) defined via a Group list in a BoM template.

Description

The filegroupGroup item type provides a container for a set of deliverables (filegroups) which are part of the same 'grouping' from the perspective of the BoM template. These are created automatically when the BoM template is read and are typically worked with as if they were filegroups. Attributes set on filegroupGroups apply to all associated filegroups unless the attribute is explicitly set on the filegroup itself.

See Also

[filegroup \(3\)](#)

Supported Attributes

[AutoloadFilegroup \(3\)](#), [ComponentOfItem \(3\)](#), [Configurable \(3\)](#), [DefaultInstallDir \(3\)](#), [DeliverableType \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [EncryptText \(3\)](#), [ExcludeLoadPatterns \(3\)](#), [Exists \(3\)](#), [FileContentType \(3\)](#), [FilePerms \(3\)](#), [HdlLibrary \(3\)](#), [Importance \(3\)](#), [Install \(3\)](#), [InstallUserWorkDir \(3\)](#), [InstallWhen \(3\)](#), [IsUpToDate \(3\)](#), [Label \(3\)](#), [Name \(3\)](#), [OneRequiredGroup \(3\)](#), [Sequence \(3\)](#), [SubstituteInFile \(3\)](#), [TypeName \(3\)](#).

NAME

filename - File name conventions supported by Tcl commands

INTRODUCTION

All Tcl commands and C procedures that take file names as arguments expect the file names to be in one of three forms, depending on the current platform. On each platform, Tcl supports file names in the standard form(s) for that platform. In addition, on all platforms, Tcl supports a Unix-like syntax intended to provide a convenient way of constructing simple file names. However, scripts that are intended to be portable should not assume a particular form for file names. Instead, portable scripts must use the **file split** and **file join** commands to manipulate file names (see the **file** manual entry for more details).

PATH TYPES

File names are grouped into three general types based on the starting point for the path used to specify the file: absolute, relative, and volume-relative. Absolute names are completely qualified, giving a path to the file relative to a particular volume and the root directory on that volume. Relative names are unqualified, giving a path to the file relative to the current working directory. Volume-relative names are partially qualified, either giving the path relative to the root directory on the current volume, or relative to the current directory of the specified volume. The **file pathtype** command can be used to determine the type of a given path.

PATH SYNTAX

The rules for native names depend on the value reported in the Tcl **platform** element of the **tcl_platform** array:

Unix On Unix and Apple MacOS X platforms, Tcl uses path names where the components are separated by slashes. Path names may be relative or absolute, and file names may contain any character other than slash. The file names **.** and **..** are special and refer to the current directory and the parent of the current directory respectively. Multiple adjacent slash characters are interpreted as a single separator. Any number of trailing slash characters at the end of a path are simply ignored, so the paths **foo**, **foo/** and **foo//** are all identical, and in particular **foo/** does not necessarily mean a directory is being referred.

The following examples illustrate various forms of path names:

/	Absolute path to the root directory.
/etc/passwd	Absolute path to the file named passwd in the directory etc in the root directory.
.	Relative path to the current directory.
foo	Relative path to the file foo in the current directory.
foo/bar	Relative path to the file bar in the directory foo in the current directory.
../foo	Relative path to the file foo in the directory above the current directory.

Windows On Microsoft Windows platforms, Tcl supports both drive-relative and UNC style names. Both **/** and **** may be used as directory separators in either type of name. Drive-relative names consist of an optional drive specifier followed by an absolute or relative path. UNC paths follow the general form **\\servername\sharename\path\file**, but must at the very least contain the server and share components, i.e. **\\servername\sharename**. In both forms, the file names **.** and **..** are special and refer to the current directory and the parent of the current directory respectively. The following examples illustrate various forms of path names:

coreTools Command Reference Index

<code>\\Host\share\file</code>	Absolute UNC path to a file called file in the root directory of the export point share on the host Host . Note that repeated use of file dirname on this path will give <code>//Host/share</code> , and will never give just <code>//Host</code> .
<code>c:foo</code>	Volume-relative path to a file foo in the current directory on drive c .
<code>c:/foo</code>	Absolute path to a file foo in the root directory of drive c .
<code>foo\bar</code>	Relative path to a file bar in the foo directory in the current directory on the current volume.
<code>\foo</code>	Volume-relative path to a file foo in the root directory of the current volume.
<code>\\foo</code>	Volume-relative path to a file foo in the root directory of the current volume. This is not a valid UNC path, so the assumption is that the extra backslashes are superfluous.

TILDE SUBSTITUTION

In addition to the file name rules described above, Tcl also supports *csh*-style tilde substitution. If a file name starts with a tilde, then the file name will be interpreted as if the first element is replaced with the location of the home directory for the given user. If the tilde is followed immediately by a separator, then the **\$HOME** environment variable is substituted. Otherwise the characters between the tilde and the next separator are taken as a user name, which is used to retrieve the user's home directory for substitution. This works on Unix, MacOS X and Windows (except very old releases).

Old Windows platforms do not support tilde substitution when a user name follows the tilde. On these platforms, attempts to use a tilde followed by a user name will generate an error that the user does not exist when Tcl attempts to interpret that part of the path or otherwise access the file. The behaviour of these paths when not trying to interpret them is the same as on Unix. File names that have a tilde without a user name will be correctly substituted using the **\$HOME** environment variable, just like for Unix.

PORTABILITY ISSUES

Not all file systems are case sensitive, so scripts should avoid code that depends on the case of characters in a file name. In addition, the character sets allowed on different devices may differ, so scripts should choose file names that do not contain special characters like: `<>:"/\\|`. The safest approach is to use names consisting of alphanumeric characters only. Care should be taken with filenames which contain spaces (common on Windows systems) and filenames where the backslash is the directory separator (Windows native path names). Also Windows 3.1 only supports file names with a root of no more than 8 characters and an extension of no more than 3 characters.

On Windows platforms there are file and path length restrictions. Complete paths or filenames longer than about 260 characters will lead to errors in most file operations.

Another Windows peculiarity is that any number of trailing dots in filenames are totally ignored, so, for example, attempts to create a file or directory with a name will result in the creation of a file/directory with name This fact is reflected in the results of **file normalize**. Furthermore, a file name consisting only of dots or dots with trailing characters is illegal.

SEE ALSO

`file(n)`, `glob(n)`

KEYWORDS

current directory, absolute file name, relative file name, volume-relative file name, portability

API Index Command Reference

File List

Here is a list of all files with brief descriptions:

[detail level 12]

man1

man2

alias.idx

apropos.idx

check_script.idx

date.idx

debug_script.idx

define_proc_attributes.idx

echo.idx

error_info.idx

exit.idx

getenv.idx

help.idx

history.idx

ls.idx

mpexpr.idx

mpformat.idx

parse_proc_arguments.idx

printenv.idx

printvar.idx

quit.idx

redirect.idx

setenv.idx

sh.idx

source.idx

unalias.idx

man2

add_command_hook.idx

after.idx

annotate_trace.idx

append.idx

apply.idx

array.idx

bgerror.idx

binary.idx

break.idx

case.idx

[catch.idx](#)
[cd.idx](#)
[chan.idx](#)
[class.idx](#)
[clock.idx](#)
[close.idx](#)
[concat.idx](#)
[continue.idx](#)
[copy.idx](#)
[coroutine.idx](#)
[create command group.idx](#)
[dde.idx](#)
[define.idx](#)
[dict.idx](#)
[encoding.idx](#)
[eof.idx](#)
[error.idx](#)
[eval.idx](#)
[exec.idx](#)
[expr.idx](#)
[fblocked.idx](#)
[fconfigure.idx](#)
[fcopy.idx](#)
[file.idx](#)
[fileevent.idx](#)
[filename.idx](#)
[flush.idx](#)
[for.idx](#)
[foreach.idx](#)
[format.idx](#)
[get app var.idx](#)
[get command hooks.idx](#)
[get command option values.idx](#)
[get current hook command.idx](#)
[get defined commands.idx](#)
[get message ids.idx](#)
[get message info.idx](#)
[get trace option.idx](#)
[get unix variable.idx](#)
[gets.idx](#)
[glob.idx](#)
[global.idx](#)

[http.idx](#)
[if.idx](#)
[incr.idx](#)
[info.idx](#)
[interp.idx](#)
[is_false.idx](#)
[is_true.idx](#)
[join.idx](#)
[lappend.idx](#)
[lassign.idx](#)
[library.idx](#)
[lindex.idx](#)
[linsert.idx](#)
[list.idx](#)
[llength.idx](#)
[lmap.idx](#)
[lminus.idx](#)
[load.idx](#)
[log_trace.idx](#)
[lrange.idx](#)
[lrepeat.idx](#)
[lreplace.idx](#)
[lreverse.idx](#)
[lsearch.idx](#)
[lset.idx](#)
[lsort.idx](#)
[man.idx](#)
[mathfunc.idx](#)
[mathop.idx](#)
[memory.idx](#)
[msgcat.idx](#)
[my.idx](#)
[namespace.idx](#)
[next.idx](#)
[object.idx](#)
[open.idx](#)
[package.idx](#)
[packagens.idx](#)
[pid.idx](#)
[pkgMkIndex.idx](#)
[platform.idx](#)
[platform_shell.idx](#)

[prefix.idx](#)
[print_message_info.idx](#)
[print_suppressed_messages.idx](#)
[proc.idx](#)
[proc_args.idx](#)
[proc_body.idx](#)
[puts.idx](#)
[pwd.idx](#)
[py_eval.idx](#)
[re_syntax.idx](#)
[read.idx](#)
[refchan.idx](#)
[regexp.idx](#)
[registry.idx](#)
[regsub.idx](#)
[remove_command_hook.idx](#)
[rename.idx](#)
[report_app_var.idx](#)
[report_trace.idx](#)
[return.idx](#)
[safe.idx](#)
[scan.idx](#)
[seek.idx](#)
[self.idx](#)
[set.idx](#)
[set_app_var.idx](#)
[set_command_option_value.idx](#)
[set_current_command_mode.idx](#)
[set_message_info.idx](#)
[set_trace_option.idx](#)
[set_unix_variable.idx](#)
[snps.cmd.idx](#)
[snps.collection.idx](#)
[snps.redirect.idx](#)
[snps.value.idx](#)
[socket.idx](#)
[split.idx](#)
[string.idx](#)
[subst.idx](#)
[suppress_message.idx](#)
[switch.idx](#)
[tailcall.idx](#)

[Tcl.idx](#)
[tcltest.idx](#)
[tclvars.idx](#)
[tell.idx](#)
[throw.idx](#)
[time.idx](#)
[timerate.idx](#)
[tm.idx](#)
[trace.idx](#)
[transchan.idx](#)
[try.idx](#)
[unknown.idx](#)
[unload.idx](#)
[unset.idx](#)
[unsetenv.idx](#)
[unsuppress message.idx](#)
[update.idx](#)
[uplevel.idx](#)
[upvar.idx](#)
[variable.idx](#)
[vwait.idx](#)
[which.idx](#)
[while.idx](#)
[write app var.idx](#)
[zlib.idx](#)
[man3](#)
[sh allow tcl with set app var.idx](#)
[sh allow tcl with set app var no message list.idx](#)
[sh arch.idx](#)
[sh command abbrev mode.idx](#)
[sh command abbrev options.idx](#)
[sh command log file.idx](#)
[sh continue on error.idx](#)
[sh deprecated is error.idx](#)
[sh dev null.idx](#)
[sh disabled is error.idx](#)
[sh enable page mode.idx](#)
[sh enable stdout redirect.idx](#)
[sh help shows group overview.idx](#)
[sh language.idx](#)
[sh new variable message.idx](#)
[sh new variable message in proc.idx](#)

coreTools Command Reference Index

[sh new variable message in script.idx](#)
[sh obsolete is error.idx](#)
[sh script stop severity.idx](#)
[sh source emits line numbers.idx](#)
[sh source logging.idx](#)
[sh source uses search path.idx](#)
[sh tcllib app dirname.idx](#)
[sh user man path.idx](#)
[synopsys program name.idx](#)
[synopsys root.idx](#)
[man4](#)
[man5](#)

find_design

Locate the specified design or specified instance of the design and create a collection that contains that design

Syntax

```
string find_design [-set] [-instance <instance_path>] design
    string <instance_path>
    string design
```

Parameters

-set	Set current_design to the design found by this command.
-instance <instance_path>	Find the design associated with the specified instance path/name.
design	The name of the design to find.

Description

The find_design command locates the specified design and creates a collection that contains the design that it finds.

A typical use for find_design is after elaboration to locate the elaborated design associated with a specified instance of a design. The name of a design changes during elaboration if the design contains parameters (VHDL generics or Verilog parameters) or if the design needs to be uniquified. Instance names do not change during elaboration. Therefore you can use the instance name (including path) to locate an instance of a design. The find_design command will find the name of the elaborated design associated with the specified instance name.

find_design returns the collection handle of the collection that it creates and prints the name of the design that it finds.

Examples

Assume that Subblock_A (instance name i_Subblock_A) of the design MyCore contains two instances of the design named Counter (i_counter1 and i_counter2). Each instance uses a different value for the width parameter of Counter. To find the name of the elaborated design associated with instance i_counter1 and make it the current design:

```
coreConsultant> current_design MyCore
```

{MyCore}

```
coreConsultant> find_design -instance i_Subblock_A/i_counter1 Counter  
{Counter_8}
```

```
coreConsultant> find_design -instance i_Subblock_A/i_counter2 Counter  
{Counter_12}
```

```
coreConsultant> current_design [find_design  
-instance i_Subblock_A/i_counter1 Counter]
```

Information: Current design changed to Counter_8. (DES-4)

```
{Counter_8}
```

```
coreConsultant> current_design
```

```
{Counter_8}
```

To create a collection that contains the design associated with instance i_Subblock_A/i_counter2 and store the collection handle in the Tcl variable i2Name:

```
coreConsultant> set i2Name [find_design -instance i_Subblock_A/i_counter2  
Counter]\fP  
{Counter_12}
```

See Also

current_design (2), find_item (2)

find_interface_instances

Locate the specified interface instances.

Syntax

```
string find_interface_instances [-name <name>] [-exported]
[-exported_from] [-attached] [-component <componentName>] [-filter
<string>] [-sort <string>]
    string <name>
    string <componentName>
    string <string>
```

Parameters

-name <name>	Find interface with the specified name
-exported	Restrict to sub-system (exported) interfaces
-exported_from	Restrict to interfaces exported from the specified component
-attached	Restrict to attached interfaces (valid: -attached -exported)
-component <componentName>	Restrict to interfaces inside this component
-filter <string>	Filter expression for instances
-sort <string>	Sort string

Description

This command is available for plugin writers who need access to interface objects in the coreAssembler data model. The command returns a collection of interfaceInstance elements which meet the specified criteria. The -filter option can be used to further restrict the list of returned elements.

Examples

Locate all exported interfaces:

```
coreAssembler> set intf [find_interface_instances -exported]
Locate all interfaces in instance i_ahb which are unused:
```

```
coreAssembler> set intf [find_interface_instances -component i_ahb
-filter Unused==1]
Get the AHB_Master interface on the i_ahb component:
```

```
coreAssembler> set intf [find_interface_instances -component i_ahb -name  
AHB_Master]
```

See Also

find_item

Create a collection of items that match the specified name pattern

Syntax

```
string find_item [-search <KB name list>] [-type <item type>] [-filter
<filter expression>] [-sort <sort expression>] [-regex] [-exact]
[-quiet] [-nocase] [-should_match <match count>] [<item name pattern>]
    string <KB name list>
    string <item type>
    string <filter expression>
    string <sort expression>
    int <match count>
    list <item name pattern>
```

Parameters

	Specifies a list of knowledge databases (KBs) to search. By default, find_item searches the current KB and all KBs required by the current KB. If you specify a search list, your search list overrides the default search list. The search list can contain the names of any loaded KBs, as well as the special keyword "\<current\>", which represents the current KB. For example, to search the KB named "work.Subblock_A" and the current KB, specify the search list as {work.Subblock_A <current>}.
-search <KB name list>	
-type <item type>	the type of item (Values: knowledgeBase, generic, file, attrDefn, item, activity, view, viewColumn, libItem, Dialog, dlgItem, dlgLabel, dlgValue, dlgButton, dlgCheckBox, dlgGroupBox, dlgStringsCombo, dlgStringsList, dlgView, design, param, port, pin, cell, cnct, net, dlgItemsCombo, dlgItemsList, Sheet, Page, GlobalActivity, DesignActivity, HierActivity, Strategy, genHierItem, busBit, menuCmd, menuCmdGroup, filegroup, dlgEditableView, formula, dlgEditableStringsList, connection, clock, state, preference, hdlFunc, dlgBitmap, timingException, filegroupGroup, bomTemplate, interfaceDefn, interfacePort, interfaceInstance, hdlType, hdlPackage, Schematic, dlgBrowser, viewRow, hierViewRow, memMap, register, registerField, addressBlock, addressBank, registerArray, intfPin, intfPort) By default, find_item returns all items that match the specified name(s) and filter. If you specify an item

coreTools Command Reference Index

	type, find_item returns the items that match the specified name(s), the filter, and have the specified type. Using the -type option to specify the item type is easier and more efficient than putting a type check in the filter expression. Expression used to filter items from the search list. This is a filter expression, which is a string containing attribute/value pairs, relational operators, and logical operators. find_item only returns items for which the filter expression is true. The simplest form of filter expression is: <i><attribute name> <relational operator> <attribute value></i> For example, "find_item -type design -filter {MapSubblocksIndividually == false}" returns all designs on which the MapSubblocksIndividually attribute is false. The supported relational operators are ==, !=, >, >=, <, <=, as well as regular expression operators =~ and !~. If you want to pass regular expression characters as literal characters (e.g. the '.' character), it must be escaped twice (Name=~"Has\\.Dots") unless the expression is enclosed in {}. In that case TCL does no interpretation of what is inside the {} so the characters are treated as literals automatically. You can build more complex expressions by using logical operators && and as well as parentheses. There is no operator precedence, so you must use parentheses to get any precedence other than simple left to right. Specifies how to sort the found items. This is a list of attribute names used to sort the returned items. Each name in the space separated list can start with '+' or '-' to indicate ascending or descending sorting. The default is ascending. For example, "find_item -type port -sort MaxOutputDelay" sorts the returned ports in order of increasing values of the MaxOutputDelay attribute. Attribute names beyond the first specified name are used only to break ties. Indicates that <item name pattern> is a regular expression. If you do not specify either -regexp or -exact, find_item uses glob-style pattern matching. Specifies that items must match <item name pattern> exactly. Do not print messages. Ignore case when matching the pattern. Specifies the expected number of items to be found. (Range: 0 to 2000000) The command will fail and return an error message if
-filter <filter expression>	
-sort <sort expression>	
-regexp	
-exact	
-quiet	
-nocase	
-should_match <match count>	

the actual number of items found differs from <match count>.

The names of the items to be found.

This can be a specific name like 'TOP', a simple regular expression like 'T*', or a Tcl list of names and expressions like {TOP BOTTOM M*}. <item name pattern> can also be a collection, in which case find_item searches the specified collection for items that match the criteria specified by the find_item options. The default <item name pattern> is *.

<item name
pattern>

Description

The find_item command finds and creates a collection of items that match the criteria specified by the find_item command options. You can specify which KBs to search, what item types to find, and how to filter and sort the found items.

You can either find items by matching the item name exactly (-exact option) or use pattern matching. If you do not specify either -exact or -regexp, find_item uses the same glob-style pattern matching that the standard Tcl "string match" command uses. If you specify the -regexp option, find_item uses the more complex and powerful regular expression pattern matching. For more information about glob and regular expressions, refer to a Tcl manual or the string and regexp man pages in the Tcl-Built-In-Commands section of the coreTool online reference.

If you specify a collection as the <item name pattern>, find_item searches the specified collection for items that match the find_item options. This enables you to use find_item to filter and/or sort the items in a collection.

find_item returns the collection handle of the collection it creates and prints the names of the items found. To use the collection later as input to another command, store the collection handle in a Tcl variable, as shown in the examples.

Examples

To find all items in the current KB:

```
coreConsultant> find_item * -search <current>
{Top Subblock_A Subblock_B Subblock_C Subblock_D Subblock_E Subblock_A1
Subblock_D Subblock_E Subblock_A1 ...}
```

To create a collection that includes all designs that begin with the letter T and store the collection handle in the variable Tdesigns:

```
coreConsultant> set Tdesigns [find_item -type design T*]
{Timer Top}
```

To find all ports that have the PinLoadCount attribute set to a value greater than or equal to 2 and have the CriticalTiming attribute set to true, and then sort the ports found by name in decreasing order:

```
coreConsultant> find_item -type port -filter  
{(PinLoadCount >= 2) && (CriticalTiming == true)}  
-sort -Name *  
{out5 out4 out3 out2 out1 out0}
```

To create a collection of all output ports of the current_design, then search the collection for all ports on which PinLoadCount is greater than or equal to 4:

```
coreConsultant> find_item -type port -filter  
{PinLoadCount >= 4} [all_outputs]  
{out0 out2 out5}
```

See Also

[find_design\(2\).](#)

find_kb

Find KB with the given name and type.

Syntax

```
string find_kb [-name <KB name>] [-type <KB type>] [-first]  
    string <KB name>  
    string <KB type>
```

Parameters

-name	
<KB name>	Name of KB to be found
-type	indicates the type of the new KB (Values: scratch,
<KB type>	integrator, builder, design, technology, install, filegroup, core, plugin, verification, assembly, system)
-first	Find first matching KB only.

Description

The find_kb command provides a faster command for locating knowledge base objects as compared to using find_item -type knowledgeBase. One or both of the -name and -type options must be specified. If -first is specified, the command returns a single KB object (the first match found).

Examples

See Also

NAME

`flush` - Flush buffered output for a channel

SYNOPSIS

`flush` *channelId*

DESCRIPTION

Flushes any output that has been buffered for *channelId*.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdout** or **stderr**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension. The channel must have been opened for writing.

If the channel is in blocking mode the command does not return until all the buffered output has been flushed to the channel. If the channel is in nonblocking mode, the command may return before all buffered output has been flushed; the remainder will be flushed in the background as fast as the underlying file or device is able to absorb it.

EXAMPLE

Prompt for the user to type some information in on the console:

```
puts -nonewline "Please type your name: "
```

```
flush stdout  
gets stdin name  
puts "Hello there, $name!"
```

SEE ALSO

`file(n)`, `open(n)`, `socket(n)`, `Tcl_StandardChannels(3)`

KEYWORDS

blocking, buffer, channel, flush, nonblocking, output

fm_shellVariable

Variable settings to be used for fm_shell. Example: set_design_attribute {fm_shellVariable[varName]} value.

Definition

Type: string

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This subscripted attribute can be used to set variables each time fm_shell is run for the design. The subscript of the attribute is the variable name and the value is the value that the variable will be set to in fm_shell.

You can set non-application variables by prefixing the attribute subscript with the text "internal:". If the subscript has this prefix then setting the variable in the tool will be done with the Tcl "set" command instead of the "set_app_var" command.

The fm_shellVariable settings are applied before any designs or libraries are loaded.

Examples

Specify some variables that are to be applied at setup when fm_shell is run.

```
coreConsultant> current_design [get_top_design]
coreConsultant> set_design_attribute {fm_shellVariable[my_var]} my_value
coreConsultant> set_design_attribute
{fm_shellVariable[internal:my_int_var]} my_value
coreConsultant> set_design_attribute {fm_shellVariable[null_var]} ""
coreConsultant> set_design_attribute {fm_shellVariable[list_var]} [list A
list of values]
```

See Also

[set_design_attribute \(2\)](#), [FormalVerificationAuxScript \(2\)](#)

[API Index](#) [Command Reference](#)

`fm_shellVariableComment`

Comment to be issued for the corresponding subscript of `fm_shellVariable`.

Definition

Type: string

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

The `fm_shellVariableComment` attribute can be used to output a comment to document why the corresponding `fm_shellVariable` is being used. The comment will be put into the file where the variable is set.

Examples

Set variable to be used in `fm_shell`, and specify a comment.

```
set_design_attribute {fm_shellVariable[myVar]} myValue
set_design_attribute {fm_shellVariableComment[myVar]} "Sample comment for
setting myVar."
```

See Also

[fm_shellVariable \(3\)](#)

NAME

`for` - 'For' loop

SYNOPSIS

`for start test next body`

DESCRIPTION

For is a looping command, similar in structure to the C **for** statement. The *start*, *next*, and *body* arguments must be Tcl command strings, and *test* is an expression string. The **for** command first invokes the Tcl interpreter to execute *start*. Then it repeatedly evaluates *test* as an expression; if the result is non-zero it invokes the Tcl interpreter on *body*, then invokes the Tcl interpreter on *next*, then repeats the loop. The command terminates when *test* evaluates to 0. If a **continue** command is invoked within *body* then any remaining commands in the current execution of *body* are skipped; processing continues by invoking the Tcl interpreter on *next*, then evaluating *test*, and so on. If a **break** command is invoked within *body* or *next*, then the **for** command will return immediately. The operation of **break** and **continue** are similar to the corresponding statements in C. **For** returns an empty string.

Note: *test* should almost always be enclosed in braces. If not, variable substitutions will be made before the **for** command starts executing, which means that variable changes made by the loop body will not be considered in the expression. This is likely to result in an infinite loop. If *test* is enclosed in braces, variable substitutions are delayed until the expression is evaluated (before each loop iteration), so changes in the variables will be visible. See below for an

example:

EXAMPLES

Print a line for each of the integers from 0 to 9:

```
for {set x 0} {$x<10} {incr x} {  
    puts "x is $x"  
}
```

Either loop infinitely or not at all because the expression being evaluated is actually the constant, or even generate an error! The actual behaviour will depend on whether the variable **x** exists before the **for** command is run and whether its value is a value that is less than or greater than/equal to ten, and this is because the expression will be substituted before the **for** command is executed.

```
for {set x 0} $x<10 {incr x} {  
    puts "x is $x"  
}
```

Print out the powers of two from 1 to 1024:

```
for {set x 1} {$x<=1024} {set x [expr {$x * 2}]} {  
    puts "x is $x"  
}
```

SEE ALSO

`break(n)`, `continue(n)`, `foreach(n)`, `while(n)`

KEYWORDS

boolean, for, iteration, loop

NAME

`foreach` - Iterate over all elements in one or more lists

SYNOPSIS

```
foreach varname list body  
foreach varlist1 list1 ?varlist2 list2 ...? body
```

DESCRIPTION

The **foreach** command implements a loop where the loop variable(s) take on values from one or more lists. In the simplest case there is one loop variable, *varname*, and one list, *list*, that is a list of values to assign to *varname*. The *body* argument is a Tcl script. For each element of *list* (in order from first to last), **foreach** assigns the contents of the element to *varname* as if the **lindex** command had been used to extract the element, then calls the Tcl interpreter to execute *body*.

In the general case there can be more than one value list (e.g., *list1* and *list2*), and each value list can be associated with a list of loop variables (e.g., *varlist1* and *varlist2*). During each iteration of the loop the variables of each *varlist* are assigned consecutive values from the corresponding *list*. Values in each *list* are used in order from first to last, and each value is used exactly once. The total number of loop iterations is large enough to use up all the values from all the value lists. If a value list does not contain enough elements for each of its loop variables in each iteration, empty values are used for the missing elements.

coreTools Command Reference Index

The **break** and **continue** statements may be invoked inside *body*, with the same effect as in the **for** command.
Foreach returns an empty string.

EXAMPLES

This loop prints every value in a list together with the square and cube of the value:

```
set values {1 3 5 7 2 4 6 8} ;# Odd numbers first, for fun!
puts "Value\tSquare\tCube"    ;# Neat-looking header
foreach x $values {           ;# Now loop and print...
    puts " $x\t [expr {$x**2}]\t [expr {$x**3}]"
}
```

The following loop uses *i* and *j* as loop variables to iterate over pairs of elements of a single list.

```
set x {}
foreach {i j} {a b c d e f} {
    lappend x $j $i
}
# The value of x is "b a d c f e"
# There are 3 iterations of the loop.
```

The next loop uses *i* and *j* to iterate over two lists in parallel.

```
set x {}
foreach i {a b c} j {d e f g} {
    lappend x $i $j
}
# The value of x is "a d b e c f {} g"
# There are 4 iterations of the loop.
```

The two forms are combined in the following example.

```
set x {}
foreach i {a b c} {j k} {d e f g} {
    lappend x $i $j $k
}
# The value of x is "a d e b f g c {} {}"
# There are 3 iterations of the loop.
```

SEE ALSO

`for(n)`, `while(n)`, `break(n)`, `continue(n)`

KEYWORDS

foreach, iteration, list, loop

`foreach_array_field_parameter`

loop through the parameters for the fields of an array.

Syntax

```
string foreach_array_field_parameter param_var index_var array_param body
    string param_var
    string index_var
    string array_param
    string body
```

Parameters

	Variable to hold this parameter
<code>param_var</code>	This variable is used within the <body> code to refer to the current sub-parameter.
<code>index_var</code>	Variable to hold the index number Values start at 0.
<code>array_param</code>	The name of the array parameter to loop over
	Script to execute per iteration
<code>body</code>	The 'param_var' and 'index_var' variables are updated prior to each invocation of the <body>.

Description

This command is used to set and/or get attributes from the individual sub-parameters within an array parameter. Array parameters are defined automatically in VHDL when you have a parameter which contains sub-types which are also arrays. See the User's Guide for example RTL fragments which result in the definition of array parameters.

Examples

Assuming an array parameter named P1, the following code will set a unique description and unique group name on each field (sub-parameter) within the array parameter.

```
foreach_array_field_parameter param idx P1 {
set_attribute $param -attr Description -value "Array parameter P1 element $idx."
set_attribute $param -attr GroupName -value "P1/$idx"
}
```

See Also

`foreach_in_collection`
Iterate over a collection

Syntax

```
string foreach_in_collection itr_var collections body
      string itr_var
      list collections
      string body
```

Parameters

<code>itr_var</code>	Iterator variable
<code>collections</code>	Collection(s) over which to iterate
<code>body</code>	Script to execute per iteration

Description

To iterate over each element in a collection, you must use the `foreach_in_collection` command. You cannot use the Tcl-supplied `foreach` command to iterate over collections because `foreach` requires a list, and a collection is not a list.

The arguments to `foreach_in_collection` parallel those of `foreach`: an iterator variable, the collections over which to iterate, and the script to apply at each iteration. All arguments are required.

During each iteration, `itr_var` is set to a collection of exactly one object. Any command that accepts collections as an argument accepts `itr_var`, because they are of the same data type (handle to a collection).

You can nest the `foreach_in_collection` command within other control structures, including `foreach_in_collection`.

Note that if the body of the iteration is modifying the knowledge database, it is possible that all or part of the collection will be deleted. The `foreach_in_collection` command is safe for such operations. If a command in the body causes the collection to be removed, at the next iteration, the iteration will end with a message indicating that the iteration ended prematurely.

Examples

An alternative to collection iteration is the use of complex filtering to create a collection which includes only the desired elements, then apply one or more commands to that collection. If the order of operations does not matter, the following are equivalent. The first is an example without iterators:

```
set s [find_item -type design core*]  
command1 $s  
command2 $s  
unset s
```

The following is the same example using `foreach_in_collection`.:

```
foreach_in_collection itr [find_item -type design core*] {  
  command1 $itr  
  command2 $itr  
}
```

For collections with large numbers of objects, the non-iterator version is more efficient, though both produce the same results if the commands are order independent.

See Also

[find_item \(2\)](#)

NAME

`format` - Format a string in the style of `sprintf`

SYNOPSIS

`format formatString ?arg arg ...?`

INTRODUCTION

This command generates a formatted string in a fashion similar to the ANSI C `sprintf` procedure. *FormatString* indicates how to format the result, using % conversion specifiers as in `sprintf`, and the additional arguments, if any, provide values to be substituted into the result. The return value from `format` is the formatted string.

DETAILS ON FORMATTING

The command operates by scanning *formatString* from left to right. Each character from the format string is appended to the result string unless it is a percent sign. If the character is a % then it is not copied to the result string. Instead, the characters following the % character are treated as a conversion specifier. The conversion specifier controls the conversion of the next successive *arg* to a particular format and the result is appended to the result string in place of the conversion specifier. If there are multiple conversion specifiers in the format string, then each one controls the conversion of one additional *arg*. The `format` command must be given enough *args* to meet the needs of

coreTools Command Reference Index

all of the conversion specifiers in *formatString*.

Each conversion specifier may contain up to six different parts: an XPG3 position specifier, a set of flags, a minimum field width, a precision, a size modifier, and a conversion character. Any of these fields may be omitted except for the conversion character. The fields that are present must appear in the order given above. The paragraphs below discuss each of these fields in turn.

OPTIONAL POSITIONAL SPECIFIER

If the % is followed by a decimal number and a \$, as in then the value to convert is not taken from the next sequential argument. Instead, it is taken from the argument indicated by the number, where 1 corresponds to the first *arg*. If the conversion specifier requires multiple arguments because of * characters in the specifier then successive arguments are used, starting with the argument given by the number. This follows the XPG3 conventions for positional specifiers. If there are any positional specifiers in *formatString* then all of the specifiers must be positional.

OPTIONAL FLAGS

The second portion of a conversion specifier may contain any of the following flag characters, in any order:

- Specifies that the converted argument should be left-justified in its field (numbers are normally right-justified with leading spaces if needed).
- + Specifies that a number should always be printed with a sign, even if positive.
- space Specifies that a space should be added to the beginning of the number if the first character is not a sign.
- 0 Specifies that the number should be padded on the left with zeroes instead of spaces.
- # Requests an alternate output form. For **o** conversions it guarantees that the first digit is always **0**. For **x** or **X** conversions, **0x** or **0X** (respectively) will be added to the beginning of the result unless it is zero. For **b** conversions, **0b** will be added to the beginning of the result unless it is zero. For all floating-point conversions (**e**, **E**, **f**, **g**, and **G**) it guarantees that the result always has a decimal point. For **g** and **G** conversions it specifies that trailing zeroes should not be removed.

OPTIONAL FIELD WIDTH

The third portion of a conversion specifier is a decimal number giving a minimum field width for this conversion. It is typically used to make columns line up in tabular printouts. If the converted argument

coreTools Command Reference Index

contains fewer characters than the minimum field width then it will be padded so that it is as wide as the minimum field width. Padding normally occurs by adding extra spaces on the left of the converted argument, but the **0** and **-** flags may be used to specify padding with zeroes on the left or with spaces on the right, respectively. If the minimum field width is specified as ***** rather than a number, then the next argument to the **format** command determines the minimum field width; it must be an integer value.

OPTIONAL PRECISION/BOUND

The fourth portion of a conversion specifier is a precision, which consists of a period followed by a number. The number is used in different ways for different conversions. For **e**, **E**, and **f** conversions it specifies the number of digits to appear to the right of the decimal point. For **g** and **G** conversions it specifies the total number of digits to appear, including those on both sides of the decimal point (however, trailing zeroes after the decimal point will still be omitted unless the **#** flag has been specified). For integer conversions, it specifies a minimum number of digits to print (leading zeroes will be added if necessary). For **s** conversions it specifies the maximum number of characters to be printed; if the string is longer than this then the trailing characters will be dropped. If the precision is specified with ***** rather than a number then the next argument to the **format** command determines the precision; it must be a numeric string.

OPTIONAL SIZE MODIFIER

The fifth part of a conversion specifier is a size modifier, which must be **ll**, **h**, or **l**. If it is **ll** it specifies that an integer value is taken without truncation for conversion to a formatted substring. If it is **h** it specifies that an integer value is truncated to a 16-bit range before converting. This option is rarely useful. If it is **l** it specifies that the integer value is truncated to the same range as that produced by the **wide()** function of the **expr** command (at least a 64-bit range). If neither **h** nor **l** are present, the integer value is truncated to the same range as that produced by the **int()** function of the **expr** command (at least a 32-bit range, but determined by the value of the **wordSize** element of the **tcl_platform** array).

MANDATORY CONVERSION TYPE

The last thing in a conversion specifier is an alphabetic character that determines what kind of conversion to perform. The following conversion characters are currently supported:

- | | |
|----------|---|
| d | Convert integer to signed decimal string. |
| u | Convert integer to unsigned decimal string. |
| i | Convert integer to signed decimal string (equivalent to d). |
| o | Convert integer to unsigned octal string. |

coreTools Command Reference Index

x or X	Convert integer to unsigned hexadecimal string, using digits for x and for X).
b	Convert integer to unsigned binary string, using digits 0 and 1.
c	Convert integer to the Unicode character it represents.
s	No conversion; just insert string.
f	Convert number to signed decimal string of the form <code>xx.yyy</code> , where the number of y 's is determined by the precision (default: 6). If the precision is 0 then no decimal point is output.
e or E	Convert number to scientific notation in the form <code>x.yyye+-zz</code> , where the number of y 's is determined by the precision (default: 6). If the precision is 0 then no decimal point is output. If the E form is used then E is printed instead of e .
g or G	If the exponent is less than -4 or greater than or equal to the precision, then convert number as for %e or %E . Otherwise convert as for %f . Trailing zeroes and a trailing decimal point are omitted.
%	No conversion: just insert %.

DIFFERENCES FROM ANSI SPRINTF

The behavior of the format command is the same as the ANSI C **sprintf** procedure except for the following differences:

- [1] Tcl guarantees that it will be working with UNICODE characters.
- [2] **%p** and **%n** specifiers are not supported.
- [3] For **%c** conversions the argument must be an integer value, which will then be converted to the corresponding character value.
- [4] The size modifiers are ignored when formatting floating-point values. The **ll** modifier has no **sprintf** counterpart. The **b** specifier has no **sprintf** counterpart.

EXAMPLES

Convert the numeric value of a UNICODE character to the character itself:

```
set value 120
set char [format %c $value]
```

Convert the output of **time** into seconds to an accuracy of hundredths of a second:

```
set us [lindex [time $someTclCode] 0]
puts [format "%.2f seconds to execute" [expr {$us / 1e6}]]
```

Create a packed X11 literal color specification:

```
# Each color-component should be in range (0..255)
set color [format "%02x%02x%02x" $r $g $b]
```

Use XPG3 format codes to allow reordering of fields (a technique that is often used in localized message catalogs; see **msgcat**) without reordering the data values passed to **format**:

```
set fmt1 "Today, %d shares in %s were bought at $%.2f each"
puts [format $fmt1 123 "Global BigCorp" 19.37]
```

```
set fmt2 "Bought %2\$s equity ($%3$.2f x %1\$d) today"
puts [format $fmt2 123 "Global BigCorp" 19.37]
```

Print a small table of powers of three:

```
# Set up the column widths
set w1 5
set w2 10

# Make a nice header (with separator) for the table first
set sep +-[string repeat - $w1]---[string repeat - $w2]--+
puts $sep
puts [format "| %-*s | %-*s |" $w1 "Index" $w2 "Power"]
puts $sep

# Print the contents of the table
set p 1
for {set i 0} {$i<=20} {incr i} {
    puts [format "| %*d | %*ld |" $w1 $i $w2 $p]
    set p [expr {wide($p) * 3}]
}

# Finish off by printing the separator again
puts $sep
```

SEE ALSO

scan(n), sprintf(3), string(n)

KEYWORDS

conversion specifier, format, sprintf, string,
substitution

generate_gtech_sim_model

Write scripts and return shell command to generate a GTECH model of the core

Syntax

```
string generate_gtech_sim_model -dir <dir> [-simple_gates]
[-include_cells <cell_list>] [-exclude_cells <cell_list>] [-ungroup_all]
[-netlist_format <format>] [-no_home_init]
[-preserve_verilog_bus_port_names <val>] [-enable_presto <val>]
[-fix_multiple_port_nets_default] [-fix_multiple_port_nets_feedthroughs]
[-fix_multiple_port_nets_outputs] [-fix_multiple_port_nets_constants]
[-fix_multiple_port_nets_buffer_constants] [-bottom_up_compile]
    string <dir>
    string <cell_list>
    string <format>
    string <val>
```

Parameters

-dir <dir>

Directory to write to
All data for the GTECH model generation will be contained below this directory. The value for this option should be a simple directory below the workspace.

-simple_gates

Use only simple gates
This option forces the set of gates in the generated netlist to only include GTECH_NOR2, GTECH_NOT, GTECH_FD2 and GTECH_FD4.

-include_cells <cell_list>

Additional GTECH cells to use.

-exclude_cells <cell_list>

GTECH cells not to use.

-ungroup_all

Ungroup all hierarchy after the compile

-netlist_format <format>

Format of the resulting netlist

This option defaults to the language of the current design. Valid values are determined by the

	NetlistFormat parameter on the DC_quick_map_strategy.
-no_home_init	Don't read \$HOME/.synopsys_dc.setup during synthesis Typically not needed, but useful if the setup file in \$HOME has errors in it.
-preserve_verilog_bus_port_names <val>	Sets dont_change_bus_members when change_names -rule verilog is run. (Values: 0, 1)
-enable_presto <val>	Explicitly control use of Presto (Values: 0, 1) If this option is not specified, then the use of Presto is determined by the value of the DC variable hdlin_enable_presto. If this option is specified, then hdlin_enable_presto is explicitly set to the given value.
-fix_multiple_port_nets_default	Force set_fix_multiple_port_nets -default. This option overrides all other fix_multiple_port_nets options
-fix_multiple_port_nets_feedthroughs	Force set_fix_multiple_port_nets -feedthroughs
-fix_multiple_port_nets_outputs	Force set_fix_multiple_port_nets -outputs
-fix_multiple_port_nets_constants	Force set_fix_multiple_port_nets -constants
-fix_multiple_port_nets_buffer_constants	Force set_fix_multiple_port_nets -buffer_constants
-bottom_up_compile	Force a bottom-up compile

Description

This command is used to generate a GTECH simulation model. This model is useful if the sources for this core have been encrypted. This command

write all necessary scripts into the directory provided by the `-dir` option, and returns a Bourne (`/bin/sh`) Shell script that will generate the simulation model.

The generated script sets the `qmap_netlist` shell variable to the location of the generated netlist, and the `qmap_errors` to the number of errors found during the model generation.

Examples

This command is generally used in a `PostPromptCmd` for an activity that will use the `launch_activity_subproc` command to run a simulation. A sample code fragment from the `PostPromptCommand` follows:

```
set cmd "$sysCmd(perl) setup_simulator.pl\"
append cmd [generate_gtech_sim_model -dir sim]
append cmd "$sysCmd(perl) run_simulation.pl\"
append cmd "$sysCmd(perl) gen_html_index.pl\"
launch_activity_subproc $act $cmd
```

See Also

[define_activity_subproc_params\(2\)](#), [launch_activity_subproc\(2\)](#)

generate_qtm

Generate a QTM (quick timing model) for the top design.

Syntax

```
string generate_qtm [-scripts_only] [-load ] [-driver ] [-setup ] [-hold
] [-clk_to_output ] [-fanout ] [-auxiliary_script ]
    string
    int
```

Parameters

-scripts_only	Generate scripts, but do not run them.
-load	Default load cell for input ports This option is for input ports which do not already have a specific value for the ExpectedLoad attribute. This cell is used in conjunction with the set_qtm_port_load command. If this option is not specified, a medium strength combinational cell will be utilized.
-driver	Default driver cell for output ports This option is for output ports which do not already have a specific value for the ExpectedDrivingCell attribute. This cell is used in conjunction with the set_qtm_port_drive command. If this option is not specified, a medium strength combinational cell will be utilized.
-setup	Cell from which to infer global setup time If this option is not specified, a medium strength sequential cell will be utilized.
-hold	Cell from which to infer global hold time If this option is not specified, a hold value of 0.0 will be utilized.
-clk_to_output	Cell from which to infer global clock to output time If this option is not specified, a medium strength sequential cell will be utilized.
-fanout	Default fanout for input ports This option is for input ports which do not have the PinLoadCount attribute explicitly set on them. A default of '1' is assumed if this option is not specified.
-auxiliary_script	Script to run prior to model generation This can be used to specify delays for combinational paths which are not otherwise

modeled, or to override other previously generated model commands.

Description

This command can be used to generate a PrimeTime QTM (quick timing model) for the current design. By default, this command will generate the TCL script required to build the model, and then invoke PrimeTime to actually build the model. The `-scripts_only` option can be used to prevent model generation if manual editing of the script is desired. In that case, the script should run by invoking the script file `'qtm_model_gen.scr'`. This file and all related files, including the generated model can be found in the `'model/qtm'` directory underneath the `coreConsultant` workspace.

Examples

To generate a QTM, assuming a default fanout of 2 on all input ports:

```
prompt> generate_qtm -fanout 2
Invoking pt_shell to generate the QTM.
Model file: <full path to model>
Log file: <full path to log file>
Errors: 0
Warnings: 2
```

See Also

generate_reports

Generate the indicated reports.

Syntax

```
string generate_reports [-cleanup] [-list] [-show] [-reports <reports>]
                        string <reports>
```

Parameters

-cleanup	Remove reports not being generated.
-list	List all reports available for generation.
-show	Show report in browser after generation.
-reports <reports>	List of reports to be generated.

Description

This command is used to generate reports associated with the current component or subsystem. Each of the reports specified with the -reports option is generated. To see a list of available reports, use 'generate_reports -list'.

Reports can be specified using the full or compressed name for a given report. The compress names are generated from the full names by removing all spaces, and the '/', and '-' characters.

Reports can also be generated when using the tool in graphical mode via the File-->Generate Reports menu or via the activity summary page shown when an activity is complete. Note that the activity summary page only shows and enables generation of reports associated with the given activity.

Reports generated in graphical mode are tracked and a corresponding call to generate_reports will be included in any batch script generated for the open workspace.

Examples

To check all available reports to generate:

```
coreAssembler> generate_reports -list
{AXI Slave Visibility} {Area Estimate} {Component Configuration}
{Component Registers} {Component Source Code} Connection I/O {Memory Map}
{Packed Structs Unions} {Power Intent} {Subsystem Contents} {Subsystem
```

Source Code}{Testbench Source Code}

Generate the I/O and Component Configuration reports:

```
coreAssembler> generate_reports -reports {I/O {Component Configuration}}
```

Do the same using the compressed report names:

```
coreAssembler> generate_reports -reports {IO ComponentConfiguration}
```

Generate the Packed Structs Unions report:

```
coreAssembler> generate_reports -reports {{Packed Structs Unions}}
```

Generate the Packed Structs Unions report using the compressed report name:

```
coreAssembler> generate_reports -reports {PackedStructsUnions}
```

See Also

[generate_views\(2\)](#)

[API Index](#) [Command Reference](#)

`generate_simulation_file_list`
Generate a list of simulation files

Syntax

```
string generate_simulation_file_list -output <file> -simulator  
<simulator>  
    string <file>  
    string <simulator>
```

Parameters

<code>-output <file></code>	File to be written
<code>-simulator <simulator></code>	Simulator to use

Description

Creates a file containing a list of all HDL files in the design for the given simulator. The name of this file can be used as input to `generate_simulation_launch_command`.

Examples

To create a file called `file_list` that contains a list of all the HDL files in the design for the VCS simulator:

```
generate_simulation_file_list -output file_list -simulator VCS
```

See Also

[generate_simulation_launch_command \(2\)](#)

`generate_simulation_launch_command`

Generate the command to launch the simulation

Syntax

string **generate_simulation_launch_command** -output *<file>* -listfile *<file>*`-simulator <simulator>`string *<file>*string *<simulator>*

Parameters

`-output <file>`

File to be written

`-listfile <file>`

File containing list of HDL files to compile

`-simulator <simulator>`

Simulator to use

Description

Creates a file containing the necessary commands to run a simulation on the given simulator. The listfile argument is the filename that contains a list of HDL files to compile and will be inserted into the command string. This list can be generated with `generate_simulation_file_list`.

Examples

To create a file called `launch_cmd` that contains the commands to invoke the VCS simulator on a design consisting of the Verilog files in `file_list`:

`generate_simulation_launch_command -output launch_cmd -listfile file_list`

\

`-simulator VCS`

See Also

[generate_simulation_file_list\(2\)](#)

`generate_views`

Generate the indicated optional views.

Syntax

```
string generate_views [-list] [-show] [-views <views>]
      string <views>
```

Parameters

<code>-list</code>	List all views available for generation.
<code>-show</code>	Show view summary after generation.
<code>-views <views></code>	List of views to be generated.

Description

This command is used to generate optional views associated with the current component or subsystem. Each of the optional views specified with the `-views` option is generated. To see a list of available views, use `'generate_views -list'`.

Views can be specified using the full or compressed name for a given view. The compress names are generated from the full names by removing all spaces, and the `'/'`, and `'-'` characters.

Views can also be generated when using the tool in graphical mode via the File-->Generate Optional Views menu or via the activity summary page shown when an activity is complete. Note that the activity summary page only shows and enables generation of optional views associated with the given activity.

Views generated in graphical mode are tracked and a corresponding call to `generate_views` will be included in any batch script generated for the open workspace.

Examples

Generate the IP-XACT Component and Component RAL views:

```
coreAssembler> generate_views -viewss {{IP-XACT Component} {Component
RAL}}
```

Do the same using the compressed view names:

```
coreAssembler> generate_views -viewss {IPXACTComponent ComponentRAL}
```

See Also

[generate_reports \(2\)](#)

[API Index](#) [Command Reference](#)

get_TMR_separation_distance

Retruns the TMR Separation Distance

Syntax

string **get_TMR_separation_distance**

Description

Examples

See Also

`get_VLNV_search_path_entries`

Get elements of given type and VLNV from the cache.

Syntax

```
string get_VLNV_search_path_entries -types <entry types> [VLNV]
      string <entry types>
      string VLNV
```

Parameters

<code>-types <entry types></code>	Types of elements to be located.
<code>VLNV</code>	The {Vendor Library Name Version} of the element(s)

Description

This command is used to determine which coreKits and/or SPIRIT files are available via the current search path. All entries which match the specified type and VLNV criteria are returned as a list of lists. Each list entry represents one coreKit or SPIRIT file. The entry is itself a list containing two elements. The first is the VLNV of the coreKit or SPIRIT file and the second is the full path to the coreKit installation area (for a coreKit) or to the actual XML file for a SPIRIT file.

Multiple types can be specified with the `-types` option and results are "anded" together. Legal type values include: `coreKit`, `component`, `busDefinition`, `pmd`, `generatorChain`, `design`, and `designConfiguration`.

The VLNV specified must contain four fields (vendor, library, name, and version). If you do not care about the value of a particular field, set it to `"\S+"` so that it will match all values.

Examples

Get all coreKits and SPIRIT component files:

```
get_VLNV_search_path_entries -types {coreKit component}
Get all bus definitions with Synopsys as the vendor:
```

```
get_VLNV_search_path_entries -types busDefinition {Synopsys \S+ \S+ \S+}
```

See Also

`get_activity_parameter`

Get the value of parameter for an activity

Syntax

```
string get_activity_parameter [-param] [-quiet] [-component
componentName] activityName paramName
```

*string componentName**string activityName**string paramName*

Parameters

<code>-param</code>	Return parameter instead of its value.
<code>-quiet</code>	Do not issue user error message if not found.
<code>-component componentName</code>	Component associated with the activity. This option is used within coreAssembler to specify which component an activity is associated with. It is only needed when the same activity appears in more than one component.
<code>activityName</code>	The name of the activity for which you want to get the parameter.
<code>paramName</code>	The name of the parameter on the specified activity.

Description

The `get_activity_parameter` command returns the current value of the specified parameter of the specified activity. When using a coreTool, you can use `get_activity_parameter` to get the value of an activity parameter when working in command line mode or when writing batch mode command files.

For coreBuilder users, a common use for `get_activity_parameter` is to retrieve the value of an activity parameter for a custom activity for use as input to the Tcl command procedure the performs the action associated with the activity. When you create custom activities, you use the coreBuilder `create_custom_activity_parameter` command to create an activity parameter and attach the parameter to your custom activity. In the Tcl code that implements the custom activity, you can use the `get_activity_parameter` command to retrieve the user-specified value for the activity parameter. Refer to the *coreBuilder User Guide* for examples of how to use `get_activity_parameter` with custom activities.

For all the coreTools, if you specify the -param option, `get_activity_parameter` does not return the parameter's current value. Instead, `get_activity_parameter` creates a collection that contains the specified parameter and returns the collection handle.

Examples

To get the current value of the `DoCompletenessCheck` parameter on the `VerifyBudgets` activity:

```
coreConsultant> get_activity_parameter VerifyBudgets DoCompletenessCheck  
1
```

The following line in a Tcl command procedure for a custom activity gets the user-specified value for the `Simulator` parameter on the custom activity named `SimulateDesign` and stores the value in a Tcl variable named `sim`:

```
set sim [get_activity_parameter SimulateDesign Simulator]
```

The following command creates a Tcl collection that contains the `StrategyName` parameter on the `Synthesize` activity and stores the collection handle in the Tcl variable `strategy`:

```
coreConsultant> set strategy [get_activity_parameter -param Synthesize  
StrategyName] {StrategyName}
```

In `coreAssembler`, two different components contain an activity named `Simulate` with a parameter named `Simulator`. To get the simulator for the activity associated with component `U1`:

```
coreAssembler> set sim [get_activity_parameter -component U1 Simulate  
Simulator]
```

See Also

[report_activity_parameters \(2\)](#), [set_activity_parameter \(2\)](#),
[create_custom_activity \(2\)](#), [create_custom_activity_parameter \(2\)](#)

[API Index](#) [Command Reference](#)

`get_address_bank_attribute`

Get the value of an attribute from an address bank.

Syntax

```
string get_address_bank_attribute bank attr  
    string bank  
    string attr
```

Parameters

`bank` Name of the address bank to get attribute from.

`attr` Name of the attribute to get.

Description

Gets an attribute from an address bank

Examples

```
coreBuilder> get_address_bank_attribute map/bank BaseAddress
```

See Also

[addressBank \(3\)](#), [set_address_bank_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

`get_address_block_attribute`

Get the value on an attribute from an address block.

Syntax

```
string get_address_block_attribute block attr  
    string block  
    string attr
```

Parameters

<code>block</code>	Name of the address block.
<code>attr</code>	Name of the attribute to get.

Description

Gets an attribute from an address block.

Examples

```
coreBuilder> get_address_block_attribute map/block1 BaseAddress
```

See Also

[addressBlock \(3\).set_address_block_attribute \(2\)](#)

`get_all_bits`

Return all bits of the specified port/pin, or port/pin itself if it is a single bit item.

Syntax

```
string get_all_bits [-context <context>] port_or_pin  
    string <context>  
    string port_or_pin
```

Parameters

<code>-context <context></code>	Context in which to look for port/pin
<code>port_or_pin</code>	A port/pin name to get bits for.

Description

This command returns a collection of busbits of the specified port/pin. If the specified port/pin is a single bit item, it will return the specified port/pin itself.

Examples

If we have top design port `topa[7:0]` connected to pin `U1/a[7:0]`, use the following command to get all the busbits of port `topa`:

```
coreBuilder> get_all_bits topa
```

Use the following command to get all the busbits of pin `U1/a`:

```
coreBuilder> get_all_bits U1/a
```

See Also

[get_connections \(2\)](#)

NAME

get_app_var Gets the value of an application variable.

SYNTAX

```
string get_app_var  
    [-default | -details | -list]  
    [-only_changed_vars]  
    var
```

Data Types
var string

ARGUMENTS

-default	Gets the default value.
-details	Gets additional variable information.
-list	Returns a list of variables matching the pattern. When this option is used, then the <i>var</i> argument is interpreted as a pattern instead of a variable name.
-only_changed_vars	Returns only the variables matching the pattern that are not set to their default values, when specified with -list .
var	Specifies the application variable to get.

DESCRIPTION

The **get_app_var** command returns the value of an application variable.

There are four legal forms for this command:

- o **get_app_var <var>**
Returns the current value of the variable.
- o **get_app_var <var> -default**
Returns the default value of the variable.
- o **get_app_var <var> -details**
Returns more detailed information about the variable. See below for details.
- o **get_app_var -list [-only_changed_vars] <pattern>**
Returns a list of variables matching the pattern. If *-only_changed_vars* is specified, then only variables that are changed from their default values are returned.

In all cases, if the specified variable is not an application variable, then a Tcl error is returned, unless the application variable **sh_allow_tcl_with_set_app_var** is set to true. See the **sh_allow_tcl_with_set_app_var** man page for details.

When **-details** is specified, the return value is a Tcl list that is suitable as input to the Tcl **array set** command. The returned value is a list with an even number of arguments. Each odd-numbered element in the list is a key, and each even-numbered element in the list is the value of the previous key.

The supported keys are as follows:

<i>name</i>	This key contains the name of the variable. This key is always present.
<i>value</i>	This key contains the current value of the variable. This key is always present.
<i>default</i>	This key contains the default value of the variable. This key is always present.
<i>help</i>	This key contains the help string for the variable. This key is always present, but sometimes the value is empty.
<i>type</i>	This key contains the type of the application variable. Legal values of

coreTools Command Reference Index

for this key are: string, bool, int, real. This key is always present.

<i>constraint</i>	This key describes additional constraints placed on this variable. Legal values for this key are: none, list, range. This key is always present.
<i>min</i>	This key contains the min value of the application variable. This key is present if the constraint is range. The value of this key may be the empty string, in which case the variable only has a max value constraint.
<i>max</i>	This key contains the max value of the application variable. This key is present if the constraint is "range". The value of this key may be the empty string, in which case the variable only has a min value constraint.
<i>list</i>	This key contains the list of legal values for the application variable. This key is present if the constraint is "list".

EXAMPLES

The following are examples of the **get_app_var** command:

```
prompt> get_app_var sh_enable_page_mode
1
```

```
prompt> get_app_var sh_enable_page_mode -default
false
```

```
foreach {key val} [get_app_var sh_enable_page_mode -details] { echo "$key: $val" }
=> name: sh_enable_page_mode
    value: 1
    default: false
    help: Displays long reports one page at a time
    type: bool
    constraint: none
```

```
prompt> get_app_var -list sh_*message
sh_new_variable_message
```

SEE ALSO

report_app_var(2)
set_app_var(2)
write_app_var(2)

`get_associated_instance_parameter`

Get parameter value of linked instantiation parameter.

Syntax

```
string get_associated_instance_parameter iparam  
    string iparam
```

Parameters

`iparam` Interface parameter which links an instantiation parameter.

Description

Sometimes an interface parameters value should come from a linked design parameter.

Examples of this situation are when the linked parameter should not be modified by the end user.

Examples

An interrupt interface definition supports specification of an `ActiveLevel` for the interrupt signal. The linked parameter is not always Enabled or is not modifiable by the user. To force the values to be consistent do the following:

```
set_interface_parameter_attribute -instance $singleName ActiveLevel  
Visible false  
set_interface_parameter_attribute -instance $singleName ActiveLevel \  
Value {=get_associated_instance_parameter %item}  
set_interface_parameter_attribute -instance $singleName ActiveLevel \  
ReadOnlyParam true
```

See Also

get_attribute

Get the current value(s) of the specified attribute(s)

Syntax

```
string get_attribute [-no_eval] [-eval_default] [-include_names] [-attrs
<attribute names>] [-subscript <attribute subscript>] [-alias]
[-fail_if_unset] [-with_units] [-no_proxy] [<item name>]
    list <attribute names>
    string <attribute subscript>
    string <item name>
```

Parameters

-no_eval	If the attribute value is a formula, do not evaluate the formula. If you specify -no_eval, and the current value of the attribute is a formula, get_attribute returns the formula without evaluating it. The formula may be an explicitly specified formula or the default formula used to determine the default value for the attribute.
-eval_default	If the current attribute value is derived from the default formula, evaluate the default formula. If you specify -no_eval, you can use -eval_default to evaluate the current formula value for the attribute only if it is the default formula.
-include_names	Return both the name and value of each attribute specified.
-attrs <attribute names>	The names of the attribute(s) for which you want to get the values.
-subscript <attribute subscript>	Specifies which subscript of the specified attribute to get. If the subscript is valid, but the attribute is not set on the item with that particular subscript, get_attribute returns the default value for that subscript (if a default value exists). get_attribute will fail if the specified subscript is not valid on all specified attributes.
-alias	Return symbolic name instead of actual value if attribute values have symbolic names. For example, boolean attributes have symbolic names "true" and "false" for the actual values 1 and 0. If you specify -alias and the current value for the

coreTools Command Reference Index

	specified boolean attribute is 0, <code>get_attribute</code> returns the symbolic name "false". If a symbolic name is not specified for the current value, the actual current value is returned.
<code>-fail_if_unset</code>	Force <code>get_attribute</code> to fail and return an error message if the specified attribute is not set. If you do not specify <code>-fail_if_unset</code> , <code>get_attribute</code> completes successfully without returning any value if the attribute is not set.
<code>-with_units</code>	Append units to values for attributes that have a <code>UnitType</code> . If there is a dialog posted for the attribute, do not get the value that is currently in the dialog. Instead get the currently stored value for the attribute. If you do not specify <code>-no_proxy</code> , <code>get_attribute</code> returns the value that is currently posted in the dialog.
<code>-no_proxy</code>	
<code><item name></code>	The name of the item for which you want to get the specified attribute.

Description

The `get_attribute` command returns the current value of the specified attribute on the specified item. If you specify one attribute name, `get_attribute` returns the value of that attribute. If you specify more than one attribute, `get_attribute` returns a list of the values of specified attributes. If you do not specify any attribute name, `get_attribute` returns a list of the names and values of all attributes that are currently set on the specified item, either explicitly or by default.

If the current value of the attribute is a formula, `get_attribute` evaluates the formula and returns the result. If you specify the `-no_eval` option, `get_attribute` returns the formula. The formula returned is either an explicitly specified formula or the default formula that the `coreTool` uses internally to calculate the default value for the attribute. For example, the default formula for the `CanFlatten` attribute is `{=InheritValue up 0}`. If you want to evaluate the formula only if it is the default formula, specify both `-no_eval` and `-eval_default`.

For subscripted attributes, use the `-subscript` option to specify the subscript for which you want to get the value. For example, to get the value of `AuxSynthesisScript[constrain]`, use the options `"-attrs AuxSynthesisScript -subscript constrain"`.

For attributes that have units associated with their values (for example, time-related attributes), use the `-with_units` option to return the unit with the value.

There are additional commands that you can use to get values of attributes on cells, clocks, designs, ports, and parameters (for example, `get_design_attribute`). These commands are not as flexible as `get_attribute`, but can be easier to use, particularly for specifying the item. For example, a design can have a port, net, and clock that all have the name `clk`. To get the value of an attribute on the clock named `clk` with `get_attribute`, you would have specify the clock as `[find_item -type clock clk]`; whereas with the `get_clock_attribute` command, you can just specify the clock item as `"clk"`.

Examples

To get the value of the `MaxInputDelay` attribute on the port named `input_1`:

```
coreConsultant> get_attribute -attrs MaxInputDelay [find_item -type
port input_1]
2.5
```

To get the value of the `MaxInputDelay` attribute on the port named `input_1`, but do not evaluate the formula:

```
coreConsultant> get_attribute -no_eval -attrs MaxInputDelay [find_item
-type
port input_1]
=percent_of_period 25
```

To get the values of `MinInputDelay` and `MaxInputDelay` on `input_1`, including attribute names and value units:

```
coreConsultant> set inputport [find_item -type port input_1]
{input_1}
coreConsultant> get_attribute -include_name -with_units -attrs
{MinInputDelay MaxInputDelay} $inputport
MinInputDelay 2.5ns MaxInputDelay 2.5ns
```

To get the value of `CompileMapEffort[initial]` on `MyDesign`:

```
coreConsultant> get_attribute -attrs CompileMapEffort -subscript initial
MyDesign
high
```

To get the symbolic name for the current value of a parameter `MyEnum` with `EnumValues "0 1 2"` and `SymbolicNames "red blue green"`:

```
coreConsultant> get_attribute -alias -attr Value [find_item -type param
MyEnum]
green
```

See Also

[get_clock_attribute\(2\)](#), [get_design_attribute\(2\)](#), [get_port_attribute\(2\)](#), [get_cell_attribute\(2\)](#), [get_parameter_attribute\(2\)](#), [set_attribute\(2\)](#), [report_attribute\(2\)](#)

[API Index](#) [Command Reference](#)

`get_bit_driver`

Get the driver for this connection.

Syntax

```
string get_bit_driver bit  
      string bit
```

Parameters

bit bit to get the source of

Description

This command will get the driver connected to a bit or a pin/port that is 1 bit wide.

Examples

```
coreAssembler> set driver [get_bit_driver $bit]
```

See Also

[get_bit_loads \(2\)](#)

[API Index](#) [Command Reference](#)

get_bit_loads

Get the bit loads for this source connection.

Syntax

```
string get_bit_loads bit  
      string bit
```

Parameters

bit bit to get the loads of

Description

This command will get all loads connected to a bit or a pin/port that is 1 bit wide.

Examples

```
coreAssembler> set loads [get_bit_loads $bit]
```

See Also

[get_bit_driver \(2\)](#)

`get_bitfield_value`

Get value of bit range with a given parameter or numeric value.

Syntax

string **get_bitfield_value** [-param] -offset <bit offset> -size <# bits>*paramOrNum*

int <bit offset>

int <# bits>

string *paramOrNum*

Parameters

	-param	Indicates that the incoming value is a parameter.
	-offset <bit offset>	Offset into incoming value (from right)
-size <	bits>	Number of bits to extract beginning at offset
	<i>paramOrNum</i>	Parameter or numeric value

Description

Returns the indicated bit field within an explicitly specified number or from the value of the given parameter. The value is right shifted by <bit offset> and then masked to contain only <size> bits.

Examples

Get bitfield from a number:

coreAssembler> `get_bitfield_value -offset 1 -size 2 0x5`

The result in this case is '2'.

Get bitfield from a parameter name:

coreAssembler> `get_bitfield_value -param -offset 1 -size 2 TheParamName`

Get bitfield from a parameter (collection):

coreAssembler> `get_bitfield_value -param -offset 1 -size 2 $p`

See Also

get_cell

Command to get the cell from a cell name or path.

Syntax

```
string get_cell [-design] [-quiet] <name>  
      string <name>
```

Parameters

-design	Return the design not the cell.
-quiet	Do not print informational messages.
<name>	Name or full path of the cell

Description

Use this command to get a cell or design from a component name. Use the option -quiet to suppress a Tcl error message if the component or design can not be located.

Examples

```
coreAssembler> set cell [get_cell i_axi1]"  
coreAssembler> set design [get_cell -design i_axi1]"  
coreAssembler> set cell [get_cell -quiet /hier1/i_axi1]"
```

See Also

get_cell_attribute

Get the value of an attribute on a design instance

Syntax

```
string get_cell_attribute cell attr  
    string cell  
    string attr
```

Parameters

cell	The name of the cell (design instance) for which you want to get the attribute value. The name of the attribute for which you want to get the value.
attr	For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so that get_cell_attribute does not interpret the subscript as a sub-command.

Description

The get_cell_attribute command returns the current value of the specified attribute on the specified cell of the current_design. Specify the cell by the cell (instance) name. To set the value of an attribute on a cell, use the set_cell_attribute command.

Examples

To get the value of the DontTouch attribute on the cell named u1 in the current_design:

```
coreConsultant> get_cell_attribute u1 DontTouch  
0
```

See Also

[set_cell_attribute \(2\)](#)

get_children

Get the child/children of the given item

Syntax

string **get_children** [-name <name>] [-type <item type>] [-filter <filter expression>] [-sort <sort expression>] [-match] <hierarchical item>

string <name>
string <item type>
string <filter expression>
string <sort expression>
string <hierarchical item>

Parameters

-name <name>	Name of child to be retrieved.
-type <item type>	the type of item (Values: knowledgeBase, generic, file, attrDefn, item, activity, view, viewColumn, libItem, Dialog, dlgItem, dlgLabel, dlgValue, dlgButton, dlgCheckBox, dlgGroupBox, dlgStringsCombo, dlgStringsList, dlgView, design, param, port, pin, cell, cnct, net, dlgItemsCombo, dlgItemsList, Sheet, Page, GlobalActivity, DesignActivity, HierActivity, Strategy, genHierItem, busBit, menuCmd, menuCmdGroup, filegroup, dlgEditableView, formula, dlgEditableStringsList, connection, clock, state, preference, hdlFunc, dlgBitmap, timingException, filegroupGroup, bomTemplate, interfaceDefn, interfacePort, interfaceInstance, hdlType, hdlPackage, Schematic, dlgBrowser, viewRow, hierViewRow, memMap, register, registerField, addressBlock, addressBank, registerArray, intfPin, intfPort)
-filter <filter expression>	Filter to be applied to the collection.
-sort <sort expression>	Sort expression to apply to returned collection.
-match	Match expected. Error return if no items found.
<hierarchical item>	The item whose child/children are requested.

Description

Examples

See Also

get_clock_attribute

Get the value of an attribute on a clock

Syntax

```
string get_clock_attribute clock attr  
    string clock  
    string attr
```

Parameters

clock	The name of the clock for which you want to get the attribute value. The name of the attribute for which you want to get the value. For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so that get_clock_attribute does not interpret the subscript as a sub-command.
attr	

Description

The `get_clock_attribute` command returns the current value of the specified attribute on the specified clock. For a list of available attributes, see the man page for the clock object type referenced below.

Examples

To get the value of the CycleTime attribute on the clock named clk:

```
coreConsultant> get_clock_attribute clk CycleTime  
10
```

See Also

[set_clock_attribute \(2\)](#), [clock \(3\)](#)

`get_clocks`

Create a collection of all clocks associated with the specified design

Syntax

```
string get_clocks [-unmerged] [-merged] [<design name>]
                string <design name>
```

Parameters

<code>-unmerged</code>	Don't merge clocks with common system clock name.
<code>-merged</code>	Merge clocks with common system clock name.
<code><design name></code>	The design for which you want to create a collection of clocks.

Description

The `get_clocks` command returns a collection of all clocks defined for the specified design. The collection includes clocks on input ports, clocks on output ports (generated clock sources), and virtual clocks.

The `-merged` and `-unmerged` options are applicable to `coreAssembler` only. After completion of the Initialize Subsystem Constraints activity it is possible that multiple component level clocks from the same component may be merged into a single clock at the system level (due to connectivity). The default behavior of `get_clocks` is to merge component clocks that are merged at the system-level in the returned clock list unless the command is invoked within a component specific activity. The `-merged` and `-unmerged` options can be used to get the opposite behavior. To get merged clocks when invoking `get_clocks` within a component specific activity, use the `-merged` option. To get unmerged clocks when invoking `get_clocks` outside of the context of a component specific activity, use the `-unmerged` option.

Examples

To create a collection of all clocks defined for the current design:

```
coreConsultant> get_clocks
{clk}
```

To create a collection of all clocks defined for the design named `Subblock_A`, store the collection handle in the Tcl variable `A_clocks`:

```
coreConsultant> set A_clocks [get_clocks Subblock_A]
```

{clk1 clk2}

See Also

[create_virtual_clock \(2\)](#), [get_clock_attribute \(2\)](#), [set_clock_attribute \(2\)](#), [ClockName \(3\)](#)

NAME

get_command_hooks

Get registered hooks for this command

SYNTAX

```
string get_command_hooks commandName
```

```
string     commandName
```

ARGUMENTS

commandName Command to query

DESCRIPTION

The **get_command_hooks** command returns information about any hooks registered on the specified command. The data returned is in Tcl dict format and may be queried using that command.

EXAMPLES

When a command has both before and after hooks registered the data will look like this

```
prompt> get_command_hooks place_opt  
before {before0 report_timing}  
after  {after0 {echo "Done with placement"}}
```

SEE ALSO

`add_command_hook(2)`
`remove_command_hook(2)`
`get_current_hook_command(2)`
`dict(2)`

NAME

get_command_option_values
Queries current or default option values.

SYNTAX

```
get_command_option_values  
    [-default | -current]  
    -command command_name
```

Data Types
command_name string

ARGUMENTS

-default Gets the default option values, if available.

-current Gets the current option values, if available.

-command *command_name*
 Gets the option values for this command.

DESCRIPTION

This command attempts to query a default or current value for each option (of the command) that has default and/or current-value-tracking enabled. Details of how the option value is queried depend on whether one of the **-current** or **-default** options is specified (see below).

A "Tcl array set compatible" (possibly empty) list of option names and values is returned as the Tcl result. The even-numbered entries in the list are the names of options that were enabled for default-value-tracking or current-value-tracking and had at least one of these

coreTools Command Reference Index

values set to a not-undefined value). Each odd-numbered entry in the list is the default or current value of the option name preceding it in the list.

Any options that were not enabled for either default-value-tracking nor current-value-tracking are omitted from the output list. Similarly, options that were enabled for default-value-tracking or current-value-tracking, but for which no (not-undefined) default or current value is set, are omitted from the result list. If neither **-current** nor **-default** is specified, then for each command option that has either default-value-tracking or current-value-tracking (or both) enabled, the value returned is as follows:

- o The current value is returned if current-value-tracking is enabled and a (not-undefined) current value has been set;
- o Otherwise the default value is returned if default-value-tracking is enabled and a (not-undefined) default value has been set;
- o Otherwise the name and value pair for the option is not included in the result list.

If **-current** is specified, the value returned for an option is the current value if current-value-tracking is enabled, and a (not-undefined) current value has been set; otherwise the name and value pair for the option is omitted from the result list.

If **-default** is specified, the value returned for an option is the default value if default-value-tracking is enabled, and a (not-undefined) default value has been set; otherwise the name and value pair for the option is omitted from the result list.

The result list from **get_command_option_values** includes option values of both dash options and positional options (assuming that both kinds of options of a command have been enabled for value-tracking).

The command issues a Tcl error in a variety of situations, such as if an invalid command name was passed in with **-command**.

EXAMPLES

The following example shows the use of **get_command_option_values**:

```
prompt> test -opt1 10 -opt2 20
1

prompt> get_command_option_values -command test
-bar1 10 -bar2 20
```

SEE ALSO

`preview(2)`
`set_command_option_value(2)`

`get_component_design`

Command to get the elaborated or unelaborated design for a component.

Syntax

```
string get_component_design [-elaborated] [-unelaborated] [-current]  
[-all] [-quiet] <component>  
    string <component>
```

Parameters

<code>-elaborated</code>	Return the elaborated design or NULL if not elaborated.
<code>-unelaborated</code>	Return the unelaborated design.
<code>-current</code>	Return the referenced design regardless of elaboration state.
<code>-all</code>	Return the top design and all of the sub-designs.
<code>-quiet</code>	Do not print informational messages.
<code><component></code>	Name or full path of the component.

Description

Examples

See Also

[API Index](#) [Command Reference](#)`get_component_name`

Get the component name of the given item.

Syntax

```
string get_component_name item  
    string item
```

Parameters

`item` Item whose component is needed

Description

This command can be used to determine the name of the component containing the give item. This is typically used in plugins when you have an item and need to know in which component it belongs. If the item is not contained within a component, the command will return the empty string. The most common case where this will occur is when asking for the component name of an exported interface.

Examples

Get the component name of the given interface instance.

```
set intf [index_collection [find_item -type interfaceInstance Intr] 0]  
set compname [get_component_name $intf]
```

See Also

[set_current_component\(2\), get_current_component\(2\)](#)

get_component_view

Get the view associated with the current/selected component.

Syntax

```
string get_component_view [-component <component name>]  
    string <component name>
```

Parameters

-component <component name>	Get the view associated with the given component.
-----------------------------	---

Description

This command returns the name of the currently selected view of the specified component. Components instantiated via a SPIRIT IP-XACT component description can have multiple views and the selected view impacts the processing of the component. For example a component may have both a Verilog and a VHDL view with different RTL source files associated with each view. The currently selected view can be set using the `set_component_view` command.

Examples

```
set view [get_component_view -component i_uart]
```

See Also

[set_component_view \(2\)](#)

get_configuration_parameter

Get configuration (design) parameter value

Syntax

```
string get_configuration_parameter [-component <component>] [-param]
[-design] parameter
      string <component>
      string parameter
```

Parameters

-component <component>	Component on which you want to get the parameter value. This option is only needed and can only be used in coreAssembler.
-param	Return parameter instead of its value.
-design	Get the design parameter when both a design and package parameter are present.
parameter	The name of the parameter for which you want to get the value. An optional index or index range can be used with the parameter name for bitfield parameters only.

Description

This is the command that should be used to get design configuration parameters. The optional index range can be used to get the value of the parameter for that specific range. This command is typically used in customization scripts in order to perform some type of configuration specific action.

Examples

To get the value of the width parameter:

```
prompt> set value [get_configuration_parameter width]
```

To get the value for range [3:0] of a bitfield parameter '[7:0] P = 0xf6':

```
prompt> set value [get_configuration_parameter P[3:0]]
0x6
```

To get value for range [7:4] of a bitfield parameter '[7:0] P = 0xf6' for component 'C' (in coreAssembler):

```
prompt> set value [get_configuration_parameter -component C P[7:4]]  
0xf
```

See Also

[get_configuration_parameter_attribute\(2\), set_configuration_parameter](#)
(2)

get_configuration_parameter_attribute

Get the value of an attribute from the specified configuration parameter

Syntax

```
string get_configuration_parameter_attribute [-component <component>]
[-design] parameterName attr
    string <component>
    string parameterName
    string attr
```

Parameters

	Parent component for the configuration parameter; "" identifies current.
-component <component>	Identifies a specific component in a coreAssembler subsystem to which the named parameter belongs. This allows configuration of individual components from coreAssembler's subsystem perspective. Note that inside a subsystem component the "" annotates the component itself and not the subsystem, as set_current_component would do.
-design	Get the design parameter when both a design and package parameter are present.
parameterName	The configuration parameter to get the attribute value from
attr	The name of the attribute to get the value from

Description

This command is used to access the value of the named attribute on the specified configuration parameter. This command is not required in standard usage of the tools, but is provided as a complement to set_configuration_parameter_attribute. This command is typically used in customization scripts in order to perform some type of configuration specific action.

Examples

To check whether parameter PHY0 is enabled in component i_usb (coreAssembler only):

```
prompt> set enabled [get_configuration_parameter_attribute -component
i_usb PHY0 Enabled]
```

To check whether parameter PHY0 is enabled (all tools):

```
prompt> set enabled [get_configuration_parameter_attribute PHY0 Enabled]
```

See Also

get_configuration_parameter (2), set_configuration_parameter_attribute
(2), set_current_component (2)

`get_connections`

get connections for the specified single bit port/pin/busbit

Syntax

```
string get_connections [-names] [-hierarchy] [-sort] [-withcontext]
[-bitlevel] [-passthrough <attr>] [-fullpath] [-quiet] <port/pin name>
    string <attr>
    string <port/pin name>
```

Parameters

<code>-names</code>	return a list of user names of the connections
<code>-hierarchy</code>	get connection end-points crossing hierarchical boundaries
<code>-sort</code>	sort returned connections by direction and name
<code>-withcontext</code>	return context integrated into the result
<code>-bitlevel</code>	work at bitlevel even if bussed connection is fully connected
<code>-passthrough</code> <code><attr></code>	do not stop at a connection with <code><attr></code> set on it
<code>-fullpath</code>	return full path details
<code>-quiet</code>	don't complain if pin is not found
<code><port/pin name></code>	port/pin to get connections for

Description

This command will traverse the design hierarchy to get the connections for the specified port/pins. A single bit port/pin or a busbit must be specified to get connections for. It will return a collection of connected port/pins for the specified port/pin including the specified port/pin itself. use `-names` if you want to get a list of UserNames of the connections.

In coreAssembler, using the `-hierarchy` option returns connection end points crossing hierarchical boundaries. When this option is used, the global variable `::get_connections_contexts` is set to a list of component contexts corresponding to the component context of each return connection object (or name if `-names` is used). This can be used to determine the exact object being returned in the case of shared hierarchical components where the leaf object or pin names returned can be duplicates from different instances of shared hierarchies.

With the `-withcontext` option, the result will contain the concatenated context information along with the connections. It will be a list of path where each path element is a pair of `<receiver>` and `<driver>`. If `-fullpath` option is used, then each path element also may contains intermediate connection info if coupled with `-hierarchy` option. If `-names` option is not specified, each sub element returns as a list of `<context path>` and `<object collection>`.

If `-withcontext` is specified, it is legal to pass a bussed connection as input. If all connections are to the entire bus, it is treated as scalar from a tracing and output perspective unless `-bitlevel` option is also specified. If `-bitlevel` option is specified then results will be returned per bit of input connections.

For `-bitlevel` and `-fullpath` options, `-withcontext` option will be enabled by default.

With `-passthrough` option, tracing will continue on a connection if the specified `<attr>` is set on it. The value of the `<attr>` is expected to be a name of a pin within the same component context, from which connection tracing should continue. The `<attr>` name also supports subscript if required as `<attr[subscript]>`. With `-fullpath` option, the output will contain the pin name as an intermediate connection info.

Examples

If we have top design port `topa[7:0]` connected to pin `U1/a[7:0]`, use the following command to get the connections for each of the busbits of port `topa`:

```
coreBuilder> foreach_in_collection bit [get_all_bits topa] {
get_connections $bit
}
```

Use the following command to get the connections for each of the busbit of pin `U1/a`:

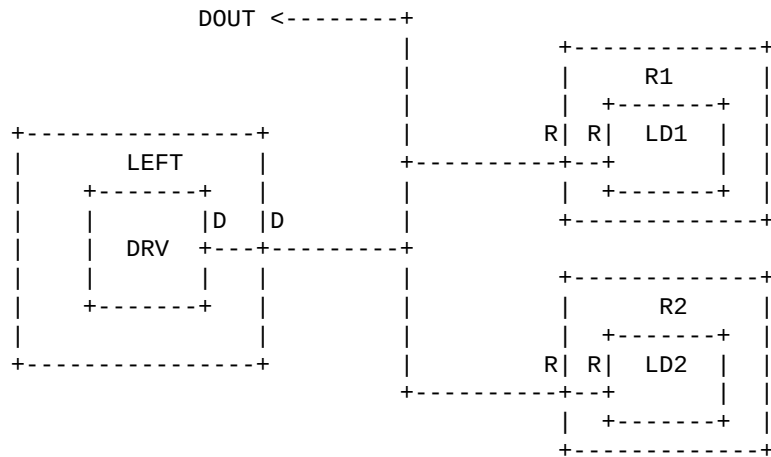
```
coreBuilder> foreach_in_collection bit [get_all_bits U1/a] {
get_connections $bit
}
```

If we have following design data where, here are some examples based on that.

- Instance `DRV` with output port `D` is instantiated inside instance `LEFT` with output port `D` which is connected to `DRV/D`
- Instance `LD1` with input port `R` is instantiated inside instance `R1` with input port `R` which is connected to `LD1/R`
- Instance `LD2` with input port `R` is instantiated inside instance `R2` with input port `R` which is connected to `LD2/R`
- At the top level `LEFT/D` is connected `R1/R` and `R2/R` and goes out via top-level port `DOUT`

coreTools Command Reference Index

Diagram::



The result will be as follow for the `get_connection` command.

```
coreAssembler> get_connections LEFT/D
Output: {DOUT R2/R R1/R LEFT/D}
```

The output returns the collection of objects.

```
coreAssembler> get_connections -names LEFT/D
Output: DOUT R2/R R1/R LEFT/D
```

The output returns the name of objects.

```
coreAssembler> get_connections -names -hierarchy LEFT/D
Output: DOUT DRV/D LD1/R LD2/R
```

The output returns the connection end point names crossing the hierarchy.

```
coreAssembler> get_connections -names -sort LEFT/D
Output: LEFT/D DOUT R1/R R2/R
```

Returns the connection name in sorted order.

```
coreAssembler> get_connections -names -withcontext LEFT/D
Output: {DOUT LEFT/D} {R2/R LEFT/D} {R1/R LEFT/D}
```

With `-withcontext`, each path will be a pair of <receiver> <driver>. Where each sub path contains the context information concatenated with connection name. With `-hierarchy` option, it contains the full hierarchical path. See the next example.

```
coreAssembler> get_connections -names -withcontext -hierarchy LEFT/D
Returns: {R1/LD1/R LEFT/DRV/D} {R2/LD2/R LEFT/DRV/D} {DOUT LEFT/DRV/D}
```

For `-fullpath` option, the output contains all intermediate hierachical connection names for each pair of path component.

Examples

coreTools Command Reference Index

```
coreAssembler> get_connections -hier -withcontext -names -fullpath LEFT/D
Returns: {R1/LD1/R R1/R LEFT/D LEFT/DRV/D} {R2/LD2/R R2/R LEFT/D
LEFT/DRV/D} {DOUT LEFT/D LEFT/DRV/D}
```

Without -names option, the output contains the object reference instead of name of connection. Each sub path comes as a list of context path and collection of objects. See the example below.

```
coreAssembler> get_connections -hier -withcontext -fullpath LEFT/D
Returns: {{R1 <C:LD1/R>} {/ <C:R1/R>} {/ <C:LEFT/D>} {LEFT <C:DRV/D>}}
{{R2 <C:LD2/R>} {/ <C:R2/R>} {/ <C:LEFT/D>} {LEFT <C:DRV/D>}} {{/
<C:DOUT>} {/ <C:LEFT/D>} {LEFT <C:DRV/D>}}
```

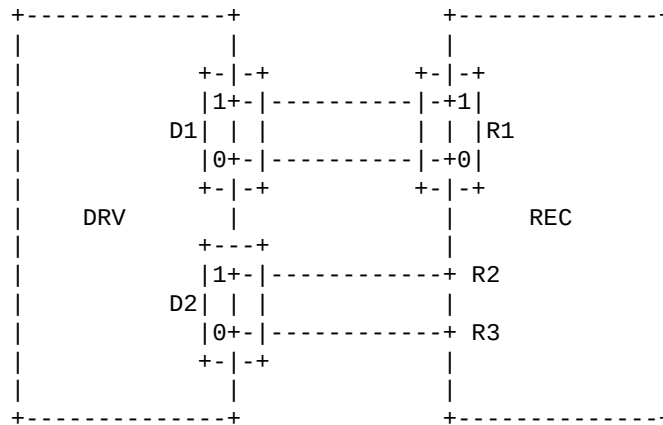
For the sub element {R1 <C:LD1/R>}

- R1 is the context information for LD1/R pin.
- <C:LD1/R> is a collection containing the named pin.

Now a simple example to show how the bussed connection would work.

- Instance DRV with output port D1[1:0] connected to instance REC with input port R1[1:0]
- Same instance DRV has output port D2[1:0] with D2[1] connected to REC/R2 and D2[0] connected to REC/R3

Diagram::



```
coreAssembler> get_connections -withcontext -names DRV/D1
Returns: {REC/R1 DRV/D1}
```

```
coreAssembler> get_connections -withcontext -bitlevel DRV/D1
Returns: {REC/R1[1] DRV/D1[1]} {REC/R1[0] DRV/D1[0]}
```

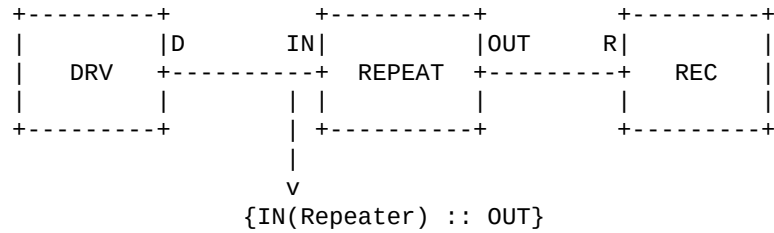
```
coreAssembler> get_connections -withcontext DRV/D2
Returns: {REC/R2 DRV/D2[1]} {REC/R3 DRV/D2[0]}
```

Example to show how passthrough option would work.

coreTools Command Reference Index

- Instance DRV with output port D connected to repeater pin REPEAT/IN
- The repeater output REPEAT/OUT connected to instance REC with input port R
- The pin REPEAT/IN has the attribute "Repeater" set to "OUT"

Diagram::



```
coreAssembler> get_connections -withcontext -names -fullpath DRV/D
Returns: {REPEAT/IN DRV/D}
```

```
coreAssembler> get_connections -withcontext -names -fullpath DRV/D
-passthrough Repeater
Returns: {REC/R REPEAT/OUT REPEAT/IN DRV/D}
```

```
coreAssembler> get_connections -withcontext -names DRV/D -passthrough
Repeater
Returns: {REC/R DRV/D}
```

See Also

[get_all_bits\(2\)](#)

`get_current_component`
Returns the current component.

Syntax

```
string get_current_component [-name <item>] [-unfolded] [-cell]
[-kbprefix]
    string <item>
```

Parameters

	Expand this item or name in the current context.
-name <item>	This option is used to return the component name associated with the given item (if a collection is passed in). Otherwise it returns the current component name with the passed name appended to it. (if a collection is passed in), or the given string
-unfolded	Return unfolded name when sharing is enabled.
-cell	Get the cell for the current component
-kbprefix	Get the specified kb for the current component.

Description

This command is used to return the name of the current component. The current component indicates which coreAssembler component is currently being processed. The current component is set using the `set_current_component` command. When a component becomes 'current', its associated workspace becomes the current workspace (accessible using `get_workspace_kb` or `get_workspace_name`), and the design of which the component is an instantiation becomes the current design.

When working in coreConsultant, the current component is always "". This is also true in coreAssembler when working at the sub-system level (i.e. at the level which instantiates each of the components).

Examples

To print the value of the current component:

```
coreAssembler> echo "Current component: [get_current_component]"
```

To print the value of the unfolded component name of a given shared cell instance (stored in variable 'cell'):


```
coreAssembler> echo "Component: [get_current_component -unfolded -name  
$cell]"
```

See Also

[set_current_component\(2\), get_workspace_kb\(2\), get_workspace_name\(2\),
get_component_name\(2\)](#)

NAME

get_current_hook_command
Get the currently running command string

SYNTAX

string **get_current_hook_command**

ARGUMENTS

DESCRIPTION

The **get_current_hook_command** command may only be run from within a command hook. The command returns the command string used to invoke the command.

Note that option values may have been abbreviated when the hooked command was executed. In that case the information returned will also include abbreviated option values.

Python Support

If the command hook is written in Python, then this command returns a tuple containing three elements. The first item is the command that was run. The second are the positional arguments used. The third argument is a dictionary of the keyword arguments specified to the command invocation.

SEE ALSO

add_command_hook(2)
remove_command_hook(2)
get_command_hooks(2)

NAME

get_defined_commands

Get information on defined commands and groups.

SYNTAX

```
string get_defined_commands [-details]
                        [-groups] [pattern]
```

```
string     pattern
```

ARGUMENTS

-details Get detailed information on specific command or group.

-groups Search groups rather than commands

pattern Return commands or groups matching *pattern*. The default value of this argument is "".

DESCRIPTION

The **get_defined_commands** gets information about defined commands and command groups. By default the command returns a list of commands that match the specified *pattern*.

When **-details** is specified, the return value is a list that is suitable as input to the **array set** command. The returned value is a list with an even number of arguments. Each odd-numbered element in the list is a key name, and each even-numbered element in the list is

coreTools Command Reference Index

the value of the previous key. The **-details** option is only legal if the pattern matches exactly one command or group.

When **-group** is specified with **-details**, the supported keys are as follows:

<i>name</i>	This key contains the name of the group.
<i>info</i>	This key contains the short help for the group.
<i>commands</i>	This key contains the commands in the group

When **-details** is used with a command, the supported keys are as follows:

<i>name</i>	This key contains the name of the command.
<i>info</i>	This key contains the short help for the command.
<i>groups</i>	This key contains the group names that this command belongs to.
<i>options</i>	This key contains the options defined for the command. The value is a list.
<i>return</i>	This key contains the return type for the command.

Each element in the *options* list also follows the key value pattern. The set of available keys for options are as follows:

<i>name</i>	This key contains the name of the option.
<i>info</i>	This key contains the short help for the option.
<i>value_info</i>	This key contains the short help string for the value

coreTools Command Reference Index

<i>type</i>	The type of the option.
<i>required</i>	The value will be 0 or 1 depending if the option is optional or required.
<i>is_list</i>	Will be 1 if the option requires a list.
<i>list_length</i>	If a list contains the list length constraint. One of any, even, odd, non_empty or a number of elements.
<i>allowed_values</i>	The allowed values if the option has specified allowed values.
<i>min_value</i>	The minimum allowed value if the option has specified one.
<i>max_value</i>	The maximum allowed value if the option has specified one.

EXAMPLES

```
prompt> get_defined_commands *collection
add_to_collection append_to_collection copy_collection filter_collection
foreach_in_collection index_collection sort_collection
prompt> get_defined_commands -details sort_collection
name sort_collection info {Create a sorted copy of the collection}
groups {} options {{name -descending info {Sort in descending order}
value_info {} type boolean required 1 is_list 0} {name -dictionary info
{Sort strings dictionary order.} value_info {} type boolean required 1
is_list 0} {name collection info {Collection to sort} value_info
collection type string required 0 is_list 0} {name criteria info {Sort
criteria - list of attributes} value_info criteria type list required 0
is_list 1 list_length non_empty}}
```

SEE ALSO

```
help(2)
man(2)
```

get_design_attribute

Get the value of an attribute on the specified design

Syntax

```
string get_design_attribute [-design <design>] attr
      string <design>
      string attr
```

Parameters

-design	The design for which you want to get the attribute value
<design>	(default = current_design).
	The name of the attribute for which you want to get the value.
attr	For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so that get_design_attribute does not interpret the subscript as a sub-command.

Description

The get_design_attribute returns the current value of the specified attribute on the specified design. If you do not include the -design option, get_design_attribute returns the current value of the specified attribute on the current_design. To set the value of a design attribute, use the set_design_attribute command.

Examples

To get the value of the PreserveHierarchy attribute on the current_design:

```
coreConsultant> get_design_attribute PreserveHierarchy
1
```

To get the value of CompileMapEffort[initial] on the design named Subblock_A:

```
coreConsultant> get_design_attribute -design Subblock_A
{CompileMapEffort[initial]}
high
```

To find the design named Subblock_B, then get the value of its Overconstrain attribute and store the value in the variable "B_overconst":

```
coreConsultant> set B [find_design Subblock_B]
{Subblock_B}
coreConsultant> set B_overconst [get_design_attribute -design $B
Overconstrain]
0.1
```

See Also

[set_design_attribute \(2\)](#)

`get_design_prefix`

Returns the prefix to be associated with each design name.

Syntax

```
string get_design_prefix [-component <component>]  
    string <component>
```

Parameters

-component <component> hierarchical path to the component.

Description

This command returns the name of the prefix to be associated with a design. It is typically used to ensure that renamed designs are referenced properly in verification environments and in post-elaboration intent files. Note that the value returned is actually <prefix>_ or "" (i.e. the empty string), depending on whether or not a prefix is defined. This makes using the prefix in substitution pragmas easier (see Examples).

In coreConsultant, a single prefix can be associated with each design. When the source code for the design is written to disk, this prefix is prepended (as <prefix>_<original name>) to the definition and each instantiation of every design in the core. The prefix is specified as part of the workspace creation process and can not be changed.

In coreAssembler, a prefix is automatically associated with each component instantiated in the subsystem. The prefix is set to the name of the component instantiation (i.e. the cell name). As in coreConsultant, as source files are written to disk, the designs and their instantiations are renamed as <prefix>_<original name>. Also, In coreAssembler, Design prefixing is enabled if the components support it. Design prefixing for each component can be get by passing the component path to the command if prefixing is enabled and set.

Examples

The following is a TCL code fragment used to ensure that post-elaboration intent is set on the proper design:

```
# Set post-elaboration intent on design 'top'.  
current_design "[get_design_prefix]top"  
set_design_attribute MapBlockIndividually true
```

coreTools Command Reference Index

The following is a Verilog code fragment from a testbench instantiating a core:

```
-- Instantiate core 'top'.  
-- reuse-pragma startSub [format "%s%s" [get_design_prefix] "top"]  
top  
U_top();  
The following command gets the Designprefix in assembler on the i_enabled  
component:
```

```
coreAssembler> get_design_prefix -component /i_enabled
```

See Also

set_design_prefix (2), set_macro_prefix (2), get_macro_prefix (2),
set_file_prefix (2), get_file_prefix (2)

`get_field_parameter_for_array`

returning the field parameter for a given index of an array parameter.

Syntax

```
string get_field_parameter_for_array array_param index
      string array_param
      int index
```

Parameters

<code>array_param</code>	The name of the array parameter
<code>index</code>	The index of the field parameter to get

Description

The coreTools support array parameters which are used to set the values of fields within a bitfield parameter. `get_field_parameter_for_array` returns the field parameter object for a given index defined for an array parameter. The field parameters can be defined using `define_array_field_parameters` command.

Examples

The following command defines three field parameters based on the array parameter AP, and get the value of each field parameter:

```
coreBuilder> create_parameter -type bitfield -package pac1 AP
coreBuilder> set_attribute AP -attr IsArray -value 1
coreBuilder> set_attribute AP -attr ArrayStart -value 1
coreBuilder> set_attribute AP -attr ArrayEnd -value 3
coreBuilder> set_attribute AP -attr ArrayFieldSize -value 16
coreBuilder> set_attribute AP -attr IsHexVal -value 1
coreBuilder> set_attribute AP -attr Value -value 0xaaaabbbbcccc
coreBuilder> define_array_field_parameters AP
coreBuilder> get_attribute [get_field_parameter_for_array AP 1] -attr
Value
0xaaaa
coreBuilder> get_attribute [get_field_parameter_for_array AP 2] -attr
Value
0xbbbb
coreBuilder> get_attribute [get_field_parameter_for_array AP 3] -attr
Value
0xcccc
```

See Also

define_array_field_parameters (2), foreach_array_field_parameter (2),
IsArray (3), ArrayStart (3), ArrayEnd (3), ArrayFieldSize (3) _____

get_file_prefix

Returns the file prefix to be associated with each design name.

Syntax

```
string get_file_prefix [-component <component>]  
    string <component>
```

Parameters

-component <component> hierachical path to the component.

Description

This command returns the name of the prefix to be associated with each RTL file in the scenario where file prefixing is enabled. File prefixing is used to enable renaming of RTL files such that multiple instances of a component can be guaranteed to not have overlapping file names.

In coreConsultant, a single prefix is used for all files. When the source files are written to disk, this prefix is prepended (as <prefix>_<original name>) to each file name. The prefix is specified in the SetDesignAndFilePrefix activity, provided that the coreKit being used was packaged with file prefixing enabled.

In coreAssembler, file prefixing is enabled if the components support it. Each component gets a unique file prefix. The file prefixing for each component can be get by passing the component path to the command if prefixing is set and enabled. The design prefix is based on the hierarchical path to the component instance.

Examples

The following command gets the fileprefix in assembler on the i_enabled component:

```
coreAssembler> get_file_prefix -component /i_enabled
```

See Also

[set_file_prefix \(2\)](#), [get_design_prefix \(2\)](#), [set_design_prefix \(2\)](#), [get_macro_prefix \(2\)](#), [set_macro_prefix \(2\)](#)

[API Index](#) [Command Reference](#)

get_filegroup_attribute

Get an attribute from a filegroup

Syntax

```
string get_filegroup_attribute fgroup attr  
    string fgroup  
    string attr
```

Parameters

<i>fgroup</i>	The filegroup to get the attribute value from The (subscripted) attribute to get.
<i>attr</i>	For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so that Tcl does not interpret the subscript as a sub-command.

Description

The get_filegroup_attribute command returns the current value of the specified attribute on the specified filegroup. To set the value of an attribute on a filegroup, use the set_filegroup attribute command.

Examples

To get the set of parameters used to configure the testbench filegroup:

```
coreConsultant> get_filegroup_attribute testbench Parameters
```

See Also

[set_filegroup_attribute \(2\)](#)

get_filegroup_parameter
Get a parameter for a filegroup

Syntax

```
string get_filegroup_parameter [-param] [-quiet] fgroup paramName
      string fgroup
      string paramName
```

Parameters

-param	Return parameter instead of its value
-quiet	Don't issue user error message if not found
fgroup	The filegroup to get the attribute value from
paramName	The name of the parameter you want to get.

Description

The `get_filegroup_parameter` command returns the current value of the specified parameter of the specified filegroup. When using any coreTool, you can use `get_filegroup_parameter` to get the value of a filegroup parameter when working in command line mode or when writing batch mode command files.

For coreBuilder users, a common use for `get_filegroup_parameter` is to retrieve the value of a filegroup parameter for a configurable filegroup. This would typically be done in a post-prompt command associated with the filegroup.

For all coreTools, if you specify the `-param` option, `get_filegroup_parameter` does not return the parameter's current value. Instead, `get_filegroup_parameter` creates a collection that contains the specified parameter and returns the collection handle.

Examples

To get the name of simulator to be used with filegroup `SimulationStuff`:

```
set sim [get_filegroup_parameter SimulationStuff SimulatorName]
```

See Also

[set_filegroup_parameter \(2\)](#)

[API Index](#) [Command Reference](#)`get_generator_parameter`

Get component generator parameter value

Syntax

string **get_generator_parameter** [-component *<component_name>*] *generator*
parameter

*string <component_name>**string generator**string parameter*

Parameters

`-component``<component_name>`

component name of the component generator

`generator`Name of the generator the parameter
belongs to`parameter`The name of the parameter that you want
to set.

Description

This command get the component generator parameter value.

Examples

to get the parameter 'param1' of generator 'Generator':

```
coreAssembler>get_generator_parameter Generator param1
```

See Also

[set_generator_parameter \(2\), invoke_generator \(2\)](#)

get_hdl_file_list

Return the list of HDL files that DC must analyze (in order of analysis)

Syntax

```
string get_hdl_file_list [-includes] [-all] [-items] [-library <library
name>] [-component <component name>] [-incdirs]
    string <library name>
    string <component name>
```

Parameters

	Return only files that are referenced by Verilog 'include.
-includes	This options does not return the files that are explicitly included in the list of files to be analyzed. For VHDL designs, this option does not return any files.
-all	List both the files are referenced by Verilog include and those that are in the list of files to be analyzed explicitly. The files referenced byinclude appear at the beginning of the list. This option has no effect with VHDL designs.
-items	Return a collection of HDL file items instead of a list of HDL file names. If you want to set or get attributes on the the HDL file items returned by get_hdl_file_list, you need to use the -items option. Without the -items option, get_hdl_file_list returns a list of HDL file names (including path).
-library <library name>	Return only the files that are specified to be analyzed into <library name>. If you do not specify the -library option, get_hdl_file_list returns all HDL files without indicating which library they are to be analyzed into. The -library option is only useful for VHDL designs that use multiple HDL libraries.
-component <component name>	Return only the files that are associated with <component name>. This option is valid in coreAssembler only. If this option is not specified within coreAssembler, then only the top-level file is returned. If the special value "*" is passed as the component name, results are returned for all components in the subsystem.
-incdirs	Return the path to be included w.r.t the src folder

Description

The `get_hdl_file_list` command returns the list of HDL source files that Design Compiler (DC) must analyze for a core, in the order in which DC must analyze the files. In coreBuilder, you must first complete the Load Designs activity before you can successfully execute `get_hdl_file_list`.

Verilog designs may include files that are referenced by `'include` and do not need to be explicitly included in the list of files to be analyzed.

By default, `get_hdl_file_list` only returns the list of files that are specified to be analyzed explicitly and not the files that are referenced by `include`. To list the files that are referenced by `include`, use either `-includes` (list only `include` files) or `-all` (list both `include` files and files that are to be analyzed explicitly).

For VHDL designs, all source files must be analyzed explicitly. Therefore, the `-includes` option does not return files and the `-all` option returns the same list that `get_hdl_file_list` returns without any options.

For VHDL designs that use multiple HDL libraries, you can use the `-library <library name>` option to list only the VHDL files that are to be analyzed into a specific library.

The `-incdirs` option returns only explicitly defined include directories. These would come from IP-XACT components where the source code fileSet includes `ipxact:dependency` elements. The full set of include directories associated with a component includes this set and the names of directories associated with files analyzed in coreBuilder without being explicitly specified (i.e. analyzed via `'include`). Paths specified via the `IncludePath` parameter in the Load Designs activity in coreBuilder will not appear in this list unless an include file was actually found in the given include path entry.

Examples

To return the HDL file list for the currently loaded VHDL coreKit:

```
coreBuilder> get_hdl_file_list
../MyCore_package.vhd ../src/Subblock_A.vhd ../src/Subblock_B.vhd
../src/TOP.vhd
```

To return the list of `'include` files for the currently loaded Verilog coreKit:

```
coreBuilder> get_hdl_file_list -includes
../src/MyCore_package.v
```

To return the list of all HDL files for the currently loaded Verilog coreKit, including the files referenced by `'include`:

```
coreBuilder> get_hdl_file_list -all
```

coreTools Command Reference Index

```
../src/MyCore_package.v ../src/Subblock_A.v ../src/Subblock_B.v  
../src/TOP.v
```

To return the list of all VHDL to be analyzed into MyLib:

```
coreBuilder> get_hdl_file_list -library MyLib  
../src/Subblock_A.vhd ../src/Subblock_B.vhd ../src/TOP.vhd
```

To return a collection of HDL file items, then report attributes
HdlLibrary and FilePerms on all items in the collection:

```
coreBuilder> report_attribute -attrs {HdlLibrary FilePerms}  
[get_hdl_file_list -items]  
Report : Attributes  
Item(s) : _sel594  
Version: 1999.05-CB1.0.6  
Date : Fri Feb 4 15:10:16 2000  
Item HdlLibrary FilePerms
```

```
-----  
Subblock_A.vhd MyLib 0644  
Subblock_B.vhd MyLib 0644  
TOP.vhd MyLib 0644
```

See Also

[get_hdl_library_names \(2\)](#), [report_attribute \(2\)](#), [HdlLibrary \(3\)](#),
[FilePerms \(3\)](#)

`get_hdl_language`

Get the HDL languages used

Syntax

string **get_hdl_language** [-component <component>]
 string <component>

Parameters

-component <component> Component to get the HDL language from.

Description

This command is used to query the language of an IP in coreConsultant or coreAssembler. In coreAssembler it is possible for this command to return multiple languages if the -component option is not used and their are components of differing languages instantiated. The same is true if a single component is of mixed language.

Examples

Get the language of component i_uart in a coreAssembler subsystem:

```
prompt> set lang [get_hdl_language -component i_uart]
```

See Also

[get_hdl_file_list\(2\)](#)

`get_hdl_library_names`

Return the list of HDL libraries used by the currently loaded core

Syntax

```
string get_hdl_library_names [-component <component name>]  
    string <component name>
```

Parameters

<code>-component</code>	Return only the libraries that are associated with <component name>.
<code><component name></code>	This option is only useful within <code>coreAssembler</code> .

Description

This command is used to determine the list of libraries into which source code is going to be analyzed. For Verilog based designs, this will return only 'WORK'. For VHDL based designs, the list of libraries returned is defined by the core developer, as they indicate which files are to be analyzed into which libraries as part of the design loading process.

Examples

See Also

[get_hdl_file_list\(2\)](#)

get_hdl_pragma

Get reuse-pragma statement external to the HDL

Syntax

```
string get_hdl_pragma -ignore -attr <attribute> -process_ifdef <mode>
[-value <value>] -macro <name> -design <name> -package <name> -line <num>
-start_line <num> [-end_line <num>] -ignore_file <name> [-param <name>]
[-function <name>] [-port <name>] [-cell <name>] [-signal <name>] [-type
<name>] [-file <name>] [-library <name>]
    string <attribute>
    string <mode>
    string <value>
    string <name>
    int <num>
```

Parameters

-ignore	Ignore the object or line
-attr <attribute>	Set attribute on the object
-process_ifdef <mode>	Change parameter macro ifdef mode (Values: all_branches, standard)
-value <value>	Set the value of the given attribute
-macro <name>	Select the macro
-design <name>	Select the design
-package <name>	Select the package
-line <num>	Select the specified line
-start_line <num>	Select a range of lines
-end_line <num>	End a range lines
-ignore_file <name>	Specify the included file to be ignored in post processing
-param <name>	Select a parameter
-function <name>	Select a function
-port <name>	Select a port
-cell <name>	Select a cell
-signal <name>	Select a signal
-type <name>	Select a type
-file <name>	Select the file
-library <name>	Select the library

Description

Examples

See Also

Returns the macro prefix to be associated with each macro name.

Syntax

```
string get_macro_prefix [-component <component>]  
                        string <component>
```

Parameters

-component <component> hierachical path to the component.

Description

Examples

See Also

get_hierarchies

Get information about the subsystem hierarchies.

Syntax

```
string get_hierarchies [-filter <string>] [-top] [-unique] [-reverse]  
string <string>
```

Parameters

-filter <string>	Filter expression for instances
-top	Return top design in the list.
-unique	Return only unique non-shared items.
-reverse	Reverse NestLevel based sorting order.

Description

Examples

See Also

[API Index](#) [Command Reference](#)

`get_include_dirs`

return all include dirs for master leaf components in current or provided context

Syntax

```
string get_include_dirs [-context <context>]  
    string <context>
```

Parameters

<code>-context</code>	Context to run the command in (default is the
<code><context></code>	current context).

Description

For consultant or assembler, this command returns the include dirs for all the leaf level master components in the current context or the context provided with the `-context` argument.

Examples

```
# Here compI inside H1 is a duplicate component  
coreAssembler> all_components -recursive  
/compI /H1/compI  
coreAssembler> get_include_dirs  
/path/to/workspace_dir/Subsystem/components/compI/src/include_dir  
/path/to/workspace_dir/Subsystem/components/compI/src
```

See Also

[get_hdl_file_list \(2\)](#) [get_logical_dir \(2\)](#)

`get_installation_dir`

Get full installation directory name for the current core

Syntax

string `get_installation_dir`

Description

The `get_installation_dir` command returns the full directory name of the coreKit installation directory for the currently loaded workspace.

In coreAssembler this command will return the coreKit installation directory of an instantiated component. If invoked within a hierarchical component, it returns the logical directory of the component (matching `get_logical_dir`).

Use the `get_installation_dir` command to get the full name of the directory where you originally installed the coreKit.

Examples

To get the coreKit installation directory for the currently loaded workspace and store the directory name in the Tcl variable `install_dir`:

```
coreConsultant> set install_dir [get_installation_dir]  
/remote/repository/cores/DWPCI
```

See Also

[get_logical_dir \(2\)](#)

`get_installed_component_names`

Return installed component names and versions (for array set).

Syntax

string `get_installed_component_names`

Description

This command returns the set of all components installed in the search path. In coreAssembler, the SearchPath parameter of the AddSubsystemComponents activity is used. For coreConsultant, the setting of the global variable component_search_path is used.

The data is returned in a format suitable for the array set command. When used in this manner, the keys of the array are the names of the components found. Each array entry is a two element list. The first element is the path to the install directory for that component. The second element is the version of the installed component.

Examples

See the coreTools home page in the tool installation area in the doc/dware directory for an example use within a web page.

See Also

get_instance_parameter_value

Command to get the value of instance parameter in terms of the subsystem parameter.

Syntax

```
string get_instance_parameter_value <param>  
      string <param>
```

Parameters

<param> Name of the instance parameter

Description

This command is used to get (query) the value of a parameter on a component instance within a coreAssembler subsystem. This would typically only be needed when used in conjunction with create_subsystem_parameter and set_instance_parameter. Otherwise the parameter values are all specified from within the containing module.

Examples

Get value of a parameter set on an instance. Value set via an expression.

```
cA> set_instance_parameter -instance U1 -parameter DataWidth @width  
cA> echo "Parameter value is [get_instance_parameter U1/DataWidth]"
```

See Also

[set_instance_parameter \(2\)](#), [create_subsystem_parameter \(2\)](#)

get_interface_attribute

Get the value of an attribute from the specified interface

Syntax

string **get_interface_attribute** [-instance] [-refitem *intfItem*] *interface*
attr

string *intfItem*
string *interface*
string *attr*

Parameters

	The interface name identifies an instance instead of the definition
-instance	This option is only needed in coreBuilder, as the specified name is assumed to be that of an interface instance in coreAssembler because interface definitions can not be modified in coreAssembler. In coreBuilder, the specified name is assumed to indicate an interface definition, unless this option is specified.
	The interface is an interface instance in the same component as this item
-refitem <i>intfItem</i>	This option allows easy access to interface attributes and values from parallel interface instances in the same component. In many cases the option is used itself in the context of another item which has the same attribute value as the interface instance object. In this case "-refitem \"%item\"" is the object which needs the value, and get_interface_attribute takes this value from the parallel instance interface.
interface	The interface to get the attribute value from
attr	The name of the attribute to get the value from

Description

This command is used to access the value of the named attribute on the specified interface instance or interface definition. This command is not required in standard usage of the tools, but is provided as a complement to set_interface_attribute.

Examples

To get the MaxConsumers attribute for exported interface instance
AHBexported:

```
prompt> set value [get_interface_attribute AHBexported MaxConsumers]
```

To get the MaxConsumers attribute for Inputs from parallel interface
instance Outputs:

```
set_interface_attribute -instance Inputs MaxConsumers  
{=get_interface_attribute -refitem %item Outputs  
MaxConsumers}
```

See Also

[set_interface_attribute \(2\)](#)

`get_interface_link`

Get evaluated InterfaceLink for the given interface port/param.

Syntax

string **get_interface_link** [-context <interface instance collection>]*linkItem* [value]

string <interface instance collection>

string *linkItem*string *value*

Parameters

-context <interface instance
collection>Interface (collection) to be used
for parameter evaluation*linkItem*Interface port or parameter
(collection) for which the
InterfaceLink is desired

Description

This command can be used to get the evaluated value of the InterfaceLink attribute from the indicated item. It is equivalent to using `get_attribute` and getting the InterfaceLink attribute, except in cases where the InterfaceLink value contains parameter references. This command will evaluate those references to return the specific name of the linked port or parameter.

Examples

```
coreAssembler> set intf [find_interface_instance -component i_ahb -name
AHB_Slave]
```

```
coreAssembler> set intfPort [get_attribute $intf -attr Children
-subscript hrdata]
```

```
coreAssembler> set linkVal [get_interface_link $intfPort]
```

See Also

[InterfaceLink \(2\)](#)

`get_interface_parameter`

Get a parameter for an interface

Syntax

```
string get_interface_parameter [-instance] [-refitem intfItem] [-param]
[-quiet] interface paramName
    string intfItem
    string interface
    string paramName
```

Parameters

	The interface name identifies an instance instead of the definition
-instance	This option is only needed in coreBuilder, as the specified name is assumed to be that of an interface instance in coreAssembler because interface definitions can not be modified in coreAssembler. In coreBuilder, the specified name is assumed to indicate an interface definition, unless this option is specified.
	The interface is an interface instance in the same component as this item
-refitem <i>intfItem</i>	In many cases the option is used itself in the context of another parameter which has the same value as the interface instance object. In this case "-refitem %item" is the object which needs the value, and <code>get_interface_parameter</code> takes this value from the parallel instance interface.
-param	Return parameter instead of its value
-quiet	Do not issue user error message if parameter not found
<i>interface</i>	The interface to get the parameter from
<i>paramName</i>	Name of the parameter you want to get

Description

The `get_interface_parameter` command returns the current value of the specified parameter of the specified interface instance (or definition). This command can be used to retrieve interface parameter values set by the `set_interface_parameter` command or values set in the GUI.

This command is not required in standard usage of the tools, but is provided as a complement to `set_interface_parameter`.

If you specify the `-param` option, `get_interface_parameter` does not return the parameter's current value. Instead, `get_interface_parameter` creates a collection that contains the specified parameter and returns the collection handle.

Examples

To get the value (in `coreBuilder`) of the 'width' parameter from the bus interface:

```
coreBuilder> get_interface_parameter BusInterface width
1
```

To get the value of parameter `Slots` for instance `Inputs` from parallel interface instance `Outputs` and its parameter `Slots`:

```
set_interface_parameter -instance Inputs Slots
{=get_interface_parameter -refitem %item Outputs Slots}
```

To set the value of `ActiveLevel` of attached interface `IRQ2` (component instance `i_dma`) equal to the one of the packaged `IRQ` interface:

```
set_interface_parameter i_dma/IRQ2 ActiveLevel {=get_interface_parameter
-refitem %item IRQ ActiveLevel}
```

See Also

[set_interface_parameter \(2\)](#)

`get_interface_parameter_attribute`

Get the value of an attribute from the specified interface parameter

Syntax

`string get_interface_parameter_attribute [-instance] [-refitem intfItem]``interface parameterName attr``string intfItem``string interface``string parameterName``string attr`

Parameters

	The interface name identifies an instance instead of the definition
-instance	This option is only needed in coreBuilder, as the specified name is assumed to be that of an interface instance in coreAssembler because interface definitions can not be modified in coreAssembler. In coreBuilder, the specified name is assumed to indicate an interface definition, unless this option is specified.
	The interface is an interface instance in the same component as this item
-refitem	In many cases the option is used itself in the context of another item which has the same attribute value as the interface instance object. In this case "-refitem %item" is the object which needs the value, and <code>get_interface_parameter_attribute</code> takes this value from the parallel instance interface.
intfItem	
interface	The interface to get the parameter from
parameterName	The interface parameter to get the attribute value from
attr	The name of the attribute to get the value from

Description

This command is used to access the value of the named attribute on the specified interface parameter. This command is not required in standard usage of the tools, but is provided as a complement to `set_interface_parameter_attribute`.

Examples

To get the value of DataWidth parameter (i.e. the Value attribute):

```
prompt> set value [get_interface_parameter_attribute AHBExported  
DataWidth Value]
```

To get the MaxValue attribute for parameter Inputs/Slots from parallel interface instance Outputs and its parameter Slots:

```
set_interface_parameter_attribute -instance Inputs Slots MaxValue  
{=get_interface_parameter_attribute  
-refitem %item Outputs Slots MaxValue}
```

See Also

[set_interface_parameter_attribute \(2\)](#)

get_interface_port_attribute

Get the value of an attribute from the specified interface port

Syntax

string **get_interface_port_attribute** [-instance] [-refitem *intfItem*]

interface portName attr

string *intfItem*

string *interface*

string *portName*

string *attr*

Parameters

	The interface name identifies an instance instead of the definition
-instance	This option is only needed in coreBuilder, as the specified name is assumed to be that of an interface instance in coreAssembler because interface definitions can not be modified in coreAssembler. In coreBuilder, the specified name is assumed to indicate an interface definition, unless this option is specified.
	The interface is an interface instance in the same component as this item
-refitem <i>intfItem</i>	In many cases the option is used itself in the context of another item which has the same attribute value as the interface instance object. In this case "-refitem %item" is the object which needs the value, and get_interface_port_attribute takes this value from the parallel instance interface.
<i>interface</i>	The interface to get the port from
<i>portName</i>	The interface port to get the attribute value from
<i>attr</i>	The name of the attribute to get the value from

Description

This command is used to access the value of the named attribute on the specified interface port. This command is not required in standard usage of the tools, but is provided as a complement to set_interface_port_attribute.

Examples

To get the InterfaceLink attribute for the pwdata interface port:

```
prompt> set value [get_interface_port_attribute AHBexported pwdata  
InterfaceLink]
```

The attached interface Intr2 from component instance i_dma must connect to the same design port as the packaged interface Intr and its interface port irq:

```
set_interface_port_attribute i_dma/Intr2 irq InterfaceLink  
{=get_interface_port_attribute  
-refitem %item Intr irq InterfaceLink}
```

See Also

[set_interface_port_attribute \(2\)](#)

`get_launch_activities_status`

Get status for all the activities or few activities launched with
`launch_activities_on_grid`

Syntax

```
string get_launch_activities_status [-activities ] [-launch_job_names ]
[-info]
    string
```

Parameters

<code>-activities</code>	Reads in specific activity or list of activities and returns the job ids,launch job names,status of jobs,and activity names
<code>-launch_job_names</code>	Reads into specific launch session and gives details of underlting launched jobs
<code>-info</code>	Print the Information of launch

Description

The command returns the following information in a TCL Dict structure. When the command is run without any switch, it will return the status for all the jobs launched.

If the command is specified with `-activities` option, it returns the status of specified activities launched under any launch name.

If the command is specified with `-launch_job_names` option, it returns the status of all the activities of the given launch names.

If both the switches i.e. `-activities` and `-launch_job_names` are specified, information is fetched for given activities within those launch names.

Consider an example of job status returned in the form of TCL dict by the command,

```
launch1 {Synthesize {JobId 151643 Status done WorkDirectory launch1_dir}
RemoveGaurd {JobId 151659 Status pending}}
```

This dict's info can be easily extracted using a foreach loop. Following is an example printing the dict in key value pair fashion using foreach loop.

```
launchName : launch1
Activity   : Synthesize
JobId      : 151643
Status     : done
WorkDirectory : launch1_dir
```

Note : The current dict structure does not have a WorkDirectory associated with RemoveGuard

Examples

Consider following 3 differenet launches.

```
launch1 : Synthesize, GenerateTestVectors, FormalVerification,
DW_apb_rap_Simulate
launch2 : Synthesize, GenerateTestVectors, StaticTimingAnalysis,
runSpyglass
launch3 : Synthesize, StaticTimingAnalysis, GenerateGtechModel
```

Use `get_launch_activities_status` to get information of launches and launched activities.

Basic example, without any switch.

```
coreConsultant> get_launch_activities_status
launch3 {GenerateGtechModel {JobId 153252 Status done WorkDirectory gtech}
RemoveGaurd {JobId 153259 Status done} StaticTimingAnalysis {JobId 153250 Status
done WorkDirectory launch3_dir HoldJob Synthesize HoldId 153247} Synthesize
{JobId 153247 Status done WorkDirectory launch3_dir}} launch1
{GenerateTestVectors {JobId 151649 Status done WorkDirectory launch1_dir HoldJob
Synthesize HoldId 151643} RemoveGaurd {JobId 151659 Status done}
DW_apb_rap_Simulate {JobId 151653 Status done WorkDirectory sim} Synthesize
{JobId 151643 Status done WorkDirectory launch1_dir} FormalVerification {JobId
151647 Status done WorkDirectory launch1_dir HoldJob Synthesize HoldId 151643}}
launch2 {RemoveGaurd {JobId 155614 Status done} GenerateTestVectors {JobId 155607 Status d
```

Filter launch using switch `-launch_job_names`

```
coreConsultant> get_launch_activities_status -launch_job_names launch2
launch2 {RemoveGaurd {JobId 155614 Status pending} GenerateTestVectors {JobId 155607 Statu

coreConsultant> get_launch_activities_status -launch_job_names "launch2
launch3"
launch3 {GenerateGtechModel {JobId 153252 Status done WorkDirectory gtech}
RemoveGaurd {JobId 153259 Status done} StaticTimingAnalysis {JobId 153250 Status
done WorkDirectory launch3_dir HoldJob Synthesize HoldId 153247} Synthesize
{JobId 153247 Status done WorkDirectory launch3_dir}} launch2 {RemoveGaurd {JobId 155614 S
```

Filter activity using switch `-activities`

```
coreConsultant> get_launch_activities_status -activities "Synthesize
FormalVerification DW_apb_rap_Simulate"
launch3 {Synthesize {JobId 153247 Status done WorkDirectory launch3_dir}} launch1 {DW_apb_
```

Filter launch and activity by using `-launch_job_names` and `-activities` switches together

```
coreConsultant> get_launch_activities_status -activities "Synthesize
FormalVerification DW_apb_rap_Simulate" -launch_job_names "launch1
launch3"
```

launch3 {Synthesize {JobId 153247 Status done WorkDirectory launch3_dir}} launch1 {DW_apb_

See Also

launch_activities_on_grid (1) wait_activities_on_grid (2)

`get_logical_dir`

Get full logical directory name for the specified workspace directory

Syntax

```
string get_logical_dir [dir]  
    string dir
```

Parameters

dir The name of the workspace logical directory.

Description

The `get_logical_dir` command returns the full path name of the specified logical directory in a coreTool workspace. If you do not specify the optional `<dir>` argument, `get_logical_dir` returns the full path name of the current workspace directory.

If you mapped the logical directory to a different physical directory when you created the workspace, `get_logical_dir` returns the path name of the logical directory, not the physical directory.

Examples

To return the full path name of the logical directory `kb` of the current workspace and store the path name in the Tcl variable `kb_dir`:

```
coreConsultant> set kb_dir [get_logical_dir kb]  
/users/joe/cores/BusCore/config1/kb
```

To return the full path name of the current workspace root directory:

```
coreConsultant> get_logical_dir  
/users/joe/cores/BusCore/config1
```

See Also

[get_synthesis_dir \(2\)](#), [get_installation_dir \(2\)](#)

[API Index](#) [Command Reference](#)

`get_memory_map_attribute`

Get the value on an attribute from a memory map.

Syntax

```
string get_memory_map_attribute map attr
      string map
      string attr
```

Parameters

`map` Name of the memory map.
`attr` Name of the attribute to get.

Description

This command gets an attribute on a memory map.

Examples

The following command gets the MemoryAccess type of the memory map 'map1':

```
coreBuilder> get_memory_map_attribute map1 MemoryAccess
```

See Also

[memMap \(3\)](#), [set_memory_map_attribute \(2\)](#), _____

[API Index](#) [Command Reference](#)

`get_memory_mode`

Get the current memory mode associated with the current/selected design

Syntax

string **get_memory_mode**

Description

This command is used to get the current memory mode of the current/selected design. By default to 'DEFAULT' mode.

This current memory mode will be used to evaluate the register/registerField attributes value.

Examples

The following command get the current memory mode.

```
coreBuilder> get_memory_mode
```

See Also

[get_register_field_attribute \(2\)](#), [set_register_attribute \(3\)](#),
[set_memory_mode \(4\)](#), [create_memory_mode \(5\)](#)

NAME

get_message_ids Get application message ids

SYNTAX

```
string get_message_ids [-type severity] [pattern]
```

```
string      severity  
string      pattern
```

ARGUMENTS

-type *severity* Filter ids based on type (Values: Info, Warning, Error, Severe, Fatal)

pattern Get IDs matching pattern (default: *)

DESCRIPTION

The **get_message_ids** command retrieves the error, warning and informational messages used by the application. The result of this command is a Tcl formatted list of all message ids. Information about the id can be queried with the **get_message_info** command.

EXAMPLES

The following code finds all error messages and makes the application stop script execution when one of these messages is encountered.

coreTools Command Reference Index

```
foreach id [get_message_ids -type Error] {  
  set_message_info -stop_on -id $id  
}
```

SEE ALSO

```
print_message_info(2)  
set_message_info(2)  
suppress_message(2)
```

NAME

get_message_info

Returns information about diagnostic messages.

SYNTAX

```
integer get_message_info
    [-error_count | -warning_count | -info_count
     | -limit l_id | -occurrences o_id | -suppressed s_id | -id i_id]
```

Data Types

<i>l_id</i>	string
<i>o_id</i>	string
<i>s_id</i>	string
<i>i_id</i>	string

ARGUMENTS

-error_count	Returns the number of error messages issued so far.
-warning_count	Returns the number of warning messages issued so far.
-info_count	Returns the number of informational messages issued so far.
-limit <i>l_id</i>	Returns the current user-specified limit for a given message ID. The limit was set with the set_message_info command.
-occurrences <i>o_id</i>	Returns the number of occurrences of a given message ID.
-suppressed <i>s_id</i>	Returns the number of times a message was suppressed either using suppress_message or due to exceeding a

coreTools Command Reference Index

user-specified limit.

-id *i_id* Returns information about the specified message. The information is returned as a Tcl list compatible with the array set command.

DESCRIPTION

The **get_message_info** command retrieves information about error, warning, and informational messages. For example, if the following message is generated, information about it is recorded:

```
Error: unknown command 'wrong_command' (CMD-005)
```

It is useful to be able to retrieve recorded information about generated diagnostic messages. For example, you can stop a script after a certain number of errors have occurred, or monitor the number of messages issued by a single command.

You can also find out how many times a specific message has occurred, or how many times it has been suppressed. Also, you can find out if a limit has been set for a particular message ID.

EXAMPLES

The following example uses the **get_message_info** command to count the number of errors that occurred during execution of a specific command, and to return from the procedure if the error count exceeds a given amount:

```
prompt> proc do_command {limit} {  
    set current_errors [get_message_info -error_count]  
    command  
    set new_errors [get_message_info -error_count]  
    if {[expr $new_errors - $current_errors] > $limit} {  
        return -code error "Too many errors"  
    }  
    ...  
}
```

The following example uses the **get_message_info** command to retrieve information on the CMD-014 message:

```
prompt> get_message_info -id CMD-014  
id CMD-014 severity Error limit 0 occurrences 0 suppressed 0 message  
{Invalid %s value '%s' in list.}
```

SEE ALSO

`print_message_info(2)`
`set_message_info(2)`
`suppress_message(2)`
`get_message_ids(2)`

`get_object_name`

Converts a collection into a list of item names

Syntax

```
string get_object_name object  
      string object
```

Parameters

`object` the object (collection)

Description

This command takes a collection as argument and returns the names of all of the objects in the collection.

Examples

Print the names of the ports that begin with the letter X:

```
prompt> get_object_name [find_item -type port X*]
```

See Also

`get_parameter_attribute`

Get the value of an attribute on a parameter

Syntax

```
string get_parameter_attribute param attr
      string param
      string attr
```

Parameters

<code>param</code>	The name of the parameter for which you want to get the attribute value.
	The name of the attribute for which you want to get the value.
<code>attr</code>	For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so that <code>get_parameter_attribute</code> does not interpret the subscript as a sub-command.

Description

The `get_parameter_attribute` command returns the current value of the specified attribute on the specified parameter.

You can use `get_parameter_attribute` to get the value of any attribute on a design, activity, strategy, or formula parameter. However, for coreConsultant users, the most common use for `get_parameter_attribute` is to get the value of the Value attribute on a design parameter. (That is, to get the current value of the design parameter).

You can use `get_parameter_attribute` to get the current value of an activity parameter or strategy parameter (by specifying the Value attribute). However, it is easier and more efficient to use `get_activity_parameter` or `get_strategy_parameter` to get current parameter values for activities and strategies.

Examples

To get the current value of the design parameter named `sync_mode`:

```
coreConsultant> get_parameter_attribute sync_mode Value
0
```

To get the current value of the EnumValues attribute on the `sync_mode` parameter:

`get_parameter_attribute sync_mode EnumValues`
0 1

To get the value of the ValueRequired attribute on the StartGUIOnWorkspace parameter on the coreBuilder CreateIntegrationWorkspaces activity:

```
coreBuilder> set startGUI [get_activity_parameter -param  
CreateIntegrationWorkspaces StartGUIOnWorkspace]  
{StartGUIOnWorkspace}  
coreBuilder> get_parameter_attribute $startGUI ValueRequired  
1
```

To get the value of the EnumValues attribute on the QorEffort parameter of the synthesis strategy named DC_opto_strategy:

```
coreBuilder> set qoreffort [get_strategy_parameter -param  
DC_opto_strategy QorEffort]  
{QorEffort}  
coreBuilder> get_parameter_attribute $qoreffort EnumValues  
low medium high
```

See Also

[get_activity_parameter \(2\)](#), [get_strategy_parameter \(2\)](#)

get_port_attribute

Get the value of an attribute on a port

Syntax

```
string get_port_attribute port attr
      string port
      string attr
```

Parameters

- port** The port for which you want to get the attribute value.
 The name of the attribute for which you want to get the value.
 For subscripted attributes, enclose the attribute/subscript in
- attr** curly braces ({Attribute[subscript]}) so that
 get_port_attribute does not interpret the subscript as a
 sub-command.

Description

The get_port_attribute returns the current value of the specified attribute on the specified port. To set the value of a port attribute, use the set_port_attribute command.

Examples

To get the value of the CriticalTiming attribute on port input_1 the current_design:

```
coreConsultant> get_port_attribute input_1 CriticalTiming
1
```

To get the value of MaxOutputDelay[clk] on port output_12 of the current_design:

```
coreConsultant> get_port_attribute output_12 {MaxOutputDelay[clk]}
1.5
```

See Also

[set_port_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

get_power_domain_voltage

Returns the voltage for the specified power domain.

Syntax

string **get_power_domain_voltage** [-quiet] *domain*
string *domain*

Parameters

-quiet Don't return an error if supply not found
domain Domain Name

Description

Returns the supply voltage for the specified power domain.

Examples

```
coreConsultant> get_power_domain_voltage PD_VMAIN
```

The above example returns the voltage for the power domain PD_VMAIN.

See Also

get_prime_profile_info

Get Prime Profile info from the data model..

Syntax

```
string get_prime_profile_info [-tag_info <Prime Profile tag>] [-current]
[-valid] [-default] [-all] [-component <hierarchical component name>]
    string <Prime Profile tag>
    string <hierarchical component name>
```

Parameters

-tag_info <Prime Profile tag>	Return information about the tag provided.
-current	Returns the name of the Prime Profile applied.
-valid	Returns the list of valid Prime Profiles.
-default	Returns the list of default Prime Profiles.
-all	Return names of all Prime Profiles available in the design.
-component <hierarchical component name>	Specify context (default: current context).

Description

This api can be used to retrieve prime profile information about the current design on the component specified.

Examples

```
coreConsultant> get_prime_profile_info -tag CustomEA
parameter dummy value 1 label {custom early access} description {custom
profile with early access enabled} expression @include_rom==1
isEarlyAccess 1 lockParams {include_rom ram_size rom_word}
analysisLockParams include_rom isExternal 1
coreConsultant> get_prime_profile_info -all
CustomEA DisableRom1 DisableRom2 DummyCheckbox DummyCheckbox2 DummyExpr1
DummyRadio LockRamSize Test003
coreAssembler> set attrVal [get_prime_profile_info -tag_info LockRamSize
-component /h0/m0]
value 2 parameter dummy label {Lock ram_size and ram_word.} isEarlyAccess
1 expression {@include_rom==1 && @ram_size==32} description {Lock
```

coreTools Command Reference Index

ram_size and ram_word to 32. In analysis allow word to change.}
lockParams {include_rom ram_size ram_word} analysisLockParams ram_size
isExternal 0
the returned string for -tag_info option is a dict and can be opened up
as follows:
coreAssembler> dict get \$attrVal parameter
dummy
coreAssembler> dict get \$attrVal value
2
coreAssembler> dict get \$attrVal expression
@include_rom==1 && @ram_size==32
coreAssembler> dict get \$attrVal label
Lock ram_size and ram_word.
coreAssembler> dict get \$attrVal description
Lock ram_size and ram_word to 32. In analysis allow word to change.
coreAssembler> dict get \$attrVal isEarlyAccess
1
coreAssembler> dict get \$attrVal isExternal
0
coreAssembler> dict get \$attrVal lockParams
include_rom ram_size ram_word
coreAssembler> dict get \$attrVal analysisLockParams
ram_size
coreAssembler> get_prime_profile_info -all -component /h0/m0
DisableRom1 DisableRom2 DummyCheckbox DummyCheckbox2 DummyExpr1
DummyRadio LockRamSize Test003
coreAssembler> get_prime_profile_info -current -component /h0/m0
Test003

See Also

[report_prime_profiles\(2\)](#)

`get_register_array_attribute`

Get the value of an attribute from a register array.

Syntax

```
string get_register_array_attribute regArray attr  
    string regArray  
    string attr
```

Parameters

<code>regArray</code>	Name of the register array to get attribute from.
<code>attr</code>	Name of the attribute to get.

Description

This command gets an attribute on a registerArray.

Examples

The following command gets the AddressOffset attribute of the register array 'regArray1'.

```
coreBuilder> get_register_array_attribute map/block/regArray1  
AddressOffset
```

See Also

[registerArray \(3\), set_register_array_attribute \(2\)](#)

`get_register_attribute`

Get the value of an attribute from a register.

Syntax

```
string get_register_attribute reg attr  
    string reg  
    string attr
```

Parameters

<code>reg</code>	Name of the register to get the attribute from.
<code>attr</code>	Name of the attribute to get.

Description

This command gets an attribute on a register.

Examples

The following command gets the AddressOffset attribute of the register 'reg1'.

```
coreBuilder> get_register_attribute map/block/reg1 AddressOffset
```

See Also

[register \(3\)](#), [set_register_attribute \(2\)](#)

`get_register_field_attribute`

Get the value of an attribute from a register field.

Syntax

```
string get_register_field_attribute reg attr  
    string reg  
    string attr
```

Parameters

<code>reg</code>	Name of the register field to get the attribute from.
<code>attr</code>	Name of the attribute to get.

Description

This command gets an attribute on a register field.

Examples

The following command gets the BitAddressOffset attribute of the register field 'field1' of register 'reg1':

```
coreBuilder> get_register_field_attribute map/block/reg1/field1  
BitAddressOffset
```

See Also

[registerField \(3\)](#), [set_register_field_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

`get_register_field_value_attribute`

Get the value of an attribute from a register field value.

Syntax

```
string get_register_field_value_attribute val attr
      string val
      string attr
```

Parameters

val Name of the register field to get the attribute from.
attr Name of the attribute to get.

Description

Returns the value of an attribute on a register field value.

Examples

To get the bit value of a field value:

```
coreBuilder> get_register_field_value_attribute \  
map/block/reg/field/val Value
```

See Also

[set register field value attribute \(2\)](#)

get_slave_base_address

Get the remap range and/or base address of the given slave interface.

Syntax

```
string get_slave_base_address [-component <component>] -interface
<interface> [-slotoffset <slot>] [-mode <mode>] [-range] [-address]
    string <component>
    string <interface>
    int <slot>
    string <mode>
```

Parameters

-component <component>	The slave component or name.
-interface <interface>	The exported or slave instance or name.
-slotoffset <slot>	The slotoffset the remap object is associated with for multi-slot interfaces.
-mode <mode>	The mode to use when setting the base address.
-range	Get the remap range attribute.
-address	Get the remap address attribute.

Description

Gets the base memory address or range for the slave interface specified with the -component and -interface options. For an exported interface, the interface name is specified by -interface without the -component option.

Examples

Get the address on a slave interface:

```
cA> get_slave_base_address -component i_ahb -interface AHB_Slave -mode
Normal -address
```

Get the range for a multi-slot interface:

```
cA> get_slave_base_address -component i_icm -interface Layer1 -slotoffset
1 -range
```

Get the range of an exported interface:

```
cA> get_slave_base_address -interface ex_i_xml_AHB_Slave1 -range
```

See Also

set slave base address (2)

get_solution_data

Get all solution data or an individual field.

Syntax

```
string get_solution_data [field]  
      string field
```

Parameters

field	Solution value to be returned
-------	----------------------------------

Description

Information about an instantiated solution is stored in the top-level design of the instantiated solution. This information can be queried using the `get_solution_data` command. When called without arguments, all of the stored information is returned in TCL 'array set' format. If instead, a field name is passed in, then only the value of the given field is returned.

Fields exist for each of the arguments to the `define_solution` command used to define the solution. Argument and field names are aligned. For example, the field 'env' contains the value of the `-env` argument to `define_solution`. In addition to fields for each of the arguments to `define_solution`, there are fields for:

- `dir` - directory containing the definition of the instantiated solution
- `family` - the solution "family" name
- `userSysCfgResult` - system configuration information specified by the user either in the call to `instantiate_solution` or in the system configuration dialog if instantiated in GUI mode

Examples

Get all solution data and store in an array:

```
array set solution_data [get_solution_data]
```

Get the value of the directory from which the solution was instantiated:

```
set root [get_solution_data dir]
```

See Also

[define_solution\(2\)](#), [instantiate_solution\(2\)](#), [report_solutions\(2\)](#)

[API Index](#) [Command Reference](#)

get_solution_file

Get IP solution file if present.

Syntax

string **get_solution_file** *file*
 string *file*

Parameters

file Solution file to be returned

Description

Examples

See Also

`get_solution_root`

Get root context of current DesignWare IP Solution.

Syntax

string **`get_solution_root`** [-quiet]

Parameters

-quiet Don't complain if not actually inside a DW IP Solution.

Description

If called within an instantiated solution, this command returns the root hierarchical path of the solution. An error is returned if this command is called outside of an instantiated solution.

Consider a subsystem where a solution is instantiated using the `instantiate_solution` command in the hierarchical component `/mySoln`, and within the solution a component named `U1` is instantiated. If this command is called within `/mySoln` or `/mySoln/U1`, it will return `/mySoln`. If invoked in `/`, an error is returned because that is outside of any instantiated solution.

Examples

Get root of solution instantiated at the `/` context:

```
create_workspace
instantiate_solution <arguments>
set root [get_solution_root]
```

See Also

[`instantiate\_solution` \(2\)](#)

[API Index](#) [Command Reference](#)

get_solution_variable

Get all solution variables or an individual variable.

Syntax

string **get_solution_variable** [-quiet] [*varname*]
 string *varname*

Parameters

-quiet	Don't complain if variable is not defined.
varname	Solution variable to be returned

Description

This command is used to query the value of a variable defined as part of a DW IP Solution. Variables are defined via the solution definition command (define_solution). If the variable does not exist a TCL error will occur unless -quiet is specified.

Examples

Get the value of the numlanes variable:

```
prompt> set lanes [get_solution_variable numlanes]
```

See Also

[define_solution \(2\)](#)

get_strategy_parameter

Get the value of a parameter for a strategy

Syntax

```
string get_strategy_parameter [-param] strategyName paramName
      string strategyName
      string paramName
```

Parameters

-param	Return the parameter instead of its value.
strategyName	The name of the strategy for which you want to get the parameter.
paramName	The name of the parameter on the specified strategy.

Description

The `get_strategy_parameter` command returns the current value of the specified parameter of the specified coreTool synthesis strategy.

If you specify the `-param` option, `get_strategy_parameter` does not return the parameter's current value. Instead, `get_strategy_parameter` creates a collection that contains the specified parameter and returns the collection handle. You can store the collection handle in a Tcl variable and reference the variable in future commands to access the strategy parameter.

The synthesis strategies currently available and their associated parameters are:

- `DC_opto_strategy` - Parameters: `QorEffort`, `UseDesignBudgets`, `NetlistFormat`
- `DC_incremental_opto_strategy` - Parameters: `QorEffort`, `UseDesignBudgets`, `ResumeAfter`, `NetlistFormat`
- `DC_design_budgeting_strategy` - Parameters: `QorEffort`, `ResumeAfter`, `NetlistFormat`
- `DC_quick_map_strategy` - Parameters: `StopAtGtech`, `NetlistFormat`
- `PT_model_extraction` - Parameters: `AuxiliaryScript`, `ContextBorrow`, `ModelFormat`, `ModelStage`, `OutputFileRoot`, `RemoveInternalArcs`, `ModelOperatingConditions`, `ModelAsLibCell`, `LatchLevels`

For detailed descriptions of the above strategies and parameters, refer to the *coreConsultant User Guide*.

To set the value of a synthesis strategy parameter, use the `set_strategy_parameter` command.

Examples

To get the current value of the `QorEffort` parameter on the synthesis strategy named `DC_opto_strategy`:

```
coreConsultant> get_strategy_parameter DC_opto_strategy QorEffort
medium
```

To create a Tcl collection that contains the `ModelOperatingConditions` parameter on the `PT_model_extraction` strategy and store the collection handle in the Tcl variable `OpCon`:

```
coreConsultant> set OpCon [get_strategy_parameter -param
PT_model_extraction ModelOperatingConditions]
{ModelOperatingConditions}
```

See Also

[set_strategy_parameter\(2\)](#)

get_supply_voltage

Command used to insert a voltage value into a set_voltage command in a power intent file.

Syntax

```
string get_supply_voltage -type <power|ground> -mode <best|worst>
-default <voltage>
    string <power|ground>
    string <best|worst>
    double <voltage>
```

Parameters

-type <power ground>	Voltage type (Values: power, ground)
-mode <best worst>	Voltage mode. (Values: best, worst)
-default <voltage>	Default voltage.

Description

Examples

See Also

[API Index](#) [Command Reference](#)

get_sv_expr

Convert a tcl expression to equivalent sv expression

Syntax

```
string get_sv_expr [-purge <item names>] <expression>  
    string <item names>  
    string <expression>
```

Parameters

-purge <item names>	Items to be purged from sv expression.
<expression>	Expression to be converted to sv equivalent.

Description

Examples

See Also

get_synthesis_dir

Return the full directory name for the specified synthesis subdirectory

Syntax

```
string get_synthesis_dir internalSubdirectoryName [internalStageName]  
    string internalSubdirectoryName  
    string internalStageName
```

Parameters

<i>internalSubdirectoryName</i>	Name of desired synthesis sub-directory.
<i>internalStageName</i>	Name of synthesis stage in which the sub-directory resides.

Description

The `get_synthesis_dir` command returns the full path name to the synthesis subdirectory specified by `<internalSubdirectoryName>`.

In `coreConsultant` and `coreAssembler`, the `syn` subdirectory of a workspace contains subdirectories for each synthesis stage (setup, constrain, initial, and so on). Within each stage subdirectory, there are leaf-level subdirectories for different types of data (db, log, report, and so on).

Use `get_synthesis_dir` to determine the full path to the specified stage/leaf directory for the currently loaded workspace.

Examples

To return the path to the log directory for the initial synthesis stage:

```
coreConsultant> set dirname [get_synthesis_dir initial/log]  
/usr/fred/MyCore/Config/syn/initial/log
```

See Also

[get_logical_dir\(2\)](#), [get_installation_dir\(2\)](#)

`get_tcl_expr`
convert a SV expression to tcl.

Syntax

string **get_tcl_expr** [-expand] [-prepend] [-xpath] <sv_expression>
string <sv_expression>

Parameters

-expand	expand operators in expression to our tcl procs.
-prepend	prepend '@' symbol before each identifier.
-xpath	convert SV expression into xpath expression.
<sv_expression>	SV expression to be converted to tcl.

Description

Examples

See Also

[API Index](#) [Command Reference](#)

get_template_naming_style

Get the template_naming_style value.

Syntax

string `get_template_naming_style`

Description

Examples

See Also

[API Index](#) [Command Reference](#)

get_template_parameter_style

Get the template_parameter_style value.

Syntax

string **get_template_parameter_style**

Description

Examples

See Also

[API Index](#) [Command Reference](#)

get_template_separator_style

Get the template_separator_style value.

Syntax

string **get_template_separator_style**

Description

Examples

See Also

[API Index](#) [Command Reference](#)

get_tool_preference

Get the tool preference parameter

Syntax

```
string get_tool_preference -pref pref  
      string pref
```

Parameters

-pref *pref* Tool preference parameter

Description

Gets the value of tool preference

Examples

```
get_tool_preference -pref retain_static_port_range
```

See Also

[set_tool_preference](#)

`get_tool_root`

Get the root directory for the specified tool.

Syntax

```
string get_tool_root [-64bit] tool
      string tool
```

Parameters

<code>-64bit</code>	Is 64bit mode requested? Indicates which tool root to get (Values: <code>dc_shell</code> , <code>pt_shell</code> , <code>fm_shell</code> , <code>tcm</code> , <code>fc_shell</code> , <code>rtl_shell</code> , <code>tmax</code> , <code>dft_shell</code> , <code>synplify</code> , <code>protocompiler</code> , <code>vivado</code> , <code>dc_shell-t</code> , <code>de_shell</code> , <code>vcs</code> , <code>vcsi</code> , <code>vera</code> , <code>vsim</code> , <code>ncsim</code> , <code>leda</code> , <code>mvtools</code> , <code>spyglass</code> , <code>vcstatic</code> , <code>verdi</code>)
<code>tool</code>	

Description

This command is used to get the root directory of the installation area of the specified tool. This information is used within the tools to properly invoke them, and is made available for external access via the `get_tool_root` command.

When using the tools in graphical mode, the tool roots can be viewed and modified via the View->Specify Tool Roots menu option.

If the `-64bit` option is used, then the command returns 1 if 64 bit operation was requested for the tool, and 0 for 32 bit operation.

Examples

To determine the location of the `dc_shell` executable:

```
prompt> set exec [get_tool_root dc_shell]/$::sh_arch/syn/bin/dc_shell
```

See Also

[set_tool_root\(2\)](#)

get_top_design

Returns the top design.

Syntax

string **get_top_design** [-quiet]

Parameters

-quiet Do not print error messages.

Description

This command can be used to return a collection containing the top-level design. This is useful for writing customizations which require access to the top-level design of a core, but which don't know the name of that design. By default, if there is not top-level design, this command will return a TCL error. If the -quiet option is specified, the command will return "" without a TCL error in the event that there is no top-level design.

In coreAssembler, this command can return different designs depending on the current component. If the current component is "" (i.e. you are in the top-level of subsystem), then the command will return the true top-level design. This is the design created within coreAssembler which represents the subsystem. However, if you are currently inside a component, then this command will return the top-level design associated with that component. This is so that core-specific plugins will work the same in coreAssembler as they do in coreConsultant.

Examples

To set the current design to the top-design:

```
prompt> current_design [get_top_design]
```

See Also

[get_top_design_name \(2\)](#), [get_current_component \(2\)](#), [set_current_component \(2\)](#)

`get_top_design_name`

Return the name of the top level design

Syntax

string **`get_top_design_name`** [-as_filename]

Parameters

	Return top design name as used as synthesis script filename.
-as_filename	While elaborated design names can become very long because of parameters, the filename length is limited by the operating system. Long names (over about 100 characters) use the original design name with an unique number suffix. This option is useful in plug-ins when you need access to <top>.db files etc.

Description

The command returns the name of the top-level design of the current core.

Examples

To change back to the top-level design:

```
prompt> current_design [get_top_design_name]
```

To print the final synthesis db filename:

```
prompt> echo [file join [get_synthesis_dir db final]  
"[get_top_design_name -as_filename].db"]
```

See Also

NAME

set_trace_option

Get the current option value controlling behavior of command tracing and output annotation..

SYNTAX

```
get_trace_option [-command name
                  |-profile
                  |-memory_threshold
                  |-cpu_threshold
                  |-time_treshhold]
                  [-annotate | -is_traced]
```

string *name*

ARGUMENTS

- command *name* This option is mutually exclusive with -profile, -memory_threshold, -cpu_threshold, and -time_threshold. The option identifies the command for which tracing or annotation option is returned.
- annotate If this option is given, then the *annotate* setting for the given command is returned. This option is mutually exclusive with -is_traced and is meaningful only in combination with the -command option.
- is_traced If this option is given, then returns 1 if the given command is traced and logged, 0 otherwise. This option is mutually exclusive with -annotate and is meaningful only in combination with the -command option.
- profile This option is mutually exclusive with all others. The option returns a list of the currently enabled profile

coreTools Command Reference Index

metrics, or "all" if all metrics are enabled.

-memory_threshold

This option is mutually exclusive with all others. The option returns the currently set memory profile threshold, if any, in kilobytes. If no threshold is set, then it returns the string "unset".

-cpu_threshold This option is mutually exclusive with all others. The option returns the currently set cpu profile threshold, if any, in seconds. If no threshold is set, then it returns the string "unset".

-time_threshold

This option is mutually exclusive with all others. The option returns the currently set wall clock time profile threshold, if any, in kilobytes. If no threshold is set, then it returns the string "unset".

DESCRIPTION

The command is used to query the current setting for the command annotation option that is set with the `set_trace_option` command, or profile metric settings.

EXAMPLES

The following example sets the annoation type of the `get_attribute` command.

```
prompt> get_trace_option -command get_attribute -annotate  
annotate
```

The following example sets, then gets, the currently enabled profile metrics.

```
prompt> set_trace_topion -profile {cpu time}  
prompt> get_trace_topion -profile  
cpu time
```

The following example gets the threshold for memory profile metric before one has been set.

```
prompt> get_trace_option -memory_threshold  
unset
```

coreTools Command Reference Index

The following example will list the commands in the Builtins command group that are not traced and logged.

```
prompt> foreach cmd [dict get [get_defined_commands -details -groups Builtins]  
?  if {[get_trace_option -command $cmd -is_traced]}  
?    {echo $cmd}  
? }
```

SEE ALSO

```
log_trace(2)  
annotate_trace(2)  
set_trace_option(2)
```

NAME

get_unix_variable

This is a synonym for the **getenv** command.

SEE ALSO

getenv(2), **printenv(2)**, **printvar(2)**, **set(2)**,
setenv(2), **sh(2)**, **unset(2)**.

`get_upf_attribute`

Get the value of an attribute on a UPF item

Syntax

```
string get_upf_attribute item attr
      string item
      string attr
```

Parameters

- `item` The name of the UPF item for which you want to get the attribute value.
- `attr` The name of the attribute for which you want to get the value.

Description

The `get_upf_attribute` returns the current value of the specified attribute on the specified UPF item. To set the the value of UPF item(s), use the `set_upf_attribute` command.

The following information can be used to determine how to find a specific UPF object in the coreTools data model.

All UPF objects are of type `genHierItem`. The object names are based on the UPF construct/command for which they are created. To find a specific object you need to search with a filter because the names are just <fixed prefix><index>. For example, power domain objects are named `UPF_power_domain<index>`. You can find all power domains by finding all `genHierItem` objects that match `UPF_power_domain_*`. Then on these objects you can get the `UPFDomain` attribute to get the actual power domain name. Following is a list of objects and attributes.

UPF Command Item Names Attributes Value

```
-----
map_isolation_cell UPF_isolation_X UPFStrategy isolation_strategy
UPFDomain domain
UPFHardCodedCells {lib_cells}
map_level_shifter_cell UPF_level_shift_X UPFStrategy
level_shifter_strategy
UPFDomain domain
UPFHardCodedCells {lib_cells}
map_retention_strategy UPF_retention_X UPFStrategy retention_strategy
UPFDomain domain
UPFHardCodedCells {lib_cells}
```

coreTools Command Reference Index

add_port_state UPF_port_state_X UPFSupply supply_port_name
UPFState state_name
UPFStateValue state_value
add_power_state UPF_power_state_X UPFSupply supply_set_name
UPFState state_name
UPFSupplyExpr supply_expr
get_supply_voltage UPF_supply_<type>_<mode>
create_power_domain UPF_power_domain_X UPFDomain domain_name
UPFIsTopDomain (has -elements)
set_voltage UPF_supply_<name>_<mode>

Examples

To get the value of UPFDomain attribute on item UPF_retention_1

```
coreConsultant> get_upf_attribute UPF_retention_1 UPFDomain
```

See Also

set_upf_attribute(2).

[API Index](#) [Command Reference](#)`get_upf_port_names`

Returns the names of the ports in in the specified power domain.

Syntax

```
string get_upf_port_names -domain [-direction ]
      string
```

Parameters

<code>-domain</code>	Domain Name
<code>-direction</code>	Port Direction

Description

This command is used to return the names of ports in a given power domain. The power domain of a port is defined by the `PowerDomain` attribute. The optional `-direction` option can be used to further filter the list of ports returned. This command is typically used within configurable UPF files to build list of ports to which other UPF commands are applied.

Examples

See Also

[PowerDomain \(3\)](#)

`get_value_from_interface`

Get value for interface parameter from the specified interface on the same component.

Syntax

```
string get_value_from_interface interface [paramName]  
    string interface  
    string paramName
```

Parameters

<code>interface</code>	Name of the interface to get the value from
<code>paramName</code>	Name of the parameter to get, if different

Description

This command is used as a formula on interface parameters. Sometimes a component provides multiple interfaces which all have parameter settings that are global to the component. For example an AHB bus has the settings for BigEndian, DataWidth, and AddressWidth. These settings need to be the same for both the master and slave provider interface instance on this component. Use this formula to ensure that.

If the `paramName` option is not specified, then the parameter name of the parameter the formula is set on is used.

Examples

To set the DataWidth parameter on the ABH_Slave interface to be the same as that on the AHB_Master interface:

```
set_interface_parameter_attribute -instance AHB_Slave \  
DataWidth Value \  
{=get_value_from_interface AHB_Master}
```

See Also

`get_workspace_kb`

Returns the current top-level knowledge database.

Syntax

string **get_workspace_kb** [-quiet] [-top]

Parameters

-quiet Don't error if not found

-top Get the top kb

Description

The `get_workspace_kb` command returns a collection that contains the top-level knowledge database (KB) for the currently loaded workspace.

Examples

To create a collection that contains the top-level workspace KB and store the collection handle in the variable `wkb`:

```
coreConsultant> set wkb [get_workspace_kb]  
{Config1}
```

See Also

[get_workspace_name \(2\)](#)

`get_workspace_license_info`

Get details about license and license type associated with workspace creation.

Syntax

string `get_workspace_license_info`

Description

This command is used to return licensing information associated with the currently open workspace. It returns two pieces of information in as a two element list. The first element in the list is the license feature used to create the workspace. The second element in the list is a set of tags indicating the type of license used to create the workspace. Valid tags are: restricted automotive. The restricted tag indicates a workspace created for restricted configuration(s), and the automotive tag indicates a workspace created for automotive product.

Examples

Get workspace license info for the current automotive workspace:

```
prompt> set data [get_workspace_license_info]  
DesignWare automotive
```

See Also

`get_workspace_name`

Returns the name of the current top-level knowledge database.

Syntax

string **`get_workspace_name`** [-quiet]

Parameters

-quiet Don't error if not found

Description

The `get_workspace_name` command returns the name of the knowledge database associated with the current workspace. This name will match the workspace name in all cases, except in `coreAssembler` in the case where the current component is not the top-level component. In that scenario, the name returned is the component instance path with any internal `'/'` characters replaced with a `' '` character. See the examples below for more information.

Examples

Get the workspace name in `coreConsultant` within a workspace named `Config1`:

```
coreConsultant> set wsname [get_workspace_name]
Config1
```

The remaining examples apply to a `coreAssembler` workspace named `Subsystem1` containing the following instances: `i_leaf1`, `i_hier1`, and `i_hier1/i_leaf2`.

```
coreAssembler> set_current_component /
coreAssembler> set wsname [get_workspace_name]
Subsystem1
coreAssembler> set_current_component /i_leaf1
coreAssembler> set wsname [get_workspace_name]
i_leaf1
coreAssembler> set_current_component /i_hier1/i_leaf2
coreAssembler> set wsname [get_workspace_name]
i_hier1%i_leaf2
```

See Also

[get_workspace_kb \(2\)](#), [get_current_component \(2\)](#)

NAME

getenv Returns the value of a system environment variable.

SYNTAX

```
string getenv  
      variable_name
```

Data Types
 variable_name string

ARGUMENTS

variable_name Specifies the name of the environment variable to be retrieved.

DESCRIPTION

The **getenv** command searches the system environment for the specified *variable_name* and sets the result of the command to the value of the environment variable. If the variable is not defined in the environment, the command returns a Tcl error. The command is catchable.

Environment variables are stored in the **env** Tcl array variable. The **getenv**, **setenv**, and **printenv** environment commands are convenience functions to interact with this array.

The application you are running inherited the initial values for environment variables from its parent process (that is, the shell from which you invoked the application). If you set the variable to a new value using the **setenv** command, you see the new value within the application and within any new child processes you initiate from the application using the **exec** command. However, these new values are not exported to the parent process. Further, if you set an environment variable using the appropriate system command in a shell you invoke using the **exec** command, that value is

not reflected in the current application.

See the **set**, **unset**, and **printvar** commands for information about working with non-environment variables.

EXAMPLES

In the following example, **getenv** returns you to your home directory:

```
prompt> set home [getenv "HOME"]
/users/disk1/bill

prompt> cd $home

prompt> pwd
/users/disk1/bill
```

In the following example, **setenv** changes the value of an environment variable:

```
prompt> getenv PRINTER
laser1

prompt> setenv PRINTER "laser3"
laser3

prompt> getenv PRINTER
laser3
```

In the following example, the requested environment variable is not defined. The error message shows that the Tcl variable **env** was indexed with the value **UNDEFINED**, which resulted in an error. In the second command, **catch** is used to suppress the message.

```
prompt> getenv "UNDEFINED"
Error: can't read "env(UNDEFINED)": no such element in array
Use error_info for more info. (CMD-013)

prompt> if {[catch {getenv "UNDEFINED"} msg]} {
    setenv UNDEFINED 1
}
```

SEE ALSO

catch(2)
exec(2)
printenv(2)
printvar(2)
set(2)
setenv(2)
unsetenv(2)
unset(2)

NAME

`gets` - Read a line from a channel

SYNOPSIS

`gets channelId ?varName?`

DESCRIPTION

This command reads the next line from *channelId*, returns everything in the line up to (but not including) the end-of-line character(s), and discards the end-of-line character(s).

ChannelId must be an identifier for an open channel such as the Tcl standard input channel (**stdin**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension. The channel must have been opened for input.

If *varName* is omitted the line is returned as the result of the command. If *varName* is specified then the line is placed in the variable by that name and the return value is a count of the number of characters returned.

If end of file occurs while scanning for an end of line, the command returns whatever input is available up to the end of file. If *channelId* is in non-blocking mode and there is not a full line of input available, the command returns an empty string and does not consume any input. If *varName* is specified and an empty string is returned in *varName* because of end-of-file or because of insufficient data in non-blocking mode, then the return count is -1. Note that if

coreTools Command Reference Index

varName is not specified then the end-of-file and no-full-line-available cases can produce the same results as if there were an input line consisting only of the end-of-line character(s). The **eof** and **fblocked** commands can be used to distinguish these three cases.

EXAMPLE

This example reads a file one line at a time and prints it out with the current line number attached to the start of each line.

```
set chan [open "some.file.txt"]
set lineNumber 0
while {[gets $chan line] >= 0} {
    puts "[incr lineNumber]: $line"
}
close $chan
```

SEE ALSO

`file(n)`, `eof(n)`, `fblocked(n)`, `Tcl_StandardChannels(3)`

KEYWORDS

blocking, channel, end of file, end of line, line, non-blocking, read

NAME

`glob` - Return names of files that match patterns

SYNOPSIS

`glob` *?switches? ?pattern ...?*

DESCRIPTION

This command performs file name in a fashion similar to the `csh` shell or `bash` shell. It returns a list of the files whose names match any of the *pattern* arguments. No particular order is guaranteed in the list, so if a sorted list is required the caller should use `lsort`.

OPTIONS

If the initial arguments to **glob** start with `-` then they are treated as switches. The following switches are currently supported:

-directory *directory*

Search for files which match the given patterns starting in the given *directory*. This allows searching of directories whose name contains glob-sensitive characters without the need to quote such characters explicitly. This option may not be used in conjunction with **-path**, which is used to allow searching for complete file paths whose names may contain glob-sensitive characters.

-join

The remaining pattern arguments, after option processing, are treated as a single pattern obtained by joining the arguments with directory separators.

-nocomplain

Allows an empty list to be returned without error;

coreTools Command Reference Index

without this switch an error is returned if the result list would be empty.

-path *pathPrefix*

Search for files with the given *pathPrefix* where the rest of the name matches the given patterns. This allows searching for files with names similar to a given file (as opposed to a directory) even when the names contain glob-sensitive characters. This option may not be used in conjunction with **-directory**. For example, to find all files with the same root name as *\$path*, but differing extensions, you should use which will work even if *\$path* contains numerous glob-sensitive characters.

-tails

Only return the part of each file found which follows the last directory named in any **-directory** or **-path** path specification. Thus is equivalent to For **-path** specifications, the returned names will include the last path segment, so will return paths like **foo.aux** **foo.bib** **foo.tex** etc.

-types *typeList*

Only list files or directories which match *typeList*, where the items in the list have two forms. The first form is like the **-type** option of the Unix find command: **b** (block special file), **c** (character special file), **d** (directory), **f** (plain file), **l** (symbolic link), **p** (named pipe), or **s** (socket), where multiple types may be specified in the list. **Glob** will return all files which match at least one of the types given. Note that symbolic links will be returned both if **-types l** is given, or if the target of a link matches the requested type. So, a link to a directory will be returned if **-types d** was specified.

The second form specifies types where all the types given must match. These are **r**, **w**, **x** as file permissions, and **readonly**, **hidden** as special permission cases. On the Macintosh, MacOS types and creators are also supported, where any item which is four characters long is assumed to be a MacOS type (e.g. **TEXT**). Items which are of the form *{macintosh type XXXX}* or *{macintosh creator XXXX}* will match types or creators respectively. Unrecognized types, or specifications of multiple MacOS types/creators will signal an error.

The two forms may be mixed, so **-types {d f r w}** will find all regular files OR directories that have both read AND write permissions. The following are equivalent:

```
glob -type d *
glob */
```

except that the first case doesn't return the trailing and is more platform independent.

--

Marks the end of switches. The argument following this one will be treated as a *pattern* even if it starts with a **-**.

coreTools Command Reference Index

GLOBBING PATTERNS

The *pattern* arguments may contain any of the following special characters, which are a superset of those supported by **string match**:

? Matches any single character.

***** Matches any sequence of zero or more characters.

[chars] Matches any single character in *chars*. If *chars* contains a sequence of the form **a-b** then any character between **a** and **b** (inclusive) will match.

\x Matches the character **x**.

{a,b,...} Matches any of the sub-patterns **a**, **b**, etc.

On Unix, as with **csh**, **a** at the beginning of a file's name or just after **a** must be matched explicitly or with a **{}** construct, unless the **-types hidden** flag is given (since at the beginning of a file's name indicates that it is hidden). On other platforms, files beginning with **a** are handled no differently to any others, except the special directories and which must be matched explicitly (this is to avoid a recursive pattern like from recursing up the directory hierarchy as well as down). In addition, all characters must be matched explicitly.

If the first character in a *pattern* is then it refers to the home directory for the user whose name follows the **If** the is followed immediately by then the value of the **HOME** environment variable is used.

The **glob** command differs from **csh** globbing in two ways. First, it does not sort its result list (use the **lsort** command if you want the list sorted). Second, **glob** only returns the names of files that actually exist; in **csh** no check for existence is made unless a pattern contains a **?**, *****, or **[]** construct.

When the **glob** command returns relative paths whose filenames start with a tilde (for example through **glob *** or **glob -tails**, the returned list will not quote the tilde with **T**his means care must be taken if those names are later to be used with **file join**, to avoid them being interpreted as absolute paths pointing to a given user's home directory.

WINDOWS PORTABILITY ISSUES

For Windows UNC names, the servername and sharename components of the path may not contain **?**, *****, or **[]** constructs. On Windows NT, if *pattern* is of the form it refers to the home directory of the user whose account information resides on the specified NT domain server.

coreTools Command Reference Index

Otherwise, user account information is obtained from the local computer.

Since the backslash character has a special meaning to the glob command, glob patterns containing Windows style path separators need special care. The pattern is interpreted as where will match the single character and will match the single character and will not be interpreted as a wildcard character. One solution to this problem is to use the Unix style forward slash as a path separator. Windows style paths can be converted to Unix style paths with the command or

EXAMPLES

Find all the Tcl files in the current directory:

```
glob *.tcl
```

Find all the Tcl files in the user's home directory, irrespective of what the current directory is:

```
glob -directory ~ *.tcl
```

Find all subdirectories of the current directory:

```
glob -type d *
```

Find all files whose name contains an a or the sequence

```
glob -type f *{a,b,cde}*
```

SEE ALSO

file(n)

KEYWORDS

exist, file, glob, pattern

NAME

`global` - Access global variables

SYNOPSIS

`global ?varname ...?`

DESCRIPTION

This command has no effect unless executed in the context of a proc body. If the **global** command is executed in the context of a proc body, it creates local variables linked to the corresponding global variables (though these linked variables, like those created by **upvar**, are not included in the list returned by **info locals**).

If *varname* contains namespace qualifiers, the local variable's name is the unqualified name of the global variable, as determined by the **namespace tail** command.

varname is always treated as the name of a variable, not an array element. An error is returned if the name looks like an array element, such as **a(b)**.

EXAMPLES

This procedure sets the namespace variable `::a::x`

```
proc reset {} {  
    global a::x  
    set x 0  
}
```

coreTools Command Reference Index

This procedure accumulates the strings passed to it in a global buffer, separated by newlines. It is useful for situations when you want to build a message piece-by-piece (as if with **puts**) but send that full message in a single piece (e.g. over a connection opened with **socket** or as part of a counted HTTP response).

```
proc accum {string} {  
    global accumulator  
    append accumulator $string \n  
}
```

SEE ALSO

namespace(n), upvar(n), variable(n)

KEYWORDS

global, namespace, procedure, variable

group_components

Group the following components into a lower hierarchical level

Syntax

```
string group_components -name <name> [-design <name>] components  
    string <name>  
    list components
```

Parameters

-name <name>	The new hierarchical cell name.
-design <name>	The new hierarchical cell design name.
components	The list of components to include in the group.

Description

Use this command to create a new level of hierarchy moving the specified components into the new hierarchical level. The command tries to preserve the existing connections by exporting the appropriate interfaces from the new hierarchical level and reconnecting the new hierarchical component at the current level.

Examples

```
group_components [list myComp1 myComp2 myComp2]
```

See Also

[ungroup_component \(2\).](#)

guiBehavior

Array variable that specifies different aspects of the GUI behavior.

Syntax

```
string guiBehavior = "\<array?\>"
```

Description

guiBehavior is a global array variable that holds values that determine different aspects of the coreTool GUI behavior. The guiBehavior array subscripts and their meanings are:

- `max_error_dlgs` - Specifies how many error messages to display in dialog boxes before sending error messages to the console window only. Until the `max_error_dlgs` is reached, the coreTools send each error message to a dialog box and to the console. The default value is 3. To change the value of `guiBehavior(max_error_dlgs)`, edit the `guiBehavior(max_error_dlgs)` value in your `.synopsys_rt.setup` file. In GUI mode, use the console `File>Preferences` command to edit your `.synopsys_rt.setup` file.
- `dont_prompt_on_exit` - Specifies whether the coreTool will prompt you to save modified knowledge databases prior to exiting. The default value is false (prompt before exiting). Synopsys does not advise changing the value of `guiBehavior(dont_prompt_on_exit)`. However, if you must change the behavior to "don't prompt on exit", add the following line to your `.synopsys_rt.setup` file: `set guiBehavior(dont_prompt_on_exit) true`

Examples

See Also

`gui_get_setting`

Get a setting on the specified window

Syntax

```
string gui_get_setting -window <window> [-setting <setting>] [-list]  
    string <window>  
    string <setting>
```

Parameters

<code>-window <window></code>	The target window to apply this setting.
<code>-setting <setting></code>	The name of the setting to get.
<code>-list</code>	When specified the list of settings is returned.

Description

Examples

See Also

gui_set_setting

Set a setting on the specified window

Syntax

```
string gui_set_setting -window <window> -setting <setting> -value <value>  
    string <window>  
    string <setting>  
    string <value>
```

Parameters

-window <window>	The target window to apply this setting.
-setting <setting>	The name of the setting to set.
-value <value>	The value of the setting.

Description

Examples

See Also

[API Index](#) [Command Reference](#)

`gui_source`

Source script in the GUI without GUI updates.

Syntax

string **gui_source** [-echo] [-continue_on_error] [-verbose] *file_name*
 string *file_name*

Parameters

-echo	Echo all commands
-continue_on_error	Don't stop script on errors
-verbose	Display intermediate results
<i>file_name</i>	Script file to be read

Description

Examples

See Also

gui_start
Start GUI

Syntax

```
string gui_start [-no_windows] [-offscreen sbool] [-file script] [  
xargs ]  
        boolean sbool  
        string script  
        list xargs
```

Parameters

-no_windows	set flag not to load windows for this cmd
-offscreen <i>sbool</i>	display gui to offscreen buffer
-file <i>script</i>	file to source on gui_start
– <i>xargs</i>	arguments to pass to the X connection

Description

Starts the GUI in a batch mode operation. Accepts standard X options.

Examples

See Also

[gui_stop\(2\)](#)

gui_stop
Stop GUI

Syntax

string **gui_stop**

Description

If the GUI is running, this puts the tool back into batch mode operation and shuts down all windows.

Examples

See Also

[gui_start\(2\)](#)

hdlFunc

Internal model of a VHDL function

Description

The hdlFunc item type represents the coreTool model of a VHDL function. An hdlFunc item type is required for any VHDL function that is required for design configuration. For example, if a core uses a VHDL function to calculate the value of a design parameter, you need to create an hdlFunc item so that coreConsultant can model the function and calculate the required parameter value when the core integrator executes the Specify Configuration activity.

To create an hdlFunc item, use a reuse-pragma annotation in the design's VHDL source code to assign the coreBuilder FunctionDefinition attribute on the function. When coreBuilder parses the code, it sets the FunctionDefinition attribute on the function and creates the hdlFunc item.

See Also

[FunctionDefinition \(3\)](#)

Supported Attributes

[ComponentOfItem \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [FunctionDefinition \(3\)](#), [Label \(3\)](#), [Name \(3\)](#), [Sequence \(3\)](#), [TypeName \(3\)](#), _____

[API Index](#) [Command Reference](#)

hdlin_enable_upf_compatible_naming

Tracks the value of the like-named dc_shell variable

Definition

Type: string

Flags:

Default value: true

Valid on:

Description

Examples

See Also

NAME

help Displays quick help for one or more commands.

SYNTAX

```
string help
      [-verbose]
      [-groups]
      [pattern]
```

Data Types

pattern string

ARGUMENTS

-verbose Displays the command options; for example *command_name -help*.

-groups Displays a list of command groups only.

pattern Displays commands matching the specified pattern.

DESCRIPTION

The **help** command is used to get quick help for one or more commands or procedures. This is not the same as the **man** command that displays reference manual pages for a command. There are many levels of help.

By typing the **help** command, a brief informational message is printed followed by the available command groups.

List all of the commands in a group by typing the group name as the argument to **help**. Each command is followed by a one-line description of the command.

coreTools Command Reference Index

To get a one-line help description for a single command, type **help** followed by the command name. You can specify a wildcard pattern for the name; for example, all commands containing the string "alias". Use the **-verbose** option to get syntax help for one or more commands. Use the **-groups** option to show only the command groups in the application. This option cannot be combined with any other option.

EXAMPLES

The following example lists the commands by command group:

```
prompt> help
Specify a command name or wild card pattern to get help on individual
commands. Use the '-verbose' option to get detailed option help for a
command. Commands also provide help when the '-help' option is passed to
the command.'
```

You can also specify a command group to get the commands available in that group. Available groups are:

```
Procedures:           Miscellaneous procedures
Help:                 Help commands
Builtins:             Generic Tcl commands
...
```

The following example displays the list of procedures in the Procedures group:

```
prompt> help procedures
ls                # List files
sh                # Execute a shell command
```

The following example uses a wildcard character to display a one-line description of all commands beginning with **a**:

```
prompt> help a*
alias            # Create a command which expands to words.
append          # Builtin
array           # Builtin
```

This example displays option information for the **source** command:

```
prompt> help -verbose source
source          # Read a file and execute it as a script
[-echo]         (Echo all commands)
[-verbose]      (Display intermediate results)
file_name       (Script file to read)
```

SEE ALSO

`man(2)`
`sh_help_shows_group_overview(3)`

NAME

history Displays or modifies the commands recorded in the history list.

SYNTAX

```
string history
      [-h]
      [-r]
      [argument_list]
```

Data Types

argument_list list

ARGUMENTS

-h	Displays the history list without the leading numbers. You can use this for creating scripts from existing history. You can then source the script with the source command. You can combine this option with only a single numeric argument. Note that this option is a nonstandard extension to Tcl.
-r	Reverses the order of output so that most recent history entries display first rather than the oldest entries first. You can combine this option with only a single numeric argument. Note that this option is a nonstandard extension to Tcl.
<i>argument_list</i>	Additional arguments to history (see DESCRIPTION).

DESCRIPTION

The **history** command performs one of several operations related to recently-executed commands recorded in a history list. Each of these recorded commands is referred to as an "event." The most commonly used forms of the command are described below. You can combine each with either the **-h** or **-r** option, but not both.

- o With no arguments, the **history** command returns a formatted string (intended for you to read) giving the event number and contents for each of the events in the history list.
- o If a single, integer argument *count* is specified, only the most recent *count* events are returned. Note that this option is a nonstandard extension to Tcl.
- o Initially, 20 events are retained in the history list. You can change the length of the history list using the following:

```
history keep count
```

Tcl supports many additional forms of the **history** command. See the "Advanced Tcl History" section below.

EXAMPLES

The following examples show the basic forms of the **history** command. The first is an example of how to limit the number of events shown using a single numeric argument:

```
prompt> history 3
7  set base_name "my_file"
8  set fname [format "%s.db" $base_name]
9  history 3
```

Using the **-r** option creates the history listing in reverse order:

```
prompt> history -r 3
9  history -r 3
8  set fname [format "%s.db" $base_name]
7  set base_name "my_file"
```

Using the **-h** option removes the leading numbers from each history line:

```
prompt> history -h 3
set base_name "my_file"
set fname [format "%s.db" $base_name]
history -h 3
```

coreTools Command Reference Index

Advanced Tcl History

The **history** command performs one of several operations related to recently-executed commands recorded in a history list. Each of these recorded commands is referred to as an "event." When specifying an event to the **history** command, the following forms may be used:

- o A number, which if positive, refers to the event with that number (all events are numbered starting at 1). If the number is negative, it selects an event relative to the current event; for example, **-1** refers to the previous event, **-2** to the one before that, and so on. Event **0** refers to the current event.
- o A string selects the most recent event that matches the string. An event is considered to match the string either if the string is the same as the first characters of the event, or if the string matches the event in the sense of the **string match** command.

The **history** command can take any of the following forms:

history Same as **history info**, described below.

history add *command [exec]*

Adds the *command* argument to the history list as a new event. If **exec** is specified (or abbreviated), the command is also executed and its result is returned. If **exec** is not specified, an empty string is returned.

history change *newValue [event_number]*

Replaces the value recorded for an event with *newValue*. The *event_number* specifies the event to replace, and defaults to the *current* event (not event **-1**). This command is intended for use in commands that implement new forms of history substitution and want to replace the current event (that invokes the substitution) with the command created through substitution. The return value is an empty string.

history clear Erases the history list. The current keep limit is retained. The history event numbers are reset.

history event *[event_number]*

Returns the value of the event given by *event_number*. The default value of *event_number* is **-1**.

history info *[count]*

Returns a formatted string, giving the

coreTools Command Reference Index

event number and contents for each of the events in the history list except the current event. If *count* is specified, then only the most recent *count* events are returned.

history keep [*count*]

Changes the size of the history list to *count* events. Initially, 20 events are retained in the history list. If *count* is not specified, the current keep limit is returned.

history nextid Returns the number of the next event to be recorded in the history list. Use this for printing the event number in command-line prompts.

history redo [*event_number*]

Reruns the command indicated by *event* and returns its result. The default value of *event_number* is -1. This command results in history revision. See the following section for details.

History Revision

Pre-8.0 Tcl had a complex history revision mechanism. The current mechanism is more limited, and the **substitute** and **words** history operations have been removed. The **clear** operation was added.

The **redo** history option results in much simpler "history revision." When this option is invoked, the most recent event is modified to eliminate the history command and replace it with the result of the history command. If you want to redo an event without modifying the history, use the **event** operation to retrieve an event, and use the **add** operation to add it to history and execute it.

NAME

http - Client-side implementation of the HTTP/1.1 protocol

SYNOPSIS

```
package require http ?2.9?

::http::config ?-option value ...?
::http::geturl url ?-option value ...?
::http::formatQuery key value ?key value ...?
::http::quoteString value
::http::reset token ?why?
::http::wait token
::http::status token
::http::size token
::http::code token
::http::ncode token
::http::meta token
::http::data token
::http::error token
::http::cleanup token
::http::register proto port command
::http::registerError port ?message?
::http::unregister proto
```

DESCRIPTION

The **http** package provides the client side of the HTTP/1.1 protocol, as defined in RFC 7230 to RFC 7235, which supersede RFC 2616. The package implements the GET, POST, and HEAD operations of HTTP/1.1. It allows configuration of a proxy host to get through firewalls. The package is compatible with the **Safesock** security policy, so it can be used by untrusted applets to do URL fetching from a restricted set of hosts. This package can be extended to support additional HTTP transport protocols, such as HTTPS, by providing a custom **socket** command, via **::http::register**.

The **::http::geturl** procedure does a HTTP transaction. Its *options* determine whether a GET, POST, or HEAD transaction is performed. The return value of **::http::geturl** is a token for the transaction. The value is also the name of an array in the **::http** namespace that contains state information about the transaction. The elements of this array are described in the **STATE ARRAY** section.

If the **-command** option is specified, then the HTTP operation is done in the background. **::http::geturl** returns immediately after generating the HTTP request and the callback is invoked when the transaction completes. For this to work, the Tcl event loop must be active. In Tk applications this is always true. For pure-Tcl applications, the caller can use **::http::wait** after calling **::http::geturl** to start the event loop.

Note: The event queue is even used without the **-command** option. As a side effect, arbitrary commands may be processed while **http::geturl** is running.

COMMANDS

::http::config ?options?

The **::http::config** command is used to set and query the name of the proxy server and port, and the User-Agent name used in the HTTP requests. If no options are specified, then the current configuration is returned. If a single argument is specified, then it should be one of the flags described below. In this case the current value of that setting is returned. Otherwise, the options should be a set of flags and values that define the configuration:

-accept *mimetypes*

coreTools Command Reference Index

The Accept header of the request. The default is `*/*`, which means that all types of documents are accepted. Otherwise you can supply a comma-separated list of mime type patterns that you are willing to receive. For example,

-pipeline *boolean*

Specifies whether HTTP/1.1 transactions on a persistent socket will be pipelined. See the **PERSISTENT SOCKETS** section for details. The default is 1.

-postfresh *boolean*

Specifies whether requests that use the **POST** method will always use a fresh socket, overriding the **-keepalive** option of command **http::geturl**. See the **PERSISTENT SOCKETS** section for details. The default is 0.

-proxyhost *hostname*

The name of the proxy host, if any. If this value is the empty string, the URL host is contacted directly.

-proxyport *number*

The proxy port number.

-proxyfilter *command*

The command is a callback that is made during **::http::geturl** to determine if a proxy is required for a given host. One argument, a host name, is added to *command* when it is invoked. If a proxy is required, the callback should return a two-element list containing the proxy server and proxy port. Otherwise the filter should return an empty list. The default filter returns the values of the **-proxyhost** and **-proxyport** settings if they are non-empty.

-repost *boolean*

Specifies what to do if a POST request over a persistent connection fails because the server has half-closed the connection. If boolean **true**, the request will be automatically retried; if boolean **false** it will not, and the application that uses **http::geturl** is expected to seek user confirmation before retrying the POST. The value **true** should be used only under certain conditions. See the **PERSISTENT SOCKETS** section for details. The default is 0.

-urlencoding *encoding*

The *encoding* used for creating the x-url-encoded URLs with **::http::formatQuery** and **::http::quoteString**. The default is **utf-8**, as specified by RFC 2718. Prior to http 2.5 this was unspecified, and that behavior can be returned by specifying the empty string (**{}**), although *iso8859-1* is recommended to restore similar behavior but without the **::http::formatQuery** or **::http::quoteString** throwing an error processing non-latin-1 characters.

-useragent *string*

The value of the User-Agent header in the HTTP request. In an unsafe interpreter, the default value depends upon the operating system, and the version numbers of

coreTools Command Reference Index

http and **Tcl**, and is (for example) A safe interpreter cannot determine its operating system, and so the default in a safe interpreter is to use a Windows 10 value with the current version numbers of **http** and **Tcl**.

-zip *boolean*

If the value is boolean **true**, then by default requests will send a header. If the value is boolean **false**, then by default this header will not be sent. In either case the default can be overridden for an individual request by supplying a custom **Accept-Encoding** header in the **-headers** option of **http::geturl**. The default is 1.

::http::geturl *url ?options?*

The **::http::geturl** command is the main procedure in the package. The **-query** option causes a POST operation and the **-validate** option causes a HEAD operation; otherwise, a GET operation is performed. The **::http::geturl** command returns a *token* value that can be used to get information about the transaction. See the **STATE ARRAY** and **ERRORS** section for details. The **::http::geturl** command blocks until the operation completes, unless the **-command** option specifies a callback that is invoked when the HTTP transaction completes. **::http::geturl** takes several options:

-binary *boolean*

Specifies whether to force interpreting the URL data as binary. Normally this is auto-detected (anything not beginning with a **text** content type or whose content encoding is **gzip** or **compress** is considered binary data).

-blocksize *size*

The block size used when reading the URL. At most *size* bytes are read at once. After each block, a call to the **-progress** callback is made (if that option is specified).

-channel *name*

Copy the URL contents to channel *name* instead of saving it in **state(body)**.

-command *callback*

Invoke *callback* after the HTTP transaction completes. This option causes **::http::geturl** to return immediately. The *callback* gets an additional argument that is the *token* returned from **::http::geturl**. This token is the name of an array that is described in the **STATE ARRAY** section. Here is a template for the callback:

```
proc httpCallback {token} {
    upvar #0 $token state
    # Access state as a Tcl array
}
```

-handler *callback*

Invoke *callback* whenever HTTP data is available; if present, nothing else will be done with the HTTP data. This procedure gets two additional arguments: the

coreTools Command Reference Index

socket for the HTTP data and the *token* returned from `::http::geturl`. The token is the name of a global array that is described in the **STATE ARRAY** section. The procedure is expected to return the number of bytes read from the socket. Here is a template for the callback:

```
proc httpHandlerCallback {socket token} {
    upvar #0 $token state
    # Access socket, and state as a Tcl array
    # For example...
    ...
    set data [read $socket 1000]
    set nbytes [string length $data]
    ...
    return $nbytes
}
```

The `http::geturl` code for the **-handler** option is not compatible with either compression or chunked transfer-encoding. If **-handler** is specified, then to work around these issues `http::geturl` will reduce the HTTP protocol to 1.0, and override the **-zip** option (i.e. it will not send the header **"Accept-Encoding: gzip,deflate,compress"**).

If options **-handler** and **-channel** are used together, the handler is responsible for copying the data from the HTTP socket to the specified channel. The name of the channel is available to the handler as element **-channel** of the token array.

-headers *keyvaluelist*

This option is used to add headers not already specified by `::http::config` to the HTTP request. The *keyvaluelist* argument must be a list with an even number of elements that alternate between keys and values. The keys become header field names. Newlines are stripped from the values so the header cannot be corrupted. For example, if *keyvaluelist* is **Pragma no-cache** then the following header is included in the HTTP request:

Pragma: no-cache

-keepalive *boolean*

If boolean **true**, attempt to keep the connection open for servicing multiple requests. Default is 0.

-method *type*

Force the HTTP request method to *type*. `::http::geturl` will auto-select GET, POST or HEAD based on other options, but this option enables choices like PUT and DELETE for webdav support.

-myaddr *address*

Pass an specific local address to the underlying **socket** call in case multiple interfaces are available.

-progress *callback*

The *callback* is made after each transfer of data from the URL. The callback gets three additional arguments:

coreTools Command Reference Index

the *token* from **::http::geturl**, the expected total size of the contents from the **Content-Length** meta-data, and the current number of bytes transferred so far. The expected total size may be unknown, in which case zero is passed to the callback. Here is a template for the progress callback:

```
proc httpProgress {token total current} {  
    upvar #0 $token state  
}
```

-protocol *version*

Select the HTTP protocol version to use. This should be 1.0 or 1.1 (the default). Should only be necessary for servers that do not understand or otherwise complain about HTTP/1.1.

-query *query*

This flag causes **::http::geturl** to do a POST request that passes the *query* as payload verbatim to the server. The content format (and encoding) of *query* is announced by the header field **content-type** set by the option **-type**. *query* is an x-url-encoding formatted query, if used for html forms. The **::http::formatQuery** procedure can be used to do the formatting.

-queryblocksize *size*

The block size used when posting query data to the URL. At most *size* bytes are written at once. After each block, a call to the **-queryprogress** callback is made (if that option is specified).

-querychannel *channelID*

This flag causes **::http::geturl** to do a POST request that passes the data contained in *channelID* to the server. The data contained in *channelID* must be an x-url-encoding formatted query unless the **-type** option below is used. If a Content-Length header is not specified via the **-headers** options, **::http::geturl** attempts to determine the size of the post data in order to create that header. If it is unable to determine the size, it returns an error.

-queryprogress *callback*

The *callback* is made after each transfer of data to the URL (i.e. POST) and acts exactly like the **-progress** option (the callback format is the same).

-strict *boolean*

Whether to enforce RFC 3986 URL validation on the request. Default is 1.

-timeout *milliseconds*

If *milliseconds* is non-zero, then **::http::geturl** sets up a timeout to occur after the specified number of milliseconds. A timeout results in a call to **::http::reset** and to the **-command** callback, if specified. The return value of **::http::status** is **timeout** after a timeout has occurred.

-type *mime-type*

Use *mime-type* as the **Content-Type** value, instead of the

coreTools Command Reference Index

default value (**application/x-www-form-urlencoded**) during a POST operation.

-validate *boolean*

If *boolean* is non-zero, then **::http::geturl** does an HTTP HEAD request. This request returns meta information about the URL, but the contents are not returned. The meta information is available in the **state(meta)** variable after the transaction. See the **STATE ARRAY** section for details.

::http::formatQuery *key value ?key value ...?*

This procedure does x-url-encoding of query data. It takes an even number of arguments that are the keys and values of the query. It encodes the keys and values, and generates one string that has the proper & and = separators. The result is suitable for the **-query** value passed to **::http::geturl**.

::http::quoteString *value*

This procedure does x-url-encoding of string. It takes a single argument and encodes it.

::http::reset *token ?why?*

This command resets the HTTP transaction identified by *token*, if any. This sets the **state(status)** value to *why*, which defaults to **reset**, and then calls the registered **-command** callback.

::http::wait *token*

This is a convenience procedure that blocks and waits for the transaction to complete. This only works in trusted code because it uses **vwait**. Also, it is not useful for the case where **::http::geturl** is called *without* the **-command** option because in this case the **::http::geturl** call does not return until the HTTP transaction is complete, and thus there is nothing to wait for.

::http::data *token*

This is a convenience procedure that returns the **body** element (i.e., the URL data) of the state array.

::http::error *token*

This is a convenience procedure that returns the **error** element of the state array.

::http::status *token*

This is a convenience procedure that returns the **status** element of the state array.

::http::code *token*

This is a convenience procedure that returns the **http** element of the state array.

::http::ncode *token*

This is a convenience procedure that returns just the numeric return code (200, 404, etc.) from the **http** element of the state array.

::http::size *token*

This is a convenience procedure that returns the

coreTools Command Reference Index

currentsize element of the state array, which represents the number of bytes received from the URL in the **::http::geturl** call.

::http::meta *token*

This is a convenience procedure that returns the **meta** element of the state array which contains the HTTP response headers. See below for an explanation of this element.

::http::cleanup *token*

This procedure cleans up the state associated with the connection identified by *token*. After this call, the procedures like **::http::data** cannot be used to get information about the operation. It is *strongly* recommended that you call this function after you are done with a given HTTP request. Not doing so will result in memory not being freed, and if your app calls **::http::geturl** enough times, the memory leak could cause a performance hit...or worse.

::http::register *proto port command*

This procedure allows one to provide custom HTTP transport types such as HTTPS, by registering a prefix, the default port, and the command to execute to create the Tcl **channel**. E.g.:

```
package require http
package require tls
```

```
::http::register https 443 ::tls::socket
```

```
set token [::http::geturl https://my.secure.site/]
```

::http::registerError *port ?message?*

This procedure allows a registered protocol handler to deliver an error message for use by **http**. Calling this command does not raise an error. The command is useful when a registered protocol detects a problem (for example, an invalid TLS certificate) that will cause an error to propagate to **http**. The command allows **http** to provide a precise error message rather than a general one. The command returns the value provided by the last call with argument *message*, or the empty string if no such call has been made.

::http::unregister *proto*

This procedure unregisters a protocol handler that was previously registered via **::http::register**, returning a two-item list of the default port and handler command that was previously installed (via **::http::register**) if there was such a handler, and an error if there was no such handler.

ERRORS

The **::http::geturl** procedure will raise errors in the following cases: invalid command line options, an invalid URL, a URL on a non-existent host, or a URL at

coreTools Command Reference Index

a bad port on an existing host. These errors mean that it cannot even start the network transaction. It will also raise an error if it gets an I/O error while writing out the HTTP request header. For synchronous `::http::geturl` calls (where `-command` is not specified), it will raise an error if it gets an I/O error while reading the HTTP reply headers or data. Because `::http::geturl` does not return a token in these cases, it does all the required cleanup and there is no issue of your app having to call `::http::cleanup`.

For asynchronous `::http::geturl` calls, all of the above error situations apply, except that if there is any error while reading the HTTP reply headers or data, no exception is thrown. This is because after writing the HTTP headers, `::http::geturl` returns, and the rest of the HTTP transaction occurs in the background. The command callback can check if any error occurred during the read by calling `::http::status` to check the status and if its *error*, calling `::http::error` to get the error message.

Alternatively, if the main program flow reaches a point where it needs to know the result of the asynchronous HTTP request, it can call `::http::wait` and then check status and error, just as the callback does.

In any case, you must still call `::http::cleanup` to delete the state array when you are done.

There are other possible results of the HTTP transaction determined by examining the status from `::http::status`. These are described below.

ok

If the HTTP transaction completes entirely, then status will be **ok**. However, you should still check the `::http::code` value to get the HTTP status. The `::http::ncode` procedure provides just the numeric error (e.g., 200, 404 or 500) while the `::http::code` procedure returns a value like

eof

If the server closes the socket without replying, then no error is raised, but the status of the transaction will be **eof**.

error

The error message will also be stored in the **error** status array element, accessible via `::http::error`.

timeout

A timeout occurred before the transaction could complete

reset

user-reset

Another error possibility is that `::http::geturl` is unable to write all the post query data to the server before the server responds and closes the socket. The

error message is saved in the **posterror** status array element and then **::http::geturl** attempts to complete the transaction. If it can read the server's response it will end up with an **ok** status, otherwise it will have an **eof** status.

STATE ARRAY

The **::http::geturl** procedure returns a *token* that can be used to get to the state of the HTTP transaction in the form of a Tcl array. Use this construct to create an easy-to-use array variable:

```
upvar #0 $token state
```

Once the data associated with the URL is no longer needed, the state array should be unset to free up storage. The **::http::cleanup** procedure is provided for that purpose. The following elements of the array are supported:

binary

This is boolean **true** if (after decoding any compression specified by the response header) the HTTP response is binary. It is boolean **false** if the HTTP response is text.

body

The contents of the URL. This will be empty if the **-channel** option has been specified. This value is returned by the **::http::data** command.

charset

The value of the charset attribute from the **Content-Type** meta-data value. If none was specified, this defaults to the RFC standard **iso8859-1**, or the value of **::http::defaultCharset**. Incoming text data will be automatically converted from this charset to utf-8.

coding

A copy of the **Content-Encoding** meta-data value.

currentsize

The current number of bytes fetched from the URL. This value is returned by the **::http::size** command.

error

If defined, this is the error string seen when the HTTP transaction was aborted.

http

The HTTP status reply from the server. This value is returned by the **::http::code** command. The format of this value is:

HTTP/1.1 code string

The *code* is a three-digit number defined in the HTTP standard. A code of 200 is OK. Codes beginning with 4

coreTools Command Reference Index

or 5 indicate errors. Codes beginning with 3 are redirection errors. In this case the **Location** meta-data specifies a new URL that contains the requested information.

meta

The HTTP protocol returns meta-data that describes the URL contents. The **meta** element of the state array is a list of the keys and values of the meta-data. This is in a format useful for initializing an array that just contains the meta-data:

```
array set meta $state(meta)
```

Some of the meta-data keys are listed below, but the HTTP standard defines more, and servers are free to add their own.

Content-Type

The type of the URL contents. Examples include **text/html**, **image/gif**, **application/postscript** and **application/x-tcl**.

Content-Length

The advertised size of the contents. The actual size obtained by `::http::geturl` is available as **state(currentsize)**.

Location

An alternate URL that contains the requested data.

posterror

The error, if any, that occurred while writing the post query data to the server.

status

See description in the chapter **ERRORS** above for a list and description of **status**. During the transaction this value is the empty string.

totalsize

A copy of the **Content-Length** meta-data value.

type

A copy of the **Content-Type** meta-data value.

url

The requested URL.

PERSISTENT CONNECTIONS

BASICS

See RFC 7230 Sec 6, which supersedes RFC 2616 Sec 8.1.

A persistent connection allows multiple HTTP/1.1 transactions to be carried over the same TCP connection. Pipelining allows a client to make multiple requests over a persistent connection without waiting for each response. The server sends responses

coreTools Command Reference Index

in the same order that the requests were received.

If a POST request fails to complete, typically user confirmation is needed before sending the request again. The user may wish to verify whether the server was modified by the failed POST request, before sending the same request again.

A HTTP request will use a persistent socket if the call to **http::geturl** has the option **-keepalive true**. It will use pipelining where permitted if the **http::config** option **-pipeline** is boolean **true** (its default value).

The http package maintains no more than one persistent connection to each server (i.e. each value of **http::geturl** is called to make a request over a persistent connection while the connection is busy with another request, the new request will be held in a queue until the connection is free.

The http package does not support HTTP/1.0 persistent connections controlled by the **Keep-Alive** header.

SPECIAL CASES

This subsection discusses issues related to closure of the persistent connection by the server, automatic retry of failed requests, the special treatment necessary for POST requests, and the options for dealing with these cases.

In accordance with RFC 7230, **http::geturl** does not pipeline requests that use the POST method. If a POST uses a persistent connection and is not the first request on that connection, **http::geturl** waits until it has received the response for the previous request; or (if **http::config** option **-postfresh** is boolean **true**) it uses a new connection for each POST.

If the server is processing a number of pipelined requests, and sends a response header with one of the responses (other than the last), then subsequent responses are unfulfilled. **http::geturl** will send the unfulfilled requests again over a new connection.

A difficulty arises when a HTTP client sends a request over a persistent connection that has been idle for a while. The HTTP server may half-close an apparently idle connection while the client is sending a request, but before the request arrives at the server: in this case (an the request will fail. The difficulty arises because the client cannot be certain whether the POST modified the state of the server. For HEAD or GET requests, **http::geturl** opens another connection and retransmits the failed request. However, if the request was a POST, RFC 7230 forbids automatic retry by default, suggesting either user confirmation, or confirmation by user-agent software that has semantic understanding of the application. The **http::config** option **-repost** allows for either possibility.

Asynchronous close events can occur only in a short interval of time. The **http** package monitors each persistent connection for closure by the server. Upon

coreTools Command Reference Index

detection, the connection is also closed at the client end, and subsequent requests will use a fresh connection.

If the **http::geturl** command is called with option **-keepalive true**, then it will both try to use an existing persistent connection (if one is available), and it will send the server a request header asking to keep the connection open for future requests.

The **http::config** options **-pipeline**, **-postfresh**, and **-repost** relate to persistent connections.

Option **-pipeline**, if boolean **true**, will pipeline GET and HEAD requests made over a persistent connection. POST requests will not be pipelined - if the POST is not the first transaction on the connection, its request will not be sent until the previous response has finished. GET and HEAD requests made after a POST will not be sent until the POST response has been delivered, and will not be sent if the POST fails.

Option **-postfresh**, if boolean **true**, will override the **http::geturl** option **-keepalive**, and always open a fresh connection for a POST request.

Option **-repost**, if **true**, permits automatic retry of a POST request that fails because it uses a persistent connection that the server has half-closed (an Subsequent GET and HEAD requests in a failed pipeline will also be retried. *The -repost option should be used only if the application understands that the retry is appropriate* - specifically, the application must know that if the failed POST successfully modified the state of the server, a repeat POST would have no adverse effect.

EXAMPLE

This example creates a procedure to copy a URL to a file while printing a progress meter, and prints the meta-data associated with the URL.

```
proc httpcopy { url file {chunk 4096} } {
    set out [open $file w]
    set token [::http::geturl $url -channel $out \
        -progress httpCopyProgress -blocksize $chunk]
    close $out

    # This ends the line started by httpCopyProgress
    puts stderr ""

    upvar #0 $token state
    set max 0
    foreach {name value} $state(meta) {
        if {[string length $name] > $max} {
            set max [string length $name]
        }
    }
}
```

coreTools Command Reference Index

```
if {[regexp -nocase ^location$ $name]} {  
    # Handle URL redirects  
    puts stderr "Location:$value"  
    return [httpcopy [string trim $value] $file $chunk]  
}  
}  
incr max  
foreach {name value} $state(meta) {  
    puts [format "%-*s %s" $max $name: $value]  
}  
  
return $token  
}  
proc httpCopyProgress {args} {  
    puts -nonewline stderr .  
    flush stderr  
}
```

SEE ALSO

safe(n), socket(n), safesock(n)

KEYWORDS

internet, security policy, socket, www

NAME

`if` - Execute scripts conditionally

SYNOPSIS

```
if expr1 ?then? body1 elseif expr2 ?then? body2 elseif  
... ?else? ?bodyN?
```

DESCRIPTION

The `if` command evaluates *expr1* as an expression (in the same way that **expr** evaluates its argument). The value of the expression must be a boolean (a numeric value, where 0 is false and anything is true, or a string value such as **true** or **yes** for true and **false** or **no** for false); if it is true then *body1* is executed by passing it to the Tcl interpreter. Otherwise *expr2* is evaluated as an expression and if it is true then **body2** is executed, and so on. If none of the expressions evaluates to true then *bodyN* is executed. The **then** and **else** arguments are optional to make the command easier to read. There may be any number of **elseif** clauses, including zero. *BodyN* may also be omitted as long as **else** is omitted too. The return value from the command is the result of the body script that was executed, or an empty string if none of the expressions was non-zero and there was no *bodyN*.

EXAMPLES

A simple conditional:

coreTools Command Reference Index

```
if {$vbl == 1} { puts "vbl is one" }
```

With an **else**-clause:

```
if {$vbl == 1} {  
    puts "vbl is one"  
} else {  
    puts "vbl is not one"  
}
```

With an **elseif**-clause too:

```
if {$vbl == 1} {  
    puts "vbl is one"  
} elseif {$vbl == 2} {  
    puts "vbl is two"  
} else {  
    puts "vbl is not one or two"  
}
```

Remember, expressions can be multi-line, but in that case it can be a good idea to use the optional **then** keyword for clarity:

```
if {  
    $vbl == 1  
    || $vbl == 2  
    || $vbl == 3  
} then {  
    puts "vbl is one, two or three"  
}
```

SEE ALSO

`expr(n)`, `for(n)`, `foreach(n)`

KEYWORDS

boolean, conditional, else, false, if, true

import_component

Import an unpackaged component into the subsystem.

Syntax

```
string import_component -name <name> -design <name> -language <language>
[-library <libName>] [-include <dirList>] [-VCSOptions <options>]
[-intent <dirList>] [-linklibs <dbList>] [-noshare]
    string <name>
    string <language>
    string <fileList>
    string <libName>
    string <dirList>
    string <options>
    string <dbList>
```

Parameters

-name <name>	New component cell name
-design <name>	Name of top-level design coreAssembler will only keep and maintain design intent for this design. All other entities and modules are ignored and under full responsibility of the component developer.
-language <language>	Chooses the HDL input language (Values: vhdl, verilog, systemverilog)
-files <fileList>	List of HDL source and .lst files in analysis order The source files will be analyzed and the design information for the top-level design is captured. All files are copied internally: Any change of the source files after import_component requires a full re-import. The analysis order is important for synthesis: import_component filters for the top-level design but synthesis requires all files in correct order. Please note that missing files for lower-level designs are not recognized until subsystem-synthesis. In many cases the source files just represent an HDL wrapper around an already synthesized block. Then this wrapper will link against the link-libraries during synthesis. Be aware that a top-level design is still required as wrapper. The list of source files can include two-element lists consisting of {library LibName}: This changes the working library of the following source files in case that the design requires multiple VHDL libraries. The list of HDL files with optional library and include path

coreTools Command Reference Index

	specifications can be specified in a file with a .lst extension.
-library <libName>	Name of VHDL working library (default: WORK) If the language is VHDL then this library represents the name of the first working library. For a sequence of source files you can change the working library for some sources as described above under -files.
-include <dirList>	List of directories to search in for Verilog include files
-VCSOptions <options>	Options for compiling HDL source with VCS
	List of directories to search in for Tcl & SDC intent files Please be aware that the design contains at least one clock, and that a top-level design port must identify this clock with ClockName. Set the port attribute in a stand-alone Tcl intent file if you do not have SDC file(s). The name of the intent file must be <top-level-source-file>.tcl (without source file suffix .v or .vhd etc). If SDC constraints exist then the constraints are read if the file name is <top-level-source-file>.sdc (no HDL suffix inside <..>). To find the intent files the -intent option lists the directories where the files should be searched.
-intent <dirList>	List of synthesis libraries the top-level is linked with In most cases the source files represent a structural wrapper around a cell which is pre-synthesized, or a netlist of multiple pre-synthesized cells. The wrapper is linked against the listed synthesis libraries (db files). Whether most of the design is contained in HDL source files and synthesized with a default top-down strategy, or whether it is just a wrapper around a pre-synthesized netlist, or some mixture, is fully flexible. Make sure that a wrapper around a pre-synthesized "don't-touch" netlist requires design intent PreserveMapping on the top-level design. Like the source files the link-netlist files are copied internally: Any change to the netlist requires a re-import of the component.
-linklibs <dbList>	
-noshare	Don't auto share if sharing is enabled

Description

coreAssembler allows the import of HDL designs into a subsystem even if the design is not packaged (and installed) as a coreKit. import_component keeps design information about the indicated top-level design but allows synthesis of the entire source file list in DC with the given analysis order. In most cases the source files just represent a HDL wrapper around

a pre-synthesized netlist; then this netlist is loaded and linked as db files against the wrapper.

Obviously imported components do not offer the capabilities of a fully packaged coreKit but their use enables the integration of your own components into the subsystem.

Because coreAssembler supports automatic connection between components and consistent cross-component configuration, all imported components require each at least one interface attached to the imported component from a packaged, instantiated component.

After `import_component`, the imported component behaves like a packaged component with just a top-level design. To add interfaces to the component use `attach_interface`.

Examples

To import an own UART component into the subsystem (entire source code):

```
coreAssembler> import_component -name Uuart
-design qw_uart
-language vhd
-files {src/qw_uart.vhd src/qw_uart_async.vhd}
-intent {src}
```

To import your own DMA component into the subsystem (wrapper around netlist):

```
coreAssembler> import_component -name DMA
-design dma_wrapper
-language verilog
-files {src/dma_wrapper.v}
-intent {sdc}
-linklibs {db/dma.db}
```

To use a file to specify the list of HDL source files:

```
coreAssembler> import_component -name U1
-design designTop
-language systemverilog
-files hdl_files.lst
# HDL list file "hdl_files.lst"
include=inc1
mod1.sv
mod2.sv
mod3.sv
designTop.sv
```

See Also

instantiate_component (2), remove_component (2), all_components (2), _____
attach_interface (2) duplicate_component (2) _____

import_ipxact_data

Import partial IP-XACT component data.

Syntax

```
string import_ipxact_data xmlfilename  
      string xmlfilename
```

Parameters

<code>xmlfilename</code>	The name of the IP-XACT component file to import.
--------------------------	---

Description

This command is used to import portions of an IP-XACT component file into the current coreBuilder workspace. The imported data includes memory maps and vendor extensions on the component and component ports.

Examples

Import memory map definition from an existing IP-XACT component file:

```
coreBuilder> import_ipxact_data mycomponent.xml
```

See Also

import_workspace_as_component

Import the workspace as a component.

Syntax

string **import_workspace_as_component** -file <tarfile/directory> [-name <name>] [-readonly]

string <tarfile/directory>

string <name>

Parameters

-file <tarfile/directory>	The compressed tar file or exported workspace directory to import from.
-name <name>	The name of the component instance.
-readonly	Keep the component read only.

Description

coreAssembler allows the import of a component workspace file set or tar file that was generated using the export_workspace_as_component command. For the -file argument you would specify the top directory of the workspace file set or the name of the compressed tar file.

If a workspace you import is elaborated then it will be read-only and you will not be able to make changes to that workspace component or its subcomponents.

If the workspace you import is not elaborated then you can make changes unless you import it using the -readonly commandline argument.

Examples

```
coreAssembler> import_workspace_as_component
-name i_ws1
-file ./export/workspace_component/Subsystem1
coreAssembler> import_workspace_as_component
-name i_ws1
-file ./Subsystem1.tar.gz
coreAssembler> replace_component
-name i_ws1
./Subsystem1.tar.gz
```


See Also

[export_workspace_as_component \(2\), replace_component \(2\)](#)

`in_autocomplete_activity`

Is an activity currently being auto-completed?

Syntax

string `in_autocomplete_activity`

Description

This command can be used to indicate whether or not the processing of the current activity caused to a call to the `autocomplete_activity` command. In certain cases, activity processing commands (e.g. the `PostPromptCmd`) will behave differently when being auto-completed. This command is provided so that these commands can make that determination.

Examples

The following is a sample proc fragment utilizing `in_autocomplete_activity`:

```
if {[in_autocomplete_activity]} {  
  # Processing for auto-completion  
} else {  
  # Standard processing  
}
```

See Also

NAME

`incr` - Increment the value of a variable

SYNOPSIS

`incr varName ?increment?`

DESCRIPTION

Increments the value stored in the variable whose name is *varName*. The value of the variable must be an integer. If *increment* is supplied then its value (which must be an integer) is added to the value of variable *varName*; otherwise 1 is added to *varName*. The new value is stored as a decimal string in variable *varName* and also returned as result.

Starting with the Tcl 8.5 release, the variable *varName* passed to **incr** may be unset, and in that case, it will be set to the value *increment* or to the default increment value of **1**.

EXAMPLES

Add one to the contents of the variable **x**:

```
incr x
```

Add 42 to the contents of the variable **x**:

```
incr x 42
```

Add the contents of the variable **y** to the contents of

the variable `x`:

```
incr x $y
```

Add nothing at all to the variable `x` (often useful for checking whether an argument to a procedure is actually integral and generating an error if it is not):

```
incr x 0
```

SEE ALSO

`expr(n)`, `set(n)`

KEYWORDS

add, increment, variable, value

coreTools Command Reference Index

Information by Category

[Application-Programming-Interface](#)

[Activity](#)

[Attributes](#)

[Commands](#)

[Executables](#)

[Item Types](#)

[Strategy](#)

[TCL-Built-in-Commands](#)

[Variables](#)

Alphabetical Listing

[A](#) [B](#) [C](#) [D](#) [E](#) [F](#) [G](#) [H](#) [I](#) [J](#) [K](#) [L](#) [M](#) [N](#) [O](#) [P](#) [Q](#) [R](#) [S](#) [T](#) [U](#) [V](#) [W](#) [X](#) [Y](#) [Z](#) _ _ _ _ _

A

[Abstraction](#)

Indicates the abstraction of an interface instance or the abstraction in which a given interface port should be present.

[Activity](#)

Activity bound to a menu item.

[ActivityGroup](#)

Group/Menu to put this to-level activity in

[AddInterfaces](#)

Add interfaces to the top-level of the core for automated assembly of this core in a System-on-a-Chip subsystem.

[AddLockUpLatch](#)

When true lockup latches will be inserted between clock domain boundaries on scan chains.

[addressBank](#)

This item represents an address bank in a memory map.

[addressBlock](#)

This item represents a block of memory in a memory map.

[AddressOffset](#)

Offset from the base address of the address block for this register or registerArray

[AddressSpaceRef](#)

The name of the address space.

[add activity hook](#)

Add a hook to an existing activity

[add command hook](#)

Add hook into command execution

coreTools Command Reference Index

<u>add files to filegroup</u>	Add files to the specified file group in the Bill of Materials; create the file group if it does not already exist
<u>add hdl pragma</u>	A reuse-pragma statement external to the HDL
<u>add help menu item</u>	Add an item to the console Help menu and link it to relevant documentation
<u>add html report link</u>	Add an HTML report link to the index page
<u>add instantiate component hook</u>	Add hook to execute a Tcl plugin command when instantiate_component is completed
<u>add to collection</u>	Add object(s) to a collection. Result is new collection
<u>add verify subsystem hook</u>	Execute a Tcl checker command to verify subsystem at various places
<u>add workspace hook</u>	Execute a Tcl script when a workspace operation occurs
<u>after</u>	Execute a command after a time delay
<u>alias</u>	Creates a pseudo-command that expands to one or more words, or lists current alias definitions.
<u>AllowOpenProvider</u>	Flag disabling the requirement that provider interfaces must provide a driver when the consumer has an input port.
<u>all components</u>	Return sorted list of all currently loaded components in the sub-system.
<u>all designs</u>	Create a collection of all currently loaded designs
<u>all inputs</u>	Return a collection of all input and inout ports of the current_design
<u>all outputs</u>	Return a collection of all output and inout ports of the current_design
<u>all subdesigns</u>	Return a collection of all subdesigns of the current_design
<u>AltValue</u>	Alternate value for the attribute.
<u>analyze filegroup</u>	Source intent files for this filegroup.
<u>annotate trace</u>	Control annotation of traced command execution to the shell output.
<u>API coreAssembler</u>	Defines the application programming interface for coreAssembler
<u>API coreBuilder</u>	Defines the application programming interface for coreBuilder
<u>API coreConsultant</u>	Defines the application programming interface for coreConsultant
<u>append</u>	Append to variable
<u>append to collection</u>	Add object(s) to a collection. Modifies variable

coreTools Command Reference Index

<u>apply</u>	Apply an anonymous function
<u>apropos</u>	Searches the command database for a pattern.
<u>AreaEstimate</u>	Estimated cell area of a design. Indicates relative importance of a parameter with respect to area estimation. By default all parameters have a weight of 1 but parameters that have a bigger influence on the area of a design can have their match importance increased by raising the AreaWeight value.
<u>AreaWeight</u>	
<u>array</u>	Manipulate array variables
<u>ArrayEnd</u>	Final index for the array entries.
<u>ArrayFieldSize</u>	Number of bits per element in the array
<u>ArrayStart</u>	Initial index for the array entries.
<u>AssociationComplete</u>	Is true if all ports and parameters of the interface instance are completely 'linked'. This attribute does NOT indicate whether the association is valid and error-free. Provides specific completeness information on interface ports and parameters.
<u>AssociationFormula</u>	Formula to determine the alternative value to use during interface-design association.
<u>as_collection</u>	Find a collection or iter_collection by name; result a collection
<u>AtpgTclAuxScript</u>	Specifies a TestMAX Tcl script to be executed at specific points during test vector generation.
<u>AtpgTclAuxScriptComment</u>	Specifies a comment to be issued for the corresponding AtpgTclAuxScript
<u>ATPG basic scan strategy</u>	The ATPG_basic_scan_strategy will create test vectors using the Basic_scan_only engine of TetraMax. This is the recommended starting point for generating vectors, and it will provide fast, good coverage for full scan designs.
<u>attach_interface</u>	Attach a component interface to another component.
<u>attrDefn</u>	Definition of an attribute
<u>autocomplete activity</u>	Automatically complete activities
<u>AutoConnectComplete</u>	Indicates this design was auto connected.
<u>AutoConnectIfUnconnected</u>	Connection to make if left unconnected during auto-connection step.

coreTools Command Reference Index

<u>AutoConnectionPreCallback</u>	Defines user-callback to be invoked before making all automatic connections at a given level of hierarchy.
<u>AutoConnectWhen</u>	Indicates if/when the given interface should be automatically connected.
<u>AutoFixAsync</u>	Enables/Disables automatic fixing of asynchronous preset/clear violations. When set to gate, specifies that the asynchronous sets and resets are autofixed with a gate driven by the test_mode signal. When set to mux sets and resets are with a multiplexer. The select input of the mux is the test_mode signal and a primary input is controlling the asynchronous set or reset in test mode.
<u>AutoFixAsyncLogicGate</u>	
<u>AutoFixAsyncTestData</u>	Port to be used as TestData for fixing set/reset violations. The port must have been defined with a DftExistingSignalType of Reset.
<u>AutoFixAsyncWithScanEnable</u>	When true the test scan enable signal will be used to autofix asynchronous set and reset violations.
<u>AutoFixBidirectional</u>	Enables/Disables automatic bidirectional disabling.
<u>AutoFixClocks</u>	Enables/Disables automatic fixing of uncontrollable clock violations Port to be used as TestData for fixing clock violations. The port must be associated with a clock that has TestClock==1 or have been defined with a DftExistingSignalType of a clock type.
<u>AutoFixClockTestData</u>	
<u>AutoloadFilegroup</u>	Are the files in this group loaded automatically?

B

<u>BalanceBistSegments</u>	By default, SoCBIST rebalances the scan channels in BIST mode. Setting BalanceBistSegments to false will override this, and SoCBIST will not concatenate the scan segments.
<u>BankAlignment</u>	The memory alignment of this bank
<u>BaseAddress</u>	The base address of this item.
<u>batch_install</u>	Install a packaged coreKit into the specified installation root directory.

coreTools Command Reference Index

<u>bgererror</u>	Command invoked to process background errors
<u>BidirectionalMode</u>	Determines whether bidirectional ports are treated as inputs, outputs or left untouched during scan shift.
<u>binary</u>	Insert and extract fields from binary strings
<u>bind tech cell mapping</u>	Specify binding requirements for mapping to a technology specific cell.
<u>BistAutoFixBusses</u>	Indicates floating bus and bus contention issues should be fixed automatically.
<u>BistAutoFixXprop</u>	Should automatic fixing of x propagation be enabled?
<u>BistBlockIndividually</u>	Causes insertion of BIST logic into this design. By default, DFT Compiler concatenates lower level scan segments to arrive at the optimal number of scan chains during BIST controller insertion. You can override this number by specifying the scan configuration
<u>BistChainCount</u>	NumberOfScanChains or MaximumScanChainLength attributes. You must also use BistChainCount to control how many channels are hooked up to the controller. Setting this attribute to 0 allows DFT compiler to choose the number of scan chains.
<u>BistCodecCount</u>	Number of BIST CODECS
<u>BistDiagOutputs</u>	The number of diagnostic outputs for DBIST
<u>BistInvertPrpgClock</u>	Invert the BIST clock that feeds the PRPG Registers Controls control the size of the counter, which increases the default upper limit on the length of the reconfigured scan chains.
<u>BistMaxChainLength</u>	Setting BistMaxChainLength to 0 will allow DFT Compiler to determine the size of the counter. BIST operations to be performed. Choosing 'all' will cause the design to be made bist ready and for the codec to be inserted and integrated with the controller. For hierarchical bist insertion choose
<u>BistMode</u>	codec_insertion for the lower levels of hierarchy and all at the level where the controller is to be integrated with the codecs. Choose bist_ready to make many BIST scan chains or channels within the design without X-propagation and without wrapping the design.
<u>BistObserveOutputs</u>	Controls the number of scan output pins that are inserted into the design for BistMode == xdbist. By default, the number of scan outputs

coreTools Command Reference Index

	created depends on the number of scan chains: scan outputs = (scan chains/32). Setting this attribute to 0 in coreTools lets DFT compiler use the default value. By default, SoCBIST creates a 479-bit PRPG. You can override this by setting BistPrpgLength to 257. This will cause a 257-bit PRPG to be created. For DBIST, you can use up to 512 scan chains with the 257-bit PRPG. For X-tolerant DBIST, you can use 2048 scan chains with the 257-bit PRPG. By default, DFT Compiler creates from one to twelve shadow scan chains for the PRPG shadow. The number of shadow scan chains created is optimized based on the length of the longest scan chain in the design. You can stipulate that DFT Compiler create a specific number of shadow scan chains instead by setting BistPrpgShadowSi to n, where n is a number from one to twelve. Setting BistPrpgShadowSi to default allows DFT Compiler to determine the number of shadow scan chains. BistPrpgShadowSi does not affect the length of the longest scan chain. It only affects the number of shadow scan chains. The number of selector shadow chains. Setting BistSelectorShadowSi will allow DFT Compiler to determine the number of selector shadow chains based on the length of the longest scan chain.
<u>BistPrpgLength</u>	
<u>BistPrpgShadowSi</u>	
<u>BistSelectorShadowSi</u>	
<u>BistSubblocksIndividually</u>	Causes BIST logic to be inserted into the subdesigns of this design.
<u>BistType</u>	Type of BIST to implement
<u>BistUseTristateMux</u>	Specifies that tri-state muxes be used to construct the XDBIST codec
<u>BitAddressOffset</u>	Offset in bits from the base address of the register or memory block
<u>BitRange</u>	Returns the bit range of this field as [start]:[end]. Range is not evaluated if the start or end bit indicator is a formula.
<u>BitsInLAU</u>	The number of bits in the Least Addressable Unit (LAU)
<u>BitsToRepresent</u>	The number of bits needed to represent valid values for this type.
<u>BitWidth</u>	The number of bits in the bitfield.
<u>break</u>	Abort looping command
<u>Bridge</u>	Indicates that there is a bridge from this interface across the component.

coreTools Command Reference Index

<u>BuildcoreKit</u>	Build the coreKit for this core.
<u>BuildIPXACTComponent</u>	Build the IPX-ACT for this core.
<u>BuildSubsystemcoreKit</u>	Run coreBuilder and package the subsystem as a coreKit.
<u>build_debug_tarfile</u>	Build a debug tar-file Usually an interface port needs 'full' association to a design port with the same number of bits as InterfaceSize. Some vectored ports (busses) allow partial connection to an opposite design port. BusAlignment indicates how a narrower design port aligns within a bus of full interface size, and whether it connects on the left or right hand side of a full opposite port, or by slice-indices on a full port with a 0..InterfaceSize-1 indexing and same bit order. The bitwise alignment value is currently only available for IP-XACT components and should not be used within coreBuilder.
<u>BusAlignment</u>	
<u>busBit</u>	Individual bit of a bus
<u>byte_compile_file</u>	Byte compile a TCL file or text string.
<u>C</u>	
<u>CanUngroup</u>	Indicates that the design can be replaced by a hierarchy component.
<u>CanUngroupInfo</u>	Information about design HDL file and/or design contents when CanUngroup value is false.
<u>Capacitance</u>	Explicit load capacitance.
<u>case</u>	Evaluate one of several scripts, depending on a given value Static value that the port or pin will take on for the synthesis session.
<u>CaseAnalysisValue</u>	Unlike ConstantPort, CaseAnalysisValue will not cause logic to be optimized away.
<u>catch</u>	Evaluate script and trap exceptional returns
<u>cA supports hierarchy</u>	Controls whether or not hierarchy is supported by coreAssembler.
<u>cd</u>	Change working directory
<u>cell</u>	Cell item
<u>chan</u>	Read, write and manipulate channels
<u>Channel</u>	Indicates that there is a channel from this interface across the component.

coreTools Command Reference Index

<u>CHeaderValue</u>	Defines the value for the C header file for the given attribute on this register or register field.
<u>CheckCmd</u>	Command used to check values
<u>CheckExpr</u>	Check expression for a parameter
<u>CheckExprMessage</u>	Check expression message for a parameter
<u>CheckExprWhenDisabled</u>	Specifies whether or not CheckExpr expressions should be checked for non-visible or non-enabled parameters.
<u>check bom</u>	Check Bill Of Materials
<u>check component for parameter loops</u>	Check design parameters for the top design for loops return error count.
<u>check env vars</u>	Check for the existence of a list of environment variables.
<u>check error list</u>	List of messages IDs that should cause Design Compiler to stop in the case where one or more of the message appears during a call to the 'compile' command.
<u>check executables</u>	Check for the existence of a list of executables.
<u>check file for errors</u>	Check the specified file for error messages and return the number of error messages found
<u>check license</u>	Get information about a license
<u>check ParameterInfo</u>	Check the syntax and semantics of a ParameterInfo list string
<u>check parameter attribute formulas</u>	Check the formula value of an attribute on a parameter for any issues like dependancy loops.
<u>check script</u>	Statically check a script using the a static Tcl syntax checker based on TclPro Procheck and the Synopsys checking extensions for this tool.
<u>check workspace project id</u>	Compares the Project ID specified against the Project ID used to create the workspace
<u>class</u>	class of all classes
<u>clear prime profile validation exprs</u>	Clears all validation expressions for Prime Profiles.
<u>clock</u>	Clock item
<u>ClockFallLatency</u>	The time it takes a clock signal to propagate from the clock definition point to a register clock pin for a falling transition at the clock

coreTools Command Reference Index

<u>ClockGatingFallHoldCheck</u>	definition point. Falling edge hold check for gating control signals for this clock.
<u>ClockGatingFallSetupCheck</u>	Falling edge setup check for gating control signals for this clock.
<u>ClockGatingRiseHoldCheck</u>	Rising edge hold check for gating control signals for this clock.
<u>ClockGatingRiseSetupCheck</u>	Rising edge setup check for gating control signals for this clock.
<u>ClockGatingSignals</u>	Use this attribute to specify a list of signals to include/exclude when inferring flip-flops with gated clocks.
<u>ClockHoldUncertainty</u>	Hold uncertainty applied to all endpoints of the given clock.
<u>ClockMixing</u>	Specifies whether cells from different clock domains may be put in the same scan chain
<u>ClockName</u>	Name of the clock object (possibly associated with a port) The time it takes a clock signal to propagate from the clock definition point to a register clock pin for a rising transition at the clock definition point.
<u>ClockRiseLatency</u>	Setup uncertainty applied to all endpoints of the given clock.
<u>ClockSetupUncertainty</u>	The time it takes a clock signal to propagate from its ideal waveform origin point to the clock definition point for a falling transition at the waveform origin point.
<u>ClockSourceFallLatency</u>	The time it takes a clock signal to propagate from its ideal waveform origin point to the clock definition point for a rising transition at the waveform origin point.
<u>ClockSourceRiseLatency</u>	clone a component in the subsystem.
<u>clone component</u>	Close an open channel
<u>close</u>	Close the current workspace.
<u>close workspace</u>	Connection class(es) for this connection.
<u>CnctClass</u>	Number of item names to print when creating a collection.
<u>collection print item limit</u>	Inherit attribute values from other items and combine values to form a single result
<u>CombineInheritValues</u>	

coreTools Command Reference Index

<u>combineValues</u>	Combine the values of the specified attribute on the specified items according to the specified command
<u>compare dc version</u>	Compare a version string with the version of Design Compiler currently in use
<u>CompleteConnections</u>	Connect and disconnect component pins and (new) subsystem ports manually until the subsystem design is complete.
<u>complete custom activity definition</u>	Complete a custom activity definition
<u>complete interface definition</u>	Complete an interface definition, and mark the definition as unchangable
<u>ComponentOfItem</u>	The component name of an item
<u>compress sdc</u>	Inputs an SDC file and writes out a bus-compressed version of the input file
<u>concat</u>	Join lists together
<u>ConfigActivity</u>	The activity name that to configure this filegroup.
<u>ConfigDependsOnActivities</u>	List of activity names that should be completed (if they exist) before this filegroup is configured.
<u>ConfigDependsOnGroup</u>	List of filegroups that must be configured before this group.
<u>ConfigIntentSearchPath</u>	A list of directories to search for intent files
<u>Configurable</u>	Is this filegroup configurable?
<u>ConnectionDialogCmd</u>	Defines a custom command to be called for interactive connections.
<u>ConnectionRequired</u>	Used to indicate that this interface does not require a connection.
<u>ConnectToExportedInstance</u>	This instance need to connect to its exported instance
<u>connect interface</u>	Connect interfaces
<u>connect or export unconnected pins</u>	Connect or export unconnected pins in current context.
<u>ConstantPort</u>	Indicates that a port has a fixed logic value.
<u>ConstraintAnalysis</u>	Generate and (optionally) run ptc_shell synthesis constraint analysis scripts.
<u>ConstraintsVerification</u>	set up the rtl config file and generate the specified assertions
<u>ConstraintsVerificationWithAssertions</u>	Execute the simulation generated list of log files
<u>continue</u>	Skip to the next iteration of a loop
<u>ControlPointsPerScanCell</u>	

coreTools Command Reference Index

	Specifies the number of control points that can be shared per scan cell. When TestabilityMethod==control_and_observe for XG mode the larger of ObservePointsPerScanCell and ControlPointsPerScanCell will be used for the -test_points_per_scan_cell value for set_testability_configuration.
<u>ConvertSingleBitBus</u>	Convert single bit bus to bit type
<u>copy</u>	create copies of objects and classes Should this component be copied into the template? A value of false means that the template must be used with an install area for this component. A value of true means that the template does not require an install area for this component
<u>CopyToTemplate</u>	
<u>copy_configuration</u>	Copy configuration parameter values from source to target(s).
<u>copy_target_interface_parameters</u>	Copy parameter values from given interfaces to opposite target interfaces.
<u>coreAssembler</u>	Invokes the coreAssembler tool in either GUI or shell mode.
<u>coreBuilder</u>	Invokes the coreBuilder tool in either GUI or shell mode.
<u>coreBuilderBomTemplateVersion</u>	The version of the Bill-of-Materials template syntax used by a template.
<u>coreConsultant</u>	Invokes the coreConsultant tool in either GUI or shell mode.
<u>coretools_home_page</u>	Page to show as the tool home page.
<u>coroutine</u>	Create and produce values from coroutines Specify the set of files that are to be incorporated into the bill of materials. This is the definition of the set of all files that are part of the core design kit.
<u>CreateBillOfMaterials</u>	Set to true for designs that are created using create_hierarchical_component -fromComponent.
<u>CreatedViaUngrouping</u>	
<u>CreateIntegrationWorkspaces</u>	This activity packages up a core kit, installs the core kit, and then creates one or more integration (coreConsultant) work spaces from that

coreTools Command Reference Index

<u>CreateSkewGroup</u>	installation. These work spaces can be used to do validation of the core kit. Specifies the clock for which the skew group needs to be created. The skew group created will follow the naming convention SKEW_GROUP_QiK_CK_GEN_\ Refer to man page for 'create_clock_skew_group' in Fusion Compiler reference documentation.
<u>CreateSubsystemTemplate</u>	Create a subsystem template. The template is created as a single file and is located in the ./export directory of the current workspace.
<u>CreateTestbenchWorkspace</u>	Create testbench workspace for the current DUT.
<u>create address bank</u>	Create an address bank in a memory map or bank.
<u>create address block</u>	Create an address block in a memory map or bank.
<u>create address space</u>	Create a new address space or update an existing one.
<u>create autoload filegroup</u>	Create an autoloading filegroup
<u>create command group</u>	Creates a new command group.
<u>create component view</u>	Create an IP-XACT component view.
<u>create configuration parameter</u>	Create a parameter for a configurable filegroup
<u>create connection</u>	Create connections in between component pins and subsystem ports.
<u>create custom activity</u>	Create a custom activity
<u>create custom activity parameter</u>	Create a parameter for a custom activity
<u>create custom tech cell</u>	Specify binding requirements for mapping to a technology specific cell.
<u>create generated clock</u>	Create a generated clock for use within the specified design
<u>create hierarchical component</u>	Create a new hierarchical component in the subsystem
<u>create interface</u>	Create named interface definition
<u>create interface instance</u>	Create an instance of an interface to associate it on the core
<u>create interface parameter</u>	Create interface parameter on an interface definition
<u>create interface port</u>	Create interface port on an interface definition
<u>create memory map</u>	Create a memory map.
<u>create memory mode</u>	

coreTools Command Reference Index

<u>create_plugin_kb</u>	Create a memory mode for the current/selected design
<u>create_prime_profile_file</u>	Create a plug-in KB from the files in the given directory
<u>create_reference_clock</u>	Creates a Prime Profile file to be packaged with the coreKit or shipped externally.
<u>create_register</u>	Create a reference clock for use within the specified design
<u>create_register_array</u>	Create register in an address block.
<u>create_register_field</u>	Create a register array in a memory block.
<u>create_register_field_value</u>	Create a register field in a register.
<u>create_subsystem_parameter</u>	Create a value for a register field.
<u>create_virtual_clock</u>	Create a parameter on the subsystem
<u>create_workspace</u>	Create a virtual clock for use within the specified design
<u>CriticalRange</u>	Create a new workspace (working area).
<u>CriticalRangeCoveringViolators</u>	Default critical range for this design.
<u>CriticalTiming</u>	Set the critical_range value to cover all violating paths in the design.
<u>crm_file_patterns</u>	Indicates whether the port will likely be part of a critical timing path.
	Identifies files that are part of your source code control system.
<u>CTLModels</u>	CTLModels design attribute lists the ctl models to be read in by the synthesis tool when bottom-up synthesis is enabled. It is applicable only for hierarchical designs which have SynthesizeAsSubblock set to 1. CTL files for hardened blocks which are already synthesized may be specified at this level. For example a Subsystem has an IP (A), IP (A) is to be synthesized as a sub-block with DFT models cell1.ctl and cell2.ctl. So one can set the design attributes to include the test models during synthesis.
<u>current_activity</u>	Return current activity
<u>current_design</u>	Set or get the current design
<u>current_kb</u>	Set or get the current knowledge database (KB)
<u>CustomizedTestbenchCode</u>	Customized testbench code to be inserted
<u>CycleTime</u>	Cycle time for a clock.

D

<u>date</u>	Returns a string containing the current date and time.
<u>DCLSPaths</u>	Specify the paths as a list-of-list. Generates scripts that do a 'quick map' of this core to the target technology. This is the fastest possible mapping to gates, and should be used as an estimation path only. You can also select the Stop at GTECH parameter to elaborate your configured core to the Synopsys technology-independent GTECH library, without actually compiling the core to your target technology. Generating a GTECH version of the core is an easy way to create a simulation model for a core that is delivered as encrypted HDL.
<u>DCTCL quick_map_strategy</u>	
<u>dc_shellStopMessageIds</u>	Messages IDs which should cause Design Compiler to terminate.
<u>dc_shellVariable</u>	Variable settings to be used for dc_shell. Example: set_design_attribute {dc_shellVariable[varName]} value.
<u>dc_shellVariableComment</u>	Comment to be issued for the corresponding subscript of dc_shellVariable.
<u>dde</u>	Execute a Dynamic Data Exchange command
<u>debug_script</u>	Debug a script using the TclDevKit Tcl debugger.
<u>DedicatedScanPorts</u>	Determines whether dedicated scan out ports will be created, or whether the functional ports will be used as scan ports.
<u>DedicatedWrapperCell</u>	Specifies the default dedicated wrapper cell type.
<u>DefaultConstantPort</u>	Specifies a default value to be connected to a design input port when the interface connection only links to a design port on one side

coreTools Command Reference Index

<u>DefaultHdlValue</u>	of connected interfaces. Store the Default HdlValue based on the unconfigured core
<u>DefaultInstallDir</u>	Default directory into which to install a file or file group
<u>DefaultLoadPath</u>	A list glob-style patterns specifying how to load this deliverable. Relative paths start with the root of the current workspace.
<u>DefaultParamRange</u>	Store the Parameter Range based on the unconfigured core
<u>DefaultValue</u>	Default value for a parameter
<u>DefaultValueMessage</u>	Message to display along with the default value
<u>DeferEvaluation</u>	If true, port will be present in elaborated design irrespective of generateIf condition. Its conditionality will be preserved in generated toplevel netlist on subsystem level.
<u>define</u>	define and configure classes and objects
<u>DefineComponentViews</u>	Add or delete IPXACT views for the component.
<u>DefineLibClocks</u>	Defines hard macro libraries containing clocks to be defined for constraints.
<u>DefineMemoryMaps</u>	This activity is used to create and view memory maps.
<u>define activity subproc params</u>	Define sub-process launch/result-viewing parameters for an activity or filegroup
<u>define array field parameters</u>	define parameters for each field of an array.
<u>define lib clocks</u>	Extract clock definitions from a hard macro library file
<u>define proc attributes</u>	Defines attributes of a Tcl procedure, including an information string for help, a command group, a set of argument descriptions for help, and so on. The command returns the empty string.
<u>define report</u>	Support creation and viewing of user defined reports
<u>define solution</u>	Define a DesignWare IP Solution

coreTools Command Reference Index

<u>define_split_interface</u>	Group the interfaces belonging to two different components.
<u>define_tech_cell_mapping</u>	Define mapping requirements to a technology specific cell.
<u>DeliverableType</u>	The type of information stored in this deliverable.
<u>Description</u>	Brief description of the item
<u>design</u>	Design item
<u>DesignConfigurationIntent</u>	Review/specify intent attributes associated with the HDL parameters for the core design.
<u>Design Compiler Reference Methodology</u>	Performs a one pass, top down compile from RTL using compile_ultra. This flow implements the Design Compiler Reference Methodology associated with the version of Design Compiler being used.
<u>detach_interface</u>	Remove an attached interface instance from the given component.
<u>DftExistingSignalActiveState</u>	Active logic state for the signal.
<u>DftExistingSignalTestMode</u>	The name of the test mode the dft signal specification applies to.
<u>DftExistingSignalTiming</u>	Waveform to be used for the test signal.
<u>DftExistingSignalType</u>	Test signal to be used for the 'existing_dft' view of set_dft_signal. These are signals that describe structures which already exist that must be understood for the design to pass DRC (dft_drc): clocks, resets, constants, existing scan chains, etc.
<u>DftExistingSignalUsage</u>	The usage scenario(s) the dft signal specification applies to.
<u>DftSpecSignalActiveState</u>	Active logic state for the signal.
<u>DftSpecSignalTestMode</u>	The name of the test mode the dft signal specification applies to.
<u>DftSpecSignalType</u>	Test signal to be used for the 'spec' view of set_dft_signal. These are signals that describe structures that DFT Compiler should use for insert_dft: scan in, scan out, scan enable, etc.
<u>DftSpecSignalUsage</u>	The usage scenario(s) the dft signal specification applies to.

coreTools Command Reference Index

<u>dict</u>	Manipulate dictionaries
<u>DigitsPrecision</u>	Number of digits of precision to interpret and display
<u>DisableAggregation</u>	Set on a pin/port to disable aggregate connections to it in the netlist.
<u>disable_prime_profile</u>	Disable Prime Profile related commands in builder, consultant and assembler (Synopsys only).
<u>DocAddressOffset</u>	Text relating to the AddressOffset attribute, for inclusion in documentation
<u>DocBlockTableAddressOffset</u>	Text relating to the BlockTableAddressOffset attribute, for inclusion in documentation
<u>docbook to html</u>	Convert the given DocBook XML file to HTML format.
<u>DocDefaultValue</u>	Documentation oriented default value for a parameter.
<u>DocDescription</u>	Documentation oriented version of the description of the given item. The text may include DocBook tags.
<u>DocDescriptionField</u>	Defines extra tagged information to be included in the table cell describing the given item. Each value is included, tagged with its corresponding subscript. The text may include DocBook tags.
<u>DocEnabled</u>	Documentation oriented description of when the given item is enabled. The text may include DocBook tags. Must be set for each subscript for which Enabled is set.
<u>DocGenerateIf</u>	Documentation oriented description of when this port exists. The text may include DocBook tags.
<u>DocGroup</u>	Indicates that this object belongs to a group of objects that will be documented as a group.
<u>DocGroupName</u>	Grouping value used to group objects for documentation generation.
<u>DocInclude</u>	Used to conditionally include an item in generated documentation.
<u>DocLabelName</u>	Report oriented label and name combo for a parameter.
<u>DocMaster</u>	

	Indicates that this object is the 'master' element among the group of elements with the same value for DocGroup, and is the element from which the documentation should be generated. The value of this attribute indicates the name of the looping variable and the min and max loop indices for the group (for documentation purposes). It can contain three entries "var min max" or five entries, with the two added entries being the post-elaboration min/max values. The latter two values can be parameter expressions if needed. Note a 4th/6th argument was added that allows a list of attribute/value pairs which can override the existing attribute value [list Name "UseMyName" GroupName "UseMyGroupName"].
<u>DocMemoryAccess</u>	Text relating to the MemoryAccess attribute, for inclusion in documentation
<u>DocOffset</u>	Replaces register table pretext overriding GlobalAddressOffset value normally displayed.
<u>DocPortWidth</u>	Documentation oriented description of the width of the port.
<u>DocPowerDomain</u>	Documentation oriented description of the power domain associated with a port.
<u>DocRangeDecoratedName</u>	Report oriented value for the RangeDecoratedName of a port.
<u>DocRegistered</u>	Documentation oriented description of whether or not a port is registered.
<u>DocRegisteredBy</u>	Documentation oriented description of whether or not a port is registered.
<u>DocRegisterResetMask</u>	Documentation oriented description of the RegisterResetMask attribute. The text may include DocBook tags.
<u>DocRegisterResetValue</u>	Documentation oriented description of the reset value for this register or field. The text may

<u>DocRegisterSize</u>	include DocBook tags. Documentation oriented description of the size of this register or field. The text may include DocBook tags.
<u>DocRegTableAddressOffset</u>	Text relating to the RegTableAddressOffset attribute, for inclusion in documentation
<u>DocShortDescription</u>	Shortened version of the regular DocDescription.
<u>DocSynchronousTo</u>	Documentation oriented description of the clocks to which the port is synchronized.
<u>DocTestable</u>	Documentation oriented description of the Testable attribute. The text may include DocBook tags.
<u>DocVisible</u>	Documentation oriented description of when this register or field is visible. The text may include DocBook tags.
<u>DocVolatileMemory</u>	Documentation oriented description of the VolatileMemory attribute. The text may include DocBook tags.
<u>DocWaiver</u>	Used to document waivers for use of Doc* attributes. Subscript indicates attribute the waiver applies to.
<u>DontTouchNetwork</u>	Set dont_touch_network on this port.
<u>Drive</u>	Drive resistance for input or inout ports.
<u>DrivingCell</u>	Specifies how to generate a set_driving_cell constraint for an input port.
<u>duplicate_component</u>	Duplicate a component in the subsystem
<u>duplicate_workspace</u>	Save the current workspace as duplicate.

E

<u>echo</u>	Echos arguments to standard output.
<u>Enabled</u>	Enable or disable this parameter in the GUI
<u>EnableDftAutoFix</u>	Enables the DFT Compiler AutoFix utility.
<u>EnableDftShadowLogic</u>	

coreTools Command Reference Index

<u>EnableScanCompression</u>	Enables the DFT Compiler Shadow LogicDFT utility for this design. When true enables scan compression. For details on DFTMAX compression options, please refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.
<u>enable_prime_profile</u>	Enable Prime Profile related commands in builder, consultant and assembler (Synopsys only).
<u>encoding</u>	Manipulate encodings
<u>EncryptText</u>	Encrypt text as it is being written.
<u>encrypt_solution</u>	Encrypt TCL files associated with a given DW IP Solution directory.
<u>EndBit</u>	Tag specification ending position. Indicates the original EndBit expression (subscripted representation) which was read in from RTL file in LoadDesigns activity
<u>EndBitExpr</u>	Indicates the original EndBits complete expression which was read in from RTL file in LoadDesigns activity
<u>EndBitsExpr</u>	Are the elements in this address block or interface big or little endian
<u>Endian</u>	an enumeration of the legal attribute values
<u>EnumValues</u>	EnvCheck is a subscripted attribute that may be attached to a coreKit. The user may call verify_environment from coreConsultant to verify the environment based on the CheckEnv attribute.
<u>EnvCheck</u>	Check for end of file condition on channel
<u>eof</u>	Generate an error
<u>error</u>	Prints extended information on errors from the last command.
<u>error_info</u>	Converts escaped name to internally compatible mangled name
<u>escaped_name</u>	Evaluate a Tcl script
<u>eval</u>	evaluates the code in component context
<u>eval_in_component</u>	evaluate a SV expression from ipxact file.
<u>eval_ipxact_expr</u>	Evaluate the given parameter expression.
<u>eval_param</u>	Will cause a set_dont_use \ true to be placed on the library cells specified.
<u>ExcludeLibCells</u>	A list of glob-style patterns to match file names against. Files that match an
<u>ExcludeLoadPatterns</u>	

coreTools Command Reference Index

	entry in this list will not be added to the coreKit.
<u>exec</u>	Invoke subprocesses
<u>Exists</u>	Does this deliverable exist?
<u>exit</u>	End the application
<u>explicit capacitance</u>	Formula to specify an explicit capacitance value for PinLoadType
	Generate and (optionally) run RTL Architect scripts. This activity is used to generate RTLA Reference Methodology scripts. By default, the RTLArchitect run is started immediately after the scripts are generated, but this behavior can be changed by checking the 'Generate Scripts Only' check-box. Once the RTLA Activity is finished, results will be available in the following locations: Logs - rtl/run_'flow type'/logs Reports - rtl/run_'flow type'/reports Outputs - rtl/run_'flow type'/outputs If you choose to run the rtl yourself, or wish to review the generated scripts or constraints, these can be found in the following locations: Flat Makefile: rm_setup/Makefile_rtl_flat Explore Makefile: rm_setup/Makefile_rtl_explore
<u>ExploreRTLArchitecture</u>	Indicates a 'define whose value should be written to the indicated file as part of the configuration process.
<u>ExportDefineToFile</u>	Export all interfaces in the current context that are set up for auto-export.
<u>export all interfaces</u>	Export a component interface out of the subsystem.
<u>export interface</u>	Export pins currently connected to open
<u>export open pins</u>	Export the pin to a port on the subsystem.
<u>export pin</u>	Export port description attributes up one level of hierarchy.
<u>export port descriptions</u>	Export unconnected interfaces in the current hierarchy.
<u>export unconnected interfaces</u>	Export unconnected pins in the current hierarchy.
<u>export unconnected pins</u>	Export the workspace as a component.
<u>export workspace as component</u>	Evaluate an expression
<u>expr</u>	Determines the disabling option during scan shift for all tristate nets that drive output ports of a design.
<u>ExternalTristates</u>	disable_all disables all drivers.

coreTools Command Reference Index

	enable_one disables all but one driver. no_disabling specifies not to insert disabling logic.
<u>ExtraSearchPathAppend</u>	List of additional appended search_path entries for Design Compiler.
<u>ExtraSearchPathPrepend</u>	List of additional prepended search_path entries for Design Compiler.
E	
<u>fblocked</u>	Test whether the last input operation exhausted all available input
<u>FcAuxSynthesisScript</u>	Specifies a FC script to be executed prior to specific optimization phase.
<u>FcAuxSynthesisScriptComment</u>	Specifies a comment to be issued for the corresponding FcAuxSynthesisScript.
<u>FCExcludeLibCells</u>	The attribute specifies the purposes for the library cells. The purpose is passed into the set_lib_cell_purpose command.
<u>FCIncludeLibCells</u>	The attribute specifies the purposes for the library cells. The purpose is passed into the set_lib_cell_purpose command.
<u>fconfigure</u>	Set and get options on a channel
<u>fcopy</u>	Copy data from one channel to another
<u>fc_shellAppOption</u>	Sets application options to the specified value on the specified block. Settings to be used for Fusion Compiler (fc_shell).
<u>fc_shellAppOptionComment</u>	Add a comment for the application options being introduced in the tool intent.
<u>FeedThroughConnect</u>	This input port feeds-through to the output ports listed. Used in automatic connections to prevent connections to these pins. Connections will bypass the component and connect items that would be connected to both pins directly.
<u>file</u>	File item

coreTools Command Reference Index

<u>FileContentType</u>	Content type of a file or file group
<u>fileevent</u>	Execute a script when a channel becomes readable or writable
<u>filegroup</u>	File group item Represents a collection of deliverables (filegroups) defined via a Group list in a BoM template.
<u>filegroupGroup</u>	File name conventions supported by Tcl commands
<u>filename</u>	Read/write/execute permissions for a file or file group
<u>FilePerms</u>	Locate the specified design or specified instance of the design and create a collection that contains that design
<u>find design</u>	Locate the specified interface instances.
<u>find interface instances</u>	Create a collection of items that match the specified name pattern
<u>find item</u>	Find KB with the given name and type.
<u>find kb</u>	Fix hold time violations for this clock.
<u>FixHold</u>	Flush buffered output for a channel
<u>flush</u>	Run formal verification on a design.
<u>FM formal verification strategy</u>	Variable settings to be used for fm_shell. Example: set_design_attribute {fm_shellVariable[varName]} value.
<u>fm_shellVariable</u>	Comment to be issued for the corresponding subscript of fm_shellVariable.
<u>fm_shellVariableComment</u>	'For' loop
<u>for</u>	If GenericAttr[ComponentUndefFile] is set on the design, use that while writing out technology cell macro undef directives. The file option is used otherwise.
<u>ForceTCBToUseComponentUndefFile</u>	Iterate over all elements in one or more lists
<u>foreach</u>	loop through the parameters for the fields of an array.
<u>foreach array field parameter</u>	

coreTools Command Reference Index

<u>foreach in collection</u>	Iterate over a collection
<u>FormalVerification</u>	Generate and (optionally) run formal verification scripts. Specifies a script that is to be run in Formality after the reference and implementation designs have been loaded but just before the verify command.
<u>FormalVerificationAuxScript</u>	Specifies a comment to be issued for the corresponding FormalVerificationAuxScript.
<u>FormalVerificationAuxScriptComment</u>	Format a string in the style of sprintf
<u>format</u>	Arguments to be passed to the set_fpga_pad_type command for FPGA synthesis.
<u>FpgaPadType</u>	Specifies that this port is a primary I/O for FPGA synthesis.
<u>FpgaPortIsPad</u>	Specifies which synthetic library FPGA synthesis will use for synthetic operators. See the man page for fpga_prefer_tmg in fpga_shell.
<u>FpgaPreferTmg</u>	Generate and (optionally) run FPGA synthesis scripts. This activity is used to generate FPGA synthesis scripts for SynplifyPro or SynplifyPremier, customized for the particular core configuration, Synplify version and FPGA device. By default, the Synplify run is started immediately after the scripts are generated, but this behavior can be changed by checking the 'Generate Scripts Only' check-box. Once the synthesis is finished, results will be available in the following locations: EDIF netlist - syn/final/db/<top_design>.edif HDL netlist - syn/final/db/<top_design>.v[hdl] Reports - syn/final/report If you choose to run the synthesis yourself, or wish to review the generated scripts or constraints, these can be found in the following locations; Launch script - syn/run.scr Synplify script -
<u>FPGASynthesis</u>	

FPGA optimization strategy

FunctionDefinition

Fusion Compiler Reference Methodology

syn/synplify.tcl Constraints -
syn/constrain/script/*.cscr
Performs synthesis from RTL
targeting an FPGA using synplify.
The Tcl code to implement this
function
Performs a one pass, top down
compile from RTL using
compile_fusion. This flow
implements the Fusion Compiler
Reference Methodology associated
with the version of Fusion
Compiler being used.

G

GenerateIf

GenerateSubsystemRTL

GenerateTestVectors

generate_gtech_sim_model

generate_qtm

generate_reports

generate_simulation_file_list

generate_simulation_launch_command

generate_views

getenv

gets

get_activity_parameter

get_address_bank_attribute

get_address_block_attribute

get_all_bits

This cell or port is conditionally
generated
Generate the HDL source code for
top-level design of the subsystem.
Generate test vectors using
TestMAX.
Write scripts and return shell
command to generate a GTECH model
of the core
Generate a QTM (quick timing
model) for the top design.
Generate the indicated reports.
Generate a list of simulation
files
Generate the command to launch the
simulation
Generate the indicated optional
views.
Returns the value of a system
environment variable.
Read a line from a channel
Get the value of parameter for an
activity
Get the value of an attribute from
an address bank.
Get the value on an attribute from
an address block.
Return all bits of the specified
port/pin, or port/pin itself if it
is a single bit item.

coreTools Command Reference Index

<u>get_app_var</u>	Gets the value of an application variable.
<u>get_associated_instance_parameter</u>	Get parameter value of linked instantiation parameter.
<u>get_attribute</u>	Get the current value(s) of the specified attribute(s)
<u>get_bitfield_value</u>	Get value of bit range with a given parameter or numeric value.
<u>get_bit_driver</u>	Get the driver for this connection.
<u>get_bit_loads</u>	Get the bit loads for this source connection.
<u>get_cell</u>	Command to get the cell from a cell name or path.
<u>get_cell_attribute</u>	Get the value of an attribute on a design instance
<u>get_children</u>	Get the child/children of the given item
<u>get_clocks</u>	Create a collection of all clocks associated with the specified design
<u>get_clock_attribute</u>	Get the value of an attribute on a clock
<u>get_command_hooks</u>	Get registered hooks for this command
<u>get_command_option_values</u>	Queries current or default option values.
<u>get_component_design</u>	Command to get the elaborated or unelaborated design for a component.
<u>get_component_name</u>	Get the component name of the given item.
<u>get_component_view</u>	Get the view associated with the current/selected component.
<u>get_configuration_parameter</u>	Get configuration (design) parameter value
<u>get_configuration_parameter_attribute</u>	Get the value of an attribute from the specified configuration parameter
<u>get_connections</u>	get connections for the specified single bit port/pin/busbit
<u>get_current_component</u>	Returns the current component.
<u>get_current_hook_command</u>	Get the currently running command string
<u>get_defined_commands</u>	Get information on defined commands and groups.

coreTools Command Reference Index

<u>get design attribute</u>	Get the value of an attribute on the specified design
<u>get design prefix</u>	Returns the prefix to be associated with each design name.
<u>get field parameter for array</u>	returning the field parameter for a given index of an array parameter.
<u>get filegroup attribute</u>	Get an attribute from a filegroup
<u>get filegroup parameter</u>	Get a parameter for a filegroup
<u>get file prefix</u>	Returns the file prefix to be associated with each design name.
<u>get generator parameter</u>	Get component generator parameter value
<u>get hdl file list</u>	Return the list of HDL files that DC must analyze (in order of analysis)
<u>get hdl language</u>	Get the HDL languages used
<u>get hdl library names</u>	Return the list of HDL libraries used by the currently loaded core
<u>get hdl pragma</u>	Get reuse-pragma statement external to the HDL
<u>get hierarchies</u>	Get information about the subsystem hierarchies.
<u>get include dirs</u>	return all include dirs for master leaf components in current or provided context
<u>get installation dir</u>	Get full installation directory name for the current core
<u>get installed component names</u>	Return installed component names and versions (for array set).
<u>get instance parameter value</u>	Command to get the value of instance parameter in terms of the subsystem parameter.
<u>get interface attribute</u>	Get the value of an attribute from the specified interface
<u>get interface link</u>	Get evaluated InterfaceLink for the given interface port/param.
<u>get interface parameter</u>	Get a parameter for an interface
<u>get interface parameter attribute</u>	Get the value of an attribute from the specified interface parameter
<u>get interface port attribute</u>	Get the value of an attribute from the specified interface port
<u>get launch activities status</u>	Get status for all the activities or few activities launched with launch_activities_on_grid
<u>get logical dir</u>	

coreTools Command Reference Index

<u>get_macro_prefix</u>	Get full logical directory name for the specified workspace directory
<u>get_memory_map_attribute</u>	Returns the macro prefix to be associated with each macro name.
<u>get_memory_mode</u>	Get the value on an attribute from a memory map.
<u>get_message_ids</u>	Get the current memory mode associated with the current/selected design
<u>get_message_info</u>	Get application message ids
<u>get_object_name</u>	Returns information about diagnostic messages.
<u>get_parameter_attribute</u>	Converts a collection into a list of item names
<u>get_port_attribute</u>	Get the value of an attribute on a parameter
<u>get_power_domain_voltage</u>	Get the value of an attribute on a port
<u>get_prime_profile_info</u>	Returns the voltage for the specified power domain.
<u>get_register_array_attribute</u>	Get Prime Profile info from the data model..
<u>get_register_attribute</u>	Get the value of an attribute from a register array.
<u>get_register_field_attribute</u>	Get the value of an attribute from a register.
<u>get_register_field_value_attribute</u>	Get the value of an attribute from a register field.
<u>get_slave_base_address</u>	Get the value of an attribute from a register field value.
<u>get_solution_data</u>	Get the remap range and/or base address of the given slave interface.
<u>get_solution_file</u>	Get all solution data or an individual field.
<u>get_solution_root</u>	Get IP solution file if present.
<u>get_solution_variable</u>	Get root context of current DesignWare IP Solution.
<u>get_strategy_parameter</u>	Get all solution variables or an individual variable.
<u>get_supply_voltage</u>	Get the value of a parameter for a strategy
	Command used to insert a voltage value into a set_voltage command in a power intent file.

coreTools Command Reference Index

<u>get sv expr</u>	Convert a tcl expression to equivalent sv expression
<u>get synthesis dir</u>	Return the full directory name for the specified synthesis subdirectory
<u>get tcl expr</u>	convert a SV expression to tcl.
<u>get template naming style</u>	Get the template_naming_style value.
<u>get template parameter style</u>	Get the template_parameter_style value.
<u>get template separator style</u>	Get the template_separator_style value.
<u>get TMR separation distance</u>	Retruns the TMR Separation Distance
<u>get tool preference</u>	Get the tool preference parameter
<u>get tool root</u>	Get the root directory for the specified tool.
<u>get top design</u>	Returns the top design.
<u>get top design name</u>	Return the name of the top level design
<u>get trace option</u>	Get the current option value controlling behavior of command tracing and output annotation..
<u>get unix variable</u>	This is a synonym for the \fBgetenv\fP command.
<u>get upf attribute</u>	Get the value of an attribute on a UPF item
<u>get upf port names</u>	Returns the names of the ports in in the specified power domain.
<u>get value from interface</u>	Get value for interface parameter from the specified interface on the same component.
<u>get VLNV search path entries</u>	Get elements of given type and VLNV from the cache.
<u>get workspace kb</u>	Returns the current top-level knowledge database.
<u>get workspace license info</u>	Get details about license and license type associated with workspace creation.
<u>get workspace name</u>	Returns the name of the current top-level knowledge database.
<u>glob</u>	Return names of files that match patterns
<u>global</u>	Access global variables
<u>GlobalSlotNumber</u>	Provides access to the global slot number stored in the SlotNumber

<u>GroupImage</u>	parameter on the given interface Image to use in pre-text for signal, parameter, and memory map groups.
<u>GroupImageAttrs</u>	Attributes to use for xml docbook:imagedata generated from GroupImage.
<u>GroupImageTitle</u>	Title to use for figures displayed with GroupText or GroupImage.
<u>GroupName</u>	Group similar items together in GUI displays
<u>GroupText</u>	Text that is displayed as pre-text for signal, parameter, and memory map groups.
<u>GroupUserLabel</u>	User label of the parameter group
<u>group_components</u>	Group the following components into a lower hierarchical level
<u>guiBehavior</u>	Array variable that specifies different aspects of the GUI behavior.
<u>gui_get setting</u>	Get a setting on the specified window
<u>gui_set setting</u>	Set a setting on the specified window
<u>gui_source</u>	Source script in the GUI without GUI updates.
<u>gui_start</u>	Start GUI
<u>gui_stop</u>	Stop GUI
 H	
<u>HasLibertyModel</u>	Indicates that this design has a Liberty model and therefore no HDL file analysis is required for synthesis.
<u>hdlFunc</u>	Internal model of a VHDL function
<u>hdlin_enable_upf_compatible_naming</u>	Tracks the value of the like-named dc_shell variable
<u>HdlLibrary</u>	HDL library into which to analyze this file or file group
<u>HdlSourceFileNames</u>	Name(s) of the file that defines this design or package, and any included files.
<u>HdlTypeName</u>	HDL type name of this parameter
<u>HdlUnquoted</u>	

coreTools Command Reference Index

	Indicates that when string param is written in HDL it should not be quoted.
<u>HdlValue</u>	How the value is formatted in the given language
<u>HeaderNameFormat</u>	Defines the name format for the header reference for the given attribute on this register or register field.
<u>help</u>	Displays quick help for one or more commands.
<u>HelpUrl</u>	URL of the documentation file for an item
<u>HierarchicalIsolation</u>	When true, hierarchical isolation logic is built, so that dedicated subdesign scan-out signals are gated by the design scan enable signal.
<u>HighFanout</u>	This input port has a high fanout.
<u>history</u>	Displays or modifies the commands recorded in the history list.
<u>http</u>	Client-side implementation of the HTTP/1.1 protocol
 L	
<u>IconPath</u>	Path to an icon for this item
<u>IdealPort</u>	This port is 'ideal'. It has infinite drive, no resistance, and directly attached nets are not checked for transition or capacitance violations. When true , insert_dft/compile -scan identifies shift-registers in the design and keep only the first flop as a scan flop. Scan Architecting in preview_dft/insert_dft will group the flops that form shift registers as a single scan group named shift_register_identified_??. preview_dft -show all will show the shift registers identified. preview_dft -script writes out the shift-registers as scan groups. If the scan style is not multiplexed_flip_flop, insert_dft/compile -scan ignores this option.
<u>IdentifyShiftRegisters</u>	
<u>if</u>	Execute scripts conditionally

coreTools Command Reference Index

<u>IfUnconnected</u>	Connection to make if left unconnected.
<u>Importance</u>	Indicates whether the deliverables for this filegroup are required or optional
<u>ImportedWorkspaceComponentFile</u>	Indicates the file used to import a workspace component.
<u>import_component</u>	Import an unpackaged component into the subsystem.
<u>import_ipxact_data</u>	Import partial IP-XACT component data.
<u>import_workspace_as_component</u>	Import the workspace as a component.
<u>IncludeIf</u>	Replace text if the parameter expression is true
<u>IncludeLibCells</u>	Will cause a set_dont_use \ false to be placed on the library cells specified.
<u>incr</u>	Increment the value of a variable
<u>index_collection</u>	Extract object from collection. Result is new collection
<u>infinite_drive</u>	Formula to indicate infinite drive for an input port
<u>info</u>	Return information about the state of the Tcl interpreter
<u>InheritValue</u>	Inherit an attribute value from an item up or down the design hierarchy or from a pin
<u>InitializeSubsystemConstraints</u>	Create system level clocks, and initialize system level constraints from component level constraints.
<u>InputDelay</u>	Delay constraint for an input port
<u>InputResistance</u>	Specifies the resistance on the net driven by an input port.
<u>InsertEndOfChainLockupLatch</u>	When true lockup latches will be inserted at the end of scan chains.
<u>InsertTechCellBindMacroAtStart</u>	Set to true to the file item if tech cell defn insertion should be done at top
<u>InsertTestPoints</u>	Controls whether observe and control points are inserted into this design to enhance testability.
<u>Install</u>	Should this group be installed
<u>InstallUserWorkDir</u>	How to install a file or file group into a user workspace
<u>InstallWhen</u>	When should file group be installed?
<u>install_filegroup</u>	

coreTools Command Reference Index

<u>InstanceClockNames</u>	Install the specified filegroup from the Bill of Materials
<u>instances of</u>	Names to use when creating clocks during InitializeSubsystemConstraints.
<u>instantiate component</u>	Get instances of given design name, recursively.
<u>instantiate dut</u>	Instantiate a component in the subsystem.
<u>instantiate interface</u>	Instantiate a DUT into the testbench subsystem
<u>instantiate solution</u>	Instantiate an exported interface without any attachment.
<u>IntentFilesProcessed</u>	Initiate creation of a DesignWare IP Solution
<u>InterClockHoldFallFallUncertainty</u>	Set to true after intent files or processed.
<u>InterClockHoldFallRiseUncertainty</u>	Uncertainty applied from falling edge of source clock to the falling edge of the destination clock for hold delay calculations
<u>InterClockHoldRiseFallUncertainty</u>	Uncertainty applied from falling edge of source clock to the rising edge of the destination clock for hold delay calculations
<u>InterClockHoldRiseRiseUncertainty</u>	Uncertainty applied from rising edge of source clock to the falling edge of the destination clock for hold delay calculations
<u>InterClockSetupFallFallUncertainty</u>	Uncertainty applied from rising edge of source clock to the rising edge of the destination clock for hold delay calculations
<u>InterClockSetupFallRiseUncertainty</u>	Uncertainty applied from falling edge of source clock to the falling edge of the destination clock for setup delay calculations
<u>InterClockSetupRiseFallUncertainty</u>	Uncertainty applied from falling edge of source clock to the rising edge of the destination clock for setup delay calculations
<u>InterClockSetupRiseRiseUncertainty</u>	Uncertainty applied from rising edge of source clock to the rising edge of the destination clock for setup delay calculations

coreTools Command Reference Index

	the destination clock for setup delay calculations
<u>interfaceDefn</u>	interfaceDefn item
<u>InterfaceGroup</u>	Used to indicate that an interface is part of a 'connection' group.
<u>interfaceInstance</u>	interfaceInstance item
<u>InterfaceIsUsed</u>	Indicates that the interface object is used on its parent interface instance.
<u>InterfaceLabel</u>	The label to be used in the display of interfaces/connections in the ad component tree dialog
<u>InterfaceLink</u>	Indicates the association to a design port or parameter (top-level core design). The design name can include \@\' and {\} in the attribute value, and allows a [\@\'] index annotation.
<u>interfacePort</u>	interfacePort item
<u>InterfaceSize</u>	Number of required bits on a design port associated with this interface port.
<u>InterfaceType</u>	Indicates the interface type for the given interface.
<u>InterfaceTypeName</u>	The customized name for each interface type.
<u>InternalTristates</u>	Determines the disabling option during scan shift for all tristate nets that do not drive output ports of a design. disable_all disables all drivers. enable_one disables all but one driver. no_disabling specifies not to insert disabling logic.
<u>interp</u>	Create and manipulate Tcl interpreters
<u>intfPin</u>	Represents an interface instance on a cell.
<u>intfPort</u>	Represents an interface instance on a design.
<u>invoke_generator</u>	invoke a component generator.
<u>invoke_ralgen</u>	Invoke ralgen from \$VCS_HOME or within cT image.
<u>in_autocomplete activity</u>	Is an activity currently being autocompleted?
<u>IPEnvList</u>	Allows additional IP specific env vars to be considered for running an activity. These will be overridden by UserEnvList activity parameter if

coreTools Command Reference Index

	same exist
<u>IsAddressable</u>	Is true if the interface bus definition is addressable.
<u>IsArray</u>	Indicates that the parameter is an array or not.
<u>IsBehavioral</u>	Indicates that the given port/net represents a behavioral connection.
	Concurrent Clock and Data (CCD) Optimization technology to enable users to meet frequency goals on high performance designs. When the attribute is set to true the clock is becomes part of the path groups for targeted concurrent clock and data (CCD) optimization. Refer to man page for 'ccd.targeted_ccd_path_groups' in Fusion Compiler reference documentation.
<u>IsCCDTarget</u>	
<u>IsComplete</u>	This activity is complete
<u>IsConnected</u>	Is the subsystem port, component pin, or interface connected?
	Indicates that the interface parameter value is used as control-only value. As a consequence the parameter value impacts interface setup but not the interface configuration.
<u>IsControlValue</u>	
	Indicates whether an interface can be left without the corresponding consumer/provider.
<u>IsDanglingIntfAllowed</u>	
<u>IsEnabled</u>	This activity is enabled
	The attribute is set to true when any subscript of ResetProperties is explicitly set.
<u>IsExternalReset</u>	
<u>IsFullyModeled</u>	Indicates that the design contents are fully modeled in coreTools.
<u>IsHexVal</u>	This value is in hexadecimal format
<u>IsTriState</u>	Is the port a tri-state?
<u>IsUpToDate</u>	Is this deliverable up-to-date when compared to disk?
<u>IsWorkspaceComponent</u>	Indicates the file used to import a workspace component.
<u>is_false</u>	Tests the value of a specified variable, and returns 1 if the value is 0 or the case-insensitive string \bfalse\bP; returns 0 if the value is 1 or the case-insensitive string

coreTools Command Reference Index

<u>is_prime_profile_enabled</u>	\fBtrue\fP. Returns 1 if Prime Profile is enabled in this session, 0 otherwise.
<u>is_true</u>	Tests the value of a specified variable, and returns 1 if the value is 1 or the case-insensitive string \fBtrue\fP; returns 0 if the value is 0 or the case-insensitive string \fBfalse\fP.

J

<u>join</u>	Create a string by joining together list elements
-------------	---

K

<u>KbVersion</u>	Knowledge base version identifier.
<u>knowledgeBase</u>	Knowledge database item

L

<u>Label</u>	Label to be used for this item in GUI dialogs
<u>lappend</u>	Append list elements onto a variable
<u>lassign</u>	Assign list elements to variables
<u>launch_activities_on_grid</u>	Launch the sub-processes associated with multiple activities on a grid from a user workspace in coreConsultant
<u>launch_activity_subproc</u>	Launch a sub-process for an activity or filegroup
<u>library</u>	standard library of Tcl procedures
<u>LicenseForOpen</u>	The license used for opening the knowledgebase.
<u>lindex</u>	Retrieve an element from a list
<u>linsert</u>	Insert elements into a list
<u>list</u>	Create a list
<u>list_kb</u>	List all currently loaded knowledge databases
<u>llength</u>	Count the number of elements in a list
<u>lmap</u>	Iterate over all elements in one or more lists and collect results
<u>lminus</u>	Removes one or more named elements from a list and returns a new list.
<u>load</u>	Load machine code and initialize new commands
<u>LoadConsultantPlugins</u>	

coreTools Command Reference Index

	This activity is used to create plugin KBs that are used by the core integrator of this IP. These KBs are added to the Plugins file group for installation in the consultant workspace. As a side-effect, all non-object Tcl files are sourced. This activity is used to get design data into coreBuilder. Currently the tool supports input from either VHDL, Verilog or SystemVerilog Hdl. The HDL files may contain special comment based annotations. After the HDL is analyzed, then Tcl (and SDC) based intent files are sourced.
<u>LoadDesigns</u>	
<u>LoadInterfaceDefinitions</u>	Load definitions for each interface to be instantiated in the top-level of this core for automated assembly within coreAssembler.
<u>load autoload filegroup</u>	Populate an autoloading filegroup
<u>load component memory maps</u>	Loads all visible component memory maps from knowledgebase files in cC and cA into memory.
<u>load file into kb</u>	Load a file or directory into the current knowledge database
<u>load interface definitions</u>	Load interface definitions.
<u>load plugin</u>	Load a plugin KB and install non-configurable files.
<u>load solution definition</u>	Load a DesignWare IP Solution definition file.
<u>LockedInTemplate</u>	Indicate if the parameter/cell/interfaceinstance should be locked in template
<u>lock parameter</u>	Lock the configuration (design) parameter value so it cannot be changed.
<u>LogicalLeft</u>	Indicates the logical left bit index of the interface port to be connected.
<u>LogicalName</u>	Indicates the logical (abstract) name for this port/pin. Used to enable connections with matching LogicalName values.
<u>LogicalRight</u>	Indicates the logical right bit index of the interface port to be connected.
<u>log trace</u>	Control creation of a replay log of traced commands.
<u>lrange</u>	Return one or more adjacent elements from a list
<u>lrepeat</u>	Build a list by repeating elements
<u>lreplace</u>	Replace elements in a list with new elements
<u>lreverse</u>	Reverse the order of a list
<u>ls</u>	Lists the contents of a directory.

coreTools Command Reference Index

<u>lsearch</u>	See if a list contains a particular element
<u>lset</u>	Change an element in a list
<u>lsort</u>	Sort the elements of a list

M

<u>MacroPrefixFileList</u>	User specified list of files which need to be processed for macro prefixing. The paths should correspond to the installation path in the workspace.
<u>man</u>	Displays reference manual pages.
<u>MapBlockIndividually</u>	Map this design individually in Design Compiler. Note that this attribute does not control mapping for PSYN strategies.
<u>MapSubblocksIndividually</u>	Map this design by individually compiling its subdesigns.
<u>mathfunc</u>	Mathematical functions for Tcl expressions
<u>mathop</u>	Mathematical operators as Tcl commands
<u>MaxArea</u>	Maximum area constraint for current design.
<u>MaxCap</u>	Maximum capacitance value for a port or design.
<u>MaxControlPoints</u>	Maximum number of control points to be inserted to enhance testability. When TestabilityMethod==control_and_observe for XG mode the larger of MaxObservePoints and MaxControlPoints will be used for the -max_test_points value for set_testability_configuration.
<u>MaxFallInputDelay</u>	Maximum falling edge delay constraint for an input port
<u>MaxFallInputDelayFallingEdge</u>	Maximum falling edge delay constraint for an input port relative to a falling clock edge
<u>MaxFallOutputDelay</u>	Maximum falling edge delay constraint for an output port
<u>MaxFallOutputDelayFallingEdge</u>	Maximum falling edge delay constraint for an output port relative to a falling clock edge
<u>MaxFallTransitionDelay</u>	Maximum transition fall time for an ideal clock.
<u>MaxFanout</u>	Maximum fanout for an input port or a design.
<u>MaximumScanChainLength</u>	The maximum length a scan chain may be. Setting the value to 0 will allow DFT Compiler to determine the maximum scan

coreTools Command Reference Index

	chain length.
<u>maximum bit blast size</u>	Largest size bus that can be bit blasted.
<u>MaxInputDelay</u>	Maximum delay constraint for an input port
<u>MaxInputDelayFallingEdge</u>	Maximum delay constraint for an input port (relative to the falling clock edge)
<u>MaxLoads</u>	Maximum number of loads for this connection.
	Maximum number of observe points to be inserted to enhance testability. When TestabilityMethod==control_and_observe for XG mode the larger of MaxObservePoints and MaxControlPoints will be used for the -max_test_points value for set_testability_configuration.
<u>MaxObservePoints</u>	
<u>MaxOutputDelay</u>	Maximum delay constraint for an output port
<u>MaxOutputDelayFallingEdge</u>	Maximum delay constraint for an output port (relative to the falling clock edge)
<u>MaxRiseInputDelay</u>	Maximum rising edge delay constraint for an input port
<u>MaxRiseInputDelayFallingEdge</u>	Maximum rising edge delay constraint for an input port relative to a falling clock edge
<u>MaxRiseOutputDelay</u>	Maximum rising edge delay constraint for an output port
<u>MaxRiseOutputDelayFallingEdge</u>	Maximum rising edge delay constraint for an output port relative to a falling clock edge
<u>MaxRiseTransitionDelay</u>	Maximum transition rise time for an ideal clock.
<u>MaxTransition</u>	Maximum transition time for an input port or design.
<u>MaxValue</u>	Maximum value allowed
<u>MaxWriteConstraint</u>	Defines a maximum value constraint to be met when writing the given register field.
<u>memMap</u>	Describes a memory map in this component.
<u>memory</u>	Control Tcl memory debugging capabilities
<u>MemoryAccess</u>	Access type for this memory item
<u>MemoryAccessDescription</u>	Access type for this memory item
<u>MemoryMap</u>	Holds the memory map that this slave interface refers to.
<u>MemoryMapSpecification</u>	This activity is used to configure memory map.
<u>MemoryMode</u>	Set all the valid Memory Modes active for the design
<u>MemoryRange</u>	

coreTools Command Reference Index

	Specifies the address range of the registerArray, addressBlock or addressBank
<u>MemoryUsage</u>	Specifies the usage of this addressBlock or addressBank.
<u>MemoryWidth</u>	Specifies the bit width of an address block or bank
<u>MergeCandidate</u>	Indicates that a pin/port is a candidate for port merging via merge_ports.
<u>merge_ports</u>	Merge ports in current hierarchy, which are exporting adjacent bits of same pin.
<u>MessageLimitDC</u>	For each subscript, injects 'set_message_info -id \ -limit \' in the synthesis RM scripts.
<u>MessageLimitFC</u>	For each subscript, injects 'set_message_info -id \ -limit \' in the synthesis RM scripts.
<u>MessageStopOnDC</u>	If true, for each subscript, injects 'set_message_info -id \ -stop_on' in the synthesis RM scripts.
<u>MessageStopOnFC</u>	If true, for each subscript, injects 'set_message_info -id \ -stop_on' in the synthesis RM scripts.
<u>MinCap</u>	Minimum capacitance value for a port or a design.
<u>MinFallInputDelay</u>	Minimum falling edge delay constraint for an input port
<u>MinFallInputDelayFallingEdge</u>	Minimum falling edge delay constraint for an input port relative to a falling clock edge
<u>MinFallOutputDelay</u>	Minimum falling edge delay constraint for an output port
<u>MinFallOutputDelayFallingEdge</u>	Minimum falling edge delay constraint for an output port relative to a falling clock edge
<u>MinFallTransitionDelay</u>	Minimum transition fall time for an ideal clock.
<u>MinInputDelay</u>	Minimum delay constraint for an input port
<u>MinInputDelayFallingEdge</u>	Minimum delay constraint for an input port (relative to the falling clock edge)
<u>MinOutputDelay</u>	Minimum delay constraint for an output port
<u>MinOutputDelayFallingEdge</u>	Minimum delay constraint for an output port (relative to the falling clock edge)
<u>MinRiseInputDelay</u>	Minimum rising edge delay constraint for an input port
<u>MinRiseInputDelayFallingEdge</u>	

coreTools Command Reference Index

	Minimum rising edge delay constraint for an input port relative to a falling clock edge
<u>MinRiseOutputDelay</u>	Minimum rising edge delay constraint for an output port
<u>MinRiseOutputDelayFallingEdge</u>	Minimum rising edge delay constraint for an output port relative to a falling clock edge
<u>MinRiseTransitionDelay</u>	Minimum transition rise time for an ideal clock.
<u>MinValue</u>	Minimum value allowed
<u>MinWriteConstraint</u>	Defines a minimum value constraint to be met when writing the given register field.
<u>MonitoredComponent</u>	Defines a component as a monitor and indicates which type of component can be monitored. Value should be a component VLNV.
<u>MonitoredInterface</u>	Defines a component as application side VIP, identifying the type of interface to be connected to (SPIRIT VLN), the SPIRIT interface type, and the name of the VIP interface to connect to.
<u>move_components</u>	Move one or more components into a different hierarchical level.
<u>mpexpr</u>	Evaluate an expression with multiple precision math
<u>mpformat</u>	Evaluate an expression with multiple precision math
<u>msgcat</u>	Tcl message catalog
<u>msg_box</u>	Displays a modal message box
<u>my</u>	invoke any method of current object
 N	
<u>Name</u>	The name of the item
<u>namespace</u>	create and manipulate contexts for commands and variables
<u>net</u>	Net item
<u>next</u>	invoke superclass method implementations
<u>NumberOfScanChains</u>	The number of scan chains to be created. This value is passed to the set_scan_configuration command with the -chain_count option. For Internal_scan, this is also the number of scan IO ports created. The default value is the number of test clocks in the design.

coreTools Command Reference Index

<u>NumberOfScanCompressionChains</u>	In ScanCompression_mode, this parameter sets the number of scan chains to be created internally. Setting it to 0 will let coreTools decide the number of scan chains in the design based on the number of ScanCompressionIOCount strategy parameter.
<u>Q</u>	
<u>object</u>	root class of the class hierarchy
<u>ObservePointsPerScanCell</u>	Specifies the number of observe points that can be shared per scan cell. When TestabilityMethod==control_and_observe for XG mode the larger of ObservePointsPerScanCell and ControlPointsPerScanCell will be used for the -test_points_per_scan_cell value for set_testability_configuration.
<u>OneRequiredGroup</u>	Specifies that either this deliverable (filegroup) or another deliverable with the same value for this attribute in the the parent group is required.
<u>OneWrapperClock</u>	If the option is set to TRUE, a single wrapper clock is used for all wrapper cells of the design. If shared wrapper cells are used in wrapping the design, the functional clocks of the shared wrapper cells are muxed with the wrapper clock in wrapper test modes by the core wrapper application. If the option is set to FALSE, a dedicated wrapper clock is used for all dedicated wrapper cells of the design and functional clocks are used for shared wrapper cells of the design.
<u>open</u>	Open a file-based or command pipeline channel
<u>OpenAllTreeItems</u>	should all tree items be displayed open for tree dialogs
<u>open_workspace</u>	Open the workspace.
<u>OperatingConditionsBest</u>	Best-case operating condition to use when evaluating timing for this design.
<u>OperatingConditionsWorst</u>	Worst-case operating condition to use when evaluating timing for this design.
<u>OptimizationPriorities</u>	Primary and secondary implementation goals for this design.
<u>OptimizeArithmetic</u>	Should this design be optimized at the arithmetic level prior to initial mapping?
<u>Optional</u>	Indicates that the interface object may not be linked to a design object, or may not exist.

coreTools Command Reference Index

<u>OptionalAssociation</u>	Indicates that the interface object must not be associated to a design object in the parent interface instance.
<u>OutputDelay</u>	Delay constraint for an output port
<u>Overconstrain</u>	Percentage by which delay constraints should be tightened during initial mapping.
 P	
<u>package</u>	Facilities for package loading and version control
<u>packaged_prime_profile_report</u>	Generate an HTML report for Prime Profiles
<u>packagens</u>	Construct an appropriate 'package ifneeded' command for a given package specification
<u>ParallelJobCpuLimit</u>	Specifies the number of parallel compile jobs that may be run.
<u>param</u>	Parameter item
<u>ParameterCheckCmd</u>	Global check command for a number of parameters.
<u>ParameterInfo</u>	This attribute holds hints to build parameter dialog.
<u>Parameters</u>	Parameters used to configure this item. Subscript for that param.
<u>ParamValueFromDesign</u>	Indicates that the interface parameter comes from the corresponding design parameter.
<u>ParentWireLoad</u>	wireload model that will be used one level above this design.
<u>parse_proc_arguments</u>	Parses the arguments passed into a Tcl procedure.
<u>percent_of_period</u>	Define delay as a function of clock's cycle time
<u>permanent_proc</u>	Wrapper around proc to make a permanent hidden proc.
<u>PhysicalLeft</u>	Indicates the physical left bit index of the interface port to be connected.
<u>PhysicalRegion</u>	Denotes that this design is to be contained in one contiguous physical region of the chip. Also controls the mapping of the the design in Physical Compiler as MapBlockIndividually controls the mapping of designs in Design

coreTools Command Reference Index

	Compiler. In Design Compiler there are no restrictions on subdesigns during compilation, but in Physical Compiler a design that is compiled may not contain any subdesigns that are have been compiled separately. When in doubt leave this attribute set to FALSE.
<u>PhysicalRight</u>	Indicates the physical right bit index of the interface port to be connected.
<u>pid</u>	Retrieve process identifiers
<u>pin</u>	Pin item
<u>PingTestMask</u>	Obsolete attribute - Use UVM testing now.
<u>PinLoadCount</u>	Expected external fanout value for a port.
<u>PinLoadType</u>	Type of cell expected to load this port (externally).
<u>pin_cap_of</u>	Return the capacitance of the specified pin.
<u>pkgMkIndex</u>	Build an index for automatic loading of packages
<u>platform</u>	System identification support code and utilities
<u>platform_shell</u>	System identification support code and utilities
<u>plugin_proc</u>	Define a procedure in coreKit context or workspace context
<u>port</u>	Port item
<u>PortDirection</u>	Signal flow direction of this port
<u>PortFunction</u>	Function of the port/pin/bit: S(ource), L(oad), or B(oth)
<u>PortWidth</u>	Indicates the width of the port.
<u>PostPromptCmd</u>	Command(s) to be evaluated when completing an activity, after the parameters are successfully validated.
<u>PowerDomain</u>	Indicates that the given port is associated with the indicated power domain (which must be enumerated in the PowerDomains attribute on the containing design).
<u>PowerDomains</u>	Defines the set of valid power domain names for this design.
<u>PredefinedInoutPorts</u>	List of inout ports to create prior

<u>PredefinedInputPorts</u>	to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.
<u>PredefinedOutputPorts</u>	List of input ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.
<u>prefix</u>	List of output ports to create prior to making automatic connections. This enables creating ports which are of a larger dimension than needed for auto-connection.
<u>prefix defines</u>	facilities for prefix matching
<u>prefix identifiers</u>	Prefix parameters in RTL
<u>prefix macros</u>	Prefix the specified identifiers in the listed source files.
<u>PrePromptCmd</u>	Prefix all of the macros found in source files.
<u>prereq activities complete</u>	Command(s) to be evaluated before querying and validating the parameters for an activity
<u>PreserveDesignWare</u>	Check if prerequisite activities have completed yet!
<u>PreserveHierarchyFromTop</u>	Indicates that the DesignWare parts in this design should not be ungrouped.
<u>PreserveMuxOps</u>	Defines lower level designs and cells to be preserved during synthesis.
<u>PrimeProfileAllowNone</u>	Indicates that the MuxOps in this design should not be ungrouped.
<u>PrimeProfileCurrent</u>	If true, users are allowed to skip Prime Profile application in SelectPrimeProfile activity.
<u>PrimeProfileDefaults</u>	Design attribute to store the current prime profile.
<u>PrimeProfileNote</u>	Design attribute to store Default Prime Profile tags.
<u>PrimeProfilesValid</u>	A note to be displayed in SelectPrimeProfile activity.
<u>printenv</u>	Design attribute to store valid prime profile tags.
<u>printvar</u>	Prints the value of environment variables.

coreTools Command Reference Index

<u>print_message_info</u>	Prints information about diagnostic messages that have occurred or have been limited.
<u>print_suppressed_messages</u>	Displays an alphabetical list of message IDs that are currently suppressed.
<u>proc</u>	Create a Tcl procedure
<u>process_solution_files</u>	Explicitly process solution files for the indicated phase.
<u>proc_args</u>	Displays the formal parameters of a procedure.
<u>proc_body</u>	Displays the body of a procedure.
<u>ProjectID</u>	Used to record the project ID associated with a workspace.
<u>ProjectRootDir</u>	The logical root directory of a core project.
<u>propagate_memory_map</u>	Propagate the memory maps visible from an exported slave interface to that slave
<u>protected_proc</u>	Wrapper around proc to make a protected (permanent but visible) proc.
<u>pt_shellVariable</u>	Variable settings to be used for pt_shell. Example: set_design_attribute {pt_shellVariable[varName]} value.
<u>pt_shellVariableComment</u>	Comment to be issued for the corresponding subscript of pt_shellVariable.
<u>PT_static_timing_analysis_strategy</u>	Run Static Timing Analysis on a design using PrimeTime
<u>puts</u>	Write to a channel
<u>pwd</u>	Return the absolute path of the current working directory
<u>py_eval</u>	Evaluate python code. This is an optional feature, not all applications support Python.

Q

quit Exits the shell.

R

RALAccessType

<u>RALAdditionalFieldAttributeText</u>	Defines the VMM RAL access type for this register field. In most cases this is calculated via the MemoryAccess, WriteBehavior, and ReadAction values for the field. This is typically only set explicitly when the a0 or a1 types are desired. For RAL (Register Abstraction Language) files, this attribute defines additional text to be added to the definition of each RAL field. This attribute can be used to specify RAL constraints.
<u>RalListInfo</u>	Specifies the Ral list info for tb mode used for the workspace.
<u>RALSuppressionComment</u>	Reason for skipping test on register, register field or register array. Must not be left empty.
<u>randomize_parameters</u>	randomize the parameters with the help of VCS constraints solver
<u>rand_proc</u>	Create SV function which is equivalent of tcl proc which will be used by randomize_parameters command. Generally created in coreBuilder via plugin files. Can be overridden in cC and cA
<u>read</u>	Read from a channel
<u>ReadAction</u>	Indicates that reading the given field will cause the field value to be modified as described by the value of this attribute.
<u>ReadActionDescription</u>	Indicates that reading the given field will cause the field value to be modified as described by the value of this attribute.
<u>ReadActionModifier</u>	Indicates user-defined reading action when ReadAction is set to 'modify'. It implies this read action is not covered by any of other allowed ReadAction values which are: {clear, set}. This attribute is only valid when ReadAction is set to 'modify'
<u>ReadOnlyInterface</u>	Indicates that this interface connection cannot be modified manually.

coreTools Command Reference Index

<u>ReadOnlyParam</u>	The value of this parameter is read-only
<u>ReadOnlyWorkspaceComponent</u>	Indicates the workspace component will remain read-only after completing specify subsystem.
<u>read_attribute table</u>	Read an attribute table file.
<u>read_component constraints</u>	Read component SDC files to apply component constraints.
<u>read_ipxact file</u>	Read and process an IP-XACT file of the specified type.
<u>read_parameter info table</u>	Read a parameter info table file.
<u>read_parameter table</u>	Read a parameter table file.
<u>read_pin connection table</u>	Read a pin connection table file.
<u>read_sdc</u>	Read a Synopsys Design Constraints format script.
<u>read_subsystem table</u>	Read a subsystem table file.
<u>redirect</u>	Redirects the output of a command to a file.
<u>refchan</u>	command handler API of reflected channels
<u>ReferenceClock</u>	Indicates a generated clock defined for reference purposes only. A reference clock is used to enable clock linking in subsystem assembly but is not written to any SDC file.
<u>ReferenceClockName</u>	For a virtual clock, stores name of the reference real clock in the same hierarchy.
<u>refItem</u>	Defines a reference to a memory map in a component.
<u>regexp</u>	Match a regular expression against a string
<u>register</u>	Describes a register within an address block.
<u>registerArray</u>	Describes a registerArray within an address block or register array.
<u>RegisterArrayDimensions</u>	Specifies the size of the RegisterArray.
<u>Registered</u>	Indicates that the given port is registered within the component. It is either directly connected to a register or connected via only buffers or inverters with no intervening logic.
<u>RegisteredBy</u>	Indicates that the given port is registered within the component. It

coreTools Command Reference Index

<u>registerField</u>	is either directly connected to a register or connected via only buffers or inverters with no intervening logic. Describes a field within a register.
<u>RegisterResetMask</u>	Mask to be anded before comparing to the RegisterResetValue
<u>RegisterResetValue</u>	Value of the register at reset
<u>RegisterSize</u>	Specifies the size of the Register or register field in bits
<u>register_netlist</u>	Register a user-defined netlist for use in Generate Subsystem RTL.
<u>registry</u>	Manipulate the Windows registry
<u>regsub</u>	Perform substitutions based on regular expression pattern matching
<u>reinitialize component</u>	Re-initialize a component through resetting all design parameters in coreAssembler or coreConsultant.
<u>reinitialize workspace</u>	Re-initialize workspace through resetting all design parameters and activity parameters for each given activity
<u>RemoveIfEmpty</u>	Specifies that this file can be deleted from disk if it is empty.
<u>remove address bank</u>	Remove an address bank.
<u>remove address block</u>	Remove an address block.
<u>remove address space</u>	Remove an address space.
<u>remove allow_paths</u>	Remove -allow_paths from the named clock group.
<u>remove area estimates</u>	Remove area estimation values for the current design.
<u>remove attached interface</u>	Delete attached interfaces.
<u>remove command hook</u>	Remove hook from command execution
<u>remove component</u>	Remove the specified component instance from the subsystem.
<u>remove component view</u>	Remove an IP-XACT component view.
<u>remove connection</u>	Remove the specified connections.
<u>remove constraints</u>	Remove constraints from the specified or current component and from all levels above.
<u>remove exported interface</u>	Remove an exported interface instance from the subsystem.
<u>remove from collection</u>	Remove object(s) from a collection. Result is new collection

coreTools Command Reference Index

<u>remove generated clock</u>	Remove the specified generated clock
<u>remove html report link</u>	Remove an HTML report link from the index page
<u>remove interface</u>	Remove an interface definition and all its instances
<u>remove interface instance</u>	Remove an instance of an interface associate with the core
<u>remove ipxact file</u>	remove a spirit file and related objects defined in the file.
<u>remove memory map</u>	Remove a memory map.
<u>remove register</u>	Remove a register with the given name.
<u>remove register array</u>	Remove a registerArray.
<u>remove register field</u>	Remove a register field.
<u>remove register field value</u>	Remove a register field_value.
<u>remove virtual clock</u>	Remove the specified virtual clock
<u>rename</u>	Rename or delete a command
<u>rename component</u>	rename a component in the subsystem.
<u>rename exported interface</u>	Rename an exported or attached interface instance in the subsystem.
<u>rename port</u>	Rename a hierarchical port
<u>ReplaceConstantParam</u>	Replace the definition of a constant
<u>ReplaceDesignParams</u>	Replace a set of design parameters with user-specified values
<u>ReplaceFormatParam</u>	Replace text with a formatted parameter value
<u>ReplaceInHDL</u>	If true on a VHDL package parameter, then automatically replace the parameter definition in the source code. Defaults to false if there are no annotations on the parameter in the VHDL. For verilog 2001, set to false for conditional cells defined through generate-conditional statements.
<u>ReplaceSingleBitBus</u>	Convert single bit declarations into bit type
<u>replace component</u>	replace a component in the subsystem.
<u>replace interface</u>	This command takes a connected interface (with SpecifySubsystem completed) and removes the auto

coreTools Command Reference Index

<u>replay connections</u>	connections of the interfaces and replaces them with manual connections all without incompleting SpecifySubsystem. Replay manual connections.
<u>report activities</u>	Lists all activities and their current states
<u>report activity parameters</u>	List all parameters for each activity in the activity list.
<u>report activity subproc</u>	Report on activity/filegroup sub-process.
<u>report app var</u>	Shows the application variables.
<u>report attribute</u>	Report on the specified (or all) attributes for the specified item(s)
<u>report bom</u>	Report Bill Of Materials
<u>report connections</u>	Report on connected and unconnected component pins and subsystem ports.
<u>report filegroup</u>	Report file groups
<u>report interfaces</u>	generate textual or HDL description of the given interface definitions
<u>report interface instances</u>	generate textual or HDL description of the given interface instances
<u>report ip</u>	Report on Synopsys IP used.
<u>report license</u>	List all currently checked out licenses
<u>report memory maps</u>	Report on memory maps in a component.
<u>report prime profiles</u>	Generates a Prime Profile report for a hierarchy/component.
<u>report prime profile validation expr</u>	Report validation expressions for Prime Profiles.
<u>report solutions</u>	Generate simple report of available solutions.
<u>report suppressed ral tests</u>	Generates a JSON report for all memory items that are untested and items which are tested with a non-empty string in the RALSuppressionComment attribute
<u>report tech cell mapping</u>	Report technology specific cell mapping definitions and bindings.
<u>report timing exceptions</u>	Report timing exception objects
<u>report trace</u>	Produce a report of performance profile metrics collected for traced commands.
<u>RequiredExPortDirection</u>	

coreTools Command Reference Index

<u>RequiredPortDirection</u>	Required design port direction when associated with an exported interface port.
<u>Reserved</u>	Required design port direction when associated with an interface port. Indicates that the bits Reserved (meaning unspecified or undocumented bitFields). Support modeling of external reset signals. active-low : active low reset active-high : active high reset synchronous : reset is synchronous ideal : treat as ideal network for synthesis no-dft : do not include DFT reset signal definition no-vcspyglass : do not include SpyGlass reset signal definition no-synthesis : do not include in synthesis constraints
<u>ResetProperties</u>	Reset specified paths to single cycle timing
<u>reset_path</u>	Restructure the current workspace.
<u>restructure_workspace</u>	Return from a procedure, or set return code of a script
<u>return</u>	A type of stylized comment used to configure text files when used in coreConsultant and coreAssembler. For a more detailed description with more examples, see the coreBuilder manual.
<u>reuse-pragma</u>	Syntax of Tcl regular expressions
<u>re_syntax</u>	Specifies a RTLA script to be executed prior to specific optimization phase.
<u>RtlaAuxScript</u>	Specifies a comment to be issued for the corresponding RtlaAuxScript.
<u>RtlaAuxScriptComment</u>	Perform Design Verification of the subsystem and its components.
<u>RTLStaticChecking</u>	The attribute indicates that design needs abstract generation and to verified using bottom up flow using VC Spyglass in coreAssembler.
<u>RTLStaticCheckingAsSubblock</u>	Runs RTL Architect under the RTL Architect Reference Methodology associated with the version of RTL Architect being used.
<u>RTL_Architect_Reference_Methodology</u>	

<u>rtl_shellAppOption</u>	Sets application options to the specified value on the specified block. Settings to be used for RTL Architect (rtl_shell).
<u>rtl_shellAppOptionComment</u>	Add a comment for the application options being introduced in the tool intent.
<u>RunOptions</u>	Additional options for the given run style. For remote this should be a hostname. For lsf and uge this specifies options to the bsub and qsub commands, respectively. For bsub you will typically need to specify the queue name (e.g. '-q normal'). For qsub you can specify a project or other options (e.g. '-P bnormal -l arch=glinux'). The -V option is included automatically for qsub.
 <u>S</u>	
<u>safe</u>	Creating and manipulating safe interpreters
<u>save_workspace</u>	Save the current workspace.
<u>scale_to_current_units</u>	Scale a time, area, or capacitance value to the unit currently in use
<u>scan</u>	Parse string using conversion specifiers in the style of sscanf
<u>ScanBlockIndividually</u>	Causes scan insertion to be performed on this design when it is stand-alone. The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, please refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.
<u>ScanCompressionConfiguration</u>	
<u>ScanCompressionTestClock</u>	Sets a test clock for scan compression from the list of test clocks available.
<u>ScanCompressionUsage</u>	The value passed along with -usage option for ScanCompression_mode test mode in define_test_mode command. See

ScanExcludeScanInternalClocksScanJumpBuffersInvertersScanMethodologyScanReplaceScanStyleScanSubblocksIndividuallyScratchInstallIndex

DFTMAX Compression and DFTMAX Ultra/SEQ Compression sections in the TestMAX DFT user guide for more details.

List of sequential cells, hierarchical cells containing sequential elements, references, library cells, or designs that should not be replaced by scan cells.

When true, internal clocks are treated as separate clocks for creating scan chains. In XG mode, setting this attribute to false is equivalent to '-internal_clocks none'. When this attribute is set to TRUE it is Used in conjunction with ScanJumpBuffersInverters.

This attribute will only have an effect when ScanInternalClocks is set to TRUE. For XG mode, the value of this attribute will be the value used for the -internal_clocks option to set_scan_configuration. When not in XG mode, setting

ScanJumpBuffersInverters controls the setting of the DC variable test_jump_over_bufs_invs. Setting ScanJumpBuffersInverters to single sets test_jump_over_bufs_invs to true. Setting the attribute ScanJumpBuffersInverters to 'multi' sets test_jump_over_bufs_invs to false.

Selects the scan methodology. In full_scan methodology insert_scan makes all sequential cells scannable if they do not violate scan design rules. A setting of none selects no scan methodology for the design.

Should non-scan registers be replaced by scannable registers?

Sets the test scan implementation style for this design.

Causes scan insertion to be performed on the sub-designs directly instantiated in this design.

Indicates the index for the components being installed in scratch/ungroup folder by

<u>scratch_dir</u>	'create_hierarchical_component -fromComponent'. On the Subsystem KB this attribute is used to store the current count, on the design it is used to indicate the index to the install path. The install path can be created as Vscratch/ungroup/Install_\
<u>SDCConstraints</u>	Directory for storage of temporary files.
<u>SDCPropagateText</u>	Specifies information about an externally provided SDC file for a component.
<u>SDCWriteCallback</u>	SDC text to be propagated to next level of hierarchy. The text insertion point is based on the subscript specified.
<u>seek</u>	Name of callback command to invoke after writing SDC file
<u>SelectPrimeProfile</u>	Change the access position for an open channel
<u>select_by_class</u>	Prime Profiles set on components can be selected in this activity.
<u>select_by_function</u>	Select a representative library cell by its function class
<u>select_by_name</u>	Select a representative library cell by its logic function
<u>self</u>	Select a library cell (or pin) by name
<u>SeqCompressionInputs</u>	method call internal introspection
<u>SeqCompressionOutputs</u>	Total No of SEQ Compression Input Ports
<u>Sequence</u>	Total No of SEQ Compression Output Ports
<u>set</u>	Sequence number of an item
<u>SetDesignFileMacroPrefixes</u>	Read and write variables
	Set the design, file, and macro prefixes. The design prefix is used to make each design in your core have a unique name. Setting the design prefix is only needed when you need to have two or more versions of one component, i.e., you plan to instantiate two or more versions of one component in the same design database, each with different values of configuration parameters. When the

<u>setenv</u>	RTL code is written to disk, the prefix is prepended to every design name reference \ (and package name references for VHDL\). All design names and references to design names will be changed to {prefix}_{original name}. The macro prefix is used to replace any 'define {var's} and usages of the '{var}' with a prefixed value throughout the RTL code.
<u>SetInHex</u>	Sets the value of a system environment variable. This parameter value can be, must be, or cannot be specified in hexadecimal format
<u>SetUpHelpMenu</u>	Add documentation links to the Help menu in the console window.
<u>set activity incomplete</u>	Set the specified activities to the incomplete state
<u>set activity parameter</u>	Set the value of an activity parameter
<u>set address bank attribute</u>	Set the value of an attributes on an address bank.
<u>set address block attribute</u>	Set the value of an attribute on an address block.
<u>set app var</u>	Sets the value of an application variable.
<u>set area estimate</u>	Store area estimation value for a given technology and configuration.
<u>set attribute</u>	Set the value of an attribute on one or more items
<u>set cell attribute</u>	Set the value of an attribute on a list of design instances
<u>set clock attribute</u>	Set the value of an attribute on a list of clocks
<u>set command option value</u>	Set a command option default or current value.
<u>set component view</u>	Set the view associated with the current/selected component.
<u>set configuration parameter</u>	Set configuration (design) parameter value
<u>set configuration parameter attribute</u>	Set an attribute on the specified configuration parameter
<u>set current command mode</u>	
<u>set current component</u>	Set the current component to enable component level operations.

coreTools Command Reference Index

<u>set default prime profiles</u>	Set Default Prime Profiles on the design in order of priority.
<u>set dependent controls</u>	Set dependent controls for a parameter dialog.
<u>set design attribute</u>	Set the value of an attribute on one or more designs
<u>set design prefix</u>	Sets the prefix to be prepended to each design name.
<u>set disable timing</u>	Disable timing arcs in a circuit
<u>set editing mode</u>	Set workspace editing mode
<u>set false path</u>	Removes timing constraints from particular paths
<u>set filegroup attribute</u>	Set an attribute on a list of filegroups
<u>set filegroup parameter</u>	Set a parameter for a filegroup
<u>set file prefix</u>	Sets the file prefix to be prepended to each design name.
<u>set generator parameter</u>	Set component generator parameter value
<u>set hdl file list</u>	Defines an alternate HDL file list for a component.
<u>set instance parameter</u>	Command to set the value of instance parameter in terms of the subsystem parameter.
<u>set instance wire names</u>	Store mapping of port names to wire names.
<u>set interface attribute</u>	Set an attribute on the specified interface
<u>set interface parameter</u>	Set a parameter on an interface
<u>set interface parameter attribute</u>	Set an attribute on the specified interface parameter
<u>set interface port attribute</u>	Set an attribute on the specified interface port
<u>set macro prefix</u>	Sets the macro prefix to be prepended to each macro name.
<u>set max delay</u>	Specifies a maximum delay target for paths in the current design
<u>set memory map attribute</u>	Set the value of an attribute on a memory map.
<u>set memory mode</u>	Set the memory mode associated with the current/selected design
<u>set message info</u>	Set some information about diagnostic messages.
<u>set min delay</u>	Specifies a minimum delay target for paths in the current design

coreTools Command Reference Index

<u>set_multicycle_path</u>	Modifies the single-cycle timing relationship of a constrained path
<u>set_parameter attribute</u>	Set the value an attribute on one or more parameters
<u>set_port attribute</u>	Set the value of an attribute on one or more ports
<u>set_prime_profile</u>	Set a Prime Profile.
<u>set_prime_profile validation expr</u>	Set a validation expression for Prime Profiles.
<u>set_register_array attribute</u>	Set the value of an attributes on a register array.
<u>set_register attribute</u>	Set the value of an attribute on a register.
<u>set_register field attribute</u>	Set the value of an attribute on a register field.
<u>set_register field value attribute</u>	Set the value of an attribute on a register field value.
<u>set_slave base address</u>	Set the remap range and/or base address of the given slave interface.
<u>set_solution preference</u>	Set solution preference
<u>set_strategy parameter</u>	Set the value of a parameter for a stragety
<u>set_synthesis dir</u>	Set the name of the directory where synthesis will be performed.
<u>set_tclconfig variables</u>	Set TCL variables for design parameters defined to have a TCL configuration variable.
<u>set_template naming style</u>	Set the template_naming_style value.
<u>set_template parameter style</u>	Set the template_parameter_style value.
<u>set_template separator style</u>	Set the template_separator_style value.
<u>set_tool preference</u>	Set the tool preference parameter
<u>set_tool root</u>	Set the root directory for the specified tool.
<u>set_trace option</u>	Set an option value controlling behavior of command tracing and output annotation..
<u>set_unix variable</u>	This is a synonym for the \fBsetenv\fP command.
<u>set_unused interface</u>	set the Unused attribute for unused provider
<u>set_upf attribute</u>	Set the value of an attribute on a list of UPF items
<u>set_upf cells</u>	User routine to specify technology cells for UPF file configuration.

coreTools Command Reference Index

<u>set upf supply voltage</u>	Set the voltage on a supply created from set_voltage.
<u>set upf voltage</u>	User routine to specify voltage values for UPF file configuration.
<u>set workspace options</u>	Set one or more workspace options, customizing the user flow.
<u>sh</u>	Executes a command in a child process.
<u>SharedWrapperCell</u>	Specifies the default shared wrapper cell type.
<u>ShareTestPointsAcrossHierarchy</u>	Specifies that test point scan cells may be shared across hierarchy.
<u>ShowIcons</u>	When shown in a tree display an icon if it is specified.
<u>ShowRoute</u>	Indicates whether or not routes from this component should be drawn in the schematic window.
<u>show color dialog</u>	Displays a color browser dialog
<u>show exit dialog</u>	Displays a color browser dialog
<u>show prime profile prompt</u>	Show/Don't show Prime Profile selection prompt in consultant and assembler.
<u>show spreadsheet for connections</u>	Generate and show a spreadsheet for making connections to interfaces with the type of the given interface.
<u>show url</u>	Display URL internally
<u>show url external</u>	Display URL externally
<u>show user parameter dialog</u>	Show the user parameter dialog to build and return the values of a set of user defined parameters
<u>sh allow tcl with set app var</u>	Allows the \fBset_app_var\fP and \fBget_app_var\fP commands to work with application variables.
<u>sh allow tcl with set app var no message list</u>	Suppresses CMD-104 messages for variables in this list.
<u>sh arch</u>	Indicates the system architecture of your machine.
<u>sh command abbrev mode</u>	Sets the command abbreviation mode for interactive convenience.
<u>sh command abbrev options</u>	Turns off abbreviation of command dash option names when false.
<u>sh command log file</u>	Specifies the name of the file to which the application logs the commands you executed during the session.
<u>sh continue on error</u>	

<u>sh deprecated is error</u>	Allows processing to continue when errors occur during script execution with the \fBsource\fP command.
<u>sh dev null</u>	Raise a Tcl error when a deprecated command is executed.
<u>sh disabled is error</u>	Indicates the current null device.
<u>sh enable page mode</u>	Raise a Tcl error when a disabled command is executed.
<u>sh enable stdout redirect</u>	Displays long reports one page at a time (similar to the UNIX \fBmore\fP command).
<u>sh executable name</u>	Allows the redirect command to capture output to the Tcl stdout channel.
<u>sh help shows group overview</u>	Full path name of executable for the current tool session.
<u>sh language</u>	Changes the behavior of the "help" command.
<u>sh new variable message</u>	Controls the tool language used by the application interactive console.
<u>sh new variable message in proc</u>	Controls a debugging feature for tracing the creation of new variables.
<u>sh new variable message in script</u>	Controls a debugging feature for tracing the creation of new variables in a Tcl procedure.
<u>sh obsolete is error</u>	Controls a debugging feature for tracing the creation of new variables within a sourced script.
<u>sh product version</u>	Raise a Tcl error when an obsolete command is executed.
<u>sh script stop severity</u>	Tracks the value of the like-named pt_shell variable
<u>sh show tcl stack on error</u>	Indicates the error message severity level that would cause a script to stop running before it completes.
<u>sh source emits line numbers</u>	Print or do not print the TCL stack when a TCL error occurs.
<u>sh source logging</u>	Indicates the error message severity level that causes an informational message to be issued, listing the script name and line number where that message occurred.
<u>sh source uses search path</u>	Indicates if individual commands from a sourced script should be logged to the command log file.

<u>sh Tcllib app dirname</u>	Indicates if the \fBsource\fP command uses the \fBsearch_path\fP variable to search for files.
<u>sh user man path</u>	Indicates the name of a directory where application-specific Tcl files are found.
<u>SideConstraint</u>	Indicates a directory root where you can store man pages for display with the \fBman\fP command.
<u>SideEffects</u>	Specifies the definite pin side details. The values for DesignCompiler are left, right, top, bottom \& for FusionCompiler the side should be an integer value starting from 1.
<u>SimTieOff</u>	Obsolete attribute - Use UVM testing now.
<u>SimulateSubsystemTestbench</u>	Indicates how a pin should be tied off in the testbench if left unconnected.
<u>SimulationModel</u>	Simulate the current subsystem testbench.
<u>sizeof collection</u>	Indicates that this design is a simulation model and is not to be synthesized.
<u>SlotNumber</u>	Get the number of objects in collection
<u>SlotNumberOffset</u>	Indicates the slot number on the bus for the given interface.
<u>SlotOrder</u>	Indicates the offset (relative to 0) for numbering slots associated with this interface.
<u>SlotWidth</u>	Indicates the order of the slot on the bus for the given interface.
<u>snps.cmd</u>	Indicates the number of continuous slots allocated by attribute SlotNumber and any other 'unique-value' interface parameter on the instance. The attribute annotates an eval_param expression in context of the instance.
<u>snps.collection</u>	Python object that provides access to application commands.
<u>snps.redirect</u>	Python class for Synopsys collection objects.
	Python class used to redirect application output.

coreTools Command Reference Index

<u>snps.value</u>	Python class for application command return values.
<u>socket</u>	Open a TCP network connection
<u>SolutionData</u>	Stores the definition of the solution (from define_solution).
<u>SolutionEnvVarValues</u>	The solution environment vars stored on disk for use in open workspace.
<u>SolutionItemPhase</u>	Indicates the solution phase the item was created in when installing a solution.
<u>SolutionProcessed</u>	Internal status attribute indicating whether solution files have been processed or not.
<u>source</u>	Read a file and evaluate it as a Tcl script.
<u>source solution script</u>	Invoke specified solution script.
<u>SpecifyClocks</u>	Define the set of clocks (virtual and real) within the core design.
<u>SpecifyConfiguration</u>	Specify values for HDL configuration parameters.
<u>SpecifyOperatingConditionsAndWireLoads</u>	Specify information which defines the context in which the core is going to be used (operating conditions and wireloads)
<u>SpecifyPortConstraints</u>	Set and/or review intent attributes on ports.
<u>SpecifyPowerIntent</u>	This activity is used to capture power intent scripts.
<u>SpecifySubsystem</u>	Select and configure the components and any associated memory maps for the subsystem.
<u>SpecifySynthesisMethodology</u>	Set and/or review intent attributes on designs.
<u>SpecifyTestMethododology</u>	Set and/or review Design for Test intent attributes on designs.
<u>SpecifyTimingExceptions</u>	Define the timing exceptions within the core design.
<u>SpiritContainer</u>	A place holder for spirit data that we don't model so we can export as is
<u>SpiritFile</u>	Indicates the file from which a SPIRIT element originated.
<u>SpiritFileType</u>	Specifies the value of the fileType element. Default is based on FileContentType and LoadDesigns language version.
<u>SpiritInterfaceType</u>	SPIRIT interface type

coreTools Command Reference Index

<u>SpiritLibrary</u>	IP-XACT library name
<u>SpiritName</u>	IP-XACT name
<u>SpiritVendor</u>	IP-XACT vendor name
<u>SpiritVersion</u>	IP-XACT version
<u>split</u>	Split a string into a proper Tcl list
<u>SplitInterfaceMembers</u>	Indicates the list of interfaces linked with the master interface
<u>SplitInterfaceName</u>	Indicates the Master interface pointed from the slave interface.
<u>StartBit</u>	Tag specification starting position. Indicates the original StartBit expression (subscripted representation) which was read in from RTL file in LoadDesigns activity
<u>StartBitExpr</u>	Indicates the original StartBits complete expression which was read in from RTL file in LoadDesigns activity
<u>StartBitsExpr</u>	
<u>StaticChecking</u>	Perform static checking of the subsystem and its components.
<u>StaticDefineExpr</u>	Defines the expression which causes the given port to exist. Assumed to be an expression of parameters that are not in the coreTools data model.
<u>StaticDefineExprs</u>	Specifies which static defines referenced in StaticDefineExpr values on ports should be treated as defined for constraint generation purposes.
<u>StaticTimingAnalysis</u>	Generate and (optionally) run static timing analysis scripts.
<u>StaticTimingAuxScript</u>	Specifies a script that is to be run in PrimeTime after the design has been loaded and all clocks and constraints have been applied, just before the 'update_timing' command.
<u>StaticTimingAuxScriptComment</u>	Specifies a comment to be issued for the corresponding StaticTimingAuxScript.
<u>Strategy</u>	Item used to model a strategy (command script) to drive a design tool
<u>string</u>	Manipulate strings
<u>subst</u>	Perform backslash, command, and variable substitutions
<u>SubstituteInFile</u>	Substitute text in this file at the time indicated by the subscript
<u>suppress_message</u>	

<u>switch</u>	Disables printing of one or more informational or warning messages.
<u>SymbolicNames</u>	Evaluate one of several scripts, depending on a given value
<u>SymbolLibraryPath</u>	List of symbolic names for valid values
<u>SymbolName</u>	The path to the symbol library to be loaded when the component is displayed in a schematic.
<u>SymbolPinPrefix</u>	The name of the desired symbol to use in the schematic display.
<u>SymbolPinSide</u>	The desired pin prefix name (Master).
<u>SymbolType</u>	The desired side of the symbol to place the pin
<u>SymmetricBus</u>	The desired symbol type to be generated
<u>SymmetricBusLink</u>	Indicates the interface is symmetric which allow direct master-slave connection. The bus definition should be defined as 'slave' type, then 'master' type is just symmetric to the slave. Imply 'provider-consumer' interface type for interface instance.
<u>SynchronousTo</u>	Indicates the association to a design port when a symmetric bus is used as a provider or a consumer. The design name can include \@ and {} in the attribute value, and allows a [\@] index annotation.
<u>synopsys program name</u>	Indicates that the given port is synchronous with respect to the given clock or other designer specified conditions.
<u>synopsys root</u>	Indicates the name of the program currently running.
<u>SynplifyAuxSynthesisScript</u>	Indicates the root directory from which the application was run.
<u>SynplifyAuxSynthesisScriptComment</u>	Specifies a synplify script to be executed prior to specific optimization phase.
<u>Synthesis API</u>	Specifies a comment to be issued for the corresponding SynplifyAuxSynthesisScript.
	This man page describes the different methodologies available for generating a gate-level

Synthesize

implementation of a component configured in coreConsultant or a subsystem assembled in coreAssembler.

Generate and (optionally) run synthesis scripts. This activity is used to generate synthesis scripts customized for the particular core configuration, DC/FC version compatibility, and target technology. Now user have the option between Design Compiler and Fusion Compiler to run synthesis. By default synthesis is fired right after the scripts are generated, using the preferred tool. But this behavior can be changed by checking the Generate Scripts Only check-box. Once the synthesis is finished, results will be available in the following locations: For DC only : DDC file - syn/final/db/<top design>.ddc For DC & FC : Netlist - syn/final/db/<top design>.v[hd] Reports - syn/final/report/ If you choose to run the synthesis yourself, or wish to review the generated scripts or constraints, these files can be found in the following locations: Launch script - syn/[scriptFile] Where the name of the 'scriptFile' can be obtained with the command: `get_activity_parameter Synthesize ScriptFile Constraints - syn/constrain/script/*.[sdc/tcl]` If you wish to integrate the generated scripts into your own chip-level synthesis environment, please see the 'Synthesis_API' man page for details about how to access the generated scripts from your own environment.

The attribute indicates that design needs to be synthesized separately and a DDC (when synthesized with Design Compiler) or NDM libs (when synthesized with Fusion Compiler) will be generated for the design. The generated DDC or NDM will be consumed by the top design during hierarchical synthesis flow in coreAssembler.

SynthesizeAsSubblocksysCmd

Global array variable containing paths to different system commands.

I

<u>tailcall</u>	Replace the current procedure with another command
<u>TargetConnectionBoundary</u>	Indicates that this is a TargetConnection boundry point for exported workspace components.
<u>TCBDisableUndefMacros</u>	Disable writing out undefine directives for tech cell bound in TechnologyCellBinding activity.
<u>Tcl</u>	Tool Command Language
<u>TclAuxSynthesisScript</u>	Specifies a DC Tcl script to be executed prior to specific optimization phase.
<u>TclAuxSynthesisScriptComment</u>	Specifies a comment to be issued for the correcsponding DcTclAuxSynthesisScript.
<u>TCLConfigVariable</u>	Indicates a parameter to be defined as a TCL variable by set_tclconfig_variables.
<u>TCLConfigVariableName</u>	Indicates a parameter to be defined as a TCL variable by set_tclconfig_variables. The attribute value defines the TCL variable name.
<u>TCLConfigVariableSetArgs</u>	Defines default arguments for the set_tclconfig_variables command when invoked with arguments in the context of this design.
<u>TclPhysicalConstraintFile</u>	Tcl format physical constraints to be placed on the design during synthesis.
<u>tcltest</u>	Test harness support code and utilities
<u>tclvars</u>	Variables used by Tcl
<u>TechCellMappingBind</u>	Used to store technology cell mapping bindings from specify_tech_cell_mapping.
<u>TechCellMappingDefn</u>	Used to store technology cell mapping definitions from define_tech_cell_mapping.
<u>TechnologyCellBinding</u>	This activity implements the technology cell mapping functionality.
<u>TechSetup</u>	This activity gets variable settings from DesignCompiler, and extracts technology specific information from

coreTools Command Reference Index

<u>tell</u>	the specified technology libraries. Return current access position for an open channel
<u>TestabilityClockSignal</u>	Specifies the clock to be used for control or observe registers.
<u>TestabilityClockType</u>	Specifies the clock type to be used for testability enhancements. Selects the control signal to be used for test points. The signal should be predefined as having the TestMode signal type. If no control signal is specified, the test points will be controlled by any signal with the TestMode signal type. If no TestMode signal is found, DFT Compiler will create a new TestMode port.
<u>TestabilityControlSignal</u>	Specifies the maximum percentage of area overhead due to the observe test point structure insertion in the design. Setting this attribute to 0 leaves this as unspecified in DFT Compiler.
<u>TestabilityMaxArea</u>	Testability method to be used. Note that the tdvr and observe methods are identical. The tdvr setting is for backward compatibility.
<u>TestabilityMethod</u>	Indicates that the given field is testable in a simple register test, provided the specified constraints are met.
<u>Testable</u>	This clock is used in test mode.
<u>TestClock</u>	Cycle time for clock in test mode.
<u>TestClockCycleTime</u>	Default cycle time for test clocks.
<u>TestClockDefaultCycleTime</u>	When true, dedicated test clocks are created according to different system clocks. Thus, scan cells that have different system clocks have different test clocks. When false, the default, test clocks are created without considering system clocks.
<u>TestClocksFollowSystemDomains</u>	Waveform for clock in test mode.
<u>TestClockWaveform</u>	Causes set_test_isolate to be issued for a port. This functionality is not supported in XG mode.
<u>TestIsolate</u>	Adds logic structure (and gate) for the power saving on the XOR tree
<u>TestpointPowerSavingOn</u>	
<u>TestsuiteSetup</u>	

coreTools Command Reference Index

<u>throw</u>	Specify testsuite filegroups and parameters.
<u>time</u>	Generate a machine-readable error
<u>timerate</u>	Time the execution of a script
<u>timingException</u>	Calibrated performance measurements of script execution time
<u>TimingExceptionCmd</u>	Item used to model a timing exception command
<u>TimingExceptionFall fromValue</u>	Command associated with this timing exception
<u>TimingExceptionFall throughValue</u>	List of path startpoints. The path must fall from objects specified
<u>TimingExceptionFall toValue</u>	List of path through points. Applied to paths with a falling transition at specified objects
<u>TimingExceptionFromValue</u>	List of path endpoints. Applies to paths falling at the endpoint
<u>TimingExceptionGroup path</u>	List of path startpoints
<u>TimingExceptionOptions</u>	Name of group for paths
<u>TimingExceptionRise fromValue</u>	Options associated with this timing exception
<u>TimingExceptionRise throughValue</u>	List of path startpoints. The path must rise from objects specified
<u>TimingExceptionRise toValue</u>	List of path through points. Applies to paths with a rising transition at specified objects
<u>TimingExceptionThroughValue</u>	List of path endpoints. Applies to paths rising at the endpoint
<u>TimingExceptionToValue</u>	List of path through points
<u>TimingExceptionValue</u>	List of path endpoints
<u>tm</u>	Value for timing exception command
<u>TMAX Reference Methodology</u>	Facilities for locating and loading of Tcl Modules
<u>tmax shellVariable</u>	This flow implements the TestMAX Reference Methodology. For details refer to the TMAX Reference Methodology.
<u>tmax shellVariableComment</u>	Sets application options to the specified value on the specified block. Settings to be used for Synopsys TestMAX (tmax). The app variables can only be applied at setup and report generation.
	Add a comment for the application options being introduced in the tool intent.

coreTools Command Reference Index

<u>TmmAuxScript</u>	Specifies a TestMaxManager script to be executed prior to specific phase.
<u>TmmAuxScriptComment</u>	Specifies a comment to be issued for the corresponding TmmAuxScript. The attribute is used for storing the updated values for the Test Max Manager Reference Methodology variables, which will then be updated in the Reference Methodology scripts if the strategy parameter EnableTestMaxManagerFlow is set to 1
<u>TMMRMVariable</u>	Sets application options to the specified value on the specified block. Settings to be used for TestMax Manager (tmm_shell).
<u>tmm_shellAppOption</u>	Add a comment for the application options being introduced in the tool intent.
<u>tmm_shellAppOptionComment</u>	Specifies a TMR intent script to be executed post elaboration.
<u>TMRIntentFile</u>	Specifies a comment to be issued for the corresponding TMR Intent file exported in the tool intent.
<u>TMRIntentFileComment</u>	Monitor variable accesses, command usages and command executions
<u>trace</u>	command handler API of channel transforms
<u>transchan</u>	This command takes a connected interface (with SpecifySubsystem completed) and removes the auto connections of the interfaces and replaces them with manual connections all without incompleting SpecifySubsystem.
<u>translate interface connection</u>	Translate a specified design from HDL files to IP-XACT.
<u>translate netlist</u>	Translate Hierarchical Pin SDC constraints to be compatible with DW IP Solution flow
<u>translate phy constraints</u>	Traverse a component boundary returning the above/below equivalent connection item.
<u>traverse boundary</u>	Traverse a component connections for the connection object.
<u>traverse connection</u>	Trap and process errors and exceptions
<u>try</u>	Generic class of SV typedef for the item: type, enum, struct, union.
<u>TypedefType</u>	

coreTools Command Reference Index

TypeName

The type of an item

U

<u>unalias</u>	Removes one or more aliases.
<u>UnconnectedPort</u>	Indicates that an output port remains unconnected.
<u>unconnect interface</u>	Unconnect interfaces
<u>UndefinedBitValue</u>	Indicates the default reset value for undefined bits in a register.
<u>Underconstrain</u>	Percentage by which I/O delay constraints should be relaxed during initial mapping.
<u>UnelaboratedName</u>	the unelaborated name for this design
<u>ungroup component</u>	Ungroup the hierarchical component into current level.
<u>ungroup filegroup</u>	ungroup a filegroup
<u>union collection</u>	Combine two collections with set union semantics.
<u>UniqueConsumerConnections</u>	Show each consumer connection for this provider independently on schematic.
<u>unknown</u>	Handle attempts to use non-existent commands
<u>unload</u>	Unload machine code
<u>unload autoload filegroup</u>	Removes loaded files from an autoloaded filegroup
<u>unload component memory maps</u>	Unloads all non-visible component memory maps from knowledgebase files in cC and cA from memory or a specific one if -map option is used.
<u>unload file from kb</u>	Write the contents from a file item to the host system.
<u>unload plugin</u>	Unload a plugin KB.
<u>unset</u>	Delete variables
<u>unsetenv</u>	Removes a system environment variable.
<u>unsuppress message</u>	Enables printing of one or more suppressed informational or suppressed warning messages.
<u>unused interface instance</u>	Returns 1 if the given interface instance is not being used.
<u>update</u>	Process pending events and idle callbacks
<u>update dut</u>	Update the DUT to the latest version if possible.
<u>update workspace links</u>	Change symbolic links in a workspace
<u>UPFFile</u>	

coreTools Command Reference Index

	Specifies a UPF file to load with load_upf in Design Compiler.
<u>uplevel</u>	Execute a script in a different stack frame
<u>upvar</u>	Create link to variable in a different stack frame
	Specifies condition whether the interface port or parameter is used in the parent interface instance. The condition can include \@\'s in the attribute value, which makes the interface usage conditional.
<u>UsedOnInstance</u>	
	Indicates a port name specified with create_connection -port to use with merge_ports.
<u>UserPortName</u>	
	Indicates how utilized the given interface is.
	This is a read-only attribute. This stores UVM Ral Attributes based on MemoryAccess ReadAction, WriteBehaviour. Please refer to UVM 1.1 users guide section 5.5.1.4. Note : As per IP-Xact mapping table, the User-defined fields will be se as rw as UVM RALF format does not support any appropriate field type
<u>UVMRALAccessType</u>	
	This feature provides the user ability to add user specific steps in the UVM Test phases of RAL tests in the UVM RAL Subsystem flow. The user can configure, drive and control signals in any core by using this feature. Not all phases of the RAL test can be modified, the most important ones are available for modification.
<u>UVMTestText</u>	
 <u>V</u>	
<u>Value</u>	Value of this parameter
<u>ValueRequired</u>	Value is required for this parameter
<u>variable</u>	create and initialize a namespace variable
<u>verbose messages</u>	Enables verbose messaging mode.
<u>VerifyIntent</u>	This activity attempts to flag common mistakes that are made when specifying constraints.
<u>verify_dwf</u>	Verify the DesignWare Foundation version.
<u>verify_environment</u>	Verify the user's environment using the EnvCheck attribute.
<u>verify_tool</u>	Verify the proper environment variable (if it has one; otherwise, verify that the tool's

coreTools Command Reference Index

	executable is in PATH), version and licensing for a tool.
<u>VerilogHeaderValue</u>	Defines the value for the Verilog header file for the given attribute on this register or register field.
<u>Version</u>	The version of the Bill-of-Materials template.
<u>VHDLDesignLibrary</u>	VHDL library for source code generated by coreAssembler
<u>VhdlHeaderValue</u>	Defines the value for the VHDL header file for the given attribute on this register or register field.
<u>VipConnection</u>	Indicates the name of the interface to be connected to. If two values are returned, indicates an alternate instance to be connected to. Can access global variable \::iip_instance_name via formulas if needed.
<u>VipInitializationCall</u>	Indicates the string required to be inserted into the testbench to initialize the instantiated VIP component.
<u>VipModelName</u>	Specifies the name of the vmm vip model
<u>VipPackage</u>	Specifies the name of the vmm vip package
<u>VipRandomizable</u>	Specifies if the specified VMM variable is randomizable
<u>VipRandomRange</u>	Specifies the VMM variable's range of possible settings
<u>VipReasonableRandomRange</u>	Specifies the VMM variable's range of reasonable settings
<u>VipScope</u>	Specifies the scope of the VIP configuration field
<u>Visible</u>	Make this parameter visible in the GUI
<u>visible fields</u>	Formula that can be used on a register to determine if there are visible fields.
<u>VolatileMemory</u>	True if this item refers to volatile memory
<u>vwait</u>	Process events until a variable is written
 <u>W</u>	
<u>wait activities on grid</u>	Wait for all or a specified set of activities started with launch_activities_on_grid to complete.
<u>wait for activity subproc</u>	Wait for an activity sub-process started with launch_activity_subproc to complete.
<u>Waveform</u>	clock waveform
<u>which</u>	

coreTools Command Reference Index

	Locates a file and displays its pathname.
<u>while</u>	Execute script repeatedly as long as a condition is met
<u>WireLoad</u>	Wireload model to use for a design.
<u>WireLoadGroup</u>	Wireload model selection group to use for automatic wireload model selection.
<u>WireLoadMinBlockSize</u>	Minimum block size to use for automatic wireload selection.
<u>WireLoadMode</u>	Indicates the wireload mode for the design.
<u>WrapBlockIndividually</u>	Causes test wrapper insertion to be performed on this design.
<u>WrapperDefaultSafeState</u>	Specifies the default safe state for the wrapper cells.
<u>WrapperExcludePorts</u>	List of ports that should not be wrapped. If the option is set to swap registers connected to ports of the design are replaced with equivalent shared wrapper cells. If the option is set to in_place, additional glue logic will be added to registers attached to ports of the design to reuse the registers in core wrapper chains.
<u>WrapperRegisterImplementation</u>	If the option is set to TRUE, shared wrapper cells will be used for register bound ports. The registers are reused in core wrapper chains.
<u>WrapperUseRegisterIO</u>	Causes test wrapper insertion to be performed on the subdesigns of this design.
<u>WrapSubblocksIndividually</u>	Indicates that writing the given field will cause a write of a value other than the one written. The written value will be modified as described by the value of this attribute.
<u>WriteBehavior</u>	Indicates user-defined writing action when WriteBehaviour is set to 'modify'. It implies this write action is not covered by any of other allowed WriteBehavior values which are: {oneToClear, oneToSet, oneToToggle, zeroToClear, zeroToSet, zeroToToggle, clear, set}. This attribute is only valid when WriteBehavior is set to 'modify'
<u>WriteBehaviorModifier</u>	

coreTools Command Reference Index

<u>WriteComponentInHDL</u>	Set to false if this component should not be written in the final netlist written by coreAssembler.
<u>WriteConstraint</u>	Defines a constraint to be met when writing the given register field.
<u>write_app_var</u>	Writes a script to set the current variable values.
<u>write_attribute table</u>	Write an attribute table file.
<u>write_automotive synthesis files</u>	Export the Automotive Synthesis files
<u>write_batch script</u>	Generate a script to recreate this workspace
<u>write_component register headers</u>	Write Component Register Headers files for component
<u>write_config tar file</u>	Write out a coreConsultant configuration tar file.
<u>write_failure modes file</u>	To export SGDC files
<u>write_ipxact abstractor</u>	Export an IP-XACT description of the current abstractor.
<u>write_ipxact component</u>	Export an IP-XACT description of the current component.
<u>write_ipxact design</u>	Export an IP-XACT description of the current design.
<u>write_ipxact designConfiguration</u>	Export an IP-XACT design configuration file.
<u>write_ipxact typeDefinitions</u>	Export an IP-XACT description of the current typeDefinition.
<u>write_ParameterInfo</u>	Write the configuration intent for a set of parameters out in the format of a template for use as a starting point when generating ParameterInfo.
<u>write_pin connection table</u>	Write a pin connection table file.
<u>write_ral file</u>	Write RAL (Register Abstraction Layer) file for component
<u>write_sdc</u>	Writes out a script in Synopsys Design Constraints (SDC) format.
<u>write_subsystem table</u>	Write a subsystem table file.

X

<u>xml_translate</u>	Export an IP-XACT description of the current component.
<u>xml_verify</u>	Verify IP-XACT syntax and semantics.

Y Z

zlib

compression and decompression operations

`index_collection`

Extract object from collection. Result is new collection

Syntax

```
string index_collection collection1 index
      string collection1
      int index
```

Parameters

<code>collection1</code>	Collection to index
<code>index</code>	Index (zero based)

Description

You can use the `index_collection` command to extract a single object from a collection. The result is a new collection containing that object. The range of indices is from 0 to one less than the size of the collection. If the specified index is outside of that range, an error is generated.

Commands that create a collection of objects do not impose a specific order on the collection, but they do generate the objects in the same, predictable order each time. Applications that support the sorting of collections allow you to impose a specific order on a collection.

You can use the empty string for the 'collection' argument. However, by definition, any index into the empty collection is invalid. So using `index_collection` with the empty collection always generates the empty string as a result and issues an error message.

Examples

This example shows how the first object in a collection can be extracted:

```
dwc_shell> set x [find_item -type design {d1 d2}]
_sel3
dwc_shell> index_collection $x 0
{"d1"}
```

See Also

[sizeof_collection\(2\)](#)

`infinite_drive`

Formula to indicate infinite drive for an input port

Syntax

string **`infinite_drive`**

Description

The `infinite_drive` command is a formula command that you can use as a value for the `DrivingCell` attribute on an input port to indicate that the input port should be assumed to have infinite drive.

A typical use for the `infinite_drive` formula as a `DrivingCell` value is on clock and reset ports that will be buffered by external tools. The coreTools set `DrivingCell` to `{=infinite_drive}` by default on clock and reset ports.

Examples

To set infinite drive on the `clk` port:

```
coreConsultant> set_port_attribute clk DrivingCell {=infinite_drive}
```

See Also

[select_by_class \(2\)](#), [select_by_function \(2\)](#), [select_by_name \(2\)](#), [DrivingCell \(3\)](#)

NAME

`info` - Return information about the state of the Tcl interpreter

SYNOPSIS

`info option ?arg arg ...?`

DESCRIPTION

This command provides information about various internals of the Tcl interpreter. The legal *options* (which may be abbreviated) are:

info args *procname*

Returns a list containing the names of the arguments to procedure *procname*, in order. *Procname* must be the name of a Tcl command procedure.

info body *procname*

Returns the body of procedure *procname*. *Procname* must be the name of a Tcl command procedure.

info class *subcommand class ?arg ...*

Returns information about the class, *class*. The *subcommands* are described in **CLASS INTROSPECTION** below.

info cmdcount

Returns a count of the total number of commands that have been invoked in this interpreter.

info commands *?pattern?*

If *pattern* is not specified, returns a list of names of all the Tcl commands visible (i.e. executable without using a qualified name) to the current namespace, including both the built-in commands written in C and the command procedures defined using the **proc** command.

coreTools Command Reference Index

If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for **string match**. *pattern* can be a qualified name like **Foo::print***. That is, it may specify a particular namespace using a sequence of namespace names separated by double colons (::), and may have pattern matching special characters at the end to specify a set of commands in that namespace. If *pattern* is a qualified name, the resulting list of command names has each one qualified with the name of the specified namespace, and only the commands defined in the named namespace are returned.

info complete *command*

Returns 1 if *command* is a complete Tcl command in the sense of having no unclosed quotes, braces, brackets or array element names. If the command does not appear to be complete then 0 is returned. This command is typically used in line-oriented input environments to allow users to type in commands that span multiple lines; if the command is not complete, the script can delay evaluating it until additional lines have been typed to complete the command.

info coroutine

Returns the name of the currently executing **coroutine**, or the empty string if either no coroutine is currently executing, or the current coroutine has been deleted (but has not yet returned or yielded since deletion).

info default *procname arg varname*

Procname must be the name of a Tcl command procedure and *arg* must be the name of an argument to that procedure. If *arg* does not have a default value then the command returns 0. Otherwise it returns 1 and places the default value of *arg* into variable *varname*.

info errorstack *?interp?*

Returns, in a form that is programmatically easy to parse, the function names and arguments at each level from the call stack of the last error in the given *interp*, or in the current one if not specified.

This form is an even-sized list alternating tokens and parameters. Tokens are currently either **CALL**, **UP**, or **INNER**, but other values may be introduced in the future. **CALL** indicates a procedure call, and its parameter is the corresponding **info level** 0. **UP** indicates a shift in variable frames generated by **uplevel** or similar, and applies to the previous **CALL** item. Its parameter is the level offset. **INNER** identifies the which is the innermost atomic command or bytecode instruction that raised the error, along with its arguments when available. While **CALL** and **UP** allow to follow complex call paths, **INNER** homes in on the offending operation in the innermost procedure call, even going to sub-expression granularity.

This information is also present in the **-errorstack** entry of the options dictionary returned by 3-argument **catch**; **info errorstack** is a convenient way of retrieving it for uncaught errors at top-level in an

coreTools Command Reference Index

interactive **tclsh**.

info exists *varName*

Returns **1** if the variable named *varName* exists in the current context (either as a global or local variable) and has been defined by being given a value, returns **0** otherwise.

info frame *?number?*

This command provides access to all frames on the stack, even those hidden from **info level**. If *number* is not specified, this command returns a number giving the frame level of the command. This is 1 if the command is invoked at top-level. If *number* is specified, then the result is a dictionary containing the location information for the command at the *numbered* level on the stack.

If *number* is positive (> 0) then it selects a particular stack level (1 refers to the outer-most active command, 2 to the command it called, and so on, up to the current frame level which refers to **info frame** itself); otherwise it gives a level relative to the current command (0 refers to the current command, i.e., **info frame** itself, -1 to its caller, and so on).

This is similar to how **info level** works, except that this subcommand reports all frames, like **sourced** scripts, **evals**, **uplevels**, etc.

Note that for nested commands, like only will be seen by an **info frame** invoked within This is the same as for **info level** and error stack traces.

The result dictionary may contain the keys listed below, with the specified meanings for their values:

type

This entry is always present and describes the nature of the location for the command. The recognized values are **source**, **proc**, **eval**, and **precompiled**.

source

means that the command is found in a script loaded by the **source** command.

proc

means that the command is found in dynamically created procedure body.

eval

means that the command is executed by **eval** or **uplevel**.

precompiled

means that the command is found in a pre-compiled script (loadable by the package **tbcload**), and no further information will be available.

line

This entry provides the number of the line the command is at inside of the script it is a part of. This information is not present for type **precompiled**. For

coreTools Command Reference Index

type **source** this information is counted relative to the beginning of the file, whereas for the last two types the line is counted relative to the start of the script.

file

This entry is present only for type **source**. It provides the normalized path of the file the command is in.

cmd

This entry provides the string representation of the command. This is usually the unsubstituted form, however for commands which are a canonically-constructed list (e.g., as produced by the **list** command) executed by **eval** it is the substituted form as they have no other string representation. Care is taken that the canonicity property of the latter is not spoiled.

proc

This entry is present only if the command is found in the body of a regular Tcl procedure. It then provides the name of that procedure.

lambda

This entry is present only if the command is found in the body of an anonymous Tcl procedure, i.e. a lambda. It then provides the entire definition of the lambda in question.

level

This entry is present only if the queried frame has a corresponding frame returned by **info level**. It provides the index of this frame, relative to the current level (0 and negative numbers).

A thing of note is that for procedures statically defined in files the locations of commands in their bodies will be reported with type **source** and absolute line numbers, and not as type **proc**. The same is true for procedures nested in statically defined procedures, and literal eval scripts in files or statically defined procedures.

In contrast, procedure definitions and **eval** within a dynamically **evaluated** environment count line numbers relative to the start of their script, even if they would be able to count relative to the start of the outer dynamic script. That type of number usually makes more sense.

A different way of describing this behaviour is that file based locations are tracked as deeply as possible, and where this is not possible the lines are counted based on the smallest possible **eval** or procedure body, as that scope is usually easier to find than any dynamic outer scope.

The syntactic form **{*}** is handled like **eval**. I.e. if it is given a literal list argument the system tracks the line number within the list words as well, and otherwise all line numbers are counted relative to the

coreTools Command Reference Index

start of each word (smallest scope)

info functions *?pattern?*

If *pattern* is not specified, returns a list of all the math functions currently defined. If *pattern* is specified, only those functions whose name matches *pattern* are returned. Matching is determined using the same rules as for **string match**.

info globals *?pattern?*

If *pattern* is not specified, returns a list of all the names of currently-defined global variables. Global variables are variables in the global namespace. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for **string match**.

info hostname

Returns the name of the computer on which this invocation is being executed. Note that this name is not guaranteed to be the fully qualified domain name of the host. Where machines have several different names (as is common on systems with both TCP/IP (DNS) and NetBIOS-based networking installed,) it is the name that is suitable for TCP/IP networking that is returned.

info level *?number?*

If *number* is not specified, this command returns a number giving the stack level of the invoking procedure, or 0 if the command is invoked at top-level. If *number* is specified, then the result is a list consisting of the name and arguments for the procedure call at level *number* on the stack. If *number* is positive then it selects a particular stack level (1 refers to the top-most active procedure, 2 to the procedure it called, and so on); otherwise it gives a level relative to the current level (0 refers to the current procedure, -1 to its caller, and so on). See the **uplevel** command for more information on what stack levels mean.

info library

Returns the name of the library directory in which standard Tcl scripts are stored. This is actually the value of the **tcl_library** variable and may be changed by setting **tcl_library**.

info loaded *?interp?*

Returns a list describing all of the packages that have been loaded into *interp* with the **load** command. Each list element is a sub-list with two elements consisting of the name of the file from which the package was loaded and the name of the package. For statically-loaded packages the file name will be an empty string. If *interp* is omitted then information is returned for all packages loaded in any interpreter in the process. To get a list of just the packages in the current interpreter, specify an empty string for the *interp* argument.

info locals *?pattern?*

coreTools Command Reference Index

If *pattern* is not specified, returns a list of all the names of currently-defined local variables, including arguments to the current procedure, if any. Variables defined with the **global**, **upvar** and **variable** commands will not be returned. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for **string match**.

info nameofexecutable

Returns the full path name of the binary file from which the application was invoked. If Tcl was unable to identify the file, then an empty string is returned.

info object *subcommand object ?arg ...*

Returns information about the object, *object*. The *subcommands* are described in **OBJECT INTROSPECTION** below.

info patchlevel

Returns the value of the global variable **tcl_patchLevel**, which holds the exact version of the Tcl library by default.

info procs *?pattern?*

If *pattern* is not specified, returns a list of all the names of Tcl command procedures in the current namespace. If *pattern* is specified, only those procedure names in the current namespace matching *pattern* are returned. Matching is determined using the same rules as for **string match**. If *pattern* contains any namespace separators, they are used to select a namespace relative to the current namespace (or relative to the global namespace if *pattern* starts with ::) to match within; the matching pattern is taken to be the part after the last namespace separator.

info script *?filename?*

If a Tcl script file is currently being evaluated (i.e. there is a call to **Tcl_EvalFile** active or there is an active invocation of the **source** command), then this command returns the name of the innermost file being processed. If *filename* is specified, then the return value of this command will be modified for the duration of the active invocation to return that name. This is useful in virtual file system applications. Otherwise the command returns an empty string.

info sharedlibextension

Returns the extension used on this platform for the names of files containing shared libraries (for example, **.so** under Solaris). If shared libraries are not supported on this platform then an empty string is returned.

info tclversion

Returns the value of the global variable **tcl_version**, which holds the major and minor version of the Tcl library by default.

info vars *?pattern?*

If *pattern* is not specified, returns a list of all the names of currently-visible variables. This includes

coreTools Command Reference Index

locals and currently-visible globals. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for **string match**. *pattern* can be a qualified name like **Foo::option***. That is, it may specify a particular namespace using a sequence of namespace names separated by double colons (::), and may have pattern matching special characters at the end to specify a set of variables in that namespace. If *pattern* is a qualified name, the resulting list of variable names has each matching namespace variable qualified with the name of its namespace. Note that a currently-visible variable may not yet if it has not been set (e.g. a variable declared but not set by **variable**).

CLASS INTROSPECTION

The following *subcommand* values are supported by **info class**:

info class call *class method*

Returns a description of the method implementations that are used to provide a stereotypical instance of *class*'s implementation of *method* (stereotypical instances being objects instantiated by a class without having any object-specific definitions added). This consists of a list of lists of four elements, where each sublist consists of a word that describes the general type of method implementation (being one of **method** for an ordinary method, **filter** for an applied filter, and **unknown** for a method that is invoked as part of unknown method handling), a word giving the name of the particular method invoked (which is always the same as *method* for the **method** type, and for the **unknown** type), a word giving the fully qualified name of the class that defined the method, and a word describing the type of method implementation (see **info class methodtype**).

Note that there is no inspection of whether the method implementations actually use **next** to transfer control along the call chain.

info class constructor *class*

This subcommand returns a description of the definition of the constructor of class *class*. The definition is described as a two element list; the first element is the list of arguments to the constructor in a form suitable for passing to another call to **proc** or a method definition, and the second element is the body of the constructor. If no constructor is present, this returns the empty list.

info class definition *class method*

This subcommand returns a description of the definition of the method named *method* of class *class*. The definition is described as a two element list; the first element is the list of arguments to the method in a form suitable for passing to another call to **proc** or a method definition, and the second element is the body of the method.

coreTools Command Reference Index

info class destructor *class*

This subcommand returns the body of the destructor of class *class*. If no destructor is present, this returns the empty string.

info class filters *class*

This subcommand returns the list of filter methods set on the class.

info class forward *class method*

This subcommand returns the argument list for the method forwarding called *method* that is set on the class called *class*.

info class instances *class ?pattern?*

This subcommand returns a list of instances of class *class*. If the optional *pattern* argument is present, it constrains the list of returned instances to those that match it according to the rules of **string match**.

info class methods *class ?options...?*

This subcommand returns a list of all public (i.e. exported) methods of the class called *class*. Any of the following *options* may be specified, controlling exactly which method names are returned:

-all

If the **-all** flag is given, the list of methods will include those methods defined not just by the class, but also by the class's superclasses and mixins.

-private

If the **-private** flag is given, the list of methods will also include the private (i.e. non-exported) methods of the class (and superclasses and mixins, if **-all** is also given).

info class methodtype *class method*

This subcommand returns a description of the type of implementation used for the method named *method* of class *class*. When the result is **method**, further information can be discovered with **info class definition**, and when the result is **forward**, further information can be discovered with **info class forward**.

info class mixins *class*

This subcommand returns a list of all classes that have been mixed into the class named *class*.

info class subclasses *class ?pattern?*

This subcommand returns a list of direct subclasses of class *class*. If the optional *pattern* argument is present, it constrains the list of returned classes to those that match it according to the rules of **string match**.

info class superclasses *class*

This subcommand returns a list of direct superclasses of class *class* in inheritance precedence order.

info class variables *class*

This subcommand returns a list of all variables that

coreTools Command Reference Index

have been declared for the class named *class* (i.e. that are automatically present in the class's methods, constructor and destructor).

OBJECT INTROSPECTION

The following *subcommand* values are supported by **info object**:

info object call *object method*

Returns a description of the method implementations that are used to provide *object*'s implementation of *method*. This consists of a list of lists of four elements, where each sublist consists of a word that describes the general type of method implementation (being one of **method** for an ordinary method, **filter** for an applied filter, and **unknown** for a method that is invoked as part of unknown method handling), a word giving the name of the particular method invoked (which is always the same as *method* for the **method** type, and for the **unknown** type), a word giving what defined the method (the fully qualified name of the class, or the literal string **object** if the method implementation is on an instance), and a word describing the type of method implementation (see **info object methodtype**).

Note that there is no inspection of whether the method implementations actually use **next** to transfer control along the call chain.

info object class *object ?className?*

If *className* is unspecified, this subcommand returns class of the *object* object. If *className* is present, this subcommand returns a boolean value indicating whether the *object* is of that class.

info object definition *object method*

This subcommand returns a description of the definition of the method named *method* of object *object*. The definition is described as a two element list; the first element is the list of arguments to the method in a form suitable for passing to another call to **proc** or a method definition, and the second element is the body of the method.

info object filters *object*

This subcommand returns the list of filter methods set on the object.

info object forward *object method*

This subcommand returns the argument list for the method forwarding called *method* that is set on the object called *object*.

info object isa *category object ?arg?*

This subcommand tests whether an object belongs to a particular category, returning a boolean value that indicates whether the *object* argument meets the criteria for the category. The supported categories are:

info object isa class *object*

This returns whether *object* is a class (i.e. an

coreTools Command Reference Index

instance of **oo::class** or one of its subclasses).

info object isa metaclass *object*

This returns whether *object* is a class that can manufacture classes (i.e. is **oo::class** or a subclass of it).

info object isa mixin *object class*

This returns whether *class* is directly mixed into *object*.

info object isa object *object*

This returns whether *object* really is an object.

info object isa typeof *object class*

This returns whether *class* is the type of *object* (i.e. whether *object* is an instance of *class* or one of its subclasses, whether direct or indirect).

info object methods *object ?option...?*

This subcommand returns a list of all public (i.e. exported) methods of the object called *object*. Any of the following *options* may be specified, controlling exactly which method names are returned:

-all

If the **-all** flag is given, the list of methods will include those methods defined not just by the object, but also by the object's class and mixins, plus the superclasses of those classes.

-private

If the **-private** flag is given, the list of methods will also include the private (i.e. non-exported) methods of the object (and classes, if **-all** is also given).

info object methodtype *object method*

This subcommand returns a description of the type of implementation used for the method named *method* of object *object*. When the result is **method**, further information can be discovered with **info object definition**, and when the result is **forward**, further information can be discovered with **info object forward**.

info object mixins *object*

This subcommand returns a list of all classes that have been mixed into the object named *object*.

info object namespace *object*

This subcommand returns the name of the internal namespace of the object named *object*.

info object variables *object*

This subcommand returns a list of all variables that have been declared for the object named *object* (i.e. that are automatically present in the object's methods).

info object vars *object ?pattern?*

This subcommand returns a list of all variables in the private namespace of the object named *object*. If the optional *pattern* argument is given, it is a filter (in

coreTools Command Reference Index

the syntax of a **string match** glob pattern) that constrains the list of variables returned. Note that this is different from the list returned by **info object variables**; that can include variables that are currently unset, whereas this can include variables that are not automatically included by any of *object's* methods (or those of its class, superclasses or mixins).

EXAMPLES

This command prints out a procedure suitable for saving in a Tcl script:

```
proc printProc {procName} {
    set result [list proc $procName]
    set formal {}
    foreach var [info args $procName] {
        if {[info default $procName $var def]} {
            lappend formal [list $var $def]
        } else {
            # Still need the list-quoting because variable
            # names may properly contain spaces.
            lappend formal [list $var]
        }
    }
    puts [lappend result $formal [info body $procName]]
}
```

EXAMPLES WITH OBJECTS

Every object necessarily knows what its class is; this information is trivially extractable through introspection:

```
oo::class create c
c create o
puts [info object class o]
    -> prints "::c"
puts [info object class c]
    -> prints "::oo::class"
```

The introspection capabilities can be used to discover what class implements a method and get how it is defined. This procedure illustrates how:

```
proc getDef {obj method} {
    foreach inf [info object call $obj $method] {
        lassign $inf calltype name locus methodtype
        # Assume no forwards or filters, and hence no $calltype|
        # or $methodtype checks...
        if {$locus eq "object"} {
            return [info object definition $obj $name]
        } else {
            return [info class definition $locus $name]
        }
    }
    error "no definition for $method"
}
```

coreTools Command Reference Index

This is an alternate way of looking up the definition; it is implemented by manually scanning the list of methods up the inheritance tree. This code assumes that only single inheritance is in use, and that there is no complex use of mixed-in classes (in such cases, using **info object call** as above is the simplest way of doing this by far):

```
proc getDef {obj method} {  
    if {$method in [info object methods $obj]} {  
        # Assume no forwards  
        return [info object definition $obj $method]  
    }  
    set cls [info object class $obj]  
    while {$method ni [info class methods $cls]} {  
        # Assume the simple case  
        set cls [lindex [info class superclass $cls] 0]  
        if {$cls eq ""} {  
            error "no definition for $method"  
        }  
    }  
    # Assume no forwards  
    return [info class definition $cls $method]  
}
```

SEE ALSO

global(n), oo::class(n), oo::define(n), oo::object(n),
proc(n), self(n), tcl_library(n), tcl_patchLevel(n),
tcl_version(n)

KEYWORDS

command, information, interpreter, introspection,
level, namespace, object, procedure, variable

install_filegroup

Install the specified filegroup from the Bill of Materials

Syntax

```
string install_filegroup [-substitute when] filegroup
      string when
      string filegroup
```

Parameters

	Specifies when to perform text substitution. The value that you specify for <when> must match the SubstituteInFile attribute subscript for the files in the file group that require text substitution. The default value for <when> is OnWrite (perform text substitution when files in this file group are written to disk).
-substitute when	
filegroup	The name of the file group to install.

Description

The install_filegroup command unloads the specified file group from the Bill of Materials (BoM) and installs the files in the file group according to the attributes set on the file group and its members (files).

For example, you can use install_filegroup in the Tcl command procedures for a custom activity to install the files needed for that activity when the user executes the activity.

Files in the file group may contain reuse-pragmas that enable text substitution when the file is written. coreConsultant uses text substitution to set user-specified parameter values in HDL source files as part of the Specify Configuration activity.

You can also use text substitution in files that are installed as part of custom activity execution. For example, if your coreKit includes configurable testbench/test suite files, you can use text substitution to automatically configure the testbench files according to user-specified values for activity parameters.

There are two things that determine when text substitution occurs: the value of the SubstituteInFile attribute on files and file groups, and the value of the -substitute option on the install_filegroup command. For

text substitution to occur, the SubstituteInFile subscript that is set to true must match the value of the install_filegroup -substitute option. For example, if SubstituteInFile [OnWrite] is true on a file and you specify "install_filegroup -substitute OnWrite" when installing the associated file group, then text substitution occurs on the file each time coreConsultant writes the file to disk.

The default subscript for SubstituteInFile and the default value for the install_filegroup -substitute option is OnWrite. Therefore, if you set SubstituteInFile (no subscript specified) to true on a file and omit the -substitute option with install_filegroup, coreConsultant will perform text substitution on the file each time it writes the file to disk, including during coreKit installation and each time coreConsultant writes the file to a user workspace.

If you do not want to perform text substitution during coreKit installation (for example, if the user-specified parameter values are not available), specify something other than OnWrite for the SubstituteInFile subscript and the install_filegroup -substitute option. You can specify any text string other than OnWrite. For example, if you are creating a command procedure for a custom activity that generates a configured set of testbench files, you may not want to perform text substitution during coreKit installation because the user-specified testbench configuration parameter values are not yet available. In such a case, use the same non-OnWrite value for the SubstituteInFile subscript and install_filegroup -substitute option. For example, set SubstituteInFile[ConfigTB] to true and specify "install_filegroup -substitute ConfigTB" in the command procedure that generates the configured testbench files.

Examples

To install a file group and use the default (OnWrite) text substitution method, use parameter_conditional_text with reuse-pragma in the file(s) that require text substitution (my_tb_file.txt in this example) and then set SubstituteInFile on files that require text substitution:

```
set_attribute -attr SubstituteInFile -value true my_tb_file.txt
```

To add the file(s) to a file group:

```
add_files_to_filegroup testbench -files  
my_tb_file.txt -install test_bench
```

To install the file group as part of a custom activity command procedure, include the following command in the custom command procedure:

```
install_filegroup testbench
```

The above set of commands cause text substitution to occur in my_tb_file.txt during coreKit installation and each time coreConsultant writes my_tb_file.txt to a user workspace. To prevent text substitution during coreKit installation and only perform text substitution when the

user specifies the desired testbench configuration as part of a custom activity, use `parameter_conditional_text` with `reuse-pragma` in the file(s) that require text substitution (my_tb_file.txt in this example) and set `SubstituteInFile` on files that require text substitution:

```
set_attribute -attr SubstituteInFile -subscript ConfigTB -value true  
my_tb_file.txt
```

To add the file(s) to a file group:

```
add_files_to_filegroup testbench -files  
my_tb_file.txt -install test_bench
```

To install the file group as part of a custom activity command procedure, include the following command in the custom command procedure:

```
install_filegroup -substitute ConfigTB testbench
```

See Also

[add_files_to_filegroup \(2\)](#), [IncludeIf \(2\)](#), [set_attribute \(2\)](#),
[SubstituteInFile \(3\)](#)

instances_of

Get instances of given design name, recursively.

Syntax

```
string instances_of design_name  
    string design_name
```

Parameters

<code>design_name</code>	Design name of instances to be returned
--------------------------	---

Description

This command returns the full hierarchical path to each instance of the specified design name. Only instances visible from the current hierarchy (recursively) are returned.

Note that calls occurring after completion of SpecifySubsystem will generally want to pass a regular expression to account for design name prefixing.

Examples

Find all instances (after SpecifyConfiguration) of DWC_pcie_ctl.

```
set instances [instances_of .*DWC_pcie_ctl]
```

See Also

instantiate_component

Instantiate a component in the subsystem.

Syntax

```
string instantiate_component -name <name> [-consumer <componentName>]
[-noexport] [-noprefix] [-instance <name value pairs>] [-noshare]
[-connection <connection info>] [-prime_profile <None> | <Prime Profile
tag> | <External File> ] [-enable_analysis_mode] component
    string <name>
    string <componentName>
    string <name value pairs>
    list <connection info>
    string <None> | <Prime Profile tag> | <External File>
    string component
```

Parameters

-name <name>	New component cell name
-consumer <componentName>	Associated consumer for an embedded 'consumer-monitor' interface This option is only specified for components which act in the role of monitoring a particular consumer. The component name specified with the -consumer option is assumed to be the name of the component being monitored by this consumer monitor.
-noexport	Do not auto export interfaces Set the PreventDesignPrefix attribute for this and subsequent instances of the component. This option prevents prefixing of designs within the instantiated component.
-noprefix	Prefixing is done by default for components that are configured using 'defines because the RTL must be unique for each instance. Use this option to prevent the prefixing. This is required if the component will be shared via duplicate_component (multiple instances sharing the same RTL).
-instance <name value pairs>	Instantiation parameter configuration values.
-noshare	Don't auto share if sharing is enabled

coreTools Command Reference Index

-connection <connection info>	List of connection info (<new input port name> <new output port name> <connection pin name list>) that need to reconnect via the new component
-prime_profile <None> <Prime Profile tag> <External File>	Apply a packaged or an external Prime Profile or explicitly select <None> to skip applying one. When Prime Profile is specified with command, enables analysis mode, ignored otherwise.
-enable_analysis_mode	The name of the component to instantiate. Available components search path can be specified in the SearchPath parameter of the AddSubsystemComponents activity or by specifying a component XML file. The component in the search path are identified by using the following attributes {Vendor Library Name Version}. The component in the search path could be coreKit component or IP-XACT components.
component	

Description

This command is used to add components to the subsystem in batch mode. It is equivalent to the 'Add Component' dialog within the schematic window of the Add Subsystem Components activity. The installed components is searched using the SearchPath parameter of the AddSubsystemComponents activity. Any referenced consumers must already exist before this command is called.

This command creates a cell for the new component, and then creates a coreConsultant-like workspace for the component, underneath the coreAssembler workspace. All files associated with the core are either linked or copied into the workspace, just as in coreConsultant.

To configure the added instance, parameter configuration info can be provided as name/value pairs with -instance option.

This command can also be used to split some existing connections and reconnect them via the added component. The -connection option is used to provide the information of the connected pins which need to be reconnected through new component pins. If the command is called with -connection option after Specify Subsystem activity is completed, it will not incomplete the activity as it will work in silent mode.

For the -connection option, it supports list of <connection info> where each <connection info> item need to follow specific input format '{{<New Input Port Name>} {<New Output Port Name>} {<Connection Pin Name List>}}'.

coreTools Command Reference Index

- The <New Input Port Name> will contain single input port name of the new component.
- The <New Output Port Name> will contain single output port name of the new component.
- The <connection pin name list> can contain
 - ◆ single driver pin name.
 - ◆ single/multiple receiver pin names of same bit size driven by the same driver.

The driver pin will be reconnected to new input pin and the new output pin will drive all the connected receivers.

Each pin name of <connection pin name list> should be a full hierarchical name.

It supports name of single bit pin or bussed pin. But for bussed pin, the driver can be connected to multiple receivers from same or different bit index. For the receiver, it should not be connected to multiple drivers at same bit. But different bits can be driven by different drivers. It also supports partially connected to open or any constant value for any pin and that is not going to change after the re-connection.

This feature is in ALPHA development.

To split the connections through different ports of new component(if supported), a separate entry has to be added in <connection info> list for each connection.

With -connection option, no other optional options are supported except -instance option. Other options like -noshare , -noexport, -noprefix or -consumer will throw error if provided.

Applying Prime Profiles

Prime Profiles can be applied on component instantiation with the -prime_profile option.

- If allowed by the component, a '<None>' argument can be provided to skip Prime Profile application.
- For packaged Prime Profiles, the Prime Profile tag can be provided.
- For external Prime Profiles, the path to the external Prime Profile file can be provided.

Whether a packaged, external or no Prime Profile is being applied, is inferred from the string provided. If the string ends in '.pp.gz', it is assumed to be an external Prime Profile. If the string is '<None>', Prime Profile application is skipped. In all other cases, the string is assumed to be a tag of a packaged Prime Profile for the component.

When GUI is up, adding a component in a coreAssembler schematic window invokes `instantiate_component` command. If the component has valid Prime Profiles (that do not use triggers) packaged, a Prime Profile selection GUI will appear. On pressing 'OK', the selected profile will be applied immediately.

Examples

To instantiate a 2.04a version of DW APB Rap component using the {Vendor Library Name Version} available in the search path. The component could be a packaged component (installed coreKit) or IP-XACT component.

```
coreAssembler> instantiate_component -name i_rap {Synopsys
DesignWareLibrary DW_apb_rap 2.04a}
```

To instantiate a IP-XACT component using a component XML (mycore.xml)

```
coreAssembler> instantiate_component -name i_mycore
/u/prj/spirit/component/mycore.xml
```

To add a new AHB component:

```
coreAssembler> instantiate_component -name AHBinst AHB
```

To add a new component with design prefixing prevented for all instances:

```
coreAssembler> instantiate_component -name compA_slave compA_slave
-noprefix
```

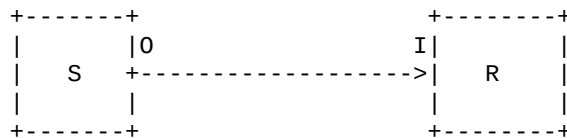
To configure the new component use `-instance` option with name value pairs.

```
coreAssembler> instantiate_component -name AHBinst AHB -instance
{{<param1> 3} {<param2> 2}}
```

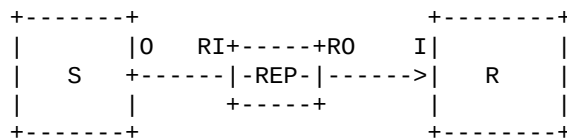
To split existing connections and reconnect through new component, check the below example for command usage.

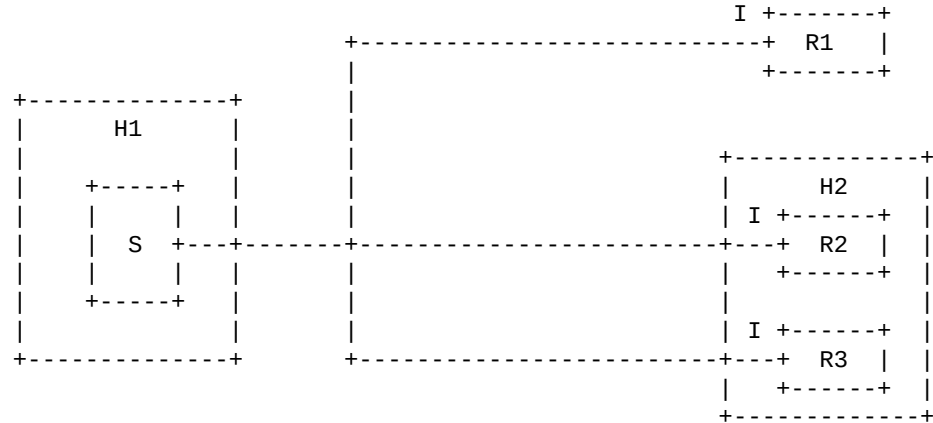
For component REP, input (RI) and output (RO) port sizes are same as S/O.

CASE 1 :: single driver is connected to single receiver



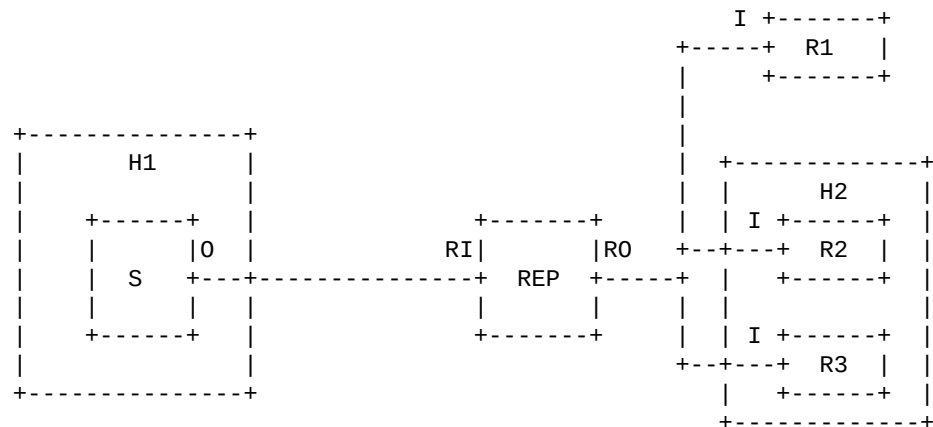
```
coreAssembler> instantiate_component -name REP REP -connection { {RI}
{RO} {S/O} }
```



CASE 2:: Single driver is connected to multiple receivers with/without hierarchy

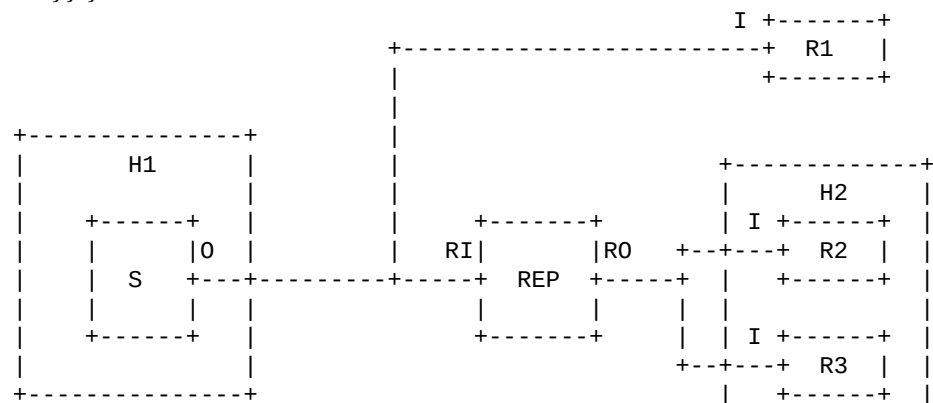
Scenario :: i To add new component between driver and all receivers, use the source pin or all load pins.

```
coreAssembler> instantiate_component -name REP REP -connection { {{RI}
{RO} {/H1/S/O}} }
```



Scenario :: ii To add new component between R2 and R3 only, specify particular load pins.

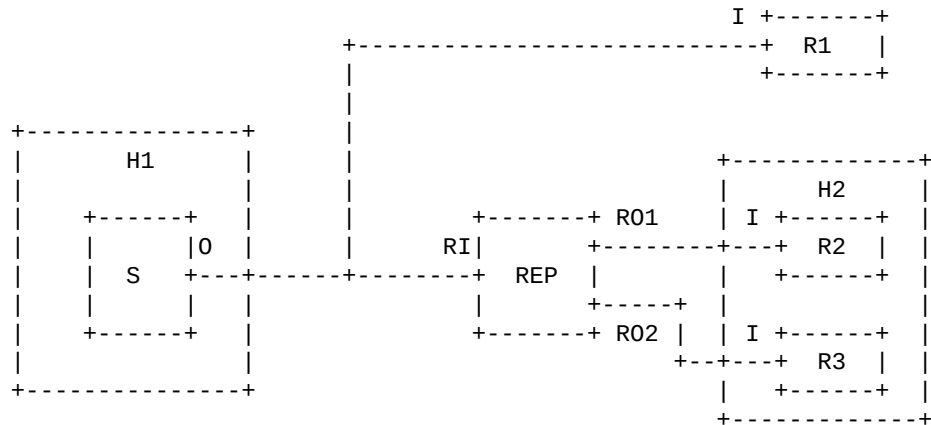
```
coreAssembler> instantiate_component -name REP REP -connection { {{RI}
{RO} {/H2/R2/I /H2/R3/I}} }
```



+-----+

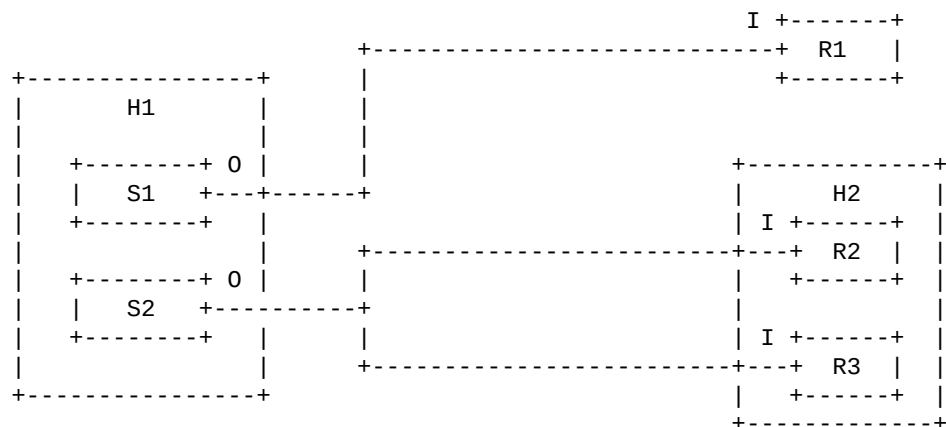
Scenario :: iii If new component has single input but multiple output ports , then to connect pin with different output ports, add a separate entry for each connection in the connection info list.

```
coreAssembler> instantiate_component -name REP REP -connection { {{RI}
{RO1} {/H2/R2/I}} {{RI} {RO2} {/H2/R3/I}} }
```



CASE 3:: Multiple driver pins of same or different components are connected

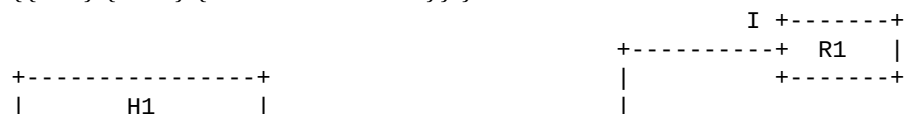
If new component has multiple input and output ports then connections can be made through single component. Check the usage as follow.



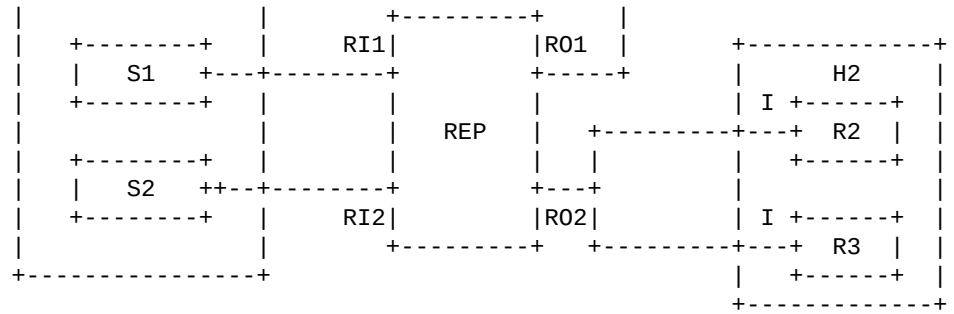
For component REP,

- input RI1 and output RO1 port sizes are same as S1/O
- input RI2 and output RO2 port sizes are same as S2/O

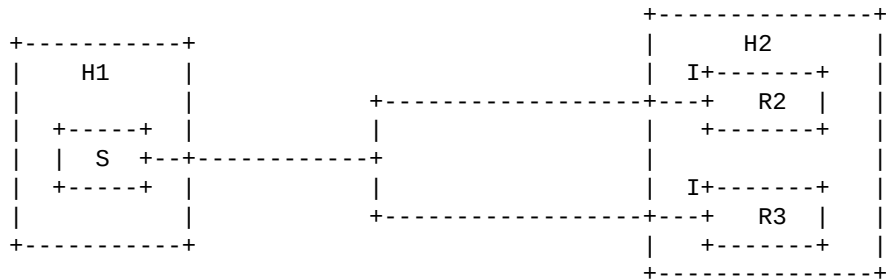
```
coreAssembler> instantiate_component -name REP REP -connection { {{RI1}
{RO1} {/H1/S1/O}} {{RI2} {RO2} {/H2/R2/I /H2/R3/I}} }
```



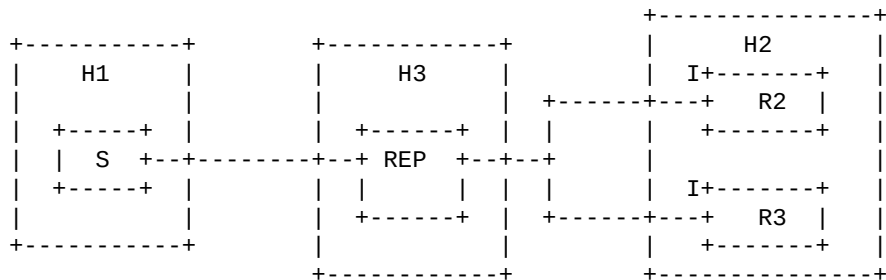
coreTools Command Reference Index



CASE 4:: If the connections exist accross hierarchy level, you call the command from diffrent hier level

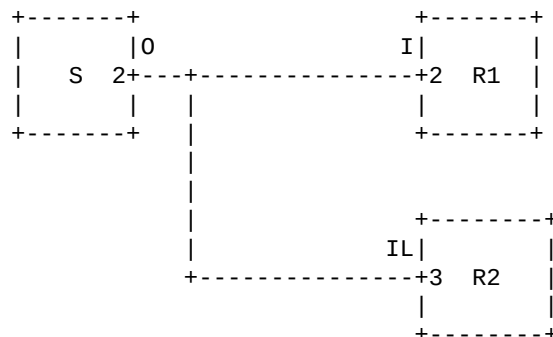


```
coreAssembler> set_current_component /H3
coreAssembler> instantiate_component -name REP REP -connection { {{RI}
{RO} {/H1/S/O}} }
```



Partial bussed pin connection:: Connection is partially connected to constant

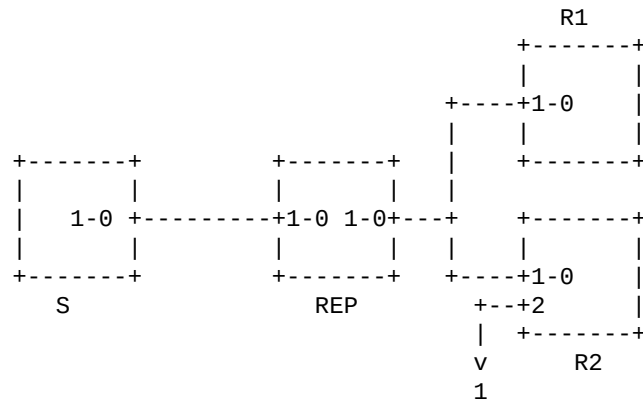
- {S/O[1:0]} {R1/I[1:0]} {R2/IL[1:0]} are connected.
- R2/IL[2] is logic one.



coreTools Command Reference Index

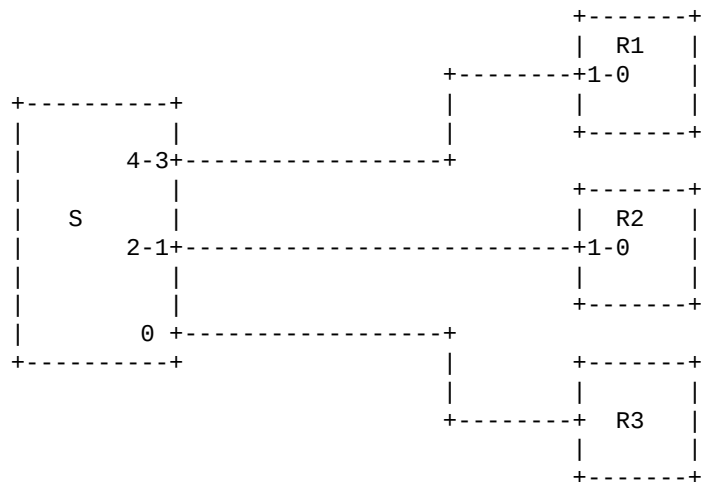
For component REP, input RI and output RI port sizes are same as S1/O (2 bits).

```
coreAssembler> instantiate_component -name REP REP -connection { {{RI}
{RO} {S/O}} }
```



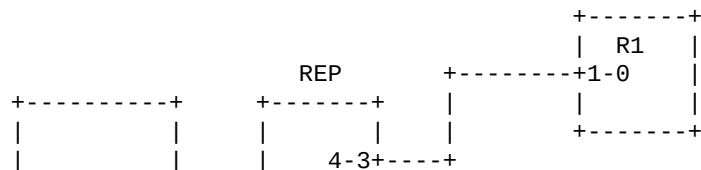
Multi connection:: Different index bits are connected to different sources

- {S/O[4:3]} {R1/I[1:0]} are connected
- {S/O[2:1]} {R2/I[1:0]} are connected
- {S/O[0]} {R3/I} are connected

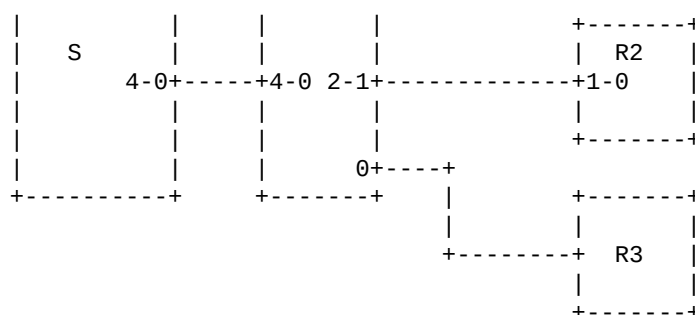


For component REP, input RI and output RI port sizes are same as S1/O (5 bits).

```
coreAssembler> instantiate_component -name REP REP -connection { {{RI}
{RO} {S/O}} }
```

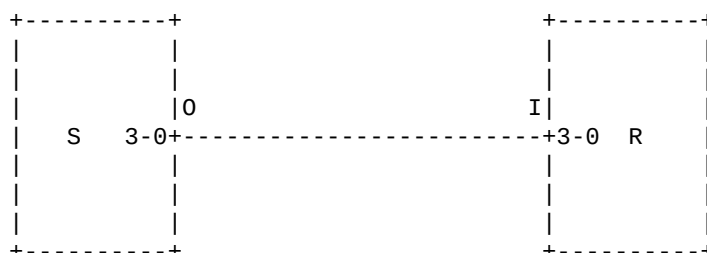


coreTools Command Reference Index



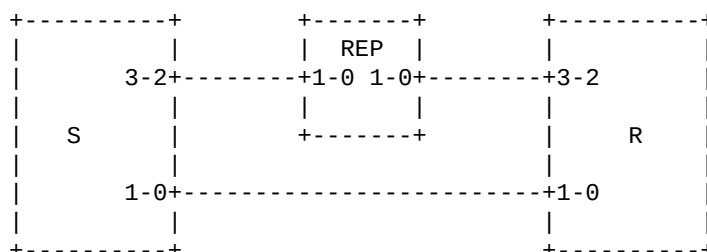
Partial range support:: Reconnect only a particular range.

- {S/O[3:0]} {R/I[3:0]} are connected.



For component REP, input RI and output RI port sizes are of 2 bits.

```
coreAssembler> instantiate_component -name REP REP -connection { {{RI}
{RO} {S/O[3:2]}} }
```



Applying Prime Profiles

Applying a packaged Prime Profile

```
coreAssembler> instantiate_component -prime_profile "LockRamSize" -name
m0 -noauto mycore
```

Skip Prime Profile application

```
coreAssembler> instantiate_component -prime_profile "<None>" -name m1
-noauto mycore
```

Applying an External Prime Profile

```
coreAssembler> instantiate_component -prime_profile
"/path/to/external/prime/profile/ExternalProfile.pp.gz" -name m2 -noauto
mycore
```

Apply a Prime Profile with analysis mode

```
coreAssembler> instantiate_component -prime_profile "LockRamSize"
-enable_analysis_mode -name m3 -noauto mycore
```

See Also

import_component (2), duplicate_component (2), remove_component (2),
all_components (2), instantiate_dut (2) set_prime_profile (2)

`instantiate_dut`

Instantiate a DUT into the testbench subsystem

Syntax

`string instantiate_dut -name <name> [-workspace <wsPath>] [-spirit
<xmlFile>]``string <name>
string <wsPath>
string <xmlFile>`

Parameters

<code>-name <name></code>	New component cell name
<code>-workspace <wsPath></code>	The coreAssembler workspace to instantiate This option specifies a coreAssembler workspace. The information that is imported is from the last successful completion of the GenerateSubsystemRTL activity.
<code>-spirit <xmlFile></code>	The path to SPIRIT component to instantiate

Description

This command instantiates a SPIRIT component into the current testbench workspace. If the component is a hierarchical component, then the entire hierarchy is visible within coreAssembler, and interface and consumer monitors may be attached to interfaces within that DUT.

Examples

To put a DUT into the subsystem:

```
coreAssembler> instantiate_dut -name dut -spirit ../spirit/core.xml
```

See Also

[instantiate_component \(2\)](#)

instantiate_interface

Instantiate an exported interface without any attachment.

Syntax

```
string instantiate_interface -name <newName> -type <type> [-interface
<definition>] [-abstraction <abstraction VLNV>] [-file <file>] [-vlnv
<vlnv list>] [-prefix] [-format <format>] [-channel] [-leaveUnconnected]
    string <newName>
    string <type>
    string <definition>
    string <abstraction VLNV>
    string <file>
    string <vlnv list>
    string <format>
```

Parameters

-name <newName>	New exported interface instance name
-type <type>	The type of the exported interface instance (Values: provider, consumer, master, slave, system, mirroredMaster, mirroredSlave, mirroredSystem)
-interface <definition>	The name of the interface definition to be instantiated
-abstraction <abstraction VLNV>	The abstraction for the new interface instance
-file <file>	The name of the TCL file defining the interface definition
-vlnv <vlnv list>	SPIRIT {vendor library name version}
-prefix	Always prefix subsystem ports with the interface name, even 'common' ports
-format <format>	The format to use for prefixing exported interfaces.
-channel	Symmetric SPIRIT interface to be connected to a channel
-leaveUnconnected	Leave interface unconnected

Description

This command can be used to instantiate a 'stand-alone' exported interface. This is an exported interface which is not (yet) associated with any instantiated component. The exported interface can later be connected to any components that are added to the subsystem provided that

they have the proper interface type.

The interface can be defined in one of two ways. First, by defining the name of the interface definition and the file containing the commands to define it. This is done by using the `-interface` and `-file` options, respectively. Secondly, by defining the SPIRIT VLNV (vendor library name version) of the SPIRIT bus definition which corresponds to the desired coreAssembler interface definition.

NOTE - for the moment, if you use the `-vlnv` option, the SPIRIT bus definition must contain explicit mapping information to a Synopsys interface definition.

The legal values for the `-type` option depend on which method is used to determine the interface definition. If you specify `-file` and `-interface`, then the valid values for `-type` are: `provider`, and `consumer`. If you specify the `-vlnv` option instead, the value values for `-type` are: `master`, `slave`, `system`, `mirroredMaster`, `mirroredSlave`, and `mirroredSystem`.

The file name specified with the `-file` option should specify a file which contains the TCL commands required to define the interface definition specified via the `-interface` option.

The `-channel` option is used only in conjunction with symmetric busses as defined by the SPIRIT consortium. If you are exporting a SPIRIT master interface of a symmetric bus and expect to connect it to a `mirroredMaster` interface instead of a slave interface, the `-channel` option must be used. Otherwise coreAssembler assumes that symmetric SPIRIT interfaces represent direct master to slave connections.

Interfaces added with the `instantiate_interface` command can be removed using the `remove_exported_interface` command.

Examples

Add a stand-alone exported interface for HCLK:

```
coreAssembler> instantiate_interface -name HCLK -type provider \
-interface DW-SoC::AHB-Clock -file ~/DW-interface_definitions.tcl.inc
```

Add a stand-alone interface associated with a simple SPIRIT bus definition:

```
coreAssembler> instantiate_interface -name Master -type master \
-vlnv {myVendor myLibrary myBus 1.0}
```

Instantiate an unconnected interface associated with a definition:

```
coreAssembler> instantiate_interface -leaveUnconnected -noauto \
-name AXIClock -interface DW-SoC::AXI-Clock -type provider
```

See Also

export_interface (2), remove_exported_interface (2),
translate_interface_connection (2)

instantiate_solution

Initiate creation of a DesignWare IP Solution

Syntax

```
string instantiate_solution [-workspace <name>] [-design <design>]
[-query] [-family <family>] [-version <version>] [-syscfg
<name-value-list>] [-noplugins] solution_name
    string <name>
    string <design>
    string <family>
    string <version>
    string <name-value-list>
    string solution_name
```

Parameters

-workspace <name>	Name for the coreAssembler workspace
-design <design>	Name for the top-level design
-query	Query user for missing information
-family <family>	Solution family name
-version <version>	Version of solution family
-syscfg <name-value-list>	System configuration values
-noplugins	Do not automatically load subsystem plugins This option disables the automatic loading of plugins associated with the solution. Plugins can be loaded at a later time using the load_plugin command.
solution_name	Solution name

Description

The instantiate_solution command is used to instantiate a DesignWare IP Solution product. These are multi-IP subsystems available from a DESIGNWARE_HOME installation within the \$DESIGNWARE_HOME/solutions directory. Solutions are instantiated in a level of hierarchy in coreAssembler, and completely define that level of hierarchy.

The -family, -version, and "solution_name" options indicate which solution is to be instantiated. The -family and -version options are only required when multiple solutions exist with the same name. To see the list of available solutions in batch mode, use the report_solutions command.

The `-workspace` and `-design` options are available to override the default names for the workspace and design, respectively. Alternatively, if using the tool in GUI mode, the `-query` option can be used to enter these details via a dialog box.

The `-syscfg` option is used to pass system configuration parameter values to the solution. The value for this option is a list of alternating variable names and values. The variable names must correspond to variable names used in the definition of the solution.

Examples

Instantiate a solution named "Controller and PHY" with all default details:

```
instantiate_solution "Controller and PHY"
```

Instantiate a solution named "Controller and PHY". Override the name of the workspace and design, and pass system configuration details:

```
instantiate_solution -workspace ws_example -design my_design \  
-syscfg "NLANES 4" "Controller and PHY"
```

See Also

[report_solutions \(2\)](#) [define_solution \(2\)](#) [load_plugin \(2\)](#)

interfaceDefn
interfaceDefn item

Description

A documented definition of a set of ports and parameters which defines a connection point to a component. Interfaces are typically 'provided' by one component and 'consumed' by one or more other components. For example, a bus IP might provide a master interface while each master component in the sub-system would consume this interface.

See Also

[create_interface \(2\)](#), [set_interface_attribute \(2\)](#),
[get_interface_attribute \(2\)](#), [create_interface_parameter \(2\)](#),
[report_interfaces \(2\)](#), [interfaceInstance \(3\)](#), [interfacePort \(3\)](#)

Supported Attributes

[AutoConnectWhen \(3\)](#), [ComponentOffItem \(3\)](#), [ConnectionDialogCmd \(3\)](#),
[Description \(3\)](#), [DocInclude \(3\)](#), [InterfaceLabel \(3\)](#), [InterfaceTypeName \(3\)](#),
[IsAddressable \(3\)](#), [Label \(3\)](#), [Name \(3\)](#), [Sequence \(3\)](#), [ShowRoute \(3\)](#),
[SlotNumberOffset \(3\)](#), [SlotWidth \(3\)](#), [SpiritFile \(3\)](#), [SpiritInterfaceType \(3\)](#),
[SpiritLibrary \(3\)](#), [SpiritName \(3\)](#), [SpiritVendor \(3\)](#), [SpiritVersion \(3\)](#),
[SymmetricBus \(3\)](#), [TypeName \(3\)](#), [Visible \(3\)](#)

[API Index](#) [Command Reference](#)

interfaceInstance
interfaceInstance item

Description

Instantiation of an interfaceDefn.

See Also

[create interface instance \(2\)](#), [remove interface instance \(2\)](#),
[interfaceDefn \(3\)](#), [interfacePort \(3\)](#)

Supported Attributes

[Abstraction \(3\)](#), [AddressSpaceRef \(3\)](#), [AssociationComplete \(3\)](#),
[AutoConnectWhen \(3\)](#), [BaseAddress \(3\)](#), [BitsInLAU \(3\)](#), [Bridge \(3\)](#), [Channel](#)
(3), [ComponentOfItem \(3\)](#), [ConnectToExportedInstance \(3\)](#),
[ConnectionDialogCmd \(3\)](#), [ConnectionRequired \(3\)](#), [Description \(3\)](#),
[DocInclude \(3\)](#), [Endian \(3\)](#), [GlobalSlotNumber \(3\)](#), [GroupName \(3\)](#),
[InterfaceGroup \(3\)](#), [InterfaceLabel \(3\)](#), [InterfaceType \(3\)](#),
[InterfaceTypeName \(3\)](#), [IsAddressable \(3\)](#), [IsConnected \(3\)](#),
[IsDanglingIntfAllowed \(3\)](#), [Label \(3\)](#), [LockedInTemplate \(3\)](#), [MemoryMap](#)
(3), [Name \(3\)](#), [ReadOnlyInterface \(3\)](#), [Sequence \(3\)](#), [ShowRoute \(3\)](#),
[SlotNumber \(3\)](#), [SlotNumberOffset \(3\)](#), [SlotOrder \(3\)](#), [SlotWidth \(3\)](#),
[SolutionItemPhase \(3\)](#), [SpiritContainer \(3\)](#), [SpiritFile \(3\)](#),
[SpiritInterfaceType \(3\)](#), [SpiritLibrary \(3\)](#), [SpiritName \(3\)](#), [SpiritVendor](#)
(3), [SpiritVersion \(3\)](#), [SplitInterfaceMembers \(3\)](#), [SplitInterfaceName](#)
(3), [SymbolPinPrefix \(3\)](#), [SymbolPinSide \(3\)](#), [TargetConnectionBoundary](#)
(3), [TypeName \(3\)](#), [UniqueConsumerConnections \(3\)](#), [Utilization \(3\)](#),
[VipConnection \(3\)](#), [Visible \(3\)](#),

[API Index](#) [Command Reference](#)

interfacePort
interfacePort item

Description

An interface definition consists of a set of interfacePorts, along with parameters and a set of attributes set on those objects to define a connection point to a component.

See Also

[create interface port \(2\)](#), [interfaceDefn \(3\)](#), [interfaceInstance \(3\)](#)

Supported Attributes

[Abstraction \(3\)](#), [AllowOpenProvider \(3\)](#), [AssociationComplete \(3\)](#),
[BusAlignment \(3\)](#), [ComponentOfItem \(3\)](#), [DefaultConstantPort \(3\)](#),
[Description \(3\)](#), [DocInclude \(3\)](#), [InterfaceIsUsed \(3\)](#), [InterfaceLink \(3\)](#),
[InterfaceSize \(3\)](#), [Label \(3\)](#), [LogicalLeft \(3\)](#), [LogicalRight \(3\)](#), [Name](#) _____
(3), [Optional \(3\)](#), [OptionalAssociation \(3\)](#), [PhysicalLeft \(3\)](#), _____
[PhysicalRight \(3\)](#), [RequiredExportDirection \(3\)](#), [RequiredPortDirection](#) _____
(3), [Sequence \(3\)](#), [SymmetricBusLink \(3\)](#), [TypeName \(3\)](#), [UsedOnInstance](#) _____
(3),

NAME

`interp` - Create and manipulate Tcl interpreters

SYNOPSIS

`interp subcommand ?arg arg ...?`

DESCRIPTION

This command makes it possible to create one or more new Tcl interpreters that co-exist with the creating interpreter in the same application. The creating interpreter is called the *parent* and the new interpreter is called a *child*. A parent can create any number of children, and each child can itself create additional children for which it is parent, resulting in a hierarchy of interpreters.

Each interpreter is independent from the others: it has its own name space for commands, procedures, and global variables. A parent interpreter may create connections between its children and itself using a mechanism called an *alias*. An *alias* is a command in a child interpreter which, when invoked, causes a command to be invoked in its parent interpreter or in another child interpreter. The only other connections between interpreters are through environment variables (the `env` variable), which are normally shared among all interpreters in the application, and by resource limit exceeded callbacks. Note that the name space for files (such as the names returned by the `open` command) is no longer shared between interpreters. Explicit commands are provided to share files and to transfer references to open files from one interpreter to another.

The `interp` command also provides support for *safe* interpreters. A safe interpreter is a child whose

coreTools Command Reference Index

functions have been greatly restricted, so that it is safe to execute untrusted scripts without fear of them damaging other interpreters or the application's environment. For example, all IO channel creation commands and subprocess creation commands are made inaccessible to safe interpreters. See **SAFE INTERPRETERS** below for more information on what features are present in a safe interpreter. The dangerous functionality is not removed from the safe interpreter; instead, it is *hidden*, so that only trusted interpreters can obtain access to it. For a detailed explanation of hidden commands, see **HIDDEN COMMANDS**, below. The alias mechanism can be used for protected communication (analogous to a kernel call) between a child interpreter and its parent. See **ALIAS INVOCATION**, below, for more details on how the alias mechanism works.

A qualified interpreter name is a proper Tcl list containing a subset of its ancestors in the interpreter hierarchy, terminated by the string naming the interpreter in its immediate parent. Interpreter names are relative to the interpreter in which they are used. For example, if *is* is a child of the current interpreter and it has a child which in turn has a child the qualified name of *in* is the list

The **interp** command, described below, accepts qualified interpreter names as arguments; the interpreter in which the command is being evaluated can always be referred to as **{}** (the empty list or string). Note that it is impossible to refer to a parent (ancestor) interpreter by name in a child interpreter except through aliases. Also, there is no global name by which one can refer to the first interpreter created in an application. Both restrictions are motivated by safety concerns.

THE INTERP COMMAND

The **interp** command is used to create, delete, and manipulate child interpreters, and to share or transfer channels between interpreters. It can have any of several forms, depending on the *subcommand* argument:

interp alias srcPath srcToken

Returns a Tcl list whose elements are the *targetCmd* and *args* associated with the alias represented by *srcToken* (this is the value returned when the alias was created; it is possible that the name of the source command in the child is different from *srcToken*).

interp alias srcPath srcToken {}

Deletes the alias for *srcToken* in the child interpreter identified by *srcPath*. *srcToken* refers to the value returned when the alias was created; if the source command has been renamed, the renamed command will be deleted.

coreTools Command Reference Index

interp alias *srcPath srcCmd targetPath targetCmd ?arg arg ...?*

This command creates an alias between one child and another (see the **alias** child command below for creating aliases between a child and its parent). In this command, either of the child interpreters may be anywhere in the hierarchy of interpreters under the interpreter invoking the command. *SrcPath* and *srcCmd* identify the source of the alias. *SrcPath* is a Tcl list whose elements select a particular interpreter. For example, identifies an interpreter which is a child of interpreter which is a child of the invoking interpreter. An empty list specifies the interpreter invoking the command. *srcCmd* gives the name of a new command, which will be created in the source interpreter. *TargetPath* and *targetCmd* specify a target interpreter and command, and the *arg* arguments, if any, specify additional arguments to *targetCmd* which are prepended to any arguments specified in the invocation of *srcCmd*. *TargetCmd* may be undefined at the time of this call, or it may already exist; it is not created by this command. The alias arranges for the given target command to be invoked in the target interpreter whenever the given source command is invoked in the source interpreter. See **ALIAS INVOCATION** below for more details. The command returns a token that uniquely identifies the command created *srcCmd*, even if the command is renamed afterwards. The token may but does not have to be equal to *srcCmd*.

interp aliases *?path?*

This command returns a Tcl list of the tokens of all the source commands for aliases defined in the interpreter identified by *path*. The tokens correspond to the values returned when the aliases were created (which may not be the same as the current names of the commands).

interp bgerror *path ?cmdPrefix?*

This command either gets or sets the current background exception handler for the interpreter identified by *path*. If *cmdPrefix* is absent, the current background exception handler is returned, and if it is present, it is a list of words (of minimum length one) that describes what to set the interpreter's background exception handler to. See the **BACKGROUND EXCEPTION HANDLING** section for more details.

interp cancel *?-unwind? ?-? ?path? ?result?*

Cancels the script being evaluated in the interpreter identified by *path*. Without the **-unwind** switch the evaluation stack for the interpreter is unwound until an enclosing catch command is found or there are no further invocations of the interpreter left on the call stack. With the **-unwind** switch the evaluation stack for the interpreter is unwound without regard to any intervening catch command until there are no further invocations of the interpreter left on the call stack. The **--** switch can be used to mark the end of switches; it may be needed if *path* is an unusual value such as **-safe**. If *result* is present, it will be used as the error message string; otherwise, a default error

coreTools Command Reference Index

message string will be used.

interp create *?-safe? ?-? ?path?*

Creates a child interpreter identified by *path* and a new command, called a *child command*. The name of the child command is the last component of *path*. The new child interpreter and the child command are created in the interpreter identified by the path obtained by removing the last component from *path*. For example, if *path* is **a b c** then a new child interpreter and child command named **c** are created in the interpreter identified by the path **a b**. The child command may be used to manipulate the new interpreter as described below. If *path* is omitted, Tcl creates a unique name of the form **interp*x***, where *x* is an integer, and uses it for the interpreter and the child command. If the **-safe** switch is specified (or if the parent interpreter is a safe interpreter), the new child interpreter will be created as a safe interpreter with limited functionality; otherwise the child will include the full set of Tcl built-in commands and variables. The **--** switch can be used to mark the end of switches; it may be needed if *path* is an unusual value such as **-safe**. The result of the command is the name of the new interpreter. The name of a child interpreter must be unique among all the children for its parent; an error occurs if a child interpreter by the given name already exists in this parent. The initial recursion limit of the child interpreter is set to the current recursion limit of its parent interpreter.

interp debug *path ?-frame ?bool??*

Controls whether frame-level stack information is captured in the child interpreter identified by *path*. If no arguments are given, option and current setting are returned. If **-frame** is given, the debug setting is set to the given boolean if provided and the current setting is returned. This only affects the output of **info frame**, in that exact frame-level information for command invocation at the bytecode level is only captured with this setting on.

For example, with code like

```
proc mycontrol{... script} {
    ...
    uplevel 1 $script
    ...
}

proc dosomething {...} {
    ...
    mycontrol {
        somecode
    }
}
```

the standard setting will provide a relative line number for the command **somecode** and the relevant frame will be of type **eval**. With frame-debug active on the other hand the tracking extends so far that the system will be able to determine the file and absolute line

coreTools Command Reference Index

number of this command, and return a frame of type **source**. This more exact information is paid for with slower execution of all commands.

Note that once it is on, this flag cannot be switched back off: such attempts are silently ignored. This is needed to maintain the consistency of the underlying interpreter's state.

interp delete *?path ...?*

Deletes zero or more interpreters given by the optional *path* arguments, and for each interpreter, it also deletes its children. The command also deletes the child command for each interpreter deleted. For each *path* argument, if no interpreter by that name exists, the command raises an error.

interp eval *path arg ?arg ...?*

This command concatenates all of the *arg* arguments in the same fashion as the **concat** command, then evaluates the resulting string as a Tcl script in the child interpreter identified by *path*. The result of this evaluation (including all **return** options, such as **-errorinfo** and **-errorcode** information, if an error occurs) is returned to the invoking interpreter. Note that the script will be executed in the current context stack frame of the *path* interpreter; this is so that the implementations (in a parent interpreter) of aliases in a child interpreter can execute scripts in the child that find out information about the child's current state and stack frame.

interp exists *path*

Returns **1** if a child interpreter by the specified *path* exists in this parent, **0** otherwise. If *path* is omitted, the invoking interpreter is used.

interp expose *path hiddenName ?exposedCmdName?*

Makes the hidden command *hiddenName* exposed, eventually bringing it back under a new *exposedCmdName* name (this name is currently accepted only if it is a valid global name space name without any ::), in the interpreter denoted by *path*. If an exposed command with the targeted name already exists, this command fails.

Hidden commands are explained in more detail in **HIDDEN COMMANDS**, below.

interp hide *path exposedCmdName ?hiddenCmdName?*

Makes the exposed command *exposedCmdName* hidden, renaming it to the hidden command *hiddenCmdName*, or keeping the same name if *hiddenCmdName* is not given, in the interpreter denoted by *path*. If a hidden command with the targeted name already exists, this command fails. Currently both *exposedCmdName* and *hiddenCmdName* can not contain namespace qualifiers, or an error is raised. Commands to be hidden by **interp hide** are looked up in the global namespace even if the current namespace is not the global one. This prevents children from fooling a parent interpreter into hiding the wrong command, by making the current namespace be different from the global one. Hidden commands are explained in more detail in **HIDDEN COMMANDS**, below.

interp hidden path

Returns a list of the names of all hidden commands in the interpreter identified by *path*.

interp invokehidden path ?-option ...? hiddenCmdName ?arg ...?

Invokes the hidden command *hiddenCmdName* with the arguments supplied in the interpreter denoted by *path*. No substitutions or evaluation are applied to the arguments. Three *-options* are supported, all of which start with -: **-namespace** (which takes a single argument afterwards, *nsName*), **-global**, and **--**. If the **-namespace** flag is present, the hidden command is invoked in the namespace called *nsName* in the target interpreter. If the **-global** flag is present, the hidden command is invoked at the global level in the target interpreter; otherwise it is invoked at the current call frame and can access local variables in that and outer call frames. The **--** flag allows the *hiddenCmdName* argument to start with a character, and is otherwise unnecessary. If both the **-namespace** and **-global** flags are present, the **-namespace** flag is ignored. Note that the hidden command will be executed (by default) in the current context stack frame of the *path* interpreter. Hidden commands are explained in more detail in **HIDDEN COMMANDS**, below.

interp issafe ?path?

Returns **1** if the interpreter identified by the specified *path* is safe, **0** otherwise.

interp limit path limitType ?-option? ?value ...?

Sets up, manipulates and queries the configuration of the resource limit *limitType* for the interpreter denoted by *path*. If no *-option* is specified, return the current configuration of the limit. If *-option* is the sole argument, return the value of that option. Otherwise, a list of *-option/value* argument pairs must be supplied. See **RESOURCE LIMITS** below for a more detailed explanation of what limits and options are supported.

interp marktrusted path

Marks the interpreter identified by *path* as trusted. Does not expose the hidden commands. This command can only be invoked from a trusted interpreter. The command has no effect if the interpreter identified by *path* is already trusted.

interp recursionlimit path ?newlimit?

Returns the maximum allowable nesting depth for the interpreter specified by *path*. If *newlimit* is specified, the interpreter recursion limit will be set so that nesting of more than *newlimit* calls to **Tcl_Eval** and related procedures in that interpreter will return an error. The *newlimit* value is also returned. The *newlimit* value must be a positive integer between 1 and the maximum value of a non-long integer on the platform.

The command sets the maximum size of the Tcl call stack only. It cannot by itself prevent stack overflows on

coreTools Command Reference Index

the C stack being used by the application. If your machine has a limit on the size of the C stack, you may get stack overflows before reaching the limit set by the command. If this happens, see if there is a mechanism in your system for increasing the maximum size of the C stack.

interp share *srcPath channelId destPath*

Causes the IO channel identified by *channelId* to become shared between the interpreter identified by *srcPath* and the interpreter identified by *destPath*. Both interpreters have the same permissions on the IO channel. Both interpreters must close it to close the underlying IO channel; IO channels accessible in an interpreter are automatically closed when an interpreter is destroyed.

interp slaves *?path?*

Returns a Tcl list of the names of all the child interpreters associated with the interpreter identified by *path*. If *path* is omitted, the invoking interpreter is used.

interp children

?path?

Synonym for . **interp slaves** *?path?*

interp target *path alias*

Returns a Tcl list describing the target interpreter for an alias. The alias is specified with an interpreter path and source command name, just as in **interp alias** above. The name of the target interpreter is returned as an interpreter path, relative to the invoking interpreter. If the target interpreter for the alias is the invoking interpreter then an empty list is returned. If the target interpreter for the alias is not the invoking interpreter or one of its descendants then an error is generated. The target command does not have to be defined at the time of this invocation.

interp transfer *srcPath channelId destPath*

Causes the IO channel identified by *channelId* to become available in the interpreter identified by *destPath* and unavailable in the interpreter identified by *srcPath*.

child COMMAND

For each child interpreter created with the **interp** command, a new Tcl command is created in the parent interpreter with the same name as the new interpreter. This command may be used to invoke various operations on the interpreter. It has the following general form:

child command ?arg arg ...?

child is the name of the interpreter, and *command* and the *args* determine the exact behavior of the command. The valid forms of this command are:

coreTools Command Reference Index

child aliases

Returns a Tcl list whose elements are the tokens of all the aliases in *child*. The tokens correspond to the values returned when the aliases were created (which may not be the same as the current names of the commands).

child alias srcToken

Returns a Tcl list whose elements are the *targetCmd* and *args* associated with the alias represented by *srcToken* (this is the value returned when the alias was created; it is possible that the actual source command in the *child* is different from *srcToken*).

child alias srcToken {}

Deletes the alias for *srcToken* in the *child* interpreter. *srcToken* refers to the value returned when the alias was created; if the source command has been renamed, the renamed command will be deleted.

child alias srcCmd targetCmd ?arg ..?

Creates an alias such that whenever *srcCmd* is invoked in *child*, *targetCmd* is invoked in the parent. The *arg* arguments will be passed to *targetCmd* as additional arguments, prepended before any arguments passed in the invocation of *srcCmd*. See **ALIAS INVOCATION** below for details. The command returns a token that uniquely identifies the command created *srcCmd*, even if the command is renamed afterwards. The token may but does not have to be equal to *srcCmd*.

child berror ?cmdPrefix?

This command either gets or sets the current background exception handler for the *child* interpreter. If *cmdPrefix* is absent, the current background exception handler is returned, and if it is present, it is a list of words (of minimum length one) that describes what to set the interpreter's background exception handler to. See the **BACKGROUND EXCEPTION HANDLING** section for more details.

child eval arg ?arg ..?

This command concatenates all of the *arg* arguments in the same fashion as the **concat** command, then evaluates the resulting string as a Tcl script in *child*. The result of this evaluation (including all **return** options, such as **-errorinfo** and **-errorcode** information, if an error occurs) is returned to the invoking interpreter. Note that the script will be executed in the current context stack frame of *child*; this is so that the implementations (in a parent interpreter) of aliases in a *child* interpreter can execute scripts in the *child* that find out information about the *child*'s current state and stack frame.

child expose hiddenName ?exposedCmdName?

This command exposes the hidden command *hiddenName*, eventually bringing it back under a new *exposedCmdName* name (this name is currently accepted only if it is a valid global name space name without any **::**), in *child*. If an exposed command with the targeted name already exists, this command fails. For more details on hidden

coreTools Command Reference Index

commands, see **HIDDEN COMMANDS**, below.

child **hide** *exposedCmdName* ?*hiddenCmdName*?

This command hides the exposed command *exposedCmdName*, renaming it to the hidden command *hiddenCmdName*, or keeping the same name if the argument is not given, in the *child* interpreter. If a hidden command with the targeted name already exists, this command fails. Currently both *exposedCmdName* and *hiddenCmdName* can not contain namespace qualifiers, or an error is raised. Commands to be hidden are looked up in the global namespace even if the current namespace is not the global one. This prevents children from fooling a parent interpreter into hiding the wrong command, by making the current namespace be different from the global one. For more details on hidden commands, see **HIDDEN COMMANDS**, below.

child **hidden**

Returns a list of the names of all hidden commands in *child*.

child **invokehidden** ?-option ...? *hiddenName* ?arg ..?

This command invokes the hidden command *hiddenName* with the supplied arguments, in *child*. No substitutions or evaluations are applied to the arguments. Three -options are supported, all of which start with -: **-namespace** (which takes a single argument afterwards, *nsName*), **-global**, and **--**. If the **-namespace** flag is given, the hidden command is invoked in the specified namespace in the child. If the **-global** flag is given, the command is invoked at the global level in the child; otherwise it is invoked at the current call frame and can access local variables in that or outer call frames. The **--** flag allows the *hiddenCmdName* argument to start with a character, and is otherwise unnecessary. If both the **-namespace** and **-global** flags are given, the **-namespace** flag is ignored. Note that the hidden command will be executed (by default) in the current context stack frame of *child*. For more details on hidden commands, see **HIDDEN COMMANDS**, below.

child **issafe**

Returns **1** if the child interpreter is safe, **0** otherwise.

child **limit** *limitType* ?-option? ?value ...?

Sets up, manipulates and queries the configuration of the resource limit *limitType* for the child interpreter. If no -option is specified, return the current configuration of the limit. If -option is the sole argument, return the value of that option. Otherwise, a list of -option/value argument pairs must be supplied. See **RESOURCE LIMITS** below for a more detailed explanation of what limits and options are supported.

child **marktrusted**

Marks the child interpreter as trusted. Can only be invoked by a trusted interpreter. This command does not expose any hidden commands in the child interpreter. The command has no effect if the child is already trusted.

child **recursionlimit** ?*newlimit*?

Returns the maximum allowable nesting depth for the *child* interpreter. If *newlimit* is specified, the recursion limit in *child* will be set so that nesting of more than *newlimit* calls to **Tcl_Eval()** and related procedures in *child* will return an error. The *newlimit* value is also returned. The *newlimit* value must be a positive integer between 1 and the maximum value of a non-long integer on the platform.

The command sets the maximum size of the Tcl call stack only. It cannot by itself prevent stack overflows on the C stack being used by the application. If your machine has a limit on the size of the C stack, you may get stack overflows before reaching the limit set by the command. If this happens, see if there is a mechanism in your system for increasing the maximum size of the C stack.

SAFE INTERPRETERS

A safe interpreter is one with restricted functionality, so that is safe to execute an arbitrary script from your worst enemy without fear of that script damaging the enclosing application or the rest of your computing environment. In order to make an interpreter safe, certain commands and variables are removed from the interpreter. For example, commands to create files on disk are removed, and the **exec** command is removed, since it could be used to cause damage through subprocesses. Limited access to these facilities can be provided, by creating aliases to the parent interpreter which check their arguments carefully and provide restricted access to a safe subset of facilities. For example, file creation might be allowed in a particular subdirectory and subprocess invocation might be allowed for a carefully selected and fixed set of programs.

A safe interpreter is created by specifying the **-safe** switch to the **interp create** command. Furthermore, any child created by a safe interpreter will also be safe.

A safe interpreter is created with exactly the following set of built-in commands:

after	append	apply	array
binary	break	catch	chan
clock	close	concat	continue
dict	eof	error	eval
expr	fblocked	fcopy	fileevent
flush	for	foreach	format
gets	global	if	incr
info	interp	join	lappend
lassign	lindex	linsert	list
llength	lrange	lrepeat	lreplace
lsearch	lset	lsort	namespace
package	pid	proc	puts

coreTools Command Reference Index

read	regexp	regsub	rename
return	scan	seek	set
split	string	subst	switch
tell	time	trace	unset
update	uplevel	upvar	variable
vwait	while		

The following commands are hidden by **interp create** when it creates a safe interpreter:

cd	encoding	exec	exit
fconfigure	file	glob	load
open	pwd	socket	source
unload			

These commands can be recreated later as Tcl procedures or aliases, or re-exposed by **interp expose**.

The following commands from Tcl's library of support procedures are not present in a safe interpreter:

auto_exec_ok	auto_import	auto_load
auto_load_index	auto_qualify	unknown

Note in particular that safe interpreters have no default **unknown** command, so Tcl's default autoloading facilities are not available. Autoload access to Tcl's commands that are normally autoloaded:

auto_mkindex	auto_mkindex_old
auto_reset	history
parray	pkg_mkIndex
::pkg::create	::safe::interpAddToAccessPath
::safe::interpCreate	::safe::interpConfigure
::safe::interpDelete	::safe::interpFindInAccessPath
::safe::interpInit	::safe::setLogCmd
tcl_endOfWord	tcl_findLibrary
tcl_startOfNextWord	tcl_startOfPreviousWord
tcl_wordBreakAfter	tcl_wordBreakBefore

can only be provided by explicit definition of an **unknown** command in the safe interpreter. This will involve exposing the **source** command. This is most easily accomplished by creating the safe interpreter with Tcl's **Safe-Tcl** mechanism. **Safe-Tcl** provides safe versions of **source**, **load**, and other Tcl commands needed to support autoloading of commands and the loading of packages.

In addition, the **env** variable is not present in a safe interpreter, so it cannot share environment variables with other interpreters. The **env** variable poses a security risk, because users can store sensitive information in an environment variable. For example, the PGP manual recommends storing the PGP private key protection password in the environment variable **PGPPASS**. Making this variable available to untrusted code executing in a safe interpreter would incur a security risk.

If extensions are loaded into a safe interpreter, they may also restrict their own functionality to eliminate

coreTools Command Reference Index

unsafe commands. For a discussion of management of extensions for safety see the manual entries for **Safe-Tcl** and the **load** Tcl command.

A safe interpreter may not alter the recursion limit of any interpreter, including itself.

ALIAS INVOCATION

The alias mechanism has been carefully designed so that it can be used safely in an untrusted script which is being executed in a safe interpreter even if the target of the alias is not a safe interpreter. The most important thing in guaranteeing safety is to ensure that information passed from the child to the parent is never evaluated or substituted in the parent; if this were to occur, it would enable an evil script in the child to invoke arbitrary functions in the parent, which would compromise security.

When the source for an alias is invoked in the child interpreter, the usual Tcl substitutions are performed when parsing that command. These substitutions are carried out in the source interpreter just as they would be for any other command invoked in that interpreter. The command procedure for the source command takes its arguments and merges them with the *targetCmd* and *args* for the alias to create a new array of arguments. If the words of *srcCmd* were the new set of words will be where *targetCmd* and *args* are the values supplied when the alias was created. *TargetCmd* is then used to locate a command procedure in the target interpreter, and that command procedure is invoked with the new set of arguments. An error occurs if there is no command named *targetCmd* in the target interpreter. No additional substitutions are performed on the words: the target command procedure is invoked directly, without going through the normal Tcl evaluation mechanism. Substitutions are thus performed on each word exactly once: *targetCmd* and *args* were substituted when parsing the command that created the alias, and *arg1* - *argN* are substituted when the alias's source command is parsed in the source interpreter.

When writing the *targetCmds* for aliases in safe interpreters, it is very important that the arguments to that command never be evaluated or substituted, since this would provide an escape mechanism whereby the child interpreter could execute arbitrary code in the parent. This in turn would compromise the security of the system.

HIDDEN COMMANDS

Safe interpreters greatly restrict the functionality available to Tcl programs executing within them.

coreTools Command Reference Index

Allowing the untrusted Tcl program to have direct access to this functionality is unsafe, because it can be used for a variety of attacks on the environment. However, there are times when there is a legitimate need to use the dangerous functionality in the context of the safe interpreter. For example, sometimes a program must be **sourced** into the interpreter. Another example is Tk, where windows are bound to the hierarchy of windows for a specific interpreter; some potentially dangerous functions, e.g. window management, must be performed on these windows within the interpreter context.

The **interp** command provides a solution to this problem in the form of *hidden commands*. Instead of removing the dangerous commands entirely from a safe interpreter, these commands are hidden so they become unavailable to Tcl scripts executing in the interpreter. However, such hidden commands can be invoked by any trusted ancestor of the safe interpreter, in the context of the safe interpreter, using **interp invoke**. Hidden commands and exposed commands reside in separate name spaces. It is possible to define a hidden command and an exposed command by the same name within one interpreter.

Hidden commands in a child interpreter can be invoked in the body of procedures called in the parent during alias invocation. For example, an alias for **source** could be created in a child interpreter. When it is invoked in the child interpreter, a procedure is called in the parent interpreter to check that the operation is allowable (e.g. it asks to source a file that the child interpreter is allowed to access). The procedure then it invokes the hidden **source** command in the child interpreter to actually source in the contents of the file. Note that two commands named **source** exist in the child interpreter: the alias, and the hidden command.

Because a parent interpreter may invoke a hidden command as part of handling an alias invocation, great care must be taken to avoid evaluating any arguments passed in through the alias invocation. Otherwise, malicious child interpreters could cause a trusted parent interpreter to execute dangerous commands on their behalf. See the section on **ALIAS INVOCATION** for a more complete discussion of this topic. To help avoid this problem, no substitutions or evaluations are applied to arguments of **interp invokehidden**.

Safe interpreters are not allowed to invoke hidden commands in themselves or in their descendants. This prevents them from gaining access to hidden functionality in themselves or their descendants.

The set of hidden commands in an interpreter can be manipulated by a trusted interpreter using **interp expose** and **interp hide**. The **interp expose** command moves a hidden command to the set of exposed commands in the interpreter identified by *path*, potentially renaming the command in the process. If an exposed command by the targeted name already exists, the operation fails. Similarly, **interp hide** moves an exposed command to the

coreTools Command Reference Index

set of hidden commands in that interpreter. Safe interpreters are not allowed to move commands between the set of hidden and exposed commands, in either themselves or their descendants.

Currently, the names of hidden commands cannot contain namespace qualifiers, and you must first rename a command in a namespace to the global namespace before you can hide it. Commands to be hidden by **interp hide** are looked up in the global namespace even if the current namespace is not the global one. This prevents children from fooling a parent interpreter into hiding the wrong command, by making the current namespace be different from the global one.

RESOURCE LIMITS

Every interpreter has two kinds of resource limits that may be imposed by any parent interpreter upon its children. Command limits (of type **command**) restrict the total number of Tcl commands that may be executed by an interpreter (as can be inspected via the **info cmdcount** command), and time limits (of type **time**) place a limit by which execution within the interpreter must complete. Note that time limits are expressed as *absolute* times (as in **clock seconds**) and not relative times (as in **after**) because they may be modified after creation.

When a limit is exceeded for an interpreter, first any handler callbacks defined by parent interpreters are called. If those callbacks increase or remove the limit, execution within the (previously) limited interpreter continues. If the limit is still in force, an error is generated at that point and normal processing of errors within the interpreter (by the **catch** command) is disabled, so the error propagates outwards (building a stack-trace as it goes) to the point where the limited interpreter was invoked (e.g. by **interp eval**) where it becomes the responsibility of the calling code to catch and handle.

LIMIT OPTIONS

Every limit has a number of options associated with it, some of which are common across all kinds of limits, and others of which are particular to the kind of limit.

-command

This option (common for all limit types) specifies (if non-empty) a Tcl script to be executed in the global namespace of the interpreter reading and writing the option when the particular limit in the limited interpreter is exceeded. The callback may modify the limit on the interpreter if it wishes the limited interpreter to continue executing. If the callback generates an exception, it is reported through the background exception mechanism (see **BACKGROUND EXCEPTION HANDLING**). Note that the callbacks defined

coreTools Command Reference Index

by one interpreter are completely isolated from the callbacks defined by another, and that the order in which those callbacks are called is undefined.

-granularity

This option (common for all limit types) specifies how frequently (out of the points when the Tcl interpreter is in a consistent state where limit checking is possible) that the limit is actually checked. This allows the tuning of how frequently a limit is checked, and hence how often the limit-checking overhead (which may be substantial in the case of time limits) is incurred.

-milliseconds

This option specifies the number of milliseconds after the moment defined in the **-seconds** option that the time limit will fire. It should only ever be specified in conjunction with the **-seconds** option (whether it was set previously or is being set this invocation.)

-seconds

This option specifies the number of seconds after the epoch (see **clock seconds**) that the time limit for the interpreter will be triggered. The limit will be triggered at the start of the second unless specified at a sub-second level using the **-milliseconds** option. This option may be the empty string, which indicates that a time limit is not set for the interpreter.

-value

This option specifies the number of commands that the interpreter may execute before triggering the command limit. This option may be the empty string, which indicates that a command limit is not set for the interpreter.

Where an interpreter with a resource limit set on it creates a child interpreter, that child interpreter will have resource limits imposed on it that are at least as restrictive as the limits on the creating parent interpreter. If the parent interpreter of the limited parent wishes to relax these conditions, it should hide the **interp** command in the child and then use aliases and the **interp invokehidden** subcommand to provide such access as it chooses to the **interp** command to the limited parent as necessary.

BACKGROUND EXCEPTION HANDLING

When an exception happens in a situation where it cannot be reported directly up the stack (e.g. when processing events in an **update** or **vwait** call) the exception is instead reported through the background exception handling mechanism. Every interpreter has a background exception handler registered; the default exception handler arranges for the **bgerror** command in the interpreter's global namespace to be called, but other exception handlers may be installed and process

coreTools Command Reference Index

background exceptions in substantially different ways.

A background exception handler consists of a non-empty list of words to which will be appended two further words at invocation time. The first word will be the interpreter result at time of the exception, typically an error message, and the second will be the dictionary of return options at the time of the exception. These are the same values that **catch** can capture when it controls script evaluation in a non-background situation. The resulting list will then be executed in the interpreter's global namespace without further substitutions being performed.

CREDITS

The safe interpreter mechanism is based on the Safe-Tcl prototype implemented by Nathaniel Borenstein and Marshall Rose.

EXAMPLES

Creating and using an alias for a command in the current interpreter:

```
interp alias {} getIndex {} lsearch {alpha beta gamma delta}  
set idx [getIndex delta]
```

Executing an arbitrary command in a safe interpreter where every invocation of **lappend** is logged:

```
set i [interp create -safe]  
interp hide $i lappend  
interp alias $i lappend {} loggedLappend $i  
proc loggedLappend {i args} {  
    puts "logged invocation of lappend $args"  
    interp invokehidden $i lappend {*} $args  
}  
interp eval $i $someUntrustedScript
```

Setting a resource limit on an interpreter so that an infinite loop terminates.

```
set i [interp create]  
interp limit $i command -value 1000  
interp eval $i {  
    set x 0  
    while {1} {  
        puts "Counting up... [incr x]"  
    }  
}
```

SEE ALSO

`bgerror(n)`, `load(n)`, `safe(n)`, `Tcl_CreateChild(3)`,
`Tcl_Eval(3)`, `Tcl_BackgroundException(3)`

KEYWORDS

alias, parent interpreter, safe interpreter, child
interpreter

intfPin

Represents an interface instance on a cell.

Description

This is a new item type still under development - used to model instance specific connections between interfaces.

See Also

Supported Attributes

[AddressSpaceRef \(3\)](#), [AssociationComplete \(3\)](#), [AutoConnectWhen \(3\)](#),
[BaseAddress \(3\)](#), [BitsInLAU \(3\)](#), [Bridge \(3\)](#), [Channel \(3\)](#), [ComponentOfItem \(3\)](#),
[ConnectToExportedInstance \(3\)](#), [ConnectionDialogCmd \(3\)](#),
[ConnectionRequired \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [Endian \(3\)](#),
[GlobalSlotNumber \(3\)](#), [GroupName \(3\)](#), [InterfaceGroup \(3\)](#), [InterfaceLabel \(3\)](#),
[InterfaceType \(3\)](#), [IsAddressable \(3\)](#), [Label \(3\)](#), [LockedInTemplate \(3\)](#),
[MemoryMap \(3\)](#), [Name \(3\)](#), [ReadOnlyInterface \(3\)](#), [Sequence \(3\)](#),
[ShowRoute \(3\)](#), [SlotWidth \(3\)](#), [SpiritContainer \(3\)](#), [SpiritFile \(3\)](#),
[SpiritInterfaceType \(3\)](#), [SplitInterfaceMembers \(3\)](#), [SplitInterfaceName \(3\)](#),
[SymbolPinPrefix \(3\)](#), [SymbolPinSide \(3\)](#), [TypeName \(3\)](#),
[UniqueConsumerConnections \(3\)](#), [VipConnection \(3\)](#),

intfPort

Represents an interface instance on a design.

Description

This is a new item type still under development - used as a proxy for the interfaceInstance stored internal to the design.

See Also

Supported Attributes

[AddressSpaceRef \(3\)](#), [AssociationComplete \(3\)](#), [AutoConnectWhen \(3\)](#),
[BaseAddress \(3\)](#), [BitsInLAU \(3\)](#), [Bridge \(3\)](#), [Channel \(3\)](#), [ComponentOfItem \(3\)](#),
[ConnectToExportedInstance \(3\)](#), [ConnectionDialogCmd \(3\)](#),
[ConnectionRequired \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [Endian \(3\)](#),
[GlobalSlotNumber \(3\)](#), [GroupName \(3\)](#), [InterfaceGroup \(3\)](#), [InterfaceLabel \(3\)](#),
[InterfaceType \(3\)](#), [IsAddressable \(3\)](#), [Label \(3\)](#), [LockedInTemplate \(3\)](#),
[MemoryMap \(3\)](#), [Name \(3\)](#), [ReadOnlyInterface \(3\)](#), [Sequence \(3\)](#),
[ShowRoute \(3\)](#), [SlotWidth \(3\)](#), [SpiritContainer \(3\)](#), [SpiritFile \(3\)](#),
[SpiritInterfaceType \(3\)](#), [SplitInterfaceMembers \(3\)](#), [SplitInterfaceName \(3\)](#),
[SymbolPinPrefix \(3\)](#), [SymbolPinSide \(3\)](#), [TypeName \(3\)](#),
[UniqueConsumerConnections \(3\)](#), [VipConnection \(3\)](#),

invoke_generator
invoke a component generator.

Syntax

string **invoke_generator** [-component <component_name>] generatorName
 string <component_name>
 string generatorName

Parameters

-component	The component name in which the component generator is registered.
<component_name>	
generatorName	The name of the file or component generator to invoke

Description

This command is the integration of all 3 steps to run a component generator. A component generator is registered through the imported Spirit component. First, it generates the needed loose generator interface files. Second, it invokes the generator and finally processes the generator change file.

Examples

to run a component generator named 'generator1':
coreAssembler>invoke_generator generator1

See Also

invoke_ralgen

Invoke ralgen from \$VCS_HOME or within cT image.

Syntax

```
string invoke_ralgen cmdArgs catchVar runDir  
    string cmdArgs  
    string catchVar  
    string runDir
```

Parameters

<i>cmdArgs</i>	ralgen command line arguments
<i>catchVar</i>	name of variable in which catch output is returned
<i>runDir</i>	directory in which ralgen should be invoked

Description

invoke_ralgen provides a simple TCL wrapper around the ralgen utility used to translate RALF files into System Verilog source code.

The invoke_ralgen command supports the same command line arguments as the ralgen program.

Examples

Invoking ralgen to generate SV model for the UVM RAL file for the design 'cTTop'. If the command fails the error log is captured in the 'msg' variable.

```
if {[invoke_ralgen "-l sv -uvm -t cTTop cTTop.ralf" msg]} {  
    echo "Error invoking ralgen: $msg"  
}
```

See Also

NAME

is_false Tests the value of a specified variable, and returns 1 if the value is 0 or the case-insensitive string **false**; returns 0 if the value is 1 or the case-insensitive string **true**.

SYNTAX

```
status is_false
      value
```

Data Types
value string

ARGUMENTS

value Specifies the name of the variable whose value is to be tested.

DESCRIPTION

This command tests the value of a specified variable, and returns 1 if the value is either 0 or the case-insensitive string **false**. The command returns 0 if the value is either 1 or the case-insensitive string **true**. Any value other than 1, 0, **true**, or **false** generates a Tcl error message.

The **is_false** command is used in writing scripts that test Boolean variables that can be set to 1, 0, or the case-insensitive strings **true** or **false**. When such variables are set to **true** or **false**, they cannot be tested in the negative in an **if** statement by simple variable substitution, because they do not constitute a true or false condition. The following example is not legal Tcl:

```
set x FALSE
if { !$x } {
    set y TRUE
```

```
}
```

This results in a Tcl error message, indicating that you cannot use a non-numeric string as the operand of "!". So, although you can test the positive condition, **is_false** allows you to test both conditions safely.

EXAMPLES

The following example shows the use of the **is_false** command:

```
prompt> set x TRUE
TRUE

prompt> if { ![is_false $x] } {
?      set y TRUE
?      }
TRUE

prompt>
```

SEE ALSO

```
expr(2)
if(2)
is_true(2)
```


[API Index](#) [Command Reference](#)

is_prime_profile_enabled

Returns 1 if Prime Profile is enabled in this session, 0 otherwise.

Syntax

string **is_prime_profile_enabled**

Description

Returns whether Prime Profile feature is enabled in the current session for the current workspace.

Examples

```
coreConsultant> is_prime_profile_enabled
1
coreConsultant> disable_prime_profile
Warning: Prime Profile feature is disabled. Prime Profile related
commands will not have any effect. (CMDS-288)
1
coreConsultant> is_prime_profile_enabled
0
coreConsultant> enable_prime_profile
Information: Prime Profile feature and its related commands will be
enabled. (CMDS-297)
0
coreConsultant> is_prime_profile_enabled
1
```

See Also

[disable_prime_profile \(2\)](#) [enable_prime_profile \(2\)](#) [set_prime_profile \(2\)](#)
[create_prime_profile_file \(2\)](#)

NAME

is_true Tests the value of a specified variable, and returns 1 if the value is 1 or the case-insensitive string **true**; returns 0 if the value is 0 or the case-insensitive string **false**.

SYNTAX

```
status is_true
      value
```

Data Types
value string

ARGUMENTS

value Specifies the name of the variable whose value is to be tested.

DESCRIPTION

This command tests the value of a specified variable, and returns 1 if the value is either 1 or the case-insensitive string **true**. The command returns 0 if the value is either 0 or the case-insensitive string **false**. Any value other than 1, 0, **true**, or **false** generates a Tcl error message.

The **is_true** command is used in writing scripts that test Boolean variables that can be set to 1, 0, or the case-insensitive strings **true** or **false**. When such variables are set to **true** or **false**, they cannot be tested in the negative in an **if** statement by simple variable substitution, because they do not constitute a true or false condition. The following example is not legal Tcl:

```
set x TRUE
if { !$x } {
    set y FALSE
```

```
}
```

This results in a Tcl error message, indicating that you cannot use a non-numeric string as the operand of "!". So, although you can test the positive condition, **is_true** allows you to test both conditions safely.

EXAMPLES

The following example shows the use of the **is_true** command:

```
prompt> set x FALSE
FALSE

prompt> if { ![is_true $x] } {
?      set y FALSE
?      }

FALSE

prompt>
```

SEE ALSO

```
expr(2)
if(2)
is_false(2)
```


NAME

`join` - Create a string by joining together list elements

SYNOPSIS

```
join list ?joinString?
```

DESCRIPTION

The *list* argument must be a valid Tcl list. This command returns the string formed by joining all of the elements of *list* together with *joinString* separating each adjacent pair of elements. The *joinString* argument defaults to a space character.

EXAMPLES

Making a comma-separated list:

```
set data {1 2 3 4 5}
join $data ", "
-> 1, 2, 3, 4, 5
```

Using `join` to flatten a list by a single level:

```
set data {1 {2 3} 4 {5 {6 7} 8}}
join $data
-> 1 2 3 4 5 {6 7} 8
```

SEE ALSO

`list(n)`, `lappend(n)`, `split(n)`

KEYWORDS

`element`, `join`, `list`, `separator`

knowledgeBase

Knowledge database item

Description

The knowledgeBase item represents a coreTool knowledge database (KB). The coreTools use KBs to store information about a design, coreKit, or workspace. For example, a design KB contains the design and all intent specified for the design.

The coreTools automatically maintain the KBs associated with a workspace, loading and saving KBs as needed. As a coreTool user, you primarily work with your workspace KB. You create a workspace to begin a design project, save your work in the workspace KB, and reload your workspace KB to continue your work.

See Also

[save_workspace \(2\)](#) [close_workspace \(2\)](#) [open_workspace \(2\)](#)

Supported Attributes

[Activity \(3\)](#), [ComponentOfItem \(3\)](#), [CreatedViaUngrouping \(3\)](#), [Description \(3\)](#), [KbVersion \(3\)](#), [Label \(3\)](#), [LicenseForOpen \(3\)](#), [Name \(3\)](#), [ScratchInstallIndex \(3\)](#), [Sequence \(3\)](#), [TypeName \(3\)](#),

NAME

`lappend` - Append list elements onto a variable

SYNOPSIS

lappend *varName* *?value value value ...?*

DESCRIPTION

This command treats the variable given by *varName* as a list and appends each of the *value* arguments to that list as a separate element, with spaces between elements. If *varName* does not exist, it is created as a list with elements given by the *value* arguments. **Lappend** is similar to **append** except that the *values* are appended as list elements rather than raw text. This command provides a relatively efficient way to build up large lists. For example, is much more efficient than when **\$a** is long.

EXAMPLE

Using **lappend** to build up a list of numbers.

```
% set var 1
1
% lappend var 2
1 2
% lappend var 3 4 5
1 2 3 4 5
```

SEE ALSO

`list(n)`, `lindex(n)`, `linsert(n)`, `llength(n)`, `lset(n)`,
`lsort(n)`, `lrange(n)`

KEYWORDS

`append`, `element`, `list`, `variable`

NAME

`lassign` - Assign list elements to variables

SYNOPSIS

`lassign list ?varName ...?`

DESCRIPTION

This command treats the value *list* as a list and assigns successive elements from that list to the variables given by the *varName* arguments in order. If there are more variable names than list elements, the remaining variables are set to the empty string. If there are more list elements than variables, a list of unassigned elements is returned.

EXAMPLES

An illustration of how multiple assignment works, and what happens when there are either too few or too many elements.

```
lassign {a b c} x y z      ;# Empty return
puts $x                    ;# Prints "a"
puts $y                    ;# Prints "b"
puts $z                    ;# Prints "c"
```

```
lassign {d e} x y z        ;# Empty return
puts $x                    ;# Prints "d"
puts $y                    ;# Prints "e"
puts $z                    ;# Prints ""
```

coreTools Command Reference Index

```
lassign {f g h i} x y      ;# Returns "h i"  
puts $x                     ;# Prints "f"  
puts $y                     ;# Prints "g"
```

The **lassign** command has other uses. It can be used to create the analogue of the command in many shell languages like this:

```
set ::argv [lassign $::argv argumentToReadOff]
```

SEE ALSO

lindex(n), list(n), lrange(n), lset(n), set(n)

KEYWORDS

assign, element, list, multiple, set, variable

launch_activities_on_grid

Launch the sub-processes associated with multiple activities on a grid from a user workspace in coreConsultant

Syntax

```
string launch_activities_on_grid [-name name] [-add_dependency ]
[-syn_dir syn_dir] [-norun] actlist
    string name
    string
    string syn_dir
    string actlist
```

Parameters

-name <i>name</i>	Launch name, same will embed with the name of the script.
-add_dependency	Specify custom dependency for the activity to be launched. It supports activity name and associated dependent activity names as list of list,ex {<Activity Name1> { list of Dependent Activity Names} <Activity Name2> {list of Dependent Activity Names}}
-syn_dir <i>syn_dir</i>	Name of the synthesis directory to be used to run synthesis related activities
-norun	Generate launch script but don't run it.
<i>actlist</i>	Activity Names to be launched

Description

The launch_activities_on_grid command allows a user to launch groups of activity sub-commands within a single workspace. This allows scripts to be generated for multiple activities and then run without keeping the coreTools slot in the farm while the jobs complete. The command supports the following

1. Launches a single or multiple subprocesses for activities like Synthesize, StaticTimingAnalysis, FormalVerification, Simulation, Spyglass etc.
2. Launch multiple groups of activities simultaneously by executing multiple launch_activities_on_grid commands
3. Wait on one group of activities using wait_activities_on_grid command
4. Generate unique names for launch scripts using -name option

5. Separate directories for synthesis and related activities using `-syn_dir`
6. User can also provide a list of dependent activities using `-add_dependency` option

The following needs to be true before calling `launch_activities_on_grid`. This applies to all activities being run:

- `RunStyle` set to `uge` or `lsf`
- `RunOptions` should be set and valid
- `ScriptsOnly` mode is enabled
- The activity must be autocompleted

Dependent Activities :

Activities will be processed in order based on activity dependencies. You can add additional dependencies using the `-add_dependency` option. For example, a dependency could be added to the simulation activity to support running gate level simulation after synthesis.

Multiple Synthesis Related Runs :

A user can run multiple synthesis and related activities (.e.g. `FormalVerification`) using the `-syn_dir` option to indicate the synthesis directory. Scripts for the activities must be generated in the indicated synthesis directory in advance using the `set_synthesis_dir` command prior to activity completion.

For example:

```
coreConsultant> set_synthesis_dir dir1
coreConsultant> autocomplete_activity Synthesize
coreConsultant> set_synthesis_dir dir2
coreConsultant> autocomplete_activity Synthesize
coreConsultant> launch_activities_on_grid "Synthesize" -name "launch1"
-syn_dir "dir1"
coreConsultant> launch_activities_on_grid "Synthesize" -name "launch2"
-syn_dir "dir2"
```

Examples

Autocomplete & Launch of multiple activities

This will launch the unit-level testbench (`DW_apb_rap_Simulate`) along with the synthesis (`Synthesize`), formal verification (`FormalVerification`), and static timing analysis (`StaticTimingAnalysis`) processes. The jobs are launched and controlled based on dependencies of the associated activities. Therefore, synthesis and simulation can occur in parallel, as can formal verification and static timing analysis, but the latter 2 will not launch until synthesis is finished.

```
coreConsultant> set_activity_parameter DW_apb_rap_Simulate RunStyle grd
```

coreTools Command Reference Index

```
coreConsultant> set_activitiy_parameter DW_apb_rap_Simulate RunOptions {
-P bnormal -l 'os_version=CS7.0'}
coreConsultant> autocomplete_activity DW_apb_rap_Simulate
coreConsultant> set_activitiy_parameter Synthesize RunStyle grd
coreConsultant> set_activitiy_parameter Synthesize RunOptions { -P
bnormal -l 'os_version=CS7.0'}
coreConsultant> autocomplete_activity Synthesize
coreConsultant> autocomplete_activity FormalVerification
coreConsultant> autocomplete_activity StaticTimingAnalysis
coreConsultant> launch_activities_on_grid "Synthesize
StaticTimingAnalysis FormalVerification DW_apb_rap_Simulate"
Information: Executing Command
      <path to workspace>/scratch/launch_activities_on_grid.grd.scr  (TCLSH-18)
Information: Successfully launched jobs
Primary Activity Jobs
Synthesize          | Job Id:7972505
StaticTimingAnalysis | Job Id:7972525
FormalVerification  | Job Id:7972543
DW_apb_rap_Simulate | Job Id:7972557
(CMDS-243)
```

Launch Synthesis related activities in separate folder

The following explains launching synthesis related activities in syn1 folder and associates a launch name launch1

```
coreConsultant> launch_activities_on_grid "Synthesize FormalVerification
GenerateTestVectors" -syn_dir syn1 -name launch1
Information: Executing Command
      /<path to workspace>/coreConsultant/launch/i_rap/scratch/launch_activities_on_grid
Information: Successfully launched jobs
Primary Activity Jobs
Synthesize          | Job Id :4318422
FormalVerification  | Job Id :4318426 | Hold Job Id :4418422
GenerateTestVectors | Job Id :5378121 | Hold Job Id :4418422
DW_apb_rap_Simulate | Job Id :4318432
(CMDS-243)
```

Writing launch scripts on disk

The following command launches the scripts on disk without executing them

```
coreConsultant> launch_activities_on_grid "FormalVerification
StaticTimingAnalysis GenerateGtechModel" -name launchfile_SO -syn_dir
SO_dir -norun
Information: Successfully Generated scripts <path to workspace>/scratch/scriptsonly.grd.sc
1
```

Executing launch command with user dependencies

The -add_dependency supports launch of activities with custom dependencies.

Example 1 :

Consider the following activities

1) runvclp

Examples

- 2) Synthesize
- 3) X
- 4) Y
- 5) runSpyGlass

X, Y are two custom activities for this workspace

User wants to launch a custom activity runVclp when {Synthesize X Y} completes and runSpyglass when { runVclp X Y} completes.

```
coreConsultant> launch_activities_on_grid "runVclp runSpyglass  
FormalVerification X Y Synthesize" -add_dependency { runVclp {Synthesize  
X Y} runSpyglass { runVclp X Y}}
```

Example 2 : User wants to add STA as a custom dependency for FormalVerification. FormalVerification already by default is dependent on Synthesize.

```
coreConsultant> launch_activities_on_grid "FormalVerification  
StaticTimingAnalysis" -add_dependency {FormalVerification  
{StaticTimingAnalysis}}
```

See Also

See following reference on how to wait on activities launched using launch_activities_on_grid command.

[wait activities on grid \(2\)](#) [get launch_activities status \(2\)](#)
[set synthesis dir \(2\)](#)

launch_activity_subproc

Launch a sub-process for an activity or filegroup

Syntax

string **launch_activity_subproc** *activity cmdstr* [*scriptname*]

string *activity*

string *cmdstr*

string *scriptname*

Parameters

	Activity or filegroup associated with the command This is required in order to obtain the 'launch' parameters which determine how to run the job (remotely, via LSF, etc.), what the results file name is, whether to send e-mail when the job completes, and other process related information.
activity	
cmdstr	Command string to execute.
scriptname	Name of shell script to generate from cmdstr and execute If the sub-process script name is not run.scr a link is created from the scriptname to run.scr for the user.

Description

This command executes the command string entered using the sub-process parameter information specified on the activity. The parameters define where and how the command is run. Commands can be executed using LSF, grd, remote shells, and as a local process. Process information is available and can be obtained using the `report_activity_subproc` command. After a process is complete or stopped, email can be sent to the user. A default mail file is generated but users can set the MailFile parameter on the activity to use a different file.

Examples

launch_activity_subproc ListDirectory "ls -F \$home"

launch_activity_subproc ListDirectory "ls -F \$home" ListDirScript.scr

See Also

[define activity_subproc_params \(2\)](#), [report_activity_subproc \(2\)](#).

[wait_for_activity_subproc \(2\)](#), [prereq_activities_complete \(2\)](#)

NAME

`auto_execok`, `auto_import`, `auto_load`, `auto_mkindex`,
`auto_qualify`, `auto_reset`, `tcl_findLibrary`, `parray`,
`tcl_endOfWord`, `tcl_startOfNextWord`,
`tcl_startOfPreviousWord`, `tcl_wordBreakAfter`,
`tcl_wordBreakBefore` - standard library of Tcl
procedures

SYNOPSIS

```
auto_execok cmd  
auto_import pattern  
auto_load cmd  
auto_mkindex dir pattern pattern ...  
auto_qualify command namespace  
auto_reset  
tcl_findLibrary basename version patch initScript enVarName varName  
parray arrayName ?pattern?  
tcl_endOfWord str start  
tcl_startOfNextWord str start  
tcl_startOfPreviousWord str start  
tcl_wordBreakAfter str start  
tcl_wordBreakBefore str start
```

INTRODUCTION

Tcl includes a library of Tcl procedures for commonly-needed functions. The procedures defined in the Tcl library are generic ones suitable for use by many different applications. The location of the Tcl library is returned by the **info library** command. In addition to the Tcl library, each application will normally have its own library of support procedures as well; the location of this library is normally given by the value of the **\$app_library** global variable, where *app* is the name of the application. For example, the location of the Tk library is kept in the variable

tk_library.

To access the procedures in the Tcl library, an application should source the file **init.tcl** in the library, for example with the Tcl command

```
source [file join [info library] init.tcl]
```

If the library procedure **Tcl_Init** is invoked from an application's **Tcl_AppInit** procedure, this happens automatically. The code in **init.tcl** will define the **unknown** procedure and arrange for the other procedures to be loaded on-demand using the auto-load mechanism defined below.

COMMAND PROCEDURES

The following procedures are provided in the Tcl library:

auto_execok *cmd*

Determines whether there is an executable file or shell builtin by the name *cmd*. If so, it returns a list of arguments to be passed to **exec** to execute the executable file or shell builtin named by *cmd*. If not, it returns an empty string. This command examines the directories in the current search path (given by the **PATH** environment variable) in its search for an executable file named *cmd*. On Windows platforms, the search is expanded with the same directories and file extensions as used by **exec**. **Auto_execok** remembers information about previous searches in an array named **auto_execs**; this avoids the path search in future calls for the same *cmd*. The command **auto_reset** may be used to force **auto_execok** to forget its cached information.

auto_import *pattern*

Auto_import is invoked during **namespace import** to see if the imported commands specified by *pattern* reside in an autoloaded library. If so, the commands are loaded so that they will be available to the interpreter for creating the import links. If the commands do not reside in an autoloaded library, **auto_import** does nothing. The pattern matching is performed according to the matching rules of **namespace import**.

auto_load *cmd*

This command attempts to load the definition for a Tcl command named *cmd*. To do this, it searches an *auto-load path*, which is a list of one or more directories. The auto-load path is given by the global variable **auto_path** if it exists. If there is no **auto_path** variable, then the **TCLLIBPATH** environment variable is used, if it exists. Otherwise the auto-load path consists of just the Tcl library directory. Within each directory in the auto-load path there must be a file **tclIndex** that describes one or more commands defined in that directory and a script to evaluate to

coreTools Command Reference Index

load each of the commands. The **tclIndex** file should be generated with the **auto_mkindex** command. If *cmd* is found in an index file, then the appropriate script is evaluated to create the command. The **auto_load** command returns 1 if *cmd* was successfully created. The command returns 0 if there was no index entry for *cmd* or if the script did not actually define *cmd* (e.g. because index information is out of date). If an error occurs while processing the script, then that error is returned. **Auto_load** only reads the index information once and saves it in the array **auto_index**; future calls to **auto_load** check for *cmd* in the array rather than re-reading the index files. The cached index information may be deleted with the command **auto_reset**. This will force the next **auto_load** command to reload the index database from disk.

auto_mkindex *dir pattern pattern ...*

Generates an index suitable for use by **auto_load**. The command searches *dir* for all files whose names match any of the *pattern* arguments (matching is done with the **glob** command), generates an index of all the Tcl command procedures defined in all the matching files, and stores the index information in a file named **tclIndex** in *dir*. If no pattern is given a pattern of ***.tcl** will be assumed. For example, the command

auto_mkindex **foo *.tcl**

will read all the **.tcl** files in subdirectory **foo** and generate a new index file **foo/tclIndex**.

Auto_mkindex parses the Tcl scripts by sourcing them into a child interpreter and monitoring the proc and namespace commands that are executed. Extensions can use the (undocumented) **auto_mkindex_parser** package to register other commands that can contribute to the **auto_load** index. You will have to read through **auto.tcl** to see how this works.

Auto_mkindex_old (which has the same syntax as **auto_mkindex**) parses the Tcl scripts in a relatively unsophisticated way: if any line contains the word as its first characters then it is assumed to be a procedure definition and the next word of the line is taken as the procedure's name. Procedure definitions that do not appear in this way (e.g. they have spaces before the **proc**) will not be indexed. If your script contains code, such as global initialization code or procedure names with special characters like **\$**, *****, **[** or **]**, you are safer using **auto_mkindex_old**.

auto_reset

Destroys all the information cached by **auto_execok** and **auto_load**. This information will be re-read from disk the next time it is needed. **Auto_reset** also deletes any procedures listed in the **auto-load** index, so that fresh copies of them will be loaded the next time that they are used.

auto_qualify *command namespace*

Computes a list of fully qualified names for *command*.

coreTools Command Reference Index

This list mirrors the path a standard Tcl interpreter follows for command lookups: first it looks for the command in the current namespace, and then in the global namespace. Accordingly, if *command* is relative and *namespace* is not ::, the list returned has two elements: *command* scoped by *namespace*, as if it were a command in the *namespace* namespace; and *command* as if it were a command in the global namespace. Otherwise, if either *command* is absolute (it begins with ::), or *namespace* is ::, the list contains only *command* as if it were a command in the global namespace.

Auto_qualify is used by the auto-loading facilities in Tcl, both for producing auto-loading indexes such as *pkgIndex.tcl*, and for performing the actual auto-loading of functions at runtime.

varName

tcl_findLibrary *basename version patch initScript*
enVarName

This is a standard search procedure for use by extensions during their initialization. They call this procedure to look for their script library in several standard directories. The last component of the name of the library directory is normally *basenameversion* (e.g., tk8.0), but it might be when in the build hierarchies. The *initScript* file will be sourced into the interpreter once it is found. The directory in which this file is found is stored into the global variable *varName*. If this variable is already defined (e.g., by C code during application initialization) then no searching is done. Otherwise the search looks in these directories: the directory named by the environment variable *enVarName*; relative to the Tcl library directory; relative to the executable file in the standard installation bin or bin/*arch* directory; relative to the executable file in the current build tree; relative to the executable file in a parallel build tree.

parray *arrayName ?pattern?*

Prints on standard output the names and values of all the elements in the array *arrayName*, or just the names that match *pattern* (using the matching rules of **string match**) and their values if *pattern* is given. *ArrayName* must be an array accessible to the caller of **parray**. It may be either local or global.

tcl_endOfWord *str start*

Returns the index of the first end-of-word location that occurs after a starting index *start* in the string *str*. An end-of-word location is defined to be the first non-word character following the first word character after the starting point. Returns -1 if there are no more end-of-word locations after the starting point. See the description of **tcl_wordchars** and **tcl_nonwordchars** below for more details on how Tcl determines which characters are word characters.

tcl_startOfNextWord *str start*

Returns the index of the first start-of-word location that occurs after a starting index *start* in the string *str*. A start-of-word location is defined to be the first word character following a non-word character. Returns -1 if there are no more start-of-word locations after the starting point.

tcl_startOfPreviousWord *str start*

Returns the index of the first start-of-word location that occurs before a starting index *start* in the string *str*. Returns -1 if there are no more start-of-word locations before the starting point.

tcl_wordBreakAfter *str start*

Returns the index of the first word boundary after the starting index *start* in the string *str*. Returns -1 if there are no more boundaries after the starting point in the given string. The index returned refers to the second character of the pair that comprises a boundary.

tcl_wordBreakBefore *str start*

Returns the index of the first word boundary before the starting index *start* in the string *str*. Returns -1 if there are no more boundaries before the starting point in the given string. The index returned refers to the second character of the pair that comprises a boundary.

VARIABLES

The following global variables are defined or used by the procedures in the Tcl library. They fall into two broad classes, handling unknown commands and packages, and determining what are words.

AUTOLOADING AND PACKAGE MANAGEMENT VARIABLES

auto_execs

Used by **auto_execok** to record information about whether particular commands exist as executable files.

auto_index

Used by **auto_load** to save the index information read from disk.

auto_noexec

If set to any value, then **unknown** will not attempt to auto-exec any commands.

auto_noload

If set to any value, then **unknown** will not attempt to auto-load any commands.

auto_path

If set, then it must contain a valid Tcl list giving directories to search during auto-load operations (including for package index files when using the default **package unknown** handler). This variable is initialized during startup to contain, in order: the

coreTools Command Reference Index

directories listed in the **TCLLIBPATH** environment variable, the directory named by the **tcl_library** global variable, the parent directory of **tcl_library**, the directories listed in the **tcl_pkgPath** variable. Additional locations to look for files and package indices should normally be added to this variable using **lappend**.

env(TCL_LIBRARY)

If set, then it specifies the location of the directory containing library scripts (the value of this variable will be assigned to the **tcl_library** variable and therefore returned by the command **info library**). If this variable is not set then a default value is used.

env(TCLLIBPATH)

If set, then it must contain a valid Tcl list giving directories to search during auto-load operations. Directories must be specified in Tcl format, using as the path separator, regardless of platform. This variable is only used when initializing the **auto_path** variable.

WORD BOUNDARY DETERMINATION VARIABLES

These variables are only used in the **tcl_endOfWord**, **tcl_startOfNextWord**, **tcl_startOfPreviousWord**, **tcl_wordBreakAfter**, and **tcl_wordBreakBefore** commands.

tcl_nonwordchars

This variable contains a regular expression that is used by routines like **tcl_endOfWord** to identify whether a character is part of a word or not. If the pattern matches a character, the character is considered to be a non-word character. On Windows platforms, spaces, tabs, and newlines are considered non-word characters. Under Unix, everything but numbers, letters and underscores are considered non-word characters.

tcl_wordchars

This variable contains a regular expression that is used by routines like **tcl_endOfWord** to identify whether a character is part of a word or not. If the pattern matches a character, the character is considered to be a word character. On Windows platforms, words are comprised of any character that is not a space, tab, or newline. Under Unix, words are comprised of numbers, letters or underscores.

SEE ALSO

env(n), **info(n)**, **re_syntax(n)**

KEYWORDS

auto-exec, **auto-load**, **library**, **unknown**, **word**, **whitespace**

NAME

`lindex` - Retrieve an element from a list

SYNOPSIS

```
lindex list ?index ...?
```

DESCRIPTION

The **lindex** command accepts a parameter, *list*, which it treats as a Tcl list. It also accepts zero or more *indices* into the list. The indices may be presented either consecutively on the command line, or grouped in a Tcl list and presented as a single argument.

If no indices are presented, the command takes the form:

```
lindex list
```

or

```
lindex list {}
```

In this case, the return value of **lindex** is simply the value of the *list* parameter.

When presented with a single index, the **lindex** command treats *list* as a Tcl list and returns the *index*'th element from it (0 refers to the first element of the list). In extracting the element, **lindex** observes the same rules concerning braces and quotes and backslashes as the Tcl command interpreter; however, variable substitution and command substitution do not occur. If *index* is negative or greater than or equal to the number of elements in *value*, then an empty string is returned. The interpretation of each simple *index*

coreTools Command Reference Index

value is the same as for the command **string index**, supporting simple index arithmetic and indices relative to the end of the list.

If additional *index* arguments are supplied, then each argument is used in turn to select an element from the previous indexing operation, allowing the script to select elements from sublists. The command,

```
lindex $a 1 2 3
```

or

```
lindex $a {1 2 3}
```

is synonymous with

```
lindex [lindex [lindex $a 1] 2] 3
```

EXAMPLES

Lists can be indexed into from either end:

```
lindex {a b c} 0  
-> a  
lindex {a b c} 2  
-> c  
lindex {a b c} end  
-> c  
lindex {a b c} end-1  
-> b
```

Lists or sequences of indices allow selection into lists of lists:

```
lindex {a b c}  
-> a b c  
lindex {a b c} {}  
-> a b c  
lindex {{a b c} {d e f} {g h i}} 2 1  
-> h  
lindex {{a b c} {d e f} {g h i}} {2 1}  
-> h  
lindex {{{a b} {c d}} {{e f} {g h}}} 1 1 0  
-> g  
lindex {{{a b} {c d}} {{e f} {g h}}} {1 1 0}  
-> g
```

List indices may also perform limited computation, adding or subtracting fixed amounts from other indices:

```
set idx 1  
lindex {a b c d e f} $idx+2  
-> d  
set idx 3  
lindex {a b c d e f} $idx+2  
-> f
```

SEE ALSO

`list(n)`, `lappend(n)`, `linsert(n)`, `llength(n)`,
`lsearch(n)`, `lset(n)`, `lsort(n)`, `lrange(n)`, `lreplace(n)`,
`string(n)`

KEYWORDS

`element`, `index`, `list`

NAME

linsert - Insert elements into a list

SYNOPSIS

```
linsert list index ?element element ...?
```

DESCRIPTION

This command produces a new list from *list* by inserting all of the *element* arguments just before the *index*'th element of *list*. Each *element* argument will become a separate element of the new list. If *index* is less than or equal to zero, then the new elements are inserted at the beginning of the list, and if *index* is greater or equal to the length of *list*, it is as if it was **end**. As with **string index**, the *index* value supports both simple index arithmetic and end-relative indexing.

Subject to the restrictions that indices must refer to locations inside the list and that the *elements* will always be inserted in order, insertions are done so that when *index* is start-relative, the first *element* will be at that index in the resulting list, and when *index* is end-relative, the last *element* will be at that index in the resulting list.

EXAMPLE

Putting some values into a list, first indexing from the start and then indexing from the end, and then chaining them together:

coreTools Command Reference Index

```
set oldList {the fox jumps over the dog}
set midList [linsert $oldList 1 quick]
set newList [linsert $midList end-1 lazy]
# The old lists still exist though...
set newerList [linsert [linsert $oldList end-1 quick] 1 lazy]
```

SEE ALSO

list(n), lappend(n), lindex(n), llength(n), lsearch(n),
lset(n), lsort(n), lrange(n), lreplace(n), string(n)

KEYWORDS

element, insert, list

NAME

`list` - Create a list

SYNOPSIS

`list ?arg arg ...?`

DESCRIPTION

This command returns a list comprised of all the *args*, or an empty string if no *args* are specified. Braces and backslashes get added as necessary, so that the **index** command may be used on the result to re-extract the original arguments, and also so that **eval** may be used to execute the resulting list, with *arg1* comprising the command's name and the other *args* comprising its arguments. **List** produces slightly different results than **concat**: **concat** removes one level of grouping before forming the list, while **list** works directly from the original arguments.

EXAMPLE

The command

```
list a b "c d e " " f {g h}"
```

will return

```
a b {c d e } { f {g h}}
```

while **concat** with the same arguments will return

a b c d e f {g h}

SEE ALSO

`lappend(n)`, `lindex(n)`, `linsert(n)`, `llength(n)`,
`lrange(n)`, `lrepeat(n)`, `lreplace(n)`, `lsearch(n)`,
`lset(n)`, `lsort(n)`

KEYWORDS

element, list, quoting

`list_kb`

List all currently loaded knowledge databases

Syntax

string `list_kb`

Description

The `list_kb` command lists all of the currently loaded knowledge databases (KBs). The returned list includes the name and complete filename of each currently loaded KB.

Examples

To list all currently loaded KBs:

```
coreConsultant> list_kb
<KB name> <source file>
=====
work.subblock_b /usr/fred/MyCore/Config/kb/work.subblock_b.kb
work.subblock_a /usr/fred/MyCore/Config/kb/work.subblock_a.kb
ieee.std_logic_1164 /tools/synopsys/coreConsultant/libraries/dware/
ieee.std_logic_1164.kb
lca500k.db /usr/fred/MyCore/Config/kb/lca500k.db.kb
work.top /usr/fred/MyCore/Config/kb/work.top.kb
ieee.std_logic_unsigned /tools/synopsys/coreConsultant/libraries/dware/
ieee.std_logic_unsigned.kb
core /usr/fred/MyCore/Config/kb/core.kb
std.standard /tools/synopsys/coreConsultant/libraries/dware/
std.standard.kb
> HdrConfig /usr/fred/MyCore/Config/kb/HdrConfig.kb
_auxDesignData /usr/fred/MyCore/Config/kb/_auxDesignData.kb
```

See Also

[current_kb \(2\)](#), [get_workspace_kb \(2\)](#)

NAME

`llength` - Count the number of elements in a list

SYNOPSIS

`llength list`

DESCRIPTION

Treats *list* as a list and returns a decimal string giving the number of elements in it.

EXAMPLES

The result is the number of elements:

```
% llength {a b c d e}
5
% llength {a b c}
3
% llength {}
0
```

Elements are not guaranteed to be exactly words in a dictionary sense of course, especially when quoting is used:

```
% llength {a b {c d} e}
4
% llength {a b { } c d e}
6
```

An empty list is not necessarily an empty string:

```
% set var { }; puts "[string length $var],[length $var]"  
1,0
```

SEE ALSO

list(n), lappend(n), lindex(n), linsert(n), lsearch(n),
lset(n), lsort(n), lrange(n), lreplace(n)

KEYWORDS

element, list, length

NAME

`lmap` - Iterate over all elements in one or more lists and collect results

SYNOPSIS

```
lmap varname list body  
lmap varlist1 list1 ?varlist2 list2 ...? body
```

DESCRIPTION

The `lmap` command implements a loop where the loop variable(s) take on values from one or more lists, and the loop returns a list of results collected from each iteration.

In the simplest case there is one loop variable, *varname*, and one list, *list*, that is a list of values to assign to *varname*. The *body* argument is a Tcl script. For each element of *list* (in order from first to last), `lmap` assigns the contents of the element to *varname* as if the `lindex` command had been used to extract the element, then calls the Tcl interpreter to execute *body*. If execution of the body completes normally then the result of the body is appended to an accumulator list. `lmap` returns the accumulator list.

In the general case there can be more than one value list (e.g., *list1* and *list2*), and each value list can be associated with a list of loop variables (e.g., *varlist1* and *varlist2*). During each iteration of the loop the variables of each *varlist* are assigned consecutive values from the corresponding *list*. Values in each *list* are used in order from first to last, and each value is used exactly once. The total number of loop iterations is large enough to use up all the values from all the value lists. If a value list does

coreTools Command Reference Index

not contain enough elements for each of its loop variables in each iteration, empty values are used for the missing elements.

The **break** and **continue** statements may be invoked inside *body*, with the same effect as in the **for** and **foreach** commands. In these cases the body does not complete normally and the result is not appended to the accumulator list.

EXAMPLES

Zip lists together:

```
set list1 {a b c d}
set list2 {1 2 3 4}
set zipped [lmap a $list1 b $list2 {list $a $b}]
# The value of zipped is "{a 1} {b 2} {c 3} {d 4}"
```

Filter a list to remove odd values:

```
set values {1 2 3 4 5 6 7 8}
proc isEven {n} {expr {($n % 2) == 0}}
set goodOnes [lmap x $values {expr {
    [isEven $x] ? $x : [continue]
}}]
# The value of goodOnes is "2 4 6 8"
```

Take a prefix from a list based on the contents of the list:

```
set values {8 7 6 5 4 3 2 1}
proc isGood {counter} {expr {$n > 3}}
set prefix [lmap x $values {expr {
    [isGood $x] ? $x : [break]
}}]
# The value of prefix is "8 7 6 5 4"
```

SEE ALSO

`break(n)`, `continue(n)`, `for(n)`, `foreach(n)`, `while(n)`

KEYWORDS

`foreach`, `iteration`, `list`, `loop`, `map`

NAME

lminus Removes one or more named elements from a list and returns a new list.

SYNTAX

```
list lminus
    [-exact]
    original_list
    elements
```

Data Types

<i>original_list</i>	list
<i>elements</i>	list

ARGUMENTS

[-exact] Specifies that the exact pattern is to be matched. By default, **lminus** uses the default match mode of **lsearch**, the **-glob** mode.

original_list Specifies the list to copy and modify.

elements Specifies a list of elements to remove from *original_list*.

DESCRIPTION

The **lminus** command removes elements from a list by using the element itself, rather than the index of the element in the list (as in **lreplace**). The **lminus** command uses the **lsearch** and **lreplace** commands to find the elements and replace them with nothing.

If none of the elements are found, a copy of *original_list* is returned.

The **lminus** command is often used in the translation of

coreTools Command Reference Index

Design Compiler scripts that use the subtraction operator (-) to remove elements from a list.

EXAMPLES

The following example shows the use of the **lminus** command. Notice that no error message is issued if a specified element is not in the list.

```
prompt> set l1 {a b c}
a b c

prompt> set l2 [lminus $l1 {a b d}]
c

prompt> set l3 [lminus $l1 d]
a b c
```

The following example illustrates the use of **lminus** with the **-exact** option:

```
prompt> set l1 {a a[1] a* b[1] b c}
a a[1] a* b[1] b c

prompt> set l2 [lminus $l1 a*]
{b[1]} b c

prompt> set l3 [lminus -exact $l1 a*]
a {a[1]} {b[1]} b c

prompt> set l4 [lminus -exact $l1 {a[1] b[1]} ]
a a* b c
```

SEE ALSO

lreplace(2)
lsearch(2)

NAME

`load` - Load machine code and initialize new commands

SYNOPSIS

```
load ?-global? ?-lazy? ?--? fileName
load ?-global? ?-lazy? ?--? fileName packageName
load ?-global? ?-lazy? ?--? fileName packageName interp
```

DESCRIPTION

This command loads binary code from a file into the application's address space and calls an initialization procedure in the package to incorporate it into an interpreter. *fileName* is the name of the file containing the code; its exact form varies from system to system but on most systems it is a shared library, such as a `.so` file under Solaris or a DLL under Windows. *packageName* is the name of the package, and is used to compute the name of an initialization procedure. *interp* is the path name of the interpreter into which to load the package (see the **interp** manual entry for details); if *interp* is omitted, it defaults to the interpreter in which the **load** command was invoked.

Once the file has been loaded into the application's address space, one of two initialization procedures will be invoked in the new code. Typically the initialization procedure will add new commands to a Tcl interpreter. The name of the initialization procedure is determined by *packageName* and whether or not the target interpreter is a safe one. For normal interpreters the name of the initialization procedure will have the form *pkg_Init*, where *pkg* is the same as *packageName* except that the first letter is converted to upper case and all other letters are converted to

coreTools Command Reference Index

lower case. For example, if *packageName* is **foo** or **F0o**, the initialization procedure's name will be **Foo_Init**.

If the target interpreter is a safe interpreter, then the name of the initialization procedure will be *pkg_SafeInit* instead of *pkg_Init*. The *pkg_SafeInit* function should be written carefully, so that it initializes the safe interpreter only with partial functionality provided by the package that is safe for use by untrusted code. For more information on Safe-Tcl, see the **safe** manual entry.

The initialization procedure must match the following prototype:

```
typedef int Tcl_PackageInitProc(  
    Tcl_Interp *interp);
```

The *interp* argument identifies the interpreter in which the package is to be loaded. The initialization procedure must return **TCL_OK** or **TCL_ERROR** to indicate whether or not it completed successfully; in the event of an error it should set the interpreter's result to point to an error message. The result of the **load** command will be the result returned by the initialization procedure.

The actual loading of a file will only be done once for each *fileName* in an application. If a given *fileName* is loaded into multiple interpreters, then the first **load** will load the code and call the initialization procedure; subsequent **loads** will call the initialization procedure without loading the code again. For Tcl versions lower than 8.5, it is not possible to unload or reload a package. From version 8.5 however, the **unload** command allows the unloading of libraries loaded with **load**, for libraries that are aware of the Tcl's unloading mechanism.

The **load** command also supports packages that are statically linked with the application, if those packages have been registered by calling the **Tcl_StaticPackage** procedure. If *fileName* is an empty string, then *packageName* must be specified.

If *packageName* is omitted or specified as an empty string, Tcl tries to guess the name of the package. This may be done differently on different platforms. The default guess, which is used on most UNIX platforms, is to take the last element of *fileName*, strip off the first three characters if they are **lib**, and use any following alphabetic and underline characters as the module name. For example, the command **load libxyz4.2.so** uses the module name **xyz** and the command **load bin/last.so {}** uses the module name **last**.

If *fileName* is an empty string, then *packageName* must be specified. The **load** command first searches for a statically loaded package (one that has been registered by calling the **Tcl_StaticPackage** procedure) by that name; if one is found, it is used. Otherwise, the **load** command searches for a dynamically loaded package by

coreTools Command Reference Index

that name, and uses it if it is found. If several different files have been **loaded** with different versions of the package, Tcl picks the file that was loaded first.

If **-global** is specified preceding the filename, all symbols found in the shared library are exported for global use by other libraries. The option **-lazy** delays the actual loading of symbols until their first actual use. The options may be abbreviated. The option **--** indicates the end of the options, and should be used if you wish to use a filename which starts with **-** and you provide a packageName to the **load** command.

On platforms which do not support the **-global** or **-lazy** options, the options still exist but have no effect. Note that use of the **-global** or **-lazy** option may lead to crashes in your application later (in case of symbol conflicts resp. missing symbols), which cannot be detected during the **load**. So, only use this when you know what you are doing, you will not get a nice error message when something is wrong with the loaded library.

PORTABILITY ISSUES

Windows

When a load fails with error, it is also possible that a dependent library was not found. To see the dependent libraries, type in a DOS console to see what the library must import. When loading a DLL in the current directory, Windows will ignore as a path specifier and use a search heuristic to find the DLL instead. To avoid this, load the DLL with:

```
load [file join [pwd] mylib.DLL]
```

BUGS

If the same file is **loaded** by different *fileNames*, it will be loaded into the process's address space multiple times. The behavior of this varies from system to system (some systems may detect the redundant loads, others may not).

EXAMPLE

The following is a minimal extension:

```
#include <tcl.h>
#include <stdio.h>
static int fooCmd(ClientData clientData,
                  Tcl_Interp *interp, int objc, Tcl_Obj *const objv[]) {
```

coreTools Command Reference Index

```
printf("called with %d arguments\n", objc);
return TCL_OK;
}
int Foo_Init(Tcl_Interp *interp) {
    if (Tcl_InitStubs(interp, "8.1", 0) == NULL) {
        return TCL_ERROR;
    }
    printf("creating foo command");
    Tcl_CreateObjCommand(interp, "foo", fooCmd, NULL, NULL);
    return TCL_OK;
}
```

When built into a shared/dynamic library with a suitable name (e.g. **foo.dll** on Windows, **libfoo.so** on Solaris and Linux) it can then be loaded into Tcl with the following:

```
# Load the extension
switch $tcl_platform(platform) {
    windows {
        load [file join [pwd] foo.dll]
    }
    unix {
        load [file join [pwd] libfoo[info sharedlibextension]]
    }
}

# Now execute the command defined by the extension
foo
```

SEE ALSO

```
info sharedlibextension, package(n),
Tcl_StaticPackage(3), safe(n)
```

KEYWORDS

binary code, dynamic library, load, safe interpreter, shared library

[API Index](#) [Command Reference](#)

load_autoload_filegroup
Populate an autoloaded filegroup

Syntax

string **load_autoload_filegroup** *group*
 string *group*

Parameters

group BOM File group

Description

An auto loaded filegroup is a group that is specified as a list of glob-style patterns. When the BuildcoreKit activity is completed, these patterns determine the files that are put into the coreKit.

This command is used to bring the set of files into the tool before the BuildcoreKit activity is completed. This is not usually needed, unless the user wants to override some attribute settings on some of the files.

Examples

To populate the Documentation group that was created with create_autoload_filegroup:

```
coreBuilder> load_autoload_filegroup Documentation
```

See Also

[create_autoload_filegroup \(2\)](#), [unload_autoload_filegroup \(2\)](#),
[DefaultLoadPath \(3\)](#)

[API Index](#) [Command Reference](#)

load_component_memory_maps

Loads all visible component memory maps from knowledgebase files in cC and cA into memory.

Syntax

```
string load_component_memory_maps [-all] [-map <mapname>]  
      string <mapname>
```

Parameters

-all	Loads all the current components memory maps including nonvisible ones from disk into memory.
-map <mapname>	Loads a single memMap from a map knowledgebase file located in the current components kb/memmaps directory.

Description

Examples

See Also

[unload component memory maps \(2\)](#)

`load_file_into_kb`

Load a file or directory into the current knowledge database

Syntax

```
string load_file_into_kb [-external] [-contents <file contents>] [-empty
<storage type>] [-force] [-install_dir <installation directory>] [-quiet]
[-rename <new_name>] <filename>
    string <file contents>
    string <storage type>
    string <installation directory>
    string <new_name>
    string <filename>
```

Parameters

<code>-external</code>	Create a file that contents are still on disk (not in memory) Specifies text to store in the file. Use the <code>-contents</code> to add content to a new file before storing the file in the current KB. If you use the <code>-contents</code> option when adding an existing file to the KB, <code>load_file_into_kb</code> replaces the existing contents of the specified file with <code><file contents></code> when loading the file into the current KB.
<code>-contents <file contents></code>	Create an empty file or empty directory. (Values: file, directory) If you specify the <code>-empty</code> option, <code>load_file_into_kb</code> loads the specified file or directory as empty even if you are specifying an existing file or directory that has content.
<code>-empty <storage type></code>	Overwrite existing file item with same name in the current KB.
<code>-force</code>	Specifies the directory in which to install the file when unloaded.
<code>-install_dir <installation directory></code>	Suppress file creation message.
<code>-quiet</code>	Rename the specified file or directory before loading.
<code>-rename <new_name> <filename></code>	The name of the file or directory to load You can specify a file or directory that currently exists on disk or create the new file or directory and add it directly into the current KB. If you are creating and adding a file, you can also

specify the file content with the -contents option.

Description

The `load_file_into_kb` reads the specified file or directory and stores it in the current KB. To determine the current KB, use the `current_kb` command.

You can load an existing file or directory or create a new file or directory and add load it in the current KB. To load the file or directory as an empty file or directory, use the `-empty` option.

To add content to a new file or replace the content of a current disk file when adding it to the current KB, use the `-contents` option.

`load_file_into_kb` automatically compresses directories when adding them to the current KB. If you want to compress a file, use the `-compress` option.

To unload a file or directory from the current KB (that is, write out the file or directory to disk), use the `unload_file_from_kb` command.

To specify a directory into which to unload the file, use the `-install_dir` option when you execute the `load_file_into_kb` command. When you use the `-install_dir`, `load_file_into_kb` prepends the specified directory <installation directory> to the file name. To access the file object with the `unload_file_from_kb` command, you must refer to the file as <installation directory><filename>.

Examples

To load the file `myfile` into the current KB:

```
coreBuilder> load_file_into_kb myfile
Information: Created file: myfile (CMD5-39)
{myfile}
```

To load the directory `mydir` into the current KB:

```
coreBuilder> load_file_into_kb mydir
Information: Created file: mydir (CMD5-39)
{mydir}
```

To load the file `myfile` into the current KB, but compress it first:

```
coreBuilder> load_file_into_kb -compress myfile
Information: Created file: myfile (CMD5-39)
{myfile}
```

See Also

[unload file from kb \(2\)](#)

[API Index](#) [Command Reference](#)

load_interface_definitions
Load interface definitions.

Syntax

string **load_interface_definitions** *files*
 string *files*

Parameters

files List of tcl or XML files

Description

This command is used to load definitions of interfaces into coreAssembler. This is only needed for the case where you want to use an interface definition that is not already in use in your subsystem. Files loaded can contain either TCL interface definitions (e.g. defined by create_interface) or IP-XACT bus and abstraction definitions in XML format. The type of file is determined by the file extension. IP-XACT XML files must end with .xml and TCL interface definition files must end with .tcl.

Interface definitions loaded with load_interface_definitions are then available for use with the attach_interface command.

Examples

```
coreAssembler> load_interface_definitions mybus.tcl  
coreAssembler> load_interface_definitions yourbus.xml
```

See Also

[attach_interface \(2\)](#)

load_plugin

Load a plugin KB and install non-configurable files.

Syntax

```
string load_plugin [-copy] name  
      string name
```

Parameters

- copy Copy the plugin KB to workspace instead of linking to it.
 Name of the plugin KB.
- name The name may include a full pathname. The .kb extension is
 optional.

Description

The load_plugin command reads a plugin knowledge database (KB) that was created with the create_plugin_kb command. A plugin is used to alter the default behavior of a coreTool by creating or modifying activities, defining TCL procs, and adding files to the workspace.

The load_plugin command can be executed only when a workspace is open. The KB is read using command read_kb and a link is made to the plugin KB file under the workspace kb directory. If the -copy option is used, then the KB file is copied. The plugin KB object is returned if load_plugin is successful.

Any non-configurable files included in filegroups in the plugin are installed by load_plugin. Configurable files install based on the associated activity.

In coreAssembler, a plugin can be associated with a particular component by running load_plugin with the current component set to the component. Activities defined in the plugin will apply to the component. Also, files in the plugin filegroups will be written in the component workspace (instead of at the top-level of the Subsystem). For a plugin to work correctly when loaded into more than one component, it should be built with the -no_search flag of the create_plugin_kb command.

The plugin can be unloaded with the unload_plugin command.

Examples

To load a plugin KB named MyPlugIn.kb:

```
coreConsultant> load_plugin MyPlugin
```

To load a plugin in a component context in coreAssembler:

```
coreAssembler> set_current_component /i_myComp
```

```
coreAssembler> load_plugin MyPlugin
```

See Also

[create_plugin_kb \(2\), unload_plugin \(2\)](#)

[API Index](#) [Command Reference](#)

load_solution_definition

Load a DesignWare IP Solution definition file.

Syntax

```
string load_solution_definition solnDefnFile  
      string solnDefnFile
```

Parameters

solnDefnFile Solution definition file to load.

Description

Examples

See Also

lock_parameter

Lock the configuration (design) parameter value so it cannot be changed.

Syntax

```
string lock_parameter [-component <component>] [-activity <activity>]
[-strategy <strategy>] parameter
    string <component>
    string <activity>
    string <strategy>
    string parameter
```

Parameters

-component <component>	Component on which you want to lock the parameter value.
-activity <activity>	Activity on which you want to lock the parameter value.
-strategy <strategy>	Strategy on which you want to lock the parameter value.
parameter	The name of the parameter that you want to lock.

Description

This command is used to lock a parameter to a particular value. Once locked, the value cannot be changed in the given workspace.

Examples

```
coreConsultant> lock_parameter -strategy
Design_Compiler_Reference_Methodology -setValue -value MyOptions
CompileUltraInitOptions
coreAssembler> lock_parameter -activity TechSetup -setValue -value
MyOptions CreateMwDesignOptions
coreAssembler> lock_parameter -component MyComp1 Param3
coreAssembler> set_current_component MyComp1
coreAssembler> lock_parameter Param3
```

See Also

NAME

log_trace Control creation of a replay log of traced commands.

SYNTAX

```
string log_trace [-start  
file_name|-stop|-pause|-resume] [-log log_string]  
[-quiet]
```

```
string file_name string log_string
```

ARGUMENTS

-start *file-Name*

This option is mutually exclusive with **-stop**, **-pause**, and **-resume**. The argument is the pathname of the file to which the replay trace log is output. The option opens a file for output and begins logging command execution strings and application variable changes. Note that if the file already exists, the command overwrites the existing file. There can only be a single replay trace log output at any time.

-stop

This option is mutually exclusive with **-start**, **-pause**, and **-resume**. This option stops replay trace log output and closes the log file. Once stopped, the replay log cannot be reopened for appending.

-pause

This option is mutually exclusive with **-start**, **-stop**, and **-resume**. This option pauses replay trace log output. Execution of all commands and application variable settings are ignored and omitted from the log until output is resumed with **-resume**.

Note that pausing replay log output may render the resulting log unable to

coreTools Command Reference Index

- replay or incapable of replicating tool results.
- resume This option is mutually exclusive with -start, -stop, and -pause. This option resumes replay trace log output.
- log *log_string* This option outputs the given *log_string*, verbatim, to the replay trace log. The option may be used to explicitly log a command invocation or a comment. If the string is a comment, it must start with the Tcl comment character '#'.
- quiet If given, this option causes the command to ignore error conditions such as an attempt to pause the trace log output before it is started. The command will exit quietly when it encounters an error condition.

DESCRIPTION

The **log_trace** command performs operations related to creating a flat replay log of traced command execution and application variable changes. All options are optional. Upon successful completion, the command returns the pathname of the trace log file, if any, or an error message if the command fails.

Trace log output contains a comment header with the tool name and version, and the date and time when trace log output was started.

The resulting replay log is not 100% guaranteed to be able to replay or to reproduce tool results. One reason is that the tool execution environment may be different. If any operations are performed during log creation that would further compromise the ability to replay the resulting log or to replicate tool results, a warning message is appended to the log. Such operations including pausing log output.

Invoking the **log_trace** with no options will report the current status of logging: on, paused, or off, followed by the currently open log file name, if any.

EXAMPLES

The following example shows reporting of the log status before starting trace logging.

```
prompt> log_trace  
off
```

The following example starts trace log creation.

```
prompt> log_trace -start ./tracelog.tcl  
/an/absolute/path/.tracelog.tcl  
prompt> log_trace  
on: /an/absolute/path/.tracelog.tcl
```

The following example pauses, then resumes log creation.

```
prompt> log_trace -pause  
/an/absolute/path/.tracelog.tcl  
prompt> log_trace  
paused: /an/absolute/path/.tracelog.tcl  
prompt> log_trace -resume  
/an/absolute/path/.tracelog.tcl  
prompt> log_trace  
on: /an/absolute/path/.tracelog.tcl
```

SEE ALSO

```
set_trace_option(2)  
get_trace_option(2)  
annotate_trace(2)
```

NAME

`lrange` - Return one or more adjacent elements from a list

SYNOPSIS

`lrange list first last`

DESCRIPTION

List must be a valid Tcl list. This command will return a new list consisting of elements *first* through *last*, inclusive. The index values *first* and *last* are interpreted the same as index values for the command **string index**, supporting simple index arithmetic and indices relative to the end of the list. If *first* is less than zero, it is treated as if it were zero. If *last* is greater than or equal to the number of elements in the list, then it is treated as if it were **end**. If *first* is greater than *last* then an empty string is returned. Note: does not always produce the same result as (although it often does for simple fields that are not enclosed in braces); it does, however, produce exactly the same results as

EXAMPLES

Selecting the first two elements:

```
% lrange {a b c d e} 0 1
a b
```

Selecting the last three elements:

coreTools Command Reference Index

```
% lrange {a b c d e} end-2 end  
c d e
```

Selecting everything except the first and last element:

```
% lrange {a b c d e} 1 end-1  
b c d
```

Selecting a single element with **lrange** is not the same as doing so with **lindex**:

```
% set var {some {elements to} select}  
some {elements to} select  
% lindex $var 1  
elements to  
% lrange $var 1 1  
{elements to}
```

SEE ALSO

`list(n)`, `lappend(n)`, `lindex(n)`, `linsert(n)`, `llength(n)`,
`lsearch(n)`, `lset(n)`, `lreplace(n)`, `lsort(n)`, `string(n)`

KEYWORDS

element, list, range, sublist

NAME

`lrepeat` - Build a list by repeating elements

SYNOPSIS

`lrepeat` *count* *?element ...?*

DESCRIPTION

The **lrepeat** command creates a list of size *count* * *number of elements* by repeating *count* times the sequence of elements *element* *count* must be a non-negative integer, *element* can be any Tcl value. Note that **lrepeat 1 element ...** is identical to **list element**

EXAMPLES

```
lrepeat 3 a
-> a a a
lrepeat 3 [lrepeat 3 0]
-> {0 0 0} {0 0 0} {0 0 0}
lrepeat 3 a b c
-> a b c a b c a b c
lrepeat 3 [lrepeat 2 a] b c
-> {a a} b c {a a} b c {a a} b c
```

SEE ALSO

`list(n)`, `lappend(n)`, `linsert(n)`, `llength(n)`, `lset(n)`

KEYWORDS

`element`, `index`, `list`

NAME

`lreplace` - Replace elements in a list with new elements

SYNOPSIS

`lreplace list first last ?element element ...?`

DESCRIPTION

lreplace returns a new list formed by replacing zero or more elements of *list* with the *element* arguments. *first* and *last* are index values specifying the first and last elements of the range to replace. The index values *first* and *last* are interpreted the same as index values for the command **string index**, supporting simple index arithmetic and indices relative to the end of the list. 0 refers to the first element of the list, and **end** refers to the last element of the list.

If either *first* or *last* is less than zero, it is considered to refer to before the first element of the list. This allows **lreplace** to prepend elements to *list*. If either *first* or *last* indicates a position greater than the index of the last element of the list, it is treated as if it is an index one greater than the last element. This allows **lreplace** to append elements to *list*.

If *last* is less than *first*, then any specified elements will be inserted into the list before the element specified by *first* with no elements being deleted.

The *element* arguments specify zero or more new elements to be added to the list in place of those that were deleted. Each *element* argument will become a separate element of the list. If no *element* arguments are specified, then the elements between *first* and *last* are

simply deleted.

EXAMPLES

Replacing an element of a list with another:

```
% lreplace {a b c d e} 1 1 foo
a foo c d e
```

Replacing two elements of a list with three:

```
% lreplace {a b c d e} 1 2 three more elements
a three more elements d e
```

Deleting the last element from a list in a variable:

```
% set var {a b c d e}
a b c d e
% set var [lreplace $var end end]
a b c d
```

A procedure to delete a given element from a list:

```
proc lremove {listVariable value} {
  upvar 1 $listVariable var
  set idx [lsearch -exact $var $value]
  set var [lreplace $var $idx $idx]
}
```

Appending elements to the list; note that **end+2** will initially be treated as if it is **6** here, but both that and **12345** are greater than the index of the final item so they behave identically:

```
% set var {a b c d e}
a b c d e
% set var [lreplace $var 12345 end+2 f g h i]
a b c d e f g h i
```

SEE ALSO

list(n), lappend(n), lindex(n), linsert(n), llength(n), lsearch(n), lset(n), lrange(n), lsort(n), string(n)

KEYWORDS

element, list, replace

NAME

`lreverse` - Reverse the order of a list

SYNOPSIS

`lreverse list`

DESCRIPTION

The **lreverse** command returns a list that has the same elements as its input list, *list*, except with the elements in the reverse order.

EXAMPLES

```
lreverse {a a b c}  
-> c b a a  
lreverse {a b {c d} e f}  
-> f e {c d} b a
```

SEE ALSO

`list(n)`, `lsearch(n)`, `lsort(n)`

KEYWORDS

element, list, reverse

NAME

ls Lists the contents of a directory.

SYNTAX

```
string ls [filename ...]
```

```
string     filename
```

ARGUMENTS

filename Provides the name of a directory or filename, or a pattern which matches files or directories.

DESCRIPTION

If no argument is specified, the contents of the current directory are listed. For each *filename* matching a directory, **ls** lists the contents of that directory. If *filename* matches a file name, the file name is listed.

EXAMPLES

```
shell> ls *.db *.pt
test1.pt            c1.db                c3.db                c5.db
test2.pt            c2.db                c4.db                c6.db
```

SEE ALSO

cd(2), **pwd(2)**.

NAME

`lsearch` - See if a list contains a particular element

SYNOPSIS

`lsearch ?options? list pattern`

DESCRIPTION

This command searches the elements of *list* to see if one of them matches *pattern*. If so, the command returns the index of the first matching element (unless the options **-all** or **-inline** are specified.) If not, the command returns **-1** or (if options **-all** or **-inline** are specified) the empty string. The *option* arguments indicates how the elements of the list are to be matched against *pattern* and must have one of the values below:

MATCHING STYLE OPTIONS

If all matching style options are omitted, the default matching style is **-glob**. If more than one matching style is specified, the last matching style given takes precedence.

-exact

Pattern is a literal string that is compared for exact equality against each list element.

-glob

Pattern is a glob-style pattern which is matched against each list element using the same rules as the **string match** command.

-regexp

Pattern is treated as a regular expression and matched against each list element using the rules described in

coreTools Command Reference Index

the **re_syntax** reference page.

-sorted

The list elements are in sorted order. If this option is specified, **lsearch** will use a more efficient searching algorithm to search *list*. If no other options are specified, *list* is assumed to be sorted in increasing order, and to contain ASCII strings. This option is mutually exclusive with **-glob** and **-regexp**, and is treated exactly like **-exact** when either **-all** or **-not** are specified.

GENERAL MODIFIER OPTIONS

These options may be given with all matching styles.

-all

Changes the result to be the list of all matching indices (or all matching values if **-inline** is specified as well.) If indices are returned, the indices will be in numeric order. If values are returned, the order of the values will be the order of those values within the input *list*.

-inline

The matching value is returned instead of its index (or an empty string if no value matches.) If **-all** is also specified, then the result of the command is the list of all values that matched.

-not

This negates the sense of the match, returning the index of the first non-matching value in the list.

-start index

The list is searched starting at position *index*. The interpretation of the *index* value is the same as for the command **string index**, supporting simple index arithmetic and indices relative to the end of the list.

CONTENTS DESCRIPTION OPTIONS

These options describe how to interpret the items in the list being searched. They are only meaningful when used with the **-exact** and **-sorted** options. If more than one is specified, the last one takes precedence. The default is **-ascii**.

-ascii

The list elements are to be examined as Unicode strings (the name is for backward-compatibility reasons.)

-dictionary

The list elements are to be compared using dictionary-style comparisons (see **lsort** for a fuller description). Note that this only makes a meaningful difference from the **-ascii** option when the **-sorted** option is given, because values are only dictionary-equal when exactly equal.

-integer

The list elements are to be compared as integers.

coreTools Command Reference Index

-nocase

Causes comparisons to be handled in a case-insensitive manner. Has no effect if combined with the

-dictionary, **-integer**, or **-real** options.

-real

The list elements are to be compared as floating-point values.

SORTED LIST OPTIONS

These options (only meaningful with the **-sorted** option) specify how the list is sorted. If more than one is given, the last one takes precedence. The default option is **-increasing**.

-decreasing

The list elements are sorted in decreasing order. This option is only meaningful when used with **-sorted**.

-increasing

The list elements are sorted in increasing order. This option is only meaningful when used with **-sorted**.

-bisect

Inexact search when the list elements are in sorted order. For an increasing list the last index where the element is less than or equal to the pattern is returned. For a decreasing list the last index where the element is greater than or equal to the pattern is returned. If the pattern is before the first element or the list is empty, -1 is returned. This option implies **-sorted** and cannot be used with either **-all** or **-not**.

NESTED LIST OPTIONS

These options are used to search lists of lists. They may be used with any other options.

-index *indexList*

This option is designed for use when searching within nested lists. The *indexList* argument gives a path of indices (much as might be used with the **lindex** or **lset** commands) within each element to allow the location of the term being matched against.

-subindices

If this option is given, the index result from this command (or every index result when **-all** is also specified) will be a complete path (suitable for use with **lindex** or **lset**) within the overall list to the term found. This option has no effect unless the **-index** is also specified, and is just a convenience short-cut.

EXAMPLES

Basic searching:

```
lsearch {a b c d e} c
-> 2
```

coreTools Command Reference Index

```
lsearch -all {a b c a b c} c  
-> 2 5
```

Using **lsearch** to filter lists:

```
lsearch -inline {a20 b35 c47} b*  
-> b35  
lsearch -inline -not {a20 b35 c47} b*  
-> a20  
lsearch -all -inline -not {a20 b35 c47} b*  
-> a20 c47  
lsearch -all -not {a20 b35 c47} b*  
-> 0 2
```

This can even do a removal operation:

```
lsearch -all -inline -not -exact {a b c a d e a f g a} a  
-> b c d e f g
```

Searching may start part-way through the list:

```
lsearch -start 3 {a b c a b c} c  
-> 5
```

It is also possible to search inside elements:

```
lsearch -index 1 -all -inline {{a abc} {b bcd} {c cde}} *bc*  
-> {a abc} {b bcd}
```

SEE ALSO

`foreach(n)`, `list(n)`, `lappend(n)`, `lindex(n)`, `linsert(n)`,
`llength(n)`, `lset(n)`, `lsort(n)`, `lrange(n)`, `lreplace(n)`,
`string(n)`

KEYWORDS

binary search, linear search, list, match, pattern,
regular expression, search, string

NAME

`lset` - Change an element in a list

SYNOPSIS

```
lset varName ?index ...? newValue
```

DESCRIPTION

The `lset` command accepts a parameter, *varName*, which it interprets as the name of a variable containing a Tcl list. It also accepts zero or more *indices* into the list. The indices may be presented either consecutively on the command line, or grouped in a Tcl list and presented as a single argument. Finally, it accepts a new value for an element of *varName*.

If no indices are presented, the command takes the form:

```
lset varName newValue
```

or

```
lset varName {} newValue
```

In this case, *newValue* replaces the old value of the variable *varName*.

When presented with a single index, the `lset` command treats the content of the *varName* variable as a Tcl list. It addresses the *index*'th element in it (0 refers to the first element of the list). When interpreting the list, `lset` observes the same rules concerning braces and quotes and backslashes as the Tcl command interpreter; however, variable substitution and command substitution do not occur. The command

coreTools Command Reference Index

constructs a new list in which the designated element is replaced with *newValue*. This new list is stored in the variable *varName*, and is also the return value from the **lset** command.

If *index* is negative or greater than the number of elements in *\$varName*, then an error occurs.

If *index* is equal to the number of elements in *\$varName*, then the given element is appended to the list.

The interpretation of each simple *index* value is the same as for the command **string index**, supporting simple index arithmetic and indices relative to the end of the list.

If additional *index* arguments are supplied, then each argument is used in turn to address an element within a sublist designated by the previous indexing operation, allowing the script to alter elements in sublists (or append elements to sublists). The command,

```
lset a 1 2 newValue
```

or

```
lset a {1 2} newValue
```

replaces element 2 of sublist 1 with *newValue*.

The integer appearing in each *index* argument must be greater than or equal to zero. The integer appearing in each *index* argument must be less than or equal to the length of the corresponding list. In other words, the **lset** command can change the size of a list only by appending an element (setting the one after the current end). If an index is outside the permitted range, an error is reported.

EXAMPLES

In each of these examples, the initial value of **x** is:

```
set x [list [list a b c] [list d e f] [list g h i]]  
      -> {a b c} {d e f} {g h i}
```

The indicated return value also becomes the new value of **x** (except in the last case, which is an error which leaves the value of **x** unchanged.)

```
lset x {j k l}  
      -> j k l  
lset x {} {j k l}  
      -> j k l  
lset x 0 j  
      -> j {d e f} {g h i}  
lset x 2 j  
      -> {a b c} {d e f} j
```

coreTools Command Reference Index

```
lset x end j  
-> {a b c} {d e f} j  
lset x end-1 j  
-> {a b c} j {g h i}  
lset x 2 1 j  
-> {a b c} {d e f} {g j i}  
lset x {2 1} j  
-> {a b c} {d e f} {g j i}  
  
lset x {2 3} j  
-> list index out of range
```

In the following examples, the initial value of **x** is:

```
set x [list [list [list a b] [list c d]] \  
      [list [list e f] [list g h]]]  
-> {{a b} {c d}} {{e f} {g h}}
```

The indicated return value also becomes the new value of **x**.

```
lset x 1 1 0 j  
-> {{a b} {c d}} {{e f} {j h}}  
lset x {1 1 0} j  
-> {{a b} {c d}} {{e f} {j h}}
```

SEE ALSO

list(n), lappend(n), lindex(n), linsert(n), llength(n),
lsearch(n), lsort(n), lrange(n), lreplace(n), string(n)

KEYWORDS

element, index, list, replace, set

NAME

`lsort` - Sort the elements of a list

SYNOPSIS

`lsort ?options? list`

DESCRIPTION

This command sorts the elements of *list*, returning a new list in sorted order. The implementation of the **lsort** command uses the merge-sort algorithm which is a stable sort that has $O(n \log n)$ performance characteristics.

By default ASCII sorting is used with the result returned in increasing order. However, any of the following options may be specified before *list* to control the sorting process (unique abbreviations are accepted):

-ascii

Use string comparison with Unicode code-point collation order (the name is for backward-compatibility reasons.) This is the default.

-dictionary

Use dictionary-style comparison. This is the same as **-ascii** except (a) case is ignored except as a tie-breaker and (b) if two strings contain embedded numbers, the numbers compare as integers, not characters. For example, in **-dictionary** mode, **bigBoy** sorts between **bigbang** and **bigboy**, and **x10y** sorts between **x9y** and **x11y**. Overrides the **-nocase** option.

-integer

Convert list elements to integers and use integer

coreTools Command Reference Index

comparison.

-real

Convert list elements to floating-point values and use floating comparison.

-command *command*

Use *command* as a comparison command. To compare two elements, evaluate a Tcl script consisting of *command* with the two elements appended as additional arguments. The script should return an integer less than, equal to, or greater than zero if the first element is to be considered less than, equal to, or greater than the second, respectively.

-increasing

Sort the list in increasing order This is the default.

-decreasing

Sort the list in decreasing order

-indices

Return a list of indices into *list* in sorted order instead of the values themselves.

-index *indexList*

If this option is specified, each of the elements of *list* must itself be a proper Tcl sublist (unless **-stride** is used). Instead of sorting based on whole sublists, **lsort** will extract the *indexList*'th element from each sublist (as if the overall element and the *indexList* were passed to **lindex**) and sort based on the given element. For example,

```
lsort -integer -index 1 \  
    {{First 24} {Second 18} {Third 30}}
```

returns **{Second 18} {First 24} {Third 30}**,

```
lsort -index end-1 \  
    {{a 1 e i} {b 2 3 f g} {c 4 5 6 d h}}
```

returns **{c 4 5 6 d h} {a 1 e i} {b 2 3 f g}**, and

```
lsort -index {0 1} {  
    {{b i g} 12345}  
    {{d e m o} 34512}  
    {{c o d e} 54321}  
}
```

returns **{{d e m o} 34512} {{b i g} 12345} {{c o d e} 54321}** (because **e** sorts before **i** which sorts before **o**.) This option is much more efficient than using **-command** to achieve the same effect.

-stride *strideLength*

If this option is specified, the list is treated as consisting of groups of *strideLength* elements and the groups are sorted by either their first element or, if the **-index** option is used, by the element within each group given by the first index passed to **-index** (which is then ignored by **-index**). Elements always remain in the same position within their group.

coreTools Command Reference Index

The list length must be an integer multiple of *strideLength*, which in turn must be at least 2.

For example,

```
lsort -stride 2 {carrot 10 apple 50 banana 25}
```

returns and

```
lsort -stride 2 -index 1 -integer {carrot 10 apple 50 banana 25}
```

returns

-nocase

Causes comparisons to be handled in a case-insensitive manner. Has no effect if combined with the

-dictionary, **-integer**, or **-real** options.

-unique

If this option is specified, then only the last set of duplicate elements found in the list will be retained. Note that duplicates are determined relative to the comparison used in the sort. Thus if **-index 0** is used, **{1 a}** and **{1 b}** would be considered duplicates and only the second element, **{1 b}**, would be retained.

NOTES

The options to **lsort** only control what sort of comparison is used, and do not necessarily constrain what the values themselves actually are. This distinction is only noticeable when the list to be sorted has fewer than two elements.

The **lsort** command is reentrant, meaning it is safe to use as part of the implementation of a command used in the **-command** option.

EXAMPLES

Sorting a list using ASCII sorting:

```
% lsort {a10 B2 b1 a1 a2}  
B2 a1 a10 a2 b1
```

Sorting a list using Dictionary sorting:

```
% lsort -dictionary {a10 B2 b1 a1 a2}  
a1 a2 a10 b1 B2
```

Sorting lists of integers:

```
% lsort -integer {5 3 1 2 11 4}  
1 2 3 4 5 11  
% lsort -integer {1 2 0x5 7 0 4 -1}
```

coreTools Command Reference Index

-1 0 1 2 4 0x5 7

Sorting lists of floating-point numbers:

```
% lsort -real {5 3 1 2 11 4}
1 2 3 4 5 11
% lsort -real {.5 0.07e1 0.4 6e-1}
0.4 .5 6e-1 0.07e1
```

Sorting using indices:

```
% # Note the space character before the c
% lsort {{a 5} { c 3} {b 4} {e 1} {d 2}}
{ c 3} {a 5} {b 4} {d 2} {e 1}
% lsort -index 0 {{a 5} { c 3} {b 4} {e 1} {d 2}}
{a 5} {b 4} { c 3} {d 2} {e 1}
% lsort -index 1 {{a 5} { c 3} {b 4} {e 1} {d 2}}
{e 1} {d 2} { c 3} {b 4} {a 5}
```

Sorting a dictionary:

```
% set d [dict create c d a b h i f g c e]
c e a b h i f g
% lsort -stride 2 $d
a b c e f g h i
```

Sorting using striding and multiple indices:

```
% # Note the first index value is relative to the group
% lsort -stride 3 -index {0 1} \
    {{Bob Smith} 25 Audi {Jane Doe} 40 Ford}
    {{Jane Doe} 40 Ford {Bob Smith} 25 Audi}
```

Stripping duplicate values using sorting:

```
% lsort -unique {a b c a b c a b c}
a b c
```

More complex sorting using a comparison function:

```
% proc compare {a b} {
    set a0 [lindex $a 0]
    set b0 [lindex $b 0]
    if {$a0 < $b0} {
        return -1
    } elseif {$a0 > $b0} {
        return 1
    }
    return [string compare [lindex $a 1] [lindex $b 1]]
}
% lsort -command compare \
    {{3 apple} {0x2 carrot} {1 dingo} {2 banana}}
    {1 dingo} {2 banana} {0x2 carrot} {3 apple}
```

SEE ALSO

list(n), lappend(n), lindex(n), linsert(n), llength(n),
lsearch(n), lset(n), lrange(n), lreplace(n)

KEYWORDS

element, list, order, sort

NAME

man Displays reference manual pages.

SYNTAX

string **man**
 topic

Data Types
 topic string

ARGUMENTS

topic Specifies the subject to display.
 Available topics include commands,
 variables, and error messages

DESCRIPTION

The **man** command displays the online manual page for a command, variable, or error message. You can write man pages for your own Tcl procedures and access them with the **man** command by setting the **sh_user_man_path** variable to an appropriate value. See the man page for the **sh_user_man_path** variable for details.

EXAMPLES

The following command displays the man page for the **echo** command:

```
prompt> man echo
```

The following command displays the man page for the error message CMD-025:

```
prompt> man CMD-025
```

SEE ALSO

help(2)
sh_user_man_path(3)

NAME

mathfunc - Mathematical functions for Tcl expressions

SYNOPSIS

```
package require Tcl 8.5
```

```
::tcl::mathfunc::abs arg
::tcl::mathfunc::acos arg
::tcl::mathfunc::asin arg
::tcl::mathfunc::atan arg
::tcl::mathfunc::atan2 y x
::tcl::mathfunc::bool arg
::tcl::mathfunc::ceil arg
::tcl::mathfunc::cos arg
::tcl::mathfunc::cosh arg
::tcl::mathfunc::double arg
::tcl::mathfunc::entier arg
::tcl::mathfunc::exp arg
::tcl::mathfunc::floor arg
::tcl::mathfunc::fmod x y
::tcl::mathfunc::hypot x y
::tcl::mathfunc::int arg
::tcl::mathfunc::isqrt arg
::tcl::mathfunc::log arg
::tcl::mathfunc::log10 arg
::tcl::mathfunc::max arg ?arg ...?
::tcl::mathfunc::min arg ?arg ...?
::tcl::mathfunc::pow x y
::tcl::mathfunc::rand
::tcl::mathfunc::round arg
::tcl::mathfunc::sin arg
::tcl::mathfunc::sinh arg
::tcl::mathfunc::sqrt arg
::tcl::mathfunc::srand arg
::tcl::mathfunc::tan arg
::tcl::mathfunc::tanh arg
::tcl::mathfunc::wide arg
```

DESCRIPTION

The **expr** command handles mathematical functions of the form **sin(\$x)** or **atan2(\$y,\$x)** by converting them to calls of the form **[tcl::mathfunc::sin [expr {\$x}]]** or **[tcl::mathfunc::atan2 [expr {\$y}] [expr {\$x}]]**. A number of math functions are available by default within the namespace **::tcl::mathfunc**; these functions are also available for code apart from **expr**, by invoking the given commands directly.

Tcl supports the following mathematical functions in expressions, all of which work solely with floating-point numbers unless otherwise noted:

abs	acos	asin	atan
atan2	bool	ceil	cos
cosh	double	entier	exp
floor	fmod	hypot	int
isqrt	log	log10	max
min	pow	rand	round
sin	sinh	sqrt	srand
tan	tanh	wide	

In addition to these predefined functions, applications may define additional functions by using **proc** (or any other method, such as **interp alias** or **Tcl_CreateObjCommand**) to define new commands in the **tcl::mathfunc** namespace. In addition, an obsolete interface named **Tcl_CreateMathFunc()** is available to extensions that are written in C. The latter interface is not recommended for new implementations.

DETAILED DEFINITIONS

abs *arg*

Returns the absolute value of *arg*. *Arg* may be either integer or floating-point, and the result is returned in the same form.

acos *arg*

Returns the arc cosine of *arg*, in the range $[0, \pi]$ radians. *Arg* should be in the range $[-1, 1]$.

asin *arg*

Returns the arc sine of *arg*, in the range $[-\pi/2, \pi/2]$ radians. *Arg* should be in the range $[-1, 1]$.

atan *arg*

Returns the arc tangent of *arg*, in the range $[-\pi/2, \pi/2]$ radians.

atan2 *y x*

Returns the arc tangent of *y/x*, in the range $[-\pi, \pi]$ radians. *x* and *y* cannot both be 0. If *x* is greater

coreTools Command Reference Index

than **0**, this is equivalent to

bool *arg*

Accepts any numeric value, or any string acceptable to **string is boolean**, and returns the corresponding boolean value **0** or **1**. Non-zero numbers are true. Other numbers are false. Non-numeric strings produce boolean value in agreement with **string is true** and **string is false**.

ceil *arg*

Returns the smallest integral floating-point value (i.e. with a zero fractional part) not less than *arg*. The argument may be any numeric value.

cos *arg*

Returns the cosine of *arg*, measured in radians.

cosh *arg*

Returns the hyperbolic cosine of *arg*. If the result would cause an overflow, an error is returned.

double *arg*

The argument may be any numeric value. If *arg* is a floating-point value, returns *arg*, otherwise converts *arg* to floating-point and returns the converted value. May return **Inf** or **-Inf** when the argument is a numeric value that exceeds the floating-point range.

entier *arg*

The argument may be any numeric value. The integer part of *arg* is determined and returned. The integer range returned by this function is unlimited, unlike **int** and **wide** which truncate their range to fit in particular storage widths.

exp *arg*

Returns the exponential of *arg*, defined as **e**arg**. If the result would cause an overflow, an error is returned.

floor *arg*

Returns the largest integral floating-point value (i.e. with a zero fractional part) not greater than *arg*. The argument may be any numeric value.

fmod *x y*

Returns the floating-point remainder of the division of *x* by *y*. If *y* is **0**, an error is returned.

hypot *x y*

Computes the length of the hypotenuse of a right-angled triangle, approximately except for being more numerically stable when the two arguments have substantially different magnitudes.

int *arg*

The argument may be any numeric value. The integer part of *arg* is determined, and then the low order bits of that integer value up to the machine word size are returned as an integer value. For reference, the number of bytes in the machine word are stored in the

coreTools Command Reference Index

wordSize element of the **tcl_platform** array.

isqrt *arg*

Computes the integer part of the square root of *arg*. *Arg* must be a positive value, either an integer or a floating point number. Unlike **sqrt**, which is limited to the precision of a floating point number, **isqrt** will return a result of arbitrary precision.

log *arg*

Returns the natural logarithm of *arg*. *Arg* must be a positive value.

log10 *arg*

Returns the base 10 logarithm of *arg*. *Arg* must be a positive value.

max *arg ...*

Accepts one or more numeric arguments. Returns the one argument with the greatest value.

min *arg ...*

Accepts one or more numeric arguments. Returns the one argument with the least value.

pow *x y*

Computes the value of *x* raised to the power *y*. If *x* is negative, *y* must be an integer value.

rand

Returns a pseudo-random floating-point value in the range (0,1). The generator algorithm is a simple linear congruential generator that is not cryptographically secure. Each result from **rand** completely determines all future results from subsequent calls to **rand**, so **rand** should not be used to generate a sequence of secrets, such as one-time passwords. The seed of the generator is initialized from the internal clock of the machine or may be set with the **srand** function.

round *arg*

If *arg* is an integer value, returns *arg*, otherwise converts *arg* to integer by rounding and returns the converted value.

sin *arg*

Returns the sine of *arg*, measured in radians.

sinh *arg*

Returns the hyperbolic sine of *arg*. If the result would cause an overflow, an error is returned.

sqrt *arg*

The argument may be any non-negative numeric value. Returns a floating-point value that is the square root of *arg*. May return **Inf** when the argument is a numeric value that exceeds the square of the maximum value of the floating-point range.

srand *arg*

The *arg*, which must be an integer, is used to reset the

coreTools Command Reference Index

seed for the random number generator of **rand**. Returns the first random number (see **rand**) from that seed. Each interpreter has its own seed.

tan *arg*

Returns the tangent of *arg*, measured in radians.

tanh *arg*

Returns the hyperbolic tangent of *arg*.

wide *arg*

The argument may be any numeric value. The integer part of *arg* is determined, and then the low order 64 bits of that integer value are returned as an integer value.

SEE ALSO

`expr(n)`, `mathop(n)`, `namespace(n)`

COPYRIGHT

Copyright (C) 1993 The Regents of the University of California.
Copyright (C) 1994-2000 Sun Microsystems Incorporated.
Copyright (C) 2005, 2006 Kevin B. Kenny <kennykb@acm.org>.

NAME

mathop - Mathematical operators as Tcl commands

SYNOPSIS

```
package require Tcl 8.5
```

```

::tcl::mathop::! number
::tcl::mathop::~ number
::tcl::mathop::+ ?number ...?
::tcl::mathop::- number ?number ...?
::tcl::mathop::* ?number ...?
::tcl::mathop::/ number ?number ...?
::tcl::mathop::% number number
::tcl::mathop:** ?number ...?
::tcl::mathop::& ?number ...?
::tcl::mathop::| ?number ...?
::tcl::mathop::^ ?number ...?
::tcl::mathop::<< number number
::tcl::mathop::>> number number
::tcl::mathop::== ?arg ...?
::tcl::mathop::!= arg arg
::tcl::mathop::< ?arg ...?
::tcl::mathop::<= ?arg ...?
::tcl::mathop::>= ?arg ...?
::tcl::mathop::> ?arg ...?
::tcl::mathop::eq ?arg ...?
::tcl::mathop::ne arg arg
::tcl::mathop::in arg list
::tcl::mathop::ni arg list

```

DESCRIPTION

The commands in the `::tcl::mathop` namespace implement the same set of operations as supported by the `expr` command. All are exported from the namespace, but are

coreTools Command Reference Index

not imported into any other namespace by default. Note that renaming, reimplementing or deleting any of the commands in the namespace does *not* alter the way that the **expr** command behaves, and nor does defining any new commands in the **::tcl::mathop** namespace.

The following operator commands are supported:

~	!	+	-	*
/	%	**	&	
^	>>	<<	==	eq
!=	ne	<	<=	>
>=	in	ni		

MATHEMATICAL OPERATORS

The behaviors of the mathematical operator commands are as follows:

! *boolean*

Returns the boolean negation of *boolean*, where *boolean* may be any numeric value or any other form of boolean value (i.e. it returns truth if the argument is falsity or zero, and falsity if the argument is truth or non-zero).

+ ?*number* ...?

Returns the sum of arbitrarily many arguments. Each *number* argument may be any numeric value. If no arguments are given, the result will be zero (the summation identity).

- *number* ?*number* ...?

If only a single *number* argument is given, returns the negation of that numeric value. Otherwise returns the number that results when all subsequent numeric values are subtracted from the first one. All *number* arguments must be numeric values. At least one argument must be given.

*** ?*number* ...?**

Returns the product of arbitrarily many arguments. Each *number* may be any numeric value. If no arguments are given, the result will be one (the multiplicative identity).

/ *number* ?*number* ...?

If only a single *number* argument is given, returns the reciprocal of that numeric value (i.e. the value obtained by dividing 1.0 by that value). Otherwise returns the number that results when the first numeric argument is divided by all subsequent numeric arguments. All *number* arguments must be numeric values. At least one argument must be given.

Note that when the leading values in the list of arguments are integers, integer division will be used for those initial steps (i.e. the intermediate results will be as if the functions *floor* and *int* are applied to them, in that order). If all values in the operation are integers, the result will be an integer.

% *number number*

coreTools Command Reference Index

Returns the integral modulus (i.e., remainder) of the first argument with respect to the second. Each *number* must have an integral value. Also, the sign of the result will be the same as the sign of the second *number*, which must not be zero.

Note that Tcl defines this operation exactly even for negative numbers, so that the following command returns a true value (omitting the namespace for clarity):

```
== [* [/ x y] y] [- x [% x y]]
```

**** ?number ...?**

Returns the result of raising each value to the power of the result of recursively operating on the result of processing the following arguments, so is the same as `pow`. Each *number* may be any numeric value, though the second number must not be fractional if the first is negative. The maximum exponent value that Tcl can handle if the first number is an integer > 1 is 268435455. If no arguments are given, the result will be one, and if only one argument is given, the result will be that argument. The result will have an integral value only when all arguments are integral values.

COMPARISON OPERATORS

The behaviors of the comparison operator commands (most of which operate preferentially on numeric arguments) are as follows:

== ?arg ...?

Returns whether each argument is equal to the arguments on each side of it in the sense of the **expr** `==` operator (i.e., numeric comparison if possible, exact string comparison otherwise). If fewer than two arguments are given, this operation always returns a true value.

eq ?arg ...?

Returns whether each argument is equal to the arguments on each side of it using exact string comparison. If fewer than two arguments are given, this operation always returns a true value.

!= arg arg

Returns whether the two arguments are not equal to each other, in the sense of the **expr** `!=` operator (i.e., numeric comparison if possible, exact string comparison otherwise).

ne arg arg

Returns whether the two arguments are not equal to each other using exact string comparison.

< ?arg ...?

Returns whether the arbitrarily-many arguments are ordered, with each argument after the first having to be strictly more than the one preceding it. Comparisons are performed preferentially on the numeric values, and are otherwise performed using UNICODE string comparison. If fewer than two arguments are present, this operation always returns a true value. When the arguments are numeric but should be compared

coreTools Command Reference Index

as strings, the **string compare** command should be used instead.

<= ?arg ...?

Returns whether the arbitrarily-many arguments are ordered, with each argument after the first having to be equal to or more than the one preceding it. Comparisons are performed preferentially on the numeric values, and are otherwise performed using UNICODE string comparison. If fewer than two arguments are present, this operation always returns a true value. When the arguments are numeric but should be compared as strings, the **string compare** command should be used instead.

> ?arg ...?

Returns whether the arbitrarily-many arguments are ordered, with each argument after the first having to be strictly less than the one preceding it. Comparisons are performed preferentially on the numeric values, and are otherwise performed using UNICODE string comparison. If fewer than two arguments are present, this operation always returns a true value. When the arguments are numeric but should be compared as strings, the **string compare** command should be used instead.

>= ?arg ...?

Returns whether the arbitrarily-many arguments are ordered, with each argument after the first having to be equal to or less than the one preceding it. Comparisons are performed preferentially on the numeric values, and are otherwise performed using UNICODE string comparison. If fewer than two arguments are present, this operation always returns a true value. When the arguments are numeric but should be compared as strings, the **string compare** command should be used instead.

BIT-WISE OPERATORS

The behaviors of the bit-wise operator commands (all of which only operate on integral arguments) are as follows:

~ *number*

Returns the bit-wise negation of *number*. *Number* may be an integer of any size. Note that the result of this operation will always have the opposite sign to the input *number*.

& ?*number* ...?

Returns the bit-wise AND of each of the arbitrarily many arguments. Each *number* must have an integral value. If no arguments are given, the result will be minus one.

| ?*number* ...?

Returns the bit-wise OR of each of the arbitrarily many arguments. Each *number* must have an integral value. If no arguments are given, the result will be zero.

^ ?*number* ...?

coreTools Command Reference Index

Returns the bit-wise XOR of each of the arbitrarily many arguments. Each *number* must have an integral value. If no arguments are given, the result will be zero.

<< *number number*

Returns the result of bit-wise shifting the first argument left by the number of bits specified in the second argument. Each *number* must have an integral value.

>> *number number*

Returns the result of bit-wise shifting the first argument right by the number of bits specified in the second argument. Each *number* must have an integral value.

LIST OPERATORS

The behaviors of the list-oriented operator commands are as follows:

in *arg list*

Returns whether the value *arg* is present in the list *list* (according to exact string comparison of elements).

ni *arg list*

Returns whether the value *arg* is not present in the list *list* (according to exact string comparison of elements).

EXAMPLES

The simplest way to use the operators is often by using **namespace path** to make the commands available. This has the advantage of not affecting the set of commands defined by the current namespace.

```
namespace path {::tcl::mathop ::tcl::mathfunc}
```

```
# Compute the sum of some numbers
```

```
set sum [+ 1 2 3]
```

```
# Compute the average of a list
```

```
set list {1 2 3 4 5 6}
```

```
set mean [/ [+ {*} $list] [double [llength $list]]]
```

```
# Test for list membership
```

```
set gotIt [in 3 $list]
```

```
# Test to see if a value is within some defined range
```

```
set inRange [<= 1 $x 5]
```

```
# Test to see if a list is sorted
```

```
set sorted [<= {*} $list]
```

SEE ALSO

`expr(n)`, `mathfunc(n)`, `namespace(n)`

KEYWORDS

`command`, `expression`, `operator`

maximum_bit_blast_size

Largest size bus that can be bit blasted.

Syntax

```
string maximum_bit_blast_size = "128"
```

Description

maximum_bit_blast_size is a global variable that specifies the largest size bus that the coreTool will blast so that you can specify bit-level intent.

The coreTools automatically blast a bus when you set a port attribute on an individual bit of the bus, either through the GUI "Bit Level Intent" button or by executing "set_port_attribute {Bus[bit]}". However, the coreTools will not blast a bus that is larger than the current value of the maximum_bit_blast_size variable. If you attempt to set bit-level intent on a bus that is larger than maximum_bit_blast_size, the coreTool returns an error.

If you want to set bit-level intent on a bus that is larger than the current value of maximum_bit_blast_size, then set maximum_bit_blast_size to a larger value.

Examples

If you want to blast buses that are up to 256 bits wide so that you can specify bit-level intent, add the following line to your .synopsys_rt.setup file:

```
set maximum_bit_blast_size 256
```

See Also

[set_port_attribute \(2\)](#)

memMap

Describes a memory map in this component.

Description

A memory map is a container used to hold different address regions. The address regions are described by adding address blocks and address banks to the memory map.

The memory map is then associated to an interface instance with the [MemoryMap](#) attribute to describe the memory exposed by that interface.

See Also

[addressBlock \(3\)](#), [addressBank \(3\)](#), [create_memory_map \(2\)](#),
[remove_memory_map \(2\)](#), [MemoryMap \(3\)](#)

Supported Attributes

[BitsInLAU \(3\)](#), [ComponentOfItem \(3\)](#), [Description \(3\)](#), [DocGroup \(3\)](#),
[DocInclude \(3\)](#), [DocMaster \(3\)](#), [DocMemoryAccess \(3\)](#), [DocVisible \(3\)](#),
[DocWaiver \(3\)](#), [GroupImage \(3\)](#), [GroupImageAttrs \(3\)](#), [GroupImageTitle \(3\)](#),
[GroupText \(3\)](#), [Label \(3\)](#), [MemoryAccess \(3\)](#), [MemoryAccessDescription \(3\)](#),
[Name \(3\)](#), [Sequence \(3\)](#), [TypeName \(3\)](#), [UndefinedBitValue \(3\)](#), [Visible \(3\)](#)

NAME

`memory` - Control Tcl memory debugging capabilities

SYNOPSIS

`memory option ?arg arg ...?`

DESCRIPTION

The **memory** command gives the Tcl developer control of Tcl's memory debugging capabilities. The **memory** command has several suboptions, which are described below. It is only available when Tcl has been compiled with memory debugging enabled (when **TCL_MEM_DEBUG** is defined at compile time), and after **Tcl_InitMemory** has been called.

memory active file

Write a list of all currently allocated memory to the specified *file*.

memory break_on_malloc count

After the *count* allocations have been performed, **ckalloc** outputs a message to this effect and that it is now attempting to enter the C debugger. Tcl will then issue a **SIGINT** signal against itself. If you are running Tcl under a C debugger, it should then enter the debugger command mode.

memory info

Returns a report containing the total allocations and frees since Tcl began, the current packets allocated (the current number of calls to **ckalloc** not met by a corresponding call to **ckfree**), the current bytes allocated, and the maximum number of packets and bytes allocated.

coreTools Command Reference Index

memory init [on|off]

Turn on or off the pre-initialization of all allocated memory with bogus bytes. Useful for detecting the use of uninitialized values.

memory objs *file*

Causes a list of all allocated Tcl_Obj values to be written to the specified *file* immediately, together with where they were allocated. Useful for checking for leaks of values.

memory onexit *file*

Causes a list of all allocated memory to be written to the specified *file* during the finalization of Tcl's memory subsystem. Useful for checking that memory is properly cleaned up during process exit.

memory tag *string*

Each packet of memory allocated by **ckalloc** can have associated with it a string-valued tag. In the lists of allocated memory generated by **memory active** and **memory onexit**, the tag for each packet is printed along with other information about the packet. The **memory tag** command sets the tag value for subsequent calls to **ckalloc** to be *string*.

memory trace [on|off]

Turns memory tracing on or off. When memory tracing is on, every call to **ckalloc** causes a line of trace information to be written to *stderr*, consisting of the word *ckalloc*, followed by the address returned, the amount of memory allocated, and the C filename and line number of the code performing the allocation. For example:

```
ckalloc 40e478 98 tclProc.c 1406
```

Calls to **ckfree** are traced in the same manner.

memory trace_on_at_malloc *count*

Enable memory tracing after *count* **ckallocs** have been performed. For example, if you enter **memory trace_on_at_malloc 100**, after the 100th call to **ckalloc**, memory trace information will begin being displayed for all allocations and frees. Since there can be a lot of memory activity before a problem occurs, judicious use of this option can reduce the slowdown caused by tracing (and the amount of trace information produced), if you can identify a number of allocations that occur before the problem sets in. The current number of memory allocations that have occurred since Tcl started is printed on a guard zone failure.

memory validate [on|off]

Turns memory validation on or off. When memory validation is enabled, on every call to **ckalloc** or **ckfree**, the guard zones are checked for every piece of memory currently in existence that was allocated by **ckalloc**. This has a large performance impact and should only be used when overwrite problems are strongly suspected. The advantage of enabling memory validation is that a guard zone overwrite can be

detected on the first call to **ckalloc** or **ckfree** after the overwrite occurred, rather than when the specific memory with the overwritten guard zone(s) is freed, which may occur long after the overwrite occurred.

SEE ALSO

`ckalloc`, `ckfree`, `Tcl_ValidateAllMemory`,
`Tcl_DumpActiveMemory`, `TCL_MEM_DEBUG`

KEYWORDS

memory, debug

merge_ports

Merge ports in current hierarchy, which are exporting adjacent bits of same pin.

Syntax

```
string merge_ports [-exclude <port or pin names>] [-ignore_attrs]
                        string <port or pin names>
```

Parameters

-exclude <port or pin names>	Name of pins or ports to be ignored in merging.
-ignore_attrs	Don't compare interface attribute values when determining ports to merge.

Description

This command merges redundant ports under the following conditions:

- Ports connected to adjacent bits of the same pin
- Output pin driving multiple output ports
- Input ports driven by the same pin in the containing hierarchy

These situations can occur explicitly due to calls to create_connection and export_pin, or may occur implicitly due to use of create_connection with the -hierarchy option.

The -exclude option can be used to specify ports or pins that should not be considered for merging. The same behavior can be obtained by setting the MergeCandidate attribute to false on ports/pins to be excluded.

The merge_ports command merges ports in the current hierarchy only. Therefore to ensure maximal merging occurs, the command should be invoked in each hierarchical component using a bottom up traversal.

The command returns the count of the number of ports merged.

Examples

To merge ports in current hierarchy, while ignoring ports ex_in_0 and ex_in_1

```
coreAssembler> merge_ports -exclude {ex_in1_0 ex_in1_1}
```

See Also

[create_connection \(2\)](#), [export_pin \(2\)](#), [MergeCandidate \(3\)](#)

`move_components`

Move one or more components into a different hierarchical level.

Syntax

```
string move_components components hierName
      list components
      string hierName
```

Parameters

<code>components</code>	List of component names for the components to move.
<code>hierName</code>	The name of the hierarchical cell to move components to.

Description

Move the components into a different hierarchy

Examples

Move the components `'/i_apb /i_gpio'` into hierarchy `'/H1'`

```
move_components "/i_apb /i_gpio" /H1
```

Move the component `'L2/i_ahb'` to current hierarchy

```
move_components L2/i_ahb .
```

Move the components `'i_apb'` and `'i_gpio'` to hierarchy one level above current context

```
"i_apb i_gpio" ".."
```

See Also

NAME

mpexpr - Evaluate an expression with multiple precision
math

SYNOPSIS

```
package require Mpexpr
mpexpr arg ?arg arg ...?
mpformat formatString ?arg arg ...?
global mp_precision
```

DESCRIPTION

Mpexpr is based on Tcl's native *expr* command, and shares many similarities with *expr*. Mpexpr performs all of its calculations using an arbitrary precision math package.

Mpexpr concatenates *arg*'s (adding separator spaces between them), evaluates the result as a Tcl expression, and returns the value. The operators permitted in Tcl expressions are a subset of the operators permitted in C expressions, and they have the same meaning and precedence as the corresponding C operators. Expressions almost always yield numeric results (integer or floating-point values). For example, the expression

```
mpexpr 8.2 + 6
```

evaluates to 14.2. Tcl expressions differ from C expressions in the way that operands are specified. Also, Tcl expressions support non-numeric operands and string comparisons.

OPERANDS

A Tcl expression consists of a combination of operands, operators, and parentheses. White space may be used between the operands and operators and parentheses; it is ignored by the expression processor. Where possible, operands are interpreted as integer values.

coreTools Command Reference Index

Integer values may be specified in decimal (the normal case), in octal (if the first character of the operand is **0**), or in hexadecimal (if the first two characters of the operand are **0x**). If an operand does not have one of the integer formats given above, then it is treated as a floating-point number if that is possible. Floating-point numbers may be specified in any of the ways accepted by an ANSI-compliant C compiler (except that the **'f'**, **'F'**, **'l'**, and **'L'** suffixes will not be permitted in most installations). For example, all of the following are valid floating-point numbers: 2.1, 3., 6e4, 7.91e+16. If no numeric interpretation is possible, then an operand is left as a string (and only a limited set of operators may be applied to it).

Operands may be specified in any of the following ways:

- [1] As an numeric value, either integer or floating-point.
- [2] As a Tcl variable, using standard **\$** notation. The variable's value will be used as the operand.
- [3] As a string enclosed in double-quotes. The expression parser will perform backslash, variable, and command substitutions on the information between the quotes, and use the resulting value as the operand
- [4] As a string enclosed in braces. The characters between the open brace and matching close brace will be used as the operand without any substitutions.
- [5] As a Tcl command enclosed in brackets. The command will be executed and its result will be used as the operand.
- [6] As a mathematical function whose arguments have any of the above forms for operands, such as **'sin(\$x)'**. See below for a list of defined functions.

Where substitutions occur above (e.g. inside quoted strings), they are performed by the expression processor. However, an additional layer of substitution may already have been performed by the command parser before the expression processor was called. As discussed below, it is usually best to enclose expressions in braces to prevent the command parser from performing substitutions on the contents.

For some examples of simple expressions, suppose the variable **a** has the value 3 and the variable **b** has the value 6. Then the command on the left side of each of the lines below will produce the value on the following line:

mpexpr 3.1 + \$a

6.1

`mpexpr 2 + "$a.$b"`

5.6

`mpexpr 4*[length "6 2"]`

8

`mpexpr {{word one} < "word $a"}`

0

OPERATORS

The valid operators are listed below, grouped in decreasing order of precedence:

<code>- + ~ !</code>	Unary minus, unary plus, bit-wise NOT, logical NOT. None of these operands may be applied to string operands, and bit-wise NOT may be applied only to integers.
<code>* / %</code>	Multiply, divide, remainder. None of these operands may be applied to string operands, and remainder may be applied only to integers. The remainder will always have the same sign as the divisor and an absolute value smaller than the divisor.
<code>+ -</code>	Add and subtract. Valid for any numeric operands.
<code><< >></code>	Left and right shift. Valid for integer operands only. Integers in <code>mpexpr</code> are not limited to a machine word and do not use two's complement format. Therefore shifting will not include a sign bit.
<code>< > <= >=</code>	Boolean less, greater, less than or equal, and greater than or equal. Each operator produces 1 if the condition is true, 0 otherwise. These operators may be applied to strings as well as numeric operands, in which case string comparison is used.
<code>== !=</code>	Boolean equal and not equal. Each operator produces a zero/one result. Valid for all operand types.
<code>&</code>	Bit-wise AND. Valid for integer operands only.

coreTools Command Reference Index

^	Bit-wise exclusive OR. Valid for integer operands only.
 	Bit-wise OR. Valid for integer operands only.
&&	Logical AND. Produces a 1 result if both operands are non-zero, 0 otherwise. Valid for numeric operands only (integers or floating-point).
 	Logical OR. Produces a 0 result if both operands are zero, 1 otherwise. Valid for numeric operands only (integers or floating-point).
x?y:z	If-then-else, as in C. If x evaluates to non-zero, then the result is the value of y . Otherwise the result is the value of z . The x operand must have a numeric value.

See the C manual for more details on the results produced by each operator. All of the binary operators group left-to-right within the same precedence level. For example, the command

```
mpexpr 4*2 < 7
```

returns 0.

The **&&**, **||**, and **?:** operators have “lazy evaluation”, just as in C, which means that operands are not evaluated if they are not needed to determine the outcome. For example, in the command

```
mpexpr ${v} ? [a] : [b]}
```

only one of **[a]** or **[b]** will actually be evaluated, depending on the value of **\$v**. Note, however, that this is only true if the entire expression is enclosed in braces; otherwise the Tcl parser will evaluate both **[a]** and **[b]** before invoking the **expr** command.

MATH FUNCTIONS

Mpexpr supports the following mathematical functions in expressions. **x** and **y** are integer or floating point values; **i**, **j** and **c** are integer values;

Math functions compatible with *expr*:

acos(x) Arc cosine of **x**.

asin(x) Arc sine of **x**.

coreTools Command Reference Index

atan(x)	Arc tangent of <i>x</i> .
atan2(x,y)	Arc tangent of <i>x</i> / <i>y</i> .
ceil(x)	Least integral value greater than or equal to <i>x</i> .
cos(x)	Cosine of <i>x</i> .
cosh(x)	Hyperbolic cosine of <i>x</i> .
exp(x)	Exponential function e^{**x} .
floor(x)	Greatest integral value less than or equal to <i>x</i> .
fmod(x,y)	Remainder of <i>x</i> divided by <i>y</i> .
hypot(x,y)	Euclidean distance of $\sqrt{x^2 + y^2}$.
log(x)	Natural logarithm of <i>x</i> .
log10(x)	Base-10 logarithm of <i>x</i> .
pow(x,y)	<i>x</i> raised to the <i>y</i> power.
sin(x)	Sine of <i>x</i> .
sinh(x)	Hyperbolic sine of <i>x</i> .
sqrt(x)	Square root of <i>x</i> .
tan(x)	Tangent of <i>x</i> .
tanh(x)	Hyperbolic tangent of <i>x</i> .
abs(x)	Returns the absolute value of <i>x</i> . <i>x</i> may be either integer or floating-point, and the result is returned in the same form.
double(x)	If <i>x</i> is a floating value, returns <i>x</i> , otherwise converts <i>x</i> to floating and returns the converted value.
int(x)	If <i>x</i> is an integer value, returns <i>x</i> , otherwise converts <i>x</i> to integer by truncation and returns the converted value.
round(x)	If <i>x</i> is an integer value, returns <i>x</i> , otherwise converts <i>x</i> to integer by rounding and returns the converted value.
Additional mpexpr functions:	
root(x,y)	The <i>y</i> th root of <i>x</i> .
frem(x,y)	Remove all occurrence of factory from number <i>x</i> .

coreTools Command Reference Index

<i>minv(x,y)</i>	<i>Inverse of x modulo y.</i>
<i>gcd(x,y)</i>	<i>Greatest common divisor of x and y.</i>
<i>lcm(x,y)</i>	<i>Least common multiple of x and y.</i>
<i>max(x,y)</i>	<i>Maximum of x and y.</i>
<i>min(x,y)</i>	<i>Minimum of x and y.</i>
<i>pi()</i>	<i>Value of pi.</i>
<i>fib(i)</i>	<i>Fibonacci number of integer i.</i>
<i>fact(i)</i>	<i>Factorial of integer i.</i>
<i>pfact(i)</i>	<i>Product of prime numbers up to integer i.</i>
<i>lfactor(i,c)</i>	<i>Lowest prime factor of integer i, trying count c times.</i>
<i>iroot(i,j)</i>	<i>Integer root j of integer i.</i>
<i>gcdrem(i,j)</i>	<i>Relatively prime of greatest common divisor of i divided by j.</i>
<i>perm(i,j)</i>	<i>Permutations of i taking j at a time: $i! / (i - j)!$.</i>
<i>comb(i,j)</i>	<i>Combinations of i taking j at a time: $i! / (j! * (i - j)!) .$</i>
<i>prime(i,c)</i>	<i>Return 0 if i is not prime, return 1 if i probably is prime. Test for primality count c times. The chance of a non-prime passing this test is less than $(1/4)^{\text{count}}$. For example, a count of 100 fails for only 1 in 10^{60} numbers.</i>
<i>relprime(i,j)</i>	<i>Return 1 if i and j are relatively prime to each other, 0 otherwise.</i>

TYPES, OVERFLOW, AND PRECISION

Computations are performed using arbitrary fixed and floating point values. Native machine values (*int*, *long*, *IEEE 754 floating point*, etc.) and instructions are not used. Conversion among internal representations for integer, floating-point, and string operands is done automatically as needed. For arithmetic computations, integers are used until some floating-point number is introduced, after which floating-point is used. For example,

mpexpr 5 / 4

returns 1, while

```
mpexpr 5 / 4.0
mpexpr 5 / ( [string length "abcd"] + 0.0 )
```

both return 1.25. Floating-point values are always returned with a `''.'` or an `''e''` so that they will not look like integer values. For example,

```
mpexpr 20.0/5.0
```

returns `''4.0''`, not `''4''`.

The global variable **mp_precision** determines the number of significant digits that are retained during evaluation. If **mp_precision** is unset then 17 digits of precision are used. The maximum value of **mp_precision** is 10000. Note that larger values for **mp_precision** will require increasingly longer execution times. Setting **mp_precision** to an illegal value will generate an error.

STRING OPERATIONS

String values may be used as operands of the comparison operators, although the expression evaluator tries to do comparisons as integer or floating-point when it can. If one of the operands of a comparison is a string and the other has a numeric value, the numeric operand is converted back to a string using the C *sprintf* format specifier `%d` for integers and `%g` for floating-point values. For example, the commands

```
mpexpr {"0x03" > "2"} mpexpr {"0y" < "0x12"}
```

both return 1. The first comparison is done using integer comparison, and the second is done using string comparison after the second operand is converted to the string `''18''`. Because of Tcl's tendency to treat values as numbers whenever possible, it isn't generally a good idea to use operators like `==` when you really want string comparison and the values of the operands could be arbitrary; it's better in these cases to use the **string compare** command instead.

mpformat formats a string in the style of Tcl's native *format* command. **mpformat** will interpret numeric arguments as arbitrary precision numbers. **mpformat** performs limited % substitution on the output string. The following may be specified:

```
% [-] [width[.precision]] formatChar
```

-

Specifies left justification; right justification is the default.

coreTools Command Reference Index

width.precision

Specifies optional width and precision. Default precision is 8. Width and/or precision may be specified as *, in which the next argument will be used for the width or precision value.

Format character and result

d

Format next argument as integer, truncating after the decimal point.

f

Format next argument in decimal floating point.

e

Format next argument in scientific notation.

r, R

Format next argument as rational fraction x / y .

N

Format next argument as numerator only of rational fraction x / y .

D

Format next argument as denominator only of rational fraction x / y .

o

Format next argument in octal format, with leading '0'; floating point argument formatted as octal rational fraction x / y .

x

Format next argument in hexadecimal format, with leading '0x'; floating point formatted argument as hexadecimal rational fraction x / y .

b

Format next argument in binary format, with leading '0b'; floating point argument formatted as binary rational fraction x / y .

s

Format next argument as string.

c

Format next argument as single character value.

%

Format single literal %.

Other characters in format string

\n

Format ASCII newline.

\r

Format ASCII carriage return.

\t

coreTools Command Reference Index

Format ASCII tab.

\f

Format ASCII form feed.

\v

Format ASCII vertical tab.

\b

Format ASCII backspace.

NOTES

Mpexpr is based on Tcl 7.6 'tclExpr.c' and David Bell's 'Calc' program. This man page is largely borrowed from Tcl 7.6 as well, as is the mpexpr test suite.

See the files README and INSTALL for additional information.

Tcl 7.6 is Copyright (c) 1987-1994 The Regents of the University of California and Copyright (c) 1994 Sun Microsystems, Inc.

Calc is Copyright (c) 1994 David I. Bell.

AUTHOR

Tom Poindexter, tpoindex@nyx.net, Talus Technologies, Inc., Highlands Ranch, CO.
<http://www.nyx.net/~tpoindex>

Version 1.0 released November, 1998.

Copyright 1998 Tom Poindexter. See the file 'LICENSE.TERMS' for additional copyright and licensing terms.

NAME

mpexpr - Evaluate an expression with multiple precision
math

SYNOPSIS

```
package require Mpexpr  
mpexpr arg ?arg arg ...?  
mpformat formatString ?arg arg ...?  
global mp_precision
```

DESCRIPTION

Mpexpr is based on Tcl's native *expr* command, and shares many similarities with *expr*. Mpexpr performs all of its calculations using an arbitrary precision math package.

Mpexpr concatenates *arg*'s (adding separator spaces between them), evaluates the result as a Tcl expression, and returns the value. The operators permitted in Tcl expressions are a subset of the operators permitted in C expressions, and they have the same meaning and precedence as the corresponding C operators. Expressions almost always yield numeric results (integer or floating-point values). For example, the expression

```
mpexpr 8.2 + 6
```

evaluates to 14.2. Tcl expressions differ from C expressions in the way that operands are specified. Also, Tcl expressions support non-numeric operands and string comparisons.

OPERANDS

A Tcl expression consists of a combination of operands, operators, and parentheses. White space may be used between the operands and operators and parentheses; it is ignored by the expression processor. Where possible, operands are interpreted as integer values.

coreTools Command Reference Index

Integer values may be specified in decimal (the normal case), in octal (if the first character of the operand is **0**), or in hexadecimal (if the first two characters of the operand are **0x**). If an operand does not have one of the integer formats given above, then it is treated as a floating-point number if that is possible. Floating-point numbers may be specified in any of the ways accepted by an ANSI-compliant C compiler (except that the **'f'**, **'F'**, **'l'**, and **'L'** suffixes will not be permitted in most installations). For example, all of the following are valid floating-point numbers: 2.1, 3., 6e4, 7.91e+16. If no numeric interpretation is possible, then an operand is left as a string (and only a limited set of operators may be applied to it).

Operands may be specified in any of the following ways:

- [1] As an numeric value, either integer or floating-point.
- [2] As a Tcl variable, using standard **\$** notation. The variable's value will be used as the operand.
- [3] As a string enclosed in double-quotes. The expression parser will perform backslash, variable, and command substitutions on the information between the quotes, and use the resulting value as the operand
- [4] As a string enclosed in braces. The characters between the open brace and matching close brace will be used as the operand without any substitutions.
- [5] As a Tcl command enclosed in brackets. The command will be executed and its result will be used as the operand.
- [6] As a mathematical function whose arguments have any of the above forms for operands, such as **'sin(\$x)'**. See below for a list of defined functions.

Where substitutions occur above (e.g. inside quoted strings), they are performed by the expression processor. However, an additional layer of substitution may already have been performed by the command parser before the expression processor was called. As discussed below, it is usually best to enclose expressions in braces to prevent the command parser from performing substitutions on the contents.

For some examples of simple expressions, suppose the variable **a** has the value 3 and the variable **b** has the value 6. Then the command on the left side of each of the lines below will produce the value on the following line:

mpexpr 3.1 + \$a

6.1

mpexpr 2 + "\$a.\$b"

5.6

mpexpr 4*[llength "6 2"]

8

mpexpr {{word one} < "word \$a"}

0

OPERATORS

The valid operators are listed below, grouped in decreasing order of precedence:

- + ~ !	Unary minus, unary plus, bit-wise NOT, logical NOT. None of these operands may be applied to string operands, and bit-wise NOT may be applied only to integers.
* / %	Multiply, divide, remainder. None of these operands may be applied to string operands, and remainder may be applied only to integers. The remainder will always have the same sign as the divisor and an absolute value smaller than the divisor.
+ -	Add and subtract. Valid for any numeric operands.
<< >>	Left and right shift. Valid for integer operands only. Integers in mpexpr are not limited to a machine word and do not use two's complement format. Therefore shifting will not include a sign bit.
< > <= >=	Boolean less, greater, less than or equal, and greater than or equal. Each operator produces 1 if the condition is true, 0 otherwise. These operators may be applied to strings as well as numeric operands, in which case string comparison is used.
== !=	Boolean equal and not equal. Each operator produces a zero/one result. Valid for all operand types.
&	Bit-wise AND. Valid for integer operands only.

coreTools Command Reference Index

^	Bit-wise exclusive OR. Valid for integer operands only.
 	Bit-wise OR. Valid for integer operands only.
&&	Logical AND. Produces a 1 result if both operands are non-zero, 0 otherwise. Valid for numeric operands only (integers or floating-point).
 	Logical OR. Produces a 0 result if both operands are zero, 1 otherwise. Valid for numeric operands only (integers or floating-point).
x?y:z	If-then-else, as in C. If x evaluates to non-zero, then the result is the value of y . Otherwise the result is the value of z . The x operand must have a numeric value.

See the C manual for more details on the results produced by each operator. All of the binary operators group left-to-right within the same precedence level. For example, the command

```
mpexpr 4*2 < 7
```

returns 0.

The **&&**, **||**, and **?:** operators have “lazy evaluation”, just as in C, which means that operands are not evaluated if they are not needed to determine the outcome. For example, in the command

```
mpexpr ${v} ? [a] : [b]}
```

only one of **[a]** or **[b]** will actually be evaluated, depending on the value of **\$v**. Note, however, that this is only true if the entire expression is enclosed in braces; otherwise the Tcl parser will evaluate both **[a]** and **[b]** before invoking the **expr** command.

MATH FUNCTIONS

Mpexpr supports the following mathematical functions in expressions. **x** and **y** are integer or floating point values; **i**, **j** and **c** are integer values;

Math functions compatible with *expr*:

acos(x) Arc cosine of **x**.

asin(x) Arc sine of **x**.

coreTools Command Reference Index

atan(x)	Arc tangent of <i>x</i> .
atan2(x,y)	Arc tangent of <i>x</i> / <i>y</i> .
ceil(x)	Least integral value greater than or equal to <i>x</i> .
cos(x)	Cosine of <i>x</i> .
cosh(x)	Hyperbolic cosine of <i>x</i> .
exp(x)	Exponential function e^{**x} .
floor(x)	Greatest integral value less than or equal to <i>x</i> .
fmod(x,y)	Remainder of <i>x</i> divided by <i>y</i> .
hypot(x,y)	Euclidean distance of $\sqrt{x^2 + y^2}$.
log(x)	Natural logarithm of <i>x</i> .
log10(x)	Base-10 logarithm of <i>x</i> .
pow(x,y)	<i>x</i> raised to the <i>y</i> power.
sin(x)	Sine of <i>x</i> .
sinh(x)	Hyperbolic sine of <i>x</i> .
sqrt(x)	Square root of <i>x</i> .
tan(x)	Tangent of <i>x</i> .
tanh(x)	Hyperbolic tangent of <i>x</i> .
abs(x)	Returns the absolute value of <i>x</i> . <i>x</i> may be either integer or floating-point, and the result is returned in the same form.
double(x)	If <i>x</i> is a floating value, returns <i>x</i> , otherwise converts <i>x</i> to floating and returns the converted value.
int(x)	If <i>x</i> is an integer value, returns <i>x</i> , otherwise converts <i>x</i> to integer by truncation and returns the converted value.
round(x)	If <i>x</i> is an integer value, returns <i>x</i> , otherwise converts <i>x</i> to integer by rounding and returns the converted value.
Additional mpexpr functions:	
root(x,y)	The <i>y</i> th root of <i>x</i> .
frem(x,y)	Remove all occurrence of factory from number <i>x</i> .

coreTools Command Reference Index

<i>minv(x,y)</i>	<i>Inverse of x modulo y.</i>
<i>gcd(x,y)</i>	<i>Greatest common divisor of x and y.</i>
<i>lcm(x,y)</i>	<i>Least common multiple of x and y.</i>
<i>max(x,y)</i>	<i>Maximum of x and y.</i>
<i>min(x,y)</i>	<i>Minimum of x and y.</i>
<i>pi()</i>	<i>Value of pi.</i>
<i>fib(i)</i>	<i>Fibonacci number of integer i.</i>
<i>fact(i)</i>	<i>Factorial of integer i.</i>
<i>pfact(i)</i>	<i>Product of prime numbers up to integer i.</i>
<i>lfactor(i,c)</i>	<i>Lowest prime factor of integer i, trying count c times.</i>
<i>iroot(i,j)</i>	<i>Integer root j of integer i.</i>
<i>gcdrem(i,j)</i>	<i>Relatively prime of greatest common divisor of i divided by j.</i>
<i>perm(i,j)</i>	<i>Permutations of i taking j at a time: $i! / (i - j)!$.</i>
<i>comb(i,j)</i>	<i>Combinations of i taking j at a time: $i! / (j! * (i - j)!) .$</i>
<i>prime(i,c)</i>	<i>Return 0 if i is not prime, return 1 if i probably is prime. Test for primality count c times. The chance of a non-prime passing this test is less than $(1/4)^{\text{count}}$. For example, a count of 100 fails for only 1 in 10^{60} numbers.</i>
<i>relprime(i,j)</i>	<i>Return 1 if i and j are relatively prime to each other, 0 otherwise.</i>

TYPES, OVERFLOW, AND PRECISION

Computations are performed using arbitrary fixed and floating point values. Native machine values (*int*, *long*, *IEEE 754 floating point*, etc.) and instructions are not used. Conversion among internal representations for integer, floating-point, and string operands is done automatically as needed. For arithmetic computations, integers are used until some floating-point number is introduced, after which floating-point is used. For example,

mpexpr 5 / 4

returns 1, while

```
mpexpr 5 / 4.0
mpexpr 5 / ( [string length "abcd"] + 0.0 )
```

both return 1.25. Floating-point values are always returned with a `''.'` or an `''e''` so that they will not look like integer values. For example,

```
mpexpr 20.0/5.0
```

returns `''4.0''`, not `''4''`.

The global variable **mp_precision** determines the number of significant digits that are retained during evaluation. If **mp_precision** is unset then 17 digits of precision are used. The maximum value of **mp_precision** is 10000. Note that larger values for **mp_precision** will require increasingly longer execution times. Setting **mp_precision** to an illegal value will generate an error.

STRING OPERATIONS

String values may be used as operands of the comparison operators, although the expression evaluator tries to do comparisons as integer or floating-point when it can. If one of the operands of a comparison is a string and the other has a numeric value, the numeric operand is converted back to a string using the C *sprintf* format specifier `%d` for integers and `%g` for floating-point values. For example, the commands

```
mpexpr {"0x03" > "2"} mpexpr {"0y" < "0x12"}
```

both return 1. The first comparison is done using integer comparison, and the second is done using string comparison after the second operand is converted to the string `''18''`. Because of Tcl's tendency to treat values as numbers whenever possible, it isn't generally a good idea to use operators like `==` when you really want string comparison and the values of the operands could be arbitrary; it's better in these cases to use the **string compare** command instead.

mpformat formats a string in the style of Tcl's native *format* command. **mpformat** will interpret numeric arguments as arbitrary precision numbers. **mpformat** performs limited % substitution on the output string. The following may be specified:

```
% [-] [width[.precision]] formatChar
```

-

Specifies left justification; right justification is the default.

coreTools Command Reference Index

width.precision

Specifies optional width and precision. Default precision is 8. Width and/or precision may be specified as *, in which the next argument will be used for the width or precision value.

Format character and result

d

Format next argument as integer, truncating after the decimal point.

f

Format next argument in decimal floating point.

e

Format next argument in scientific notation.

r, R

Format next argument as rational fraction x / y .

N

Format next argument as numerator only of rational fraction x / y .

D

Format next argument as denominator only of rational fraction x / y .

o

Format next argument in octal format, with leading '0'; floating point argument formatted as octal rational fraction x / y .

x

Format next argument in hexadecimal format, with leading '0x'; floating point formatted argument as hexadecimal rational fraction x / y .

b

Format next argument in binary format, with leading '0b'; floating point argument formatted as binary rational fraction x / y .

s

Format next argument as string.

c

Format next argument as single character value.

%

Format single literal %.

Other characters in format string

\n

Format ASCII newline.

\r

Format ASCII carriage return.

\t

coreTools Command Reference Index

Format ASCII tab.

\f

Format ASCII form feed.

\v

Format ASCII vertical tab.

\b

Format ASCII backspace.

NOTES

Mpexpr is based on Tcl 7.6 'tclExpr.c' and David Bell's 'Calc' program. This man page is largely borrowed from Tcl 7.6 as well, as is the mpexpr test suite.

See the files README and INSTALL for additional information.

Tcl 7.6 is Copyright (c) 1987-1994 The Regents of the University of California and Copyright (c) 1994 Sun Microsystems, Inc.

Calc is Copyright (c) 1994 David I. Bell.

AUTHOR

Tom Poindexter, tpoindex@nyx.net, Talus Technologies, Inc., Highlands Ranch, CO.
<http://www.nyx.net/~tpoindex>

Version 1.0 released November, 1998.

Copyright 1998 Tom Poindexter. See the file 'LICENSE.TERMS' for additional copyright and licensing terms.

msg_box

Displays a modal message box

Syntax

```
string msg_box [-msgType <messageType>] [-icon <iconType>] [-dfltBtn
<buttonNumber>] [-title <title_str>] [-auto_close_time <seconds>]
```

<message>

string <messageType>

string <iconType>

int <buttonNumber>

string <title_str>

int <seconds>

string <message>

Parameters

-msgType

<messageType>

Specifies the buttons available.

Legal values: ok, okcancel, yesno, yesnocancel, retrycancel, abortretryignore
Default value: ok

-icon <iconType>

Icon to be displayed.

Legal values: exclamation, info, question, stop, none
Default value: none

-dfltBtn

<buttonNumber>

Button number for the default button. (Range: 1 to 3)

-title <title_str>

String to be displayed in the title bar.

-auto_close_time

<seconds>

Automatically close the MessageBox window after the specified time.

<message>

Text string to be displayed.

Description

This command is used to create simple dialog boxes to be presented to a user. They will contain an icon, a simple message string, and one or more buttons which the user will select to cause processing to continue down particular paths.

Examples

To display a message box with only an OK button and exclamation mark for the icon:

```
coreAssembler> msg_box -msgType ok -icon exclamation  
-title "Test Dialog" "This is a test MessageBox dialog."  
Display a message when an error occurs prior to stopping execution.
```

```
if {[catch {some-command} msg]} {  
  msg_box -icon exclamation -title "An error occurred" $msg  
  error $msg  
}
```

See Also

NAME

msgcat - Tcl message catalog

SYNOPSIS

```

package require Tcl 8.5

package require msgcat 1.6

::msgcat::mc src-string ?arg arg ...?

::msgcat::mcmax ?src-string src-string ...?

::msgcat::mcexists ?-exactnamespace? ?-exactlocale?
src-string

::msgcat::mclocale ?newLocale?

::msgcat::mcpreferences

::msgcat::mcloadedlocales subcommand ?locale?

::msgcat::mcload dirname

::msgcat::mcset locale src-string ?translate-string?

::msgcat::mcmset locale src-trans-list

::msgcat::mcflset src-string ?translate-string?

::msgcat::mcflmset src-trans-list

::msgcat::mcunknown locale src-string ?arg arg ...?

::msgcat::mcpackagelocale subcommand ?locale?

::msgcat::mcpackageconfig subcommand option ?value?

::msgcat::mcforgetpackage

```

DESCRIPTION

The **msgcat** package provides a set of functions that can be used to manage multi-lingual user interfaces. Text strings are defined in a which is independent from the application, and which can be edited or localized without modifying the application source code. New languages or locales may be provided by adding a new file to the message catalog.

msgcat distinguishes packages by its namespace. Each package has its own message catalog and configuration settings in **msgcat**.

A *locale* is a specification string describing a user language like **de_ch** for Swiss German. In **msgcat**, there is a global locale initialized by the system locale of the current system. Each package may decide to use the global locale or to use a package specific locale.

The global locale may be changed on demand, for example by a user initiated language change or within a multi user application like a web server.

COMMANDS

::msgcat::mc *src-string* ?*arg* *arg* ...?

Returns a translation of *src-string* according to the current locale. If additional arguments past *src-string* are given, the **format** command is used to substitute the additional arguments in the translation of *src-string*.

::msgcat::mc will search the messages defined in the current namespace for a translation of *src-string*; if none is found, it will search in the parent of the current namespace, and so on until it reaches the global namespace. If no translation string exists, **::msgcat::mcunknown** is called and the string returned from **::msgcat::mcunknown** is returned.

::msgcat::mc is the main function used to localize an application. Instead of using an English string directly, an application can pass the English string through **::msgcat::mc** and use the result. If an application is written for a single language in this fashion, then it is easy to add support for additional languages later simply by defining new message catalog entries.

coreTools Command Reference Index

::msgcat::mcmax *?src-string src-string ...?*

Given several source strings, **::msgcat::mcmax** returns the length of the longest translated string. This is useful when designing localized GUIs, which may require that all buttons, for example, be a fixed width (which will be the width of the widest button).

::msgcat::mcexists *?-exactnamespace? ?-exactlocale?*

src-string

Return true, if there is a translation for the given *src-string*.

The search may be limited by the option **-exactnamespace** to only check the current namespace and not any parent namespaces.

It may also be limited by the option **-exactlocale** to only check the first preferred locale (e.g. first element returned by **::msgcat::mcpreferences** if global locale is used).

::msgcat::mclocale *?newLocale?*

This function sets the locale to *newLocale*. If *newLocale* is omitted, the current locale is returned, otherwise the current locale is set to *newLocale*. msgcat stores and compares the locale in a case-insensitive manner, and returns locales in lowercase. The initial locale is determined by the locale specified in the user's environment. See **LOCALE SPECIFICATION** below for a description of the locale string format.

If the locale is set, the preference list of locales is evaluated. Locales in this list are loaded now, if not yet loaded.

::msgcat::mcpreferences

Returns an ordered list of the locales preferred by the user, based on the user's language specification. The list is ordered from most specific to least preference. The list is derived from the current locale set in msgcat by **::msgcat::mclocale**, and cannot be set independently. For example, if the current locale is *en_US_funky*, then **::msgcat::mcpreferences** returns **{en_us_funky en_us en {}}**.

::msgcat::mcloadedlocales *subcommand ?locale?*

This group of commands manage the list of loaded locales for packages not setting a package locale.

The subcommand **get** returns the list of currently loaded locales.

The subcommand **present** requires the argument *locale* and returns true, if this locale is loaded.

The subcommand **clear** removes all locales and their data, which are not in the current preference list.

::msgcat::mcload *dirname*

Searches the specified directory for files that match

coreTools Command Reference Index

the language specifications returned by
::msgcat::mcloadedlocales get (or
msgcat::mcpackagelocale preferences if a package locale
is set) (note that these are all lowercase), extended
by the file extension. Each matching file is read in
order, assuming a UTF-8 encoding. The file contents
are then evaluated as a Tcl script. This means that
Unicode characters may be present in the message file
either directly in their UTF-8 encoded form, or by use
of the backslash-u quoting recognized by Tcl
evaluation. The number of message files which matched
the specification and were loaded is returned.

In addition, the given folder is stored in the **msgcat**
package configuration option *mcfolder* to eventually
load message catalog files required by a locale change.

::msgcat::mcset locale src-string ?translate-string?
Sets the translation for *src-string* to *translate-string*
in the specified *locale* and the current namespace. If
translate-string is not specified, *src-string* is used
for both. The function returns *translate-string*.

::msgcat::mcmset locale src-trans-list
Sets the translation for multiple source strings in
src-trans-list in the specified *locale* and the current
namespace. *src-trans-list* must have an even number of
elements and is in the form {*src-string translate-*
string ?src-string translate-string ...?}
::msgcat::mcmset can be significantly faster than
multiple invocations of **::msgcat::mcset**. The function
returns the number of translations set.

::msgcat::mcflset src-string ?translate-string?
Sets the translation for *src-string* to *translate-string*
in the current namespace for the locale implied by the
name of the message catalog being loaded via
::msgcat::mcload. If *translate-string* is not
specified, *src-string* is used for both. The function
returns *translate-string*.

::msgcat::mcflmset src-trans-list
Sets the translation for multiple source strings in
src-trans-list in the current namespace for the locale
implied by the name of the message catalog being loaded
via **::msgcat::mcload**. *src-trans-list* must have an even
number of elements and is in the form {*src-string*
translate-string ?src-string translate-string ...?}
::msgcat::mcflmset can be significantly faster than
multiple invocations of **::msgcat::mcflset**. The function
returns the number of translations set.

::msgcat::mcunknown locale src-string ?arg arg ...?
This routine is called by **::msgcat::mc** in the case when
a translation for *src-string* is not defined in the
current locale. The default action is to return *src-*
string passed by format if there are any arguments.
This procedure can be redefined by the application, for
example to log error messages for each unknown string.
The **::msgcat::mcunknown** procedure is invoked at the
same stack context as the call to **::msgcat::mc**. The
return value of **::msgcat::mcunknown** is used as the

return value for the call to <code>::msgcat::mc</code> .		
Note that this routine is only called if the concerned package did not set a package locale unknown command name.		
<code>::msgcat::mcforgetpackage</code>		
The calling package clears all its state within the <code>msgcat</code> package including all settings and translations.		

LOCALE SPECIFICATION

The locale is specified to **`msgcat`** by a locale string passed to `::msgcat::mclocale`. The locale string consists of a language code, an optional country code, and an optional system-specific code, each separated by a hyphen. The country and language codes are specified in standards ISO-639 and ISO-3166. For example, the locale specifies English and specifies U.S. English.

When the **`msgcat`** package is first loaded, the locale is initialized according to the user's environment. The variables **`env(LC_ALL)`**, **`env(LC_MESSAGES)`**, and **`env(LANG)`** are examined in order. The first of them to have a non-empty value is used to determine the initial locale. The value is parsed according to the XPG4 pattern

```
language[_country][.codeset][@modifier]
```

to extract its parts. The initial locale is then set by calling `::msgcat::mclocale` with the argument

```
language[_country][_modifier]
```

On Windows and Cygwin, if none of those environment variables is set, **`msgcat`** will attempt to extract locale information from the registry. From Windows Vista on, the RFC4747 locale name "lang-script-country-options" is transformed to the locale as "lang_country_script" (Example: sr-Latn-CS -> sr_cs_latin). For Windows XP, the language id is transformed analogously (Example: 0c1a -> sr_yu_cyrillic). If all these attempts to discover an initial locale from the user's environment fail, **`msgcat`** defaults to an initial locale of

When a locale is specified by the user, a search is performed during string translation. For example, if a user specifies `en_GB_Funky`, the locales and (the empty string) are searched in order until a matching translation string is found. If no translation string is available, then the unknown handler is called.

NAMESPACES AND MESSAGE CATALOGS

Strings stored in the message catalog are stored relative to the namespace from which they were added. This allows multiple packages to use the same strings without fear of collisions with other packages. It also allows the source string to be shorter and less prone to typographical error.

For example, executing the code

```
::msgcat::mcset en hello "hello from ::"
namespace eval foo {
    ::msgcat::mcset en hello "hello from ::foo"
}
puts [::msgcat::mc hello]
namespace eval foo {puts [::msgcat::mc hello]}

will print

hello from ::
hello from ::foo
```

When searching for a translation of a message, the message catalog will search first the current namespace, then the parent of the current namespace, and so on until the global namespace is reached. This allows child namespaces to messages from their parent namespace.

For example, executing (in the locale) the code

```
::msgcat::mcset en m1 ":: message1"
::msgcat::mcset en m2 ":: message2"
::msgcat::mcset en m3 ":: message3"
namespace eval ::foo {
    ::msgcat::mcset en m2 "::foo message2"
    ::msgcat::mcset en m3 "::foo message3"
}
namespace eval ::foo::bar {
    ::msgcat::mcset en m3 "::foo::bar message3"
}
namespace import ::msgcat::mc
puts "[mc m1]; [mc m2]; [mc m3]"
namespace eval ::foo {puts "[mc m1]; [mc m2]; [mc m3]"}
namespace eval ::foo::bar {puts "[mc m1]; [mc m2]; [mc m3]"}
will print

:: message1; :: message2; :: message3
:: message1; ::foo message2; ::foo message3
:: message1; ::foo message2; ::foo::bar message3
```

LOCATION AND FORMAT OF MESSAGE FILES

Message files can be located in any directory, subject to the following conditions:

- [1] All message files for a package are in

coreTools Command Reference Index

the same directory.

- [2] The message file name is a msgcat locale specifier (all lowercase) followed by For example:

```
es.msg      -- spanish
en_gb.msg   -- United Kingdom English
```

Exception: The message file for the root locale is called This exception is made so as not to cause peculiar behavior, such as marking the message file as on Unix file systems.

- [3] The file contains a series of calls to **mcflset** and **mcflmset**, setting the necessary translation strings for the language, likely enclosed in a **namespace eval** so that all source strings are tied to the namespace of the package. For example, a short **es.msg** might contain:

```
namespace eval ::mypackage {
    ::msgcat::mcflset "Free Beer" "Cerveza Gratis"
}
```

RECOMMENDED MESSAGE SETUP FOR PACKAGES

If a package is installed into a subdirectory of the **tcl_pkgPath** and loaded via **package require**, the following procedure is recommended.

- [1] During package installation, create a subdirectory **msgs** under your package directory.
- [2] Copy your *.msg files into that directory.
- [3] Add the following command to your package initialization script:

```
# load language files, stored in msgs subdirectory
::msgcat::mcload [file join [file dirname [info script]] msgs]
```

POSITIONAL CODES FOR FORMAT AND SCAN COMMANDS

It is possible that a message string used as an argument to **format** might have positionally dependent parameters that might need to be repositioned. For example, it might be syntactically desirable to rearrange the sentence structure while translating.

```
format "We produced %d units in location %s" $num $city
format "In location %s we produced %d units" $city $num
```

coreTools Command Reference Index

This can be handled by using the positional parameters:

```
format "We produced %1$d units in location %2$s" $num $city
format "In location %2$s we produced %1$d units" $num $city
```

Similarly, positional parameters can be used with **scan** to extract values from internationalized strings. Note that it is not necessary to pass the output of **::msgcat::mc** to **format** directly; by passing the values to substitute in as arguments, the formatting substitution is done directly.

```
msgcat::mc {Produced %1$d at %2$s} $num $city
# ... where that key is mapped to one of the
# human-oriented versions by msgcat::mcset
```

Package private locale |

A package using **msgcat** may choose to use its own package private locale and its own set of loaded locales, independent to the global locale set by **::msgcat::mclocale**.

This allows a package to change its locale without causing any locales load or removal in other packages and not to invoke the global locale change callback (see below).

This action is controlled by the following ensemble:

```
::msgcat::mcpackagelocale set
?locale?
|
Set or change a package private locale. The package
private locale is set to the given locale if the locale
is given. If the option locale is not given, the
package is set to package private locale mode, but no
locale is changed (e.g. if the global locale was valid
for the package before, it is copied to the package
private locale).
```

This command may cause the load of locales.

```
::msgcat::mcpackagelocale
get
|
Return the package private locale or the global locale,
if no package private locale is set.
```

```
::msgcat::mcpackagelocale
preferences
|
Return the package private preferences or the global
preferences, if no package private locale is set.
```

```
::msgcat::mcpackagelocale
loaded
|
Return the list of locales loaded for this package.
```

```
::msgcat::mcpackagelocale
isset
|
Returns true, if a package private locale is set.
```

```

::msgcat::mcpackagelocale
unset
Unset the package private locale and use the globale
locale. Load and remove locales to adjust the list of
loaded locales for the package to the global loaded
locales list.

::msgcat::mcpackagelocale present
locale
Returns true, if the given locale is loaded for the
package.

::msgcat::mcpackagelocale
clear
Clear any loaded locales of the package not present in
the package preferences.

```

Changing package options |

Each package using msgcat has a set of options within **msgcat**. The package options are described in the next sectionPackage options. Each package option may be set or unset individually using the following ensemble:

```

::msgcat::mcpackageconfig get
option
Return the current value of the given option. This
call returns an error if the option is not set for the
package.

::msgcat::mcpackageconfig isset
option
Returns 1, if the given option is set for the package,
0 otherwise.

::msgcat::mcpackageconfig set option
value
Set the given option to the given value. This may
invoke additional actions in dependency of the option.
The return value is 0 or the number of loaded packages
for the option mcfolder.

::msgcat::mcpackageconfig unset
option
Unsets the given option for the package. No action is
taken if the option is not set for the package. The
empty string is returned.

```

Package options

The following package options are available for each package:

```

mcfolder
This is the message folder of the package. This option
is set by mclload and by the subcommand set. Both are
identical and both return the number of loaded message
catalog files.

```

||

coreTools Command Reference Index

Setting or changing this value will load all locales contained in the preferences valid for the package. This implies also to invoke any set loadcmd (see below).

Unsetting this value will disable message file load for the package.

loadcmd

This callback is invoked before a set of message catalog files are loaded for the package which has this property set.

This callback may be used to do any preparation work for message file load or to get the message data from another source like a data base. In this case, no message files are used (mcfolder is unset).

See section **callback invocation** below. The parameter list appended to this callback is the list of locales to load.

If this callback is changed, it is called with the preferences valid for the package.

changecmd

This callback is invoked when a default local change was performed. Its purpose is to allow a package to update any dependency on the default locale like showing the GUI in another language.

See the callback invocation section below. The parameter list appended to this callback is **mcpreferences**. The registered callbacks are invoked in no particular order.

unknowncmd

Use a package locale mcunknown procedure instead of the standard version supplied by the msgcat package (msgcat::mcunknown).

The called procedure must return the formatted message which will finally be returned by msgcat::mc.

A generic unknown handler is used if set to the empty string. This consists in returning the key if no arguments are given. With given arguments, format is used to process the arguments.

See section **callback invocation** below. The appended arguments are identical to **::msgcat::mcunknown**.

Callback invocation

A package may decide to register one or multiple callbacks, as described above.

Callbacks are invoked, if:

1. the callback command is set,
2. the command is not the empty string,

coreTools Command Reference Index

3. the registering namespace exists.

If a called routine fails with an error, the **bgerror** routine for the interpreter is invoked after command completion. Only exception is the callback **unknowncmd**, where an error causes the invoking **mc**-command to fail with that error.

Examples

Packages which display a GUI may update their widgets when the global locale changes. To register to a callback, use:

```
namespace eval gui {
    msgcat::mcpackageconfig changecmd updateGUI

    proc updateGui args {
        puts "New locale is '[lindex $args 0]'."
    }
}
% msgcat::mclocale fr
fr
% New locale is 'fr'.
```

If locales (or additional locales) are contained in another source like a data base, a package may use the load callback and not mcload:

```
namespace eval db {
    msgcat::mcpackageconfig loadcmd loadMessages

    proc loadMessages args {
        foreach locale $args {
            if {[LocaleInDB $locale]} {
                msgcat::mcmset $locale [GetLocaleList $locale]
            }
        }
    }
}
```

The **clock** command implementation uses **msgcat** with a package locale to implement the command line parameter **-locale**. Here are some sketches of the implementation:

First, a package locale is initialized and the generic unknown function is deactivated:

```
msgcat::mcpackagelocale set
msgcat::mcpackageconfig unknowncmd ""
```

As an example, the user requires the week day in a certain locale as follows:

```
clock format clock seconds -format %A -locale fr
clock sets the package locale to fr and looks for the day name as follows:
```

```
msgcat::mcpackagelocale set $locale
return [lindex [msgcat::mc DAYS_OF_WEEK_FULL] $day]
### Returns "mercredi"
```

Within **clock**, some message-catalog items are heavy in computation and thus are dynamically cached using:

```
proc ::tcl::clock::LocalizeFormat { locale format } {
    set key FORMAT_$format
    if { [::msgcat::mcexists -exactlocale -exactnamespace $key] } {
        return [mc $key]
    }
}
```

coreTools Command Reference Index

```
#...expensive computation of format clipped...  
mcset $locale $key $format  
return $format  
}
```

CREDITS

The message catalog code was developed by Mark Harrison.

SEE ALSO

format(n), scan(n), namespace(n), package(n)

KEYWORDS

internationalization, i18n, localization, l10n,
message, text, translation

NAME

`my` - invoke any method of current object

SYNOPSIS

```
package require Tcl00
```

```
my methodName ?arg ...?
```

DESCRIPTION

The **my** command is used to allow methods of objects to invoke any method of the object (or its class). In particular, the set of valid values for *methodName* is the set of all methods supported by an object and its superclasses, including those that are not exported. The object upon which the method is invoked is always the one that is the current context of the method (i.e. the object that is returned by **self object**) from which the **my** command is invoked.

Each object has its own **my** command, contained in its instance namespace.

EXAMPLES

This example shows basic use of **my** to use the **variables** method of the **oo::object** class, which is not publicly visible by default:

```
oo::class create c {  
    method count {} {  
        my variable counter
```

coreTools Command Reference Index

```
        puts [incr counter]
    }
}
c create o
o count      -> prints "1"
o count      -> prints "2"
o count      -> prints "3"
```

SEE ALSO

`next(n)`, `oo::object(n)`, `self(n)`

KEYWORDS

`method`, `method visibility`, `object`, `private method`,
`public method`

NAME

`namespace` - create and manipulate contexts for commands and variables

SYNOPSIS

`namespace` *?subcommand?* *?arg ...?*

DESCRIPTION

The **namespace** command lets you create, access, and destroy separate contexts for commands and variables. See the section **WHAT IS A NAMESPACE?** below for a brief overview of namespaces. The legal values of *subcommand* are listed below. Note that you can abbreviate the *subcommands*.

namespace children *?namespace?* *?pattern?*

Returns a list of all child namespaces that belong to the namespace *namespace*. If *namespace* is not specified, then the children are returned for the current namespace. This command returns fully-qualified names, which start with a double colon (::). If the optional *pattern* is given, then this command returns only the names that match the glob-style pattern. The actual pattern used is determined as follows: a pattern that starts with double colon (::) is used directly, otherwise the namespace *namespace* (or the fully-qualified name of the current namespace) is prepended onto the pattern.

namespace code script

Captures the current namespace context for later execution of the script *script*. It returns a new script in which *script* has been wrapped in a **namespace** **inscope** command. The new script has two important properties. First, it can be evaluated in any

coreTools Command Reference Index

namespace and will cause *script* to be evaluated in the current namespace (the one where the **namespace code** command was invoked). Second, additional arguments can be appended to the resulting script and they will be passed to *script* as additional arguments. For example, suppose the command **set script [namespace code {foo bar}]** is invoked in namespace **::a::b**. Then **eval \$script [list x y]** can be executed in any namespace (assuming the value of **script** has been passed in properly) and will have the same effect as the command **::namespace eval ::a::b {foo bar x y}**. This command is needed because extensions like Tk normally execute callback scripts in the global namespace. A scoped command captures a command together with its namespace context in a way that allows it to be executed properly later. See the section **SCOPED SCRIPTS** for some examples of how this is used to create callback scripts.

namespace current

Returns the fully-qualified name for the current namespace. The actual name of the global namespace is (i.e., an empty string), but this command returns **::** for the global namespace as a convenience to programmers.

namespace delete ?namespace namespace ...?

Each namespace *namespace* is deleted and all variables, procedures, and child namespaces contained in the namespace are deleted. If a procedure is currently executing inside the namespace, the namespace will be kept alive until the procedure returns; however, the namespace is marked to prevent other code from looking it up by name. If a namespace does not exist, this command returns an error. If no namespace names are given, this command does nothing.

namespace ensemble subcommand ?arg ...?

Creates and manipulates a command that is formed out of an ensemble of subcommands. See the section **ENSEMBLES** below for further details.

namespace eval namespace arg ?arg ...?

Activates a namespace called *namespace* and evaluates some code in that context. If the namespace does not already exist, it is created. If more than one *arg* argument is specified, the arguments are concatenated together with a space between each one in the same fashion as the **eval** command, and the result is evaluated.

If *namespace* has leading namespace qualifiers and any leading namespaces do not exist, they are automatically created.

namespace exists namespace

Returns **1** if *namespace* is a valid namespace in the current context, returns **0** otherwise.

namespace export ?-clear? ?pattern pattern ...?

Specifies which commands are exported from a namespace. The exported commands are those that can be later

coreTools Command Reference Index

imported into another namespace using a **namespace import** command. Both commands defined in a namespace and commands the namespace has previously imported can be exported by a namespace. The commands do not have to be defined at the time the **namespace export** command is executed. Each *pattern* may contain glob-style special characters, but it may not include any namespace qualifiers. That is, the pattern can only specify commands in the current (exporting) namespace. Each *pattern* is appended onto the namespace's list of export patterns. If the **-clear** flag is given, the namespace's export pattern list is reset to empty before any *pattern* arguments are appended. If no *patterns* are given and the **-clear** flag is not given, this command returns the namespace's current export list.

namespace forget *?pattern pattern ...?*

Removes previously imported commands from a namespace. Each *pattern* is a simple or qualified name such as **x**, **foo::x** or **a::b::p***. Qualified names contain double colons (::) and qualify a name with the name of one or more namespaces. Each is qualified with the name of an exporting namespace and may have glob-style special characters in the command name at the end of the qualified name. Glob characters may not appear in a namespace name. For each this command deletes the matching commands of the current namespace that were imported from a different namespace. For this command first finds the matching exported commands. It then checks whether any of those commands were previously imported by the current namespace. If so, this command deletes the corresponding imported commands. In effect, this un-does the action of a **namespace import** command.

namespace import *?-force? ?pattern pattern ...?*

Imports commands into a namespace, or queries the set of imported commands in a namespace. When no arguments are present, **namespace import** returns the list of commands in the current namespace that have been imported from other namespaces. The commands in the returned list are in the format of simple names, with no namespace qualifiers at all. This format is suitable for composition with **namespace forget** (see **EXAMPLES** below).

When *pattern* arguments are present, each *pattern* is a qualified name like **foo::x** or **a::p***. That is, it includes the name of an exporting namespace and may have glob-style special characters in the command name at the end of the qualified name. Glob characters may not appear in a namespace name. When the namespace name is not fully qualified (i.e., does not start with a namespace separator) it is resolved as a namespace name in the way described in the **NAME RESOLUTION** section; it is an error if no namespace with that name can be found.

All the commands that match a *pattern* string and which are currently exported from their namespace are added to the current namespace. This is done by creating a

coreTools Command Reference Index

new command in the current namespace that points to the exported command in its original namespace; when the new imported command is called, it invokes the exported command. This command normally returns an error if an imported command conflicts with an existing command. However, if the **-force** option is given, imported commands will silently replace existing commands. The **namespace import** command has snapshot semantics: that is, only requested commands that are currently defined in the exporting namespace are imported. In other words, you can import only the commands that are in a namespace at the time when the **namespace import** command is executed. If another command is defined and exported in this namespace later on, it will not be imported.

namespace inscope *namespace script ?arg ...?*

Executes a script in the context of the specified *namespace*. This command is not expected to be used directly by programmers; calls to it are generated implicitly when applications use **namespace code** commands to create callback scripts that the applications then register with, e.g., Tk widgets. The **namespace inscope** command is much like the **namespace eval** command except that the *namespace* must already exist, and **namespace inscope** appends additional *args* as proper list elements.

namespace inscope **::foo** **\$script** **\$x** **\$y** **\$z**

is equivalent to

namespace eval **::foo** [**concat** **\$script** [**list** **\$x** **\$y** **\$z**]]

thus additional arguments will not undergo a second round of substitution, as is the case with **namespace eval**.

namespace origin *command*

Returns the fully-qualified name of the original command to which the imported command *command* refers. When a command is imported into a namespace, a new command is created in that namespace that points to the actual command in the exporting namespace. If a command is imported into a sequence of namespaces *a*, *b*, ..., *n* where each successive namespace just imports the command from the previous namespace, this command returns the fully-qualified name of the original command in the first namespace, *a*. If *command* does not refer to an imported command, the command's own fully-qualified name is returned.

namespace parent *?namespace?*

Returns the fully-qualified name of the parent namespace for namespace *namespace*. If *namespace* is not specified, the fully-qualified name of the current namespace's parent is returned.

namespace path *?namespaceList?*

Returns the command resolution path of the current namespace. If *namespaceList* is specified as a list of named namespaces, the current namespace's command

coreTools Command Reference Index

resolution path is set to those namespaces and returns the empty list. The default command resolution path is always empty. See the section **NAME RESOLUTION** below for an explanation of the rules regarding name resolution.

namespace qualifiers *string*

Returns any leading namespace qualifiers for *string*. Qualifiers are namespace names separated by double colons (::). For the *string ::foo::bar::x*, this command returns *::foo::bar*, and for *::* it returns an empty string. This command is the complement of the **namespace tail** command. Note that it does not check whether the namespace names are, in fact, the names of currently defined namespaces.

namespace tail *string*

Returns the simple name at the end of a qualified string. Qualifiers are namespace names separated by double colons (::). For the *string ::foo::bar::x*, this command returns *x*, and for *::* it returns an empty string. This command is the complement of the **namespace qualifiers** command. It does not check whether the namespace names are, in fact, the names of currently defined namespaces.

namespace upvar *namespace ?otherVar myVar ...?*

This command arranges for zero or more local variables in the current procedure to refer to variables in *namespace*. The namespace name is resolved as described in section **NAME RESOLUTION**. The command **namespace upvar \$ns a b** has the same behaviour as **upvar 0 \${ns}::a b**, with the sole exception of the resolution rules used for qualified namespace or variable names. **namespace upvar** returns an empty string.

namespace unknown *?script?*

Sets or returns the unknown command handler for the current namespace. The handler is invoked when a command called from within the namespace cannot be found in the current namespace, the namespace's path nor in the global namespace. The *script* argument, if given, should be a well formed list representing a command name and optional arguments. When the handler is invoked, the full invocation line will be appended to the script and the result evaluated in the context of the namespace. The default handler for all namespaces is *::unknown*. If no argument is given, it returns the handler for the current namespace.

namespace which *?-command? ?-variable? name*

Looks up *name* as either a command or variable and returns its fully-qualified name. For example, if *name* does not exist in the current namespace but does exist in the global namespace, this command returns a fully-qualified name in the global namespace. If the command or variable does not exist, this command returns an empty string. If the variable has been created but not defined, such as with the **variable** command or through a **trace** on the variable, this command will return the fully-qualified name of the variable. If no flag is given, *name* is treated as a command name. See the section **NAME RESOLUTION** below for an explanation of the

rules regarding name resolution.

WHAT IS A NAMESPACE?

A namespace is a collection of commands and variables. It encapsulates the commands and variables to ensure that they will not interfere with the commands and variables of other namespaces. Tcl has always had one such collection, which we refer to as the *global namespace*. The global namespace holds all global variables and commands. The **namespace eval** command lets you create new namespaces. For example,

```
namespace eval Counter {
    namespace export bump
    variable num 0

    proc bump {} {
        variable num
        incr num
    }
}
```

creates a new namespace containing the variable **num** and the procedure **bump**. The commands and variables in this namespace are separate from other commands and variables in the same program. If there is a command named **bump** in the global namespace, for example, it will be different from the command **bump** in the **Counter** namespace.

Namespace variables resemble global variables in Tcl. They exist outside of the procedures in a namespace but can be accessed in a procedure via the **variable** command, as shown in the example above.

Namespaces are dynamic. You can add and delete commands and variables at any time, so you can build up the contents of a namespace over time using a series of **namespace eval** commands. For example, the following series of commands has the same effect as the namespace definition shown above:

```
namespace eval Counter {
    variable num 0
    proc bump {} {
        variable num
        return [incr num]
    }
}
namespace eval Counter {
    proc test {args} {
        return $args
    }
}
namespace eval Counter {
    rename test ""
}
```

coreTools Command Reference Index

Note that the **test** procedure is added to the **Counter** namespace, and later removed via the **rename** command.

Namespaces can have other namespaces within them, so they nest hierarchically. A nested namespace is encapsulated inside its parent namespace and can not interfere with other namespaces.

QUALIFIED NAMES

Each namespace has a textual name such as **history** or **::safe::interp**. Since namespaces may nest, qualified names are used to refer to commands, variables, and child namespaces contained inside namespaces. Qualified names are similar to the hierarchical path names for Unix files or Tk widgets, except that **::** is used as the separator instead of **/** or **..**. The topmost or global namespace has the name (i.e., an empty string), although **::** is a synonym. As an example, the name **::safe::interp::create** refers to the command **create** in the namespace **interp** that is a child of namespace **::safe**, which in turn is a child of the global namespace, **::**.

If you want to access commands and variables from another namespace, you must use some extra syntax. Names must be qualified by the namespace that contains them. From the global namespace, we might access the **Counter** procedures like this:

```
Counter::bump 5
Counter::Reset
```

We could access the current count like this:

```
puts "count = $Counter::num"
```

When one namespace contains another, you may need more than one qualifier to reach its elements. If we had a namespace **Foo** that contained the namespace **Counter**, you could invoke its **bump** procedure from the global namespace like this:

```
Foo::Counter::bump 3
```

You can also use qualified names when you create and rename commands. For example, you could add a procedure to the **Foo** namespace like this:

```
proc Foo::Test {args} {return $args}
```

And you could move the same procedure to another namespace like this:

```
rename Foo::Test Bar::Test
```

There are a few remaining points about qualified names that we should cover. Namespaces have nonempty names except for the global namespace. **::** is disallowed in

coreTools Command Reference Index

simple command, variable, and namespace names except as a namespace separator. Extra colons in any separator part of a qualified name are ignored; i.e. two or more colons are treated as a namespace separator. A trailing `::` in a qualified variable or command name refers to the variable or command named `{}`. However, a trailing `::` in a qualified namespace name is ignored.

NAME RESOLUTION

In general, all Tcl commands that take variable and command names support qualified names. This means you can give qualified names to such commands as **set**, **proc**, **rename**, and **interp alias**. If you provide a fully-qualified name that starts with a `::`, there is no question about what command, variable, or namespace you mean. However, if the name does not start with a `::` (i.e., is *relative*), Tcl follows basic rules for looking it up:

- o **Variable names** are always resolved by looking first in the current namespace, and then in the global namespace.
- o **Command names** are always resolved by looking in the current namespace first. If not found there, they are searched for in every namespace on the current namespace's command path (which is empty by default). If not found there, command names are looked up in the global namespace (or, failing that, are processed by the appropriate **namespace unknown** handler.)
- o **Namespace names** are always resolved by looking in only the current namespace.

In the following example,

```
set traceLevel 0
namespace eval Debug {
    printTrace $traceLevel
}
```

Tcl looks for **traceLevel** in the namespace **Debug** and then in the global namespace. It looks up the command **printTrace** in the same way. If a variable or command name is not found in either context, the name is undefined. To make this point absolutely clear, consider the following example:

```
set traceLevel 0
namespace eval Foo {
    variable traceLevel 3

    namespace eval Debug {
        printTrace $traceLevel
    }
}
```

```
}
```

Here Tcl looks for **traceLevel** first in the namespace **Foo::Debug**. Since it is not found there, Tcl then looks for it in the global namespace. The variable **Foo::traceLevel** is completely ignored during the name resolution process.

You can use the **namespace which** command to clear up any question about name resolution. For example, the command:

```
namespace eval Foo::Debug {namespace which -variable traceLevel}
```

returns **::traceLevel**. On the other hand, the command,

```
namespace eval Foo {namespace which -variable traceLevel}
```

returns **::Foo::traceLevel**.

As mentioned above, namespace names are looked up differently than the names of variables and commands. Namespace names are always resolved in the current namespace. This means, for example, that a **namespace eval** command that creates a new namespace always creates a child of the current namespace unless the new namespace name begins with **::**.

Tcl has no access control to limit what variables, commands, or namespaces you can reference. If you provide a qualified name that resolves to an element by the name resolution rule above, you can access the element.

You can access a namespace variable from a procedure in the same namespace by using the **variable** command. Much like the **global** command, this creates a local link to the namespace variable. If necessary, it also creates the variable in the current namespace and initializes it. Note that the **global** command only creates links to variables in the global namespace. It is not necessary to use a **variable** command if you always refer to the namespace variable using an appropriate qualified name.

IMPORTING COMMANDS

Namespaces are often used to represent libraries. Some library commands are used so frequently that it is a nuisance to type their qualified names. For example, suppose that all of the commands in a package like BLT are contained in a namespace called **Blt**. Then you might access these commands like this:

```
Blt::graph .g -background red
Blt::table . .g 0,0
```

If you use the **graph** and **table** commands frequently, you may want to access them without the **Blt::** prefix. You can do this by importing the commands into the current

coreTools Command Reference Index

namespace, like this:

```
namespace import Blt::*
```

This adds all exported commands from the **Blt** namespace into the current namespace context, so you can write code like this:

```
graph .g -background red
table . .g 0,0
```

The **namespace import** command only imports commands from a namespace that that namespace exported with a **namespace export** command.

Importing *every* command from a namespace is generally a bad idea since you do not know what you will get. It is better to import just the specific commands you need. For example, the command

```
namespace import Blt::graph Blt::table
```

imports only the **graph** and **table** commands into the current context.

If you try to import a command that already exists, you will get an error. This prevents you from importing the same command from two different packages. But from time to time (perhaps when debugging), you may want to get around this restriction. You may want to reissue the **namespace import** command to pick up new commands that have appeared in a namespace. In that case, you can use the **-force** option, and existing commands will be silently overwritten:

```
namespace import -force Blt::graph Blt::table
```

If for some reason, you want to stop using the imported commands, you can remove them with a **namespace forget** command, like this:

```
namespace forget Blt::*
```

This searches the current namespace for any commands imported from **Blt**. If it finds any, it removes them. Otherwise, it does nothing. After this, the **Blt** commands must be accessed with the **Blt::** prefix.

When you delete a command from the exporting namespace like this:

```
rename Blt::graph ""
```

the command is automatically removed from all namespaces that import it.

EXPORTING COMMANDS

You can export commands from a namespace like this:

```
namespace eval Counter {
    namespace export bump reset
    variable Num 0
    variable Max 100

    proc bump {{by 1}} {
        variable Num
        incr Num $by
        Check
        return $Num
    }
    proc reset {} {
        variable Num
        set Num 0
    }
    proc Check {} {
        variable Num
        variable Max
        if {$Num > $Max} {
            error "too high!"
        }
    }
}
```

The procedures **bump** and **reset** are exported, so they are included when you import from the **Counter** namespace, like this:

```
namespace import Counter::*
```

However, the **Check** procedure is not exported, so it is ignored by the import operation.

The **namespace import** command only imports commands that were declared as exported by their namespace. The **namespace export** command specifies what commands may be imported by other namespaces. If a **namespace import** command specifies a command that is not exported, the command is not imported.

SCOPED SCRIPTS

The **namespace code** command is the means by which a script may be packaged for evaluation in a namespace other than the one in which it was created. It is used most often to create event handlers, Tk bindings, and traces for evaluation in the global context. For instance, the following code indicates how to direct a variable **trace** callback into the current namespace:

```
namespace eval a {
    variable b
    proc theTraceCallback { n1 n2 op } {
        upvar 1 $n1 var
```

coreTools Command Reference Index

```
puts "the value of $n1 has changed to $var"
return
}
trace add variable b write [namespace code theTraceCallback]
}
set a::b c
```

When executed, it prints the message:

the value of a::b has changed to c

ENSEMBLES

The **namespace ensemble** is used to create and manipulate ensemble commands, which are commands formed by grouping subcommands together. The commands typically come from the current namespace when the ensemble was created, though this is configurable. Note that there may be any number of ensembles associated with any namespace (including none, which is true of all namespaces by default), though all the ensembles associated with a namespace are deleted when that namespace is deleted. The link between an ensemble command and its namespace is maintained however the ensemble is renamed.

Three subcommands of the **namespace ensemble** command are defined:

namespace ensemble create *?option value ...?*

Creates a new ensemble command linked to the current namespace, returning the fully qualified name of the command created. The arguments to **namespace ensemble create** allow the configuration of the command as if with the **namespace ensemble configure** command. If not overridden with the **-command** option, this command creates an ensemble with exactly the same name as the linked namespace. See the section **ENSEMBLE OPTIONS** below for a full list of options supported and their effects.

namespace ensemble configure *command ?option? ?value ...?*

Retrieves the value of an option associated with the ensemble command named *command*, or updates some options associated with that ensemble command. See the section **ENSEMBLE OPTIONS** below for a full list of options supported and their effects.

namespace ensemble exists *command*

Returns a boolean value that describes whether the command *command* exists and is an ensemble command. This command only ever returns an error if the number of arguments to the command is wrong.

When called, an ensemble command takes its first argument and looks it up (according to the rules described below) to discover a list of words to replace the ensemble command and subcommand with. The

coreTools Command Reference Index

resulting list of words is then evaluated (with no further substitutions) as if that was what was typed originally (i.e. by passing the list of words through **Tcl_EvalObjv**) and returning the result of the command. Note that it is legal to make the target of an ensemble rewrite be another (or even the same) ensemble command. The ensemble command will not be visible through the use of the **uplevel** or **info level** commands.

ENSEMBLE OPTIONS

The following options, supported by the **namespace ensemble create** and **namespace ensemble configure** commands, control how an ensemble command behaves:

-map

When non-empty, this option supplies a dictionary that provides a mapping from subcommand names to a list of prefix words to substitute in place of the ensemble command and subcommand words (in a manner similar to an alias created with **interp alias**; the words are not reparsed after substitution); if the first word of any target is not fully qualified when set, it is assumed to be relative to the *current* namespace and changed to be exactly that (that is, it is always fully qualified when read). When this option is empty, the mapping will be from the local name of the subcommand to its fully-qualified name. Note that when this option is non-empty and the **-subcommands** option is empty, the ensemble subcommand names will be exactly those words that have mappings in the dictionary.

-parameters

This option gives a list of named arguments (the names being used during generation of error messages) that are passed by the caller of the ensemble between the name of the ensemble and the subcommand argument. By default, it is the empty list.

-prefixes

This option (which is enabled by default) controls whether the ensemble command recognizes unambiguous prefixes of its subcommands. When turned off, the ensemble command requires exact matching of subcommand names.

-subcommands

When non-empty, this option lists exactly what subcommands are in the ensemble. The mapping for each of those commands will be either whatever is defined in the **-map** option, or to the command with the same name in the namespace linked to the ensemble. If this option is empty, the subcommands of the namespace will either be the keys of the dictionary listed in the **-map** option or the exported commands of the linked namespace at the time of the invocation of the ensemble command.

-unknown

When non-empty, this option provides a partial command (to which all the words that are arguments to the ensemble command, including the fully-qualified name of the ensemble, are appended) to handle the case where an ensemble subcommand is not recognized and would

coreTools Command Reference Index

otherwise generate an error. When empty (the default) an error (in the style of `Tcl_GetIndexFromObj`) is generated whenever the ensemble is unable to determine how to implement a particular subcommand. See **UNKNOWN HANDLER BEHAVIOUR** for more details.

The following extra option is allowed by **namespace ensemble create**:

-command

This write-only option allows the name of the ensemble created by **namespace ensemble create** to be anything in any existing namespace. The default value for this option is the fully-qualified name of the namespace in which the **namespace ensemble create** command is invoked.

The following extra option is allowed by **namespace ensemble configure**:

-namespace

This read-only option allows the retrieval of the fully-qualified name of the namespace which the ensemble was created within.

UNKNOWN HANDLER BEHAVIOUR

If an unknown handler is specified for an ensemble, that handler is called when the ensemble command would otherwise return an error due to it being unable to decide which subcommand to invoke. The exact conditions under which that occurs are controlled by the **-subcommands**, **-map** and **-prefixes** options as described above.

To execute the unknown handler, the ensemble mechanism takes the specified **-unknown** option and appends each argument of the attempted ensemble command invocation (including the ensemble command itself, expressed as a fully qualified name). It invokes the resulting command in the scope of the attempted call. If the execution of the unknown handler terminates normally, the ensemble engine reparses the subcommand (as described below) and tries to dispatch it again, which is ideal for when the ensemble's configuration has been updated by the unknown subcommand handler. Any other kind of termination of the unknown handler is treated as an error.

The result of the unknown handler is expected to be a list (it is an error if it is not). If the list is an empty list, the ensemble command attempts to look up the original subcommand again and, if it is not found this time, an error will be generated just as if the **-unknown** handler was not there (i.e. for any particular invocation of an ensemble, its unknown handler will be called at most once.) This makes it easy for the unknown handler to update the ensemble or its backing namespace so as to provide an implementation of the desired subcommand and reparse.

When the result is a non-empty list, the words of that list are used to replace the ensemble command and subcommand, just as if they had been looked up in the

coreTools Command Reference Index

-map. It is up to the unknown handler to supply all namespace qualifiers if the implementing subcommand is not in the namespace of the caller of the ensemble command. Also note that when ensemble commands are chained (e.g. if you make one of the commands that implement an ensemble subcommand into an ensemble, in a manner similar to the **text** widget's tag and mark subcommands) then the rewrite happens in the context of the caller of the outermost ensemble. That is to say that ensembles do not in themselves place any namespace contexts on the Tcl call stack.

Where an empty **-unknown** handler is given (the default), the ensemble command will generate an error message based on the list of commands that the ensemble has defined (formatted similarly to the error message from **Tcl_GetIndexFromObj**). This is the error that will be thrown when the subcommand is still not recognized during reparsing. It is also an error for an **-unknown** handler to delete its namespace.

EXAMPLES

Create a namespace containing a variable and an exported command:

```
namespace eval foo {
    variable bar 0
    proc grill {} {
        variable bar
        puts "called [incr bar] times"
    }
    namespace export grill
}
```

Call the command defined in the previous example in various ways.

```
# Direct call
::foo::grill
```

```
# Use the command resolution path to find the name
namespace eval boo {
    namespace path ::foo
    grill
}
```

```
# Import into current namespace, then call local alias
namespace import foo::grill
grill
```

```
# Create two ensembles, one with the default name and one with a
# specified name. Then call through the ensembles.
```

```
namespace eval foo {
    namespace ensemble create
    namespace ensemble create -command ::foobar
}
foo grill
foobar grill
```

coreTools Command Reference Index

Look up where the command imported in the previous example came from:

```
puts "grill came from [namespace origin grill]"
```

Remove all imported commands from the current namespace:

```
namespace forget {*}[namespace import]
```

Create an ensemble for simple working with numbers, using the **-parameters** option to allow the operator to be put between the first and second arguments.

```
namespace eval do {  
    namespace export *  
    namespace ensemble create -parameters x  
    proc plus {x y} {expr { $x + $y }}  
    proc minus {x y} {expr { $x - $y }}  
}
```

```
# In use, the ensemble works like this:  
puts [do 1 plus [do 9 minus 7]]
```

SEE ALSO

interp(n), upvar(n), variable(n)

KEYWORDS

command, ensemble, exported, internal, variable

net
Net item

Description

The net item type represents a net in the coreConsultant or coreBuilder model of the design. When coreBuilder parses the HDL source code for the core, it creates a net item for each net port/pin to port/pin connection that it finds.

See Also

Supported Attributes

[ComponentOfItem \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [EndBit \(3\)](#),
[IsBehavioral \(3\)](#), [Label \(3\)](#), [Name \(3\)](#), [Sequence \(3\)](#), [StartBit \(3\)](#),
[TypeName \(3\)](#),

NAME

`next`, `nextto` - invoke superclass method implementations

SYNOPSIS

```
package require Tcl00

next ?arg ...?
nextto class ?arg ...?
```

DESCRIPTION

The **next** command is used to call implementations of a method by a class, superclass or mixin that are overridden by the current method. It can only be used from within a method. It is also used within filters to indicate the point where a filter calls the actual implementation (the filter may decide to not go along the chain, and may process the results of going along the chain of methods as it chooses). The result of the **next** command is the result of the next method in the method chain; if there are no further methods in the method chain, the result of **next** will be an error. The arguments, *arg*, to **next** are the arguments to pass to the next method in the chain.

The **nextto** command is the same as the **next** command, except that it takes an additional *class* argument that identifies a class whose implementation of the current method chain (see **info object call**) should be used; the method implementation selected will be the one provided by the given class, and it must refer to an existing non-filter invocation that lies further along the chain than the current implementation.

THE METHOD CHAIN

When a method of an object is invoked, things happen in several stages:

- [1] The structure of the object, its class, superclasses, filters, and mixins, are examined to build a *method chain*, which contains a list of method implementations to invoke.
- [2] The first method implementation on the chain is invoked.
- [3] If that method implementation invokes the **next** command, the next method implementation is invoked (with its arguments being those that were passed to **next**).
- [4] The result from the overall method call is the result from the outermost method implementation; inner method implementations return their results through **next**.
- [5] The method chain is cached for future use.

METHOD SEARCH ORDER

When constructing the method chain, method implementations are searched for in the following order:

- [1] In the classes mixed into the object, in class traversal order. The list of mixins is checked in natural order.
- [2] In the classes mixed into the classes of the object, with sources of mixing in being searched in class traversal order. Within each class, the list of mixins is processed in natural order.
- [3] In the object itself.
- [4] In the object's class.
- [5] In the superclasses of the class, following each superclass in a depth-first fashion in the natural order of the superclass list.

Any particular method implementation always comes as late in the resulting list of implementations as possible; this means that if some class, A, is both mixed into a class, B, and is also a superclass of B, the instances of B will always treat A as a superclass from the perspective of inheritance. This is true even when the multiple inheritance is processed indirectly.

coreTools Command Reference Index

FILTERS

When an object has a list of filter names set upon it, or is an instance of a class (or has mixed in a class) that has a list of filter names set upon it, before every invocation of any method the filters are processed. Filter implementations are found in class traversal order, as are the lists of filter names (each of which is traversed in natural list order). Explicitly invoking a method used as a filter will cause that method to be invoked twice, once as a filter and once as a normal method.

Each filter should decide for itself whether to permit the execution to go forward to the proper implementation of the method (which it does by invoking the **next** command as filters are inserted into the front of the method call chain) and is responsible for returning the result of **next**.

Filters are invoked when processing an invocation of the **unknown** method because of a failure to locate a method implementation, but *not* when invoking either constructors or destructors. (Note however that the **destroy** method is a conventional method, and filters are invoked as normal when it is called.)

EXAMPLES

This example demonstrates how to use the **next** command to call the (super)class's implementation of a method. The script:

```
oo::class create theSuperclass {
  method example {args} {
    puts "in the superclass, args = $args"
  }
}
oo::class create theSubclass {
  superclass theSuperclass
  method example {args} {
    puts "before chaining from subclass, args = $args"
    next a {*} $args b
    next pureSynthesis
    puts "after chaining from subclass"
  }
}
theSubclass create obj
oo::objdefine obj method example args {
  puts "per-object method, args = $args"
  next x {*} $args y
  next
}
obj example 1 2 3
```

prints the following:

```
per-object method, args = 1 2 3
before chaining from subclass, args = x 1 2 3 y
in the superclass, args = a x 1 2 3 y b
```


coreTools Command Reference Index

in the superclass, args = pureSynthesis
after chaining from subclass
before chaining from subclass, args =
in the superclass, args = a b
in the superclass, args = pureSynthesis
after chaining from subclass

This example demonstrates how to build a simple cache class that applies memoization to all the method calls of the objects it is mixed into, and shows how it can make a difference to computation times:

```
oo::class create cache {
  filter Memoize
  method Memoize args {
    # Do not filter the core method implementations
    if {[lindex [self target] 0] eq "::oo::object"} {
      return [next {*}$args]
    }

    # Check if the value is already in the cache
    my variable ValueCache
    set key [self target],$args
    if {[info exist ValueCache($key)]} {
      return $ValueCache($key)
    }

    # Compute value, insert into cache, and return it
    return [set ValueCache($key) [next {*}$args]]
  }
  method flushCache {} {
    my variable ValueCache
    unset ValueCache
    # Skip the caching
    return -level 2 ""
  }
}

oo::object create demo
oo::objdefine demo {
  mixin cache
  method compute {a b c} {
    after 3000 ;/# Simulate deep thought
    return [expr {$a + $b * $c}]
  }
  method compute2 {a b c} {
    after 3000 ;/# Simulate deep thought
    return [expr {$a * $b + $c}]
  }
}

puts [demo compute 1 2 3]      -> prints "7" after delay
puts [demo compute2 4 5 6]    -> prints "26" after delay
puts [demo compute 1 2 3]      -> prints "7" instantly
puts [demo compute2 4 5 6]    -> prints "26" instantly
puts [demo compute 4 5 6]      -> prints "34" after delay
puts [demo compute 4 5 6]      -> prints "34" instantly
puts [demo compute 1 2 3]      -> prints "7" instantly
demo flushCache
puts [demo compute 1 2 3]      -> prints "7" after delay
```

SEE ALSO

`oo::class(n)`, `oo::define(n)`, `oo::object(n)`, `self(n)`

KEYWORDS

`call`, `method`, `method chain`

NAME

`oo::object` - root class of the class hierarchy

SYNOPSIS

```
package require Tcl00
```

```
oo::object method ?arg ...?
```

CLASS HIERARCHY

```
oo::object
```

DESCRIPTION

The **oo::object** class is the root class of the object hierarchy; every object is an instance of this class. Since classes are themselves objects, they are instances of this class too. Objects are always referred to by their name, and may be **renamed** while maintaining their identity.

Instances of objects may be made with either the **create** or **new** methods of the **oo::object** object itself, or by invoking those methods on any of the subclass objects; see **oo::class** for more details. The configuration of individual objects (i.e., instance-specific methods, mixed-in classes, etc.) may be controlled with the **oo::objdefine** command.

Each object has a unique namespace associated with it, the instance namespace. This namespace holds all the instance variables of the object, and will be the

coreTools Command Reference Index

current namespace whenever a method of the object is invoked (including a method of the class of the object). When the object is destroyed, its instance namespace is deleted. The instance namespace contains the object's **my** command, which may be used to invoke non-exported methods of the object or to create a reference to the object for the purpose of invocation which persists across renamings of the object.

CONSTRUCTOR

The **oo::object** class does not define an explicit constructor.

DESTRUCTOR

The **oo::object** class does not define an explicit destructor.

EXPORTED METHODS

The **oo::object** class supports the following exported methods:

obj destroy

This method destroys the object, *obj*, that it is invoked upon, invoking any destructors on the object's class in the process. It is equivalent to using **rename** to delete the object command. The result of this method is always the empty string.

NON-EXPORTED METHODS

The **oo::object** class supports the following non-exported methods:

obj eval ?arg ...?

This method concatenates the arguments, *arg*, as if with **concat**, and then evaluates the resulting script in the namespace that is uniquely associated with *obj*, returning the result of the evaluation.

obj unknown ?methodName? ?arg ...?

This method is called when an attempt to invoke the method *methodName* on object *obj* fails. The arguments that the user supplied to the method are given as *arg* arguments. If *methodName* is absent, the object was invoked with no method name at all (or any other arguments). The default implementation (i.e., the one defined by the **oo::object** class) generates a suitable error, detailing what methods the object supports given whether the object was invoked by its public name or through the **my** command.

obj variable ?varName ...?

This method arranges for each variable called *varName* to be linked from the object *obj*'s unique namespace into the caller's context. Thus, if it is invoked from inside a procedure then the namespace variable in the object is linked to the local variable in the procedure. Each *varName* argument must not have any namespace separators in it. The result is the empty string.

obj varname varName

This method returns the globally qualified name of the

coreTools Command Reference Index

variable *varName* in the unique namespace for the object *obj*.

obj <**cloned**> *sourceObjectName*

This method is used by the **oo::object** command to copy the state of one object to another. It is responsible for copying the procedures and variables of the namespace of the source object (*sourceObjectName*) to the current object. It does not copy any other types of commands or any traces on the variables; that can be added if desired by overriding this method in a subclass.

EXAMPLES

This example demonstrates basic use of an object.

```
set obj [oo::object new]
$obj foo          -> error "unknown method foo"
oo::objdefine $obj method foo {} {
    my variable count
    puts "bar[incr count]"
}
$obj foo          -> prints "bar1"
$obj foo          -> prints "bar2"
$obj variable count -> error "unknown method variable"
$obj destroy
$obj foo          -> error "unknown command obj"
```

SEE ALSO

my(n), oo::class(n)

KEYWORDS

base class, class, object, root class

NAME

`open` - Open a file-based or command pipeline channel

SYNOPSIS

```
open fileName  
open fileName access  
open fileName access permissions
```

DESCRIPTION

This command opens a file, serial port, or command pipeline and returns a channel identifier that may be used in future invocations of commands like **read**, **puts**, and **close**. If the first character of *fileName* is not `|` then the command opens a file: *fileName* gives the name of the file to open, and it must conform to the conventions described in the **filename** manual entry.

The *access* argument, if present, indicates the way in which the file (or command pipeline) is to be accessed. In the first form *access* may have any of the following values:

r	Open the file for reading only; the file must already exist. This is the default value if <i>access</i> is not specified.
r+	Open the file for both reading and writing; the file must already exist.
w	Open the file for writing only. Truncate it if it exists. If it does not exist, create a new file.
w+	Open the file for reading and writing. Truncate it if it exists. If it does

coreTools Command Reference Index

not exist, create a new file.

- a** Open the file for writing only. If the file does not exist, create a new empty file. Set the file pointer to the end of the file prior to each write.
- a+** Open the file for reading and writing. If the file does not exist, create a new empty file. Set the initial access position to the end of the file.

All of the legal access values above may have the character **b** added as the second or third character in the value to indicate that the opened channel should be configured as if with the **fconfigure -translation binary** option, making the channel suitable for reading or writing of binary data.

In the second form, access consists of a list of any of the following flags, all of which have the standard POSIX meanings. One of the flags must be either **RDONLY**, **WRONLY** or **RDWR**.

- RDONLY** Open the file for reading only.
- WRONLY** Open the file for writing only.
- RDWR** Open the file for both reading and writing.
- APPEND** Set the file pointer to the end of the file prior to each write.
- BINARY** Configure the opened channel with the **-translation binary** option.
- CREAT** Create the file if it does not already exist (without this flag it is an error for the file not to exist).
- EXCL** If **CREAT** is also specified, an error is returned if the file already exists.
- NOCTTY** If the file is a terminal device, this flag prevents the file from becoming the controlling terminal of the process.
- NONBLOCK** Prevents the process from blocking while opening the file, and possibly in subsequent I/O operations. The exact behavior of this flag is system- and device-dependent; its use is discouraged (it is better to use the **fconfigure** command to put a file in nonblocking mode). For details refer to your system documentation on the **open** system call's **O_NONBLOCK** flag.
- TRUNC** If the file exists it is truncated to zero length.

coreTools Command Reference Index

If a new file is created as part of opening it, *permissions* (an integer) is used to set the permissions for the new file in conjunction with the process's file mode creation mask. *Permissions* defaults to 0666.

COMMAND PIPELINES

If the first character of *fileName* is then the remaining characters of *fileName* are treated as a list of arguments that describe a command pipeline to invoke, in the same style as the arguments for **exec**. In this case, the channel identifier returned by **open** may be used to write to the command's input pipe or read from its output pipe, depending on the value of *access*. If write-only access is used (e.g. *access* is then standard output for the pipeline is directed to the current standard output unless overridden by the command. If read-only access is used (e.g. *access* is standard input for the pipeline is taken from the current standard input unless overridden by the command. The id of the spawned process is accessible through the **pid** command, using the channel id returned by **open** as argument.

If the command (or one of the commands) executed in the command pipeline returns an error (according to the definition in **exec**), a Tcl error is generated when **close** is called on the channel unless the pipeline is in non-blocking mode then no exit status is returned (a silent **close** with *-blocking 0*).

It is often useful to use the **fileevent** command with pipelines so other processing may happen at the same time as running the command in the background.

SERIAL COMMUNICATIONS

If *fileName* refers to a serial port, then the specified serial port is opened and initialized in a platform-dependent manner. Acceptable values for the *fileName* to use to open a serial port are described in the PORTABILITY ISSUES section.

The **fconfigure** command can be used to query and set additional configuration options specific to serial ports (where supported):

-mode *baud,parity,data,stop*

This option is a set of 4 comma-separated values: the baud rate, parity, number of data bits, and number of stop bits for this serial port. The *baud* rate is a simple integer that specifies the connection speed. *Parity* is one of the following letters: **n**, **o**, **e**, **m**, **s**; respectively signifying the parity options of or *Data* is the number of data bits and should be an integer from 5 to 8, while *stop* is the number of stop bits and

coreTools Command Reference Index

should be the integer 1 or 2.

-handshake *type*

(Windows and Unix). This option is used to setup automatic handshake control. Note that not all handshake types maybe supported by your operating system. The *type* parameter is case-independent.

If *type* is **none** then any handshake is switched off. **rtscts** activates hardware handshake. Hardware handshake signals are described below. For software handshake **xonxoff** the handshake characters can be redefined with **-xchar**. An additional hardware handshake **dtrdsr** is available only under Windows. There is no default handshake configuration, the initial value depends on your operating system settings. The **-handshake** option cannot be queried.

-queue

(Windows and Unix). The **-queue** option can only be queried. It returns a list of two integers representing the current number of bytes in the input and output queue respectively.

-timeout *msec*

(Windows and Unix). This option is used to set the timeout for blocking read operations. It specifies the maximum interval between the reception of two bytes in milliseconds. For Unix systems the granularity is 100 milliseconds. The **-timeout** option does not affect write operations or nonblocking reads. This option cannot be queried.

-ttycontrol *{signal boolean signal boolean ...}*

(Windows and Unix). This option is used to setup the handshake output lines (see below) permanently or to send a BREAK over the serial line. The *signal* names are case-independent. **{RTS 1 DTR 0}** sets the RTS output to high and the DTR output to low. The BREAK condition (see below) is enabled and disabled with **{BREAK 1}** and **{BREAK 0}** respectively. It is not a good idea to change the **RTS** (or **DTR**) signal with active hardware handshake **rtscts** (or **dtrdsr**). The result is unpredictable. The **-ttycontrol** option cannot be queried.

-ttystatus

(Windows and Unix). The **-ttystatus** option can only be queried. It returns the current modem status and handshake input signals (see below). The result is a list of signal,value pairs with a fixed order, e.g. **{CTS 1 DSR 0 RING 1 DCD 0}**. The *signal* names are returned upper case.

-xchar *{xonChar xoffChar}*

(Windows and Unix). This option is used to query or change the software handshake characters. Normally the operating system default should be DC1 (0x11) and DC3 (0x13) representing the ASCII standard XON and XOFF characters.

-pollinterval *msec*

coreTools Command Reference Index

(Windows only). This option is used to set the maximum time between polling for fileevents. This affects the time interval between checking for events throughout the Tcl interpreter (the smallest value always wins). Use this option only if you want to poll the serial port more or less often than 10 msec (the default).

-sysbuffer *inSize*

-sysbuffer {*inSize outSize*}

(Windows only). This option is used to change the size of Windows system buffers for a serial channel. Especially at higher communication rates the default input buffer size of 4096 bytes can overrun for latent systems. The first form specifies the input buffer size, in the second form both input and output buffers are defined.

-lasterror

(Windows only). This option is query only. In case of a serial communication error, **read** or **puts** returns a general Tcl file I/O error. **fconfigure -lasterror** can be called to get a list of error details. See below for an explanation of the various error codes.

SERIAL PORT SIGNALS

RS-232 is the most commonly used standard electrical interface for serial communications. A negative voltage (-3V..-12V) define a mark (on=1) bit and a positive voltage (+3..+12V) define a space (off=0) bit (RS-232C). The following signals are specified for incoming and outgoing data, status lines and handshaking. Here we are using the terms *workstation* for your computer and *modem* for the external device, because some signal names (DCD, RI) come from modems. Of course your external device may use these signal lines for other purposes.

TXD (output)	Transmitted Data: Outgoing serial data.
RXD (input)	Received Data: Incoming serial data.
RTS (output)	Request To Send: This hardware handshake line informs the modem that your workstation is ready to receive data. Your workstation may automatically reset this signal to indicate that the input buffer is full.
CTS (input)	Clear To Send: The complement to RTS. Indicates that the modem is ready to receive data.
DTR (output)	Data Terminal Ready: This signal tells the modem that the workstation is ready to establish a link. DTR is often enabled automatically whenever a serial port is opened.

coreTools Command Reference Index

DSR(input)	Data Set Ready: The complement to DTR. Tells the workstation that the modem is ready to establish a link.
DCD(input)	Data Carrier Detect: This line becomes active when a modem detects a signal.
RI(input)	Ring Indicator: Goes active when the modem detects an incoming call.
BREAK	A BREAK condition is not a hardware signal line, but a logical zero on the TXD or RXD lines for a long period of time, usually 250 to 500 milliseconds. Normally a receive or transmit data signal stays at the mark (on=1) voltage until the next character is transferred. A BREAK is sometimes used to reset the communications line or change the operating mode of communications hardware.

ERROR CODES (Windows only)

A lot of different errors may occur during serial read operations or during event polling in background. The external device may have been switched off, the data lines may be noisy, system buffers may overrun or your mode settings may be wrong. That is why a reliable software should always **catch** serial read operations. In cases of an error Tcl returns a general file I/O error. Then **fconfigure -lasterror** may help to locate the problem. The following error codes may be returned.

RXOVER	Windows input buffer overrun. The data comes faster than your scripts reads it or your system is overloaded. Use fconfigure -sysbuffer to avoid a temporary bottleneck and/or make your script faster.
TXFULL	Windows output buffer overrun. Complement to RXOVER. This error should practically not happen, because Tcl cares about the output buffer status.
OVERRUN	UART buffer overrun (hardware) with data lost. The data comes faster than the system driver receives it. Check your advanced serial port settings to enable the FIFO (16550) buffer and/or setup a lower(1) interrupt threshold value.
RXPARITY	A parity error has been detected by your UART. Wrong parity settings with fconfigure -mode or a noisy data line (RXD) may cause this error.

coreTools Command Reference Index

- FRAME** A stop-bit error has been detected by your UART. Wrong mode settings with **fconfigure -mode** or a noisy data line (RXD) may cause this error.
- BREAK** A BREAK condition has been detected by your UART (see above).

PORTABILITY ISSUES

Windows

Valid values for *fileName* to open a serial port are of the form **comX**, where **X** is a number, generally from 1 to 9. A legacy form accepted as well is **comX:**. This notation only works for serial ports from 1 to 9. An attempt to open a serial port that does not exist or has a number greater than 9 will fail. An alternate form of opening serial ports is to use the filename **\\./comX**, where **X** is any number that corresponds to a serial port.

When running Tcl interactively, there may be some strange interactions between the real console, if one is present, and a command pipeline that uses standard input or output. If a command pipeline is opened for reading, some of the lines entered at the console will be sent to the command pipeline and some will be sent to the Tcl evaluator. If a command pipeline is opened for writing, keystrokes entered into the console are not visible until the pipe is closed. These problems only occur because both Tcl and the child application are competing for the console at the same time. If the command pipeline is started from a script, so that Tcl is not accessing the console, or if the command pipeline does not use standard input or output, but is redirected from or to a file, then the above problems do not occur.

Unix

Valid values for *fileName* to open a serial port are generally of the form **/dev/ttyX**, where **X** is **a** or **b**, but the name of any pseudo-file that maps to a serial port may be used. Advanced configuration options are only supported for serial ports when Tcl is built to use the POSIX serial interface.

When running Tcl interactively, there may be some strange interactions between the console, if one is present, and a command pipeline that uses standard input. If a command pipeline is opened for reading, some of the lines entered at the console will be sent to the command pipeline and some will be sent to the Tcl evaluator. This problem only occurs because both Tcl and the child application are competing for the console at the same time. If the command pipeline is started from a script, so that Tcl is not accessing the console, or if the command pipeline does not use standard input, but is redirected from a file, then the above problem does not occur.

coreTools Command Reference Index

See the **PORTABILITY ISSUES** section of the **exec** command for additional information not specific to command pipelines about executing applications on the various platforms

EXAMPLE

Open a command pipeline and catch any errors:

```
set fl [open "| ls this_file_does_not_exist"]
set data [read $fl]
if {[catch {close $fl} err]} {
    puts "ls command failed: $err"
}
```

SEE ALSO

file(n), close(n), filename(n), fconfigure(n), gets(n), read(n), puts(n), exec(n), pid(n), fopen(3)

KEYWORDS

access mode, append, create, file, non-blocking, open, permissions, pipeline, process, serial

open_workspace
Open the workspace.

Syntax

```
string open_workspace [-gui] [ws_dir]
      string ws_dir
```

Parameters

	Prompt before creating workspace.
-gui	When you invoke open_workspace from the coreTool console, the -gui option causes the coreTool to display the "Open Workspace" dialog where you can specify the root directory of the workspace to be read. If you specify the -gui option when using the coreTool in shell mode, open_workspace does not post the the "Open Workspace" dialog.
ws_dir	The root directory of the workspace to be read The workspace Knowledge database (KB) name will be the same as the tail of the root directory name. ws_dir is default to the current directory.

Description

The open_workspace command opens an existing workspace by specifying its root directory and loads the required knowledge databases for the workspace.

After the workspace is opened, the current knowledge database is set to the workspace knowledge database, use get_workspace_kb and get_workspace_name to get the current workspace.

In the coreTool GUI, you can open a workspace from the menu File->Open.

Examples

The following command opens a coreConsultant workspace from the root directory /usr/fred/workspaces/consultant/config1:

```
coreConsultant> open_workspace /usr/fred/workspaces/consultant/config1
```

The following command opens a coreBuilder workspace from the root directory /usr/fred/workspaces/builder/Builder1, and invokes the "Open Workspace" dialog:

```
coreConsultant> open_workspace -gui /usr/fred/workspaces/builder/Builder1
```

See Also

[create_workspace \(2\)](#), [save_workspace \(2\)](#), [close_workspace \(2\)](#),
[add_workspace_hook \(2\)](#), [get_workspace_kb \(2\)](#), [get_workspace_name \(2\)](#)

NAME

`package` - Facilities for package loading and version control

SYNOPSIS

```
package forget ?package package ...?  
package ifneeded package version ?script?  
package names  
package present package ?requirement...?  
package present -exact package version  
package provide package ?version?  
package require package ?requirement...?  
package require -exact package version  
package unknown ?command?  
package vcompare version1 version2  
package versions package  
package vsatisfies version requirement...  
package prefer ?latest|stable?
```

DESCRIPTION

This command keeps a simple database of the packages available for use by the current interpreter and how to load them into the interpreter. It supports multiple versions of each package and arranges for the correct version of a package to be loaded based on what is needed by the application. This command also detects and reports version clashes. Typically, only the **package require** and **package provide** commands are invoked in normal Tcl scripts; the other commands are used primarily by system scripts that maintain the package database.

The behavior of the **package** command is determined by its first argument. The following forms are permitted:

coreTools Command Reference Index

package forget *?package package ...?*

Removes all information about each specified package from this interpreter, including information provided by both **package ifneeded** and **package provide**.

package ifneeded *package version ?script?*

This command typically appears only in system configuration scripts to set up the package database. It indicates that a particular version of a particular package is available if needed, and that the package can be added to the interpreter by executing *script*. The script is saved in a database for use by subsequent **package require** commands; typically, *script* sets up auto-loading for the commands in the package (or calls **load** and/or **source** directly), then invokes **package provide** to indicate that the package is present. There may be information in the database for several different versions of a single package. If the database already contains information for *package* and *version*, the new *script* replaces the existing one. If the *script* argument is omitted, the current script for version *version* of package *package* is returned, or an empty string if no **package ifneeded** command has been invoked for this *package* and *version*.

package names

Returns a list of the names of all packages in the interpreter for which a version has been provided (via **package provide**) or for which a **package ifneeded** script is available. The order of elements in the list is arbitrary.

package present *?-exact? package ?requirement...?*

This command is equivalent to **package require** except that it does not try and load the package if it is not already loaded.

package provide *package ?version?*

This command is invoked to indicate that version *version* of package *package* is now present in the interpreter. It is typically invoked once as part of an **ifneeded** script, and again by the package itself when it is finally loaded. An error occurs if a different version of *package* has been provided by a previous **package provide** command. If the *version* argument is omitted, then the command returns the version number that is currently provided, or an empty string if no **package provide** command has been invoked for *package* in this interpreter.

package require *package ?requirement...?*

This command is typically invoked by Tcl code that wishes to use a particular version of a particular package. The arguments indicate which package is wanted, and the command ensures that a suitable version of the package is loaded into the interpreter. If the command succeeds, it returns the version number that is loaded; otherwise it generates an error.

A suitable version of the package is any version which satisfies at least one of the requirements, per the rules of **package vsatisfies**. If multiple versions are

coreTools Command Reference Index

suitable the implementation with the highest version is chosen. This last part is additionally influenced by the selection mode set with **package prefer**.

In the selection mode the command will select the highest stable version satisfying the requirements, if any. If no stable version satisfies the requirements, the highest unstable version satisfying the requirements will be selected. In the selection mode the command will accept the highest version satisfying all the requirements, regardless of its stableness.

If a version of *package* has already been provided (by invoking the **package provide** command), then its version number must satisfy the *requirements* and the command returns immediately. Otherwise, the command searches the database of information provided by previous **package ifneeded** commands to see if an acceptable version of the package is available. If so, the script for the highest acceptable version number is evaluated in the global namespace; it must do whatever is necessary to load the package, including calling **package provide** for the package. If the **package ifneeded** database does not contain an acceptable version of the package and a **package unknown** command has been specified for the interpreter then that command is evaluated in the global namespace; when it completes, Tcl checks again to see if the package is now provided or if there is a **package ifneeded** script for it. If all of these steps fail to provide an acceptable version of the package, then the command returns an error.

package require -exact *package version*

This form of the command is used when only the given *version* of *package* is acceptable to the caller. This command is equivalent to **package require package version-version**.

package unknown *?command?*

This command supplies a command to invoke during **package require** if no suitable version of a package can be found in the **package ifneeded** database. If the *command* argument is supplied, it contains the first part of a command; when the command is invoked during a **package require** command, Tcl appends one or more additional arguments giving the desired package name and requirements. For example, if *command* is **foo bar** and later the command **package require test 2.4** is invoked, then Tcl will execute the command **foo bar test 2.4** to load the package. If no requirements are supplied to the **package require** command, then only the name will be added to invoked command. If the **package unknown** command is invoked without a *command* argument, then the current **package unknown** script is returned, or an empty string if there is none. If *command* is specified as an empty string, then the current **package unknown** script is removed, if there is one.

package vcompare *version1 version2*

Compares the two version numbers given by *version1* and *version2*. Returns -1 if *version1* is an earlier version

coreTools Command Reference Index

than *version2*, 0 if they are equal, and 1 if *version1* is later than *version2*.

package versions *package*

Returns a list of all the version numbers of *package* for which information has been provided by **package ifneeded** commands.

package vsatisfies *version requirement...*

Returns 1 if the *version* satisfies at least one of the given requirements, and 0 otherwise. Each *requirement* is allowed to have any of the forms:

min

This form is called

min-

This form is called

min-max

This form is called

where *and* are valid version numbers. The legacy syntax is a special case of the extended syntax, keeping backward compatibility. Regarding satisfaction the rules are:

- [1] The *version* has to pass at least one of the listed *requirements* to be satisfactory.
- [2] A version satisfies a requirement when
 - [a] For *min* equal to the *max* if, and only if the *version* is equal to the *min*.
 - [b] Otherwise if, and only if the *version* is greater than or equal to the *min*, and less than the *max*, where both *min* and *max* have been padded internally with Note that while the comparison to *min* is inclusive, the comparison to *max* is exclusive.
- [3] A requirement is a requirement in disguise, with the *max* part implicitly specified as the next higher major version number of the *min* part. A version satisfies it per the rules above.
- [4] A *version* satisfies a requirement if, and only if it is greater than or equal to the *min*, where the *min* has been padded internally with There is no constraint to a maximum.

package prefer ?latest|stable?

With no arguments, the command returns either or whichever describes the current mode of selection logic used by **package require**.

When passed the argument it sets the selection logic mode to

When passed the argument if the mode is already that value is kept. If the mode is already then the attempt to set it back to is ineffective and the mode value remains

When passed any other value as an argument, raise an invalid argument error.

When an interpreter is created, its initial selection mode value is set to unless the environment variable **TCL_PKG_PREFER_LATEST** is set. If that environment variable is defined (with any value) then the initial (and permanent) selection mode value is set to

VERSION NUMBERS

Version numbers consist of one or more decimal numbers separated by dots, such as 2 or 1.162 or 3.1.13.1. The first number is called the major version number. Larger numbers correspond to later versions of a package, with leftmost numbers having greater significance. For example, version 2.1 is later than 1.3 and version 3.4.6 is later than 3.3.5. Missing fields are equivalent to zeroes: version 1.3 is the same as version 1.3.0 and 1.3.0.0, so it is earlier than 1.3.1 or 1.3.0.2. In addition, the letters (alpha) and/or (beta) may appear exactly once to replace a dot for separation. These letters semantically add a negative specifier into the version, where is -2, and is -1. Each may be specified only once, and or are mutually exclusive in a specifier. Thus 1.3a1 becomes (semantically) 1.3.-2.1, 1.3b1 is 1.3.-1.1. Negative numbers are not directly allowed in version specifiers. A version number not containing the letters or as specified above is called a **stable** version, whereas presence of the letters causes the version to be called is **unstable**. A later version number is assumed to be upwards compatible with an earlier version number as long as both versions have the same major version number. For example, Tcl scripts written for version 2.3 of a package should work unchanged under versions 2.3.2, 2.4, and 2.5.1. Changes in the major version number signify incompatible changes: if code is written to use version 2.1 of a package, it is not guaranteed to work unmodified with either version 1.7.3 or version 3.1.

PACKAGE INDICES

The recommended way to use packages in Tcl is to invoke **package require** and **package provide** commands in scripts, and use the procedure **pkg_mkIndex** to create package index files. Once you have done this, packages will be loaded automatically in response to **package require** commands. See the documentation for **pkg_mkIndex** for details.

EXAMPLES

To state that a Tcl script requires the Tk and http packages, put this at the top of the script:

```
package require Tk  
package require http
```

To test to see if the Snack package is available and load it if it is (often useful for optional enhancements to programs where the loss of the functionality is not critical) do this:

```
if {[catch {package require Snack}]} {  
    # Error thrown - package not found.  
    # Set up a dummy interface to work around the absence  
} else {  
    # We have the package, configure the app to use it  
}
```

SEE ALSO

msgcat(n), packagens(n), pkgMkIndex(n)

KEYWORDS

package, version

packaged_prime_profile_report

Generate an HTML report for Prime Profiles

Syntax

```
string packaged_prime_profile_report -file <File Name>  
      string <File Name>
```

Parameters

-file <File Name>	Name of the HTML packaged prime profile report file
-------------------	---

Description

This is a coreBuilder command that generates an html report that contains detailed information of the prime profiles created for a design. The user must specify the report html file name using the file option. When completing the BuildcoreKit activity, if the activity parameter 'GeneratePrimeProfileReport' is enabled, the packaged prime profile HTML report is generated in the report folder (<workspace_name>/report/PackagedPrimeProfiles.html)

Examples

To generate Prime Profile report as PrimeProfiles.html use the command:

```
packaged_prime_profile_report -file PrimeProfiles.html
```

See Also

NAME

`pkg::create` - Construct an appropriate 'package
ifneeded' command for a given package specification

SYNOPSIS

```
::pkg::create -name packageName -version packageVersion  
?-load filespec? ... ?-source filespec? ...
```

DESCRIPTION

`::pkg::create` is a utility procedure that is part of the standard Tcl library. It is used to create an appropriate **package ifneeded** command for a given package specification. It can be used to construct a **pkgIndex.tcl** file for use with the **package** mechanism.

OPTIONS

The parameters supported are:

-name *packageName*

This parameter specifies the name of the package. It is required.

-version *packageVersion*

This parameter specifies the version of the package. It is required.

-load *filespec*

This parameter specifies a binary library that must be

coreTools Command Reference Index

loaded with the **load** command. *filespec* is a list with two elements. The first element is the name of the file to load. The second, optional element is a list of commands supplied by loading that file. If the list of procedures is empty or omitted, **::pkg::create** will set up the library for direct loading (see **pkg_mkIndex**). Any number of **-load** parameters may be specified.

-source *filespec*

This parameter is similar to the **-load** parameter, except that it specifies a Tcl library that must be loaded with the **source** command. Any number of **-source** parameters may be specified.

At least one **-load** or **-source** parameter must be given.

SEE ALSO

package(n)

KEYWORDS

auto-load, index, package, version

[API Index](#) [Command Reference](#)

Related Pages

Here is a list of all related documentation pages:

[coreAssembler](#)
[coreBuilder](#)
[coreConsultant](#)
[add activity hook](#)
[add files to filegroup](#)
[add hdl pragma](#)
[add help menu item](#)
[add html report link](#)
[add instantiate component hook](#)
[add to collection](#)
[add verify subsystem hook](#)
[add workspace hook](#)
[all components](#)
[all designs](#)
[all inputs](#)
[all outputs](#)
[all subdesigns](#)
[analyze filegroup](#)
[append to collection](#)
[as collection](#)
[attach interface](#)
[autocomplete activity](#)
[batch install](#)
[bind tech cell mapping](#)
[build debug tarfile](#)
[byte compile file](#)
[check bom](#)
[check component for parameter loops](#)
[check env vars](#)
[check executables](#)
[check file for errors](#)
[check license](#)
[check parameter attribute formulas](#)
[check ParameterInfo](#)
[check workspace project id](#)
[clear prime profile validation exprs](#)
[clone component](#)
[close workspace](#)

[CombineInheritValues](#)
[combineValues](#)
[compare dc version](#)
[complete custom activity definition](#)
[complete interface definition](#)
[compress sdc](#)
[connect interface](#)
[connect or export unconnected pins](#)
[copy configuration](#)
[copy target interface parameters](#)
[create address bank](#)
[create address block](#)
[create address space](#)
[create autoload filegroup](#)
[create component view](#)
[create configuration parameter](#)
[create connection](#)
[create custom activity](#)
[create custom activity parameter](#)
[create custom tech cell](#)
[create generated clock](#)
[create hierarchical component](#)
[create interface](#)
[create interface instance](#)
[create interface parameter](#)
[create interface port](#)
[create memory map](#)
[create memory mode](#)
[create plugin kb](#)
[create prime profile file](#)
[create reference clock](#)
[create register](#)
[create register array](#)
[create register field](#)
[create register field value](#)
[create subsystem parameter](#)
[create virtual clock](#)
[create workspace](#)
[current activity](#)
[current design](#)
[current kb](#)
[define activity subproc params](#)

[define array field parameters](#)
[define lib clocks](#)
[define report](#)
[define solution](#)
[define split interface](#)
[define tech cell mapping](#)
[detach interface](#)
[disable prime profile](#)
[docbook to html](#)
[duplicate component](#)
[duplicate workspace](#)
[enable prime profile](#)
[encrypt solution](#)
[escaped name](#)
[eval in component](#)
[eval ipxact expr](#)
[eval param](#)
[explicit capacitance](#)
[export all interfaces](#)
[export interface](#)
[export open pins](#)
[export pin](#)
[export port descriptions](#)
[export unconnected interfaces](#)
[export unconnected pins](#)
[export workspace as component](#)
[find design](#)
[find interface instances](#)
[find item](#)
[find kb](#)
[foreach array field parameter](#)
[foreach in collection](#)
[generate gtech sim model](#)
[generate qtm](#)
[generate reports](#)
[generate simulation file list](#)
[generate simulation launch command](#)
[generate views](#)
[get activity parameter](#)
[get address bank attribute](#)
[get address block attribute](#)
[get all bits](#)

[get associated instance parameter](#)
[get attribute](#)
[get bit driver](#)
[get bit loads](#)
[get bitfield value](#)
[get cell](#)
[get cell attribute](#)
[get children](#)
[get clock attribute](#)
[get clocks](#)
[get component design](#)
[get component name](#)
[get component view](#)
[get configuration parameter](#)
[get configuration parameter attribute](#)
[get connections](#)
[get current component](#)
[get design attribute](#)
[get design prefix](#)
[get field parameter for array](#)
[get file prefix](#)
[get filegroup attribute](#)
[get filegroup parameter](#)
[get generator parameter](#)
[get hdl file list](#)
[get hdl language](#)
[get hdl library names](#)
[get hdl pragma](#)
[get hierarchies](#)
[get include dirs](#)
[get installation dir](#)
[get installed component names](#)
[get instance parameter value](#)
[get interface attribute](#)
[get interface link](#)
[get interface parameter](#)
[get interface parameter attribute](#)
[get interface port attribute](#)
[get launch activities status](#)
[get logical dir](#)
[get memory map attribute](#)
[get memory mode](#)

get object name
get parameter attribute
get port attribute
get power domain voltage
get prime profile info
get register array attribute
get register attribute
get register field attribute
get register field value attribute
get slave base address
get solution data
get solution file
get solution root
get solution variable
get strategy parameter
get supply voltage
get sv expr
get synthesis dir
get tcl expr
get template naming style
get template parameter style
get template separator style
get TMR separation distance
get tool preference
get tool root
get top design
get top design name
get upf attribute
get upf port names
get value from interface
get VLNV search path entries
get workspace kb
get workspace license info
get workspace name
group components
gui get setting
gui set setting
gui source
gui start
gui stop
import component
import ipxact data

[import workspace as component](#)
[in autocomplete activity](#)
[IncludeIf](#)
[index collection](#)
[infinite drive](#)
[InheritValue](#)
[install filegroup](#)
[instances of](#)
[instantiate component](#)
[instantiate dut](#)
[instantiate interface](#)
[instantiate solution](#)
[invoke generator](#)
[invoke ralgen](#)
[is prime profile enabled](#)
[launch activities on grid](#)
[launch activity subproc](#)
[list kb](#)
[load autoload filegroup](#)
[load component memory maps](#)
[load file into kb](#)
[load interface definitions](#)
[load plugin](#)
[load solution definition](#)
[lock parameter](#)
[merge ports](#)
[move components](#)
[msg box](#)
[open workspace](#)
[packaged prime profile report](#)
[percent of period](#)
[permanent proc](#)
[pin cap of](#)
[plugin proc](#)
[prefix defines](#)
[prefix identifiers](#)
[prefix macros](#)
[prereq activities complete](#)
[process solution files](#)
[propagate memory map](#)
[protected proc](#)
[rand proc](#)

[randomize parameters](#)
[read attribute table](#)
[read component constraints](#)
[read ipxact file](#)
[read parameter info table](#)
[read parameter table](#)
[read pin connection table](#)
[read sdc](#)
[read subsystem table](#)
[register netlist](#)
[reinitialize component](#)
[reinitialize workspace](#)
[remove address bank](#)
[remove address block](#)
[remove address space](#)
[remove allow paths](#)
[remove area estimates](#)
[remove attached interface](#)
[remove component](#)
[remove component view](#)
[remove connection](#)
[remove constraints](#)
[remove exported interface](#)
[remove from collection](#)
[remove generated clock](#)
[remove html report link](#)
[remove interface](#)
[remove interface instance](#)
[remove ipxact file](#)
[remove memory map](#)
[remove register](#)
[remove register array](#)
[remove register field](#)
[remove register field value](#)
[remove virtual clock](#)
[rename component](#)
[rename exported interface](#)
[rename port](#)
[replace component](#)
[replace interface](#)
[ReplaceConstantParam](#)
[ReplaceDesignParams](#)

ReplaceFormatParam
ReplaceSingleBitBus
replay connections
report activities
report activity parameters
report activity subproc
report attribute
report bom
report connections
report filegroup
report interface instances
report interfaces
report ip
report license
report memory maps
report prime profile validation expr
report prime profiles
report solutions
report suppressed ral tests
report tech cell mapping
report timing exceptions
reset path
restructure workspace
reuse-pragma
save workspace
scale to current units
select by class
select by function
select by name
set activity incomplete
set activity parameter
set address bank attribute
set address block attribute
set area estimate
set attribute
set cell attribute
set clock attribute
set component view
set configuration parameter
set configuration parameter attribute
set current component
set default prime profiles

[set dependant controls](#)
[set design attribute](#)
[set design prefix](#)
[set disable timing](#)
[set editing mode](#)
[set false path](#)
[set file prefix](#)
[set filegroup attribute](#)
[set filegroup parameter](#)
[set generator parameter](#)
[set hdl file list](#)
[set instance parameter](#)
[set instance wire names](#)
[set interface attribute](#)
[set interface parameter](#)
[set interface parameter attribute](#)
[set interface port attribute](#)
[set macro prefix](#)
[set max delay](#)
[set memory map attribute](#)
[set memory mode](#)
[set min delay](#)
[set multicycle path](#)
[set parameter attribute](#)
[set port attribute](#)
[set prime profile](#)
[set prime profile validation expr](#)
[set register array attribute](#)
[set register attribute](#)
[set register field attribute](#)
[set register field value attribute](#)
[set slave base address](#)
[set solution preference](#)
[set strategy parameter](#)
[set synthesis dir](#)
[set tclconfig variables](#)
[set template naming style](#)
[set template parameter style](#)
[set template separator style](#)
[set tool preference](#)
[set tool root](#)
[set unused interface](#)

[set upf attribute](#)
[set upf cells](#)
[set upf supply voltage](#)
[set upf voltage](#)
[set workspace options](#)
[show color dialog](#)
[show exit dialog](#)
[show prime profile prompt](#)
[show spreadsheet for connections](#)
[show url](#)
[show url external](#)
[show user parameter dialog](#)
[sizeof collection](#)
[source solution script](#)
[Synthesis API](#)
[translate interface connection](#)
[translate netlist](#)
[translate phy constraints](#)
[traverse boundary](#)
[traverse connection](#)
[unconnect interface](#)
[ungroup component](#)
[ungroup filegroup](#)
[union collection](#)
[unload autoload filegroup](#)
[unload component memory maps](#)
[unload file from kb](#)
[unload plugin](#)
[unused interface instance](#)
[update dut](#)
[update workspace links](#)
[verify dwf](#)
[verify environment](#)
[verify tool](#)
[visible fields](#)
[wait activities on grid](#)
[wait for activity subproc](#)
[write attribute table](#)
[write automotive synthesis files](#)
[write batch script](#)
[write component register headers](#)
[write config tar file](#)

[write failure modes file](#)
[write ipxact abstractor](#)
[write ipxact component](#)
[write ipxact design](#)
[write ipxact designConfiguration](#)
[write ipxact typeDefinitions](#)
[write ParameterInfo](#)
[write pin connection table](#)
[write ral file](#)
[write sdc](#)
[write subsystem table](#)
[xml translate](#)
[xml verify](#)
[Abstraction](#)
[Activity](#)
[ActivityGroup](#)
[AddLockUpLatch](#)
[addressBank](#)
[addressBlock](#)
[AddressOffset](#)
[AddressSpaceRef](#)
[AllowOpenProvider](#)
[AltValue](#)
[AreaEstimate](#)
[AreaWeight](#)
[ArrayEnd](#)
[ArrayFieldSize](#)
[ArrayStart](#)
[AssociationComplete](#)
[AssociationFormula](#)
[AtpgTclAuxScript](#)
[AtpgTclAuxScriptComment](#)
[attrDefn](#)
[AutoConnectComplete](#)
[AutoConnectIfUnconnected](#)
[AutoConnectionPreCallback](#)
[AutoConnectWhen](#)
[AutoFixAsync](#)
[AutoFixAsyncLogicGate](#)
[AutoFixAsyncTestData](#)
[AutoFixAsyncWithScanEnable](#)
[AutoFixBidirectional](#)

[AutoFixClocks](#)
[AutoFixClockTestData](#)
[AutoloadFilegroup](#)
[BalanceBistSegments](#)
[BankAlignment](#)
[BaseAddress](#)
[BidirectionalMode](#)
[BistAutoFixBusses](#)
[BistAutoFixXprop](#)
[BistBlockIndividually](#)
[BistChainCount](#)
[BistCodecCount](#)
[BistDiagOutputs](#)
[BistInvertPrpgClock](#)
[BistMaxChainLength](#)
[BistMode](#)
[BistObserveOutputs](#)
[BistPrpgLength](#)
[BistPrpgShadowSi](#)
[BistSelectorShadowSi](#)
[BistSubblocksIndividually](#)
[BistType](#)
[BistUseTristateMux](#)
[BitAddressOffset](#)
[BitRange](#)
[BitsInLAU](#)
[BitsToRepresent](#)
[BitWidth](#)
[Bridge](#)
[BusAlignment](#)
[busBit](#)
[cA supports hierarchy](#)
[CanUngroup](#)
[CanUngroupInfo](#)
[Capacitance](#)
[CaseAnalysisValue](#)
[cell](#)
[Channel](#)
[CHeaderValue](#)
[check_error_list](#)
[CheckCmd](#)
[CheckExpr](#)

[CheckExprMessage](#)
[CheckExprWhenDisabled](#)
[clock](#)
[ClockFallLatency](#)
[ClockGatingFallHoldCheck](#)
[ClockGatingFallSetupCheck](#)
[ClockGatingRiseHoldCheck](#)
[ClockGatingRiseSetupCheck](#)
[ClockGatingSignals](#)
[ClockHoldUncertainty](#)
[ClockMixing](#)
[ClockName](#)
[ClockRiseLatency](#)
[ClockSetupUncertainty](#)
[ClockSourceFallLatency](#)
[ClockSourceRiseLatency](#)
[CnctClass](#)
[collection print item limit](#)
[ComponentOfItem](#)
[ConfigActivity](#)
[ConfigDependsOnActivities](#)
[ConfigDependsOnGroup](#)
[ConfigIntentSearchPath](#)
[Configurable](#)
[ConnectionDialogCmd](#)
[ConnectionRequired](#)
[ConnectToExportedInstance](#)
[ConstantPort](#)
[ControlPointsPerScanCell](#)
[ConvertSingleBitBus](#)
[CopyToTemplate](#)
[coreBuilderBomTemplateVersion](#)
[coretools home page](#)
[CreatedViaUngrouping](#)
[CreateSkewGroup](#)
[CriticalRange](#)
[CriticalRangeCoveringViolators](#)
[CriticalTiming](#)
[crm file patterns](#)
[CTLModels](#)
[CustomizedTestbenchCode](#)
[CycleTime](#)

dc_shellStopMessageIds
dc_shellVariable
dc_shellVariableComment
DCLSPaths
DedicatedScanPorts
DedicatedWrapperCell
DefaultConstantPort
DefaultHdlValue
DefaultInstallDir
DefaultLoadPath
DefaultParamRange
DefaultValue
DefaultValueMessage
DeferEvaluation
DefineLibClocks
DeliverableType
Description
design
DftExistingSignalActiveState
DftExistingSignalTestMode
DftExistingSignalTiming
DftExistingSignalType
DftExistingSignalUsage
DftSpecSignalActiveState
DftSpecSignalTestMode
DftSpecSignalType
DftSpecSignalUsage
DigitsPrecision
DisableAggregation
DocAddressOffset
DocBlockTableAddressOffset
DocDefaultValue
DocDescription
DocDescriptionField
DocEnabled
DocGenerateIf
DocGroup
DocGroupName
DocInclude
DocLabelName
DocMaster
DocMemoryAccess

DocOffset
DocPortWidth
DocPowerDomain
DocRangeDecoratedName
DocRegistered
DocRegisteredBy
DocRegisterResetMask
DocRegisterResetValue
DocRegisterSize
DocRegTableAddressOffset
DocShortDescription
DocSynchronousTo
DocTestable
DocVisible
DocVolatileMemory
DocWaiver
DontTouchNetwork
Drive
DrivingCell
Enabled
EnableDftAutoFix
EnableDftShadowLogic
EnableScanCompression
EncryptText
EndBit
EndBitExpr
EndBitsExpr
Endian
EnumValues
EnvCheck
ExcludeLibCells
ExcludeLoadPatterns
Exists
ExportDefineToFile
ExternalTristates
ExtraSearchPathAppend
ExtraSearchPathPrepend
fc_shellAppOption
fc_shellAppOptionComment
FcAuxSynthesisScript
FcAuxSynthesisScriptComment
FCExcludeLibCells

FCIncludeLibCells
FeedThroughConnect
file
FileContentType
filegroup
filegroupGroup
FilePerms
FixHold
fm_shellVariable
fm_shellVariableComment
ForceTCBToUseComponentUndefFile
FormalVerificationAuxScript
FormalVerificationAuxScriptComment
FpgaPadType
FpgaPortIsPad
FpgaPreferTmg
FunctionDefinition
GenerateIf
GlobalSlotNumber
GroupImage
GroupImageAttrs
GroupImageTitle
GroupName
GroupText
GroupUserLabel
guiBehavior
HasLibertyModel
hdlFunc
hdlin enable upf compatible naming
HdlLibrary
HdlSourceFileNames
HdlTypeName
HdlUnquoted
HdlValue
HeaderNameFormat
HelpUrl
HierarchicalIsolation
HighFanout
IconPath
IdealPort
IdentifyShiftRegisters
IfUnconnected

Importance
ImportedWorkspaceComponentFile
IncludeLibCells
InputDelay
InputResistance
InsertEndOfChainLockupLatch
InsertTechCellBindMacroAtStart
InsertTestPoints
Install
InstallUserWorkDir
InstallWhen
InstanceClockNames
IntentFilesProcessed
InterClockHoldFallFallUncertainty
InterClockHoldFallRiseUncertainty
InterClockHoldRiseFallUncertainty
InterClockHoldRiseRiseUncertainty
InterClockSetupFallFallUncertainty
InterClockSetupFallRiseUncertainty
InterClockSetupRiseFallUncertainty
InterClockSetupRiseRiseUncertainty
interfaceDefn
InterfaceGroup
interfaceInstance
InterfaceIsUsed
InterfaceLabel
InterfaceLink
interfacePort
InterfaceSize
InterfaceType
InterfaceTypeName
InternalTristates
intfPin
intfPort
IPEnvList
IsAddressable
IsArray
IsBehavioral
IsCCDTarget
IsComplete
IsConnected
IsControlValue

IsDanglingIntfAllowed
IsEnabled
IsExternalReset
IsFullyModeled
IsHexVal
IsTriState
IsUpToDate
IsWorkspaceComponent
KbVersion
knowledgeBase
Label
LicenseForOpen
LockedInTemplate
LogicalLeft
LogicalName
LogicalRight
MacroPrefixFileList
MapBlockIndividually
MapSubblocksIndividually
MaxArea
MaxCap
MaxControlPoints
MaxFallInputDelay
MaxFallInputDelayFallingEdge
MaxFallOutputDelay
MaxFallOutputDelayFallingEdge
MaxFallTransitionDelay
MaxFanout
maximum bit blast size
MaximumScanChainLength
MaxInputDelay
MaxInputDelayFallingEdge
MaxLoads
MaxObservePoints
MaxOutputDelay
MaxOutputDelayFallingEdge
MaxRiseInputDelay
MaxRiseInputDelayFallingEdge
MaxRiseOutputDelay
MaxRiseOutputDelayFallingEdge
MaxRiseTransitionDelay
MaxTransition

MaxValue
MaxWriteConstraint
memMap
MemoryAccess
MemoryAccessDescription
MemoryMap
MemoryMode
MemoryRange
MemoryUsage
MemoryWidth
MergeCandidate
MessageLimitDC
MessageLimitFC
MessageStopOnDC
MessageStopOnFC
MinCap
MinFallInputDelay
MinFallInputDelayFallingEdge
MinFallOutputDelay
MinFallOutputDelayFallingEdge
MinFallTransitionDelay
MinInputDelay
MinInputDelayFallingEdge
MinOutputDelay
MinOutputDelayFallingEdge
MinRiseInputDelay
MinRiseInputDelayFallingEdge
MinRiseOutputDelay
MinRiseOutputDelayFallingEdge
MinRiseTransitionDelay
MinValue
MinWriteConstraint
MonitoredComponent
MonitoredInterface
Name
net
NumberOfScanChains
NumberOfScanCompressionChains
ObservePointsPerScanCell
OneRequiredGroup
OneWrapperClock
OpenAllTreeItems

OperatingConditionsBest
OperatingConditionsWorst
OptimizationPriorities
OptimizeArithmetic
Optional
OptionalAssociation
OutputDelay
Overconstrain
ParallelJobCpuLimit
param
ParameterCheckCmd
ParameterInfo
Parameters
ParamValueFromDesign
ParentWireLoad
PhysicalLeft
PhysicalRegion
PhysicalRight
pin
PingTestMask
PinLoadCount
PinLoadType
port
PortDirection
PortFunction
PortWidth
PostPromptCmd
PowerDomain
PowerDomains
PredefinedInoutPorts
PredefinedInputPorts
PredefinedOutputPorts
PrePromptCmd
PreserveDesignWare
PreserveHierarchyFromTop
PreserveMuxOps
PrimeProfileAllowNone
PrimeProfileCurrent
PrimeProfileDefaults
PrimeProfileNote
PrimeProfilesValid
ProjectID

ProjectRootDir
pt_shellVariable
pt_shellVariableComment
RALAccessType
RALAdditionalFieldAttributeText
RalListInfo
RALSuppressionComment
ReadAction
ReadActionDescription
ReadActionModifier
ReadOnlyInterface
ReadOnlyParam
ReadOnlyWorkspaceComponent
ReferenceClock
ReferenceClockName
refItem
register
registerArray
RegisterArrayDimensions
Registered
RegisteredBy
registerField
RegisterResetMask
RegisterResetValue
RegisterSize
RemoveIfEmpty
ReplaceInHDL
RequiredExPortDirection
RequiredPortDirection
Reserved
ResetProperties
rtl_shellAppOption
rtl_shellAppOptionComment
RtlaAuxScript
RtlaAuxScriptComment
RTLStaticCheckingAsSubblock
RunOptions
ScanBlockIndividually
ScanCompressionConfiguration
ScanCompressionTestClock
ScanCompressionUsage
ScanExclude

ScanInternalClocks
ScanJumpBuffersInverters
ScanMethodology
ScanReplace
ScanStyle
ScanSubblocksIndividually
scratch_dir
ScratchInstallIndex
SDCConstraints
SDCPropagateText
SDCWriteCallback
SeqCompressionInputs
SeqCompressionOutputs
Sequence
SetInHex
sh_executable_name
sh_product_version
sh_show_tcl_stack_on_error
SharedWrapperCell
ShareTestPointsAcrossHierarchy
ShowIcons
ShowRoute
SideConstraint
SideEffects
SimTieOff
SimulationModel
SlotNumber
SlotNumberOffset
SlotOrder
SlotWidth
SolutionData
SolutionEnvVarValues
SolutionItemPhase
SolutionProcessed
SpiritContainer
SpiritFile
SpiritFileType
SpiritInterfaceType
SpiritLibrary
SpiritName
SpiritVendor
SpiritVersion

[SplitInterfaceMembers](#)
[SplitInterfaceName](#)
[StartBit](#)
[StartBitExpr](#)
[StartBitsExpr](#)
[StaticDefineExpr](#)
[StaticDefineExprs](#)
[StaticTimingAuxScript](#)
[StaticTimingAuxScriptComment](#)
[Strategy](#)
[SubstituteInFile](#)
[SymbolicNames](#)
[SymbolLibraryPath](#)
[SymbolName](#)
[SymbolPinPrefix](#)
[SymbolPinSide](#)
[SymbolType](#)
[SymmetricBus](#)
[SymmetricBusLink](#)
[SynchronousTo](#)
[SynplifyAuxSynthesisScript](#)
[SynplifyAuxSynthesisScriptComment](#)
[SynthesizeAsSubblock](#)
[sysCmd](#)
[TargetConnectionBoundary](#)
[TCBDisableUndefMacros](#)
[TclAuxSynthesisScript](#)
[TclAuxSynthesisScriptComment](#)
[TCLConfigVariable](#)
[TCLConfigVariableName](#)
[TCLConfigVariableSetArgs](#)
[TclPhysicalConstraintFile](#)
[TechCellMappingBind](#)
[TechCellMappingDefn](#)
[TestabilityClockSignal](#)
[TestabilityClockType](#)
[TestabilityControlSignal](#)
[TestabilityMaxArea](#)
[TestabilityMethod](#)
[Testable](#)
[TestClock](#)
[TestClockCycleTime](#)

TestClockDefaultCycleTime
TestClocksFollowSystemDomains
TestClockWaveform
TestIsolate
TestpointPowerSavingOn
timingException
TimingExceptionCmd
TimingExceptionFall fromValue
TimingExceptionFall throughValue
TimingExceptionFall toValue
TimingExceptionFromValue
TimingExceptionGroup path
TimingExceptionOptions
TimingExceptionRise fromValue
TimingExceptionRise throughValue
TimingExceptionRise toValue
TimingExceptionThroughValue
TimingExceptionToValue
TimingExceptionValue
tmax shellVariable
tmax shellVariableComment
tmm shellAppOption
tmm shellAppOptionComment
TmmAuxScript
TmmAuxScriptComment
TMMRMVariable
TMRIntentFile
TMRIntentFileComment
TypedefType
TypeName
UnconnectedPort
UndefinedBitValue
Underconstrain
UnelaboratedName
UniqueConsumerConnections
UPFFile
UsedOnInstance
UserPortName
Utilization
UVMRALAccessType
UVMTestText
Value

[ValueRequired](#)
[verbose messages](#)
[VerilogHeaderValue](#)
[Version](#)
[VHDLDesignLibrary](#)
[VhdlHeaderValue](#)
[VipConnection](#)
[VipInitializationCall](#)
[VipModelName](#)
[VipPackage](#)
[VipRandomizable](#)
[VipRandomRange](#)
[VipReasonableRandomRange](#)
[VipScope](#)
[Visible](#)
[VolatileMemory](#)
[Waveform](#)
[WireLoad](#)
[WireLoadGroup](#)
[WireLoadMinBlockSize](#)
[WireLoadMode](#)
[WrapBlockIndividually](#)
[WrapperDefaultSafeState](#)
[WrapperExcludePorts](#)
[WrapperRegisterImplementation](#)
[WrapperUseRegisterIO](#)
[WrapSubblocksIndividually](#)
[WriteBehavior](#)
[WriteBehaviorModifier](#)
[WriteComponentInHDL](#)
[WriteConstraint](#)
[Add Interfaces](#)
[Build coreKit](#)
[Build IP-XACT Component](#)
[Build Subsystem coreKit](#)
[Complete Connections](#)
[Constraint Analysis](#)
[Constraints Verification](#)
[Constraints Verification With Assertions](#)
[Create Bill of Materials](#)
[Create Integration Workspaces](#)
[Create Subsystem Template](#)

[Create Testbench Workspace](#)
[Define Component Views](#)
[Define Memory Maps](#)
[Design Configuration Intent](#)
[Explore RTL Architecture](#)
[Formal Verification](#)
[Perform FPGA Synthesis](#)
[Generate Subsystem RTL](#)
[Generate Test Vectors](#)
[Initialize Subsystem Constraints](#)
[Load Consultant Plugins](#)
[Load Designs](#)
[Load Interface Definitions](#)
[Memory Map Specification](#)
[RTL Static Checking](#)
[Select Prime Profile](#)
[Set Design, File, and Macro Prefixes](#)
[Set Up Help Menu](#)
[Simulate Subsystem Testbench](#)
[Specify Clocks](#)
[Specify Configuration](#)
[Specify Operating Conditions & WireLoads](#)
[Specify Port Constraints](#)
[Specify Power Intent](#)
[Specify Subsystem](#)
[Specify Synthesis Methodology](#)
[Specify Test Methodology](#)
[Specify Timing Exceptions](#)
[Static Checking](#)
[Static Timing Analysis](#)
[Perform ASIC Synthesis](#)
[Technology Cell Binding](#)
[Specify Reference Technology](#)
[Testsuite Set-up](#)
[Verify Intent](#)
[ATPG basic scan strategy](#)
[DCTCL quick map strategy](#)
[Design Compiler Reference Methodology](#)
[FM formal verification strategy](#)
[FPGA optimization strategy](#)
[Fusion Compiler Reference Methodology](#)
[PT static timing analysis strategy](#)

RTL Architect Reference Methodology

TMAX Reference Methodology

Deprecated List

coreTools Command Reference Index

DocWaiver (3), Enabled (3), EndBit (3), EndBitExpr (3), EndBitsExpr (3),
ExportDefineToFile (3), GenerateIf (3), GroupImage (3), GroupImageAttrs
(3), GroupImageTitle (3), GroupName (3), GroupText (3), GroupUserLabel
(3), HdlTypeName (3), HdlUnquoted (3), HdlValue (3), InterfacesUsed (3),
InterfaceLink (3), IsArray (3), IsControlValue (3), Label (3),
LockedInTemplate (3), MaxValue (3), MemoryAccess (3),
MemoryAccessDescription (3), MinValue (3), Name (3), OptionalAssociation
(3), ParamValueFromDesign (3), ReadOnlyParam (3), ReplaceInHDL (3),
Sequence (3), SpiritContainer (3), StartBit (3), StartBitExpr (3),
StartBitsExpr (3), StaticDefineExpr (3), TCLConfigVariable (3),
TCLConfigVariableName (3), TypeName (3), UsedOnInstance (3), Value (3),
ValueRequired (3), VipRandomRange (3), VipRandomizable (3),
VipReasonableRandomRange (3), VipScope (3), Visible (3),

NAME

parse_proc_arguments

Parses the arguments passed into a Tcl procedure.

SYNTAX

```
string parse_proc_arguments -args arg_list result_array

list arg_list
string result_array
```

ARGUMENTS

-args *arg_list* Specified the list of arguments passed in to the Tcl procedure.

result_array Specifies the name of the array into which the parsed arguments should be stored.

DESCRIPTION

The **parse_proc_arguments** command is used within a Tcl procedure to enable use of the -help option, and to support argument validation. It should typically be the first command called within a procedure. Procedures that use **parse_proc_arguments** will validate the semantics of the procedure arguments and generate the same syntax and semantic error messages as any application command (see the examples that follow).

When a procedure that uses **parse_proc_arguments** is invoked with the -help option, **parse_proc_arguments** will print help information (in the same style as using **help -verbose**) and will then cause the calling procedure to return. Similarly, if there was any type of error with the arguments (missing required arguments, invalid value, and so on), **parse_proc_arguments** will return a Tcl error and the calling procedure will terminate and return.

coreTools Command Reference Index

If you didn't specify `-help`, and the specified arguments were valid, the array variable `result_array` will contain each of the argument values, subscripted with the argument name. Note that the argument name here is NOT the names of the arguments in the procedure definition, but rather the names of the arguments as defined using the `define_proc_attributes` command.

The `parse_proc_arguments` command cannot be used outside of a procedure.

EXAMPLES

The following procedure shows how `parse_proc_arguments` is typically used. The `argHandler` procedure parses the arguments it receives. If the parse is successful, `argHandler` prints the options or values actually received.

```
proc argHandler args {
    parse_proc_arguments -args $args results
    foreach argname [array names results] {
        echo " $argname = $results($argname)"
    }
}
define_proc_attributes argHandler -info "argument processor" \
    -define_args \
    {{-Oos "oos help" AnOos one_of_string {required value_help {values {a b}}}}
    {-Int "int help" AnInt int optional}
    {-Float "float help" AFloat float optional}
    {-Bool "bool help" "" boolean optional}
    {-String "string help" AString string optional}
    {-List "list help" AList list optional}}
    {-IDup int dup AIDup int {optional merge_duplicates}}}
```

Invoking `argHandler` with the `-help` option generates the following:

```
prompt> argHandler -help
Usage: argHandler      # argument processor
      -Oos AnOos      (oos help:
                       Values: a, b)
      [-Int AnInt]    (int help)
      [-Float AFloat] (float help)
      [-Bool]         (bool help)
      [-String AString] (string help)
      [-List AList]   (list help)
```

Invoking `argHandler` with an invalid option causes the following output (and a Tcl error):

```
prompt> argHandler -Int z
Error: value 'z' for option '-Int' not of type 'integer' (CMD-009)
Error: Required argument '-Oos' was not found (CMD-007)
```

Invoking `argHandler` with valid arguments generates the following:

```
prompt> argHandler -Int 6 -Oos a  
-Oos = a  
-Int = 6
```

SEE ALSO

define_proc_attributes(2), help(2), proc(2).

percent_of_period

Define delay as a function of clock's cycle time

Syntax

```
string percent_of_period percentage [clock]
      double percentage
      string clock
```

Parameters

percentage	The percentage of the specified clock's period.
	The name of the clock.
clock	You do not need to specify <clock> if there is only one clock in the design.

Description

The percent_of_period command returns a value that is equal to the specified percentage of the specified clock's cycle time. The most common use for percent_of_period is as a formula to specify values for input and output delay attributes, or any other time-based attribute, as a function of a clock's cycle time.

When you use percent_of_period as a formula to specify input and output delay attributes on ports, coreConsultant evaluates the formula to generate input and output delay constraints. If you change the value of the clock's CycleTime attribute, coreConsultant automatically propagates that change to the input and output delay constraints.

If there is only one clock in the current_design, you do not need to specify <clock>. The coreTools use the single existing clock by default.

Examples

To return a value that is 25 percent of the default clock's period:

```
coreConsultant> percent_of_period 25
2650.00
```

To specify the InputDelay on a port as a function of the default clock's period:

```
coreConsultant> set_port_attribute d_bus InputDelay
{=percent_of_period 25.0}
```

To specify the OutputDelay on a port as a function of the test clock's (tclk) period:

```
coreConsultant> set_port_attribute scan_out {OutputDelay[tclk]}  
{=percent_of_period 15.0 tclk}
```

See Also

[set_port_attribute \(2\)](#)

permanent_proc

Wrapper around proc to make a permanent hidden proc.

Syntax

```
string permanent_proc procName procArgs procBody [procHelp] [procArgDefn]
    string procName
    string procArgs
    string procBody
    string procHelp
    string procArgDefn
```

Parameters

procName	procedure name
procArgs	procedure arguments
procBody	procedure body
procHelp	procedure help
procArgDefn	procedure argument definitions

Description

This command is used to create a proc which is both permanent (cannot be redefined) and hidden (not visible to help or info). It is defined such that it can be called multiple times without issue so is useful for defining procs that need to be permanent but whose definitions might get sourced multiple times (e.g. plugin procedures).

The optional help argument will not be visible externally since the command is hidden. The optional argument definition option expects input consistent with the -define_args argument to the define_proc_attributes command.

If your proc needs to be permanent but not hidden use protected_proc instead.

Examples

Example definition for a simple permanent hidden proc:

```
permanent_proc myProc {myProcArg1 myProcArg2} {
echo "Arg1 is $myProcArg1"
echo "Arg2 is $myProcArg2"
}
```

See Also

protected proc

NAME

`pid` - Retrieve process identifiers

SYNOPSIS

`pid ?fileId?`

DESCRIPTION

If the *fileId* argument is given then it should normally refer to a process pipeline created with the **open** command. In this case the **pid** command will return a list whose elements are the process identifiers of all the processes in the pipeline, in order. The list will be empty if *fileId* refers to an open file that is not a process pipeline. If no *fileId* argument is given then **pid** returns the process identifier of the current process. All process identifiers are returned as decimal strings.

EXAMPLE

Print process information about the processes in a pipeline using the SysV **ps** program before reading the output of that pipeline:

```
set pipeline [open "| zcat somefile.gz | grep foofoo | sort -u"]
# Print process information
exec ps -fp [pid $pipeline] >@stdout
# Print a separator and then the output of the pipeline
puts [string repeat - 70]
puts [read $pipeline]
```

`close $pipeline`

SEE ALSO

`exec(n)`, `open(n)`

KEYWORDS

`file`, `pipeline`, `process identifier`

pin
Pin item

Description

The pin item type represents a port on a cell (design instance) in the coreBuilder or coreConsultant model of a design.

See Also

[port](#) (3)

Supported Attributes

[AutoConnectIfUnconnected](#) (3), [CaseAnalysisValue](#) (3), [CnctClass](#) (3),
[ComponentOfItem](#) (3), [DeferEvaluation](#) (3), [Description](#) (3),
[DisableAggregation](#) (3), [DocInclude](#) (3), [EndBitExpr](#) (3), [EndBitsExpr](#) (3),
[GenerateIf](#) (3), [IfUnconnected](#) (3), [IsConnected](#) (3), [Label](#) (3),
[LogicalName](#) (3), [MaxLoads](#) (3), [Name](#) (3), [PortFunction](#) (3), [RegisteredBy](#)
(3), [Sequence](#) (3), [SimTieOff](#) (3), [StartBitExpr](#) (3), [StartBitsExpr](#) (3),
[StaticDefineExpr](#) (3), [TypeName](#) (3), [TypedefType](#) (3), [UserPortName](#) (3)

`pin_cap_of`

Return the capacitance of the specified pin.

Syntax

```
string pin_cap_of pin
      string pin
```

Parameters

`pin` The hierarchal path to the pin.

Description

Return the capacitance of the specified pin. Return "" if the pin argument is not specified as CELL/PIN or if the pin does not exist on the cell.

Examples

```
coreConsultant> pin_cap_of ND2/A
1
coreConsultant> set_port_attribute out_port PinLoadType
{=explicit_capacitance [pin_cap_of ND2/A]}
coreConsultant> get_attribute out_port -attr PinLoadType
explicit 1
```

See Also

NAME

`pkg_mkIndex` - Build an index for automatic loading of packages

SYNOPSIS

`pkg_mkIndex ?options...? dir ?pattern pattern ...?`

DESCRIPTION

`Pkg_mkIndex` is a utility procedure that is part of the standard Tcl library. It is used to create index files that allow packages to be loaded automatically when **package require** commands are executed. To use `pkg_mkIndex`, follow these steps:

- [1] Create the package(s). Each package may consist of one or more Tcl script files or binary files. Binary files must be suitable for loading with the **load** command with a single argument; for example, if the file is **test.so** it must be possible to load this file with the command **load test.so**. Each script file must contain a **package provide** command to declare the package and version number, and each binary file must contain a call to **Tcl_PkgProvide**.
- [2] Create the index by invoking **pkg_mkIndex**. The *dir* argument gives the name of a directory and each *pattern* argument is a **glob**-style pattern that selects script or binary files in *dir*. The default pattern is ***.tcl** and ***.[info sharedlibextension]**.

coreTools Command Reference Index

Pkg_mkIndex will create a file **pkgIndex.tcl** in *dir* with package information about all the files given by the *pattern* arguments. It does this by loading each file into a child interpreter and seeing what packages and new commands appear (this is why it is essential to have **package provide** commands or **Tcl_PkgProvide** calls in the files, as described above). If you have a package split among scripts and binary files, or if you have dependencies among files, you may have to use the **-load** option or adjust the order in which **pkg_mkIndex** processes the files. See **COMPLEX CASES** below.

- [3] Install the package as a subdirectory of one of the directories given by the **tcl_pkgPath** variable. If **\$tcl_pkgPath** contains more than one directory, machine-dependent packages (e.g., those that contain binary shared libraries) should normally be installed under the first directory and machine-independent packages (e.g., those that contain only Tcl scripts) should be installed under the second directory. The subdirectory should include the package's script and/or binary files as well as the **pkgIndex.tcl** file. As long as the package is installed as a subdirectory of a directory in **\$tcl_pkgPath** it will automatically be found during **package require** commands.

If you install the package anywhere else, then you must ensure that the directory containing the package is in the **auto_path** global variable or an immediate subdirectory of one of the directories in **auto_path**. **Auto_path** contains a list of directories that are searched by both the auto-loader and the package loader; by default it includes **\$tcl_pkgPath**. The package loader also checks all of the subdirectories of the directories in **auto_path**. You can add a directory to **auto_path** explicitly in your application, or you can add the directory to your **TCLLIBPATH** environment variable: if this environment variable is present, Tcl initializes **auto_path** from it during application startup.

- [4] Once the above steps have been taken, all you need to do to use a package is to invoke **package require**. For example, if versions 2.1, 2.3, and 3.1 of package **Test** have been indexed by **pkg_mkIndex**, the command **package require Test** will make version 3.1 available and the

command **package require -exact Test 2.1** will make version 2.1 available. There may be many versions of a package in the various index files in **auto_path**, but only one will actually be loaded in a given interpreter, based on the first call to **package require**. Different versions of a package may be loaded in different interpreters.

OPTIONS

The optional switches are:

- direct** The generated index will implement direct loading of the package upon **package require**. This is the default.

- lazy** The generated index will manage to delay loading the package until the use of one of the commands provided by the package, instead of loading it immediately upon **package require**. This is not compatible with the use of *auto_reset*, and therefore its use is discouraged.

- load pkgPat** The index process will pre-load any packages that exist in the current interpreter and match *pkgPat* into the child interpreter used to generate the index. The pattern match uses string match rules, but without making case distinctions. See **COMPLEX CASES** below.

- verbose** Generate output during the indexing process. Output is via the **tclLog** procedure, which by default prints to stderr.

- End of the flags, in case *dir* begins with a dash.

PACKAGES AND THE AUTO-LOADER

The package management facilities overlap somewhat with the auto-loader, in that both arrange for files to be loaded on-demand. However, package management is a higher-level mechanism that uses the auto-loader for the last step in the loading process. It is generally better to index a package with **pkg_mkIndex** rather than **auto_mkindex** because the package mechanism provides version control: several versions of a package can be made available in the index files, with different applications using different versions based on **package require** commands. In contrast, **auto_mkindex** does not understand versions so it can only handle a single

version of each package. It is probably not a good idea to index a given package with both **pkg_mkIndex** and **auto_mkindex**. If you use **pkg_mkIndex** to index a package, its commands cannot be invoked until **package require** has been used to select a version; in contrast, packages indexed with **auto_mkindex** can be used immediately since there is no version control.

HOW IT WORKS

Pkg_mkIndex depends on the **package unknown** command, the **package ifneeded** command, and the auto-loader. The first time a **package require** command is invoked, the **package unknown** script is invoked. This is set by Tcl initialization to a script that evaluates all of the **pkgIndex.tcl** files in the **auto_path**. The **pkgIndex.tcl** files contain **package ifneeded** commands for each version of each available package; these commands invoke **package provide** commands to announce the availability of the package, and they setup auto-loader information to load the files of the package. If the **-lazy** flag was provided when the **pkgIndex.tcl** was generated, a given file of a given version of a given package is not actually loaded until the first time one of its commands is invoked. Thus, after invoking **package require** you may not see the package's commands in the interpreter, but you will be able to invoke the commands and they will be auto-loaded.

DIRECT LOADING

Some packages, for instance packages which use namespaces and export commands or those which require special initialization, might select that their package files be loaded immediately upon **package require** instead of delaying the actual loading to the first use of one of the package's command. This is the default mode when generating the package index. It can be overridden by specifying the **-lazy** argument.

COMPLEX CASES

Most complex cases of dependencies among scripts and binary files, and packages being split among scripts and binary files are handled OK. However, you may have to adjust the order in which files are processed by **pkg_mkIndex**. These issues are described in detail below.

If each script or file contains one package, and packages are only contained in one file, then things are easy. You simply specify all files to be indexed

coreTools Command Reference Index

in any order with some glob patterns.

In general, it is OK for scripts to have dependencies on other packages. If scripts contain **package require** commands, these are stubbed out in the interpreter used to process the scripts, so these do not cause problems. If scripts call into other packages in global code, these calls are handled by a stub **unknown** command. However, if scripts make variable references to other package's variables in global code, these will cause errors. That is also bad coding style.

If binary files have dependencies on other packages, things can become tricky because it is not possible to stub out C-level APIs such as **Tcl_PkgRequire** API when loading a binary file. For example, suppose the BLT package requires Tk, and expresses this with a call to **Tcl_PkgRequire** in its **Blt_Init** routine. To support this, you must run **pkg_mkIndex** in an interpreter that has Tk loaded. You can achieve this with the **-load pkgPat** option. If you specify this option, **pkg_mkIndex** will load any packages listed by **info loaded** and that match **pkgPat** into the interpreter used to process files. In most cases this will satisfy the **Tcl_PkgRequire** calls made by binary files.

If you are indexing two binary files and one depends on the other, you should specify the one that has dependencies last. This way the one without dependencies will get loaded and indexed, and then the package it provides will be available when the second file is processed. You may also need to load the first package into the temporary interpreter used to create the index by using the **-load** flag; it will not hurt to specify package patterns that are not yet loaded.

If you have a package that is split across scripts and a binary file, then you should avoid the **-load** flag. The problem is that if you load a package before computing the index it masks any other files that provide part of the same package. If you must use **-load**, then you must specify the scripts first; otherwise the package loaded from the binary file may mask the package defined by the scripts.

SEE ALSO

`package(n)`

KEYWORDS

auto-load, index, package, version

NAME

`platform` - System identification support code and utilities

SYNOPSIS

```
package require platform ?1.0.10?
```

```
platform::generic  
platform::identify  
platform::patterns identifier
```

DESCRIPTION

The **platform** package provides several utility commands useful for the identification of the architecture of a machine running Tcl.

Whilst Tcl provides the **tcl_platform** array for identifying the current architecture (in particular, the platform and machine elements) this is not always sufficient. This is because (on Unix machines) **tcl_platform** reflects the values returned by the **uname** command and these are not standardized across platforms and architectures. In addition, on at least one platform (AIX) the **tcl_platform(machine)** contains the CPU serial number.

Consequently, individual applications need to manipulate the values in **tcl_platform** (along with the output of system specific utilities) - which is both inconvenient for developers, and introduces the potential for inconsistencies in identifying architectures and in naming conventions.

The **platform** package prevents such fragmentation - i.e., it establishes a standard naming convention for

coreTools Command Reference Index

architectures running Tcl and makes it more convenient for developers to identify the current architecture a Tcl program is running on.

COMMANDS

platform::identify

This command returns an identifier describing the platform the Tcl core is running on. The returned identifier has the general format *OS-CPU*. The *OS* part of the identifier may contain details like kernel version, libc version, etc., and this information may contain dashes as well. The *CPU* part will not contain dashes, making the preceding dash the last dash in the result.

platform::generic

This command returns a simplified identifier describing the platform the Tcl core is running on. In contrast to **platform::identify** it leaves out details like kernel version, libc version, etc. The returned identifier has the general format *OS-CPU*.

platform::patterns *identifier*

This command takes an identifier as returned by **platform::identify** and returns a list of identifiers describing compatible architectures.

EXAMPLE

This can be used to allow an application to be shipped with multiple builds of a shared library, so that the same package works on many versions of an operating system. For example:

package require platform

```
# Assume that app script is ../theapp/bin/theapp.tcl
set binDir [file dirname [file normalize [info script]]]
set libDir [file join $binDir .. lib]
set platLibDir [file join $libDir [platform::identify]]
load [file join $platLibDir support[info sharedlibextension]]
```

KEYWORDS

operating system, cpu architecture, platform, architecture

NAME

`platform::shell` - System identification support code and utilities

SYNOPSIS

`package require platform::shell ?1.1.4?`

```
platform::shell::generic shell
platform::shell::identify shell
platform::shell::platform shell
```

DESCRIPTION

The `platform::shell` package provides several utility commands useful for the identification of the architecture of a specific Tcl shell.

This package allows the identification of the architecture of a specific Tcl shell different from the shell running the package. The only requirement is that the other shell (identified by its path), is actually executable on the current machine.

While for most platform this means that the architecture of the interrogated shell is identical to the architecture of the running shell this is not generally true. A counter example are all platforms which have 32 and 64 bit variants and where a 64bit system is able to run 32bit code. For these running and interrogated shell may have different 32/64 bit settings and thus different identifiers.

For applications like a code repository it is important to identify the architecture of the shell which will actually run the installed packages, versus the architecture of the shell running the repository

software.

COMMANDS

platform::shell::identify *shell*

This command does the same identification as **platform::identify**, for the specified Tcl shell, in contrast to the running shell.

platform::shell::generic *shell*

This command does the same identification as **platform::generic**, for the specified Tcl shell, in contrast to the running shell.

platform::shell::platform *shell*

This command returns the contents of **tcl_platform(platform)** for the specified Tcl shell.

KEYWORDS

operating system, cpu architecture, platform, architecture

plugin_proc

Define a procedure in coreKit context or workspace context

Syntax

```
string plugin_proc [-info info_text] [-hidden] [-define_args arg_defs]
[-command_group group_name] [-namespace namespace_name] procName procArgs
procBody
    string info_text
    string arg_defs
    string group_name
    string namespace_name
    string procName
    string procArgs
    string procBody
```

Parameters

-info <i>info_text</i>	Help string for the procedure
-hidden	Procedure does not show up in help or info
-define_args <i>arg_defs</i>	Procedure argument definitions for verbose help
-command_group <i>group_name</i>	Command group for procedure. (default: Plugin Procedures)
-namespace <i>namespace_name</i>	Namespace in which to define the proc.
<i>procName</i>	Name of new global procedure (as for 'proc') The procedure name is global by default, like a prepended "namespace ::", until the name starts with ::.
<i>procArgs</i>	Procedure arguments list
<i>procBody</i>	Procedure body script

Description

plugin_proc declares a procedure like proc, and the procedure is part of the coreKit. For this reason the overall behaviour is like a procedure declared in a core-specific plug-in. plugin_proc is dedicated to provide a simple way for procedure definitions required by intent specifications (rare case of configuration intent and/or synthesis intent procedures) and TRE specifications, especially filegroup attribute PostPromptCmd.

Under the hood the procedure is defined for the lifetime of the workspace, and in case of coreBuilder the procedure is exported to the

coreKit and all coreConsultant workspaces.

A `plugin_proc` cannot be used to declare a plug-in procedure (during `create_plugin_kb`).

`plugin_proc` directly provides procedure attributes. Please refer to `define_proc_attributes` for details about the options `-command_group`, `-define_args`, `-hidden` and `-info`. The `define_proc_attributes` option `-permanent` is not supported (in all cases where `plugin_proc` is useful, especially intent files, the source files could be read multiple times during a `coreBuilder` session; `-permanent` would fail in this case).

Examples

The core needs a special function for attribute formulas (`percent_of_period`, `select_by_name` etc). The intent file attached to the HDL file defines an custom `percent_of_period` function and uses it for the Delay attributes:

```
set_port_attribute clk ClockName clock
set_clock_attribute clock CycleTime 100ns
# default: 10
set_port_attribute cs {InputDelay[clock]} {myPOP }
set_port_attribute wr {InputDelay[clock]} {myPOP 20 }
plugin_proc myPOP {args} \
-info "A special delay dependency function" \
-command_group "POP" \
-define_args {
{percentage "percentage of clock period" percentage int optional}
} \
{
set opts(percentage) 10
parse_proc_arguments -args $args opts
return [percent_of_period $opts(percentage)]
}
```

Now look at help for this command:

```
coreBuilder> help -v myPOP
myPOP # A special delay dependency function
[percentage] (percentage of clock period)
coreBuilder> help -v POP
POP:
myPOP # A special delay dependency function
[percentage] (percentage of clock period)
coreBuilder> myPOP -help
myPOP # A special delay dependency function
[percentage] (percentage of clock period)
```


See Also

[define_proc_attributes \(2\)](#), [parse_proc_arguments \(2\)](#), [add_workspace_hook \(2\)](#), [PostPromptCmd \(3\)](#), [create_plugin_kb \(2\)](#)

port
Port item

Description

The port item type represents a port on a design. When coreBuilder parses the HDL source code for a core, it creates a port object for each port that it finds on each design.

For designs with hierarchy, coreBuilder creates ports for each design's ports and pins for the subdesign ports within each design. That is, the ports of a subdesign are pins in the parent design.

See Also

[pin](#) (3)

Supported Attributes

[AutoConnectIfUnconnected](#) (3), [Capacitance](#) (3), [CaseAnalysisValue](#) (3), [ClockName](#) (3), [CnctClass](#) (3), [ComponentOfItem](#) (3), [ConstantPort](#) (3), [ConvertSingleBitBus](#) (3), [CriticalTiming](#) (3), [DeferEvaluation](#) (3), [Description](#) (3), [DftExistingSignalActiveState](#) (3), [DftExistingSignalTestMode](#) (3), [DftExistingSignalTiming](#) (3), [DftExistingSignalType](#) (3), [DftExistingSignalUsage](#) (3), [DftSpecSignalActiveState](#) (3), [DftSpecSignalTestMode](#) (3), [DftSpecSignalType](#) (3), [DftSpecSignalUsage](#) (3), [DisableAggregation](#) (3), [DocDescriptionField](#) (3), [DocEnabled](#) (3), [DocGenerateIf](#) (3), [DocGroup](#) (3), [DocGroupName](#) (3), [DocInclude](#) (3), [DocMaster](#) (3), [DocPortWidth](#) (3), [DocPowerDomain](#) (3), [DocRangeDecoratedName](#) (3), [DocRegisteredBy](#) (3), [DocSynchronousTo](#) (3), [DocWaiver](#) (3), [DontTouchNetwork](#) (3), [Drive](#) (3), [DrivingCell](#) (3), [EndBit](#) (3), [EndBitExpr](#) (3), [EndBitsExpr](#) (3), [FeedThroughConnect](#) (3), [FpgaPadType](#) (3), [FpgaPortIsPad](#) (3), [GenerateIf](#) (3), [GroupImage](#) (3), [GroupImageAttrs](#) (3), [GroupImageTitle](#) (3), [GroupName](#) (3), [GroupText](#) (3), [HighFanout](#) (3), [IdealPort](#) (3), [IfUnconnected](#) (3), [InputDelay](#) (3), [InputResistance](#) (3), [IsBehavioral](#) (3), [IsConnected](#) (3), [IsExternalReset](#) (3), [IsTriState](#) (3), [Label](#) (3), [LogicalName](#) (3), [MaxCap](#) (3), [MaxFallInputDelay](#) (3), [MaxFallInputDelayFallingEdge](#) (3), [MaxFallOutputDelay](#) (3), [MaxFallOutputDelayFallingEdge](#) (3), [MaxFanout](#) (3), [MaxInputDelay](#) (3), [MaxInputDelayFallingEdge](#) (3), [MaxLoads](#) (3), [MaxOutputDelay](#) (3), [MaxOutputDelayFallingEdge](#) (3), [MaxRiseInputDelay](#) (3), [MaxRiseInputDelayFallingEdge](#) (3), [MaxRiseOutputDelay](#) (3), [MaxRiseOutputDelayFallingEdge](#) (3), [MaxTransition](#) (3), [MinCap](#) (3), [MinFallInputDelay](#) (3), [MinFallInputDelayFallingEdge](#) (3), [MinFallOutputDelay](#) (3), [MinFallOutputDelayFallingEdge](#) (3), [MinInputDelay](#) (3), [MinInputDelayFallingEdge](#) (3), [MinOutputDelay](#) (3),

coreTools Command Reference Index

MinOutputDelayFallingEdge (3), MinRiseInputDelay (3),
MinRiseInputDelayFallingEdge (3), MinRiseOutputDelay (3),
MinRiseOutputDelayFallingEdge (3), Name (3), OutputDelay (3),
PinLoadCount (3), PinLoadType (3), PortDirection (3), PortFunction (3),
PortWidth (3), PowerDomain (3), RegisteredBy (3), ReplaceInHDL (3),
ResetProperties (3), Sequence (3), SideConstraint (3), SimTieOff (3),
SpiritContainer (3), StartBit (3), StartBitExpr (3), StartBitsExpr (3),
StaticDefineExpr (3), SynchronousTo (3), TestIsolate (3), TypeName (3),
TypedefType (3), UnconnectedPort (3), UserPortName (3), Visible (3),
WireLoad (3)

NAME

tcl::prefix - facilities for prefix matching

SYNOPSIS

```
::tcl::prefix all table string
::tcl::prefix longest table string
::tcl::prefix match ?option ...? table string
```

DESCRIPTION

This document describes commands looking up a prefix in a list of strings. The following commands are supported:

::tcl::prefix all *table string*

Returns a list of all elements in *table* that begin with the prefix *string*.

::tcl::prefix longest *table string*

Returns the longest common prefix of all elements in *table* that begin with the prefix *string*.

::tcl::prefix match *?options?* *table string*

If *string* equals one element in *table* or is a prefix to exactly one element, the matched element is returned. If not, the result depends on the **-error** option. (It is recommended that the *table* be sorted before use with this subcommand, so that the list of matches presented in the error message also becomes sorted, though this is not strictly necessary for the operation of this subcommand itself.)

-exact

Accept only exact matches.

-message *string*

coreTools Command Reference Index

Use *string* in the error message at a mismatch. Default is

-error options

The *options* are used when no match is found. If *options* is empty, no error is generated and an empty string is returned. Otherwise the *options* are used as **return** options when generating the error message. The default corresponds to setting Example: If is used, an error would be generated as:

```
return -errorcode MyError -level 1 -code error \
    "ambiguous option ..."
```

EXAMPLES

Basic use:

```
namespace import ::tcl::prefix
prefix match {apa bepa cepa} apa
    -> apa
prefix match {apa bepa cepa} a
    -> apa
prefix match -exact {apa bepa cepa} a
    -> bad option "a": must be apa, bepa, or cepa
prefix match -message "switch" {apa ada bepa cepa} a
    -> ambiguous switch "a": must be apa, ada, bepa, or cepa
prefix longest {fblocked fconfigure fcopy file fileevent flush} fc
    -> fco
prefix all {fblocked fconfigure fcopy file fileevent flush} fc
    -> fconfigure fcopy
```

Simplifying option matching:

```
array set opts {-apa 1 -bepa "" -cepa 0}
foreach {arg val} $args {
    set opts([prefix match {-apa -bepa -cepa} $arg]) $val
}
```

Creating a **switch** that supports prefixes:

```
switch [prefix match {apa bepa cepa} $arg] {
    apa { }
    bepa { }
    cepa { }
}
```

SEE ALSO

lsearch(n), namespace(n), string(n),
Tcl_GetIndexFromObj(3)

KEYWORDS

prefix, table lookup

prefix_defines
Prefix parameters in RTL

Syntax

string **prefix_defines** -prefix <prefix> [-out <directory>] [-files <file list>]

string <prefix>
string <directory>
string <file list>

Parameters

-prefix <prefix>	Prefix to be applied to RTL configuration parameters
-out <directory>	Output directory
-files <file list>	File containing a list of files for processing

Description

This command can be used to specify a prefix to prepend to the configuration parameters used in your RTL. Every parameter will be renamed <prefix>_<original_name>. This can be useful when instantiating two different configurations of the same core in a higher-level design.

If an output directory is not specified, the modified RTL containing the new prefixed parameters will be placed in <workspace>/export/prefixed_src. The original, unmodified RTL will be preserved in its original location. Overwriting this original RTL is forbidden and will cause an error.

The files on which prefixing is to be performed may be specified in a single file using the "-files" switch. If this argument is not specified, then all of the RTL files associated with your core will be processed.

If this command is used in coreAssembler, it should be used separately for each component instance. See the examples below for one way to accomplish this.

Examples

To prefix all design parameters with the string "U_prefix"

`prefix_defines -prefix U_prefix`

See Also

`set design prefix (2)`

prefix_identifiers

Prefix the specified identifiers in the listed source files.

Syntax

```
string prefix_identifiers -file_list <file_list> -names <name_list>
-prefix <prefix> [-output_dir <output_dir>] [-index]
    string <file_list>
    string <name_list>
    string <prefix>
    string <output_dir>
```

Parameters

-file_list <file_list>	The list of files to be processed.
-names <name_list>	List of names for interfaces, packages, etc. to prefix.
-prefix <prefix>	The prefix to use for replacement.
-output_dir <output_dir>	The output directory to write to.
-index	Return a list of names which were matched in the file(s). Don't write files.

Description

This command prefixes the specified identifiers with the indicated prefix. The command is run automatically in coreConsultant and coreAssembler when design prefixing is enabled to perform prefixing of SystemVerilog interface and package names in configured RTL source files.

Examples

```
prefix_identifiers -file_list "file1 file2" -prefix myprefix -names
"intf1 intf2 pkg1" -out outDir
```

See Also

prefix_macros

Prefix all of the macros found in source files.

Syntax

```
string prefix_macros -file_list <file_list> [-prefix <prefix>]
[-output_dir <output_dir>] [-unprefix] [-custom_file_list <file_list>]
    string <file_list>
    string <prefix>
    string <output_dir>
```

Parameters

-file_list <file_list>	The list of files to be processed.
-prefix <prefix>	The prefix to use for replacement.
-output_dir <output_dir>	The output directory to write to.
-unprefix	If set generate a file to undo the prefixing in scratch.
-custom_file_list <file_list>	The list of files in which the macros will be prefixed. Macros defined in these files will not be prefixed.

Description

This command prefixes macros in the RTL files where macro prefixing is enabled.

In coreConsultant, by default a single prefix is used for RTL file macros. This command is automatically run when you complete SpecifyConfiguration and if a prefix is set and prefixing enabled the files will be prefixed.

In coreAssembler each components RTL file macros get a unique macro prefix which matches the file prefix and is based off the full component path. This command is automatically run when you complete SpecifySubsystem and if any components have prefixing enabled the files will be prefixed.

You can also manually prefix files using this command as long as the required SpecifyConfiguration or SpecifySubsystem activities have been completed. Note if PreventMacroPrefix was false files may already be prefixed.

Examples

See Also

`get_macro_prefix (2)`, `set_macro_prefix (2)`_____

[API Index](#) [Command Reference](#)

`prereq_activities_complete`

Check if prerequisite activities have completed yet!

Syntax

string **prereq_activities_complete** *activity*
 string *activity*

Parameters

activity Activity associated with the command

Description

Checks if an activity and its sub-process have completed.

Examples

`prereq_activity_complete` Synthesize

See Also

`launch_activity_subproc(2), define_activity_subproc_params(2),`
`report_activity_subproc(2), wait_for_activity_subproc(2)`

NAME

print_message_info

Prints information about diagnostic messages that have occurred or have been limited.

SYNTAX

```
string print_message_info
      [-ids id_list]
      [-summary]
```

Data Types

id_list list

ARGUMENTS

- ids *id_list* Specifies a list of message identifiers to report. Each entry can be a specific message or a glob-style pattern that matches one or more messages. If this option is omitted and no other options are given, then all messages that have occurred or have been limited are reported.
- summary Generates a summary of error, warning, and informational messages that have occurred so far.

DESCRIPTION

The **print_message_info** command enables you to print summary information about error, warning, and informational messages that have occurred or have been limited with the **set_message_info** command. For example, if the following message is generated, information about it is recorded:

```
Error: unknown command 'wrong_command' (CMD-005)
```

coreTools Command Reference Index

It is useful to be able to summarize all recorded information about generated diagnostic messages. Much of this can be done using the **get_message_info** command, but you need to know a specific message ID. By default, **print_message_info** summarizes all of the information. It provides a single line for each message that has occurred or has been limited, and one summary line that shows the total number of errors, warnings, and informational messages that have occurred so far. If an *id_list* is given, then only messages matching those patterns are displayed. If **-summary** is given, then a summary is displayed.

Using a pattern in the *id_list* is intended to show a specific message prefix, for example, "CMD*". Note that this does not show all messages with that prefix. It shows only the messages that have occurred or have been limited.

EXAMPLES

The following example uses **print_message_info** to show a few specific messages:

```
prompt> print_message_info -ids [list "CMD*" APP-99]
```

Id	Limit	Occurrences	Suppressed
CMD-005	0	7	2
APP-99	1	0	0

At the end of the session, you might want to generate some information about a set of interesting messages, such as how many times each occurred (which includes suppressions), how many times each was suppressed, and whether a limit was set for any of them. The following example uses **print_message_info** to get this information:

```
prompt> print_message_info
```

Id	Limit	Occurrences	Suppressed
CMD-005	0	12	0
APP-027	100	150	50
APP-99	0	1	0

Diagnostics summary: 12 errors, 150 warnings, 1 informational

Note that the suppressed count is not necessarily the difference between the limit and the occurrences, since the limit can be dynamically changed with the **set_message_info** command.

SEE ALSO

get_message_info(2)
set_message_info(2)
suppress_message(2)

NAME

print_suppressed_messages

Displays an alphabetical list of message IDs that are currently suppressed.

SYNTAX

```
string print_suppressed_messages
```

ARGUMENTS

The **print_suppressed_messages** command has no arguments.

DESCRIPTION

The **print_suppressed_messages** command displays all messages that you suppressed using the **suppress_message** command. The messages are listed in alphabetical order. You only can suppress informational and warning messages. The result of **print_suppressed_messages** is always the empty string.

EXAMPLES

The following example shows the output from the **print_suppressed_messages** command:

```
prompt> print_suppressed_messages  
No messages are suppressed
```

```
prompt> suppress_message {XYZ-001 CMD-029 UI-1}
```

```
prompt> print_suppressed_messages  
The following 3 messages are suppressed:  
CMD-029, UI-1, XYZ-001
```


SEE ALSO

`suppress_message(2)`
`unsuppress_message(2)`

NAME

printenv Prints the value of environment variables.

SYNTAX

```
string printenv
      [variable_name]
```

Data Types

<i>variable_name</i>	string
----------------------	--------

ARGUMENTS

variable_name Specifies the name of a single environment variable to print.

DESCRIPTION

The **printenv** command prints the values of the environment variables inherited from the parent process or set from within the application with the **setenv** command. When no arguments are specified, all environment variables are listed. When a single *variable_name* is specified, if that variable exists, its value is printed. There is no useful return value from **printenv**.

To retrieve the value of a single environment variable, use the **getenv** command.

EXAMPLES

The following examples show the output from the **printenv** command:

```
prompt> printenv SHELL
```

/bin/csh

```
prompt> printenv EDITOR  
emacs
```

SEE ALSO

getenv(2)
setenv(2)

NAME

printvar Prints the values of one or more variables.

SYNTAX

```
string printvar  
      [pattern]  
      [-user_defined | -application]
```

Data Types

pattern string

ARGUMENTS

pattern Prints variable names that match *pattern*. The optional *pattern* argument can include the wildcard characters * (asterisk) and ? (question mark). If not specified, all variables are printed.

-user_defined Shows only user-defined variables. This option is mutually exclusive with the **-application** option.

-application Shows only application variables. This option is mutually exclusive with the **-user_defined** option.

DESCRIPTION

The **printvar** command prints the values of one or more variables. If the *pattern* argument is not specified, the command prints out the values of application and user-defined variables. The **-user_defined** option limits the variables to those that you have defined. The **-application** option limits the variables to those created by the application. The two options are

coreTools Command Reference Index

mutually exclusive and cannot be used together.

Some variables are suppressed from the output of **printvar**. For example, the Tcl environment variable array *env* is not shown. To see the environment variables, use the **printenv** command. Also, the Tcl variable **errorInfo** is not shown. To see the error stack, use the **error_info** command.

EXAMPLES

The following command prints the values of all of the variables:

```
prompt> printvar
```

The following command prints the values of all application variables that match the pattern *sh*a**:

```
prompt> printvar -application sh*a*
sh_arch           = "sparcOS5"
sh_command_abbrev_mode = "Anywhere"
sh_enable_page_mode  = "false"
sh_new_variable_message = "false"
sh_new_variable_message_in_proc = "false"
sh_source_uses_search_path = "false"
```

The following command prints the value of the **search_path** variable:

```
prompt> printvar search_path
search_path       = ". /designs/newcpu/v1.6 /lib/cmos"
```

The following command prints the values of the user-defined variables:

```
prompt> printvar -user_defined
a                = "6"
designDir         = "/u/designs"
```

SEE ALSO

error_info(2)
printenv(2)
setenv(2)

NAME

`proc` - Create a Tcl procedure

SYNOPSIS

`proc` *name args body*

DESCRIPTION

The **proc** command creates a new Tcl procedure named *name*, replacing any existing command or procedure there may have been by that name. Whenever the new command is invoked, the contents of *body* will be executed by the Tcl interpreter. Normally, *name* is unqualified (does not include the names of any containing namespaces), and the new procedure is created in the current namespace. If *name* includes any namespace qualifiers, the procedure is created in the specified namespace. *Args* specifies the formal arguments to the procedure. It consists of a list, possibly empty, each of whose elements specifies one argument. Each argument specifier is also a list with either one or two fields. If there is only a single field in the specifier then it is the name of the argument; if there are two fields, then the first is the argument name and the second is its default value. Arguments with default values that are followed by non-defaulted arguments become required arguments; enough actual arguments must be supplied to allow all arguments up to and including the last required formal argument.

When *name* is invoked a local variable will be created for each of the formal arguments to the procedure; its value will be the value of corresponding argument in the invoking command or the argument's default value. Actual arguments are assigned to formal arguments strictly in order. Arguments with default values need

coreTools Command Reference Index

not be specified in a procedure invocation. However, there must be enough actual arguments for all the formal arguments that do not have defaults, and there must not be any extra actual arguments. Arguments with default values that are followed by non-defaulted arguments become de-facto required arguments, though this may change in a future version of Tcl; portable code should ensure that all optional arguments come after all required arguments.

There is one special case to permit procedures with variable numbers of arguments. If the last formal argument has the name `then` a call to the procedure may contain more actual arguments than the procedure has formal arguments. In this case, all of the actual arguments starting at the one that would be assigned to **args** are combined into a list (as if the **list** command had been used); this combined value is assigned to the local variable **args**.

When *body* is being executed, variable names normally refer to local variables, which are created automatically when referenced and deleted when the procedure returns. One local variable is automatically created for each of the procedure's arguments. Other variables can only be accessed by invoking one of the **global**, **variable**, **upvar** or **namespace upvar** commands. The current namespace when *body* is executed will be the namespace that the procedure's name exists in, which will be the namespace that it was created in unless it has been changed with **rename**.

The **proc** command returns an empty string. When a procedure is invoked, the procedure's return value is the value specified in a **return** command. If the procedure does not execute an explicit **return**, then its return value is the value of the last command executed in the procedure's body. If an error occurs while executing the procedure body, then the procedure-as-a-whole will return that same error.

EXAMPLES

This is a procedure that takes two arguments and prints both their sum and their product. It also returns the string to the caller as an explicit result.

```
proc printSumProduct {x y} {
    set sum [expr {$x + $y}]
    set prod [expr {$x * $y}]
    puts "sum is $sum, product is $prod"
    return "OK"
}
```

This is a procedure that accepts arbitrarily many arguments and prints them out, one by one.

```
proc printArguments args {
    foreach arg $args {
```


coreTools Command Reference Index

```
    puts $arg  
  }  
}
```

This procedure is a bit like the **incr** command, except it multiplies the contents of the named variable by the value, which defaults to **2**:

```
proc mult {varName {multiplier 2}} {  
    upvar 1 $varName var  
    set var [expr {$var * $multiplier}]  
}
```

SEE ALSO

info(n), unknown(n)

KEYWORDS

argument, procedure

NAME

proc_args Displays the formal parameters of a procedure.

SYNTAX

```
string proc_args
      proc_name
```

Data Types

<i>proc_name</i>	string
------------------	--------

ARGUMENTS

proc_name Specifies the name of the procedure.

DESCRIPTION

The **proc_args** command is used to display the names of the formal parameters of a user-defined procedure. This command is essentially a synonym for the Tcl builtin command **info** with the *args* argument.

EXAMPLES

This example shows the output of **proc_args** for a simple procedure:

```
prompt> proc plus {a b} { return [expr $a + $b] }

prompt> proc_args plus
a b

prompt> info args plus
a b

prompt>
```

SEE ALSO

info(2)
proc(2)
proc_body(2)

NAME

proc_body Displays the body of a procedure.

SYNTAX

```
string proc_body  
      proc_name
```

Data Types
 proc_name string

ARGUMENTS

proc_name Specifies the name of the procedure.

DESCRIPTION

The **proc_body** command is used to display the body (contents) of a user-defined procedure. This command is essentially a synonym for the Tcl builtin command **info** with the *body* argument.

EXAMPLES

This example shows the output of **proc_body** for a simple procedure:

```
prompt> proc plus {a b} { return [expr $a + $b] }  
  
prompt> proc_body plus  
      return [expr $a + $ b]  
  
prompt> info body plus  
      return [expr $a + $ b]  
  
prompt>
```

SEE ALSO

`info(2)`
`proc(2)`
`proc_args(2)`

`process_solution_files`

Explicitly process solution files for the indicated phase.

Syntax

```
string process_solution_files -phase <phase>  
      string <phase>
```

Parameters

<code>-phase</code>	Phase of files to be sourced. (Values: Instances,
<code><phase></code>	Interfaces)

Description

This command is used to cause DW IP Solution TCL files to be processed for the indicated phase. This command is only needed when instantiating solutions within hierarchical components. When instantiated at the top (/ context) it is not required as all of the solution files are sourced automatically by coreAssembler.

Use this command with `-phase Interfaces` to cause the solution level interface connections to be made. This will then enable manual connection of interfaces above the hierarchies where the solution is instantiated.

Examples

Give a solution instantiated in hierarchy `/u_soln`, cause solution interface connections to occur.

```
prompt> process_solution_files -phase Interfaces
```

See Also

propagate_memory_map

Propagate the memory maps visible from an exported slave interface to that slave

Syntax

```
string propagate_memory_map -interface <interface> -map <memory map name>
[-address <base address>]
    string <interface>
    string <memory map name>
    string <base address>
```

Parameters

-interface <interface>	The exported interface to propagate the memory map to.
-map <memory map name>	The name of the new memory map.
-address <base address>	Override base address for 1 slave

Description

This command is used to propagate a memory map upward within a subsystem. The upward propagation is only supported when the subsystem contains an interface exported as a slave. Specifically, when a portion of the subsystem can be viewed as a slave in a higher level context. In this situation, the memory maps visible from that slave interface can be merged into a single subsystem-level memory map.

Examples

See Also

protected_proc

Wrapper around proc to make a protected (permanent but visible) proc.

Syntax

```
string protected_proc procName procArgs procBody [procHelp] [procArgDefn]
    string procName
    string procArgs
    string procBody
    string procHelp
    string procArgDefn
```

Parameters

procName	procedure name
procArgs	procedure arguments
procBody	procedure body
procHelp	procedure help
procArgDefn	procedure argument definitions

Description

This command is used to create a proc which is user visible but also permanent (cannot be redefined). It is defined such that it can be called multiple times without issue so is useful for defining procs that need to be permanent but whose definitions might get sourced multiple times (e.g. plugin procedures).

The optional help argument can be used to define help for the command if the command uses `parse_proc_args`. The optional argument definition option expects input consistent with the `-define_args` argument to the `define_proc_attributes` command.

If your proc needs to be permanent and hidden use `permanent_proc` instead.

Examples

Example definition for a simple protected proc:

```
protected_proc myProc {myProcArg1 myProcArg2} {
echo "Arg1 is $myProcArg1"
echo "Arg2 is $myProcArg2"
} {Dump input arguments.}
```

See Also

permanent_proc

pt_shellVariable

Variable settings to be used for pt_shell. Example: set_design_attribute {pt_shellVariable[varName]} value.

Definition

Type: string

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This subscripted attribute can be used to set variables each time pt_shell is run for designs in the core or subsystem. If a design does not have an explicit setting for a subscript the value is inherited from its parent design. The subscript of the attribute is the variable name and the value is the value that the variable will be set to in pt_shell.

You can set non-application variables by prefixing the attribute subscript with the text "internal:". If the subscript has this prefix then setting the variable in the tool will be done with the Tcl "set" command instead of the "set_app_var" command.

Examples

Specify some variables that are to be applied at setup each time pt_shell is run.

```
coreConsultant> current_design [get_top_design]
coreConsultant> set_design_attribute {pt_shellVariable[my_var]} my_value
coreConsultant> set_design_attribute
{pt_shellVariable[internal:my_int_var]} my_value
coreConsultant> set_design_attribute {pt_shellVariable[null_var]} ""
coreConsultant> set_design_attribute {pt_shellVariable[list_var]} [list A
list of values]
```

See Also

[set_design_attribute \(2\)](#), [dc_shellVariable \(2\)](#), _____

pt_shellVariableComment

Comment to be issued for the corresponding subscript of pt_shellVariable.

Definition

Type: string

Flags: subscripted

Default value: =InheritValue up {}

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

The pt_shellVariableComment attribute can be used to output a comment to document why the corresponding pt_shellVariable is being used. The comment will be put into the file where the variable is set.

Examples

Set variable to be used in pt_shell, and specify a comment.

```
set_design_attribute {pt_shellVariable[myVar]} myValue
set_design_attribute {pt_shellVariableComment[myVar]} "Sample comment for
setting myVar."
```

See Also

[pt_shellVariable \(3\)](#)

NAME

`puts` - Write to a channel

SYNOPSIS

`puts ?-nonewline? ?channelId? string`

DESCRIPTION

Writes the characters given by *string* to the channel given by *channelId*.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdout** or **stderr**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension. The channel must have been opened for output.

If no *channelId* is specified then it defaults to **stdout**. **Puts** normally outputs a newline character after *string*, but this feature may be suppressed by specifying the **-nonewline** switch.

Newline characters in the output are translated by **puts** to platform-specific end-of-line sequences according to the current value of the **-translation** option for the channel (for example, on PCs newlines are normally replaced with carriage-return-linefeed sequences. See the **fconfigure** manual entry for a discussion on ways in which **fconfigure** will alter output.

Tcl buffers output internally, so characters written with **puts** may not appear immediately on the output file or device; Tcl will normally delay output until the buffer is full or the channel is closed. You can force output to appear immediately with the **flush** command.

When the output buffer fills up, the **puts** command will normally block until all the buffered data has been accepted for output by the operating system. If *channelId* is in nonblocking mode then the **puts** command will not block even if the operating system cannot accept the data. Instead, Tcl continues to buffer the data and writes it in the background as fast as the underlying file or device can accept it. The application must use the Tcl event loop for nonblocking output to work; otherwise Tcl never finds out that the file or device is ready for more output data. It is possible for an arbitrarily large amount of data to be buffered for a channel in nonblocking mode, which could consume a large amount of memory. To avoid wasting memory, nonblocking I/O should normally be used in an event-driven fashion with the **fileevent** command (do not invoke **puts** unless you have recently been notified via a file event that the channel is ready for more output data).

EXAMPLES

Write a short message to the console (or wherever **stdout** is directed):

```
puts "Hello, World!"
```

Print a message in several parts:

```
puts -nonewline "Hello, "  
puts "World!"
```

Print a message to the standard error channel:

```
puts stderr "Hello, World!"
```

Append a log message to a file:

```
set chan [open my.log a]  
set timestamp [clock format [clock seconds]]  
puts $chan "$timestamp - Hello, World!"  
close $chan
```

SEE ALSO

[file\(n\)](#), [fileevent\(n\)](#), [Tcl_StandardChannels\(3\)](#)

KEYWORDS

channel, newline, output, write

NAME

`pwd` - Return the absolute path of the current working directory

SYNOPSIS

`pwd`

DESCRIPTION

Returns the absolute path name of the current working directory.

EXAMPLE

Sometimes it is useful to change to a known directory when running some external command using `exec`, but it is important to keep the application usually running in the directory that it was started in (unless the user specifies otherwise) since that minimizes user confusion. The way to do this is to save the current directory while the external command is being run:

```
set tarFile [file normalize somefile.tar]
set savedDir [pwd]
cd /tmp
exec tar -xf $tarFile
cd $savedDir
```


SEE ALSO

`file(n)`, `cd(n)`, `glob(n)`, `filename(n)`

KEYWORDS

working directory

NAME

py_eval Evaluate python code. This is an optional feature, not all applications support Python.

SYNTAX

```
string py_eval -file <path>
      code [-verbose]
```

```
string    <path>
string    code
string    dict
```

ARGUMENTS

-file <path> Evaluate python code in the specified file. This option cannot be specified with code.

-locals dict Use the supplied dictionary value as the Python locals value. When specified the Python code evaluates as if it is an unnamed function definition. When this option is specified you must use the Python "global" keyword to indicate that a variable name refers to a global, rather than a local variable. This option allows you to "pass" data from Tcl into your Python script directly.

code Evaluate python statements embed within a Tcl script. This option cannot be specified with -file.

-verbose Echo commands and display results during evaluation. This option is only legal when combined with -file.

DESCRIPTION

This command evaluates Python code in the application. The Python language uses the "snps" module to control tool behavior. That module is implicitly imported into the Python global name space.

When this command is run from within Python (i.e. `cmd.py_eval(...)`) the specified code (or file) is evaluated in the current module namespace. When evaluated from Tcl the code is evaluated in global namespace.

The Python snps module

Synopsys applications add a module called "snps" into the embedded Python interpreter. The snps module defines classes and objects that are used to interact with application builtin commands. Please see the following man pages for more details:

snps.cmd Python object that provides access to application commands.

snps.collection Python class that provides access to Synopsys collection values.

snps.value Python class that provides access to application command return values.

snps.redirect Python class used to redirect application output.

The Python interactive console

When the app_var `sh_language` is set to "python", the interactive language of the application is Python instead of Tcl. In this mode all input must be a valid Python expression or statement. When the input language is Python, the prompt will include "-py" to indicate this. The following example shows how to change the input language from Tcl, into Python, and then back again. Notice the changes to the prompt display:

```
fc_shell> set_app_var sh_language python
fc_shell-py> cmd.set_app_var('sh_language', 'tcl')
fc_shell>
```

The interactive Python console has a few extensions compared to the plain Python interpreter:

history() This command prints the interactive command history and is an alias for the `cmd.history()` command. The tool keeps a

coreTools Command Reference Index

separate history for interactive Python and Tcl. In Python mode you may evaluate commands from the history using the traditional "!" syntax.

source(fileName)

This command is an alias for the `cmd.py_eval(file=fileName)` command.

EXAMPLES

Evaluate the Python file in the global namespace.

```
prompt> py_eval -file pyFile.py
```

Find `pyfile.py` in the Python `sys.path`. Creates a new module (and namespace). If code in `pyFile.py` depends on features of the `snps` module, those must be imported manually into the module namespace.

```
prompt> py_eval {import pyFile.py}
```

The return value of the `py_eval` can be set by assigning to the `cmd.result` property. In the following example the Tcl variable `"x"` holds the value `"10"`.

```
prompt> set x [py_eval {cmd.result = 10}]
```

SEE ALSO

`sh_language(3)` `snps.cmd(2)` `snps.collection(2)`
`snps.value(2)` `snps.redirect(2)`

NAME

quit Exits the shell.

SYNTAX

string **quit**

ARGUMENTS

The **quit** command has no arguments.

DESCRIPTION

The **quit** command exits from the application. It is basically a synonym for the **exit** command with no arguments.

EXAMPLES

The following example exits the current session:

```
prompt> quit
```

SEE ALSO

`exit(2)`

rand_proc

Create SV function which is equivalent of tcl proc which will be used by randomize_parameters command. Generally created in coreBuilder via plugin files. Can be overridden in cC and cA

Syntax

```
string rand_proc -tcl_proc tcl_proc -sv_function sv_function
[-internal_args internal_args] [-force] [-replace_external_args
replace_external_args] body
    string tcl_proc
    string sv_function
    string internal_args
    string replace_external_args
    string body
```

Parameters

-tcl_proc <i>tcl_proc</i>	Name of the tcl proc to convert equivalent SV function
-sv_function <i>sv_function</i>	Name of equivalent SV function
-internal_args <i>internal_args</i>	Parameters which are used inside the tcl proc and not passed in argument list must be provided as arguments in SV function. These will be added to arguments passed with tcl proc in general cases
-force	Allow to override existing SV proc implemented in plugin files
-replace_external_args <i>replace_external_args</i>	Option to replace, remove external arguments used in tcl procs. Position, replaced name pair . Example [check_data_width item]. rand_proc -tcl_proc {check_data_width} -replace_external_args {{0 ""}}
<i>body</i>	Actualy SV function with function name body and return value. This will be a replaced in randomize_parameters whenever a tcl_proc of same name is encountered

Description

rand_proc command

Use rand_proc to define System Verilog code implementing a function

within a TCL file .

- Provide implementation of TCL procedures co-located with TCL definitions
- Function definitions stored in coreKit and automatically included in generated SV code
- Automatic substitution into expressions when a rand_proc definition is found during randomize_parameters
- Arguments must either be static or parameter references

Note :

- The command **MUST** be used in coreBuilder during packaging. The command can be overridden in coreConsultant by -force option
- If the user wants to package the rand_proc along with the coreKit in that case the user can specify the rand_proc command in the TCL intent search (refer activity parameter IntentSearchPath).

rand_proc example (coreBuilder)

File : plugin.tcl

```
proc myMin {A B} {return [expr {($A < $B) ? $A : $B}]}
```

```
rand_proc -tcl_proc myMin \
-sv_function {function longint myMin(longint A; longint B);
return((A>B)?A:B);
endfunction}
<myIP>_cc_constants.tcl
```

```
//reuse-pragma attr DefaultValue [myMin @P1 @P2] + 1
`define P0 4'b0
```

Auto-generated SV code for constraints

```
class `CT_BASE_CLASS;
....
....
constraint P0_constraint {
// DefaultValue Expr
P0 == myMin(P1, P2) + 1;
}
....
....
```

```
function longint myMin(longint A; longint B);
return((A>B)?A:B);
endfunction
endclass
```


rand_proc example (coreConsultant)

```
rand_proc -force -tcl_proc myMin -sv_function myMin \
{
function longint myMin(longint A; longint B);
return((A>B)?A:B);
endfunction}
```

TCL to SV conversion requirements

- ALL the parameters used inside the SV functions MUST be passed as arguments
- Make sure you remove item and other related tcl arguments which are not supported by SV
- There MUST be a return type for SV function
- User MUST make sure all parameters in argument list are available in randomize_parameters . If some arguments doesnt need to be randomized then pass them in -default list

Adding Internal Arguments (-internal_args) If a TCL proc contains get_configuration_parameter calls, then that becomes an argument to SV function.

So that needs to be passed along with current set of arguments in TCL proc. See the following example

DWC_GMAC :

```
set p1 [find_item -type param CSR_DATAWIDTH]
get_attribute $p1 -attr RandDefaultValue
CSR_DATAWIDTH == ((@CSR_PORT==3) ? @DATAWIDTH : (@CSR_PORT!=2) ? 32
: [getCaCSR_datawidth "%item"]);
```

#cC_plugin

```
proc getCaCSR_datawidth {param} {
set val 32
if { [string equal $::shell_activity_mode "assembler"] == 1 } {
eval_in_component $param {
set val [get_configuration_parameter DATAWIDTH]
}
}
return $val
}
```

Here DATAWIDTH is used inside the proc. So the rand_proc command will be similar to

```
rand_proc -tcl_proc getCaCSR_datawidth -sv_function
getCaCSR_datawidth -internal_args {DATAWIDTH} -replace_external_args
{{0 ""}} \
{
```

coreTools Command Reference Index

```
function int getCaCSR_datawidth(longint DATAWIDTH);
int tempDW ;
tempDW = 32;
if (DATAWIDTH > tempDW)
return DATAWIDTH;
else
return tempDW;
endfunction
}
```

```
randomize_parameters -randomize {CSR_DATAWIDTH}
```

Output :

```
class 'CT_BASE_CLASS
```

```
...
```

```
...
```

```
// Random Constraint
constraint CSR_DATAWIDTH_constraint {
// Enabled Expr
en_CSR_DATAWIDTH == (( ( CSR_PORT==2) ) );
if (!en_CSR_DATAWIDTH) {
// DefaultValue Expr
CSR_DATAWIDTH == ((CSR_PORT==3) ? DATAWIDTH : (CSR_PORT!=2) ? 32 :
getCaCSR_datawidth(DATAWIDTH));
} else {
// CheckExpr
(((CSR_PORT!=2)||((CSR_SLV_CLK!=0)||((CSR_DATAWIDTH<=DATAWIDTH)))));
}
}
```

```
function int getCaCSR_datawidth(longint DATAWIDTH);
int tempDW ;
tempDW = 32;
if (DATAWIDTH > tempDW)
return DATAWIDTH;
else
return tempDW;
endfunction
```

```
endclass
```

NOTE : -internal_args always preceeds external args

Replacing external arguments (-replace_external_args)

If a tcl proc has arguments which needs to be replaced or removed, then thats done using -replace_external_args Consider the example showed above. The item needs to be removed in this case.

Format : `-replace_external_args { {<POS> <PARAM/Empty String>} }`

Example : `DWC_GMAC`

```
set p1 [find_item -type param CSR_DATAWIDTH]
get_attribute $p1 -attr RandDefaultValue
CSR_DATAWIDTH == ((@CSR_PORT==3) ? @DATAWIDTH : (@CSR_PORT!=2) ? 32
: [getCaCSR_datawidth "%item"]);
```

```
rand_proc -tcl_proc getCaCSR_datawidth -sv_function
getCaCSR_datawidth -replace_external_args {{0 ""}} -internal_args
{DATA_WIDTH} .....
```

Where : 0 : item is at position 0. Its replaced with empty string ""

Internal arguments , External arguments precedence

When `-internal_args` and `-replace_external_args` are used, the order is , the arguments provided by `-internal_args` precedes the args provided in tcl proc

Example Consider the following proc using C and D

```
proc myMin {A B} {
set c_val [get_configuraiton_parameter C]
set d_val [get_configuration_parameter D]
return [expr {($A < $B) ? $c_val : $d_val}]
rand_proc -tcl_proc myMin -sv_function myMin -internal_args {C D}
.....
SV output
int function myMin(int C, ind D, int A, int B) ;
....
...
endfunction
```

Examples

See Also

[randomize parameters](#)

randomize_parameters

randomize the parameters with the help of VCS constraints solver

Syntax

```
string randomize_parameters [-randomize randomize] [-randc randomize]
[-enable enabled] [-disable disabled] [-fixed fixed] [-default default]
[-follow] [-iterations iterations] [-user user] [-files user] [-seed
seed] [-svOut svOut] [-writeOnly] [-comment_procs comment_procs]
[-vcs_compile vcs_compile] [-vcs_exec vcs_execute] [-report_procs]
[-coverage_threshold coverage] [-attempts attempts] [-maxconfig ]
[-initialize_coverage initialize_config_path] [-urg_args ]
[-prepend_to_config ]
    string randomize
    string enabled
    string disabled
    string fixed
    string default
    string iterations
    string user
    string seed
    string svOut
    string comment_procs
    string vcs_compile
    string vcs_execute
    string coverage
    string attempts
    string
    string initialize_config_path
```

Parameters

-randomize <i>randomize</i>	Randomize the given parameters using 'rand'. Parameters may or may not be enabled in result
-randc <i>randomize</i>	Randomize the given parameters using 'randc'. Parameters may or may not be enabled in result
-enable <i>enabled</i>	Parameters are guaranteed to be enabled, and are also randomized
-disable <i>disabled</i>	Parameters are guaranteed to be disabled and take default values
-fixed <i>fixed</i>	Parameters in the list that retain their current values irrespective of the state (Enabled or Disabled)

coreTools Command Reference Index

-default default	Parameters that should maintain their default value, could be enabled or disabled
-follow	Randomize all parameters in the dependency tree of all parameters on the command line
-iterations iterations	How many configurations to generate (default is 1)
-user user	params for which all constraints are user defined. Tool will not create any base constraint
-files user	Takes a .lst file or sv files with path containing user constraints and coverage
-seed seed	Seed value for VCS to enable duplicate results. A generated seed will be retained in logs which can be reused
-svOut svOut	Generated SV file Name
-writeOnly	Write out the scripts but dont execute.
-comment_procs comment_procs	Specify proc names to be commented. Specify "all" to comment all procs This option should be used with caution as it modifies the definition of parameters or their dependencies by removing references to the procs that are commented out. This can lead to generation of invalid configurations.
-vcs_compile vcs_compile	VCS compile time options
-vcs_exec vcs_execute	VCS simulation options
-report_procs	Generates report of TCL procedures that are used in attribute expressions. It also writes out diffrent attribute specific lists of unmodeled parameters
-coverage_threshold coverage	Expected coverage value for multiple unique configurations (default is 99).
-attempts attempts	Maximum number of unsuccessful attempts in a row for generating next unique configuration (default is 99.99 or 0.25% of iterations).
-maxconfig	Generate maximum number of unique randomized configurations for given parameters
-initialize_coverage initialize_config_path	Path contains fixed configurations which will be used to initialize the coverage model. Supported config file extensions are .tcl, .config and .tbc.
-urg_args	Options to be passed into Synopsys' Unified Report Generator

-prepend_to_config	list of coretools commands which prepends to each set of configurations in config.log file
--------------------	--

Description

randomize_parameters The randomize_parameters command is used to generate random configurations of the design parameters using the System Verilog and VCS Constraints solver. This feature is intended to replace the current perl based random configs generation.

The command

- Takes all the parameters given in input list and generates its entire dependency tree based on CheckExpr , DefaultValue , Enabled, MinValue and MaxValue attributes
- Generates SV constraints file and randomizes parameters using VCS
- Compiles , Simulates and generates a Coverage Report for parameters given in the input list
- Reads the simulation log and generates and returns a list of lists containing <PARAM VALUE> pairs. list of list is , one list of <PARAM VALUE> pairs of parameters for one iteration.

Features

- SV constraints generation of the given parameters and its dependencies
- Randomize the parameters with random behaviour (-random)
- Ensure certain parameters can take fixed or default values (-fixed , -default)
- Ensure certain parameter can always be kept enabled or disabled (-enabled, -disabled)
- Ability to randomize the dependent parameters (not given in user list) or keep them at default state (-follow)
- Entire SV setup generated is available for analysis
- Introduced Rand* Attributes (RandEnabled RandCheckExpr RandDefaultValue RandMinValue RandMaxValue) which are SV format alike, for debugging and can be overwritten by user
- Can generate multiple iterations for multiple configurations. (-iterations)
- Ability to reproduce the same random configuration set (-seed)
- Support for User defined Constraints (-files)
- Support for User to override the complete constraints. The tool will not generate any constraints for parameters provided in -user. The SV constraints should be provided in user files via -files (-user)
- Support to increase the ntbv_solver_cpu_limit
- Write out SV files only for Analysis (-writeOnly)
- Prepend configuration commands required before each constraint set generated

coreTools Command Reference Index

Prerequisites The `randomize_parameters` command is supported in `coreConsultant`. To generate parameters one needs to take a default IP core (say in `coreConsultant`). You might need to retain certain configured parameters values (based on your requirement) such that they are enabled, take current default value, disabled or take any pre-fixed values (`set_configuration_parameter`). To ensure this behavior is retained in your random generations, you would have to preconfigure your core before calling `randomize_parameters`.

For Example

- ◆ Certain parameters may be required to be enabled so make sure they are enabled in `coreConsultant` before calling `randomize_parameters` and put them in `-enabled` list.
- ◆ Certain parameters may be required to take fixed values so configure those parameters before calling `randomize_parameters` and put them in `-fixed` list.

NOTE : The `coreConsultant` configuration invoked to run `randomize_parameters` is generally in default state. If you intend to change default value of any parameter using `set_configuration_parameter` and if that happens to be used in `randomize_parameters` directly or via dependency list , then you must ensure its value is in `-fixed` list else the command will issue an Error

Output Format

The output of the `randomize_parameters` command is a list of list of `<PARAM VALUE>` pairs.

Example : DW_apb_timer

```
randomize_parameters -randomize APB_DATA_WIDTH -enable  
NUM_TIMERS -iterations 2
```

Output

```
{{APB_DATA_WIDTH 32} {NUM_TIMERS 4}} {{APB_DATA_WIDTH 16}  
{NUM_TIMERS 3}}
```

Complete Random Mode (-random)

The parameters passed in `-random` list are completely randomized. The generated value is available as output only if the parameter gets enabled in a particular iteration.

Example : DWC_usb_31

```
randomize_parameters -randomize DWC_USB31_NUM_SSIC_PORTS  
SV constraint
```

```
// Random Constraint
constraint DWC_USB31_NUM_SSIC_PORTS_constraint {
// Enabled Expr
if (!en_DWC_USB31_NUM_SSIC_PORTS) {
// DefaultValueExpr
DWC_USB31_NUM_SSIC_PORTS == (0);
} else {
// CheckExpr
( DWC_USB31_NUM_SSIC_PORTS!=0)-> (check_license());
(DWC_USB31_MODE ==0)-> (DWC_USB31_NUM_SSIC_PORTS <= 1);
( DWC_USB31_NUM_SSIC_PORTS !=0)-> (DWC_USB31_NUM_U3_ROOT_PORTS
>= DWC_USB31_NUM_SSIC_PORTS);
}
}
```

Configuring certain params to remain Enabled (-enable)

The -enable switch ensures that parameters in the -enable list will always be enabled after randomization. Before passing a parameter in -enable list , ensure that you explicitly enable the parameter if its disabled by default. The tool will Error out if a parameter is not enabled before passing to -enable list. The parameter which enables the parameter of interest , then MUST be kept in fixed list. See example below

Example : DWC_gmac

Scenario 1 Parameter to be passed in -enabled : ENDIAN_NESS.
This parameter is in Enabled State in default mode.

```
randomize_parameters -enabled ENDIAN_NESS
SV constraint
//Enabled constraint
constraint ENDIAN_NESS_constraint {
// Enabled Expr
en_ENDIAN_NESS == 1;
en_ENDIAN_NESS== 1;
}
```

Note : Ignore the occurrence of en_ENDIAN_NESS coming twice.
Once is explicitly set by tool and other is taken from Enabled Expr

Scenario 2 Parameter to be passed in -enabled : CSR_DATAWIDTH
This parameter is not enabled in default config mode. The parameter needs CSR_PORT to be set to 2 and requires CSR_PORT also to be passed in fixed list set_configuration_parameter CSR_PORT 2

```
randomize_parameters -enabled CSR_DATAWIDTH -fixed CSR_PORT
```


SV constraint

```
// Enabled constraint
constraint CSR_DATAWIDTH_constraint {
// Enabled Expr
en_CSR_DATAWIDTH == (( ( CSR_PORT==2) ) );
en_CSR_DATAWIDTH == 1;
// CheckExpr
(((CSR_PORT!=2)||((CSR_SLV_CLK!=0)||((CSR_DATAWIDTH<=DATAWIDTH)))));
}
```

// Fixed constraint

```
constraint CSR_PORT_constraint {
// Enabled Expr
en_CSR_PORT == 1;
en_CSR_PORT == 1;
CSR_PORT == 2;
// CheckExpr
((GMAC==0) ? (CSR_PORT!=0&&CSR_PORT!=3) : (GMAC ==4) ?
(CSR_PORT!=0) : (CSR_PORT!=2&&CSR_PORT!=3));
}
```

Configuring certain params to take SET value (-fixed)

The -fixed option ensures the following values are retained after randomization for parameters in -fixed list

- 1) SET value of parameter which is set via set_configuration_parameter.
- 2) Default value of a parameter
- 3)The value in fixed list can be Enabled or Disabled

Example : DWC_gmac

```
randomize_parameters -fixed CSR_PORT
SV Constraint
// Fixed constraint
constraint CSR_PORT_constraint {
// Enabled Expr
en_CSR_PORT == 1;
en_CSR_PORT == 1;
CSR_PORT == 2;
// CheckExpr
}
```

Configuring certain params to remain in Default State (-default)

coreTools Command Reference Index

The -default option will ensure params in -default list take DefaultValue or DefaultValue Expr after randomization. This is useful for

1) Getting Coverage on ReadOnly Params

2) Getting Coverage on Params in Default State

NOTE : The -default value can be applied to Both Enabled or Disabled parameters without any restriction

Example : DWC_gmac

```
randomize_parameters -default {CSR_PORT DATAWIDTH}
SV Constraint
// Default Constraint
constraint CSR_PORT_constraint {
// Enabled Expr
en_CSR_PORT == 1;
// DefaultValue Expr
CSR_PORT == ((GMAC==0) ? 2 : (GMAC==4) ? 3 : 0) ;
if (en_CSR_PORT) {
// CheckExpr
}
}

// Default Constraint
{
// Enabled Expr
en_DATAWIDTH == 1;
// DefaultValue Expr
DATAWIDTH == (32);
}
```

Configuring certain params to remain Disabled (-disable) The -disable option ensures params in the -disable list are in disables state after randomization.

Note : An Enabled Parameter cannot be provided with -disable. The command will give an Error

Example : DWC_gmac

```
randomize_parameters -disable {GOW}
SV Constraint
// Disabled constraint
constraint GOW_constraint {
// Enabled Expr
en_GOW == (( ( GPIO_EN) ) ) ;
en_GOW == 0;
```

```
// DefaultValue Expr
GOW == (GPIO_EN ? 1 : 0) ;
}
```

Configuring dependent params (-follow)

In Normal flow when a parameter is randomized using any (-randomize, -fixed, -enable , -disable) options, the dependent parameters always take Default Value. The -follow option ensures all the parameters which are under the dependency tree of parameters provided in any (-randomize, -fixed, -enable , -disable) list , will generate a Complete Random Value.

Note : If a Dependent Parameter of a parameter in any input list (-randomize, -fixed, -enable , -disable) is found to be in the input list , then it will respect its input format and not -follow or Default format.

Example : GOW Is Enabled at GPIO_EN. It means GPIO_EN becomes a dependent parameter.

If Both are provided as inputs (randomize_parameters -randomize GOW -enable GPIO_EN) , then GPIO_EN will take the format of -enable in this case

Example : DWC_GMAC

Without (-follow) option

```
randomize_parameters -randomize {GOW}
SV Constraint
```

```
// Random Constraint
constraint GOW_constraint {
// Enabled Expr
en_GOW == (( ( GPIO_EN) ) ) ;
if (!en_GOW) {
// DefaultValue Expr
GOW == (GPIO_EN ? 1 : 0) ;
} else {
// CheckExpr
(GPIO_EN ? GOW>0 : GOW==0);
}
}
```

```
// Default Constraint
constraint GPIO_EN_constraint {
// Enabled Expr
```

coreTools Command Reference Index

```
en_GPIO_EN == (( 1 ) );  
// DefaultValue Expr  
GPIO_EN == (0);  
}
```

With (-follow) option

```
randomize_parameters -randomize {GOW} -follow  
SV Constraint
```

```
// Random Constraint  
constraint GOW_constraint {  
// Enabled Expr  
en_GOW == (( ( GPIO_EN ) ) );  
if (!en_GOW) {  
// DefaultValue Expr  
GOW == (GPIO_EN ? 1 : 0);  
} else {  
// CheckExpr  
(GPIO_EN ? GOW>0 : GOW==0);  
}  
}
```

```
// Random Constraint  
constraint GPIO_EN_constraint {  
// Enabled Expr  
en_GPIO_EN == (( 1 ) );  
if (!en_GPIO_EN) {  
// DefaultValue Expr  
GPIO_EN == (0);  
}  
// Ranges  
GPIO_EN >= (0);  
GPIO_EN <= (1);  
}
```

Prepend commands to configuration set (-prepend_to_config) The

-prepend_to_config option will prepend any command to configuration set in config.log file. This helps user to pre-set any configurations or any other commands required before setting the configuration parameters generated by randomize parameters For Example :

The output in config.log file will look like this
Tracking: attempt=002 uCoverage=44.666668%

```
-----  
CONSTRAINT SET -1  
-----
```

----- Prepend Start ---

set_configuration_parameter DATA_WIDTH 16

set_configuration_parameter ENABLE_PROFILE 1

set_prime_profile -tag ProfileE

----- Prepend End ---

set_configuration_parameter ENABLE_INTERRUPT 0x1

set_configuration_parameter INTERRUPT_POLARITY 0x1

set_configuration_parameter PROFILE_PARAM0 1

set_configuration_parameter RAND_PARAM0_DEP_LOCKED 2

set_configuration_parameter RAND_PARAM0_DEP_RO 2

Note : The configuration values from -prepend_to_config will not be appened to the output list returned by randomize_parameters command

Locked Parameter Example The Locked parameters in a configuration must first be locked using lock_parameter command. The locked parameters cannot be specified in -random list.

Example :

Lets consider the following example

- 1) X and Y are to be randomized
- 2) W is to be prepended to constraint set
- 3) Z is locked using locked_param and to be prepended to constraint set
- 4) check_license -authorize MY_LIC to be prepended to constraint set

randomize_parameters -randomize {X Y} -iterations 10

-prepend_to_config {{set_configuration_parameter W 5}

{lock_parameter Z} {check_license -authorize MY_LIC}}

Configuring report generation (-report_procs)

The -report_procs option with randomize_parameters command will generate two report files, named "proc_statistics.rpt" and "unmodeled_parameters.rpt". The information of unmodeled TCL procedures, which are participated in attribute expression, will be found in "proc_statistics.rpt" report. "unmodeled_parameters.rpt" report file will contain list of parameters for each attribute whose expressions refer to unmodeled TCL procedures in SV Constraint.

Configuring expected coverage (-coverage_threshold)

coreTools Command Reference Index

The `-coverage_threshold` option will set exit criteria for randomized configs generation. Randomization will be stopped when it will achieve required coverage. Default value is 99.

Configuring unique value generation (-attempts)

The `-attempts` option value will set the number of continuous unsuccessful attempts for randomization. If it hits maximum unsuccessful attempts in a row without improving coverage, randomization will be stopped. Default value is 99.99 or 0.25% of no of iterations.

Configuring unique value generation (-maxconfig)

The `-maxconfig` option sets an early exit criteria for random config generation. Randomization will stop if the number of successful configurations meets the count set by the user. If not specified then the exit criteria is based on other exit criterias like (`-iterations`), (`-coverage_threshold`) or (`-attempts`).

Configuring coverage initialization (-initialize_coverage)

The `-initialize_coverage` option accepts a path of available fixed configurations as input. To initialize coverage, internally it will create a header file `"config_header.vh"` which will contain multiple sets of param-value pair. Each param-value set will be populated after sourcing each config file and fetching the values of all parameters given for randomization. Finally, the coverage value, after sourcing that header file from simulation top file, will be used to initialize coverage model. Only the files having supported (`.tcl`, `.config`, `.tbc`) extension in the given path will be considered as config files. Also a report file `"initialize_coverage.rpt"` will be generated, which will show which configurations contribute to coverage.

Controlling Synopsys' Unified Report Generator (-urg_args)

The `-urg_args` option accepts a list of arguments that are passed to Synopsys' Unified Report Generator application. If no option is passed to `-urg_args` then the tool will pass `'-group suppress_implicit_cps'` to Unified Report Generator application. If the user has specified custom cover points via parameter attribute `'URGCoverpointBinText'` or `'URGCoverpointBins'` in that case the tool will append `'-group ratio'` to the Unified Report Generator arguments. The call to the `'urg'` application is made from the shell script `'cov.csh'` exported in the `<workspace>/export/rand` folder.

Invocation Examples

coreTools Command Reference Index

Single Iteration Randomize A and B and all dependent parameters

randomize_parameters -randomize {A B} -follow
Randomize A-D ensuring B is enabled, E disabled, F at default value

randomize_parameters -randomize {A C D} \
-enabled B -disable E -default F
Randomize A and B, and based on that result, C or D

```
array set result [randomize_parameters randomize {A B}]  
if {[info exists result(A)] && $result(A) > 2} {  
array set result [randomize_parameters -randomize C]  
} else {  
array set result [randomize_parameters -randomize D]  
}
```

Generate 10 configuration files

```
set params {A B C}  
set enabled {D E}  
set results [randomize_parameters -randomize $params \  
-enable $enabled \  
-iterations 10]  
set index -1  
set cmd "set_configuration_parameter %s %s"  
foreach result $results {  
set file config[incr index].tcl  
foreach {pName pValue} $results {  
redirect $file append {format $cmd $pName $pValue}  
}  
}
```

Rand* parameter attributes and its usage

The following attributes are introduced to support randomize_parameters command

- ◆ RandEnabled
- ◆ RandDefaultValue
- ◆ RandCheckExpr
- ◆ RandMinValue
- ◆ RandMaxValue

These attributes generate a constraint expression string which is used for SV constraint generation. The randomize_parameters makes use of these attrs to create constraints.

Example : DWC_gmac

```
set p [find_item -type param RXFIFO_SIZE]  
get_attribute $p -attr RandEnabled  
Output : en_RXFIFO_SIZE == (( ( @GMAC != 1) ) ) ;
```

coreTools Command Reference Index

Usage

- ◆ These attributes can be used for Users to Debug Issues during `randomize_parameters`
- ◆ User can **OVERRIDE** any `Rand*Attr` expr with its own SV expr. This is needed when exprs contain TCL procs or exprs are incorrectly coded. *NOTE* : The `randomize_parameters` command still needs `<PARAM>` in the `Rand*attr` for further processing
- ◆ The user `Rand*Attr` Expressions **MUST** be SV compliant except `<PARAM>` format

Overriding Rand*Attr Example (DWC_GMAC)

`CSR_DATAWIDTH` has the following `DefaultValue` Expr

```
set p1 [find_item -type param CSR_DATAWIDTH]
get_attribute $p1 -attr RandDefaultValue
Output CSR_DATAWIDTH == ((@CSR_PORT==3) ? @DATAWIDTH :
(@CSR_PORT!=2) ? 32 : [getCaCSR_datawidth "%item"]);
```

Override with following

```
set str {CSR_DATAWIDTH == ((\@CSR_PORT==3) ? \@DATAWIDTH :
(\@CSR_PORT!=1) ? 32 : [getCaCSR_datawidth "%item"]);}
set_attribute $p1 -attr RandDefaultValue -value $str
Final SV Constraint
```

```
randomize_parameters -randomize {CSR_DATAWIDTH}
```

```
// Random Constraint
constraint CSR_DATAWIDTH_constraint {
// Enabled Expr
en_CSR_DATAWIDTH == (( ( CSR_PORT==2) ) );

if (!en_CSR_DATAWIDTH) {
// DefaultValue Expr
CSR_DATAWIDTH == ((CSR_PORT==3) ? DATAWIDTH : (CSR_PORT!=1) ?
32 : getCaCSR_datawidth(item));
} else {
// CheckExpr
(((CSR_PORT!=2)||((CSR_SLV_CLK!=0)||((CSR_DATAWIDTH<=DATAWIDTH)))));

}
}
```

Reverting Back to Original Rand*Attr To Revert Back to original `RandDefaultValue` for `CSR_DATAWIDTH`

```
remove_attribute $p1 -attr RandDefaultValue
```

Creating User Constraints file

coreTools Command Reference Index

A user constraints file can be created for allowing users to set their own constraints. A sample user constraints file looks like this one

```
ifndef CT_USER_CLASS_INCLUDE <br>define CT_USER_CLASS_INCLUDE

'define CT_USER_CLASS gmac_user_class

class CT_USER_CLASS extendsCT_BASE_CLASS;

constraint user_const1 {
  GMAC inside {[0:2]};
}

covergroup my_cov_cg @(CT_EVENT);
coverpoint GMAC;
endgroup

function new();
super.new();
$display ("Initializing user class cg\n");
my_cov_cg = new();
endfunction
endclass
'endif
```

Requirements

The following requirements must be met

- ◆ The top level User class MUST have a `define CT_USER_CLASS`
The user class must extend the base class with `CT_BASE_CLASS`
- ◆ The `new()` functions MUST have `super.new()`
- ◆ The `covergroup` MUST be instantiated
- ◆ The parameters used in User class file MUST be specified in `-randomize_parameters` in following ways
- ◆ can be specified in `-default` if you dont want to randomize OR
- ◆ can occur in other input lists of `-randomize_parameter` OR
- ◆ can be present in dependency param in taht case you dont have to pass as input in `-randomize_parameters` OR
- ◆ can be specified in `-user` list in which case the tool wont generate a base constraint

SV Files Path

The randomize parameter files are available in path
<config>/export/randomize_params

Files and Directories : DWC_GMAC

*DWC_gmac_top_const.sv comp.csh cov.csh ct_top.sv inc.f run.csh
urgReport/*

Debugging Constraint Solver and Compilation Issues

Before reporting any tool issues please see that these things are done correctly

Compilation Issues

- ◆ The functions are not implemented. Please use rand_proc and create SV functions
- ◆ The Expressions are not written correctly
- ◆ User constraints file is provided and parameters used there is not passed via randomize_parameters. A variable definition wont get created

Solver Failures

Constraint solver failures can occur due to following

- ◇ Wrongly coded constraint in parameter Expressions
- ◇ Constraint specified in User constraint file is not in-lines with base constraints generated by the tool

Examples

Please refer to examples above

See Also

rand_proc URGCoverpointBinText URGCoverpointBins

NAME

re_syntax - Syntax of Tcl regular expressions

DESCRIPTION

A *regular expression* describes strings of characters. It's a pattern that matches certain strings and does not match others.

DIFFERENT FLAVORS OF REs

Regular expressions as defined by POSIX, come in two flavors: *extended* REs and *basic* REs. EREs are roughly those of the traditional *egrep*, while BREs are roughly those of the traditional *ed*. This implementation adds a third flavor, *advanced* REs basically EREs with some significant extensions.

This manual page primarily describes AREs. BREs mostly exist for backward compatibility in some old programs; they will be discussed at the end. POSIX EREs are almost an exact subset of AREs. Features of AREs that are not present in EREs will be indicated.

REGULAR EXPRESSION SYNTAX

Tcl regular expressions are implemented using the package written by Henry Spencer, based on the 1003.2 spec and some (not quite all) of the Perl5 extensions (thanks, Henry!). Much of the description of regular expressions below is copied verbatim from his manual entry.

coreTools Command Reference Index

An ARE is one or more *branches*, separated by matching anything that matches any of the branches.

A branch is zero or more *constraints* or *quantified atoms*, concatenated. It matches a match for the first, followed by a match for the second, etc; an empty branch matches the empty string.

QUANTIFIERS

A quantified atom is an *atom* possibly followed by a single *quantifier*. Without a quantifier, it matches a single match for the atom. The quantifiers, and what a so-quantified atom matches, are:

- `*` a sequence of 0 or more matches of the atom
- `+` a sequence of 1 or more matches of the atom
- `?` a sequence of 0 or 1 matches of the atom
- `{m}` a sequence of exactly *m* matches of the atom
- `{m,}` a sequence of *m* or more matches of the atom
- `{m,n}` a sequence of *m* through *n* (inclusive) matches of the atom; *m* may not exceed *n*
- `*? +? ?? {m}? {m,}? {m,n}?`
non-greedy quantifiers, which match the same possibilities, but prefer the smallest number rather than the largest number of matches (see **MATCHING**)

The forms using `{` and `}` are known as *bounds*. The numbers *m* and *n* are unsigned decimal integers with permissible values from 0 to 255 inclusive.

ATOMS

An atom is one of:

- `(re))` matches a match for *re* (*re* is any regular expression) with the match noted for possible reporting
- `(?:re))` as previous, but does no reporting (a set of parentheses)
- `()` matches an empty string, noted for possible reporting
- `(?:)` matches an empty string, without reporting
- `[chars]` a bracket expression, matching any one of the chars (see **BRACKET EXPRESSIONS** for more detail)
- `.` matches any single character
- `\k` matches the non-alphanumeric character *k* taken as an ordinary character, e.g. `\\` matches a backslash character

coreTools Command Reference Index

<code>\c</code>	where <i>c</i> is alphanumeric (possibly followed by other characters), an escape (AREs only), see ESCAPES below
<code>{</code>	when followed by a character other than a digit, matches the left-brace character when followed by a digit, it is the beginning of a bound (see above)
<code>x</code>	where <i>x</i> is a single character with no other significance, matches that character.

CONSTRAINTS

A constraint matches an empty string when specific conditions are met. A constraint may not be followed by a quantifier. The simple constraints are as follows; some more constraints are described later, under **ESCAPES**.

<code>^</code>	matches at the beginning of a line
<code>\$</code>	matches at the end of a line
<code>(?=re)</code>	positive lookahead (AREs only), matches at any point where a substring matching <i>re</i> begins
<code>(?!re)</code>	negative lookahead (AREs only), matches at any point where no substring matching <i>re</i> begins

The lookahead constraints may not contain back references (see later), and all parentheses within them are considered non-capturing.

An RE may not end with

BRACKET EXPRESSIONS

A *bracket expression* is a list of characters enclosed in `[]`. It normally matches any single character from the list (but see below). If the list begins with `^` it matches any single character (but see below) *not* from the rest of the list.

If two characters in the list are separated by a hyphen `-` this is shorthand for the full *range* of characters between those two (inclusive) in the collating sequence, e.g. `[0-9]` in Unicode matches any conventional decimal digit. Two ranges may not share an endpoint, so `[0-10]` is illegal. Ranges in Tcl always use the Unicode collating sequence, but other programs may use other collating sequences and this can be a source of incompatibility between programs.

To include a literal `]` or `-` in the list, the simplest method is to enclose it in `[.]` and `[.]` to make it a

coreTools Command Reference Index

collating element (see below). Alternatively, make it the first character (following a possible or (AREs only) precede it with Alternatively, for make it the last character, or the second endpoint of a range. To use a literal - as the first endpoint of a range, make it a collating element or (AREs only) precede it with With the exception of these, some combinations using [(see next paragraphs), and escapes, all other special characters lose their special significance within a bracket expression.

CHARACTER CLASSES

Within a bracket expression, the name of a *character class* enclosed in [: and :] stands for the list of all characters (not all collating elements!) belonging to that class. Standard character classes are:

alpha	A letter.
upper	An upper-case letter.
lower	A lower-case letter.
digit	A decimal digit.
xdigit	A hexadecimal digit.
alnum	An alphanumeric (letter or digit).
print	A "printable" (same as graph, except also including space).
blank	A space or tab character.
space	A character producing white space in displayed text.
punct	A punctuation character.
graph	A character with a visible representation (includes both alnum and punct).
cntrl	A control character.

A locale may provide others. A character class may not be used as an endpoint of a range.

(*Note:* the current Tcl implementation has only one locale, the Unicode locale, which supports exactly the above classes.)

BRACKETED CONSTRAINTS

There are two special cases of bracket expressions: the bracket expressions and are constraints, matching empty strings at the beginning and end of a word respectively. A word is defined as a sequence of word characters that is neither preceded nor followed by word characters. A word character is an *alnum* character or an underscore These special bracket expressions are deprecated; users of AREs should use constraint escapes instead (see below).

COLLATING ELEMENTS

Within a bracket expression, a collating element (a character, a multi-character sequence that collates as if it were a single character, or a collating-sequence name for either) enclosed in [. and .] stands for the sequence of characters of that collating element. The sequence is a single element of the bracket expression's list. A bracket expression in a locale that has multi-character collating elements can thus match more than one character. So (insidiously), a bracket expression that starts with ^ can match multi-character collating elements even if none of them appear in the bracket expression!

(Note: Tcl has no multi-character collating elements. This information is only for illustration.)

For example, assume the collating sequence includes a **ch** multi-character collating element. Then the RE (zero or more followed by matches the first five characters of Also, the RE matches all of (because matches the multi-character

EQUIVALENCE CLASSES

Within a bracket expression, a collating element enclosed in [= and =] is an equivalence class, standing for the sequences of characters of all collating elements equivalent to that one, including itself. (If there are no other equivalent collating elements, the treatment is as if the enclosing delimiters were and For example, if **o** and **o** are the members of an equivalence class, then and are all synonymous. An equivalence class may not be an endpoint of a range.

(Note: Tcl implements only the Unicode locale. It does not define any equivalence classes. The examples above are just illustrations.)

ESCAPES

Escapes (AREs only), which begin with a \ followed by an alphanumeric character, come in several varieties: character entry, class shorthands, constraint escapes, and back references. A \ followed by an alphanumeric character but not constituting a valid escape is illegal in AREs. In EREs, there are no escapes: outside a bracket expression, a \ followed by an alphanumeric character merely stands for that character as an ordinary character, and inside a bracket expression, \ is an ordinary character. (The latter is the one actual incompatibility between EREs and AREs.)

CHARACTER-ENTRY ESCAPES

Character-entry escapes (AREs only) exist to make it easier to specify non-printing and otherwise inconvenient characters in RES:

\a alert (bell) character, as in C

coreTools Command Reference Index

- `\b` backspace, as in C
- `\B` synonym for `\` to help reduce backslash doubling in some applications where there are multiple levels of backslash processing
- `\cX` (where *X* is any character) the character whose low-order 5 bits are the same as those of *X*, and whose other bits are all zero
- `\e` the character whose collating-sequence name is or failing that, the character with octal value 033
- `\f` formfeed, as in C
- `\n` newline, as in C
- `\r` carriage return, as in C
- `\t` horizontal tab, as in C
- `\uvwxyz`
(where *wxyz* is one up to four hexadecimal digits) the Unicode character `U+wxyz` in the local byte ordering
- `\Ustuvwxyz`
(where *stuvwxyz* is one up to eight hexadecimal digits) reserved for a Unicode extension up to 21 bits. The digits are parsed until the first non-hexadecimal character is encountered, the maximum of eight hexadecimal digits are reached, or an overflow would occur in the maximum value of `U+10ffff`.
- `\v` vertical tab, as in C are all available.
- `\xhh` (where *hh* is one or two hexadecimal digits) the character whose hexadecimal value is `0xhh`.
- `\0` the character whose value is `0`
- `\xyz` (where *xyz* is exactly three octal digits, and is not a *back reference* (see below)) the character whose octal value is `0xyz`. The first digit must be in the range 0-3, otherwise the two-digit form is assumed.
- `\xy` (where *xy* is exactly two octal digits, and is not a *back reference* (see below)) the character whose octal value is `0xy`

Hexadecimal digits are and Octal digits are

The character-entry escapes are always taken as ordinary characters. For example, `\135` is `]` in Unicode, but `\135` does not terminate a bracket expression. Beware, however, that some applications (e.g., C compilers and the Tcl interpreter if the regular expression is not quoted with braces) interpret

coreTools Command Reference Index

such sequences themselves before the regular-expression package gets to see them, which may require doubling (quadrupling, etc.) the

CLASS-SHORTHAND ESCAPES

Class-shorthand escapes (AREs only) provide shorthands for certain commonly-used character classes:

<code>\d</code>	<code>[[:digit:]]</code>
<code>\s</code>	<code>[[:space:]]</code>
<code>\w</code>	<code>[[:alnum:]]</code> (note underscore)
<code>\D</code>	<code>[^[:digit:]]</code>
<code>\S</code>	<code>[^[:space:]]</code>
<code>\W</code>	<code>[^[:alnum:]]</code> (note underscore)

Within bracket expressions, and lose their outer brackets, and are illegal. (So, for example, is equivalent to Also, which is equivalent to is illegal.)

CONSTRAINT ESCAPES

A constraint escape (AREs only) is a constraint, matching the empty string if specific conditions are met, written as an escape:

<code>\A</code>	matches only at the beginning of the string (see MATCHING , below, for how this differs from
<code>\m</code>	matches only at the beginning of a word
<code>\M</code>	matches only at the end of a word
<code>\y</code>	matches only at the beginning or end of a word
<code>\Y</code>	matches only at a point that is not the beginning or end of a word
<code>\Z</code>	matches only at the end of the string (see MATCHING , below, for how this differs from
<code>\m</code>	(where <i>m</i> is a nonzero digit) a <i>back reference</i> , see below
<code>\mnn</code>	(where <i>m</i> is a nonzero digit, and <i>nn</i> is some more digits, and the decimal value <i>mnn</i> is not greater than the number of closing capturing parentheses seen so far) a <i>back reference</i> , see below

A word is defined as in the specification of and above. Constraint escapes are illegal within bracket expressions.

BACK REFERENCES

A back reference (AREs only) matches the same string matched by the parenthesized subexpression specified by the number, so that (e.g.) matches or but not The subexpression must entirely precede the back reference

coreTools Command Reference Index

in the RE. Subexpressions are numbered in the order of their leading parentheses. Non-capturing parentheses do not define subexpressions.

There is an inherent historical ambiguity between octal character-entry escapes and back references, which is resolved by heuristics, as hinted at above. A leading zero always indicates an octal escape. A single non-zero digit, not followed by another digit, is always taken as a back reference. A multi-digit sequence not starting with a zero is taken as a back reference if it comes after a suitable subexpression (i.e. the number is in the legal range for a back reference), and otherwise is taken as octal.

METASYNTAX

In addition to the main syntax described above, there are some special forms and miscellaneous syntactic facilities available.

Normally the flavor of RE being used is specified by application-dependent means. However, this can be overridden by a *director*. If an RE of any flavor begins with the rest of the RE is an ARE. If an RE of any flavor begins with the rest of the RE is taken to be a literal string, with all characters considered ordinary characters.

An ARE may begin with *embedded options*: a sequence (*?xyz*) (where *xyz* is one or more alphabetic characters) specifies options affecting the rest of the RE. These supplement, and can override, any options specified by the application. The available option letters are:

- b** rest of RE is a BRE
- c** case-sensitive matching (usual default)
- e** rest of RE is an ERE
- i** case-insensitive matching (see **MATCHING**, below)
- m** historical synonym for **n**
- n** newline-sensitive matching (see **MATCHING**, below)
- p** partial newline-sensitive matching (see **MATCHING**, below)
- q** rest of RE is a literal string, all ordinary characters
- s** non-newline-sensitive matching (usual default)
- t** tight syntax (usual default; see below)
- w** inverse partial newline-sensitive matching (see **MATCHING**, below)

x *expanded syntax (see below)*

Embedded options take effect at the) terminating the sequence. They are available only at the start of an ARE, and may not be used later within it.

*In addition to the usual (tight) RE syntax, in which all characters are significant, there is an expanded syntax, available in all flavors of RE with the **-expanded** switch, or in AREs with the embedded **x** option. In the expanded syntax, white-space characters are ignored and all characters between a # and the following newline (or the end of the RE) are ignored, permitting paragraphing and commenting a complex RE. There are three exceptions to that basic rule:*

- o** a white-space character or preceded by
 is retained*
- o** white space or within a bracket
 expression is retained*
- o** white space and comments are illegal
 within multi-character symbols like the
 ARE or the BRE*

Expanded-syntax white-space characters are blank, tab, newline, and any character that belongs to the space character class.

*Finally, in an ARE, outside bracket expressions, the sequence (where *t* is any text not containing a *is* a comment, completely ignored. Again, this is not allowed between the characters of multi-character symbols like *Such comments are more a historical artifact than a useful facility, and their use is deprecated; use the expanded syntax instead.**

None of these metasyntax extensions is available if the application (or an initial director) has specified that the user's input be treated as a literal string rather than as an RE.

MATCHING

In the event that an RE could match more than one substring of a given string, the RE matches the one starting earliest in the string. If the RE could match more than one substring starting at that point, its choice is determined by its *preference*: either the longest substring, or the shortest.

Most atoms, and all constraints, have no preference. A parenthesized RE has the same preference (possibly none) as the RE. A quantified atom with quantifier **{m}** or **{m}?** has the same preference (possibly none) as the atom itself. A quantified atom with other normal quantifiers (including **{m,n}** with *m* equal to *n*) prefers

coreTools Command Reference Index

longest match. A quantified atom with other non-greedy quantifiers (including `{m,n}?` with `m` equal to `n`) prefers shortest match. A branch has the same preference as the first quantified atom in it which has a preference. An RE consisting of two or more branches connected by the `|` operator prefers longest match.

Subject to the constraints imposed by the rules for matching the whole RE, subexpressions also match the longest or shortest possible substrings, based on their preferences, with subexpressions starting earlier in the RE taking priority over ones starting later. Note that outer subexpressions thus take priority over their component subexpressions.

The quantifiers `{1,1}` and `{1,1}?` can be used to force longest and shortest preference, respectively, on a subexpression or a whole RE.

NOTE: This means that you can usually make a RE be non-greedy overall by putting `{1,1}?` after one of the first non-constraint atoms or parenthesized sub-expressions in it. *It pays to experiment* with the placing of this non-greediness override on a suitable range of input texts when you are writing a RE if you are using this level of complexity.

For example, this regular expression is non-greedy, and will match the shortest substring possible given that will be matched as early as possible (the quantifier does not change that):

```
ab{1,1}?c.*x.*cba
```

The atom has no greediness preference, we explicitly give one for and the remaining quantifiers are overridden to be non-greedy by the preceding non-greedy quantifier.

Match lengths are measured in characters, not collating elements. An empty string is considered longer than no match at all. For example, matches the three middle characters of matches all ten characters of when is matched against the parenthesized subexpression matches all three characters, and when is matched against both the whole RE and the parenthesized subexpression match an empty string.

If case-independent matching is specified, the effect is much as if all case distinctions had vanished from the alphabet. When an alphabetic that exists in multiple cases appears as an ordinary character outside a bracket expression, it is effectively transformed into a bracket expression containing both cases, so that `x` becomes `X` when it appears inside a bracket expression, all case counterparts of it are added to the bracket expression, so that `[x]` becomes `[Xx]` and becomes

If newline-sensitive matching is specified, `.` and bracket expressions using `^` will never match the

coreTools Command Reference Index

newline character (so that matches will never cross newlines unless the RE explicitly arranges it) and `^` and `$` will match the empty string after and before a newline respectively, in addition to matching at beginning and end of string respectively. ARE `\A` and `\Z` continue to match beginning or end of string *only*.

If partial newline-sensitive matching is specified, this affects `.` and bracket expressions as with newline-sensitive matching, but not `^` and `$`.

If inverse partial newline-sensitive matching is specified, this affects `^` and `$` as with newline-sensitive matching, but not `.` and bracket expressions. This is not very useful but is provided for symmetry.

LIMITS AND COMPATIBILITY

No particular limit is imposed on the length of REs. Programs intended to be highly portable should not employ REs longer than 256 bytes, as a POSIX-compliant implementation can refuse to accept such REs.

The only feature of AREs that is actually incompatible with POSIX EREs is that `\` does not lose its special significance inside bracket expressions. All other ARE features use syntax which is illegal or has undefined or unspecified effects in POSIX EREs; the `***` syntax of directors likewise is outside the POSIX syntax for both BRES and ERES.

Many of the ARE extensions are borrowed from Perl, but some have been changed to clean them up, and a few Perl extensions are not present. Incompatibilities of note include the lack of special treatment for a trailing newline, the addition of complemented bracket expressions to the things affected by newline-sensitive matching, the restrictions on parentheses and back references in lookahead constraints, and the longest/shortest-match (rather than first-match) matching semantics.

The matching rules for REs containing both normal and non-greedy quantifiers have changed since early beta-test versions of this package. (The new rules are much simpler and cleaner, but do not work as hard at guessing the user's real intentions.)

Henry Spencer's original 1986 *regex* package, still in widespread use (e.g., in pre-8.1 releases of Tcl), implemented an early version of today's EREs. There are four incompatibilities between *regex*'s near-EREs and AREs. In roughly increasing order of significance:

- o In AREs, `\` followed by an alphanumeric character is either an escape or an error, while in RREs, it was just another way of writing the alphanumeric. This should not be a problem because

coreTools Command Reference Index

there was no reason to write such a sequence in RREs.

- o { followed by a digit in an ARE is the beginning of a bound, while in RREs, { was always an ordinary character. Such sequences should be rare, and will often result in an error because following characters will not look like a valid bound.
- o In AREs, \ remains a special character within so a literal \ within [] must be written \\ also gives a literal \ within [] in RREs, but only truly paranoid programmers routinely doubled the backslash.
- o AREs report the longest/shortest match for the RE, rather than the first found in a specified search order. This may affect some RREs which were written in the expectation that the first match would be reported. (The careful crafting of RREs to optimize the search order for fast matching is obsolete (AREs examine all possible matches in parallel, and their performance is largely insensitive to their complexity) but cases where the search order was exploited to deliberately find a match which was *not* the longest/shortest will need rewriting.)

BASIC REGULAR EXPRESSIONS

BREs differ from EREs in several respects. and ? are ordinary characters and there is no equivalent for their functionality. The delimiters for bounds are \{ and with { and } by themselves ordinary characters. The parentheses for nested subexpressions are \(and with (and) by themselves ordinary characters. ^ is an ordinary character except at the beginning of the RE or the beginning of a parenthesized subexpression, \$ is an ordinary character except at the end of the RE or the end of a parenthesized subexpression, and * is an ordinary character if it appears at the beginning of the RE or the beginning of a parenthesized subexpression (after a possible leading Finally, single-digit back references are available, and \< and \> are synonyms for and respectively; no other escapes are available.

SEE ALSO

`RegExp(3)`, `regexp(n)`, `regsub(n)`, `lsearch(n)`, `switch(n)`,
`text(n)`

KEYWORDS

`match`, `regular expression`, `string`

NAME

`read` - Read from a channel

SYNOPSIS

`read ?-nonewline? channelId`

`read channelId numChars`

DESCRIPTION

In the first form, the **read** command reads all of the data from *channelId* up to the end of the file. If the **-nonewline** switch is specified then the last character of the file is discarded if it is a newline. In the second form, the extra argument specifies how many characters to read. Exactly that many characters will be read and returned, unless there are fewer than *numChars* left in the file; in this case all the remaining characters are returned. If the channel is configured to use a multi-byte encoding, then the number of characters read may not be the same as the number of bytes read.

ChannelId must be an identifier for an open channel such as the Tcl standard input channel (**stdin**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension. The channel must have been opened for input.

If *channelId* is in nonblocking mode, the command may not read as many characters as requested: once all available input has been read, the command will return the data that is available rather than blocking for more input. If the channel is configured to use a multi-byte encoding, then there may actually be some

coreTools Command Reference Index

bytes remaining in the internal buffers that do not form a complete character. These bytes will not be returned until a complete character is available or end-of-file is reached. The **-nonewline** switch is ignored if the command returns before reaching the end of the file.

Read translates end-of-line sequences in the input into newline characters according to the **-translation** option for the channel. See the **fconfigure** manual entry for a discussion on ways in which **fconfigure** will alter input.

USE WITH SERIAL PORTS

For most applications a channel connected to a serial port should be configured to be nonblocking: **fconfigure channelId -blocking 0**. Then **read** behaves much like described above. Care must be taken when using **read** on blocking serial ports:

read channelId numChars

In this form **read** blocks until *numChars* have been received from the serial port.

read channelId

In this form **read** blocks until the reception of the end-of-file character, see **fconfigure -eofchar**. If there no end-of-file character has been configured for the channel, then **read** will block forever.

EXAMPLE

This example code reads a file all at once, and splits it into a list, with each line in the file corresponding to an element in the list:

```
set fl [open /proc/meminfo]
set data [read $fl]
close $fl
set lines [split $data \n]
```

SEE ALSO

file(n), eof(n), fblocked(n), fconfigure(n),
Tcl_StandardChannels(3)

KEYWORDS

blocking, channel, end of line, end of file,
nonblocking, read, translation, encoding

read_attribute_table

Read an attribute table file.

Syntax

```
string read_attribute_table [-tables <table names>] [-columns <list of
column name value pairs>] [-map <list of column name map pairs>]
[-separator <table column separator>] [-create] [-update] [-loops]
[-blanks] [-comment_indicator <table comment indicator>]
[-string_delimiter <table text delimiter>] <filename>
    list <table names>
    list <list of column name value pairs>
    list <list of column name map pairs>
    string <table column separator>
    string <table comment indicator>
    string <table text delimiter>
    string <filename>
```

Parameters

-tables <table names>	List of tables names to be read in from the file. If this option is not specified, then all tables in the table file will be processed
-columns <list of column name value pairs>	List of column name and value pairs to insert into read in table. This option defines static columns that should be assumed to exist in all processed tables. The <column data> is a list of two-element lists where the first element is a column name and the second element is the value to appear in that column for every table row. If a column is defined in this manner and explicitly exists in the table, then a table reading error occurs.
-map <list of column name map pairs>	List of column mapped name table name pairs. This option defines column header name mapping. Different tables have different requirements regarding the names of columns in the table. This option can be used to map from the names seen in the table to the names expected by the table reading command. This enables user-defined column names while still providing the table reader with known names. The <column map> is a list of two-element lists where the first element is the name of a column in the

coreTools Command Reference Index

	table and the second element is the name expected by the table reading command.
-separator <table column separator>	The character used to separate columns in the table file.
-create	Create the items read in from the tables.
-update	Update the items read in from the tables.
-loops	Process table data for loop expressions.
-blanks	Allow blanks rows in named tables.
-comment_indicator <table comment indicator>	The character used to indicate a comment line in the table file.
-string_delimiter <table text delimiter>	The character used to delimit strings in the table.
<filename>	The name of the table file to read.

Description

This command is used to set attributes on objects in the coreTools data model. Attribute tables may have the have the following columns:

Column Name	Optional	Description
Type	No	Indicates the object type
Name	No	Indicates the object name
<attribute name>	Yes	Indicates name of the attribute to be set. An arbitrary number of these columns is allowed.

It is used to set the indicated attributes on the indicated objects.

Objects are located using the item type and item name. When a table contains objects of multiple types, scoping is handled automatically when switching between object types that have a parent-child relationship. For example port lines appearing after a design line are assumed to indicate ports within that design. Scoping is supported for the following object type transitions:

- design > port
- memoryMap > addressBlock
- addressBlock > register
- addressBlock > registerArray
- registerArray > register
- register > registerField

As each of these transitions occurs the current context is changed by pushing the prior element onto the context stack. When the reverse transitions occur elements are popped from the stack as appropriate. This scoping means that names specified for items can always be simple local names without context in the name itself.

If neither create nor update option is specified, each line is assumed to refer to an object that already exists in the data model. If create is specified, each object is assumed to not exist, and will be created before setting any attributes on it. If update is specified, a search will be done and the object will be created only if it does not already exist.

If the loops option is specified, each name will be scanned for loop syntax. The loop syntax is {<loop var>:<start index>..<end index>}. When seen, the line will be replicated for each inclusive index value. Each attribute value will be scanned for \$<loop var> and that string will be replaced with the loop index value. When a line with loop syntax is used for scoping, all scoped objects will be created within each replicated scoping object.

For example, the partial table shown below would result in two registers being created, each with two fields, F1 and F2.

Type	Name	Size	Description
register	Interrupt{i:0..1}	32	Interrupt \$i
field	F{j:0..1}	16	Field \$j in interrupt \$i

Examples

```
coreBuilder> read_attribute_table -comment_indicator "#" -separator ":" \
-map {{MappedMaxLoads MaxLoads}} ./attrTables_cB.txt
```

See Also

write_attribute_table(2), read_pin_connection_table(2),
write_pin_connection_table(2), read_subsystem_table(2),
write_subsystem_table(2)

read_component_constraints

Read component SDC files to apply component constraints.

Syntax

string **read_component_constraints**

Description

This command is used to cause coreAssembler to read any component SDC files that have been associated with leaf component instances but which have not already been loaded (read). This typically applies to IP-XACT components which reference external SDC files. These files will be read automatically during the Initialize Subsystem Constraints activity, but if the SDC files define clocks it may be necessary to load the files explicitly in order to set attributes on the clocks. The primary usage would be to enable setting SystemClockName in batch mode.

Examples

Cause all external component level SDC files to be read:

```
prompt> read_component_constraints
```

See Also

`read_ipxact_file`

Read and process an IP-XACT file of the specified type.

Syntax

string **read_ipxact_file** -type *<file type>* [-strict] *filename*string *<file type>*string *filename*

Parameters

<code>-type <file type></code>	The top-level element type of the IP-XACT file. (Values: design, designConfiguration, generatorChain, component, typeDefinitions)
<code>-strict</code>	Applicable only for component file type. If present, top-level ports and interface instances can not be changed.
<code>filename</code>	The name of the IP-XACT file to be processed.

Description

This command is used to read (import) an IP-XACT format file into coreAssembler. The currently supported files types are 'design', 'generatorChain', 'designConfiguration' and 'component'. Reading an IP-XACT design file causes the associated design to be created in the current level of hierarchy, which must be empty when the design is read. Reading an IP-XACT generatorChain file makes the generator chain available for invocation via the Generators menu. All generators referenced by the generator chain must already be loaded into coreAssembler or the reading of the generator chain will fail.

-strict option may only be specified when file type is 'component'. If specified, it prevents deletion of any interface instances that were loaded from xml file. It also prevents user from creating new ports or exporting any interface instances on top-level design.

Examples

Create a new top-level design from myDesign.xml:

```
coreAssembler> create_workspace
```

```
coreAssembler> read_ipxact_file -type design myDesign.xml
```

Read an IP-XACT generator so that it can be used to invoke multiple generators:

coreTools Command Reference Index

```
coreAssembler> read_ipxact_file -type generatorChain myChain.xml
```

Read an IP-XACT component from myComponent.xml with -strict option:

```
coreAssembler> read_ipxact_file -type component myComponent.xml -strict
```

See Also

read_parameter_info_table
Read a parameter info table file.

Syntax

```
string read_parameter_info_table [-tables <table names>] [-columns <list
of column name value pairs>] [-map <list of column name map pairs>]
[-separator <table column separator>] [-loops] [-blanks]
[-comment_indicator <table comment indicator>] [-container_type
<container type>] [-string_delimiter <table text delimiter>] <filename>
    list <table names>
    list <list of column name value pairs>
    list <list of column name map pairs>
    string <table column separator>
    string <table comment indicator>
    string <container type>
    string <table text delimiter>
    string <filename>
```

Parameters

-tables <table names>	List of tables names to be read in from the file.
-columns <list of column name value pairs>	List of column name and value pairs to insert into read in table.
-map <list of column name map pairs>	List of column mapped name table name pairs.
-separator <table column separator>	The character used to separate columns in the table file.
-loops	Process table data for loop expressions.
-blanks	Allow blanks rows in named tables.
-comment_indicator <table comment indicator>	The character used to indicate a comment line in the table file.
-container_type <container type>	The container type to use for the dialog parameter info.
-string_delimiter <table text delimiter>	The character used to delimit strings in the table.
<filename>	The name of the table file to read.

Description

Examples

See Also

read_parameter_table

Read a parameter table file.

Syntax

```
string read_parameter_table [-tables <table names>] [-columns <list of
column name value pairs>] [-map <list of column name map pairs>]
[-separator <table column separator>] [-design] [-package <package name>]
[-generic <for item>] [-loops] [-blanks] [-comment_indicator <table
comment indicator>] [-string_delimiter <table text delimiter>] <filename>
    list <table names>
    list <list of column name value pairs>
    list <list of column name map pairs>
    string <table column separator>
    string <package name>
    string <for item>
    string <table comment indicator>
    string <table text delimiter>
    string <filename>
```

Parameters

-tables <table names>	List of tables names to be read in from the file.
-columns <list of column name value pairs>	List of column name and value pairs to insert into read in table.
-map <list of column name map pairs>	List of column mapped name table name pairs.
-separator <table column separator>	The character used to separate columns in the table file.
-design	Specify parameter to be a design parameter.
-package <package name>	Package that this parameter should go into.
-generic <for item>	Specify parameter to be a non-HDL generic parameter to the given item.
-loops	Process table data for loop expressions.
-blanks	Allow blanks rows in named tables.
-comment_indicator <table comment indicator>	The character used to indicate a comment line in the table file.
-string_delimiter <table text delimiter>	The character used to delimit strings in the table.
<filename>	The name of the table file to read.

Description

This command reads a table containing information defining parameters and creates the corresponding parameters in the coreTools data model. This applies only to parameters associated with custom activities, as it is not appropriate to create design parameters in this fashion. The table syntax is similar to that of the other table reading commands (see the coreBuilder UG for details). The parameter tables can contain the following columns:

- Name: required column indicating the parameter name
- Type: required column indicating the parameter type
- <attribute>: optional column(s) whose name matches that of an attribute which can be set on parameter. Typical columns would include DefaultValue, Enabled, MinValue, MaxValue, EnumValues, etc.

When the table is read, a parameter is created for each row in the table, based on the cell values in that row. The owner (parent) of the parameters is indicated by the -design/-generic/-package option specified to the command itself.

Examples

See Also

`read_pin_connection_table`
Read a pin connection table file.

Syntax

```
string read_pin_connection_table [-tables <table names>] [-columns <list
of column name value pairs>] [-map <list of column name map pairs>]
[-separator <table column separator>] [-blanks] [-comment_indicator
<table comment indicator>] [-hierarchy] [-nosource] [-string_delimiter
<table text delimiter>] <filename>
    list <table names>
    list <list of column name value pairs>
    list <list of column name map pairs>
    string <table column separator>
    string <table comment indicator>
    string <table text delimiter>
    string <filename>
```

Parameters

<code>-tables <table names></code>	List of tables names to be read in from the file. List of column name and value pairs to insert into read in table. This option defines static columns that should be assumed to exist in all processed tables. The <column data> is a list of two-element lists where the first element is a column name and the second element is the value to appear in that column for every table row. If a column is defined in this manner and explicitly exists in the table, then a table reading error occurs.
<code>-columns <list of column name value pairs></code>	List of column mapped name table name pairs. This option defines column header name mapping. Different tables have different requirements regarding the names of columns in the table. This option can be used to map from the names seen in the table to the names expected by the table reading command. This enables user-defined column names while still providing the table reader with known names. The <column map> is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table reading command.
<code>-map <list of column name map pairs></code>	

coreTools Command Reference Index

-separator <table column separator>	The character used to separate columns in the table file.
-blanks	Allow blanks rows in named tables.
-comment_indicator <table comment indicator>	The character used to indicate a comment line in the table file.
-hierarchy	Connection are to be made through hierarchy when appropriate.
-nosource	All connections in the table are assumed to be load to load.
-string_delimiter <table text delimiter>	The character used to delimit strings in the table.
<filename>	The name of the table file to read.

Description

The pin connection table is available for coreAssembler only and is used to define manual connections for pins and ports. Pin connection tables can be used to represent calls to the create_connection, export_pin, create_port, and set_current_component commands. See the coreAssembler User Guide for details about the support columns in this table.

If the -hierarchy option is specified, each pin name in a connect. or export. line is scanned to see if it is hierarchical. If it is, then hierarchy is passed to the corresponding generated command. If the nosource option is specified, then the nosource option is passed to every generated create_connection command.

Examples

Read the connection table and store connections to be applied at the start of the Complete Connections activity. The connections are read in from the inFile and the column separator used is ','.

```
coreAssembler> read_pin_connection_table inFile -separator "," -hierarchy
```

See Also

write_pin_connection_table (2), read_attribute_table (2),
write_attribute_table (2), read_subsystem_table (2),
write_subsystem_table (2)

read_sdc

Read a Synopsys Design Constraints format script.

Syntax

```
string read_sdc [-script <sdc_script>] [-echo] [-reset] [-syntax]
[-version <sdc_version>] [-passthrough] [SDCFilename]
    string <sdc_script>
    string <sdc_version>
    string SDCFilename
```

Parameters

	SDC script to execute
-script <sdc_script>	The SDC commands should be provided as a TCL list immediately following the -script option. The commands in <sdc_script> are processed as if the text of <sdc_script> was written into a file named my.sdc and then 'read_sdc my.sdc' was invoked.
-echo	Echo all commands
-reset	Reset pass-through constraint values
-syntax	Verifies the legality of the SDC file only Warning messages will be issued for SDC command with wrong syntax. This option does not set constraints read from the SDC file to design objects.
-version <sdc_version>	SDC Version (default is 2.1) (Values: 1.0, 1.1, 1.2, 1.3, 1.5, 1.6, 1.7, 1.8, 1.9, 2.0, 2.1) Specify the version of SDC to be checked against. If the version of the SDC file doesn't match the specified version, an error message will be issued.
-passthrough	Tcl or SDC commands to be stored and dumped as it is by write_sdc
SDCFilename	SDC file to read The file should be in SDC version 1.5 format.

Description

The read_sdc command reads in a script file in Synopsys Design Constraints (SDC) format, SDC formatted script files are Tcl scripts that use a subset of the commands supported by the implementation tools. The entire SDC command set is not supported by the coreTools. Supported commands alter the coreTools data model and are then written out by [write_sdc](#). Unsupported commands (called pass-through commands for

coreTools) are passed through from the SDC file that is read to the SDC file that is written out using `write_sdc`. Pass-through commands do not alter the state of the design. They are written out verbatim.

Another category of SDC commands that are written out verbatim are commands with unsupported switches. These switches are called unsupported-switches. Unsupported-switches are options to a command that will cause the command to be passed-through (similar to a pass-through command). These commands are not passed through if none of these special switches are invoked with it.

Note that SDC syntax does not allow command abbreviation. Similar to the TCL source command, the exit and quit commands do not cause the application to exit, but they do cause the reading of the SDC file to stop.

The `read_sdc` command can be used to apply constraints in SDC format to a design. This is mostly commonly used in coreTools for timing exception commands since standard constraints (I/O delays, clock definitions, drive and load constraints) are normally defined using technology independent syntax that is not supported via SDC.

The `read_sdc` and `write_sdc` commands provide tool integration between the coreTools and other tools which support the SDC format.

Examples

The following command reads SDC file `top.sdc`:

```
coreBuilder> read_sdc top.sdc
```

The following command reads the SDC file `top.sdc`, does syntax check only, and verifies if the SDC file is of SDC version 2.0:

```
coreBuilder> read_sdc designA.sdc -syntax -version 2.0
```

The following command reads SDC commands from the command line directly:

```
coreBuilder> read_sdc -script {  
set_false_path -from reset  
create_clock -name myClk [get_ports clk1] -period 10  
}
```

See Also

[write_sdc \(2\)](#)

read_subsystem_table

Read a subsystem table file.

Syntax

```
string read_subsystem_table [-tables <table names>] [-columns <list of
column name value pairs>] [-map <list of column name map pairs>]
[-separator <table column separator>] [-blanks] [-comment_indicator
<table comment indicator>] [-string_delimiter <table text delimiter>]
<filename>
    list <table names>
    list <list of column name value pairs>
    list <list of column name map pairs>
    string <table column separator>
    string <table comment indicator>
    string <table text delimiter>
    string <filename>
```

Parameters

-tables <table names>	List of tables names to be read in from the file. List of column name and value pairs to insert into read in table. This option defines static columns that should be assumed to exist in all processed tables. The <column data> is a list of two-element lists where the first element is a column name and the second element is the value to appear in that column for every table row. If a column is defined in this manner and explicitly exists in the table, then a table reading error occurs.
-columns <list of column name value pairs>	List of column mapped name table name pairs. This option defines column header name mapping. Different tables have different requirements regarding the names of columns in the table. This option can be used to map from the names seen in the table to the names expected by the table reading command. This enables user-defined column names while still providing the table reader with known names. The <column map> is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table reading command.
-map <list of column name map pairs>	

coreTools Command Reference Index

-separator <table column separator>	The character used to separate columns in the table file.
-blanks	Allow blanks rows in named tables.
-comment_indicator <table comment indicator>	The character used to indicate a comment line in the table file.
-string_delimiter <table text delimiter>	The character used to delimit strings in the table.
<filename>	The name of the table file to read.

Description

A subsystem table can be used to define the contents of a subsystem in coreAssembler. The tables can contain information about components to instantiate, configuration information, and interface and pin-level connection details. The table below defines the columns that can appear in the table. Any additional columns appearing in the table will result in warnings and ignored.

Column Name	Optional	Description
Type	No	Indicates the type of information in the table line. Each type maps to a command used to define the subsystem. Legal values are instance, interface, unused, connect, and configure which correspond to the instantiate_interface, export_interface, set_unused_interface, connect_interface, and set_configuration_parameter or set_interface_parameter commands, respectively.
VLNV	Yes	Indicates the desired VLVN for a component instantiation.
TargetInstanceName	No	Indicates the desired instance name for a newly created instance or the target instance name for a command.
TargetInterfaceName	Yes	Indicates the desired interface name for a newly exported interface or the target interface name for a command.
SourceInstanceName	Yes	Indicates the source instance name for a command requiring both a source and target instance.
SourceInterfaceName	Yes	Indicates the source interface name for a command requiring both a source and target interface.
ParameterName	Yes	Indicates the name of a parameter to be configured. Either a component parameter or

coreTools Command Reference Index

Column Name	Optional	Description
		an interface parameter depending on whether or not the TargetInterfaceName column contains a value.
ParameterValue	Yes	Indicates the value to be applied to the named parameter.

The following table indicates which columns are required based on the line type.

Line type	Required Table Columns
instance	VLNV, TargetInstanceName
interface	TargetInterfaceName, SourceInstanceName, SourceInterfaceName
unused	TargetInstanceName, TargetInterfaceName
connect	TargetInstanceName, TargetInterfaceName, SourceInstanceName, SourceInterfaceName
configure	TargetInstanceName, ParameterName, ParameterValue. (TargetInterfaceName required if setting a interface parameter)

Examples

```
coreAssembler> read_subsystem_table ./subsystemTable.txt \
-tables SUBSYSTEM_TABLE_8 \
-separator "@" \
-map {{Instance TargetInstanceName} \
{Interface TargetInterfaceName} \
{Instance2 SourceInstanceName} \
{Interface2 SourceInterfaceName} \
{Param ParameterName} \
{Value ParameterValue } }
```

See Also

write subsystem table (2) read attribute table (2), write attribute table
(2), read pin connection table (2), write pin connection table (2).

NAME

redirect Redirects the output of a command to a file.

SYNTAX

```
string redirect
[-append] [-tee] [-file | -variable | -channel] [-compress]
target
{command_string}

string    target
string    command_string
```

ARGUMENTS

-append	Appends the output to <i>target</i> .
-tee	Like the unix command of the same name, sends output to the current output channel as well as to the <i>target</i> .
-file	Indicates that <i>target</i> is a file name, and redirection is to that file. This is the default. It is exclusive of <i>-variable</i> and <i>-channel</i> .
-variable	Indicates that <i>target</i> is a variable name, and redirection is to that Tcl variable. It is exclusive of <i>-file</i> and <i>-channel</i> .
-channel	Indicates tha <i>target</i> is a Tcl channel, and redirection is to that channel. It is exclusive of <i>-variable</i> and <i>-file</i> .
-compress	Compress when writing to file. If redirecting to a file, then this option specifies that the output should be compressed as it is written. The output file is in a format recognizable by

coreTools Command Reference Index

"gzip -d". You cannot specify this option with *-append*.

target Indicates the target of the output redirection. This is either a filename, Tcl variable, or Tcl channel depending on the command arguments.

command_string The command to execute. Intermediate output from this command, as well as the result of the command, will be redirected to *target*. The *command_string* should be rigidly quoted with curly braces.

DESCRIPTION

The **redirect** command performs the same function as the traditional unix-style redirection operators `>` and `>>`. The *command_string* must be rigidly quoted (that is, enclosed in curly braces) in order for the operation to succeed.

Output is redirected to a file by default. Output can be redirected to a Tcl variable by using the *-variable* option, or to a Tcl channel by using the *-channel* option.

Output can be sent to the current output device as well as the redirect target by using the *-tee* option. See the examples section for an example.

The result of a **redirect** command which does not generate a Tcl error is the empty string. Screen output occurs only if errors occurred during execution of the *command_string* (other than opening the redirect file). When errors occur, a summary message is printed. See the examples.

Although the result of a successful **redirect** command is the empty string, it is still possible to get and use the result of the command that you redirected. Construct a **set** command in which you set a variable to the result of your command. Then, redirect the **set** command. The variable holds the result of your command. See the examples.

The **redirect** command is much more flexible than traditional unix redirection operators. With **redirect**, you can redirect multiple commands or an entire script. See the examples for an example of how to construct such a command.

Note that the builtin Tcl command **puts** does not respond

coreTools Command Reference Index

to output redirection of any kind. Use the builtin **echo** command instead.

EXAMPLES

In the following example, the output of the `plus` procedure is redirected. The echoed string and the result of the `plus` operation is in the output file. Notice that the result was not echoed to the screen.

coreTools Command Reference Index

```
prompt> proc plus {a b} {echo "In plus" ; return [expr $a + $b]}
prompt> redirect p.out {plus 12 13}
prompt> exec cat p.out
In plus
25
```

In this example, a typo in the command created an error condition. The error message indicates that you can use **error_info** to trace the error, but you should first check the output file.

```
prompt> redirect p.out {plus2 12 13}
Error: Errors detected during redirect
      Use error_info for more info. (CMD-013)
prompt> exec cat p.out
Error: unknown command 'plus2' (CMD-005)
```

In this example, we explore the usage of results from redirected commands. Since the result of **redirect** for a command which does not generate a Tcl error is the empty string, use the **set** command to trap the result of the command. For example, assume that there is a command to read a file which has a result of "1" if it succeeds, and "0" if it fails. If you redirect only the command, there is no way to know if it succeeded.

```
redirect p.out { read_a_file "a.txt" }
# Now what?  How can I redirect and use the result?
```

But if you set a variable to the result, then it is possible to use that result in a conditional expression, etc.

```
redirect p.out { set rres [read_a_file "a.txt"] }
if { $rres == 1 } {
    echo "Read ok!"
}
```

The **redirect** command is not limited to redirection of a single command. You can redirect entire blocks of a script with a single **redirect** command. This simple example with **echo** demonstrates this feature:

```
prompt> redirect p.out {
?      echo -n "Hello "
?      echo "world"
?      }
prompt> exec cat p.out
Hello world
prompt>
```

The **redirect** command allows you to tee output to the previous output device and also to redirect output to a variable. This simple example with **echo** demonstrates these features:

```
prompt> set y "This is "
This is
```

coreTools Command Reference Index

```
prompt> redirect -tee x.out {  
    echo XXX  
    redirect -variable y -append {  
        echo YYY  
        redirect -tee -variable z {  
            echo ZZZ  
        }  
    }  
}  
XXX  
prompt> exec cat x.out  
XXX  
prompt> echo $y  
This is YYY  
ZZZ  
  
prompt> echo $z  
ZZZ
```

SEE ALSO

echo(2), error_info(2), set(2).

refItem

Defines a reference to a memory map in a component.

Description

The refItem is used as placeholder for a memory map. It always exists in memory while the corresponding (like-named) memory map is only loaded into the data model after configuration. The refItem has the same Visible settings and is used to determine which memory maps should be loaded after configuration is complete.

See Also

[memMap](#) (3),

Supported Attributes

NAME

refchan - command handler API of reflected channels

SYNOPSIS

`cmdPrefix option ?arg arg ...?`

DESCRIPTION

The Tcl-level handler for a reflected channel has to be a command with subcommands (termed an *ensemble*, as it is a command such as that created by **namespace ensemble create**, though the implementation of handlers for reflected channel *is not* tied to **namespace ensembles** in any way; see **EXAMPLE** below for how to build an **oo::class** that supports the API). Note that *cmdPrefix* is whatever was specified in the call to **chan create**, and may consist of multiple arguments; this will be expanded to multiple words in place of the prefix.

Of all the possible subcommands, the handler *must* support **initialize**, **finalize**, and **watch**. Support for the other subcommands is optional.

MANDATORY SUBCOMMANDS

cmdPrefix **initialize** *channelId* *mode*

An invocation of this subcommand will be the first call the *cmdPrefix* will receive for the specified new *channelId*. It is the responsibility of this subcommand to set up any internal data structures required to keep track of the channel and its state.

The return value of the method has to be a list containing the names of all subcommands supported by the *cmdPrefix*. This also tells the Tcl core which version of the API for reflected channels is used by this command handler.

coreTools Command Reference Index

Any error thrown by the method will abort the creation of the channel and no channel will be created. The thrown error will appear as error thrown by **chan create**. Any exception other than an **error** (e.g., **break**, etc.) is treated as (and converted to) an error.

Note: If the creation of the channel was aborted due to failures here, then the **finalize** subcommand will not be called.

The *mode* argument tells the handler whether the channel was opened for reading, writing, or both. It is a list containing any of the strings **read** or **write**. The list will always contain at least one element.

The subcommand must throw an error if the chosen mode is not supported by the *cmdPrefix*.

cmdPrefix **finalize** *channelId*

An invocation of this subcommand will be the last call the *cmdPrefix* will receive for the specified *channelId*. It will be generated just before the destruction of the data structures of the channel held by the Tcl core. The command handler *must not* access the *channelId* anymore in no way. Upon this subcommand being called, any internal resources allocated to this channel must be cleaned up.

The return value of this subcommand is ignored.

If the subcommand throws an error the command which caused its invocation (usually **chan close**) will appear to have thrown this error. Any exception beyond **error** (e.g., **break**, etc.) is treated as (and converted to) an error.

This subcommand is not invoked if the creation of the channel was aborted during **initialize** (See above).

cmdPrefix **watch** *channelId* *eventspec*

This subcommand notifies the *cmdPrefix* that the specified *channelId* is interested in the events listed in the *eventspec*. This argument is a list containing any of **read** and **write**. The list may be empty, which signals that the channel does not wish to be notified of any events. In that situation, the handler should disable event generation completely.

Warning: Any return value of the subcommand is ignored. This includes all errors thrown by the subcommand, **break**, **continue**, and custom return codes.

This subcommand interacts with **chan postevent**. Trying to post an event which was not listed in the last call to **watch** will cause **chan postevent** to throw an error.

OPTIONAL SUBCOMMANDS

cmdPrefix **read** *channelId* *count*

This *optional* subcommand is called when the user requests data from the channel *channelId*. *count* specifies how many *bytes* have been requested. If the

coreTools Command Reference Index

subcommand is not supported then it is not possible to read from the channel handled by the command.

The return value of this subcommand is taken as the requested data *bytes*. If the returned data contains more bytes than requested, an error will be signaled and later thrown by the command which performed the read (usually **gets** or **read**). However, returning fewer bytes than requested is acceptable.

Note that returning nothing (0 bytes) is a signal to the higher layers that **EOF** has been reached on the channel. To signal that the channel is out of data right now, but has not yet reached **EOF**, it is necessary to throw the error "EAGAIN", i.e. to either

```
return -code error EAGAIN
or
error EAGAIN
```

For extensibility any error whose value is a negative integer number will cause the higher layers to set the C-level variable "**errno**" to the absolute value of this number, signaling a system error. However, note that the exact mapping between these error numbers and their meanings is operating system dependent.

For example, while on Linux both

```
return -code error -11
and
error -11
```

are equivalent to the examples above, using the more readable string "EAGAIN", this is not true for BSD, where the equivalent number is -35.

The symbolic string however is the same across systems, and internally translated to the correct number. No other error value has such a mapping to a symbolic string.

If the subcommand throws any other error, the command which caused its invocation (usually **gets**, or **read**) will appear to have thrown this error. Any exception beyond **error**, (e.g., **break**, etc.) is treated as and converted to an error.

cmdPrefix **write** *channelId* *data*

This *optional* subcommand is called when the user writes data to the channel *channelId*. The *data* argument contains *bytes*, not characters. Any type of transformation (EOL, encoding) configured for the channel has already been applied at this point. If this subcommand is not supported then it is not possible to write to the channel handled by the command.

The return value of the subcommand is taken as the number of bytes written by the channel. Anything non-numeric will cause an error to be signaled and later thrown by the command which performed the write. A

coreTools Command Reference Index

negative value implies that the write failed. Returning a value greater than the number of bytes given to the handler, or zero, is forbidden and will cause the Tcl core to throw an error.

To signal that the channel is not able to accept data for writing right now, it is necessary to throw the error "EAGAIN", i.e. to either

```
return -code error EAGAIN
or
error EAGAIN
```

For extensibility any error whose value is a negative integer number will cause the higher layers to set the C-level variable "**errno**" to the absolute value of this number, signaling a system error. However, note that the exact mapping between these error numbers and their meanings is operating system dependent.

For example, while on Linux both

```
return -code error -11
and
error -11
```

are equivalent to the examples above, using the more readable string "EAGAIN", this is not true for BSD, where the equivalent number is -35.

The symbolic string however is the same across systems, and internally translated to the correct number. No other error value has such a mapping to a symbolic string.

If the subcommand throws any other error the command which caused its invocation (usually **puts**) will appear to have thrown this error. Any exception beyond **error** (e.g., **break**, etc.) is treated as and converted to an error.

cmdPrefix **seek** *channelId* *offset* *base*

This *optional* subcommand is responsible for the handling of **chan seek** and **chan tell** requests on the channel *channelId*. If it is not supported then seeking will not be possible for the channel.

The *base* argument is the same as the equivalent argument of the builtin **chan seek**, namely:

start	Seeking is relative to the beginning of the channel.
current	Seeking is relative to the current seek position.
end	Seeking is relative to the end of the channel.

The *offset* is an integer number specifying the amount of **bytes** to seek forward or backward. A positive number should seek forward, and a negative number should seek backward. A channel may provide only limited seeking.

coreTools Command Reference Index

For example sockets can seek forward, but not backward.

The return value of the subcommand is taken as the (new) location of the channel, counted from the start. This has to be an integer number greater than or equal to zero. If the subcommand throws an error the command which caused its invocation (usually **chan seek**, or **chan tell**) will appear to have thrown this error. Any exception beyond **error** (e.g., **break**, etc.) is treated as and converted to an error.

The offset/base combination of 0/**current** signals a **chan tell** request, i.e., seek nothing relative to the current location, making the new location identical to the current one, which is then returned.

*cmdPrefix **configure** channelId option value*

This *optional* subcommand is for setting the type-specific options of channel *channelId*. The *option* argument indicates the option to be written, and the *value* argument indicates the value to set the option to.

This subcommand will never try to update more than one option at a time; that is behavior implemented in the Tcl channel core.

The return value of the subcommand is ignored.

If the subcommand throws an error the command which performed the (re)configuration or query (usually **fconfigure** or **chan configure**) will appear to have thrown this error. Any exception beyond **error** (e.g., **break**, etc.) is treated as and converted to an error.

*cmdPrefix **cget** channelId option*

This *optional* subcommand is used when reading a single type-specific option of channel *channelId*. If this subcommand is supported then the subcommand **cgetall** must be supported as well.

The subcommand should return the value of the specified *option*.

If the subcommand throws an error, the command which performed the (re)configuration or query (usually **fconfigure** or **chan configure**) will appear to have thrown this error. Any exception beyond *error* (e.g., **break**, etc.) is treated as and converted to an error.

*cmdPrefix **cgetall** channelId*

This *optional* subcommand is used for reading all type-specific options of channel *channelId*. If this subcommand is supported then the subcommand **cget** has to be supported as well.

The subcommand should return a list of all options and their values. This list must have an even number of elements.

coreTools Command Reference Index

If the subcommand throws an error the command which performed the (re)configuration or query (usually **fconfigure** or **chan configure**) will appear to have thrown this error. Any exception beyond **error** (e.g., **break**, etc.) is treated as and converted to an error.

cmdPrefix **blocking** *channelId* *mode*

This *optional* subcommand handles changes to the blocking mode of the channel *channelId*. The *mode* is a boolean flag. A true value means that the channel has to be set to blocking, and a false value means that the channel should be non-blocking.

The return value of the subcommand is ignored.

If the subcommand throws an error the command which caused its invocation (usually **fconfigure** or **chan configure**) will appear to have thrown this error. Any exception beyond **error** (e.g., **break**, etc.) is treated as and converted to an error.

NOTES

Some of the functions supported in channels defined in Tcl's C interface are not available to channels reflected to the Tcl level.

The function **Tcl_DriverGetHandleProc** is not supported; i.e., reflected channels do not have OS specific handles.

The function **Tcl_DriverHandlerProc** is not supported. This driver function is relevant only for stacked channels, i.e., transformations. Reflected channels are always base channels, not transformations.

The function **Tcl_DriverFlushProc** is not supported. This is because the current generic I/O layer of Tcl does not use this function anywhere at all. Therefore support at the Tcl level makes no sense either. This may be altered in the future (through extending the API defined here and changing its version number) should the function be used at some time in the future.

EXAMPLE

This demonstrates how to make a channel that reads from a string.

```
oo::class create stringchan {
    variable data pos
    constructor {string {encoding {}}} {
        if {$encoding eq ""} {set encoding [encoding system]}
        set data [encoding convertto $encoding $string]
        set pos 0
    }
}
```

coreTools Command Reference Index

```
method initialize {ch mode} {
    return "initialize finalize watch read seek"
}
method finalize {ch} {
    my destroy
}
method watch {ch events} {
    # Must be present but we ignore it because we do not
    # post any events
}

# Must be present on a readable channel
method read {ch count} {
    set d [string range $data $pos [expr {$pos+$count-1}]]
    incr pos [string length $d]
    return $d
}

# This method is optional, but useful for the example below
method seek {ch offset base} {
    switch $base {
        start {
            set pos $offset
        }
        current {
            incr pos $offset
        }
        end {
            set pos [string length $data]
            incr pos $offset
        }
    }
    if {$pos < 0} {
        set pos 0
    } elseif {$pos > [string length $data]} {
        set pos [string length $data]
    }
    return $pos
}

# Now we create an instance...
set string "The quick brown fox jumps over the lazy dog.\n"
set ch [chan create read [stringchan new $string]]

puts [gets $ch];    # Prints the whole string

seek $ch -5 end;
puts [read $ch];    # Prints just the last word
```

SEE ALSO

chan(n), transchan(n)

KEYWORDS

API, channel, ensemble, prefix, reflection

NAME

`regexp` - Match a regular expression against a string

SYNOPSIS

```
regexp ?switches? exp string ?matchVar? ?subMatchVar  
subMatchVar ...?
```

DESCRIPTION

Determines whether the regular expression *exp* matches part or all of *string* and returns 1 if it does, 0 if it does not, unless **-inline** is specified (see below). (Regular expression matching is described in the **re_syntax** reference page.)

If additional arguments are specified after *string* then they are treated as the names of variables in which to return information about which part(s) of *string* matched *exp*. *MatchVar* will be set to the range of *string* that matched all of *exp*. The first *subMatchVar* will contain the characters in *string* that matched the leftmost parenthesized subexpression within *exp*, the next *subMatchVar* will contain the characters that matched the next parenthesized subexpression to the right in *exp*, and so on.

If the initial arguments to **regexp** start with **-** then they are treated as switches. The following switches are currently supported:

-about	Instead of attempting to match the regular expression, returns a list containing information about the regular expression. The first element of the list is a subexpression count. The second element is a list of property
---------------	---

coreTools Command Reference Index

	names that describe various attributes of the regular expression. This switch is primarily intended for debugging purposes.
-expanded	Enables use of the expanded regular expression syntax where whitespace and comments are ignored. This is the same as specifying the (?x) embedded option (see the re_syntax manual page).
-indices	Changes what is stored in the <i>matchVar</i> and <i>subMatchVars</i> . Instead of storing the matching characters from <i>string</i> , each variable will contain a list of two decimal strings giving the indices in <i>string</i> of the first and last characters in the matching range of characters.
-line	Enables newline-sensitive matching. By default, newline is a completely ordinary character with no special meaning. With this flag, bracket expressions and never match newline, matches an empty string after any newline in addition to its normal function, and matches an empty string before any newline in addition to its normal function. This flag is equivalent to specifying both -linestop and -lineanchor , or the (?n) embedded option (see the re_syntax manual page).
-linestop	Changes the behavior of bracket expressions and so that they stop at newlines. This is the same as specifying the (?p) embedded option (see the re_syntax manual page).
-lineanchor	Changes the behavior of ^ and \$ (the so they match the beginning and end of a line respectively. This is the same as specifying the (?w) embedded option (see the re_syntax manual page).
-nocase	Causes upper-case characters in <i>string</i> to be treated as lower case during the matching process.
-all	Causes the regular expression to be matched as many times as possible in the string, returning the total number of matches found. If this is specified with match variables, they will contain information for the last match only.
-inline	Causes the command to return, as a list, the data that would otherwise be placed in match variables. When using -inline , match variables may not be specified. If used with -all , the list will be concatenated at each iteration, such

coreTools Command Reference Index

that a flat list is always returned. For each match iteration, the command will append the overall match data, plus one element for each subexpression in the regular expression. Examples are:

```
regex -inline -- {\w(\w)} " inlined "  
-> in n  
regex -all -inline -- {\w(\w)} " inlined "  
-> in n li i ne e
```

- start index** Specifies a character index offset into the string to start matching the regular expression at. The *index* value is interpreted in the same manner as the *index* argument to **string index**. When using this switch, will not match the beginning of the line, and \A will still match the start of the string at *index*. If **-indices** is specified, the indices will be indexed starting from the absolute beginning of the input string. *index* will be constrained to the bounds of the input string.
- Marks the end of switches. The argument following this one will be treated as *exp* even if it starts with a **-**.

If there are more *subMatchVars* than parenthesized subexpressions within *exp*, or if a particular subexpression in *exp* does not match the string (e.g. because it was in a portion of the expression that was not matched), then the corresponding *subMatchVar* will be set to **if -indices** has been specified or to an empty string otherwise.

EXAMPLES

Find the first occurrence of a word starting with **foo** in a string that is not actually an instance of **foobar**, and get the letters following it up to the end of the word into a variable:

```
regex {\mfoo(?!bar\M)(\w*)} $string -> restOfWord
```

Note that the whole matched substring has been placed in the variable which is a name chosen to look nice given that we are not actually interested in its contents.

Find the index of the word **badger** (in any case) within a string and store that in the variable **location**:

```
regex -indices {(?i)\mbadger\M} $string location
```

This could also be written as a *basic* regular expression (as opposed to using the default syntax of

coreTools Command Reference Index

advanced regular expressions) match by prefixing the expression with a suitable flag:

```
regex -indices {(?ib)\<badger\>} $string location
```

This counts the number of octal digits in a string:

```
regex -all {[0-7]} $string
```

This lists all words (consisting of all sequences of non-whitespace characters) in a string, and is useful as a more powerful version of the **split** command:

```
regex -all -inline {\S+} $string
```

SEE ALSO

re_syntax(n), regsub(n), string(n)

KEYWORDS

match, parsing, pattern, regular expression, splitting, string

register

Describes a register within an address block.

Description

This item describes a register within an address block. The register can also hold (as children) other items to describe the internal structure (fields) of the register.

See Also

[addressBlock \(3\)](#), [create_register \(2\)](#) [registerField \(3\)](#)

Supported Attributes

[AddressOffset \(3\)](#), [CHdrValue \(3\)](#), [ComponentOfItem \(3\)](#), [Description \(3\)](#), [DocAddressOffset \(3\)](#), [DocBlockTableAddressOffset \(3\)](#),
[DocDescriptionField \(3\)](#), [DocEnabled \(3\)](#), [DocGroup \(3\)](#), [DocInclude \(3\)](#),
[DocMaster \(3\)](#), [DocMemoryAccess \(3\)](#), [DocOffset \(3\)](#),
[DocRegTableAddressOffset \(3\)](#), [DocRegisterResetMask \(3\)](#),
[DocRegisterResetValue \(3\)](#), [DocRegisterSize \(3\)](#), [DocVisible \(3\)](#), [DocWaiver \(3\)](#),
[GroupImage \(3\)](#), [GroupImageAttrs \(3\)](#), [GroupImageTitle \(3\)](#), [GroupText \(3\)](#),
[HeaderNameFormat \(3\)](#), [Label \(3\)](#), [MemoryAccess \(3\)](#),
[MemoryAccessDescription \(3\)](#), [Name \(3\)](#), [PingTestMask \(3\)](#),
[RALSuppressionComment \(3\)](#), [RegisterResetMask \(3\)](#), [RegisterResetValue \(3\)](#),
[RegisterSize \(3\)](#), [Sequence \(3\)](#), [SideEffects \(3\)](#), [SpiritContainer \(3\)](#),
[TypeName \(3\)](#), [UndefinedBitValue \(3\)](#), [VerilogHeaderValue \(3\)](#),
[VhdlHeaderValue \(3\)](#), [Visible \(3\)](#)

registerArray

Describes a registerArray within an address block or register array.

Description

This item describes a registerArray within an address block or register array. The register can also hold (as children) other items to describe the internal structure (fields) of the registerArray.

See Also

[addressBlock \(3\)](#), [create_register_array \(2\)](#) [remove_register_array \(2\)](#)
[set_register_array_attribute \(2\)](#), [get_register_array_attribute \(2\)](#),
[AddressOffset \(3\)](#), [MemoryRange \(3\)](#), [RegisterArrayDimensions \(3\)](#), [register](#)
[\(3\)](#)

Supported Attributes

[AddressOffset \(3\)](#), [ComponentOfItem \(3\)](#), [Description \(3\)](#), [DocAddressOffset](#)
[\(3\)](#), [DocBlockTableAddressOffset \(3\)](#), [DocGroup \(3\)](#), [DocInclude \(3\)](#),
[DocMaster \(3\)](#), [DocMemoryAccess \(3\)](#), [DocRegTableAddressOffset \(3\)](#),
[DocWaiver \(3\)](#), [GroupImage \(3\)](#), [GroupImageAttrs \(3\)](#), [GroupImageTitle \(3\)](#),
[GroupText \(3\)](#), [Label \(3\)](#), [MemoryAccess \(3\)](#), [MemoryAccessDescription \(3\)](#),
[MemoryRange \(3\)](#), [Name \(3\)](#), [RALSuppressionComment \(3\)](#),
[RegisterArrayDimensions \(3\)](#), [Sequence \(3\)](#), [SpiritContainer \(3\)](#), [TypeName](#)
[\(3\)](#), [UndefinedBitValue \(3\)](#), [Visible \(3\)](#)

registerField

Describes a field within a register.

Description

The register field is used to describe the structure of a register in further detail. These items describe the individual bits and their uses.

See Also

[register \(3\)](#), [create_register_field \(2\)](#)

Supported Attributes

[BitAddressOffset \(3\)](#), [BitRange \(3\)](#), [CHeaderValue \(3\)](#), [ComponentOffItem \(3\)](#), [Description \(3\)](#), [DocAddressOffset \(3\)](#), [DocBlockTableAddressOffset \(3\)](#), [DocDescriptionField \(3\)](#), [DocEnabled \(3\)](#), [DocInclude \(3\)](#), [DocMemoryAccess \(3\)](#), [DocRegTableAddressOffset \(3\)](#), [DocRegisterResetMask \(3\)](#), [DocRegisterResetValue \(3\)](#), [DocRegisterSize \(3\)](#), [DocTestable \(3\)](#), [DocVisible \(3\)](#), [DocWaiver \(3\)](#), [HeaderNameFormat \(3\)](#), [Label \(3\)](#), [MaxWriteConstraint \(3\)](#), [MemoryAccess \(3\)](#), [MemoryAccessDescription \(3\)](#), [MinWriteConstraint \(3\)](#), [Name \(3\)](#), [RALAccessType \(3\)](#), [RALAdditionalFieldAttributeText \(3\)](#), [RALSuppressionComment \(3\)](#), [ReadAction \(3\)](#), [ReadActionDescription \(3\)](#), [ReadActionModifier \(3\)](#), [RegisterResetMask \(3\)](#), [RegisterResetValue \(3\)](#), [RegisterSize \(3\)](#), [Reserved \(3\)](#), [Sequence \(3\)](#), [SideEffects \(3\)](#), [SpiritContainer \(3\)](#), [Testable \(3\)](#), [TypeName \(3\)](#), [UVMRALAccessType \(3\)](#), [VerilogHeaderValue \(3\)](#), [VhdlHeaderValue \(3\)](#), [Visible \(3\)](#), [WriteBehavior \(3\)](#), [WriteBehaviorModifier \(3\)](#), [WriteConstraint \(3\)](#)

register_netlist

Register a user-defined netlist for use in Generate Subsystem RTL.

Syntax

```
string register_netlist -language <language> [-callback <proc name>]
[-unregister]
    string <language>
    string <proc name>
```

Parameters

-language <language>	Netlist output language (Values: SystemC, SystemVerilog, UserDefined)
-callback <proc name>	Procedure to be called to invoke netlist
-unregister	Unregister a user-defined netlist

Description

This command can be utilized to register a user-defined netlist within coreAssembler. When a netlist is registered and associated with a particular language, then the language will show up as an additional choice in the list of languages available in the Generate Subsystem RTL activity. If that language is then selected, the user-defined netlist will be run when the activity is completed (by clicking 'Apply' in the GUI or when the activity is completed in batch mode with the autocomplete_activity command).

The netlist must be defined before it is invoked. Definition of the netlist and its inclusion into coreAssembler are the responsibility of the end user. The most common way to make this happen is through the definition of a plugin which defines the command.

The netlist will be invoked exactly once for each hierarchical design within the subsystem which requires a netlist to be generated. It will be invoked without any arguments and should return a list of the files it generated. The returned list can be either a simple list of files (i.e. {<file 1> <file 2> ...}) or it can be a list of two element sub-lists where the first sub-element is the file name and the second sub-element is a brief abstract describing the file contents. If provided, the abstract information will be included in the report page generated upon completion of the Generate Subsystem RTL activity.

The outer wrapper of the netlister must be written in TCL but the core of the functionality can be written in either TCL or as an IP-XACT tight generator. In the case of a generator, the TCL wrapper would directly invoke the generator using the `invoke_generator` command. A simple example of this is shown below.

```
proc mySystemCnetlister {} {  
  invoke_generator mySystemCgenerator  
  set files [glob [get_logical_dir src] *.*]  
  return $files  
}
```

Examples

Register a System C netlister:

```
register_netlister -language SystemC -callback mySystemCnetlister
```

See Also

NAME

registry - Manipulate the Windows registry

SYNOPSIS

package require registry 1.3

registry *?-mode? option keyName ?arg arg ...?*

DESCRIPTION

The **registry** package provides a general set of operations for manipulating the Windows registry. The package implements the **registry** Tcl command. This command is only supported on the Windows platform. Warning: this command should be used with caution as a corrupted registry can leave your system in an unusable state.

KeyName is the name of a registry key. Registry keys must be one of the following forms:

\\hostname\rootname\keypath

rootname\keypath

rootname

Hostname specifies the name of any valid Windows host that exports its registry. The *rootname* component must be one of **HKEY_LOCAL_MACHINE**, **HKEY_USERS**, **HKEY_CLASSES_ROOT**, **HKEY_CURRENT_USER**, **HKEY_CURRENT_CONFIG**, **HKEY_PERFORMANCE_DATA**, or **HKEY_DYN_DATA**. The *keypath* can be one or more registry key names separated by backslash (\) characters.

The optional *-mode* argument indicates which registry to |

coreTools Command Reference Index

work with; when it is **-32bit** the 32-bit registry will be used, and when it is **-64bit** the 64-bit registry will be used. If this argument is omitted, the system's default registry will be the subject of the requested operation.

Option indicates what to do with the registry key name. Any unique abbreviation for *option* is acceptable. The valid options are:

registry broadcast *keyName* *?-timeout milliseconds?*

Sends a broadcast message to the system and running programs to notify them of certain updates. This is necessary to propagate changes to key registry keys like Environment. The timeout specifies the amount of time, in milliseconds, to wait for applications to respond to the broadcast message. It defaults to 3000. The following example demonstrates how to add a path to the global Environment and notify applications of the change without requiring a logoff/logon step (assumes admin privileges):

```
set regPath [join {
    HKEY_LOCAL_MACHINE
    SYSTEM
    CurrentControlSet
    Control
    {Session Manager}
    Environment
} "\\"]
set curPath [registry get $regPath "Path"]
registry set $regPath "Path" "$curPath;$addPath"
registry broadcast "Environment"
```

registry delete *keyName* *?valueName?*

If the optional *valueName* argument is present, the specified value under *keyName* will be deleted from the registry. If the optional *valueName* is omitted, the specified key and any subkeys or values beneath it in the registry hierarchy will be deleted. If the key could not be deleted then an error is generated. If the key did not exist, the command has no effect.

registry get *keyName* *valueName*

Returns the data associated with the value *valueName* under the key *keyName*. If either the key or the value does not exist, then an error is generated. For more details on the format of the returned data, see **SUPPORTED TYPES**, below.

registry keys *keyName* *?pattern?*

If *pattern* is not specified, returns a list of names of all the subkeys of *keyName*. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for **string match**. If the specified *keyName* does not exist, then an error is generated.

registry set *keyName* *?valueName data ?type??*

If *valueName* is not specified, creates the key *keyName* if it does not already exist. If *valueName* is specified, creates the key *keyName* and value *valueName*

coreTools Command Reference Index

if necessary. The contents of *valueName* are set to *data* with the type indicated by *type*. If *type* is not specified, the type **sz** is assumed. For more details on the data and type arguments, see **SUPPORTED TYPES** below.

registry type *keyName valueName*

Returns the type of the value *valueName* in the key *keyName*. For more information on the possible types, see **SUPPORTED TYPES**, below.

registry values *keyName ?pattern?*

If *pattern* is not specified, returns a list of names of all the values of *keyName*. If *pattern* is specified, only those names matching *pattern* are returned.

Matching is determined using the same rules as for **string match**.

SUPPORTED TYPES

Each value under a key in the registry contains some data of a particular type in a type-specific representation. The **registry** command converts between this internal representation and one that can be manipulated by Tcl scripts. In most cases, the data is simply returned as a Tcl string. The type indicates the intended use for the data, but does not actually change the representation. For some types, the **registry** command returns the data in a different form to make it easier to manipulate. The following types are recognized by the registry command:

binary	The registry value contains arbitrary binary data. The data is represented exactly in Tcl, including any embedded nulls.
none	The registry value contains arbitrary binary data with no defined type. The data is represented exactly in Tcl, including any embedded nulls.
sz	The registry value contains a null-terminated string. The data is represented in Tcl as a string.
expand_sz	The registry value contains a null-terminated string that contains unexpanded references to environment variables in the normal Windows style (for example, %var%). The data is represented in Tcl as a string.
dword	The registry value contains a little-endian 32-bit number. The data is represented in Tcl as a decimal string.
dword_big_endian	The registry value contains a big-endian 32-bit number. The data is represented in Tcl as a decimal string.

coreTools Command Reference Index

string.

link	The registry value contains a symbolic link. The data is represented exactly in Tcl, including any embedded nulls.
multi_sz	The registry value contains an array of null-terminated strings. The data is represented in Tcl as a list of strings.
resource_list	The registry value contains a device-driver resource list. The data is represented exactly in Tcl, including any embedded nulls.

In addition to the symbolically named types listed above, unknown types are identified using a 32-bit integer that corresponds to the type code returned by the system interfaces. In this case, the data is represented exactly in Tcl, including any embedded nulls.

PORTABILITY ISSUES

The registry command is only available on Windows.

EXAMPLE

Print out how double-clicking on a Tcl script file will invoke a Tcl interpreter:

```
package require registry
set ext .tcl

# Read the type name
set type [registry get HKEY_CLASSES_ROOT\\$ext {}]
# Work out where to look for the command
set path HKEY_CLASSES_ROOT\\$type\\Shell\\Open\\command
# Read the command!
set command [registry get $path {}]

puts "$ext opens with $command"
```

KEYWORDS

registry

NAME

`regsub` - Perform substitutions based on regular expression pattern matching

SYNOPSIS

`regsub` *?switches? exp string subSpec ?varName?*

DESCRIPTION

This command matches the regular expression *exp* against *string*, and either copies *string* to the variable whose name is given by *varName* or returns *string* if *varName* is not present. (Regular expression matching is described in the **re_syntax** reference page.) If there is a match, then while copying *string* to *varName* (or to the result of this command if *varName* is not present) the portion of *string* that matched *exp* is replaced with *subSpec*. If *subSpec* contains a `|` or `&` then it is replaced in the substitution with the portion of *string* that matched *exp*. If *subSpec* contains a `[n]` where *n* is a digit between 1 and 9, then it is replaced in the substitution with the portion of *string* that matched the *n*'th parenthesized subexpression of *exp*. Additional backslashes may be used in *subSpec* to prevent special interpretation of and backslashes. The use of backslashes in *subSpec* tends to interact badly with the Tcl parser's use of backslashes, so it is generally safest to enclose *subSpec* in braces if it includes backslashes.

If the initial arguments to **regsub** start with `-` then they are treated as switches. The following switches are currently supported:

-all

All ranges in *string* that match *exp* are found and

coreTools Command Reference Index

substitution is performed for each of these ranges. Without this switch only the first matching range is found and substituted. If **-all** is specified, then and sequences are handled for each substitution using the information from the corresponding match.

-expanded

Enables use of the expanded regular expression syntax where whitespace and comments are ignored. This is the same as specifying the **(?x)** embedded option (see the **re_syntax** manual page).

-line

Enables newline-sensitive matching. By default, newline is a completely ordinary character with no special meaning. With this flag, bracket expressions and never match newline, matches an empty string after any newline in addition to its normal function, and matches an empty string before any newline in addition to its normal function. This flag is equivalent to specifying both **-linestop** and **-lineanchor**, or the **(?n)** embedded option (see the **re_syntax** manual page).

-linestop

Changes the behavior of bracket expressions and so that they stop at newlines. This is the same as specifying the **(?p)** embedded option (see the **re_syntax** manual page).

-lineanchor

Changes the behavior of **^** and **\$** (the so they match the beginning and end of a line respectively. This is the same as specifying the **(?w)** embedded option (see the **re_syntax** manual page).

-nocase

Upper-case characters in *string* will be converted to lower-case before matching against *exp*; however, substitutions specified by *subSpec* use the original unconverted form of *string*.

-start index

Specifies a character index offset into the string to start matching the regular expression at. The *index* value is interpreted in the same manner as the *index* argument to **string index**. When using this switch, will not match the beginning of the line, and **\A** will still match the start of the string at *index*. *index* will be constrained to the bounds of the input string.

--

Marks the end of switches. The argument following this one will be treated as *exp* even if it starts with a **-**.

If *varName* is supplied, the command returns a count of the number of matching ranges that were found and replaced, otherwise the string after replacement is returned. See the manual entry for **regexp** for details on the interpretation of regular expressions.

EXAMPLES

Replace (in the string in variable *string*) every instance of **foo** which is a word by itself with **bar**:

```
regsub -all {\mfoo\M} $string bar string
```

or (using the syntax):

```
regsub -all {(?b)\<foo\>} $string bar string
```

Insert double-quotes around the first instance of the word **interesting**, however it is capitalized.

```
regsub -nocase {\yinteresting\y} $string {"&} string
```

Convert all non-ASCII and Tcl-significant characters into \u escape sequences by using **regsub** and **subst** in combination:

```
# This RE is just a character class for almost everything "bad"
set RE {[{};#\$\ \r\t\u0080-\uffff]}
```

```
# We will substitute with a fragment of Tcl script in brackets
set substitution {[format \\\u%04x [scan "\&" %c]]}
```

```
# Now we apply the substitution to get a subst-string that
# will perform the computational parts of the conversion. Note
# that newline is handled specially through string map since
# backslash-newline is a special sequence.
set quoted [subst [string map {\n {\u000a}} \
    [regsub -all $RE $string $substitution]]]
```

SEE ALSO

regexp(n), re_syntax(n), subst(n), string(n)

KEYWORDS

match, pattern, quoting, regular expression, substitution

`reinitialize_component`

Re-initialize a component through resetting all design parameters in `coreAssembler` or `coreConsultant`.

Syntax

```
string reinitialize_component [-component <component>]  
    string <component>
```

Parameters

<code>-component</code>	Re-initialize all the visible parameters for
<code><component></code>	given component.

Description

Resets all parameters for a given component by doing most of what `reinitialize_workspace` does except limiting it to a component context. For assembler, the current context is assumed if the component option is not used. If component option is provided and it turns out to be a hierarchical component, the procedure returns with an error.

Examples

Assume 3 hierarchies and a component in each of them

```
coreAssembler> all_components -recursive -exclude_hier  
/h0/m0 /h1/m1 /h2/m2 /h3/m3  
coreAssembler> reinitialize_component -component "/h1/m1"  
coreAssembler> set_current_component /h0/m0  
coreAssembler> reinitialize_component  
coreAssembler> set_current_component /h3  
coreAssembler> reinitialize_component -component m3
```

See Also

[reinitialize_workspace \(2\)](#)

Re-initialize workspace through resetting all design parameters and activity parameters for each given activity

Syntax

```
string reinitialize_workspace [-activity activity]  
    string activity
```

Parameters

-activity	Re-initialize all the visible parameters of given
activity	activities.

Description

reinitialize_workspace This command is used to reinitialize all the non-readOnly and visible design parameters. It also takes a list of activities and resets all the visible parameters of each activity given in the input list.

Examples

The following command only reinitializes all visible and non-readOnly design parameters.

```
coreConsultant> reinitialize_workspace
```

The following command reinitializes all design parameters and DW_apb_rap_Simulate activity parameters.

```
coreConsultant> reinitialize_workspace -activity {DW_apb_rap_Simulate}
```

See Also

[API Index](#) [Command Reference](#)

remove_address_bank

Remove an address bank.

Syntax

string **remove_address_bank** *name*
 string *name*

Parameters

name Name of the address bank to remove.

Description

Removes an address bank. Removes the bank and all of its children.

Examples

```
coreBuilder> remove_address_bank map/bank1  
.
```

See Also

[create_address_bank \(2\)](#)

[API Index](#) [Command Reference](#)

remove_address_block
Remove an address block.

Syntax

string **remove_address_block** *block*
string *block*

Parameters

block Name of the address block to remove.

Description

Examples

See Also

[API Index](#) [Command Reference](#)

remove_address_space

Remove an address space.

Syntax

string **remove_address_space** *name*
 string *name*

Parameters

name The unique name of the address space.

Description

This command is used to create or modify an addressSpace defined on a design.

Examples

remove_address_space -name lPort

See Also

[create_address_space \(2\)](#), [BaseAddress \(3\)](#), [AddressSpaceRef \(3\)](#)

remove_allow_paths

Remove -allow_paths from the named clock group.

Syntax

string **remove_allow_paths** [*names*]
list *names*

Parameters

names Clock groups from which -allow_paths option should be removed.

Description

This command is used to remove the -allow_paths option from previously processed set_clock_groups commands. This is only required in a coreAssembler context when propagating constraints where some components define clock groups using -allow_paths and some components do not. In that case the propagated set_clock_group commands will be inconsistent at the system level. Ideally, all leaf components would use this option consistently, but when they do not, removing the -allow_paths option is a viable work-around.

If no clock group name is specified, -allow_paths is removed from all groups.

Examples

Remove -allow_paths from previously defined clock group MYGROUP:

```
remove_allow_paths MYGROUP
```

Remove -allow_paths from all clock groups:

```
remove_allow_paths
```

See Also

remove_area_estimates

Remove area estimation values for the current design.

Syntax

string **remove_area_estimates** [-configuration <configuration>] [-label <label>]

string <configuration>

string <label>

Parameters

-configuration

<configuration>

-label <label>

Configuration values for the area estimate to be removed

Label for the configuration to be removed

Description

This command is used to remove area estimates from a design. It will typically only be needed in coreAssembler when additional estimates have been added manually, and there is a subsequent desire to remove them. Without any arguments, all area estimates are removed. If either the -label or -configuration argument is specified, then any matching area estimates are removed.

Examples

Remove an area estimate created with a configuration label.

```
prompt> remove_area_estimates -label Max
```

See Also

[set_area_estimate \(2\)](#)

[API Index](#) [Command Reference](#)

remove_attached_interface
Delete attached interfaces.

Syntax

string **remove_attached_interface** *interfaces*
 string *interfaces*

Parameters

interfaces List of attached interfaces to be removed

Description

This command is used to remove an 'attached' interface instance from a component.

Examples

Remove attached interface 'myIntf' from component 'theSlave':

```
coreAssembler> remove_attached_interface theSlave/myIntf
```

See Also

[attach_interface \(3\)](#)

NAME

remove_command_hook

Remove hook from command execution

SYNTAX

```
string remove_command_hook -before name  
                        -after name -replace name commandName
```

```
string    name  
string    commandName
```

ARGUMENTS

-before *name* Remove the before hook named *name*

-after *name* Remove the after hook named *name*

-replace *name* Remove the replace hook named *name*

commandName Command to update

DESCRIPTION

The **remove_command_hook** removes a hook previously added to a command. When a hook is created via the **add_command_hook** the name of the hook is returned. Use that name to remove the hook. The `get_command_hooks` command can also be used to determine the name of each registered hook.

EXAMPLES

```
prompt> remove_command_hook place_opt -before before0
```

SEE ALSO

```
add_command_hook(2)  
get_current_hook_command(2)  
get_command_hooks(2)
```

remove_component

Remove the specified component instance from the subsystem.

Syntax

string **remove_component** *instanceName*
 string *instanceName*

Parameters

instanceName Component instantiation to be removed.

Description

This command is used to remove the named component from the subsystem. The cell associated with the component is removed from the subsystem-level design, and the associated component level workspace is removed. This command is the batch inverse of the `instantiate_component` and `import_component` commands. It is not typically called directly but can be called in batch environment if needed.

If the component has interfaces attached to it then all these interfaces are detached prior to remove the component cell.

You cannot remove a component which is source for an exported interface or an attached interface: You must remove these interfaces first (just 'unconnect_interface' is not satisfying).

Examples

To add and then remove the component AHBinst:

```
prompt> instantiate_component -name AHBinst -installation ./installations
prompt> remove_component AHBinst
```

See Also

[instantiate_component \(2\)](#), [import_component \(2\)](#), [detach_interface \(2\)](#), [remove_exported_interface \(2\)](#)

[API Index](#) [Command Reference](#)

`remove_component_view`

Remove an IP-XACT component view.

Syntax

string **`remove_component_view`** *name*
 string *name*

Parameters

name Name of the view to be removed.

Description

Deletes the named IPXACT view.

Examples

`remove_component_view my_view`

See Also

`create_component_view` (2)

remove_connection

Remove the specified connections.

Syntax

```
string remove_connection [-keep_ports] [-hierarchy] conns  
string conns
```

Parameters

-keep_ports	Keep disconnected ports for unconnected items
-hierarchy	Allows for removal of connections at lower hierarchy if specified
conns	ports and pins to unconnect

Description

This command is used to remove connections. It removes each connection point from the set of connections that it is currently connected to. It is used to undo connections created by `export_pin` and `create_connection`. This command is not typically called directly because GUI users will typically use the GUI connection removal interface, and batch users will simply update their batch scripts and recreate the workspace.

Examples

To create a connection from port A to two different loads and then disconnect one of the loads:

```
coreAssembler> create_connection {A U1/Z U2/Z}  
coreAssembler> remove_connection U2/Z
```

See Also

[create_connection \(2\)](#), [export_pin \(2\)](#), [translate_interface_connection \(2\)](#)

remove_constraints

Remove constraints from the specified or current component and from all levels above.

Syntax

```
string remove_constraints [component]  
      string component
```

Parameters

component	Name of the component to remove constraints from. The default is the current component.
-----------	---

Description

The remove_constraints command removes design constraints from ports and designs in the specified or current component. In coreAssembler, design constraints are also removed from all levels above the specified component. This command can be used to remove constraints that are created by applying an SDC file with the read_sdc command.

The remove_constraints command must be run after the design is elaborated, and in coreAssembler when the activity InitializeSubsystemConstraints is incomplete.

Examples

In coreAssembler, the following command will remove design constraints from the component "i_hier_comp" and from all levels above.

```
remove_constraints i_hier_comp
```

See Also

[read_sdc\(2\)](#)

remove_exported_interface

Remove an exported interface instance from the subsystem.

Syntax

string **remove_exported_interface** *name*
 string *name*

Parameters

name Name of the exported interface instance to remove

Description

This command is used to remove an exported interface from a coreAssembler subsystem. Executing this command will cause the named exported interface to be removed. Connections to the exported interface, and ports associated with the interface are also automatically removed.

Examples

To remove an exported interface associated with the USB PHY:

```
prompt> remove_exported_interface USB_PHY
```

See Also

[export_interface \(2\)](#), [detach_interface \(2\)](#), [disconnect_interface \(2\)](#),
[remove_component \(2\)](#)

remove_from_collection

Remove object(s) from a collection. Result is new collection

Syntax

```
string remove_from_collection collection1 object_spec
      string collection1
      list object_spec
```

Parameters

	Base collection
collection1	This is copied to the result collection and objects matching object_spec are removed from the new collection.
	Object(s) to remove
object_spec	This is a list of named objects or collections. If the name matches an existing collection, the collection is used. Otherwise, the objects are searched for in the knowledge database as if the name had been passed to the find_item command.

Description

Because Tcl is a command-driven language, traditional operators like plus (+) and minus (-) have no special meaning unless a particular command (like expr) imposes some meaning. The remove_from_collection command can remove elements from a collection.

If nothing matches the object_spec, the resulting collection is a copy of the base collection. If everything in collection matches the object_spec, the result is the empty string.

Examples

The following example gets all input ports with the exception of "CLOCK". This is analogous to all_inputs() - "CLOCK" in dc_shell.

```
dwc_shell> set cPorts [remove_from_collection [all_inputs]
CLOCK] _sel7
```

See Also

add to collection (2), all inputs (2), find item (2) _____

[API Index](#) [Command Reference](#)`remove_generated_clock`

Remove the specified generated clock

Syntax

string **remove_generated_clock**

Description

Removes the specified generated clocks. An error is produced if the generated clock cannot be found, or if the specified clock is not a generated clock.

Examples

The following command removes the generated clock 'genClock':

```
remove_generated_clock genClock
```

See Also

[`create\_generated\_clock \(2\)`](#), [`get\_clocks \(2\)`](#)

[API Index](#) [Command Reference](#)

`remove_html_report_link`

Remove an HTML report link from the index page

Syntax

string **`remove_html_report_link`** *title*
 string *title*

Parameters

title The title of the link to remove

Description

Examples

See Also

[add_html_report_link\(2\)](#)

[API Index](#) [Command Reference](#)

`remove_interface`

Remove an interface definition and all its instances

Syntax

```
string remove_interface name  
      string name
```

Parameters

`name` Name of the interface to remove

Description

This command can be used (in coreBuilder only) to remove an interface definition and all of its instances. This command should not typically be needed as a better approach to removing an interface definition is to simply remove the name of the file defining the interface definition from the list of interface definition files in the LoadInterfaceDefns activity in coreBuilder.

Examples

To remove the AHB interface definition and all associated interface instances:

```
coreBuilder> remove_interface AHB
```

See Also

[remove_interface instance \(2\)](#)

[API Index](#) [Command Reference](#)

`remove_interface_instance`

Remove an instance of an interface associate with the core

Syntax

string **remove_interface_instance** *name*
 string *name*

Parameters

name Name of the instance to remove

Description

This command is used (in coreBuilder only) to remove an interface instance from a core. Other instances of the associated interface definition, and the interface definition itself are left unchanged.

Examples

To remove just the AHBMaster instance from a coreBuilder core:

```
coreBuilder> remove_interface_instance AHBMaster
```

See Also

[remove_interface \(2\)](#) [translate_interface_connection \(2\)](#)

`remove_ipxact_file`

remove a spirit file and related objects defined in the file.

Syntax

```
string remove_ipxact_file -type <file name> filename
      string <file name>
      string filename
```

Parameters

<code>-type <file name></code>	The top-level element type of the IP-XACT file. (Values: generatorChain)
<code>filename</code>	The name of the IP-XACT file to be processed.

Description

This command is used to remove IP-XACT generators and other elements that have been read into coreAssembler using the `read_ipxact_file` command.

This may be needed if you have updated the IP-XACT file on disk and want to see the updates within coreAssembler.

Examples

Add and then remove a generator chain:

```
coreAssembler> read_ipxact_file -type generatorChain mychain.xml
coreAssembler> remove_ipxact_file -type generatorChain mychain.xml
```

See Also

[read_ipxact_file \(2\)](#)

[API Index](#) [Command Reference](#)

remove_memory_map

Remove a memory map.

Syntax

string **remove_memory_map** [-quiet] *map*
string *map*

Parameters

-quiet	Remove the memory map silently.
map	Name of the memory map to remove.

Description

This command remove the named memory map.

Examples

The following command removes a memory map 'map1'

```
coreBuilder> remove_memory_map map1
```

See Also

[create_memory_map \(2\)](#), [set_memory_map_attribute \(2\)](#),
[get_memory_map_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

remove_register

Remove a register with the given name.

Syntax

string **remove_register** *name*
 string *name*

Parameters

 name Name of the register

Description

This command removes a register with given name.

Examples

The following command removes a register 'reg1' in memory map 'map1'.

```
coreBuilder>remove_register map1/block1/reg1
```

See Also

[create_register \(2\)](#)

[API Index](#) [Command Reference](#)

remove_register_array
Remove a registerArray.

Syntax

string **remove_register_array** *name*
 string *name*

Parameters

name Name of the registerArray to remove.

Description

This command removes a register array with given name.

Examples

The following command removes a register array 'regArray1' address block 'block1'.

```
coreBuilder>remove_register_array map1/block1/regArray1
```

See Also

[create_register_array \(2\).](#)

[API Index](#) [Command Reference](#)

remove_register_field
Remove a register field.

Syntax

string **remove_register_field** *name*
 string *name*

Parameters

name Name of the register field to remove.

Description

This command removes the given register field.

Examples

The following command removes a register field 'field1' in the given register 'reg1'.

```
coreBuilder> remove_register_field map1/block1/reg1/field1
```

See Also

[create_register_field \(2\)](#)

[API Index](#) [Command Reference](#)

remove_register_field_value
Remove a register field_value.

Syntax

string **remove_register_field_value** *name*
 string *name*

Parameters

name Name of the register field value to remove.

Description

Removes a register field value.

Examples

```
coreBuilder> remove_register_field_value \  
map1/block1/reg1/field1/val
```

See Also

[create_register_field_value \(2\)](#)

remove_virtual_clock

Remove the specified virtual clock

Syntax

```
string remove_virtual_clock [-design <design>] name
    string <design>
    string name
```

Parameters

	Specifies the design from which to remove the virtual clock.
-design <design>	If you do not specify a design, remove_virtual_clock removes the specified virtual clock from the current_design.
name	The name of the virtual clock to be removed.

Description

The remove_virtual_clock command removes the specified virtual clock from the specified design. If you do not specify a design, remove_virtual_clock removes the specified virtual clock from the current_design.

To create a virtual clock, use the create_virtual_clock command.

Examples

To remove the virtual clock named vclk from the current_design:

```
coreConsultant> remove_virtual_clock vclk
```

To remove the virtual clock named vclk from the design named Subblock_B:

```
coreConsultant> remove_virtual_clock -design Subblock_B vclk
```

See Also

[create_virtual_clock \(2\)](#), [get_clocks \(2\)](#)

NAME

`rename` - Rename or delete a command

SYNOPSIS

`rename` *oldName newName*

DESCRIPTION

Rename the command that used to be called *oldName* so that it is now called *newName*. If *newName* is an empty string then *oldName* is deleted. *oldName* and *newName* may include namespace qualifiers (names of containing namespaces). If a command is renamed into a different namespace, future invocations of it will execute in the new namespace. The **rename** command returns an empty string as result.

EXAMPLE

The **rename** command can be used to wrap the standard Tcl commands with your own monitoring machinery. For example, you might wish to count how often the **source** command is called:

```
rename ::source ::theRealSource
set sourceCount 0
proc ::source args {
    global sourceCount
    puts "called source for the [incr sourceCount]'th time"
    uplevel 1 ::theRealSource $args
}
```


SEE ALSO

`namespace(n)`, `proc(n)`

KEYWORDS

`command`, `delete`, `namespace`, `rename`

rename_component

rename a component in the subsystem.

Syntax

```
string rename_component oldComponentName newComponentName  
    string oldComponentName  
    string newComponentName
```

Parameters

oldComponentName	The name of the existing component to rename.
newComponentName	The new name to rename to.

Description

This command rename an old component in the subsystem to a new name.

Examples

To rename the existing i_ahb component name to u_ahb:

```
coreAssembler> rename_component i_ahb u_ahb
```

See Also

[rename_exported_interface \(2\), instantiate_component \(2\),](#)
[import_component \(2\), remove_component \(2\), all_components \(2\)](#)

rename_exported_interface

Rename an exported or attached interface instance in the subsystem.

Syntax

```
string rename_exported_interface oldInterfaceName newInterfaceName
      string oldInterfaceName
      string newInterfaceName
```

Parameters

<code>oldInterfaceName</code>	The name of the existing exported interface to rename.
<code>newInterfaceName</code>	The new name to rename to.

Description

This command rename an existing exported interface or attached interface to a new name.

Examples

To rename the existing `ex_AHB_MASTER` interface to `exported_master`:

```
coreAssembler> rename_exported_interface ex_AHB_MASTER exported_master
```

See Also

[rename_component \(2\)](#), [instantiate_component \(2\)](#), [import_component \(2\)](#), [remove_component \(2\)](#), [all_components \(2\)](#)

rename_port

Rename a hierarchical port

Syntax

```
string rename_port oldport newport  
      string oldport  
      string newport
```

Parameters

oldport	The name of the port to rename.
newport	The new name for the port.

Description

This command is used to rename a port created on a hierarchical design within coreAssembler. All existing connections to the port are preserved.

Examples

Rename port 'ABC' to have name 'DEF':

```
cA> rename_port ABC DEF
```

replace_component

replace a component in the subsystem.

Syntax

```
string replace_component -name <name> [-all] [-mapinfo ] component
      string <name>
      string
      string component
```

Parameters

-name <name>	name of the component to replace
-all	replace all components with the same design name if version updated
-mapinfo	Component mapping data list. The name of the component to instantiate.
component	The component to instantiate has to be available through the SearchPath parameter of the AddSubsystemComponents activity.

Description

This command is used to replace an existing component in the subsystem with a new component, usually due to version upgrade. The new component needs to be available through the SearchPath parameter of the AddSubsystemComponents activity. The component name will be the same after replacement.

Examples

To replace the existing AHB component i_ahb in the subsystem with a new version coreKit DW_ahb:

```
coreAssembler> replace_component -name i_ahb DW_ahb
```

See Also

[instantiate_component \(2\)](#), [remove_component \(2\)](#), [rename_component \(2\)](#),
[all_components \(2\)](#)

replace_interface

This command takes a connected interface (with SpecifySubsystem completed) and removes the auto connections of the interfaces and replaces them with manual connections all without incompleting SpecifySubsystem.

Syntax

```
string replace_interface -component <component_name> -interface
<interface_name> [-splitconnected] interface_list
[connected_interface_list]
    string <component_name>
    string <interface_name>
    string interface_list
    string connected_interface_list
```

Parameters

-component <component_name>	Component cell name
-interface <interface_name>	Specific interface to split appart
-splitconnected	Split both sides of a connected interface through any hierarchy to the target connection
interface_list	Specified interface gets split into this list of interfaces
connected_interface_list	Specified connected interface gets split into this list of interfaces and each interface connected

Description

The command splits the supplied interface instance into multiple interfaces. Then finds the interfacePorts for the master interface and moves the InterfaceLink setting to the split interfaces interfacePorts. The split interfaces are not modifiable after the command is complete. Interface connections are not modified.

When using the -splitconnected argument both sides of the connected interfaces are split through any hierarchy and the split interfaces are connected. The interfacePorts InterfaceLink attributes are updated. The master and split interfaces as well as the connections are user modifiable when the command is complete.

Examples

```
coreAssembler> replace_interface -component i_s4 -interface myConsumer
[list {myConsumer1 myIntf1} {myConsumer2 myIntf2} {myConsumer3 myIntf3}
{myConsumer4 myIntf4} {myConsumer5 myIntf5}]
coreAssembler> replace_interface -component i_s5 -interface myConsumer
-splitconnected [list {myConsumer1 myIntf1} {myConsumer2 myIntf2}
{myConsumer3 myIntf3} {myConsumer4 myIntf4} {myConsumer5 myIntf5}] [list
{myProvider1 myIntf1} {myProvider2 myIntf2} {myProvider3 myIntf3}
{myProvider4 myIntf4} {myProvider5 myIntf5}]
coreAssembler> replace_interface -component i_s5 -interface myConsumer
-splitconnected [list {myConsumer1 myIntf1} {myConsumer2 myIntf2}
{myConsumer3 myIntf3} {myConsumer4 myIntf4} {myConsumer5 myIntf5}] ]
```

See Also

translate_interface_connection (2), export_interface (2), _____
remove_exported_interface (2), unconnect_interface (2), _____
set_unused_interface (2), create_connection (2), remove_connection (2), _____
export_pin (2)

replay_connections

Replay manual connections.

Syntax

string **replay_connections** [-reset]

Parameters

-reset	Reset manual connections saved for all components or for the components specified by the -components option.
--------	--

Description

Examples

See Also

`report_activities`

Lists all activities and their current states

Syntax

string **report_activities**

Description

The `report_activities` command returns a list of all activities (in sequential order) associated with the currently loaded workspace and the status (enabled/complete) of each activity.

The `TypeName` column indicates the type (global or hierarchical) of the activity. A `GlobalActivity` is an activity that you execute; it is enabled when all of its prerequisite activities are completed. A `GlobalActivity` is complete when it is successfully executed.

A `HierActivity` is a parent "container" for a set of other activities, either global activities or other hierarchical activities. A `HierActivity` is enabled when all of its prerequisite activities are complete. A `HierActivity` is complete when all of its children (sub-activities) are complete.

Examples

To report activities for the current workspace:

```
coreConsultant> report_activities
Name TypeName Complete Enabled
-----
TechSetup GlobalActivity true true
SpecifyConfiguration GlobalActivity true true
SpecifyContextInformation GlobalActivity true true
SpecifyDesignIntent GlobalActivity true true
SpecifyPortIntent GlobalActivity true true
VerifyBudgets GlobalActivity true true
Synthesize GlobalActivity true true
AnalyzeResults GlobalActivity true true
ModelGeneration HierActivity true true
GenerateVmcmModel GlobalActivity true true
```

See Also

[report_activity_parameters \(2\)](#)

report_activity_parameters

List all parameters for each activity in the activity list.

Syntax

```
string report_activity_parameters [-hidden] activityList
      string activityList
```

Parameters

-hidden	Show hidden parameters.
activityList	List of activities (can be *).

Description

The report_activity_parameters command returns an indented list of all parameters associated with the specified activities and the values of the HelpText and Value attributes for each parameter.

The default behavior is to report the values of the HelpText and Value attributes for each parameter. Different attributes can be reported by adding their names to the global variable, report_activity_parameters_attributes.

Examples

To list parameters and attribute values associated with the VerifyBudgets activity, and report the value of the HelpText, Value and Sequence attributes:

```
coreConsultant> set ::report_activity_parameters_attributes
{HelpText Value Sequence}
HelpText Value Sequence
coreConsultant> report_activity_parameters {VerifyBudgets}
VerifyBudgets
DoConsistencyCheck
HelpText Description: Ensure budgets are consistent.
All electrically equivalent ports
must have equivalent values for
input/output delay, driving cell,
and load type. Clocks has period
greater than delay and uncertainty.
DefaultValue 1
Value 1
```

Sequence 1

DoCompletenessCheck

HelpText Description: Ensure constraints are complete.

All clocks have a period specified.

Ports have input/output delay
and driving cell set.

DefaultValue 1

Value 1

Sequence 0

See Also

get_activity_parameter (2), report_activities (2), set_activity_parameter (2)

report_activity_subproc

Report on activity/filegroup sub-process.

Syntax

```
string report_activity_subproc [-action <action>] activity
      string <action>
      string activity
```

Parameters

	Action to perform (Values: info, infolist, kill)
-action	Using '-action info' generates a formatted report containing sub-process information, including job id, job status, etc. Using the '-action infolist' option returns the same information, but in this case the result is returned as a TCL list so that it can be processed, as opposed to being reported. Using the '-action kill' option will kill the sub-process associated with the specified activity or filegroup.
<action>	
activity	Activity or filegroup associated with this process.

Description

This command is used to report on or kill the sub-process associated with the given activity or filegroup. The action taken is controlled by the -action switch. Valid options are described above.

Examples

```
report_activity_subproc Synthesize -action info
```

See Also

[launch_activity_subproc\(2\)](#), [define_activity_subproc_params\(2\)](#),
[wait_for_activity_subproc\(2\)](#), [prereq_activities_complete\(2\)](#)

NAME

report_app_var Shows the application variables.

SYNTAX

```
string report_app_var  
    [-verbose]  
    [-only_changed_vars]  
    [pattern]
```

Data Types

pattern string

ARGUMENTS

-verbose Shows detailed information.

-only_changed_vars
Reports only changed variables.

pattern Reports on variables matching the
pattern. The default is "*".

DESCRIPTION

The **report_app_var** command prints information about application variables matching the supplied pattern. By default, all descriptive information for the variable is printed, except for the help text.

If no variables match the pattern, an error is returned. Otherwise, this command returns the empty string.

If the **-verbose** option is used, then the command also prints the help text for the variable. This text is printed after the variable name and all lines of the help text are prefixed with " #".

coreTools Command Reference Index

The Constraints column can take the following forms:

<code>{val1 ...}</code>	The valid values must belong to the displayed list.
<code>val <= a</code>	The value must be less than or equal to "a".
<code>val >= b</code>	The value must be greater than or equal to "b".
<code>b <= val <= a</code>	The value must be greater than or equal to "b", and less than or equal to "a".

EXAMPLES

The following are examples of the **report_app_var** command:

```
prompt> report_app_var sh*
Variable          Value      Type      Default  Constraints
-----
sh_continue_on_error  false    bool      false
sh_script_stop_severity none      string    none      {none W E}
...

prompt> report_app_var sh* -verbose
Variable          Value      Type      Default  Constraints
-----
sh_continue_on_error  false    bool      false
# Allows source to continue after an error
sh_script_stop_severity none      string    none      {none W E}
# Indicates the error message severity level which would cause
# a script to stop executing before it completes
...
```

SEE ALSO

```
get_app_var(2)
set_app_var(2)
write_app_var(2)
```


report_attribute

Report on the specified (or all) attributes for the specified item(s)

Syntax

```
string report_attribute [-attrs <attribute names>] [-no_split]
[-subscript <attribute subscript>] [<item names>]
    list <attribute names>
    string <attribute subscript>
    list <item names>
```

Parameters

-attrs	Specifies the names of the attributes to include in the report.
<attribute names>	If you do not specify the -attrs option, report_attribute generates a report for all valid attributes on the specified item(s).
-no_split	Do not split long lines in report.
-subscript	Specifies which subscript of the specified attribute(s) to report.
<attribute subscript>	The report_attribute command will report on <attribute subscript> for each specified attribute. Use the -subscript option only if the specified subscript is valid on all specified attributes.
	The names of the items for which you want to report attributes.
<item names>	You can specify a single item, a list of items, or specify the items as a Tcl collection. If you do not specify <item names>, report_attribute reports the specified attributes on the items defined in the current_collection variable.

Description

The report_attribute command generates a tabular list of the current attribute values for the specified items.

report_attribute differs from get_attribute and similar commands (get_design_attribute, get_clock_attribute, and so on) in that it retrieves multiple attribute values on multiple items and returns the information in a tabular report format. If the current attribute value is a formula, report_attribute displays the formula without evaluating it.

If the rows in the generated table are excessively wide (many attributes or attributes have long names/values), `report_attribute` splits the rows for better readability. To maintain a single line of output for each item, use the `-no_split` option.

Examples

To report `MinInputDelay[clk]` and `MaxInputDelay[clk]` on all input ports of the `current_design`, except clocks and resets and not split lines:

```
coreConsultant> report_attribute -no_split -attrs
{MinInputDelay MaxInputDelay}
-subscript clk
[all_inputs -no_clocks -no_resets]
Report : Attributes
Item(s) : _sel1590
Version: 1999.05-CB1.0.1
Date : Wed Oct 27 18:19:06 1999
Item MinInputDelay[clk] MaxInputDelay[clk]
-----
input_1 =percent_of_period 15 =percent_of_period 15
input_2 =percent_of_period 15 =percent_of_period 15
input_3 =percent_of_period 25 =percent_of_period 25
input_4 =percent_of_period 25 =percent_of_period 25
input_5 =percent_of_period 10 =percent_of_period 10
To report the values of PreserveHierarchy and PreserveMapping on all
designs that are to be compiled individually:
```

```
coreConsultant> set current_collection [find_item -type
design -filter {MapBlockIndividually==true}]
{Subblock_A Subblock_B Subblock_C Subblock_A1 Subblock_A2 Subblock_A3}
coreConsultant> report_attribute -attrs {PreserveHierarchy
PreserveMapping}
Report : Attributes
Item(s) : <current collection>
Version: 1999.05-CB1.0.1
Date : Wed Oct 27 18:34:14 1999
Item PreserveHierarchy PreserveMapping
-----
Subblock_A true =InheritValue up 0
Subblock_B true =InheritValue up 0
Subblock_C true =InheritValue up 0
Subblock_A1 true =InheritValue up 0
Subblock_A2 true =InheritValue up 0
Subblock_A3 true =InheritValue up 0
```

See Also

[get_clock_attribute\(2\)](#), [get_design_attribute\(2\)](#), [get_port_attribute\(2\)](#), [get_cell_attribute\(2\)](#), [get_parameter_attribute\(2\)](#)

report_bom
Report Bill Of Materials

Syntax

```
string report_bom [-deliverables_only] [-required] [-missing] [-filter
<filter_expression>] [-attrs <attrs>]
    string <filter_expression>
    string <attrs>
```

Parameters

-deliverables_only	Limit the report to deliverables only. By default filegroupGroups will be included in the report, they are indicated by a preappended "*" to the name. This option Excludes filegroupGroups to be reported.
-required	Limit the report to show required deliverables only. Required deliverables are filegroups with attribute Importance set to "Required".
-missing	Limit the report to show missing deliverables only. Missing deliverables are required filegroups but missing file members, i.e., attribute Exists is "false".
-filter <filter_expression>	Limit the report to show a paticular set of deliverables. Specifies a filter expression to filter out a paticular set of filegroups to report. Please refer to the command find_item for filter expression.
-attrs <attrs>	List of attribute for deliverable to report. Specifies the names of the attributes to report. If you do not specify the -attrs option, report_bom will report a default set of attributes on filegroups. These attributes are: {DeliverableType Importance Exists IsUpToDate AutoloadFilegroup Configurable SourceDir}

Description

The report_bom command generates a tabular list of the current attribute values for the Bill of Materials (BoM) file groups in the current

workspace.

There are a couple of options you can specify to generate report for file groups you are interested. Note that option `-required`, `-missing`, `-filter`, `-attrs` imply `-deliverables_only`. If the current attribute value is a formula, `report_bom` displays the evaluated value of the formula. If the attribute value is a collection, it reports the name of each item in the collection.

If `report_bom` is invoked from GUI, the report will be displayed in the user's web browser. In addition, in `coreBuilder` GUI, `report_bom` can be invoked from the dialog for the Create Bill of Materials activity.

Examples

The following command reports missing BoM filegroups and only reports attributes Importance, Exists, and IsUpToDate:

```
coreBuilder> report_bom -missing -attrs {Importance Exists IsUpToDate}
coreBuilder> report_bom -filter "Importance==Required && Exists==0"
-attrs {Importance Exists IsUpToDate}
```

The following command reports the Test Regression Environment:

```
coreBuilder> report_bom -filter "DeliverableType==Verification ||
DeliverableType==TRE"
```

See Also

[report_attribute \(2\)](#), [check_bom \(2\)](#)

report_connections

Report on connected and unconnected component pins and subsystem ports.

Syntax

```
string report_connections [-connected] [-unconnected] [-format <format>]  
string <format>
```

Parameters

-connected	Report connected pins and ports only
-unconnected	Report unconnected pins and ports only
	The format of reporting: text is written to stdout. (Values: text, html, textConverted)
-format <format>	'text' is the default. Text format reports are written to the console. In the case of the 'html' format, the name of the generated HTML file is returned instead of the actual report. 'textConverted' format writes text to the console by converting generating HTML to text.

Description

This command is used to generate a report about the current 'connection state' of a subsystem. It can be used to list all connected and/or unconnected ports and pins in the subsystem. This command is not typically invoked directly but is available in the Manual Connections GUI by pressing the 'Report Connections' button.

Examples

To generate and show an HTML version of the connections report:

```
prompt> set file [report_connections -unconnected -format html]  
prompt> show_url $file
```

See Also

[report_interfaces\(2\)](#)

report_filegroup
Report file groups

Syntax

```
string report_filegroup [-attrs <attrs>] filegroups
      string <attrs>
      string filegroups
```

Parameters

	List of attributes to report.
-attrs	Specifies the names of the attributes to report. If you do not specify the -attrs option, report_filegroup will report a default set of attributes on filegroups. These attributes are: {FileContentType Modified HierName EncryptText DefaultInstallDir SourceDir}
<attrs>	
	List of filegroups to report.
filegroups	The names of the filegroups to report on. You can specify a single filegroup, a list of filegroups, or specify the filegroups as a Tcl collection. Wildcards can be used for a glob style match with the hierarchical name of the filegroup.

Description

The report_filegroup command generates a tabular list of the current attribute values for each file member of the the specified filegroups. If the current attribute value is a formula, report_filegroup displays the evaluated value of the formula. If the attribute value is a collection, it reports the name of each item in the collection.

Examples

The following command reports all top level filegroups on attributes FileContentType and Modified:

```
coreBuilder> report_filegroup * -attrs {FileContentType Modified}
```

See Also

[report_attribute \(2\)](#), [report_bom \(2\)](#), _____

report_interface_instances

generate textual or HDL description of the given interface instances

Syntax

string **report_interface_instances** [-vhdl] [-list] *instance*
string *instance*

Parameters

	Report in VHDL format
-vhdl	In this format, interface ports are generated as ports of a VHDL design and interface parameters as parameters of the VHDL entity.
-list	list interface instance names to report only
	List of interfaceInstances to report
instance	If none are specified, all instances are included in the report.

Description

This command is used to generate a report of the specified interface instances. Interface instances are associated with cores in coreBuilder or in the subsystem when an interface is exported. Each instance has a set of ports and parameters, as defined in the interface definition associated with the given interface instance. The generated report indicates how the interface is linked to it's core or top-level ports (for exported interfaces), and indicates the current values for all parameters.

For information about interface definitions (as opposed to instances), see the report_interfaces command.

Examples

To generate a report for all exported interface instances:

```
prompt> report_interface_instances [find_interface_instances -exported]
```

```
-----
-- Report : interfaceInstance
-- Interface : HCLK
-- Definition : DW-SoC::AHB-Clock
-- Version : 7.0
-- Component :
```

-- InterfaceType : AHB Clock Source
-- Tool Version : T-2022.06-SP2
-- Date : Mon Sep 19 07:29:48 PDT 2022

Port LinkDirection PortDirection InterfaceLink
hclk fromProvider out HCLK_hclk
Parameter Type Value InterfaceLink
NumConsumers integer 1 <open>
SlotNumber integer -1 <open>

See Also

[report_interfaces \(2\)](#)

report_interfaces

generate textual or HDL description of the given interface definitions

Syntax

string **report_interfaces** [-vhdl] [-list] *interface*
string *interface*

Parameters

	Report in VHDL format
-vhdl	In this format, interface ports are generated as ports of a VHDL design and interface parameters as parameters of the VHDL entity.
-list	list interface definition names to report only
	List of the interfaceDefns to report
interface	If none are specified, all definitions are included in the report.

Description

This command is used to generate a report of the specified interface definitions. Interface definitions are instantiated on cores in coreBuilder or in the subsystem when an interface is exported. It is the interface definition which defines the ports and parameters which make up a given interface.

For information about interface instances (as opposed to definitions), see the report_interface_instances command.

Examples

To generate a report for the DW-SoC::AHB-Clock interface definition:

```
prompt> report_interfaces DW-SoC::AHB-Clock
```

```
-----
-- Report : interfaceDefn
-- Interface : DW-SoC::AHB-Clock
-- Version : 7.0
-- Component :
-- Tool Version : T-2022.06-SP2
-- Date : Mon Sep 19 07:40:51 PDT 2022
-----
```

```
Port LinkDirection PortDirection InterfaceLink
```


hclk fromProvider out
Parameter Type Value InterfaceLink
NumConsumers integer 1 <open>
SlotNumber integer -1 <open>

See Also

[report interface instances \(2\)](#)

report_ip

Report on Synopsys IP used.

Syntax

```
string report_ip [-type <type>]  
    string <type>
```

Parameters

-type <type>	The type of the report to generate and display (Values: text, html)
--------------	--

Description

Generates an HTML or text report of the currently used Synopsys IP. By editing the application preferences for automatic IP checking you can configure the application to perform automatic IP checking. This insures that after an elapsed time period if you open a workspace or add a new component to an existing workspace a report is automatically generated for used Synopsys IP. Use the -type option to determine whether you are generating HTML or text.

Examples

See Also

report_license

List all currently checked out licenses

Syntax

string **report_license** [-authorized] [-opened] [-all] [-summary]

Parameters

-authorized	Returns a list of all licenses that are currently checked out or which have passed authorized checking
-opened	Returns union of unique LicenseForOpen values across all KBs in cC or KBs of a component in cA.
-all	Returns all license info checked out, authorized, and opened.
-summary	Summary of all authorized and checked out licenses.

Description

Without any options the report_license command lists the licenses that you have checked out in your current coreTool session and the reason that each license is checked out.

The -authorized, and -opened options report licenses for which authorization checks have been done, and those that have been checked out to open your workspace, respectively.

The -all options lists the current licenses (as with no options), along with the same output as -authorized and -opened.

The -summary option list a brief summary of all licenses authorized or checked out at any time during the coreTools session.

Examples

```
coreBuilder> report_license
Report : Checked Out Licenses
Version: W-2024.09-SP1
Date : Tue Sep 7 16:30:16 2024
License Name Reason Checked Out
coreBuilder Builder mode requested at startup.
```

See Also

[check_license \(2\)](#), [ProjectID \(3\)](#), [LicenseForOpen \(3\)](#)

report_memory_maps

Report on memory maps in a component.

Syntax

```
string report_memory_maps [-format <type>] [-file <file>] [-type <type>]  
    string <type>  
    string <file>
```

Parameters

-format <type>	The format of the report to generate and display (Values: text, html, xml)
-file <file>	The output file name for the report
-type <type>	Type of report to be generated (Values: basic, advanced, documentation)

Description

Generates an HTML or text report of the currently defined memory maps, memory registers, and register fields. Use the -type option to determine whether you are generating HTML or text. Use the -attributes option to select which group of attributes to display. "-attributes basic" displays Address Offset, Register Size, Reset Value and Description information, "-attributes advanced" displays information about IP-XACT Vendor Extensions, Endianness and Register Reset Masks;

Examples

See Also

[create_memory_map \(2\)](#), [create_register \(2\)](#), [create_register_field \(2\)](#)

report_prime_profile_validation_expr
Report validation expressions for Prime Profiles.

Syntax

```
string report_prime_profile_validation_expr [-out <file name>] [-format
<value>] [-component <leaf component object/name>]
    string <file name>
    string <value>
    string <leaf component object/name>
```

Parameters

-out <file name>	Write out a file.
-format <value>	Report format. (Values: tcl, yaml, json)
-component <leaf component object/name>	Prime Profile for a leaf component in coreAssembler.

Description

This command generates a report of all validation expressions defined for Prime Profiles in a given component. In coreAssembler, it must be invoked in the top level context. It takes an optional component argument without which Prime Profile validation expressions for all master components are reported. The output is in a YAML file format by default. An optional -file option to redirect the output is also provided.

Examples

```
coreConsultant> report_prime_profile_validation_expr
mycore:
0:
exprTxt: "@dummy == 1"
tagList: DisableRom1 DisableRom2
msg: "DisableRom(1|2) profiles must be applied when setting dummy to 1."
1:
exprTxt: "@dummy == 2"
tagList: LockRamSize
msg: ""
2:
exprTxt: "@dummy == 3"
tagList: Test003 BadLicense2
msg: ""
```

coreTools Command Reference Index

```
3:
exprTxt: "@ram_size == 32 &&@ram_word==64&& !@include_rom"
tagList: DisableRom2
msg: ""
4:
exprTxt: "@dummy == 5"
tagList: DontLockProfile
msg: ""
5:
exprTxt: "@dummy == 4"
tagList: BadLicense
msg: ""
6:
exprTxt: "@dummy_radio == 1"
tagList: DummyRadio
msg: ""
7:
exprTxt: "@dummy_expression == 1"
tagList: DummyExpr1
msg: ""
8:
exprTxt: "@dummy_checkbox == 1"
tagList: DummyCheckbox DummyCheckbox2
msg: ""
9:
exprTxt: "@ram_size == 32 && @ram_word == 64 && @include_rom && @rom_size
== 8 && @rom_word == 32"
tagList: DummyCheckbox2
msg: ""
coreConsultant> report_prime_profile_validation_expr -format tcl
mycore {0 {exprTxt {@dummy == 1} tagList {DisableRom1 DisableRom2} msg
{DisableRom(1|2) profiles must be applied when setting dummy to 1.}} 1
{exprTxt {@dummy == 2} tagList LockRamSize msg {}} 2 {exprTxt {@dummy ==
3} tagList {Test003 BadLicense2} msg {}} 3 {exprTxt {@ram_size == 32
&&@ram_word==64&& !@include_rom} tagList DisableRom2 msg {}} 4 {exprTxt
{@dummy == 5} tagList DontLockProfile msg {}} 5 {exprTxt {@dummy == 4}
tagList BadLicense msg {}} 6 {exprTxt {@dummy_radio == 1} tagList
DummyRadio msg {}} 7 {exprTxt {@dummy_expression == 1} tagList DummyExpr1
msg {}} 8 {exprTxt {@dummy_checkbox == 1} tagList {DummyCheckbox
DummyCheckbox2} msg {}} 9 {exprTxt {@ram_size == 32 && @ram_word == 64 &&
@include_rom && @rom_size == 8 && @rom_word == 32} tagList DummyCheckbox2
msg {}}
coreConsultant> report_prime_profile_validation_expr -format json -out
reports.json
{
"mycore": {
"0": {
"exprTxt": "@dummy == 1",
"tagList": "DisableRom1 DisableRom2",
"msg": "DisableRom(1|2) profiles must be applied when setting dummy to
1."

```

```

},
"1": {
  "exprTxt": "@dummy == 2",
  "tagList": "LockRamSize",
  "msg": ""
},
"2": {
  "exprTxt": "@dummy == 3",
  "tagList": "Test003 BadLicense2",
  "msg": ""
},
"3": {
  "exprTxt": "@ram_size == 32 &&@ram_word==64&& !@include_rom",
  "tagList": "DisableRom2",
  "msg": ""
},
"4": {
  "exprTxt": "@dummy == 5",
  "tagList": "DontLockProfile",
  "msg": ""
},
"5": {
  "exprTxt": "@dummy == 4",
  "tagList": "BadLicense",
  "msg": ""
},
"6": {
  "exprTxt": "@dummy_radio == 1",
  "tagList": "DummyRadio",
  "msg": ""
},
"7": {
  "exprTxt": "@dummy_expression == 1",
  "tagList": "DummyExpr1",
  "msg": ""
},
"8": {
  "exprTxt": "@dummy_checkbox == 1",
  "tagList": "DummyCheckbox DummyCheckbox2",
  "msg": ""
},
"9": {
  "exprTxt": "@ram_size == 32 && @ram_word == 64 && @include_rom &&
@rom_size == 8 && @rom_word == 32",
  "tagList": "DummyCheckbox2",
  "msg": ""
}
}
}

```

See Also

set_prime_profile_validation_expr (2)

clear_prime_profile_validation_exprs (2) create_prime_profile_file (2)

set_prime_profile (2) report_prime_profiles (2)

report_prime_profiles

Generates a Prime Profile report for a hierarchy/component.

Syntax

```
string report_prime_profiles [-out <file name>] [-format <value>]
[-verbose] [-component <leaf component object/name>]
    string <file name>
    string <value>
    string <leaf component object/name>
```

Parameters

-out <file name>	Write out a file.
-format <value>	Report format. (Values: tcl, yaml, json)
-verbose	Report includes the description.
-component <leaf component object/name>	Prime Profile for a leaf component in coreAssembler.

Description

This command generates a report of all Prime Profiles defined on a given component. In coreAssembler, it must be invoked in the top level context. It takes an optional component argument without which Prime Profiles for all master components are reported. The output is in a YAML file format. An optional -file option to redirect the output is also provided.

With Prime Profiles without triggers, valid Prime Profiles are listed (Prime Profiles with valid licenses). An additional `isValid` field is added under each Prime Profile. Profiles with defaults have `isDefault` field set to 1.

- For coreBuilder, reports also include a `config` key (with `cfg_file` or `body` strings as subkeys) with the path to the config file used in the `-cfg_file` option of the `create_prime_profile_file` command or the contents of the body used with the `-body` option. The config keys only show up if the `report_prime_profiles` command was issued in the same coreBuilder session as the one in which the `create_prime_profile_file` commands were issued. If a builder workspace is reopened, this information will not be found in the reports.
- External Prime Profile files have the key `isExternal` set to 1. For components where an external Prime Profiles has been applied, the report keys `setPrimeProfile` and `triggeredPrimeProfiles` keys store

the name of the external Prime Profile.

Note

The `-include_license` hidden option can be specified to report license strings in the report. This option is only allowed at Synopsys sites.

Examples

```
coreBuilder> report_prime_profiles -verbose
coreConsultant> report_prime_profiles -out prime_profiles.yaml
coreConsultant> report_prime_profiles -format tcl -verbose
coreConsultant> report_prime_profiles -format json -verbose -out
reports.json
coreAssembler> report_prime_profiles -component /h1/m1 -verbose
```

See Also

[set_prime_profile \(2\)](#) [create_prime_profile file \(2\)](#)

[report_prime_profile_validation_expr \(2\)](#)

[API Index](#) [Command Reference](#)

report_solutions

Generate simple report of available solutions.

Syntax

string **report_solutions** [-keys] [-ip] [-internal] [-recurse]

Parameters

-keys	Just return the solution key names.
-ip	Include IP requirements
-internal	Include internal solutions
-recurse	Include solutions defined within solutions

Description

This command is used to generate a simple report of the currently defined solution products in coreAssembler. Listed products can be instantiated using the `instantiate_solution` command.

If the `-keys` option is specified, the command returns the key names for all defined solutions. This is meant primarily for use within coreAssembler itself.

If the `-internal` option is specified, internal solutions will be included in the report.

Examples

See Also

[define_solution \(2\)](#)

report_suppressed_ral_tests

Generates a JSON report for all memory items that are untested and items which are tested with a non-empty string in the RALSuppressionComment attribute

Syntax

```
string report_suppressed_ral_tests -fileName [-itemType <type>
(default=all)] [-testType <test> (default=all)] [-verbose]
    string
    list <type> (default=all)
    list <test> (default=all)
```

Parameters

-fileName	Output file
-itemType <type> (default=all)	String list of the type of memory map item requested: all, register, registerField.
-testType <test> (default=all)	String list of the names of the tests: all, bit_bash, hw_reset, reg_access
-verbose	Also print json to screen

Description

Examples

See Also

report_tech_cell_mapping

Report technology specific cell mapping definitions and bindings.

Syntax

```
string report_tech_cell_mapping [-defines] [-bindings] [-file <file
name>]
    string <file name>
```

Parameters

-defines	Report technology cell map definitions
-bindings	Report technology cell map bindings
-file <file name>	File to redirect the output

Description

This command is used to dump information from the coreTools data model regarding technology cell binding definitions and the actual cell bindings (in coreConsultant and coreAssembler). See the examples below for information about dumped data.

Examples

List current technology cell binding definitions:

```
coreBuilder> report_tech_cell_mapping -defines
```

```
# Begin Tech Cell Mapping Definitions:
```

```
Module: DWbb_bcm21 Macro: DWC_AND_2_SRC Design: design
```

```
Definition: 1 / {{ Vectorized 2-input AND Gate } U_AND {[AWIDTH-1:0]} {i_a
i_b o_z} { "Input A" "Input B" "Output Z" } DWbb_bcm21.v {} 0 {}
INVERTED_OUT 0}
```

```
/slowfs/us01dwt2p070/clients/ct/us01_cb95_apbld_mnoll/apbld/unit/stress/solution/rtest.65295a/cB_test/ws_cb_DWbb_bcm21
```

This shows a single binding definition for DWbb_bcm21. The Module line indicates the component name and the binding macro name (DWC_AND_2_SRC).

The Definition line is a TCL list of data values associated with the binding definition. The first entry indicates whether the binding definition is valid or not. The second is the instance name (always / in coreBuilder). Next is a sub-list containing the details from the binding definition command. This includes the description, instance name, width, port names, port descriptions, source file name, filter text, enable macro args, extra macro args, macro extension for inverted output support, and a flag indicating whether or not the binding is mandatory. Following that is the file containing the binding attributes (e.g.

size-only).

In the next example we list the actual binding details along with the binding definition.

```
coreConsultant> report_tech_cell_mapping
# Begin Tech Cell Mapping Definitions.
Module: BCM05 Macro: BCM05_map Design: design
Definition: 1 / {{Sample tech map} I005 {[width-1:0]} {q1 q2 q3} {{q1
port} {q2 port} {q3 port}} compA.v {} 0 {} {} 0}
/slowfs/us01dwt2p070/clients/ct/us01_cb95_apbld_mnoll/apbld/unit/stress/solution/rtest.45982a/cB_test/compA_conf
# End Tech Cell Mapping Definitions.
# Begin Tech Cell Mapping Binds:
Module: BCM05 Macro: BCM05_map Design: design
Binding: OR2 {{A 0} {B 0} {Z 0}} / {{Sample tech map} I005 {[width-1:0]}
{q1 q2 q3} {{q1 port} {q2 port} {q3 port}} compA.v {} 0 {} {} 0}
{dont_touch dont_retime size_only} {} {}
```

In this example the definition details are analogous to the prior example. The binding information includes a Module line with the same information from the definition. The Binding line indicates the name of the technology cell, the pin mapping (pin and inversion flag), and then everything from the instance name onward matches information from the Definition line of the definition.

See Also

[define_tech_cell_mapping\(2\)](#) [bind_tech_cell_mapping\(2\)](#)

report_timing_exceptions
Report timing exception objects

Syntax

```
string report_timing_exceptions [-designs <designs>] [-recursive]
[-brief]
    list <designs>
```

Parameters

-designs	Specifies a list of designs for which you want to
<designs>	report timing exceptions (default = current_design).
-recursive	Report timing exceptions recursively in subdesigns of
	the specified design(s).
-brief	List the timing exception objects only (no report
	header).

Description

The report_timing_exceptions command lists all the timing exception objects associated with the specified designs.

When you create a timing exception object by executing set_false_path, set_multicycle_path, set_max_delay, set_min_delay, reset_path, or set_disable_timing, the coreTool creates a timing exception object in the design knowledge database of the specified design. The coreTool also automatically assigns a name to the timing exception object.

The report_timing_exceptions command returns the same command syntax that you used to create the timing exceptions, with addition of the automatically generated -name option. You can use report_timing_exceptions to determine the names of timing exception objects so that you can remove a timing exception object by using the remove_item command.

If you specify the -brief option, report_timing_exceptions does not include a report header or design names in the report.

Examples

To list timing exceptions associated with the current_design:

```
coreConsultant> report_timing_exceptions
```

Report : Timing Exceptions

Version : 1999.05-CB1.0.3

Date : Thu Nov 18 13:21:45 PST 1999

Design: top

Timing Exceptions:

set_false_path -name FP0 -from int1 -to

set_multicycle_path -name MCP1 -to dout -from din 2

set_max_delay -name MaxD2 -to dout -from din 100

set_disable_timing -name DT3 u1

To list timing exceptions for Subblock_A and show timing exception commands only:

coreConsultant> report_timing_exceptions -brief -designs Subblock_A

set_multicycle_path -name MCP0 -to out_tf2 2 -design Subblock_A

set_multicycle_path -name MCP1 -to out_t2_of1 3 -design Subblock_A

See Also

set_false_path (2), set_multicycle_path (2), set_max_delay (2), _____
set_min_delay (2), reset_path (2), set_disable_timing (2), _____

NAME

report_trace Produce a report of performance profile metrics collected for traced commands.

SYNTAX

```
report_trace [-start|-stop|resume]
              [-profile profile_type]
              [-command command_name]
              [-cumulative count]
              [-top count]
              [-quiet]
```

string *profile_type* string *command_name* int *count*

ARGUMENTS

-start	This option is mutually exclusive with -stop, -resume, -profile, -top, -cumulative, -command. The option starts the collection of performance profile statics for each traced command invocation. If metric collection had previously been stopped with -stop, then previously collected data are deleted before collection starts.
-stop	This option is mutually exclusive with -start, -resume, -profile, -top, -cumulative, -command. The option stops the collection of performance profile statics for traced command invocations.
-resume	This option is mutually exclusive with -start, -stop, -profile, -top, -cumulative, -command. The option resume the collection of performance profile statics for traced command invocations. Previous gathered statics remain and newly collected metrics are added to pre-existing data. This option may be used after -stop to resume data collection without deleting previously collected data.

coreTools Command Reference Index

`-profile profile_type`

This option is mutually exclusive with `-start`, `-stop`, and `-resume`. This option selects the profile metrics that are included in the report. Allowed valuesA valid value is a list of permitted values {memory, cpu, time, count}, or the value all to select all metrics. If `-profile` is given, then the report output contains performance profile data of given types only. If the option is omitted, then it defaults to all.

`-cumulative count`

This optional option is mutually exclusive with `-start`, `-stop`, `-resume`, and `-command`. This option limits the number of commands reported for the top resource consumer accumulated over all invocations. However, all commands that share the same usage metric as the minimum consumer included in the count will be shown. If the option is omitted, the report output contains the top 10 consumers.

`-top count`

This optional option is mutually exclusive with `-start`, `-stop`, `-resume` and `-command`. This option limits the number of commands reported for the top resource consumer per single invocation. However, all commands that share the same usage metric as the minimum consumer included in the count will be shown. If the option is omitted, the report output contains the top 10 consumers.

`-command command_name`

This option is mutually exclusive with `-start`, `-stop`, `-resume`, `-top` and `-cumulative`. This option causes a performance profile report to be displayed for the given command. Cumulative and top single invocation metrics are shown. If the command was not invoked, then a message with this information is displayed instead of a report.

`-quiet`

If given, this option causes the command to ignore error conditions such as an attempt to start profile gathering when it has already been started or reporting on a non-existent command. The command

coreTools Command Reference Index

will exit quietly when it encounters an error condition.

DESCRIPTION

The **report_trace** command collects performance profile metrics on all traced command invocations since the last -start. Stopping the gathering is not necessary to produce a report. When performance profile collection is started with **report_trace -start**, each traced command will be measured for the available performance metrics: memory, CPU, wallclock time, and call count. The accumulated profile data can be reported at any time by calling this command

Stopping the metric gathering with **-stop** freezes the gathered metrics until the next **-start**. When gathering is restarted, all previously gathered metrics are deleted. Alternatively, metric collection may be restarted without deleting previously collected data using **-resume**.

The **report_trace** with no options will report the top 10 consumers in both cumulative and single call metrics for all profile types collected thus far.

EXAMPLES

The following example starts the collection of performance profile metrics.

```
prompt> report_trace -start  
on
```

The following example produces a report of top 10 commands with most usage for all profile types for both cumulative and individual call consumption. 10 is the default number reported for **-top** and **-cumulative** when these options are omitted.

```
prompt> report_trace  
on
```

The following example produces a report of cumulative and the single most expensive invocation for each profile type for the command **update_timing**.

```
prompt> report_trace -command update_timing  
on
```

The following example produces a report of top 20 consumers for cumulative and single invocation collected thus far.

coreTools Command Reference Index

```
prompt> report_trace -top 20 -cumulative 20  
on
```

The following example produces a report of top 5 cumulative consumption of memory.

```
prompt> report_trace -profile memory -cumulative 5  
on
```

The following example produces a report of top 15 commands with most invocations.

```
prompt> report_trace -profile count -cumulative 15  
on
```

The following example produces a report of top 10 commands with most cpu and wallclock time usage for both cumulative and individual call consumption. 10 is the default number reported for -top and -cumulative when these options are omitted.

```
prompt> report_trace -profile {cpu time}  
on
```

SEE ALSO

```
set_trace_option(2)  
get_trace_option(2)  
log_trace(2)  
annotate_trace(2)
```

reset_path

Reset specified paths to single cycle timing

Syntax

string **reset_path** [-setup] [-hold] [-rise] [-fall] [-from <from_list>]

[-through <through_list>] [-to <to_list>]

string <from_list>

string <through_list>

string <to_list>

Parameters

-setup	Reset setup evaluation to single-cycle If you do not specify either -setup or -hold, both setup and hold timing are reset.
-hold	Reset hold evaluation to single-cycle If you do not specify either -setup or -hold, both setup and hold timing are reset.
-rise	Reset rising delays to single-cycle If you do not specify either -rise or -fall, both rise and fall timing are reset.
-fall	Reset falling delays to single-cycle If you do not specify either -rise or -fall, both rise and fall timing are reset.
-from <from_list>	List of path startpoints If you do not specify a <from_list>, all paths to end points in <to_list> are reset. <from_list> can include clocks, pins, or ports. If you specify a clock, all path start points related to the specified clock are affected.
-through <through_list>	List of path through points Nets are interpreted to imply the leaf-level driver pins. If you do not specify -through, all timing paths specified using the -from and -to options are reset. You can specify multiple groups of <through_list> by using multiple -through options. Design Compiler uses a logic OR function to combine objects within one -through option; a path is reset if it passes through any object in the list. Design Compiler uses a logic AND function to combine objects from separate -through options; a path is reset if it passes through one or more objects in each list. For PrimeTime, the value of the environment

variable `timing_through_compatibility` determines how PrimeTime interprets `-through` options. When `timing_through_compatibility` is false (the default), objects specified with one or more `-through` options are interpreted as previously described for Design Compiler. When `timing_through_compatibility` is true, PrimeTime does not allow specification of through objects using multiple `-through` options. PrimeTime uses a logic AND function to group objects specified within one `-through` option; a path is reset if it passes through all objects in the list.

List of path endpoints

If you do not specify `<to_list>`, all paths from start points in `<from_list>` are reset. The `<to_list>` can include clocks, pins, or ports. If you specify a clock, all path endpoints related to the specified clock are reset.

`-to <to_list>`

Description

The `reset_path` command resets specified timing paths to the default single-cycle behavior. Use `reset_path` to undo timing exceptions specified using the following commands: `set_max_delay`, `set_min_delay`, `set_false_path`, `set_multicycle_path`.

The `reset_path` command creates a new timing exception object in the design knowledge database, with the next available sequence number. During the synthesis, the timing exceptions commands are executed in sequential order so that the `reset_path` command restores the default single-cycle timing to the specified path.

Another way to remove a timing exception in a coreTool is to remove the timing exception object by using the `remove_item` command.

You can also use `reset_path` to narrow the scope of a previous timing exception command. For example, if you executed a `set_false_path` command with multiple through objects specified with a single `-through` option, the through objects are combined as a logical OR. If you execute `reset_path` with the same options, but fewer through objects, you essentially remove paths through those objects from the set of paths that were marked as false.

By default, the coreTools check the validity of the `reset_path` command to ensure that the nodes specified in the `reset_path` command options exist in the `current_design` and are of the correct type. However, because the coreTools only model the structural part of a design, some timing exception start/end/through points may not exist in the coreTool model of the design. In such cases, you can use the `-quiet` option to suppress

validity checking. If you use the -quiet option and you specify incorrect or invalid arguments to reset_path, an error will occur during synthesis.

Examples

To reset the timing relation between u5/X and u10/Y to the default single cycle:

```
coreConsultant> reset_path -from {u5/X} -to {u10/Y}
```

To reset rising delay to single cycle for all paths ending at u18:

```
coreConsultant> reset_path -rise -to u18
```

See Also

set_false_path (2), set_multicycle_path (2), set_max_delay (2),
set_min_delay (2), set_disable_timing (2),

restructure_workspace

Restructure the current workspace.

Syntax

```
string restructure_workspace [-hide_activities <activities>]
[-transfer_activity_settings <activities>]
[-exclude_component_constraints] [-techsetup]
    string <activities>
```

Parameters

-hide_activities <activities>

List of activities to be hidden in the restructured workspace

-transfer_activity_settings
<activities>

List of activities for which the activity and strategy parameters need to be transferred from the original workspace to the restructured workspace.

-exclude_component_constraints

When specified Subsystem level SDC constraints are consumed in the restructured workspace

-techsetup

The switch enables transferring the TechSetup activity parameters from the original workspace to the restructured workspace

Description

Examples

See Also

NAME

`return` - Return from a procedure, or set return code of a script

SYNOPSIS

`return ?result?`

`return ?-code code? ?result?`

`return ?option value ...? ?result?`

DESCRIPTION

In its simplest usage, the **return** command is used without options in the body of a procedure to immediately return control to the caller of the procedure. If a *result* argument is provided, its value becomes the result of the procedure passed back to the caller. If *result* is not specified then an empty string will be returned to the caller as the result of the procedure.

The **return** command serves a similar function within script files that are evaluated by the **source** command. When **source** evaluates the contents of a file as a script, an invocation of the **return** command will cause script evaluation to immediately cease, and the value *result* (or an empty string) will be returned as the result of the **source** command.

EXCEPTIONAL RETURN CODES

In addition to the result of a procedure, the return code of a procedure may also be set by **return** through use of the **-code** option. In the usual case where the **-code** option is not specified the procedure will return normally. However, the **-code** option may be used to generate an exceptional return from the procedure. *Code* may have any of the following values:

ok (or **0**) Normal return: same as if the option is omitted. The return code of the procedure is 0 (**TCL_OK**).

error (or **1**) Error return: the return code of the procedure is 1 (**TCL_ERROR**). The procedure command behaves in its calling context as if it were the command **error result**. See below for additional options.

return (or **2**) The return code of the procedure is 2 (**TCL_RETURN**). The procedure command behaves in its calling context as if it were the command **return** (with no arguments).

break (or **3**) The return code of the procedure is 3 (**TCL_BREAK**). The procedure command behaves in its calling context as if it were the command **break**.

continue (or **4**) The return code of the procedure is 4 (**TCL_CONTINUE**). The procedure command behaves in its calling context as if it were the command **continue**.

value *Value* must be an integer; it will be returned as the return code for the current procedure.

When a procedure wants to signal that it has received invalid arguments from its caller, it may use **return -code error** with *result* set to a suitable error message. Otherwise usage of the **return -code** option is mostly limited to procedures that implement a new control structure.

The **return -code** command acts similarly within script files that are evaluated by the **source** command. During the evaluation of the contents of a file as a script by **source**, an invocation of the **return -code code** command will cause the return code of **source** to be *code*.

RETURN OPTIONS

In addition to a result and a return code, evaluation of a command in Tcl also produces a dictionary of return options. In general usage, all *option value* pairs given as arguments to **return** become entries in the return options dictionary, and any values at all are acceptable except as noted below. The **catch** command may be used to capture all of this information -- the return code, the result, and the return options dictionary -- that arise from evaluation of a script.

As documented above, the **-code** entry in the return options dictionary receives special treatment by Tcl. There are other return options also recognized and treated specially by Tcl. They are:

-errorcode *list*

The **-errorcode** option receives special treatment only when the value of the **-code** option is **TCL_ERROR**. Then the *list* value is meant to be additional information about the error, presented as a Tcl list for further processing by programs. If no **-errorcode** option is provided to **return** when the **-code error** option is provided, Tcl will set the value of the **-errorcode** entry in the return options dictionary to the default value of **NONE**. The **-errorcode** return option will also be stored in the global variable **errorCode**.

-errorinfo *info*

The **-errorinfo** option receives special treatment only when the value of the **-code** option is **TCL_ERROR**. Then *info* is the initial stack trace, meant to provide to a human reader additional information about the context in which the error occurred. The stack trace will also be stored in the global variable **errorInfo**. If no **-errorinfo** option is provided to **return** when the **-code error** option is provided, Tcl will provide its own initial stack trace value in the entry for **-errorinfo**. Tcl's initial stack trace will include only the call to the procedure, and stack unwinding will append information about higher stack levels, but there will be no information about the context of the error within the procedure. Typically the *info* value is supplied from the value of **-errorinfo** in a return options dictionary captured by the **catch** command (or from the copy of that information stored in the global variable **errorInfo**).

-errorstack *list*

The **-errorstack** option receives special treatment only when the value of the **-code** option is **TCL_ERROR**. Then *list* is the initial error stack, recording actual argument values passed to each proc level. The error stack will also be reachable through **info errorstack**. If no **-errorstack** option is provided to **return** when the **-code error** option is provided, Tcl will provide its own initial error stack in the entry for **-errorstack**. Tcl's initial error stack will include only the call to the procedure, and stack unwinding will append information about higher stack levels, but there will

be no information about the context of the error within the procedure. Typically the *list* value is supplied from the value of **-errorstack** in a return options dictionary captured by the **catch** command (or from the copy of that information from **info errorstack**).

-level *level*

The **-level** and **-code** options work together to set the return code to be returned by one of the commands currently being evaluated. The *level* value must be a non-negative integer representing a number of levels on the call stack. It defines the number of levels up the stack at which the return code of a command currently being evaluated should be *code*. If no **-level** option is provided, the default value of *level* is 1, so that **return** sets the return code that the current procedure returns to its caller, 1 level up the call stack. The mechanism by which these options work is described in more detail below.

-options *options*

The value *options* must be a valid dictionary. The entries of that dictionary are treated as additional *option value* pairs for the **return** command.

RETURN CODE HANDLING MECHANISMS

Return codes are used in Tcl to control program flow. A Tcl script is a sequence of Tcl commands. So long as each command evaluation returns a return code of **TCL_OK**, evaluation will continue to the next command in the script. Any exceptional return code (non-**TCL_OK**) returned by a command evaluation causes the flow on to the next command to be interrupted. Script evaluation ceases, and the exceptional return code from the command becomes the return code of the full script evaluation. This is the mechanism by which errors during script evaluation cause an interruption and unwinding of the call stack. It is also the mechanism by which commands like **break**, **continue**, and **return** cause script evaluation to terminate without evaluating all commands in sequence.

Some of Tcl's built-in commands evaluate scripts as part of their functioning. These commands can make use of exceptional return codes to enable special features. For example, the built-in Tcl commands that provide loops -- such as **while**, **for**, and **foreach** -- evaluate a script that is the body of the loop. If evaluation of the loop body returns the return code of **TCL_BREAK** or **TCL_CONTINUE**, the loop command can react in such a way as to give the **break** and **continue** commands their documented interpretation in loops.

Procedure invocation also involves evaluation of a script, the body of the procedure. Procedure invocation provides special treatment when evaluation of the procedure body returns the return code **TCL_RETURN**. In that circumstance, the **-level** entry in

coreTools Command Reference Index

the return options dictionary is decremented. If after decrementing, the value of the **-level** entry is 0, then the value of the **-code** entry becomes the return code of the procedure. If after decrementing, the value of the **-level** entry is greater than zero, then the return code of the procedure is **TCL_RETURN**. If the procedure invocation occurred during the evaluation of the body of another procedure, the process will repeat itself up the call stack, decrementing the value of the **-level** entry at each level, so that the *code* will be the return code of the current command level levels up the call stack. The **source** command performs the same handling of the **TCL_RETURN** return code, which explains the similarity of **return** invocation during a **source** to **return** invocation within a procedure.

The return code of the **return** command itself triggers this special handling by procedure invocation. If **return** is provided the option **-level 0**, then the return code of the **return** command itself will be the value *code* of the **-code** option (or **TCL_OK** by default). Any other value for the **-level** option (including the default value of 1) will cause the return code of the **return** command itself to be **TCL_RETURN**, triggering a return from the enclosing procedure.

EXAMPLES

First, a simple example of using **return** to return from a procedure, interrupting the procedure body.

```
proc printOneLine {} {
    puts "line 1"      ;# This line will be printed.
    return
    puts "line 2"      ;# This line will not be printed.
}
```

Next, an example of using **return** to set the value returned by the procedure.

```
proc returnX {} {return X}
puts [returnX]      ;# prints "X"
```

Next, a more complete example, using **return -code error** to report invalid arguments.

```
proc factorial {n} {
    if {![string is integer $n] || ($n < 0)} {
        return -code error \
            "expected non-negative integer,\
            but got \"$n\""
    }
    if {$n < 2} {
        return 1
    }
    set m [expr {$n - 1}]
    set code [catch {factorial $m} factor]
    if {$code != 0} {
        return -code $code $factor
    }
}
```

coreTools Command Reference Index

```
}
set product [expr {$n * $factor}]
if {$product < 0} {
    return -code error \
        "overflow computing factorial of $n"
}
return $product
}
```

Next, a procedure replacement for **break**.

```
proc myBreak {} {
    return -code break
}
```

With the **-level 0** option, **return** itself can serve as a replacement for **break**, with the help of **interp alias**.

```
interp alias {} Break {} return -level 0 -code break
```

An example of using **catch** and **return -options** to re-raise a caught error:

```
proc doSomething {} {
    set resource [allocate]
    catch {
        # Long script of operations
        # that might raise an error
    } result options
    deallocate $resource
    return -options $options $result
}
```

Finally an example of advanced use of the **return** options to create a procedure replacement for **return** itself:

```
proc myReturn {args} {
    set result ""
    if {[llength $args] % 2} {
        set result [lindex $args end]
        set args [lrange $args 0 end-1]
    }
    set options [dict merge {-level 1} $args]
    dict incr options -level
    return -options $options $result
}
```

SEE ALSO

`break(n)`, `catch(n)`, `continue(n)`, `dict(n)`, `error(n)`,
`errorCode(n)`, `errorInfo(n)`, `proc(n)`, `source(n)`,
`throw(n)`, `try(n)`

KEYWORDS

break, catch, continue, error, exception, procedure,
result, return

reuse-pragma

A type of stylized comment used to configure text files when used in coreConsultant and coreAssembler. For a more detailed description with more examples, see the coreBuilder manual.

Description

A reuse-pragma annotation is inserted into comments in files that must be configured by either coreAssembler or coreConsultant. There are a number of styles of reuse-pragma statements that can be used in files.

The first set of styles are only valid when used in files that are analyzed by the LoadDesigns activity in coreBuilder. These annotations can all be specified via the add_hdl_pragma command as well.

The types of annotations used by LoadDesigns follow:

- `reuse-pragma attr attrName attrValue`
This annotation applies to the next object found within the source file. It sets the attribute *attrName* to the value specified as *attrValue*
- `reuse-pragma startAttr attrName`
attrValue1
attrValue2
...
`reuse-pragma endAttr`
This annotation is like the previous one, except it allows for attribute values that must span multiple lines. This form is commonly used with the Description attribute.
- `reuse-pragma attr if (attrFilter) attrName attrValue`
This annotation applies to multiple objects. When specified within the port list, the attribute is applied to all ports matched by the *attrFilter*. The syntax of the filter is identical to that used in the -filter argument of the find_item command. If this annotation syntax is used outside of a port list (i.e. outside of a module/entity, then the attribute would apply to all matching designs in that file.
- `reuse-pragma translate_off`
...
`reuse-ppragma translate_on`
This annotation tells the source code analyzer to ignore any objects within the translate_off/translate_on region.
- `reuse-pragma process_ifdef all_branches|standard`
This annotation tells the Verilog compiler how to handle 'ifdef statements when the referenced parameter is a parameter macro.

The next set of styles are processed when coreConsultant and coreAssembler write configurable filegroups to disk. All the source files specified in

LoadDesigns belong to a configurable filegroup. You can also create your own configurable filegroup in coreBuilder.

- `reuse-pragma startSub [<Tcl_command>] endSub`
Replaces the current line with the result of the <Tcl_command>
- `reuse-pragma startSub [<Tcl_command>]`
Like the previous one, but replaces the current line, and the next line. The Tcl command string may include the special string %subText, that string is expanded to include the next line of input.
- `reuse-pragma startSub <key> [<Tcl_command>]`
...
`reuse-pragma endSub <key>`
This is a multi-line replacement. All the lines between the startSub and endSub and the actual reuse-pragma lines are replaced by the results of the Tcl command. You may use any string for <key>, but they must match on the startSub and endSub lines. You may nest any type of startSub within this version. The Tcl command string may include the special string %subText, that string is expanded to include all the lines between the startSub and the endSub (but not including those lines).

In the above replacement annotations, you may use any Tcl command, including commands defined in plugins for your core. Some common commands used are:

- IncludeIf - Include text in the file, if a given parameter expression returns true.
- ReplaceFormatParam - Replace the value of a parameter with a formatted value.

Examples

Please see the coreBuilder manual, and the referenced commands for specific examples using these special comments.

See Also

add_hdl_pragma (2), IncludeIf (2), ReplaceFormatParam (2),
analyze_filegroup (2), create_configuration_parameter (2),
create_autoload_filegroup (2), add_files_to_filegroup (2), GenerateIf
(3), ConfigActivity (3), MinValue (3), MaxValue (3), EnumValues (3),
Label (3), Description (3)

rtl_shellAppOption

Sets application options to the specified value on the specified block.

Settings to be used for RTL Architect (rtl_shell).

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This subscripted attribute can be used to issue set_app_options commands in the RTL Architect tool each time rtl_shell is run.

For application options that should be set at the start of the init_design stage, the attribute subscript indicates the application option name and the attribute value is the value to assign to the application option. For application options that should be set at points other than init_design, the insertion point must be specified as part of the attribute subscript. For example, "read_rtl-hdlin.elaborate.option" would define an application option "hdlin.elaborate.option" which is set before RTL is analyzed and elaborated. The valid insertion points are defined in the table below.

Insertion points for app options in RTLARM

Insertion Point	Description
init_design	At the start of init_design stage (Data preparation)
read_rtl	Before the RTL is analyzed and elaborated
rtl_opt	At the start of rtl_opt stage in (Flat flow) (before rtl_opt is called)
metrics	Before metrics generation commands are issued (Flat flow)
explore_design	Before the explore_design command is issued (Explore flow)

Examples

Issue the command 'set_app_options -name place.coarse.continue_on_missing_scandef -value true' in the generated reference methodology scripts at the start of init_design stage:

coreTools Command Reference Index

```
coreConsultant> set_design_attribute  
{rtl_shellAppOption[place.coarse.continue_on_missing_scandef]} true  
Issue the command 'set_app_options -name  
hdlin.elaborate.maximum_while_loop_iterations -value 400000' in the  
generated reference methodology scripts before RTL is analysed and  
elaborated
```

```
coreConsultant> set_design_attribute  
{rtl_shellAppOption[read_rtl-hdlin.elaborate.maximum_while_loop_iterations]}  
400000
```

See Also

rtl_shellAppOptionComment

Add a comment for the application options being introduced in the tool intent.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

The rtl_shellAppOptionComment attribute can be used to output a comment to document why the corresponding rtl_shellAppOption is being used. The comment will be put into the file where the app option is set.

Examples

```

coreConsultant> current_design [get_top_design]
coreConsultant> set_design_attribute
{rtl_shellAppOptionComment[place.coarse.continue_on_missing_scandef]}
"Overriding app option to true"
coreConsultant> set_design_attribute
{rtl_shellAppOption[place.coarse.continue_on_missing_scandef]} true
Generates the following code fragment in the script
# Overriding app option to true
set_app_options -name place.coarse.continue_on_missing_scandef -value
true

```

See Also

NAME

safe - Creating and manipulating safe interpreters

SYNOPSIS

```
::safe::interpCreate ?child? ?options...?  
::safe::interpInit child ?options...?  
::safe::interpConfigure child ?options...?  
::safe::interpDelete child  
::safe::interpAddToAccessPath child directory  
::safe::interpFindInAccessPath child directory  
::safe::setLogCmd ?cmd arg...?
```

OPTIONS

```
?-accessPath pathList? ?-statics boolean? ?-noStatics?  
?-nested boolean? ?-nestedLoadOk? ?-deleteHook script?
```

DESCRIPTION

Safe Tcl is a mechanism for executing untrusted Tcl scripts safely and for providing mediated access by such scripts to potentially dangerous functionality.

Safe Tcl ensures that untrusted Tcl scripts cannot harm the hosting application. It prevents integrity and privacy attacks. Untrusted Tcl scripts are prevented from corrupting the state of the hosting application or computer. Untrusted scripts are also prevented from disclosing information stored on the hosting computer or in the hosting application to any party.

coreTools Command Reference Index

Safe Tcl allows a parent interpreter to create safe, restricted interpreters that contain a set of predefined aliases for the **source**, **load**, **file**, **encoding**, and **exit** commands and are able to use the auto-loading and package mechanisms.

No knowledge of the file system structure is leaked to the safe interpreter, because it has access only to a virtualized path containing tokens. When the safe interpreter requests to source a file, it uses the token in the virtual path as part of the file name to source; the parent interpreter transparently translates the token into a real directory name and executes the requested operation (see the section **SECURITY** below for details). Different levels of security can be selected by using the optional flags of the commands described below.

All commands provided in the parent interpreter by Safe Tcl reside in the **safe** namespace.

COMMANDS

The following commands are provided in the parent interpreter:

::safe::interpCreate *?child? ?options...?*

Creates a safe interpreter, installs the aliases described in the section **ALIASES** and initializes the auto-loading and package mechanism as specified by the supplied *options*. See the **OPTIONS** section below for a description of the optional arguments. If the *child* argument is omitted, a name will be generated.

::safe::interpCreate always returns the interpreter name.

The interpreter name *child* may include namespace separators, but may not have leading or trailing namespace separators, or excess colon characters in namespace separators. The interpreter name is qualified relative to the global namespace **::**, not the namespace in which the **::safe::interpCreate** command is evaluated.

::safe::interpInit *child ?options...?*

This command is similar to **interpCreate** except it that does not create the safe interpreter. *child* must have been created by some other means, like **interp create -safe**. The interpreter name *child* may include namespace separators, subject to the same restrictions as for **interpCreate**.

::safe::interpConfigure *child ?options...?*

If no *options* are given, returns the settings for all options for the named safe interpreter as a list of options and their current values for that *child*. If a single additional argument is provided, it will return a list of 2 elements *name* and *value* where *name* is the full name of that option and *value* the current value

coreTools Command Reference Index

for that option and the *child*. If more than two additional arguments are provided, it will reconfigure the safe interpreter and change each and only the provided options. See the section on **OPTIONS** below for options description. Example of use:

```
# Create new interp with the same configuration as "$i0":
set i1 [safe::interpCreate {}][safe::interpConfigure $i0]
# Get the current deleteHook
set dh [safe::interpConfigure $i0 -del]
```

```
# Change (only) the statics loading ok attribute of an
# interp and its deleteHook (leaving the rest unchanged):
safe::interpConfigure $i0 -delete {foo bar} -statics 0
```

::safe::interpDelete *child*

Deletes the safe interpreter and cleans up the corresponding parent interpreter data structures. If a *deleteHook* script was specified for this interpreter it is evaluated before the interpreter is deleted, with the name of the interpreter as an additional argument.

::safe::interpFindInAccessPath *child directory*

This command finds and returns the token for the real directory *directory* in the safe interpreter's current virtual access path. It generates an error if the directory is not found. Example of use:

```
$child eval [list set tk_library \
    [::safe::interpFindInAccessPath $name $tk_library]]
```

::safe::interpAddToAccessPath *child directory*

This command adds *directory* to the virtual path maintained for the safe interpreter in the parent, and returns the token that can be used in the safe interpreter to obtain access to files in that directory. If the directory is already in the virtual path, it only returns the token without adding the directory to the virtual path again. Example of use:

```
$child eval [list set tk_library \
    [::safe::interpAddToAccessPath $name $tk_library]]
```

::safe::setLogCmd *?cmd arg...?*

This command installs a script that will be called when interesting life cycle events occur for a safe interpreter. When called with no arguments, it returns the currently installed script. When called with one argument, an empty string, the currently installed script is removed and logging is turned off. The script will be invoked with one additional argument, a string describing the event of interest. The main purpose is to help in debugging safe interpreters. Using this facility you can get complete error messages while the safe interpreter gets only generic error messages. This prevents a safe interpreter from seeing messages about failures and other events that might contain sensitive information such as real directory names.

Example of use:

coreTools Command Reference Index

```
::safe::setLogCmd puts stderr
```

Below is the output of a sample session in which a safe interpreter attempted to source a file not found in its virtual access path. Note that the safe interpreter only received an error message saying that the file was not found:

```
NOTICE for child interp10 : Created
NOTICE for child interp10 : Setting accessPath=(/foo/bar) staticsok=1 nestedok=0
NOTICE for child interp10 : auto_path in interp10 has been set to {$p(:0:)}
ERROR for child interp10 : /foo/bar/init.tcl: no such file or directory
```

OPTIONS

The following options are common to

::safe::interpCreate, **::safe::interpInit**, and **::safe::interpConfigure**. Any option name can be abbreviated to its minimal non-ambiguous name. Option names are not case sensitive.

-accessPath *directoryList*

This option sets the list of directories from which the safe interpreter can **source** and **load** files. If this option is not specified, or if it is given as the empty list, the safe interpreter will use the same directories as its parent for auto-loading. See the section **SECURITY** below for more detail about virtual paths, tokens and access control.

-statics *boolean*

This option specifies if the safe interpreter will be allowed to load statically linked packages (like **load {} Tk**). The default value is **true** : safe interpreters are allowed to load statically linked packages.

-noStatics

This option is a convenience shortcut for **-statics false** and thus specifies that the safe interpreter will not be allowed to load statically linked packages.

-nested *boolean*

This option specifies if the safe interpreter will be allowed to load packages into its own sub-interpreters. The default value is **false** : safe interpreters are not allowed to load packages into their own sub-interpreters.

-nestedLoadOk

This option is a convenience shortcut for **-nested true** and thus specifies the safe interpreter will be allowed to load packages into its own sub-interpreters.

-deleteHook *script*

When this option is given a non-empty *script*, it will be evaluated in the parent with the name of the safe interpreter as an additional argument just before actually deleting the safe interpreter. Giving an empty value removes any currently installed deletion hook script for that safe interpreter. The default value (**{}**) is not to have any deletion call back.

ALIASES

The following aliases are provided in a safe interpreter:

source *fileName*

The requested file, a Tcl source file, is sourced into the safe interpreter if it is found. The **source** alias can only source files from directories in the virtual path for the safe interpreter. The **source** alias requires the safe interpreter to use one of the token names in its virtual path to denote the directory in which the file to be sourced can be found. See the section on **SECURITY** for more discussion of restrictions on valid filenames.

load *fileName*

The requested file, a shared object file, is dynamically loaded into the safe interpreter if it is found. The filename must contain a token name mentioned in the virtual path for the safe interpreter for it to be found successfully. Additionally, the shared object file must contain a safe entry point; see the manual page for the **load** command for more details.

file *?subCmd args...?*

The **file** alias provides access to a safe subset of the subcommands of the **file** command; it allows only **dirname**, **join**, **extension**, **root**, **tail**, **pathname** and **split** subcommands. For more details on what these subcommands do see the manual page for the **file** command.

encoding *?subCmd args...?*

The **encoding** alias provides access to a safe subset of the subcommands of the **encoding** command; it disallows setting of the system encoding, but allows all other subcommands including **system** to check the current encoding.

exit

The calling interpreter is deleted and its computation is stopped, but the Tcl process in which this interpreter exists is not terminated.

SECURITY

Safe Tcl does not attempt to completely prevent annoyance and denial of service attacks. These forms of attack prevent the application or user from temporarily using the computer to perform useful work, for example by consuming all available CPU time or all available screen real estate. These attacks, while aggravating, are deemed to be of lesser importance in general than integrity and privacy attacks that Safe Tcl is to prevent.

The commands available in a safe interpreter, in

coreTools Command Reference Index

addition to the safe set as defined in **interp** manual page, are mediated aliases for **source**, **load**, **exit**, and safe subsets of **file** and **encoding**. The safe interpreter can also auto-load code and it can request that packages be loaded.

Because some of these commands access the local file system, there is a potential for information leakage about its directory structure. To prevent this, commands that take file names as arguments in a safe interpreter use tokens instead of the real directory names. These tokens are translated to the real directory name while a request to, e.g., source a file is mediated by the parent interpreter. This virtual path system is maintained in the parent interpreter for each safe interpreter created by **::safe::interpCreate** or initialized by **::safe::interpInit** and the path maps tokens accessible in the safe interpreter into real path names on the local file system thus preventing safe interpreters from gaining knowledge about the structure of the file system of the host on which the interpreter is executing. The only valid file names arguments for the **source** and **load** aliases provided to the child are path in the form of **[file join token filename]** (i.e. when using the native file path formats: *token/filename* on Unix and *token\filename* on Windows), where *token* is representing one of the directories of the *accessPath* list and *filename* is one file in that directory (no sub directories access are allowed).

When a token is used in a safe interpreter in a request to source or load a file, the token is checked and translated to a real path name and the file to be sourced or loaded is located on the file system. The safe interpreter never gains knowledge of the actual path name under which the file is stored on the file system.

To further prevent potential information leakage from sensitive files that are accidentally included in the set of files that can be sourced by a safe interpreter, the **source** alias restricts access to files meeting the following constraints: the file name must fourteen characters or shorter, must not contain more than one dot must end up with the extension or be called

Each element of the initial access path list will be assigned a token that will be set in the child **auto_path** and the first element of that list will be set as the **tcl_library** for that child.

If the access path argument is not given or is the empty list, the default behavior is to let the child access the same packages as the parent has access to (Or to be more precise: only packages written in Tcl (which by definition cannot be dangerous as they run in the child interpreter) and C extensions that provides a **_SafeInit** entry point). For that purpose, the parent's **auto_path** will be used to construct the child access path. In order that the child successfully loads the Tcl library files (which includes the auto-loading

coreTools Command Reference Index

mechanism itself) the **tcl_library** will be added or moved to the first position if necessary, in the child access path, so the child **tcl_library** will be the same as the parent's (its real path will still be invisible to the child though). In order that auto-loading works the same for the child and the parent in this by default case, the first-level sub directories of each directory in the parent **auto_path** will also be added (if not already included) to the child access path. You can always specify a more restrictive path for which sub directories will never be searched by explicitly specifying your directory list with the **-accessPath** flag instead of relying on this default mechanism.

When the *accessPath* is changed after the first creation or initialization (i.e. through **interpConfigure -accessPath list**), an **auto_reset** is automatically evaluated in the safe interpreter to synchronize its **auto_index** with the new token list.

SEE ALSO

interp(n), **library(n)**, **load(n)**, **package(n)**, **source(n)**, **unknown(n)**

KEYWORDS

alias, auto-loading, auto_mkindex, load, parent interpreter, safe interpreter, child interpreter, source

save_workspace

Save the current workspace.

Syntax

string **save_workspace**

Description

The save_workspace command saves the currently loaded workspace. If there is one loaded, it calls save_all_modified_kbs to write out each knowledge database (KB) that is currently marked as having been modified in the workspace.

In the coreTool GUI, if you try to open/create a workspace with a currently loaded workspace, or exit the tool, you will be prompted to save the current workspace.

Examples

The following command saves the currently loaded workspace:

```
coreConsultant> save_workspace
```

See Also

[duplicate_workspace \(2\)](#), [create_workspace \(2\)](#), [open_workspace \(2\)](#), [close_workspace \(2\)](#), _____

scale_to_current_units

Scale a time, area, or capacitance value to the unit currently in use

Syntax

```
string scale_to_current_units <unitType> <value>
    string <unitType>
    string <value>
```

Parameters

<unitType> (time area capacitance) (Values: time, area, capacitance)
 <value> The value (including units) to be scaled.

Description

The `scale_to_current_units` command scales a time, area, or capacitance value to the unit currently being used by the coreTool. The coreTool automatically sets the current unit (for example, ns for time values) to the unit used by the currently loaded technology library. In addition, for values that are expressed as simple constants, the coreTool automatically scale time, area, and capacitance values to the current unit. For example, if you specify `MinInputDelay` as 10ns, then switch to a technology library that uses ps as the time value, coreConsultant or coreBuilder will automatically scale the 10ns value to 10,000ps.

However, the coreTools do not automatically scale values that are embedded in formulas. The coreConsultant commands that you typically use in formulas automatically scale units. For example, "`percent_of_period 15 clk`" automatically scales the 15-percent-of-clk-period value to the current time unit. However, to add a specific time period to the `percent_of_period` result, you need to use `scale_to_current_units` to scale the value to be added to the current time unit, as shown in the examples below.

Examples

To scale 25ps to the current time unit (ns):

```
coreConsultant> scale_to_current_units time 25ps
0.025
```

To set `MinOutputDelay` on port `dout` to 15 percent of the clock cycle time plus 10ps:

```
coreConsultant> set_port_attribute dout MinOutputDelay  
{=[percent_of_period 15] + [scale_to_current_units time 10ps]}
```

In the above example, coreConsultant calculates 15 percent of the clock period in units of the currently loaded technology library (current time unit). Then, coreConsultant scales 10ps to the current time unit and adds the scaled value to the percent_of_period result to generate a minimum output delay constraint in the time units used by the target technology library.

See Also

[percent_of_period\(2\)](#)

NAME

`scan` - Parse string using conversion specifiers in the style of `sscanf`

SYNOPSIS

`scan string format ?varName varName ...?`

INTRODUCTION

This command parses substrings from an input string in a fashion similar to the ANSI C **`sscanf`** procedure and returns a count of the number of conversions performed, or -1 if the end of the input string is reached before any conversions have been performed. *String* gives the input to be parsed and *format* indicates how to parse it, using % conversion specifiers as in **`sscanf`**. Each *varName* gives the name of a variable; when a substring is scanned from *string* that matches a conversion specifier, the substring is assigned to the corresponding variable. If no *varName* variables are specified, then **`scan`** works in an inline manner, returning the data that would otherwise be stored in the variables as a list. In the inline case, an empty string is returned when the end of the input string is reached before any conversions have been performed.

DETAILS ON SCANNING

`Scan` operates by scanning *string* and *format* together. If the next character in *format* is a blank or tab then it matches any number of white space characters in *string* (including zero). Otherwise, if it is not a %

coreTools Command Reference Index

character then it must match the next character of *string*. When a % is encountered in *format*, it indicates the start of a conversion specifier. A conversion specifier contains up to four fields after the %: a XPG3 position specifier (or a * to indicate the converted value is to be discarded instead of assigned to any variable); a number indicating a maximum substring width; a size modifier; and a conversion character. All of these fields are optional except for the conversion character. The fields that are present must appear in the order given above.

When **scan** finds a conversion specifier in *format*, it first skips any white-space characters in *string* (unless the conversion character is [or c). Then it converts the next input characters according to the conversion specifier and stores the result in the variable given by the next argument to **scan**.

OPTIONAL POSITIONAL SPECIFIER

If the % is followed by a decimal number and a \$, as in then the variable to use is not taken from the next sequential argument. Instead, it is taken from the argument indicated by the number, where 1 corresponds to the first *varName*. If there are any positional specifiers in *format* then all of the specifiers must be positional. Every *varName* on the argument list must correspond to exactly one conversion specifier or an error is generated, or in the inline case, any position can be specified at most once and the empty positions will be filled in with empty strings.

OPTIONAL SIZE MODIFIER

The size modifier field is used only when scanning a substring into one of Tcl's integer values. The size modifier field dictates the integer range acceptable to be stored in a variable, or, for the inline case, in a position in the result list. The syntactically valid values for the size modifier are **h**, **L**, **l**, and **ll**. The **h** size modifier value is equivalent to the absence of a size modifier in the the conversion specifier. Either one indicates the integer range to be stored is limited to the same range produced by the **int()** function of the **expr** command. The **L** size modifier is equivalent to the **l** size modifier. Either one indicates the integer range to be stored is limited to the same range produced by the **wide()** function of the **expr** command. The **ll** size modifier indicates that the integer range to be stored is unlimited.

MANDATORY CONVERSION CHARACTER

The following conversion characters are supported:

d

The input substring must be a decimal integer. It is read in and the integer value is stored in the variable, truncated as required by the size modifier value.

o

The input substring must be an octal integer. It is read in and the integer value is stored in the

coreTools Command Reference Index

variable, truncated as required by the size modifier value.

x or X

The input substring must be a hexadecimal integer. It is read in and the integer value is stored in the variable, truncated as required by the size modifier value.

b

The input substring must be a binary integer. It is read in and the integer value is stored in the variable, truncated as required by the size modifier value.

u

The input substring must be a decimal integer. The integer value is truncated as required by the size modifier value, and the corresponding unsigned value for that truncated range is computed and stored in the variable as a decimal string. The conversion makes no sense without reference to a truncation range, so the size modifier **ll** is not permitted in combination with conversion character **u**.

i

The input substring must be an integer. The base (i.e. decimal, octal, or hexadecimal) is determined by the C convention (leading 0 for octal; prefix 0x for hexadecimal). The integer value is stored in the variable, truncated as required by the size modifier value.

c

A single character is read in and its Unicode value is stored in the variable as an integer value. Initial white space is not skipped in this case, so the input substring may be a white-space character.

s

The input substring consists of all the characters up to the next white-space character; the characters are copied to the variable.

e or f or g or E or G

The input substring must be a floating-point number consisting of an optional sign, a string of decimal digits possibly containing a decimal point, and an optional exponent consisting of an **e** or **E** followed by an optional sign and a string of decimal digits. It is read in and stored in the variable as a floating-point value.

[chars]

The input substring consists of one or more characters in *chars*. The matching string is stored in the variable. If the first character between the brackets is a **]** then it is treated as part of *chars* rather than the closing bracket for the set. If *chars* contains a sequence of the form **a-b** then any character between **a** and **b** (inclusive) will match. If the first or last character between the brackets is a **-**, then it is

coreTools Command Reference Index

treated as part of *chars* rather than indicating a range.

[*^chars*]

The input substring consists of one or more characters not in *chars*. The matching string is stored in the variable. If the character immediately following the *^* is a *]* then it is treated as part of the set rather than the closing bracket for the set. If *chars* contains a sequence of the form *a-b* then any character between *a* and *b* (inclusive) will be excluded from the set. If the first or last character between the brackets is a *-*, then it is treated as part of *chars* rather than indicating a range value.

n

No input is consumed from the input string. Instead, the total number of characters scanned from the input string so far is stored in the variable.

The number of characters read from the input for a conversion is the largest number that makes sense for that particular conversion (e.g. as many decimal digits as possible for *%d*, as many octal digits as possible for *%o*, and so on). The input substring for a given conversion terminates either when a white-space character is encountered or when the maximum substring width has been reached, whichever comes first. If a *** is present in the conversion specifier then no variable is assigned and the next scan argument is not consumed.

DIFFERENCES FROM ANSI SSCANF

The behavior of the **scan** command is the same as the behavior of the ANSI C **sscanf** procedure except for the following differences:

- [1] **%p** conversion specifier is not supported.
- [2] For **%c** conversions a single character value is converted to a decimal string, which is then assigned to the corresponding *varName*; no substring width may be specified for this conversion.
- [3] The **h** modifier is always ignored and the **l** and **L** modifiers are ignored when converting real values (i.e. type **double** is used for the internal representation). The **ll** modifier has no **sscanf** counterpart.
- [4] If the end of the input string is reached before any conversions have been performed and no variables are given, an empty string is returned.

EXAMPLES

Convert a UNICODE character to its numeric value:

```
set char "x"
set value [scan $char %c]
```

Parse a simple color specification of the form **#RRGGBB** using hexadecimal conversions with substring sizes:

```
set string "#08D03F"
scan $string "%#2x%2x%2x" r g b
```

Parse a **HH:MM** time string, noting that this avoids problems with octal numbers by forcing interpretation as decimals (if we did not care, we would use the **%i** conversion instead):

```
set string "08:08"    ;# *Not* octal!
if {[scan $string "%d:%d" hours minutes] != 2} {
    error "not a valid time string"
}
# We have to understand numeric ranges ourselves...
if {$minutes < 0 || $minutes > 59} {
    error "invalid number of minutes"
}
```

Break a string up into sequences of non-whitespace characters (note the use of the **%n** conversion so that we get skipping over leading whitespace correct):

```
set string " a string {with braced words} + leading space "
set words {}
while {[scan $string "%s%n word length] == 2} {
    lappend words $word
    set string [string range $string $length end]
}
```

Parse a simple coordinate string, checking that it is complete by looking for the terminating character explicitly:

```
set string "(5.2,-4e-2)"
# Note that the spaces before the literal parts of
# the scan pattern are significant, and that ")" is
# the Unicode character \u0029
if {
    [scan $string " (%f ,%f %c" x y last] != 3
    || $last != 0x0029
} then {
    error "invalid coordinate string"
}
puts "X=$x, Y=$y"
```

An interactive session demonstrating the truncation of integer values determined by size modifiers:

```
% set tcl_platform(wordSize)
4
```

coreTools Command Reference Index

```
% scan 20000000000000000000 %d  
2147483647  
% scan 20000000000000000000 %ld  
9223372036854775807  
% scan 20000000000000000000 %lld  
20000000000000000000
```

SEE ALSO

`format(n)`, `sscanf(3)`

KEYWORDS

conversion specifier, parse, scan

scratch_dir

Directory for storage of temporary files.

Syntax

```
string scratch_dir = "."
```

Description

scratch_dir is a global variable that specifies the complete path to a writable directory to which you want the coreTool to store temporary files that are created for temporary use during execution of commands.

Examples

To use the writable directory /usr/tools/temp for temporary files that the coreTool creates, add the following line to your .synopsys_rt.setup file:

```
set scratch_dir "/usr/tools/temp"
```

See Also

NAME

`seek` - Change the access position for an open channel

SYNOPSIS

`seek channelId offset ?origin?`

DESCRIPTION

Changes the current access position for *channelId*.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension.

The *offset* and *origin* arguments specify the position at which the next read or write will occur for *channelId*. *Offset* must be an integer (which may be negative) and *origin* must be one of the following:

- | | |
|----------------|--|
| start | The new access position will be <i>offset</i> bytes from the start of the underlying file or device. |
| current | The new access position will be <i>offset</i> bytes from the current access position; a negative <i>offset</i> moves the access position backwards in the underlying file or device. |
| end | The new access position will be <i>offset</i> bytes from the end of the file or device. A negative <i>offset</i> places the access position before the end of file, and a positive <i>offset</i> places the access position after the end of file. |

The *origin* argument defaults to **start**.

The command flushes all buffered output for the channel before the command returns, even if the channel is in non-blocking mode. It also discards any buffered and unread input. This command returns an empty string. An error occurs if this command is applied to channels whose underlying file or device does not support seeking.

Note that *offset* values are byte offsets, not character offsets. Both **seek** and **tell** operate in terms of bytes, not characters, unlike **read**.

EXAMPLES

Read a file twice:

```
set f [open file.txt]
set data1 [read $f]
seek $f 0
set data2 [read $f]
close $f
# $data1 eq $data2 if the file wasn't updated
```

Read the last 10 bytes from a file:

```
set f [open file.data]
# This is guaranteed to work with binary data but
# may fail with other encodings...
fconfigure $f -translation binary
seek $f -10 end
set data [read $f 10]
close $f
```

SEE ALSO

file(n), open(n), close(n), gets(n), tell(n),
Tcl_StandardChannels(3)

KEYWORDS

access position, file, seek

select_by_class

Select a representative library cell by its function class

Syntax

```
string select_by_class class [drive_strength]
    string class
    string drive_strength
```

Parameters

class	The class from which to select a representative cell. (Values: combinational, sequential, tristate)
drive_strength	The relative drive strength cell to select from the specified class. (Values: low, median, high) The default is median.

Description

The select_by_class command selects a representative cell of the specified class from the currently loaded target technology libraries and returns the name of the selected cell.

The most common use for select_by_class is as a formula to specify DrivingCell on an input port or PinLoadType on an output port in a technology-independent manner. You specify the cell class and relative drive strength and coreConsultant selects the representative cell as a value for the corresponding input or output port constraint.

If you do not specify <drive_strength>, select_by_class selects the median drive strength cell of the specified class.

Examples

To return the name of the representative high drive strength combinational cell:

```
coreConsultant> select_by_class combinational high
cba_core/or2c1
```

To set DrivingCell on the data_in pin to the representative median drive strength combinational cell:

```
coreConsultant> set_port_attribute data_in DrivingCell
{=select_by_class "combinational" "median"}
```

See Also

[select_by_function \(2\)](#), [select_by_name \(2\)](#), [DrivingCell \(3\)](#), [PinLoadType](#)
(3)

select_by_function

Select a representative library cell by its logic function

Syntax

```
string select_by_function function [drive_strength]
    string function
    string drive_strength
```

Parameters

function	The logic function for which to select a representative cell. (Values: nand2, buf, inv, mux21, dff, latch, xor2)
drive_strength	The relative drive strength cell to select from the specified function. (Values: low, median, high) The default is median.

Description

The select_by_function command selects a representative cell for the specified logic function from the currently loaded target technology libraries and returns the name of the selected cell.

The most common use for select_by_function is as a formula to specify DrivingCell on an input port or PinLoadType on an output port in a technology-independent manner. You specify the cell function and relative drive strength and coreConsultant selects the representative cell as a value for the corresponding input or output port constraint.

If you do not specify <drive_strength>, select_by_function selects the median drive strength cell for the specified function.

Examples

To return the name of the representative low drive strength mux21 function cell:

```
coreConsultant> select_by_function mux21 low
cba_core/mx2a0
```

To set DrivingCell on the data_in pin to the representative median drive strength mux21 cell:

```
coreConsultant> set_port_attribute data_in DrivingCell
```

```
{=select_by_function "mux21" "median"}
```

See Also

[select_by_class \(2\)](#), [select_by_name \(2\)](#), [DrivingCell \(3\)](#), [PinLoadType \(3\)](#)

select_by_name

Select a library cell (or pin) by name

Syntax

string **select_by_name** *cell_name* [*pin_name*]

string *cell_name*

string *pin_name*

Parameters

cell_name The name of the library cell.

pin_name The name of a pin on the specified cell.

Description

The `select_by_name` command selects the specified cell by cell name and optionally by pin name. The most common use for `select_by_name` is as a formula to specify `DrivingCell` on an input port or `PinLoadType` on an output port when you know the name of the cell and, optionally, the name of the pin that you want to use for `DrivingCell` or `PinLoadType`.

As a `coreBuilder` user, the `select_by_name` formula is useful if you know that your core will always be targeted to technology libraries that use a common, well-defined naming convention. However, in general, you should avoid using technology-dependent constraints.

As a `coreConsultant` user, the `select_by_name` formula is useful if you know the name of the technology library cell and (optionally) pin that will be driving/loading an input/output pin.

Examples

To return the name of the `mx2a0` cell:

```
coreConsultant> select_by_name mx2a0  
cba_core/mx2a0
```

To set `DrivingCell` on the `data_in` pin to the library cell named `mx2a0`:

```
coreConsultant> set_port_attribute data_in DrivingCell  
{=select_by_name "mx2a0"}
```

To set `DrivingCell` on the `data_in` pin to the Q pin of the library cell named `fd1a2`:

```
coreConsultant> set_port_attribute data_in DrivingCell  
{=select_by_name "mx2a0" "Q"}
```

See Also

[select_by_class \(2\)](#), [select_by_function \(2\)](#), [DrivingCell \(3\)](#), [PinLoadType \(3\)](#)

NAME

`self` - method call internal introspection

SYNOPSIS

```
package require Tcl00
```

```
self ?subcommand?
```

DESCRIPTION

The **self** command, which should only be used from within the context of a call to a method (i.e. inside a method, constructor or destructor body) is used to allow the method to discover information about how it was called. It takes an argument, *subcommand*, that tells it what sort of information is actually desired; if omitted the result will be the same as if **self object** was invoked. The supported subcommands are:

self call

This returns a two-element list describing the method implementations used to implement the current call chain. The first element is the same as would be reported by **info object call** for the current method (except that this also reports useful values from within constructors and destructors, whose names are reported as **<constructor>** and **<destructor>** respectively), and the second element is an index into the first element's list that indicates which actual implementation is currently executing (the first implementation to execute is always at index 0).

self caller

When the method was invoked from inside another object method, this subcommand returns a three element list describing the containing object and method. The first

coreTools Command Reference Index

element describes the declaring object or class of the method, the second element is the name of the object on which the containing method was invoked, and the third element is the name of the method (with the strings **<constructor>** and **<destructor>** indicating constructors and destructors respectively).

self class

This returns the name of the class that the current method was defined within. Note that this will change as the chain of method implementations is traversed with **next**, and that if the method was defined on an object then this will fail.

If you want the class of the current object, you need to use this other construct:

```
info object class [self object]
```

self filter

When invoked inside a filter, this subcommand returns a three element list describing the filter. The first element gives the name of the object or class that declared the filter (note that this may be different from the object or class that provided the implementation of the filter), the second element is either **object** or **class** depending on whether the declaring entity was an object or class, and the third element is the name of the filter.

self method

This returns the name of the current method (with the strings **<constructor>** and **<destructor>** indicating constructors and destructors respectively).

self namespace

This returns the name of the unique namespace of the object that the method was invoked upon.

self next

When invoked from a method that is not at the end of a call chain (i.e. where the **next** command will invoke an actual method implementation), this subcommand returns a two element list describing the next element in the method call chain; the first element is the name of the class or object that declares the next part of the call chain, and the second element is the name of the method (with the strings **<constructor>** and **<destructor>** indicating constructors and destructors respectively). If invoked from a method that is at the end of a call chain, this subcommand returns the empty string.

self object

This returns the name of the object that the method was invoked upon.

self target

When invoked inside a filter implementation, this subcommand returns a two element list describing the method being filtered. The first element will be the name of the declarer of the method, and the second element will be the actual name of the method.

EXAMPLES

This example shows basic use of **self** to provide information about the current object:

```
oo::class create c {
  method foo {} {
    puts "this is the [self] object"
  }
}
c create a
c create b
a foo          -> prints "this is the ::a object"
b foo          -> prints "this is the ::b object"
```

This demonstrates what a method call chain looks like, and how traversing along it changes the index into it:

```
oo::class create c {
  method x {} {
    puts "Cls: [self call]"
  }
}
c create a
oo::objdefine a {
  method x {} {
    puts "Obj: [self call]"
    next
    puts "Obj: [self call]"
  }
}
a x          -> Obj: {{method x object method} {method x ::c method}} 0
              -> Cls: {{method x object method} {method x ::c method}} 1
              -> Obj: {{method x object method} {method x ::c method}} 0
```

SEE ALSO

info(n), next(n)

KEYWORDS

call, introspection, object

NAME

set - Read and write variables

SYNOPSIS

`set varName ?value?`

DESCRIPTION

Returns the value of variable *varName*. If *value* is specified, then set the value of *varName* to *value*, creating a new variable if one does not already exist, and return its value. If *varName* contains an open parenthesis and ends with a close parenthesis, then it refers to an array element: the characters before the first open parenthesis are the name of the array, and the characters between the parentheses are the index within the array. Otherwise *varName* refers to a scalar variable.

If *varName* includes namespace qualifiers (in the array name if it refers to an array element), or if *varName* is unqualified (does not include the names of any containing namespaces) but no procedure is active, *varName* refers to a namespace variable resolved according to the rules described under **NAME RESOLUTION** in the **namespace** manual page.

If a procedure is active and *varName* is unqualified, then *varName* refers to a parameter or local variable of the procedure, unless *varName* was declared to resolve differently through one of the **global**, **variable** or **upvar** commands.

EXAMPLES

Store a random number in the variable *r*:

```
set r [expr {rand()}]
```

Store a short message in an array element:

```
set anAry(msg) "Hello, World!"
```

Store a short message in an array element specified by a variable:

```
set elemName "msg"
set anAry($elemName) "Hello, World!"
```

Copy a value into the variable *out* from a variable whose name is stored in the *vbl* (note that it is often easier to use arrays in practice instead of doing double-dereferencing):

```
set in0 "small random"
set in1 "large random"
set vbl in[expr {rand() >= 0.5}]
set out [set $vbl]
```

SEE ALSO

[expr\(n\)](#), [global\(n\)](#), [namespace\(n\)](#), [proc\(n\)](#), [trace\(n\)](#),
[unset\(n\)](#), [upvar\(n\)](#), [variable\(n\)](#)

KEYWORDS

read, write, variable

set_activity_incomplete

Set the specified activities to the incomplete state

Syntax

string **set_activity_incomplete** *activities*
 string *activities*

Parameters

activities One or more activities to be set to the incomplete state.

Description

The set_activity_incomplete command forces an activity from the complete state to the incomplete state so that you can re-complete the activity using the autocomplete_activity command.

For activities that have parameters, you use the set_activity_parameter command to set up the data required for the activity. The set_activity_parameter command automatically sets the state of the specified activity to incomplete. You then execute autocomplete_activity to (re)complete the activity and (re)enable downstream activities that depend on that activity. This ensures that the effect of your set_activity_parameter command is propagated through the coreTool design flow.

Some activities (for example, SpecifyPortIntent) do not have activity parameters. The input data to these activities comes from commands like set_port_attribute. If you have already completed SpecifyPortIntent and activities that depend on SpecifyPortIntent (like VerifyBudgets and Synthesize), set_port_attribute will set a new attribute value on a port. However, you need to re-complete SpecifyPortIntent and its dependent activities to implement the effects of the new port attribute.

The proper way to implement the change described above is to use set_activity_incomplete to force SpecifyPortIntent to the incomplete state. Activities that depend on SpecifyPortIntent are automatically set to the incomplete, not-enabled state. You can then use autocomplete_activity to automatically (re)complete SpecifyPortIntent and its dependent activities with the new value of the port attribute.

Examples

To set SpecifyPortIntent to incomplete after updating intent values, then auto-complete SpecifyPortIntent:

```
coreConsultant> set_port_attribute sum PinLoadCount 4  
coreConsultant> set_activity_incomplete SpecifyPortIntent  
coreConsultant> autocomplete_activity SpecifyPortIntent
```

See Also

[autocomplete_activity \(2\), report_activities \(2\)](#)

`set_activity_parameter`

Set the value of an activity parameter

Syntax

`string set_activity_parameter [-component componentName] activityName
paramName value``string componentName``string activityName``string paramName``string value`

Parameters

`-component``componentName`

Component associated with the activity.

This option is only needed when the same activity appears in more than one component.

`activityName`

The name of the activity on which you want to set the parameter.

`paramName`

The name of the parameter you want to set.

`value`

The new value for the parameter.

Description

The `set_activity_parameter` command sets the value of a parameter associated with an activity. Most coreTool activities are parameter based; when you specify values in a GUI dialog for an activity, you are actually setting values of parameters on that activity.

The `set_activity_parameter` command automatically marks the associated activity as not-complete. To complete the activity with your new parameter value(s), use the `autocomplete_activity` command.

Some activities (for example, `SpecifyContextInformation`) do not have parameters. Instead, the specifications associated with the activity come from commands like `set_design_attribute`, `set_port_attribute`, and so on. For commands that do not have parameters, use the `set_activity` command to mark the activity as not-complete. Then, execute the necessary commands to set up the activity data (for example, `set_port_attribute`). To complete the activity with the new specifications, use the `autocomplete_activity` command.

To get a list and descriptions of the parameters associated with an activity, use the `report_activity_parameters` command. To get the current

value of an activity parameter, use the `get_activity_parameter` command.

Examples

To set the `StrategyName` parameter for the `Synthesize` activity to `DC_incremental_opto_strategy`, then auto-complete the `Synthesize` activity:

```
coreConsultant> set_activity_parameter Synthesize StrategyName  
DC_incremental_opto_strategy  
coreConsultant> autocomplete_activity Synthesize
```

In `coreAssembler`, two different components contain an activity named `Simulate` with a parameter named `Simulator`. To specify `VCS` as the simulator for the activity associated with component `U1`:

```
coreAssembler> set_activity_parameter -component U1 Simulate Simulator  
VCS
```

See Also

[autocomplete_activity \(2\)](#), [get_activity_parameter \(2\)](#),
[report_activity_parameters \(2\)](#)

[API Index](#) [Command Reference](#)

set_address_bank_attribute

Set the value of an attributes on an address bank.

Syntax

```
string set_address_bank_attribute bank attr [value]  
    string bank  
    string attr  
    string value
```

Parameters

<i>bank</i>	The address bank to set the attribute on.
<i>attr</i>	Name of the attribute to set.
<i>value</i>	Value of the attribute to set.

Description

Sets the value of an attribute on an address bank.

Examples

```
coreBuilder> set_address_bank_attribute map1/bank1 BaseAddress 0x10000
```

See Also

[addressBank \(3\)](#), [get_address_bank_attribute \(2\)](#), [create_address_bank \(2\)](#)

[API Index](#) [Command Reference](#)

set_address_block_attribute

Set the value of an attribute on an address block.

Syntax

```
string set_address_block_attribute block attr [value]  
    string block  
    string attr  
    string value
```

Parameters

block The address block to set the attribute on.
attr Name of the attribute to set.
value Value of the attribute to set.

Description

Sets the value of an attribute on an address block.

Examples

```
coreBuilder> set_address_block_attribute map1/block1 BaseAddress 0x1000
```

See Also

[addressBlock \(3\).get_address_block_attribute \(2\)](#)

NAME

set_app_var Sets the value of an application variable.

SYNTAX

```
string set_app_var  
      -default  
      var  
      value
```

Data Types

<i>var</i>	string
<i>value</i>	string

ARGUMENTS

-default	Resets the variable to its default value.
<i>var</i>	Specifies the application variable to set.
<i>value</i>	Specifies the value to which the variable is to be set.

DESCRIPTION

The **set_app_var** command sets the specified application variable. This command sets the variable to its default value or to a new value you specify.

This command returns the new value of the variable if setting the variable was successful. If the application variable could not be set, then an error is returned indicating the reason for the failure.

Reasons for failure include:

coreTools Command Reference Index

- o The specified variable name is not an application variable, unless the application variable **sh_allow_tcl_with_set_app_var** is set to true See the **sh_allow_tcl_with_set_app_var** man page for details.
- o The specified application variable is read only.
- o The value specified is not a legal value for this application variable

EXAMPLES

The following example attempts to set a read-only application variable:

```
prompt> set_app_var synopsis_root /tmp
Error: can't set "synopsys_root": variable is read-only
Use error_info for more info. (CMD-013)
```

In this example, the application variable name is entered incorrectly, which generates an error message:

```
prompt> set_app_var sh_enabel_page_mode 1
Error: "sh_enabel_page_mode" is not an application variable
Use error_info for more info. (CMD-013)
```

This example shows the variable name entered correctly:

```
prompt> set_app_var sh_enable_page_mode 1
1
```

This example resets the variable to its default value:

```
prompt> set_app_var sh_enable_page_mode -default
0
```

SEE ALSO

```
get_app_var(2)
report_app_var(2)
write_app_var(2)
```


set_area_estimate

Store area estimation value for a given technology and configuration.

Syntax

```
string set_area_estimate -technology <technology> [-configuration
<configuration>] [-label <label>] area
    string <technology>
    string <configuration>
    string <label>
    double area
```

Parameters

-technology <technology>	Technology associated with area estimate
-configuration <configuration>	Configuration values for this area estimate
-label <label>	Label for the given configuration
area	Area estimate

Description

This command is used to define a technology and configuration specific area estimate for a design. Tech 'technology' can be any string. The 'configuration' is a list of alternating parameter names values (e.g. {Param1 Value1 Param2 Value2}). The optional 'label' can be specified to provide a description of the given configuration (e.g. 'default', 'typical', 'max').

Area estimates are used within Area Estimation reports which can be generated in coreConsultant and coreAssembler. After a design has been configured, the report can be generated showing which area estimates are the best match for the given user configuration. A 'match' is determined by checking which configuration parameter values match those associated with an area estimate. Since some parameters have a larger impact on the area than others, a weight can be associated with each parameter by setting the AreaWeight attribute on the parameter.

Examples

Set area estimate for the default configuration.

```
prompt> set_area_estimate 456.2 -technology 90LP -label Default \
```


-configuration {P1 2 P2 7 P3 16}

See Also

remove_area_estimates (2) AreaWeight (3)

set_attribute

Set the value of an attribute on one or more items

Syntax

```
string set_attribute -attr <attribute name> -value <attribute value>
[-subscript <attribute subscript>] [-strict] [<item names>]
    string <attribute name>
    string <attribute value>
    string <attribute subscript>
    list <item names>
```

Parameters

-attr <attribute name>	Specifies the name of the attribute that you want to set.
-value <attribute value>	Specifies the value to which you want to set the attribute.
-subscript <attribute subscript>	Specifies the subscript of the specified attribute that you want to set.
-strict	Evaluate formula value for validity. If you specify -strict and specify an invalid formula for the -value option, set_attribute will fail and generate an error message. If you do not specify the -strict option, set_attribute will set the attribute to the specified formula without first evaluating the formula. When the value of the attribute is retrieved with a get_attribute command, get_attribute will evaluate the formula for validity regardless of whether you used the -strict option with set_attribute.
<item names>	The name(s) of the item(s) on which to set the attribute. You can specify a single item, a list of items, or specify the items as a Tcl collection.

Description

The set_attribute command sets the specified attribute to the specified value on the specified item(s). If more than one item exists with the specified name (for example, a port and a net with the same name), set_attribute successfully sets the attribute on the item(s) on which the

attribute is valid and fails to set the attribute on item(s) on which the attribute is not valid. `set_attribute` generates an error message if it cannot set the attribute on any specified item.

For subscripted attributes, use the `-subscript` option to specify which subscript of the specified attribute you want to set. For example, to set the `CompileMapEffort[initial]` attribute, use the options `"-attr CompileMapEffort -subscript initial"`. The `set_attribute` command will fail if you specify a subscript that is not valid for the specified attribute.

If you are specifying the attribute value as a formula (for example, `{=percent_of_period 15}`), you can specify the `-strict` option to force `set_attribute` to evaluate the formula for validity before setting the attribute. If you specify `-strict` and the formula is not valid, `set_attribute` will fail and generate an error message.

There are additional commands that you can use to set values of attributes on cells, clocks, designs, ports, and parameters (for example, `set_design_attribute`). These commands do not have all the `set_attribute` options, but can be easier to use, particularly for specifying the item. For example, a design can have a port, net, and clock that all have the name `clk`. To set the value of an attribute on the clock named `clk` with `set_attribute`, you would have to specify the clock as `[find_item -type clock clk]`; whereas with the `get_clock_attribute` command, you can just specify the clock item as `"clk"`.

Examples

To set the value of the `MaxInputDelay` attribute to 10 on the port named `input_1`:

```
coreConsultant> set_attribute -attr MaxInputDelay -value 10
[find_item -type port input_1]
2.5
```

In the above example, `coreConsultant` determines that the default subscript for `MaxInputDelay` is `clk`, because there is only one clock (`clk`) in the design.

To set the value of the `MaxInputDelay[clk]` attribute on all input ports, except clocks and resets, to the formula value, and then evaluate the formula for validity before setting the attribute:

```
coreConsultant> set_attribute -strict -attr MaxInputDelay -subscript clk
-value {=percent_of_period 15}
[all_inputs -no_clocks -no_resets]
```

See Also

[set_clock_attribute \(2\)](#), [set_design_attribute \(2\)](#), [set_port_attribute \(2\)](#), [set_cell_attribute \(2\)](#), [set_parameter_attribute \(2\)](#), [get_attribute](#)

(2), [report_attribute \(2\)](#)

set_cell_attribute

Set the value of an attribute on a list of design instances

Syntax

string **set_cell_attribute** *list attr [value]*

string *list*

string *attr*

string *value*

Parameters

- | | |
|-------|---|
| list | List of cells (design instances) on which you want to set the attribute. |
| attr | The name of the attribute that you want to set.
For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so that set_cell_attribute does not interpret the subscript as a sub-command. |
| value | The value to which you want to set the attribute.
For boolean attributes, <value> is optional (default = true). |

Description

The set_cell_attribute command sets the specified attribute to the specified value on the specified cell(s). You can specify the cells (design instances) by name (as a list if specifying more than one cell) or as a collection.

To get the current value of an attribute on a cell, use the get_cell_attribute command.

Examples

To set the DontTouch attribute to true on all cells in the current design:

```
coreConsultant> set_cell_attribute [find_item -type cell] DontTouch true
```

See Also

[get_cell_attribute \(2\)](#)

`set_clock_attribute`

Set the value of an attribute on a list of clocks

Syntax

string **set_clock_attribute** *list attr [value]*string *list*string *attr*string *value*

Parameters

- `list` List of clocks on which you want to set the attribute.
The name of the attribute that you want to set.
For subscripted attributes, enclose the attribute/subscript in
- `attr` curly braces ({Attribute[subscript]}) so that
set_clock_attribute does not interpret the subscript as a
sub-command.
- `value` The value to which you want to set the attribute.

Description

The set_clock_attribute command sets the specified attribute to the specified value on the specified clock(s). You can specify the clock(s) by name (as a list if specifying more than one clock) or as a Tcl collection.

To get the current value of an attribute on a clock, use the get_clock_attribute command.

For a list of available attributes, see the man page for the clock object type referenced below.

Examples

To set the CycleTime attribute on the clock named pclk to 25 (time units of the currently loaded technology library):

```
coreConsultant> set_clock_attribute pclk CycleTime 25
```

To set the SkewUncertainty attribute to 10ps on all clocks in the current_design:

```
coreConsultant> set_clock_attribute [get_clocks]  
SkewUncertainty 10ps
```

See Also

get_clock_attribute (2), get_clocks (2), clock (3) _____

NAME

set_command_option_value

Set a command option default or current value.

SYNTAX

```
string set_command_option_value
    -default | -current
    -command command_name
    -option option_name | -position
    positional_option_position
    -value value | -undefined
```

ARGUMENTS

-default	Set the default option value.
-current	Set the current option value.
-command <i>command_name</i>	Set the option value for an option of this command.
-option <i>option_name</i>	Set the option value for this option of the command.
-position <i>positional_option_position</i>	Set the option value for this positional option of the command.
-value <i>value</i>	Set the option value to this value.
-undefined	Set the option value to the "undefined value".

DESCRIPTION

Sets the current or default value of a command option.

Either `-default` must be specified (to specify setting the default value of the option), or `-current` must be specified (to specify setting the current value of the option). Unlike for `get_command_option_values`, one of `-current` or `-default` must be specified for `set_command_option_value`.

An option of a command must be specified (for value setting) by passing a command name via `-command` and either an option name (for an option of the command) via `-option`, or the position of a positional option via `-position`. The *option_name* passed via `-option` (for a dash option) must not have its leading dash character omitted. An option name passed via `-option` may be either the name of a dash option or the name of a positional option. A positional option may be specified either via `-option` or via `-position`. Positional option positions of a command are numbered 0, 1, 2, ... (N-1), where N is the number of positional options of the command.

To set the (current or default) value of the option, a string value should be passed via `-value`. Alternatively, you can pass `-undefined` to reset the option to have the "undefined value". (CAVEAT: For some "set value implementations" for numeric options, `-undefined` does not really "undefine" the value - instead the value is set to 0.)

Processing of the `-value` value does not include any CCI option checking of the value (such as checking whether the option value has correct syntax for its type).

The command "throws" a Tcl error for various conditions:

- * the command does not exist
- * the command exists but no such option of the command exists
- * the set operation failed (could be due to a failed conversion for one of the "set value" implementations which does a conversion from string to one of integer, double, etc.)

The `-undefined` option is a mutually exclusive alternative to the `-value` option. `-undefined` has the effect of resetting the option value to the "undefined value".

A possible power user application: power users could put `set_command_option_value` commands in their home directory Tcl startup files for their favorite dialogs/commands to "seed" initial current values for these dialogs. This may lower the need for automatically saving replay scripts (of `set_command_option_value` commands for each command/dialog option/field) on exit from the application.

EXAMPLES

The following example sets the current value for the -bar option of the foo command to a value of abc.

```
prompt> set_command_option_value -current -command foo \  
-option "-bar" -value abc
```

The following example sets the current value for the first positional argument of the foo command to a value of xyz.

```
prompt> set_command_option_value -current -command foo \  
-position 1 -value xyz
```

SEE ALSO

```
get_command_option_values(2)  
preview(2)
```

set_component_view

Set the view associated with the current/selected component.

Syntax

```
string set_component_view [-component <component name>] view  
    string <component name>  
    string view
```

Parameters

-component <component name>	Set the view associated with the given component.
view	Name of view to be set (become current).

Description

This command sets the name of the currently selected view for the specified component. Components instantiated via a SPIRIT IP-XACT component description can have multiple views and the selected view impacts the processing of the component. For example a component may have both a Verilog and a VHDL view with different RTL source files associated with each view. The currently selected view can be retrieved using the set_component_view command.

Examples

```
set_component_view -component i_uart vhdlView
```

See Also

[get_component_view \(2\)](#)

set_configuration_parameter
Set configuration (design) parameter value

Syntax

```
string set_configuration_parameter [-update_gui] [-component <component>]
[-design] [-array_extend] parameter value
    string <component>
    string parameter
    string value
```

Parameters

-update_gui	Update the gui configurion dialog if visible.
-component <component>	Component on which you want to set the parameter value. This option is only needed and can only be used in coreAssembler.
-design	Set the design parameter when both a design and package parameter are present.
-array_extend	Add elements to an array parameter and set values from the list of values.
parameter	The name of the parameter that you want to set. An optional index or index range can be used with the parameter name for bitfield parameters only.
value	The value to which you want to set the parameter.

Description

This is the command that should be used to set design configuration parameters. This is used as part of the batch mode equivalent to the SpecifyConfiguration activity in coreConsultant and the ConfigureComponents activity in coreAssembler. The value of the specified parameter is set to the specified value.

To set a particular index or range of any bitfield parameter, use the index range option with the parameter name. The configuration parameter value will be updated accordingly.

Please note that the given range alignment should match with the parameter bit alignment. Otherwise it reports alignment mismatch error.

The -array_extend option is used to add additional elements to a SystemVerilog array parameter and to set the values of the elements. The

value argument specifies a list of values for the new array elements. The array parameter is extended after elaboration. The new elements are not shown in the configuration dialog.

Examples

To set the 'width' parameter to 16:

```
prompt> set_configuration_parameter width 16
```

To set the 'depth' parameter on the FIFO component (in coreAssembler):

```
prompt> set_configuration_parameter -component FIFO depth 32
```

To set particular range [3:0] of a bitfield parameter '[7:0] P' with value 0xff:

```
prompt> set_configuration_parameter P[3:0] 4
```

The new value of parameter P will be 0xf4.

For the same above definition, it reports error for different alignment.

```
prompt> set_configuration_parameter P[0:3] 4
```

Error: Alignment mismatch.

To set value for index range [1][3:0] of a multidim bitfield parameter '[1:0][3:0] MP' with value 0x66 on component 'C' (in coreAssembler):

```
prompt> set_configuration_parameter -component C MP[1][3:0] 5
```

The new value of parameter MP will be 0x56.

To add four new elements with values 4, 5, 6, and 7 to a SystemVerilog parameter with an unpacked dimensions.

SystemVerilog parameter with unpack dimension set by another parameter.

```
parameter psize = 4;
```

```
parameter integer param1 [(psize-1):0] = {32'd0,32'd1,32'd2,32'd3};
```

```
set_configuration_parameter -component u1 psize 8
```

```
set_configuration_parameter -component u1 -array_extend param1 "0b100 5 6 0x7"
```

See Also

[set_configuration_parameter_attribute\(2\)](#), [get_configuration_parameter\(2\)](#)

set_configuration_parameter_attribute

Set an attribute on the specified configuration parameter

Syntax

```
string set_configuration_parameter_attribute [-design] [-component
<component>] parameterName attr [value]
    string <component>
    string parameterName
    string attr
    string value
```

Parameters

-design	Set the design parameter when both a design and package parameter are present.
-component <component>	Parent component for the configuration parameter; "" identifies current. Note that inside a subsystem component the "" annotates the component itself and not the subsystem, as set_current_component would do.
parameterName	The configuration parameter to set the attribute value on
attr	The name of the attribute to set the value on
value	Value for the attribute

Description

This command is used to set the value of the named attribute on the specified configuration parameter. In most cases the command is used in coreBuilder's configuration intent files to specify a specific parameter behaviour which is not easy to define in HDL.

The command can be used in coreAssembler to override a parameters behaviour for a specific component (if the named attribute allows a change).

Examples

To make the parameter read-only from now on:

```
prompt> set_configuration_parameter_attribute DataWidth ReadOnlyParam
or:
```

coreTools Command Reference Index

```
prompt> set_configuration_parameter_attribute DataWidth ReadOnlyParam  
true
```

To specify a specific default value for a component-specific parameter
(coreAssembler only):

```
prompt> set_configuration_parameter_attribute -component U_ahb DataWidth  
DefaultValue 64
```

See Also

set_configuration_parameter(2), get_configuration_parameter_attribute
(2), set_current_component(2)

NAME

set_current_command_mode

SYNTAX

```
string set_current_command_mode
      -mode command_mode | -command command

string    command_mode
string    command
```

ARGUMENTS

-mode *command_mode*
Specifies the name of the new command mode to be made current. **-mode** and **-command** are mutually exclusive; you must specify one, but not both.

-command *command*
Specifies the name of the command whose associated command mode is to be made current. **-mode** and **-command** are mutually exclusive; you must specify one, but not both.

DESCRIPTION

The **set_current_command_mode** sets the current command mode. If there is a current mode in effect, the current mode is first canceled, causing the associated clean up to be executed. The initialization for the new command mode is then executed as it is made current.

If the name of the given command mode is empty, the current command mode is cancelled and no new command mode is made current.

coreTools Command Reference Index

A current command mode stays in effect until it is displaced by a new command mode or until it is cleared by an empty command mode.

If the command completes successfully, the new current command mode name is returned. On failure, the command returns the previous command mode name which will remain current and prints an error message unless error messages are suppressed.

EXAMPLES

The following example sets the current command mode to a mode called "mode_1"

```
shell> set_current_command_mode -mode mode_1
```

The following example clears the current command mode without setting a new mode.

```
shell> set_current_command_mode -mode ""
```

The following example sets the current command mode to the mode associated with the command "modal_command"

```
shell> set_current_command_mode -command modal_command
```


set_current_component

Set the current component to enable component level operations.

Syntax

```
string set_current_component [-quiet] [<component name>]  
    string <component name>
```

Parameters

-quiet	Don't print current KB or current design messages.
<component name>	Name of the component to make 'current'

Description

This command is used to set the the current component. When a component becomes 'current', its associated workspace becomes the current workspace (accessible using `get_workspace_kb` or `get_workspace_name`), and the design of which the component is an instantiation becomes the current design.

The current component can only be set to the name of a cell instantiated at the sub-system level in `coreAssembler`. When working in `coreConsultant`, the `set_current_component` command is not available, and the current component is always "".

To unset the current component, and move back the sub-system level, issue the `set_current_component` command without a component name. This will make the `coreAssembler` workspace KB the current KB and make the sub-system level design the current design.

Examples

To set the current component to U1, and instance of the DW8051:

```
coreAssembler> set_current_component U1  
Information: Current KB changed to: U1 (CMD5-9)  
Information: Current design is DW8051. (DES-35)
```

See Also

[get_current_component\(2\)](#), [get_workspace_name\(2\)](#), [get_workspace_kb\(2\)](#),
[get_component_name\(2\)](#)

set_default_prime_profiles

Set Default Prime Profiles on the design in order of priority.

Syntax

```
string set_default_prime_profiles [-tags <Prime Profile names>] [-clear]
      string <Prime Profile names>
```

Parameters

-tags <Prime Profile names>	List of Default Prime Profile tags.
-clear	Clear list of Default Prime Profile tags.

Description

Default Prime Profiles are profiles that are applied automatically if users autocomplete component configuration without explicitly selecting a Prime Profile from the list of available Prime Profiles. The order in which the profile names are specified reflects their priority (high to low).

Note

- ◇ The Prime Profile names specified with this command must be created before using this command.
- ◇ The <None> option in SelectPrimeProfile activity is not allowed to be set as a default.

Examples

```
coreBuilder> set_default_prime_profiles -tags "ProfileC ProfileA ProfileB"
```

In the above example, ProfileC gets the highest priority and will be applied if no profile is explicitly set/selected by the user. ProfileA and ProfileB appear after and the <None> option and remaining profiles appear after in the SelectPrimeProfile activity page.

If ProfileC is unavailable (e.g. due to invalid license), ProfileA gets the highest priority and is displayed at the top of the SelectPrimeProfile activity.

To clear the default Prime Profiles list, the following can be used.

coreBuilder> set_default_prime_profiles -clear

See Also

[create_prime_profile_file \(2\)](#) [report_prime_profiles \(2\)](#)
[PrimeProfileDefaults \(3\)](#)

set_dependant_controls

Set dependent controls for a parameter dialog.

Syntax

```
string set_dependent_controls dParamName iParamName  
    string dParamName  
    string iParamName
```

Parameters

<i>dParamName</i>	The dependent parameter for the parameter dialog
<i>iParamName</i>	The independent parameter or controlling parameter for the parameter dialog

Description

Use this command in coreBuilder after the dialog has been built to set the DependantControls on dialog items for a parameter that is hidden inside of a procedures so it is does not automatically included the dependencies. Similarly you can apply the same fix in coreAssembler or coreConsultant to fix update issues in the GUI. You will need to make sure you're in the right component context so that the parameter names you specify can be found when in coreAssembler.

Examples

```
set_dependant_controls dependentParamName independentParamName
```

See Also

set_design_attribute

Set the value of an attribute on one or more designs

Syntax

```
string set_design_attribute [-designs <list>] attr [value]
    string <list>
    string attr
    string value
```

Parameters

-designs <list>	List of designs on which you want to set the attribute (default = current_design). The default design is the current_design.
attr	The name of the attribute that you want to set. For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so that set_design_attribute does not interpret the subscript as a sub-command.
value	The value to which you want to set the attribute. For boolean attributes, <value> is optional (default = true).

Description

The set_design_attribute command sets the specified attribute to the specified value on the specified design(s). You can specify a single design, a list of designs, or specify the designs as a Tcl collection.

To get the current value of a design attribute, use the get_design_attribute command.

Examples

To set the CompileMapEffort[initial] attribute to high on the current_design:

```
coreConsultant> set_design_attribute {CompileMapEffort[initial]} high
```

To set the WireLoad attribute to io30x30 on the design named My_Core:

```
coreConsultant> set_design_attribute -designs My_Core WireLoad io30x30
```

To set the MapBlockIndividually attribute to true on all subdesigns of the current_design:

```
coreConsultant> set_design_attribute -designs [all_subdesigns]  
MapBlockIndividually true
```

See Also

[get_design_attribute \(2\)](#)

set_design_prefix

Sets the prefix to be prepended to each design name.

Syntax

```
string set_design_prefix [-component <component>] prefix
    string <component>
    string prefix
```

Parameters

-component <component>	hierachical path to the component.
prefix	Design name prefix

Description

This command can be used in coreConsultant to specify a prefix to prepend to the name of each design in a core. The new design names will be <prefix>_<original name>. This can be useful when instantiating two different configurations of a core within a higher level design. The different configurations can easily end up having design name collisions if renaming is not performed.

In coreAssembler, Design prefixing is enabled if the components support it. Design prefixing for each component can be set by passing the component path to the command if prefixing is enabled. Please note that this command is not supported for cores which are packaged with a version of coreBuilder older than 3.0. If attempted, the command will fail and the prefix will not be set.

The easiest way to support this is by specifying the prefix at workspace creation time (in the creation dialog for the GUI or via the -prefix option in batch). However, the value can also be set using set_design_prefix. The only caveat is that this command can not be called after the design has been configured. If you want to go back and change the value of the prefix, the SpecifyConfiguration command must be first be undone using the 'set_activity_incomplete' command.

Examples

To prefix all designs with the string "U1_":

```
prompt> set_design_prefix U1
```

The following command sets the Designprefix in assembler on the i_enabled

component:

```
coreAssembler> set_design_prefix -component /i_enabled MyDesignPrefix
```

See Also

create_workspace (2), get_design_prefix (2), set_activity_incomplete (2),
set_macro_prefix (2), get_macro_prefix (2), set_file_prefix (2),
get_file_prefix (2)

set_disable_timing
Disable timing arcs in a circuit

Syntax

```
string set_disable_timing [-from <pin_name>] [-to <pin_name>] [-restore]
cell_pin_list
    string <pin_name>
    list cell_pin_list
```

Parameters

-from	Name of pin on specified cell
<pin_name>	You must use the -from and -to options together.
-to <pin_name>	Name of pin on specified cell
	You must use the -from and -to options together.
-restore	Restore specified arc
cell_pin_list	List of pins/cells on which to disable arcs
	If -from and -to options are used, the list must contain only cells.

Description

The set_disable_timing command creates a coreTool timing exception object for the current_design. The coreTool stores the timing exception object in the design knowledge database (KB). When coreConsultant generates a strategy for Design Compiler or PrimeTime, coreConsultant uses the timing exception object to generate a similar set_disable_timing command for Design Compiler or PrimeTime.

In Design Compiler or PrimeTime, the set_disable_timing command disables timing through the specified cells, pins, or ports.

You can disable any cell, pin, or port in the current_design or its subdesigns. For subdesigns in the hierarchy, you must specify instance names instead of design names. Specify cells at lower levels in the design hierarchy by instance name (for example, instance1/instance2/cell_name). Specify pins at lower levels in the design hierarchy as instance1/instance2/cell_name/pin_name. Specify ports at lower levels in the design hierarchy as instance1/instance2/cell_name/port_name.

When you disable timing through a cell, only those cell arcs that lead to the output pins of the cell are disabled. When you disable timing through a pin or port, only those cell arcs that lead to or from that pin or port

are disabled.

If you specify `-from` and `-to` pins, all arcs between the two specified pins on the cell are disabled.

To enable the disabled cells, pins, or ports, use `set_disable_timing -restore` or `reset_path`.

By default, the coreTools check the validity of the `set_disable_timing` command to ensure that the nodes specified in the `set_disable_timing` command options exist in the `current_design` and are of the correct type. However, because the coreTools only model the structural part of a design, some timing exception start/end/through points may not exist in the coreTool model of the design.

In such cases, you can use the `-quiet` option to suppress validity checking.

If you use the `-quiet` option and you specify incorrect or invalid arguments to `set_disable_timing`, an error will occur during synthesis.

Examples

To disable all timing arcs that lead to output pins of cells U2 and U3 of the `current_design`:

```
coreConsultant> set_disable_timing {U2, U3}
```

To disable all timing arcs that lead to output pin Z of cell U5 in the `current_design`:

```
coreConsultant> set_disable_timing U5/Z
```

To disable all timing arcs between pins A and Z on cell U1/U1 in the `current_design`:

```
coreConsultant> set_disable_timing U1/U1 -from A -to Z
```

See Also

`set_false_path(2)`, `set_multicycle_path(2)`, `set_max_delay(2)`,
`set_min_delay(2)`, `reset_path(2)`

set_editing_mode
Set workspace editing mode

Syntax

string **set_editing_mode** [-readwrite] [-readonly]

Parameters

-readwrite	Switch to read/write mode for workspaces
-readonly	Switch to read-only mode for workspace

Description

This command cannot be used when a workspace is open. Once you have set the mode to read-only if you open a workspace no lock file is created. the workspace cannot be edited or saved in that mode. Toggling to 'readwrite' can only be done once for a given tool invocation.

Examples

See Also

set_false_path

Removes timing constraints from particular paths

Syntax

```
string set_false_path [-rise] [-fall] [-setup] [-hold] [-comment <text>]
[-from <from_list>] [-rise_from <from_list>] [-fall_from <from_list>]
[-through <through_list>] [-rise_through <through_list>] [-fall_through
<through_list>] [-to <to_list>] [-rise_to <to_list>] [-fall_to <to_list>]
[-reset_path]
    string <text>
    string <from_list>
    string <through_list>
    string <to_list>
```

Parameters

-rise	Set rising delays to be false If you do not specify either -rise or -fall, both rise and fall timing are marked as false.
-fall	Set falling delays to be false If you do not specify either -rise or -fall, both rise and fall timing are marked as false.
-setup	Set maximum delays to be false If you do not specify either -setup or -hold, both setup and hold checking are disabled.
-hold	Set minimum delays to be false If you do not specify either -setup or -hold, both setup and hold checking are disabled.
-comment <text>	Associate a string description with the command for tracking purposes.
-from <from_list>	List of path startpoints If you do not specify a <from_list>, all paths to end points in <to_list> are marked as false. <from_list> can include clocks, pins, or ports. If you specify a clock, all path start points related to the specified clock are affected. If you specify an internal pin, the pin must be a path start point (for example, the clock pin of a flip-flop). If you specify a cell, one path start point on that cell is affected.
-rise_from <from_list>	List of path startpoints Same as the -from option, except that the path must rise from the objects specified.

coreTools Command Reference Index

-fall_from <from_list>	List of path startpoints Same as the -from option, except that the path must fall from the objects specified. List of path through points Nets are interpreted to imply the leaf-level driver pins. If you do not specify -through, all timing paths specified using the -from and -to options are disabled. You can specify multiple groups of <through_list> by using multiple -through options. Design Compiler uses a logic OR function to combine objects within one -through option; a path is marked false if it passes through any object in the list. Design Compiler uses a logic AND function to combine objects from separate -through options; a path is marked false if it passes through one or more objects in each list.
-through <through_list>	For PrimeTime, the value of the environment variable timing_through_compatibility determines how PrimeTime interprets -through options. When timing_through_compatibility is false (the default), objects specified with one or more -through options are interpreted as previously described for Design Compiler. When timing_through_compatibility is true, PrimeTime does not allow specification of through objects using multiple -through options. PrimeTime uses a logic AND function to group objects specified within one -through option; a path is marked false if it passes through all objects in the list.
-rise_through <through_list>	List of path through points Same as the -through option, but applies only to paths with a rising transition at the specified objects.
-fall_through <through_list>	List of path through points Same as the -through option, but applies only to paths with a falling transition at the specified objects.
-to <to_list>	List of path endpoints If you do not specify <to_list>, all paths from start points in <from_list> are marked as false. The <to_list> can include clocks, pins, or ports. If you specify a clock, all path endpoints related to the specified clock are considered. If you specify an internal pin, the pin must be a path endpoint (for example, the data pin of a

coreTools Command Reference Index

	flip-flop). If you specify a cell, one path endpoint on that cell is affected.
-rise_to <to_list>	List of path endpoints Same as the -to option, but applies only to paths rising at the endpoint.
-fall_to <to_list>	List of path endpoints Same as the -to option, but applies only to paths falling at the endpoint.
-reset_path	First reset this path Only information of the same rise/fall or setup/hold type is reset. This is equivalent to using the reset_path command with similar arguments before executing set_false_path.

Description

The set_false_path command creates a coreTool timing exception object for the current_design. The coreTool stores the timing exception object in the design knowledge database (KB). When coreConsultant generates a strategy for Design Compiler or PrimeTime, coreConsultant uses the timing exception object to generate a similar set_false_path command for Design Compiler or PrimeTime.

In Design Compiler or PrimeTime, the set_false_path command removes timing constraints from specified paths so that Design Compiler and PrimeTime do not perform setup and/or hold time checking on those paths. Use set_false_path to remove timing constraints from paths that do not occur during normal circuit operation.

By default, the coreTools check the validity of the set_false_path command to ensure that the nodes specified in the set_false_path command options exist in the current_design and are of the correct type. However, because the coreTools only model the structural part of a design, some timing exception start/end/through points may not exist in the coreTool model of the design. In such cases, you can use the -quiet option to suppress validity checking. If you use the -quiet option and you specify incorrect or invalid arguments to set_false_path, an error will occur during synthesis.

Examples

To remove timing constraints on paths from u12 to u34:

```
coreConsultant> set_false_path -from {u12} -to {u34}
```

To remove timing constraints on paths from u14/Z to u29/Reset:

```
coreConsultant> set_false_path -rise -from {u14/Z}  
-to {u29/Reset}
```


coreTools Command Reference Index

To disable hold checking (minimum delay timing) for endpoints clocked by PHI1, so the flip-flops and latches clocked by PHI1 are checked for setup violations, but not for hold violations:

```
coreConsultant> set_false_path -hold -to [get_clocks PHI1]
```

To remove timing constraints for all paths that first pass through either u1/Z or u2/Z then pass through u5/Z or u6/Z:

```
coreConsultant> set_false_path -through {u1/Z u2/Z}  
-through {u5/Z u6/Z}\fP
```

See Also

set_multicycle_path (2), set_max_delay (2), set_min_delay (2), reset_path
(2), set_disable_timing (2)

set_file_prefix

Sets the file prefix to be prepended to each design name.

Syntax

```
string set_file_prefix [-component <component>] prefix
    string <component>
    string prefix
```

Parameters

-component <component>	hierachical path to the component.
prefix	File name prefix

Description

This command sets the name of the prefix to be associated with each RTL file in the scenario where file prefixing is enabled. File prefixing is used to enable renaming of RTL files such that multiple instances of a component can be guaranteed to not have overlapping file names.

In coreConsultant, a single prefix is used for all files. When the source files are written to disk, this prefix is prepended (as <prefix>_<original name>) to each file name. The prefix is specified in the SetDesignAndFilePrefix activity, provided that the coreKit being used was packaged with file prefixing enabled.

In coreAssembler, file prefixing is enabled if the components support it. Each component gets a unique file prefix. file prefixing for each component can be set by passing the component path to the command if prefixing is enabled. The design prefix is based on the hierarchical path to the component instance.

Examples

The following command sets the fileprefix in assembler on the i_enabled component:

```
coreAssembler> set_file_prefix -component /i_enabled MyFilePrefix
```

See Also

[get_file_prefix \(2\)](#), [get_design_prefix \(2\)](#), [set_design_prefix \(2\)](#), [get_macro_prefix \(2\)](#), [set_macro_prefix \(2\)](#)

set_filegroup_attribute

Set an attribute on a list of filegroups

Syntax

string **set_filegroup_attribute** *list attr* [*value*]

string *list*

string *attr*

string *value*

Parameters

list	List of filegroups on which to set the attribute.
attr	Name of the attribute to set on the ports. For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so Tcl does not interpret the subscript as a sub-command.
value	Value for the attribute. For boolean attributes, <value> is optional (default = true).

Description

The set_filegroup_attribute command sets the specified attribute to the specified value on the specified filegroup(s). You can specify the filegroups by name (as a list if specifying more than one filegroup) or as a collection.

To get the current value of an attribute on a filegroup, use the get_filegroup_attribute command.

Examples

To set the Configurable attribute to true on the testbench filegroup:

```
coreConsultant> set_filegroup_attribute testbench Configurable true
```

See Also

[get_filegroup_attribute \(2\)](#)

set_filegroup_parameter
Set a parameter for a filegroup

Syntax

```
string set_filegroup_parameter fgroup paramName value
    string fgroup
    string paramName
    string value
```

Parameters

fgroup	The filegroup to get the attribute value from
paramName	The name of the parameter you want to set.
value	Value for the parameter.

Description

The set_filegroup_parameter command sets the value of a parameter associated with a filegroup. Note that only configurable filegroups have parameters. See the coreBuilder User Guide for details on how to define configurable filegroups and associate parameters with them.

The most common usage of this command is to set the values of filegroup parameters in a batch script. This is not typically needed within coreBuilder as default values can be assigned to the parameter as part its creation via create_configuration_parameter. In coreConsultant and coreAssembler, this command can be used in batch scripts to set parameters to specific values prior to automatically completing the activity associated with a configurable filegroup.

Examples

The following commands set parameters on a configurable filegroup and then complete the associated activity. Completing the activity will cause the files within the filegroup to be written to disk, configured based on the values of the parameters that were just set.

```
set_filegroup_parameter SimulationSetup Simulator vcs
set_filegroup_parameter SimulationSetup BatchOrGui batch
autocomplete_activity SimulationSetup
```

See Also

[create_configuration_parameter \(2\), get_filegroup_parameter \(2\)](#)

set_generator_parameter

Set component generator parameter value

Syntax

string **set_generator_parameter** [-component *<component_name>*] *generator*
parameter value

string *<component_name>*

string *generator*

string *parameter*

string *value*

Parameters

-component	
<i><component_name></i>	component name of the component generator
<i>generator</i>	Name of the generator the parameter belongs to
<i>parameter</i>	The name of the parameter that you want to set.
<i>value</i>	The value of the parameter.

Description

This command set the component generator parameter value.

Examples

to set the parameter 'param1' of generator 'Generator' value to 1:

```
coreAssembler>set_generator_parameter Generator param1 1
```

See Also

[get_generator_parameter \(2\)](#), [invoke_generator \(2\)](#)

`set_hdl_file_list`

Defines an alternate HDL file list for a component.

Syntax

```
string set_hdl_file_list [-reset] [-component <component name>] [-append]
[-incdirs <paths>] [-external] [files]
    string <component name>
    string <paths>
    string files
```

Parameters

<code>-reset</code>	Reset to use of the default (packaged) HDL file list
<code>-component</code> <code><component name></code>	Indicates the component to be updated
<code>-append</code>	Append the files to the existing file list to create the alternate file list
<code>-incdirs</code> <code><paths></code>	List of new include directories for the new file list
<code>-external</code>	File list contains file that are external to the workspace
<code>files</code>	List of files (and library/language indicators)

Description

This command can be used to alter the set of HDL files to be associated with a given component. It is typically only used to add files since files can be removed by simply removing them from the 'src' directory.

By default all files must reside within the workspace. If files are to be referenced from outside the workspace, add the `-external` option to disable checking of the file location.

Note that when this command is used to define the file list, the files are not parsed as they would be in coreBuilder. This means there is no automated process to figure out the include files. These files must therefore be specified explicitly as part of the file list. This is done by including an entry in the list of files using the format "include FileName". This is a single list element containing two strings. The keyword 'include' followed by the file name. Files specified in this manner are only returned by `get_hdl_file_list` when the `-includes` option is specified. This is required to ensure that 'get_hdl_file_list

`-incdirs` returns the correct include directory names. If files are added with `-append` then the include file information for files in the original file list is preserved and does not need to be specified.

If the new files require new include directories, they can be specified with `-incdirs` option.

Examples

Add the file `mymemory.vhd` to the list of files associated with component `i_uart`.

```
file copy mymemory.vhd [get_logical_dir src]
set newfile [get_logical_dir src]/mymemory.vhd
set files "$newfile [get_hdl_file_list]"
set_hdl_file_list $files
```

Define a simple file list including an include file.

```
set files [list top.v [list include my_incs.vh]]
set_hdl_file_list $files
```

Add new files with `-append`. The original files are added to the new list, and include file information is preserved. The `-incdirs` options is used to add a new include directory.

```
set newfile [get_logical_dir src]/mymemory.sv
set newIncfile [get_logical_dir src]/new_incDir/mymemory.sv
file copy mymemory.sv $newfile
file copy mymemory_inc.sv $newIncfile
set_hdl_file_list -append "$newfile $newIncfile" -incdirs new_incDir
```

See Also

[get_hdl_file_list\(2\)](#)

set_instance_parameter

Command to set the value of instance parameter in terms of the subsystem parameter.

Syntax

```
string set_instance_parameter -instance -parameter <value>
      string
      string <value>
```

Parameters

-instance	Instance Name
-parameter	Name of the parameter
<value>	Name of the subsystem parameter

Description

This command is used to set the value of a parameter on a component instance within a coreAssembler subsystem. The value specified can be static but is typically specified in terms of parameters within the containing design, as a means to support parameter propagation within a hierarchical subsystem.

Examples

Define a parameter width and pass into an instance via it's DataWidth parameter.

```
cA> set p [create_subsystem_parameter -type integer -default 8 width]
cA> set_attribute $p -attr MinValue 2
cA> set_attribute $p -attr MaxValue 32
cA> set_instance_parameter -instance U1 -parameter DataWidth @width
```

See Also

[create_subsystem_parameter \(2\)](#)

set_instance_wire_names

Store mapping of port names to wire names.

Syntax

```
string set_instance_wire_names [-name <name>] wireMap
    string <name>
    string wireMap
```

Parameters

-name <name>	Name for mapping
wireMap	Wire name to port name mapping

Description

The set_instance_wire_names command is used to define non-default wire names or connections to be use when generating an instance of a subsystem using write_subsystem_hdl. By default, the connections to the component instance are assumed to match the name of the port. In order to change this default behavior, a name mapping can be defined using set_instance_wire_names.

The 'wireMap' argument is specified as a list of alternating port names and desired connection names. The -name argument can be used to define multiple different mappings for the same subsystem. The default name is 'default'. The mapping name is passed to the write_subsystem_hdl command using the -instance_type argument.

Examples

All of the examples assume a simple subsystem named my_subsystem with ports in1, in2, in3, out1, out2, and out3 with directionality as indicated in the name. The default component instance would be generated as follows:

```
write_subsystem_hdl -format verilog -type testbench -tbinstancename u_DUT
-instance_type default out.v
This results in the following in out.v:
```

```
u_DUT
(.in1 (in1),
.in2 (in2),
```

```
.in3 (in3),
.out1 (out1),
.out2 (out2),
.out3 (out3));
```

Now to change the generated instance so that in1 is connected to a wire named my_in1, in2 is tied to zero, and out1 is left open, you can do the following:

```
set_instance_wire_names {in1 my_in1 in2 zero out1 open}
write_subsystem_hdl -format verilog -type testbench -tbinstancename u_DUT
-instance_type default out.v
```

This will result in the following in out.v:

```
u_DUT
(.in1 (my_in1),
.in2 ('0),
.in3 (in3),
.out1 (/*open*/),
.out2 (out2),
.out3 (out3));
```

In order to define a second mapping without impacting the first, use the -name option:

```
set_instance_wire_names -name secondary {in2 one}
write_subsystem_hdl -format verilog -type testbench -tbinstancename u_DUT
-instance_type secondary out.v
```

This will result in the following in out.v:

```
u_DUT
(.in1 (in1),
.in2 ('1),
.in3 (in3),
.out1 (out1),
.out2 (out2),
.out3 (out3));
```

See Also

set_interface_attribute

Set an attribute on the specified interface

Syntax

```
string set_interface_attribute [-instance] interface attr [value]
      string interface
      string attr
      string value
```

Parameters

	The interface name identifies an instance instead of the definition
-instance	This option is only needed in coreBuilder, as the specified name is assumed to be that of an interface instance in coreAssembler because interface definitions can not be modified in coreAssembler. In coreBuilder, the specified name is assumed to indicate an interface definition, unless this option is specified.
interface	The interface to set the attribute value on
attr	The name of the attribute to set the value on
value	Value for the attribute

Description

This command is used to set the value of the named attribute on the specified interface instance or interface definition. This command is used in files defining interfaces (interface definitions) to set attributes on the interface definitions. It is also used in coreBuilder batch scripts to set attributes on specific interface instances where the attribute value needs to be different than that on the interface definition.

Examples

To lower the setting of MaxConsumers for an AHB Master interface when the definition supports 16 but this implementation only supports 8:

```
prompt> set_interface_attribute AHBMaster MaxConsumers 8
```

See Also

[get_interface_attribute\(2\)](#)

set_interface_parameter
Set a parameter on an interface

Syntax

```
string set_interface_parameter [-instance] interface paramName value
    string interface
    string paramName
    string value
```

Parameters

	The interface name identifies an instance instead of the definition
-instance	This option is only needed in coreBuilder, as the specified name is assumed to be that of an interface instance in coreAssembler because interface definitions can not be modified in coreAssembler. In coreBuilder, the specified name is assumed to indicate an interface definition, unless this option is specified.
interface	The interface to set the parameter on
paramName	Name of the parameter you want to set
value	Value for the parameter

Description

The set_interface_parameter command is used to set the value of the specified parameter of the specified interface instance (or definition). This command is typically used in batch scripts in coreBuilder and coreAssembler to set the value of interface parameters on interface instances, and in interface definition files (loaded into coreBuilder) to set parameters on interface definitions.

Examples

The following commands set the value of the width parameter in an interface definition file. This would more typically be done using the -default option to the create_interface_parameter command, but can be done this way as well.

```
create_interface simpleIntf
create_interface_parameter width -type integer
set_interface_parameter simpleIntf width 8
```

To set the value of the width parameter in a coreBuilder batch script on the 'simple' instance of the simpleIntf interface:

```
create_interface_instance simple -interface simpleIntf -type provider  
set_interface_parameter -instance simple width 16
```

See Also

[create_interface \(2\), create_interface_instance \(2\)](#)
[create_interface_parameter \(2\), get_interface_parameter \(2\)](#)

`set_interface_parameter_attribute`

Set an attribute on the specified interface parameter

Syntax

```
string set_interface_parameter_attribute [-instance] interface
parameterName attr [value]
    string interface
    string parameterName
    string attr
    string value
```

Parameters

	The interface name identifies an instance instead of the definition
-instance	This option is only needed in coreBuilder, as the specified name is assumed to be that of an interface instance in coreAssembler because interface definitions can not be modified in coreAssembler. In coreBuilder, the specified name is assumed to indicate an interface definition, unless this option is specified.
interface	The interface to get the parameter from
parameterName	The interface parameter to set the attribute value on
attr	The name of the attribute to set the value on
value	Value for the attribute

Description

This command is used to set the value of the named attribute on the specified interface parameter. This command is used in files defining interfaces (interface definitions) to set attributes on the parameters of interface definitions. It is also used in coreBuilder batch scripts to set attributes on specific interface instance parameters where the attribute value needs to be different than that on the interface definition or where the attribute is not set on the interface definition.

Examples

To set the value of DataWidth parameter (i.e. the Value attribute):

```
prompt> set_interface_parameter_attribute AHBexported DataWidth Value 16
```


See Also

[get_interface_parameter_attribute \(2\)](#)

`set_interface_port_attribute`

Set an attribute on the specified interface port

Syntax

`string set_interface_port_attribute [-instance] interface portName attr
[value]``string interface
string portName
string attr
string value`

Parameters

	The interface name identifies an instance instead of the definition
-instance	This option is only needed in coreBuilder, as the specified name is assumed to be that of an interface instance in coreAssembler because interface definitions can not be modified in coreAssembler. In coreBuilder, the specified name is assumed to indicate an interface definition, unless this option is specified.
interface	The interface to get the port from
portName	The interface port to set the attribute value on
attr	The name of the attribute to set the value on
value	Value for the attribute

Description

This command is used to set the value of the named attribute on the specified interface port. This command is used in files defining interfaces (interface definitions) to set attributes on the ports of interface definitions. It is also used in coreBuilder batch scripts to set attributes on specific interface instance ports where the attribute value needs to be different than that on the interface definition or where the attribute is not set on the interface definition (as is the case with InterfaceLink).

Examples

To set the value of the InterfaceLink attribute on an the port of an interface instance:

```
prompt> set_interface_port_attribute AHBexported pwdata InterfaceLink
```

top_pwdata

See Also

get_interface_port_attribute (2), escaped_name (2)_____

set_macro_prefix

Sets the macro prefix to be prepended to each macro name.

Syntax

```
string set_macro_prefix [-component <component>] prefix
    string <component>
    string prefix
```

Parameters

-component <component>	hierachical path to the component.
prefix	File name prefix

Description

This command sets the name of the macro prefix to be associated with macros in the RTL files where macro prefixing is enabled. Macro prefixing is used to enable renaming of macros in the RTL files.

In coreConsultant, a single prefix is used for RTL file macros. When the source files are written to disk, this macro prefix is prepended (as <prefix>_<original name>) to each macro found in the RTL files. The prefix is specified in the SetDesignAndFilePrefix activity, provided that the coreKit being used was packaged with file prefixing enabled.

In coreAssembler each components RTL file macros get a unique macro prefix which matches the file and is based off the component path. Also, macro prefix for each component can be set by passing the component path to the command if prefixing is enabled.

Examples

The following command sets the macroPrefix in assembler on the i_enabled component:

```
coreAssembler> set_macro_prefix -component /i_enabled MyMacroPrefix
```

See Also

get_macro_prefix (2) , set_file_prefix (2) , get_file_prefix (2) ,	_____
set_design_prefix (2) , get_design_prefix (2) , prefix_macros (2)	_____

set_max_delay

Specifies a maximum delay target for paths in the current design

Syntax

```
string set_max_delay [-rise] [-fall] [-ignore_clock_latency] [-comment
<text>] [-from <from_list>] [-rise_from <from_list>] [-fall_from
<from_list>] [-through <through_list>] [-rise_through <through_list>]
[-fall_through <through_list>] [-to <to_list>] [-rise_to <to_list>]
[-fall_to <to_list>] [-group_path <group_name>] [-reset_path] delay_value
    string <text>
    string <from_list>
    string <through_list>
    string <to_list>
    string <group_name>
    string delay_value
```

Parameters

-rise	Rise delay If you do not specify -rise or -fall, both rising and falling delays are constrained.
-fall	Fall delay If you do not specify -rise or -fall, both rising and falling delays are constrained.
-ignore_clock_latency	Indicates that launch and capture clock latencies are to be ignored
-comment <text>	Associate a string description with the command for tracking purposes.
-from <from_list>	List of path startpoints If you specify a clock, all path start points related to that clock are constrained. If you specify a cell name, one path start point on that cell is constrained. All paths from these start points to the end points in the <to_list> are constrained to <delay_value>. If you do not specify <to_list>, all paths from <from_list> are constrained. This list cannot include output ports.
-rise_from <from_list>	List of path startpoints Same as the -from option, except that the path must rise from the objects specified.
-fall_from <from_list>	List of path startpoints Same as the -from option, except that the path must fall from the objects specified.

coreTools Command Reference Index

	List of path through points Nets are interpreted to imply the leaf-level driver pins. If you do not specify -through, all timing paths specified using the -from and -to options are constrained. You can specify multiple groups of <through_list> by using multiple -through options. Design Compiler uses a logic OR function to combine objects within one -through option; a path is constrained for maximum delay if it passes through any object in the list. Design Compiler uses a logic AND function to combine objects from separate -through options; a path is constrained for maximum delay if it passes through one or more objects in each list. For PrimeTime, the value of the environment variable timing_through_compatibility determines how PrimeTime interprets -through options. When timing_through_compatibility is false (the default), objects specified with one or more -through options are interpreted as previously described for Design Compiler. When timing_through_compatibility is true, PrimeTime does not allow specification of through objects using multiple -through options. PrimeTime uses a logic AND function to group objects specified within one -through option; a path is constrained for maximum delay if it passes through all objects in the list.
-through <through_list>	
-rise_through <through_list>	List of path through points Same as the -through option, but applies only to paths with a rising transition at the specified objects.
-fall_through <through_list>	List of path through points Same as the -through option, but applies only to paths with a falling transition at the specified objects.
-to <to_list>	List of path endpoints All paths from start points in the <from_list> to the end points in the <to_list> are constrained to <delay_value>. If you do not specify a <from_list>, all paths to <to_list> are constrained. If you specify a cell, one path end point on that cell is constrained. If you specify a clock, all path end points related to that clock are constrained. This list cannot include input ports.
-rise_to <to_list>	

coreTools Command Reference Index

	List of path endpoints Same as the -to option, but applies only to paths rising at the endpoint.
-fall_to <to_list>	List of path endpoints Same as the -to option, but applies only to paths falling at the endpoint.
-group_path <group_name>	Name of group for paths If the group does not already exist, set_max_delay creates the group. This is equivalent to specifying the separate command group_path -name <group_name> -from <from_list> -to <to_list> in addition to the set_max_delay command. If you do not specify this argument, the existing path grouping is not changed.
-reset_path	First reset this path If used with -to only, all paths leading to the specified end points are reset. If used with -from only, all paths leading from the specified start points are reset. If used with -from and -to, only paths between those points are reset. Only information of the same rise/fall setup/hold type is reset. This is equivalent to using the reset_path command with similar arguments before you issue set_max_delay.
delay_value	Target path delay Express <delay_value> in the time unit used by the currently loaded technology library. If a path start point is on a sequential device, clock skew is included in the computed delay. If a path start point has an input delay specified, that delay value is added to the path delay. If a path end point is on a sequential device, clock skew and library setup time are included in the computed delay. If the end point has an output delay specified, that delay is added into the path delay.

Description

The set_max_delay command creates a coreTool timing exception object for the current_design. The coreTool stores the timing exception object in the design knowledge database (KB). When coreConsultant generates a strategy for Design Compiler or PrimeTime, coreConsultant uses the timing exception object to generate a similar set_max_delay command for Design Compiler or PrimeTime.

coreTools Command Reference Index

In Design Compiler and PrimeTime, the `set_max_delay` command sets a target maximum delay for timing paths. Design Compiler uses `set_max_delay` to constrain a path for maximum delay. PrimeTime uses `set_max_delay` to set a target maximum delay for output ports in a combinational design or to override single cycle timing in cases where `set_multicycle_path` is not sufficient.

By default, the coreTools check the validity of the `set_max_delay` command to ensure that the nodes specified in the `set_max_delay` command options exist in the current_design and are of the correct type. However, because the coreTools only model the structural part of a design, some timing exception start/end/through points may not exist in the coreTool model of the design. In such cases, you can use the `-quiet` option to suppress validity checking. If you use the `-quiet` option and you specify incorrect or invalid arguments to `set_max_delay`, an error will occur during synthesis.

Examples

To set the maximum delay for any path to port Y to 10 units:

```
coreConsultant> set_max_delay -to {Y} 10.0
```

To set the maximum delay for all paths from u1 or u2 to u7 to 15.0 units:

```
coreConsultant> set_max_delay -from {u1 u2} -to {u7} 15.0
```

To set the maximum delay for all paths to endpoints clocked by PHI2 to 8.5 units:

```
coreConsultant> set_max_delay -to [get_clocks PHI2] 8.5
```

To set the maximum delay for all paths leading to ports named busA[*] to 5.0:

```
coreConsultant> set_max_delay -to "busA[*]" 5.0
```

To set the maximum delay for all timing paths from u1/X to u9/Y, that pass through one or more of {U1/Z U2/Z} and one or more of {U3/Z U4/C} to 8.0 units:

```
coreConsultant> set_max_delay -from u1/X -through {U1/Z U2/Z}  
-through {U3/Z U4/C}  
-to u9/Y 8.0
```

See Also

[set_false_path \(2\)](#), [set_multicycle_path \(2\)](#), [set_min_delay \(2\)](#),
[reset_path \(2\)](#), [set_disable_timing \(2\)](#)

[API Index](#) [Command Reference](#)

`set_memory_map_attribute`

Set the value of an attribute on a memory map.

Syntax

```
string set_memory_map_attribute map attr [value]  
    string map  
    string attr  
    string value
```

Parameters

<code>map</code>	The memory map to set the attribute on.
<code>attr</code>	Name of the attribute to set.
<code>value</code>	Value of the attribute to set.

Description

This command sets an attribute on a memory map.

Examples

The following command sets the MemoryAccess type of the memory map 'map1' to be 'read-only':

```
coreBuilder> set_memory_map_attribute map1 MemoryAccess "read-only"
```

See Also

[memMap \(3\)](#), [create_memory_map \(2\)](#), [get_memory_map_attribute \(2\)](#)

set_memory_mode

Set the memory mode associated with the current/selected design

Syntax

```
string set_memory_mode mode  
      string mode
```

Parameters

mode Name of the memory mode to set for the current/selected design.

Description

This command is used to set the current memory mode of the current/selected design. By default to 'DEFAULT' mode.

This current memory mode will be used to evaluate the register/registerField attributes value.

Examples

Assuming the 'cpu' mode has been created using create_memory_mode. The following command sets the current memory mode to 'cpu'.

```
coreBuilder> set_memory_mode cpu
```

See Also

[get_register_field_attribute \(2\)](#), [set_register_attribute \(3\)](#),
[get_memory_mode \(4\)](#), [create_memory_mode \(5\)](#)

NAME

set_message_info

Set some information about diagnostic messages.

SYNTAX

```
string set_message_info -id message_id [-limit
max_limit|-stop_on|-stop_ff]
```

```
string message_id integer max_limit
```

ARGUMENTS

-id *message_id* Information is to be set for the given *message_id*. The message must exist. Although different constraints allow different message types, no constraint allows severe or fatal messages.

-limit *max_limit*
Set the maximum number of occurrences for *message_id*. This is an integer greater than or equal to zero. If you set it to zero, that means the number of occurrences of the message is unlimited. Messages which occur after a limit is reached are automatically suppressed.

-stop_on Force Tcl error if message is emitted.

-stop_off Turn off a previous **-stop_on** directive

DESCRIPTION

The **set_message_info** command sets constraints on diagnostic messages (typically error, warning, and informational messages).

coreTools Command Reference Index

Currently, you can set a upper limit for the number of occurrences of a message. You can set this to zero to indicate that there is no limit. You can retrieve the current limit for a message using the **get_message_info** command. *When the limit is exceeded, all future occurrences of the message are automatically suppressed. A count of total occurrences (including those suppressed) can be retrieved using **get_message_info**.*

EXAMPLES

The following example uses **set_message_info** to set a limit on the number of APP-027 messages to 100. When the 101st APP-027 message is about to be issued, you will be warned that the limit has been exceeded, and that all future occurrences will be suppressed.

```
prompt> set_message_info -id APP-027 -limit 100
prompt> do_command
Warning: can't find node U27.1 (APP-027)
Warning: can't find node U27.2 (APP-027)
Warning: can't find node U27.3 (APP-027)
...
Warning: can't find node U27.100 (APP-027)
Note - message 'APP-027' limit (100) exceeded. Remainder will be suppressed.
1
```

SEE ALSO

```
get_message_info(2)
get_message_ids(2)
print_message_info(2)
suppress_message(2)
```


set_min_delay

Specifies a minimum delay target for paths in the current design

Syntax

```
string set_min_delay [-rise] [-fall] [-ignore_clock_latency] [-comment
<text>] [-from <from_list>] [-rise_from <from_list>] [-fall_from
<from_list>] [-through <through_list>] [-rise_through <through_list>]
[-fall_through <through_list>] [-to <to_list>] [-rise_to <to_list>]
[-fall_to <to_list>] [-reset_path] delay_value
    string <text>
    string <from_list>
    string <through_list>
    string <to_list>
    string delay_value
```

Parameters

-rise	Rise delay If you do not specify -rise or -fall, both rising and falling delays are constrained.
-fall	Fall delay If you do not specify -rise or -fall, both rising and falling delays are constrained.
-ignore_clock_latency	Indicates that launch and capture clock latencies are to be ignored
-comment <text>	Associate a string description with the command for tracking purposes.
-from <from_list>	List of path startpoints If you specify a clock, all path start points related to that clock are constrained. If you specify a cell name, one path start point on that cell is constrained. All paths from these start points to the end points in the <to_list> are constrained to <delay_value>. If you do not specify <to_list>, all paths from <from_list> are constrained. The <from_list> list cannot include output ports.
-rise_from <from_list>	List of path startpoints Same as the -from option, except that the path must rise from the objects specified.
-fall_from <from_list>	List of path startpoints Same as the -from option, except that the path must fall from the objects specified.

coreTools Command Reference Index

-through <through_list>	List of path through points Nets are interpreted to imply the leaf-level driver pins. If you do not specify -through, all timing paths specified using the -from and -to options are constrained. You can specify multiple groups of <through_list> by using multiple -through options. Design Compiler uses a logic OR function to combine objects within one -through option; a path is constrained for minimum delay if it passes through any object in the list. Design Compiler uses a logic AND function to combine objects from separate -through options; a path is constrained for minimum delay if it passes through one or more objects in each list. For PrimeTime, the value of the environment variable timing_through_compatibility determines how PrimeTime interprets -through options. When timing_through_compatibility is false (the default), objects specified with one or more -through options are interpreted as previously described for Design Compiler. When timing_through_compatibility is true, PrimeTime does not allow specification of through objects using multiple -through options. PrimeTime uses a logic AND function to group objects specified within one -through option; a path is constrained for minimum delay if it passes through all objects in the list.
-rise_through <through_list>	List of path through points Same as the -through option, but applies only to paths with a rising transition at the specified objects.
-fall_through <through_list>	List of path through points Same as the -through option, but applies only to paths with a falling transition at the specified objects.
-to <to_list>	List of path endpoints All paths from start points in the <from_list> to the end points in the <to_list> are constrained to <delay_value>. If you do not specify a <from_list>, all paths to <to_list> are constrained. If you specify a cell, one path end point on that cell is constrained. If you specify a clock, all path end points related to that clock are constrained. The <to_list> cannot include input ports.
-rise_to <to_list>	

coreTools Command Reference Index

	List of path endpoints Same as the -to option, but applies only to paths rising at the endpoint.
-fall_to <to_list>	List of path endpoints Same as the -to option, but applies only to paths falling at the endpoint.
-reset_path	First reset this path If used with -to only, all paths leading to the specified end points are reset. If used with -from only, all paths leading from the specified start points are reset. If used with -from and -to, only paths between those points are reset. Only information of the same rise/fall setup/hold type is reset. This is equivalent to using the reset_path command with similar arguments before you issue set_min_delay.
delay_value	Target path delay Express <delay_value> in the time unit used by the currently loaded technology library. If a path start point is on a sequential device, clock skew is included in the computed delay. If a path start point has an input delay specified, that delay value is added to the path delay. If a path end point is on a sequential device, clock skew and library setup time are included in the computed delay. If the end point has an output delay specified, that delay is added into the path delay.

Description

The set_min_delay command creates a coreTool timing exception object for the current_design. The coreTool stores the timing exception object in the design knowledge database (KB). When coreConsultant generates a strategy for Design Compiler or PrimeTime, coreConsultant uses the timing exception object to generate a similar set_min_delay command for Design Compiler or PrimeTime.

In Design Compiler and PrimeTime, the set_min_delay command sets a target minimum delay for timing paths. Design Compiler uses set_min_delay to add delay to a path as needed to meet the minimum delay requirement. PrimeTime uses set_min_delay to set a target minimum delay for output ports in a combinational design or to override single cycle timing in cases where set_multicycle_path is not sufficient.

By default, the coreTools check the validity of the set_min_delay command to ensure that the nodes specified in the set_min_delay command options

exist in the current_design and are of the correct type. However, because the coreTools only model the structural part of a design, some timing exception start/end/through points may not exist in the coreTool model of the design. In such cases, you can use the -quiet option to suppress validity checking. If you use the -quiet option and you specify incorrect or invalid arguments to set_min_delay, an error will occur during synthesis.

Examples

To set minimum delay for any path to pin Y to 12.5 units:

```
coreConsultant> set_min_delay -to Y 12.5
```

To set minimum delay for all paths from A1 or A2 to Z5 to 15.0 units:

```
coreConsultant> set_min_delay -from {A1 A2} -to {Z5} 15.0
```

To set minimum delay for all paths to endpoints clocked by CLK2 to 5 units:

```
coreConsultant> set_min_delay -to [get_clocks CLK2] 5
```

To set minimum delay for all timing paths from u6/X to u14/Y, that pass through one or more of {U1/Z U2/Z} and one or more of {U3/Z U4/C}, to 6.0 units:

```
coreConsultant> set_min_delay -from u6/X -through {U1/Z U2/Z}
-through {U3/Z U4/C} -to u14/Y 6.0
```

See Also

set_false_path (2), set_multicycle_path (2), set_max_delay (2),
reset_path (2), set_disable_timing (2)

set_multicycle_path

Modifies the single-cycle timing relationship of a constrained path

Syntax

```
string set_multicycle_path [-rise] [-fall] [-setup] [-hold] [-comment
<text>] [-start] [-end] [-from <from_list>] [-rise_from <from_list>]
[-fall_from <from_list>] [-through <through_list>] [-rise_through
<through_list>] [-fall_through <through_list>] [-to <to_list>] [-rise_to
<to_list>] [-fall_to <to_list>] [-reset_path] path_multiplier
    string <text>
    string <from_list>
    string <through_list>
    string <to_list>
    string path_multiplier
```

Parameters

-rise	Set rising delays to be multicycle The default is that both rising and falling delays are affected. Rise refers to a rising value at the path end point.
-fall	Set falling delays to be multicycle The default is that both rising and falling delays are affected. Fall refers to a falling value at the path end point.
-setup	Use path_multiplier for setup only If you do not specify -setup or -hold, <path_multiplier> is used for setup and 0 is used for hold. Note that changing the multiplier for setup affects the hold check as well.
-hold	Use path_multiplier for hold only If you do not specify -setup or -hold, <path_multiplier> is used for setup and 0 is used for hold. Note that changing the multiplier for setup affects the hold check as well.
-comment <text>	Associate a string description with the command for tracking purposes.
-start	Apply multiplier to clock at startpoints The -start and -end options are only needed for multi-frequency designs; otherwise start and end are equivalent. The start clock is the clock source related to the register or primary input at the path start point. The default is to move the setup check relative to the end clock, and the hold check

coreTools Command Reference Index

	relative to the start clock. A setup multiplier of 2 with -start moves the relation backward one cycle of the start clock. A hold multiplier of 1 with -start moves the relation forward one cycle of the start clock. Apply multiplier to clock at endpoints The -start and -end options are only needed for multi-frequency designs; otherwise start and end are equivalent. The end clock is the clock source related to the register or primary output at the path endpoint. The default is to move the setup check relative to the end clock, and the hold check relative to the start clock. A setup multiplier of 2 with -end moves the relation forward one cycle of the end clock. A hold multiplier of 1 with -end moves the relation backward one cycle of the end clock.
-end	List of path startpoints If a clock is specified, all registers and primary inputs related to that clock are used as path start points. If you specify a cell, one path start point on that cell is affected.
-from <from_list>	List of path startpoints Same as the -from option, except that the path must rise from the objects specified.
-rise_from <from_list>	List of path startpoints Same as the -from option, except that the path must fall from the objects specified.
-fall_from <from_list>	List of path through points Nets are interpreted to imply the leaf-level driver pins. If you do not specify -through, all timing paths specified using the -from and -to options are affected. You can specify multiple groups of <through_list> by using multiple -through options. Design Compiler uses a logic OR function to combine objects within one -through option; a path is affected by this command if it passes through any object in the list. Design Compiler uses a logic AND function to combine objects from separate -through options; a path is affected by this command if it passes through one or more objects in each list. For PrimeTime, the value of the environment variable timing_through_compatibility determines how PrimeTime interprets -through options. When timing_through_compatibility is false (the default), objects specified with one or more -through options are interpreted as previously described for Design Compiler. When
-through <through_list>	

coreTools Command Reference Index

	timing_through_compatibility is true, PrimeTime does not allow specification of through objects using multiple -through options. PrimeTime uses a logic AND function to group objects specified within one -through option; a path is affected by this command if it passes through all objects in the list. List of path through points
-rise_through <through_list>	Same as the -through option, but applies only to paths with a rising transition at the specified objects. List of path through points
-fall_through <through_list>	Same as the -through option, but applies only to paths with a falling transition at the specified objects. List of path endpoints
-to <to_list>	If you specify a clock, all registers and primary outputs related to that clock are used as path end points. If you specify a cell, one path end point on that cell is affected. List of path endpoints
-rise_to <to_list>	Same as the -to option, but applies only to paths rising at the endpoint. List of path endpoints
-fall_to <to_list>	Same as the -to option, but applies only to paths falling at the endpoint. First reset this path
-reset_path	If used with -to only, all paths leading to the specified end points are reset. If used with -from only, all paths leading from the specified start points are reset. If used with -from and -to, only paths between those points are reset. Only information of the same rise/fall setup/hold type is reset. This is equivalent to using the reset_path command with similar arguments before executing the set_multicycle_path command. Value for the cycle multiplier
path_multiplier	If you use -setup, this value is applied to setup path calculations. If you use -hold, this value is applied to hold path calculations. If you do not specify -setup or -hold, path_multiplier is used for setup and 0 is used for hold. Note that changing the multiplier for setup affects the hold check as well.

Description

The `set_multicycle_path` command creates a coreTools timing exception object for the current `design`. The coreTool stores the timing exception object in the design knowledge database (KB). When coreConsultant generates a strategy for Design Compiler or PrimeTime, coreConsultant uses the timing exception object to generate a similar `set_multicycle_path` command for Design Compiler or PrimeTime.

In Design Compiler or PrimeTime, the `set_multicycle_path` command modifies the single-cycle timing relationship of a constrained path. It designates timing paths that have non-default setup or hold relations. Use `set_multicycle_path` to define timing paths that require more than one clock cycle to propagate.

By default, the coreTools check the validity of the `set_multicycle_path` command to ensure that the nodes specified in the `set_multicycle_path` command options exist in the current `design` and are of the correct type. However, because coreTools only model the structural part of a design, some timing exception start/end/through points may not exist in the coreTool model of the design. In such cases, you can use the `-quiet` option to suppress validity checking. If you use the `-quiet` option and you specify incorrect or invalid arguments to `set_multicycle_path`, an error will occur during synthesis.

For more information about using `set_multicycle_path`, refer to the Design Compiler and PrimeTime documentation for timing exceptions and the `set_multicycle_path` command.

Examples

To set all paths between `u6` and `u8` to 2 cycle paths for setup and measure hold time at the previous edge of the clock at `u8`:

```
coreConsultant> set_multicycle_path -from {u6} -to {u8} 2
```

To move the hold check to the preceding edge of the start clock:

```
coreConsultant> set_multicycle_path -from {u6} -to {u8} -1
```

The following example is a two-phase level-sensitive design, where the designer expects paths from `phi1` to `phi1` to be zero cycles:

```
coreConsultant> set_multicycle_path -from phi1 -to phi1 0
```

The following example uses `-start` to specify a 3-cycle path relative to the clock at the path start point. Clock sources are specified, affecting all sequential elements clocked by that clock, or ports with input or output delay relative to that clock.

```
coreConsultant> set_multicycle_path -start -from {clk50mhz}
-to {clk10mhz} 3
```

The following example affects all paths that pass first through cell `U1` or `U2` and later pass through cell `U3`. The path resulting in a rise transition at an end point is constrained to 2 cycles, but falling paths

to 1 cycle.

```
coreConsultant> set_multicycle_path -rise -through {U1, U2}  
-through {U3} 2
```

```
coreConsultant> set_multicycle_path -fall -through {U1, U2}  
-through {U3} 1
```

See Also

set_false_path (2), set_max_delay (2), set_min_delay (2), reset_path (2),
set_disable_timing (2)

set_parameter_attribute

Set the value an attribute on one or more parameters

Syntax

string **set_parameter_attribute** *list attr [value]*

string *list*

string *attr*

string *value*

Parameters

list	List of parameters on which you want to set the attribute value.
attr	The name of the attribute that you want to set. For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so that set_parameter_attribute does not interpret the subscript as a sub-command.
value	The value to which you want to set the attribute. For boolean attributes, <value> is optional (default = true).

Description

The set_parameter_attribute command sets the specified attribute to the specified value on the specified parameter(s). You can specify a single parameter, a list of parameters, or specify the parameter(s) as a Tcl collection.

For coreConsultant, use set_parameter_attribute to set the Value attribute on design parameters. Setting the Value attribute on a design parameter sets the design parameter to the specified value when you auto-complete the Specify Configuration activity. In coreConsultant, this is the method you use to set design parameters in command line mode.

For coreBuilder, you can use set_parameter_attribute to set attribute values on design parameters and on activity parameters that you create for your custom activities. You can also use set_parameter_attribute to set attributes on formula parameters.

To get the current value of a parameter attribute, use the get_parameter_attribute command. To set the value (Value attribute) of an activity or strategy parameter, use the set_activity_parameter or set_strategy_parameter command.

Examples

To set the value of the design parameter named `sync_mode` to 1:

```
coreConsultant> set_parameter_attribute sync_mode Value 1
```

To set the value of the `EnumValues` attribute on the `sync_mode` parameter to {0 1}:

```
coreBuilder> set_parameter_attribute sync_mode EnumValues {0 1}
```

To set the value of the `ValueRequired` attribute to true on the `Iterations` parameter of the custom activity named `RandomTestSim`:

```
coreBuilder> set iter [get_activity_parameter
```

```
-param RandomTestSim Iterations]
```

```
{Iterations}
```

```
coreBuilder> set_parameter_attribute $iter ValueRequired
```

See Also

[set_activity_parameter \(2\)](#), [set_strategy_parameter \(2\)](#),
[get_parameter_attribute \(2\)](#)

set_port_attribute

Set the value of an attribute on one or more ports

Syntax

string **set_port_attribute** *list attr [value]*

string *list*

string *attr*

string *value*

Parameters

- list* List of ports on which you want to set the attribute.
The name of the attribute that you want to set.
For subscripted attributes, enclose the attribute/subscript in curly braces ({Attribute[subscript]}) so that
- attr* set_port_attribute does not interpret the subscript as a sub-command.
- value* The value to which you want to set the attribute.
For boolean attributes, <value> is optional (default = true).

Description

The set_port_attribute command sets the specified attribute to the specified value on the specified port(s). You can specify a single port, a list of ports, or specify the ports as a Tcl collection. To get the current value of a port attribute, use the get_port_attribute command.

Examples

To set maximum input delay on the int2 pin to 15 percent of the clock period with respect to the clock named clk:

```
coreConsultant> set_port_attribute int2 {MaxInputDelay[clk]}  
{=percent_of_period 15.0}
```

To set the PinLoadCount attribute to 4 on all output ports:

```
coreConsultant> set_port_attribute [all_outputs] PinLoadCount 4
```

See Also

[get_port_attribute \(2\)](#)

set_prime_profile
Set a Prime Profile.

Syntax

```
string set_prime_profile [-tag <None> | <Prime Profile Tag>] [-file <file
name>] [-component <component name>] [-analysis]
    string <None> | <Prime Profile Tag>
    string <file name>
    string <component name>
```

Parameters

-tag <None> <Prime Profile Tag>	The Prime Profile tag from the packaged Prime Profiles to apply on the component.
-file <file name>	Use file instead of a tag.
-component <component name>	Master component name to set the Prime Profile on.
-analysis	Enable analysis mode for a Prime Profile.

Description

set_prime_profile is used to apply a Prime Profile to a design. In coreConsultant, when the SelectPrimeProfile activity is autocompleted by clicking the Apply button, this command is implicitly issued. Applying a Prime Profile is equivalent to sourcing the Prime Profile file. Analysis mode can also be enabled using this command.

Note

When the <None> option is available in SelectPrimeProfile activity, users are allowed to skip Prime Profile application. The set_prime_profile -tag <None> command is written out in the batch script to make this selection explicit.

Deprecated:

In coreConsultant and coreAssembler, if Prime Profiles with triggers are packaged for a component, the trigger parameter values must be set and set_prime_profile must be used to select one of those triggered Prime Profiles. In GUI mode, this command is implicitly executed using the Prime Profile selection GUI on setting trigger parameter if set_prime_profile is not already run. In shell mode, a Prime Profile must be set explicitly even if trigger parameters are set.

Examples

coreConsultant Usage

In the following example `set_prime_profile` command applies the Prime Profile `LockRamSize` on the component. If a profile is applied prior to autocompleting the `SelectPrimeProfile` activity, no additional process is carried out on completing the activity.

```
coreConsultant> set_prime_profile -tag LockRamSize
Information: Lock ram_size and ram_word to 32. In analysis allow word to
change. (CMDS-243)
Information: Wrote knowledge base config1 to
/path/to/config1/kb/config1.kb. (CDMIO-14)
Information: Saved modified workspace 'config1' to directory
'/path/to/config1'. (CMDS-224)
Warning: The 'LockRamSize' Prime Profile applied is an early access (EA)
profile and is not suitable for tape-out. (DES-139)
Information: Prime Profile set to 'LockRamSize' on the current design.
(DES-125)
coreConsultant> autocomplete_activity SelectPrimeProfile
Information: Wrote knowledge base config1 to
/path/to/config1/kb/config1.kb. (CDMIO-14)
Information: Saved modified workspace 'config1' to directory
'/path/to/config1'. (CMDS-224)
Information: Activity 'SelectPrimeProfile' completed. (CMDS-117)
```

coreAssembler Usage

`SelectPrimeProfile` activity is not available in `coreAssembler`. For components in a Subsystem, a Prime Profile can be applied using the `-component` option. The `set_prime_profile` command for a component in a Subsystem must be issued in the top level context. The argument with the `-component` option is used to set the right context before applying the Prime Profile.

```
coreAssembler> set_prime_profile -component /h1/m1 -tag DisableRom1
-analysis
coreAssembler> set_prime_profile -component /h2/m2 -tag DisableRom2
```

Applying External Prime Profiles (-file)

External Prime Profiles can be applied using the `-file` option as shown below. Packaged Prime Profile tags cannot be used as external Prime Profile tags.

```
coreConsultant> set_prime_profile -file ../source/CustomEA.pp.gz
Information: custom profile with early access enabled (CMDS-243)
Warning: The external Prime Profile you have chosen is an early access
(EA) profile and is not suitable for tape-out. - (GUI-4)
```

Information: Wrote knowledge base config1 to
 /path/to/config1/kb/config1.kb. (CDMIO-14)
 Information: Saved modified workspace 'config1' to directory
 '/path/to/config1'. (CMDS-224)
 Warning: The 'CustomEA' Prime Profile applied is an early access (EA)
 profile and is not suitable for tape-out. (DES-139)
 Information: Prime Profile set using custom file ../source/CustomEA.pp.gz
 on the current design. (DES-125)

Analysis Mode

Analysis mode gives more flexibility to users in configuring a component once a Prime Profile is applied. However, RTL generation and the activities that follow are not allowed to complete.

```
coreConsultant> set_prime_profile -tag Test003 -analysis
```

Information: Lock down all params to 16 (CMDS-243)
 Information: Wrote knowledge base config1 to
 path/to/config1/kb/config1.kb. (CDMIO-14)
 Information: Saved modified workspace 'config1' to directory
 '/path/to/cB_test/config1'. (CMDS-224)
 Information: Prime Profile set to 'Test003' (analysis mode) on the
 current design.
 Please send the batch script generated on autocomplete to Synopsys
 support. (DES-125)

```
coreConsultant> autocomplete_activity SelectPrimeProfile
```

Information: Wrote knowledge base config1 to
 /path/to/config1/kb/config1.kb. (CDMIO-14)
 Information: Saved modified workspace 'config1' to directory
 '/path/to/config1'. (CMDS-224)
 Information: Activity 'SelectPrimeProfile' completed. (CMDS-117)

```
coreConsultant> autocomplete_activity SpecifyConfiguration
```

Information: Wrote file
 /path/to/config1/export/prime_profile_analysis_script_Test003.tcl to
 disk. (FILE-15)
 Information: Prime Profile mode for design mycore set to 'analysis'. RTL
 will not be generated. (DES-124)
 Information: Activity 'SpecifyConfiguration' not completed. (CMDS-118)
 Error: Stopping activity SpecifyConfiguration from proceeding since Prime
 Profile analysis was set on one or more components.
 Please send the autogenerated batch script
 prime_profile_analysis_script_Test003.tcl to Synopsys support.
 Could not complete activity 'SpecifyConfiguration'
 Use error_info for more info. (CMD-013)

Triggering Prime Profiles (Deprecated)

Deprecated:

If Prime Profile, Test001, has trigger parameter include_rom and
 trigger value set to 1, then the Prime Profile must be triggered
 first and a Prime Profile must be selected from the list of
 triggered tags reported above.

coreTools Command Reference Index

```
coreConsultant> set_configuration_parameter include_rom 1
Information: Triggered prime profiles for current design are: Test001
Test002. (DES-126)
coreConsultant> set_prime_profile -tag Test001
Information: Long Descr 001 (CMD5-243)
Information: Prime profile set to 'Test001' on the current design.
(DES-125)
```

Note

The command `set_prime_profile` can be issued before a Profile is triggered, however, it will be applied only if it is triggered before configuration activity is completed.

See Also

[create_prime_profile_file \(2\)](#) [report_prime_profiles \(2\)](#)

[PrimeProfileCurrent \(3\)](#) [PrimeProfilesValid \(3\)](#)

set_prime_profile_validation_expr

Set a validation expression for Prime Profiles.

Syntax

```
string set_prime_profile_validation_expr -tag_list <Prime Profile names>
-expression <expression> [-message <msg string>]
    string <Prime Profile names>
    string <expression>
    string <msg string>
```

Parameters

-tag_list <Prime Profile names>	List of Prime Profile tags.
-expression <expression>	Expression to be validated during SpecifyConfiguration or SpecifySubsystem
-message <msg string>	Error message to be displayed when expression is true but a Prime Profile from the associated list is not applied.

Description

When configuring a component in coreConsultant or coreAssembler, expressions provided here are assessed during SpecifyConfiguration or SpecifySubsystem activities. If an expression evaluates to true, the tool proceeds to verify if a Prime Profile from the associated list of Prime Profiles has indeed been applied.

Note

All these checks are skipped if an external Prime Profile has been applied since validation expr checks only apply to packaged profiles.

Examples

The following list presents sample commands that are executed in coreBuilder during packaging, after all the create_prime_profile_file commands have been issued. Please note that, in coreConsultant or coreAssembler, each expression is validated separately, regardless of the others. It is the responsibility of the IP designers to ensure that applying a Prime Profile does not result in making expressions true for a set of Prime Profiles that does not include the applied Prime Profile.

Note

It is recommended to issue these commands after all Prime Profiles have been created so that designers can identify parameters and Prime Profiles to club together in a validation expression and avoid errors arising due to expressions of disjoint Prime Profile lists becoming true.

```
coreBuilder> set_prime_profile_validation_expr -tag_list "DisableRom1
DisableRom2" -expression "@dummy == 1" -message "DisableRom(1|2) profiles
must be applied when setting dummy to 1."
coreBuilder> set_prime_profile_validation_expr -tag_list "LockRamSize"
-expression "@dummy == 2"
coreBuilder> set_prime_profile_validation_expr -tag_list "Test001
Test003" -expression "@dummy == 3"
coreBuilder> set_prime_profile_validation_expr -tag_list "DisableRom2"
-expression "@ram_size == 32 &&@ram_word==64&& !@include_rom"
coreBuilder> set_prime_profile_validation_expr -tag_list "Test004"
-expression "@dummy == 4" -message "dummy config param 4 implies Test004
must be applied."
coreBuilder> set_prime_profile_validation_expr -tag_list "ProfileB"
-expression "@dummy == 5"
```

As an example, suppose that applying the `DisableRom1` Prime Profile sets the value of the `dummy` design parameter to 1. To ensure correctness, the validation procedure verifies that a Prime Profile from the `DisableRom1` `DisableRom2` group has been applied, based on the evaluation of the `@dummy==1` expression. It should be noted that applying the `DisableRom1` profile should not result the other expressions evaluating to true (e.g. those expressions where `DisableRom1` is not in the list of associated profile names).

Suppose in another case, the expression `@ram_size == 32 &&@ram_word==64&& !@include_rom` is true along with `@dummy == 1`, then 2 of the validation expressions become true. The only common profile in those 2 validation expressions is `DisableRom2`. If the Prime Profile `DisableRom2` was applied, no errors will be issued.

See Also

[set_prime_profile \(2\) create_prime_profile_file \(2\)](#)
[report_prime_profile_validation_expr \(2\)](#)
[clear_prime_profile_validation_exprs \(2\)](#)

[API Index](#) [Command Reference](#)

`set_register_array_attribute`

Set the value of an attributes on a register array.

Syntax

```
string set_register_array_attribute regArray attr [value]  
    string regArray  
    string attr  
    string value
```

Parameters

<code>regArray</code>	The register array to set the attribute on.
<code>attr</code>	Name of the attribute to set.
<code>value</code>	Value of the attribute to set.

Description

This command sets an attribute value on a registerArray.

Examples

The following command sets the AddressOffset of the register array 'regArray1' to be 0x10.

```
coreBuilder> set_register_array_attribute map1/block1/regArray1  
AddressOffset 0x10
```

See Also

[registerArray\(3\).get_register_array_attribute\(2\)](#)

[API Index](#) [Command Reference](#)

set_register_attribute

Set the value of an attribute on a register.

Syntax

```
string set_register_attribute reg attr [value]  
    string reg  
    string attr  
    string value
```

Parameters

<i>reg</i>	Name of the register to set attribute on.
<i>attr</i>	Name of the attribute to set.
<i>value</i>	Value of the attribute to set.

Description

This command sets an attribute value on a register.

Examples

The following command sets the AddressOffset of the register 'reg1' to be 0x10.

```
coreBuilder> set_register_attribute map1/block1/reg1 AddressOffset 0x10
```

See Also

[register \(3\).get_register_attribute \(2\)](#)

`set_register_field_attribute`

Set the value of an attribute on a register field.

Syntax

```
string set_register_field_attribute reg attr [value]  
    string reg  
    string attr  
    string value
```

Parameters

<code>reg</code>	Name of the register field to set attribute on.
<code>attr</code>	Name of the attribute to set.
<code>value</code>	Value of the attribute to set.

Description

This command sets an attribute on a register field.

Examples

The following command sets the BitAddressOffset attribute of the register field 'field1' of register 'reg1' to be 2:

```
coreBuilder> set_register_field_attribute map1/block1/reg1/field1  
BitAddressOffset 2
```

See Also

[registerField \(3\)](#), [get_register_field_attribute \(2\)](#)

[API Index](#) [Command Reference](#)

set_register_field_value_attribute

Set the value of an attribute on a register field value.

Syntax

```
string set_register_field_value_attribute val attr [value]  
    string val  
    string attr  
    string value
```

Parameters

<i>val</i>	Name of the register field value to set attribute on.
<i>attr</i>	Name of the attribute to set.
<i>value</i>	Value of the attribute to set.

Description

Set sets the value of an attribute on a register field value.

Examples

To change the bit value of a field value:

```
coreBuilder> set_register_field_value_attribute \  
map1/block1/reg1/field1/val1 Value 0x3
```

See Also

[create register field value \(2\)](#), [get register field value attribute \(2\)](#)

set_slave_base_address

Set the remap range and/or base address of the given slave interface.

Syntax

```
string set_slave_base_address [-component <component>] -interface
<interface> [-slotoffset <slot>] [-mode <mode>] [-range <range>]
[-address <address>]
    string <component>
    string <interface>
    int <slot>
    string <mode>
    string <range>
    string <address>
```

Parameters

-component <component>	The slave component or name.
-interface <interface>	The exported or slave instance or name.
-slotoffset <slot>	The slotoffset the remap object is associated with for multi-slot interfaces.
-mode <mode>	The mode to use when setting the base address.
-range <range>	Set the remap range attribute.
-address <address>	Set the remap address attribute.

Description

Sets the base memory address or range on the slave interface specified with the -component and -interface options. For an exported interface, the interface name is specified by -interface without the -component option.

Examples

Set the address on a slave interface:

```
coreAssembler> set_slave_base_address -component i_ahb -interface
AHB_Slave \
-mode Normal -address 0x80009000
```

Set the range for a multi-slot interface:

coreTools Command Reference Index

```
coreAssembler> set_slave_base_address -component i_icm -interface Layer1  
-slotoffset 1 \  
-range 0x1000
```

Set the range on an exported interface:

```
coreAssembler> set_slave_base_address -interface ex_i_xml_AHB_Slave1  
-range 0x2000
```

See Also

[get_slave_base_address\(2\)](#)

set_solution_preference
Set solution preference

Syntax

string **set_solution_preference** [-readonly]

Parameters

-readonly Make the DW Solution as ReadOnly

Description

The command can be used to update the preference of the DW Solution workspace after restructuring.

Examples

The command makes the DW Solution workspace readonly, the command needs to issued before 'autocomplete_activity SpecifySubsystem'. Once issued components can not grouped or ungrouped.

```
coreAssembler> set_solution_preference -readonly
```

See Also

set_strategy_parameter

Set the value of a parameter for a strategy

Syntax

```
string set_strategy_parameter strategyName paramName value
      string strategyName
      string paramName
      string value
```

Parameters

strategyName	The name of the strategy for which you want to set a parameter value.
paramName	The name of the parameter that you want to set.
value	The value you want to set the parameter to.

Description

The set_strategy_parameter command sets the specified parameter on the specified coreConsultant synthesis strategy to the specified value. Use set_strategy_parameter to set the parameters on a synthesis strategy before you execute the strategy.

You select which strategy to execute by setting the StrategyName parameter on the Synthesize activity. The synthesis strategies currently available and their associated parameters are:

- DC_opto_strategy - Parameters: QorEffort, UseDesignBudgets, NetlistFormat
- DC_incremental_opto_strategy - Parameters: QorEffort, UseDesignBudgets, ResumeAfter, NetlistFormat
- DC_design_budgeting_strategy - Parameters: QorEffort, ResumeAfter, NetlistFormat
- DC_quick_map_strategy - Parameters: StopAtGtech, NetlistFormat
- PT_model_extraction - Parameters: AuxiliaryScript, ContextBorrow, ModelFormat, ModelStage, OutputFileRoot, RemoveInternalArcs, ModelOperatingConditions, ModelAsLibCell, LatchLevels

For detailed descriptions of the above strategies and parameters, refer to the *coreConsultant User Guide*.

To get the current value of a synthesis strategy parameter, use the get_strategy_parameter command.

Examples

To set the value of the QorEffort parameter to high on the synthesis strategy named DC_opto_strategy:

```
coreConsultant> set_strategy_parameter DC_opto_strategy QorEffort high
```

See Also

[get_strategy_parameter \(2\)](#)

set_synthesis_dir

Set the name of the directory where synthesis will be performed.

Syntax

string **set_synthesis_dir** *directory_name*
string *directory_name*

Parameters

directory_name Name of the top level synthesis directory.

Description

This command can be used to set the name of the top level synthesis directory. By default, synthesis is run in the 'syn' directory. Using this command to set the synthesis directory allows alternate or multiple synthesis directories.

Examples

Specify that the top level synthesis directory will be named defaultConfig

```
coreConsultant> set_synthesis_dir defaultConfig
```

See Also

get_logical_dir (2), get_synthesis_dir (2), get_installation_dir (2) _____

set_tclconfig_variables

Set TCL variables for design parameters defined to have a TCL configuration variable.

Syntax

```
string set_tclconfig_variables [-array <array-name>] [-namespace
<namespace>] [-script] [context]
    string <array-name>
    string <namespace>
    string context
```

Parameters

-array <array-name>	Define variables within indicated array
-namespace <namespace>	Define variables within indicated namespace
-script	Return script instead of setting variables
context	Parameter context from which to locate parameters

Description

This command is used to translate parameter objects and their current values into TCL variables with the same value. Parameters to be considered must have the TCLConfigVariable attribute set to true (on the parameter). This can be done by explicitly setting the attribute, in which case the TCL variable name will be derived from the parameter name. If a different name is desired, the TCLConfigVariableName attribute should be set on the parameter instead. In that case the value of TCLConfigVariableName will be used to derive the TCL variable name instead of the parameter name.

If the 'context' argument is not specified, the command applies to all design parameters with TCLConfigVariable or TCLConfigVariableName set on them. This command can also be applied to activity parameters by passing the activity to the command via the context argument. The TCLConfig* attributes must be set in this case as well.

By default the command will create and set a TCL variable for each parameter with TCLConfigVariable == 1. Note that if TCLConfigVariableName is set, then TCLConfigVariable defaults to 1, so only one of the two attributes need be set. If TCLConfigVariableName is set, then a TCL variable with the indicated name will be created in the global namespace. If TCLConfigVariableName is unset, then the variable name will match the

parameter name, again in the global context.

The -array and -namespace options are available to provide more control over how the variables are defined. If -array is specified, then the indicated array variable is defined with an array index for each parameter. The indices used match the parameter name or TCLConfigVariableName value as above. If the -namespace option is specified, the TCL variables are defined within the indicated namespace instead of the global namespace. Both options can be used together to define an array variable within a namespace.

Examples

All of the examples assume the following parameter definitions:

```
// reuse-pragma attr TCLConfigVariable 1
'define PARAM1 4
// reuse-pragma attr TCLConfigVariableName P2
'define PARAM2 8
```

The examples below show as output the TCL commands to set the variables. The command does not actually output 'set' commands but rather executes a set of 'set' commands. They are shown as commands to indicate what is actually set.

A default invocation such as:

```
set_tclconfig_variables
will end up defining the following:
```

```
set ::PARAM1 4
set ::P2 8
```

To set the variables via an array named ConfigParams:

```
set_tclconfig_variables -array ::ConfigParams
will end up defining the following:
```

```
set ::ConfigParams(PARAM1) 4
set ::ConfigParams(P2) 8
```

To set the variables in an array within a namespace:

```
set_tclconfig_variables -array ::ConfigParams -namespace ::myns
will end up defining the following:
```

```
set ::myns::ConfigParams(PARAM1) 4
set ::myns::ConfigParams(P2) 8
```

The command will ensure that the namespace is defined prior to setting the variables.

Assuming that one of TCLConfigVariable or TCLConfigVariableName is set on one or more activity parameters of the activity MyCustomAct, TCL

variables can set from these parameters as follows:

```
set act [find_item -type activity MyCustomAct]
set_tclconfig_variables $act
```

See Also

[TCLConfigVariable \(3\)](#), [TCLConfigVariableName \(3\)](#) [TCLConfigVariableSetArgs \(3\)](#)

[API Index](#) [Command Reference](#)

set_template_naming_style

Set the template_naming_style value.

Syntax

```
string set_template_naming_style -value <value>  
      string <value>
```

Parameters

-value <value>	New Value
----------------	-----------

Description

Examples

See Also

[API Index](#) [Command Reference](#)

set_template_parameter_style

Set the template_parameter_style value.

Syntax

```
string set_template_parameter_style -value <value>  
      string <value>
```

Parameters

-value <value>	New Value
----------------	-----------

Description

Examples

See Also

[API Index](#) [Command Reference](#)

set_template_separator_style

Set the template_separator_style value.

Syntax

```
string set_template_separator_style -value <value>  
      string <value>
```

Parameters

-value <value>	New Value
----------------	-----------

Description

Examples

See Also

[API Index](#) [Command Reference](#)

set_tool_preference

Set the tool preference parameter

Syntax

```
string set_tool_preference -pref pref -value value [-gui]
    string pref
    string value
```

Parameters

-pref <i>pref</i>	Tool preference parameter
-value <i>value</i>	Value to be set
-gui	Update the preference in UI.

Description

Sets the tool preferences via a command. To update on UI the use needs to set a -gui switch

Examples

Set the tool preference for retain_static_port_range and enable in GUI edit preferences dialog

```
set_tool_preference -pref retain_static_port_range -value true -gui
the tool preference for retain_static_port_range . This wont update in UI
edit preference dialog
```

```
set_tool_preference -pref retain_static_port_range -value true
```

See Also

[get_tool_preference\(2\),report_rm_variable\(2\)](#)

set_tool_root

Set the root directory for the specified tool.

Syntax

```
string set_tool_root -root <directory> [-64bit] tool
      string <directory>
      string tool
```

Parameters

	Tool installation root directory.
-root <directory>	For example, for dc_shell the root directory is the one typically specified by the 'SYNOPSISYS' environment variable.
-64bit	Use 64bit version of tool instead of 32bit version
tool	Indicates which tool root to set (Values: dc_shell, pt_shell, fm_shell, tcm, fc_shell, tmax, dft_shell, vcs, vcsi, vera, vsim, ncsim, synplify, rtl_shell, protocompiler, vivado, leda, mvtools, spyglass, vcstatic, verdi)

Description

This command is used to indicate the root directory of the installation area of the specified tool. This information is used to enable invoking the specified tools if needed. The tools roots are determined automatically if possible from either well known environment variables (such as SYNOPSISYS) or from the user PATH variable.

When using the tools in graphical mode, the tool roots can be viewed and modified via the View->Specify Tool Roots menu option.

Examples

To specify the path to a dc_shell installation:

```
prompt> set_tool_root dc_shell -root /remote/release/2002.05
```

See Also

[get_tool_root \(2\)](#)

NAME

set_trace_option

Set an option value controlling behavior of command tracing and output annotation..

SYNTAX

```
set_trace_option [-command name
                  |-profile profile_metric_types
                  |-memory_threshold threshold
                  |-cpu_threshold threshold
                  |-time_treshhold threshold ]
                  [-annotate annotate_type]
```

string *name* string *annotate_type* list
 rofile_metric_types integer *threshold*

ARGUMENTS

-command *name* This option is mutually exclusive with **-profile**, **-memory_threshold**, **-cpu_threshold**, and **-time_threshold**. The option identifies the command for which annotation option is being set.

-annotate *annotate_type*
 This option is meaningful only when given with the **-command** option. Set the annotation type of the given command with this option. The allowed values are {*annotate*, *omit*}.

The *annotate* option value causes the traced command execution to be annotated to the shell output. If the application supports an output log, the annotation is also output to the output log.

The *omit* option value causes the command execution string to be omitted from output annotation.

-profile *profile_metric_types*

coreTools Command Reference Index

This option is mutually exclusive with all others. The option sets the currently enabled profile metrics, or "all" if all metrics are enabled. Allowed values are a list of: memory, cpu, time, or the value all to indicate enabling of all metrics, or the value none to indicate disabling of all metrics. If none is given in a list with any other value, it is ignored. The enabled profile metrics are reported with command annotations in the tool output to shell, and to the output log if supported by the tool.

`-memory_threshold` *threshold*

This option is mutually exclusive with all others. The option sets the threshold for gathering memory metrics. The profile data annotation will be output at the end of a command invocation only when the delta memory use for a command invocation matches or exceeds the threshold. The threshold has no effect on profile data output at the beginning of command invocation when this is enabled. The threshold is expressed in kilobytes. The special value -1 unsets the threshold.

`-cpu_threshold` *threshold*

This option is mutually exclusive with all others. The option sets the threshold for gathering CPU usage metrics. The profile data annotation will be output at the end of a command invocation only when the delta CPU usage for a command invocation matches or exceeds the threshold. The threshold has no effect on profile data output at the beginning of command invocation when this is enabled. The threshold is expressed in seconds. The special value -1 unsets the threshold. If cpu threshold is unset, then the default threshold of 1 second is used. To effect no threshold, set the threshold to 0 (zero).

`-time_threshold` *threshold*

This option is mutually exclusive with all others. The option sets the threshold for gathering elapsed time metrics. The profile data annotation will be output at the end of a command invocation only when the elapsed time for a command invocation matches or exceeds the threshold. The threshold has no effect on profile data output at

the beginning of command invocation when this is enabled. The threshold is expressed in seconds. The special value -1 unsets the threshold. If time threshold is unset, then the threshold for cpu is used.

DESCRIPTION

If a command is given with `-annotate` option, then the **set_trace_option** command informs the tool on how to behave during command annotation output, which is started with the **annotate_trace** command.

If `profile`, `memory_threshold`, `cpu_threshold`, or `time_threshold` is given, the the **set_trace_option** configures the behavior of performance profile gathering and the data reported in command annotations to the output.

EXAMPLES

The following example sets profile configurations, turns annotation off for the command *get_attribute*, then starts command annotations to the output.

```
prompt> set_trace_topion -profile all
prompt> set_trace_topion -memory_threshold 10
prompt> set_trace_topion -cpu_threshold 30
prompt> set_trace_topion -command get_attribute -annotate omit
prompt> annotate_trace -start -profile on
```

SEE ALSO

log_trace(2)
annotate_trace(2)
get_trace_option(2)

NAME

set_unix_variable

This is a synonym for the **setenv** command.

SEE ALSO

getenv(2), **printenv(2)**, **printvar(2)**, **set(2)**,
setenv(2), **sh(2)**, **unset(2)**.

set_unused_interface

set the Unused attribute for unused provider

Syntax

string **set_unused_interface** [-component <componentName>] -interface
<interfaceName> [value]

string <componentName>

string <interfaceName>

string value

Parameters

-component <componentName>	Component name for core interface instance(s)
-interface <interfaceName>	Name of the core interface instance or of the exported interface
value	value to be set

Description

This command sets the value of the attribute Unused for the specified interface instance. The verify_subsystem command will verify that each interface instance in the subsystem is either connected to other interface instance or the Unused attribute is set to true.

Note that setting the interface instance to be unused does not imply anything about the design ports associated with the interface. Now that the interface is unused, the corresponding design ports will need to be connected manually.

Examples

The following command sets the instance APB_Clock of component DW_apb to be unused:

```
coreAssembler> set_unused_interface -component DW_apb -interface  
APB_Clock
```

The following command resets the Unused attribute of the instance APB_Clock of component DW_apb :

```
coreAssembler> set_unused_interface -component DW_apb -interface  
APB_Clock 0
```

See Also

[unconnect_interface \(2\), connect_interface \(2\)](#)
[translate_interface_connection \(2\)](#)

set_upf_attribute

Set the value of an attribute on a list of UPF items

Syntax

```
string set_upf_attribute list attr value
      string list
      string attr
      string value
```

Parameters

list List of UPF items on which you want to set the attribute.

attr The name of the attribute that you want to set.

value The value to which you want to set the attribute.

Description

The set_upf_attribute command sets the specified attribute to the specified value on the specified UPF item(s). You can specify a single UPF item, a list of UPF items, or specify the UPF items as a Tcl collection. To get the current value of a UPF item attribute, use the get_upf_attribute command. For information on how to find a UPF object, see the get_upf_attribute man page.

Examples

To set UPFDomain attribute to DOMAIN_1 on items UPF_isolation_1 and UPF_retention_1

```
coreConsultant> set_upf_attribute {UPF_isolation_1 UPF_retention_1}
UPFDomain DOMAIN_1
```

To set UPFStrategy attributes on all isolation items

```
coreConsultant> set upfItems [find_item -type genHierItem
UPF_isolation_*]
coreConsultant> set_upf_attribute $upfItems UPFStrategy STRATEGY_1
```

See Also

[get_upf_attribute \(2\)](#)

set_upf_cells

User routine to specify technology cells for UPF file configuration.

Syntax

```
string set_upf_cells -type <cell type> [-domain <domain>] [-strategy
<strategy>] [-novalidation] cells
    string <cell type>
    string <domain>
    string <strategy>
    string cells
```

Parameters

-type <cell type>	Cell type (Values: retention, isolation, level_shift)
-domain <domain>	Power domain name.
-strategy <strategy>	Strategy name
-novalidation	Disable cell value validation
cells	Technology library cell names

Description

This command is used to specify technology cell names to be plugged into UPF files during UPF file configuration. Typically this is done in GUI mode by end users but this command enables the same in batch mode. This command cannot be issued until after the Technology Setup activity is completed so that the cell name references can be verified.

When the UPF file is packaged, it is possible to specify that all strategies and domains have the same cell set. In this specific instance, the pre-defined domain name "All domains" and the pre-defined strategy name "All strategies" can be used. However in this instance the values need not be specified at all.

Examples

Specify use of ISO1 cell for all isolation strategies in all power domains for a core packaged with a single set of UPF isolation cells:

```
set_upf_cells -type isolation -domain "All domains" -strategy "All
strategies" ISO1
```

Specify use of all cells beginning with RET for power domain TOP and strategy Primary:

set_upf_cells -type retention -domain TOP -strategy Primary RET*

See Also

set_upf_voltage (2)

[API Index](#) [Command Reference](#)

set_upf_supply_voltage

Set the voltage on a supply created from set_voltage.

Syntax

```
string set_upf_supply_voltage -supply -mode <min|max> value
      string
      string <min|max>
      string value
```

Parameters

-supply	The name of the supply
-mode <min max>	Voltage mode (Values: min, max)
value	Voltage value

Description

Examples

See Also

set_upf_voltage

User routine to specify voltage values for UPF file configuration.

Syntax

```

string set_upf_voltage [-type <power|ground>] [-mode <best|worst>]
[-state <state>] [-supply <supply name>] voltage
    string <power|ground>
    string <best|worst>
    string <state>
    string <supply name>
    string voltage

```

Parameters

-type <power ground>	Supply voltage type (Values: power, ground)
-mode <best worst>	Supply voltage mode (Values: best, worst)
-state <state>	Power state name.
-supply <supply name>	Supply set or supply port name
voltage	Voltage value(s)

Description

The set_upf_voltage command is used to define voltage values required to support configuration of UPF files. The -type, -mode, -state, and -supply arguments are used to define what the value values apply to. The -type and -mode arguments must be used together (and without -state and -supply) to indicate the best/worst case voltage for power or ground. In this case only a single voltage can be specified.

The -state and -supply arguments are used to locate the UPF object to which voltages need to be defined. These values correlate to values of the UPFState and UPFSupply attributes associated with the UPF objects (power states and port states). In this case 1, 2, or 3 voltage values can be specified. This corresponds to specifying the desired voltage, minimum and maximum voltages, or minimum, typical, and maximum voltages, respectively.

Examples

Set worst case power voltage:

```
set_upf_voltage -type power -mode worst 3.2
```

Set minimum and maximum voltages for the SINGLE_DOMAIN.primary power supply in the SINGLE_DOMAIN_ON power state.

```
set_upf_voltage -state SINGLE_DOMAIN_ON -supply SINGLE_DOMAIN.primary  
{0.81 0.91}
```

See Also

set_workspace_options

Set one or more workspace options, customizing the user flow.

Syntax

string **set_workspace_options** [-reports] [-noreports]

Parameters

-reports	Generate reports as activities complete
-noreports	Don't Generate reports as activities complete

Description

This command is used to control which activities and/or functionality are available in coreConsultant or coreAssembler for the given workspace. Options can be added or removed to tailor the tool to your specific needs or to improve performance. This command operates on the currently open workspace only. The default for each of the specified options is controlled in the "Workspace Options" tab of the Edit-->Preferences dialog.

Examples

Prevent generation of SPIRIT IP-XACT XML files.

```
set_workspace_options -nospirit
```

See Also

NAME

setenv Sets the value of a system environment variable.

SYNTAX

```
string setenv variable_name new_value
```

```
string variable_name  
string new_value
```

ARGUMENTS

variable_name Names of the system environment variable to set.

new_value Specifies the new value for the system environment variable.

DESCRIPTION

The **setenv** command sets the specified system environment *variable_name* to the *new_value* within the application. If the variable is not defined in the environment, the environment variable is created. The **setenv** command returns the new value of *variable_name*. To develop scripts that interact with the invoking shell, use **getenv** and **setenv**.

Environment variables are stored in the Tcl array variable **env**. The environment commands **getenv**, **setenv**, and **printenv** are convenience functions to interact with this array.

The **setenv** command sets the value of a variable only within the process of your current application. Child processes initiated from the application using the **exec** command after a usage of **setenv** inherit the new variable value. However, these new values are not exported to the parent process. Further, if you set an environment variable using the appropriate system

coreTools Command Reference Index

command in a shell you invoke using the **exec** command, that value is not reflected in the current application.

EXAMPLES

The following example changes the default printer.

```
shell> getenv PRINTER
laser1
shell> setenv PRINTER "laser3"
laser3
shell> getenv PRINTER
laser3
```

SEE ALSO

exec(2), **getenv(2)**, **unsetenv(2)**, **printenv(2)**,
printvar(2), **set(2)**, **sh(2)**, **unset(2)**.

NAME

sh Executes a command in a child process.

SYNTAX

string **sh** [*args*]

string *args*

ARGUMENTS

args Command and arguments that you want to execute in the child process.

DESCRIPTION

This is very similar to the **exec** command. However, file name expansion is performed on the arguments. Remember that quoting and grouping is in terms of Tcl. Arguments which contain spaces will need to be grouped with double quotes or curly braces. Tcl special characters which are being passed to system commands will need to be quoted and/or escaped for Tcl. See the examples below.

EXAMPLES

This example shows how you can remove files with a wildcard.

```
prompt> ls aaa*
aaa1      aaa2      aaa3
prompt> sh rm aaa*
prompt> ls aaa*
Error: aaa*: No such file or directory
       Use error_info for more info. (CMD-013)
```

coreTools Command Reference Index

This example shows how to grep some files for a regular expression which contains spaces and Tcl special characters:

```
prompt> exec cat test3.out
blah blah blah
blah blah blah c blah
input [1:0] A;
output [2:0] B;
prompt>
prompt> sh egrep -v {[ ]+c[ ]+} test3.out
blah blah blah
prompt>
prompt> sh egrep {t[ ]+\[} test3.out
input [1:0] A;
output [2:0] B;
```

SEE ALSO

exec(2).

NAME

sh_allow_tcl_with_set_app_var

Allows the **set_app_var** and **get_app_var** commands to work with application variables.

TYPE

string

DEFAULT

application specific

DESCRIPTION

Normally the **get_app_var** and **set_app_var** commands only work for variables that have been registered as application variables. Setting this variable to **true** allows these commands to set a Tcl global variable instead.

These commands issue a CMD-104 error message for the Tcl global variable, unless the variable name is included in the list specified by the **sh_allow_tcl_with_set_app_var_no_message_list** variable.

SEE ALSO

get_app_var(2)
set_app_var(2)
sh_allow_tcl_with_set_app_var_no_message_list(2)

NAME

sh_allow_tcl_with_set_app_var_no_message_list
Suppresses CMD-104 messages for
variables in this list.

TYPE

string

DEFAULT

application specific

DESCRIPTION

This variable is consulted before printing the CMD-104 error message, if the **sh_allow_tcl_with_set_app_var** variable is set to **true**. All variables in this Tcl list receive no message.

SEE ALSO

get_app_var(2)
set_app_var(2)
sh_allow_tcl_with_set_app_var(2)

NAME

sh_arch Indicates the system architecture of your machine.

TYPE

string

DEFAULT

platform-dependent

DESCRIPTION

The **sh_arch** variable is set by the application to indicate the system architecture of your machine. Examples of machines being used are sparcOS5, amd64, and so on. This variable is read-only.

NAME

`sh_command_abbrev_mode`

Sets the command abbreviation mode for interactive convenience.

TYPE

string

DEFAULT

application specific

DESCRIPTION

This variable sets the command abbreviation mode as an interactive convenience. Script files should not use any command or option abbreviation, because these files are then susceptible to command changes in subsequent versions of the application.

Although the default value is **Anywhere**, it is recommended that the site startup file for the application set this variable to **Command-Line-Only**. It is also possible to set the value to **None**, which disables abbreviations altogether.

To determine the current value of this variable, use the `get_app_var sh_command_abbrev_mode` command.

SEE ALSO

`sh_command_abbrev_options(3)`
`get_app_var(2)`
`set_app_var(2)`

NAME

sh_command_abbrev_options

Turns off abbreviation of command dash option names when false.

TYPE

boolean

DEFAULT

application specific

DESCRIPTION

When command abbreviation is currently off (see `sh_command_abbrev_mode`) then setting this variable to false will also not allow abbreviation of command dash options. This variable also impacts abbreviation of the values specified to command options that expect values to be one of an allowed list of values.

This variable exists to be backward compatible with previous tool releases which always allowed abbreviation of command dash options and option values regardless of the command abbreviation mode.

It is recommended to set the value of this variable to false.

To determine the current value of this variable, use the `get_app_var sh_command_abbrev_options` command.

SEE ALSO

`sh_command_abbrev_mode(3)`
`get_app_var(2)`

set_app_var(2)

NAME

`sh_command_log_file`

Specifies the name of the file to which the application logs the commands you executed during the session.

TYPE

string

DEFAULT

empty string

DESCRIPTION

This variable specifies the name of the file to which the application logs the commands you run during the session. By default, the variable is set to an empty string, indicating that the application's default command log file name is to be used. If a file named by the default command log file name cannot be opened (for example, if it has been set to read only access), then no logging occurs during the session.

This variable can be set at any time. If the value for the log file name is invalid, the variable is not set, and the current log file persists.

To determine the current value of this variable, use the `get_app_var sh_command_log_file` command.

SEE ALSO

`get_app_var(2)`
`set_app_var(2)`

NAME

sh_continue_on_error

Allows processing to continue when errors occur during script execution with the **source** command.

TYPE

Boolean

DEFAULT

application specific

DESCRIPTION

It is recommended to use the **-continue_on_error** option to the **source** command instead of this variable because that option only applies to a single script, and not the entire application session.

When set to **true**, the **sh_continue_on_error** variable allows processing to continue when errors occur. Under normal circumstances, when executing a script with the **source** command, Tcl errors (syntax and semantic) cause the execution of the script to terminate.

When **sh_continue_on_error** is set to **false**, script execution can also terminate due to new error and warning messages based on the value of the **sh_script_stop_severity** variable.

To determine the current value of the **sh_continue_on_error** variable, use the **get_app_var sh_continue_on_error** command.

SEE ALSO

`get_app_var(2)`
`set_app_var(2)`
`source(2)`
`sh_script_stop_severity(3)`

NAME

`sh_deprecated_is_error`

Raise a Tcl error when a deprecated command is executed.

TYPE

Boolean

DEFAULT

application specific

DESCRIPTION

When set this variable causes a Tcl error to be raised when an deprecated command is executed. Normally only a warning message is issued.

SEE ALSO

`get_app_var(2)`
`set_app_var(2)`

NAME

`sh_dev_null` Indicates the current null device.

TYPE

string

DEFAULT

platform dependent

DESCRIPTION

This variable is set by the application to indicate the current null device. For example, on UNIX machines, the variable is set to `/dev/null`. This variable is read-only.

SEE ALSO

`get_app_var(2)`

NAME

`sh_disabled_is_error`

Raise a Tcl error when a disabled command is executed.

TYPE

Boolean

DEFAULT

application specific

DESCRIPTION

When set this variable causes a Tcl error to be raised when a disabled command is executed. When false, only a warning message is issued.

Disabled commands have no effect.

SEE ALSO

`get_app_var(2)`
`set_app_var(2)`

NAME

sh_enable_page_mode

Displays long reports one page at a time (similar to the UNIX **more** command).

TYPE

Boolean

DEFAULT

application specific

DESCRIPTION

This variable displays long reports one page at a time (similar to the UNIX **more** command), when set to **true**. Consult the man pages for the commands that generate reports to see if they are affected by this variable.

To determine the current value of this variable, use the **get_app_var sh_enable_page_mode** command.

SEE ALSO

get_app_var(2)
set_app_var(2)

NAME

sh_enable_stdout_redirect

Allows the redirect command to capture output to the Tcl stdout channel.

TYPE

Boolean

DEFAULT

application specific

DESCRIPTION

When set to **true**, this variable allows the redirect command to capture output sent to the Tcl stdout channel. By default, the Tcl **puts** command sends its output to the stdout channel.

SEE ALSO

get_app_var(2)
set_app_var(2)

sh_executable_name

Full path name of executable for the current tool session.

Syntax

```
string sh_executable_name = "/vobs/apbld/bin-linux/rt_shell_exec-g"
```

Description

sh_executable_name is a read-only variable that indicates the full path name to the executable file for the current coreTool session. The executable name is the actual executable file and not a shell wrapper like "coreConsultant" or "coreBuilder -shell".

Examples

See Also

NAME

`sh_help_shows_group_overview`

Changes the behavior of the "help" command.

TYPE

string

DEFAULT

application specific

DESCRIPTION

This variable changes the behavior of the **help** command when no arguments are specified to help. Normally when no arguments are specified an informational message with a list of available command groups is displayed.

When this variable is set to false the command groups and the commands in each group is printed instead. This variable exists for backward compatibility.

SEE ALSO

`help(2)`
`set_app_var(2)`

NAME

sh_language Controls the tool language used by the application interactive console.

TYPE

string

DEFAULT

tcl

DESCRIPTION

This variable controls the language used by the application interactive console. Some applications support languages in addition to Tcl as an extension language.

To determine the current value of this variable, use the **get_app_var sh_language** command.

SEE ALSO

get_app_var(2)
set_app_var(2)

NAME

sh_new_variable_message

Controls a debugging feature for tracing the creation of new variables.

TYPE

Boolean

DEFAULT

application specific

DESCRIPTION

The **sh_new_variable_message** variable controls a debugging feature for tracing the creation of new variables. Its primary debugging purpose is to catch the misspelling of an application-owned global variable. When set to **true**, an informational message (CMD-041) is displayed when a variable is defined for the first time at the command line. When set to **false**, no message is displayed.

Note that this debugging feature is superseded by the new **set_app_var** command. This command allows setting only application-owned variables. See the **set_app_var** command man page for details.

Other variables, in combination with **sh_new_variable_message**, enable tracing of new variables in scripts and Tcl procedures.

Warning: This feature has a significant negative impact on CPU performance when used with scripts and Tcl procedures. This feature should be used only when developing scripts or in interactive use. When you turn on the feature for scripts or Tcl procedures, the application issues a message (CMD-042) to warn you about the use of this feature.

To determine the current value of this variable, use

the `get_app_var` `sh_new_variable_message` command.

SEE ALSO

`get_app_var(2)`
`set_app_var(2)`
`sh_new_variable_message_in_proc(3)`
`sh_new_variable_message_in_script(3)`

NAME

sh_new_variable_message_in_proc

Controls a debugging feature for tracing the creation of new variables in a Tcl procedure.

TYPE

Boolean

DEFAULT

false

DESCRIPTION

The **sh_new_variable_message_in_proc** variable controls a debugging feature for tracing the creation of new variables in a Tcl procedure. Its primary debugging purpose is to catch the misspelling of an application-owned global variable.

Note that this debugging feature is superseded by the new **set_app_var** command. This command allows setting only application-owned variables. Please see the **set_app_var** command man page for details.

Note that the **sh_new_variable_message** variable must be set to **true** for this variable to have any effect. Both variables must be set to **true** for the feature to be enabled. Enabling the feature simply enables the **print_proc_new_vars** command. In order to trace the creation of variables in a procedure, this command must be inserted into the procedure, typically as the last statement. When all of these steps have been taken, an informational message (CMD-041) is generated for new variables defined within the procedure, up to the point that the **print_proc_new_vars** commands is executed.

Warning: This feature has a significant negative impact on CPU performance. This should be used only when

coreTools Command Reference Index

developing scripts or in interactive use. When you turn on the feature, the application issues a message (CMD-042) to warn you about the use of this feature.

To determine the current value of this variable, use the **get_app_var sh_new_variable_message_in_proc** command.

SEE ALSO

```
get_app_var(2)
print_proc_new_vars(2)
set_app_var(2)
sh_new_variable_message(3)
sh_new_variable_message_in_script(3)
```


NAME

sh_new_variable_message_in_script

Controls a debugging feature for tracing the creation of new variables within a sourced script.

TYPE

Boolean

DEFAULT

false

DESCRIPTION

The **sh_new_variable_message_in_script** variable controls a debugging feature for tracing the creation of new variables within a sourced script. Its primary debugging purpose is to catch the misspelling of an application-owned global variable.

Note that this debugging feature is superseded by the new **set_app_var** command. This command allows setting only application-owned variables. See the **set_app_var** command man page for details.

Note that the **sh_new_variable_message** variable must be set to **true** for this variable to have any effect. Both variables must be set to **true** for the feature to be enabled. In that case, an informational message (CMD-041) is displayed when a variable is defined for the first time. When **sh_new_variable_message_in_script** is set to **false** (the default), no message is displayed at the time that the variable is created. When the **source** command completes, however, you see messages for any new variables that were created in the script. This is because the state of the variables is sampled before and after the **source** command. It is not because of inter-command sampling within the script. So, this is actually a more efficient method to see if new variables were created in the script.

coreTools Command Reference Index

For example, given the following script `a.tcl`:

```
echo "Entering script"
set a 23
echo a = $a
set b 24
echo b = $b
echo "Exiting script"
```

When `sh_new_variable_message_in_script` is **false** (the default), you see the following when you source the script:

```
prompt> source a.tcl
Entering script
a = 23
b = 24
Exiting script
Information: Defining new variable 'a'. (CMD-041)
Information: Defining new variable 'b'. (CMD-041)
prompt>
```

Alternatively, when `sh_new_variable_message_in_script` is **true**, at much greater cost, you see the following when you source the script:

```
prompt> set sh_new_variable_message_in_script true
Warning: Enabled new variable message tracing -
        Tcl scripting optimization disabled. (CMD-042)
true
prompt> source a.tcl
Entering script
Information: Defining new variable 'a'. (CMD-041)
a = 23
Information: Defining new variable 'b'. (CMD-041)
b = 24
Exiting script
prompt>
```

Warning: This feature has a significant negative impact on CPU performance. This should be used only when developing scripts or in interactive use. When you turn on the feature, the application issues a message (CMD-042) to warn you about the use of this feature.

To determine the current value of this variable, use the `get_app_var sh_new_variable_message_in_script` command.

SEE ALSO

```
get_app_var(2)
set_app_var(2)
sh_new_variable_message(3)
sh_new_variable_message_in_proc(3)
```


NAME

sh_obsolete_is_error

Raise a Tcl error when an obsolete command is executed.

TYPE

Boolean

DEFAULT

application specific

DESCRIPTION

When set this variable causes a Tcl error to be raised when an obsolete command is executed. Normally only a warning message is issued.

Obsolete commands have no effect.

SEE ALSO

`get_app_var(2)`
`set_app_var(2)`

[API Index](#) [Command Reference](#)

sh_product_version

Tracks the value of the like-named pt_shell variable

Definition

Type: string

Flags:

Default value:

Valid on:

Description

Examples

See Also

NAME

sh_script_stop_severity

Indicates the error message severity level that would cause a script to stop running before it completes.

TYPE

string

DEFAULT

None

DESCRIPTION

When a script is run with the **source** command, there are several ways to get it to stop running before it completes. One is to use the **sh_script_stop_severity** variable. This variable can be set to **none**, **W**, or **E**.

- o When set to **E**, the generation of one or more error messages by a command causes a script to stop.
- o When set to **W**, the generation of one or more warning or error messages causes a script to stop.
- o When set to **none**, the generation messages does not cause the script to stop.

Note that **sh_script_stop_severity** is ignored if **sh_continue_on_error** is set to **true**.

To determine the current value of this variable, use the **get_app_var sh_script_stop_severity** command.

SEE ALSO

`get_app_var(2)`
`set_app_var(2)`
`source(2)`
`sh_continue_on_error(3)`

sh_show_tcl_stack_on_error

Print or do not print the TCL stack when a TCL error occurs.

Syntax

```
string sh_show_tcl_stack_on_error = "0"
```

Description

The sh_show_tcl_stack_on_error variable determines if the coreTool prints the Tcl stack to the console when a Tcl error occurs. Note that the stack is not printed if the error occurs from within the 'source' command. In that case error_info should be used to print the stack trace after the error occurs.

By default, sh_show_tcl_stack_on_error is set to 0 (do not print Tcl stack). To print the Tcl stack on error for diagnostic purposes, set sh_show_tcl_stack_on_error to a non-zero value.

Examples

See Also

NAME

sh_source_emits_line_numbers

Indicates the error message severity level that causes an informational message to be issued, listing the script name and line number where that message occurred.

TYPE

string

DEFAULT

application specific

DESCRIPTION

When a script is executed with the **source** command, error and warning messages can be emitted from any command within the script. Using the **sh_source_emits_line_numbers** variable, you can help isolate where errors and warnings are occurring.

This variable can be set to **none**, **W**, or **E**.

- o When set to **E**, the generation of one or more error messages by a command causes a CMD-082 informational message to be issued when the command completes, giving the name of the script and the line number of the command.
- o When set to **W**, the generation of one or more warning or error messages causes a the CMD-082 message.

The setting of **sh_script_stop_severity** affects the output of the CMD-082 message. If the setting of **sh_script_stop_severity** causes a CMD-081 message, then it takes precedence over CMD-082.

To determine the current value of this variable, use the **get_app_var sh_source_emits_line_numbers** command.

SEE ALSO

- get_app_var(2)
- set_app_var(2)
- source(2)
- sh_continue_on_error(3)
- sh_script_stop_severity(3)
- CMD-081(n)
- CMD-082(n)

NAME

sh_source_logging

Indicates if individual commands from a sourced script should be logged to the command log file.

TYPE

Boolean

DEFAULT

application specific

DESCRIPTION

When you source a script, the **source** command is echoed to the command log file. By default, each command in the script is logged to the command log file as a comment. You can disable this logging by setting **sh_source_logging** to **false**.

To determine the current value of this variable, use the **get_app_var sh_source_logging** command.

SEE ALSO

`get_app_var(2)`
`set_app_var(2)`
`source(2)`

NAME

sh_source_uses_search_path

Indicates if the **source** command uses the **search_path** variable to search for files.

TYPE

Boolean

DEFAULT

application specific

DESCRIPTION

When this variable is set to true the **source** command uses the **search_path** variable to search for files. When set to **false**, the **source** command considers its file argument literally.

To determine the current value of this variable, use the **get_app_var sh_source_uses_search_path** command.

SEE ALSO

get_app_var(2)
set_app_var(2)
source(2)
search_path(3)

NAME

sh_tcllib_app_dirname

Indicates the name of a directory where application-specific Tcl files are found.

TYPE

string

DESCRIPTION

The **sh_tcllib_app_dirname** variable is set by the application to indicate the directory where application-specific Tcl files and packages are found. This is a read-only variable.

SEE ALSO

`get_app_var(2)`

NAME

sh_user_man_path

Indicates a directory root where you can store man pages for display with the **man** command.

TYPE

list

DEFAULT

empty list

DESCRIPTION

The **sh_user_man_path** variable is used to indicate a directory root where you can store man pages for display with the **man** command. The directory structure must start with a directory named *man*. Below *man* are directories named *cat1*, *cat2*, *cat3*, and so on. The **man** command will look in these directories for files named *file.1*, *file.2*, and *file.3*, respectively. These are pre-formatted files. It is up to you to format the files. The **man** command effectively just types the file.

These man pages could be for your Tcl procedures. The combination of defining help for your Tcl procedures with the **define_proc_attributes** command, and keeping a manual page for the same procedures allows you to fully document your application extensions.

The **man** command will look in **sh_user_man_path** after first looking in application-defined paths. The user-defined paths are consulted only if no matches are found in the application-defined paths.

To determine the current value of this variable, use the **get_app_var sh_user_man_path** command.

SEE ALSO

define_proc_attributes(2)
get_app_var(2)
man(2)
set_app_var(2)

show_color_dialog

Displays a color browser dialog

Syntax

```
string show_color_dialog [-initColor <initial color>]  
    string <initial color>
```

Parameters

-initColor <initial color>

Initial color in the color dialog

Description

This command displays a color selection dialog allowing the user to determine the correct color visually. It returns an RGB color number in the format of #FFFFFF.

Examples

```
set mycolor [show_color_dialog -initColor #FF0000]
```

See Also

[API Index](#) [Command Reference](#)

show_exit_dialog

Displays a color browser dialog

Syntax

string **show_exit_dialog**

Description

Examples

See Also

[API Index](#) [Command Reference](#)

show_prime_profile_prompt

Show/Don't show Prime Profile selection prompt in consultant and assembler.

Syntax

```
string show_prime_profile_prompt val  
      string val
```

Parameters

val 1/0 imply show/don't show Prime Profile selection prompt.

Description

When a prime profile trigger parameter is set to a trigger value, a GUI dialog may pop up for prime profile selection by the user.

show_prime_profile_prompt 0/1 can disable or enable that GUI.

Examples

```
coreConsultant> show_prime_profile_prompt 0 ; # Disable prompt  
coreConsultant> show_prime_profile_prompt 1 ; # Enable prompt
```

See Also

[set_prime_profile \(2\)](#) [set_configuration_parameter \(2\)](#)

`show_spreadsheet_for_connections`

Generate and show a spreadsheet for making connections to interfaces with the type of the given interface.

Syntax

```
string show_spreadsheet_for_connections -text <text> -columns <column
names> interface
    string <text>
    string <column names>
    string interface
```

Parameters

<code>-text <text></code>	Text to show above connection spreadsheet.
<code>-columns <column names></code>	List containing names for spreadsheet columns.
<code>interface</code>	Interface used to determine which connections to show.

Description

This command is used to show a custom change connections dialog. The dialog contains a spreadsheet that allows connections between all the providers and consumers of a given interface type. This layout is especially nice when connecting up interrupt controllers.

The `-columns` option accepts a two element list. The list should contain two entries. The first entry is to label the consumers of that interface. The second column labels all the available providers of that interface.

Examples

```
set_interface_attribute Source-Interrupt \
ConnectionDialogCmd \
{show_spreadsheet_for_connections \
-text "Connect to the interrupt controller(s)." \
-columns {"IRQ pin" "Interrupt Source"} \
Source-Interrupt}
```

See Also

[ConnectionDialogCmd \(3\)](#)

show_url
Display URL internally

Syntax

```
string show_url url  
      string url
```

Parameters

url URL to display

Description

The show_url command invokes the tool's web browser (if not already invoked) to display the contents of the specified URL. If the internal browser cannot support the requested URL and external browser will be launched.

The default external web browser for coreTools can be controlled by using the application preferences dialog and setting the desired browser command string. By default it chooses the first browser found in your path searching first for firefox, then mozilla, then konqueror, and finally netscape.

Examples

To invoke the browser and go to the Synopsys web site:

```
coreConsultant> show_url "http://www.synopsys.com"
```

To invoke Netscape to view the databook for a core in the coreKit installation doc directory:

```
coreConsultant> show_url "[get_installation_dir]/doc/MyCoreDatabook.pdf"
```

See Also

[show_url_external \(2\)](#)

show_url_external
Display URL externally

Syntax

string **show_url_external** *url*
 string *url*

Parameters

url URL to display

Description

The default external web browser for coreTools can be controlled by using the application preferences dialog and setting the desired browser command string. By default it chooses the first browser found in your path searching first for firefox, then mozilla, then konqueror, and finally netscape.

The "%s" in the command string will be substituted with the appropriate URL that you are trying to display and is required to be there.

If you would like to add additional arguments into the command or specify a different browser just enter the additional text into the edit field being sure to include the "%s" what the URL is supposed go.

Example Command String: firefox openURL(s, new-tab)

Examples

To invoke the external browser and go to the Synopsys web site:

```
coreConsultant> show_url_external "http://www.synopsys.com"
```

also See also

[show_url\(2\)](#)

show_user_parameter_dialog

Show the user parameter dialog to build and return the values of a set of user defined parameters

Syntax

```
string show_user_parameter_dialog -title <Dialog title> -description
<Description> -parameters <parameter-name_type_list>
    string <Dialog title>
    string <Description>
    list <parameter-name_type_list>
```

Parameters

-title <Dialog title>	Title of the dialog
-description <Description>	Description of the dialog box
-parameters <parameter-name_type_list>	A list of lists. Each list (in the top level list) has information about one parameter and has atleast 2 elements. First element in the list is the parameter name and second element in the list is the type. Parameter type can be one of integer, bitfield, float, string, boolean. Other arguments are interpreted as Name Value pairs for attributes on the parameter

Description

Show a GUI dialog to input values for a set of parameters from the user. This command allows the creation of a GUI dialog for a set of parameters and returns the result of the users inputs for all of the parameters.

The command returns a TCL list whose elements are based on the users inputs to the GUI dialog. If the user clicks Cancel, a TCL list with two elements, namely {ButtonPressed Cancel} is returned.

If the user clicks OK, a TCL list with ButtonPressed Okay and a name-value pair for each user parameter is returned. The user is allowed to click OK only if all the values to the parameters are legal. If the invocation of show_user_parameter_dialog has 2 parameters whose names are ParamName1 and ParamName2, then, a TCL list with 6 elements is returned as shown below (if the user clicked OK). {ButtonPressed Okay ParamName1 Value1 ParamName2 Value2} See examples for more details.

The arguments to the `-parameter` switch is a list of lists. Each list that is an element in the top level list contains information about one parameter. Each parameter-list contains atleast two elements. The first two elements are the name and the type of the parameter. The remaining elements are attribute-value pairs that the user would like to set on the parameter. The parameter list has to contain an even number of parameters. If there were errors while creating the parameters or setting the attributes, the GUI dialog is not created and an error message is displayed. Some of the attributes that can be set on a parameter are: Label, Enabled, MinValue, MaxValue, EnumValues and Description. Each of these attributes have man pages with detailed pertinent information.

The arguments to `-title` and `-description` are TCL strings.

The legal parameter types are integer, bitfield, float, string and boolean. Except the boolean parameter type, all the other types result in a text-entry widget being created in the dialog. The boolean parameter type results in a checkbox in the dialog.

In batch mode (i.e non-graphical mode), the `show_user_parameter_dialog` command simulates clicking the Cancel button, i.e it returns the same value that is returned when the Cancel button is clicked in GUI mode.

Examples

`show_user_parameter_dialog -description "Some Desc" -title "a string" -parameters [list [list name_of_int integer] [list yes_or_no boolean]]`
 The above invocation creates a GUI dialog for the 2 parameters passed. The dialog will have a text entry widget for the integer field and checkbox for the boolean checkbox. Lets say the user clicked OK after entering 1671 for the integer textbox and checked the checkbox for the boolean parameter, then, the following TCL list will be returned.

`ButtonPressed Okay name_of_int 1671 yes_or_no 1`

If the user unchecked the checkbox and entered 1671 for the integer parameter `name_of_int` and clicked OK, the following value is returned. The `"{}"` indicates that the value for the parameter `yes_or_no` is false.

`ButtonPressed Okay name_of_int 1671 yes_or_no {}`

If the user clicked Cancel, the following value is returned

`ButtonPressed Cancel`

The following example illustrates the use of additional attributes for each parameter.

`show_user_parameter_dialog -description "Some Desc" -title "a string" -parameters [list [list int_6_9 integer MaxValue 9 MinValue 6]]`

The above invocation creates a GUI dialog for the integer parameter `int_6_9`. The attributes `MaxValue` and `MinValue` restrict the set of values that can be assigned to the `int_6_9` parameter. The user is not allowed to

enter a value that is lower than 6 or greater than 9.

The following example illustrates the use of parameter type bitfield.

```
show_user_parameter_dialog -description "Some Desc" -title "a string"  
-parameters [list [list large_hex_number bitfield]]
```

The above invocation creates a GUI dialog for the bitfield parameter `large_hex_number`. A valid input is a hex number in the format `0x-`. Where the dash `-` indicates one or more hex digits. If the user entered `0x1a4f` to the above command and clicked OK, the following value would be returned:

ButtonPressed Okay large_hex_number 0x1a4f

See Also

Label (3), Enabled (3), MinValue (3), MaxValue (3), EnumValues (3),
Description (3), set_parameter_attribute (2), get_parameter_attribute (2)

[API Index](#) [Command Reference](#)`sizeof_collection`

Get the number of objects in collection

Syntax

```
string sizeof_collection collection1
      string collection1
```

Parameters

<code>collection1</code>	Elements in this collection
--------------------------	-----------------------------

Description

The `sizeof_collection` command is an efficient mechanism for determining the number of objects in a collection.

The `sizeof_collection` command considers the empty string as a valid collection name, returning 0 for the size when "" is passed in as the collection name.

Examples

This example shows a simple way to find out how many objects matched a particular pattern in a `find_item` command:

```
dwc_shell> echo "Number of designs: \  
[sizeof_collection [find_item -type design]  
Number of designs: 10
```

See Also

[add_to_collection\(2\)](#), [index_collection\(2\)](#), [remove_from_collection\(2\)](#).

PYTHON

NAME

snps.cmd Python object that provides access to application commands.

DESCRIPTION

Synopsys applications are built around a series of commands. These commands are exposed into the Tcl interpreter and are also available (accessible) from within the Python interpreter as well. In Python these application commands are accessed as methods on the `snps.cmd` object).

Most application commands accept one or more arguments. The arguments can be either positional or keyword arguments. For example the following command calls the "get_cells" command to find all soft macros with names matching the glob pattern '*mem*' using Tcl syntax:

```
get_cells *mem* -hierarchical -filter is_soft_macro
```

Using Python syntax, this becomes:

```
cmd.get_cells('*mem*', hierarchical=True, filter='is_soft_macro')
```

The arguments to the various commands are described in the man page for each command. Those may be accessed via the `man` command. For example:

```
cmd.man('get_attribute')
```

Additionally, Python specific help for a command can be displayed via the Python builtin help facility. This help displays the Python calling syntax. The man pages use Tcl syntax. For example:

```
prompt-py> help(cmd.get_attribute)
get_attribute(...)    Get the value of an attribute
Usage:
    get_attribute(object_spec, attr_name, _class=class_name, quiet=False,
```

coreTools Command Reference Index

```
value_list=False, objects=object_spec, name=attr_name)
```

Positional arguments (brackets indicate optional arguments):

object_spec:	Object(s) from which to get the attribute
attr_name:	Name of the attribute

Keyword arguments (brackets indicate optional arguments):

[_class]:	If object_spec is a name, this is its class
[quiet]:	Do not report any messages
[value_list]:	Force single object return as list
objects:	Object(s) from which to get the attribute
name:	Name of the attribute

Understanding Manpages

The application man pages for commands describe the Tcl syntax for a command. The Tcl "dash" options map directly to Python keyword arguments. If the dash argument name refers to a Python keyword, then the keyword in Python uses a leading "_" as the above example shows for the "-class" option.

In Tcl, boolean arguments usually accept no value, but in Python all keyword arguments require a value. For boolean argument types you may pass the values: True, 1, False, 0.

Repeating Keyword Arguments

Some of our applications command allow for dash (keyword) arguments to be repeated. This is most common in the timing path related commands. Python does not allow repeating the name of a keyword option. In this case you need to use the "cmd.arg" class along with the Python keyword expansion syntax (**) to pass the repeated argument names into the command. For example:

```
cmd.get_timing_path(**cmd.arg('through', a, 'through', b))
```

Is equivalent to calling the Tcl command:

```
get_timing_path -through a -through b
```

The "**cmd.arg" syntax can be interspersed with regular keyword argument syntax. Both of these styles are allowed and are equivalent to the first example:

```
cmd.get_timing_path(through=a, **cmd.arg('through', b))  
cmd.get_timing_path(**cmd.arg('through', a), through=b)
```

Command Return Values

All commands return a value. The type of the value is one of the following.

<i>Int or Float</i>	Commands returning numeric values return types in one of these Python builtin types.
---------------------	--

coreTools Command Reference Index

snps.collection

Commands that return database objects (Tcl collections).

snps.value All other return types.

Communicating with Tcl

The `snps.cmd` object can be used to call any command that is registered with the Tcl environment, including Tcl defined procedures (`proc`). For example:

cmd.set('varName', value)

Sets the Tcl variable 'varName' to the specified value.

cmd.set('varName')

Retrieves the value of the Tcl variable 'varName'.

cmd.source(fileName)

Sources the fileName into the Tcl interpreter.

cmd.eval(commandScript)

Evaluates the supplied Tcl script.

SEE ALSO

`py_eval(2)`
`snps.collection(2)`
`snps.value(2)`

PYTHON

NAME

snps.collection

Python class for Synopsys collection objects.

DESCRIPTION

Synopsys applications build an internal database of objects and attributes available on those objects. Some examples for class of database objects are designs, libraries, ports, cells, nets, pins, clocks, and so on. Most commands operate on these objects.

Accessing Elements

Collections represent an ordered sequence of database objects. Collection values support the Python sequence interface. The contents of a collection may be accessed like the standard Python list type. For example (x is a collection value):

`x[0]` Retrieve the first item. Returns a single item collection.

`x[-1]` Retrieve the last item. All slice based indexing is supported.

`len(x)` Return the number of objects in the collection

`for y in x:` Loop over the items. In the loop body "y" is a single item collection.

Building Collections

Collections can be created combining one or more collections. Both operator and function based methods are supported. For example:

`x += y # x.append(y)`

coreTools Command Reference Index

Add the items in the collection y into the collection referenced by x.

`x + y # x.add(y)`
Return a new collection by combining the objects in x and y.

`x - y # x.remove(y)`
Return a new collection containing the objects in x that are not also in y. (x -= y is also supported).

`x & y # x.remove(y, intersect=True)`
Return a new collection that contains the objects that exists in both x and y. (x &= y is also supported).

Sorting and Filtering

Collections can be sorted and filtered. For example:

`x.remove_duplicates()`
Returns a new collection with duplicate objects removed.

`x.sort(criteria)`
Returns a new collection sorted by the sort criteria. This is shorthand for "cmd.sort_collection(x, criteria)"

`x.filter(expr)` Returns a new collection by evaluating the filter expression. This is shorthand for "cmd.filter_collection(x, criteria)"

Attribute Values

Collections also provide methods to query and set attribute values on objects. Usually these are used with single item collections (see attribute iterators below for optimized access to attributes on larger collections):

`x.set(attr, value)`
Calls cmd.set_attribute(x, attr, value)

`x.get(attr)` Calls cmd.get_attribute(x, attr)

`x.unset(attr)` Calls cmd.remove_attribute(x, attr)

Attribute Iterators

As mentioned above collections may be indexed via an integer or slice value. This gives access to the individual items of a collection. Additionally

coreTools Command Reference Index

collections may be indexed by a string or a tuple of strings. Indexing in this way provides access to the attributes of the collection via an iterator. For example:

```
for name, cost in x['full_name', 'cost']:
```

Iterate over the values of the full_name and cost attributes in the collection. In the body of the loop 'name' refers to the value of the full_name attribute for the current item.

```
for obj, (name, cost) in zip(x, x['full_name', 'cost']):
```

Similar to the above loop, but loop over the collection contents in addition to the attribute values.

```
x['full_name', 'cost'].to_pandas()
```

Build a Pandas DataFrame object. The DataFrame contains a column for each attribute specified, with a row for each item in the source collection. See the Pandas documentation for details about the DataFrame type. If the collection is empty, then None is returned.

```
x['cost'].to_array()
```

If a single attribute name is used, then the a NumPy array can be requested. If the collection is empty, then None is returned.

DataFrame Contents

A Pandas DataFrame is a grouping of multiple NumPy (np) arrays (each column is an array). When attribute values are converted into these arrays, the type of underlying array differs depending on the values in the attribute. Each attribute type converts like this:

<i>Boolean</i>	The array is of type np.dtype('bool'). If an attribute is unset on an object the value is 'False'.
----------------	--

<i>Int</i>	The array is of type np.dtype('int32') unless an attribute is unset on an object. In that case the type is np.dtype('float64'). Items with the unset attribute have value 'NaN'.
------------	--

<i>Double</i>	The array is of type np.dtype('float64'). If an attribute is unset on an object, the value is 'NaN'.
---------------	--

coreTools Command Reference Index

<i>Float</i>	The array is of type <code>np.dtype('float32')</code> . If an attribute is unset on an object, the value is <code>'NaN'</code> .
<i>String</i>	If the attribute values are less than 32 characters and no spaces exist in the in any attribute value (i.e. cannot be interpreted as a Tcl list), then the type is <code>nd.dtype('<UN')</code> where N is the length of the longest attribute result. Otherwise the type is <code>nd.dtype('O')</code> (i.e. Object storage) and each entry has the type <code>snps.value</code> . Any unset attribute values are the empty string.
<i>Collection</i>	The array is of type <code>nd.dtype('O')</code> and each entry is of type <code>snps.collection</code> . Unset attributes use the empty collection.

SEE ALSO

`py_eval(2)`
`collections(2)`
`filter_collection(2)`
`snps.value(2)`

PYTHON

NAME

snps.redirect Python class used to redirect application output.

CONSTRUCTOR SYNTAX

```
snps.redirect(file=fileName, compress=False, tee=False)
snps.redirect(stream=ioStream, tee=False)
```

CONSTRUCTOR ARGUMENTS

<i>file</i>	Specifies an output file. This file is created on disk.
<i>stream</i>	Specifies an already open output stream. This can be an already open Python file handle, or any object that supports a write method such as an <code>io.StringIO</code> object.
<i>tee</i>	When True, send output to the specified destination but also send the output to the current output stream.
<i>compress</i>	If the output is to a file, the file is created using <code>gzip</code> compression.

DESCRIPTION

The `redirect` type is used to redirect tool output to another location. The type is a standard Python execution context object. Output can be redirected into either a file or an in memory buffer stream. This

type is used in the Python "with" statement.

EXAMPLES

```
# Redirect to a compressed file
with snps.redirect('myFile', compresss=True):
    cmd.report_timing()
```

```
# Redirect to a variable
out = io.StringIO()
with snps.redirect(out):
    cmd.report_timing()
```

SEE ALSO

```
py_eval(2)
snps.cmd(2)
```


PYTHON

NAME

snps.value	Python class for application command return values.
-------------------	---

DESCRIPTION

Command return values that are not numeric or collections are returned as values of this type. The `snps.value` can be treated as either a Python string, sequenced container, or a dictionary as needed.

The documentation for a command will describe the return type.

List Values

For commands that retrieve a list of values, you can treat the returned value as if it is a Python list type. In the following examples, "x" is:

```
x = cmd.get_defined_commands()
```

<code>x[0]</code>	Retrieve the first item (command name). The return value is a <code>snps.value</code> object
-------------------	--

<code>x[-1]</code>	Retrieve the last item. All slice based indexing is supported.
--------------------	--

<code>len(x)</code>	Return the number of items in the value (number of commands)
---------------------	--

<code>for y in x:</code>	Loop over the items. In the loop body "y" is also a <code>snps.value</code> object
--------------------------	--

Dictionary Values

For commands that return key-value pair values, you can

coreTools Command Reference Index

treat the returned values as if it is a Python dict type. In the following examples, "x" is

```
x = cmd.get_defined_commands('get_attribute', details=True)
```

`x['name']` Access the dictionary key 'name'.

`x.keys()` Return an iterator that returns the keys.

`x.value()` Return an iterator that returns the values.

```
for k,v in x.items()
    Iterate over the keys and values.
```

String Values

In many cases the `snps.value` can also be treated as just a plain string value. However you can explicitly cast a value to a Python str object: `str(val)`

SEE ALSO

```
py_eval(2)
snps.collection(2)
snps.cmd
```

NAME

`socket` - Open a TCP network connection

SYNOPSIS

`socket` *?options? host port*

`socket` **-server** *command ?options? port*

DESCRIPTION

This command opens a network socket and returns a channel identifier that may be used in future invocations of commands like **read**, **puts** and **flush**. At present only the TCP network protocol is supported over IPv4 and IPv6; future releases may include support for additional protocols. The **socket** command may be used to open either the client or server side of a connection, depending on whether the **-server** switch is specified.

Note that the default encoding for *all* sockets is the system encoding, as returned by **encoding system**. Most of the time, you will need to use **chan configure** to alter this to something else, such as *utf-8* (ideal for communicating with other Tcl processes) or *iso8859-1* (useful for many network protocols, especially the older ones).

CLIENT SOCKETS

If the **-server** option is not specified, then the client side of a connection is opened and the command returns

coreTools Command Reference Index

a channel identifier that can be used for both reading and writing. *Port* and *host* specify a port to connect to; there must be a server accepting connections on this port. *Port* is an integer port number (or service name, where supported and understood by the host operating system) and *host* is either a domain-style name such as **www.tcl.tk** or a numerical IPv4 or IPv6 address such as **127.0.0.1** or **2001:DB8::1**. Use *localhost* to refer to the host on which the command is invoked.

The following options may also be present before *host* to specify additional information about the connection:

-myaddr *addr*

Addr gives the domain-style name or numerical IP address of the client-side network interface to use for the connection. This option may be useful if the client machine has multiple network interfaces. If the option is omitted then the client-side interface will be chosen by the system software.

-myport *port*

Port specifies an integer port number (or service name, where supported and understood by the host operating system) to use for the client's side of the connection. If this option is omitted, the client's port number will be chosen at random by the system software.

-async

This option will cause the client socket to be connected asynchronously. This means that the socket will be created immediately but may not yet be connected to the server, when the call to **socket** returns.

When a **gets** or **flush** is done on the socket before the connection attempt succeeds or fails, if the socket is in blocking mode, the operation will wait until the connection is completed or fails. If the socket is in nonblocking mode and a **gets** or **flush** is done on the socket before the connection attempt succeeds or fails, the operation returns immediately and **fblocked** on the socket returns 1. Synchronous client sockets may be switched (after they have connected) to operating in asynchronous mode using:

chan configure *chan* **-blocking** 0

See the **chan configure** command for more details.

The Tcl event loop should be running while an asynchronous connection is in progress, because it may have to do several connection attempts in the background. Running the event loop also allows you to set up a writable channel event on the socket to get notified when the asynchronous connection has succeeded or failed. See the **vwait** and the **chan** commands for more details on the event loop and channel events.

The **chan configure** option **-connecting** may be used to check if the connect is still running. To verify a

coreTools Command Reference Index

successful connect, the option **-error** may be checked when **-connecting** returned 0.

Operation without the event queue requires at the moment calls to **chan configure** to advance the internal state machine.

SERVER SOCKETS

If the **-server** option is specified then the new socket will be a server that listens on the given *port* (either an integer or a service name, where supported and understood by the host operating system; if *port* is zero, the operating system will allocate a free port to the server socket which may be discovered by using **chan configure** to read the **-sockname** option). If the host supports both, IPv4 and IPv6, the socket will listen on both address families. Tcl will automatically accept connections to the given port. For each connection Tcl will create a new channel that may be used to communicate with the client. Tcl then invokes *command* (properly a command prefix list, see the **EXAMPLES** below) with three additional arguments: the name of the new channel, the address, in network address notation, of the client's host, and the client's port number.

The following additional option may also be specified before *port*:

-myaddr *addr*

Addr gives the domain-style name or numerical IP address of the server-side network interface to use for the connection. This option may be useful if the server machine has multiple network interfaces. If the option is omitted then the server socket is bound to the wildcard address so that it can accept connections from any interface. If *addr* is a domain name that resolves to multiple IP addresses that are available on the local machine, the socket will listen on all of them.

Server channels cannot be used for input or output; their sole use is to accept new client connections. The channels created for each incoming client connection are opened for input and output. Closing the server channel shuts down the server so that no new connections will be accepted; however, existing connections will be unaffected.

Server sockets depend on the Tcl event mechanism to find out when new connections are opened. If the application does not enter the event loop, for example by invoking the **vwait** command or calling the C procedure **Tcl_DoOneEvent**, then no connections will be accepted.

If *port* is specified as zero, the operating system will allocate an unused port for use as a server socket.

coreTools Command Reference Index

The port number actually allocated may be retrieved from the created server socket using the **chan configure** command to retrieve the **-sockname** option as described below.

CONFIGURATION OPTIONS

The **chan configure** command can be used to query several readonly configuration options for socket channels:

-error

This option gets the current error status of the given socket. This is useful when you need to determine if an asynchronous connect operation succeeded. If there was an error, the error message is returned. If there was no error, an empty string is returned.

Note that the error status is reset by the read operation; this mimics the underlying `getsockopt(SO_ERROR)` call.

-sockname

For client sockets (including the channels that get created when a client connects to a server socket) this option returns a list of three elements, the address, the host name and the port number for the socket. If the host name cannot be computed, the second element is identical to the address, the first element of the list.

For server sockets this option returns a list of a multiple of three elements each group of which have the same meaning as described above. The list contains more than one group when the server socket was created without **-myaddr** or with the argument to **-myaddr** being a domain name that resolves multiple IP addresses that are local to the invoking host.

-peername

This option is not supported by server sockets. For client and accepted sockets, this option returns a list of three elements; these are the address, the host name and the port to which the peer socket is connected or bound. If the host name cannot be computed, the second element of the list is identical to the address, its first element.

-connecting

This option is not supported by server sockets. For client sockets, this option returns 1 if an asynchronous connect is still in progress, 0 otherwise.

EXAMPLES

Here is a very simple time server:

coreTools Command Reference Index

```
proc Server {startTime channel clientaddr clientport} {  
    puts "Connection from $clientaddr registered"  
    set now [clock seconds]  
    puts $channel [clock format $now]  
    puts $channel "[expr {$now - $startTime}] since start"  
    close $channel  
}
```

```
socket -server [list Server [clock seconds]] 9900  
vwait forever
```

And here is the corresponding client to talk to the server and extract some information:

```
set server localhost  
set sockChan [socket $server 9900]  
gets $sockChan line1  
gets $sockChan line2  
close $sockChan  
puts "The time on $server is $line1"  
puts "That is [lindex $line2 0]s since the server started"
```

HISTORY

Support for IPv6 was added in Tcl 8.6.

SEE ALSO

chan(n), flush(n), open(n), read(n)

KEYWORDS

asynchronous I/O, bind, channel, connection, domain name, host, network address, socket, tcp

NAME

source Read a file and evaluate it as a Tcl script.

SYNTAX

```
string source [-echo] [-verbose] [-continue_on_error]  
file
```

string *file*

ARGUMENTS

-echo Echoes each command as it is executed. Note that this option is a non-standard extension to Tcl.

-verbose Displays the result of each command executed. Note that error messages are displayed regardless. Also note that this option is a non-standard extension to Tcl.

-continue_on_error Don't stop script on errors. Similar to setting the shell variable `sh_continue_on_error` to true, but only applies to this particular script.

file Script file to read.

DESCRIPTION

The **source** command takes the contents of the specified *file* and passes it to the command interpreter as a text script. The result of the source command is the result

coreTools Command Reference Index

of the last command executed from the file. If an error occurs in evaluating the contents of the script, then the **source** command returns that error. If a return command is invoked from within the file, the remainder of the file is skipped and the source command returns normally with the result from the return command.

By default, source works quietly, like UNIX. It is possible to get various other intermediate information from the source command using the **-echo** and **-verbose** options. The **-echo** option echoes each command as it appears in the script. The **-verbose** option echoes the result of each command after execution.

NOTE: To emulate the behavior of the dc_shell **include** command, use both of these options.

The file name can be a fully expanded file name and can begin with a tilde. Under normal circumstances, the file is searched for based only on what you typed. However, if the system variable `sh_source_uses_search_path` is set to "true", the file is searched for based on the path established with the `search_path` variable.

The **source** command supports several file formats. The *file* can be a simple ascii script file, an ascii script file compressed with gzip, or a Tcl bytecode script, created by TclPro Compiler. Some applications also supported encrypted files. The **-echo** and **-verbose** options are ignored for encrypted formatted files.

EXAMPLES

This example reads in a script of aliases:

```
prompt> source -echo aliases.tcl
alias q quit
alias hv {help -verbose}
alias include {source -echo -verbose}
prompt>
```

SEE ALSO

search_path(3), *sh_source_uses_search_path(3)*.
sh_continue_on_error(3).

`source_solution_script`

Invoke specified solution script.

Syntax

```
string source_solution_script [-optional] [-once] [-context <context>]
[-uplevel] [-definition] script
    string <context>
    string script
```

Parameters

-optional	Okay for script to not exist.
-once	Don't invoke if already invoked once.
-context <context>	Context to evaluate script in.
-uplevel	Evaluate in calling context.
-definition	Source called within solution.tcl
<i>script</i>	Solution script to be sourced

Description

This command is used to source a script associated with a DesignWare IP Solution. The specified file will be taken from <solution>/tcl if the file ends with .tcl or from <solution>/tcl/include if the file ends with .tcl.inc.

If -optional is specified it is OK for the file to not exist. If -once is specified, the file will only be sourced the first time it is requested to be sourced within a given solution. If -context is specified, the script will be invoked in the indicated context. if -uplevel is specified, the script is sourced from within the calling context.

The -definition option is used to indicate the need to source a script within the solution.tcl file. This is required because the mechanism for finding the file is different when done as part of the definition of the solution.

The return value from the command is the return value of the last command sourced in the file (consistent with the TCL 'source' command).

Examples

Source optional file techsetup.tcl.inc one time:

`source_solution_script -once -optional techsetup.tcl.inc`

See Also

NAME

`split` - Split a string into a proper Tcl list

SYNOPSIS

`split string ?splitChars?`

DESCRIPTION

Returns a list created by splitting *string* at each character that is in the *splitChars* argument. Each element of the result list will consist of the characters from *string* that lie between instances of the characters in *splitChars*. Empty list elements will be generated if *string* contains adjacent characters in *splitChars*, or if the first or last character of *string* is in *splitChars*. If *splitChars* is an empty string then each character of *string* becomes a separate element of the result list. *SplitChars* defaults to the standard white-space characters.

EXAMPLES

Divide up a USENET group name into its hierarchical components:

```
split "comp.lang.tcl" .  
-> comp lang tcl
```

See how the **split** command splits on every character in *splitChars*, which can result in information loss if you are not careful:

coreTools Command Reference Index

```
split "alpha beta gamma" "temp"  
-> al {ha b} {} {a ga} {} a
```

Extract the list words from a string that is not a well-formed list:

```
split "Example with {unbalanced brace character"  
-> Example with \{unbalanced brace character
```

Split a string into its constituent characters

```
split "Hello world" {}  
-> H e l l o { } w o r l d
```

PARSING RECORD-ORIENTED FILES

Parse a Unix /etc/passwd file, which consists of one entry per line, with each line consisting of a colon-separated list of fields:

```
## Read the file  
set fid [open /etc/passwd]  
set content [read $fid]  
close $fid  
  
## Split into records on newlines  
set records [split $content "\n"]  
  
## Iterate over the records  
foreach rec $records {  
  
    ## Split into fields on colons  
    set fields [split $rec ":"]  
  
    ## Assign fields to variables and print some out...  
    lassign $fields \  
        userName password uid grp longName homeDir shell  
    puts "$longName uses [file tail $shell] for a login shell"  
}
```

SEE ALSO

`join(n)`, `list(n)`, `string(n)`

KEYWORDS

`list`, `split`, `string`

NAME

`string` - Manipulate strings

SYNOPSIS

`string option arg ?arg ...?`

DESCRIPTION

Performs one of several string operations, depending on *option*. The legal *options* (which may be abbreviated) are:

string cat *?string1? ?string2...?*

Concatenate the given *strings* just like placing them directly next to each other and return the resulting compound string. If no *strings* are present, the result is an empty string.

This primitive is occasionally handier than juxtaposition of strings when mixed quoting is wanted, or when the aim is to return the result of a concatenation without resorting to **return -level 0**, and is more efficient than building a list of arguments and using **join** with an empty join string.

string compare *?-nocase? ?-length length? string1 string2*

Perform a character-by-character comparison of strings *string1* and *string2*. Returns -1, 0, or 1, depending on whether *string1* is lexicographically less than, equal to, or greater than *string2*. If **-length** is specified, then only the first *length* characters are used in the comparison. If **-length** is negative, it is ignored. If **-nocase** is specified, then the strings are compared in a case-insensitive manner.

coreTools Command Reference Index

string equal *?-nocase? ?-length length? string1 string2*
Perform a character-by-character comparison of strings *string1* and *string2*. Returns 1 if *string1* and *string2* are identical, or 0 when not. If **-length** is specified, then only the first *length* characters are used in the comparison. If **-length** is negative, it is ignored. If **-nocase** is specified, then the strings are compared in a case-insensitive manner.

string first *needleString haystackString ?startIndex?*
Search *haystackString* for a sequence of characters that exactly match the characters in *needleString*. If found, return the index of the first character in the first such match within *haystackString*. If not found, return -1. If *startIndex* is specified (in any of the forms described in **STRING INDICES**), then the search is constrained to start with the character in *haystackString* specified by the index. For example,

string first a 0a23456789abcdef 5

will return **10**, but

string first a 0123456789abcdef 11

will return **-1**.

string index *string charIndex*
Returns the *charIndex*'th character of the *string* argument. A *charIndex* of 0 corresponds to the first character of the string. *charIndex* may be specified as described in the **STRING INDICES** section.

If *charIndex* is less than 0 or greater than or equal to the length of the string then this command returns an empty string.

string is class *?-strict? ?-failindex varname? string*
Returns 1 if *string* is a valid member of the specified character class, otherwise returns 0. If **-strict** is specified, then an empty string returns 0, otherwise an empty string will return 1 on any class. If **-failindex** is specified, then if the function returns 0, the index in the string where the class was no longer valid will be stored in the variable named *varname*. The *varname* will not be set if **string is** returns 1. The following character classes are recognized (the class name can be abbreviated):

alnum	Any Unicode alphabet or digit character.
alpha	Any Unicode alphabet character.
ascii	Any character with a value less than \u0080 (those that are in the 7-bit ascii range).
boolean	Any of the forms allowed to Tcl_GetBoolean .
control	Any Unicode control character.

coreTools Command Reference Index

digit	Any Unicode digit character. Note that this includes characters outside of the [0-9] range.
double	Any of the forms allowed to Tcl_GetDoubleFromObj .
entier	Any of the valid string formats for an integer value of arbitrary size in Tcl, with optional surrounding whitespace. The formats accepted are exactly those accepted by the C routine Tcl_GetBignumFromObj .
false	Any of the forms allowed to Tcl_GetBoolean where the value is false.
graph	Any Unicode printing character, except space.
integer	Any of the valid string formats for a 32-bit integer value in Tcl, with optional surrounding whitespace. In case of overflow in the value, 0 is returned and the <i>varname</i> will contain -1.
list	Any proper list structure, with optional surrounding whitespace. In case of improper list structure, 0 is returned and the <i>varname</i> will contain the index of the where the list parsing fails, or -1 if this cannot be determined.
lower	Any Unicode lower case alphabet character.
print	Any Unicode printing character, including space.
punct	Any Unicode punctuation character.
space	Any Unicode whitespace character, mongolian vowel separator (U+180e), zero width space (U+200b), word joiner (U+2060) or zero width no-break space (U+feff) (=BOM).
true	Any of the forms allowed to Tcl_GetBoolean where the value is true.
upper	Any upper case alphabet character in the Unicode character set.
wideinteger	Any of the valid forms for a wide integer in Tcl, with optional surrounding whitespace. In case of overflow in the value, 0 is returned and the <i>varname</i> will contain -1.
wordchar	Any Unicode word character. That is any alphanumeric character, and any Unicode

coreTools Command Reference Index

connector punctuation characters (e.g. underscore).

xdigit Any hexadecimal digit character ([0-9A-Fa-f]).

In the case of **boolean**, **true** and **false**, if the function will return 0, then the *varname* will always be set to 0, due to the varied nature of a valid boolean value.

string last *needleString haystackString ?lastIndex?*
Search *haystackString* for a sequence of characters that exactly match the characters in *needleString*. If found, return the index of the first character in the last such match within *haystackString*. If there is no match, then return -1. If *lastIndex* is specified (in any of the forms described in **STRING INDICES**), then only the characters in *haystackString* at or before the specified *lastIndex* will be considered by the search. For example,

string last a 0a23456789abcdef 15

will return **10**, but

string last a 0a23456789abcdef 9

will return **1**.

string length *string*

Returns a decimal string giving the number of characters in *string*. Note that this is not necessarily the same as the number of bytes used to store the string. If the value is a byte array value (such as those returned from reading a binary encoded channel), then this will return the actual byte length of the value.

string map *?-nocase? mapping string*

Replaces substrings in *string* based on the key-value pairs in *mapping*. *mapping* is a list of *key value key value ...* as in the form returned by **array get**. Each instance of a key in the string will be replaced with its corresponding value. If **-nocase** is specified, then matching is done without regard to case differences. Both *key* and *value* may be multiple characters. Replacement is done in an ordered manner, so the key appearing first in the list will be checked first, and so on. *string* is only iterated over once, so earlier key replacements will have no affect for later key matches. For example,

string map {abc 1 ab 2 a 3 1 0} 1abcaababcabababc

will return the string **01321221**.

Note that if an earlier *key* is a prefix of a later one, it will completely mask the later one. So if the previous example is reordered like this,

string map {1 0 ab 2 a 3 abc 1} 1abcaababcabababc

coreTools Command Reference Index

it will return the string **02c322c222c**.

string match *?-nocase? pattern string*

See if *pattern* matches *string*; return 1 if it does, 0 if it does not. If **-nocase** is specified, then the pattern attempts to match against the string in a case insensitive manner. For the two strings to match, their contents must be identical except that the following special sequences may appear in *pattern*:

- *** Matches any sequence of characters in *string*, including a null string.
- ?** Matches any single character in *string*.
- [chars]** Matches any character in the set given by *chars*. If a sequence of the form **x-y** appears in *chars*, then any character between **x** and **y**, inclusive, will match. When used with **-nocase**, the end points of the range are converted to lower case first. Whereas **{[A-z]}** matches when matching case-sensitively (since falls between the and with **-nocase** this is considered like **{[A-Za-z]}** (and probably what was meant in the first place).
- \x** Matches the single character **x**. This provides a way of avoiding the special interpretation of the characters ***?[]** in *pattern*.

string range *string first last*

Returns a range of consecutive characters from *string*, starting with the character whose index is *first* and ending with the character whose index is *last*. An index of 0 refers to the first character of the string. *first* and *last* may be specified as for the **index** method. If *first* is less than zero then it is treated as if it were zero, and if *last* is greater than or equal to the length of the string then it is treated as if it were **end**. If *first* is greater than *last* then an empty string is returned.

string repeat *string count*

Returns *string* repeated *count* number of times.

string replace *string first last ?newstring?*

Removes a range of consecutive characters from *string*, starting with the character whose index is *first* and ending with the character whose index is *last*. An index of 0 refers to the first character of the string. *first* and *last* may be specified as for the **index** method. If *newstring* is specified, then it is placed in the removed character range. If *first* is less than zero then it is treated as if it were zero, and if *last* is greater than or equal to the length of the string then it is treated as if it were **end**. If *first* is greater than *last* or the length of the initial string, or *last* is less than 0, then the initial string is returned untouched.

coreTools Command Reference Index

string reverse *string*

Returns a string that is the same length as *string* but with its characters in the reverse order.

string tolower *string* *?first?* *?last?*

Returns a value equal to *string* except that all upper (or title) case letters have been converted to lower case. If *first* is specified, it refers to the first char index in the string to start modifying. If *last* is specified, it refers to the char index in the string to stop at (inclusive). *first* and *last* may be specified using the forms described in **STRING INDICES**.

string totitle *string* *?first?* *?last?*

Returns a value equal to *string* except that the first character in *string* is converted to its Unicode title case variant (or upper case if there is no title case variant) and the rest of the string is converted to lower case. If *first* is specified, it refers to the first char index in the string to start modifying. If *last* is specified, it refers to the char index in the string to stop at (inclusive). *first* and *last* may be specified using the forms described in **STRING INDICES**.

string toupper *string* *?first?* *?last?*

Returns a value equal to *string* except that all lower (or title) case letters have been converted to upper case. If *first* is specified, it refers to the first char index in the string to start modifying. If *last* is specified, it refers to the char index in the string to stop at (inclusive). *first* and *last* may be specified using the forms described in **STRING INDICES**.

string trim *string* *?chars?*

Returns a value equal to *string* except that any leading or trailing characters present in the string given by *chars* are removed. If *chars* is not specified then white space is removed (any character for which **string is space** returns 1, and "\0").

string trimleft *string* *?chars?*

Returns a value equal to *string* except that any leading characters present in the string given by *chars* are removed. If *chars* is not specified then white space is removed (any character for which **string is space** returns 1, and "\0").

string trimright *string* *?chars?*

Returns a value equal to *string* except that any trailing characters present in the string given by *chars* are removed. If *chars* is not specified then white space is removed (any character for which **string is space** returns 1, and "\0").

OBSOLETE SUBCOMMANDS

These subcommands are currently supported, but are likely to go away in a future release as their functionality is either virtually never used or highly misleading.

string bytelength *string*

Returns a decimal string giving the number of bytes

coreTools Command Reference Index

used to represent *string* in memory when encoded as Tcl's internal modified UTF-8; Tcl may use other encodings for *string* as well, and does not guarantee to only use a single encoding for a particular *string*. Because UTF-8 uses a variable number of bytes to represent Unicode characters, the byte length will not be the same as the character length in general. The cases where a script cares about the byte length are rare.

In almost all cases, you should use the **string length** operation (including determining the length of a Tcl byte array value). Refer to the **Tcl_NumUtfChars** manual entry for more details on the UTF-8 representation.

Formally, the **string bytelength** operation returns the content of the *length* field of the **Tcl_Obj** structure, after calling **Tcl_GetString** to ensure that the *bytes* field is populated. This is highly unlikely to be useful to Tcl scripts, as Tcl's internal encoding is not strict UTF-8, but rather a modified CESU-8 with a denormalized NUL (identical to that used in a number of places by Java's serialization mechanism) to enable basic processing with non-Unicode-aware C functions. As this representation should only ever be used by Tcl's implementation, the number of bytes used to store the representation is of very low value (except to C extension code, which has direct access for the purpose of memory management, etc.)

Compatibility note: it is likely that this subcommand will be withdrawn in a future version of Tcl. It is better to use the **encoding convertto** command to convert a string to a known encoding and then apply **string length** to that.

string length [encoding convertto utf-8 \$theString]

string wordend *string charIndex*

Returns the index of the character just after the last one in the word containing character *charIndex* of *string*. *charIndex* may be specified using the forms in **STRING INDICES**. A word is considered to be any contiguous range of alphanumeric (Unicode letters or decimal digits) or underscore (Unicode connector punctuation) characters, or any single character other than these.

string wordstart *string charIndex*

Returns the index of the first character in the word containing character *charIndex* of *string*. *charIndex* may be specified using the forms in **STRING INDICES**. A word is considered to be any contiguous range of alphanumeric (Unicode letters or decimal digits) or underscore (Unicode connector punctuation) characters, or any single character other than these.

STRING INDICES

When referring to indices into a string (e.g., for **string index** or **string range**) the following formats are supported:

<i>integer</i>	For any index value that passes string is integer -strict , the char specified at this integral index (e.g., 2 would refer to the in
end	The last char of the string (e.g., end would refer to the in
end-N	The last char of the string minus the specified integer offset N (e.g., would refer to the in
end+N	The last char of the string plus the specified integer offset N (e.g., would refer to the in
M+N	The char specified at the integral index that is the sum of integer values M and N (e.g., would refer to the in
M-N	The char specified at the integral index that is the difference of integer values M and N (e.g., would refer to the in

In the specifications above, the integer value **M** contains no trailing whitespace and the integer value **N** contains no leading whitespace.

EXAMPLE

Test if the string in the variable *string* is a proper non-empty prefix of the string **foobar**.

```
set length [string length $string]
if {$length == 0} {
    set isPrefix 0
} else {
    set isPrefix [string equal -length $length $string "foobar"]
}
```

SEE ALSO

expr(n), list(n)

KEYWORDS

case conversion, compare, index, match, pattern,
string, word, equal, ctype, character, reverse

NAME

`subst` - Perform backslash, command, and variable substitutions

SYNOPSIS

`subst` *?-nobackslashes? ?-nocommands? ?-novariables?*
string

DESCRIPTION

This command performs variable substitutions, command substitutions, and backslash substitutions on its *string* argument and returns the fully-substituted result. The substitutions are performed in exactly the same way as for Tcl commands. As a result, the *string* argument is actually substituted twice, once by the Tcl parser in the usual fashion for Tcl commands, and again by the *subst* command.

If any of the **-nobackslashes**, **-nocommands**, or **-novariables** are specified, then the corresponding substitutions are not performed. For example, if **-nocommands** is specified, command substitution is not performed: open and close brackets are treated as ordinary characters with no special interpretation.

Note that the substitution of one kind can include substitution of other kinds. For example, even when the **-novariables** option is specified, command substitution is performed without restriction. This means that any variable substitution necessary to complete the command substitution will still take place. Likewise, any command substitution necessary to complete a variable substitution will take place, even when **-nocommands** is specified. See the **EXAMPLES** below.

coreTools Command Reference Index

If an error occurs during substitution, then **subst** will return that error. If a break exception occurs during command or variable substitution, the result of the whole substitution will be the string (as substituted) up to the start of the substitution that raised the exception. If a continue exception occurs during the evaluation of a command or variable substitution, an empty string will be substituted for that entire command or variable substitution (as long as it is well-formed Tcl.) If a return exception occurs, or any other return code is returned during command or variable substitution, then the returned value is substituted for that substitution. See the **EXAMPLES** below. In this way, all exceptional return codes are by **subst**. The **subst** command itself will either return an error, or will complete successfully.

EXAMPLES

When it performs its substitutions, *subst* does not give any special treatment to double quotes or curly braces (except within command substitutions) so the script

```
set a 44
subst {xyz {$a}}
```

returns not and the script

```
set a "p\} q \{r"
subst {xyz {$a}}
```

returns not

When command substitution is performed, it includes any variable substitution necessary to evaluate the script.

```
set a 44
subst -novariables {$a [format $a]}
```

returns not Similarly, when variable substitution is performed, it includes any command substitution necessary to retrieve the value of the variable.

```
proc b {} {return c}
array set a {c c [b] tricky}
subst -nocommands {[b] $a([b])}
```

returns not

The continue and break exceptions allow command substitutions to prevent substitution of the rest of the command substitution and the rest of *string* respectively, giving script authors more options when processing text using *subst*. For example, the script

```
subst {abc,[break],def}
```

returns not and the script

coreTools Command Reference Index

subst {abc,[continue;expr {1+2}],def}

returns not

Other exceptional return codes substitute the returned value

subst {abc,[return foo;expr {1+2}],def}

returns not and

subst {abc,[return -code 10 foo;expr {1+2}],def}

also returns not

SEE ALSO

Tcl(n), eval(n), break(n), continue(n)

KEYWORDS

backslash substitution, command substitution, quoting, substitution, variable substitution

NAME

suppress_message

Disables printing of one or more informational or warning messages.

SYNTAX

```
string suppress_message [message_list]
```

```
list message_list
```

ARGUMENTS

message_list A list of messages to suppress.

DESCRIPTION

The **suppress_message** command provides a mechanism to disable the printing of messages. You can suppress only informational and warning messages. The result of **suppress_message** is always the empty string.

A given message can be suppressed more than once. So, a message must be unsuppressed (using **unsuppress_message**) as many times as it was suppressed in order for it to be enabled. The **print_suppressed_messages** command displays the currently suppressed messages.

EXAMPLES

When the argument to the **unalias** command does not match any existing aliases, the CMD-029 warning message displays. This example shows how to suppress the CMD-029 message:

```
prompt> unalias q*  
Warning: no aliases matched 'q*' (CMD-029)
```

coreTools Command Reference Index

```
prompt> suppress_message CMD-029  
prompt> unalias q*  
prompt>
```

SEE ALSO

*print_suppressed_messages(2), unsuppress_message(2),
get_message_ids(2), set_message_info(2)*

NAME

`switch` - Evaluate one of several scripts, depending on a given value

SYNOPSIS

`switch` *?options?* *string pattern body ?pattern body ...?*

`switch` *?options?* *string {pattern body ?pattern body ...?}*

DESCRIPTION

The **switch** command matches its *string* argument against each of the *pattern* arguments in order. As soon as it finds a *pattern* that matches *string* it evaluates the following *body* argument by passing it recursively to the Tcl interpreter and returns the result of that evaluation. If the last *pattern* argument is **default** then it matches anything. If no *pattern* argument matches *string* and no default is given, then the **switch** command returns an empty string.

If the initial arguments to **switch** start with - then they are treated as options unless there are exactly two arguments to **switch** (in which case the first must be the *string* and the second must be the *pattern/body* list). The following options are currently supported:

- exact** Use exact matching when comparing *string* to a pattern. This is the default.
- glob** When matching *string* to the patterns, use glob-style matching (i.e. the same as implemented by the **string match** command).
- regexp** When matching *string* to the patterns, use

coreTools Command Reference Index

regular expression matching (as described in the **re_syntax** reference page).

- nocase** Causes comparisons to be handled in a case-insensitive manner.
- matchvar** *varName*
This option (only legal when **-regexp** is also specified) specifies the name of a variable into which the list of matches found by the regular expression engine will be written. The first element of the list written will be the overall substring of the input string (i.e. the *string* argument to **switch**) matched, the second element of the list will be the substring matched by the first capturing parenthesis in the regular expression that matched, and so on. When a **default** branch is taken, the variable will have the empty list written to it. This option may be specified at the same time as the **-indexvar** option.
- indexvar** *varName*
This option (only legal when **-regexp** is also specified) specifies the name of a variable into which the list of indices referring to matching substrings found by the regular expression engine will be written. The first element of the list written will be a two-element list specifying the index of the start and index of the first character after the end of the overall substring of the input string (i.e. the *string* argument to **switch**) matched, in a similar way to the **-indices** option to the **regexp** can obtain. Similarly, the second element of the list refers to the first capturing parenthesis in the regular expression that matched, and so on. When a **default** branch is taken, the variable will have the empty list written to it. This option may be specified at the same time as the **-matchvar** option.
- Marks the end of options. The argument following this one will be treated as *string* even if it starts with a -. This is not required when the matching patterns and bodies are grouped together in a single argument.

Two syntaxes are provided for the *pattern* and *body* arguments. The first uses a separate argument for each of the patterns and commands; this form is convenient if substitutions are desired on some of the patterns or commands. The second form places all of the patterns and commands together into a single argument; the argument must have proper list structure, with the elements of the list being the patterns and commands. The second form makes it easy to construct multi-line switch commands, since the braces around the whole list make it unnecessary to include a backslash at the end of each line. Since the *pattern* arguments are in

coreTools Command Reference Index

braces in the second form, no command or variable substitutions are performed on them; this makes the behavior of the second form different than the first form in some cases.

If a *body* is specified as it means that the *body* for the next pattern should also be used as the body for this pattern (if the next pattern also has a body of then the body after that is used, and so on). This feature makes it possible to share a single *body* among several patterns.

Beware of how you place comments in **switch** commands. Comments should only be placed **inside** the execution body of one of the patterns, and not intermingled with the patterns.

EXAMPLES

The **switch** command can match against variables and not just literals, as shown here (the result is **2**):

```
set foo "abc"
switch abc a - b {expr {1}} $foo {expr {2}} default {expr {3}}
```

Using glob matching and the fall-through body is an alternative to writing regular expressions with alternations, as can be seen here (this returns **1**):

```
switch -glob aaab {
  a*b      -
  b        {expr {1}}
  a*       {expr {2}}
  default  {expr {3}}
}
```

Whenever nothing matches, the **default** clause (which must be last) is taken. This example has a result of **3**:

```
switch xyz {
  a -
  b {
    # Correct Comment Placement
    expr {1}
  }
  c {
    expr {2}
  }
  default {
    expr {3}
  }
}
```

When matching against regular expressions, information about what exactly matched is easily obtained using the **-matchvar** option:

```
switch -regex -matchvar foo -- $bar {
  a(b*)c {
```

coreTools Command Reference Index

```
    puts "Found [string length [lindex $foo 1]] 'b's"
  }
  d(e*)f(g*)h {
    puts "Found [string length [lindex $foo 1]] 'e's and\
          [string length [lindex $foo 2]] 'g's"
  }
}
```

SEE ALSO

for(n), if(n), regexp(n)

KEYWORDS

switch, match, regular expression

NAME

synopsys_program_name

Indicates the name of the program currently running.

TYPE

string

DESCRIPTION

This variable is read only, and is set by the application to indicate the name of the program you are running. This is useful when writing scripts that are mostly common between some applications, but contain some differences based on the application.

To determine the current value of this variable, use **get_app_var synopsys_program_name**.

SEE ALSO

get_app_var(2).

NAME

synopsys_root Indicates the root directory from which the application was run.

TYPE

string

DESCRIPTION

This variable is read only, and is set by the application to indicate the root directory from which the application was run.

To determine the current value of this variable, use **get_app_var synopsys_root**.

SEE ALSO

get_app_var(2).

sysCmd

Global array variable containing paths to different system commands.

Syntax

```
string sysCmd = "<array?>"
```

Description

sysCmd is a global array variable that contains paths to different system commands.

For example \$sysCmd(ls) contains the platform dependent path to the ls command. If you execute "exec \$sysCmd(ls)" from the command line, the system ls command is executed by the host operating system.

The coreTools automatically set the sysCmd array values at tool start-up according to the platform on which they are installed. In general, you do not need to modify the sysCmd array values. If the values are modified, they are not persistent across tool invocations.

Examples

See Also

NAME

`tailcall` - Replace the current procedure with another command

SYNOPSIS

`tailcall` *command* *?arg ...?*

DESCRIPTION

The **tailcall** command replaces the currently executing procedure, lambda application, or method with another command. The *command*, which will have *arg ...* passed as arguments if they are supplied, will be looked up in the current namespace context, not in the caller's. Apart from that difference in resolution, it is equivalent to:

```
return [uplevel 1 [list command ?arg ...?]]
```

This command may not be invoked from within an **uplevel** into a procedure or inside a **catch** inside a procedure or lambda.

EXAMPLE

Compute the factorial of a number.

```
proc factorial {n {accum 1}} {  
  if {$n < 2} {  
    return $accum  
  }  
  tailcall factorial [expr {$n - 1}] [expr {$accum * $n}]  
}
```

```
}
```

Print the elements of a list with alternating lines having different indentations.

```
proc printList {theList} {  
    if {[llength $theList]} {  
        puts "> [lindex $theList 0]"  
        tailcall printList2 [lrange $theList 1 end]  
    }  
}  
proc printList2 {theList} {  
    if {[llength $theList]} {  
        puts "< [lindex $theList 0]"  
        tailcall printList [lrange $theList 1 end]  
    }  
}
```

SEE ALSO

`apply(n)`, `proc(n)`, `uplevel(n)`

KEYWORDS

`call`, `recursion`, `tail recursion`

NAME

tcltest - Test harness support code and utilities

SYNOPSIS

```
package require tcltest ?2.5?
```

```
tcltest::test name description ?-option value ...?
tcltest::test name description ?constraints? body result
```

```
tcltest::loadTestedCommands
tcltest::makeDirectory name ?directory?
tcltest::removeDirectory name ?directory?
tcltest::makeFile contents name ?directory?
tcltest::removeFile name ?directory?
tcltest::viewFile name ?directory?
tcltest::cleanupTests ?runningMultipleTests?
tcltest::runAllTests
```

```
tcltest::configure
tcltest::configure -option
tcltest::configure -option value ?-option value ...?
tcltest::customMatch mode command
tcltest::testConstraint constraint ?value?
tcltest::outputChannel ?channelID?
tcltest::errorChannel ?channelID?
tcltest::interpreter ?interp?
```

```
tcltest::debug ?level?
tcltest::errorFile ?filename?
tcltest::limitConstraints ?boolean?
tcltest::loadFile ?filename?
tcltest::loadScript ?script?
tcltest::match ?patternList?
tcltest::matchDirectories ?patternList?
tcltest::matchFiles ?patternList?
tcltest::outputFile ?filename?
tcltest::preserveCore ?level?
tcltest::singleProcess ?boolean?
tcltest::skip ?patternList?
tcltest::skipDirectories ?patternList?
tcltest::skipFiles ?patternList?
tcltest::temporaryDirectory ?directory?
tcltest::testsDirectory ?directory?
tcltest::verbose ?level?
```

```

tcltest::test name description optionList
tcltest::bytestring string
tcltest::normalizeMsg msg
tcltest::normalizePath pathVar
tcltest::workingDirectory ?dir?

```

DESCRIPTION

The **tcltest** package provides several utility commands useful in the construction of test suites for code instrumented to be run by evaluation of Tcl commands. Notably the built-in commands of the Tcl library itself are tested by a test suite using the **tcltest** package.

All the commands provided by the **tcltest** package are defined in and exported from the **::tcltest** namespace, as indicated in the **SYNOPSIS** above. In the following sections, all commands will be described by their simple names, in the interest of brevity.

The central command of **tcltest** is **test** that defines and runs a test. Testing with **test** involves evaluation of a Tcl script and comparing the result to an expected result, as configured and controlled by a number of options. Several other commands provided by **tcltest** govern the configuration of **test** and the collection of many **test** commands into test suites.

See **CREATING TEST SUITES WITH TCLTEST** below for an extended example of how to use the commands of **tcltest** to produce test suites for your Tcl-enabled code.

COMMANDS

```

test name description ?-option value ...?

```

Defines and possibly runs a test with the name *name* and description *description*. The name and description of a test are used in messages reported by **test** during the test, as configured by the options of **tcltest**. The remaining *option value* arguments to **test** define the test, including the scripts to run, the conditions under which to run them, the expected result, and the means by which the expected and actual results should be compared. See **TESTS** below for a complete description of the valid options and how they define a test. The **test** command returns an empty string.

```

test name description ?constraints? body result

```

coreTools Command Reference Index

This form of **test** is provided to support test suites written for version 1 of the **tcptest** package, and also a simpler interface for a common usage. It is the same as All other options to **test** take their default values. When *constraints* is omitted, this form of **test** can be distinguished from the first because all *options* begin with

loadTestedCommands

Evaluates in the caller's context the script specified by **configure -load** or **configure -loadfile**. Returns the result of that script evaluation, including any error raised by the script. Use this command and the related configuration options to provide the commands to be tested to the interpreter running the test suite.

makeFile *contents name ?directory?*

Creates a file named *name* relative to directory *directory* and write *contents* to that file using the encoding **encoding system**. If *contents* does not end with a newline, a newline will be appended so that the file named *name* does end with a newline. Because the system encoding is used, this command is only suitable for making text files. The file will be removed by the next evaluation of **cleanupTests**, unless it is removed by **removeFile** first. The default value of *directory* is the directory **configure -tmpdir**. Returns the full path of the file created. Use this command to create any text file required by a test with contents as needed.

removeFile *name ?directory?*

Forces the file referenced by *name* to be removed. This file name should be relative to *directory*. The default value of *directory* is the directory **configure -tmpdir**. Returns an empty string. Use this command to delete files created by **makeFile**.

makeDirectory *name ?directory?*

Creates a directory named *name* relative to directory *directory*. The directory will be removed by the next evaluation of **cleanupTests**, unless it is removed by **removeDirectory** first. The default value of *directory* is the directory **configure -tmpdir**. Returns the full path of the directory created. Use this command to create any directories that are required to exist by a test.

removeDirectory *name ?directory?*

Forces the directory referenced by *name* to be removed. This directory should be relative to *directory*. The default value of *directory* is the directory **configure -tmpdir**. Returns an empty string. Use this command to delete any directories created by **makeDirectory**.

viewFile *file ?directory?*

Returns the contents of *file*, except for any final newline, just as **read -nonewline** would return. This file name should be relative to *directory*. The default value of *directory* is the directory **configure -tmpdir**. Use this command as a convenient way to turn the contents of a file generated by a test into the result of that test for matching against an expected result.

coreTools Command Reference Index

The contents of the file are read using the system encoding, so its usefulness is limited to text files.

cleanupTests

Intended to clean up and summarize after several tests have been run. Typically called once per test file, at the end of the file after all tests have been completed. For best effectiveness, be sure that the **cleanupTests** is evaluated even if an error occurs earlier in the test file evaluation.

Prints statistics about the tests run and removes files that were created by **makeDirectory** and **makeFile** since the last **cleanupTests**. Names of files and directories in the directory **configure -tmpdir** created since the last **cleanupTests**, but not created by **makeFile** or **makeDirectory** are printed to **outputChannel**. This command also restores the original shell environment, as described by the global **env** array. Returns an empty string.

runAllTests

This is a main command meant to run an entire suite of tests, spanning multiple files and/or directories, as governed by the configurable options of **tcltest**. See **RUNNING ALL TESTS** below for a complete description of the many variations possible with **runAllTests**.

CONFIGURATION COMMANDS

configure

Returns the list of configurable options supported by **tcltest**. See **CONFIGURABLE OPTIONS** below for the full list of options, their valid values, and their effect on **tcltest** operations.

configure option

Returns the current value of the supported configurable option *option*. Raises an error if *option* is not a supported configurable option.

configure option value ?-option value ...?

Sets the value of each configurable option *option* to the corresponding value *value*, in order. Raises an error if an *option* is not a supported configurable option, or if *value* is not a valid value for the corresponding *option*, or if a *value* is not provided. When an error is raised, the operation of **configure** is halted, and subsequent *option value* arguments are not processed.

If the environment variable **::env(TCLTEST_OPTIONS)** exists when the **tcltest** package is loaded (by **package require tcltest**) then its value is taken as a list of arguments to pass to **configure**. This allows the default values of the configuration options to be set by the environment.

customMatch mode script

Registers *mode* as a new legal value of the **-match** option to **test**. When the **-match mode** option is passed to **test**, the script *script* will be evaluated to compare the actual result of evaluating the body of the test to

coreTools Command Reference Index

the expected result. To perform the match, the *script* is completed with two additional words, the expected result, and the actual result, and the completed script is evaluated in the global namespace. The completed script is expected to return a boolean value indicating whether or not the results match. The built-in matching modes of **test** are **exact**, **glob**, and **regex**.

testConstraint *constraint ?boolean?*

Sets or returns the boolean value associated with the named *constraint*. See **TEST CONSTRAINTS** below for more information.

interpreter *?executableName?*

Sets or returns the name of the executable to be **execed** by **runAllTests** to run each test file when **configure -singleproc** is false. The default value for **interpreter** is the name of the currently running program as returned by **info nameofexecutable**.

outputChannel *?channelID?*

Sets or returns the output channel ID. This defaults to **stdout**. Any test that prints test related output should send that output to **outputChannel** rather than letting that output default to **stdout**.

errorChannel *?channelID?*

Sets or returns the error channel ID. This defaults to **stderr**. Any test that prints error messages should send that output to **errorChannel** rather than printing directly to **stderr**.

SHORTCUT CONFIGURATION COMMANDS

debug *?level?*

Same as

errorFile *?filename?*

Same as

limitConstraints *?boolean?*

Same as

loadFile *?filename?*

Same as

loadScript *?script?*

Same as

match *?patternList?*

Same as

matchDirectories *?patternList?*

Same as

matchFiles *?patternList?*

Same as

outputFile *?filename?*

Same as

preserveCore *?level?*

Same as

coreTools Command Reference Index

singleProcess *?boolean?*

Same as

skip *?patternList?*

Same as

skipDirectories *?patternList?*

Same as

skipFiles *?patternList?*

Same as

temporaryDirectory *?directory?*

Same as

testsDirectory *?directory?*

Same as

verbose *?level?*

Same as

OTHER COMMANDS

The remaining commands provided by **tcltest** have better alternatives provided by **tcltest** or **Tcl** itself. They are retained to support existing test suites, but should be avoided in new code.

test *name description optionList*

This form of **test** was provided to enable passing many options spanning several lines to **test** as a single argument quoted by braces, rather than needing to backslash quote the newlines between arguments to **test**. The *optionList* argument is expected to be a list with an even number of elements representing *option* and *value* arguments to pass to **test**. However, these values are not passed directly, as in the alternate forms of **switch**. Instead, this form makes an unfortunate attempt to overthrow Tcl's substitution rules by performing substitutions on some of the list elements as an attempt to implement a interpretation of a brace-enclosed The result is nearly impossible to document clearly, and for that reason this form is not recommended. See the examples in **CREATING TEST SUITES WITH TCLTEST** below to see that this form is really not necessary to avoid backslash-quoted newlines. If you insist on using this form, examine the source code of **tcltest** if you want to know the substitution details, or just enclose the third through last argument to **test** in braces and hope for the best.

workingDirectory *?directoryName?*

Sets or returns the current working directory when the test suite is running. The default value for **workingDirectory** is the directory in which the test suite was launched. The Tcl commands **cd** and **pwd** are sufficient replacements.

normalizeMsg *msg*

Returns the result of removing the newlines from *msg*, where is rather imprecise. Tcl offers plenty of string processing commands to modify strings as you wish, and

coreTools Command Reference Index

customMatch allows flexible matching of actual and expected results.

normalizePath *pathVar*

Resolves symlinks in a path, thus creating a path without internal redirection. It is assumed that *pathVar* is absolute. *pathVar* is modified in place. The Tcl command **file normalize** is a sufficient replacement.

bytestring *string*

Construct a string that consists of the requested sequence of bytes, as opposed to a string of properly formed UTF-8 characters using the value supplied in *string*. This allows the tester to create denormalized or improperly formed strings to pass to C procedures that are supposed to accept strings with embedded NULL types and confirm that a string result has a certain pattern of bytes. This is exactly equivalent to the Tcl command **encoding convertfrom identity**.

TESTS

The **test** command is the heart of the **tcltest** package. Its essential function is to evaluate a Tcl script and compare the result with an expected result. The options of **test** define the test script, the environment in which to evaluate it, the expected result, and how the compare the actual result to the expected result. Some configuration options of **tcltest** also influence how **test** operates.

The valid options for **test** are summarized:

```
test name description
      ?-constraints keywordList|expression?
      ?-setup setupScript?
      ?-body testScript?
      ?-cleanup cleanupScript?
      ?-result expectedAnswer?
      ?-output expectedOutput?
      ?-errorOutput expectedError?
      ?-returnCodes codeList?
      ?-errorCode expectedErrorCode?
      ?-match mode?
```

The *name* may be any string. It is conventional to choose a *name* according to the pattern:

target-majorNum.minorNum

For white-box (regression) tests, the target should be the name of the C function or Tcl procedure being tested. For black-box tests, the target should be the name of the feature being tested. Some conventions call for the names of black-box tests to have the suffix **_bb**. Related tests should share a major number. As a test suite evolves, it is best to have the same

coreTools Command Reference Index

test name continue to correspond to the same test, so that it remains meaningful to say things like

During evaluation of **test**, the *name* will be compared to the lists of string matching patterns returned by **configure -match**, and **configure -skip**. The test will be run only if *name* matches any of the patterns from **configure -match** and matches none of the patterns from **configure -skip**.

The *description* should be a short textual description of the test. The *description* is included in output produced by the test, typically test failure messages. Good *description* values should briefly explain the purpose of the test to users of a test suite. The name of a Tcl or C function being tested should be included in the description for regression tests. If the test case exists to reproduce a bug, include the bug ID in the description.

Valid attributes and associated values are:

-constraints *keywordList|expression*

The optional **-constraints** attribute can be list of one or more keywords or an expression. If the **-constraints** value is a list of keywords, each of these keywords should be the name of a constraint defined by a call to **testConstraint**. If any of the listed constraints is false or does not exist, the test is skipped. If the **-constraints** value is an expression, that expression is evaluated. If the expression evaluates to true, then the test is run. Note that the expression form of **-constraints** may interfere with the operation of **configure -constraints** and **configure -limitconstraints**, and is not recommended. Appropriate constraints should be added to any tests that should not always be run. That is, conditional evaluation of a test should be accomplished by the **-constraints** option, not by conditional evaluation of **test**. In that way, the same number of tests are always reported by the test suite, though the number skipped may change based on the testing environment. The default value is an empty list. See **TEST CONSTRAINTS** below for a list of built-in constraints and information on how to add your own constraints.

-setup *script*

The optional **-setup** attribute indicates a *script* that will be run before the script indicated by the **-body** attribute. If evaluation of *script* raises an error, the test will fail. The default value is an empty script.

-body *script*

The **-body** attribute indicates the *script* to run to carry out the test, which must return a result that can be checked for correctness. If evaluation of *script* raises an error, the test will fail (unless the **-returnCodes** option is used to state that an error is expected). The default value is an empty script.

-cleanup *script*

coreTools Command Reference Index

The optional **-cleanup** attribute indicates a *script* that will be run after the script indicated by the **-body** attribute. If evaluation of *script* raises an error, the test will fail. The default value is an empty script.

-match *mode*

The **-match** attribute determines how expected answers supplied by **-result**, **-output**, and **-errorOutput** are compared. Valid values for *mode* are **regexp**, **glob**, **exact**, and any value registered by a prior call to **customMatch**. The default value is **exact**.

-result *expectedValue*

The **-result** attribute supplies the *expectedValue* against which the return value from script will be compared. The default value is an empty string.

-output *expectedValue*

The **-output** attribute supplies the *expectedValue* against which any output sent to **stdout** or **outputChannel** during evaluation of the script(s) will be compared. Note that only output printed using the global **puts** command is used for comparison. If **-output** is not specified, output sent to **stdout** and **outputChannel** is not processed for comparison.

-errorOutput *expectedValue*

The **-errorOutput** attribute supplies the *expectedValue* against which any output sent to **stderr** or **errorChannel** during evaluation of the script(s) will be compared. Note that only output printed using the global **puts** command is used for comparison. If **-errorOutput** is not specified, output sent to **stderr** and **errorChannel** is not processed for comparison.

-returnCodes *expectedCodeList*

The optional **-returnCodes** attribute supplies *expectedCodeList*, a list of return codes that may be accepted from evaluation of the **-body** script. If evaluation of the **-body** script returns a code not in the *expectedCodeList*, the test fails. All return codes known to **return**, in both numeric and symbolic form, including extended return codes, are acceptable elements in the *expectedCodeList*. Default value is

-errorCode *expectedErrorCode*

The optional **-errorCode** attribute supplies *expectedErrorCode*, a glob pattern that should match the error code reported from evaluation of the **-body** script. If evaluation of the **-body** script returns a code not matching *expectedErrorCode*, the test fails. Default value is If **-returnCodes** does not include **error** it is set to **error**.

To pass, a test must successfully evaluate its **-setup**, **-body**, and **-cleanup** scripts. The return code of the **-body** script and its result must match expected values, and if specified, output and error data from the test must match expected **-output** and **-errorOutput** values. If any of these conditions are not met, then the test fails. Note that all scripts are evaluated in the

coreTools Command Reference Index

context of the caller of **test**.

As long as **test** is called with valid syntax and legal values for all attributes, it will not raise an error. Test failures are instead reported as output written to **outputChannel**. In default operation, a successful test produces no output. The output messages produced by **test** are controlled by the **configure -verbose** option as described in **CONFIGURABLE OPTIONS** below. Any output produced by the test scripts themselves should be produced using **puts** to **outputChannel** or **errorChannel**, so that users of the test suite may easily capture output with the **configure -outfile** and **configure -errfile** options, and so that the **-output** and **-errorOutput** attributes work properly.

TEST CONSTRAINTS

Constraints are used to determine whether or not a test should be skipped. Each constraint has a name, which may be any string, and a boolean value. Each **test** has a **-constraints** value which is a list of constraint names. There are two modes of constraint control. Most frequently, the default mode is used, indicated by a setting of **configure -limitconstraints** to false. The test will run only if all constraints in the list are true-valued. Thus, the **-constraints** option of **test** is a convenient, symbolic way to define any conditions required for the test to be possible or meaningful. For example, a **test** with **-constraints unix** will only be run if the constraint **unix** is true, which indicates the test suite is being run on a Unix platform.

Each **test** should include whatever **-constraints** are required to constrain it to run only where appropriate. Several constraints are pre-defined in the **tcctest** package, listed below. The registration of user-defined constraints is performed by the **testConstraint** command. User-defined constraints may appear within a test file, or within the script specified by the **configure -load** or **configure -loadfile** options.

The following is a list of constraints pre-defined by the **tcctest** package itself:

singleTestInterp

This test can only be run if all test files are sourced into a single interpreter.

unix

This test can only be run on any Unix platform.

win

This test can only be run on any Windows platform.

nt

This test can only be run on any Windows NT platform.

mac

This test can only be run on any Mac platform.

unixOrWin

This test can only be run on a Unix or Windows

coreTools Command Reference Index

platform.

macOrWin

This test can only be run on a Mac or Windows platform.

macOrUnix

This test can only be run on a Mac or Unix platform.

tempNotWin

This test can not be run on Windows. This flag is used to temporarily disable a test.

tempNotMac

This test can not be run on a Mac. This flag is used to temporarily disable a test.

unixCrash

This test crashes if it is run on Unix. This flag is used to temporarily disable a test.

winCrash

This test crashes if it is run on Windows. This flag is used to temporarily disable a test.

macCrash

This test crashes if it is run on a Mac. This flag is used to temporarily disable a test.

emptyTest

This test is empty, and so not worth running, but it remains as a place-holder for a test to be written in the future. This constraint has value false to cause tests to be skipped unless the user specifies otherwise.

knownBug

This test is known to fail and the bug is not yet fixed. This constraint has value false to cause tests to be skipped unless the user specifies otherwise.

nonPortable

This test can only be run in some known development environment. Some tests are inherently non-portable because they depend on things like word length, file system configuration, window manager, etc. This constraint has value false to cause tests to be skipped unless the user specifies otherwise.

userInteraction

This test requires interaction from the user. This constraint has value false to causes tests to be skipped unless the user specifies otherwise.

interactive

This test can only be run in if the interpreter is in interactive mode (when the global `tcl_interactive` variable is set to 1).

nonBlockFiles

This test can only be run if platform supports setting files into nonblocking mode.

coreTools Command Reference Index

asyncPipeClose

This test can only be run if platform supports async flush and async close on a pipe.

unixExecs

This test can only be run if this machine has Unix-style commands **cat**, **echo**, **sh**, **wc**, **rm**, **sleep**, **fgrep**, **ps**, **chmod**, and **mkdir** available.

hasIsoLocale

This test can only be run if can switch to an ISO locale.

root

This test can only run if Unix user is root.

notRoot

This test can only run if Unix user is not root.

eformat

This test can only run if app has a working version of **sprintf** with respect to the format of floating-point numbers.

stdio

This test can only be run if **interpreter** can be **opened** as a pipe.

The alternative mode of constraint control is enabled by setting **configure -limitconstraints** to true. With that configuration setting, all existing constraints other than those in the constraint list returned by **configure -constraints** are set to false. When the value of **configure -constraints** is set, all those constraints are set to true. The effect is that when both options **configure -constraints** and **configure -limitconstraints** are in use, only those tests including only constraints from the **configure -constraints** list are run; all others are skipped. For example, one might set up a configuration with

```
configure -constraints knownBug \  
          -limitconstraints true \  
          -verbose pass
```

to run exactly those tests that exercise known bugs, and discover whether any of them pass, indicating the bug had been fixed.

RUNNING ALL TESTS

The single command **runAllTests** is evaluated to run an entire test suite, spanning many files and directories. The configuration options of **tcltest** control the precise operations. The **runAllTests** command begins by printing a summary of its configuration to **outputChannel**.

Test files to be evaluated are sought in the directory **configure -testdir**. The list of files in that directory that match any of the patterns in **configure -file** and match none of the patterns in **configure -notfile** is generated and sorted. Then each file will

be evaluated in turn. If **configure -singleproc** is true, then each file will be **sourced** in the caller's context. If it is false, then a copy of **interpreter** will be **exec'd** to evaluate each file. The multi-process operation is useful when testing can cause errors so severe that a process terminates. Although such an error may terminate a child process evaluating one file, the main process can continue with the rest of the test suite. In multi-process operation, the configuration of **tcltest** in the main process is passed to the child processes as command line arguments, with the exception of **configure -outfile**. The **runAllTests** command in the main process collects all output from the child processes and collates their results into one main report. Any reports of individual test failures, or messages requested by a **configure -verbose** setting are passed directly on to **outputChannel** by the main process.

After evaluating all selected test files, a summary of the results is printed to **outputChannel**. The summary includes the total number of **tests** evaluated, broken down into those skipped, those passed, and those failed. The summary also notes the number of files evaluated, and the names of any files with failing tests or errors. A list of the constraints that caused tests to be skipped, and the number of tests skipped for each is also printed. Also, messages are printed if it appears that evaluation of a test file has caused any temporary files to be left behind in **configure -tmpdir**.

Having completed and summarized all selected test files, **runAllTests** then recursively acts on subdirectories of **configure -testdir**. All subdirectories that match any of the patterns in **configure -relateddir** and do not match any of the patterns in **configure -asidefromdir** are examined. If a file named **all.tcl** is found in such a directory, it will be **sourced** in the caller's context. Whether or not an examined directory contains an **all.tcl** file, its subdirectories are also scanned against the **configure -relateddir** and **configure -asidefromdir** patterns. In this way, many directories in a directory tree can have all their test files evaluated by a single **runAllTests** command.

CONFIGURABLE OPTIONS

The **configure** command is used to set and query the configurable options of **tcltest**. The valid options are:

-singleproc *boolean*

Controls whether or not **runAllTests** spawns a child process for each test file. No spawning when *boolean* is true. Default value is false.

-debug *level*

Sets the debug level to *level*, an integer value

coreTools Command Reference Index

indicating how much debugging information should be printed to **stdout**. Note that debug messages always go to **stdout**, independent of the value of **configure -outfile**. Default value is 0. Levels are defined as:

- | | |
|---|---|
| 0 | Do not display any debug information. |
| 1 | Display information regarding whether a test is skipped because it does not match any of the tests that were specified using by configure -match (userSpecifiedNonMatch) or matches any of the tests specified by configure -skip (userSpecifiedSkip). Also print warnings about possible lack of cleanup or balance in test files. Also print warnings about any re-use of test names. |
| 2 | Display the flag array parsed by the command line processor, the contents of the global env array, and all user-defined variables that exist in the current namespace as they are used. |
| 3 | Display information regarding what individual procs in the test harness are doing. |

-verbose level

Sets the type of output verbosity desired to *level*, a list of zero or more of the elements **body**, **pass**, **skip**, **start**, **error**, **line**, **msec** and **usec**. Default value is Levels are defined as:

- | | |
|--------------------|--|
| body (b) | Display the body of failed tests |
| pass (p) | Print output when a test passes |
| skip (s) | Print output when a test is skipped |
| start (t) | Print output whenever a test starts |
| error (e) | Print errorInfo and errorCode , if they exist, when a test return code does not match its expected return code |
| line (l) | Print source file line information of failed tests |
| msec (m) | Print each test's execution time in milliseconds |
| usec (u) | Print each test's execution time in microseconds |

Note that the **msec** and **usec** verbosity levels are provided as indicative measures only. They do not tackle the problem of repeatability which should be considered in performance tests or benchmarks. To use these verbosity levels to thoroughly track performance degradations, consider wrapping your test bodies with **time** commands.

coreTools Command Reference Index

The single letter abbreviations noted above are also recognized so that is the same as

-preservecore *level*

Sets the core preservation level to *level*. This level determines how stringent checks for core files are. Default value is 0. Levels are defined as:

- | | |
|---|---|
| 0 | No checking -- do not check for core files at the end of each test command, but do check for them in runAllTests after all test files have been evaluated. |
| 1 | Also check for core files at the end of each test command. |
| 2 | Check for core files at all times described above, and save a copy of each core file produced in configure -tmpdir . |

-limitconstraints *boolean*

Sets the mode by which **test** honors constraints as described in **TESTS** above. Default value is false.

-constraints *list*

Sets all the constraints in *list* to true. Also used in combination with **configure -limitconstraints true** to control an alternative constraint mode as described in **TESTS** above. Default value is an empty list.

-tmpdir *directory*

Sets the temporary directory to be used by **makeFile**, **makeDirectory**, **viewFile**, **removeFile**, and **removeDirectory** as the default directory where temporary files and directories created by test files should be created. Default value is **workingDirectory**.

-testdir *directory*

Sets the directory searched by **runAllTests** for test files and subdirectories. Default value is **workingDirectory**.

-file *patternList*

Sets the list of patterns used by **runAllTests** to determine what test files to evaluate. Default value is

-notfile *patternList*

Sets the list of patterns used by **runAllTests** to determine what test files to skip. Default value is so that any SCCS lock files are skipped.

-relateddir *patternList*

Sets the list of patterns used by **runAllTests** to determine what subdirectories to search for an **all.tcl** file. Default value is

-asidefromdir *patternList*

Sets the list of patterns used by **runAllTests** to determine what subdirectories to skip when searching

coreTools Command Reference Index

for an **all.tcl** file. Default value is an empty list.

-match *patternList*

Set the list of patterns used by **test** to determine whether a test should be run. Default value is

-skip *patternList*

Set the list of patterns used by **test** to determine whether a test should be skipped. Default value is an empty list.

-load *script*

Sets a script to be evaluated by **loadTestedCommands**. Default value is an empty script.

-loadfile *filename*

Sets the filename from which to read a script to be evaluated by **loadTestedCommands**. This is an alternative to **-load**. They cannot be used together.

-outfile *filename*

Sets the file to which all output produced by **tcltest** should be written. A file named *filename* will be **opened** for writing, and the resulting channel will be set as the value of **outputChannel**.

-errfile *filename*

Sets the file to which all error output produced by **tcltest** should be written. A file named *filename* will be **opened** for writing, and the resulting channel will be set as the value of **errorChannel**.

CREATING TEST SUITES WITH TCLTEST

The fundamental element of a test suite is the individual **test** command. We begin with several examples.

[1] Test of a script that returns normally.

```
test example-1.0 {normal return} {  
    format %s value  
} value
```

[2] Test of a script that requires context setup and cleanup. Note the bracing and indenting style that avoids any need for line continuation.

```
test example-1.1 {test file existence} -setup {  
    set file [makeFile {} test]  
} -body {  
    file exists $file  
} -cleanup {  
    removeFile test  
} -result 1
```

[3] Test of a script that raises an error.

coreTools Command Reference Index

```
test example-1.2 {error return} -body {  
    error message  
} -returnCodes error -result message
```

[4] Test with a constraint.

```
test example-1.3 {user owns created files} -constraints {  
    unix  
} -setup {  
    set file [makeFile {} test]  
} -body {  
    file attributes $file -owner  
} -cleanup {  
    removeFile test  
} -result $::tcl_platform(user)
```

At the next higher layer of organization, several **test** commands are gathered together into a single test file. Test files should have names with the extension, because that is the default pattern used by **runAllTests** to find test files. It is a good rule of thumb to have one test file for each source code file of your project. It is good practice to edit the test file and the source code file together, keeping tests synchronized with code changes.

Most of the code in the test file should be the **test** commands. Use constraints to skip tests, rather than conditional evaluation of **test**.

[5] Recommended system for writing conditional tests, using constraints to guard:

```
testConstraint X [expr $myRequirement]  
test goodConditionalTest {} X {  
    # body  
} result
```

[6] Discouraged system for writing conditional tests, using **if** to guard:

```
if $myRequirement {  
    test badConditionalTest {} {  
        #body  
    } result  
}
```

Use the **-setup** and **-cleanup** options to establish and release all context requirements of the test body. Do not make tests depend on prior tests in the file. Those prior tests might be skipped. If several consecutive tests require the same context, the appropriate setup and cleanup scripts may be stored in variable for passing to each tests **-setup** and **-cleanup** options. This is a better solution than performing setup outside of **test** commands, because the setup will only be done if necessary, and any errors during setup will be reported, and not cause the test file to abort.

A test file should be able to be combined with other test files and not interfere with them, even when

coreTools Command Reference Index

configure -singleproc 1 causes all files to be evaluated in a common interpreter. A simple way to achieve this is to have your tests define all their commands and variables in a namespace that is deleted when the test file evaluation is complete. A good namespace to use is a child namespace **test** of the namespace of the module you are testing.

A test file should also be able to be evaluated directly as a script, not depending on being called by a main **runAllTests**. This means that each test file should process command line arguments to give the tester all the configuration control that **tcctest** provides.

After all **tests** in a test file, the command **cleanupTests** should be called.

```
[7]          Here is a sketch of a sample test file
              illustrating those points:

package require tcctest 2.5
eval ::tcctest::configure $argv
package require example
namespace eval ::example::test {
    namespace import ::tcctest::*
    testConstraint X [expr {...}]
    variable SETUP {#common setup code}
    variable CLEANUP {#common cleanup code}
    test example-1 {} -setup $SETUP -body {
        # First test
    } -cleanup $CLEANUP -result {...}
    test example-2 {} -constraints X -setup $SETUP -body {
        # Second test; constrained
    } -cleanup $CLEANUP -result {...}
    test example-3 {} {
        # Third test; no context required
    } {...}
    cleanupTests
}
namespace delete ::example::test
```

The next level of organization is a full test suite, made up of several test files. One script is used to control the entire suite. The basic function of this script is to call **runAllTests** after doing any necessary setup. This script is usually named **all.tcl** because that is the default name used by **runAllTests** when combining multiple test suites into one testing run.

```
[8]          Here is a sketch of a sample test suite
              main script:

package require Tcl 8.6
package require tcctest 2.5
package require example
::tcctest::configure -testdir \
    [file dirname [file normalize [info script]]]
eval ::tcctest::configure $argv
::tcctest::runAllTests
```

COMPATIBILITY

A number of commands and variables in the `::tcltest` namespace provided by earlier releases of `tcltest` have not been documented here. They are no longer part of the supported public interface of `tcltest` and should not be used in new test suites. However, to continue to support existing test suites written to the older interface specifications, many of those deprecated commands and variables still work as before. For example, in many circumstances, `configure` will be automatically called shortly after `package require tcltest 2.1` succeeds with arguments from the variable `::argv`. This is to support test suites that depend on the old behavior that `tcltest` was automatically configured from command line arguments. New test files should not depend on this, but should explicitly include

```
eval ::tcltest::configure $::argv
```

or

```
::tcltest::configure {*} $::argv
```

to establish a configuration from command line arguments.

KNOWN ISSUES

There are two known issues related to nested evaluations of `test`. The first issue relates to the stack level in which test scripts are executed. Tests nested within other tests may be executed at the same stack level as the outermost test. For example, in the following code:

```
test level-1.1 {level 1} {
    -body {
        test level-2.1 {level 2} {
        }
    }
}
```

any script executed in level-2.1 may be executed at the same stack level as the script defined for level-1.1.

In addition, while two `tests` have been run, results will only be reported by `cleanupTests` for tests at the same level as test level-1.1. However, test results for all tests run prior to level-1.1 will be available when test level-2.1 runs. What this means is that if you try to access the test results for test level-2.1, it will may say that tests have run, tests have been skipped, tests have passed and tests have failed, where and refer to tests that were run at the same test level as test level-1.1.

coreTools Command Reference Index

Implementation of output and error comparison in the `test` command depends on usage of `puts` in your application code. Output is intercepted by redefining the global `puts` command while the defined test script is being run. Errors thrown by C procedures or printed directly from C applications will not be caught by the `test` command. Therefore, usage of the `-output` and `-errorOutput` options to `test` is useful only for pure Tcl applications that use `puts` to produce output.

KEYWORDS

`test`, test harness, test suite

NAME

argc, argv, argv0, auto_path, env, errorCode,
errorInfo, tcl_interactive, tcl_library,
tcl_nonwordchars, tcl_patchLevel, tcl_pkgPath,
tcl_platform, tcl_precision, tcl_rcFileName,
tcl_traceCompile, tcl_traceExec, tcl_wordchars,
tcl_version - Variables used by Tcl

DESCRIPTION

The following global variables are created and managed automatically by the Tcl library. Except where noted below, these variables should normally be treated as read-only by application-specific code and by users.

auto_path

If set, then it must contain a valid Tcl list giving directories to search during auto-load operations (including for package index files when using the default **package unknown** handler). This variable is initialized during startup to contain, in order: the directories listed in the **TCLLIBPATH** environment variable, the directory named by the **tcl_library** global variable, the parent directory of **tcl_library**, the directories listed in the **tcl_pkgPath** variable. Additional locations to look for files and package indices should normally be added to this variable using **lappend**.

Additional variables relating to package management exist. More details are listed in the **VARIABLES** section of the **library** manual page.

env

This variable is maintained by Tcl as an array whose elements are the environment variables for the process. Reading an element will return the value of the corresponding environment variable. Setting an element of the array will modify the corresponding environment variable or create a new one if it does not already

coreTools Command Reference Index

exist. Unsetting an element of **env** will remove the corresponding environment variable. Changes to the **env** array will affect the environment passed to children by commands like **exec**. If the entire **env** array is unset then Tcl will stop monitoring **env** accesses and will not update environment variables.

Under Windows, the environment variables **PATH** and **COMSPEC** in any capitalization are converted automatically to upper case. For instance, the **PATH** variable could be exported by the operating system as etc., causing otherwise simple Tcl code to have to support many special cases. All other environment variables inherited by Tcl are left unmodified. Setting an **env** array variable to blank is the same as unsetting it as this is the behavior of the underlying Windows OS. It should be noted that relying on an existing and empty environment variable will not work on Windows and is discouraged for cross-platform usage.

The following elements of **env** are special to Tcl:

env(HOME)

This environment variable, if set, gives the location of the directory considered to be the current user's home directory, and to which a call of **cd** without arguments or with just as an argument will change into. Most platforms set this correctly by default; it does not normally need to be set by user code.

env(TCL_LIBRARY)

If set, then it specifies the location of the directory containing library scripts (the value of this variable will be assigned to the **tcl_library** variable and therefore returned by the command **info library**). If this variable is not set then a default value is used.

Note that this environment variable should *not* normally be set.

env(TCLLIBPATH)

If set, then it must contain a valid Tcl list giving directories to search during auto-load operations. Directories must be specified in Tcl format, using as the path separator, regardless of platform. This variable is only used when initializing the **auto_path** variable.

env(TCL_TZ), env(TZ)

These specify the default timezone used for parsing and formatting times and dates in the **clock** command. On many platforms, the **TZ** environment variable is set up by the operating system.

env(LC_ALL), env(LC_MESSAGES), env(LANG)

These environment variables are used by the **msgcat** package to determine what locale to format messages using.

env(TCL_INTERP_DEBUG_FRAME)

If existing, it has the same effect as running **interp debug {} -frame 1** as the very first command of each new

coreTools Command Reference Index

Tcl interpreter.

errorCode

This variable holds the value of the **-errorCode** return option set by the most recent error that occurred in this interpreter. This list value represents additional information about the error in a form that is easy to process with programs. The first element of the list identifies a general class of errors, and determines the format of the rest of the list. The following formats for **-errorCode** return options are used by the Tcl core; individual applications may define additional formats.

ARITH *code msg*

This format is used when an arithmetic error occurs (e.g. an attempt to divide zero by zero in the **expr** command). *Code* identifies the precise error and *msg* provides a human-readable description of the error. *Code* will be either DIVZERO (for an attempt to divide by zero), DOMAIN (if an argument is outside the domain of a function, such as **acos(-3)**), IOVERFLOW (for integer overflow), OVERFLOW (for a floating-point overflow), or UNKNOWN (if the cause of the error cannot be determined).

Detection of these errors depends in part on the underlying hardware and system libraries.

CHILDKILLED *pid sigName msg*

This format is used when a child process has been killed because of a signal. The *pid* element will be the process's identifier (in decimal). The *sigName* element will be the symbolic name of the signal that caused the process to terminate; it will be one of the names from the include file **signal.h**, such as **SIGPIPE**. The *msg* element will be a short human-readable message describing the signal, such as for **SIGPIPE**.

CHILDSTATUS *pid code*

This format is used when a child process has exited with a non-zero exit status. The *pid* element will be the process's identifier (in decimal) and the *code* element will be the exit code returned by the process (also in decimal).

CHILDSUSP *pid sigName msg*

This format is used when a child process has been suspended because of a signal. The *pid* element will be the process's identifier, in decimal. The *sigName* element will be the symbolic name of the signal that caused the process to suspend; this will be one of the names from the include file **signal.h**, such as **SIGTTIN**. The *msg* element will be a short human-readable message describing the signal, such as for **SIGTTIN**.

NONE

This format is used for errors where no additional information is available for an error besides the message returned with the error. In these cases the **-errorCode** return option will consist of a list containing a single element whose contents are **NONE**.

POSIX *errName msg*

If the first element is **POSIX**, then the error occurred during a POSIX kernel call. The *errName* element will contain the symbolic name of the error that occurred, such as **ENOENT**; this will be one of the values defined in the include file `errno.h`. The *msg* element will be a human-readable message corresponding to *errName*, such as for the **ENOENT** case.

TCL ...

Indicates some sort of problem generated in relation to Tcl itself, e.g. a failure to look up a channel or variable.

To set the **-errorcode** return option, applications should use library procedures such as **Tcl_SetObjErrorCode**, **Tcl_SetReturnOptions**, and **Tcl_PosixError**, or they may invoke the **-errorcode** option of the **return** command. If none of these methods for setting the error code has been used, the Tcl interpreter will reset the variable to **NONE** after the next error.

errorInfo

This variable holds the value of the **-errorinfo** return option set by the most recent error that occurred in this interpreter. This string value will contain one or more lines identifying the Tcl commands and procedures that were being executed when the most recent error occurred. Its contents take the form of a stack trace showing the various nested Tcl commands that had been invoked at the time of the error.

tcl_library

This variable holds the name of a directory containing the system library of Tcl scripts, such as those used for auto-loading. The value of this variable is returned by the **info library** command. See the **library** manual entry for details of the facilities provided by the Tcl script library. Normally each application or package will have its own application-specific script library in addition to the Tcl script library; each application should set a global variable with a name like **\$app_library** (where *app* is the application's name) to hold the network file name for that application's library directory. The initial value of **tcl_library** is set when an interpreter is created by searching several different directories until one is found that contains an appropriate Tcl startup script. If the **TCL_LIBRARY** environment variable exists, then the directory it names is checked first. If **TCL_LIBRARY** is not set or doesn't refer to an appropriate directory, then Tcl checks several other directories based on a compiled-in default location, the location of the binary containing the application, and the current working directory.

tcl_patchLevel

When an interpreter is created Tcl initializes this variable to hold a string giving the current patch level for Tcl, such as **8.4.16** for Tcl 8.4 with the first sixteen official patches, or **8.5b3** for the third

coreTools Command Reference Index

beta release of Tcl 8.5. The value of this variable is returned by the **info patchlevel** command.

tcl_pkgPath

This variable holds a list of directories indicating where packages are normally installed. It is not used on Windows. It typically contains either one or two entries; if it contains two entries, the first is normally a directory for platform-dependent packages (e.g., shared library binaries) and the second is normally a directory for platform-independent packages (e.g., script files). Typically a package is installed as a subdirectory of one of the entries in the **tcl_pkgPath** variable. The directories in the **tcl_pkgPath** variable are included by default in the **auto_path** variable, so they and their immediate subdirectories are automatically searched for packages during **package require** commands. Note: **tcl_pkgPath** is not intended to be modified by the application. Its value is added to **auto_path** at startup; changes to **tcl_pkgPath** are not reflected in **auto_path**. If you want Tcl to search additional directories for packages you should add the names of those directories to **auto_path**, not **tcl_pkgPath**.

tcl_platform

This is an associative array whose elements contain information about the platform on which the application is running, such as the name of the operating system, its current release number, and the machine's instruction set. The elements listed below will always be defined, but they may have empty strings as values if Tcl could not retrieve any relevant information. In addition, extensions and applications may add additional values to the array. The predefined elements are:

byteOrder

The native byte order of this machine: either **littleEndian** or **bigEndian**.

debug

If this variable exists, then the interpreter was compiled with and linked to a debug-enabled C run-time. This variable will only exist on Windows, so extension writers can specify which package to load depending on the C run-time library that is in use. This is not an indication that this core contains symbols.

engine

The name of the Tcl language implementation. When the interpreter is first created, this is always set to the string **Tcl**.

machine

The instruction set executed by this machine, such as **intel**, **PPC**, **68k**, or **sun4m**. On UNIX machines, this is the value returned by **uname -m**.

os

The name of the operating system running on this machine, such as **Windows NT** or **SunOS**. On UNIX

coreTools Command Reference Index

machines, this is the value returned by **uname -s**.

osVersion

The version number for the operating system running on this machine. On UNIX machines, this is the value returned by **uname -r**.

pathSeparator

The character that should be used to **split** PATH-like environment variables into their corresponding list of directory names.

platform

Either **windows**, or **unix**. This identifies the general operating environment of the machine.

pointerSize

This gives the size of the native-machine pointer in bytes (strictly, it is same as the result of evaluating *sizeof(void*)* in C.)

threaded

If this variable exists, then the interpreter was compiled with threads enabled.

user

This identifies the current user based on the login information available on the platform. This value comes from the *getuid()* and *getpwuid()* system calls on Unix, and the value from the *GetUserName()* system call on Windows.

wordSize

This gives the size of the native-machine word in bytes (strictly, it is same as the result of evaluating *sizeof(long)* in C.)

tcl_precision

This variable controls the number of digits to generate when converting floating-point values to strings. It defaults to 0. *Applications should not change this value*; it is provided for compatibility with legacy code.

The default value of 0 is special, meaning that Tcl should convert numbers using as few digits as possible while still distinguishing any floating point number from its nearest neighbours. It differs from using an arbitrarily high value for *tcl_precision* in that an inexact number like 1.4 will convert as 1.4 rather than 1.3999999999999999 even though the latter is nearer to the exact value of the binary number.

If **tcl_precision** is not zero, then when Tcl converts a floating point number, it creates a decimal representation of at most **tcl_precision** significant digits; the result may be shorter if the shorter result represents the original number exactly. If no result of at most **tcl_precision** digits is an exact representation of the original number, the one that is closest to the original number is chosen. If the original number lies precisely between two equally accurate decimal

coreTools Command Reference Index

representations, then the one with an even value for the least significant digit is chosen; for instance, if **tcl_precision** is 3, then 0.3125 will convert to 0.312, not 0.313, while 0.6875 will convert to 0.688, not 0.687. Any string of trailing zeroes that remains is trimmed.

a **tcl_precision** value of 17 digits is for IEEE floating-point in that it allows double-precision values to be converted to strings and back to binary with no loss of information. For this reason, you will often see it as a value in legacy code that must run on Tcl versions before 8.5. It is no longer recommended; as noted above, a zero value is the preferred method.

All interpreters in a thread share a single **tcl_precision** value: changing it in one interpreter will affect all other interpreters as well. Safe interpreters are not allowed to modify the variable.

Valid values for **tcl_precision** range from 0 to 17.

tcl_rcFileName

This variable is used during initialization to indicate the name of a user-specific startup file. If it is set by application-specific initialization, then the Tcl startup code will check for the existence of this file and **source** it if it exists. For example, for **wish** the variable is set to **~/.wishrc** for Unix and **~/wishrc.tcl** for Windows.

tcl_traceCompile

The value of this variable can be set to control how much tracing information is displayed during bytecode compilation. By default, **tcl_traceCompile** is zero and no information is displayed. Setting **tcl_traceCompile** to 1 generates a one-line summary in **stdout** whenever a procedure or top-level command is compiled. Setting it to 2 generates a detailed listing in **stdout** of the bytecode instructions emitted during every compilation. This variable is useful in tracking down suspected problems with the Tcl compiler.

This variable and functionality only exist if **TCL_COMPILE_DEBUG** was defined during Tcl's compilation.

tcl_traceExec

The value of this variable can be set to control how much tracing information is displayed during bytecode execution. By default, **tcl_traceExec** is zero and no information is displayed. Setting **tcl_traceExec** to 1 generates a one-line trace in **stdout** on each call to a Tcl procedure. Setting it to 2 generates a line of output whenever any Tcl command is invoked that contains the name of the command and its arguments. Setting it to 3 produces a detailed trace showing the result of executing each bytecode instruction. Note that when **tcl_traceExec** is 2 or 3, commands such as **set** and **incr** that have been entirely replaced by a sequence of bytecode instructions are not shown. Setting this variable is useful in tracking down suspected problems with the bytecode compiler and interpreter.

This variable and functionality only exist if **TCL_COMPILE_DEBUG** was defined during Tcl's compilation.

tcl_wordchars

The value of this variable is a regular expression that can be set to control what are considered characters, for instances like selecting a word by double-clicking in text in Tk. It is platform dependent. On Windows, it defaults to **\S**, meaning anything but a Unicode space character. Otherwise it defaults to **\w**, which is any Unicode word character (number, letter, or underscore).

tcl_nonwordchars

The value of this variable is a regular expression that can be set to control what are considered characters, for instances like selecting a word by double-clicking in text in Tk. It is platform dependent. On Windows, it defaults to **\s**, meaning any Unicode space character. Otherwise it defaults to **\W**, which is anything but a Unicode word character (number, letter, or underscore).

tcl_version

When an interpreter is created Tcl initializes this variable to hold the version number for this version of Tcl in the form **x.y**. Changes to **x** represent major changes with probable incompatibilities and changes to **y** represent small enhancements and bug fixes that retain backward compatibility. The value of this variable is returned by the **info tclversion** command.

OTHER GLOBAL VARIABLES

The following variables are only guaranteed to exist in **tclsh** and **wish** executables; the Tcl library does not define them itself but many Tcl environments do.

argc The number of arguments to **tclsh** or **wish**.

argv Tcl list of arguments to **tclsh** or **wish**.

argv0 The script that **tclsh** or **wish** started executing (if it was specified) or otherwise the name by which **tclsh** or **wish** was invoked.

tcl_interactive

Contains 1 if **tclsh** or **wish** is running interactively (no script was specified and standard input is a terminal-like device), 0 otherwise.

EXAMPLES

To add a directory to the collection of locations searched by **package require**, e.g., because of some

coreTools Command Reference Index

application-specific packages that are used, the **auto_path** variable needs to be updated:

```
lappend ::auto_path [file join [pwd] "theLibDir"]
```

A simple though not very robust way to handle command line arguments of the form is to load them into an array having first loaded in the default settings:

```
array set arguments {-foo 0 -bar 0 -grill 0}
array set arguments $::argv
puts "foo is $arguments(-foo)"
puts "bar is $arguments(-bar)"
puts "grill is $arguments(-grill)"
```

The **argv0** global variable can be used (in conjunction with the **info script** command) to determine whether the current script is being executed as the main script or loaded as a library. This is useful because it allows a single script to be used as both a library and a demonstration of that library:

```
if {$::argv0 eq [info script]} {
    # running as: tclsh example.tcl
} else {
    package provide Example 1.0
}
```

SEE ALSO

eval(n), library(n), tclsh(1), tkvars(n), wish(1)

KEYWORDS

arithmetic, bytecode, compiler, error, environment, POSIX, precision, subprocess, user, variables

NAME

`tell` - Return current access position for an open channel

SYNOPSIS

```
tell channelId
```

DESCRIPTION

Returns an integer string giving the current access position in *channelId*. This value returned is a byte offset that can be passed to **seek** in order to set the channel to a particular position. Note that this value is in terms of bytes, not characters like **read**. The value returned is -1 for channels that do not support seeking.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of **open** or **socket**, or the result of a channel creation command provided by a Tcl extension.

EXAMPLE

Read a line from a file channel only if it starts with **foobar**:

```
# Save the offset in case we need to undo the read...
set offset [tell $chan]
if {[read $chan 6] eq "foobar"} {
    gets $chan line
}
```

```
} else {  
    set line {}  
    # Undo the read...  
    seek $chan $offset  
}
```

SEE ALSO

file(n), open(n), close(n), gets(n), seek(n),
Tcl_StandardChannels(3)

KEYWORDS

access position, channel, seeking

NAME

`throw` - Generate a machine-readable error

SYNOPSIS

`throw` *type message*

DESCRIPTION

This command causes the current evaluation to be unwound with an error. The error created is described by the *type* and *message* arguments: *type* must contain a list of words describing the error in a form that is machine-readable (and which will form the error-code part of the result dictionary), and *message* should contain text that is intended for display to a human being.

The stack will be unwound until the error is trapped by a suitable **catch** or **try** command. If it reaches the event loop without being trapped, it will be reported through the **bgerror** mechanism. If it reaches the top level of script evaluation in **tclsh**, it will be printed on the console before, in the non-interactive case, causing an exit (the behavior in other programs will depend on the details of how Tcl is embedded and used).

By convention, the words in the *type* argument should go from most general to most specific.

EXAMPLES

The following produces an error that is identical to that produced by **expr** when trying to divide a value by zero.

```
throw {ARITH DIVZERO {divide by zero}} {divide by zero}
```

SEE ALSO

catch(n), error(n), errorCode(n), errorInfo(n),
return(n), try(n)

KEYWORDS

error, exception

NAME

`time` - Time the execution of a script

SYNOPSIS

`time script ?count?`

DESCRIPTION

This command will call the Tcl interpreter *count* times to evaluate *script* (or once if *count* is not specified). It will then return a string of the form

503.2 microseconds per iteration

which indicates the average amount of time required per iteration, in microseconds. Time is measured in elapsed time, not CPU time.

EXAMPLE

Estimate how long it takes for a simple Tcl **for** loop to count to a thousand:

```
time {
    for {set i 0} {$i<1000} {incr i} {
        # empty body
    }
}
```

SEE ALSO

`clock(n)`

KEYWORDS

`script, time`

NAME

timerate - Calibrated performance measurements of script execution time

SYNOPSIS

timerate *script* *?time?* *?max-count?*

timerate *?-direct?* *?-overhead double?* *script* *?time?* *?max-count?*

timerate *?-calibrate?* *?-direct?* *script* *?time?* *?max-count?*

DESCRIPTION

The **timerate** command does calibrated performance measurement of a Tcl command or script, *script*. The *script* should be written so that it can be executed multiple times during the performance measurement process. Time is measured in elapsed time using the finest timer resolution as possible, not CPU time; if *script* interacts with the OS, the cost of that interaction is included. This command may be used to provide information as to how well a script or Tcl command is performing, and can help determine bottlenecks and fine-tune application performance.

The first and second form will evaluate *script* until the interval *time* given in milliseconds elapses, or for 1000 milliseconds (1 second) if *time* is not specified.

The parameter *max-count* could additionally impose a further restriction by the maximal number of iterations to evaluate the script. If *max-count* is specified, the evaluation will stop either this count of iterations is reached or the time is exceeded.

coreTools Command Reference Index

It will then return a canonical tcl-list of the form:

```
0.095977 us/# 52095836 # 10419167 #/sec 5000.000 net-ms
```

which indicates:

- o the average amount of time required per iteration, in microseconds ([**lindex** \$result 0])
- o the count how many times it was executed ([**lindex** \$result 2])
- o the estimated rate per second ([**lindex** \$result 4])
- o the estimated real execution time without measurement overhead ([**lindex** \$result 6])

The following options may be supplied to the **timerate** command:

-calibrate

To measure very fast scripts as exactly as possible, a calibration process may be required. The **-calibrate** option is used to calibrate **timerate** itself, calculating the estimated overhead of the given script as the default overhead for future invocations of the **timerate** command. If the *time* parameter is not specified, the calibrate procedure runs for up to 10 seconds.

Note that calibration is not thread safe in the current implementation.

-overhead *double*

The **-overhead** parameter supplies an estimate (in microseconds) of the measurement overhead of each iteration of the tested script. This quantity will be subtracted from the measured time prior to reporting results. This can be useful for removing the cost of interpreter state reset commands from the script being measured.

-direct

The **-direct** option causes direct execution of the supplied script, without compilation, in a manner similar to the **time** command. It can be used to measure the cost of **Tcl_EvalObjEx**, of the invocation of canonical lists, and of the uncompiled versions of bytecoded commands.

As opposed to the **time** command, which runs the tested script for a fixed number of iterations, the **timerate** command runs it for a fixed time. Additionally, the compiled variant of the script will be used during the entire measurement, as if the script were part of a compiled procedure, if the **-direct** option is not specified. The fixed time period and possibility of compilation allow for more precise results and prevent

coreTools Command Reference Index

very long execution times by slow scripts, making it practical for measuring scripts with highly uncertain execution times.

EXAMPLES

Estimate how fast it takes for a simple Tcl **for** loop (including operations on variable **i**) to count to ten:

```
# calibrate
timerate -calibrate {}

# measure
timerate { for {set i 0} {$i<10} {incr i} {} } 5000
```

Estimate how fast it takes for a simple Tcl **for** loop, ignoring the overhead of the management of the variable that controls the loop:

```
# calibrate for overhead of variable operations
set i 0; timerate -calibrate {expr {$i<10}; incr i} 1000

# measure
timerate {
    for {set i 0} {$i<10} {incr i} {}
} 5000
```

Estimate the speed of calculating the hour of the day using **clock format** only, ignoring overhead of the portion of the script that prepares the time for it to calculate:

```
# calibrate
timerate -calibrate {}

# estimate overhead
set tm 0
set ovh [lindex [timerate {
    incr tm [expr {24*60*60}]
}] 0]

# measure using estimated overhead
set tm 0
timerate -overhead $ovh {
    clock format $tm -format %H
    incr tm [expr {24*60*60}]; # overhead for this is ignored
} 5000
```

SEE ALSO

time(n)

KEYWORDS

performance measurement, script, time

timingException

Item used to model a timing exception command

Description

The timingException item is a coreTool model of a Design Compiler timing exception command.

When you execute any of the coreTool timing exception commands (set_false_path, set_multicycle_path, set_max_delay, set_min_delay, reset_path, or set_disable_timing), the coreTool creates a timingException item to store the command and the data that you specify with the command (start/end points, delay values, and so on). When coreConsultant generates a synthesis strategy for the design, it generates a Design Compiler timing exception command for each timingException item associated with the design.

See Also

[set_false_path \(2\)](#), [set_multicycle_path \(2\)](#), [set_max_delay \(2\)](#), [set_min_delay \(2\)](#), [reset_path \(2\)](#), [set_disable_timing \(2\)](#)

Supported Attributes

[ComponentOfItem \(3\)](#), [Description \(3\)](#), [DocInclude \(3\)](#), [Label \(3\)](#), [Name \(3\)](#), [Sequence \(3\)](#), [TimingExceptionCmd \(3\)](#), [TimingExceptionFall fromValue \(3\)](#), [TimingExceptionFall throughValue \(3\)](#), [TimingExceptionFall toValue \(3\)](#), [TimingExceptionFromValue \(3\)](#), [TimingExceptionGroup path \(3\)](#), [TimingExceptionOptions \(3\)](#), [TimingExceptionRise fromValue \(3\)](#), [TimingExceptionRise throughValue \(3\)](#), [TimingExceptionRise toValue \(3\)](#), [TimingExceptionThroughValue \(3\)](#), [TimingExceptionToValue \(3\)](#), [TimingExceptionValue \(3\)](#), [TypeName \(3\)](#)

NAME

tm - Facilities for locating and loading of Tcl Modules

SYNOPSIS

```
::tcl::tm::path add ?path...?  
::tcl::tm::path remove ?path...?  
::tcl::tm::path list  
::tcl::tm::roots ?path...?
```

DESCRIPTION

This document describes the facilities for locating and loading Tcl Modules (see **MODULE DEFINITION** for the definition of a Tcl Module). The following commands are supported:

```
::tcl::tm::path add ?path...?
```

The paths are added at the head to the list of module paths, in order of appearance. This means that the last argument ends up as the new head of the list.

The command enforces the restriction that no path may be an ancestor directory of any other path on the list. If any of the new paths violates this restriction an error will be raised, before any of the paths have been added. In other words, if only one path argument violates the restriction then none will be added.

If a path is already present as is, no error will be raised and no action will be taken.

Paths are searched later in the order of their appearance in the list. As they are added to the front of the list they are searched in reverse order of addition. In other words, the paths added last are looked at first.

::tcl::tm::path remove *?path...?*

Removes the paths from the list of module paths. The command silently ignores all paths which are not on the list.

::tcl::tm::path list

Returns a list containing all registered module paths, in the order that they are searched for modules.

::tcl::tm::roots *?path...?*

Similar to **path add**, and layered on top of it. This command takes a list of paths, extends each with and for major version **X** of the Tcl interpreter and minor version **y** less than or equal to the minor version of the interpreter, and adds the resulting set of paths to the list of paths to search.

This command is used internally by the system to set up the system-specific default paths.

The command has been exposed to allow a build system to define additional root paths beyond those described by this document.

MODULE DEFINITION

A Tcl Module is a Tcl Package contained in a single file, and no other files required by it. This file has to be **sourceable**. In other words, a Tcl Module is always imported via:

```
source module_file
```

The **load** command is not directly used. This restriction is not an actual limitation, as some may believe. Ever since 8.4 the Tcl **source** command reads only until the first ^Z character. This allows us to combine an arbitrary Tcl script with arbitrary binary data into one file, where the script processes the attached data in any it chooses to fully import and activate the package.

The name of a module file has to match the regular expression:

```
([_[:alpha:]][_[:alnum:]]*)-([[:digit:]]*.*)\.tm
```

The first capturing parentheses provides the name of the package, the second clause its version. In addition to matching the pattern, the extracted version number must not raise an error when used in the command:

```
package vcompare $version 0
```

FINDING MODULES

The directory tree for storing Tcl modules is separate from other parts of the filesystem and independent of **auto_path**.

Tcl Modules are searched for in all directories listed in the result of the command **::tcl::tm::path list**. This is called the *Module path*. Neither the **auto_path** nor the **tcl_pkgPath** variables are used. All directories on the module path have to obey one restriction:

For any two directories, neither is an ancestor directory of the other.

This is required to avoid ambiguities in package naming. If for example the two directories and were on the path a package named **cool::ice** could be found via the names **cool::ice** or **ice**, the latter potentially obscuring a package named **ice**, unqualified.

Before the search is started, the name of the requested package is translated into a partial path, using the following algorithm:

All occurrences of **:** in the package name are replaced by the appropriate directory separator character for the platform we are on. On Unix, for example, this is

Example:

The requested package is **encoding::base64**. The generated partial path is

After this translation the package is looked for in all module paths, by combining them one-by-one, first to last with the partial path to form a complete search pattern. Note that the search algorithm rejects all files where the filename does not match the regular expression given in the section **MODULE DEFINITION**. For the remaining files *provide scripts* are generated and added to the package ifneeded database.

The algorithm falls back to the previous unknown handler when none of the found module files satisfy the request. If the request was satisfied the fall-back is ignored.

Note that packages in module form have *no* control over the *index* and *provide scripts* entered into the package database for them. For a module file **MF** the *index script* is always:

```
package ifneeded PNAME PVERSION [list source MF]
```

and the *provide script* embedded in the above is:

```
source MF
```

Both package name **PNAME** and package version **PVERSION**

coreTools Command Reference Index

are extracted from the filename **MF** according to the definition below:

MF = /module_path/**PNAME'**-**PVERSION**.tm

Where **PNAME'** is the partial path of the module as defined in section **FINDING MODULES**, and translated into **PNAME** by changing all directory separators to and **module_path** is the path (from the list of paths to search) that we found the module file under.

Note also that we are here creating a connection between package names and paths. Tcl is case-sensitive when it comes to comparing package names, but there are filesystems which are not, like NTFS. Luckily these filesystems do store the case of the name, despite not using the information when comparing.

Given the above we allow the names for packages in Tcl modules to have mixed-case, but also require that there are no collisions when comparing names in a case-insensitive manner. In other words, if a package **Foo** is deployed in the form of a Tcl Module, packages like **foo**, **f0o**, etc. are not allowed anymore.

DEFAULT PATHS

The default list of paths on the module path is computed by a **tclsh** as follows, where **X** is the major version of the Tcl interpreter and **y** is less than or equal to the minor version of the Tcl interpreter.

All the default paths are added to the module path, even those paths which do not exist. Non-existent paths are filtered out during actual searches. This enables a user to create one of the paths searched when needed and all running applications will automatically pick up any modules placed in them.

The paths are added in the order as they are listed below, and for lists of paths defined by an environment variable in the order they are found in the variable.

SYSTEM SPECIFIC PATHS

file normalize [info library]/../tclX/X.y

In other words, the interpreter will look into a directory specified by its major version and whose minor versions are less than or equal to the minor version of the interpreter.

For example for Tcl 8.4 the paths searched are:

```
[info library]/../tcl8/8.4
[info library]/../tcl8/8.3
[info library]/../tcl8/8.2
[info library]/../tcl8/8.1
[info library]/../tcl8/8.0
```

coreTools Command Reference Index

This definition assumes that a package defined for Tcl **X.y** can also be used by all interpreters which have the same major number **X** and a minor number greater than **y**.

```
file normalize EXEC/tclX/X.y  
Where EXEC is file normalize [info  
nameofexecutable]/../lib or file normalize  
[::tcl::pkgconfig get libdir, runtime]
```

This sets of paths is handled equivalently to the set coming before, except that it is anchored in **EXEC_PREFIX**. For a build with **PREFIX = EXEC_PREFIX** the two sets are identical.

SITE SPECIFIC PATHS

```
file normalize [info library]/../tclX/site-tcl  
Note that this is always a single entry because X is  
always a specific value (the current major version of  
Tcl).
```

USER SPECIFIC PATHS

```
$::env(TCLX_y_TM_PATH)  
A list of paths, separated by either : (Unix) or ;  
(Windows). This is user and site specific as this  
environment variable can be set not only by the user's  
profile, but by system configuration scripts as well.
```

```
$::env(TCLX.y_TM_PATH)  
Same meaning and content as the previous variable.  
However the use of dot '.' to separate major and minor  
version number makes this name less to non-portable and  
its use is discouraged. Support of this variable has  
been kept only for backward compatibility with the  
original specification, i.e. TIP 189.
```

These paths are seen and therefore shared by all Tcl shells in the **\$::env(PATH)** of the user.

Note that **X** and **y** follow the general rules set out above. In other words, Tcl 8.4, for example, will look at these 10 environment variables:

```
$::env(TCL8.4_TM_PATH) $::env(TCL8_4_TM_PATH)  
$::env(TCL8.3_TM_PATH) $::env(TCL8_3_TM_PATH)  
$::env(TCL8.2_TM_PATH) $::env(TCL8_2_TM_PATH)  
$::env(TCL8.1_TM_PATH) $::env(TCL8_1_TM_PATH)  
$::env(TCL8.0_TM_PATH) $::env(TCL8_0_TM_PATH)
```

SEE ALSO

package(n), Tcl Improvement Proposal #189 (online at <http://tip.tcl.tk/189.html>), Tcl Improvement Proposal #190 (online at <http://tip.tcl.tk/190.html>)

KEYWORDS

modules, package

tmax_shellVariable

Sets application options to the specified value on the specified block.

Settings to be used for Synopsys TestMAX (tmax). The app variables can only be applied at setup and report generation.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This subscripted attribute can be used to issue set_app_var commands in the TestMAX tool.

The attribute subscript indicates the application variable name and the attribute value is the value to assign to the application variable. The subscripts need to follow the following convention

{setup|report}_<application variable name> For example,

"setup-sh_source_logging" would define an application variable

"hdlin.elaborate.option" which is set at setup. The valid insertion points are setup and report.

Examples

Issue the command 'set_app_var sh_source_logging false' at report generation stage

```
coreConsultant> set_design_attribute
{tmax_shellVariable[report-sh_source_logging]} false
```

See Also

tmax_shellVariableComment

Add a comment for the application options being introduced in the tool intent.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

The tmax_shellVariableComment attribute can be used to output a comment to document why the corresponding tmax_shellVariable is being used. The comment will be put into the file where the app variable is set.

Examples

See Also

tmm_shellAppOption

Sets application options to the specified value on the specified block.
Settings to be used for TestMax Manager (tmm_shell).

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

This subscripted attribute can be used to issue set_app_options commands in the TestMax Manager tool each time dft_shell is run.

For application options that should be set before the start of the read design stage, the attribute subscript indicates the application option name and the attribute value is the value to assign to the application option.

Examples

Issue the command 'set_app_options -name design.uniquify_skip_empty_modules -value true' in the generated reference methodology scripts at the start of read constraints stage:

```
coreConsultant> set_design_attribute  
{tmm_shellAppOption[design.uniquify_skip_empty_modules]} true
```

See Also

tmm_shellAppOptionComment

Add a comment for the application options being introduced in the tool intent.

Definition

Type: string

Flags: subscripted

Default value:

Valid subscripts: <valid subscripts are not statically defined>

Default subscript:

Valid on: design_____

Description

The tmm_shellAppOptionComment attribute can be used to output a comment to document why the corresponding tmm_shellAppOption is being used. The comment will be put into the file where the app option is set.

Examples

```
coreConsultant> current_design [get_top_design] coreConsultant>
set_design_attribute
{tmm_shellAppOptionComment[design.uniquify_skip_empty_modules]}
"Overriding app option to true" coreConsultant> set_design_attribute
{tmm_shellAppOption[design.uniquify_skip_empty_modules]} true Generates
the following code fragment in the script
```

Overriding app option to true

```
set_app_options -name design.uniquify_skip_empty_modules -value true
```

See Also

NAME

`trace` - Monitor variable accesses, command usages and command executions

SYNOPSIS

`trace option ?arg arg ...?`

DESCRIPTION

This command causes Tcl commands to be executed whenever certain operations are invoked. The legal *options* (which may be abbreviated) are:

trace add *type name ops ?args?*
Where *type* is **command**, **execution**, or **variable**.

trace add command *name ops commandPrefix*
Arrange for *commandPrefix* to be executed (with additional arguments) whenever command *name* is modified in one of the ways given by the list *ops*. *Name* will be resolved using the usual namespace resolution rules used by commands. If the command does not exist, an error will be thrown.

Ops indicates which operations are of interest, and is a list of one or more of the following items:

rename
Invoke *commandPrefix* whenever the traced command is renamed. Note that renaming to the empty string is considered deletion, and will not be traced with

delete
Invoke *commandPrefix* when the traced command is deleted. Commands can be deleted explicitly by using the **rename** command to rename the command to an empty

coreTools Command Reference Index

string. Commands are also deleted when the interpreter is deleted, but traces will not be invoked because there is no interpreter in which to execute them.

When the trace triggers, depending on the operations being traced, a number of arguments are appended to *commandPrefix* so that the actual command is as follows:

commandPrefix oldName newName op

OldName and *newName* give the traced command's current (old) name, and the name to which it is being renamed (the empty string if this is a operation). *Op* indicates what operation is being performed on the command, and is one of **rename** or **delete** as defined above. The trace operation cannot be used to stop a command from being deleted. Tcl will always remove the command once the trace is complete. Recursive renaming or deleting will not cause further traces of the same type to be evaluated, so a delete trace which itself deletes the command, or a rename trace which itself renames the command will not cause further trace evaluations to occur. Both *oldName* and *newName* are fully qualified with any namespace(s) in which they appear.

trace add execution name ops commandPrefix

Arrange for *commandPrefix* to be executed (with additional arguments) whenever command *name* is executed, with traces occurring at the points indicated by the list *ops*. *Name* will be resolved using the usual namespace resolution rules used by commands. If the command does not exist, an error will be thrown.

Ops indicates which operations are of interest, and is a list of one or more of the following items:

enter

Invoke *commandPrefix* whenever the command *name* is executed, just before the actual execution takes place.

leave

Invoke *commandPrefix* whenever the command *name* is executed, just after the actual execution takes place.

enterstep

Invoke *commandPrefix* for every Tcl command which is executed from the start of the execution of the procedure *name* until that procedure finishes. *CommandPrefix* is invoked just before the actual execution of the Tcl command being reported takes place. For example if we have then an *enterstep* trace would be invoked just before is executed. Setting an *enterstep* trace on a command *name* that does not refer to a procedure will not result in an error and is simply ignored.

leavestep

Invoke *commandPrefix* for every Tcl command which is executed from the start of the execution of the procedure *name* until that procedure finishes. *CommandPrefix* is invoked just after the actual

coreTools Command Reference Index

execution of the Tcl command being reported takes place. Setting a *leavestep* trace on a command *name* that does not refer to a procedure will not result in an error and is simply ignored. When the trace triggers, depending on the operations being traced, a number of arguments are appended to *commandPrefix* so that the actual command is as follows:

For **enter** and **enterstep** operations:

commandPrefix command-string op

Command-string gives the complete current command being executed (the traced command for a **enter** operation, an arbitrary command for a **enterstep** operation), including all arguments in their fully expanded form. *Op* indicates what operation is being performed on the command execution, and is one of **enter** or **enterstep** as defined above. The trace operation can be used to stop the command from executing, by deleting the command in question. Of course when the command is subsequently executed, an error will occur.

For **leave** and **leavestep** operations:

commandPrefix command-string code result op

Command-string gives the complete current command being executed (the traced command for a **enter** operation, an arbitrary command for a **enterstep** operation), including all arguments in their fully expanded form. *Code* gives the result code of that execution, and *result* the result string. *Op* indicates what operation is being performed on the command execution, and is one of **leave** or **leavestep** as defined above. Note that the creation of many **enterstep** or **leavestep** traces can lead to unintuitive results, since the invoked commands from one trace can themselves lead to further command invocations for other traces.

CommandPrefix executes in the same context as the code that invoked the traced operation: thus the *commandPrefix*, if invoked from a procedure, will have access to the same local variables as code in the procedure. This context may be different than the context in which the trace was created. If *commandPrefix* invokes a procedure (which it normally does) then the procedure will have to use **upvar** or **uplevel** commands if it wishes to access the local variables of the code which invoked the trace operation.

While *commandPrefix* is executing during an execution trace, traces on *name* are temporarily disabled. This allows the *commandPrefix* to execute *name* in its body without invoking any other traces again. If an error occurs while executing the *commandPrefix*, then the command *name* as a whole will return that same error.

When multiple traces are set on *name*, then for *enter* and *enterstep* operations, the traced commands are invoked in the reverse order of how the traces were

coreTools Command Reference Index

originally created; and for *leave* and *leavestep* operations, the traced commands are invoked in the original order of creation.

The behavior of execution traces is currently undefined for a command *name* imported into another namespace.

trace add variable *name ops commandPrefix*

Arrange for *commandPrefix* to be executed whenever variable *name* is accessed in one of the ways given by the list *ops*. *Name* may refer to a normal variable, an element of an array, or to an array as a whole (i.e. *name* may be just the name of an array, with no parenthesized index). If *name* refers to a whole array, then *commandPrefix* is invoked whenever any element of the array is manipulated. If the variable does not exist, it will be created but will not be given a value, so it will be visible to **namespace which** queries, but not to **info exists** queries.

Ops indicates which operations are of interest, and is a list of one or more of the following items:

array

Invoke *commandPrefix* whenever the variable is accessed or modified via the **array** command, provided that *name* is not a scalar variable at the time that the **array** command is invoked. If *name* is a scalar variable, the access via the **array** command will not trigger the trace.

read

Invoke *commandPrefix* whenever the variable is read.

write

Invoke *commandPrefix* whenever the variable is written.

unset

Invoke *commandPrefix* whenever the variable is unset. Variables can be unset explicitly with the **unset** command, or implicitly when procedures return (all of their local variables are unset). Variables are also unset when interpreters are deleted, but traces will not be invoked because there is no interpreter in which to execute them.

When the trace triggers, three arguments are appended to *commandPrefix* so that the actual command is as follows:

commandPrefix name1 name2 op

Name1 and *name2* give the name(s) for the variable being accessed: if the variable is a scalar then *name1* gives the variable's name and *name2* is an empty string; if the variable is an array element then *name1* gives the name of the array and *name2* gives the index into the array; if an entire array is being deleted and the trace was registered on the overall array, rather than a single element, then *name1* gives the array name and *name2* is an empty string. *Name1* and *name2* are not necessarily the same as the name used in the **trace**

coreTools Command Reference Index

variable command: the **upvar** command allows a procedure to reference a variable under a different name. *Op* indicates what operation is being performed on the variable, and is one of **read**, **write**, or **unset** as defined above.

CommandPrefix executes in the same context as the code that invoked the traced operation: if the variable was accessed as part of a Tcl procedure, then *commandPrefix* will have access to the same local variables as code in the procedure. This context may be different than the context in which the trace was created. If *commandPrefix* invokes a procedure (which it normally does) then the procedure will have to use **upvar** or **uplevel** if it wishes to access the traced variable. Note also that *name1* may not necessarily be the same as the name used to set the trace on the variable; differences can occur if the access is made through a variable defined with the **upvar** command.

For read and write traces, *commandPrefix* can modify the variable to affect the result of the traced operation. If *commandPrefix* modifies the value of a variable during a read or write trace, then the new value will be returned as the result of the traced operation. The return value from *commandPrefix* is ignored except that if it returns an error of any sort then the traced operation also returns an error with the same error message returned by the trace command (this mechanism can be used to implement read-only variables, for example). For write traces, *commandPrefix* is invoked after the variable's value has been changed; it can write a new value into the variable to override the original value specified in the write operation. To implement read-only variables, *commandPrefix* will have to restore the old value of the variable. While *commandPrefix* is executing during a read or write trace, traces on the variable are temporarily disabled. This means that reads and writes invoked by *commandPrefix* will occur directly, without invoking *commandPrefix* (or any other traces) again. However, if *commandPrefix* unsets the variable then unset traces will be invoked.

When an unset trace is invoked, the variable has already been deleted: it will appear to be undefined with no traces. If an unset occurs because of a procedure return, then the trace will be invoked in the variable context of the procedure being returned to: the stack frame of the returning procedure will no longer exist. Traces are not disabled during unset traces, so if an unset trace command creates a new trace and accesses the variable, the trace will be invoked. Any errors in unset traces are ignored.

If there are multiple traces on a variable they are invoked in order of creation, most-recent first. If one trace returns an error, then no further traces are invoked for the variable. If an array element has a trace set, and there is also a trace set on the array as a whole, the trace on the overall array is invoked before the one on the element.

coreTools Command Reference Index

Once created, the trace remains in effect either until the trace is removed with the **trace remove variable** command described below, until the variable is unset, or until the interpreter is deleted. Unsetting an element of array will remove any traces on that element, but will not remove traces on the overall array.

This command returns an empty string.

trace remove *type name opList commandPrefix*

Where *type* is either **command**, **execution** or **variable**.

trace remove command *name opList commandPrefix*

If there is a trace set on command *name* with the operations and command given by *opList* and *commandPrefix*, then the trace is removed, so that *commandPrefix* will never again be invoked. Returns an empty string. If *name* does not exist, the command will throw an error.

trace remove execution *name opList commandPrefix*

If there is a trace set on command *name* with the operations and command given by *opList* and *commandPrefix*, then the trace is removed, so that *commandPrefix* will never again be invoked. Returns an empty string. If *name* does not exist, the command will throw an error.

trace remove variable *name opList commandPrefix*

If there is a trace set on variable *name* with the operations and command given by *opList* and *commandPrefix*, then the trace is removed, so that *commandPrefix* will never again be invoked. Returns an empty string.

trace info *type name*

Where *type* is either **command**, **execution** or **variable**.

trace info command *name*

Returns a list containing one element for each trace currently set on command *name*. Each element of the list is itself a list containing two elements, which are the *opList* and *commandPrefix* associated with the trace. If *name* does not have any traces set, then the result of the command will be an empty string. If *name* does not exist, the command will throw an error.

trace info execution *name*

Returns a list containing one element for each trace currently set on command *name*. Each element of the list is itself a list containing two elements, which are the *opList* and *commandPrefix* associated with the trace. If *name* does not have any traces set, then the result of the command will be an empty string. If *name* does not exist, the command will throw an error.

trace info variable *name*

Returns a list containing one element for each trace currently set on variable *name*. Each element of the list is itself a list containing two elements, which

coreTools Command Reference Index

are the *opList* and *commandPrefix* associated with the trace. If *name* does not exist or does not have any traces set, then the result of the command will be an empty string.

For backwards compatibility, three other subcommands are available:

trace variable *name ops command*

This is equivalent to **trace add variable** *name ops command*.

trace vdelete *name ops command*

This is equivalent to **trace remove variable** *name ops command*

trace vinfo *name*

This is equivalent to **trace info variable** *name*

These subcommands are deprecated and will likely be removed in a future version of Tcl. They use an older syntax in which **array**, **read**, **write**, **unset** are replaced by **a**, **r**, **w** and **u** respectively, and the *ops* argument is not a list, but simply a string concatenation of the operations, such as **rwua**.

EXAMPLES

Print a message whenever either of the global variables **foo** and **bar** are updated, even if they have a different local name at the time (which can be done with the **upvar** command):

```
proc tracer {varname args} {
    upvar #0 $varname var
    puts "$varname was updated to be \"$var\""
}
trace add variable foo write "tracer foo"
trace add variable bar write "tracer bar"
```

Ensure that the global variable **foobar** always contains the product of the global variables **foo** and **bar**:

```
proc doMult args {
    global foo bar foobar
    set foobar [expr {$foo * $bar}]
}
trace add variable foo write doMult
trace add variable bar write doMult
```

Print a trace of what commands are executed during the processing of a Tcl procedure:

```
proc x {} { y }
proc y {} { z }
proc z {} { puts hello }
proc report args {puts [info level 0]}
trace add execution x enterstep report
x
```

```
-> report y enterstep  
    report z enterstep  
    report {puts hello} enterstep  
    hello
```

SEE ALSO

set(n), unset(n)

KEYWORDS

read, command, rename, variable, write, trace, unset

NAME

transchan - command handler API of channel transforms

SYNOPSIS

`cmdPrefix option ?arg arg ...?`

DESCRIPTION

The Tcl-level handler for a channel transformation has to be a command with subcommands (termed an *ensemble* despite not implying that it must be created with **namespace ensemble create**; this mechanism is not tied to **namespace ensemble** in any way). Note that *cmdPrefix* is whatever was specified in the call to **chan push**, and may consist of multiple arguments; this will be expanded to multiple words in place of the prefix.

Of all the possible subcommands, the handler *must* support **initialize** and **finalize**. Transformations for writable channels must also support **write**, and transformations for readable channels must also support **read**.

Note that in the descriptions below *cmdPrefix* may be more than one word, and *handle* is the value returned by the **chan push** call used to create the transformation.

GENERIC SUBCOMMANDS

The following subcommands are relevant to all types of channel.

cmdPrefix **clear** *handle*

This optional subcommand is called to signify to the transformation that any data stored in internal buffers (either incoming or outgoing) must be cleared. It is called when a **chan seek** is performed on the channel

coreTools Command Reference Index

being transformed.

cmdPrefix **finalize** *handle*

This mandatory subcommand is called last for the given *handle*, and then never again, and it exists to allow for cleaning up any Tcl-level data structures associated with the transformation. *Warning!* Any errors thrown by this subcommand will be ignored. It is not guaranteed to be called if the interpreter is deleted.

cmdPrefix **initialize** *handle mode*

This mandatory subcommand is called first, and then never again (for the given *handle*). Its responsibility is to initialize all parts of the transformation at the Tcl level. The *mode* is a list containing any of **read** and **write**.

write

implies that the channel is writable.

read

implies that the channel is readable.

The return value of the subcommand should be a list containing the names of all subcommands supported by this handler. Any error thrown by the subcommand will prevent the creation of the transformation. The thrown error will appear as error thrown by **chan push**.

READ-RELATED SUBCOMMANDS

These subcommands are used for handling transformations applied to readable channels; though strictly **read** is optional, it must be supported if any of the others is or the channel will be made non-readable.

cmdPrefix **drain** *handle*

This optional subcommand is called whenever data in the transformation input (i.e. read) buffer has to be forced upward, i.e. towards the user or script. The result returned by the method is taken as the *binary* data to push upward to the level above this transformation (the reader or a higher-level transformation).

In other words, when this method is called the transformation cannot defer the actual transformation operation anymore and has to transform all data waiting in its internal read buffers and return the result of that action.

cmdPrefix **limit?** *handle*

This optional subcommand is called to allow the Tcl I/O engine to determine how far ahead it should read. If present, it should return an integer number greater than zero which indicates how many bytes ahead should be read, or an integer less than zero to indicate that the I/O engine may read as far ahead as it likes.

cmdPrefix **read** *handle buffer*

This subcommand, which must be present if the transformation is to work with readable channels, is called whenever the base channel, or a transformation

coreTools Command Reference Index

below this transformation, pushes data upward. The *buffer* contains the binary data which has been given to us from below. It is the responsibility of this subcommand to actually transform the data. The result returned by the subcommand is taken as the binary data to push further upward to the transformation above this transformation. This can also be the user or script that originally read from the channel.

Note that the result is allowed to be empty, or even less than the data we received; the transformation is not required to transform everything given to it right now. It is allowed to store incoming data in internal buffers and to defer the actual transformation until it has more data.

WRITE-RELATED SUBCOMMANDS

These subcommands are used for handling transformations applied to writable channels; though strictly **write** is optional, it must be supported if any of the others is or the channel will be made non-writable.

cmdPrefix flush handle

This optional subcommand is called whenever data in the transformation 'write' buffer has to be forced downward, i.e. towards the base channel. The result returned by the subcommand is taken as the binary data to write to the transformation below the current transformation. This can be the base channel as well.

In other words, when this subcommand is called the transformation cannot defer the actual transformation operation anymore and has to transform all data waiting in its internal write buffers and return the result of that action.

cmdPrefix write handle buffer

This subcommand, which must be present if the transformation is to work with writable channels, is called whenever the user, or a transformation above this transformation, writes data downward. The *buffer* contains the binary data which has been written to us. It is the responsibility of this subcommand to actually transform the data.

The result returned by the subcommand is taken as the binary data to write to the transformation below this transformation. This can be the base channel as well. Note that the result is allowed to be empty, or less than the data we got; the transformation is not required to transform everything which was written to it right now. It is allowed to store this data in internal buffers and to defer the actual transformation until it has more data.

SEE ALSO

`chan(n)`, `refchan(n)`

KEYWORDS

API, channel, ensemble, prefix, transformation

translate_interface_connection

This command takes a connected interface (with SpecifySubsystem completed) and removes the auto connections of the interfaces and replaces them with manual connections all without incompleting SpecifySubsystem.

Syntax

```
string translate_interface_connection -component <component_name>
-interface <interface_name>
    string <component_name>
    string <interface_name>
```

Parameters

-component <component_name>	Component cell name
-interface <interface_name>	Specific interface instance

Description

Examples

```
coreAssembler> translate_interface_connection -component i_s1 -interface
myProvider
```

See Also

[export_interface \(2\)](#), [remove_exported_interface \(2\)](#), [disconnect_interface \(2\)](#), [set_unused_interface \(2\)](#), [create_connection \(2\)](#), [remove_connection \(2\)](#), [export_pin \(2\)](#)

translate_netlist

Translate a specified design from HDL files to IP-XACT.

Syntax

```
string translate_netlist -top <design_name> [-components
<component_names>] -language <language> [-VCSOptions <options>] [-xml
<search_path>] [-out <directory_name>] [-include <directory_names>]
[-ungroupReport <value>] files
    string <design_name>
    list <component_names>
    string <language>
    string <options>
    string <search_path>
    string <directory_name>
    string <directory_names>
    string <value>
    list files
```

Parameters

-top <design_name>	The top-level design from the HDL files.
-components <component_names>	List of component names.
-language <language>	Language of HDL files. (Values: verilog, vhdl, systemverilog)
-VCSOptions <options>	Options for compiling HDL source with VCS
-xml <search_path>	Set component search path and don't write files for components in path.
-out <directory_name>	Output directory.
-include <directory_names>	Directories to search for include files
-ungroupReport <value>	Print report and (optional) error for RTL issues related to RTL restructuring. (Values: none, ReportOnly, ReportError)
files	List of HDL files.

Description

This command is used to create one or more IP-XACT design and component files from a given structural Verilog netlist. It can be used to create a

starting point when converting to an IP-XACT based SoC assembly methodology, given a structural netlist as a starting point. This command generates IP-XACT designs for each specified level of hierarchy and IP-XACT components for the indicated instances, when IP-XACT component files cannot be found using the given search path.

Use the `-components` option to indicate the names of the leaf level components.

Examples

Generate IP-XACT design file from a structural netlist (module top) containing instances of designs D1 and D2. An IP-XACT description for D1 already exists in the XML directory.

```
cA> translate_netlist -top top -components {D1 D2} -xml ./XML
```

See Also

translate_phy_constraints

Translate Hierarchical Pin SDC constraints to be compatible with DW IP

Solution flow

Syntax

```
string translate_phy_constraints SDCFileIn SDCFileOut  
    string SDCFileIn  
    string SDCFileOut
```

Parameters

<i>SDCFileIn</i>	Original SDC file to read
<i>SDCFileOut</i>	SDC file with translations

Description

This command is used to translate SDC files associated with Synopsys MSIP components into a format suitable for use in a Synopsys DesignWare IP Solution product. It is the default translator for this purpose. This command makes the following transformations to an SDC file: add `::PHYSDC::` namespace reference to `create_generated_clock` and `set_clock_sense` commands, replace calls `get get_cell_driver_pin` with calls to `get_pins`, and translate certain reuse-pragmas into values of the `SDCPropagateText` attribute. This command is not meant to be called directly. It is meant to be used as a value for the `SDCConstraints[translator]` attribute in component packaged with an IP-XACT target.

Examples

See Also

[SDCConstraints \(3\)](#)

[API Index](#) [Command Reference](#)

`traverse_boundary`

Traverse a component boundary returning the above/below equivalent connection item.

Syntax

string **traverse_boundary** *object context*

string *object*

string *context*

Parameters

<code>object</code>	Interface instance, pin, port, bit object
<code>context</code>	Returned variable is the object context

Description

Traverse from and interface instance, pin, port, or busBit to the above/below corresponding object. If on a leaf object or top-level port it returns nothing. The direction is determined by the object and its attributes. For example a pin would traverse to the port inside the component the pin is on. For an interface instance the direction is determined by whether it's exported or attached to a cell.

Examples

```
coreAssembler> set endIntf [traverse_boundary $startIntf contextVar]
coreAssembler> set port [traverse_boundary $pin contextVar]
coreAssembler> set pinBit [traverse_boundary $portBit contextVar]
```

See Also

[traverse_connection \(2\)](#)

`traverse_connection`

Traverse a component connections for the connection object.

Syntax

string **traverse_connection** [-monitors] *object*
string *object*

Parameters

<code>-monitors</code>	Include interface monitors
<code>object</code>	Interface instance,pin,port,bit object

Description

Traverse from an object to its connections. The supported object types are interface instance,pin, port, and busBit. If the object has more than one electrical connection it return nothing. For objects that are fully connected to a single item you get the opposite connection. If the object is a busBit then you get all of the connected busBits.

Examples

```
coreAssembler> set conns [traverse_connection $startIntf]
coreAssembler> set conns [traverse_connection $pin]
coreAssembler> set conns [traverse_connection $portBit]
```

See Also

[traverse_boundary \(2\)](#)

NAME

`try` - Trap and process errors and exceptions

SYNOPSIS

`try body ?handler...? ?finally script?`

DESCRIPTION

This command executes the script *body* and, depending on what the outcome of that script is (normal exit, error, or some other exceptional result), runs a handler script to deal with the case. Once that has all happened, if the **finally** clause is present, the *script* it includes will be run and the result of the handler (or the *body* if no handler matched) is allowed to continue to propagate. Note that the **finally** clause is processed even if an error occurs and irrespective of which, if any, *handler* is used.

The *handler* clauses are each expressed as several words, and must have one of the following forms:

on *code variableList script*

This clause matches if the evaluation of *body* completed with the exception code *code*. The *code* may be expressed as an integer or one of the following literal words: **ok**, **error**, **return**, **break**, or **continue**. Those literals correspond to the integers 0 through 4 respectively.

trap *pattern variableList script*

This clause matches if the evaluation of *body* resulted in an error and the prefix of the **-errorcode** from the interpreter's status dictionary is equal to the *pattern*. The number of prefix words taken from the **-errorcode** is equal to the list-length of *pattern*, and inter-word spaces are normalized in both the **-errorcode**

coreTools Command Reference Index

and *pattern* before comparison.

The *variableList* word in each *handler* is always interpreted as a list of variable names. If the first word of the list is present and non-empty, it names a variable into which the result of the evaluation of *body* (from the main **try**) will be placed; this will contain the human-readable form of any errors. If the second word of the list is present and non-empty, it names a variable into which the options dictionary of the interpreter at the moment of completion of execution of *body* will be placed.

The *script* word of each *handler* is also always interpreted the same: as a Tcl script to evaluate if the clause is matched. If *script* is a literal and the *handler* is not the last one, the *script* of the following *handler* is invoked instead (just like with the **switch** command).

Note that *handler* clauses are matched against in order, and that the first matching one is always selected. At most one *handler* clause will be selected. As a consequence, an **on error** will mask any subsequent **trap** in the **try**. Also note that **on error** is equivalent to **trap {}**.

If an exception (i.e. any non-**ok** result) occurs during the evaluation of either the *handler* or the **finally** clause, the original exception's status dictionary will be added to the new exception's status dictionary under the **-during** key.

EXAMPLES

Ensure that a file is closed no matter what:

```
set f [open /some/file/name a]
try {
    puts $f "some message"
    # ...
} finally {
    close $f
}
```

Handle different reasons for a file to not be openable for reading:

```
try {
    set f [open /some/file/name w]
} trap {POSIX EISDIR} {} {
    puts "failed to open /some/file/name: it's a directory"
} trap {POSIX ENOENT} {} {
    puts "failed to open /some/file/name: it doesn't exist"
}
```

SEE ALSO

`catch(n)`, `error(n)`, `return(n)`, `throw(n)`

KEYWORDS

`cleanup`, `error`, `exception`, `final`, `resource management`

NAME

unalias Removes one or more aliases.

SYNTAX

string **unalias**
 patterns

ARGUMENTS

patterns Specifies the patterns to be matched. This argument can contain more than one pattern. Each pattern can be the name of a specific alias to be removed or a pattern containing the wildcard characters * and %, which match one or more aliases to be removed.

DESCRIPTION

The **unalias** command removes aliases created by the **alias** command.

EXAMPLES

The following command removes all aliases.
prompt> **unalias ***

The following command removes all aliases beginning with f, and the alias rt100.
prompt> **unalias f* rt100**

SEE ALSO

`alias(2)`

unconnect_interface
Unconnect interfaces

Syntax

```
string unconnect_interface [-from_component <component 1>]
[-from_interface <interface instance 1>] [-to_component <component 2>]
[-to_interface <interface instance 2>] [-hierarchy]
    string <component 1>
    string <interface instance 1>
    string <component 2>
    string <interface instance 2>
```

Parameters

-from_component <component 1>	the first component name of the interface to be unconnected
-from_interface <interface instance 1>	name of the first interface instance to be unconnected
-to_component <component 2>	the second component name of the interface to be unconnected
-to_interface <interface instance 2>	name of the second interface instance to be unconnected
-hierarchy	Allows hierarchical paths to interfaces.

Description

The unconnect_interface disconnects two interface instances specified. It has the same option and usage as connect_interface command

Examples

See Also

[connect_interface \(2\)](#), [set_unused_interface \(2\)](#),
[translate_interface_connection \(2\)](#)

[API Index](#) [Command Reference](#)

ungroup_component

Ungroup the hierarchical component into current level.

Syntax

```
string ungroup_component instanceName  
    string instanceName
```

Parameters

instanceName	The name of the hierarchical cell.
--------------	------------------------------------

Description

Use this command to remove a level of hierarchy moving the underlying components into the current hierarchical level. The command tries to preserve all connections made through the hierarchical component to the underlying components by reconnecting them at the current level.

Examples

```
ungroup_component myHierCell
```

See Also

[group_components \(2\).](#)

[API Index](#) [Command Reference](#)

ungroup_filegroup
ungroup a filegroup

Syntax

string **ungroup_filegroup** -cell *<item>* -components *<component collection>*
[-debug]
 string *<item>*
 string *<component collection>*

Parameters

-cell <i><item></i>	The old item
-components <i><component collection></i>	The collection of new components
-debug	Print debugging info.

Description

Examples

See Also

union_collection

Combine two collections with set union semantics.

Syntax

```
string union_collection [-equiv <attrName>] col1 col2
    string <attrName>
    string col1
    string col2
```

Parameters

-equiv <attrName>	The attribute list to use for item equivalence. If you do not specify -equiv, union_collection considers two objects with the same name (same value for the Name attribute) to be duplicates.
col1	The collection handle of the first collection to be joined.
col2	The collection handle of the second collection to be joined.

Description

The union_collection command joins two collections together to form a third collection and automatically removes duplicate objects from the new collection. To create the combined collection, union_collection adds <col1> to the new collection, then checks <col2> for any items that are already present in <col1> (duplicates). union_collection then removes duplicate items from <col2> and appends <col2> to <col1> in the new collection. The original <col1> and <col2> remain unchanged.

By default, union_collection removes duplicate objects of the same name. However, you can use the -equiv option to specify other attributes as the basis for determining duplicates, either instead of or in addition to the Name attribute.

Examples

To combine two collections and use the item Name to determine if duplicates exist:

```
coreConsultant> union_collection $col1 $col2
```

To combine two collections of files and use both the file name and the source directory to determine if duplicates exist:

coreBuilder> union_collection -equiv {Name SourceDir} \$f1 \$f2

See Also

add_to_collection(2), remove_from_collection(2)

NAME

unknown - Handle attempts to use non-existent commands

SYNOPSIS

unknown *cmdName* *?arg arg ...?*

DESCRIPTION

This command is invoked by the Tcl interpreter whenever a script tries to invoke a command that does not exist. The default implementation of **unknown** is a library procedure defined when Tcl initializes an interpreter. You can override the default **unknown** to change its functionality, or you can register a new handler for individual namespaces using the **namespace unknown** command. Note that there is no default implementation of **unknown** in a safe interpreter.

If the Tcl interpreter encounters a command name for which there is not a defined command (in either the current namespace, or the global namespace), then Tcl checks for the existence of an unknown handler for the current namespace. By default, this handler is a command named **::unknown**. If there is no such command, then the interpreter returns an error. If the **unknown** command exists (or a new handler has been registered for the current namespace), then it is invoked with arguments consisting of the fully-substituted name and arguments for the original non-existent command. The **unknown** command typically does things like searching through library directories for a command procedure with the name *cmdName*, or expanding abbreviated command names to full-length, or automatically executing unknown commands as sub-processes. In some cases (such as expanding abbreviations) **unknown** will change the original command slightly and then (re-)execute it.

coreTools Command Reference Index

The result of the **unknown** command is used as the result for the original non-existent command.

The default implementation of **unknown** behaves as follows. It first calls the **auto_load** library procedure to load the command. If this succeeds, then it executes the original command with its original arguments. If the auto-load fails then **unknown** calls **auto_execok** to see if there is an executable file by the name *cmd*. If so, it invokes the Tcl **exec** command with *cmd* and all the *args* as arguments. If *cmd* cannot be auto-executed, **unknown** checks to see if the command was invoked at top-level and outside of any script. If so, then **unknown** takes two additional steps. First, it sees if *cmd* has one of the following three forms: **!!**, **!event**, or **^old^new?^?**. If so, then **unknown** carries out history substitution in the same way that **cs**h would for these constructs. Finally, **unknown** checks to see if *cmd* is a unique abbreviation for an existing Tcl command. If so, it expands the command name and executes the command with the original arguments. If none of the above efforts has been able to execute the command, **unknown** generates an error return. If the global variable **auto_noload** is defined, then the auto-load step is skipped. If the global variable **auto_noexec** is defined then the auto-exec step is skipped. Under normal circumstances the return value from **unknown** is the return value from the command that was eventually executed.

EXAMPLE

Arrange for the **unknown** command to have its standard behavior except for first logging the fact that a command was not found:

```
# Save the original one so we can chain to it
rename unknown _original_unknown

# Provide our own implementation
proc unknown args {
    puts stderr "WARNING: unknown command: $args"
    uplevel 1 [list _original_unknown {*}$args]
}
```

SEE ALSO

[info\(n\)](#), [proc\(n\)](#), [interp\(n\)](#), [library\(n\)](#), [namespace\(n\)](#)

KEYWORDS

error, non-existent command, unknown

NAME

`unload` - Unload machine code

SYNOPSIS

```
unload ?switches? fileName  
unload ?switches? fileName packageName  
unload ?switches? fileName packageName interp
```

DESCRIPTION

This command tries to unload shared libraries previously loaded with **load** from the application's address space. *fileName* is the name of the file containing the library file to be unload; it must be the same as the filename provided to **load** for loading the library. The *packageName* argument is the name of the package (as determined by or passed to **load**), and is used to compute the name of the unload procedure; if not supplied, it is computed from *fileName* in the same manner as **load**. The *interp* argument is the path name of the interpreter from which to unload the package (see the **interp** manual entry for details); if *interp* is omitted, it defaults to the interpreter in which the **unload** command was invoked.

If the initial arguments to **unload** start with - then they are treated as switches. The following switches are currently supported:

-nocomplain

Suppresses all error messages. If this switch is given, **unload** will never report an error.

-keeplibrary

This switch will prevent **unload** from issuing the operating system call that will unload the library from

coreTools Command Reference Index

the process.

--

Marks the end of switches. The argument following this one will be treated as a *fileName* even if it starts with a -.

UNLOAD OPERATION

When a file containing a shared library is loaded through the **load** command, Tcl associates two reference counts to the library file. The first counter shows how many times the library has been loaded into normal (trusted) interpreters while the second describes how many times the library has been loaded into safe interpreters. As a file containing a shared library can be loaded only once by Tcl (with the first **load** call on the file), these counters track how many interpreters use the library. Each subsequent call to **load** after the first simply increments the proper reference count.

unload works in the opposite direction. As a first step, **unload** will check whether the library is unloadable: an unloadable library exports a special unload procedure. The name of the unload procedure is determined by *packageName* and whether or not the target interpreter is a safe one. For normal interpreters the name of the initialization procedure will have the form *pkg_Unload*, where *pkg* is the same as *packageName* except that the first letter is converted to upper case and all other letters are converted to lower case. For example, if *packageName* is **foo** or **Foo**, the initialization procedure's name will be **Foo_Unload**. If the target interpreter is a safe interpreter, then the name of the initialization procedure will be *pkg_SafeUnload* instead of *pkg_Unload*.

If **unload** determines that a library is not unloadable (or unload functionality has been disabled during compilation), an error will be returned. If the library is unloadable, then **unload** will call the unload procedure. If the unload procedure returns **TCL_OK**, **unload** will proceed and decrease the proper reference count (depending on the target interpreter type). When both reference counts have reached 0, the library will be detached from the process.

UNLOAD HOOK PROTOTYPE

The unload procedure must match the following prototype:

```
typedef int Tcl_PackageUnloadProc(  
    Tcl_Interp *interp,  
    int flags);
```

The *interp* argument identifies the interpreter from which the library is to be unloaded. The unload procedure must return **TCL_OK** or **TCL_ERROR** to indicate whether or not it completed successfully; in the event of an error it should set the interpreter's result to point to an error message. In this case, the result of the **unload** command will be the result returned by the unload procedure.

The *flags* argument can be either **TCL_UNLOAD_DETACH_FROM_INTERPRETER** or **TCL_UNLOAD_DETACH_FROM_PROCESS**. In case the library will remain attached to the process after the unload procedure returns (i.e. because the library is used by other interpreters), **TCL_UNLOAD_DETACH_FROM_INTERPRETER** will be defined. However, if the library is used only by the target interpreter and the library will be detached from the application as soon as the unload procedure returns, the *flags* argument will be set to **TCL_UNLOAD_DETACH_FROM_PROCESS**.

NOTES

The **unload** command cannot unload libraries that are statically linked with the application. If *fileName* is an empty string, then the *packageName* argument must be specified.

If *packageName* is omitted or specified as an empty string, Tcl tries to guess the name of the package. This may be done differently on different platforms. The default guess, which is used on most UNIX platforms, is to take the last element of *fileName*, strip off the first three characters if they are **lib**, and use any following alphabetic and underline characters as the module name. For example, the command **unload libxyz4.2.so** uses the module name **xyz** and the command **unload bin/last.so {}** uses the module name **last**.

PORTABILITY ISSUES

Unix

Not all unix operating systems support library unloading. Under such an operating system **unload** returns an error (unless **-nocomplain** has been specified).

BUGS

If the same file is **loaded** by different *fileNames*, it will be loaded into the process's address space multiple times. The behavior of this varies from system to system (some systems may detect the redundant loads, others may not). In case a library has been silently detached by the operating system (and as a result Tcl thinks the library is still loaded), it may be dangerous to use **unload** on such a library (as the library will be completely detached from the application while some interpreters will continue to use it).

EXAMPLE

If an unloadable module in the file **foobar.dll** had been loaded using the **load** command like this (on Windows):

```
load c:/some/dir/foobar.dll
```

then it would be unloaded like this:

```
unload c:/some/dir/foobar.dll
```

This allows a C code module to be installed temporarily into a long-running Tcl program and then removed again (either because it is no longer needed or because it is being updated with a new version) without having to shut down the overall Tcl process.

SEE ALSO

info sharedlibextension, load(n), safe(n)

KEYWORDS

binary code, unloading, safe interpreter, shared library

[API Index](#) [Command Reference](#)`unload_autoload_filegroup`

Removes loaded files from an autoloading filegroup

Syntax

string **unload_autoload_filegroup** *group*
string *group*

Parameters

group BOM File group

Description

This command is the opposite of `load_autoload_filegroup`. If a filegroup was loaded with `load_autoload_filegroup`, then that action can be reversed with this command.

Examples

To reset the Documentation group back to autoload status:

```
coreBuilder> unload_autoload_filegroup Documentation
```

See Also

[`create\_autoload\_filegroup \(2\)`](#), [`load\_autoload\_filegroup \(2\)`](#),
[`DefaultLoadPath \(3\)`](#)

[API Index](#) [Command Reference](#)

`unload_component_memory_maps`

Unloads all non-visible component memory maps from knowledgebase files in cC and cA from memory or a specific one if -map option is used.

Syntax

string **unload_component_memory_maps** [-all] [-map <mapname>]
 string <mapname>

Parameters

-all	Removes all the current components memory maps from memory both visible and nonvisible.
-map <mapname>	Forces removal of a single component memMap from memory.

Description

Examples

See Also

[load_component_memory_maps \(2\)](#)

unload_file_from_kb

Write the contents from a file item to the host system.

Syntax

```
string unload_file_from_kb [-output <output_dir>] [-force] [-quiet]
[-rename <new_names>] [-substitute <on_time>] [-context <context_item>]
[<file_items>]
    string <output_dir>
    list <new_names>
    string <on_time>
    string <context_item>
    list <file_items>
```

Parameters

-output <output_dir>	host directory where file item contents should be written
-force	force overwrite of existing files on host system Note that you can not overwrite files for which you do not have write permission.
-quiet	Be quiet when writing files.
-rename <new_names>	new name(s) to give the file(s) after writing to disk Note that this only renames the file written to disk, the file item name stored in the knowledge base remains unchanged.
-substitute <on_time>	time when file substitution is done The value for <on_time> is matched against the subscript used with the SubstituteInFile attribute to determine if text substitution should occur.
-context <context_item>	Context to use when substituting in the file This option should not typically be needed when calling this command, except when used internally to the application.
<file_items>	name(s) of file item(s) to be written to host system

Description

This command writes the given, or selected file items from the knowledge base to the host system, optionally specifying the exact location where the files are to be installed. If the -output option is not used, the files will be written to the directory specified by the defaultInstallDir

attribute on the file item. The file items can be written out with a different name by using the `-rename` option, which renames the file after it has been written to disk. The file item name in the knowledge base remains unchanged.

Examples

To write the file item `foo` to the default installation location:

```
unload_file_from_kb foo
```

To write the currently selected file items and overwrite existing files on the host system with the same name:

```
unload_file_from_kb -force
```

To write the file item `foo` to another location:

```
unload_file_from_kb foo -output ../another
```

To write the file item `foo` and rename it to `hoo`:

```
unload_file_from_kb foo -rename hoo
```

See Also

[load_file_into_kb\(2\).](#)

[API Index](#) [Command Reference](#)

unload_plugin
Unload a plugin KB.

Syntax

string **unload_plugin** *name*
 string *name*

Parameters

name Name of the plugin KB to unload.

Description

The `unload_plugin` command removes a currently loaded plugin knowledge database (KB). A plugin is used to alter the default behavior of a `coreTool` by creating or modifying activities, defining TCL procs, and adding files to the workspace. Unloading a plugin will remove activities and filegroups that are defined by the plugin. TCL procs defined by the plugin are not removed by `unload_plugin`. In `coreAssembler`, a plugin that was associated with a component should be unloaded after setting the current component to that component.

Examples

To unload a plugin KB named `MyPlugIn.kb`:

```
coreConsultant> unload_plugin MyPlugin
```

To unload a plugin in `coreAssembler`:

```
coreAssembler> set_current_component /i_myComp
```

```
coreAssembler> unload_plugin MyPlugin
```

See Also

[create_plugin_kb \(2\)](#), [load_plugin \(2\)](#)

NAME

`unset` - Delete variables

SYNOPSIS

`unset [-nocomplain] [--] ?name name name ...?`

DESCRIPTION

This command removes one or more variables. Each *name* is a variable name, specified in any of the ways acceptable to the **set** command. If a *name* refers to an element of an array then that element is removed without affecting the rest of the array. If a *name* consists of an array name with no parenthesized index, then the entire array is deleted. The **unset** command returns an empty string as result. If **-nocomplain** is specified as the first argument, any possible errors are suppressed. The option may not be abbreviated, in order to disambiguate it from possible variable names. The option **--** indicates the end of the options, and should be used if you wish to remove a variable with the same name as any of the options. If an error occurs during variable deletion, any variables after the named one causing the error are not deleted. An error can occur when the named variable does not exist, or the name refers to an array element but the variable is a scalar, or the name refers to a variable in a non-existent namespace.

EXAMPLE

Create an array containing a mapping from some numbers to their squares and remove the array elements for non-prime numbers:

```
array set squares {
  1 1    6 36
  2 4    7 49
  3 9    8 64
  4 16   9 81
  5 25  10 100
}

puts "The squares are:"
parray squares

unset squares(1) squares(4) squares(6)
unset squares(8) squares(9) squares(10)

puts "The prime squares are:"
parray squares
```

SEE ALSO

set(n), trace(n), upvar(n)

KEYWORDS

remove, variable

NAME

unsetenv Removes a system environment variable.

SYNTAX

```
string getenv  
      variable_name
```

Data Types
 variable_name string

ARGUMENTS

variable_name Specifies the name of the environment variable to be unset.

DESCRIPTION

The **unsetenv** command searches the system environment for the specified *variable_name* and removes variable from the environment. If the variable is not defined in the environment, the command returns a Tcl error. The command is catchable.

Environment variables are stored in the **env** Tcl array variable. The **unsetenv**, commands is a convenience function to interact with this array. It is equivalent to 'unset ::env(*variable_name*)'

The application you are running inherited the initial values for environment variables from its parent process (that is, the shell from which you invoked the application). If you unset the variable using the **unsetenv** command, you remove the variable value in the application and in any new child processes you initiate from the application using the **exec** command. However, the variable is still set in the parent process.

See the **set** and **unset** commands for information about working with non-environment variables.

EXAMPLES

In the following example, **unsetenv** remove the DISPLAY variable from the environment:

```
prompt> getenv DISPLAY
host:0
prompt> unsetenv DISPLAY
prompt> getenv DISPLAY
Error: can't read "::env(DISPLAY)": no such variable
      Use error_info for more info. (CMD-013)
```

SEE ALSO

```
catch(2)
exec(2)
printenv(2)
set(2)
unset(2)
setenv(2)
getenv(2)
```


NAME

unsuppress_message

Enables printing of one or more suppressed informational or suppressed warning messages.

SYNTAX

```
string unsuppress_message [messages]
```

list *messages*

ARGUMENTS

messages A list of messages to enable.

DESCRIPTION

The **unsuppress_message** command provides a mechanism to re-enable the printing of messages which have been suppressed using **suppress_message**. You can suppress only informational and warning messages, so the **unsuppress_message** command is only useful for informational and warning messages. The result of **unsuppress_message** is always the empty string.

You can suppress a given message more than once. So, you must unsuppress a message as many times as it was suppressed in order to enable it. The **print_suppressed_messages** command displays currently suppressed messages.

EXAMPLES

When the argument to the **unalias** command does not match any existing aliases, the CMD-029 warning message displays. This example shows how to re-enable the suppressed CMD-029 message. Assume that there are no

coreTools Command Reference Index

aliases beginning with 'q'.

```
prompt> unalias q*  
prompt> unsuppress_message CMD-029  
prompt> unalias q*  
Warning: no aliases matched 'q*' (CMD-029)
```

SEE ALSO

print_suppressed_messages(2), suppress_message(2).

`unused_interface_instance`

Returns 1 if the given interface instance is not being used.

Syntax

```
string unused_interface_instance instance  
      string instance
```

Parameters

`instance` Name of interface instance to be checked.

Description

This command is available for customizing flows within coreAssembler. It can be used on cores that have mutually exclusive interfaces to determine which interface(s) are not being used. This would typically be used within a plugin to handle some form of processing which was based on the interface(s) actually being used.

Examples

Consider a proc which writes the names of simulation vector files that are applicable to a particular configuration of a core. If the A-interface is being used, the Atest must be run, but if the B-interface is being used, then the Btest should be run instead. The following code fragment could be used to accomplish this:

```
if {[unused_interface_instance A-interface] == 0} {  
  puts $file "Atest"  
} else {  
  puts $file "Btest"  
}
```

See Also

NAME

`update` - Process pending events and idle callbacks

SYNOPSIS

`update ?idletasks?`

DESCRIPTION

This command is used to bring the application by entering the event loop repeatedly until all pending events (including idle callbacks) have been processed.

If the **idletasks** keyword is specified as an argument to the command, then no new events or errors are processed; only idle callbacks are invoked. This causes operations that are normally deferred, such as display updates and window layout calculations, to be performed immediately.

The **update idletasks** command is useful in scripts where changes have been made to the application's state and you want those changes to appear on the display immediately, rather than waiting for the script to complete. Most display updates are performed as idle callbacks, so **update idletasks** will cause them to run. However, there are some kinds of updates that only happen in response to events, such as those triggered by window size changes; these updates will not occur in **update idletasks**.

The **update** command with no options is useful in scripts where you are performing a long-running computation but you still want the application to respond to events such as user interactions; if you occasionally call **update** then user input will be processed during the next call to **update**.

EXAMPLE

Run computations for about a second and then finish:

```
set x 1000
set done 0
after 1000 set done 1
while {!$done} {
    # A very silly example!
    set x [expr {log($x) ** 2.8}]

    # Test to see if our time-limit has been hit. This would
    # also give a chance for serving network sockets and, if
    # the Tk package is loaded, updating a user interface.
    update
}
```

SEE ALSO

after(n), interp(n)

KEYWORDS

asynchronous I/O, event, flush, handler, idle, update

[API Index](#) [Command Reference](#)

update_dut

Update the DUT to the latest version if possible.

Syntax

string **update_dut** [-force] *name*
 string *name*

Parameters

-force	Update, even if it does not appear out of date
name	The name of the DUT.

Description

Use this command to update the DUT in a test bench workspace to the latest version of the DUT. This command is similar to `replace_component`, but it first determines if the DUT is out of date before doing the update.

If the DUT does not appear out of date, then the `-force` option is used to force the update.

Examples

```
coreAssembler> update_dut i_dut
```

See Also

[create_workspace \(2\)](#) [replace_component \(2\)](#), [instantiate_dut \(2\)](#) _____

update_workspace_links
Change symbolic links in a workspace

Syntax

```
string update_workspace_links [-copy] [-moved_install <dir>] [workspace]
    string <dir>
    string workspace
```

Parameters

	Copy all files linked to the installation
-copy	This option is used to remove all symbolic links which go from a workspace into the installation area. This is useful to make the workspace relocatable, particularly for sending the workspace to another location (e.g. customer support). All links from the workspace to the installation area are replaced with copies of the actual files/directories.
-moved_install <dir>	New location of the installation This option is used to indicate that the installation area for the given workspace has been moved from its original location to the directory specified by <dir>. All links from within the workspace to the prior installation area are modified to point into <dir> instead.
workspace	Workspace to change

Description

This command is used to modify the symbolic links within a workspace. The links are either moved (with -moved_install), or replaced with copies of the actual files/directories (with -copy). There are two primary use models for this command. First, to fix broken symbolic links within a workspace when the IP installation directory (or directories in coreAssembler) have moved. In this use case, the -moved_install option is used. The second use model is to prepare a workspace for transfer to another file system. In that use case the -copy option is used to remove all external symbolic links from the workspace.

All relative links pointing inside the workspace are ignored for both moves and copies. For moves, links that do not point to the installation directory are ignored. The copy operation will recursively descend into directories, finding all links as it goes. Multiple passes will be made

to handle the cases where links point to links and where copied directories contain links.

After a linked directory is copied into the workspace, relative links that reference entries outside the copied tree will not resolve. When links that do not resolve are encountered during a move operation, the offending links are reported and the move is aborted, preserving the original workspace. When links that do not resolve are encountered during a copy operation, the offending links are reported, but the copy will continue.

When used in coreAssembler, each component must be processed one at a time, as indicated by using the -component option.

Examples

To remove symbolic links from workspace X:

```
prompt> update_workspace_links -copy X
```

To fix a moved installation directory in coreAssembler:

```
prompt> update_workspace_links -component U1 -moved_install  
/new/install/path
```

See Also

NAME

`uplevel` - Execute a script in a different stack frame

SYNOPSIS

`uplevel ?level? arg ?arg ...?`

DESCRIPTION

All of the *arg* arguments are concatenated as if they had been passed to **concat**; the result is then evaluated in the variable context indicated by *level*. **uplevel** returns the result of that evaluation.

If *level* is an integer then it gives a distance (up the procedure calling stack) to move before executing the command. If *level* consists of **#** followed by a number then the number gives an absolute level number. If *level* is omitted then it defaults to **1**. *Level* cannot be defaulted if the first *command* argument starts with a digit or **#**.

For example, suppose that procedure **a** was invoked from top-level, and that it called **b**, and that **b** called **c**. Suppose that **c** invokes the **uplevel** command. If *level* is **1** or **#2** or omitted, then the command will be executed in the variable context of **b**. If *level* is **2** or **#1** then the command will be executed in the variable context of **a**. If *level* is **3** or **#0** then the command will be executed at top-level (only global variables will be visible).

The **uplevel** command causes the invoking procedure to disappear from the procedure calling stack while the command is being executed. In the above example, suppose **c** invokes the command

coreTools Command Reference Index

```
uplevel 1 {set x 43; d}
```

where **d** is another Tcl procedure. The **set** command will modify the variable **x** in **b**'s context, and **d** will execute at level 3, as if called from **b**. If it in turn executes the command

```
uplevel {set x 42}
```

then the **set** command will modify the same variable **x** in **b**'s context: the procedure **c** does not appear to be on the call stack when **d** is executing. The **info level** command may be used to obtain the level of the current procedure.

Uplevel makes it possible to implement new control constructs as Tcl procedures (for example, **uplevel** could be used to implement the **while** construct as a Tcl procedure).

The **namespace eval** and **apply** commands offer other ways (besides procedure calls) that the Tcl naming context can change. They add a call frame to the stack to represent the namespace context. This means each **namespace eval** command counts as another call level for **uplevel** and **upvar** commands. For example, **info level 1** will return a list describing a command that is either the outermost procedure call or the outermost **namespace eval** command. Also, **uplevel #0** evaluates a script at top-level in the outermost namespace (the global namespace).

EXAMPLE

As stated above, the **uplevel** command is useful for creating new control constructs. This example shows how (without error handling) it can be used to create a **do** command that is the counterpart of **while** except for always performing the test after running the loop body:

```
proc do {body while condition} {
    if {$while ne "while"} {
        error "required word missing"
    }
    set conditionCmd [list expr $condition]
    while {1} {
        uplevel 1 $body
        if {![uplevel 1 $conditionCmd]} {
            break
        }
    }
}
```

SEE ALSO

[apply\(n\)](#), [namespace\(n\)](#), [upvar\(n\)](#)

KEYWORDS

context, level, namespace, stack frame, variable

NAME

`upvar` - Create link to variable in a different stack frame

SYNOPSIS

`upvar` *?level?* *otherVar* *myVar* *?otherVar myVar ...?*

DESCRIPTION

This command arranges for one or more local variables in the current procedure to refer to variables in an enclosing procedure call or to global variables. *Level* may have any of the forms permitted for the **uplevel** command, and may be omitted (it defaults to **1**). For each *otherVar* argument, **upvar** makes the variable by that name in the procedure frame given by *level* (or at global level, if *level* is **#0**) accessible in the current procedure by the name given in the corresponding *myVar* argument. The variable named by *otherVar* need not exist at the time of the call; it will be created the first time *myVar* is referenced, just like an ordinary variable. There must not exist a variable by the name *myVar* at the time **upvar** is invoked. *MyVar* is always treated as the name of a variable, not an array element. An error is returned if the name looks like an array element, such as **a(b)**. *OtherVar* may refer to a scalar variable, an array, or an array element. **Upvar** returns an empty string.

The **upvar** command simplifies the implementation of call-by-name procedure calling and also makes it easier to build new control constructs as Tcl procedures. For example, consider the following procedure:

```
proc add2 name {
```

coreTools Command Reference Index

```
    upvar $name x
    set x [expr {$x + 2}]
}
```

If *add2* is invoked with an argument giving the name of a variable, it adds two to the value of that variable. Although *add2* could have been implemented using **uplevel** instead of **upvar**, **upvar** makes it simpler for *add2* to access the variable in the caller's procedure frame.

namespace eval is another way (besides procedure calls) that the Tcl naming context can change. It adds a call frame to the stack to represent the namespace context. This means each **namespace eval** command counts as another call level for **uplevel** and **upvar** commands. For example, **info level 1** will return a list describing a command that is either the outermost procedure call or the outermost **namespace eval** command. Also, **uplevel #0** evaluates a script at top-level in the outermost namespace (the global namespace).

If an upvar variable is unset (e.g. *x* in *add2* above), the **unset** operation affects the variable it is linked to, not the upvar variable. There is no way to unset an upvar variable except by exiting the procedure in which it is defined. However, it is possible to retarget an upvar variable by executing another **upvar** command.

TRACES AND UPVAR

Upvar interacts with traces in a straightforward but possibly unexpected manner. If a variable trace is defined on *otherVar*, that trace will be triggered by actions involving *myVar*. However, the trace procedure will be passed the name of *myVar*, rather than the name of *otherVar*. Thus, the output of the following code will be rather than

```
proc traceproc { name index op } {
    puts $name
}
proc setByUpvar { name value } {
    upvar $name localVar
    set localVar $value
}
set originalVar 1
trace variable originalVar w traceproc
setByUpvar originalVar 2
```

If *otherVar* refers to an element of an array, then variable traces set for the entire array will not be invoked when *myVar* is accessed (but traces on the particular element will still be invoked). In particular, if the array is **env**, then changes made to *myVar* will not be passed to subprocesses correctly.

EXAMPLE

A **decr** command that works like **incr** except it subtracts the value from the variable instead of adding it:

```
proc decr {varName {decrement 1}} {  
    upvar 1 $varName var  
    incr var [expr {-${decrement}}]  
}
```

SEE ALSO

[global\(n\)](#), [namespace\(n\)](#), [uplevel\(n\)](#), [variable\(n\)](#)

KEYWORDS

context, frame, global, level, namespace, procedure,
upvar, variable

NAME

`variable` - create and initialize a namespace variable

SYNOPSIS

`variable` *name*

`variable` *?name value...?*

DESCRIPTION

This command is normally used within a **namespace eval** command to create one or more variables within a namespace. Each variable *name* is initialized with *value*. The *value* for the last variable is optional.

If a variable *name* does not exist, it is created. In this case, if *value* is specified, it is assigned to the newly created variable. If no *value* is specified, the new variable is left undefined. If the variable already exists, it is set to *value* if *value* is specified or left unchanged if no *value* is given. Normally, *name* is unqualified (does not include the names of any containing namespaces), and the variable is created in the current namespace. If *name* includes any namespace qualifiers, the variable is created in the specified namespace. If the variable is not defined, it will be visible to the **namespace which** command, but not to the **info exists** command.

If the **variable** command is executed inside a Tcl procedure, it creates local variables linked to the corresponding namespace variables (and therefore these variables are listed by **info vars**.) In this way the **variable** command resembles the **global** command, although the **global** command resolves variable names with respect to the global namespace instead of the current

coreTools Command Reference Index

namespace of the procedure. If any *values* are given, they are used to modify the values of the associated namespace variables. If a namespace variable does not exist, it is created and optionally initialized.

A *name* argument cannot reference an element within an array. Instead, *name* should reference the entire array, and the initialization *value* should be left off. After the variable has been declared, elements within the array can be set using ordinary **set** or **array** commands.

EXAMPLES

Create a variable in a namespace:

```
namespace eval foo {  
    variable bar 12345  
}
```

Create an array in a namespace:

```
namespace eval someNS {  
    variable someAry  
    array set someAry {  
        someName someValue  
        otherName otherValue  
    }  
}
```

Access variables in namespaces from a procedure:

```
namespace eval foo {  
    proc spong {} {  
        # Variable in this namespace  
        variable bar  
        puts "bar is $bar"  
  
        # Variable in another namespace  
        variable ::someNS::someAry  
        parray someAry  
    }  
}
```

SEE ALSO

global(n), namespace(n), upvar(n)

KEYWORDS

global, namespace, procedure, variable

verbose_messages

Enables verbose messaging mode.

Syntax

```
boolean verbose_messages = 0
```

Description

verbose_messages is a global variable that enables verbose messaging mode when set to '1'.

Examples

Turn on verbose messages:

```
set ::verbose_messages 1
```

See Also

verify_dwf

Verify the DesignWare Foundation version.

Syntax

```
string verify_dwf [-tool <tool>] [-verbose] [-quiet] min_est_ver
      string <tool>
      string min_est_ver
```

Parameters

-tool <tool>	Check designware version for tool (Values: dc_shell, fpga_shell)
-verbose	Print extra informational messages to the screen.
-quiet	Suppress all error and warning messages.
min_est_ver	The minimum acceptable version of EST DWF specified as YYMM.

Description

Check the DesignWare Foundation version of SYNOPSYS. The SYNOPSYS environment variable must be set.

NOTE: The min_est_ver argument is always expressed in YYMM format. The content of \$SYNOPSYS/dw/version may sometimes be in MMY format. For these cases, the format of the DesignWare Foundation version will be temporarily changed to YYMM prior to comparing it with the min_est_ver argument.

Here are the cases in which \$SYNOPSYS/dw/version is in (the atypical) MMY format:

- The content of \$SYNOPSYS/dw/version ends in DWF_est_####
- The content of \$SYNOPSYS/dw/version ends in DWF_0102

Return Status

1: The SYNOPSYS environment variable is set and the designware version associated with it is greater than or equal to the minimum version of DC (if supplied) or if that fails, the EST version is greater than or equal to the minimum version of EST.

0: Otherwise

Examples

For this example, SYNOPSIS is set to 2001.08 and has an EST version of 0206:

```
coreConsultant> verify_dwf 0102
1
coreConsultant> verify_dwf 0102 -verbose
Information: The DesignWare Foundation EST version is 0206
with SYNOPSIS set to /d/dwt1/synopsys/2001.08. (CMDS-222)
1
coreConsultant> verify_dwf 0207
Error: DesignWare Foundation EST version 0206
is older than the minimum limit: 0207. (CMDS-220)
0
coreConsultant> verify_dwf 0207 -quiet
0
coreConsultant> verify_dwf 0207 -tool dc_shell -verbose
Information: The DC version of SYNOPSIS is greater than or equal to
2001.08.
SYNOPSIS is set to /d/dwt1/synopsys/2001.08. (CMDS-222)
1
```

See Also

[verify_environment \(2\)](#), [EnvCheck \(3\)](#)

verify_environment

Verify the user's environment using the EnvCheck attribute.

Syntax

string **verify_environment** [-verbose] [-quiet]

Parameters

- verbose Print extra informational messages to the screen.
- quiet Suppress all error and warning messages.

Description

The verify_environment proc is called from coreConsultant, gets the subscribed values of the EnvCheck design attribute, and calls the appropriate tool(s). Refer to the EnvCheck manpage for a description and usage examples.

If the "-quiet" option is set, then all tools are called with the "-quiet" option. If the "-verbose" option is set and "-quiet" is not set, then all tools are called with the "-verbose" option and the global variable ::verify_environment_verbose, is set and can be referenced by the custom procs.

Return Status

- 1: All of the tools that were called returned good status.
- 0: Otherwise

Example custom proc:

```
# Proc to search the PATH environment variable for a string.
#
proc search_path { str } {
  if {[info exists ::verify_environment_verbose]} {
    echo "Global variable '::verify_environment_verbose' is set."
  }
  return [expr {[string first $str $::env(PATH)] != -1}]
}
```

Examples

```
coreConsultant> verify_environment -verbose
Information: Current design is simple. (DES-35)
1
coreConsultant> setenv SYNOPSYS /d/dwt1/synopsys/2000.11
```

coreTools Command Reference Index

```
coreConsultant> set_design_attribute -designs [current_design]
{EnvCheck[env_vars]} "VCS_HOME SYNOPSIS"
coreConsultant> set_design_attribute -designs [current_design]
{EnvCheck[executables]} "ls pwd"
coreConsultant> set_design_attribute -designs [current_design]
{EnvCheck[tools]} {{vcs 4.1 9.0} {dwf 0301 2001.08} vx1}
coreConsultant> append ::env(PATH) :/usr/local/lsf/bin
coreConsultant> set_design_attribute -designs [current_design]
{EnvCheck[custom]} "search_path /usr/local/lsf/bin"
coreConsultant> verify_environment
Information: Current design is simple. (DES-35)
Error: Environment variable is not set: VCS_HOME (CMDS-220)
Error: VCS_HOME environment variable is not set. (CMDS-220)
Error: Executable 'verilog' not found in PATH environment variable.
(CMDS-220)
Error: verify_environment received failure status from:
'check_env_vars "VCS_HOME SYNOPSIS"' (CMDS-220)
Error: verify_environment received failure status from:
'verify_tool vcs -min 4.1 -max 9.0' (CMDS-220)
Error: verify_environment received failure status from:
'verify_tool vx1' (CMDS-220)
0
coreConsultant> verify_environment -quiet
0
coreConsultant> verify_environment -verbose
Information: Current design is simple. (DES-35)
Error: Environment variable is not set: VCS_HOME (CMDS-220)
Information: Value of SYNOPSIS is /d/dwt1/synopsis/2000.11 (CMDS-222)
Information: Found pwd at /usr/local/bin/pwd (CMDS-222)
Information: Found ls at /usr/bin/ls (CMDS-222)
Global variable '::verify_environment_verbose' is set.
Error: VCS_HOME environment variable is not set. (CMDS-220)
Information: The DesignWare Foundation EST version is 0801
with SYNOPSIS set to /d/dwt1/synopsis/2000.11. (CMDS-222)
Error: Executable 'verilog' not found in PATH environment variable.
(CMDS-220)
Error: verify_environment received failure status from:
'check_env_vars "VCS_HOME SYNOPSIS" -verbose' (CMDS-220)
Error: verify_environment received failure status from:
'verify_tool vcs -min 4.1 -max 9.0 -verbose' (CMDS-220)
Error: verify_environment received failure status from:
'verify_tool vx1 -verbose' (CMDS-220)
Information: verify_environment received success status from:
'check_executables "ls pwd" -verbose' (CMDS-222)
Information: verify_environment received success status from:
'search_path /usr/local/lsf/bin' (CMDS-222)
Information: verify_environment received success status from:
'verify_dwf 0301 -min_dc_ver 2001.08 -verbose' (CMDS-222)
0
coreConsultant> setenv VCS_HOME /d/vcs60/vcs6.0
coreConsultant> setenv PATH [join "$::env(PATH)
```

```
/d/cds_ldv32/solaris/tools.sun4v/bin" :]
coreConsultant> setenv CDS_LIC_FILE 5280@venice:5280@void:5280@grymoire
coreConsultant> verify_environment
Information: Current design is simple. (DES-35)
Warning: Cannot find '/d/cds_ldv32/solaris/tools/lib' in LD_LIBRARY_PATH
environment variable. (CMDS-221)
Warning: LD_LIBRARY_PATH will be updated for this session only.
Please update LD_LIBRARY_PATH to permanently include
'/d/cds_ldv32/solaris/tools/lib' (CMDS-221)
1
coreConsultant> verify_environment -quiet
1
coreConsultant> verify_environment -verbose
Information: Current design is simple. (DES-35)
Information: Value of SYNOPSIS is /d/dwt1/synopsys/2000.11 (CMDS-222)
Information: Value of VCS_HOME is /d/vcs60/vcs6.0 (CMDS-222)
Information: Found pwd at /usr/local/bin/pwd (CMDS-222)
Information: Found ls at /usr/bin/ls (CMDS-222)
Global variable 'verify_environment_verbose' is set.
Information: INFO for tool 'vcs': (CMDS-222)
Information: The value of VCS_HOME is /d/vcs60/vcs6.0 (CMDS-222)
Information: The path to the tool is /d/vcs60/vcs6.0/bin/vcs (CMDS-222)
Information: The tool version is 6.0 (CMDS-222)
Information: The minimum acceptable version is 4.1 (CMDS-222)
Information: The maximum acceptable version is 9.0 (CMDS-222)
Information: The DesignWare Foundation EST version is 0801
with SYNOPSIS set to /d/dwt1/synopsys/2000.11. (CMDS-222)
Information: INFO for tool 'vxl': (CMDS-222)
Information: The path to the tool is
/d/cds_ldv32/solaris/tools/bin/verilog (CMDS-222)
Information: The tool version is 3.20.p001 (CMDS-222)
Information: verify_environment received success status from:
'check_env_vars "VCS_HOME SYNOPSIS" -verbose' (CMDS-222)
Information: verify_environment received success status from:
'check_executables "ls pwd" -verbose' (CMDS-222)
Information: verify_environment received success status from:
'search_path /usr/local/lfs/bin' (CMDS-222)
Information: verify_environment received success status from:
'verify_tool vcs -min 4.1 -max 9.0 -verbose' (CMDS-222)
Information: verify_environment received success status from:
'verify_dwf 0301 -min_dc_ver 2001.08 -verbose' (CMDS-222)
Information: verify_environment received success status from:
'verify_tool vxl -verbose' (CMDS-222)
1
```

See Also

[EnvCheck \(3\)](#), [check_env_vars \(2\)](#), [check_executables \(2\)](#), [verify_dwf \(2\)](#), [verify_tool \(2\)](#)

verify_tool

Verify the proper environment variable (if it has one; otherwise, verify that the tool's executable is in PATH), version and licensing for a tool.

Syntax

```
string verify_tool [-min <min_ver>] [-max <max_ver>] [-verbose] [-quiet]
[-getversion <variable name>] tool
    string <min_ver>
    string <max_ver>
    string <variable name>
    string tool
```

Parameters

-min <min_ver>	The minimum acceptable version of the tool.
-max <max_ver>	The maximum acceptable version of the tool.
-verbose	Print extra informational messages to the screen.
-quiet	Suppress all error and warning messages.
-getversion <variable name>	Return current version of tool in indicated variable.
<i>tool</i>	The name of the tool to be verified. The currently supported values for tool are vcs, vcsi, dc_shell, vera, mti, vxl, vnc, mti_vlog, mti_vhdl, nc_vlog, nc_vhdl, cc, gcc, fm_shell, pt_shell, synplify, mvtools, gca_shell, spyglass, verdi, vivado, focus The mti argument will do the same checks as the mti_vlog argument. The vnc argument will do the same checks as the nc_vlog argument.

Description

This command verifies that a tool is set up properly.

Return Status

1 : The tool verifies correctly by meeting the following requirements:

- The tool's environment variable is set (if it has one) or the tool's executable is in PATH.
- The tool was found using the environment variable above or by searching the PATH environment variable.

coreTools Command Reference Index

- The tool has the correct version.
- The tool is properly licensed.
- If -getversion is specified with a Tcl variable the value of the variable will be set to the current tool version.

0: Otherwise

Note: For cc and gcc, successful status is returned under the following conditions:

- The tool was found by searching the PATH environment variable.
- The tool runs successfully when invoked with simple input.
- The path to cc is not /usr/ucb.

Note: For vip:<component>, successful status is returned if:

- The DESIGNWARE_HOME environment variable is set.
- The component was found in the DESIGNWARE_HOME/vip directory.
- The component has at least one entry with the syntax of a version string.
- At least one component version is within the min/max limits if the -min and/or -max version arguments are specified.

Examples

```
coreConsultant> verify_tool vcs -min 2015.09 -max 2017.03
1
coreConsultant> verify_tool vcs -min 2015.09 -max 2017.03 -verbose
Information: INFO for tool 'vcs': (CMDS-222)
Information: The value of VCS_HOME is /d/vcs_2017.03 (CMDS-222)
Information: The path to the tool is /d/vcs_2017.03/bin/vcs (CMDS-222)
Information: The tool version is 2017.03 (CMDS-222)
Information: The minimum acceptable version is 2015.09 (CMDS-222)
Information: The maximum acceptable version is 2017.03 (CMDS-222)
1
coreConsultant> verify_tool vcs -min 2015.09 -max 2016.06 -verbose
Error: vcs version 2017.03 is newer than the maximum limit of 2016.06
(CMDS-220)
Information: INFO for tool 'vcs': (CMDS-222)
Information: The value of VCS_HOME is /d/vcs_2017.03 (CMDS-222)
Information: The path to the tool is /d/vcs_2017.03/bin/vcs (CMDS-222)
Information: The tool version is 2017.03 (CMDS-222)
Information: The minimum acceptable version is 2015.09 (CMDS-222)
Information: The maximum acceptable version is 2016.06 (CMDS-222)
0
coreConsultant> verify_tool vcs -min 2015.09 -max 2016.06
Error: vcs version 2017.03 is newer than the maximum limit of 2016.06
(CMDS-220)
0
```

```
coreConsultant> verify_tool vcs -min 2015.09 -max 2016.06 -quiet
0
coreConsultant> verify_tool -tool vcs -getversion version
coreConsultant> set version
2017.03
```

See Also

[verify_environment \(2\)](#), [EnvCheck \(3\)](#)

`visible_fields`

Formula that can be used on a register to determine if there are visible fields.

Syntax

string **visible_fields** *register*
 string *register*

Parameters

register Register to check for visible fields

Description

This command is defined to be used as a formula, most likely for setting the Visible attribute on a register. It returns 1 if the given register has visible register fields, otherwise it returns 0. It's usage enables a register to become not-visible when all of its fields are not visible.

Examples

Set Visible on register R1 such that it is not visible when its fields are not visible. Note that the passing of item is automatically translated by the tool into a collection containing the register on which the attribute is set.

```
prompt> set_register_attribute MapName/BlockName/R1 Visible  
{=visible_fields %item}
```

Check at the command line if register R2 has visible fields:

```
prompt> visible_fields [find_item -type register R2]
```

See Also

NAME

`vwait` - Process events until a variable is written

SYNOPSIS

`vwait` *varName*

DESCRIPTION

This command enters the Tcl event loop to process events, blocking the application if no events are ready. It continues processing events until some event handler sets the value of the global variable *varName*. Once *varName* has been set, the `vwait` command will return as soon as the event handler that modified *varName* completes. The *varName* argument is always interpreted as a variable name with respect to the global namespace, but can refer to any namespace's variables if the fully-qualified name is given.

In some cases the `vwait` command may not return immediately after *varName* is set. This happens if the event handler that sets *varName* does not complete immediately. For example, if an event handler sets *varName* and then itself calls `vwait` to wait for a different variable, then it may not return for a long time. During this time the top-level `vwait` is blocked waiting for the event handler to complete, so it cannot return either. (See the **NESTED VWAITS BY EXAMPLE** below.)

To be clear, *multiple vwait calls will nest and will not happen in parallel*. The outermost call to `vwait` will not return until all the inner ones do. It is recommended that code should never nest `vwait` calls (by avoiding putting them in event callbacks) but when that is not possible, care should be taken to add interlock

coreTools Command Reference Index

variables to the code to prevent all reentrant calls to **vwait** that are not *strictly* necessary. Be aware that the synchronous modes of operation of some Tcl packages (e.g., **http**) use **vwait** internally; if using the event loop, it is best to use the asynchronous callback-based modes of operation of those packages where available.

EXAMPLES

Run the event-loop continually until some event calls **exit**. (You can use any variable not mentioned elsewhere, but the name *forever* reminds you at a glance of the intent.)

```
vwait forever
```

Wait five seconds for a connection to a server socket, otherwise close the socket and continue running the script:

```
# Initialise the state
after 5000 set state timeout
set server [socket -server accept 12345]
proc accept {args} {
    global state connectionInfo
    set state accepted
    set connectionInfo $args
}

# Wait for something to happen
vwait state

# Clean up events that could have happened
close $server
after cancel set state timeout

# Do something based on how the vwait finished...
switch $state {
    timeout {
        puts "no connection on port 12345"
    }
    accepted {
        puts "connection: $connectionInfo"
        puts [lindex $connectionInfo 0] "Hello there!"
    }
}
```

A command that will wait for some time delay by waiting for a namespace variable to be set. Includes an interlock to prevent nested waits.

```
namespace eval example {
    variable v done
    proc wait {delay} {
        variable v
        if {$v ne "waiting"} {
            set v waiting
            after $delay [namespace code {set v done}]
        }
    }
}
```

coreTools Command Reference Index

```
        vwait [namespace which -variable v]
    }
    return $v
}
}
```

When running inside a **coroutine**, an alternative to using **vwait** is to **yield** to an outer event loop and to get recommenced when the variable is set, or at an idle moment after that.

```
coroutine task apply {{} {
    # simulate [after 1000]
    after 1000 [info coroutine]
    yield

    # schedule the setting of a global variable, as normal
    after 2000 {set var 1}

    # simulate [vwait var]
    proc updatedVar {task args} {
        after idle $task
        trace remove variable ::var write "updatedVar $task"
    }
    trace add variable ::var write "updatedVar [info coroutine]"
    yield
}}
```

NESTED VWAITS BY EXAMPLE

This example demonstrates what can happen when the **vwait** command is nested. The script will never finish because the waiting for the **a** variable never finishes; that **vwait** command is still waiting for a script scheduled with **after** to complete, which just happens to be running an inner **vwait** (for **b**) even though the event that the outer **vwait** was waiting for (the setting of **a**) has occurred.

```
after 500 {
    puts "waiting for b"
    vwait b
    puts "b was set"
}
after 1000 {
    puts "setting a"
    set a 10
}
puts "waiting for a"
vwait a
puts "a was set"
puts "setting b"
set b 42
```

If you run the above code, you get this output:

```
waiting for a
waiting for b
setting a
```

The script will never print until after it has printed because of the nesting of **vwait** commands, and yet **b**

coreTools Command Reference Index

will not be set until after the outer **vwait** returns, so the script has deadlocked. The only ways to avoid this are to either structure the overall program in continuation-passing style or to use **coroutine** to make the continuations implicit. The first of these options would be written as:

```
after 500 {
    puts "waiting for b"
    trace add variable b write {apply {args {
        global a b
        trace remove variable ::b write \
            [lrange [info level 0] 0 1]
        puts "b was set"
        set ::done ok
    }}}
}
after 1000 {
    puts "setting a"
    set a 10
}
puts "waiting for a"
trace add variable a write {apply {args {
    global a b
    trace remove variable a write [lrange [info level 0] 0 1]
    puts "a was set"
    puts "setting b"
    set b 42
}}}
vwait done
```

The second option, with **coroutine** and some helper procedures, is done like this:

```
# A coroutine-based wait-for-variable command
proc waitvar globalVar {
    trace add variable ::$globalVar write \
        [list apply {{v c args} {
            trace remove variable $v write \
                [lrange [info level 0] 0 3]
            after 0 $c
        }} ::$globalVar [info coroutine]]
    yield
}

# A coroutine-based wait-for-some-time command
proc waittime ms {
    after $ms [info coroutine]
    yield
}

coroutine task-1 eval {
    puts "waiting for a"
    waitvar a
    puts "a was set"
    puts "setting b"
    set b 42
}

coroutine task-2 eval {
    waittime 500
    puts "waiting for b"
    waitvar b
    puts "b was set"
```

coreTools Command Reference Index

```
    set done ok
  }
  coroutine task-3 eval {
    waittime 1000
    puts "setting a"
    set a 10
  }
  vwait done
```

SEE ALSO

global(n), update(n)

KEYWORDS

asynchronous I/O, event, variable, wait

wait_activities_on_grid

Wait for all or a specified set of activities started with launch_activities_on_grid to complete.

Syntax

string **wait_activities_on_grid** [-wait_list] [-all]
string

Parameters

-wait_list	Wait on given activities of launch_activities_on_grid session. It supports launch name and actname as list of list,ex {<lname> {actlist} <lname1> {all}}
-all	Wait on all the launch_activities_on_grid session

Description

This command is used to wait for activities launched using launch_activities_on_grid. It will block until all designated activities are complete.

The command has two options:

-wait_list : list containing key value pairs of launch_name and list of activities or "all" for all activities

-all : waits on all the activities

If both the switches are given then -all will override the -wait_list

Examples

Wait until completion of the activities launched on grid using launch_activities_on_grid command.

Ex1) Wait on random activity

```
coreConsultant> launch_activities_on_grid "FormalVerification Synthesize
StaticTimingAnalysis runSpyglass" -name launch1 -syn_dir dir1
coreConsultant> launch_activities_on_grid "FormalVerification Synthesize
StaticTimingAnalysis GenerateTestVectors " -name launch2 -syn_dir dir2
coreConsultant> wait_activities_on_grid -wait_list {launch1 { Synthesize
FormalVerification } launch2 { GenerateTestVectors }}
```

Information: Launched jobs completed

Launch Name	launch1
Synthesize	Job Id: 4439110

coreTools Command Reference Index

```
FormalVerification | Job Id: 4439117  
Completed !!
```

```
Launch Name | launch2  
GenerateTestVectors | Job Id: 4439302  
Completed !!  
(CMDS-243)
```

Ex2) Single Wait

```
coreConsultant> launch_activities_on_grid "FormalVerification Synthesize  
StaticTimingAnalysis DW_apb_rap_Simulate" -name launch1 -syn_dir dir1  
coreConsultant> wait_activities_on_grid -wait_list {launch1  
{DW_apb_rap_Simulate}}  
Information: Launched jobs completed  
Launch Name | launch1  
DW_apb_rap_Simulate | Job Id: 4441436  
Completed !!  
(CMDS-243)
```

Ex3) Wait on all

```
coreConsultant> launch_activities_on_grid "FormalVerification Synthesize  
StaticTimingAnalysis" -name launch1 -syn_dir dir1  
coreConsultant> launch_activities_on_grid "FormalVerification Synthesize  
StaticTimingAnalysis " -name launch2 -syn_dir dir2  
coreConsultant> wait_activities_on_grid -all  
Information: Launched jobs completed  
Launch Name | launch1  
StaticTimingAnalysis | Job Id: 4439844  
Synthesize | Job Id: 4439843  
runSpyglass | Job Id: 4439120  
FormalVerification | Job Id: 4439847  
Completed !!  
  
Launch Name | launch2  
GenerateTestVectors | Job Id: 4439302  
FormalVerification | Job Id: 4440081  
StaticTimingAnalysis | Job Id: 4440080  
Synthesize | Job Id: 4440079  
Completed !!  
(CMDS-243)
```

Ex4) Wait on selected activities of one launch and on all activities of other launch

```
coreConsultant> launch_activities_on_grid "FormalVerification Synthesize  
StaticTimingAnalysis runSpyglass" -name launch1 -syn_dir dir1  
coreConsultant> launch_activities_on_grid "FormalVerification  
StaticTimingAnalysis GenerateTestVectors " -name launch2 -syn_dir dir2  
coreConsultant> wait_activities_on_grid -wait_list {launch1 { Synthesize  
FormalVerification } launch2 { all }}  
Information: Launched jobs completed  
Launch Name | launch1  
Synthesize | Job Id: 4442056  
FormalVerification | Job Id: 4442058  
Completed !!
```


coreTools Command Reference Index

```
Launch Name | launch2
GenerateTestVectors | Job Id: 4439302
FormalVerification | Job Id: 4440081
StaticTimingAnalysis | Job Id: 4440080
Synthesize | Job Id: 4440079
Completed !!
(CMDS-243)
```

See Also

launch activities on grid (2) get launch_activities status (2)

`wait_for_activity_subproc`

Wait for an activity sub-process started with `launch_activity_subproc` to complete.

Syntax

```
string wait_for_activity_subproc [-component <component>] activity
      string <component>
      string activity
```

Parameters

<code>-component</code>	Component that the activity exists in
<code><component></code>	
<code>activity</code>	Activity or filegroup associated with the command

Description

Wait for an activity sub-process to stop or be killed. Waiting is accomplished by polling for the process/job status every 30 seconds. For jobs run with RunStyle 'local' or 'remote', this means checking the status of the running process using the 'ps' command. For jobs run with RunStyle 'uge' or 'lsf', checking is done using the qstat or bjobs, respectively.

Note that calling `wait_for_activity_subproc` immediately after the `launch_activity_subproc` command can lead to inconsistent results if the job scheduler has not yet scheduled a uge or lsf job. It is suggested that you use the `PostGrdDelay` activity parameter to force the launch command to wait for a while before returning to prevent this issue. The `PostGrdDelay` activity parameter indicates how many milliseconds of delay to incur before returning. A value of 2000 or 3000 should be sufficient.

Examples

```
wait_for_activity_subproc Synthesize
```

See Also

[launch_activity_subproc \(2\)](#), [define_activity_subproc_params \(2\)](#),
[report_activity_subproc \(2\)](#), [prereq_activities_complete \(2\)](#)

NAME

which Locates a file and displays its
pathname.

SYNTAX

```
string which filename_list  
list filename_list
```

ARGUMENTS

filename_list List of files to locate.

DESCRIPTION

Displays the location of the specified files. This command uses the `search_path` to find the location of the files. This command can be a useful prelude to `read_db` or `link_design`, because it shows how these commands expand filenames. The **which** command can be used to verify that a file exists in the system.

If an absolute pathname is given, the command searches for the file in the given path and returns the full pathname of the file.

EXAMPLES

The following examples are based on the following `search_path`.

```
prompt> set search_path "/u/foo  
/u/foo/test"
```

The following command searches for the file name `foo1`

coreTools Command Reference Index

in the `search_path`.

```
prompt> which foo1  
/u/foo/foo1
```

The following command searches for files `foo2`, `foo3`.

```
prompt> which {foo2 foo3}  
/u/foo/test/foo2 /u/foo/test/foo3
```

The following command returns the full pathname.

```
prompt> which ~/test/designs/sub_design.db  
/u/foo/test/designs/sub_design.db
```

SEE ALSO

link_design(2), read_db(2), search_path(3).

NAME

`while` - Execute script repeatedly as long as a condition is met

SYNOPSIS

`while test body`

DESCRIPTION

The **while** command evaluates *test* as an expression (in the same way that **expr** evaluates its argument). The value of the expression must be a proper boolean value; if it is a true value then *body* is executed by passing it to the Tcl interpreter. Once *body* has been executed then *test* is evaluated again, and the process repeats until eventually *test* evaluates to a false boolean value. **Continue** commands may be executed inside *body* to terminate the current iteration of the loop, and **break** commands may be executed inside *body* to cause immediate termination of the **while** command. The **while** command always returns an empty string.

Note: *test* should almost always be enclosed in braces. If not, variable substitutions will be made before the **while** command starts executing, which means that variable changes made by the loop body will not be considered in the expression. This is likely to result in an infinite loop. If *test* is enclosed in braces, variable substitutions are delayed until the expression is evaluated (before each loop iteration), so changes in the variables will be visible. For an example, try the following script with and without the braces around **\$x<10**:

```
set x 0
while {$x<10} {
```

```
    puts "x is $x"
    incr x
}
```

EXAMPLE

Read lines from a channel until we get to the end of the stream, and print them out with a line-number prepended:

```
set lineCount 0
while {[gets $chan line] >= 0} {
    puts "[incr lineCount]: $line"
}
```

SEE ALSO

`break(n)`, `continue(n)`, `for(n)`, `foreach(n)`

KEYWORDS

`boolean`, `loop`, `test`, `while`

write_ParameterInfo

Write the configuration intent for a set of parameters out in the format of a template for use as a starting point when generating ParameterInfo.

Syntax

string **write_ParameterInfo** [-all] *paramList*
list *paramList*

Parameters

-all	Write all parameter configuration intent If this option is not specified, only group-related info will be written.
paramList	List of params to be edited You can specify a single parameter, a list of parameters, or specify the parameters as a Tcl collection. Wildcards can be used, it finds the parameters with a glob style match with the Name of the parameter.

Description

The write_ParameterInfo command writes the configuration intent for a set of parameters out in the format of a template. This is used as a starting point when generating ParameterInfo. ParameterInfo is a Tcl list of list where each element in the list is a statement-value pair which contains structural information for the parameter configuration dialog.

Examples

The following command writes out all parameter configuration parameterInfo for current design parameters:

```
coreBuilder> set params [get_attribute [current_design] -attr Parameters]
coreBuilder> setwrite_ParameterInfo $params -all
{LayoutAs tab/groupbox/spreadsheet}
{MaxParamCount 15}
{Text {put the text here}}
{Label {put the label here}}
{GroupType subparam/layout }
{BoxType outline/indent }
{Visible {<param_expr> } }
{ColumnAttrs {Name} {Value}}
{ColumnLabels {label1} {label2} }
```

{Parameters {P1 P2 P3 P4} }

{Group Group1

{Parameters {P5} }

}

See Also

[ParameterInfo \(3\), check_ParameterInfo \(2\)](#)

NAME

write_app_var Writes a script to set the current variable values.

SYNTAX

```
string write_app_var  
    -output file  
    [-all | -only_changed_vars]  
    [pattern]
```

Data Types

<i>file</i>	string
<i>pattern</i>	string

ARGUMENTS

-output *file* Specifies the file to which to write the script.

-all Writes the default values in addition to the current values of the variables.

-only_changed_vars Writes only the changed variables. This is the default when no options are specified.

pattern Writes the variables that match the specified *pattern*. The default is "*".

DESCRIPTION

The **write_app_var** command generates a Tcl script to set all application variables to their current values. By default, variables set to their default values are not included in the script. You can force the default values to be included by specifying the **-all** option.

EXAMPLES

The following is an example of the `write_app_var` command:

```
prompt> write_app_var -output sh_settings.tcl sh*
```

SEE ALSO

`get_app_var(2)`
`report_app_var(2)`
`set_app_var(2)`

write_attribute_table

Write an attribute table file.

Syntax

```
string write_attribute_table [-name <table name>] [-append] [-separator
<table column separator>] [-comment_indicator <table comment indicator>]
[-map <list of column name map pairs>] [-columns <list of column names>]
-items <list of items> [-attributes <list of attributes>] [-defaults]
[-string_delimiter <table text delimiter>] [-formulas] <filename>
    string <table name>
    string <table column separator>
    string <table comment indicator>
    list <list of column name map pairs>
    list <list of column names>
    list <list of items>
    list <list of attributes>
    string <table text delimiter>
    string <filename>
```

Parameters

-name <table name>	Name of table to be written to the file.
-append	Append to the specified file if present.
-separator <table column separator>	The character used to separate columns in the table file.
-comment_indicator <table comment indicator>	The character used to indicate a comment line in the table file.
-map <list of column name map pairs>	List of column mapped name table name pairs. This option can be used to map from the names desired in the table to the names expected by the table writing command. This enables user-defined column names within the written tables. The <column map> is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table writing command.
-columns <list of column names>	List of column names which should be skipped from writing in table. Indicates a list of columns whose values are expected to be identical across all table rows and which should therefore be excluded from the table. If the table writer determines that

coreTools Command Reference Index

	the values are not identical, then table writing fails.
-items <list of items>	List of item name whose attributes are to be written. This can be a list of item names or collections or both.
-attributes <list of attributes>	List of attribute names whose values are to be written.
-defaults	Include default attribute values for writing.
-string_delimiter <table text delimiter>	The character used to delimit strings in the table.
-formulas	Write formulas instead of evaluated values. This option writes unevaluated formula if attribute value is a formula. Only those formulas which are explicitly set by user are written and not the default formulas. To write default formulas as well, pass -default with -formulas
<filename>	The name of table file to write.

Description

This command writes attribute data in table format to the indicated file. The resulting table will include a row for each specified item and a column for each specified attribute, plus columns for the object name and type (unless removed with columns). If there are multiple types of objects in the list to be written, only those attributes that apply to an object type will be written for a given object. If the defaults option is not specified, attributes whose value matches their default will not be written.

Examples

Following command will write AddressOffset, RegisterSize and VolatileMemory attribute for all registers into file attrTable.txt, under table name Reg_Attr

```
coreBuilder> write_attribute_table attrTable.txt -name Reg_Attr \  
-items [find_item -type register] \  
-attribute { AddressOffset RegisterSize VolatileMemory } \  
-columns { TypeName } -separator ,  
Items can be specified as a list of collections and names as given below
```

```
coreBuilder> write_attribute_table attrTable.txt -name RegField_Attr \  
-items [list [find_item -type registerField] reg1 reg2 ] \  
-attribute { AddressOffset RegisterSize VolatileMemory } \  
-map {{ Offset AddressOffset } { Size RegisterSize } }
```

{Volatile VolatileMemory}} \

-separator @ -append

See Also

read_attribute_table(2), read_pin_connection_table(2),
write_pin_connection_table(2), read_subsystem_table(2),
write_subsystem_table(2)

write_automotive_synthesis_files

Export the Automotive Synthesis files

Syntax

string **write_automotive_synthesis_files** [-target <tool name>] -distance
<VAL1> <VAL2> -intent <TMR or DCLS> [-file <fileName>] [-returnEnabled]
 string <tool name>
 string <VAL1> <VAL2>
 string <TMR or DCLS>
 string <fileName>

Parameters

-target <tool name>	Target tool for Synthesis file (dc_shell fc_shell)
-distance <VAL1> <VAL2>	Separation distance <VAL1> <VAL2>
-intent <TMR or DCLS>	Generate the intent for TMR or DCLS
-file <fileName>	Name of the generated intent file
-returnEnabled	returns the source commands

Description

Examples

See Also

write_batch_script

Generate a script to recreate this workspace

Syntax

string **write_batch_script** [-exclude] [-include] [-create_workspace]
[-design_intent] [-activities] [-default_params] *file*
string *file*

Parameters

-exclude	Include everything, except for the specified options
-include	Only include the specified options
-create_workspace	Include or exclude the create_workspace command
-design_intent	Include or exclude design intent in script
-activities	Include or exclude activity completion commands
-default_params	Include default parameter value settings
<i>file</i>	Filename for generated script

Description

This command is used to generate a batch script that can be used to recreate a workspace, either partially, or completely. By default, it will generate a script that will recreate the workspace up to the state it is in at the time write_batch_script is called. If desired, certain portions of information can be excluded, using the -exclude option, or only certain portions can be generated, using the -include option.

This command is useful to see how things done using the GUI can be recreated in batch mode, and for generating intent command files when intent is entered interactively.

In coreAssembler the default mode of operation is to write the entire subsystem. However, if you use the '-current' switch then the batch script will only be written for the current component and it's sub-components. This creates a script that can be used to recreate a portion of a design in another location as the generated script is completely relocatable.

Examples

To write a script to recreate the entire workspace:

```
prompt> write_batch_script workspace.tcl
```

To write a script that contains only the design intent data:

```
prompt> write_batch_script workspace.tcl -include -design_intent
```

See Also

write_component_register_headers

Write Component Register Headers files for component

Syntax

```
string write_component_register_headers [-outputDir <dir>] [-noprefix]
[-language <all,c,verilog>]
    string <dir>
    string <all,c,verilog>
```

Parameters

-outputDir <dir>	Output Directory. Defaults to current directory
-noprefix	Do not prepend the design prefix
-language <all,c,verilog>	Register headers language format. Both c and verilog formats are written out by default.

Description

This command is a public API to generate Component Register Header files for the component. The output directory into which the file is to be written can be specified (defaults to pwd) using the -outputDir argument. The header file formats can also be specified using the -language argument.

Examples

To write out Component Register Headers in the export/headers directory in the current workspace:

```
write_component_register_headers -outputDir [get_logical_dir
export/headers]
```

To write out Component Register Headers in C format in the current directory:

```
write_component_register_headers -language c
```

To write out Component Register Headers in the scratch directory in the current workspace:

```
write_component_register_headers -outputDir [get_logical_dir scratch]
```

See Also

`write_config_tar_file`

Write out a coreConsultant configuration tar file.

Syntax

string `write_config_tar_file`

Description

This command generates a configuration tar file for use with coreConfiguration. The tar file can be opened by the coreConfiguration tool to allow users to specify the design configuration for a component and write it out as a batch file for use in coreConsultant. The only activity available to users in coreConfiguration is the specify configuration activity.

Examples

To create a configuration tar file:

```
prompt> write_config_tar_file
```

See Also

`write_failure_modes_file`
To export SGDC files

Syntax

string **write_failure_modes_file** [-tool *FC|DC|TMAX|spyglass*] [-modes *modes*]

string *FC|DC|TMAX|spyglass*
string *modes*

Parameters

-tool	Defines the tool where the file is to be examined
<i>FC DC TMAX spyglass</i>	
-modes <i>modes</i>	Flexibility to the user to write out specific failure modes

Description

It exports the ports details with their failure mode attributes per instance. By default it only extracts the port details of the top instance. It also works on all the design in an IP but, **IncludePortsInElabDesign** attribute all has to be set for every instance, whose port details needs to be exported. The file generated is present in the export directory of the workspace.

By default the name of the file generated is <design_name>.DC.sgdc. If the file is to be generated for any other tool except DC, -tool has to be populated with the same. The file getting generated can be seen in the export directory of the workspace. You can use [**get_logical_dir export**] to get the path of the export directory.

The Content of the sgdc file looks like:

```
current_design bar
fusa_failure_mode -name <failure_mode> -full_design -fault_type
<fault_type> -observe_point <port_name>
```

Examples

Simple generation of SGDC file:

```
coreConsultant > get_top_design
bar
```

coreTools Command Reference Index

```
coreConsultant > write_failure_modes_file -mode "M6 G7"
```

Information: Wrote file /abcd/Workspace1/export/bar.DC.sgdc to disk.

Generating file for Specific tool:

```
coreConsultant > write_failure_modes_file -tool FC
```

Information: Wrote file /abcd/Workspace1/export/bar.FC.sgdc to disk.

See Also

[IncludePortsInElabDesign \(2\)](#)

[API Index](#) [Command Reference](#)

`write_ipxact_abstractor`

Export an IP-XACT description of the current abstractor.

Syntax

```
string write_ipxact_abstractor [-file <output file>] [-directory  
<directory>] [-compact]  
    string <output file>  
    string <directory>
```

Parameters

<code>-file <output file></code>	Name of the output XML file
<code>-directory <directory></code>	Directory for generated files
<code>-compact</code>	Exclude descriptive information

Description

Generates an IP-XACT format (`<spirit:abstractor>`) file for the current abstractor component. If the current component is not an abstractor, the command will fail. The `write_ipxact_component` command should be use for writing non-abstractor components.

Examples

See Also

[write_ipxact_component](#)

write_ipxact_component

Export an IP-XACT description of the current component.

Syntax

```
string write_ipxact_component [-hierarchy] [-file <output file>]
[-directory <directory>] [-prefix <file prefix>] [-replacement <string>]
[-compact] [-copy] [-bom] [-constraints] [-vendor_extensions]
[-unconfigured] [-reservedFields] [-memory_map_only]
    string <output file>
    string <directory>
    string <file prefix>
    string <string>
```

Parameters

-hierarchy	Include component hierarchy
-file <output file>	Name of the output XML file
-directory <directory>	Directory for generated files
-prefix <file prefix>	File prefix to be replaced
-replacement <string>	Replacement string for file prefix
-compact	Exclude descriptive information
-copy	Copy non-IP-XACT component source to output directory
-bom	Include all files in the BoM (coreBuilder only)
-constraints	Include constraints (coreBuilder only)
-vendor_extensions	Include Synopsys vendorExtensions (coreBuilder only)
-unconfigured	Allows writing of IP-XACT files for unconfigured designs
-reservedFields	Define all bits in a register and write out reserved fields
-memory_map_only	write out ipxact only for memory maps.

Description

Generates an IP-XACT format (<spirit:component>) file from the current subsystem. By default only the top-level component is written out to file <design>.xml. The output file name can be overridden with the -file option.

If you specify the `-hierarchy` option then all components are written recursively. In this case you can only specify the directory in which the output files should be stored. If this is not specified, the current directory is used.

If you specify the `-memory_map_only` option, only memoryMaps are written for the components.

Examples

Generate a simple top-level component file:

```
coreAssembler> write_ipxact_component
```

Generates the components file that includes only memoryMap information.

```
coreAssembler> write_ipxact_component -memory_map_only
```

Write the entire subsystem of components into directory H:

```
coreAssembler> write_ipxact_component -directory H -recursive
```

See Also

[read_ipxact_file\(2\)](#), [write_ipxact_design\(2\)](#),
[write_ipxact_designConfiguration\(2\)](#)

write_ipxact_design

Export an IP-XACT description of the current design.

Syntax

string **write_ipxact_design** [-hierarchy] [-file <output file>] [-directory <directory>]

string <output file>

string <directory>

Parameters

-hierarchy	Include component hierarchy
-file <output file>	Name of the output XML file
-directory <directory>	Directory for generated files

Description

Generates an IP-XACT format (<spirit:design>) file from the current subsystem. By default only the top-level is written out to file <design>.xml. The output file name can be overridden with the -file option.

If you specify the -hierarchy option then all components are written recursively. In this case you can only specify the directory in which the output files should be stored. If this is not specified, the current directory is used. Note that the top design is written as an IP-XACT design, but all lower level designs are written as IP-XACT components.

Examples

Generate a simple top-level design file:

```
coreAssembler> write_ipxact_design
```

Write the entire subsystem (design + components) into directory H:

```
coreAssembler> write_ipxact_design -directory H -recursive
```

read_ipxact_file (2), write_ipxact_component (2),

write_ipxact_designConfiguration (2)

write_ipxact_designConfiguration

Export an IP-XACT design configuration file.

Syntax

string **write_ipxact_designConfiguration** [-file <output file>] [-directory <directory>]

string <output file>

string <directory>

Parameters

-file <output file>	Name of the output XML file
-directory <directory>	Directory for generated files

Description

Generates an IP-XACT format (<spirit:designConfiguration>) file from the current subsystem. This file contains a reference to the current IP-XACT design, and references to all currently loaded IP-XACT 'pmd' and 'generatorChain' files.

By default, the file will be written to <top design>_config.xml in the current directory. If you specify a directory using -directory, it must already exist.

Design configuration files can be read back into coreAssembler using the read_ipxact_file command.

Examples

Write a designConfigurationFile:

```
write_ipxact_designConfiguration -file config.xml
```

See Also

[write_ipxact_design \(2\)](#), [write_ipxact_component \(2\)](#), [read_ipxact_file \(2\)](#)

write_ipxact_typeDefinitions

Export an IP-XACT description of the current typeDefinition.

Syntax

```
string write_ipxact_typeDefinitions [-file <output file>] [-directory  
<directory>] [-VLNV <VLNV>]  
    string <output file>  
    string <directory>  
    string <VLNV>
```

Parameters

-file <output file>	Name of the output XML file
-directory <directory>	Directory for generated files
-VLNV <VLNV>	VLVN of typeDefinition to write

Description

Examples

See Also

write_pin_connection_table
Write a pin connection table file.

Syntax

```
string write_pin_connection_table [-name <table name>] [-append]
[-separator <table column seperator>] [-map <list of column name map
pairs>] [-columns <list of column name value pairs>] [-recursive]
[-comment_indicator <table comment indicator>] [-string_delimiter <table
text delimiter>] <filename>
    string <table name>
    string <table column seperator>
    list <list of column name map pairs>
    list <list of column name value pairs>
    string <table comment indicator>
    string <table text delimiter>
    string <filename>
```

Parameters

-name <table name>	The name of the table to be written.
-append	Append to existing table.
-separator <table column seperator>	The character used to separate columns in the table file.
-map <list of column name map pairs>	List of column mapped name table name pairs. This option can be used to map from the names desired in the table to the names expected by the table writing command. This enables user-defined column names within the written tables. The <column map> is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table writing command.
-columns <list of column name value pairs>	List of column name and value pairs to insert into write out table. Indicates a list of columns whose values are expected to be identical across all table rows and which should therefore be excluded from the table. If the table writer determines that the values are not identical, then table writing fails.
-recursive	Write out the commands for all hierarchies.
-comment_indicator <table comment>	The character used to indicate a comment line in the table file.

indicator>	
-string_delimiter	
<table text	The character used to delimit strings in the table.
delimiter>	
<filename>	The name of the table file to written.

Description

Pin connection commands are written to the indicated <table file>. By default, connection commands are only written for the current context. Specifying the -recursive option will cause commands to be written for all hierarchies.

Examples

Write the current set of manual (pin-to-pin) connections. The connections are written to the pinConnectionTable.txt under the 'pin_table' section and the column separator used is ','.

```
cA> write_pin_connection_table -name pin_table -separator ,
pinConnectionTable.txt
```

Specifying the '-recursive' option will cause commands to be written for all hierarchies below the current hierarchy.

```
cA> write_pin_connection_table -recursive -name pin_table
pinConnectionTable.txt
```

See Also

read_pin_connection_table (2), read_attribute_table (2),
write_attribute_table (2), read_subsystem_table (2),
write_subsystem_table (2)

write_ral_file

Write RAL (Register Abstraction Layer) file for component

Syntax

```
string write_ral_file [-outputDir ] [-fileName ] [-allbits] [-instance ]
[-uvm] [-enableFieldSuppression] [-writeAlternateRegister] [-base_address
<string>]
    string
    string <string>
```

Parameters

-outputDir	Output Directory
-fileName	File Name
-allbits	Define all bits in a register
-instance	Specifies the backdoor top path to be appended. Appends a top level path at the RAL block if this string is provided.
-uvm	enables writing of uvm related info
-enableFieldSuppression	enable field-level test suppression
-writeAlternateRegister	writes the alternate register information
-base_address <string>	Provide a custom base address for component RAL file

Description

This command outputs a Register Abstraction Language (RAL) file for the component. You must specify the output directory into which the file is to be written. If you do not specify a filename for the output file using the -fileName switch, the generated file will be given the name <component_name>.ralf.

If the '-allbits' option is specified, all undefined bit ranges in a register are also documented in the RAL file.

If the '-enableFieldSuppression' option is specified, test suppression occurs at the field level instead of at the register level. This allows for better test coverage as an untestable field will not make an entire register untestable. This option is only supported for VMM RAL when using VCS 2012.09 or later, and for UVM RAL when using VCS 2014.12-SP3 or later.

In coreAssembler the instance path of a component is added to any backdoor path references in the system section. To change this behaviour you can specify a value "." for the -instance argument to indicate that the file should be written relative to the current context.

Examples

To write out a RAL description of your component to the current directory:

```
coreConsultant> write_ral_file -outputDir .
```

To write out a RAL description of your component to the current directory with a specific file name:

```
coreConsultant> write_ral_file -outputDir . -fileName  
my_ral_file_name.ralf
```

To document, defined as well as undefined bit ranges of registers of memory map, in RAL File:

```
coreConsultant> write_ral_file -outputDir ralDir -fileName ralFile.ralf  
-allbits
```

```
coreAssembler> write_ral_file -fileName ralFile.ralf -base_address  
0x400000
```

See Also

write_sdc

Writes out a script in Synopsys Design Constraints (SDC) format.

Syntax

```
string write_sdc [-include <constraint_list>] [-exclude
<constraint_list>] [-use_clk_expr] [-relative] [-force] [-upf]
[-compress] [-hierpinsdc] [-target <tool name>] [-version version]
[-split] file_name
    list <constraint_list>
    string <tool name>
    string version
    string file_name
```

Parameters

-include <constraint_list>	include only the indicated constraints
-exclude <constraint_list>	exclude the indicated constraints
-use_clk_expr	Convert constraint values with percent_of_period to an expression using clock-period variable
-relative	Convert time values to be relative to associated clock period
-force	Force writing of SDC file even if the TechSetup activity is not complete
-upf	Choose drive/load cells for ports consistent with their power domain.
-compress	Compress away identical bit-level constraints.
-hierpinsdc	Write constraints for HierPinSDC flow. May be non-standard SDC.
-target <tool name>	Target tool for SDC file (Values: dc, fc, spyglass)
-version version	Specify the SDC output version (default 2.1). (Values: 2.0, 2.1)
-split file_name	Split SDC file into multiple functional files Output file name

Description

The write_sdc command writes out coreBuilder constraints for the current design into a script file in Synopsys Design Constraints (SDC) format.

coreTools Command Reference Index

The writer is currently writing SDC version 2.0. SDC formatted script files are Tcl scripts that use a subset of the commands supported by PrimeTime and Design Compiler. They are read into supporting tools using the `read_sdc` command.

The `-split` option causes the file the SDC constraints to be partitioned into multiple files (clock constraints, port constraints, design constraints, etc.). The top-level SDC file the sources these files.

The `-upf` option is used to indicate to the writer that drive and load cells chosen for ports should take their respective power domains into account. This only has an impact if the `PowerDomain` attribute is set on the ports and libraries with multiple voltages are being used.

The `-target` option can be used to account for minor differences between how different tools support SDC. By default the `'dc'` (for Design Compiler) format is used. The value `'fc'` (for Fusion Compiler) can be used to write an SDC file that is compliant with Fusion Compiler. The value `'spglass'` is for targeting an SDC file to the SpyGlass tool.

Using the `-compress` option causes the resulting SDC file to be compressed where possible. When constraints are set identically on each bit of a bus, the constraint is written as defined on the bussed port instead of on the individual bits.

The `-use_clk_expr` option causes the SDC file to be written using variable references for clock periods and uncertainties instead of hard coded values. This can be useful when manually editing the file for constraint updates.

The `-relative` option can be used with `-use_clk_expr` to force all clock and I/O delay constraints to be written relative to the clock period even when the original constraint was not specified that way.

The `-include` and `-exclude` options are used to write specialized SDC files containing a subset of information. You can use `-include` to specify which attributes should be included in the SDC file or `-exclude` to specify which attributes should be excluded. An understanding of the mapping between coreTools attribute names and the corresponding SDC command is required to use these options.

The `-physdc` option is available specifically for use with the PHYSDC package which supports deferred specification of pin references for `create_generated_clock` and `set_clock_sense` commands. Please consult technical support for more details.

Examples

The following command writes out coreBuilder constraints to file `top.sdc`:

coreBuilder> write_sdc top.sdc

See Also

read_sdc(2),

write_subsystem_table

Write a subsystem table file.

Syntax

```
string write_subsystem_table [-name <table name>] [-append] [-separator
<table column separator>] [-comment_indicator <table comment indicator>]
[-map <list of column name map pairs>] [-columns <list of column names>]
[-types <list of types>] [-string_delimiter <table text delimiter>]
<filename>
    string <table name>
    string <table column separator>
    string <table comment indicator>
    list <list of column name map pairs>
    list <list of column names>
    list <list of types>
    string <table text delimiter>
    string <filename>
```

Parameters

-name <table name>	Name of table to be written to the file.
-append	Append to the specified file if present.
-separator <table column separator>	The character used to separate columns in the table file.
-comment_indicator <table comment indicator>	The character used to indicate a comment line in the table file.
-map <list of column name map pairs>	List of column mapped name table name pairs. This option can be used to map from the names desired in the table to the names expected by the table writing command. This enables user-defined column names within the written tables. The <column map> is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table writing command.
-columns <list of column names>	List of column name which should be skipped from writing in table. Indicates a list of columns whose values are expected to be identical across all table rows and which should therefore be excluded from the table. If the table writer determines that the values are not identical, then table

	writing fails.
-types <list of types>	List of types for which subsystem data is to be written in table.
-string_delimiter <table text delimiter>	The character used to delimit strings in the table.
<filename>	The name of table file to write.

Description

This command will write the subsystem table as indicated by options specified. The types option can be used to restrict which types of subsystem data are written. Only those types seen with -types will be written. If nothing is specified as -type, then all the types will be written. See the description of `read_subsystem_table` for description of the valid line types.

Examples

Following command will write details for every type

```
coreAssembler> write_subsystem_table \
subsystemInfoTable.txt \
-name SUBSYSTEM_TABLE_ALL \
-map {{Instance TargetInstanceName} \
{Interface TargetInterfaceName} \
{Instance2 SourceInstanceName} \
{Interface2 SourceInterfaceName} \
{Param ParameterName} \
{Value ParameterValue } } \
-separator "@"
This will write only about interface connections
```

```
coreAssembler> write_subsystem_table \
subsystemInfoTable.txt \
-name SUBSYSTEM_TABLE_CONNECT \
-map {{Instance TargetInstanceName} \
{Interface TargetInterfaceName} \
{Instance2 SourceInstanceName} \
{Interface2 SourceInterfaceName} } \
-col { VLNV ParameterName ParameterValue } \
-separator "@" \
-types {connect} \
-append
```

See Also

read_subsystem_table (2), read_attribute_table (2), write_attribute_table
(2), read_pin_connection_table (2), write_pin_connection_table (2),

xml_translate

Export an IP-XACT description of the current component.

Syntax

```
string xml_translate -xmlin <input file> -version <1.0 1.1 1.2 1.4 1.5
1685-2009 1685-2014> [-outfile <output file>]
    string <input file>
    string <1.0 1.1 1.2 1.4 1.5 1685-2009 1685-2014>
    string <output file>
```

Parameters

-xmlin <input file>	Input file needs to be translated
-version <1.0 1.1 1.2 1.4 1.5 1685-2009 1685-2014>	Outversion of given IP_XACT file
-outfile <output file>	Name of the output XML file

Description

The xml_translate command translates the component IP_XACT file into the given version and generates the output xml_file of the specified version . The outfile will be generated with the name inputfile_<version>.xml

Examples

```
xml_translate -xmlin 8.x_2014.xml -version 1.4
xml_translate takes input 1. IP_XACT file "8.x_2014.xml" , 2. version to
be converted as "1.4" , translates and generates the outfile
"8.x_2014_1.4.xml".
```

```
xml_translate -xmlin 8.x_2014.xml -version 1.4 -outfile testfile_1.4.xml
xml_translate takes input 1. IP_XACT file "8.x_2014.xml" , 2. version to
be converted as "1.4" , translates and generates the output with the
given outfile name "testfile_1.4.xml"
```

See Also

xml_verify

Verify IP-XACT syntax and semantics.

Syntax

```
string xml_verify [-files <file names>] [-vlnvs <VLNVs>] [-semantics]
[-crossfile] [-verbose]
    string <file names>
    string <VLNVs>
```

Parameters

-files <file names>	Names of IP-XACT files to be validated
-vlnvs <VLNVs>	VLNVs of IP-XACT objects to be validated
-semantics	Do semantic checking
-crossfile	Do cross-file semantic checking
-verbose	Print status messages as files are processed.

Description

The xml_verify command verifies the syntax and semantics of the IP-XACT object specified using the -vlnvs or -files option. The command refers the search path to query for the IP-XACT objects.

The command when used without the '-semantics' option only performs syntax checking for the IP-XACT object .

A detailed listing of the Semantic consistency rules (SCR) is publishes in the IEEE Std 1685-2014 . The current implementation of the command supports a partial list of SCR being checked.

Rule listing currently supported

Category	SCR Supported	SCR Not Supported
Cross Reference and VLNV	1.1 to 1.13	-
Interconnections	2.1 to 2.11	2.12 - 2.16
Channels, Bridges and abstractors	3.1 to 3.4	3.5 - 3.18
Monitor interfaces and monitor interconnections	4.1 to 4.6	-
Configurable elements	-	5.1 - 5.15
Ports	6.1 - 6.7	6.8 - 6.27
Registers	7.1 - 7.21	

coreTools Command Reference Index

		7.19 for nested register files
Memory maps	8.1 - 8.19	8.16 for banked address blocks
Addressing	-	9.1 - 9.9
Hierarchy	-	10.1 - 10.12
Hierarchy and memory map	-	11.1 - 11.4
Constraints	-	12.1 - 12.15
Design configurations	-	13.1 - 13.6
Rules requiring external knowledge	-	14.1 - 14.3

Examples

```
coreAssembler> xml_verify -semantics -crossfile -files  
Builder/export/Subsystem1.xml
```

See Also

NAME

zlib - compression and decompression operations

SYNOPSIS

zlib *subcommand arg ...*

DESCRIPTION

The **zlib** command provides access to the compression and check-summing facilities of the Zlib library by Jean-loup Gailly and Mark Adler. It has the following subcommands.

COMPRESSION SUBCOMMANDS

zlib compress *string ?level?*

Returns the zlib-format compressed binary data of the binary string in *string*. If present, *level* gives the compression level to use (from 0, which is uncompressed, to 9, maximally compressed).

zlib decompress *string ?bufferSize?*

Returns the uncompressed version of the raw compressed binary data in *string*. If present, *bufferSize* is a hint as to what size of buffer is to be used to receive the data.

zlib deflate *string ?level?*

Returns the raw compressed binary data of the binary string in *string*. If present, *level* gives the compression level to use (from 0, which is uncompressed, to 9, maximally compressed).

zlib gunzip *string ?-headerVar varName?*

Return the uncompressed contents of binary string *string*, which must have been in gzip format. If **-headerVar** is given, store a dictionary describing the

coreTools Command Reference Index

contents of the gzip header in the variable called *varName*. The keys of the dictionary that may be present are:

comment

The comment field from the header, if present.

crc

A boolean value describing whether a CRC of the header is computed.

filename

The filename field from the header, if present.

os

The operating system type code field from the header (if not the QW unknown value). See RFC 1952 for the meaning of these codes.

size

The size of the uncompressed data.

time

The time field from the header if non-zero, expected to be time that the file named by the **filename** field was modified. Suitable for use with **clock format**.

type

The type of the uncompressed data (**binary** or **text**) if known.

zlib gzip string ?-level level? ?-header dict?

Return the compressed contents of binary string *string* in gzip format. If **-level** is given, *level* gives the compression level to use (from 0, which is uncompressed, to 9, maximally compressed). If **-header** is given, *dict* is a dictionary containing values used for the gzip header. The following keys may be defined:

comment

Add the given comment to the header of the gzip-format data.

crc

A boolean saying whether to compute a CRC of the header. Note that if the data is to be interchanged with the **gzip** program, a header CRC should *not* be computed.

filename

The name of the file that the data to be compressed came from.

os

The operating system type code, which should be one of the values described in RFC 1952.

time

The time that the file named in the **filename** key was last modified. This will be in the same as is returned by **clock seconds** or **file mtime**.

coreTools Command Reference Index

type

The type of the data being compressed, being **binary** or **text**.

zlib inflate *string* *?bufferSize?*

Returns the uncompressed version of the raw compressed binary data in *string*. If present, *bufferSize* is a hint as to what size of buffer is to be used to receive the data.

CHANNEL **SUBCOMMAND**

zlib push *mode channel* *?options ...?*

Pushes a compressing or decompressing transformation onto the channel *channel*. The transformation can be removed again with **chan pop**. The *mode* argument determines what type of transformation is pushed; the following are supported:

compress

The transformation will be a compressing transformation that produces zlib-format data on *channel*, which must be writable.

decompress

The transformation will be a decompressing transformation that reads zlib-format data from *channel*, which must be readable.

deflate

The transformation will be a compressing transformation that produces raw compressed data on *channel*, which must be writable.

gunzip

The transformation will be a decompressing transformation that reads gzip-format data from *channel*, which must be readable.

gzip

The transformation will be a compressing transformation that produces gzip-format data on *channel*, which must be writable.

inflate

The transformation will be a decompressing transformation that reads raw compressed data from *channel*, which must be readable.

The following options may be set when creating a transformation via the to the **zlib push** command:

-dictionary *binData*

Sets the compression dictionary to use when working with compressing or decompressing the data to be *binData*. Not valid for transformations that work with gzip-format data. The dictionary should consist of strings (byte sequences) that are likely to be encountered later in the data to be compressed, with the most commonly used strings preferably put towards the end of the dictionary. Tcl provides no mechanism for choosing a good such dictionary for a particular data sequence.

coreTools Command Reference Index

-header *dictionary*

Passes a description of the gzip header to create, in the same format that **zlib gzip** understands.

-level *compressionLevel*

How hard to compress the data. Must be an integer from 0 (uncompressed) to 9 (maximally compressed).

-limit *readaheadLimit*

The maximum number of bytes ahead to read when decompressing.

This option has become **irrelevant**. It was originally introduced to prevent Tcl from reading beyond the end of a compressed stream in multi-stream channels to ensure that the data after was left alone for further reading, at the cost of speed.

Tcl now automatically returns any bytes it has read beyond the end of a compressed stream back to the channel, making them appear as unread to further readers.

Both compressing and decompressing channel transformations add extra configuration options that may be accessed through **chan configure**. The options are:

-checksum *checksum*

This read-only option gets the current checksum for the uncompressed data that the compression engine has seen so far. It is valid for both compressing and decompressing transforms, but not for the raw inflate and deflate formats. The compression algorithm depends on what format is being produced or consumed.

-dictionary *binData*

This read-write options gets or sets the initial compression dictionary to use when working with compressing or decompressing the data to be *binData*. It is not valid for transformations that work with gzip-format data, and should not normally be set on compressing transformations other than at the point where the transformation is stacked. Note that this cannot be used to get the current active compression dictionary mid-stream, as that information is not exposed by the underlying library.

-flush *type*

This write-only operation flushes the current state of the compressor to the underlying channel. It is only valid for compressing transformations. The *type* must be either **sync** or **full** for a normal flush or an expensive flush respectively. Flushing degrades the compression ratio, but makes it easier for a decompressor to recover more of the file in the case of data corruption.

-header *dictionary*

This read-only option, only valid for decompressing transforms that are processing gzip-format data,

coreTools Command Reference Index

returns the dictionary describing the header read off the data stream.

-limit *readaheadLimit*

This read-write option is used by decompressing channels to control the maximum number of bytes ahead to read from the underlying data source. See above for more information.

STREAMING SUBCOMMAND

zlib stream *mode ?options?*

Creates a streaming compression or decompression command based on the *mode*, and return the name of the command. For a description of how that command works, see **STREAMING INSTANCE COMMAND** below. The following modes and *options* are supported:

zlib stream compress *?-dictionary bindata? ?-level level?*

The stream will be a compressing stream that produces zlib-format output, using compression level *level* (if specified) which will be an integer from 0 to 9, and the compression dictionary *bindata* (if specified).

zlib stream decompress *?-dictionary bindata?*

The stream will be a decompressing stream that takes zlib-format input and produces uncompressed output. If *bindata* is supplied, it is a compression dictionary to use if required.

zlib stream deflate *?-dictionary bindata? ?-level level?*

The stream will be a compressing stream that produces raw output, using compression level *level* (if specified) which will be an integer from 0 to 9, and the compression dictionary *bindata* (if specified). Note that the raw compressed data includes no metadata about what compression dictionary was used, if any; that is a feature of the zlib-format data.

zlib stream gunzip

The stream will be a decompressing stream that takes gzip-format input and produces uncompressed output.

zlib stream gzip *?-header header? ?-level level?*

The stream will be a compressing stream that produces gzip-format output, using compression level *level* (if specified) which will be an integer from 0 to 9, and the header descriptor dictionary *header* (if specified; for keys see **zlib gzip**).

zlib stream inflate *?-dictionary bindata?*

The stream will be a decompressing stream that takes raw compressed input and produces uncompressed output. If *bindata* is supplied, it is a compression dictionary to use. Note that there are no checks in place to determine whether the compression dictionary is correct.

CHECKSUMMING SUBCOMMANDS

zlib adler32 *string ?initValue?*

Compute a checksum of binary string *string* using the

coreTools Command Reference Index

Adler-32 algorithm. If given, *initValue* is used to initialize the checksum engine.

zlib crc32 *string* *?initValue?*

Compute a checksum of binary string *string* using the CRC-32 algorithm. If given, *initValue* is used to initialize the checksum engine.

STREAMING INSTANCE COMMAND

Streaming compression instance commands are produced by the **zlib stream** command. They are used by calling their **put** subcommand one or more times to load data in, and their **get** subcommand one or more times to extract the transformed data.

The full set of subcommands supported by a streaming instance command, *stream*, is as follows:

stream add *?option...? data*

A short-cut for followed by

stream checksum

Returns the checksum of the uncompressed data seen so far by this stream.

stream close

Deletes this stream and frees up all resources associated with it.

stream eof

Returns a boolean indicating whether the end of the stream (as determined by the compressed data itself) has been reached. Not all formats support detection of the end of the stream.

stream finalize

A short-cut for

stream flush

A short-cut for

stream fullflush

A short-cut for

stream get *?count?*

Return up to *count* bytes from *stream*'s internal buffers with the transformation applied. If *count* is omitted, the entire contents of the buffers are returned.

stream header Return the gzip header description dictionary extracted from the stream. Only supported for streams created with their *mode* parameter set to **gunzip**.

stream put *?option...? data*

Append the contents of the binary string *data* to *stream*'s internal buffers while applying the transformation. The following *options* are supported (or an unambiguous prefix of them), which are used to

coreTools Command Reference Index

modify the way in which the transformation is applied:

-dictionary *binData*

Sets the compression dictionary to use when working with compressing or decompressing the data to be *binData*.

-finalize

Mark the stream as finished, ensuring that all bytes have been wholly compressed or decompressed. For gzip streams, this also ensures that the footer is written to the stream. The stream will need to be reset before having more data written to it after this, though data can still be read out of the stream with the **get** subcommand.

This option is mutually exclusive with the **-flush** and **-fullflush** options.

-flush

Ensure that a decompressor consuming the bytes that the current (compressing) stream is producing will be able to produce all the bytes that have been compressed so far, at some performance penalty.

This option is mutually exclusive with the **-finalize** and **-fullflush** options.

-fullflush

Ensure that not only can a decompressor handle all the bytes produced so far (as with **-flush** above) but also that it can restart from this point if it detects that the stream is partially corrupt. This incurs a substantial performance penalty.

This option is mutually exclusive with the **-finalize** and **-flush** options.

stream **reset**

Puts any stream, including those that have been finalized or that have reached eof, back into a state where it can process more data. Throws away all internally buffered data.

EXAMPLES

To compress a Tcl string, it should be first converted to a particular charset encoding since the **zlib** command always operates on binary strings.

```
set binData [encoding convertto utf-8 $string]
set compData [zlib compress $binData]
```

When converting back, it is also important to reverse the charset encoding:

```
set binData [zlib decompress $compData]
set string [encoding convertfrom utf-8 $binData]
```

coreTools Command Reference Index

The compression operation from above can also be done with streams, which is especially helpful when you want to accumulate the data by stages:

```
set strm [zlib stream compress]
$strm put [encoding convertto utf-8 $string]
# ...
$strm finalize
set compData [$strm get]
$strm close
```

SEE ALSO

binary(n), chan(n), encoding(n), Tcl_ZlibDeflate(3),
RFC1950 - RFC1952

KEYWORDS

compress, decompress, deflate, gzip, inflate, zlib

